NATURAL LANGUAGE PROCESSING

AI · ML · DS / REVISION NOTES

UDEMY · THE LAZY PROGRAMMER

Data Science: Transformers for Natural Language Processing

Attention, BERT, GPT and Hugging Face pipelines — theory and practice

6	521	625
SECTIONS	FIGURES	PAGES

Compiled October 2026

github.com/ankitgupta1729/AI-ML-DS

Contents

Welcome 6

1 Introduction 6
2 Outline 18

Getting Setup 39

1 Get Your Hands Dirty, Practical Coding Experience, Data Links 39
2 How to use Github & Extra Coding Tips (Optional) 55
3 Where to get the code, notebooks, and data 77
4 Are You Beginner, Intermediate, or Advanced All are OK! 83
5 How to Succeed in This Course 93

3 Beginner's Corner 106

1 Beginner's Corner Section Introduction 106
2 From RNNs to Attention and Transformers - Intuition 115
3 Sentiment Analysis 165
4 Sentiment Analysis in Python 185
5 Text Generation 189
6 Text Generation in Python 210
7 Masked Language Modeling (Article Spinner) 212
8 Masked Language Modeling (Article Spinner) in Python 233
9 Named Entity Recognition (NER) 235
10 Named Entity Recognition (NER) in Python 246
11 Text Summarization 247
12 Text Summarization in Python 254
13 Neural Machine Translation 256
14 Neural Machine Translation in Python 264
15 Question Answering 266
16 Question Answering in Python 273
17 Zero-Shot Classification 276
18 Zero-Shot Classification in Python 282
19 Beginner's Corner Section Summary 285
20 Suggestion Box 292

4 Fine-Tuning (Intermediate) 293

1 Fine-Tuning Section Introduction 293
2 Text Preprocessing and Tokenization Review 299
3 Models and Tokenizers 315
4 Models and Tokenizers in Python 342
5 Transfer Learning & Fine-Tuning (pt 1) 345

6 Transfer Learning & Fine-Tuning (pt 2) 357
7 Transfer Learning & Fine-Tuning (pt 3) 374
8 Fine-Tuning Sentiment Analysis and the GLUE Benchmark 385
9 Fine-Tuning Sentiment Analysis in Python 400
10 Fine-Tuning Transformers with Custom Dataset 407
11 Hugging Face AutoConfig 410
12 Fine-Tuning with Multiple Inputs (Textual Entailment) 411
13 Fine-Tuning Transformers with Multiple Inputs in Python 420
14 Fine-Tuning Section Summary 421

5 Named Entity Recognition (NER) and POS Tagging (Intermediate) 425

1 Token Classification Section Introduction 425
2 Data & Tokenizer (Code Preparation) 431
3 Data & Tokenizer (Code) 437
4 Target Alignment (Code Preparation) 439
5 Create Tokenized Dataset (Code Preparation) 448
6 Target Alignment (Code) 452
7 Data Collator (Code Preparation) 455
8 Data Collator (Code) 460
9 Metrics (Code Preparation) 461
10 Metrics (Code) 471
11 Model and Trainer (Code Preparation) 472
12 Model and Trainer (Code) 475
13 POS Tagging & Custom Datasets (Exercise Prompt) 476
14 POS Tagging & Custom Datasets (Solution) 481
15 Token Classification Section Summary 485

6 Seq2Seq and Neural Machine Translation (Intermediate) 487

1 Translation Section Introduction 487
2 Data & Tokenizer (Code Preparation) 488
3 Things Move Fast 489
4 Data & Tokenizer (Code) 490
5 Aside Seq2Seq Basics (Optional) 491
6 Model Inputs (Code Preparation) 494
7 Model Inputs (Code) 496
8 Translation Metrics (BLEU Score & BERT Score) (Code Preparation) 498
9 Translation Metrics (BLEU Score & BERT Score) (Code) 499
10 Train & Evaluate (Code Preparation) 500
11 Train & Evaluate (Code) 501
12 Translation Section Summary 502
1 Question-Answering Section Introduction 503
2 Exploring the Dataset (SQuAD) 504

3	Exploring the Dataset (SQuAD) in Python	505
4	Using the Tokenizer	506
5	Using the Tokenizer in Python	508
6	Aligning the Targets	511
7	Aligning the Targets in Python	514
8	Applying the Tokenizer	518
9	Applying the Tokenizer in Python	520
10	Question-Answering Metrics	522
11	Question-Answering Metrics in Python	523
12	From Logits to Answers	524
13	From Logits to Answers in Python	529
14	Computing Metrics	532
15	Computing Metrics in Python	534
16	Train and Evaluate	535
17	Train and Evaluate in Python	535
18	Question-Answering Section Summary	536
1	Theory Section Introduction	537
2	Basic Self-Attention	539
3	Self-Attention & Scaled Dot-Product Attention	541
4	Attention Efficiency	545
5	Attention Mask	546
6	Multi-Head Attention	547
7	Transformer Block	549
8	Positional Encodings	551
9	Encoder Architecture	553
10	Decoder Architecture	555
11	Encoder-Decoder Architecture	557
12	BERT	560
13	GPT	561
14	GPT-2	563
15	GPT-3	564
16	Theory Section Summary	565
1	Implementation Section Introduction	567
2	Encoder Implementation Plan & Outline	568
3	How to Implement Multihead Attention From Scratch	570
4	How to Implement the Transformer Block From Scratch	573
5	How to Implement Positional Encoding From Scratch	573
6	How to Implement Transformer Encoder From Scratch	575
7	Train and Evaluate Encoder From Scratch	576
8	How to Implement Causal Self-Attention From Scratch	578
9	How to Implement a Transformer Decoder (GPT) From Scratch	579

10	How to Train a Causal Language Model From Scratch	580
11	Implement a Seq2Seq Transformer From Scratch for Language Translation...	584
12	Implement a Seq2Seq Transformer From Scratch for Language Translation...	586
13	Implement a Seq2Seq Transformer From Scratch for Language Translation...	589
14	Implementation Section Summary	593
1	Anaconda Environment Setup	593
2	How to install Numpy, Scipy, Matplotlib, Pandas, IPython, Theano, and...	597
1	How to Code by Yourself (part 1)	599
2	How to Code by Yourself (part 2)	603
1	How to Succeed in this Course (Long Version)	606
2	Is this for Beginners or Experts Academic or Practical Fast or...	609
3	Machine Learning and AI Prerequisite Roadmap (pt 1)	615
4	Machine Learning and AI Prerequisite Roadmap (pt 2)	618
1	What is the Appendix	623
2	BONUS	623

SECTION 01

Welcome

1 · Introduction

FIGURE 1.1

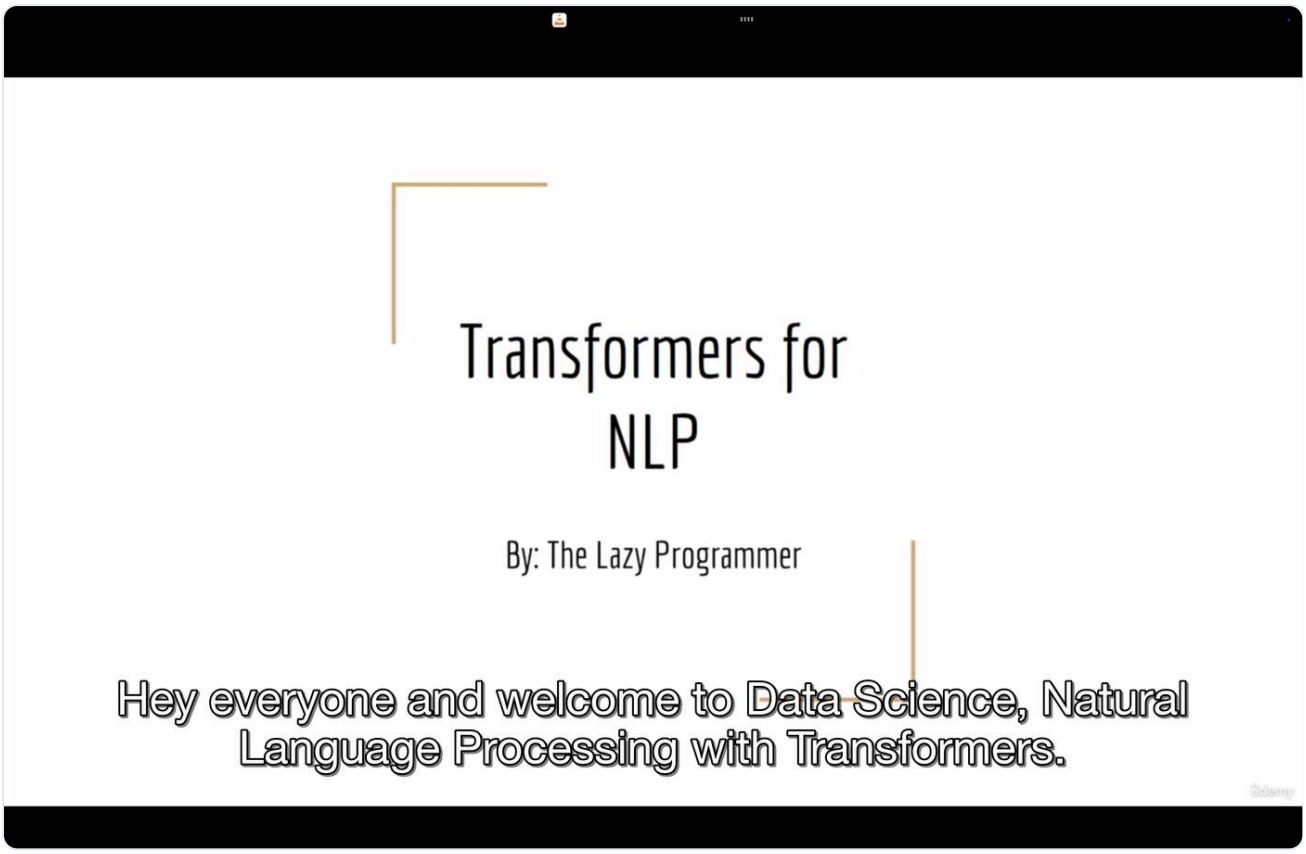

FIGURE 1.2 | Transformers for

Hey everyone and welcome to Data Science, Natural Language Processing with Transformers.

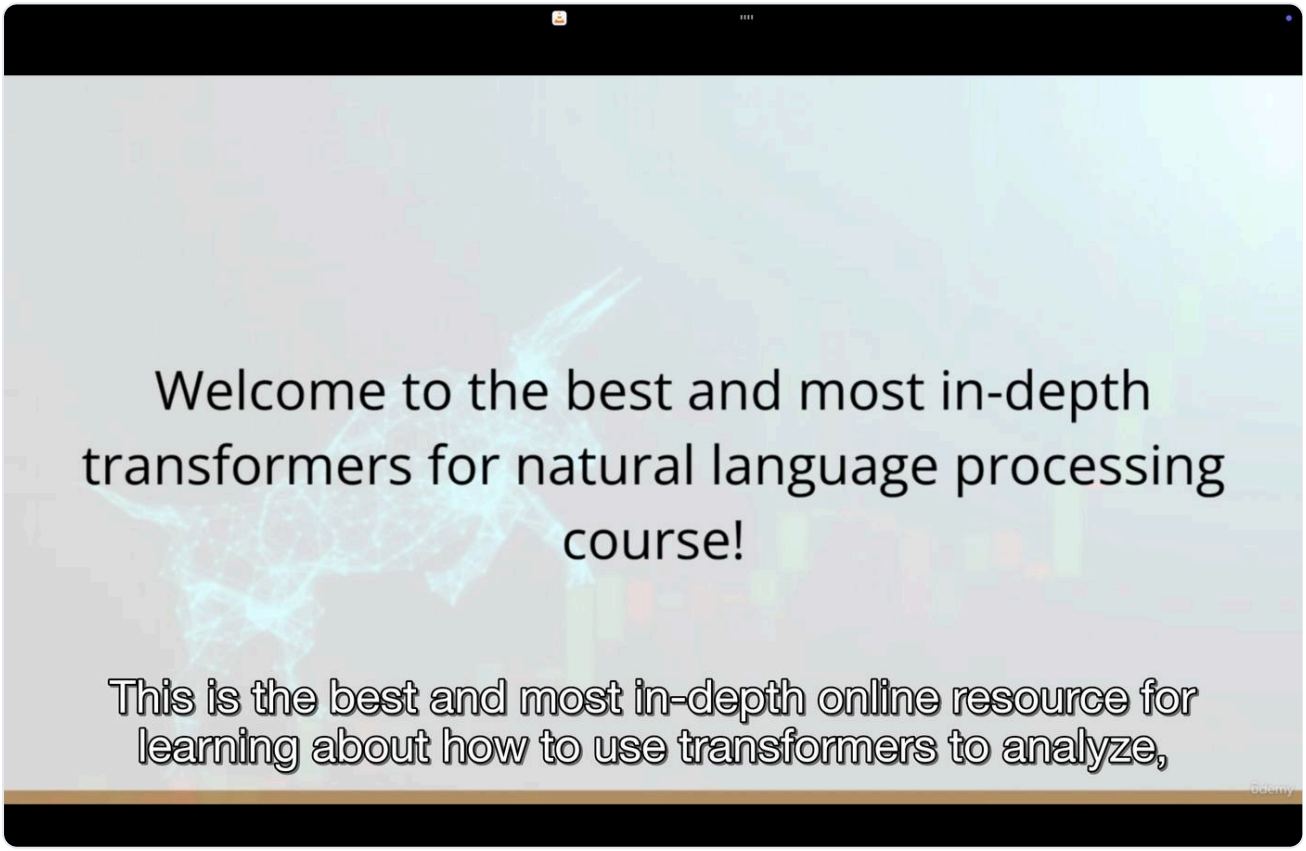

FIGURE 1.3

This is the best and most in-depth online resource for learning about how to use transformers to analyze, understand and even generate language.

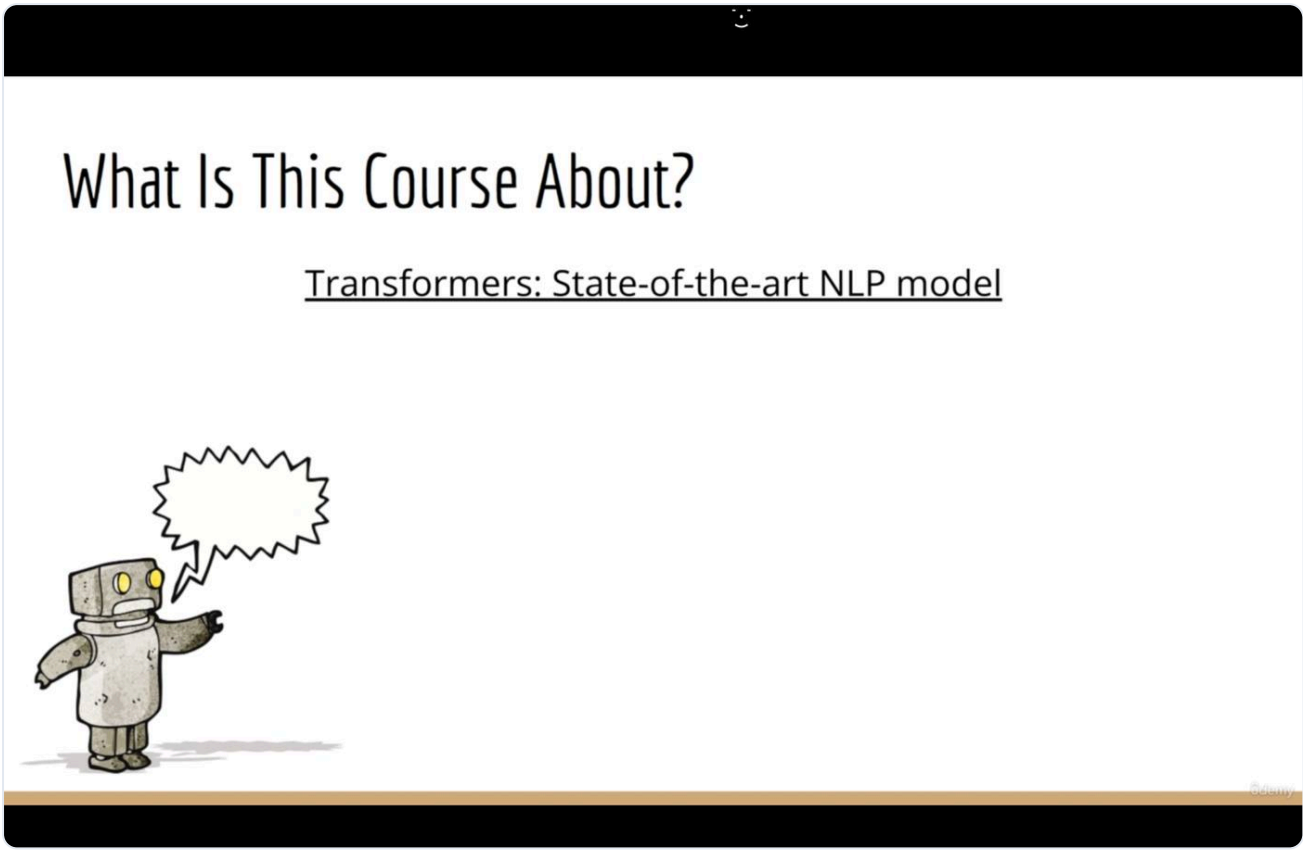

FIGURE 1.4 | What Is This Course About?

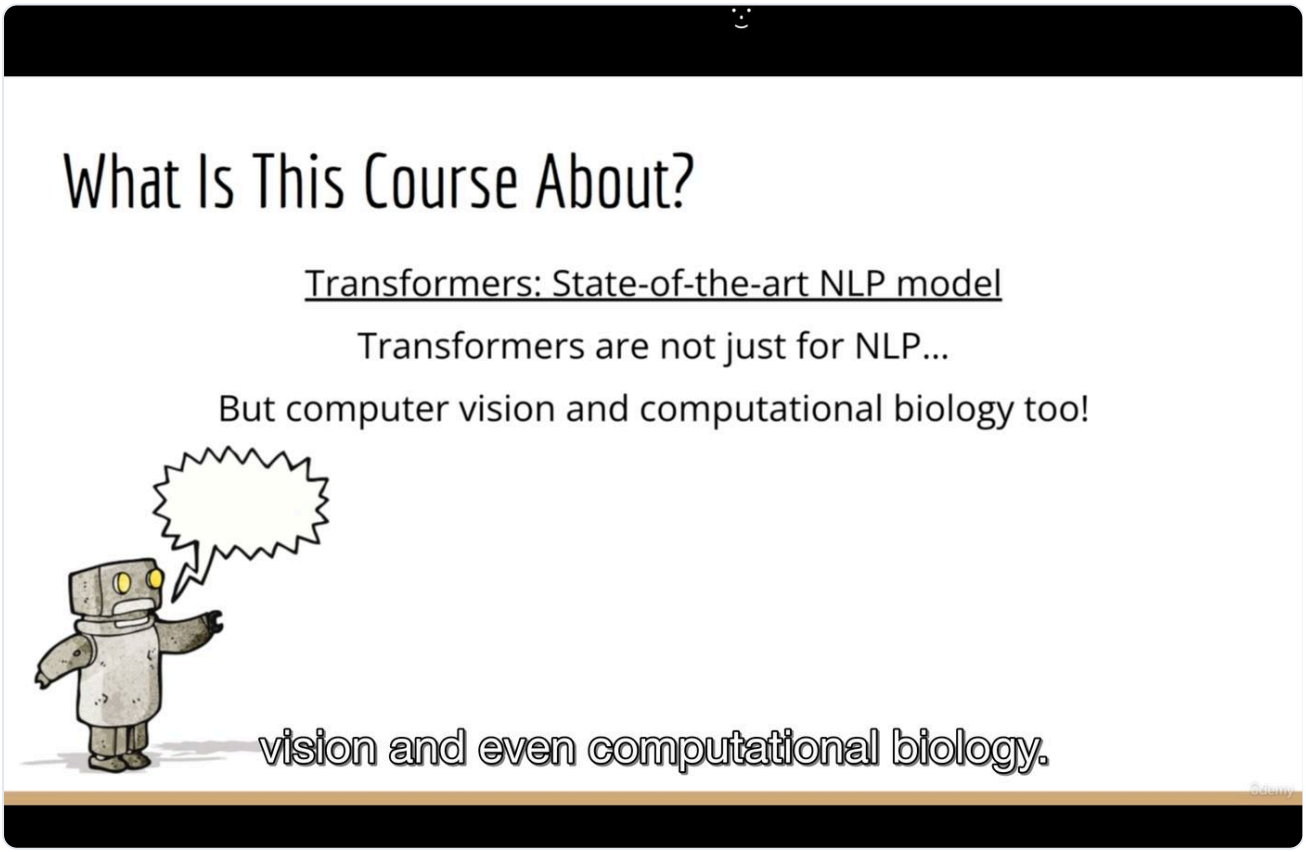

FIGURE 1.5 | What Is This Course About?

So what is this course about? This course we'll teach you about Transformers. Which are these state of the art method of applying machine learning to language? Transformers are the latest deep learning architecture to outperform all existing methods.

You might know about Transformers for NLP, but they've also made significant contributions to computer vision and even computational biology.

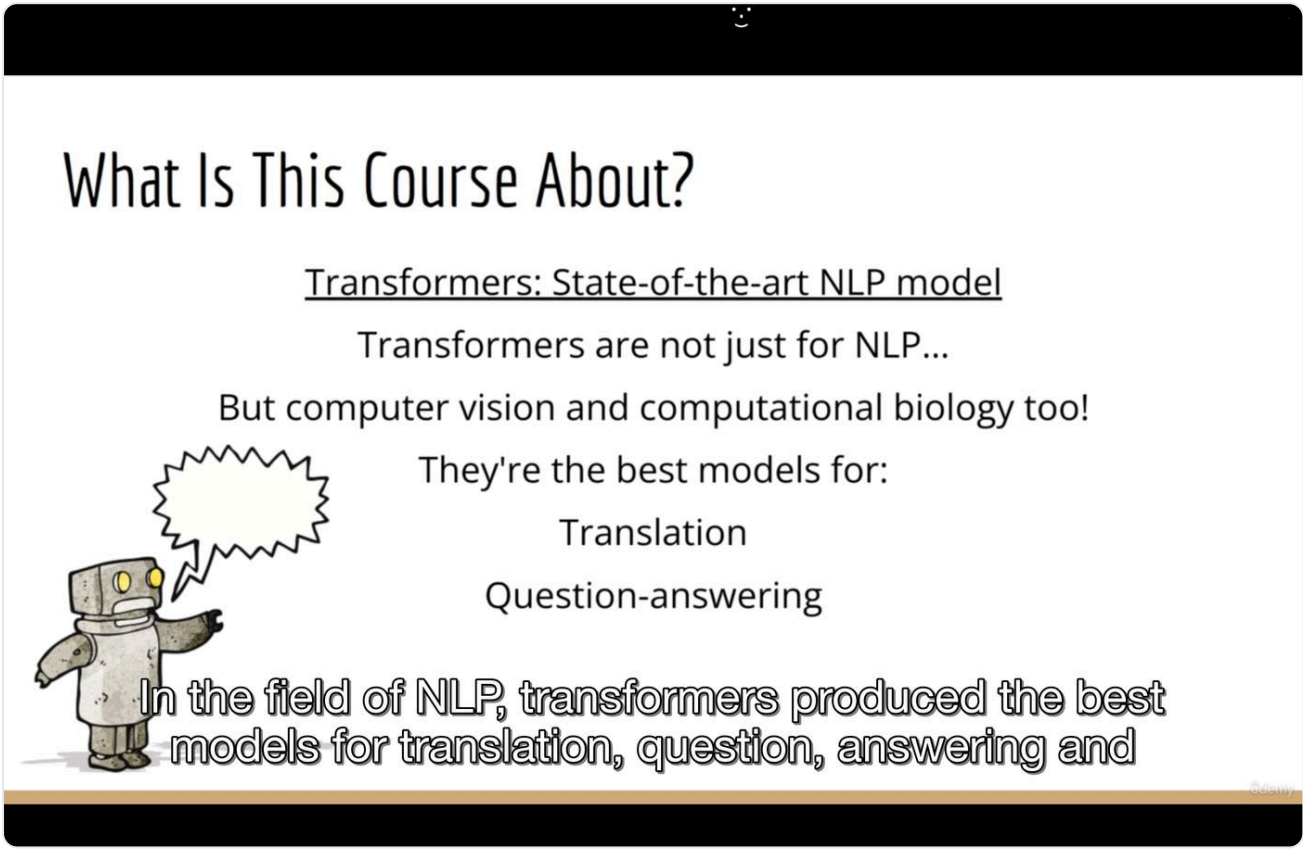

FIGURE 1.6 | What Is This Course About?

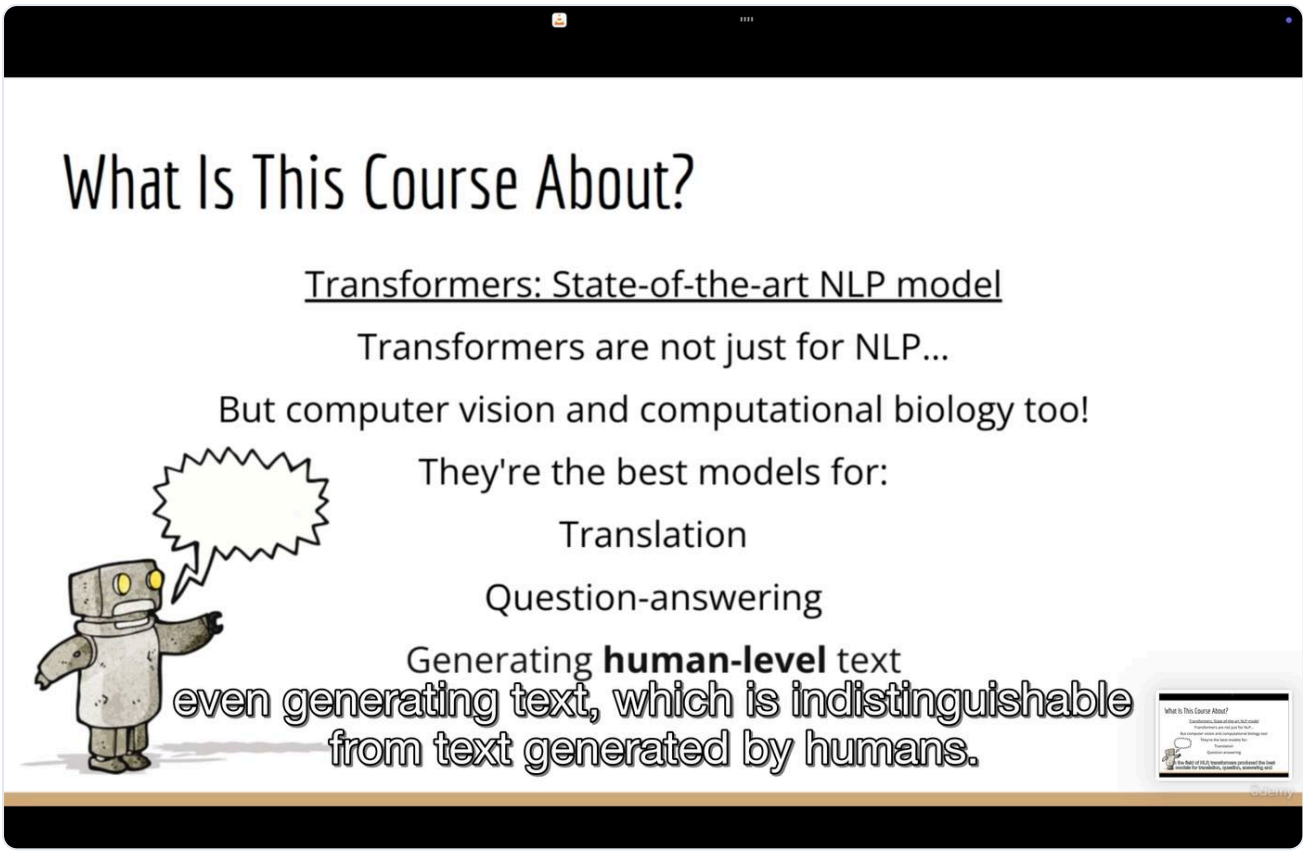

FIGURE 1.7 | What Is This Course About?

In the field of NLP, transformers produced the best models for translation, question, answering and even generating text, which is indistinguishable from text generated by humans.

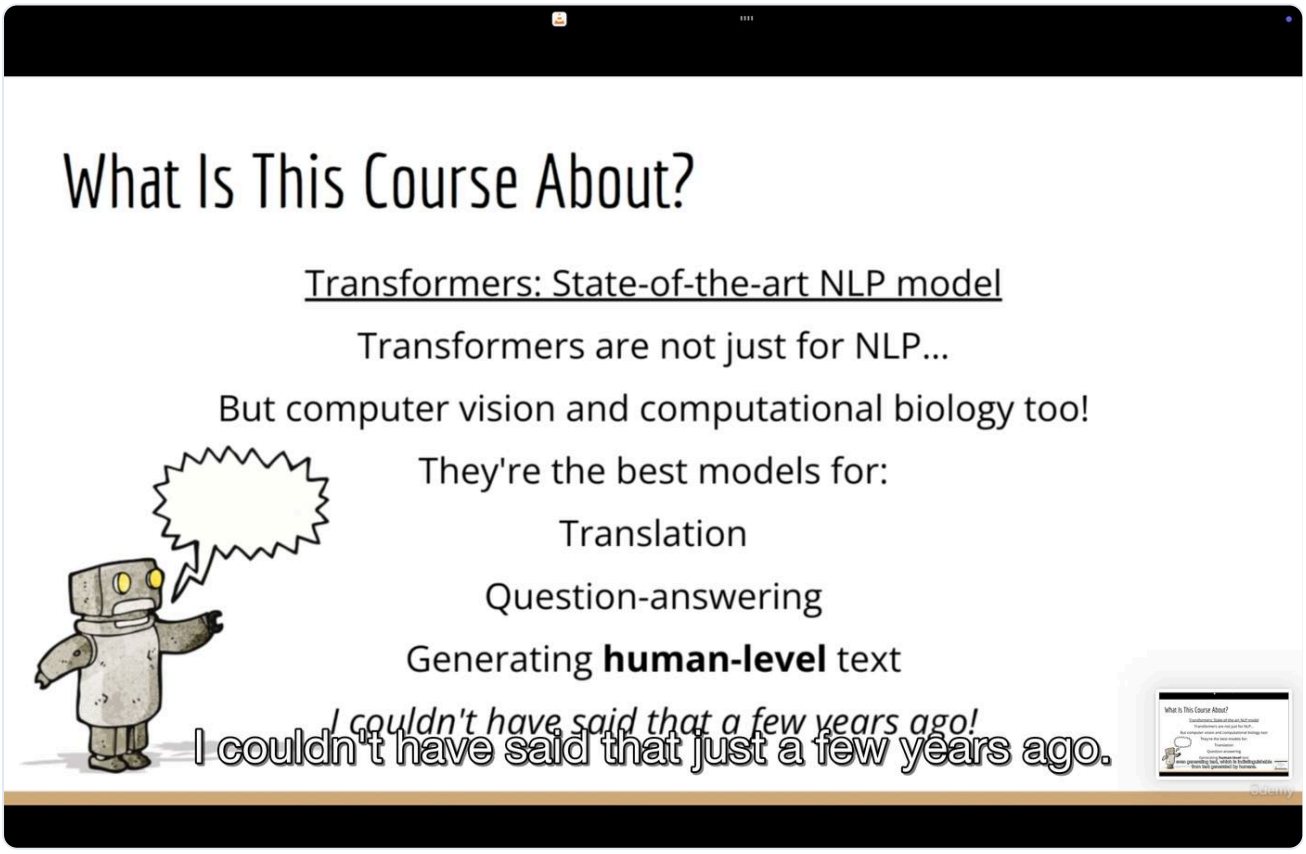

FIGURE 1.8 | What Is This Course About?

I couldn't have said that just a few years ago.

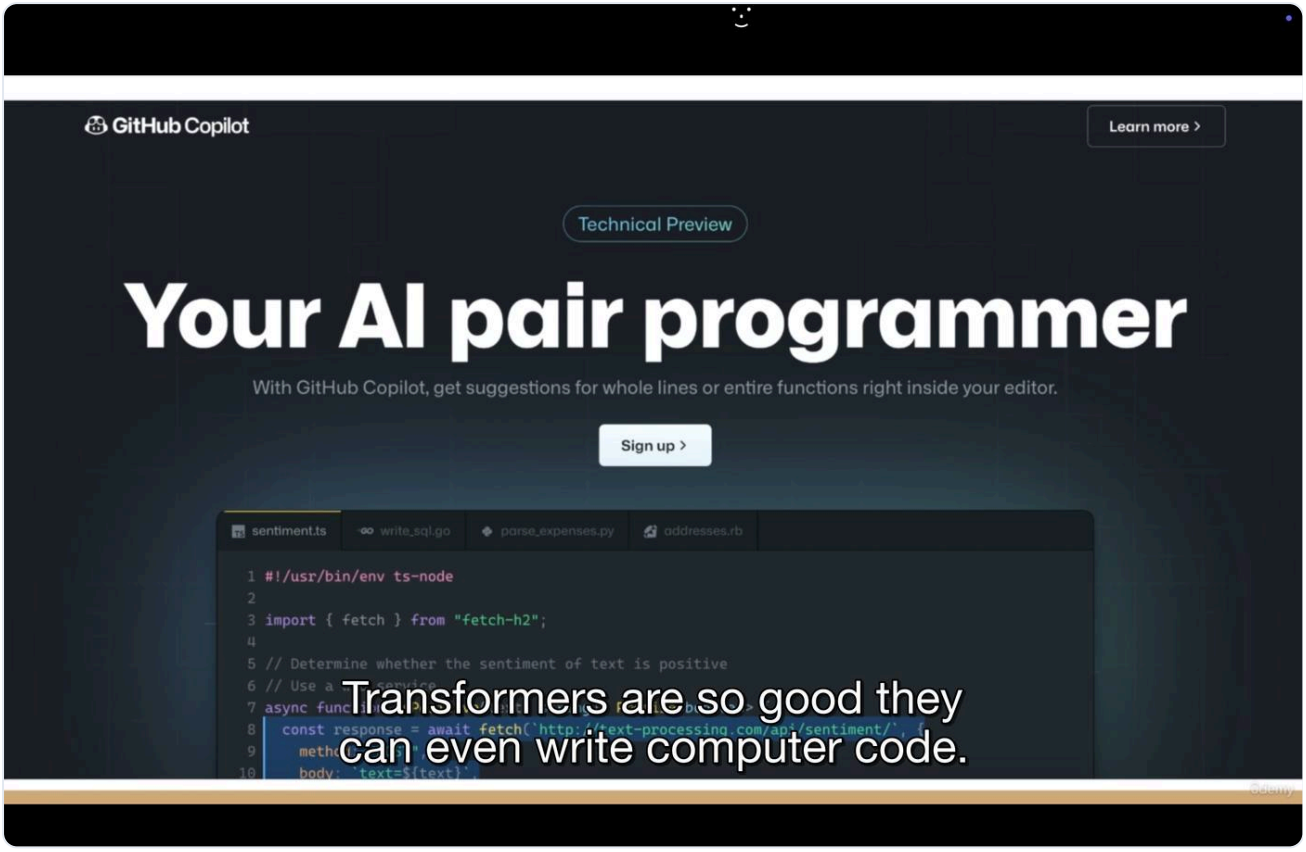

FIGURE 1.9 | Technical Preview

Transformers are so good they can even write computer code.

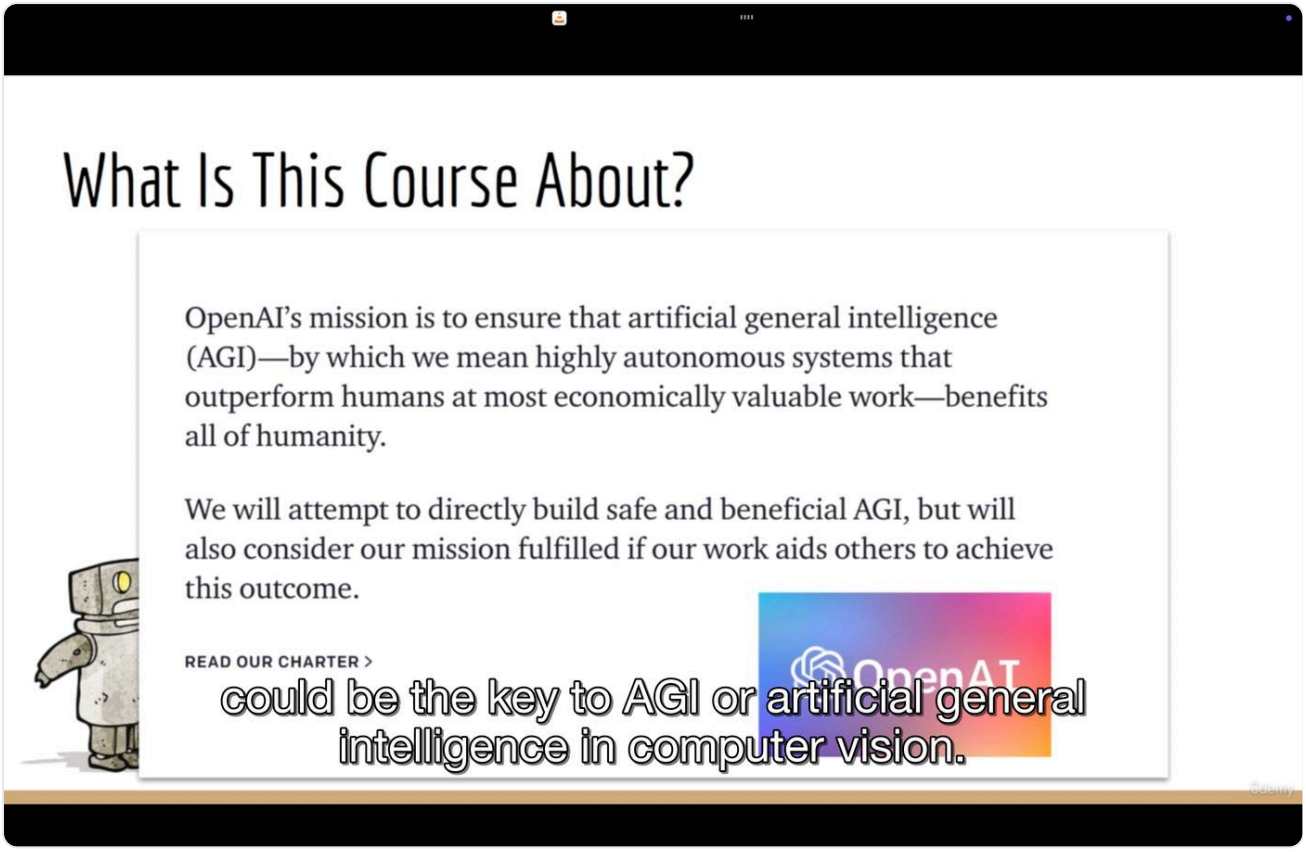

FIGURE 1.10 | What Is This Course About?

In fact, transformers are so good that models such as GPT-3 have everyone talking about how this could be the key to AGI or artificial general intelligence. In computer vision,

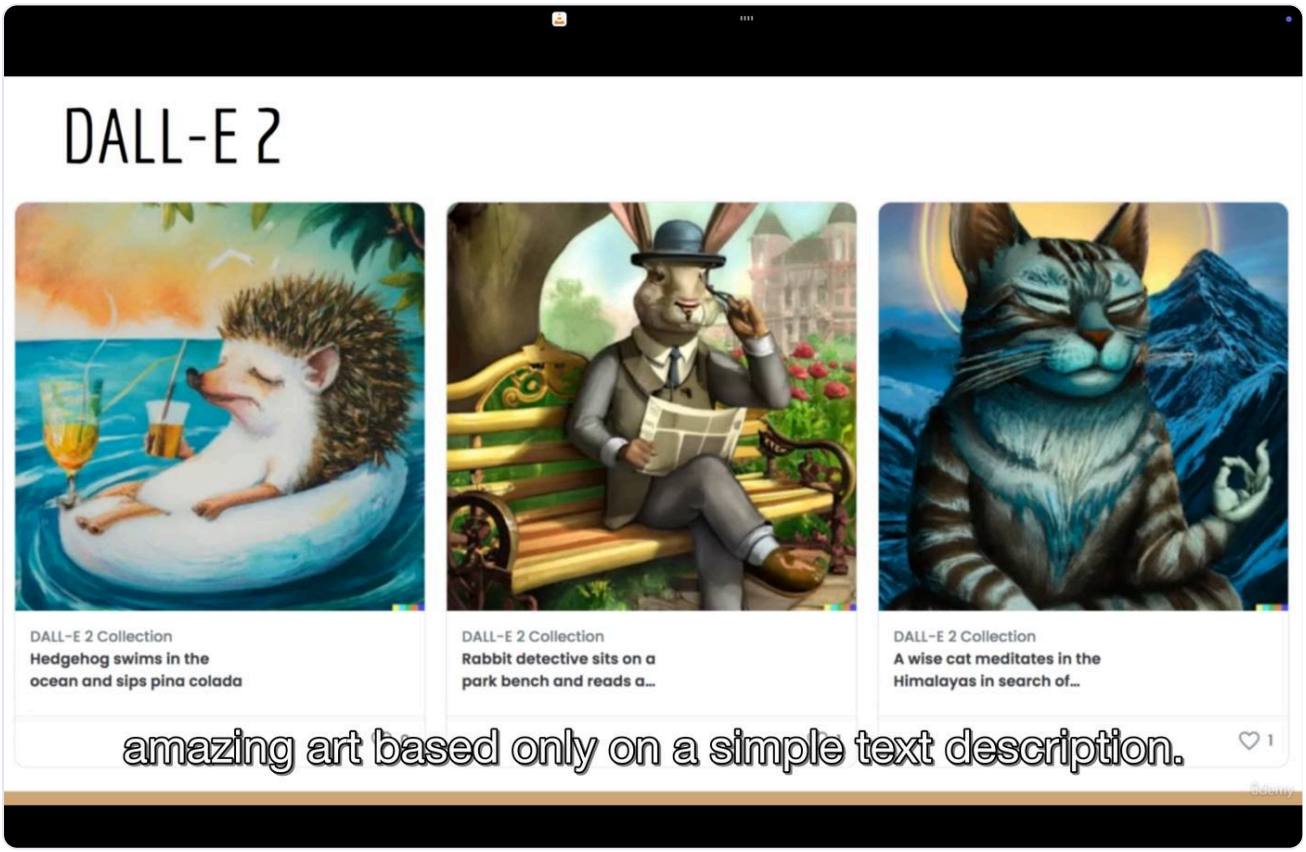

FIGURE 1.11

Transformers have impressed everyone by combining image and text representations in order to generate amazing art based only on a simple text description. In molecular biology, transformers have helped scientists to solve a longstanding problem called the protein structure prediction.

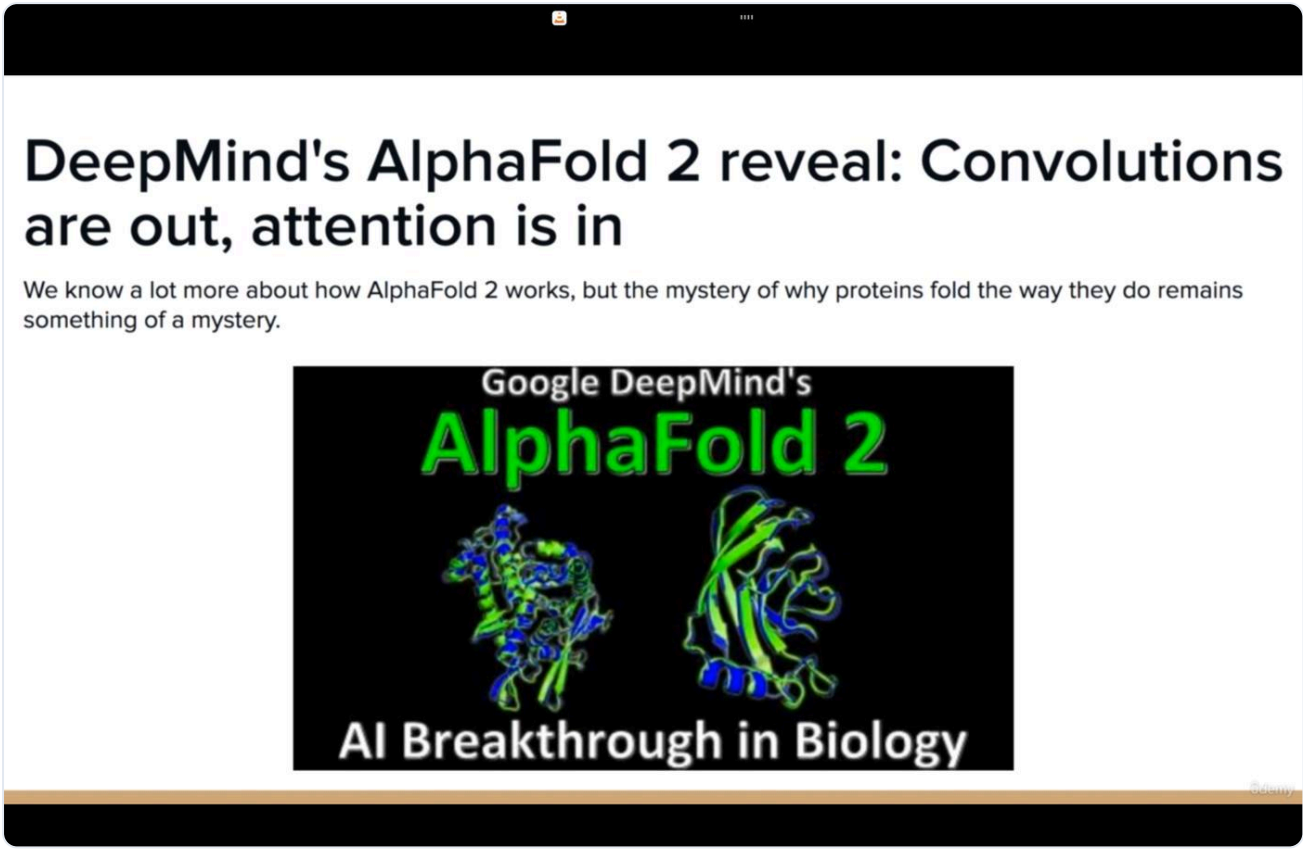

FIGURE 1.12

That is, how can we map your genetic code to actual physical proteins and what do they look like? This is the perfect task for NLP since the genetic code really is just a programming language.

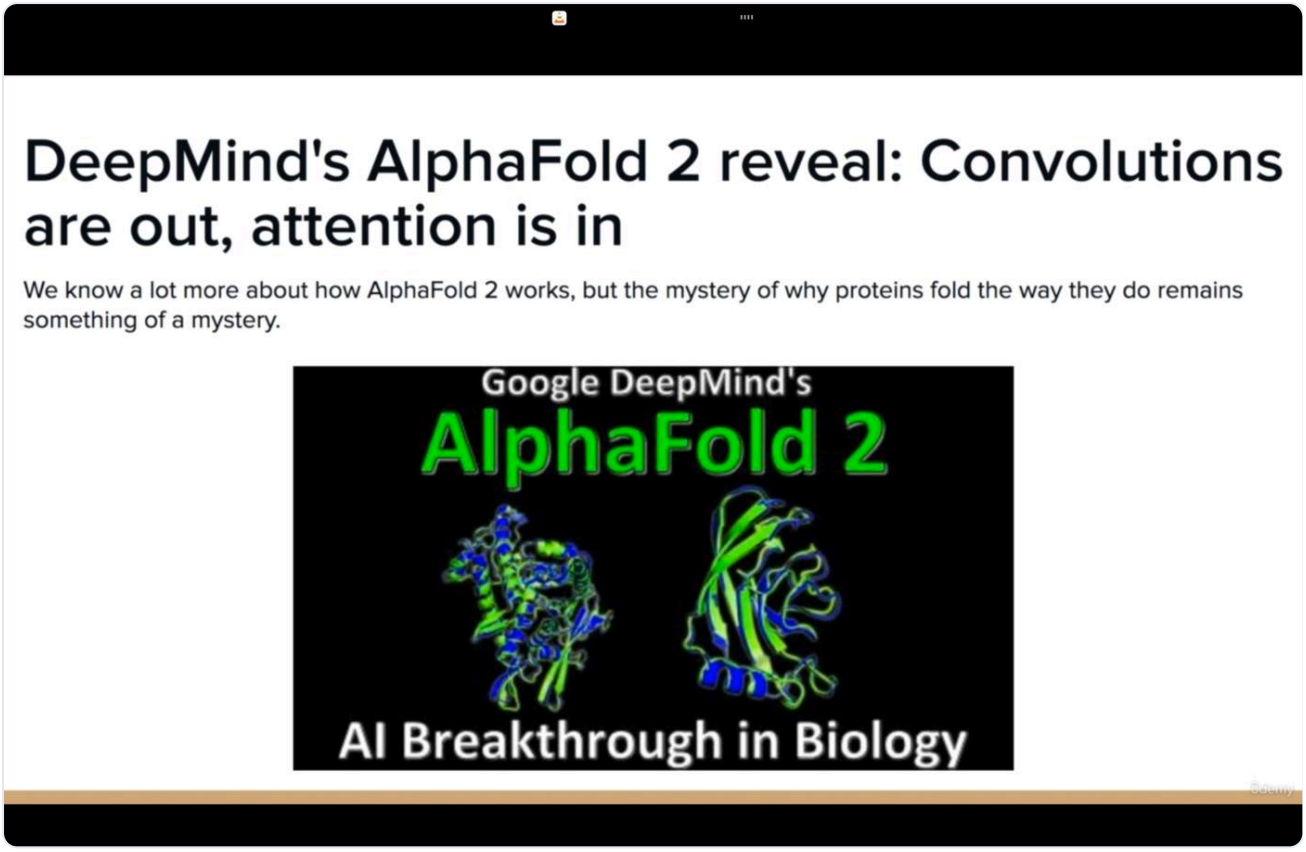

FIGURE 1.13

It is the programming language of life. So I hope you're convinced that Transformers are pretty awesome.

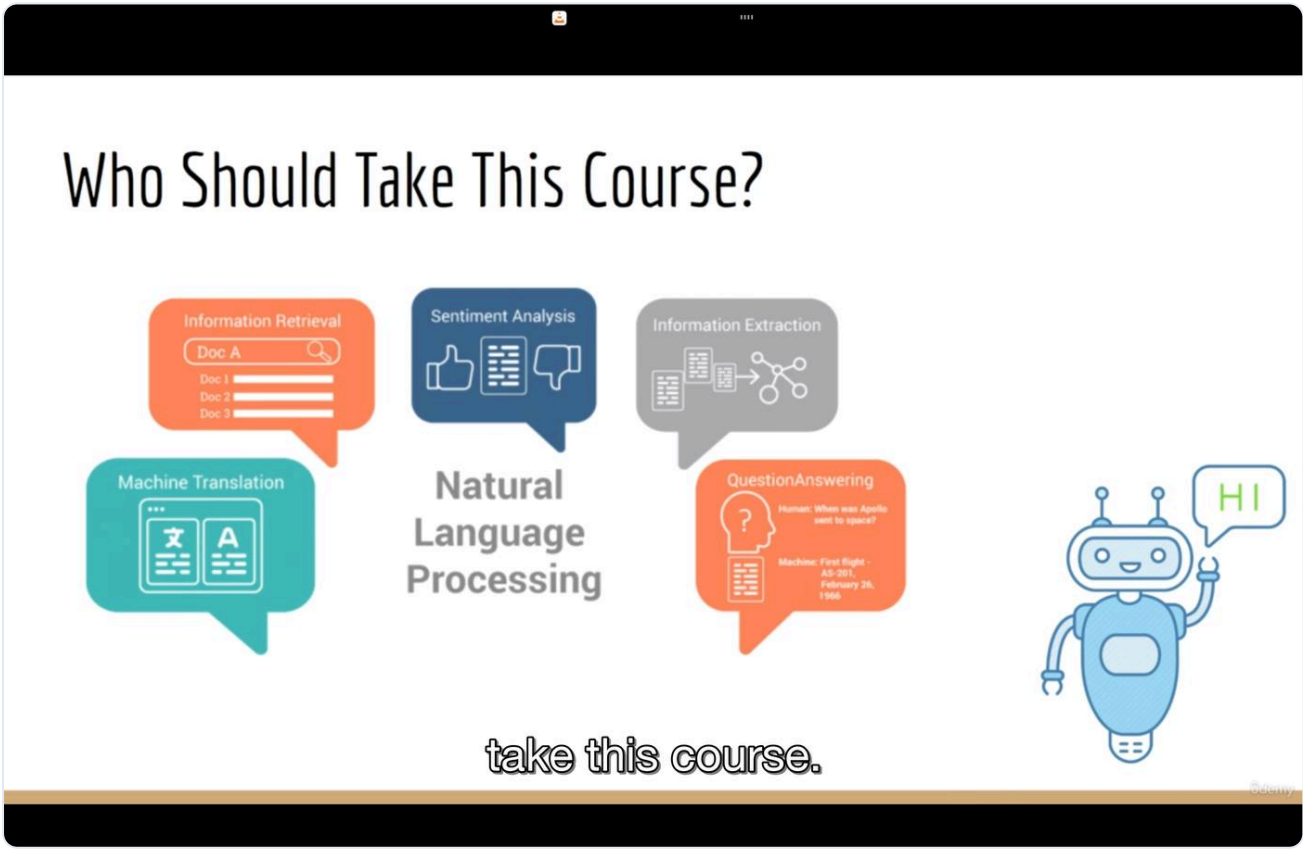

FIGURE 1.14 | Who Should Take This Course?

So who should take this course? Anyone who wants to understand and apply the latest state of the art deep learning methods to NLP should take this course. This course is something for everyone, whether you are beginner, intermediate or advanced. For developers who are more practically oriented, the first part of this course will have no theory and minimal code.

You will learn how to use the most powerful transformers with just a few minutes of efforts. For developers with a little more experience, the second part of this course will look at how to fine tune Transformers so that you can optimize them on your own data sets.

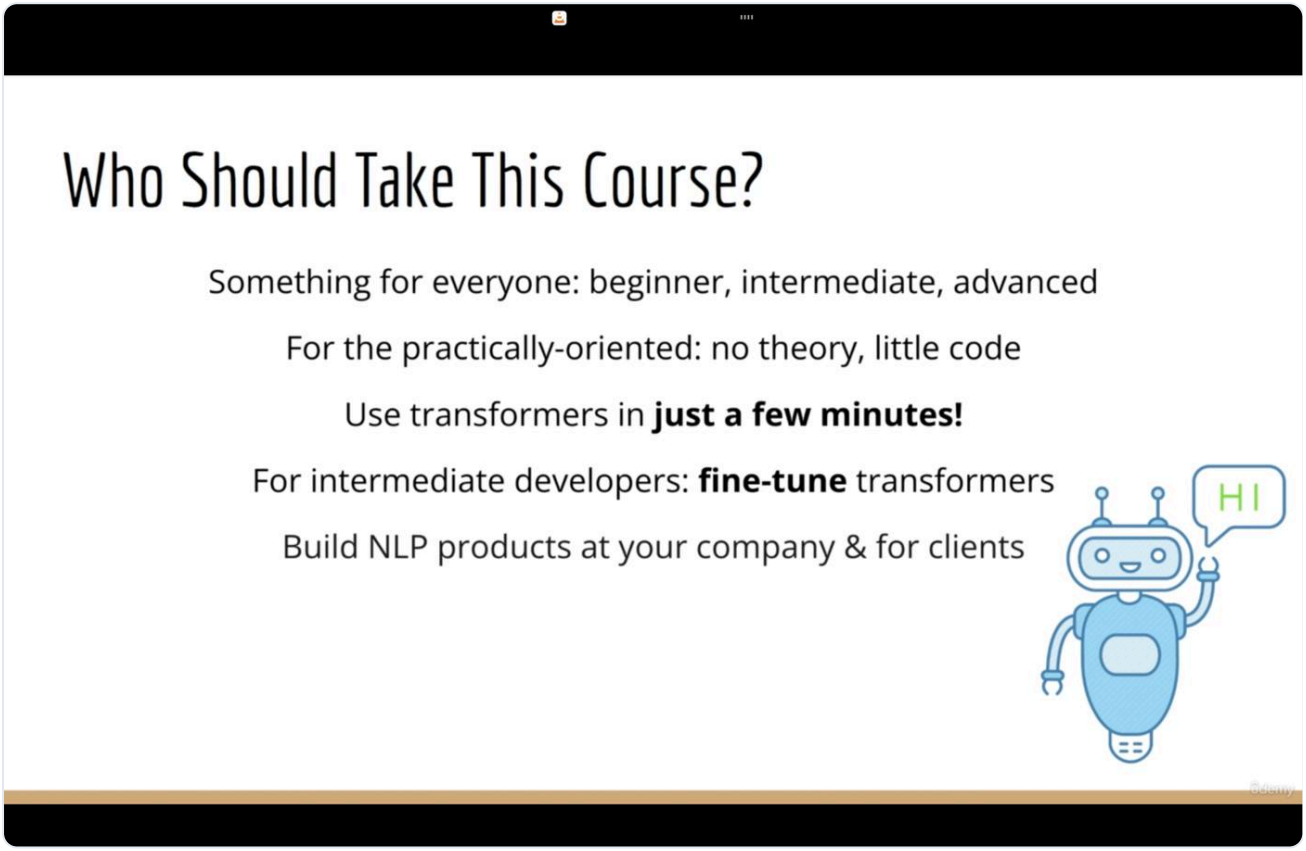

FIGURE 1.15 | Who Should Take This Course?

This will help you build state of the art NLP products at your company and for your clients. For expert developers who want to understand it. Transformers In-Depth. The third part of this course is for you.

FIGURE 1.16 | Who Should Take This Course?

In this section, we will take a deep dive into how the attention mechanism works and eventually how to build a transformer from scratch.

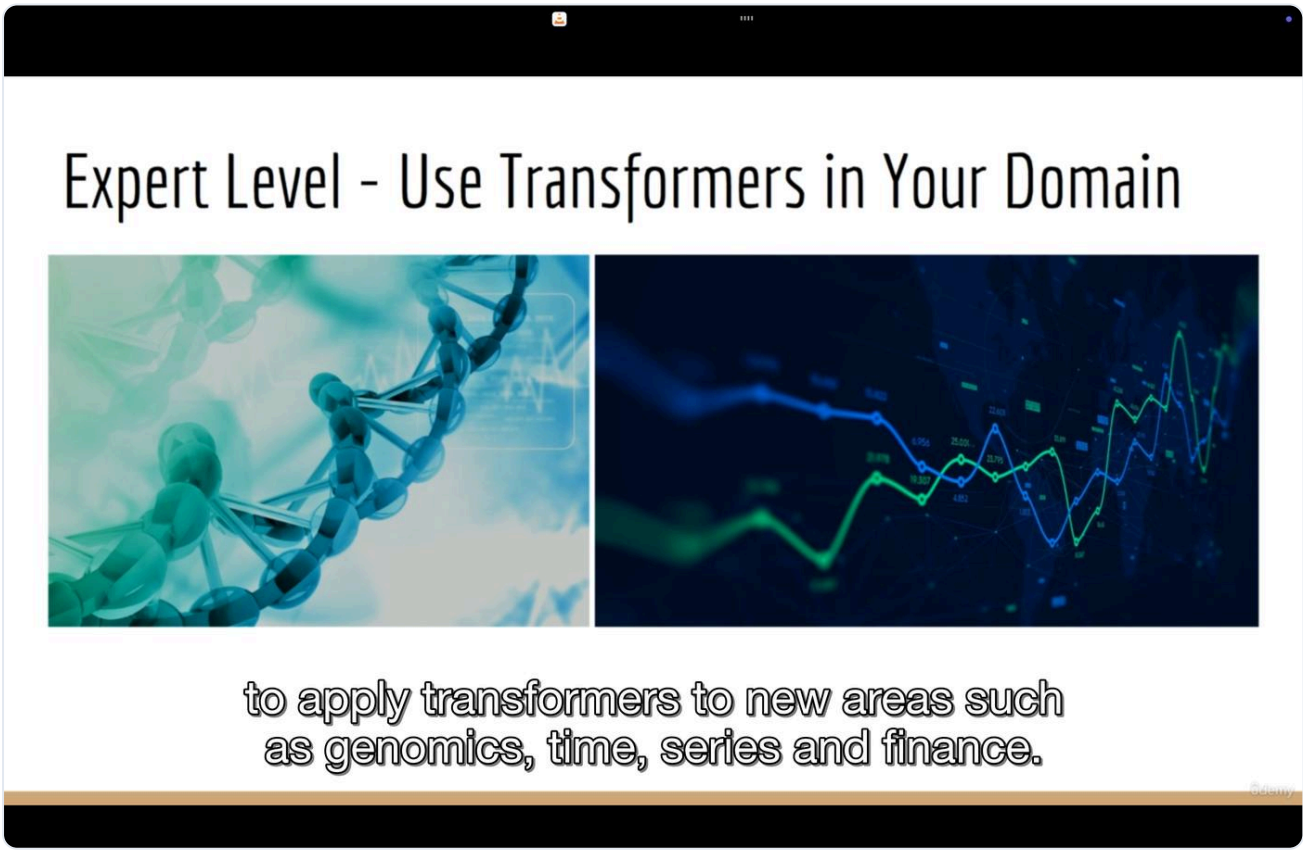

FIGURE 1.17 | Expert Level - Use Transformers in Your Domain

This will be especially useful for those of you who don't want to just fine tune NLP models, but to apply transformers to new areas such as genomics, time, series and finance.

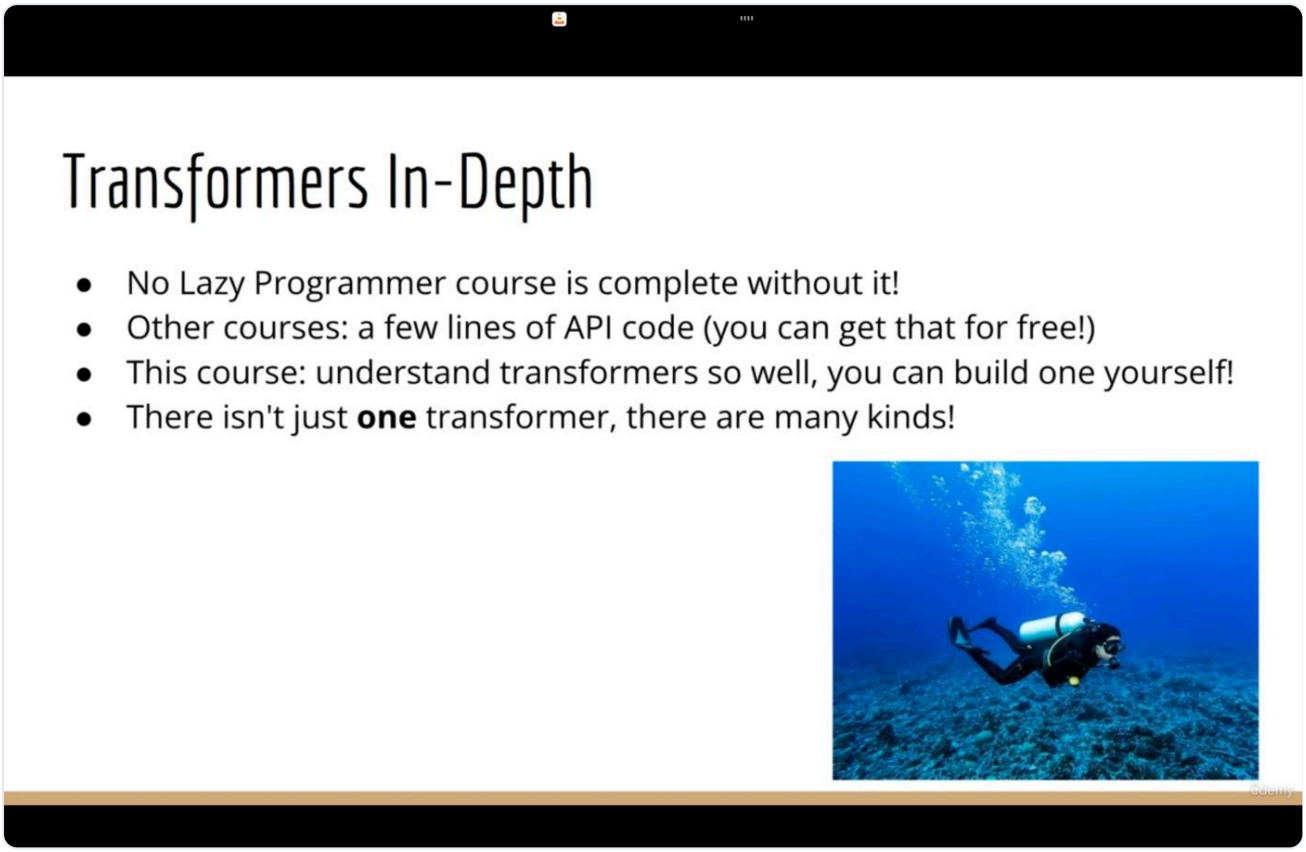

FIGURE 1.18

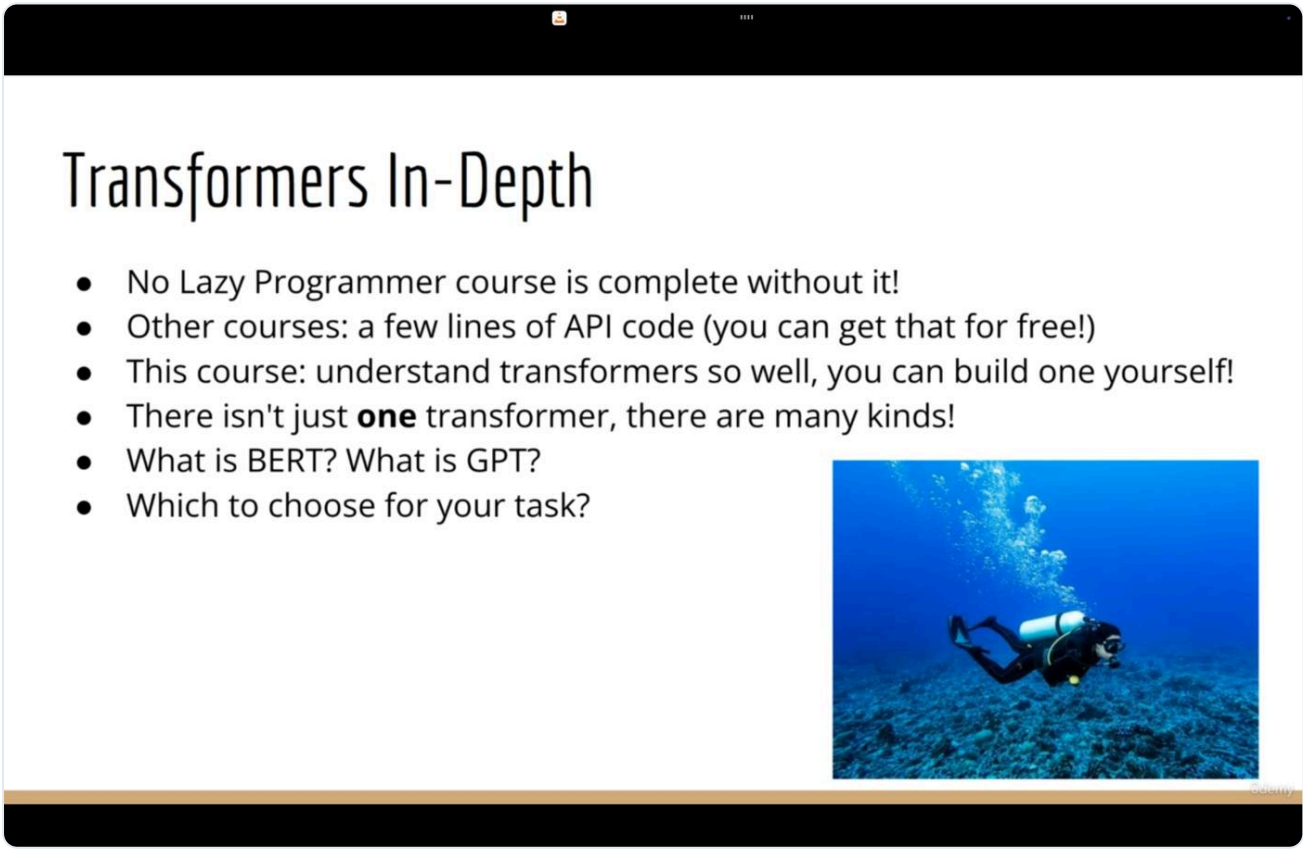

FIGURE 1.19

Now you know that no Lazy Programmer course is complete without in-depth material, and that's really what the advance section is all about.

Whereas other courses might just show you a few lines of API code. My goal in this course is to really help you understand that transformers. In fact, by the end of this course, you'll understand Transformers so well you'll be able to build one yourself from scratch. It's important to remember that transformers aren't just one thing. There are multiple kinds of transformers, such as BERT and GPT. In this course, you'll learn about how to categorize between these different types of transformers.

What is BERT and what is GPT? And how are they different? Importantly, how do you know which one to choose for a specific task? This is the real value behind this course. And by learning about transformers in depth, these skills will be yours.

2 · Outline

So in this lecture, I'm going to give you a brief overview of everything you will learn in this course. This course is quite a large course and was several years in the making.

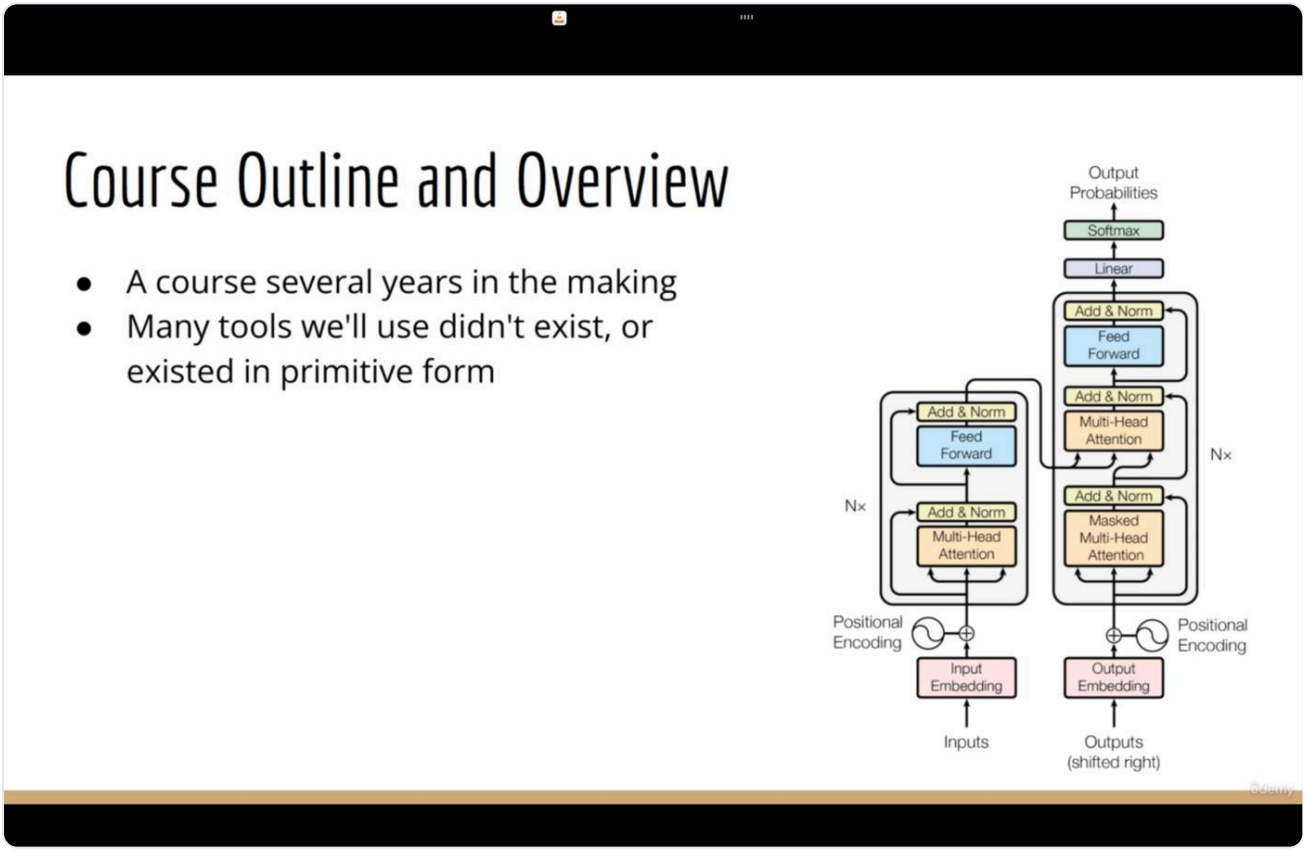

FIGURE 1.20 | Course Outline and Overview

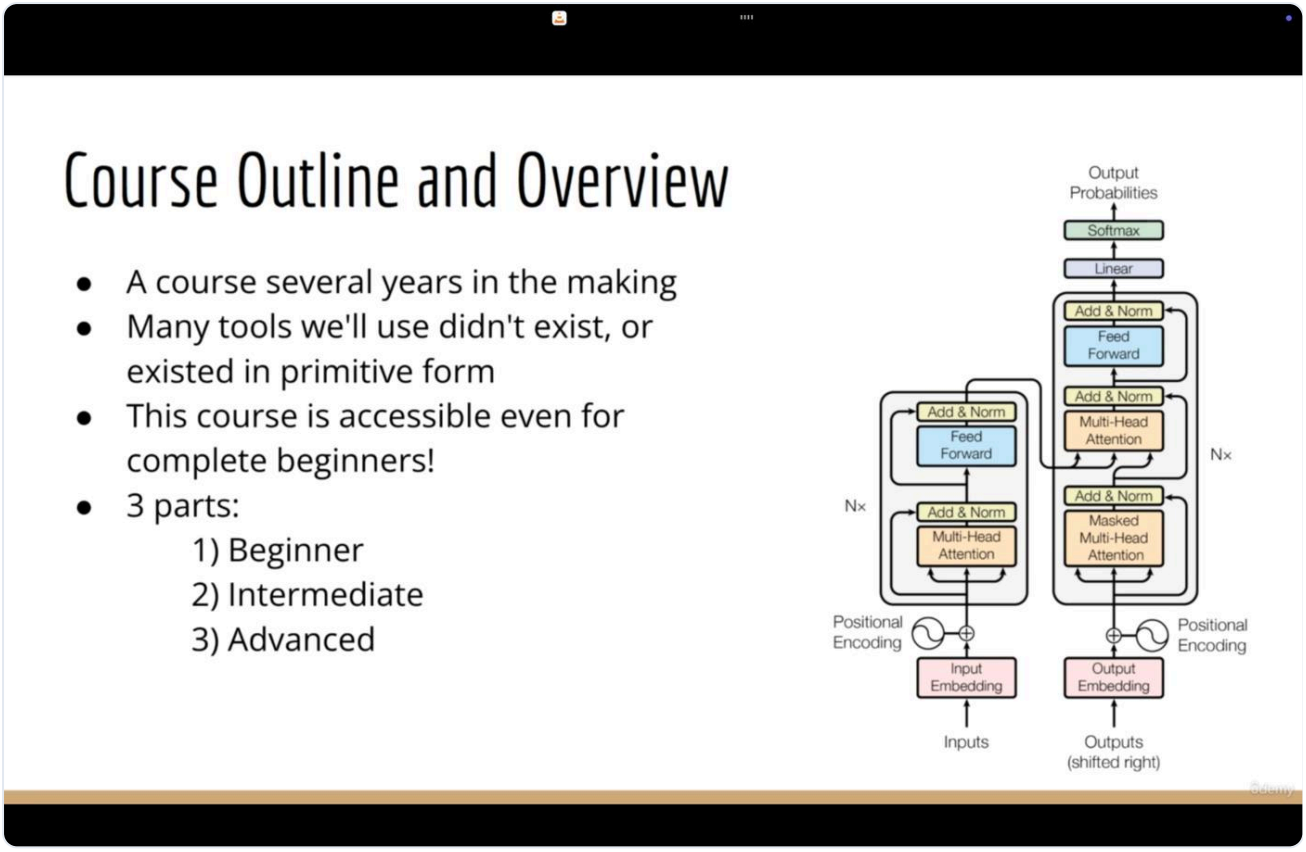

FIGURE 1.21 | Course Outline and Overview

In fact, years ago, when I began creating this course, a lot of the tools we're going to use didn't even exist or existed in primitive form, thanks to the hard work of machine learning engineers around the world. This course is very accessible, even for complete beginners. The best way to understand this course is that at a high level it's made up of three parts the beginners section, the intermediate section and the advanced section.

So the beginners corner is all about how to use Transformers as a beginner.

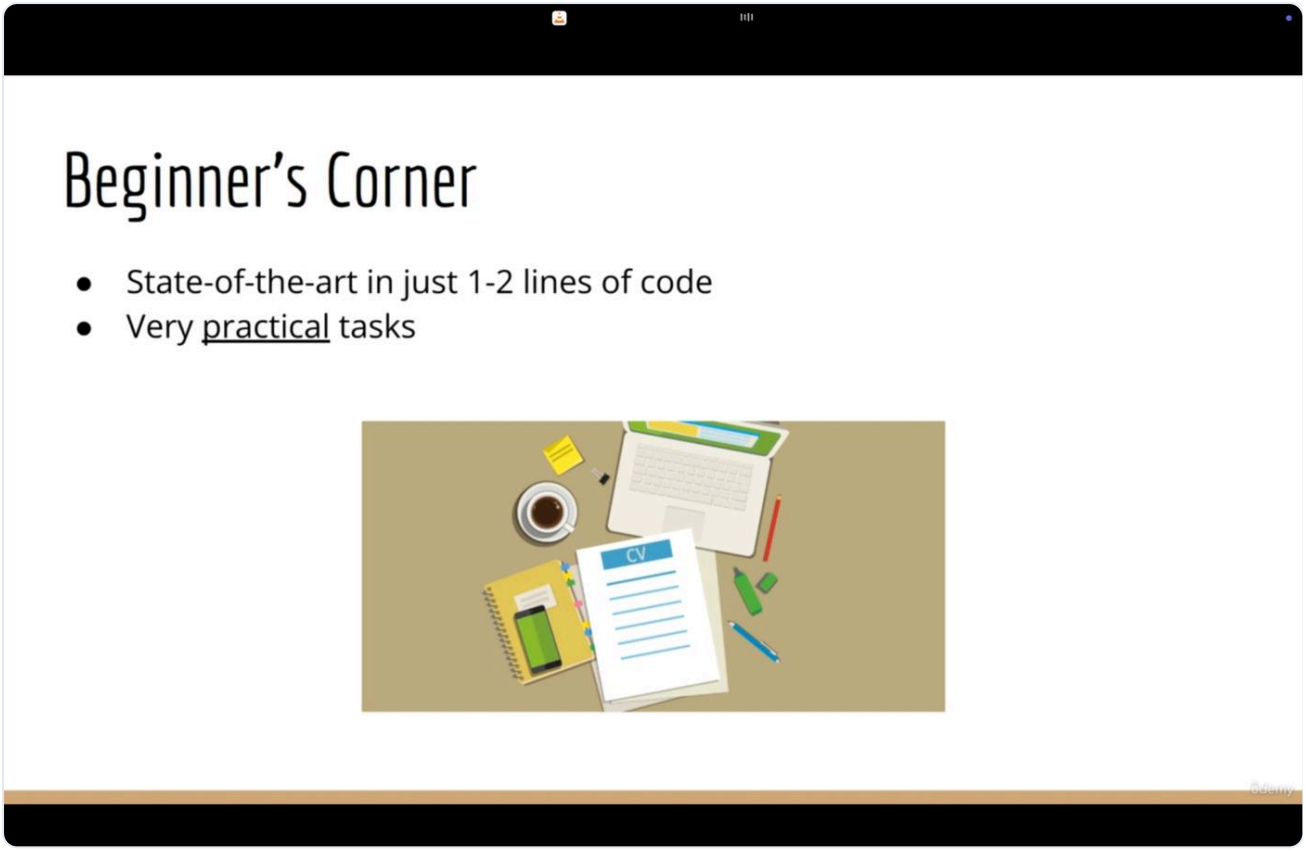

FIGURE 1.22

This means that you might not be too great with coding and you definitely do not like math and theory in this course. That's okay, at least for this part. In this section, you'll be taught how to use state of the art models with just one or two lines of code.

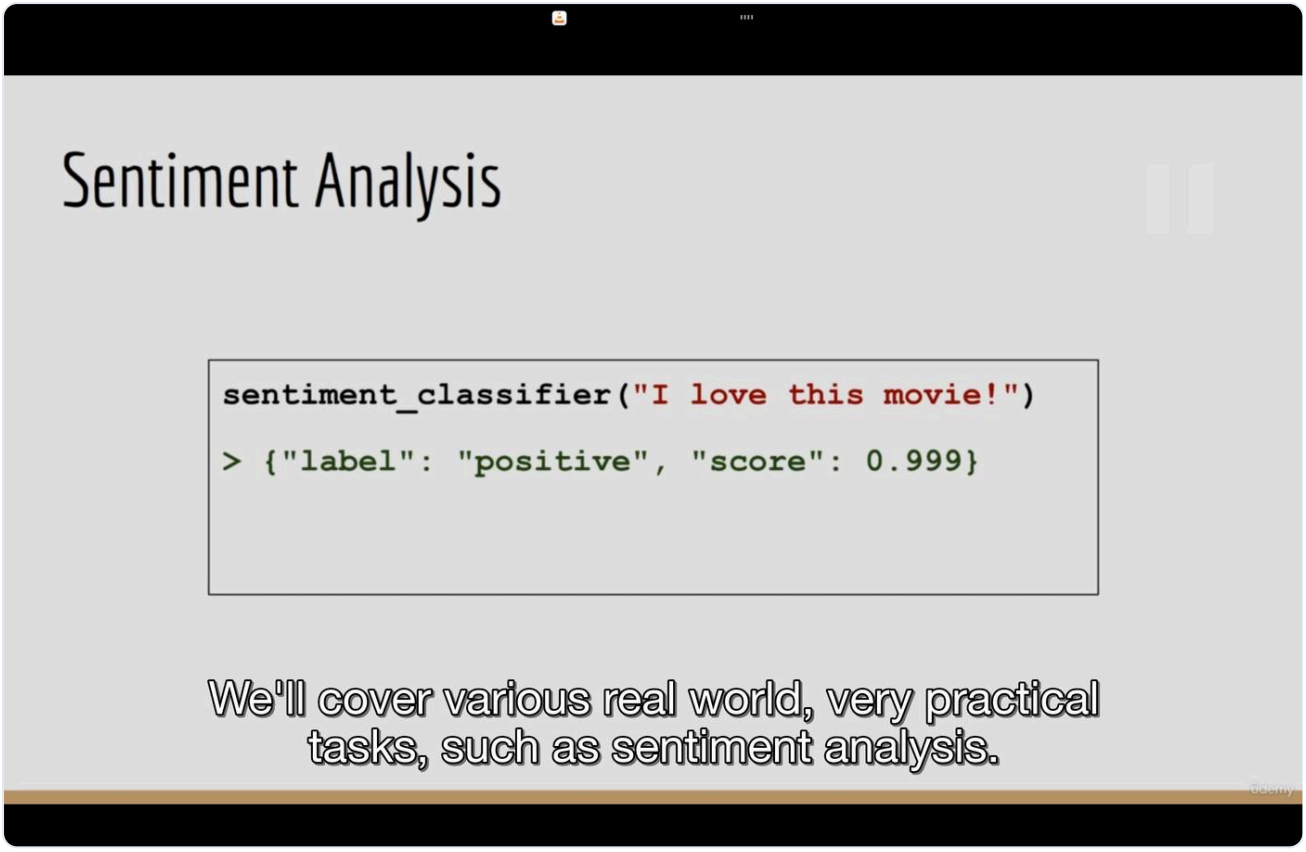

FIGURE 1.23 | Sentiment Analysis

We'll cover various real world, very practical tasks, such as sentiment analysis.

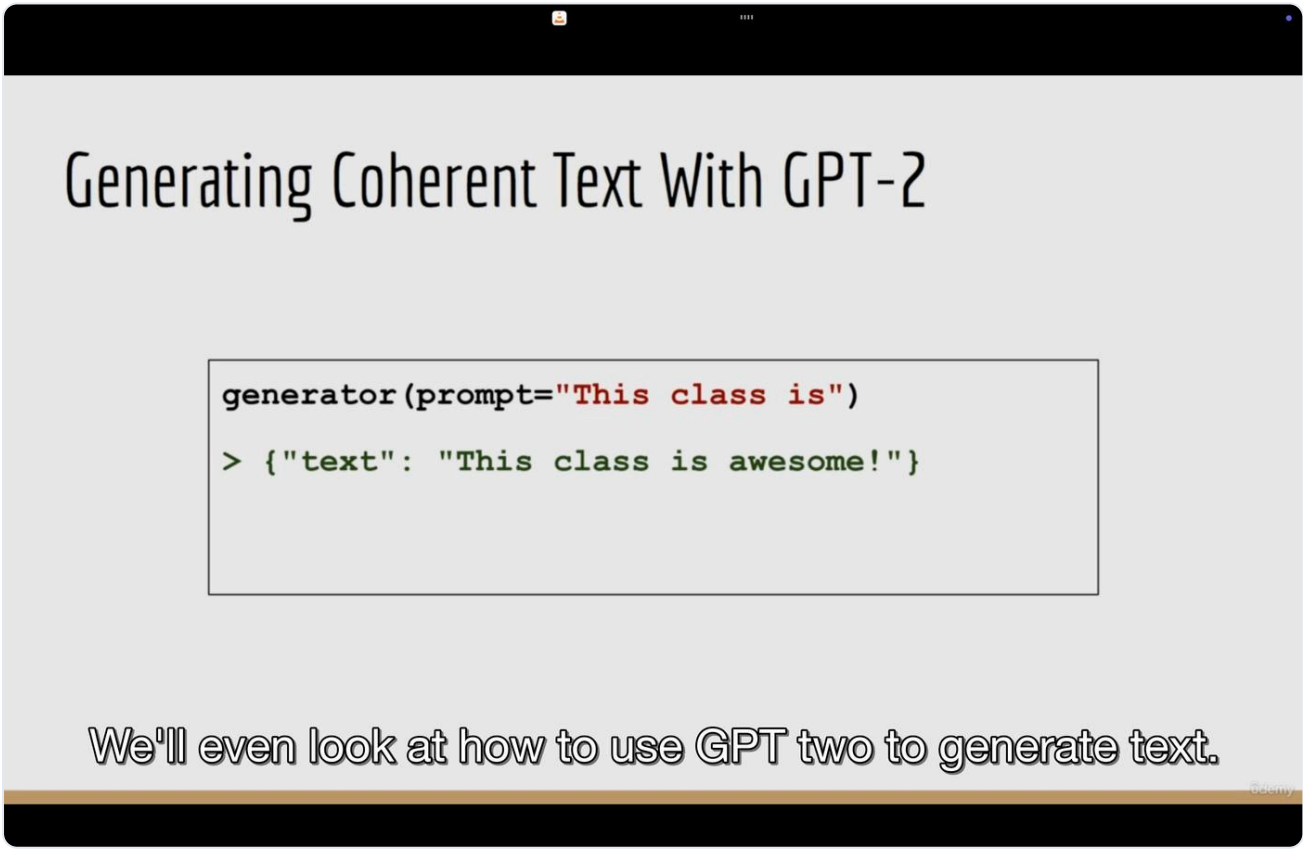

FIGURE 1.24 | Generating Coherent Text with GPT-2

We'll even look at how to use GPT-2 to generate text.

This was one of the major breakthroughs with Transformers.

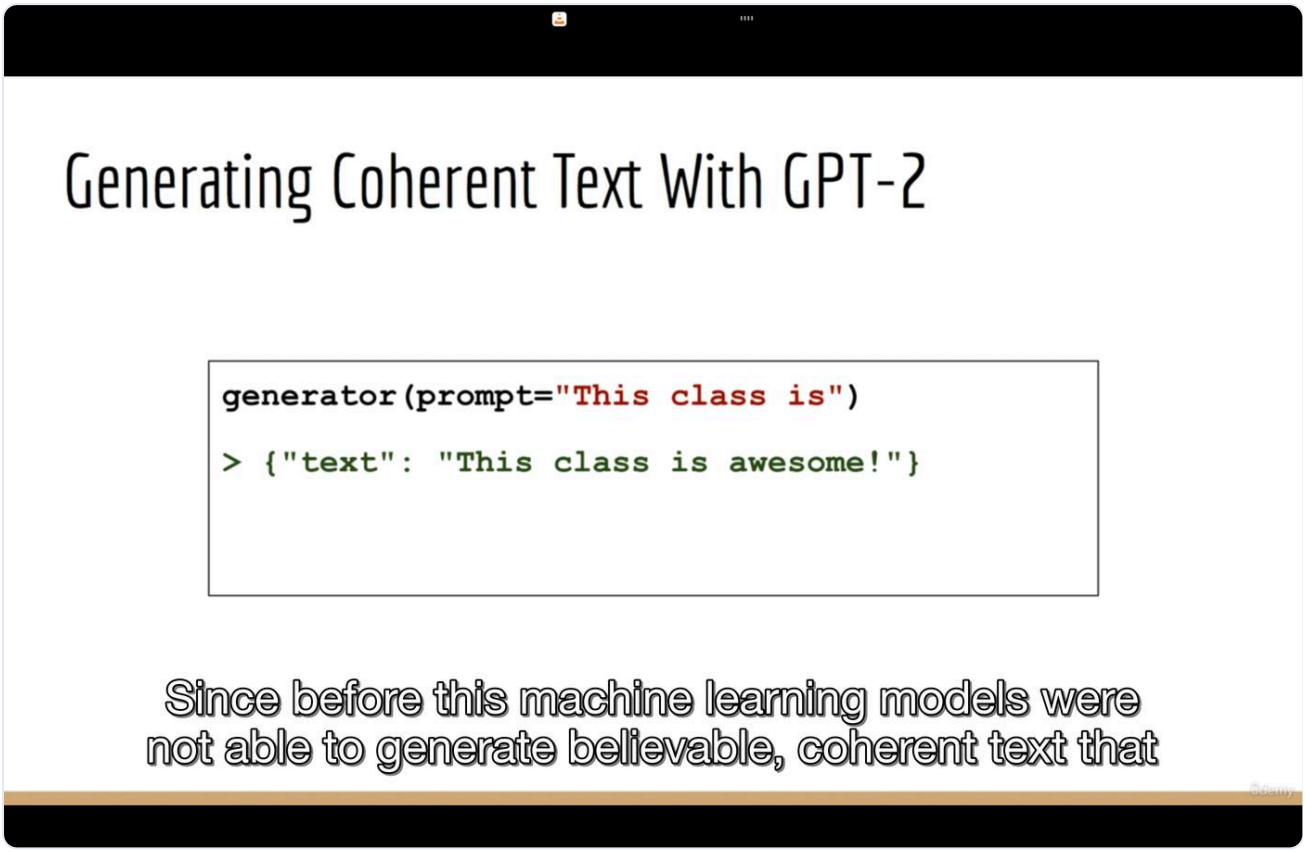

FIGURE 1.25 | Generating Coherent Text with GPT-2

Since before this machine learning models were not able to generate believable, coherent text that might be confused for what a human wrote. We'll look at how to perform classic NLP tasks like named entity recognition, text summarization and neural machine translation. Transformers have pushed the state of the art for all of these. Another interesting task is the masked language model. This might sound complicated, but in fact it's quite simple.

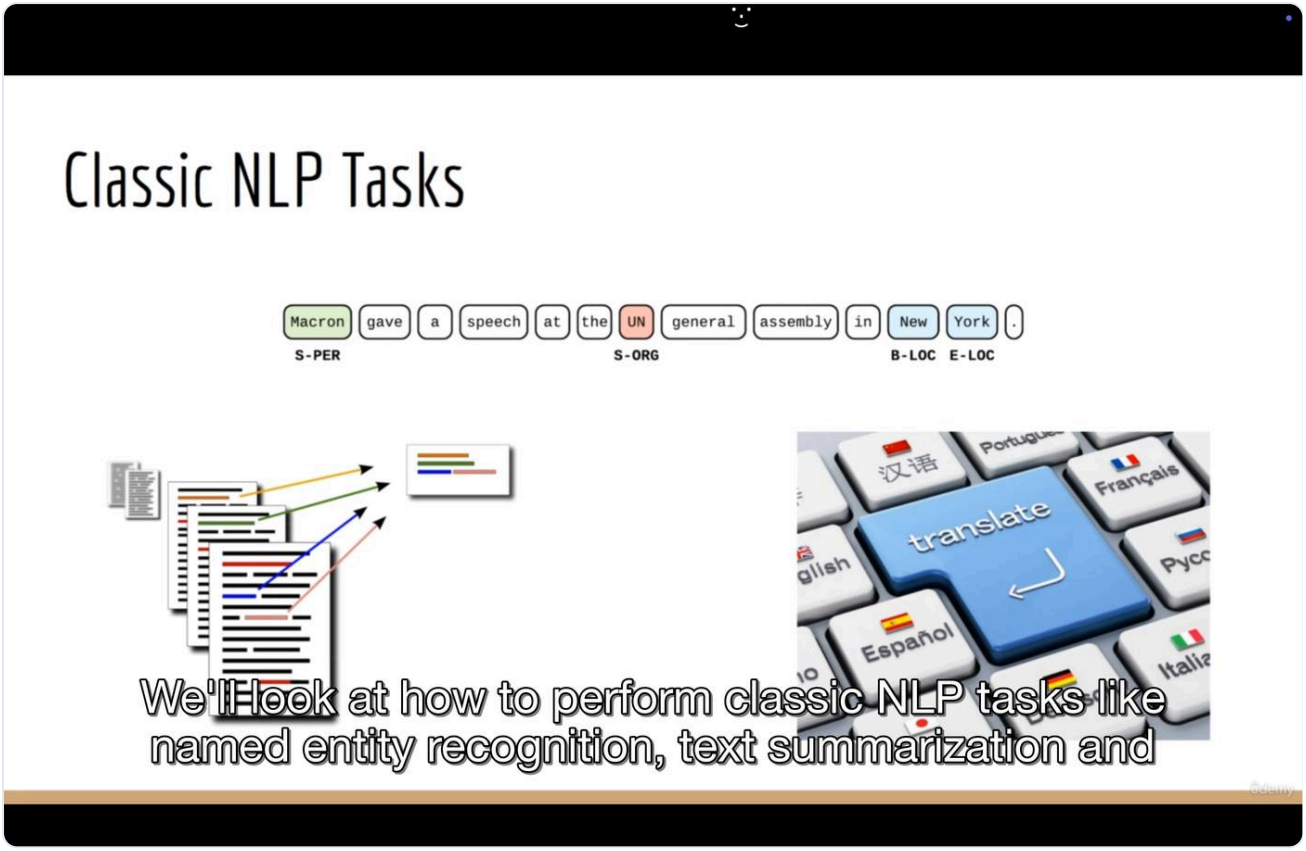

FIGURE 1.26 | Classic NLP Tasks

I introduced this idea in my first NLP course, released a few years before people even trained at Transformers to do this. I called it the article Spinner. This is because such models can be used for SEO purposes to spin existing content so that search engines can't figure out that you borrowed it from somewhere else. Now, to be clear, this is a blackhat technique, and I don't condone doing it, but nonetheless, it is something that exists.

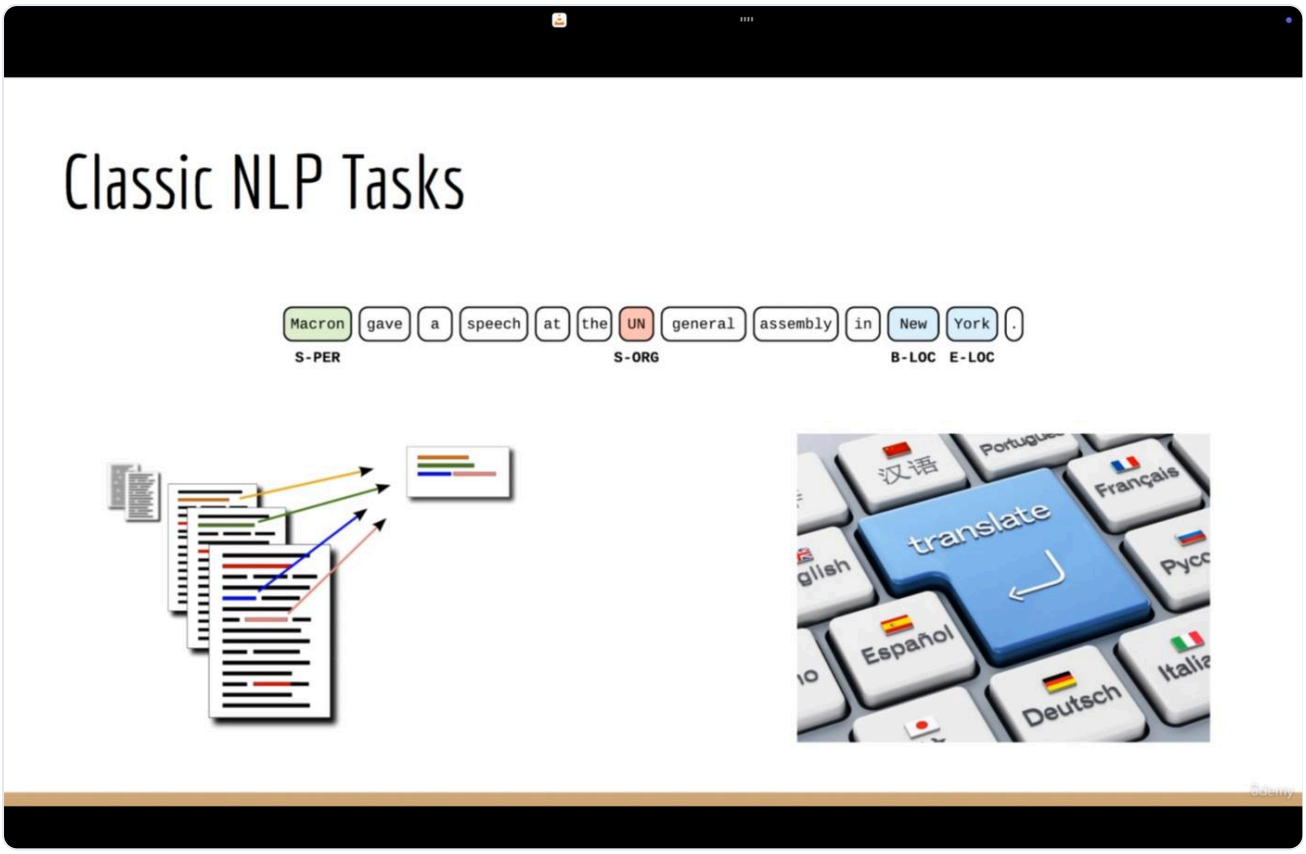

FIGURE 1.27 | Classic NLP Tasks

It is something that is very practical, and it is something that many people are willing to pay good money for.

Finally, we'll cover a few modern tasks, such as question answering, as well as zero shot classification. Question answering is a very impressive feat because it gives us the sense that the model really understands the text that it has read. You can give it a passage and then ask it a question, just like you would to schoolchildren after giving them a reading assignment. Does this mean Transformers really have achieved reading comprehension?

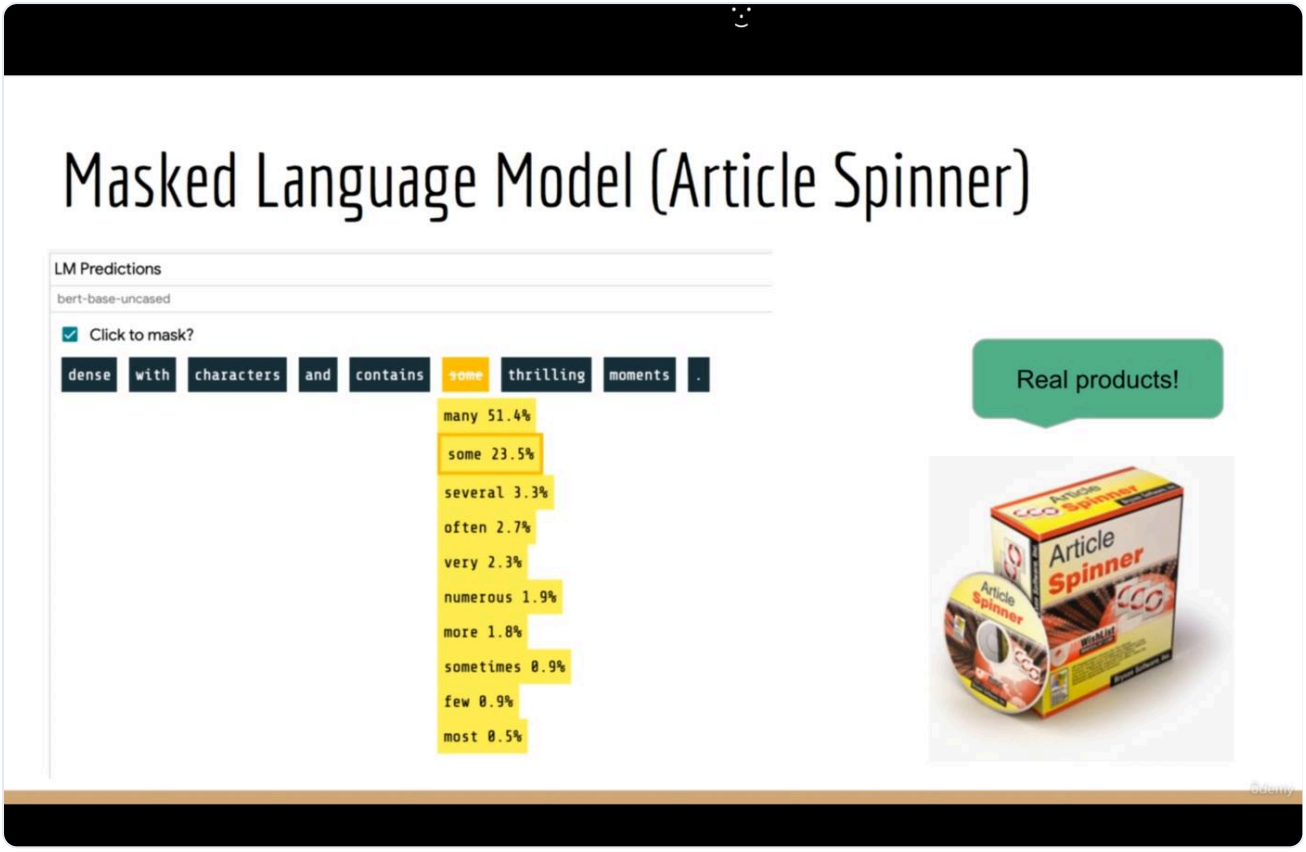

FIGURE 1.28 | Masked Language Model (Article Spinner)

I'll let you be the judge of that.

Zero shot classification is also extremely impressive. Normally, when we want to train a classifier like, say, a spam detector, we need to give it a label dataset. That means someone like you and me has to gather large amounts of data and manually assign labels to each documents one by one. As you can imagine, this is either time consuming or costly, or both.

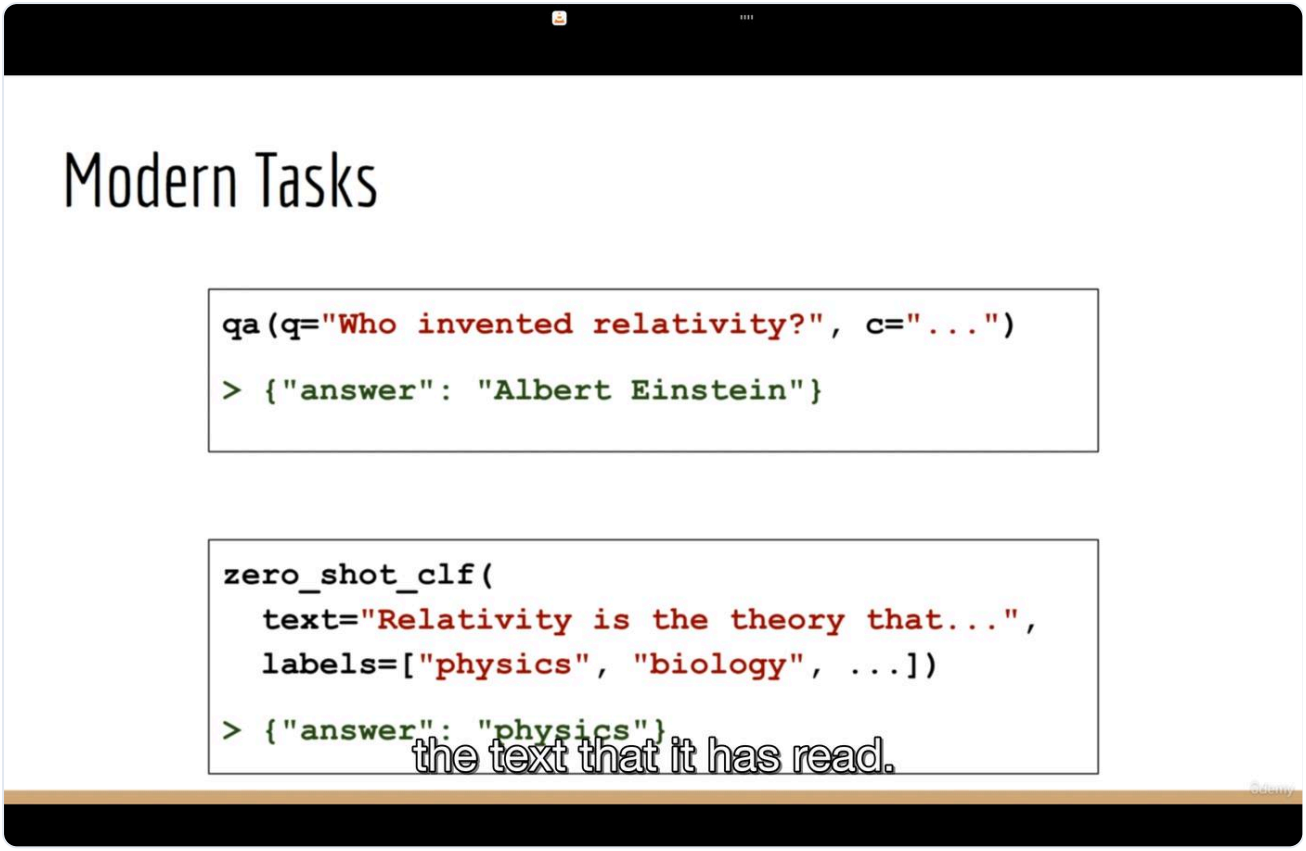

FIGURE 1.29 | Modern Tasks

With zero-shot classification, we don't have to train at all. Instead, we simply give our model some text and a set of possible labels, and it will magically understand the text well enough to predicts the correct label very often.

Now, although this all seems pretty impressive, do keep in mind that this is just the first section of the course. In the next section, we'll move on to intermediate content. This section will focus on how you can fine tune a transformer on your own data sets.

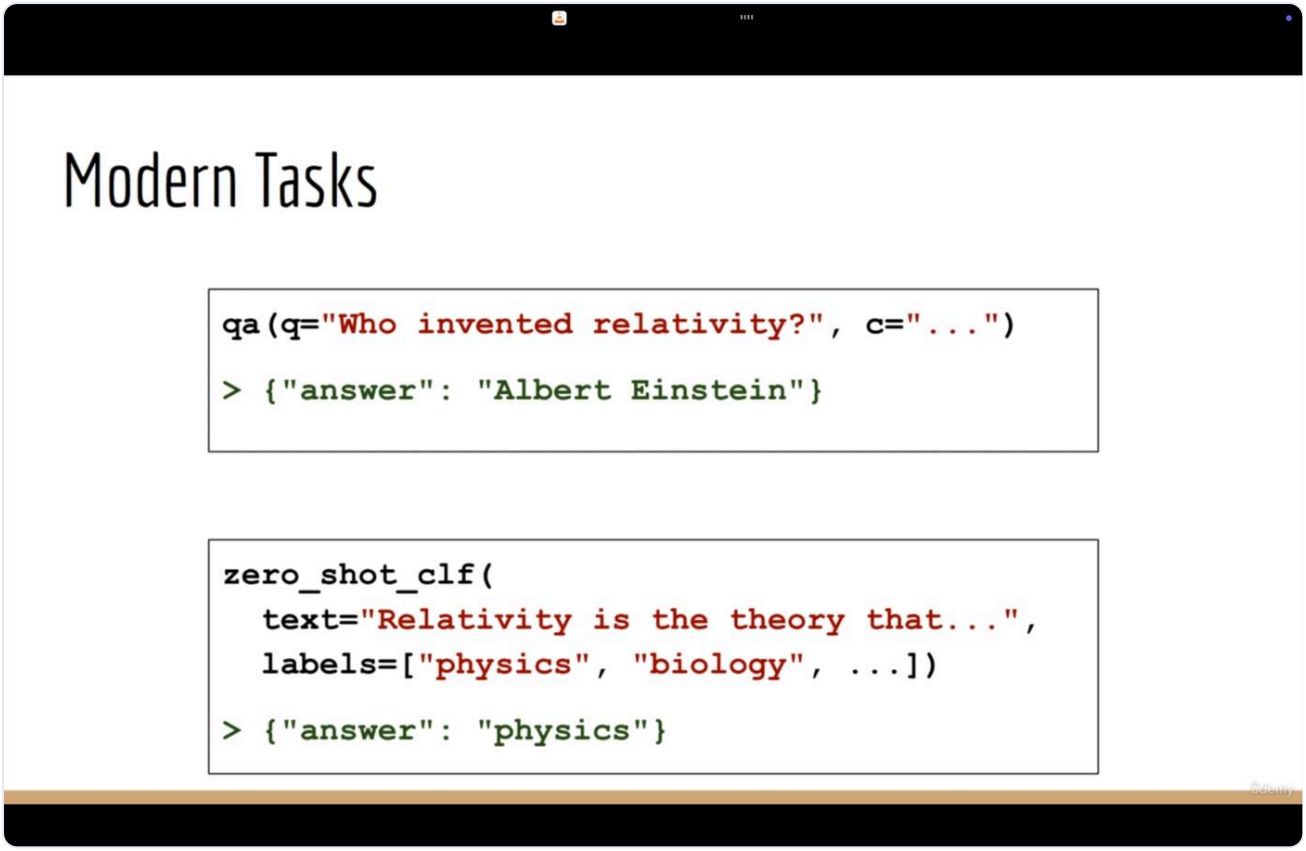

FIGURE 1.30 | Modern Tasks

By doing this, you're no longer limited to the pre-trained models from the first section, but instead you can train a state of the art transformer for any task. Transformers have borrowed a technique from computer vision known as transfer learning, which allows us to use the weights from one task and apply them to a different but similar task for state of the art performance.

In other words, the model you get from transfer learning will be superior to a model you train from scratch. We achieve this by leveraging massive amounts of unlabeled data on the internet and millions of dollars worth of training time.

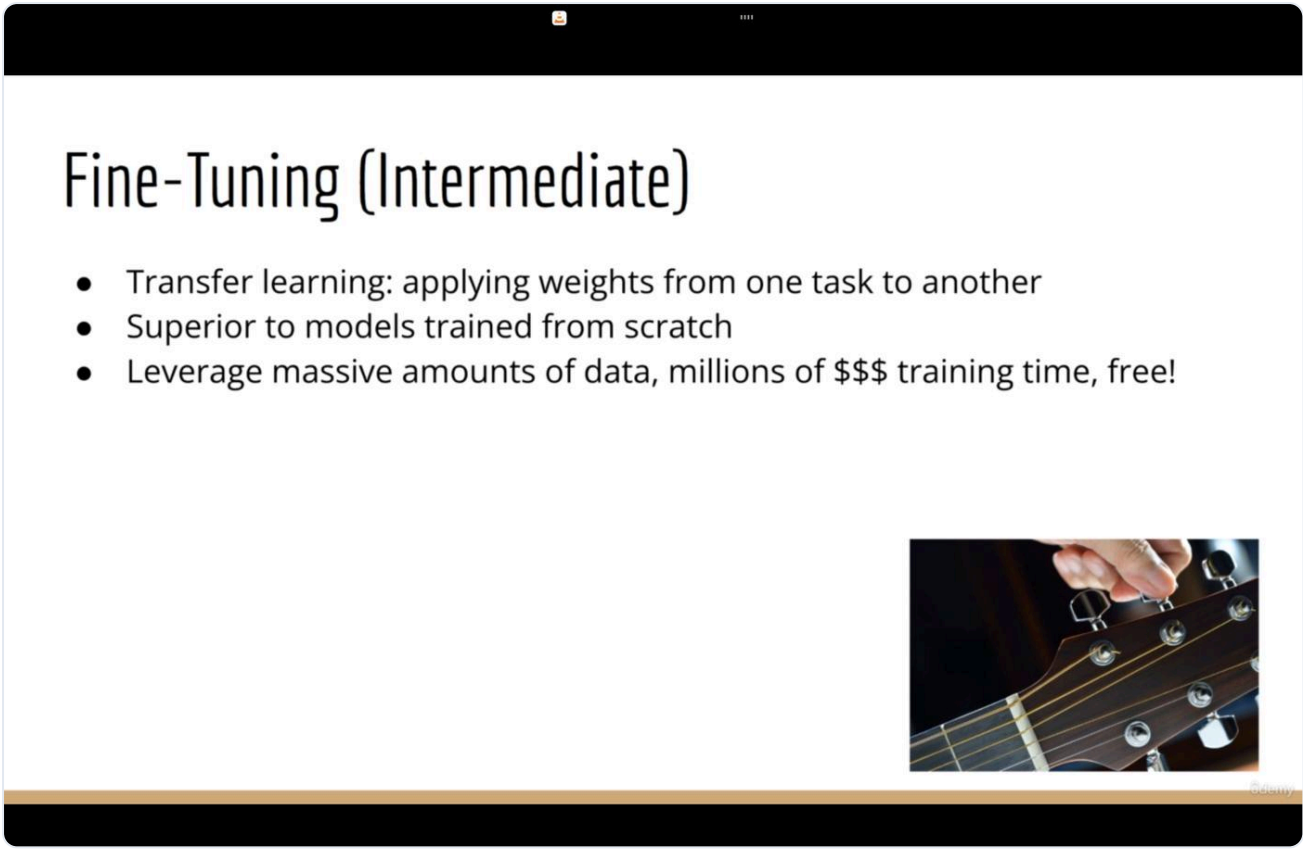

FIGURE 1.31

All for free. What this means is that you can take advantage of these neural networks that have essentially learned everything in the world and apply them to your own datasets with little effort. The researchers who built these models spent millions of dollars doing so, but you get to use them at no cost.

This is amazing in this part of the course. We'll build things up from the simplest task to more complicated tasks.

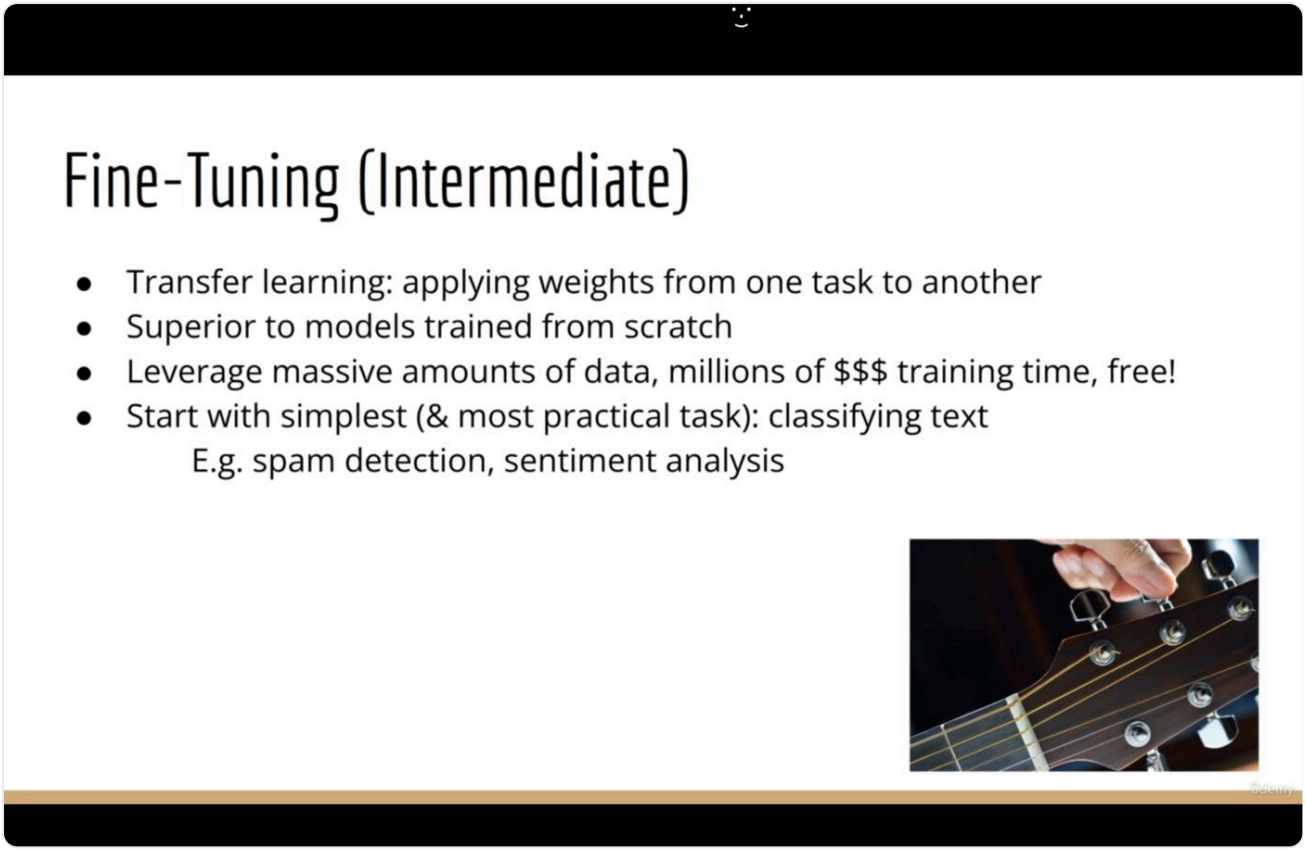

FIGURE 1.32 | Transfer Learning: Applying Weights from One Task to Another

However, don't be fooled into thinking that simple is not practical. The simplest task is probably the most practical. Specifically, we'll begin with how to fine tune a model to classify text example applications, including spam detection and sentiment analysis. In the next section, we'll look at how to fine tune a model to classify each word within a text.

Some examples are named entity recognition and parts of speech tagging.

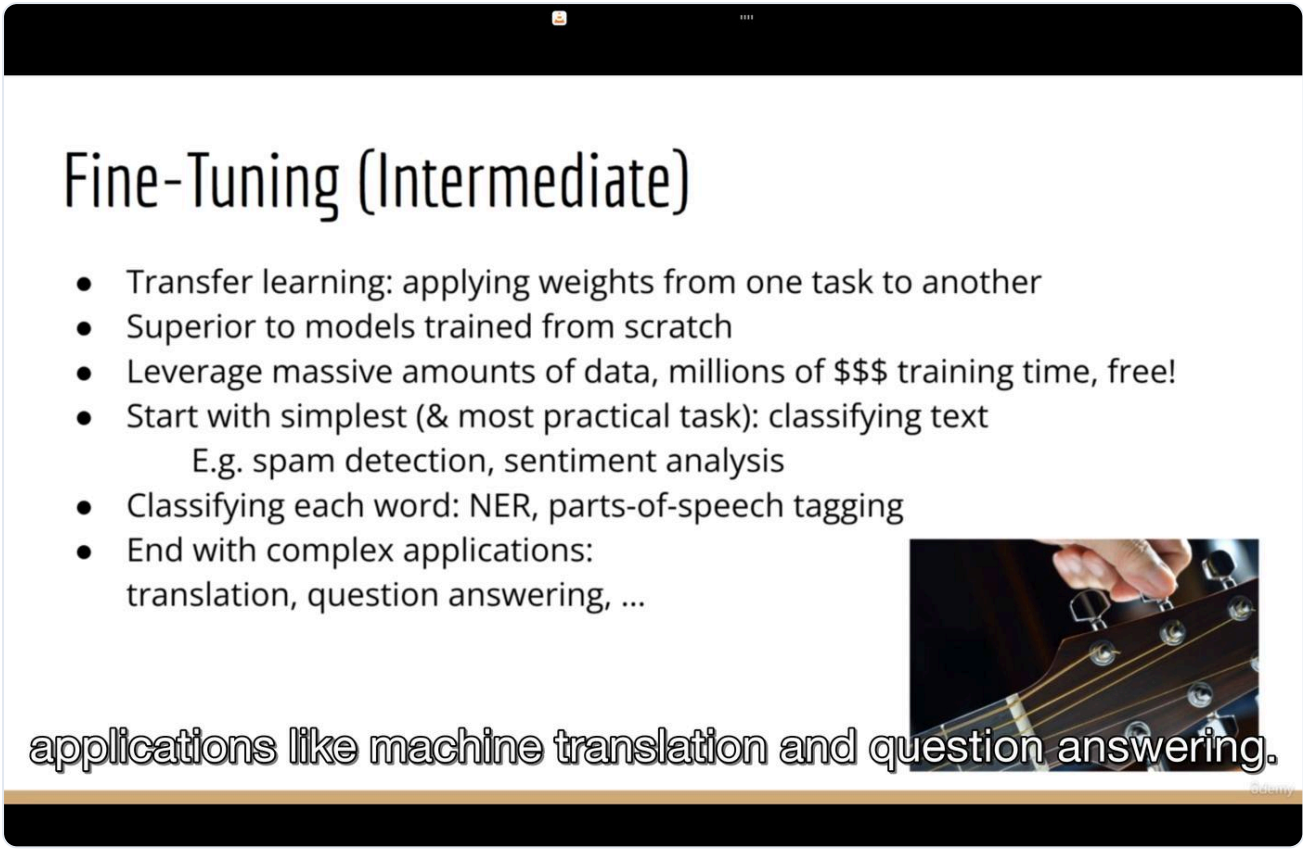

FIGURE 1.33 | Transfer Learning: Applying Weights from One Task to Another

We'll then move on to more complex applications like machine translation and question answering. In other words, you'll learn how to take the powerful transformers from the first section and make them even better. In the third part of this course, things get really interesting. And in my opinion, this is the best part.

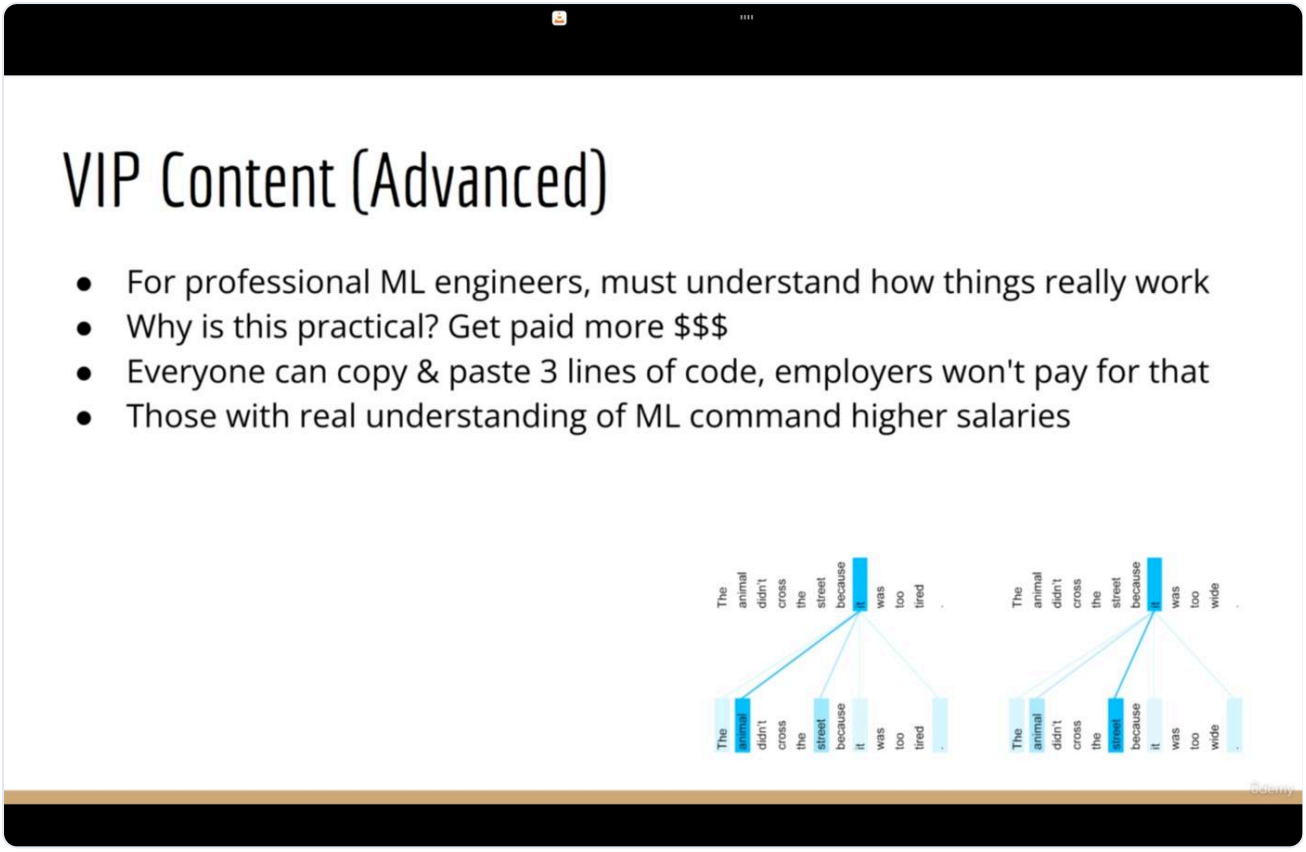

FIGURE 1.34 | VIP Content (Advanced)

This is where you get to learn how transformers actually work.

The previous sections are nice because you get to use Transformers, but for professional machine learning engineers, you often need to understand things on a deeper level. Now, don't be fooled as there's a practical component to this, too. Who wants to learn this stuff? Well, those who want the big bucks, the best and highest paying machine learning jobs require this knowledge.

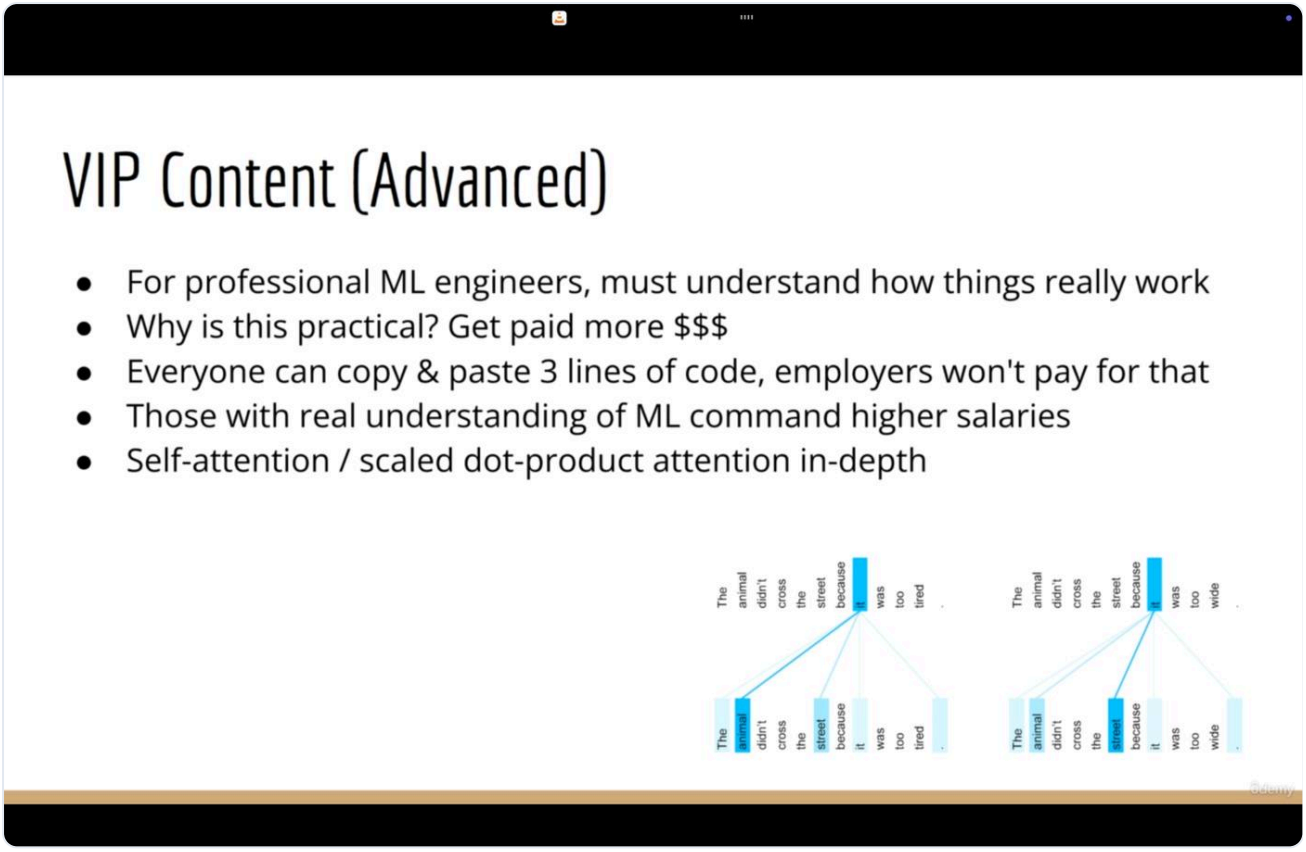

FIGURE 1.35 | VIP Content (Advanced)

Everyone can copy and paste three lines of code, and because of this, employers aren't willing to pay that much, if that's all you can do.

But those who put in the hard work to really understand machine learning at a theoretical level are able to command higher salaries. In this section, we will learn precisely how the self-attention mechanism works, which is the key component behind Transformers. We'll then extend this to build multi-head attention and then add a few more ingredients to finally build up a full transformer block. Once we have our transformer block, we'll then study specific architectures such as the encoder, the decoder and the encoder-decoder combined.

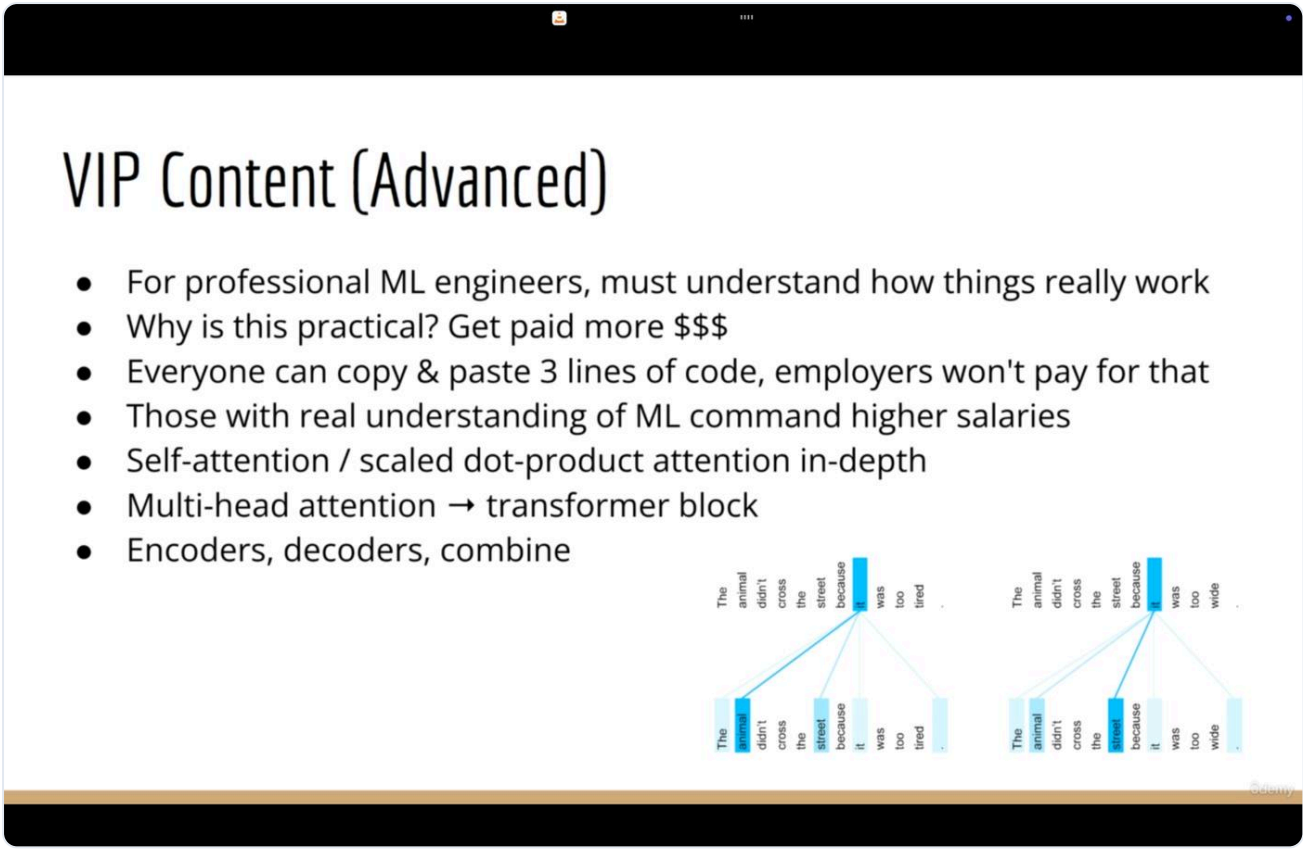

FIGURE 1.36 | VIP Content (Advanced)

We'll also look at specific models which have been prominent in the field, such as BERT and GPT. Importantly, you'll learn about how to categorize the different types of transformers so that you'll know which kind to pick for any particular task. For instance, what BERT excels at is different from what GPT excels at. And there are other things which neither BERT nor GPT can do. Understanding which type of transformer is used for which type of task is necessary if you want to use transformers in practice.

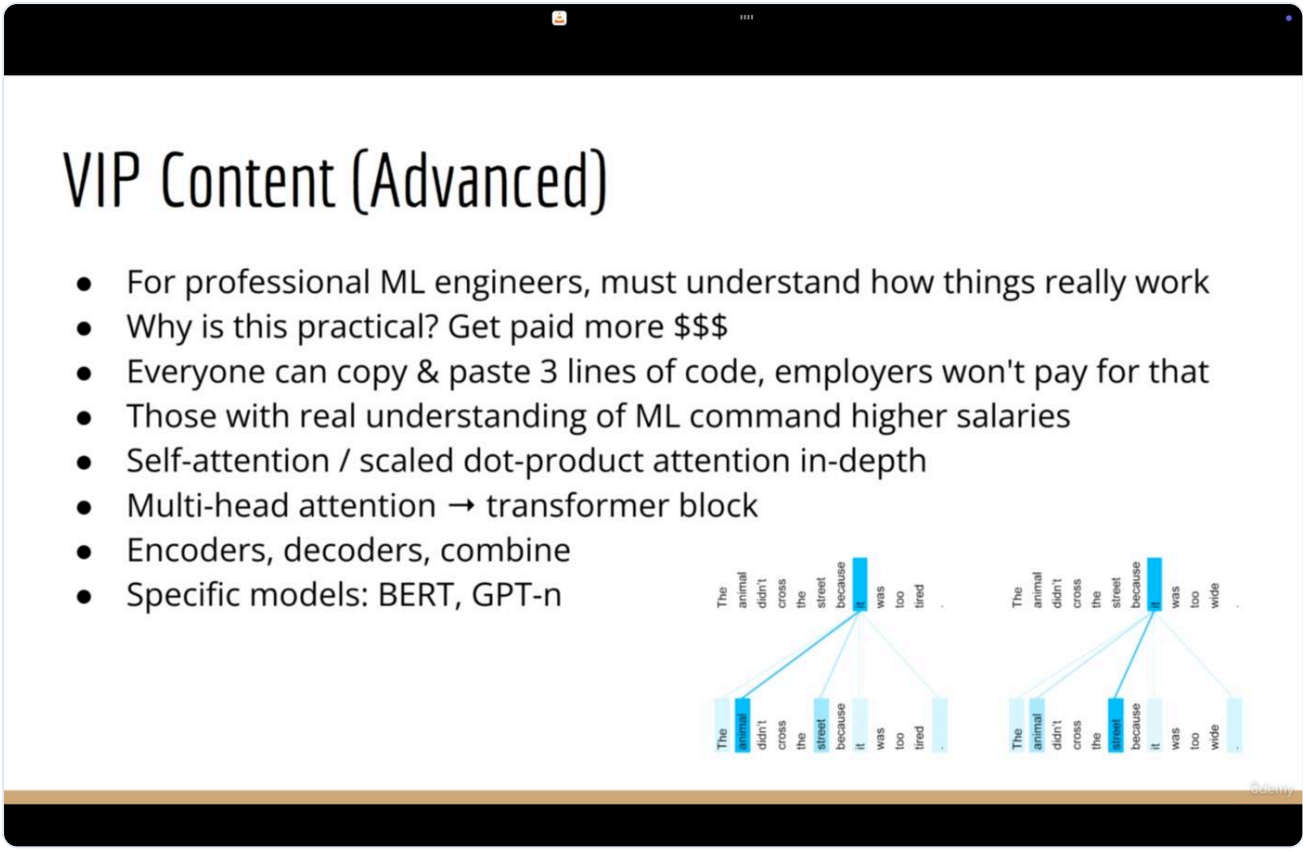

FIGURE 1.37 | VIP Content (Advanced)

After looking at the theory behind Transformers, you'll be armed with all the knowledge you need to actually implement one from scratch. Now, when I say from scratch, you should also keep in mind the correct level of abstraction. In the past, although we've implemented a simple neural networks using numpy, that would not be an appropriate tool at this stage. At this stage, when we say from scratch, we mean using a library like TensorFlow or PyTorch and possibly others.

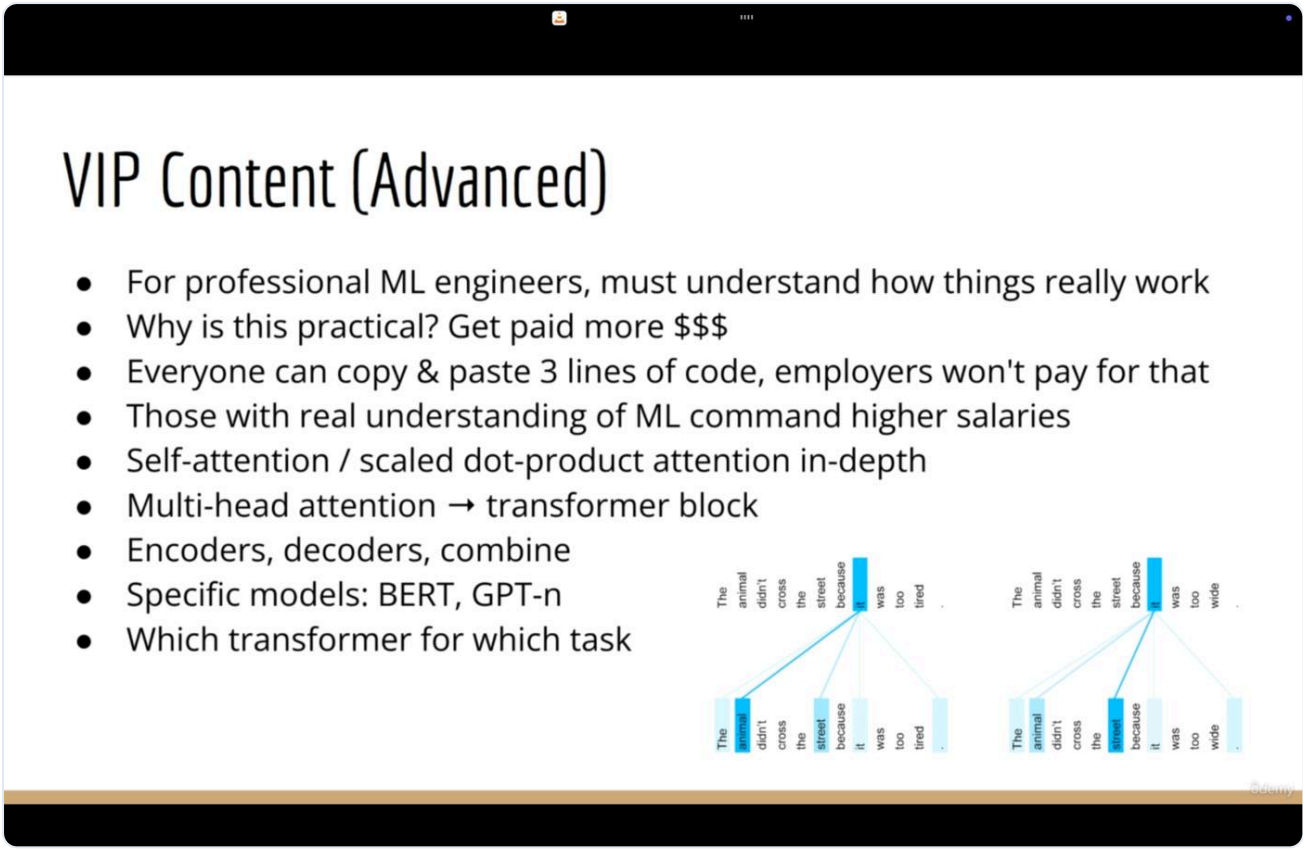

FIGURE 1.38 | VIP Content (Advanced)

And this is actually practical for two reasons. Reason number one is that transformers can really be applied to anything. They've even been applied to computer vision and computational biology with great success. Perhaps they might also be applied to computational chemistry, physics, and other important fields. But the library we're using is focused around NLP, and the PRE-TRAINED models we have are trained on text.

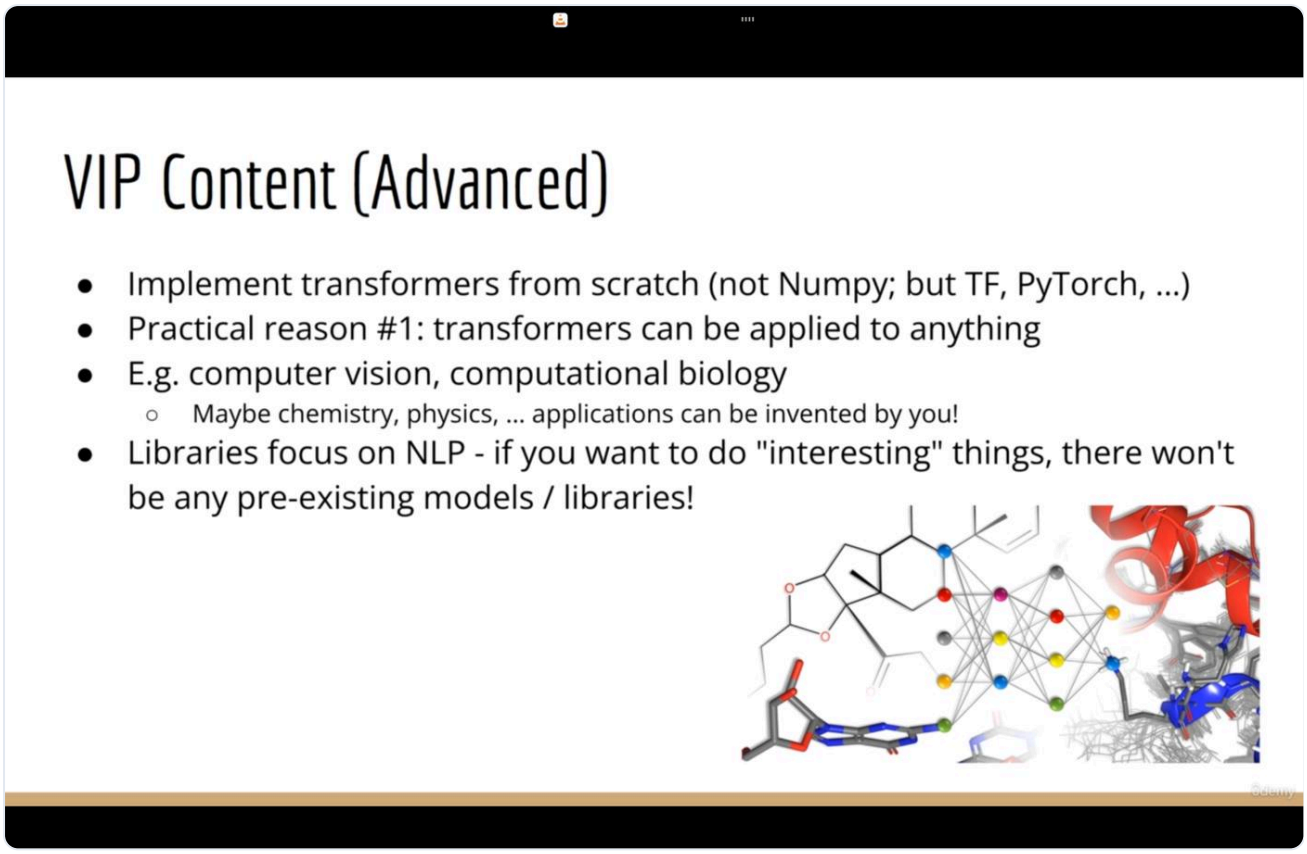

FIGURE 1.39 | VIP Content (Advanced)

So if you ever want to use transformers in other fields to advance the state of the arts, there won't be any pre-trained models for you to use.

Or maybe there will be pre-trained models, but they won't be popular enough to be part of a simple to use library like what you see in this course. You'll simply have to know how to build a transformer from scratch. Reason number two is that you don't really, truly understand Transformers until you've built one yourself. I've been saying this motto for over five years now, and it's still true.

FIGURE 1.40 | VIP Content (Advanced)

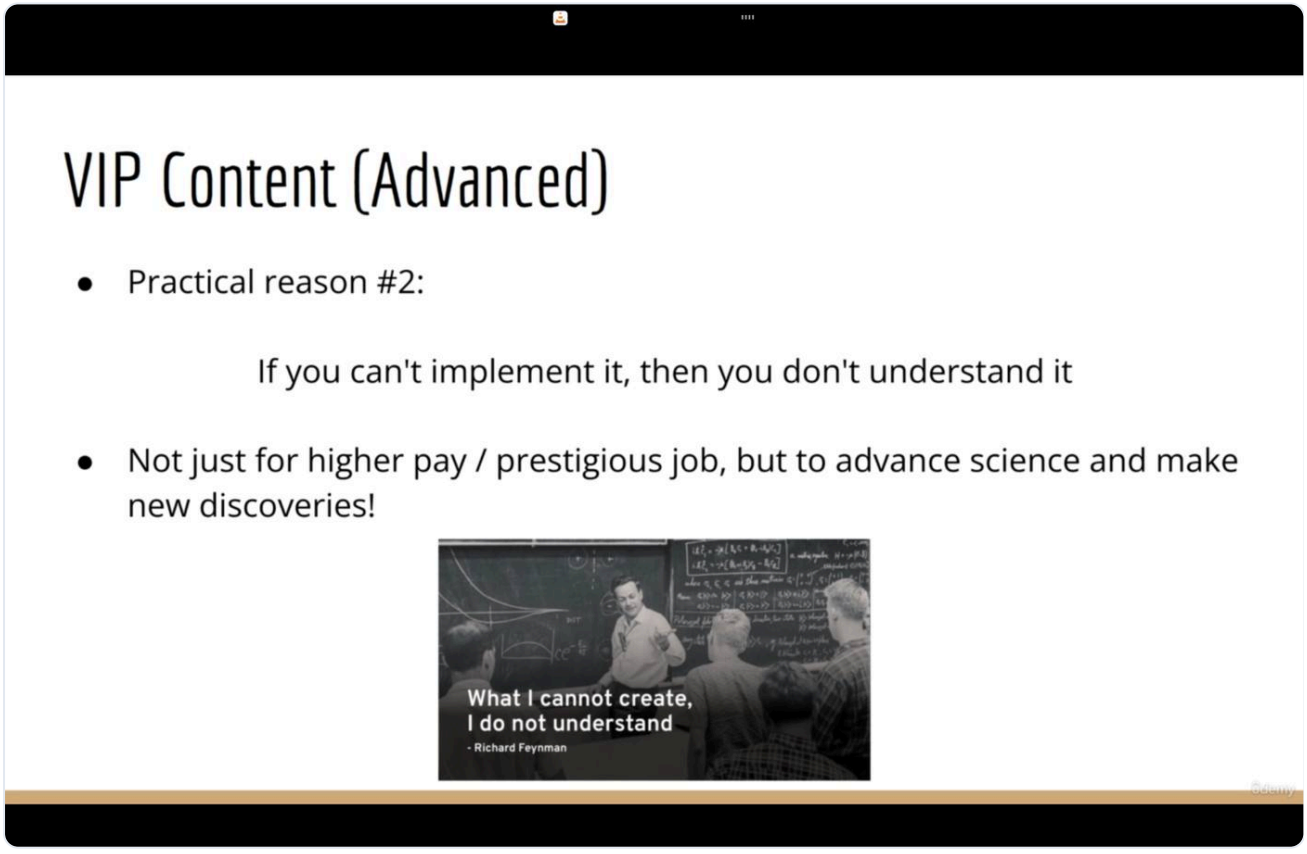

FIGURE 1.41 | VIP Content (Advanced)

If you can't implement it, then you don't understand it. The great physicist Richard Feynman said something similar.

He said, what I cannot create, I do not understand. So I hope you're convinced that understanding the theory behind Transformers so that you can build one from scratch is something worth doing, not only for higher pay and a more prestigious job, but also to advance science and to make new discoveries. Thanks for listening and I'll see you in the next lecture.

SECTION 02

Getting Setup

1 · Get Your Hands Dirty, Practical Coding Experience, Data Links

So in this lecture, I want to talk about how to practice what you've learned in this course and how to make sure you are able to carry out the tasks you've been taught to do. Now, firstly, as mentioned in the introduction to this course, this is a hands on course.

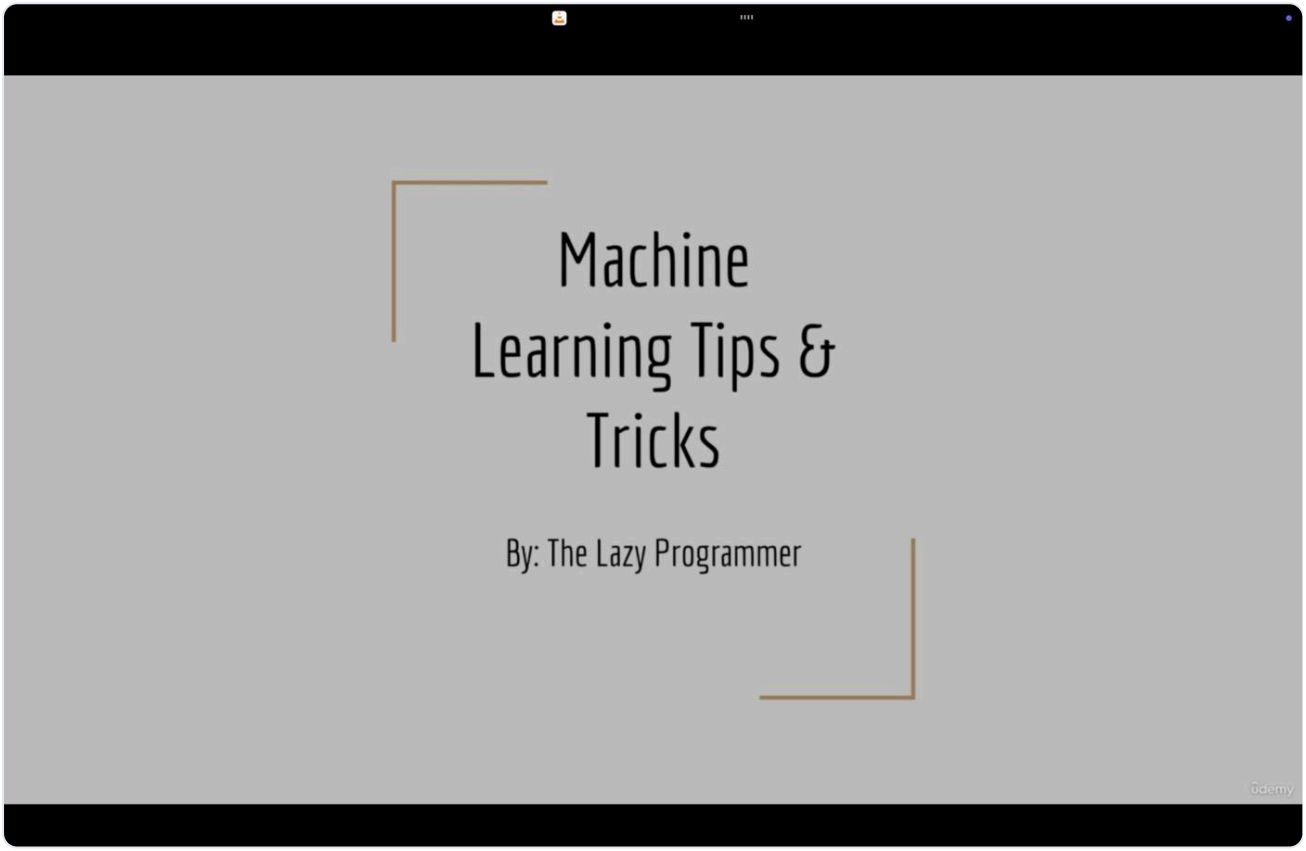

FIGURE 2.1 | Learning Tips &

So what does this mean? This means that in this course you are strongly encouraged to get your hands dirty.

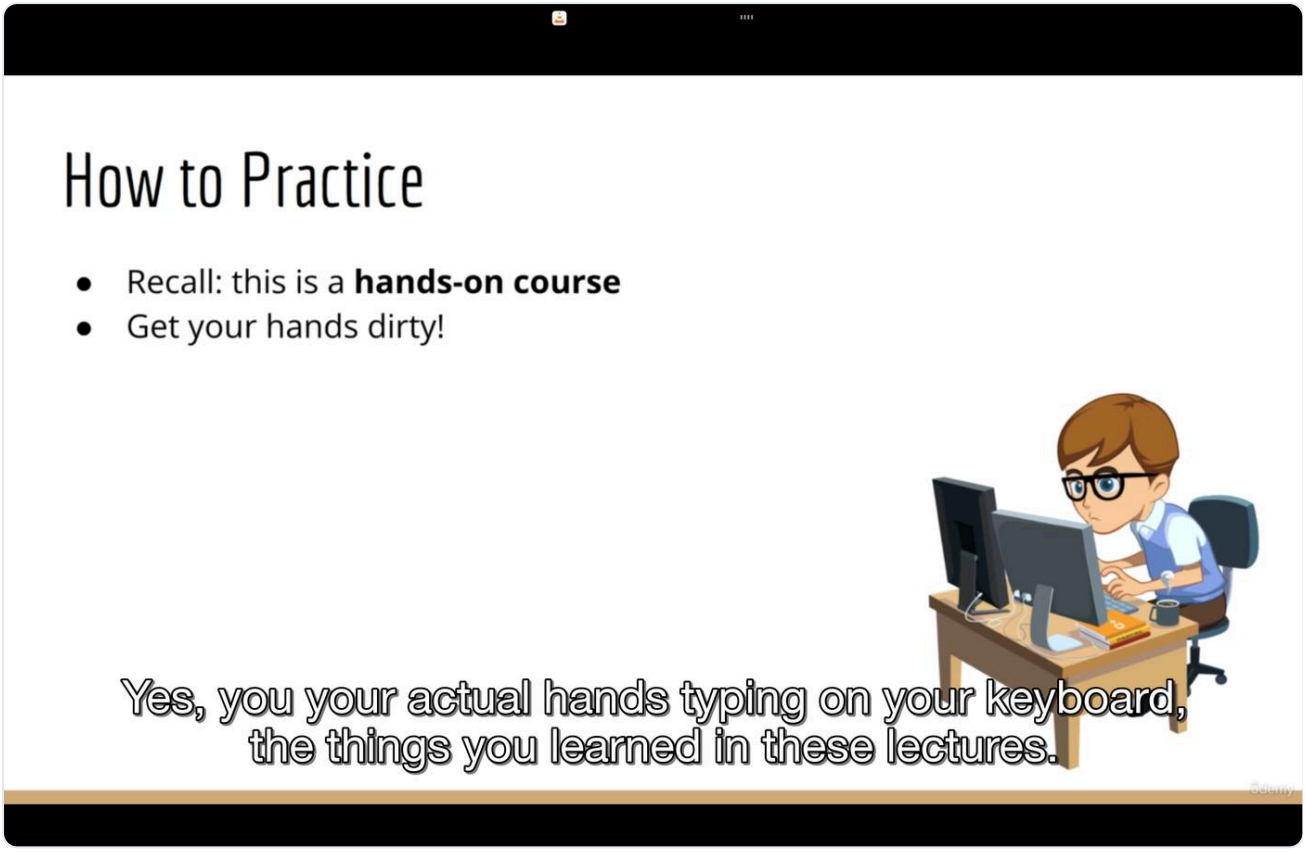

FIGURE 2.2 | How to Practice

Yes, you your actual hands typing on your keyboard, the things you learned in these lectures.

So a few of you might be wondering, how does this work? So let's talk about how this works. Basically, the lectures in this course will proceed in a two step pattern. The first step will always be to discuss the concepts. What's going on?

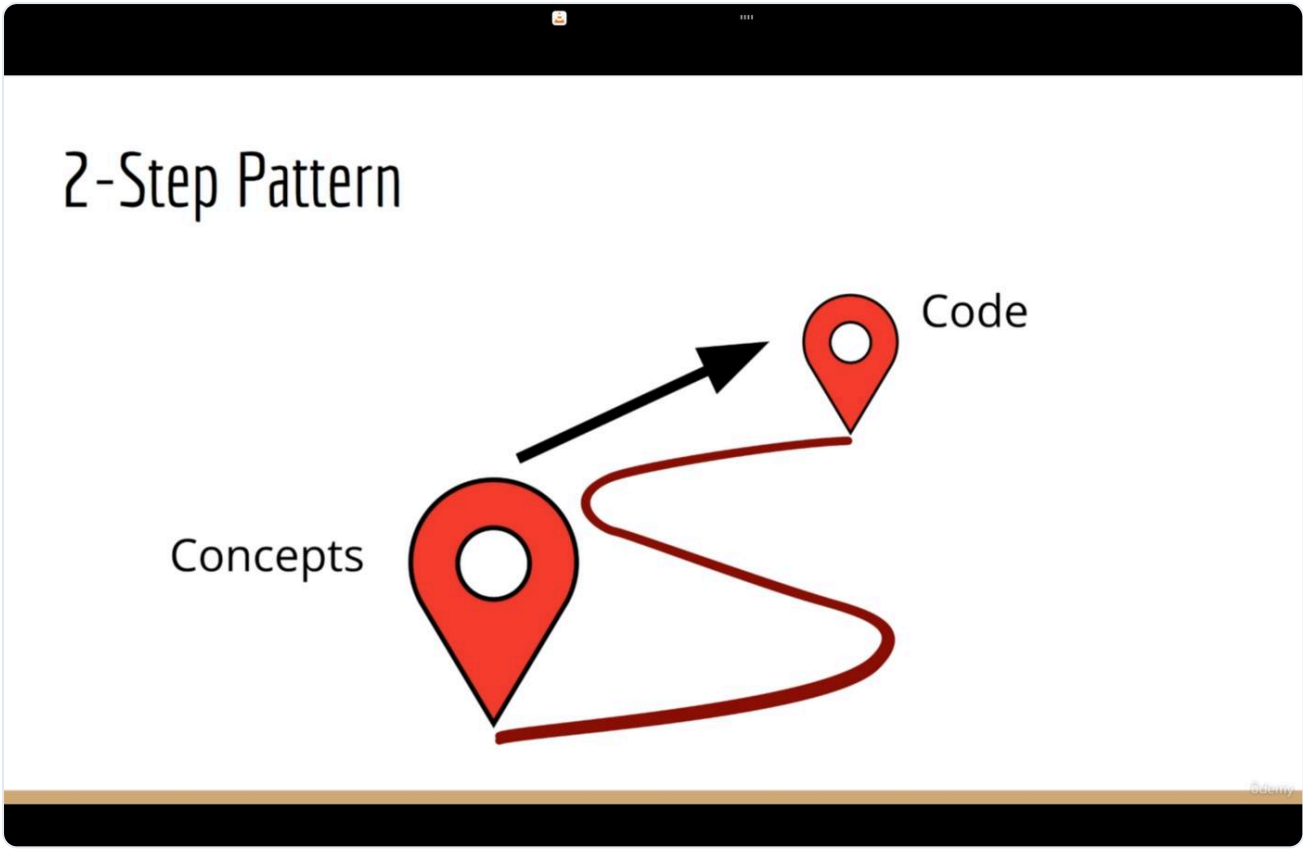

FIGURE 2.3 | 2-Step Pattern

What are we trying to accomplish, how does it work? And so on and so forth. That's the first step. The second step will be to implement those ideas in actual working code. This is where getting your hands dirty comes into play.

You see, what I want you to do is when you see these lectures, I want you to open your own a notebook or your coding environment and start typing out the stuff you see so that you learn how to implement these concepts yourself. Too often what I see is students just downloading notebooks and hitting play, and what ends up happening is that they think they know what they're doing, but in practice they are completely incapable of writing any code themselves.

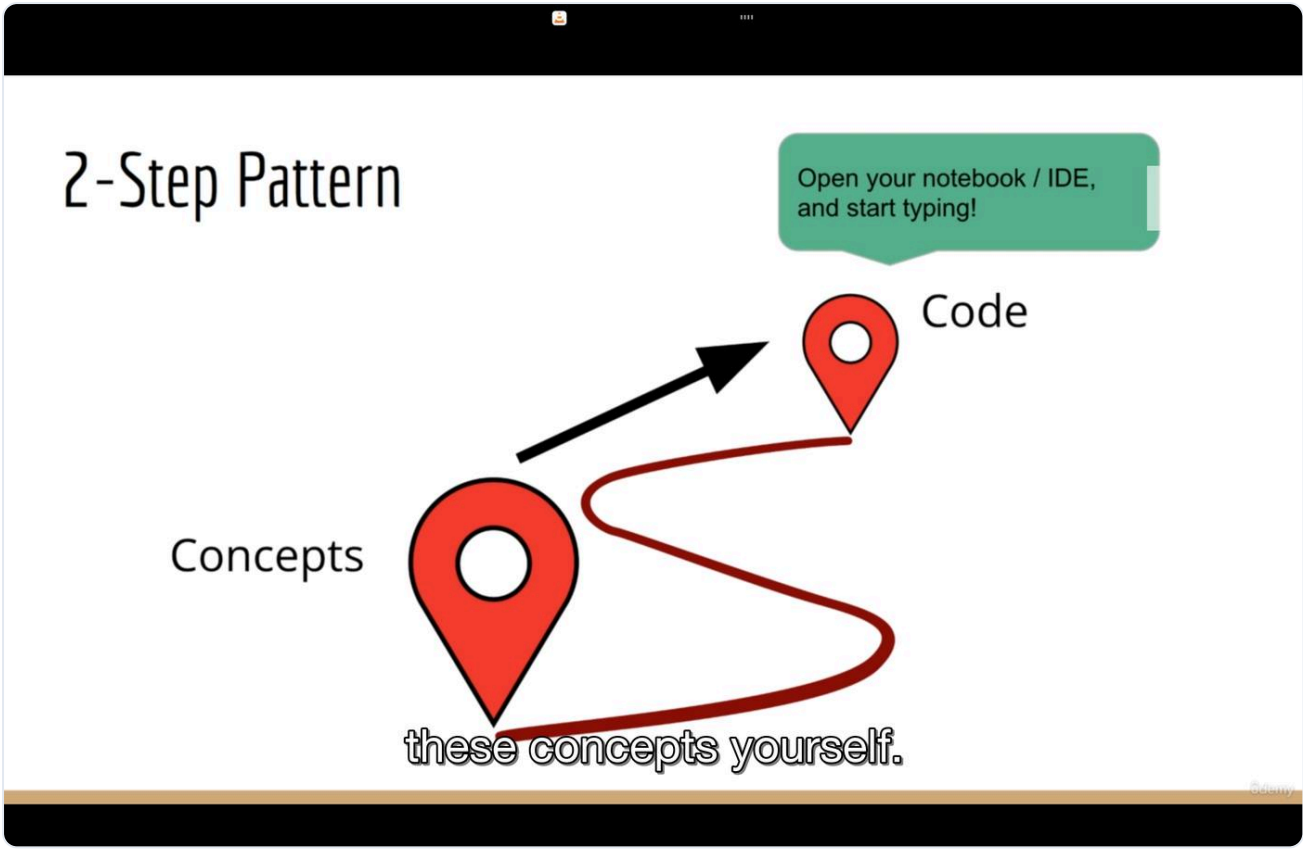

FIGURE 2.4 | 2-Step Pattern a

And if you ask them, Do you know how to write this code, they will say yes. And then if you ask them, well, how much of this code did you actually write? The answer is none of it. And, you know, if you ever get a job or work in a project, that's what you have to do. You need to write actual code, not just believe in your mind that it can be done. The real world requires you to code, not to just watch other people code or download code that's already been written.

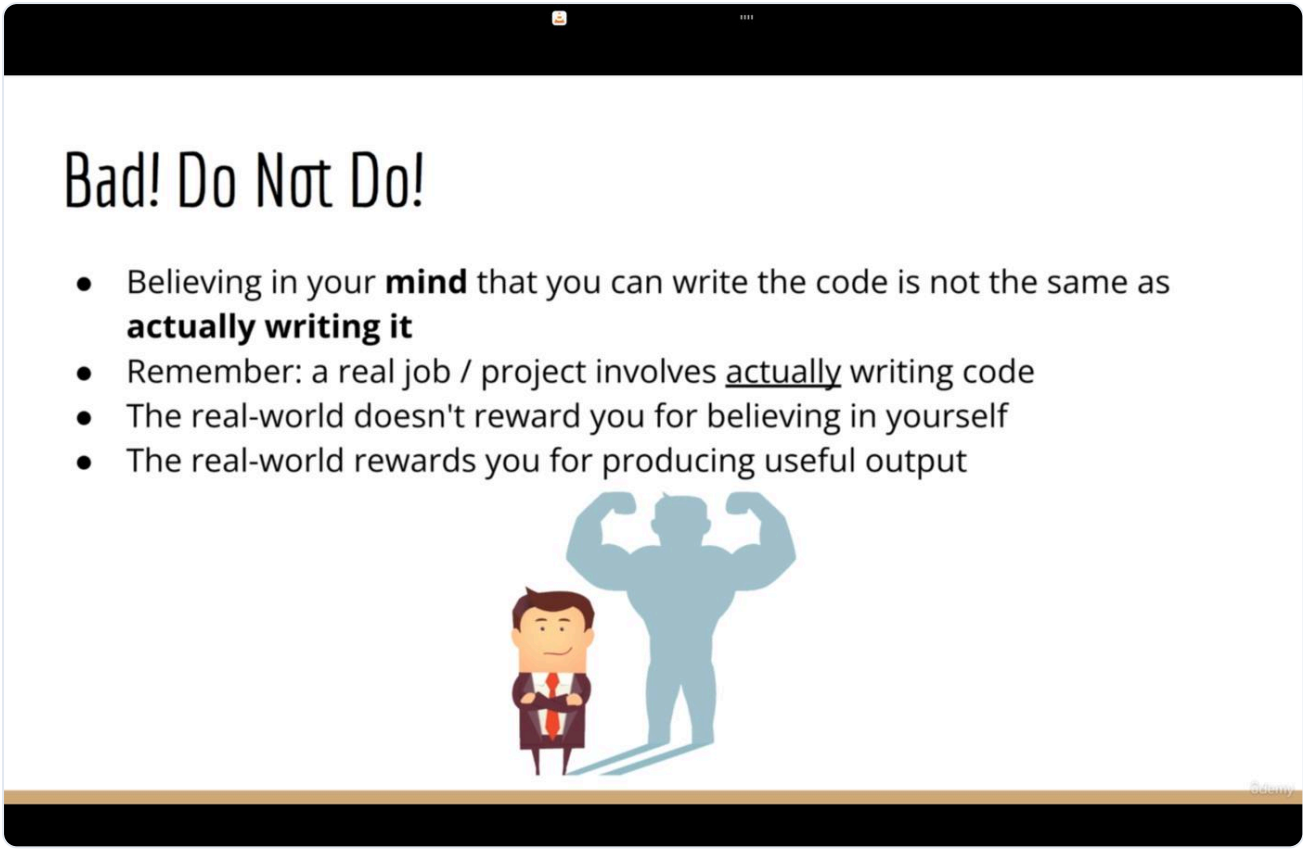

FIGURE 2.5

The real world doesn't reward you for simply believing in yourself. The real world rewards you when you produce useful outputs. So that's what you're learning to do in this course. Produce useful outputs, not just thinking you can. Now, this course is mostly an API focused course. This is opposed to a more theoretical course where you might be taught an algorithm and then be expected to implement that algorithm from scratch.

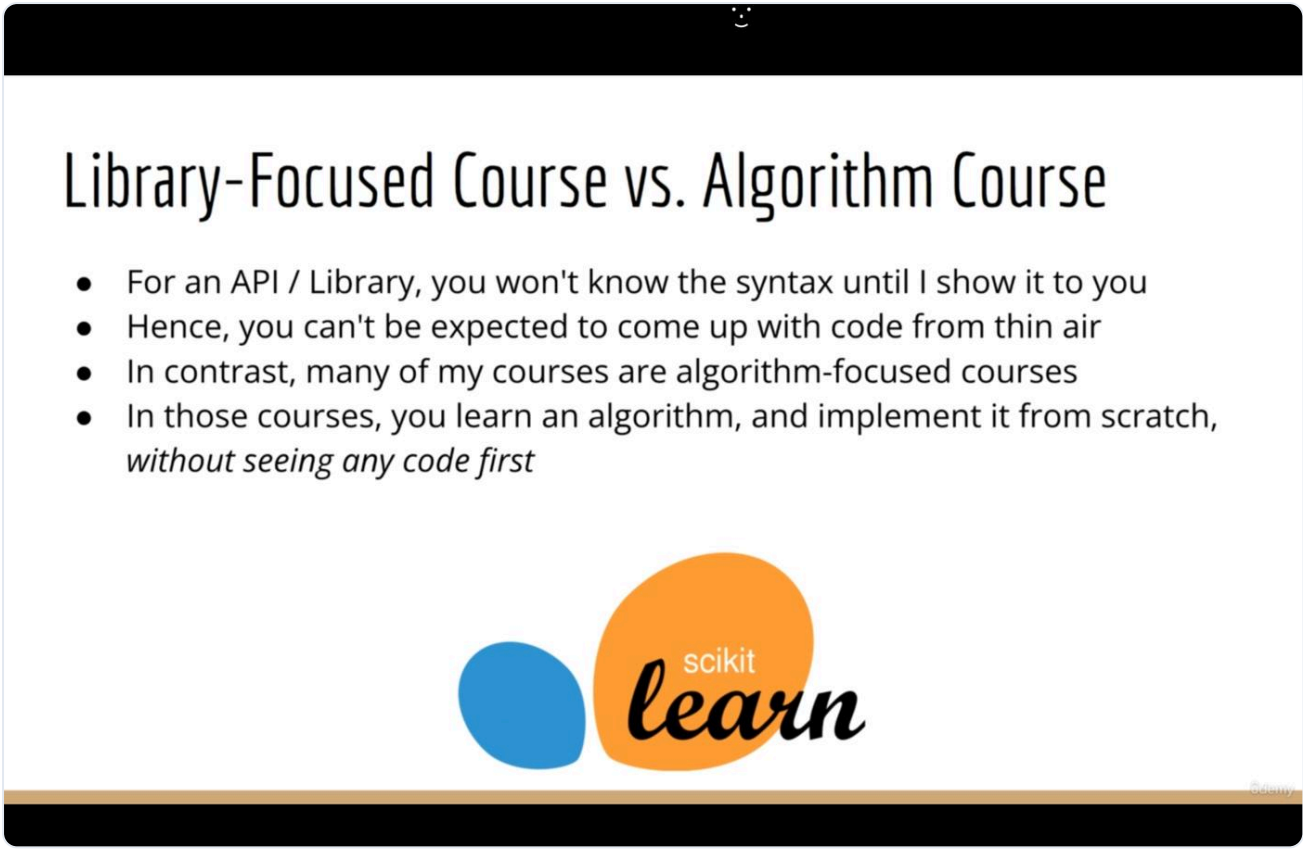

FIGURE 2.6 | Library-Focused Course vs. Algorithm Course

For this course being mostly an API focused course, you're not going to know the syntax until I actually show it to you.

So in that sense, you will need to see the code as you code. I think this is important to mention since many of my courses are courses where you are taught algorithms and then you implement those algorithms as an exercise.

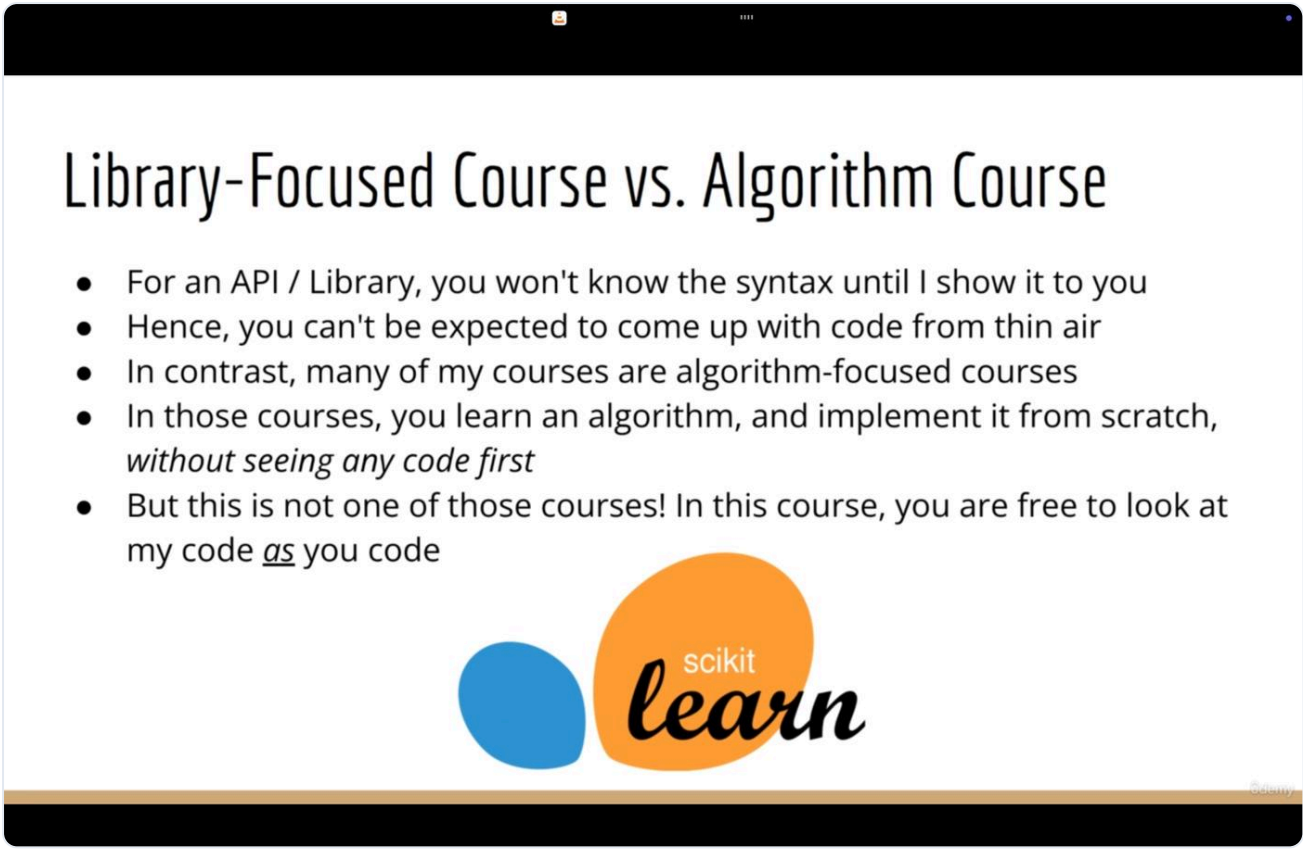

FIGURE 2.7 | Library-Focused Course vs. Algorithm Course

This, however, is not one of those courses. In this course, you will need me to show you the syntax which you will be shown in the lectures. Now you might be wondering, isn't coding a mental thing? Why do I actually physically have to code?

Isn't it enough to just see some syntax with my eyes? And, well, the answer is no. This has to do with a concept called muscle memory by typing you're building muscle memory, which will help you produce similar code in the future.

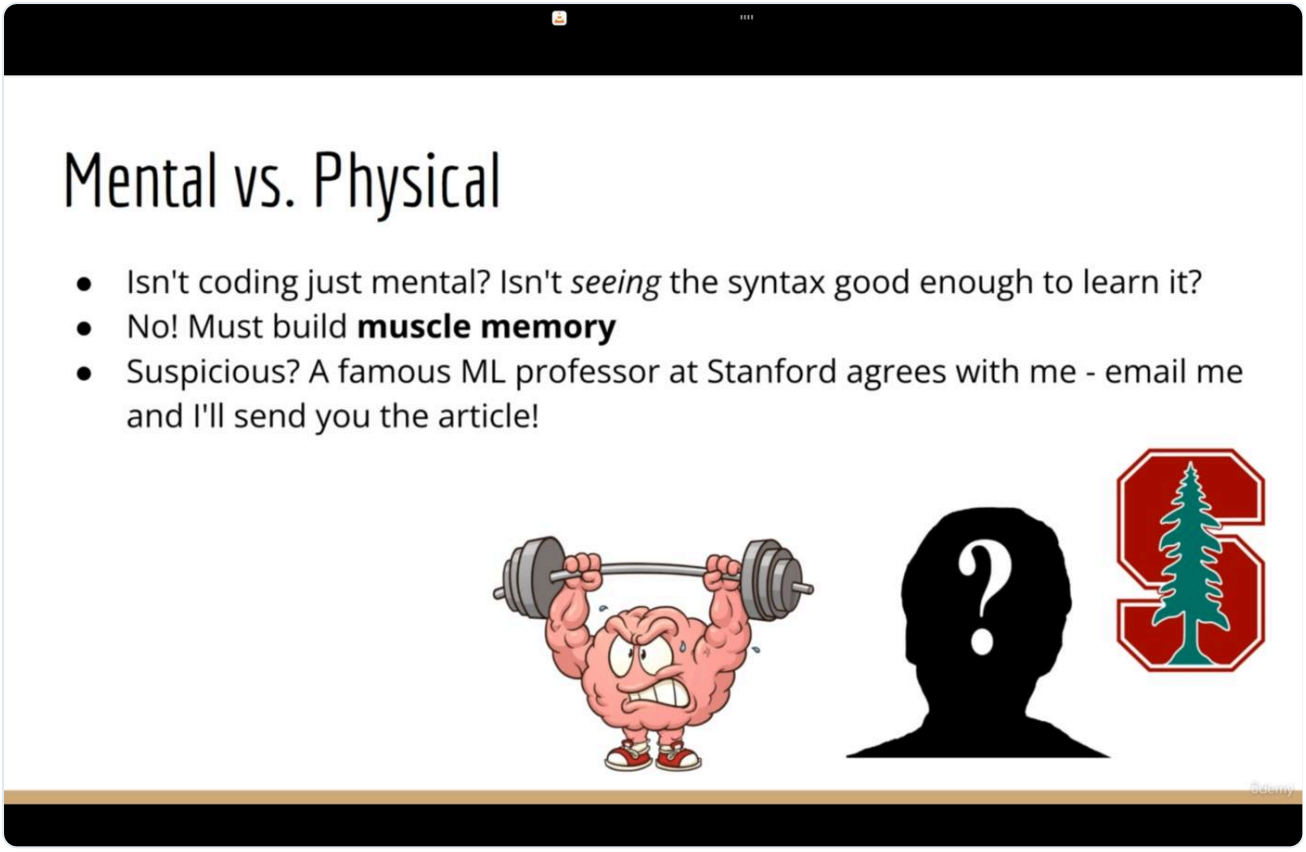

FIGURE 2.8 | Mental vs. Physical

Now, for those of you who might be suspicious of this claim, please email me and I'll send you a link from a famous machine learning professor from Stanford who recently wrote about this. It is simply undeniable that this is good for your learning experience, and if you don't trust me, then you can at least trust this Stanford professor who is very well known in the machine learning community.

Now, here's one thing I do not want you to do, which I've seen some students do in the past. And guys, please don't do this.

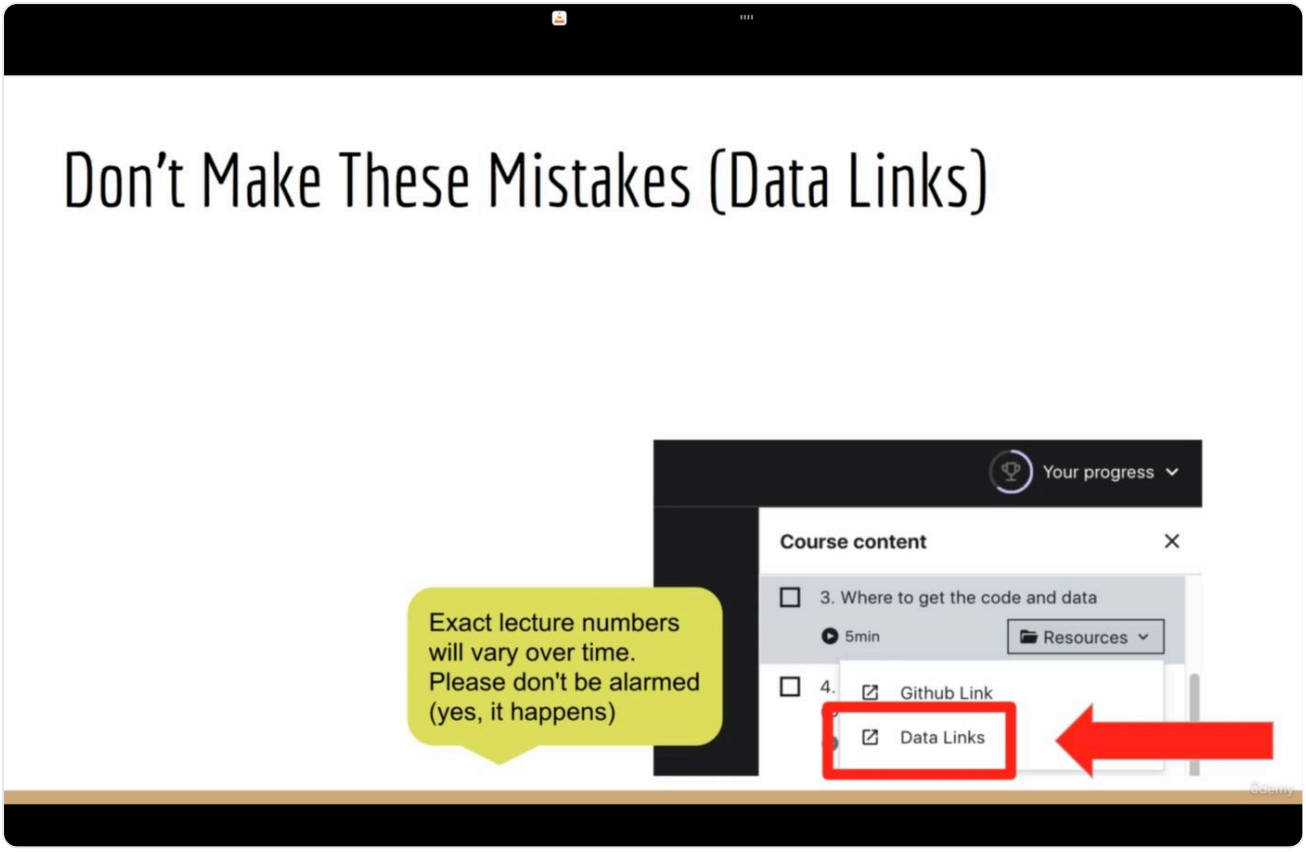

FIGURE 2.9 | Don't Make These Mistakes (Data Links)

It's a very silly. And also, if you do end up doing this, I'm going to know that you didn't pay close attention to this lecture. So what is this big mistake that I don't want you to do? I don't want you to type any data links you see by hand. And I don't want you to Google search the links for the data sets in this course.

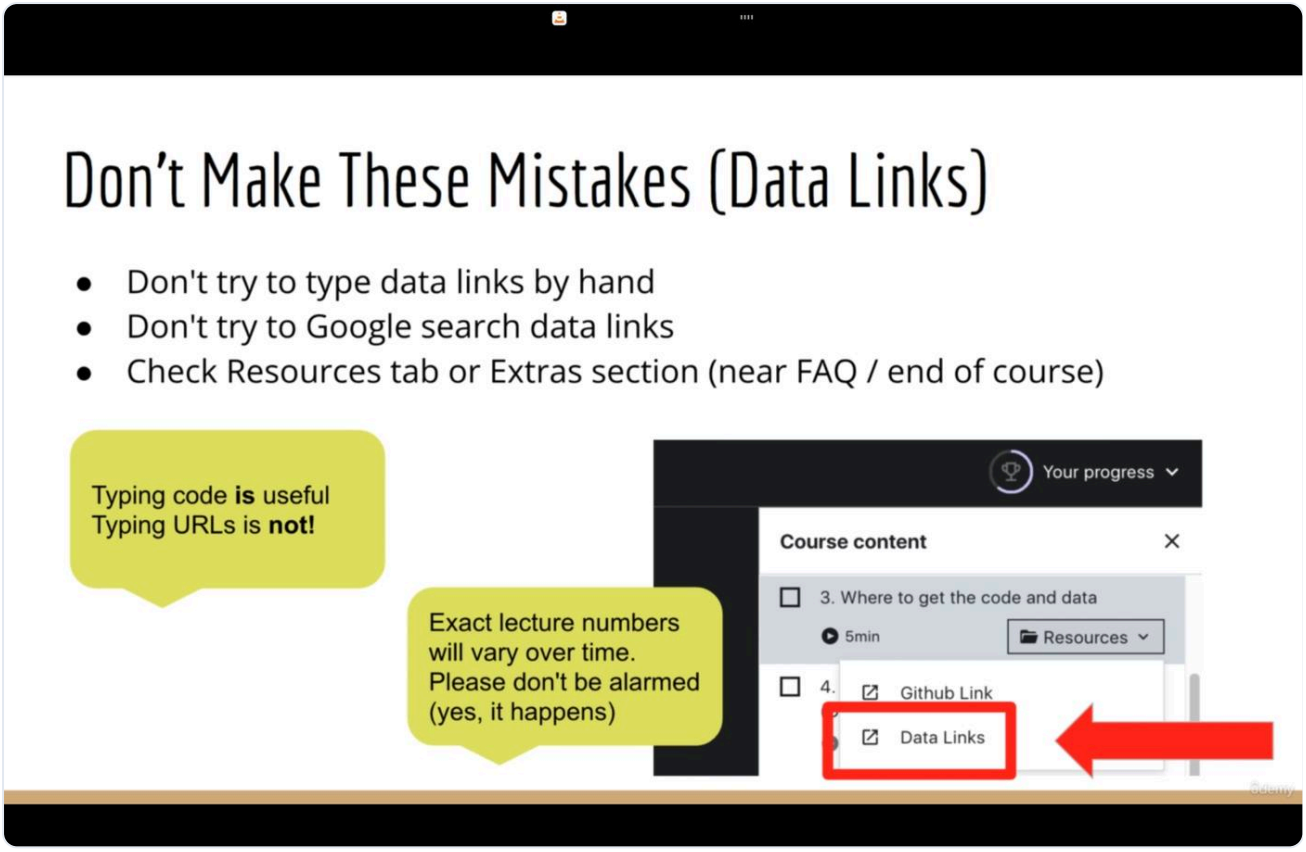

FIGURE 2.10 | Don't Make These Mistakes (Data Links)

So just to repeat that, do not type data links by hand and do not Google search the data links.

Why? Because the data links are already given in this lecture. Just click the resources tab or go to the extra section and you will see a link to a document containing all the data links for this course. So yes, for the code portions, I do want you to type all of that by hand because that's actually useful. Typing code is useful. Typing data URLs is not so useful. Now, one thing you'll notice while watching the code lectures is that I myself do not type the code during these lectures.

Of course, that's completely unnecessary. Some students in the past have made the mistake of thinking that because I wasn't typing that they also could not type. Of course, this is very silly. You don't need me to type in order for you to type. When you see any code, you can type that code. This allows me to focus on explaining things to you, which is more useful than just typing on the keyboard. And guys, just so we're clear, when you see a 20 minute lecture of just typing straight through, which is what you might find in some other course.

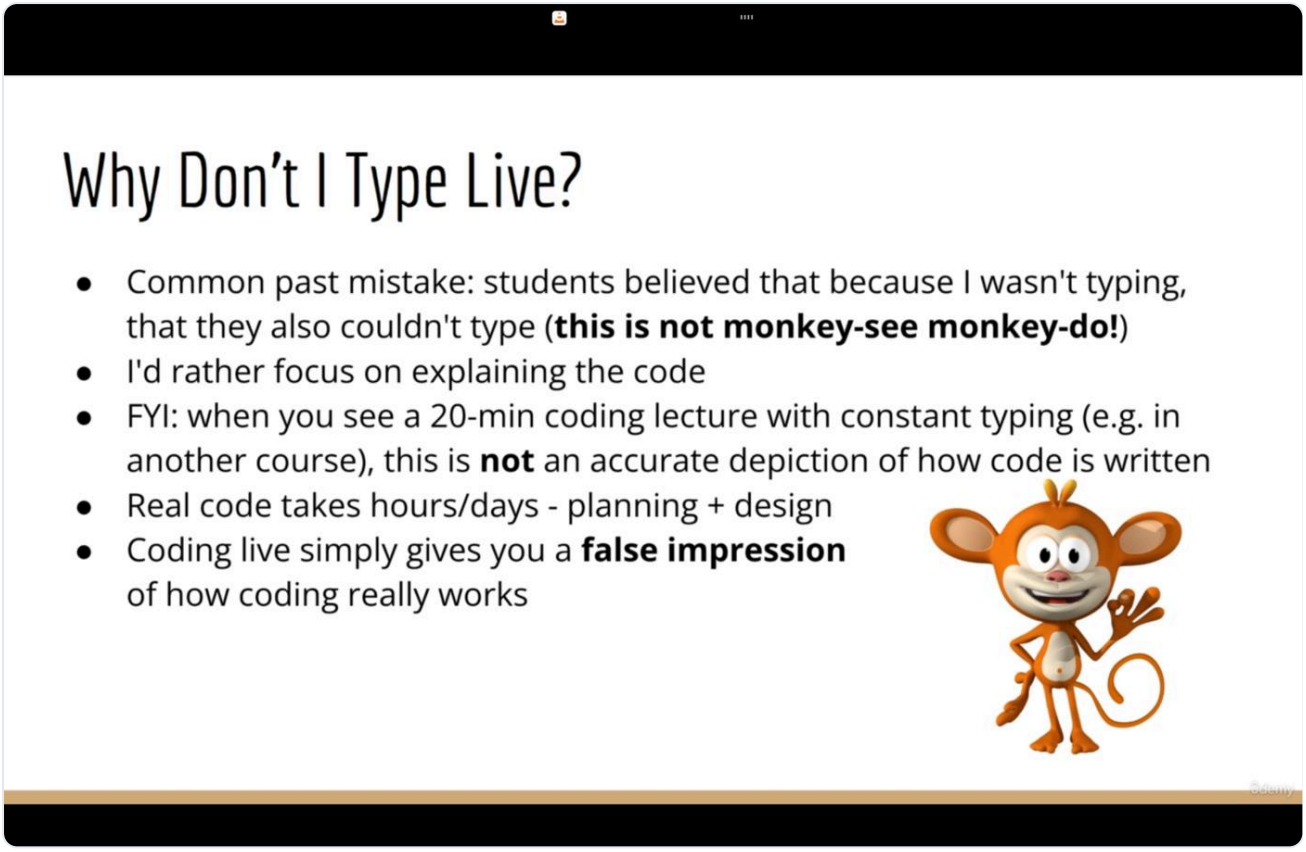

FIGURE 2.11

This is not an accurate depiction of how that code was actually written. For a lot of this kind of code, sometimes it takes hours and maybe even days to finish. So if you think watching me type a notebook from beginning to end is going to give you some idea of how it's done live, it won't.

FIGURE 2.12

It would simply lead you to a false impression of how to write code in the real world. Now, one question I get sometimes is that the code in the videos is too small or too blurry to see.

So firstly, please note that this course has been optimised to be viewed on a desktop or laptop computer.

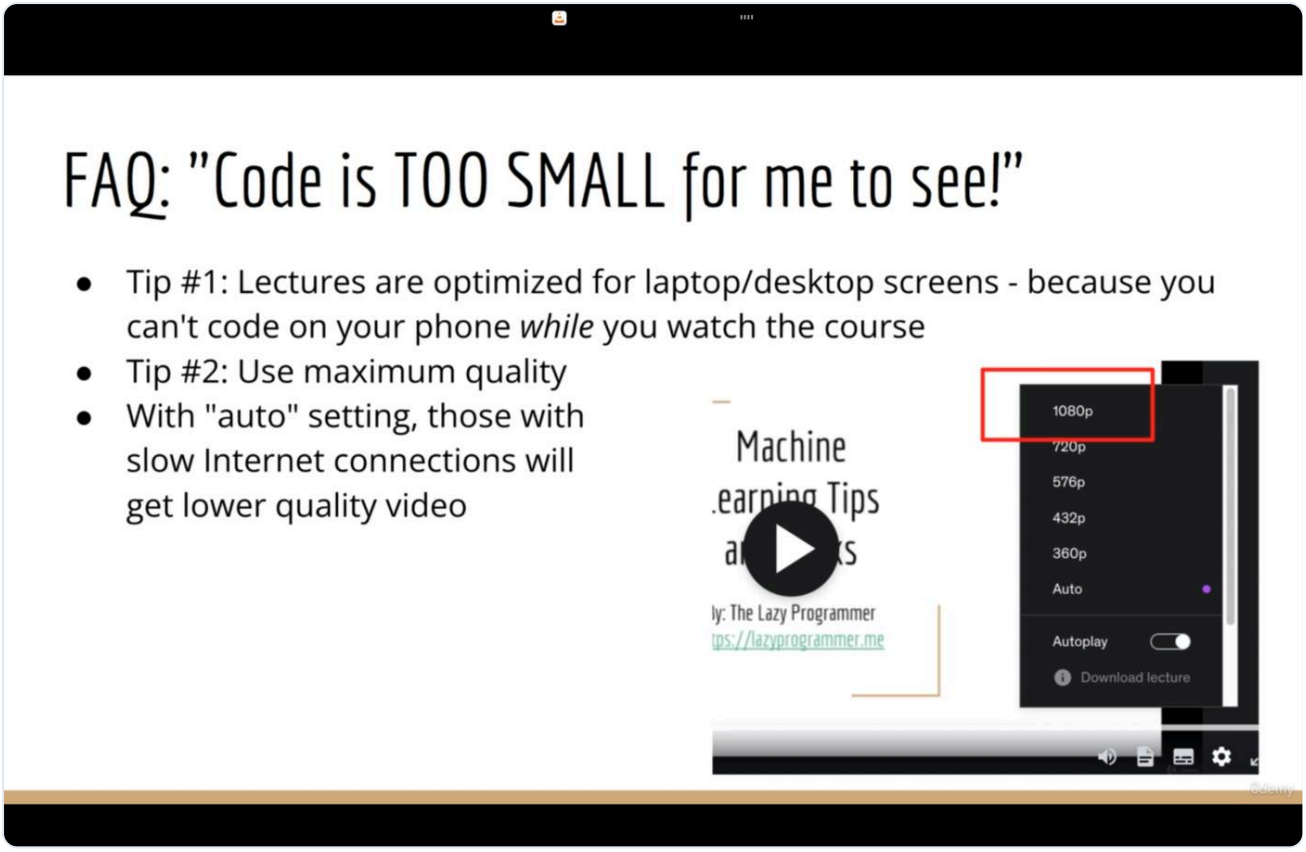

FIGURE 2.13

It wasn't really designed to be viewed on, say, your smartphone because you can't simultaneously write Python code on your smartphone, at least not with today's smartphones. So that's tip number one. Make sure you are using a full sized screen and not a smartphone screen. Tip number two Note that you can change the quality of the video. For some students with slow Internet connections.

This platform will automatically adjust the quality to have a lower resolution. The result is that the video looks blurry compared to the original. The original resolution is 1080p, and you can select this manually from the video player. And if you don't know how to do this, please contact the technical support team. And finally, tip number three, make the video full screen.

FIGURE 2.14

When you do this, the text actually becomes larger than it was on my own screen.

I've also checked this myself and can confirm that the code you see in the videos is larger than the font size in the New York Times. So as long as you're able to view regular websites, then you should be able to view the code in these lectures. So aside from just notebooks, there is yet another place to get extra resources for this course. For example, reading materials, data processing scripts, and other miscellaneous things. To get these, please go to my GitHub repo, github.com/lazyprogrammer/machine_learning_examples.

This contains materials for all my courses in a single repo and it's organized by folder. Essentially it's one folder per course, although there are some exceptions.

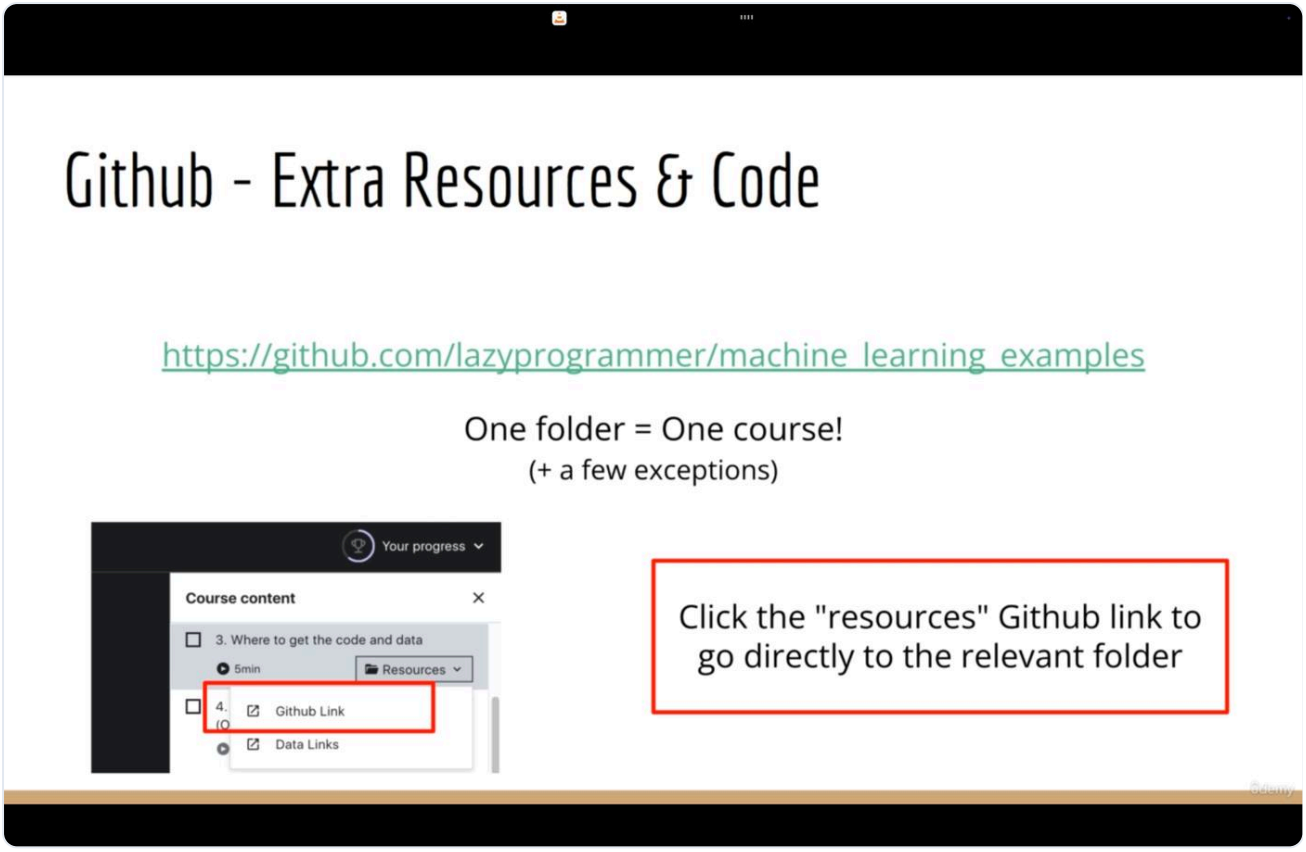

FIGURE 2.15 | Github - Extra Resources & Code

In order to go directly to the folder for this course, please click the resources link attached to this lecture. Please note that the GitHub repo also contains a file called extra_reading.txt, which includes links to papers, journal articles, tutorials, books and other useful resources for this course.

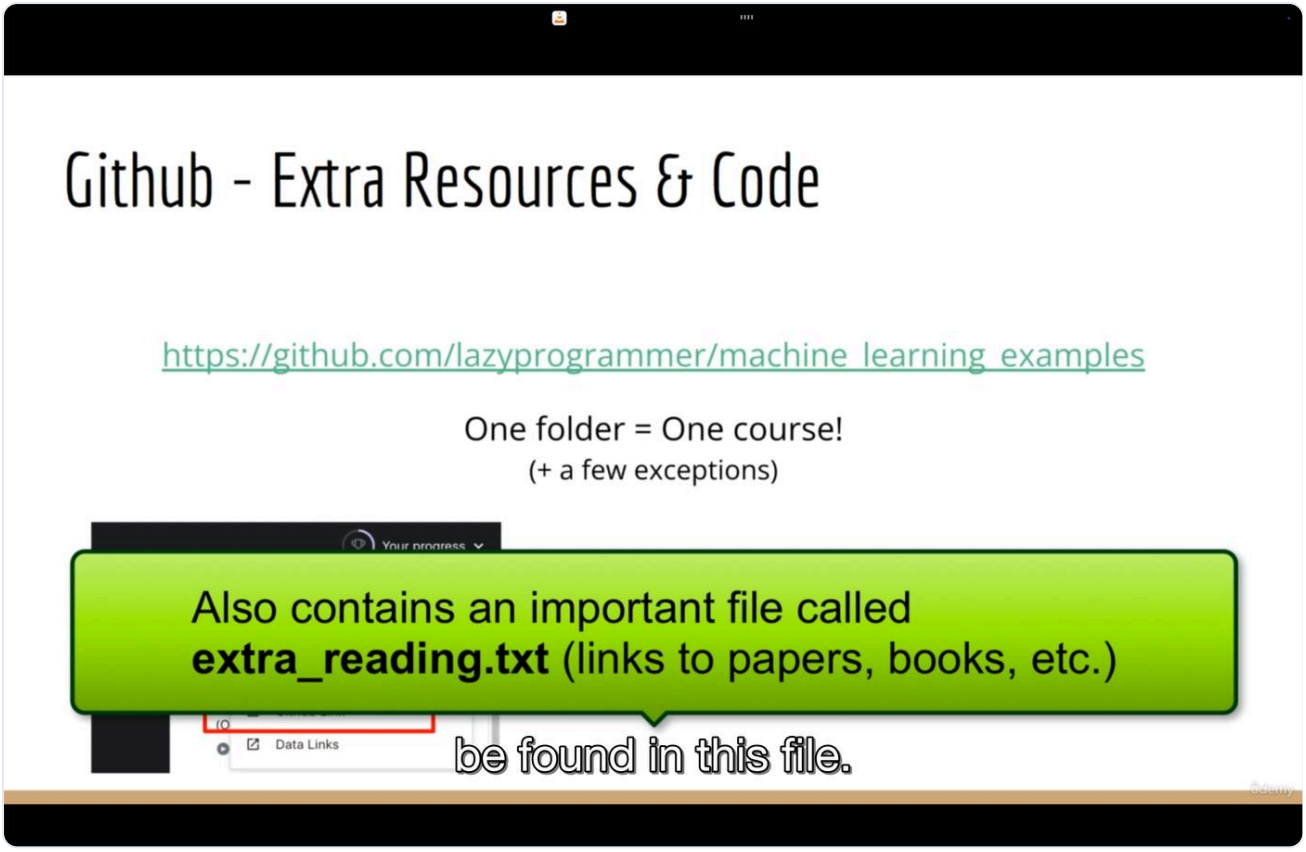

FIGURE 2.16 | Github - Extra Resources & Code

Within this course I may refer to research papers, in which case links to these research papers can be found in this file, extra_reading.txt.

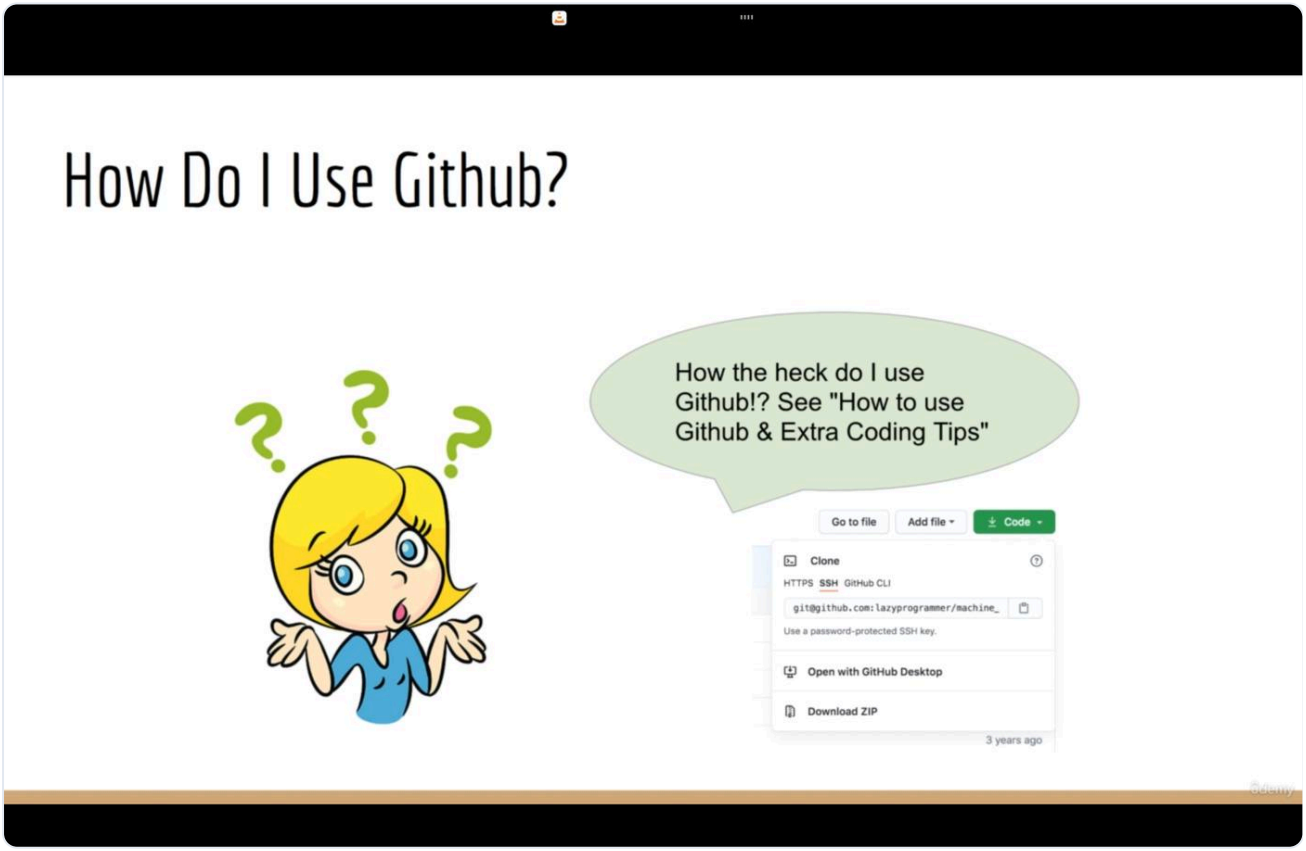

FIGURE 2.17 | Github & Extra Coding Tips

Now if you're a new to Git or GitHub and you're a bit intimidated, please see the upcoming lecture called How to use GitHub and Extra Coding Tips.

2 · How to use Github & Extra Coding Tips (Optional)

So in this lecture, I want to give you a brief rundown of how to use GitHub along with several extra coding tips. Now, if you've never used GitHub before, then there are several ways for you to download the course repository.

FIGURE 2.18 | Learning Tips &

There are also several ways you should not download the repository. So first, let's start with some options for what you should do.

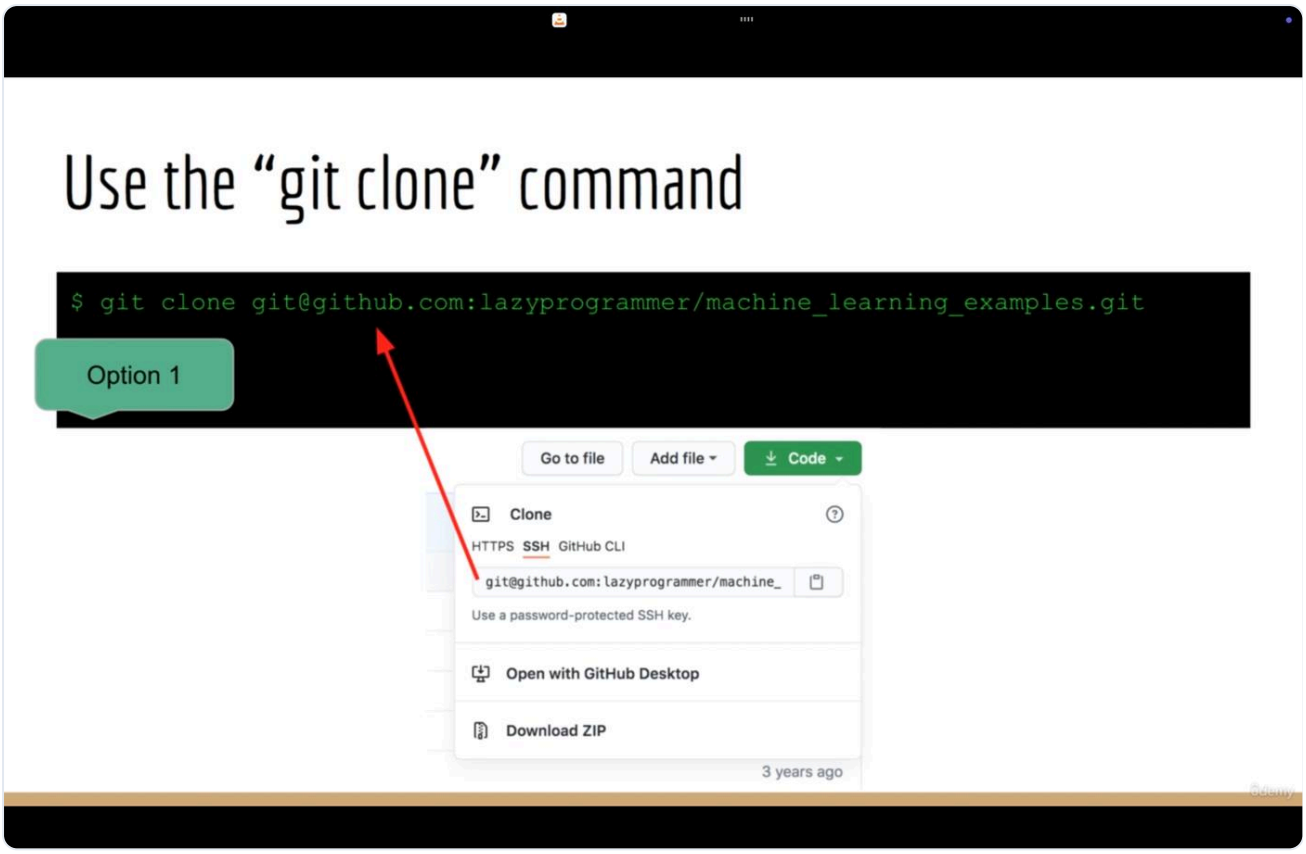

FIGURE 2.19

Option number one is to use the Git clone command.

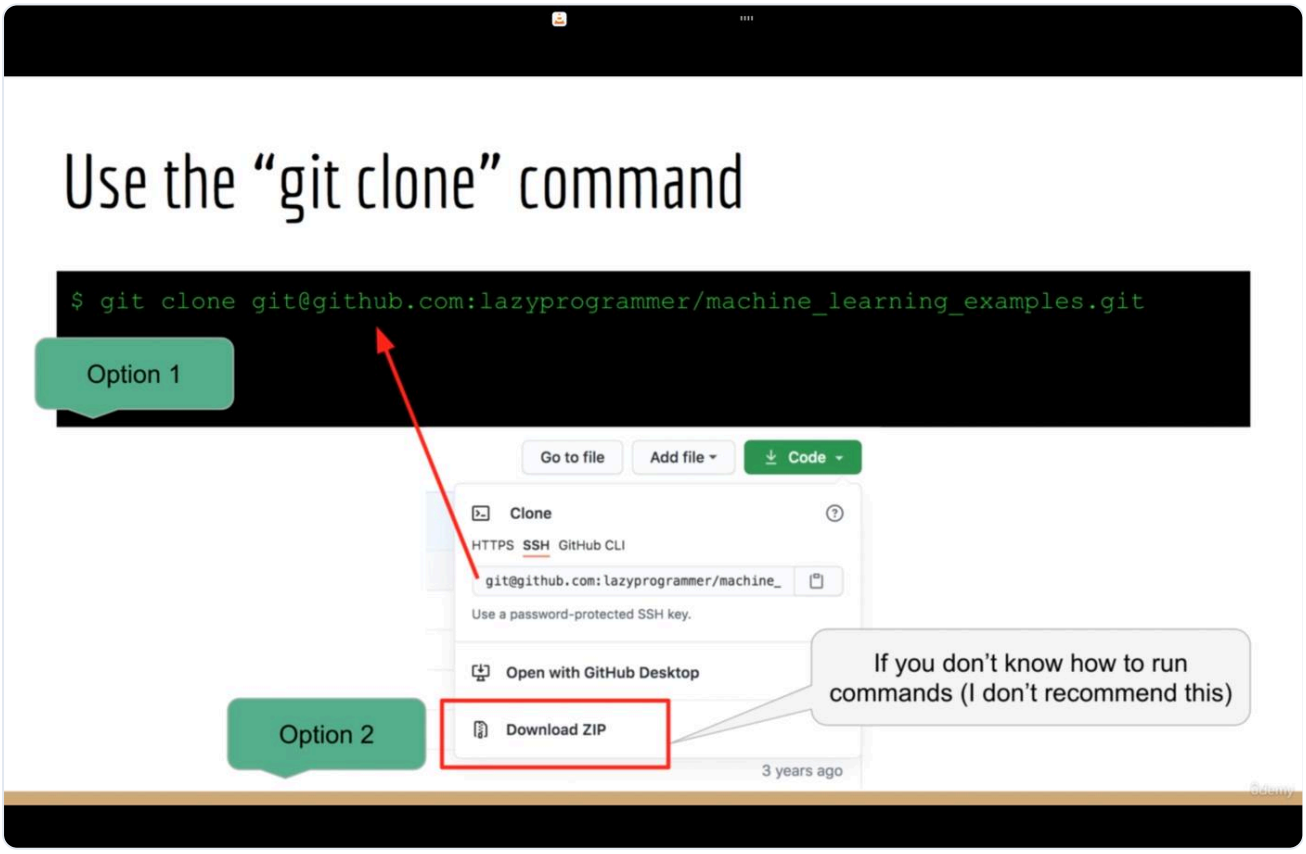

FIGURE 2.20

You can enter this in the console as you can see here.

Option number two is if you are not comfortable using the console, then you can simply use the download button, although I would still prefer option number one.

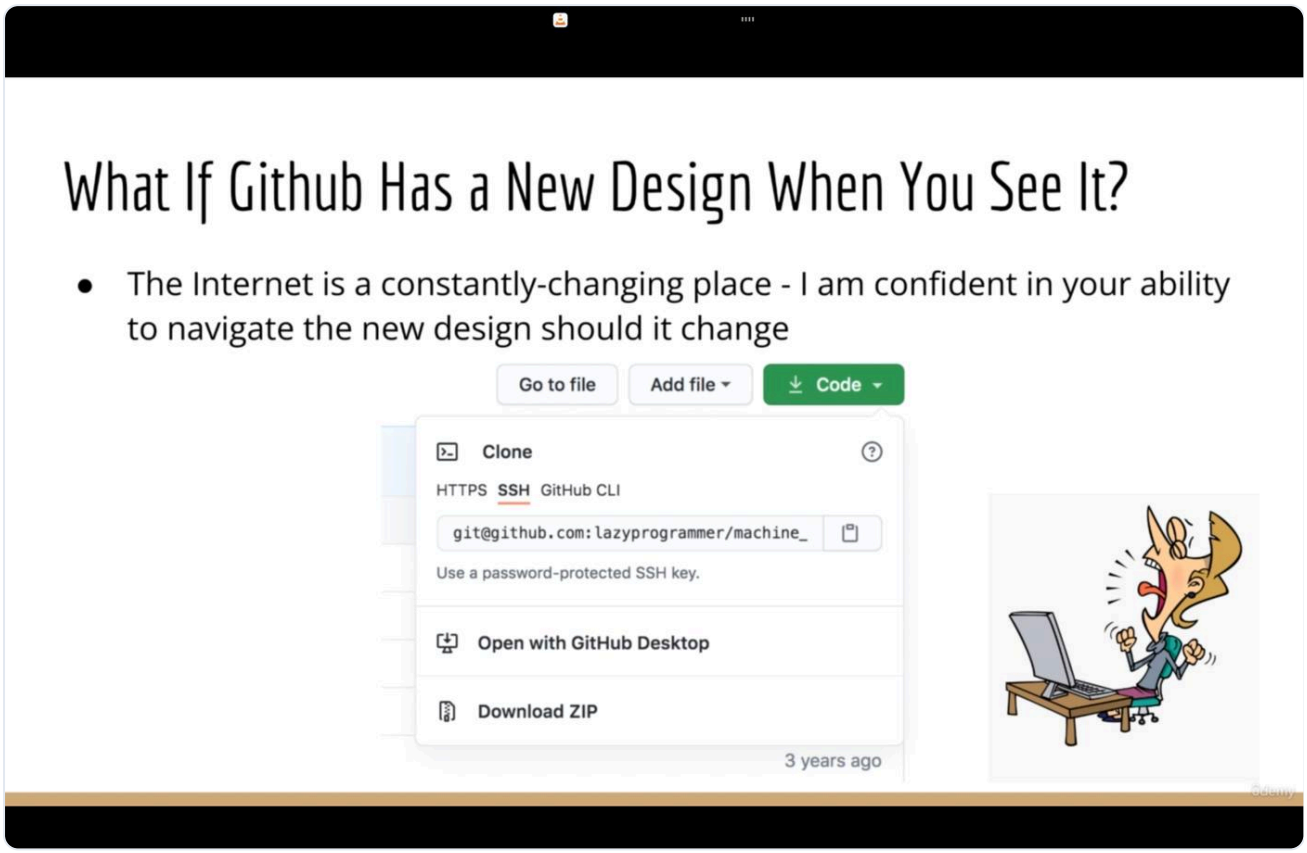

FIGURE 2.21

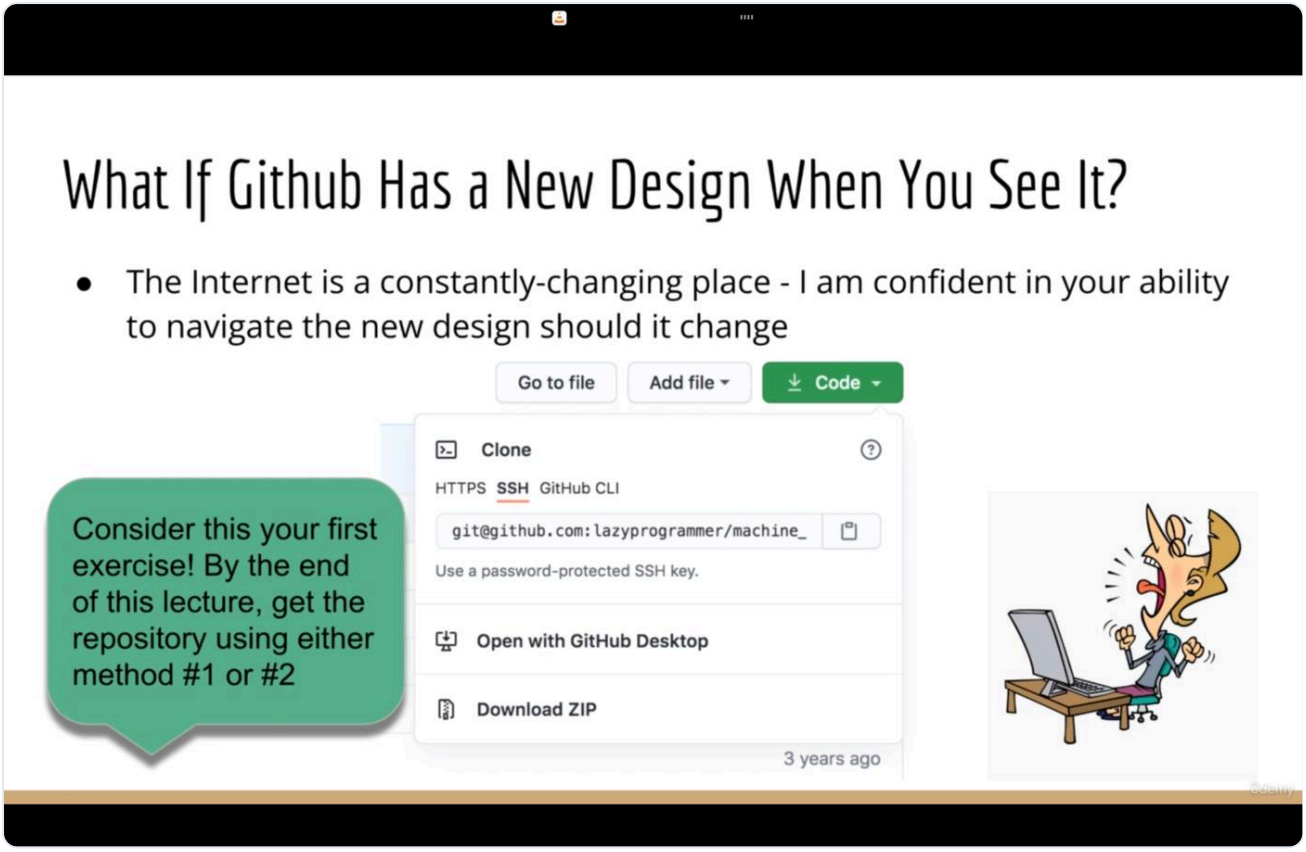

FIGURE 2.22

Now, one thing to keep in mind is that the Internet is a constantly changing place, so do not freak out if GitHub has a new design by the time you visit this page. I am confident in your ability to figure this out. And the reason I mention this is because I had one student in the past who looked at this lecture and saw that the buttons were a different color or a different shape or something, and then all of a sudden he didn't know what to do and couldn't follow the lecture.

But really, it's not that hard. All you need to do is use your pattern recognition abilities. So consider this your first exercise for the course. By the end of this lecture, you should have the entire course repository downloaded to your local machine using either of the two methods above. Now please note that if you're new to Git or GitHub and you find yourself a bit intimidated, please do not be. You do not have to know how to use git for either of the previous two methods.

Now, here are some things you should not do that I've seen several students do in the past.

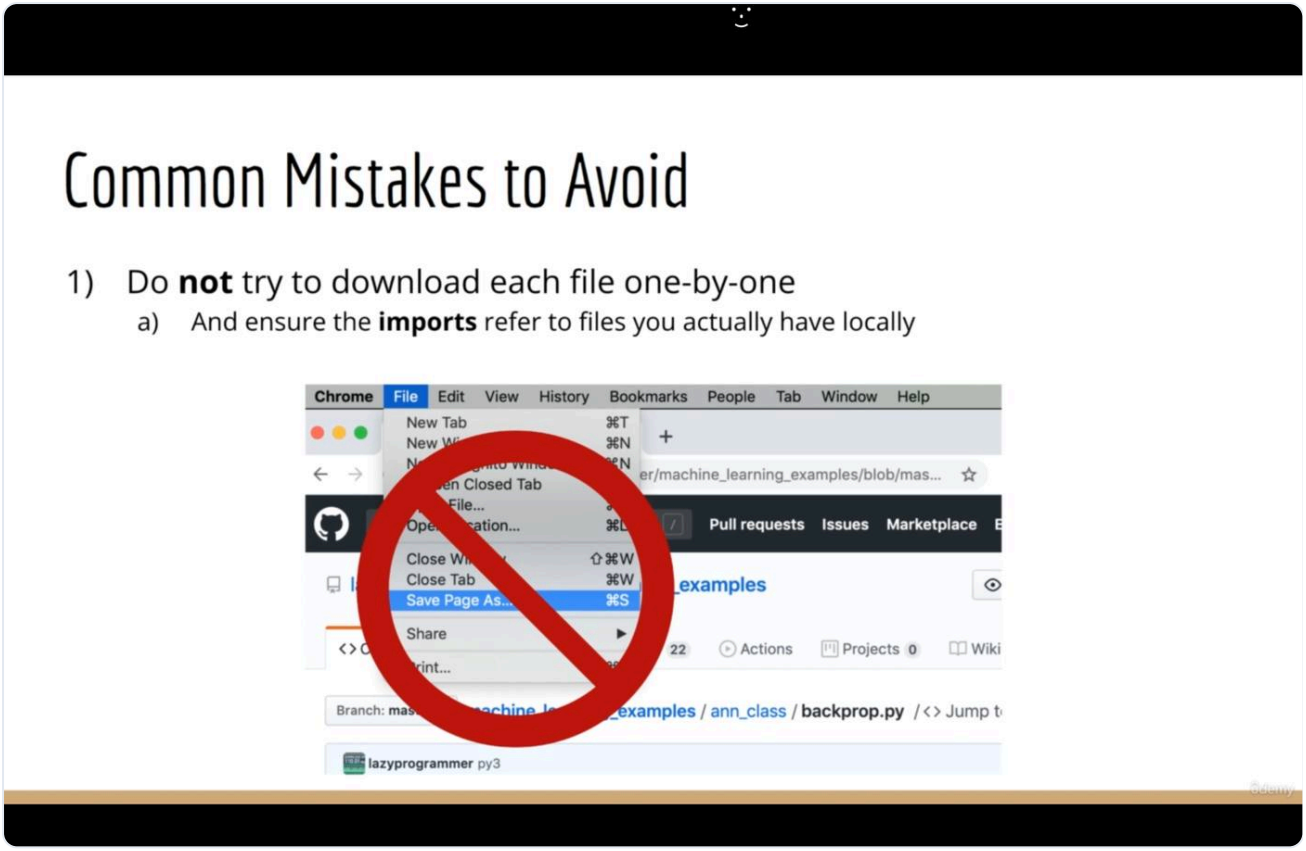

FIGURE 2.23 | Common Mistakes to Avoid

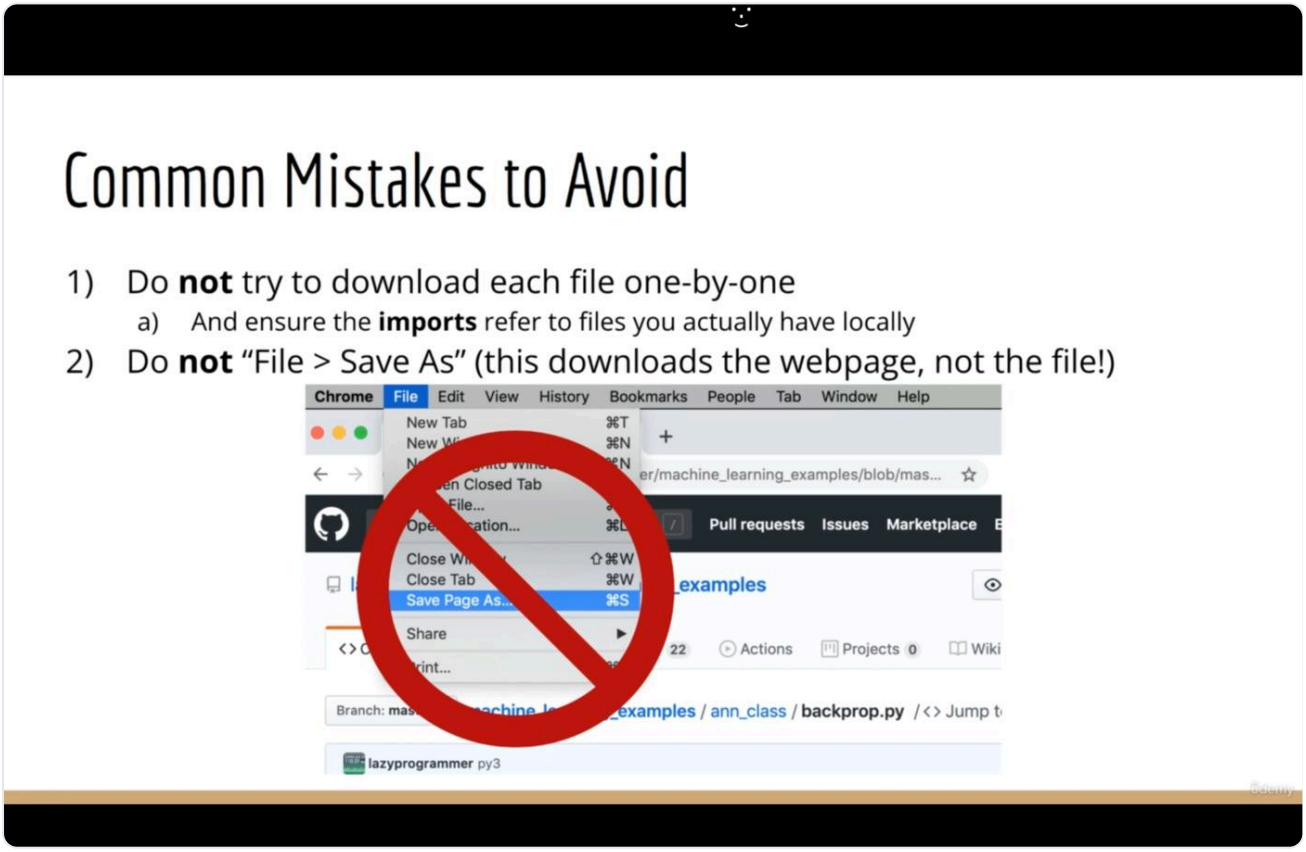

FIGURE 2.24 | Common Mistakes to Avoid

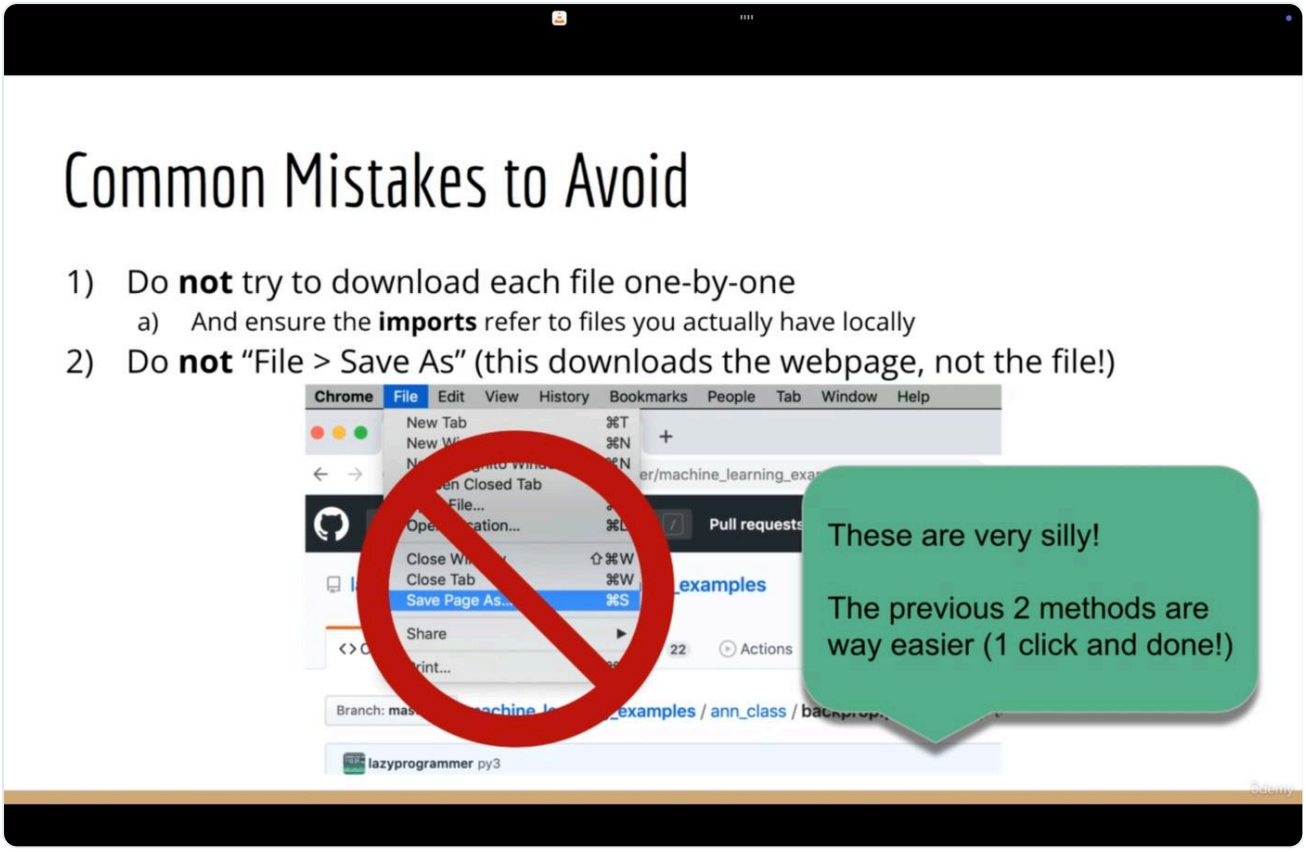

FIGURE 2.25 | Common Mistakes to Avoid

Number one, do not try to download each file one by one. As mentioned previously, this is not the way to use git. Number two. On a similar note, do not go to file and save as this is not how to download files from GitHub. Doing this will download the web page to view the file, not the file itself. So if you look at the file and you see a bunch of HTML, you know that you did it wrong because there's no HTML in this course repo.

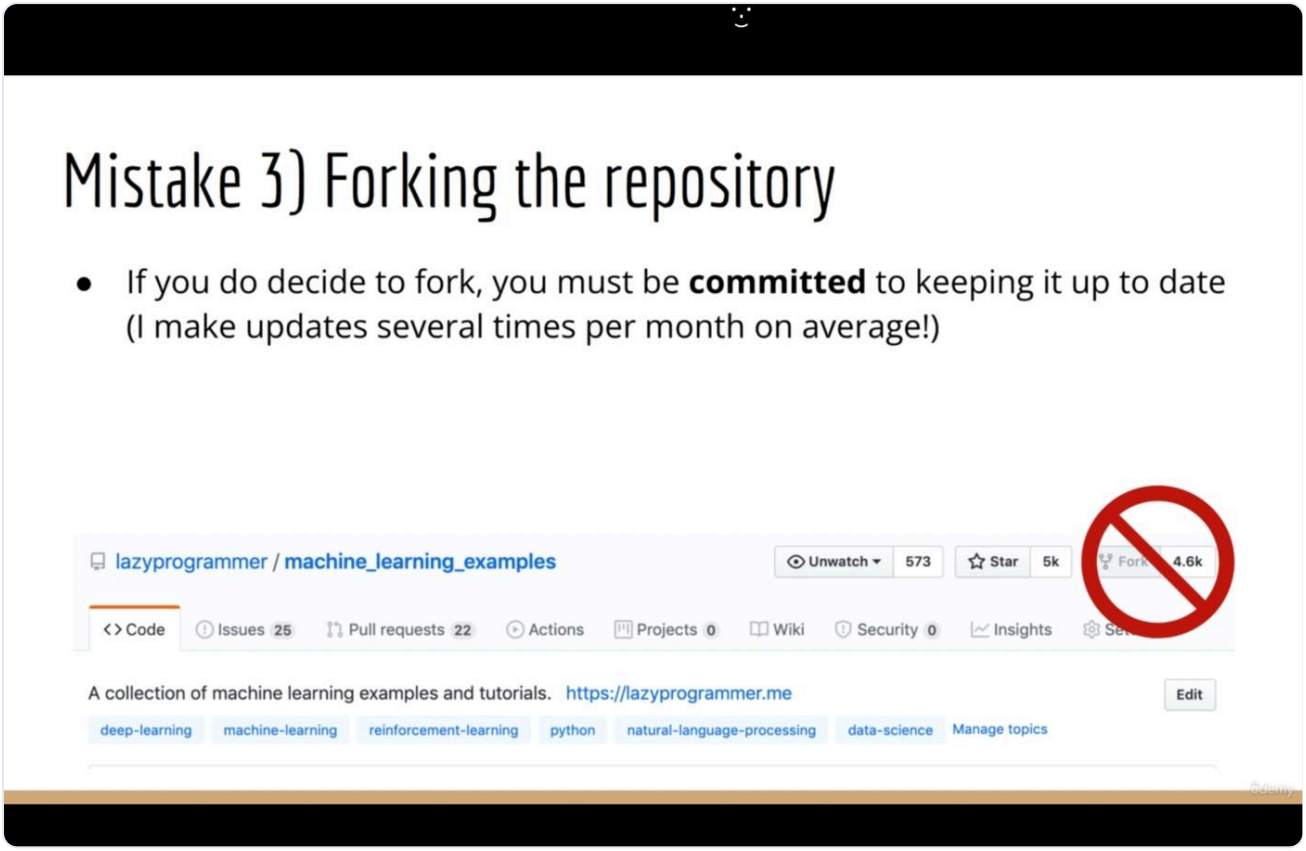

FIGURE 2.26

In fact, number one and number two are very silly because they actually take a lot more effort than simply using either of the methods I mentioned earlier. So it's actually more difficult to do this the wrong way than to do it the right way. And number three, if you want to fork my repository, please feel free to do so. But try your best to keep it up to date. I make updates constantly, so make sure you pull these updates and if you don't think you'll be able to do so, then simply use the Git clone command instead.

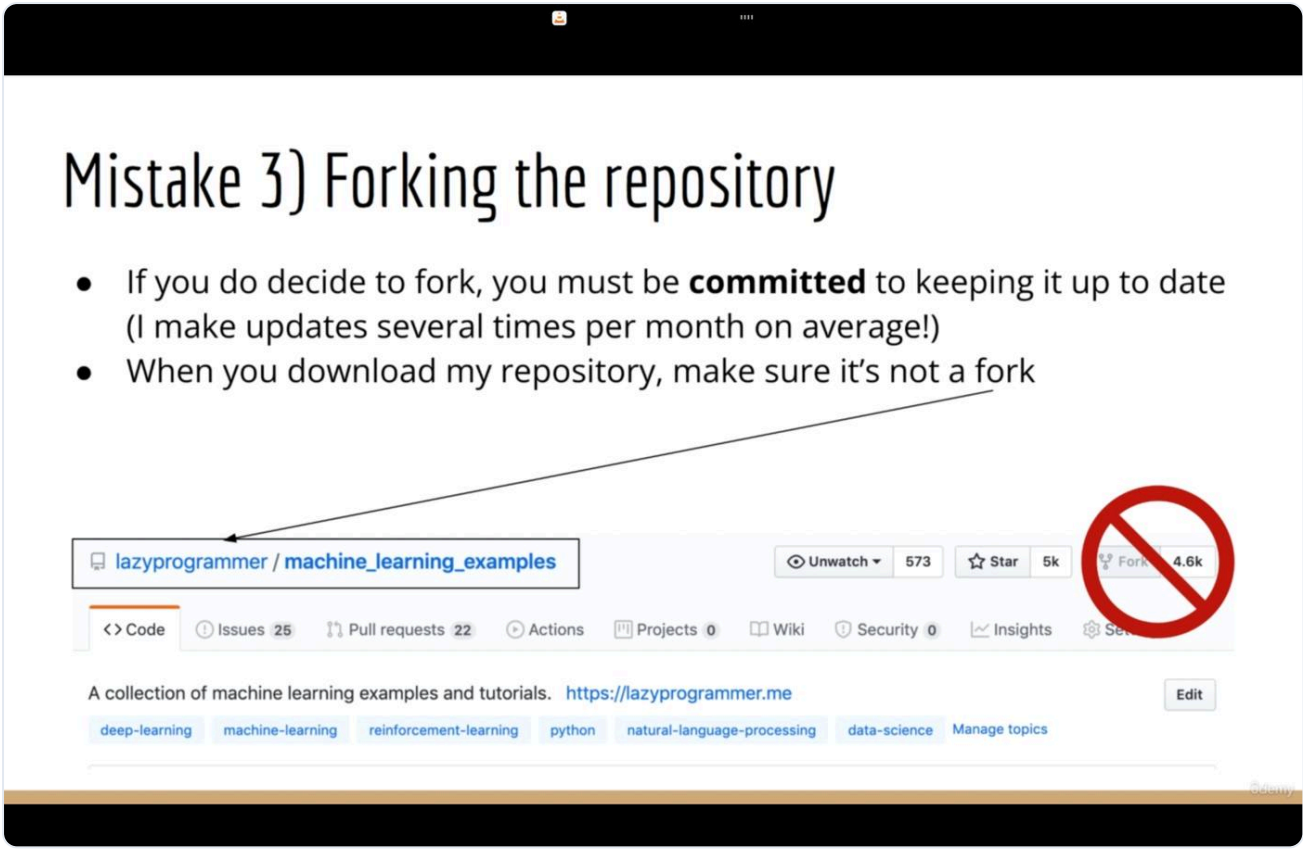

FIGURE 2.27

And on that note, when you do download my repository, make sure it's actually my URL and not someone else's fork. And number four. So this might sound silly, but your fellow students have proven to me that this needs to be said. There is no need to type anything character by character.

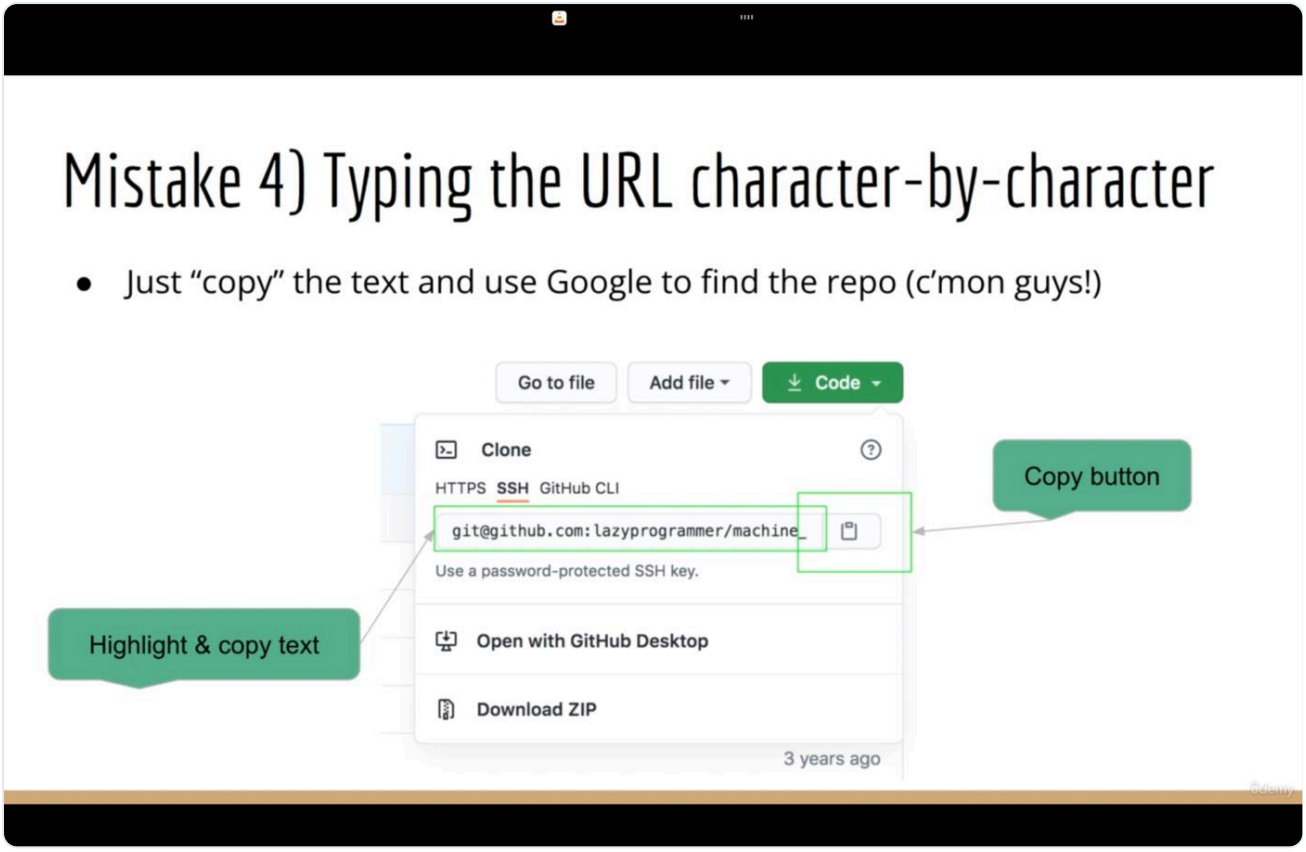

FIGURE 2.28

Simply use the copy button. So if you come back here and you say, I can't type all these characters, it's too complicated. I'm going to refer you back to this lecture. So one extra note about the GitHub repo.

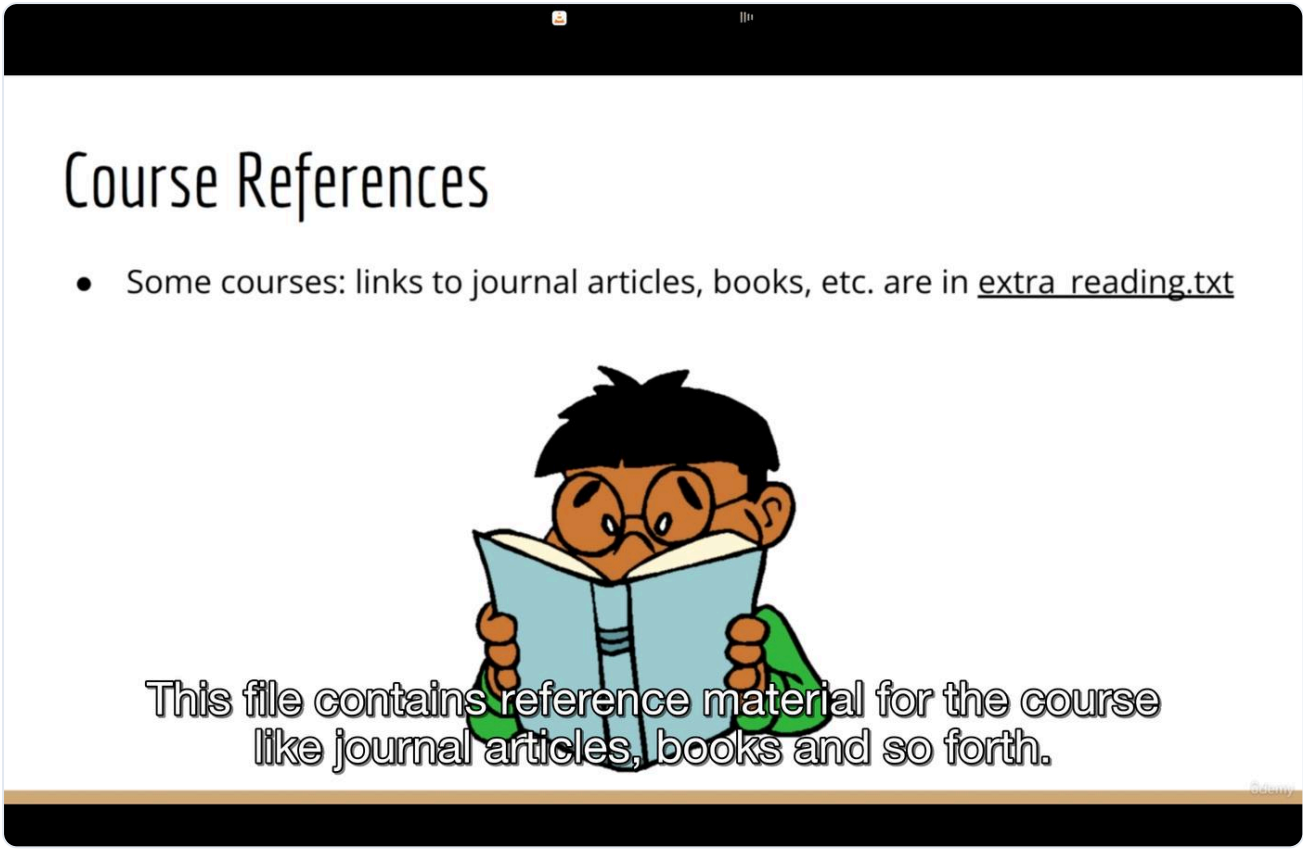

FIGURE 2.29 | Course References

Most courses also contain a file called extra_reading.txt. This file contains reference material for the course like journal articles, books and so forth. So if you hear me referring to extra_reading.txt in the lectures, this is where it can be found. Reading these materials is optional, but they are there in case you want to further your studies. So I want to mention a couple of points pertaining to the code in this course. Firstly, some of you, especially those who don't necessarily meet the prerequisites, may be asking what is Git and how do I use it?

The short answer is Git is a way for developers to manage and track changes in their code.

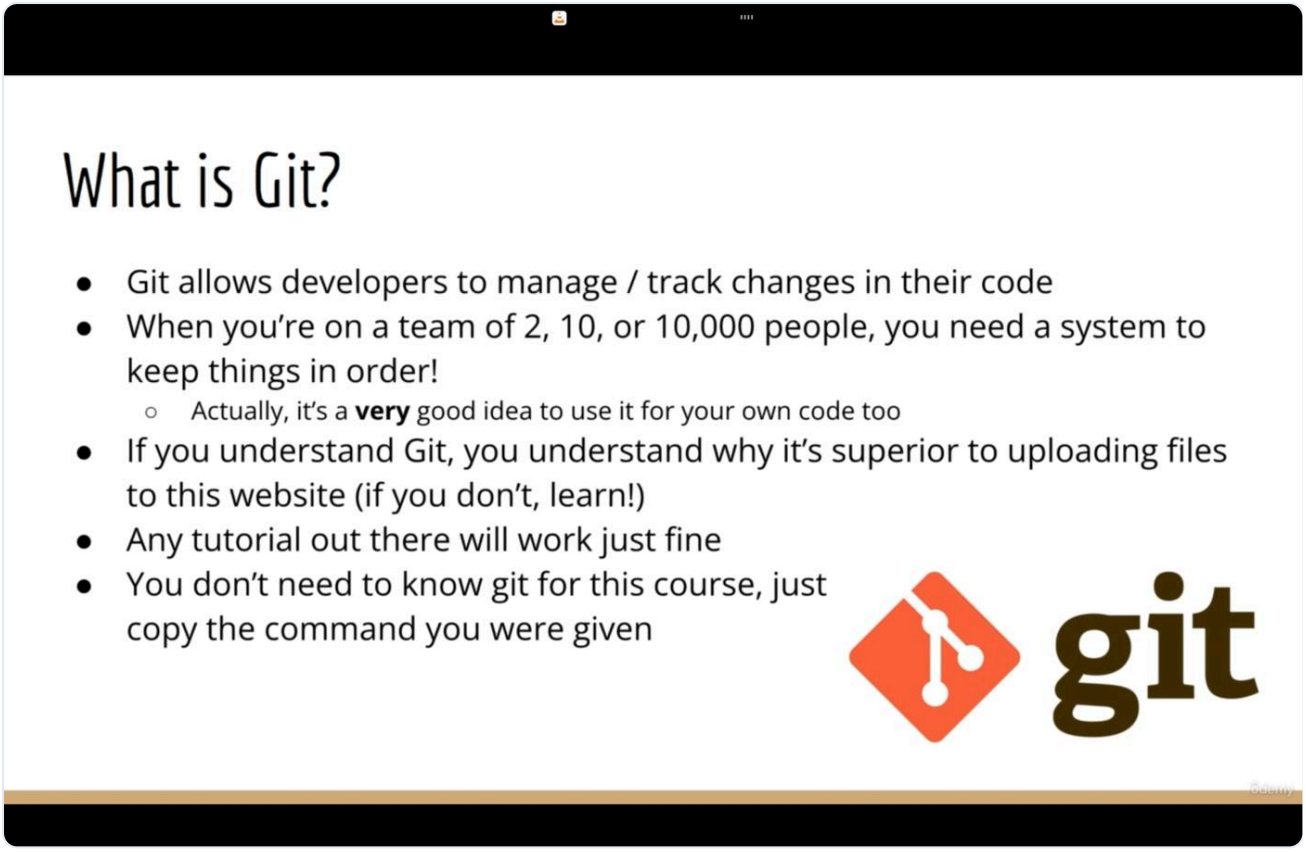

FIGURE 2.30 | What Is Git?

You might be used to writing code by yourself, but when you're working with a team of two or ten or even 10,000 people, you need a system to keep things in order. So this is why, for example, Git is superior to uploading code files to this website and learning how to use Git is very simple. There are many tutorials out there that will work just fine for this course.

You only need one command, which is the one I showed you before, so you don't even need to understand Git. You just need to copy and paste the command I gave you.

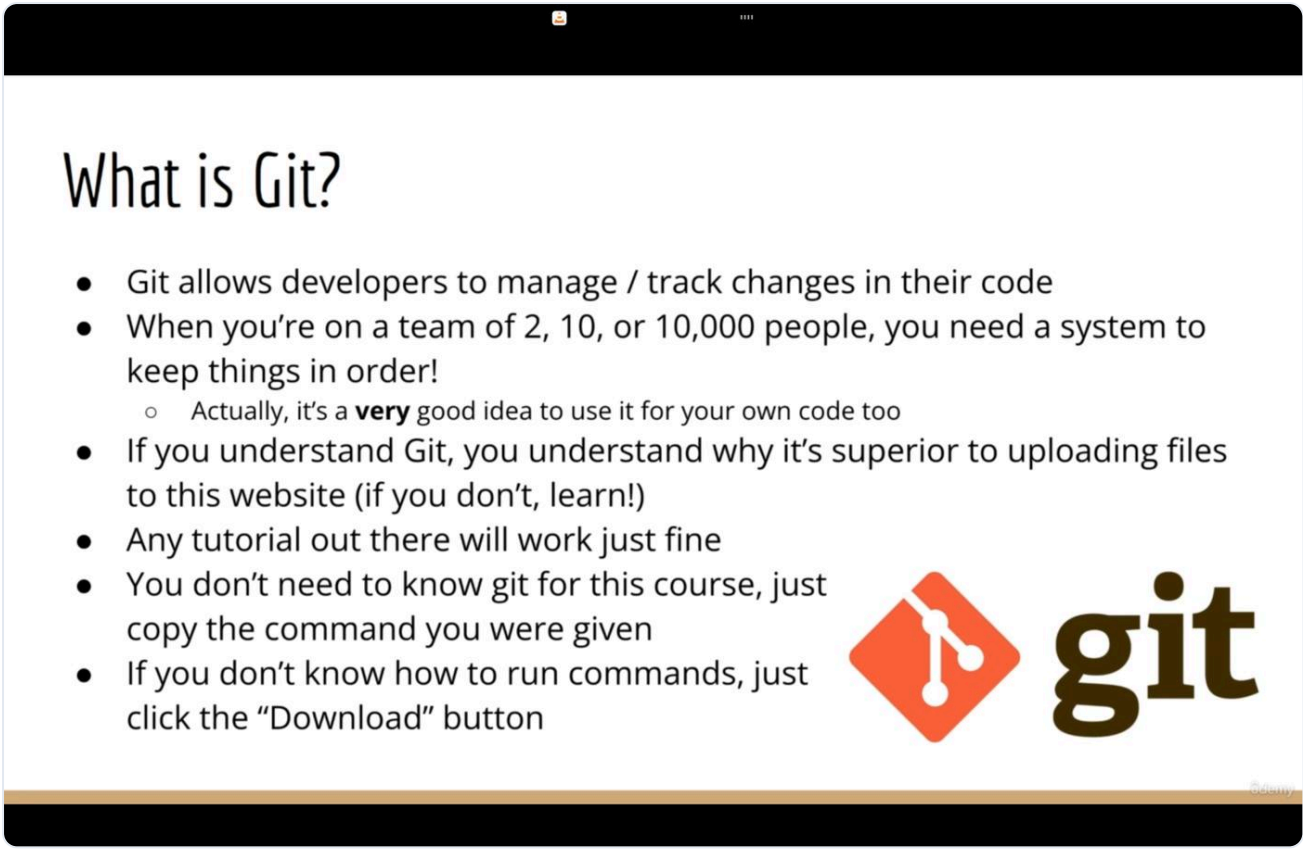

FIGURE 2.31 | What Is Git?

And therefore I don't suspect this will be an issue unless you also don't know how to run commands. But if that is the case, then as mentioned, you can simply click the button that says download. At this point, I'll assume you need no further assistance. And just as a side note, although you don't need to know how to use Git for this course, it is a valuable skill that will be necessary if you plan to work for any company where your job is to write code.

FIGURE 2.32 | You Will Need Git Skills in the Real World

I don't know any companies that write software that do not use some kind of version control, like Git. So if you do want to have a job in this field, that's something you're going to have to learn about sooner or later. Now, another common question I see from beginners is should you code along? The short answer is no. Now, you may have taken online courses in the past that encouraged you to code along.

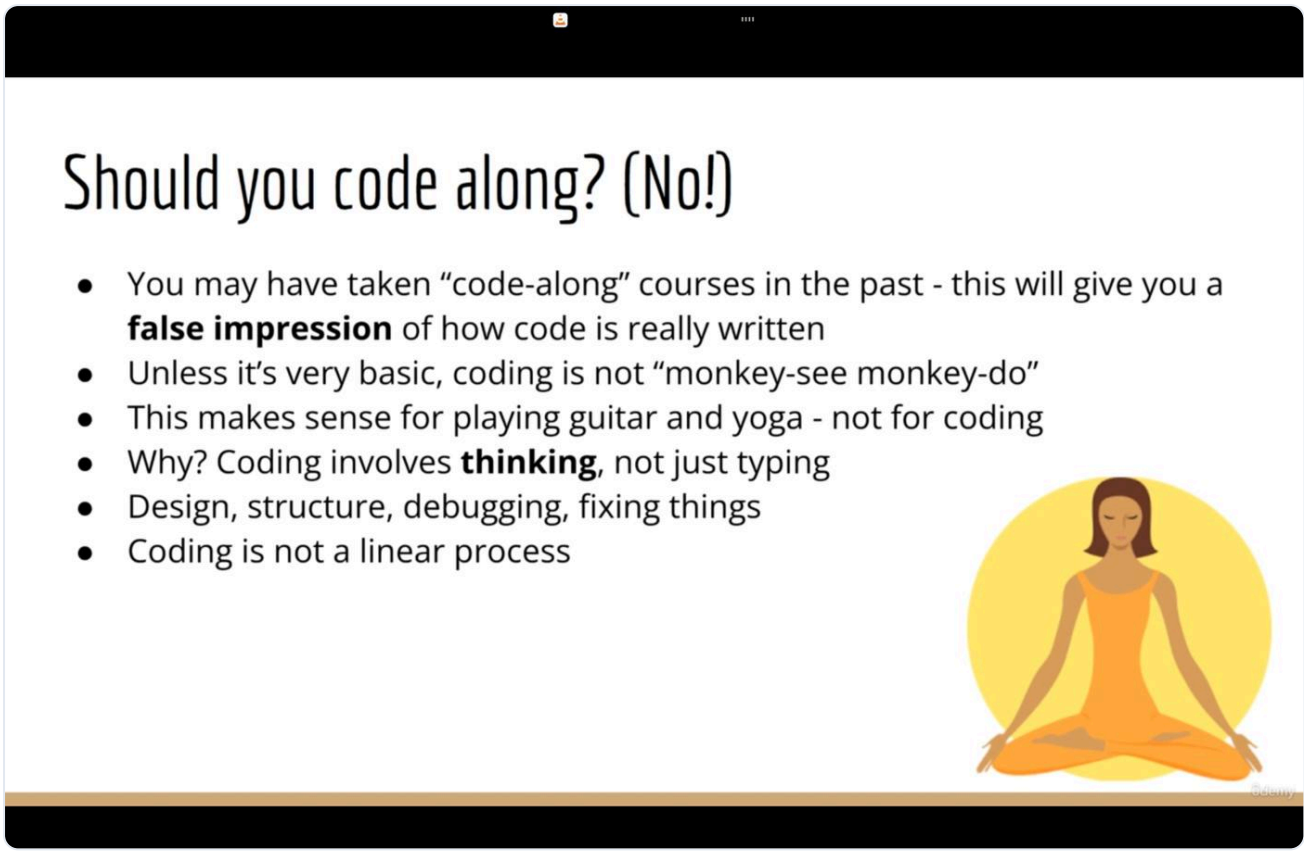

FIGURE 2.33

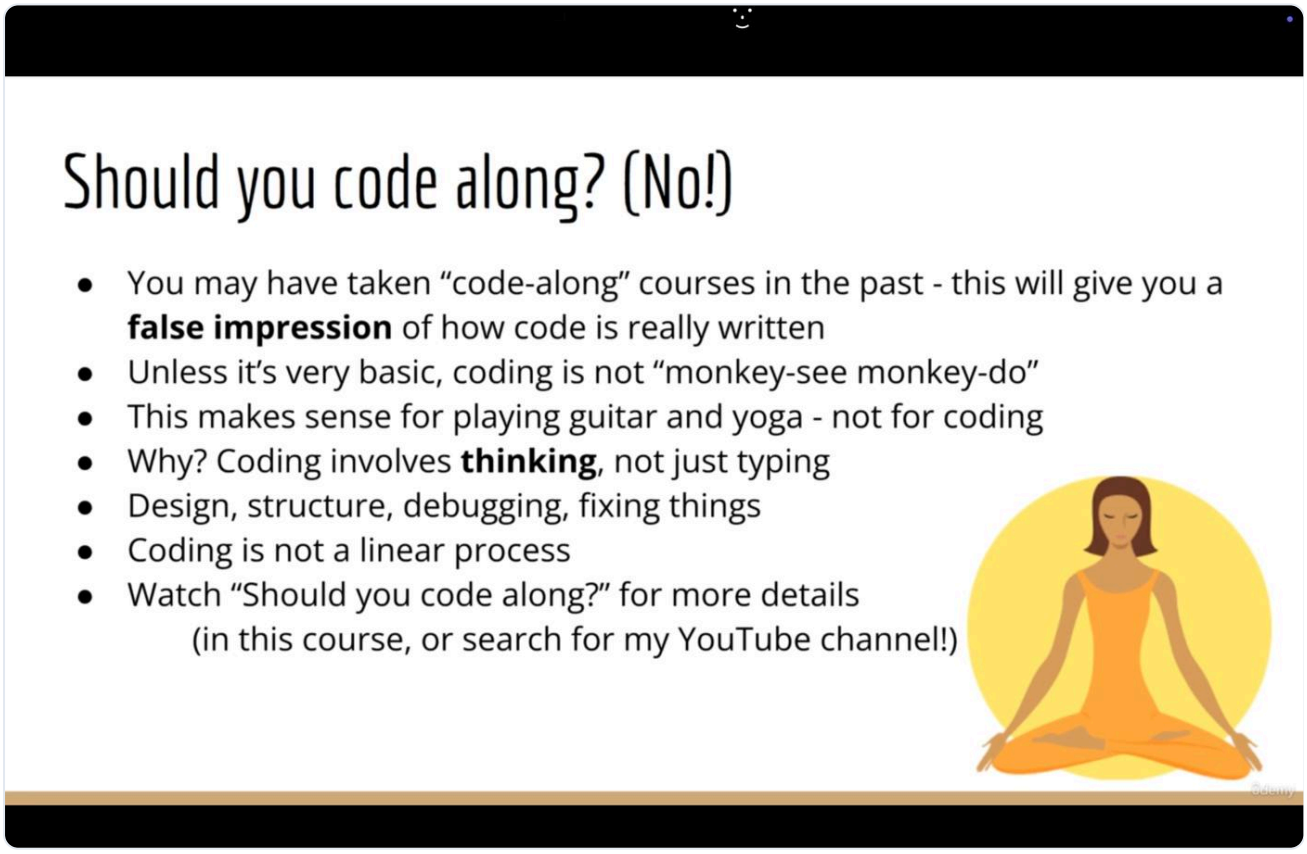

FIGURE 2.34

But the reality is this is going to give you a very false impression of how code is actually written.

Coding is not a monkey see monkey do type of thing. If you're learning to play guitar or do some yoga moves, then yes, that makes sense. But coding involves lots of thinking and not just typing. If you think you're going to write a complex algorithm in 20 minutes without actually paying attention to design or structure and without debugging and going back to fix things, think again. Real code in the real world is not written linearly from start to end.

If you've gotten this false impression from other courses, now is the time to improve your approach. I encourage you to watch the lecture, Should You Code Along for a more detailed discussion? This brings us to our next point, which is what are the exercises for this course? Now, in some cases this will be obvious because I'm going to say, here is an exercise. Please do it. But sometimes it's going to be less obvious and I'm going to trust that you've watched this lecture and understood what I'm saying.

Basically, any time that you see code that is an exercise, there are two types of code you might see.

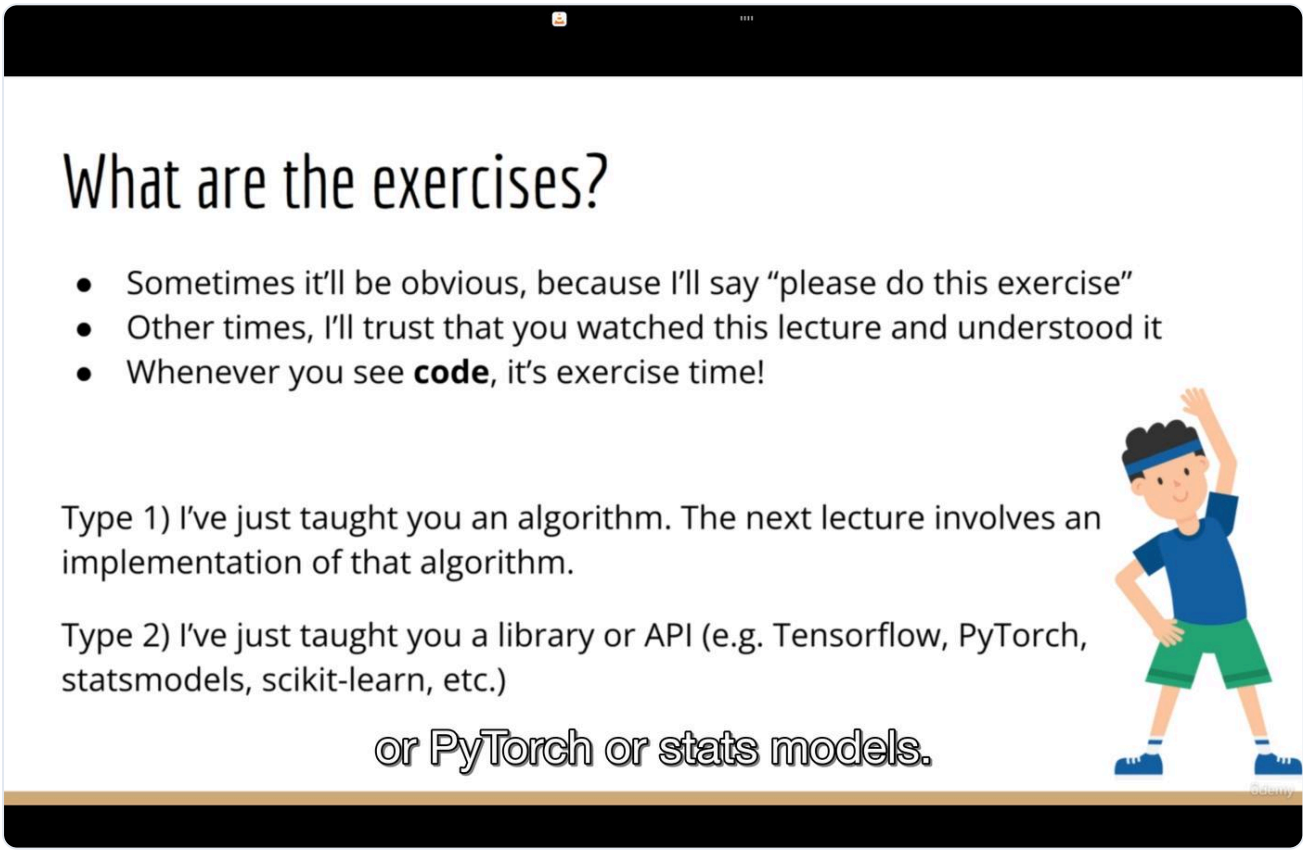

FIGURE 2.35 | What Are the Exercises?

Number one, I've just taught you an algorithm, in which case the next code lecture will involve an implementation of that algorithm. And number two, I'm going to walk you through how to use a library or an API, for example, TensorFlow or PyTorch or stats models. So what is your exercise?

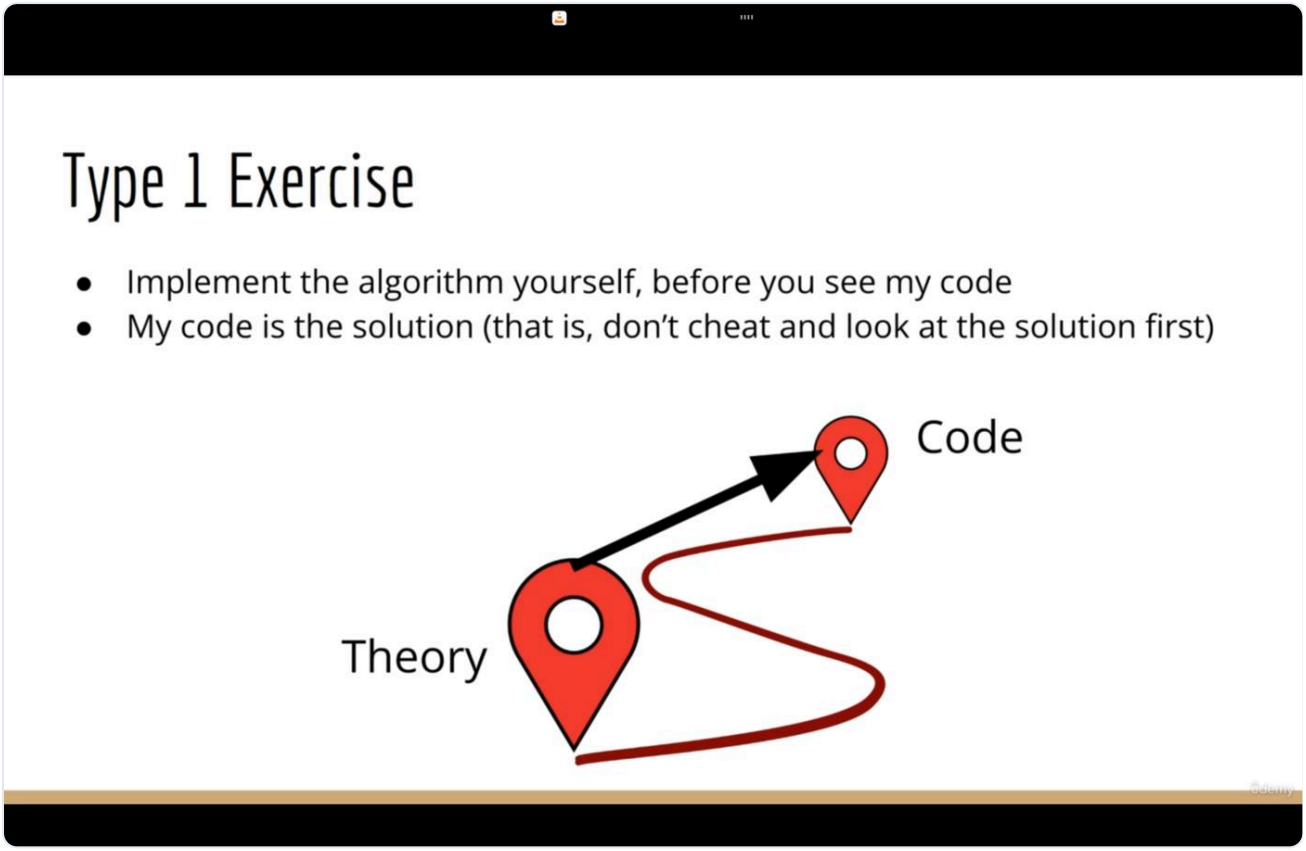

FIGURE 2.36 | Type 1 Exercise

Well, for case number one, your exercise is to implement that algorithm yourself before you ever see my code.

My code is actually the solution to that exercise.

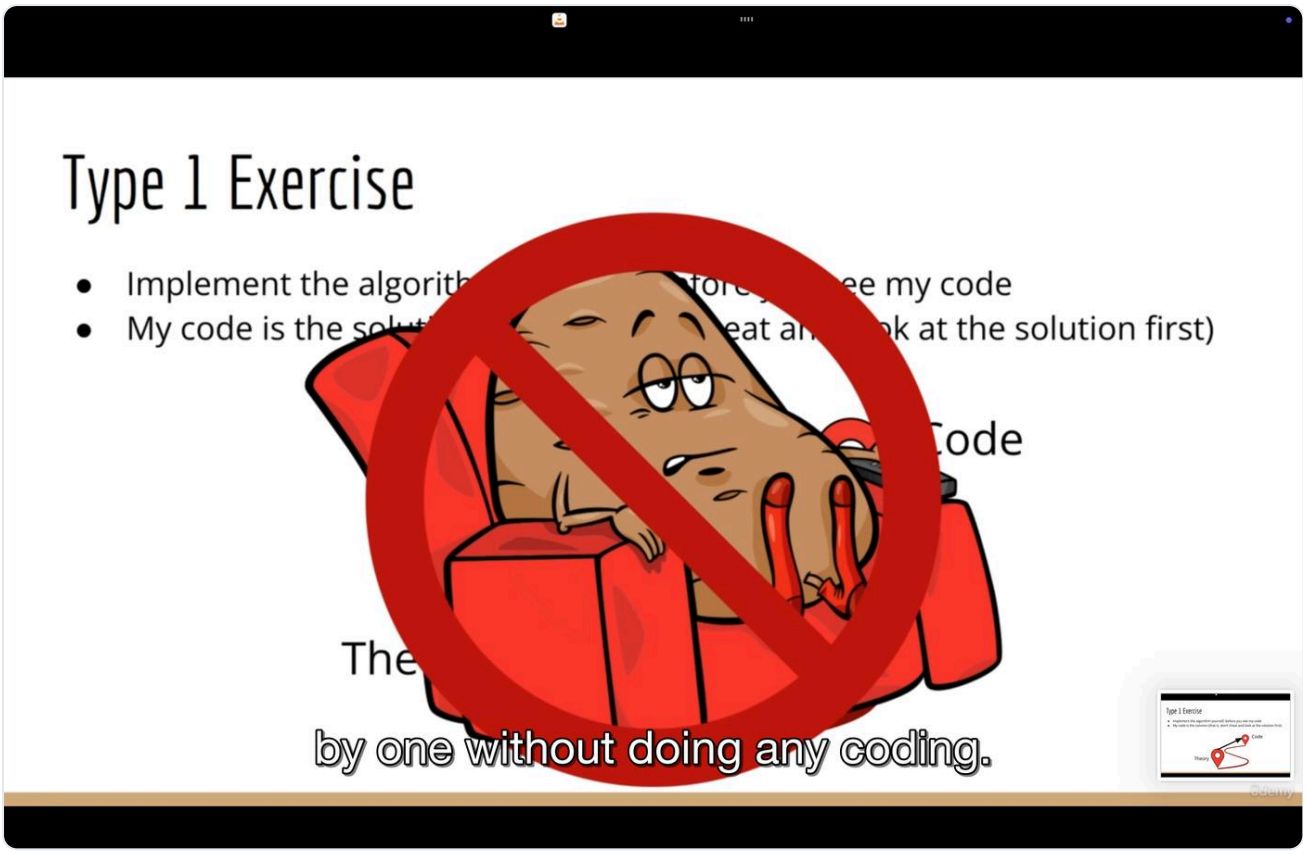

FIGURE 2.37 | Type 1 Exercise

So don't take what I call the couch potato approach where you just sit there watching each lecture one by one without doing any coding. That is not the intent of this course. And if you're looking at the solution to the exercise before you do the exercise, this is basically cheating and copying your homework. Now, obviously what you do is up to you, but I hope that you'll do the exercises as intended.

Now for case number two, it's a little bit different. Obviously, you're not going to know the syntax for a library or an API.

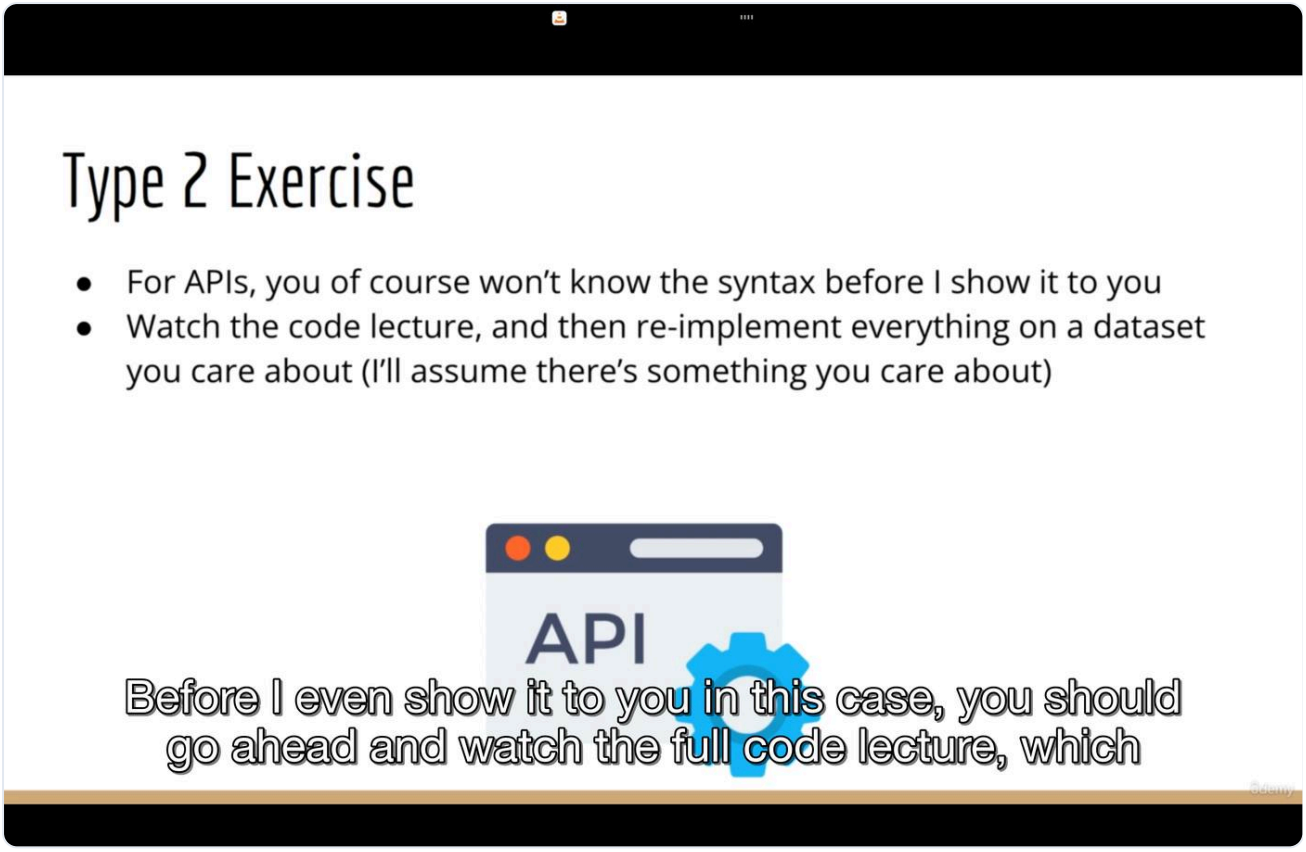

FIGURE 2.38 | Type 2 Exercise

Before I even show it to you in this case, you should go ahead and watch the full code lecture, which will show you the syntax and how things work. Replicating the code is very useful because you're building muscle memory and although this sounds like it should be easy, it's not. In fact, making mistakes when trying to replicate the code is the number one mistake I see on the Q&A.

And yes, you heard that correctly. Even after seeing all the code work perfectly right in front of their eyes, students still have trouble just copying that code without making mistakes. And for those of you not typing code at all, a lot of people mistakenly believe that just seeing the code is enough, but actually being able to type the code makes you much more capable of using it in the future. But the most important exercise comes after you've watched the lecture.

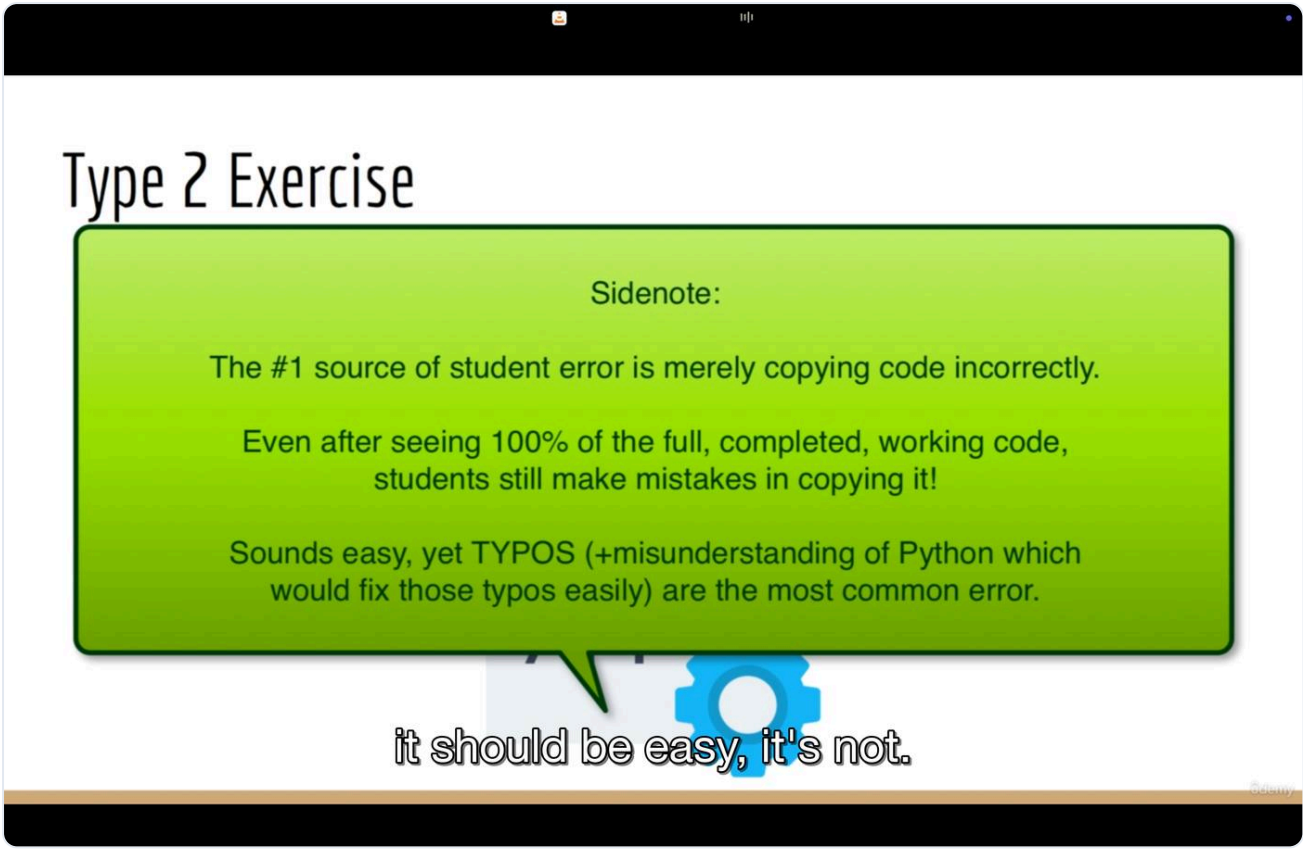

FIGURE 2.39 | Type 2 Exercise

You should be able to implement that same code yourself on another data set you care about. I assume that you're taking this course because you have some data sets you want to analyze, and if not, there are plenty of resources out there where you can find free data sets. So please choose one that interests you. Note that by this measure, most of my courses have at least 10 to 30 exercises, so try to do as many as possible. And also note that many courses contain examples of both number one and number two types of exercises, and you should be able to recognize it when you see it.

So hopefully the exercises for this course are clear, and if not, please make a question on the Q&A. Now, the next question I want to tackle is a funny one, because I honestly didn't think that this was a question I would need to answer.

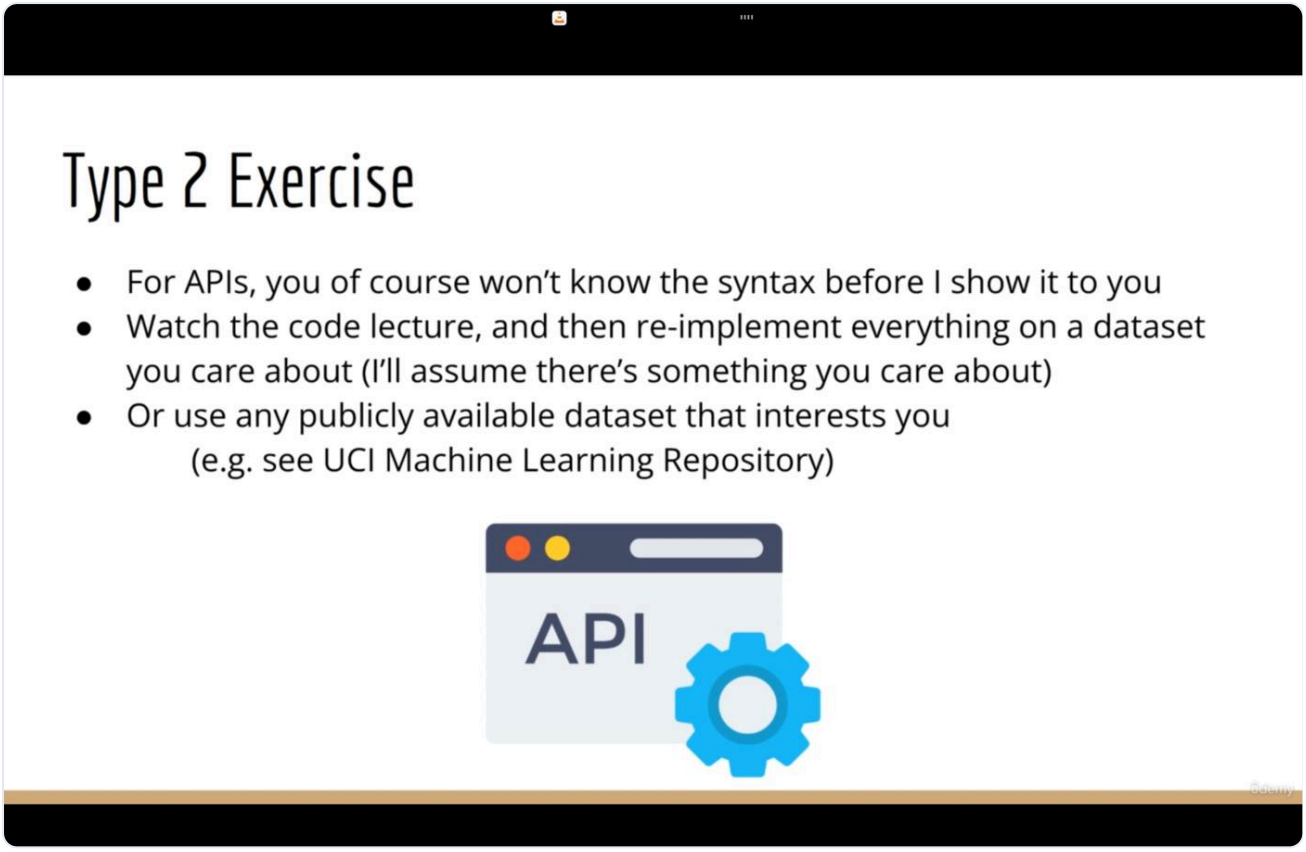

FIGURE 2.40 | Type 2 Exercise

But here we are. So some students really like Jupyter Notebook very much, while other students dislike it. Some students really like PyCharm. So you might ask, why am I not using whatever your favorite tool is? And the answer is this question is totally irrelevant.

Now, you might think I'm being dismissive of your concerns, but in fact, the reason people have this question is because they don't yet know how Python works. After this lecture, I hope you'll have a better understanding.

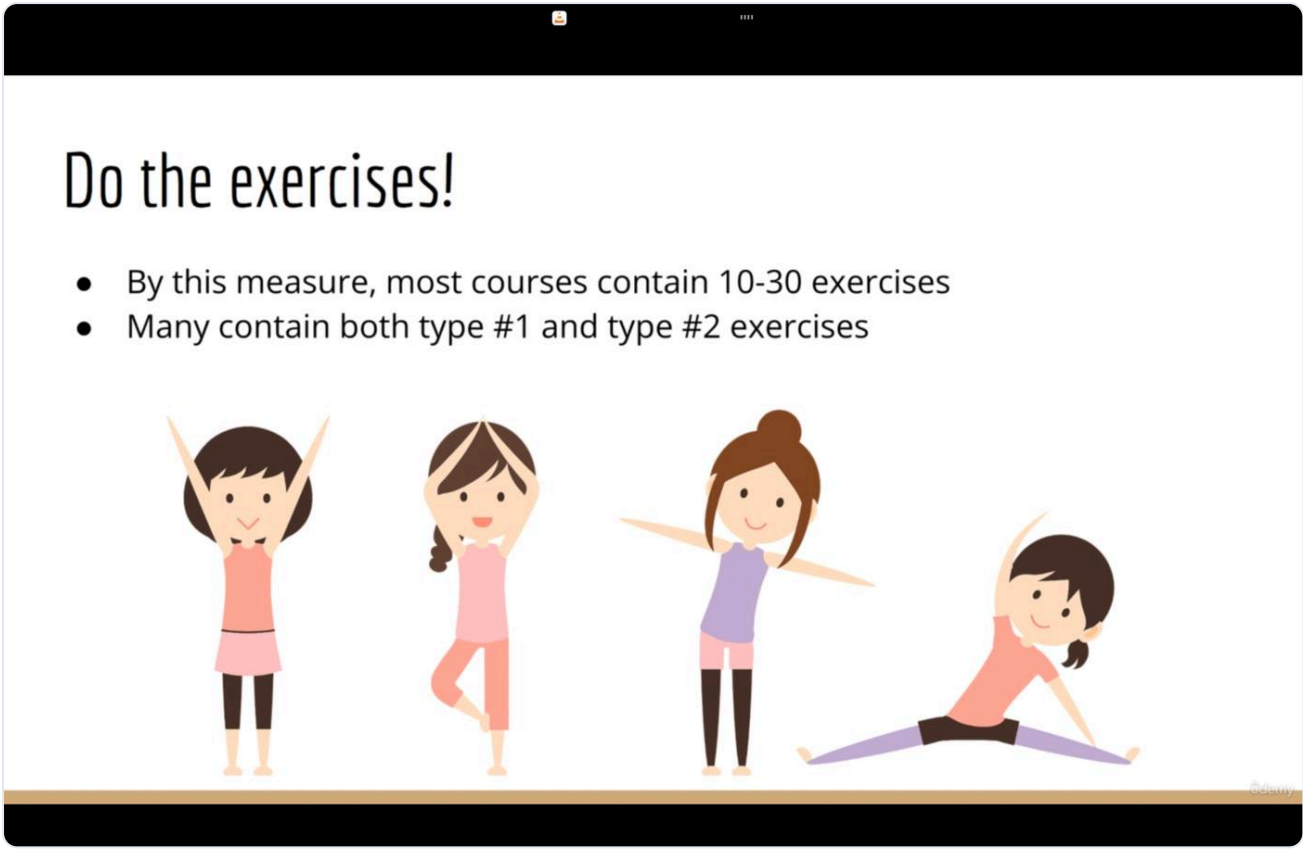

FIGURE 2.41

So in this course I'm providing you with Python code, you'll get that Python code either in a Colab notebook or a python file, but there's really no difference. Remember that in this course, this is a video course you're just seeing code on a screen.

What I use is irrelevant because you can't touch it nor interact with it. I may as well be using Microsoft Word or PowerPoint and it would make no difference. What you do with that Python code is totally up to you. You can take my Python script and paste that into a Jupyter notebook or PyCharm or open it up in Visual Studio code. It's completely your choice.

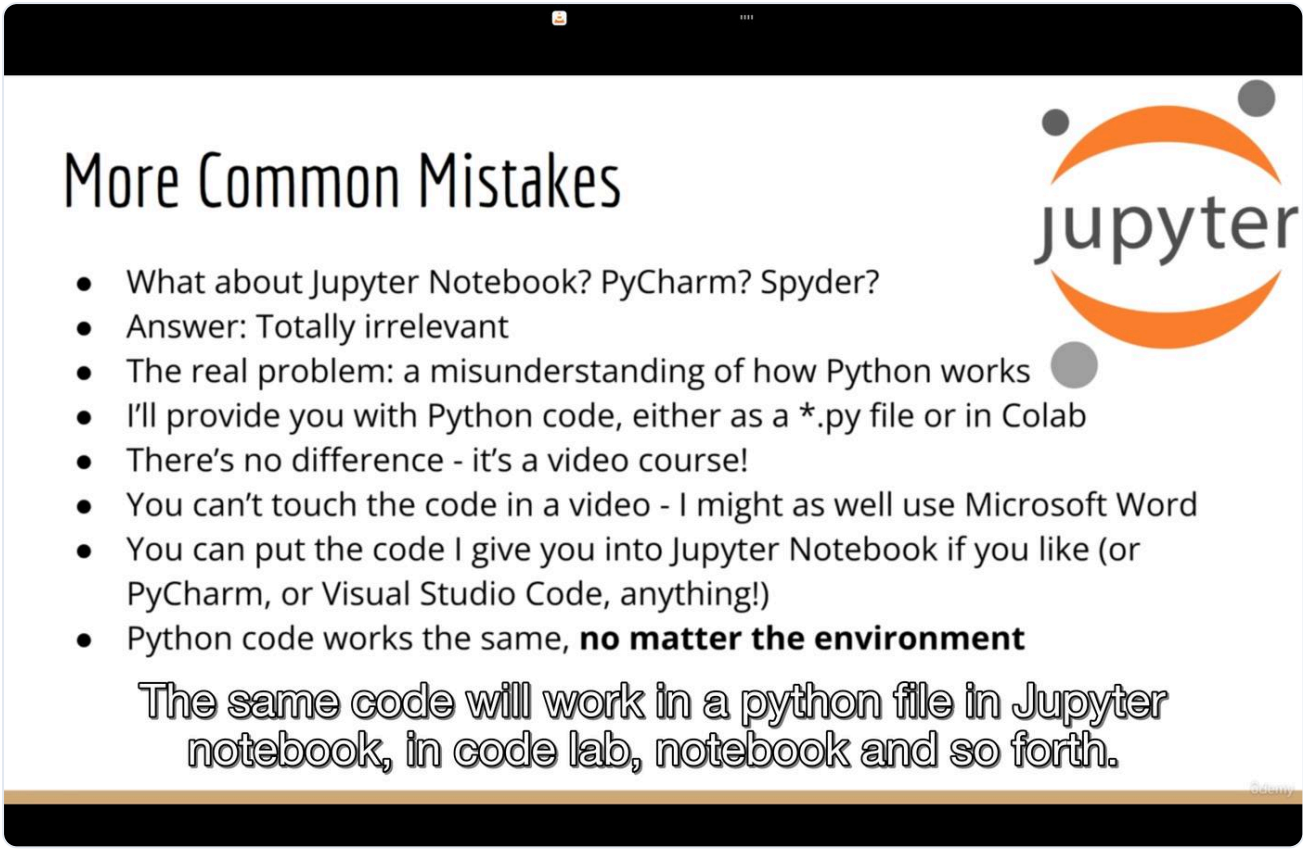

FIGURE 2.42 | More Common Mistakes Jupyter

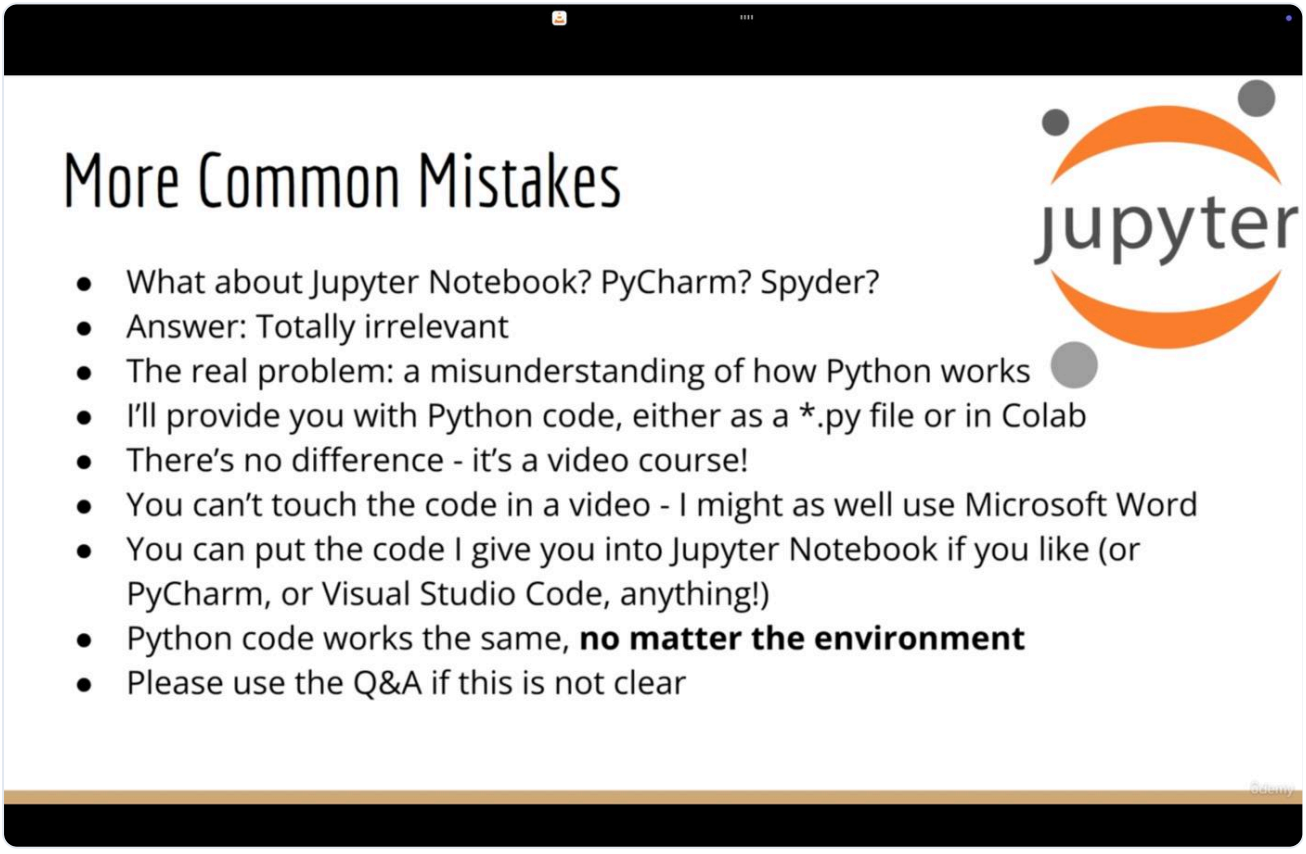

FIGURE 2.43 | More Common Mistakes Jupyter

Python code works the same no matter what environment it's in. The same code will work in a python file in Jupyter notebook, in Colab, notebook and so forth.

And if you're having trouble understanding this, please use the Q&A so I can help you sort it out.

3 · Where to get the code, notebooks, and data

So in this lecture, we are going to discuss where to get the code for this course. The code for this course mostly comes in the form of CoLab notebooks. However, it is also possible for some of the code in this course to be provided in a plain text Python files which will be hosted on GitHub. So how can you tell the difference?

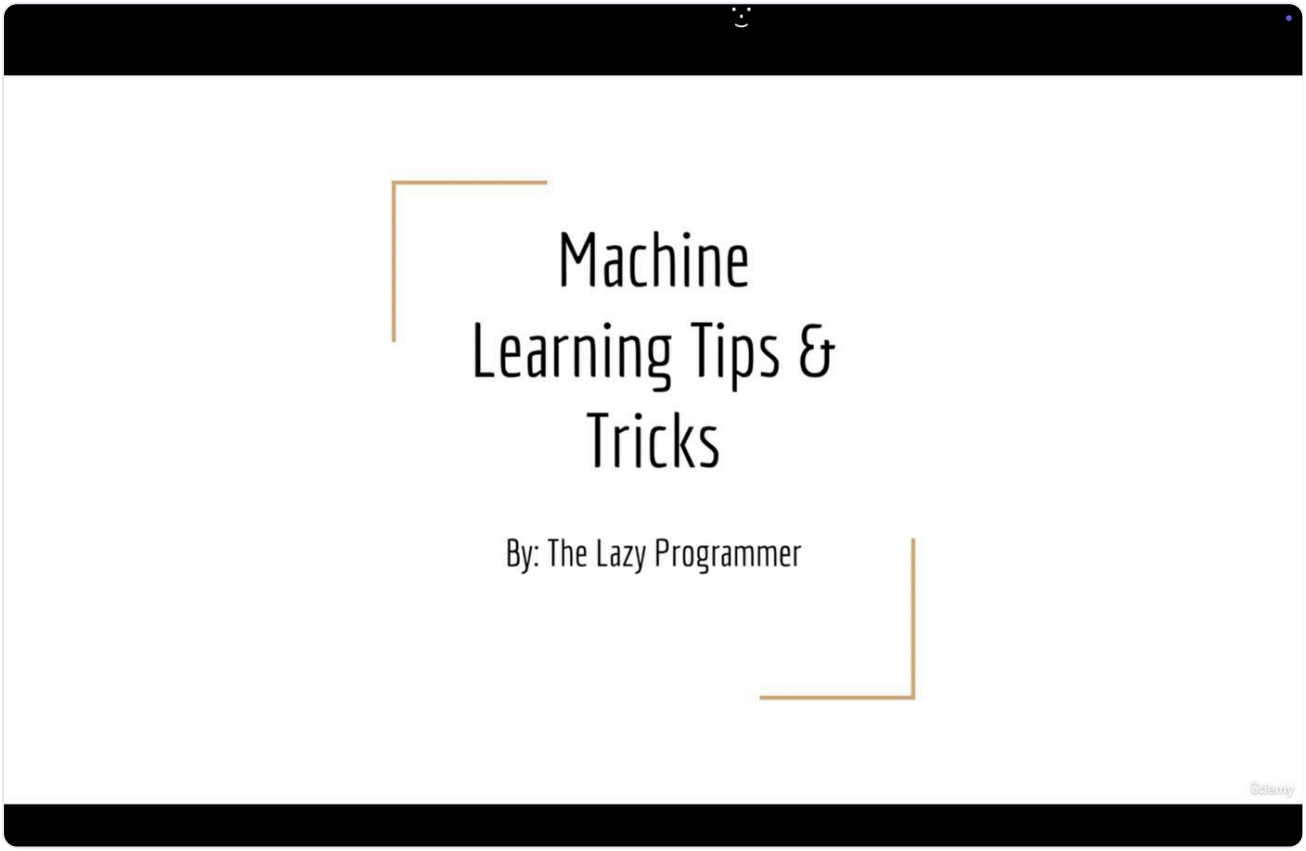

FIGURE 2.44 | Learning Lips &

Well, it's very simple. If you see a notebook interface on the screen, that means that code will be provided in the form of a notebook.

If you see Python in a text editor, then that code will be on GitHub. Now, the main thing I want you to understand is that the notebooks are not on GitHub. So the notebooks are not on GitHub.

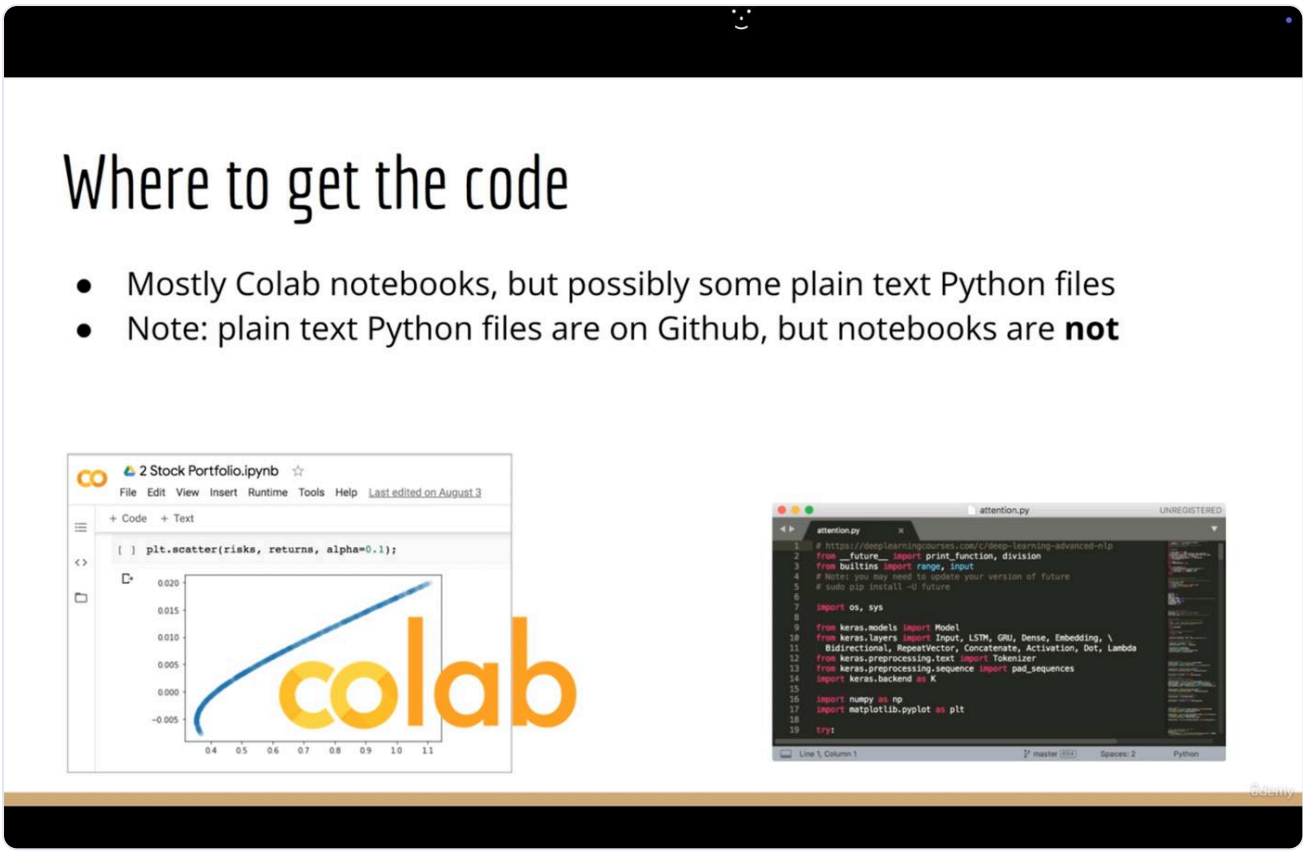

FIGURE 2.45 | Where to Get the Code

So specifically, how can you get these resources? The answer is to simply click on the resources tab attached to this lecture, then click on the link that says code link. This will take you to a page that contains a links to every code example in this course, whether it was in the form of a notebook or a python file hosted on GitHub, everything is on the same page.

So I want to make it clear that the notebooks should be considered as a bonus for this course. In a normal course, that is like a college course.

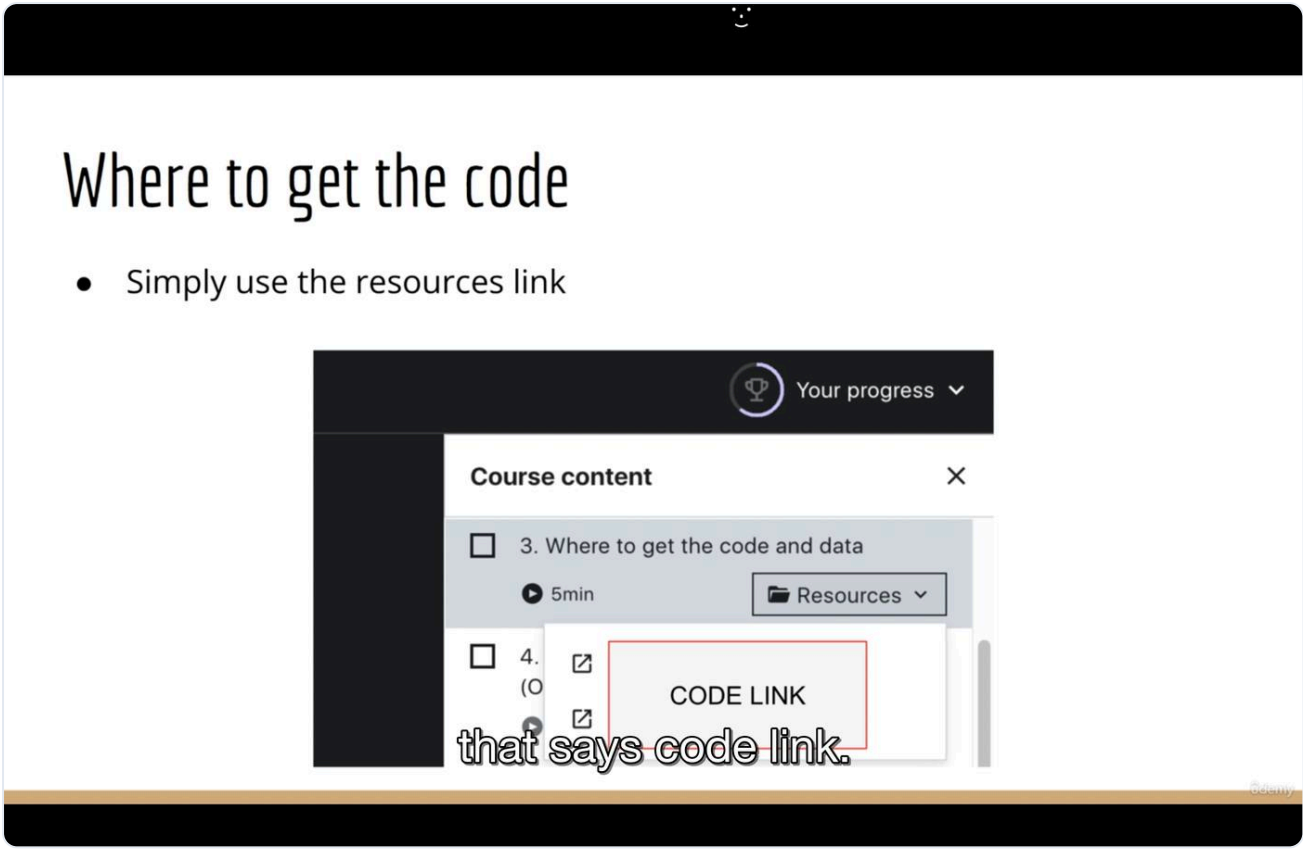

FIGURE 2.46 | Where to Get the Code

Coding would be your homework. Now, only on this website have I seen so many students believe that it's the instructor's responsibility to complete homework for the student. To me, this is wild. So let this be a warning for those students.

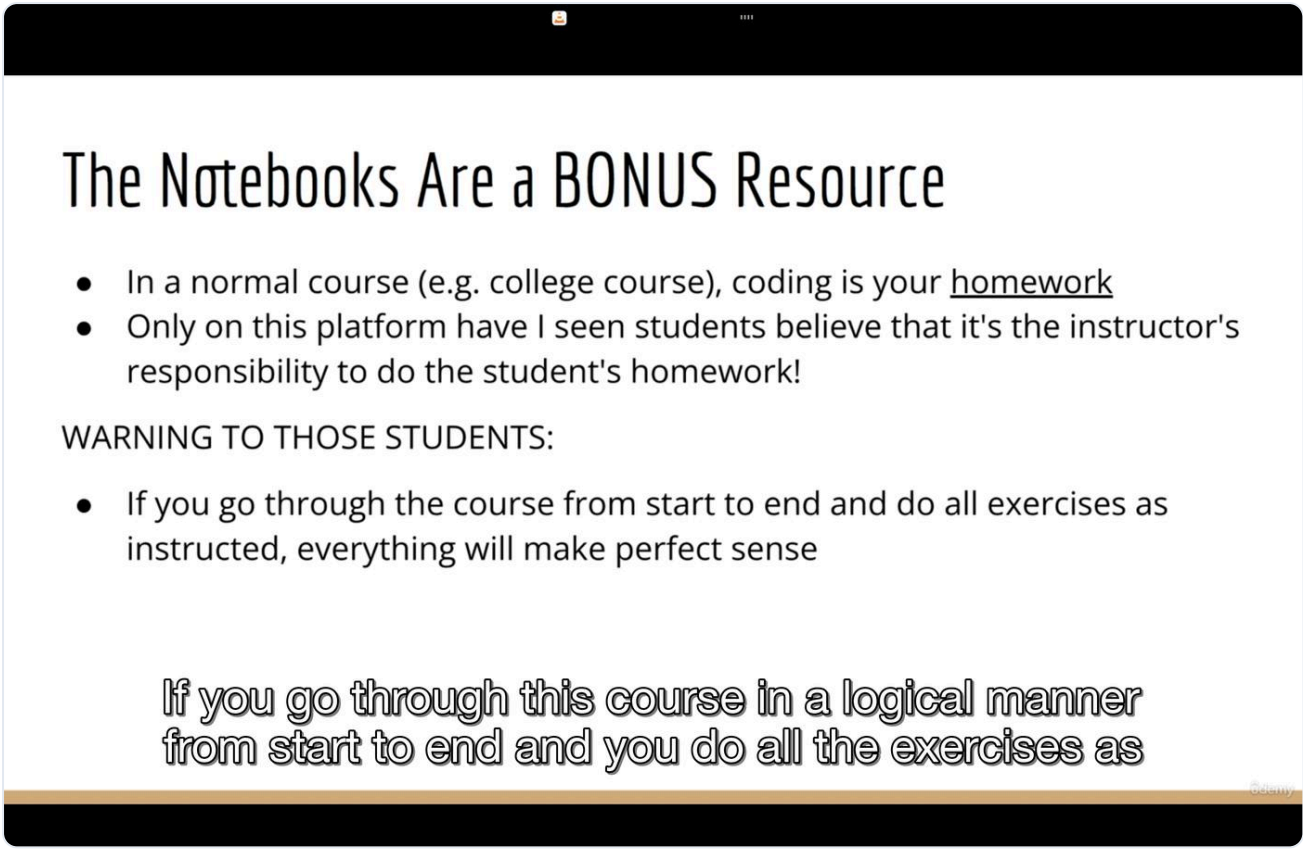

FIGURE 2.47 | The Notebooks Are a BONUS Resource

If you go through this course in a logical manner from start to end and you do all the exercises as instructed, everything will make perfect sense.

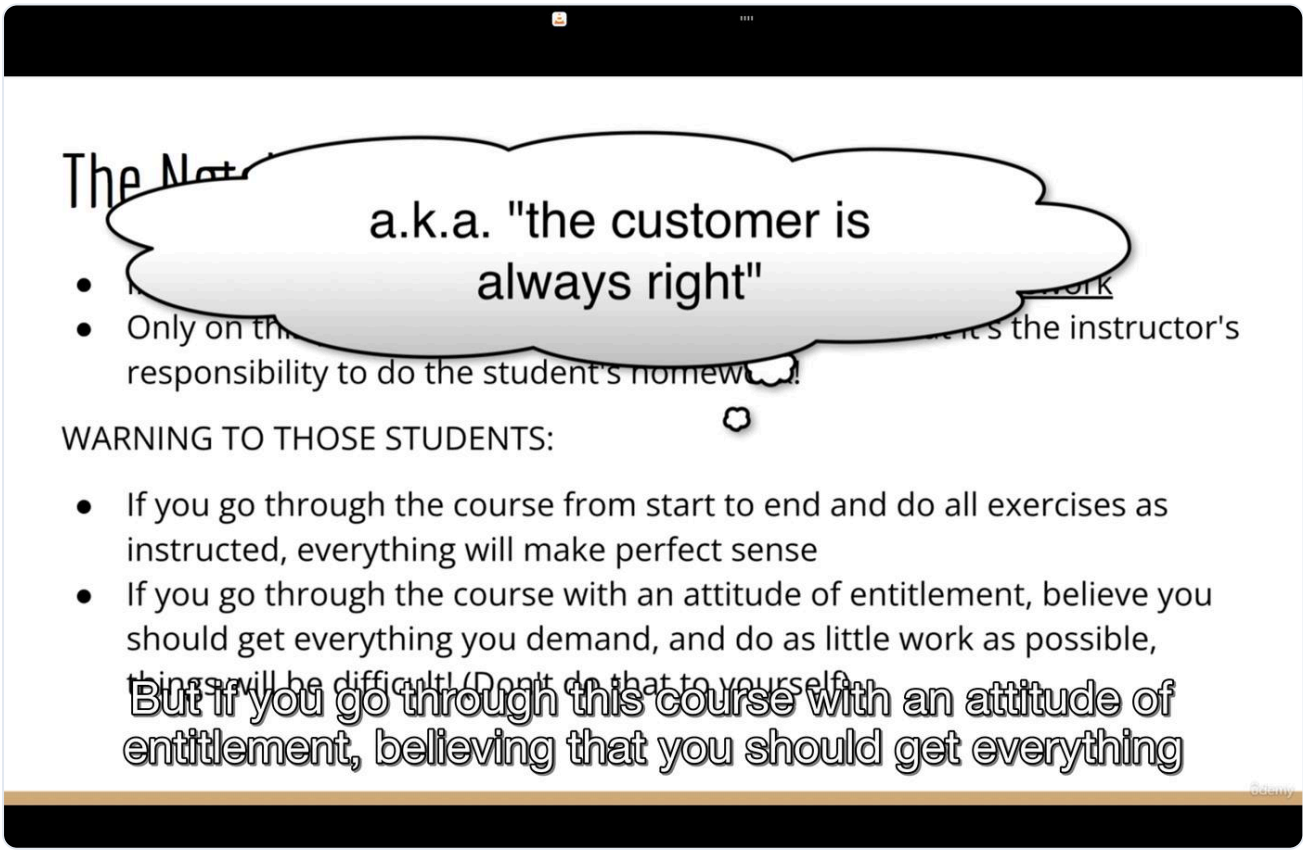

FIGURE 2.48

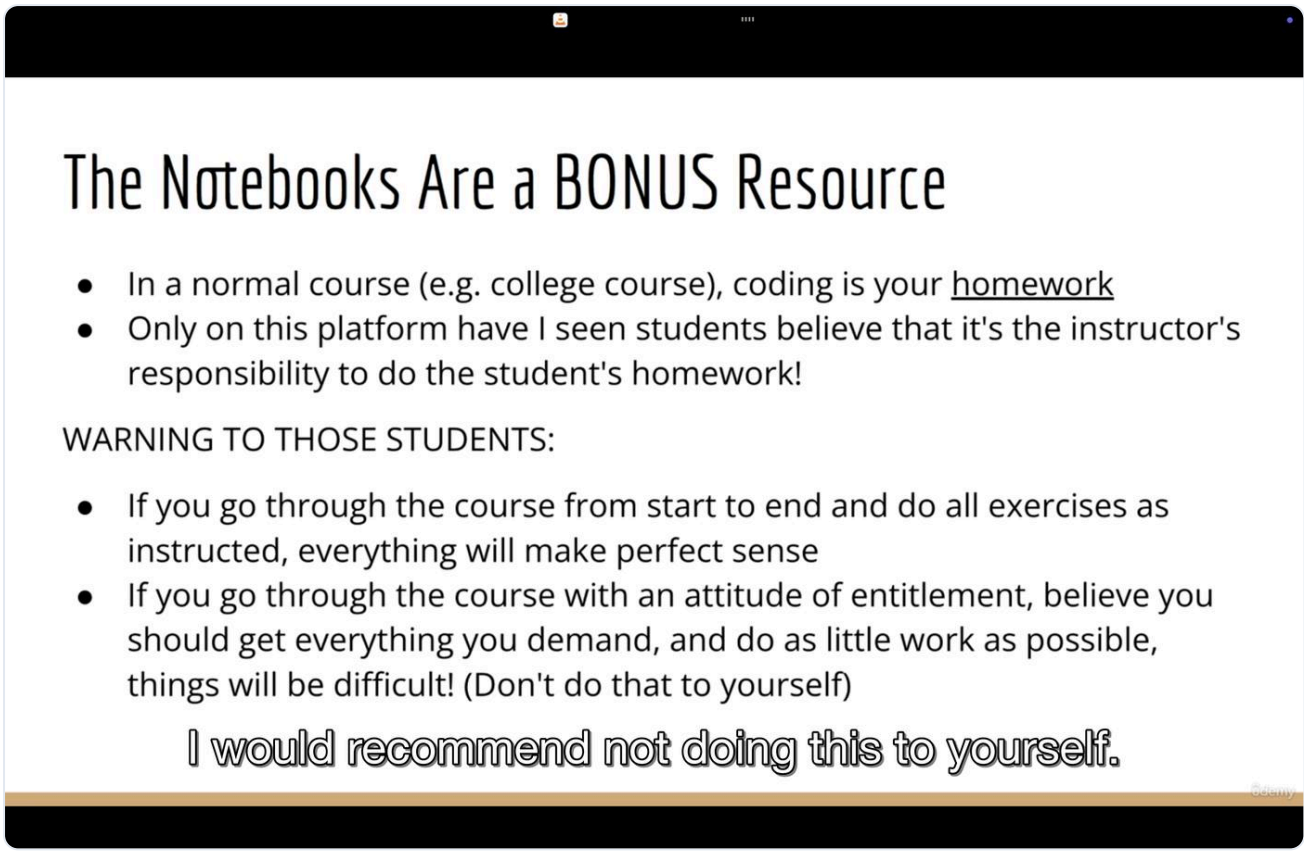

FIGURE 2.49 | The Notebooks Are a BONUS Resource

But if you go through this course with an attitude of entitlement, believing that you should get everything you demand and do as little work as possible, things will get difficult. I would recommend not doing this to yourself.

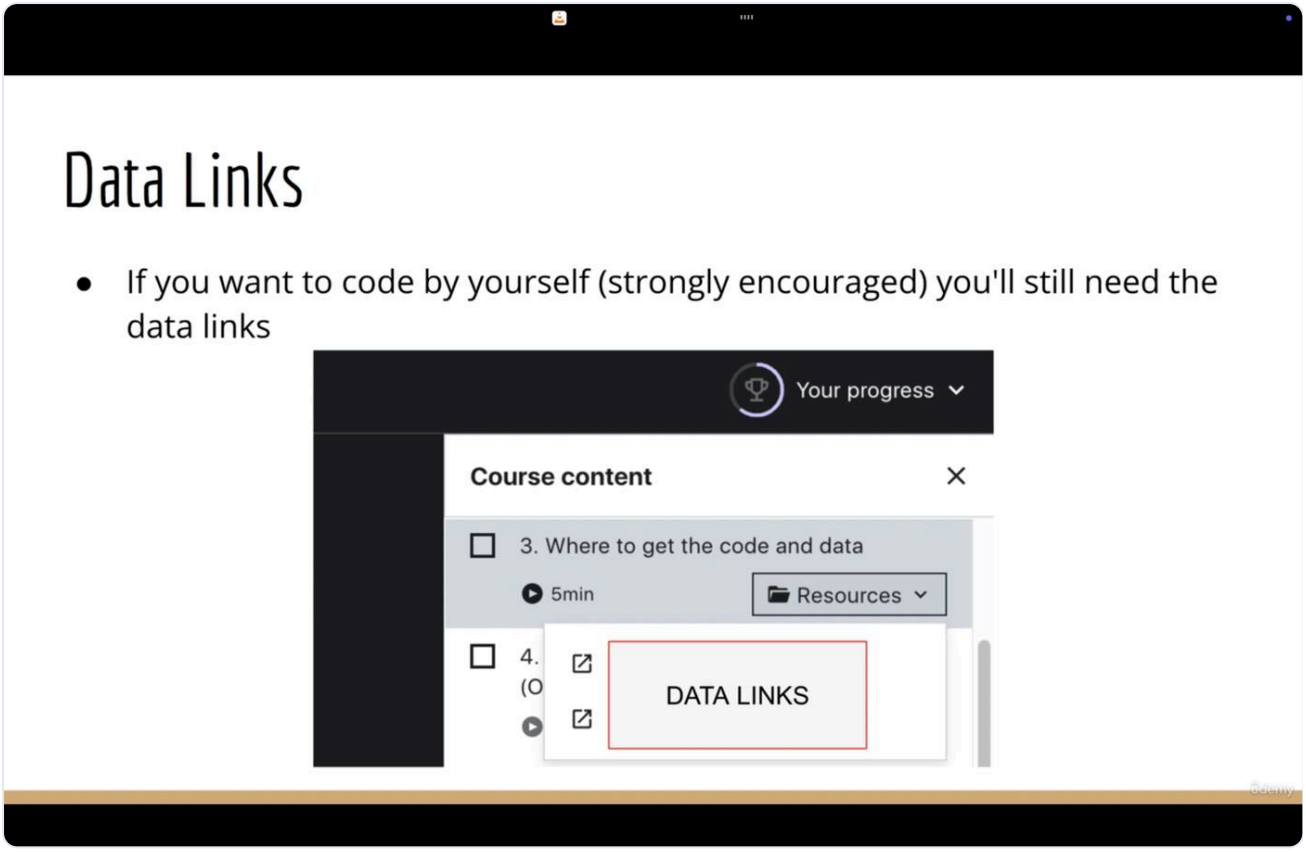

FIGURE 2.50

Now, for those of you who would rather learn to code by themselves, which I always encourage, you will still need the data links for this course. As mentioned earlier, I don't want you typing data links by hand or trying to Google.

Search the data links which usually results in you downloading the wrong data sets. So to get the data links, simply click on the resources tab and then click the link that says data links. Now, there's one mistake that many students make, even after I've mentioned it once or twice. Therefore, I found that it is necessary to mention it three or more times. So far, this seems to be the magic number.

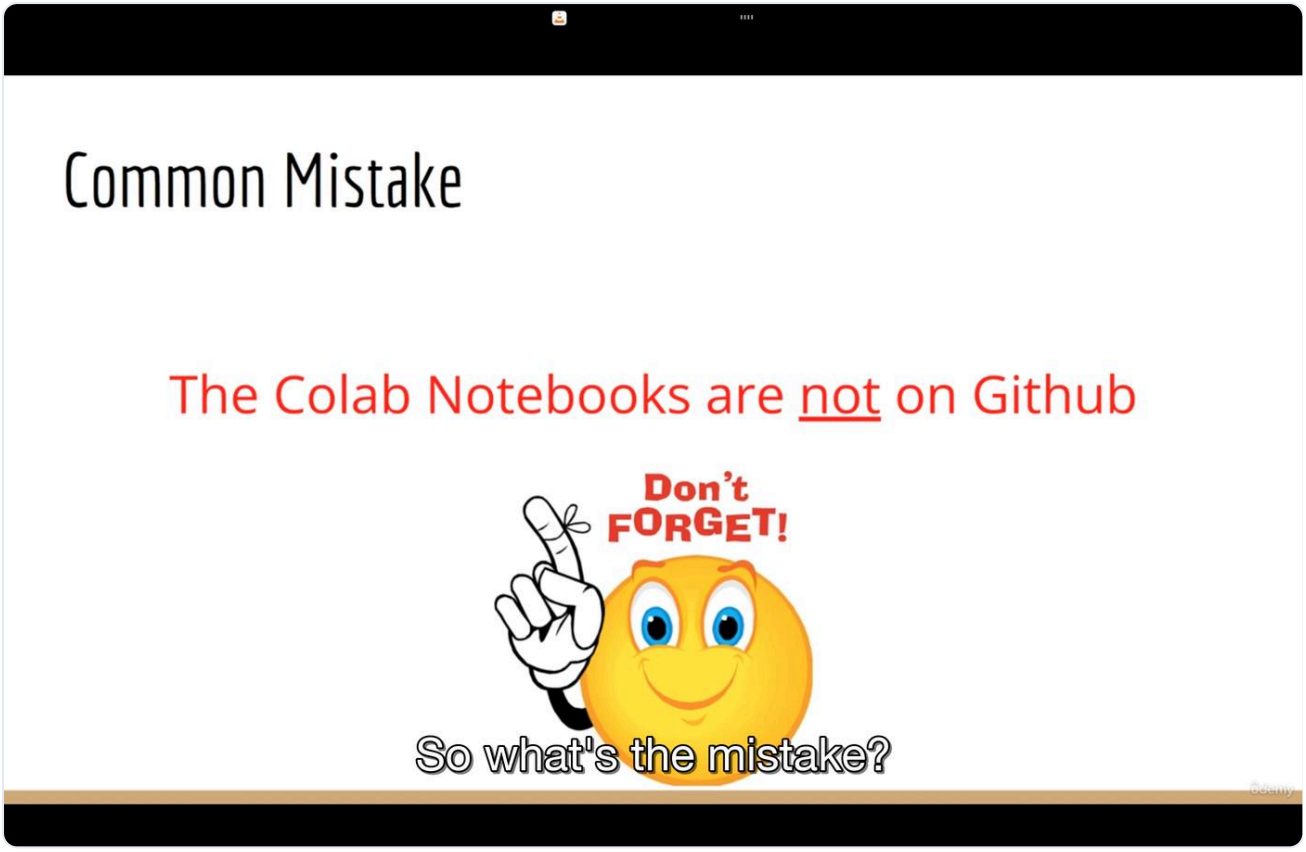

FIGURE 2.51 | Common Mistake

So what's the mistake? It's that people forget that the CoLab notebooks are not on GitHub.

So I repeat, the CoLab notebooks are not on GitHub. And let me repeat that one more time for good measure. The CoLab notebooks are not on GitHub, so hopefully that's enough to get the point across. Thanks for listening and I'll see you in the next lecture.

4 · Are You Beginner, Intermediate, or Advanced All are OK!

So in this short lecture, I'm going to explain how you, as the student, should navigate this course. This is something which didn't exist in my older courses and doesn't exist at all in other courses. So this may be new for you. For this course, there is a wide range of difficulty from topic to topic.

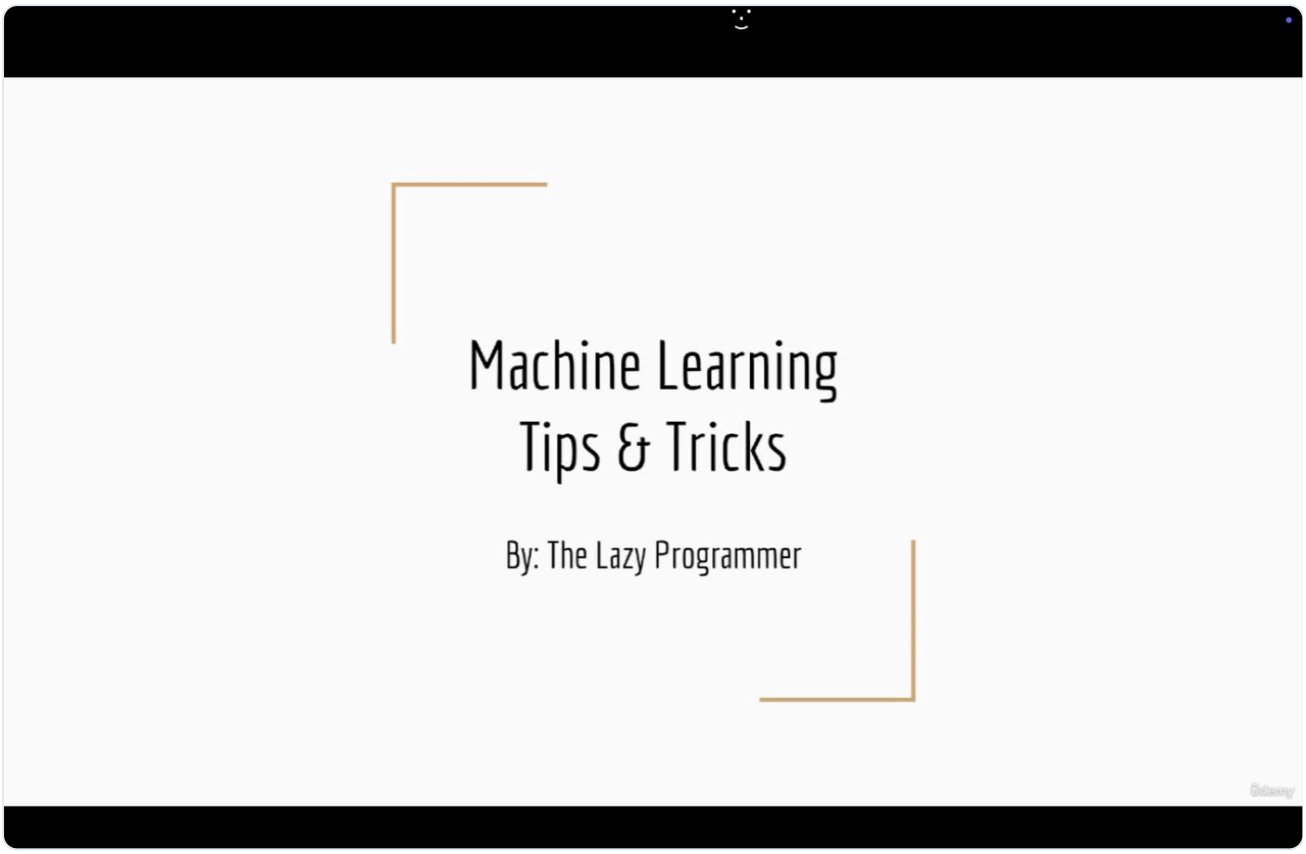

FIGURE 2.52 | Machine Learning

For this reason, I've designated some lectures and some sections by their level, which can be either beginner, intermediate or advanced. Obviously beginner content is the easiest, whereas intermediate is a bit more difficult.

Well, advanced is the most difficult. So please watch out for these designations which will be in the titles of either the lectures or full sections, typically in parentheses.

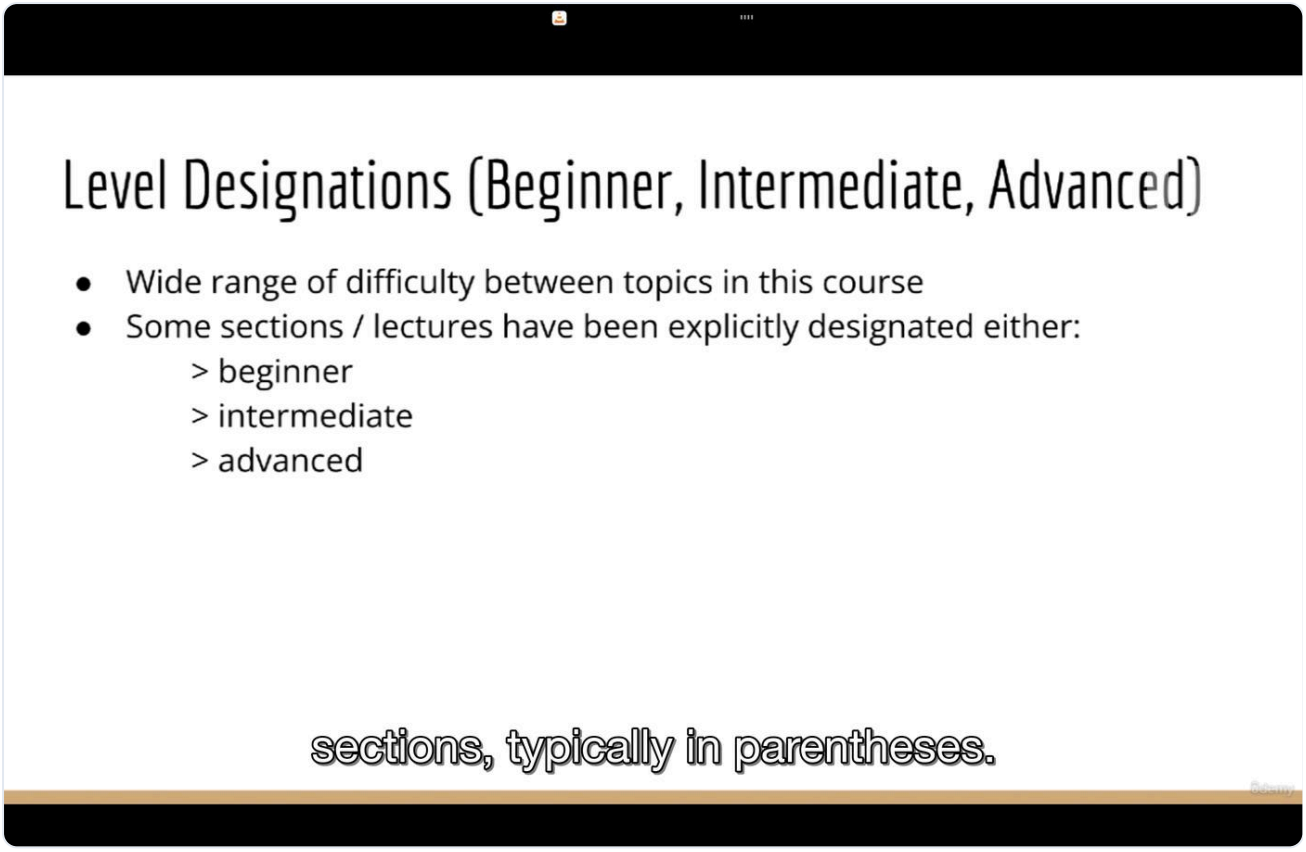

FIGURE 2.53 | Level Designations (Beginner, Intermediate, Advanced)

Now, the reason I've done this is sometimes I encounter beginner students who bite off more than they can chew, and as a result, they become frustrated when the content is too difficult for their current abilities. Of course, there is no need for that. Simply be more aware of your abilities and accept yourself as you are.

As a teacher, I always encourage you to improve your skills such that you'll be able to handle more advanced content.

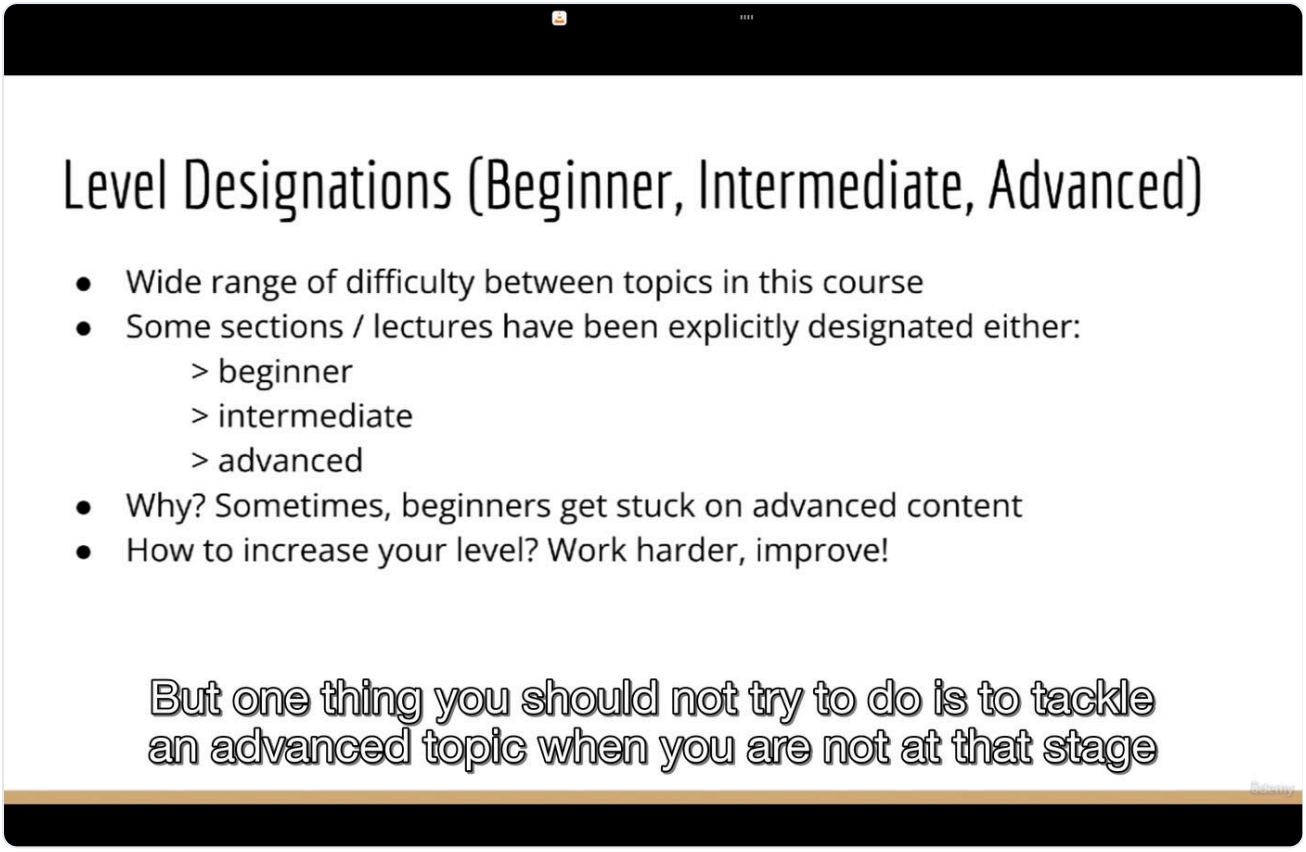

FIGURE 2.54 | Level Designations (Beginner, Intermediate, Advanced)

But one thing you should not try to do is to tackle an advanced topic when you are not at that stage in your journey.

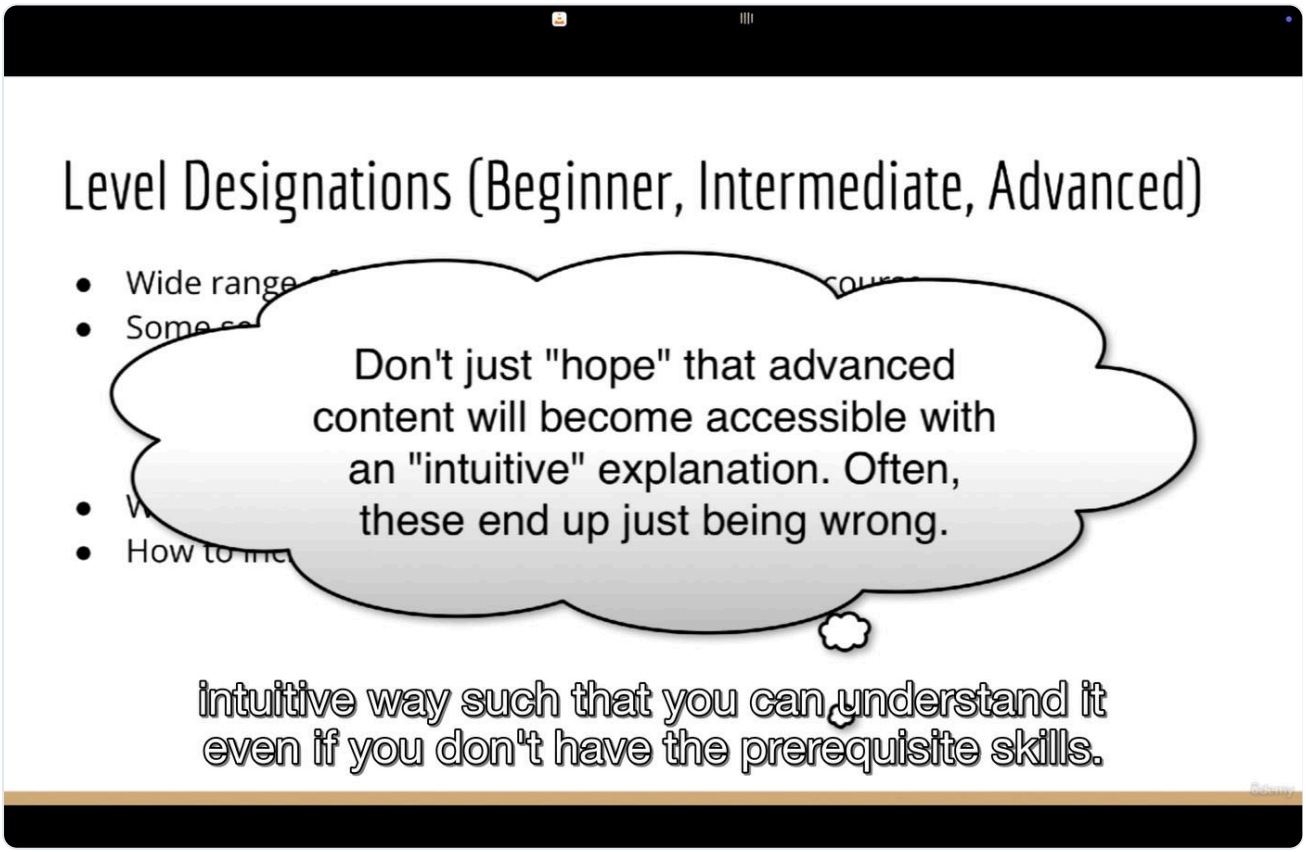

FIGURE 2.55 | Level Designations (Beginner, Intermediate, Advanced)

I don't want you to hope or expect that the advanced content will somehow be explained in a so-called intuitive way such that you can understand it even if you don't have the prerequisite skills.

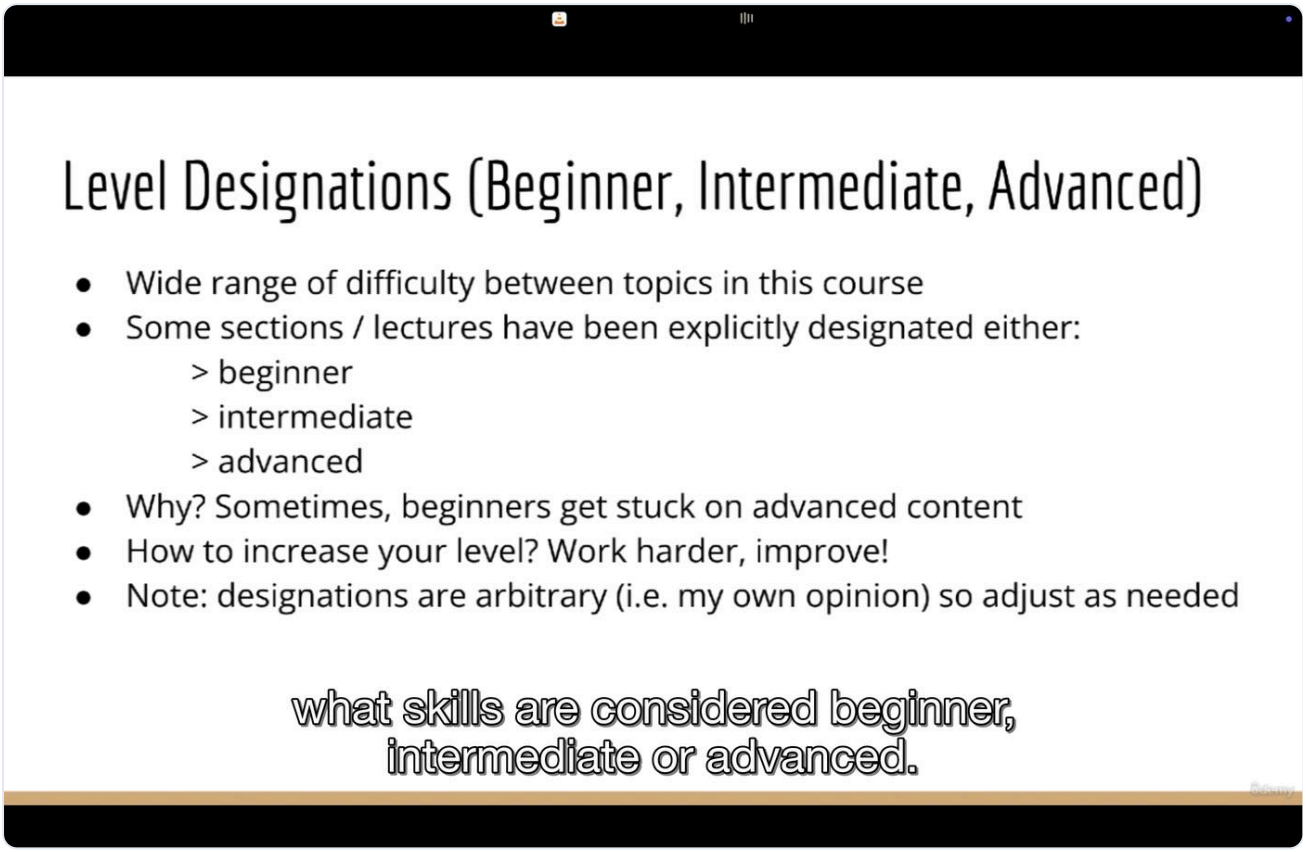

FIGURE 2.56 | Level Designations (Beginner, Intermediate, Advanced)

Now, unfortunately, these designations are somewhat arbitrary because nobody can really agree on precisely what skills are considered beginner, intermediate or advanced.

Therefore, it may take some trial and error on your part and you'll have to adjust as needed. So for example, if you try some intermediate content but it's too difficult for you, then clearly the advanced content will be even more difficult. At the same time, advanced students will get the most out of this course. That's simply because they'll have the abilities to understand all of the content from beginner to intermediate to advanced. So not only the advanced content is for the advanced students, but all the content is for the advanced students.

In this sense, advanced doesn't mean you already know the content, since at that point taking this course would be pointless. Advanced really means you are mentally prepared to handle the more challenging tasks. Conversely, beginner doesn't mean you're new and unfamiliar with these topics. It simply means you are not mentally prepared. So you should use that as motivation to get yourself to an advanced level. If you are not there yet, a lot of the time this is simply due to your experience level.

This is not something I can give to you. It's simply something you have to cultivate yourself. Nothing I can give you will replace the simple act of practice and gaining experience. I would if I could.

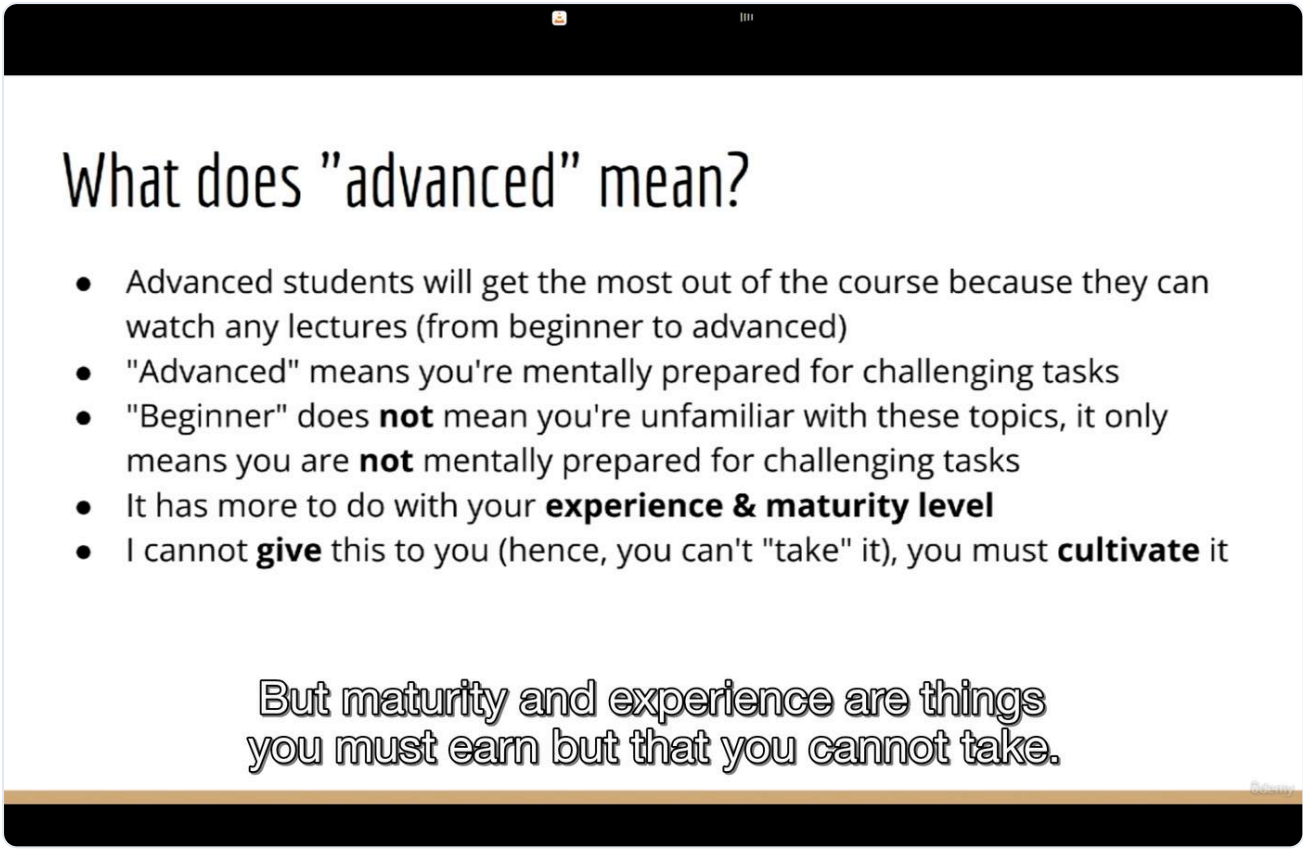

FIGURE 2.57

But maturity and experience are things you must earn but that you cannot take. For example, some projects in this course will simply require a good amount of coding. Some people are good at this, and some people not so good. But luckily, it's a skill that anyone can cultivate as long as they work hard and have patience.

So you may not understand complex code today, but with more practice and experience you will. At that point, the more advanced content will feel more accessible.

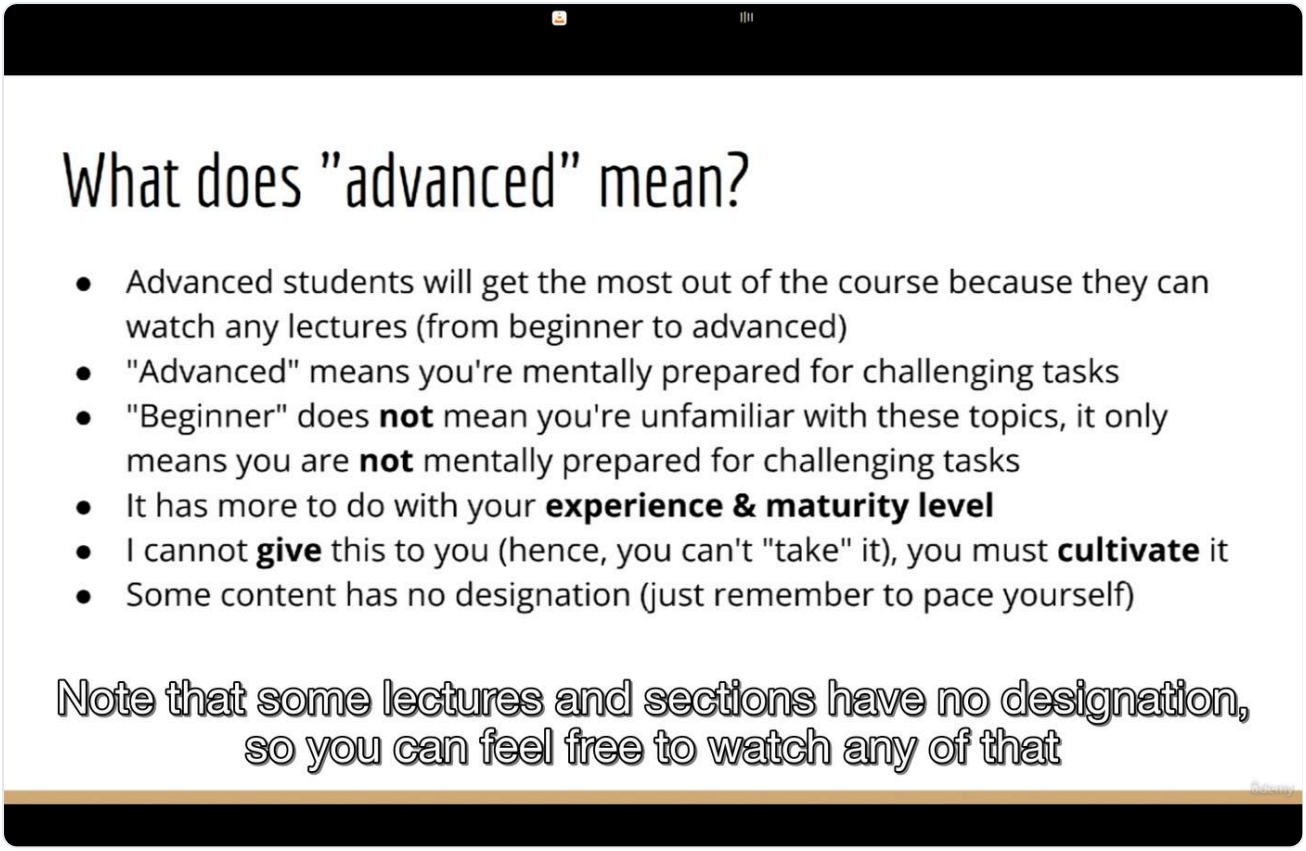

FIGURE 2.58

Note that some lectures and sections have no designation, so you can feel free to watch any of that content regardless of your level. At the same time, remember to pace yourself and remember that there might simply be some things which you do not know in those cases. You should use the Q&A to ask for help.

Now, while there are prerequisites to this course, realize that they are not the be all and end all.

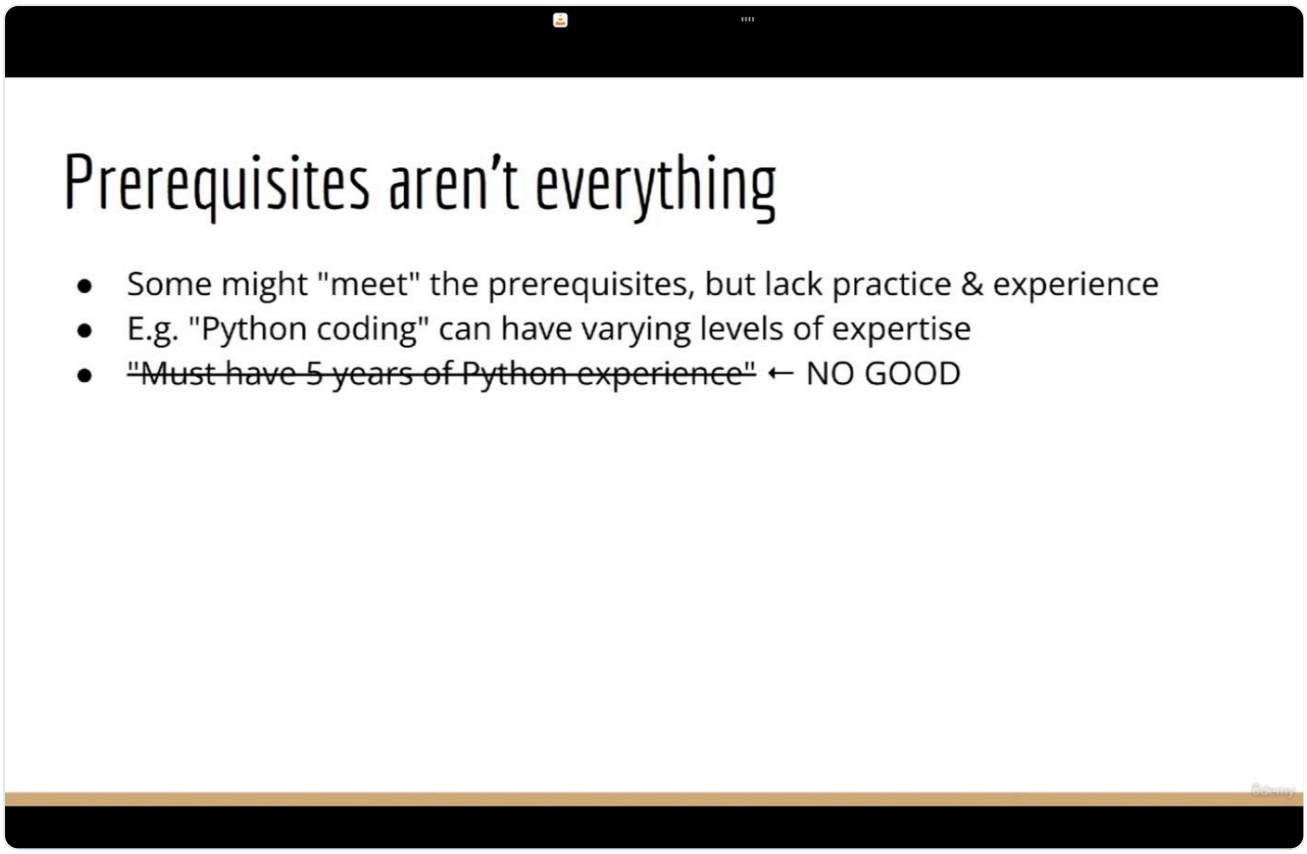

FIGURE 2.59

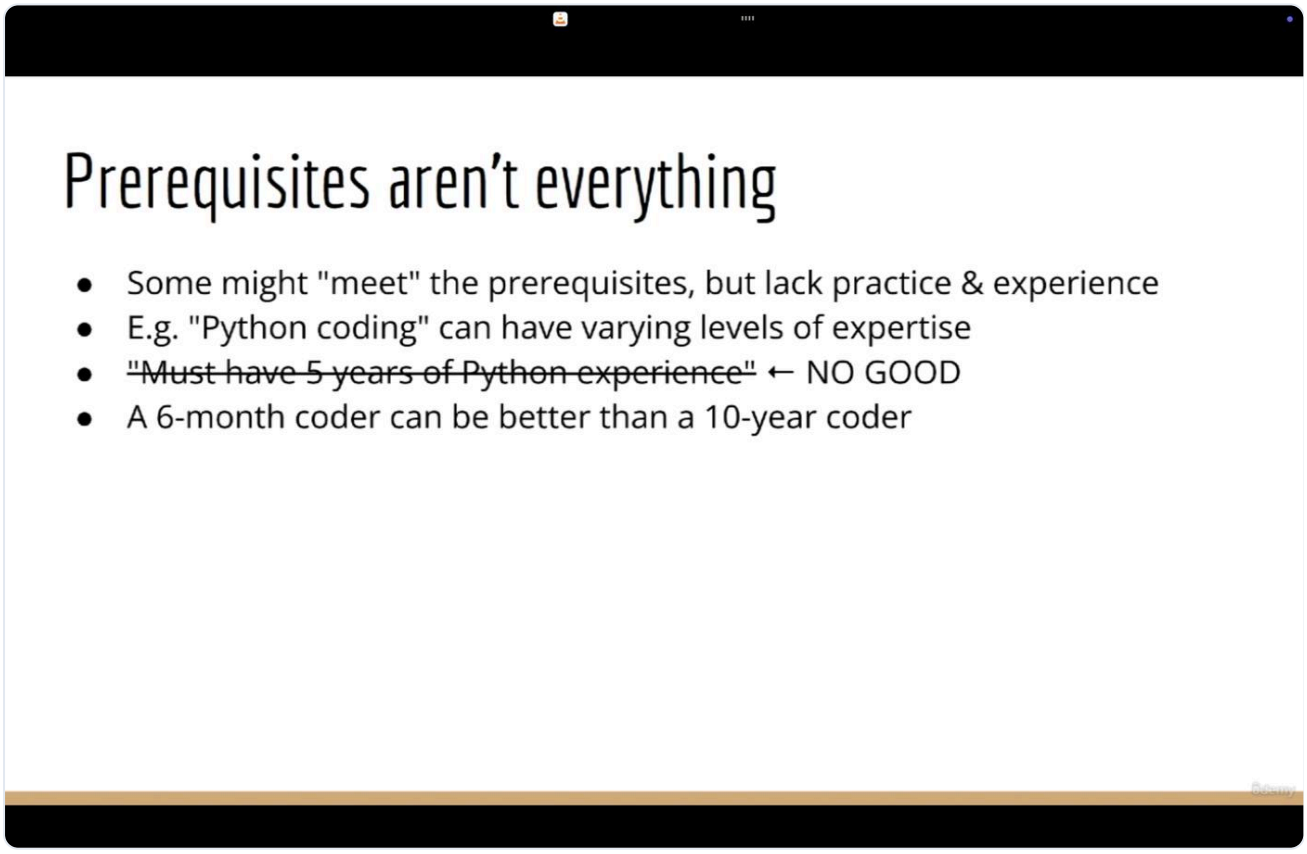

FIGURE 2.60

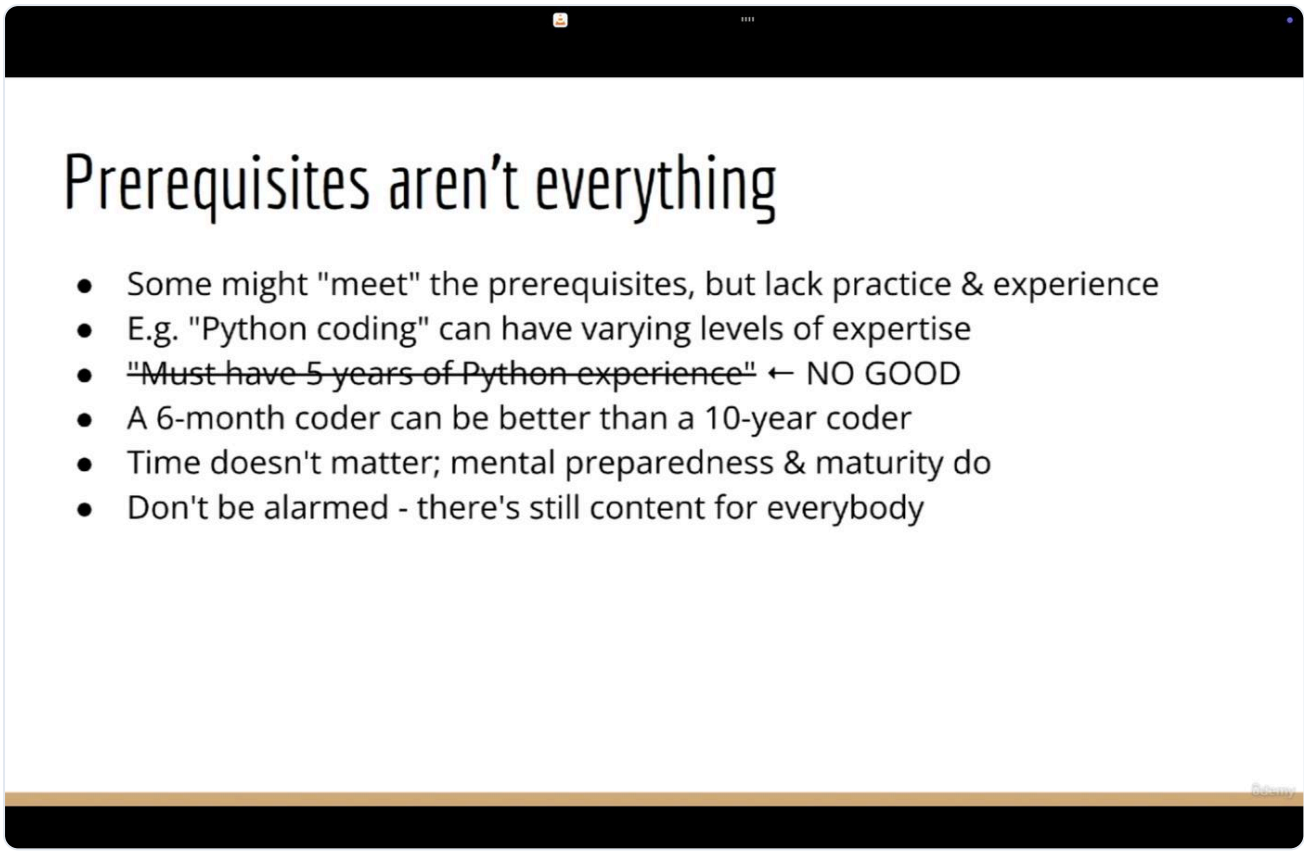

FIGURE 2.61

Sometimes you might meet the prerequisites, but simply lack practice and experience applying those prerequisites. So as an example, Python coding is a prerequisite to this course, but clearly there are varying levels of expertise. At the same time, we can't say something like must have five years of experience in Python because that wouldn't make sense.

Some people have been coding for ten years who are not very good and some people have been coding for six months and are already very good. So it's not time that matters, but rather mental preparedness and maturity. At the same time, you shouldn't be alarmed because there is still content in the course for you to complete no matter your level. So if you find that there is any content in the course that seems beyond your reach, simply cultivate your skills and come back at a later time.

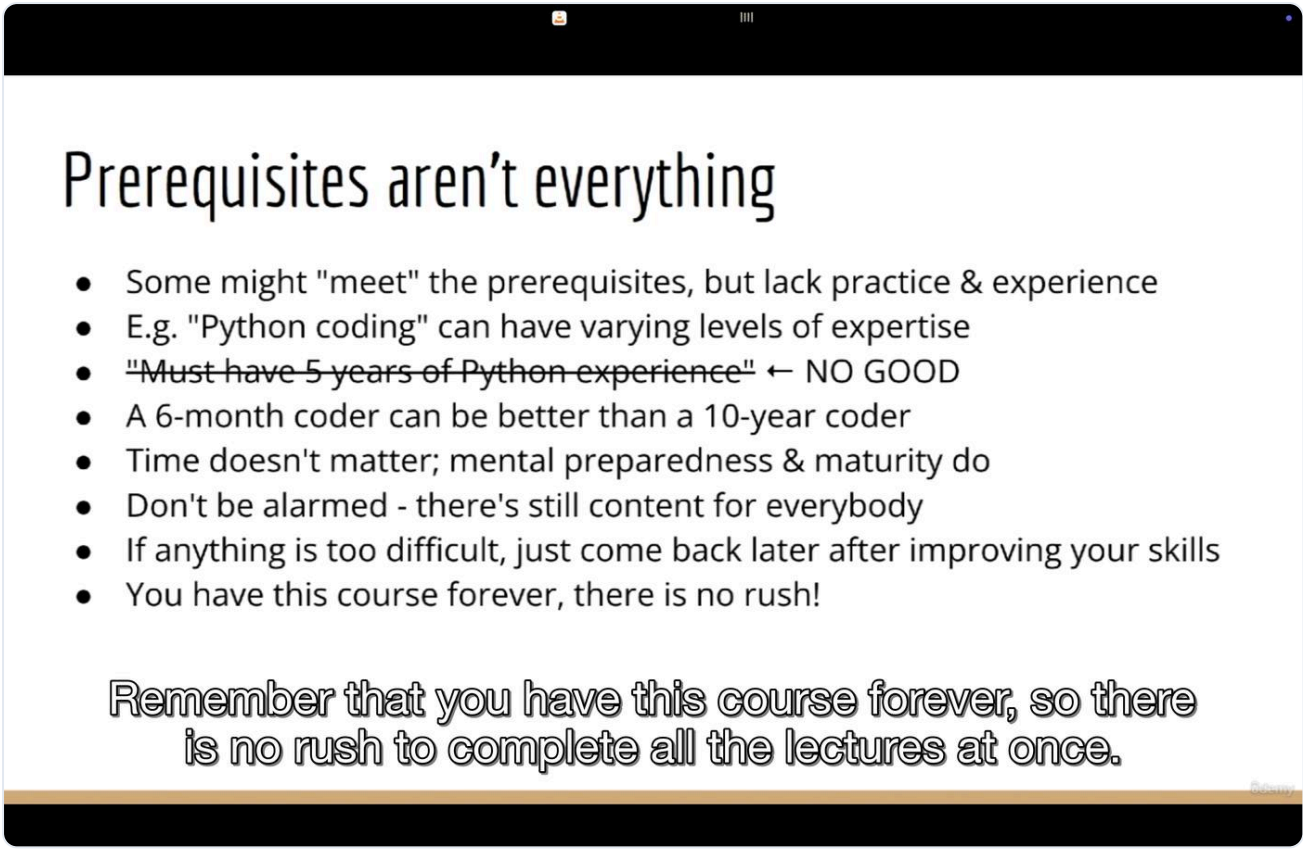

FIGURE 2.62

Remember that you have this course forever, so there is no rush to complete all the lectures at once.

5 · How to Succeed in This Course

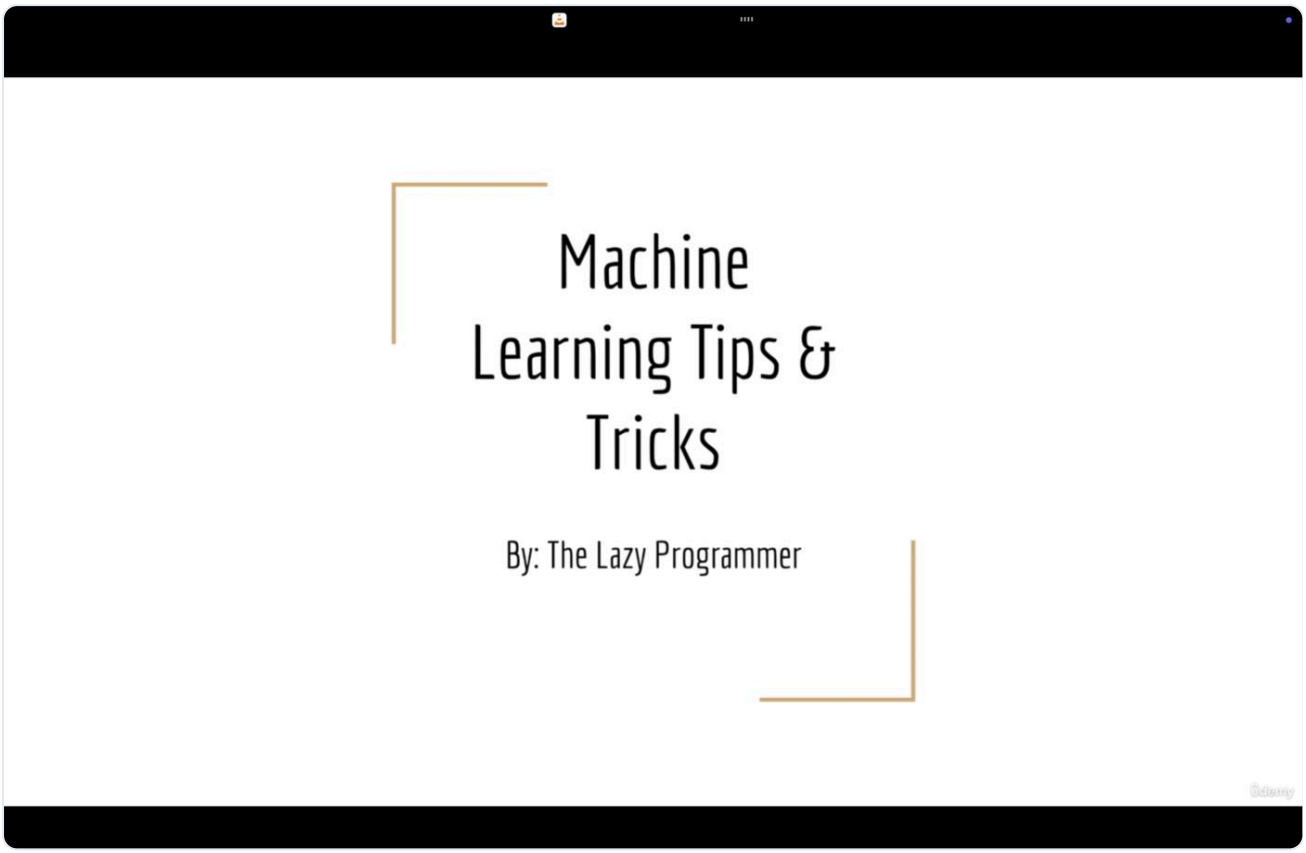

FIGURE 2.63 | Learning Tips &

So in this lecture, I want to give you a few quick tips in order to help you succeed in this course.

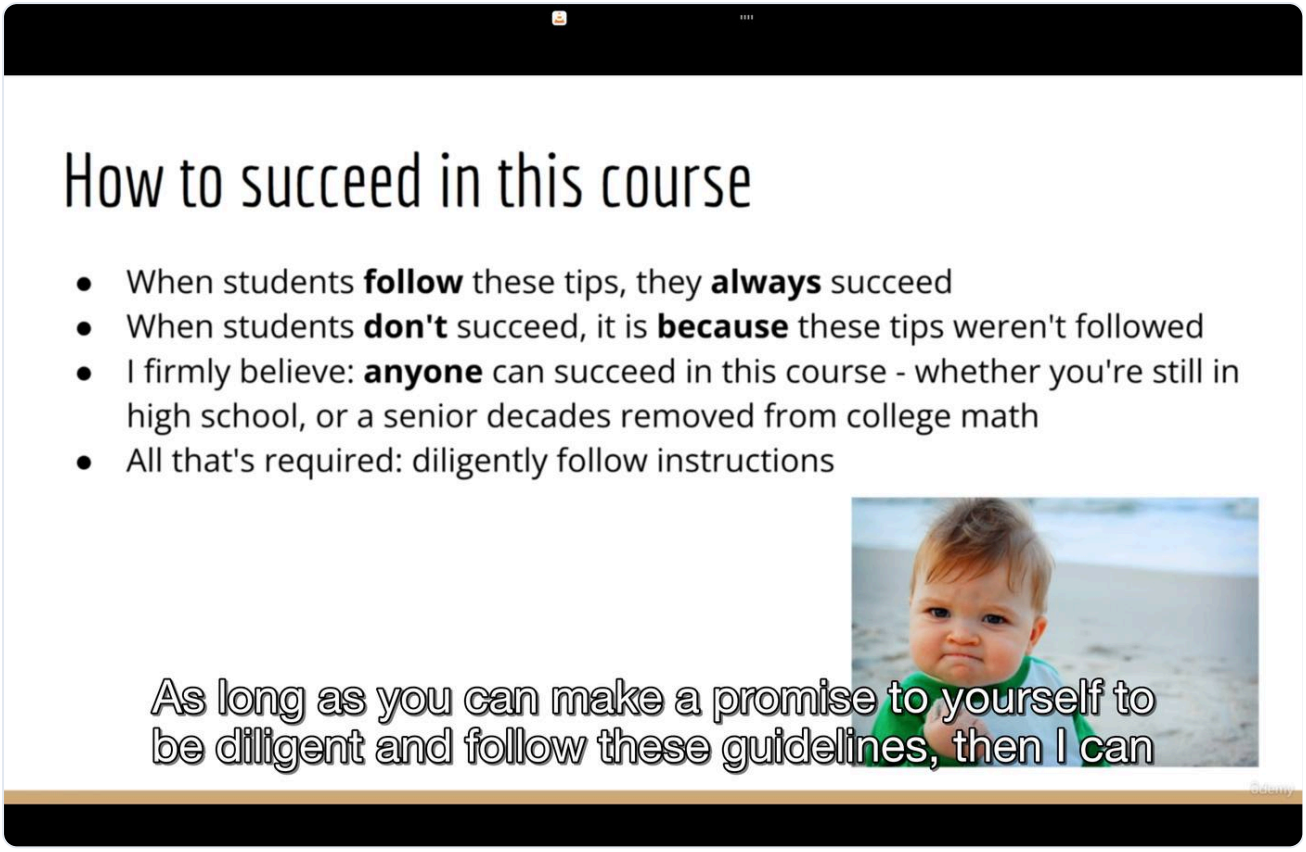

FIGURE 2.64 | How to Succeed in This Course

Now, generally speaking, I find that when students follow these tips, they always succeed. And most often when I see students that are not succeeding, it is because they are not following these tips. Now, I want to first mention that I truly believe that anyone can succeed in this course. This is the case whether you are a teenager still in high school or a senior who's retired.

And it's been decades since you've studied college math. In all cases, it really comes down to following instructions. Being diligent is key. As long as you can make a promise to yourself to be diligent and follow these guidelines, then I can promise you that you will succeed. Now, that being said, what are the tips? In brief, tip number one is to use the Q&A. Too often I see comments from students saying, oh, X, Y, Z is missing from this course. My response is typically, well, actually X, y, z is covered in lecture five.

You simply missed it. But you see, I can't tell you that if you don't ask. So what's the better outcome? You live your whole life never knowing the answer to this question, or you simply ask. Maybe bruise your ego a little bit, but at the very least improve your knowledge and skill. I prefer the outcome that results in you being better and more knowledgeable.

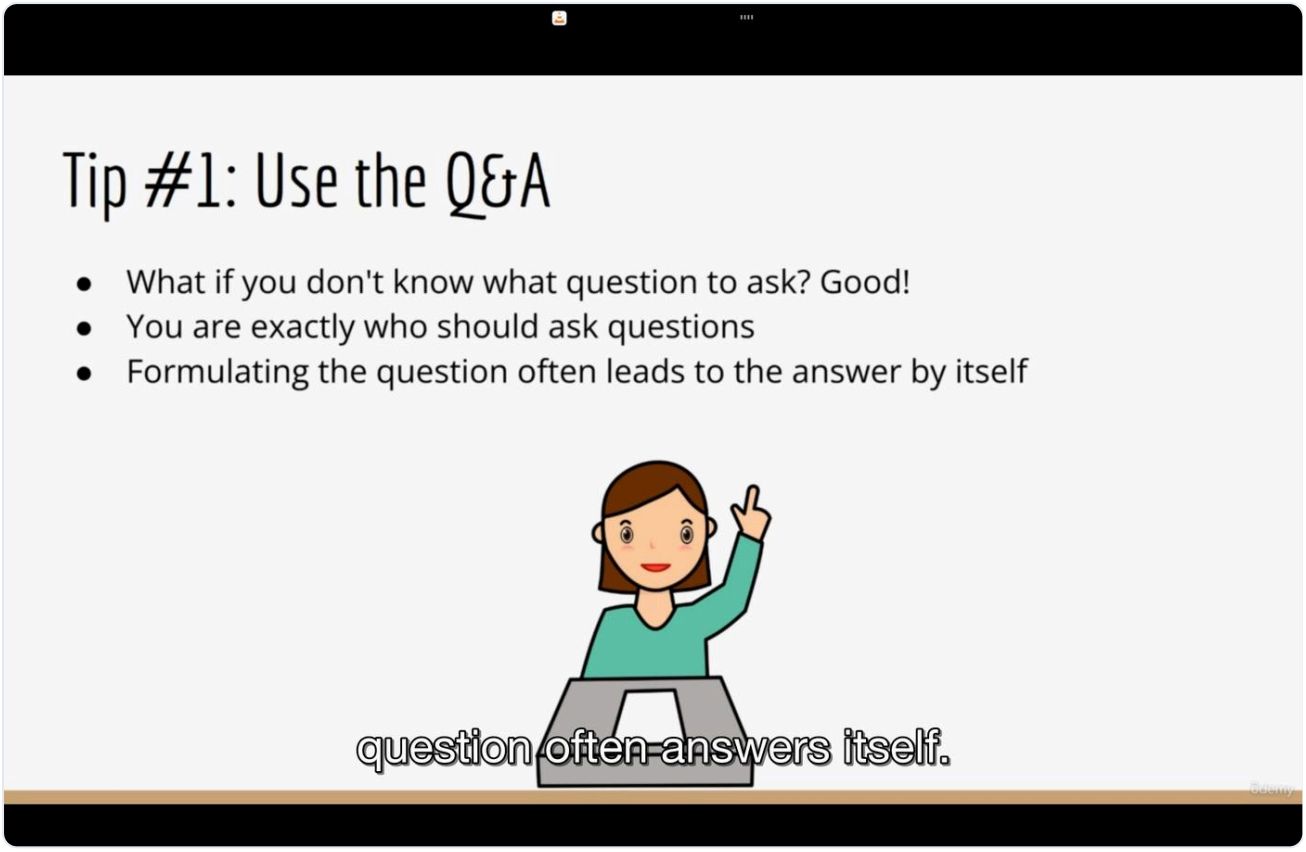

FIGURE 2.65

Now, sometimes students reject this tip and say, but I didn't know what question to ask. This is good.

You are exactly who should be asking questions. The reason you don't know how to formulate your question is because you haven't thought through the subject thoroughly enough. Most of the time, what I've seen is that in the process of trying to come up with a question, the question often answers itself. Okay, so tip number two is to meet the prerequisites to this course. Now, a lot of students think of prerequisites as a barrier to entry. They think, Oh, if the instructor mentions the prerequisites, he's punishing me by making me do more work.

And he's saying I'm dumb for not knowing this stuff. Guys, this is not true. The prerequisites are simply information which exists to help you. It's information. It's not a barrier. It's not anything to attack you. It's simply information. So how does this information help you? Well, after you have this information, if you ever see anything in this course and you think that looks weird, I've never seen this before at that point, it's now a good time to revisit the prerequisites, to make sure that there wasn't something you missed.

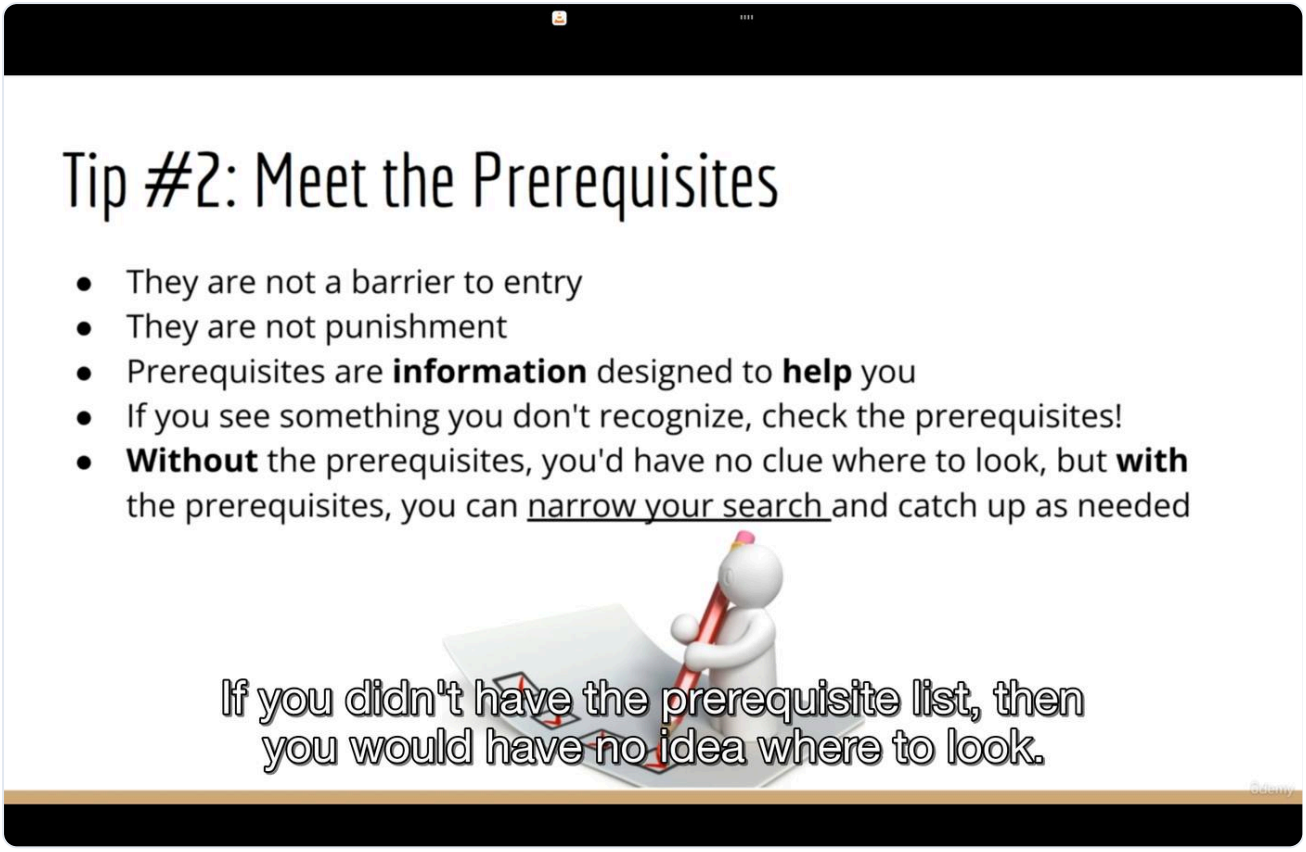

FIGURE 2.66 | They Are Not a Barrier to Entry

If you didn't have the prerequisite list, then you would have no idea where to look. But with the prerequisite list now you have a guide that tells you what you might be missing. Okay, So now on to tip number three. This is something I've mentioned already, but it's so important that it's worth it to repeat. Be actively involved and get your hands dirty in this course.

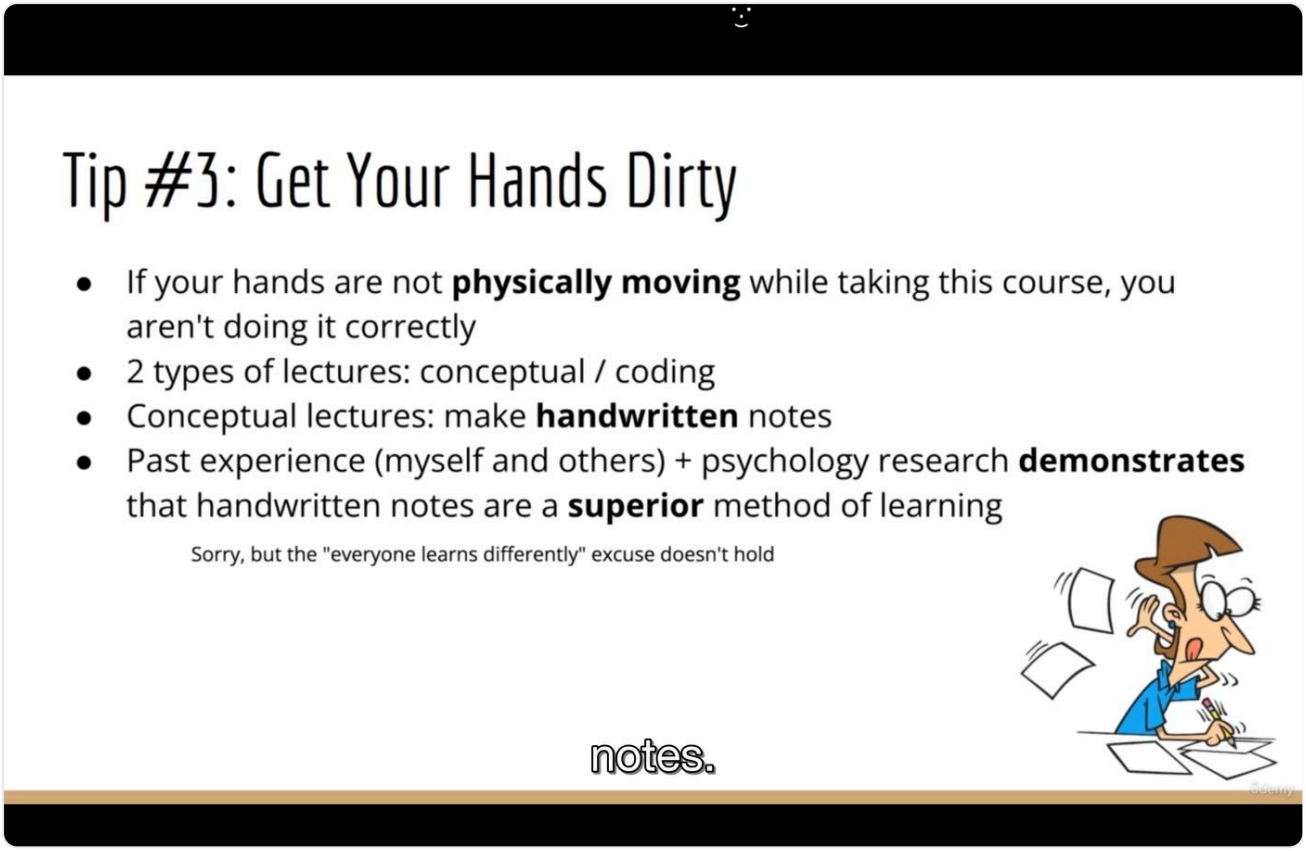

FIGURE 2.67 | 2 Types of Lectures: Conceptual / Coding

Like if your hands are not physically moving while you are taking this course, you are not doing this correctly.

There are two types of lectures in this course. Conceptual lectures and coding lectures. With conceptual lectures, you need to be taking handwritten notes. What I've seen myself in the past, what I've seen many other people say and what a lot of psychological research demonstrates, is that taking handwritten notes is a superior method of learning and absorbing information. This is better than staring at slides, better than watching videos passively and better than even typing notes.

Something about physically writing builds a superior mind muscle connection. Now, when it comes to coding lectures, you already know this, but open up your favorite coding environment and write that code. If it's a course where you're learning an algorithm, then remember that implementing the algorithm is your exercise for the course. If you're just watching the lectures all the way through and twiddling your thumbs, then you are cheating by looking at the solution before completing the exercise yourself.

But whether you're implementing an algorithm or just learning about a library, you must actually write code.

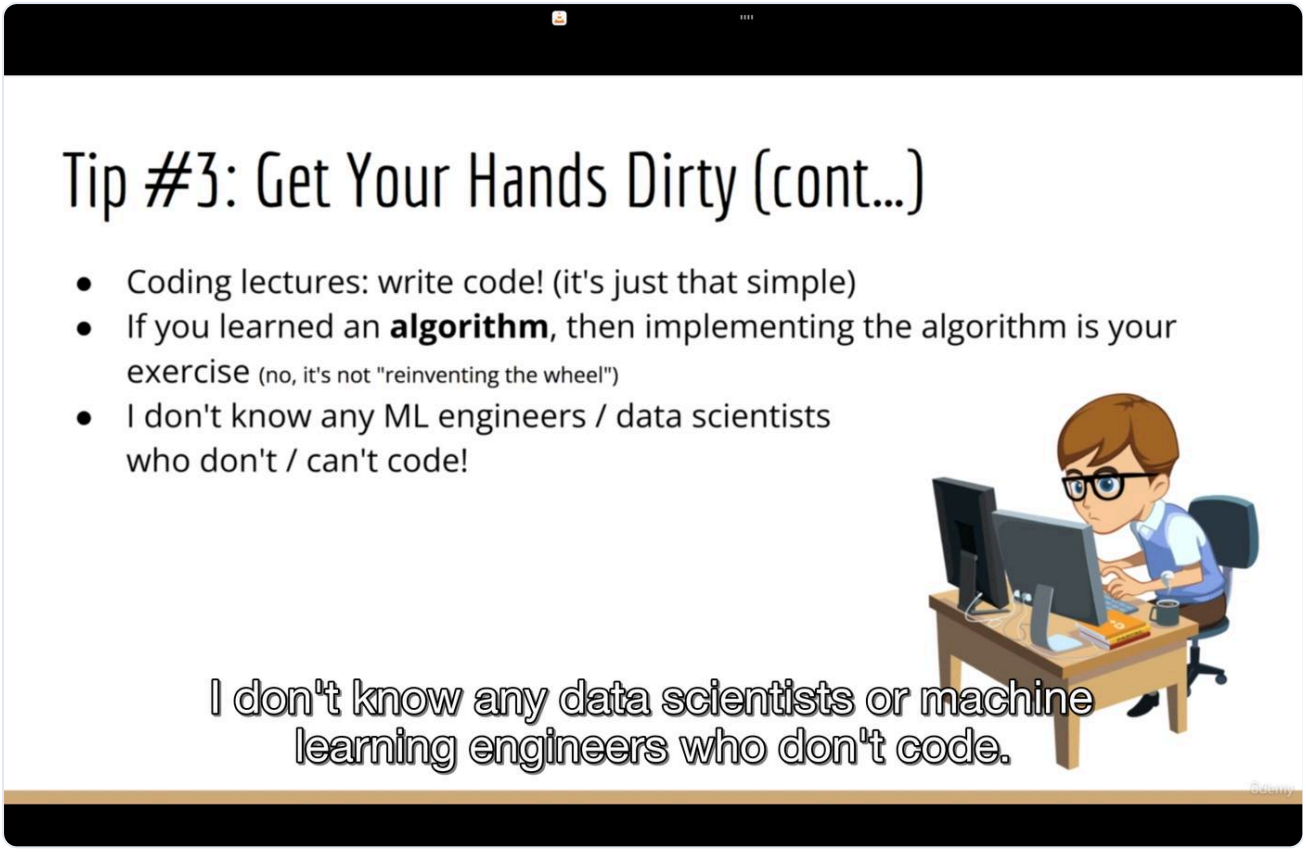

FIGURE 2.68

I don't know any data scientists or machine learning engineers who don't code. Now, I just want to mention that this list is a condensed version of the full How to Succeed Lecture, which can be found in the FAQ of this course.

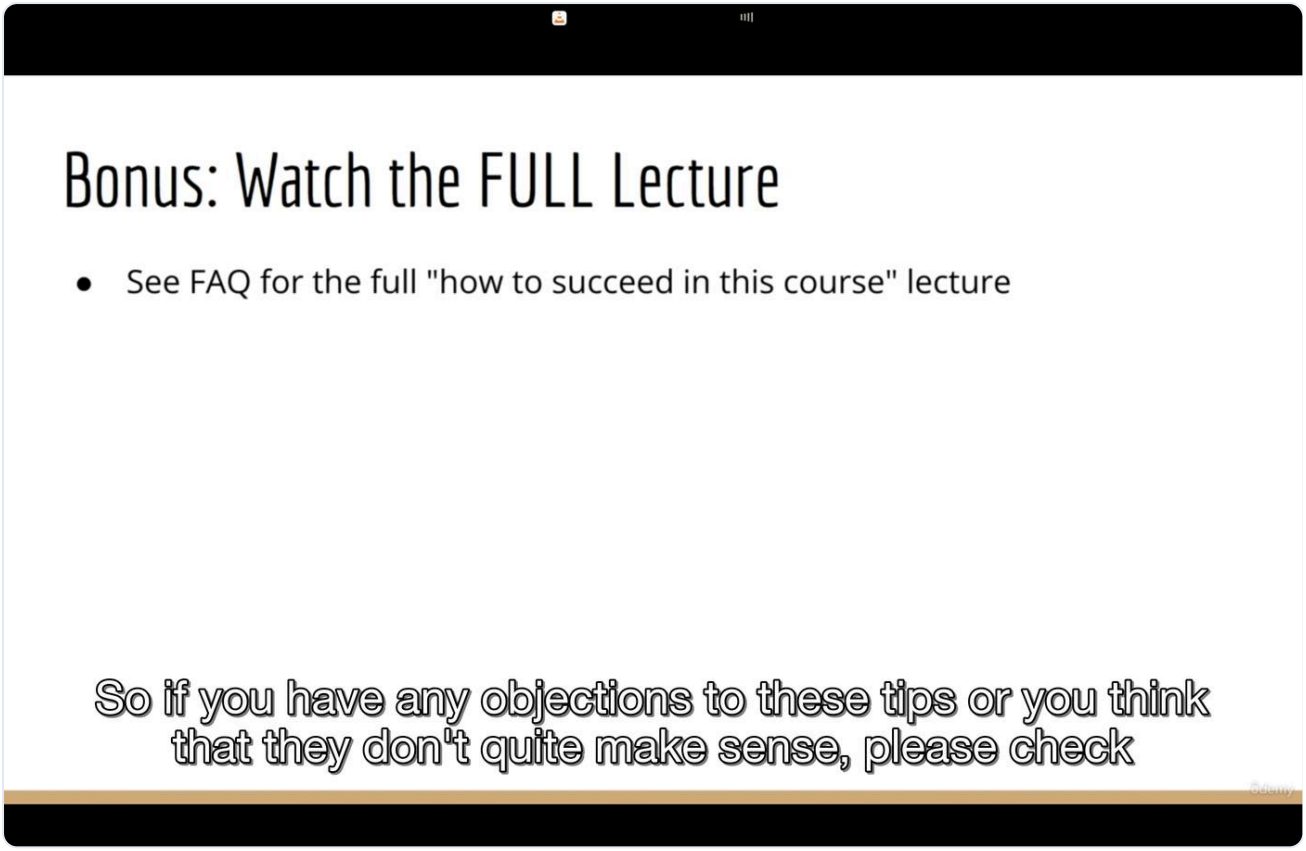

FIGURE 2.69 | Bonus: Watch the FULL Lecture

So if you have any objections to these tips or you think that they don't quite make sense, please check out that lecture for more detail.

Okay, so what is the common thread between all these tips? The common thread is that the reason students don't follow these tips is because they let their egos get in the way. In fact, ego is the number one thing holding you back from succeeding in this course. Your pride is holding you back from succeeding in this course. Here's a fun fact. I've noticed a disproportionately large number of students claiming to have PhDs, but who do very poorly. Yet I see undergrads, masters and even high school students who do better.

How can this be? Well, you see, the problem with PhDs is that they think they are so smart that they have nothing left to learn.

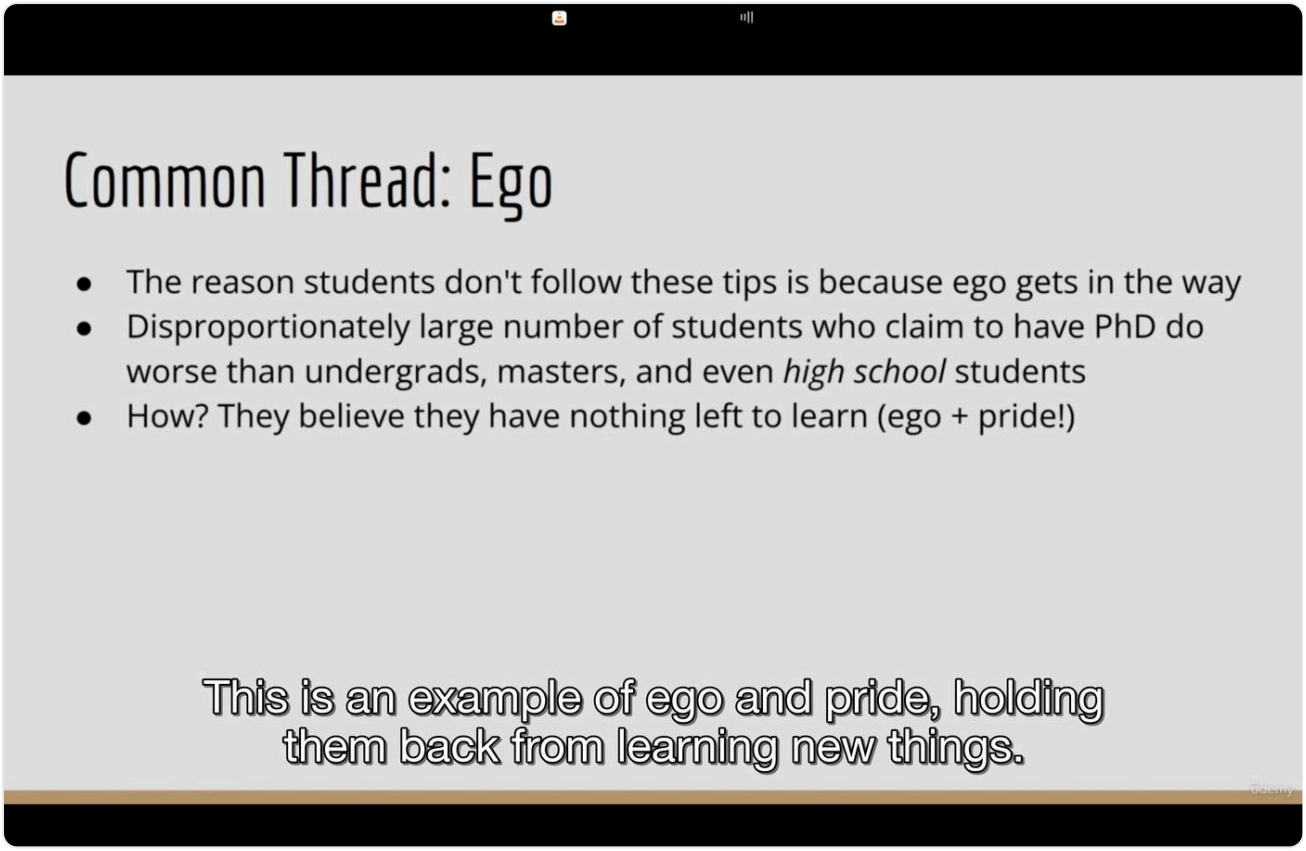

FIGURE 2.70 | Common Thread: Ego

This is an example of ego and pride, holding them back from learning new things. For tip number one, ego holds people back from using the Q&A. Often using the Q&A involves you asking a question which reveals your mistaken assumptions, which will then be corrected by whomever answers the question. This might hurt you a little bit knowing that you got something wrong.

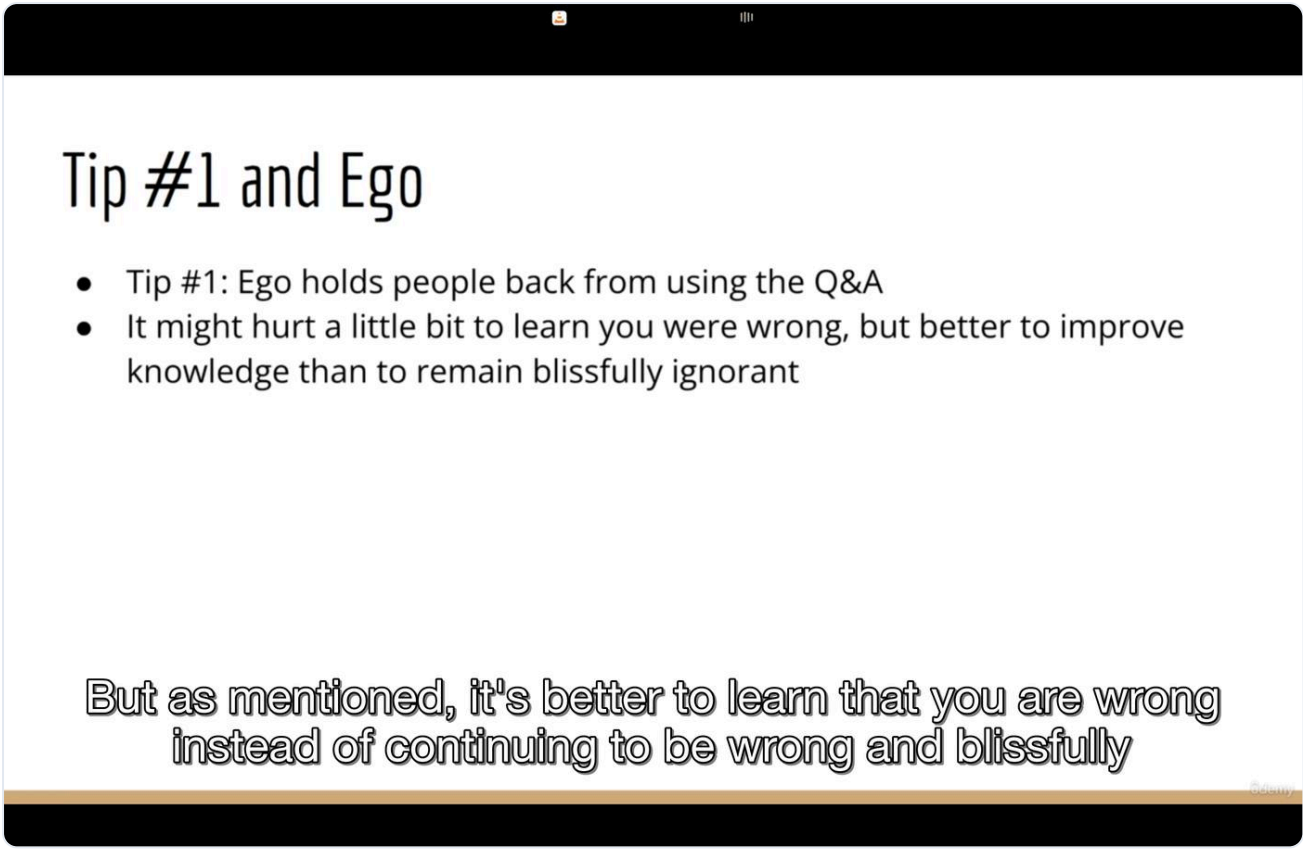

FIGURE 2.71

But as mentioned, it's better to learn that you are wrong instead of continuing to be wrong and blissfully ignorant. For tip number two, Ego holds people back from meeting the prerequisites.

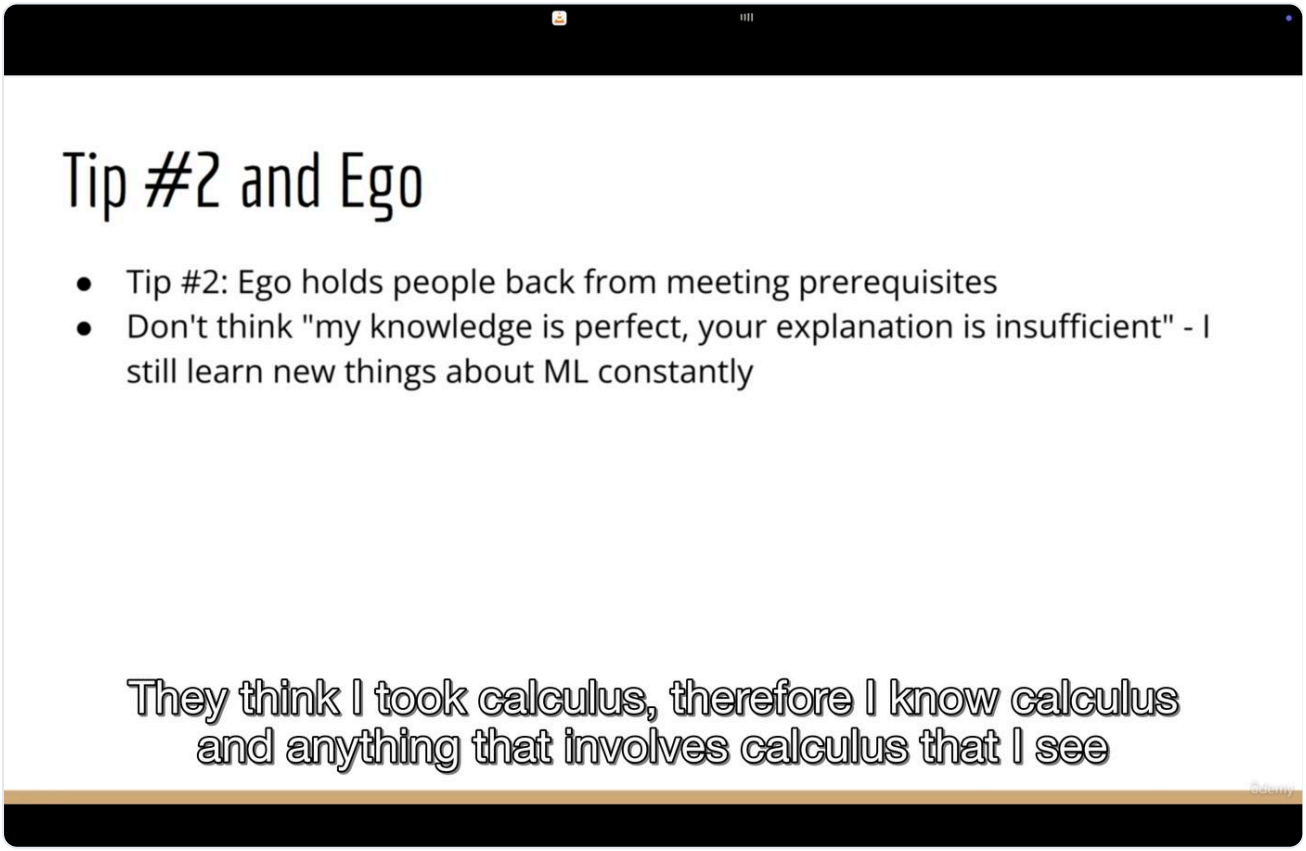

FIGURE 2.72

They think I took calculus, therefore I know calculus and anything that involves calculus that I see in the future, I must understand. Otherwise it wasn't explained correctly. Guys, this is not a good way to go through life. I learned new things about machine learning constantly, despite the fact that I took my first machine learning course over a decade ago.

And let's be clear.

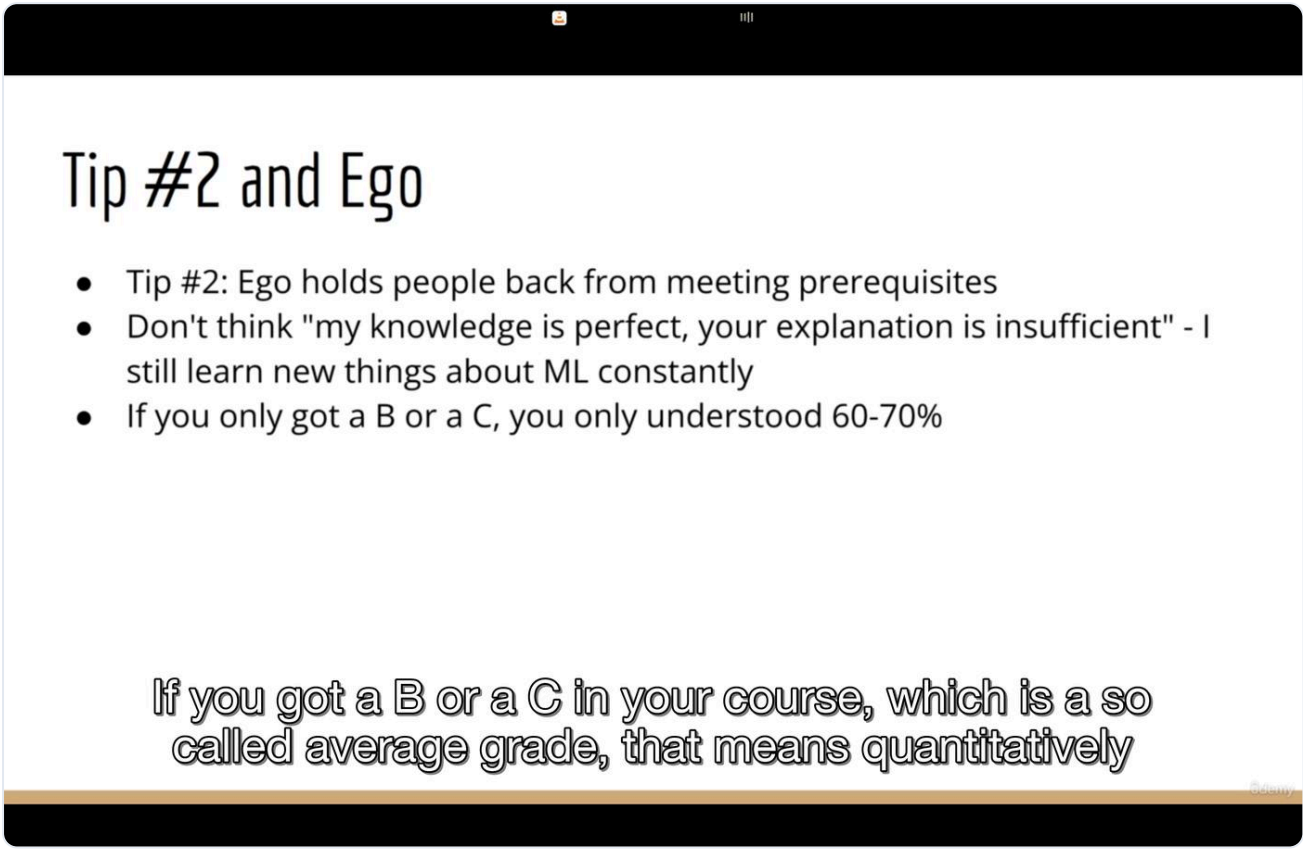

FIGURE 2.73

If you got a B or a C in your course, which is a so called average grade, that means quantitatively your understanding was only at 60 or 70%. Now, another common excuse I see is. Oh, yeah, well, I took calculus 20 years ago, so I forgot everything.

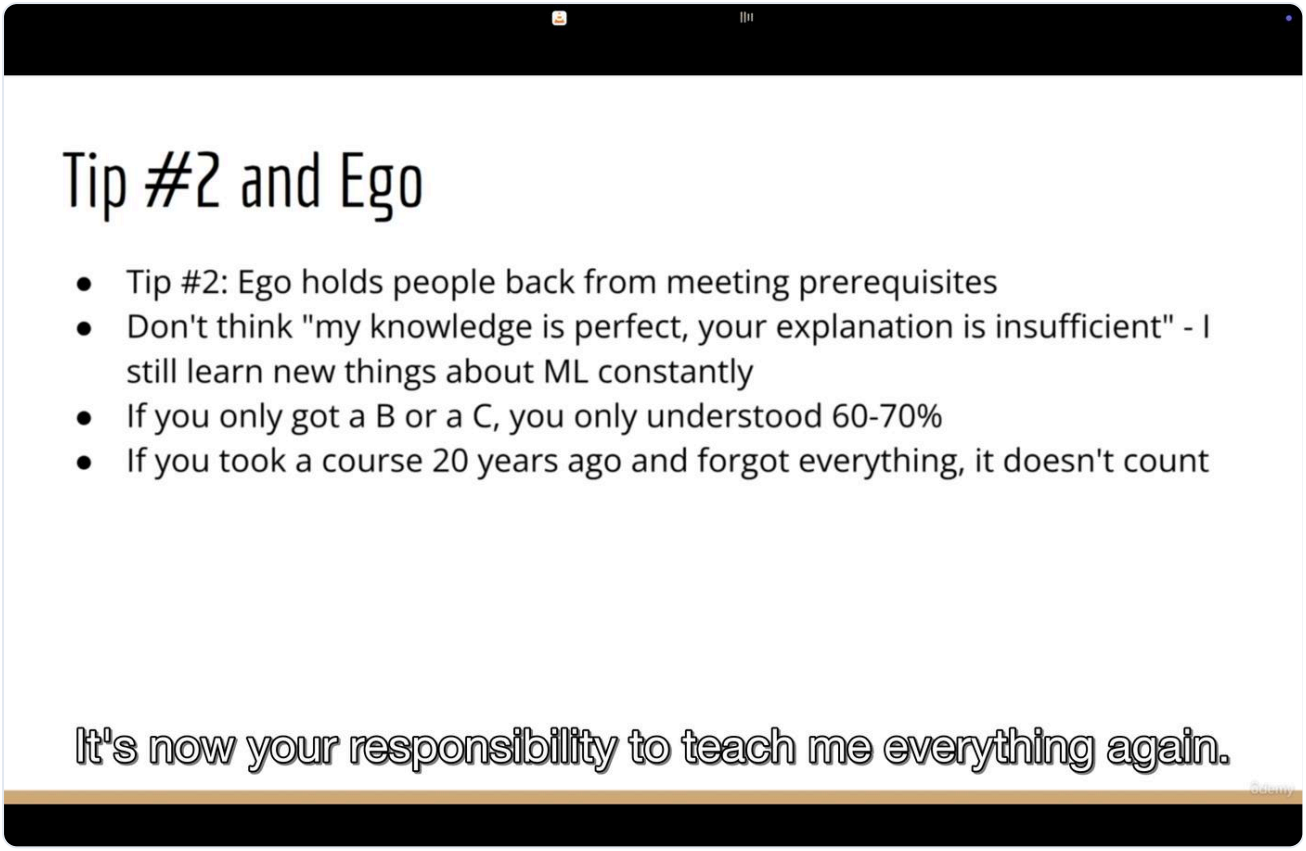

FIGURE 2.74

It's now your responsibility to teach me everything again. Guys, no, this does not count as meeting the prerequisites.

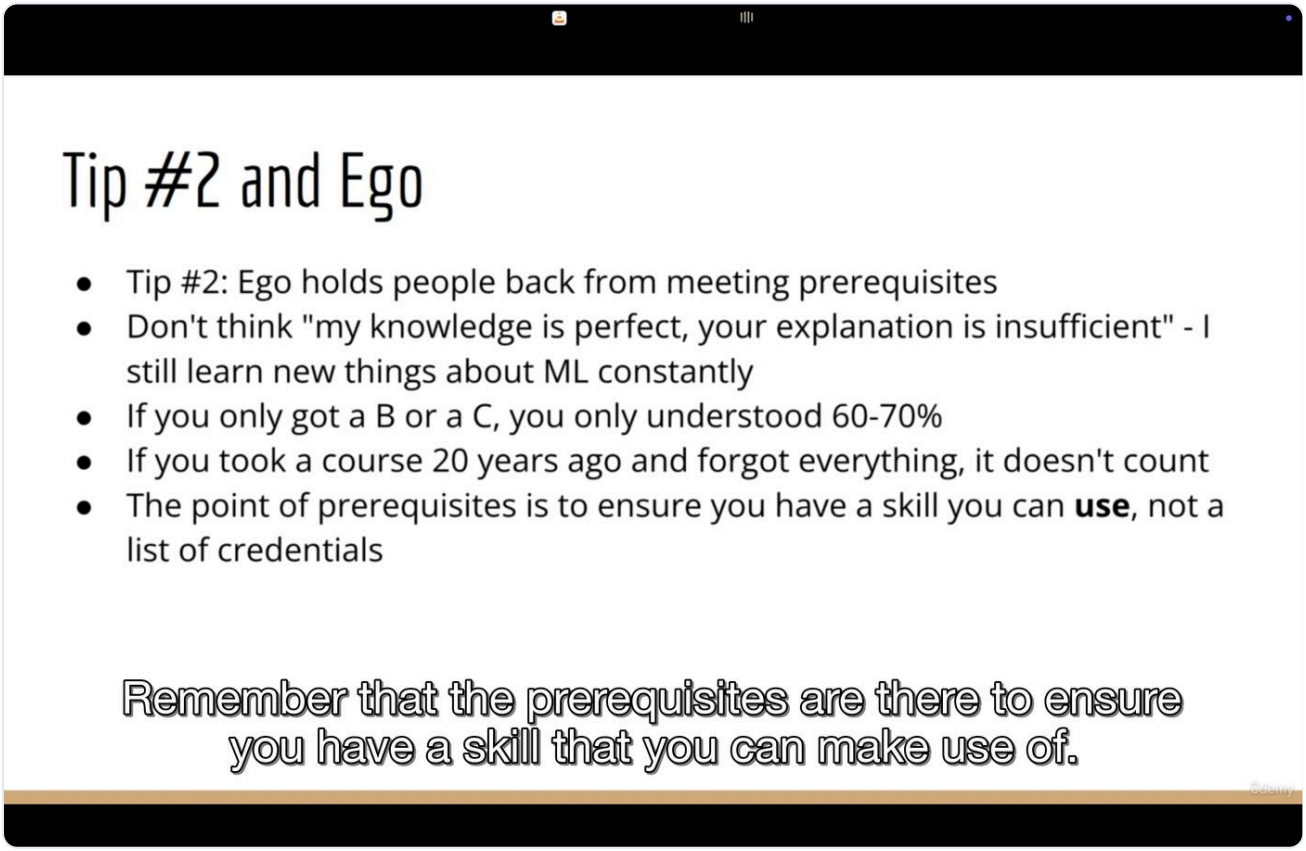

FIGURE 2.75

Remember that the prerequisites are there to ensure you have a skill that you can make use of.

It is not a list of credentials. Credentials might matter when you apply for a job, but they're irrelevant in a self-paced course when all you have to depend on is yourself. Think of it this way. Would you fly an airplane if you had a piece of paper saying that you could? But in actuality you forgot everything about how to actually fly airplanes? I think that would be very dangerous. And I don't think that any normal person would answer yes to this question.

Finally, for tip number three, ego holds people back from getting their hands dirty and taking notes and writing code. I can tell that this is the case because when I mention how writing code is a great exercise and it is literally the job you will do as a data scientist, people would rather argue with me than accept this advice. They care more about calling me names and making threats than they do about actually becoming better data scientists. They simply cannot handle the reality that muscle memory exists.

Coding is a useful exercise, and coding is literally the job description.

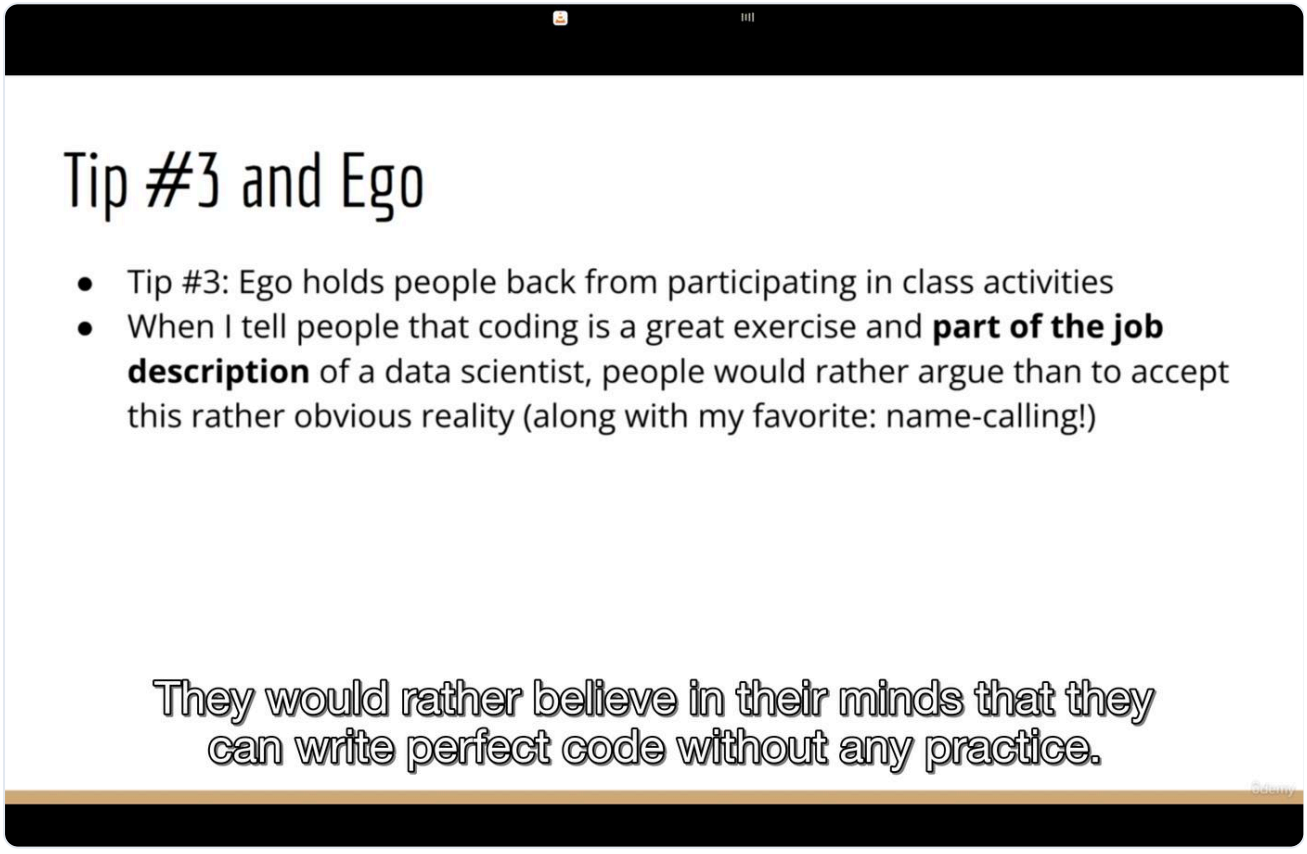

FIGURE 2.76

They would rather believe in their minds that they can write perfect code without any practice. The reason they're afraid to code is that once they start, they might make mistakes which would expose their imperfect coding abilities.

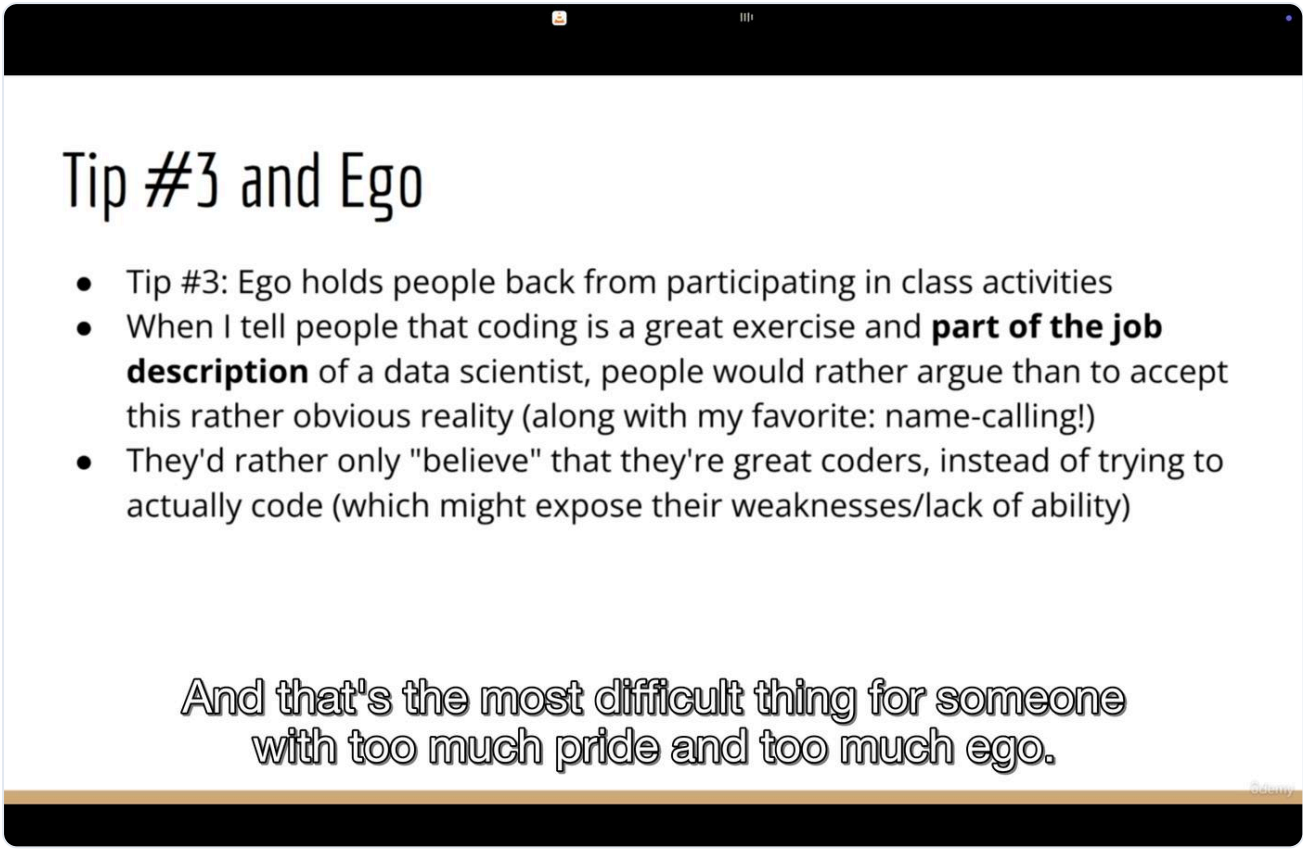

FIGURE 2.77

And that's the most difficult thing for someone with too much pride and too much ego. They simply cannot face the idea of being wrong or making mistakes.

It's too burdensome to bear. Okay, so I hope you guys are getting the idea. Do not let your ego get in the way of your success. Don't be afraid to be wrong and follow these instructions as diligently as possible. Thanks for listening and I'll see you in the next lecture.

SECTION 3

Beginner's Corner

1 · Beginner's Corner Section Introduction

So in this lecture we are going to introduce the next section of this course, which is a beginners corner on Transformers. Now, you must be wondering if you are new to my courses or even if you've taken my courses before.

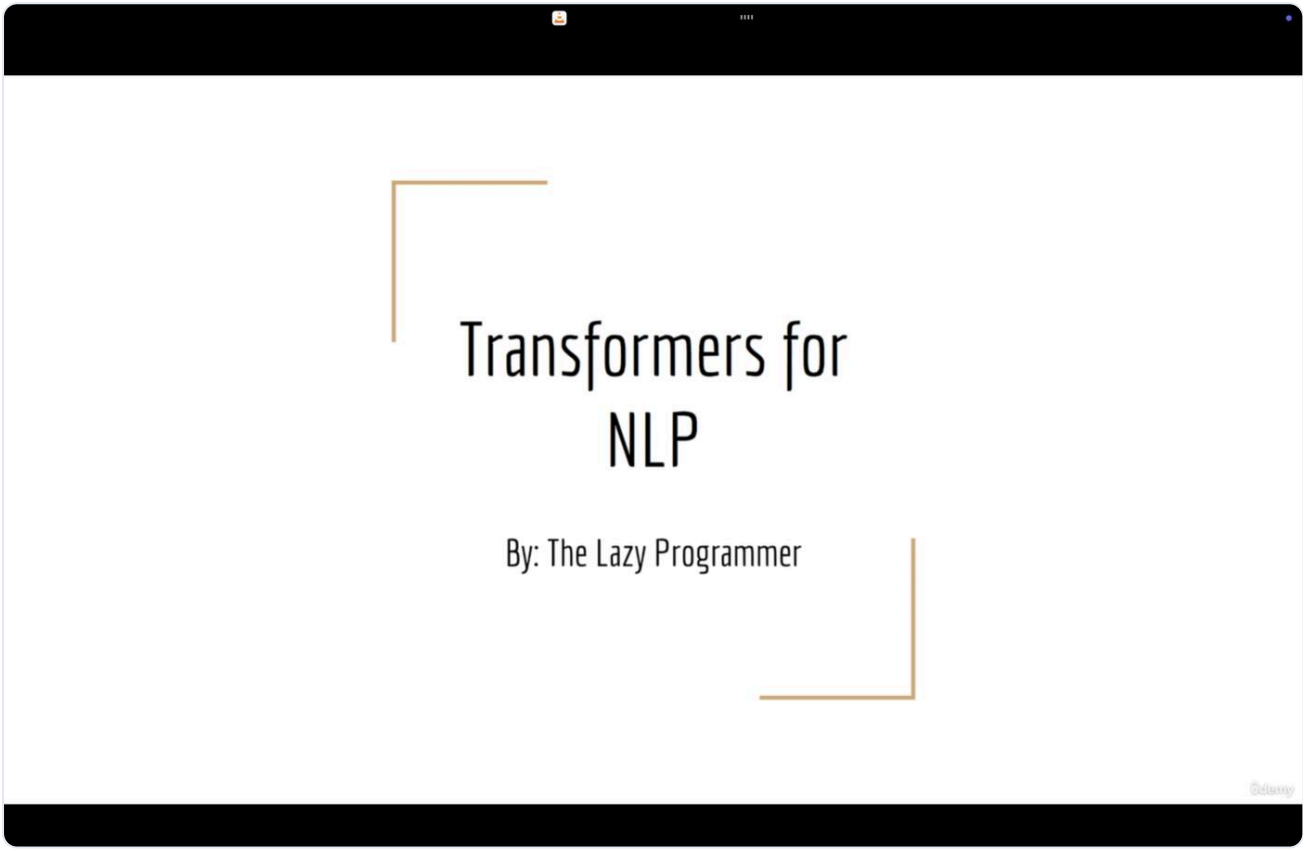

FIGURE 3.1 | Transformers for

What is a beginners corner? So to answer this question, the beginners corner section is a section that is designed with the typical beginner in mind. So what is the typical beginner?

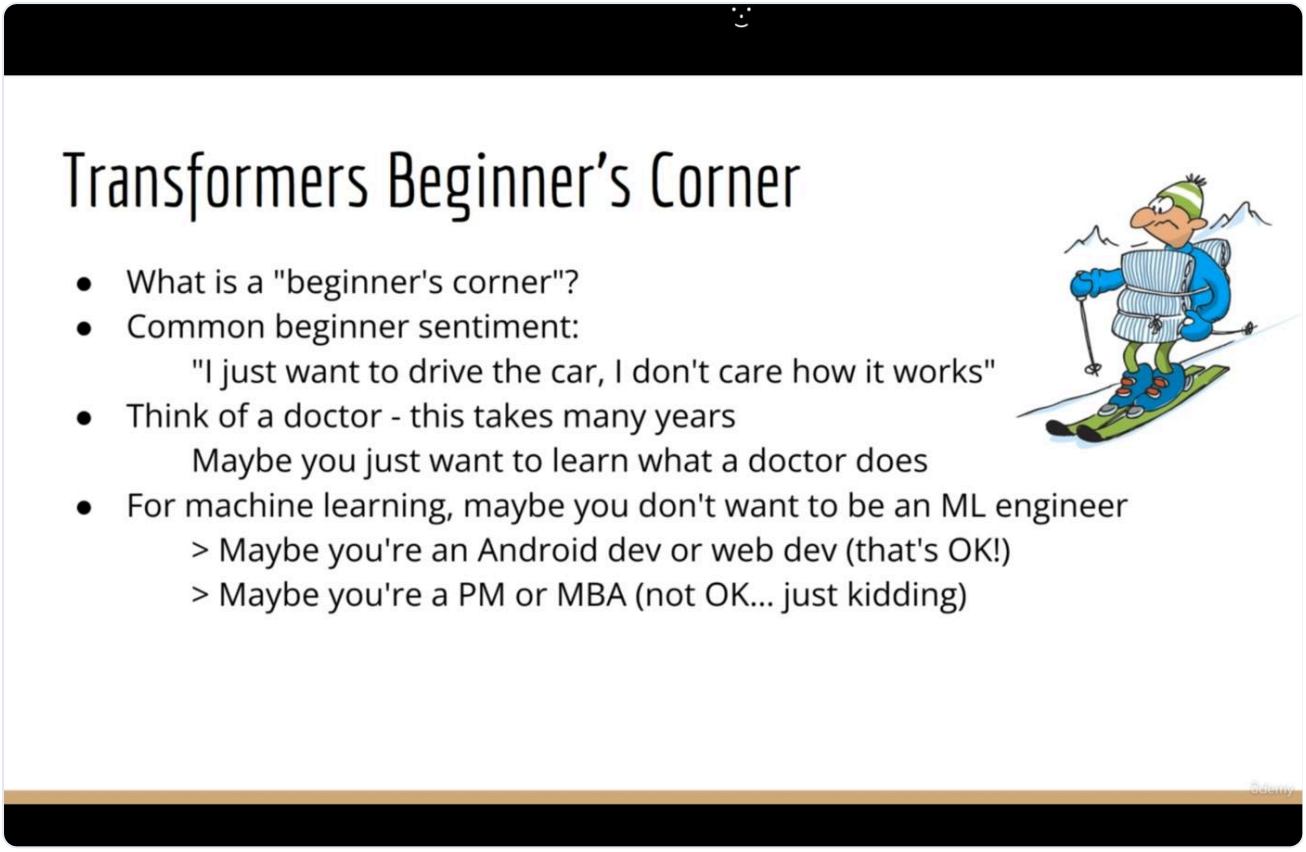

FIGURE 3.2

Well, the typical beginner sentiment is I just want to learn to drive the car.

I don't care about how the car works. You might also think of this in terms of becoming a doctor. Perhaps you don't actually want to be a doctor or to put in the years of work required to become a doctor. You just want to understand what a doctor does in their day to day job. Maybe your goal is to become a doctor's assistant. In terms of machine learning. Perhaps it's the case that you don't want to be a machine learning engineer or to understand how machine learning works.

Perhaps you're an Android developer or a web developer. And the intuition is enough. In this case, the beginners corner is for you.

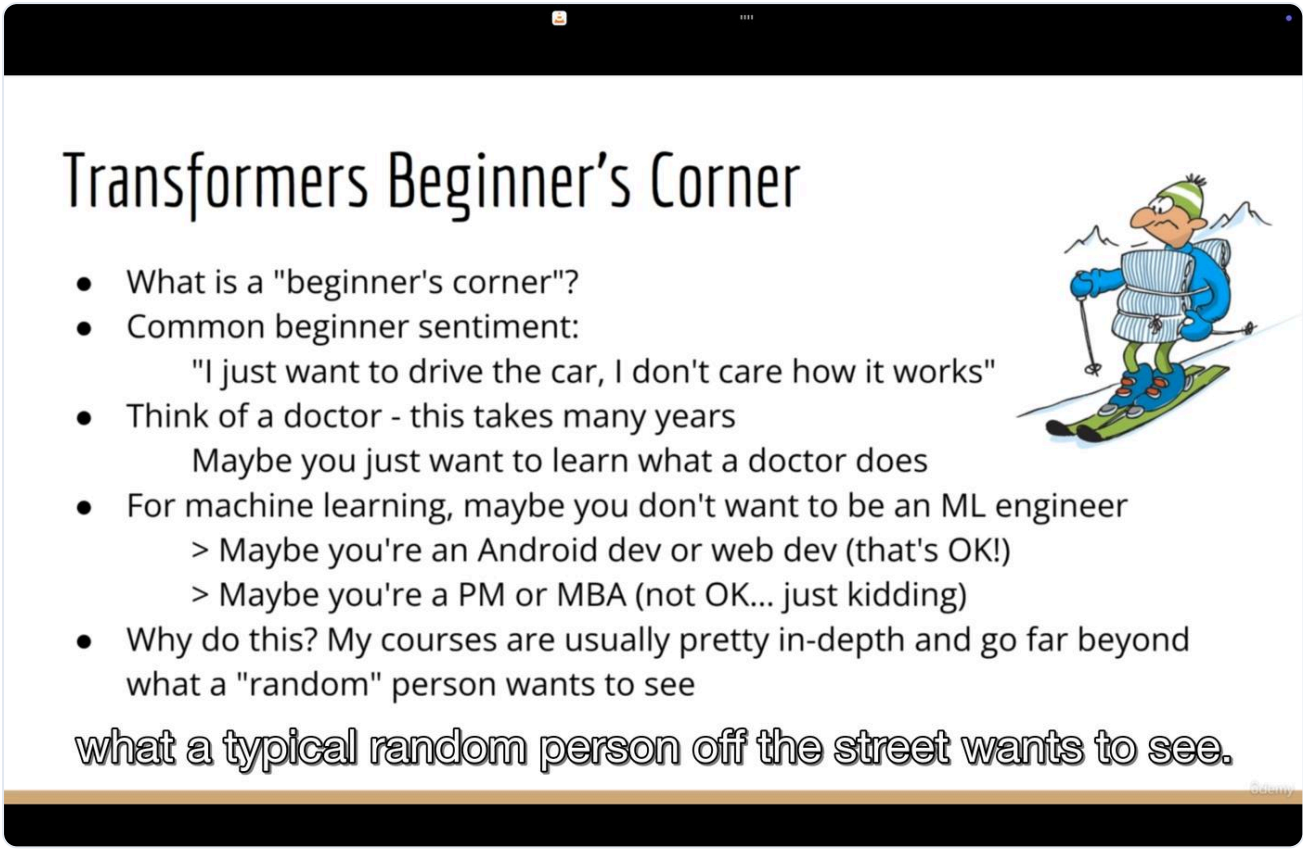

FIGURE 3.3 | Common Beginner Sentiment:

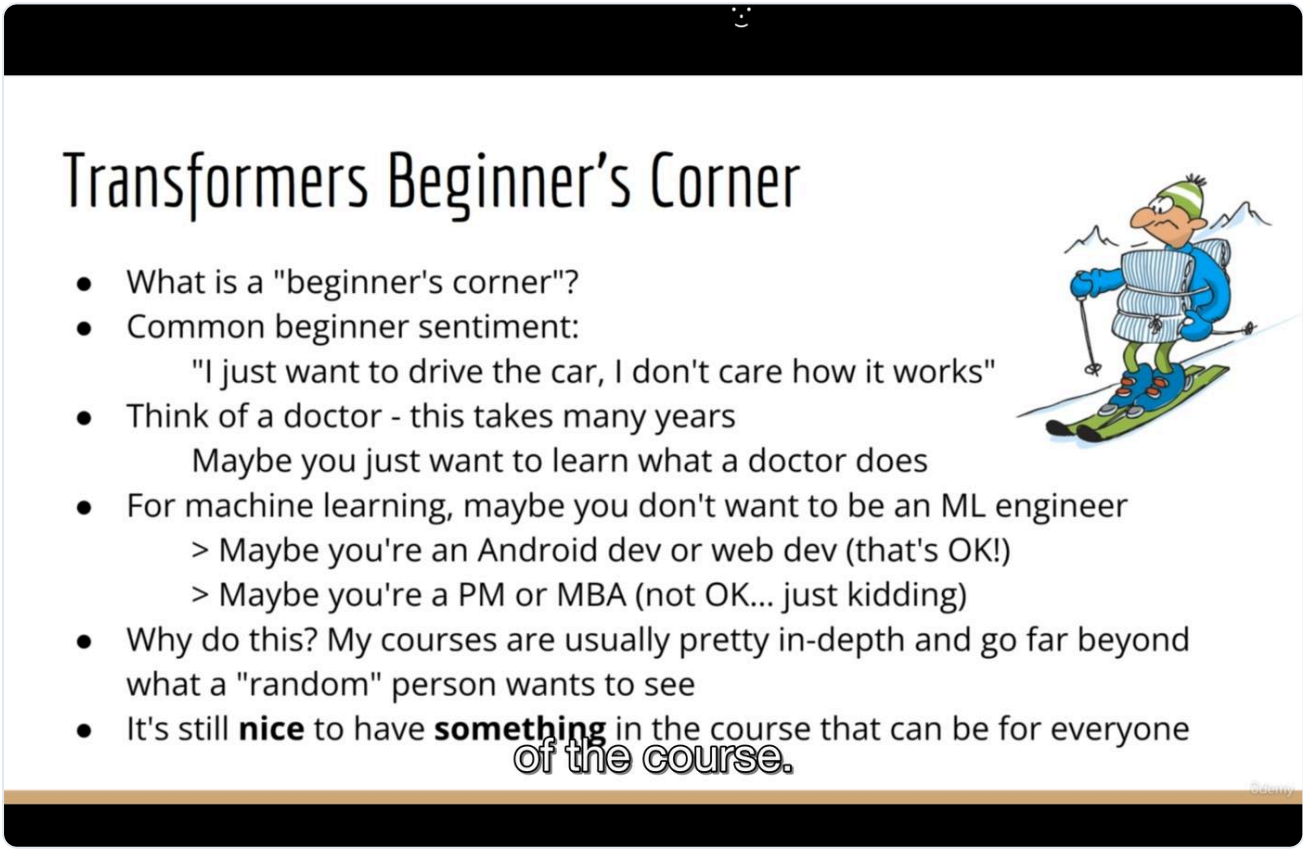

FIGURE 3.4

Now, the reason I've done this is my courses are normally pretty in-depth, and they go far beyond what a typical random person off the street wants to see. Of course, ignoring the fact that a typical random person off the street does not become a machine learning practitioner in a few months. It's still nice to give them something they can take with them from this course.

So maybe not all of the course suits your needs, but at least you might find some use in 20 to 30% of the course. This will be useful for those of you who once you know about transformers and want to use Transformers but perhaps might be too inexperienced to get into the details. Okay. So the basic thing to understand about this section is that this section is about how to use transformers, but it is not a section about how Transformers work, since that will come at a later time.

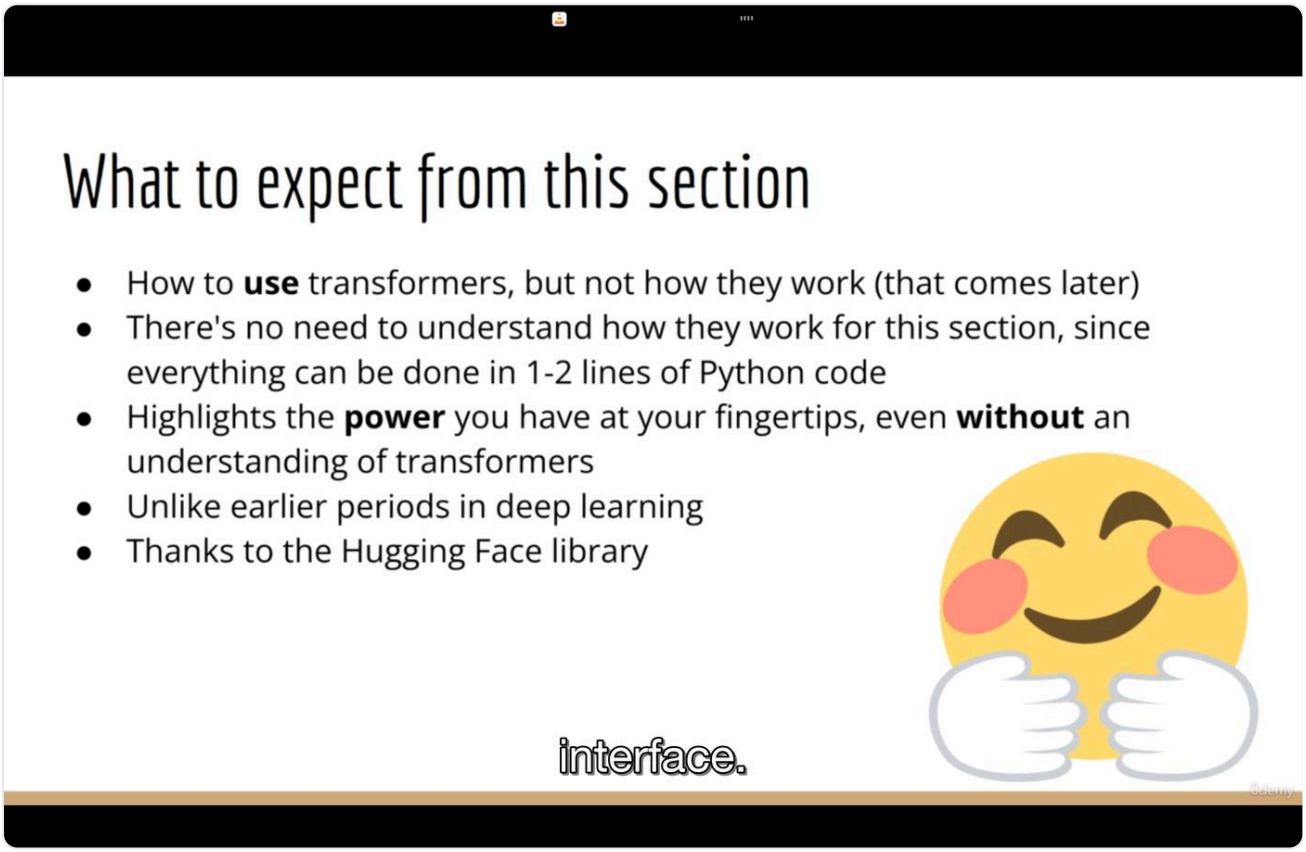

FIGURE 3.5 | What to Expect from This Section

In fact, for this section, there is no need to understand how Transformers work, since everything can be done in just one or two lines of Python code. One thing this does is it really highlights the power that you have at your fingertips, even without much of an understanding. This is unlike earlier forms of deep learning where you really needed to be pretty knowledgeable to do anything useful. And this is thanks to the Hugging Face library, which brings many powerful models into a simple universal interface.

Now for non beginners, you might be wondering, is this section useful for me if I do want to understand how transformers work? And the answer is yes. And so how can that be? Well, the answer is that by this point, if you are a non beginner, you have a pretty good idea of the steps involved in building a neural network. For NLP in particular, you understand the process of tokenization converting tokens into integers and converting those integers into embedding vectors.

You also understand how embeddings act as the initial layers in a neural network. Furthermore, you also understand how different outputs of a neural network are suited to different tasks. For example, the output of a binary spam detector would be different from the output of a text generator.

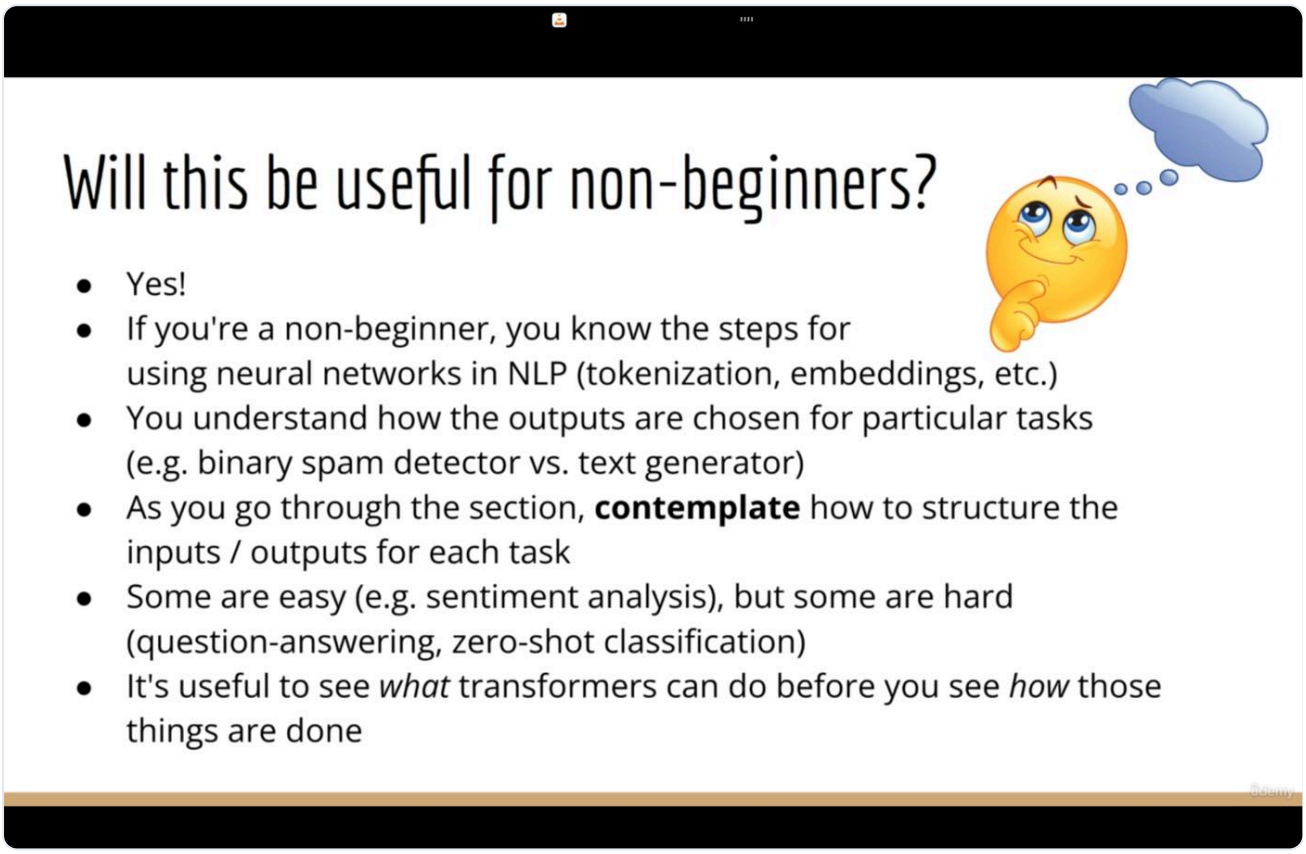

FIGURE 3.6 | Will This Be Useful for Non-beginners?

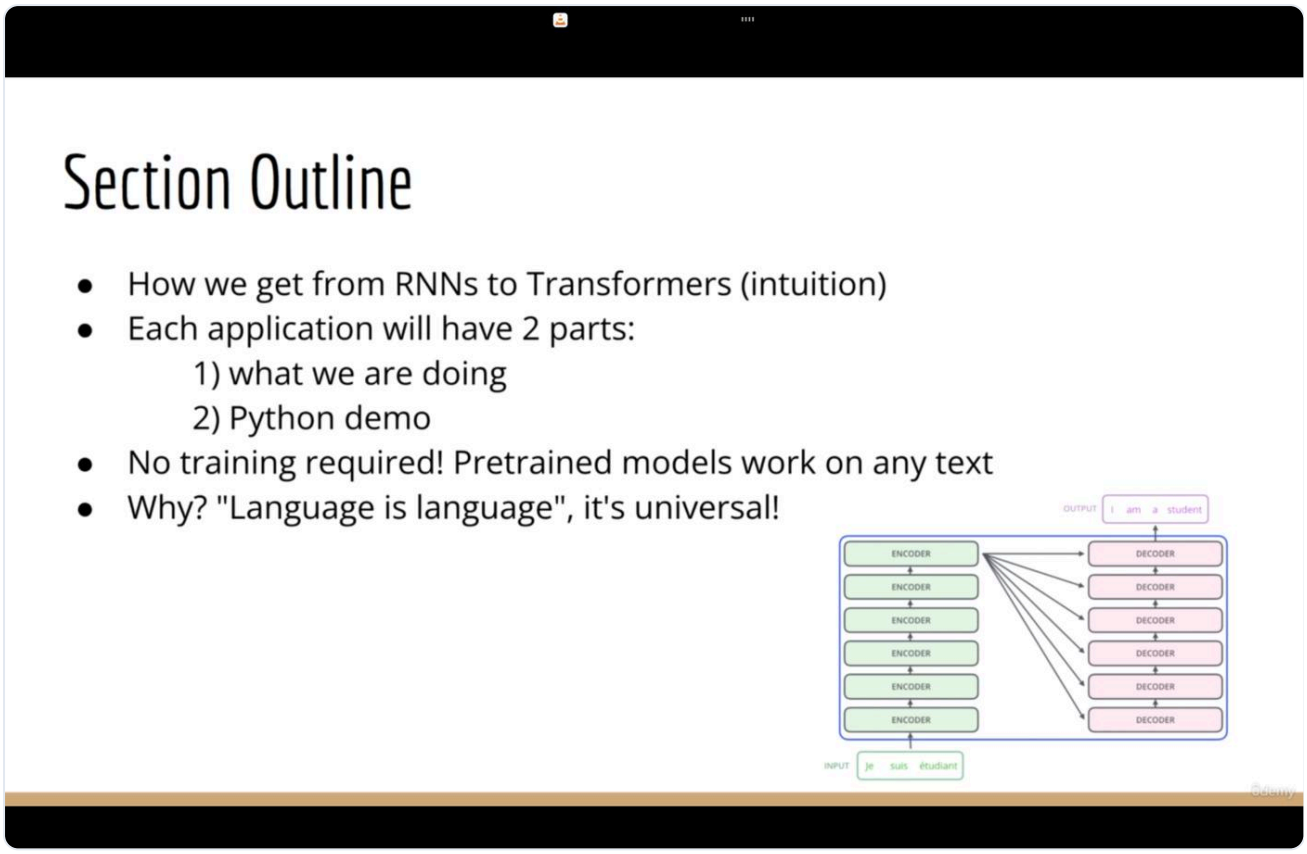

FIGURE 3.7 | Section Outline

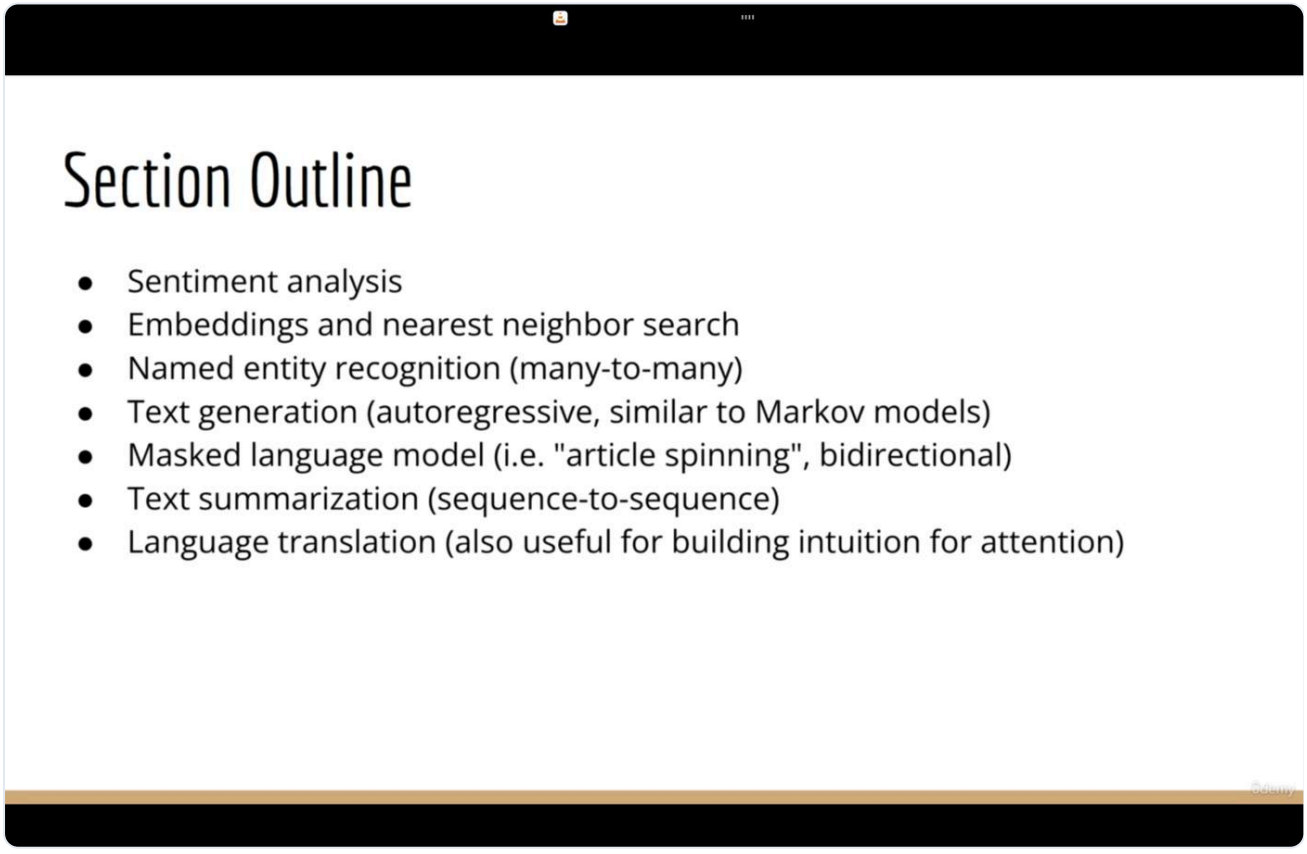

FIGURE 3.8 | Section Outline

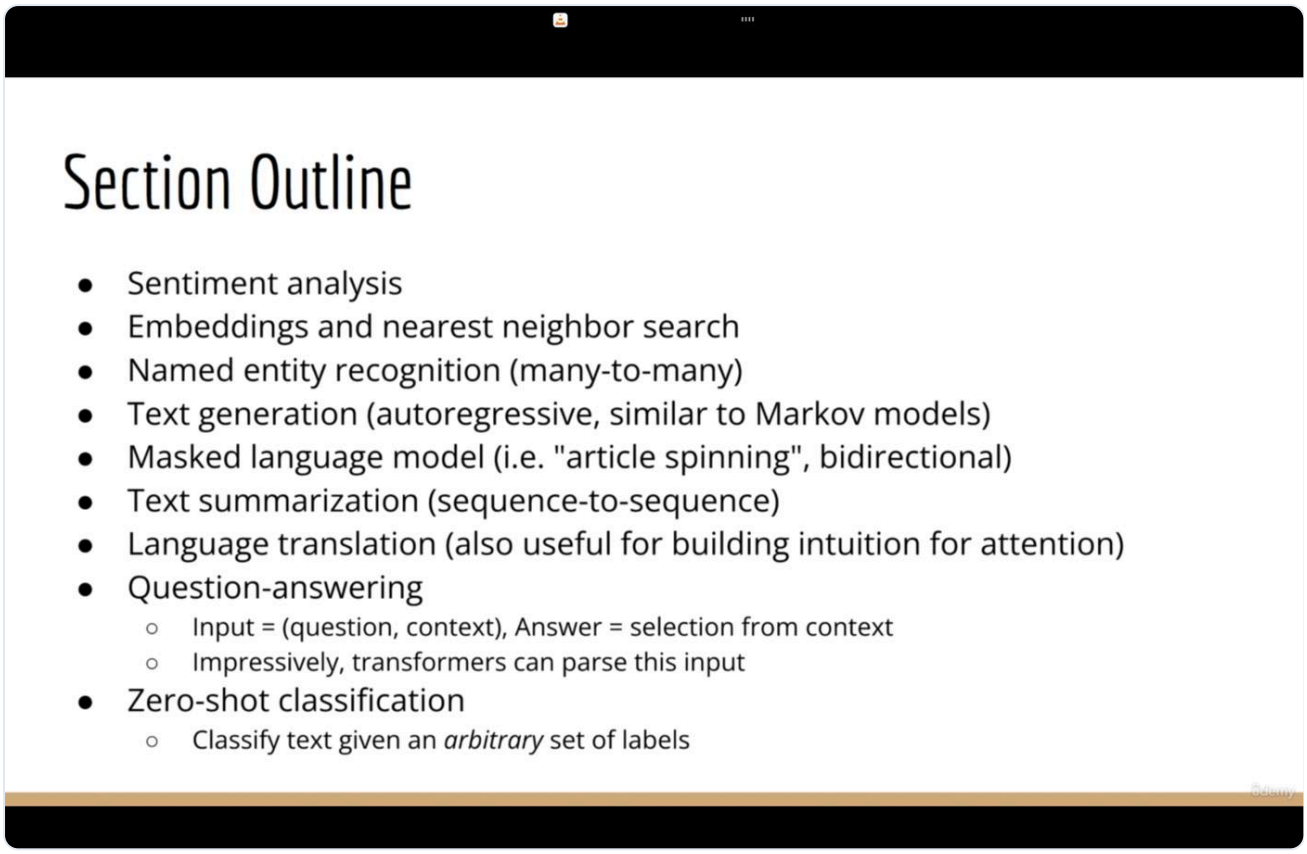

FIGURE 3.9 | Section Outline

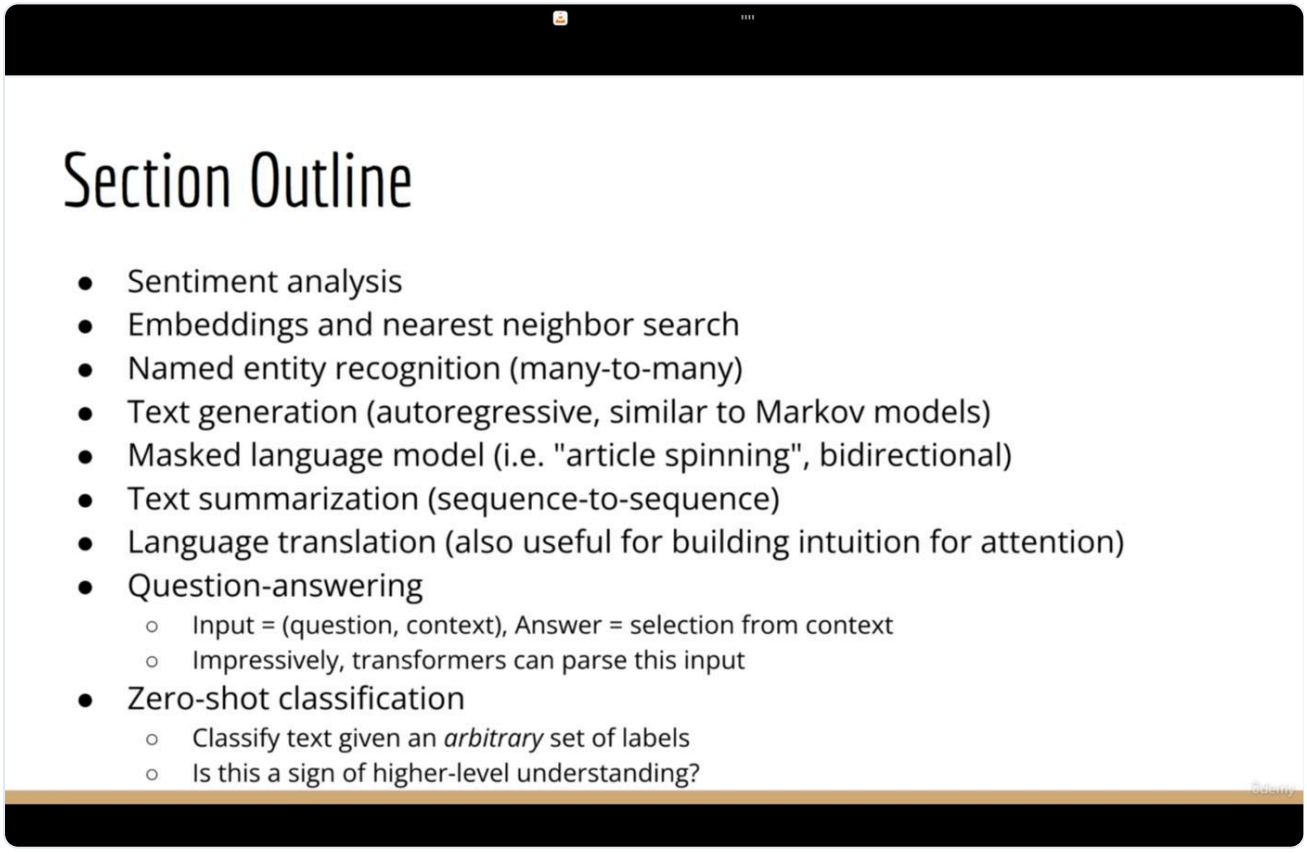

FIGURE 3.10 | Section Outline

So as you go through this section, think about how the inputs and outputs would be structured in order to complete each task. Some of these will be pretty easy. For instance, sentiment analysis, which just uses a sigmoid when the output is binary.

On the other hand, tasks like question answering and zero-shot classification are very different from what you may be used to. And because these new architectures are so different from what you've seen before, it really is useful to look only at the interface first before looking inside to see how things actually work. Okay. So let's go through an outline of what you will learn in this section. Now to begin, I don't want to leave you without any idea of what Transformers even are.

So we'll discuss the kind of derivation for how we got to Transformers, starting with RNNs after this brief introduction. We'll then begin to look at some applications for each of these applications will proceed in two steps. The first step will be to discuss what the task is and why we would want to do it, along with a brief look at the corresponding Hugging Face syntax. The second step will be to look in a notebook to demo the code, actually working on a real data set.

You'll be surprised to see that for all of these tasks there is no training required at all. Of course this makes sense since language is language no matter what data set it comes from. For example, that's why you can learn English in grade school, but after you graduate, you can still communicate with new people, despite the fact that you've never met them in the past. As another example, you can read and understand the sentiment of a movie review, even if you've never read a movie review before.

There's just a universality to language in that sense. So what are the tasks we will look at? Well, you'll recognize many of these tasks as ones you've seen before. If you've taken my courses or perhaps other courses within this field, of course this doesn't make these applications redundant, since transformers tend to provide a state of the art performance. In other words, hopefully better than what you've been able to achieve in the past. So the first task we'll look at is called sentiment analysis.

We'll see that even though the transformer we'll use has never seen our data set before, it will still do decently well. And thus this might be a suitable solution. If you want to do sentiment analysis on generic data you find on the internet or at your workplace. The next task we will look at is how to transform text into vectors or embeddings using a transformer network. Using these embeddings, we can do things like similarity search, also known as nearest neighbor search.

The next task we'll look at is called the named entity recognition. As you recall, this is an example of a many-to-many task. This is because for each token in the input, we have a target output. Either that will tell us what kind of named entity the token is or if it is not a named entity. Thus there will be many outputs one for each of the input tokens. This is opposed to a task like sentiment analysis where we only have one output for the whole input document.

The next task we'll look at is text generation, where we use an auto regressive transformer to generate text one token after another. This is very similar to how we use Markov models, except that deep neural networks can account for the entire past, whereas Markov models are limited to just one or a few tokens. We'll also look at a similar task known as masked language modeling.

In my courses we've referred to this as article spinning, where a common approach is to change random words in a document using the surrounding context to choose the most appropriate substitutions.

As you recall, this is also a form of language model, except that instead of our distribution being conditioned on past words only, it can be conditioned on both past and future words. The next task we'll look at is called text summarization. Again, this is a different task from the others since the input is a document in the output is also a document with a different length than the input, hopefully shorter. This is another example of a sequence to sequence task.

The next task we'll look at is language translation. This is yet another example of a sequence-to-sequence task. Since both the input and target will be strings of text. One interesting fact about translation is that because of how grammar works in different languages, translating is not simply a matter of mapping one word to another for every word in the source text. Instead, you might have one word mapped to multiple words. Multiple words map to one word, multiple words mapped to multiple words and re-ordering of words to match the grammar of the target language.

This task is very commonly used for building up the intuition for the attention mechanism in transformers since at each point in the output. We need to decide which input to pay attention to. The next task we'll look at is question answering. So this is one task you may have not seen before. Technically, it is a sequence to sequence task, but there are many interesting complexities. Firstly, the input to the neural network is made up of two pieces of text, the question and the context, which contains the answer to the question.

This is interesting because for most other tasks in NLP, there is only one piece of text that goes into the input. In fact, although we do have two pieces of text, what we end up doing is concatenating them into a single document. What's really impressive is that modern neural networks are still able to parse what this means and answer the question with good performance. The final task we'll look at is also a new task, which you probably haven't seen before.

This task is called zero-shot classification, which is another very interesting task in terms of what neural networks are capable of. So how this task works is you ask your model to classify a piece of text. But unlike the other classifiers we've seen, the set of possible labels have never been seen by the model before. So I can ask my model is this document about biology or physics? And you'll see that it will be able to tell me the answer, despite never being trained to classify biology and physics documents.

In my view, this shows that the neural network has attained a higher level of understanding. It's able to choose from a set of classes based only on its understanding of language. So you could do spam detection without actually training on a spam detection dataset. Okay, so I hope you're excited to learn about these topics. I'll see you in the next lecture.

2 · From RNNs to Attention and Transformers - Intuition

So in this lecture, we'll be going over a brief discussion about where Transformers came from and how this idea was derived from earlier neural network architectures, specifically recurrent neural networks. Please note that this lecture is optional since it's not necessary to be able to use the code that you will see later in this section. However, if you plan to go further in your study of Transformers, you may find this useful.

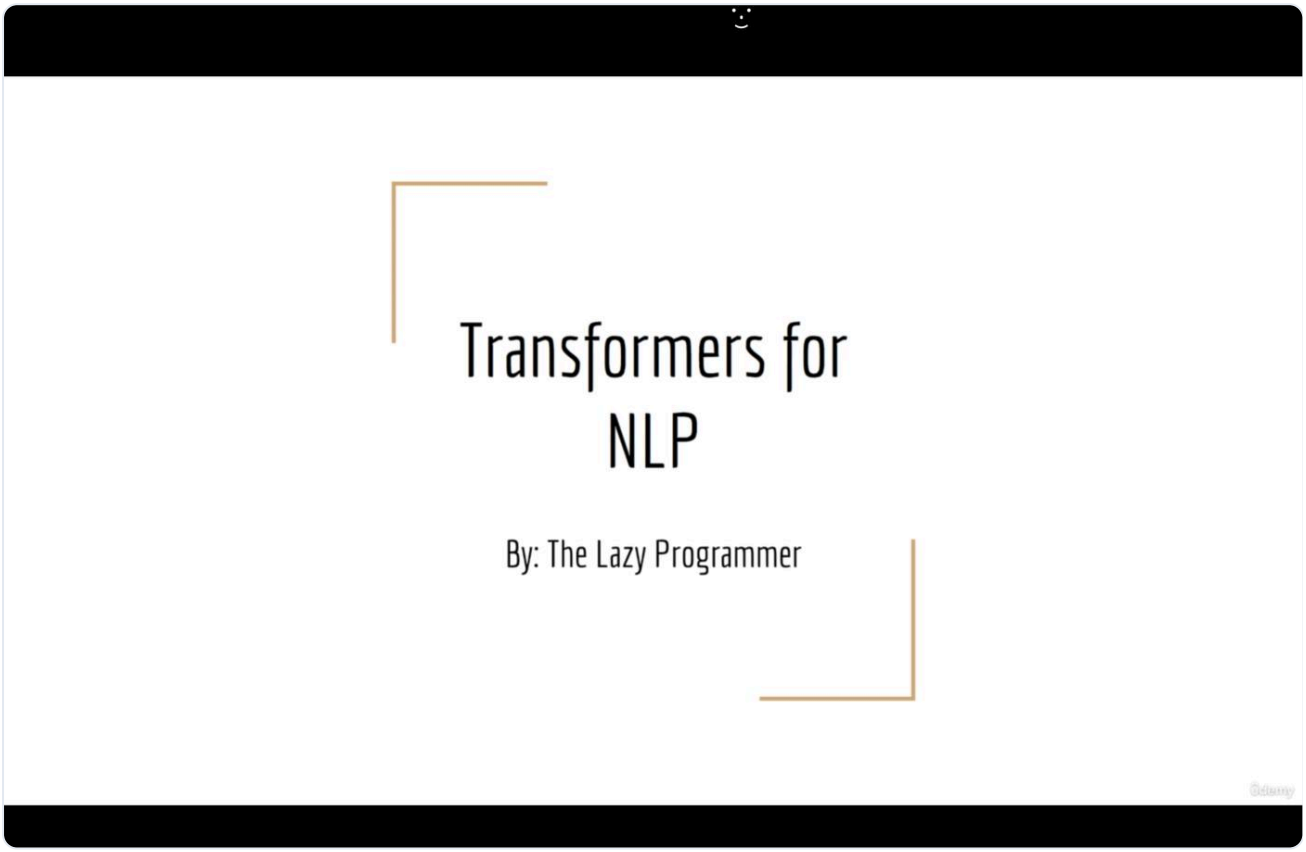

FIGURE 3.11 | Transformers for

So this lecture is for those of you who have a pretty good understanding of deep learning fundamentals, and you also want to learn the basic intuition for what transformers and attention do.

Now, for those of you who do not want to go through this lecture, I want to provide you with at least two main points to keep in mind. Point number one, the attention mechanism is what allows the neural network to learn very long term dependencies over sequences.

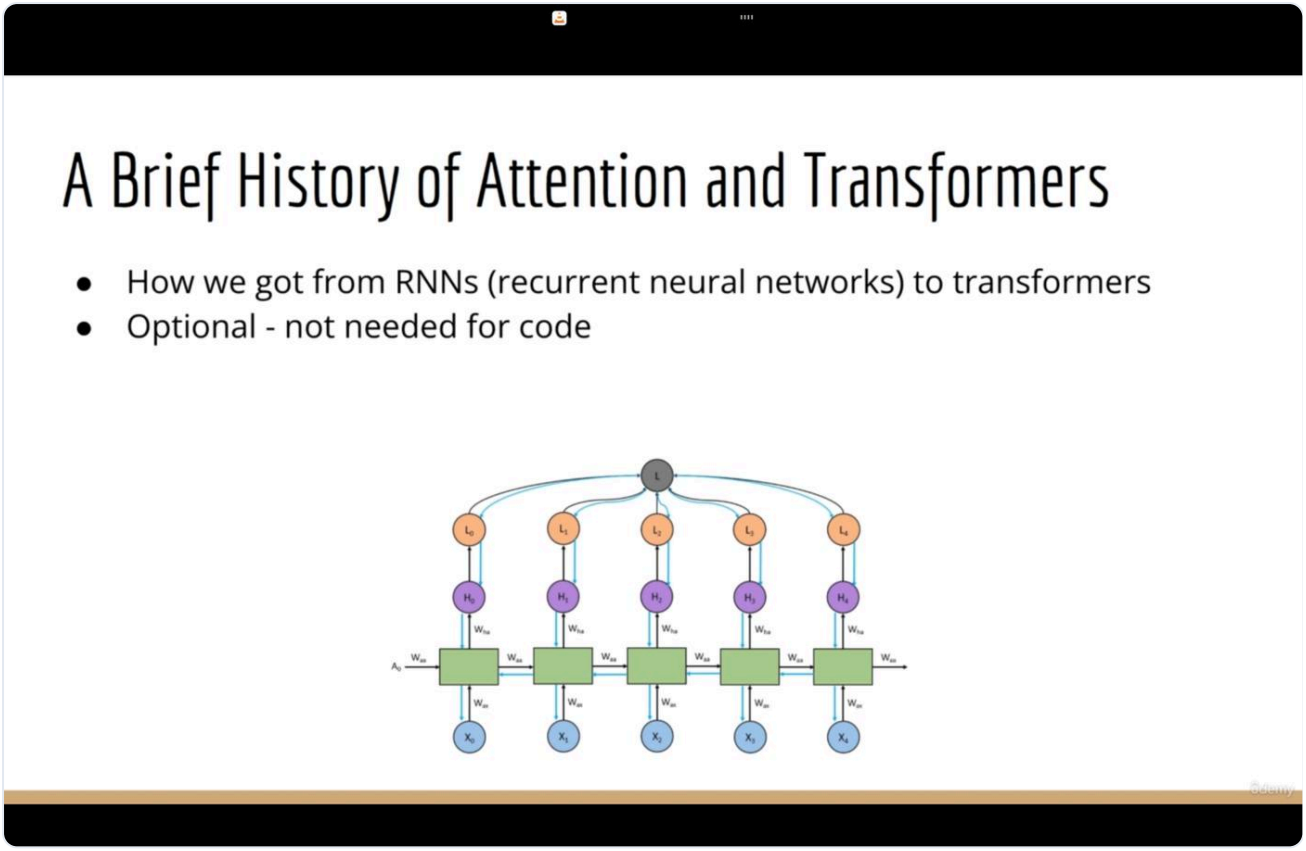

FIGURE 3.12 | A Brief History of Attention and Transformers

In fact, it does a much better job than the LSTM, which was previously the most popular architecture for sequential data sets. Recall that the LSTM is a type of recurrent neural network.

Attention originated from RNNs.

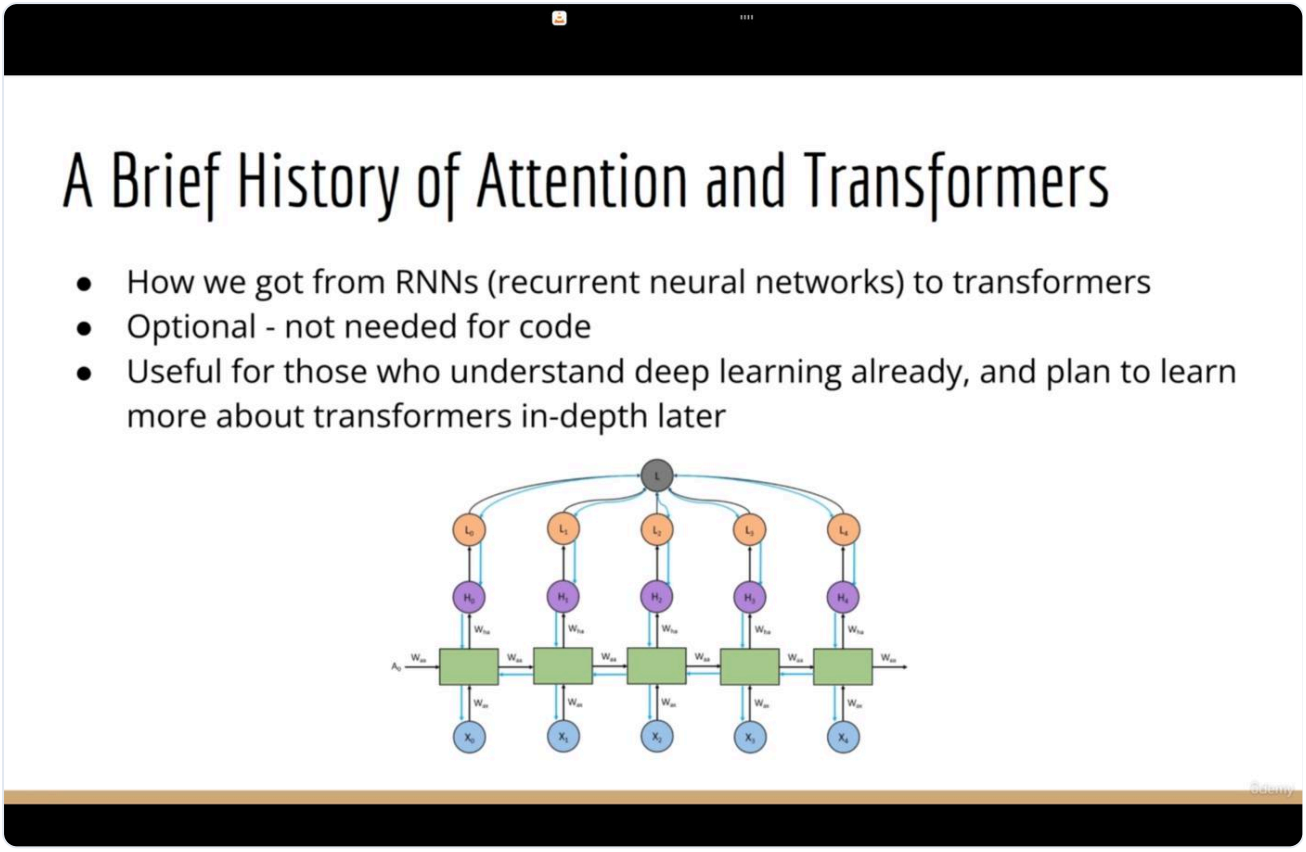

FIGURE 3.13 | A Brief History of Attention and Transformers

The transformer is essentially the attention mechanism, but without using any recurrent units to process the input. Point number two transformers are very large neural networks and as a consequence go very slow. In other words, they may take more time in computation then what you're used to.

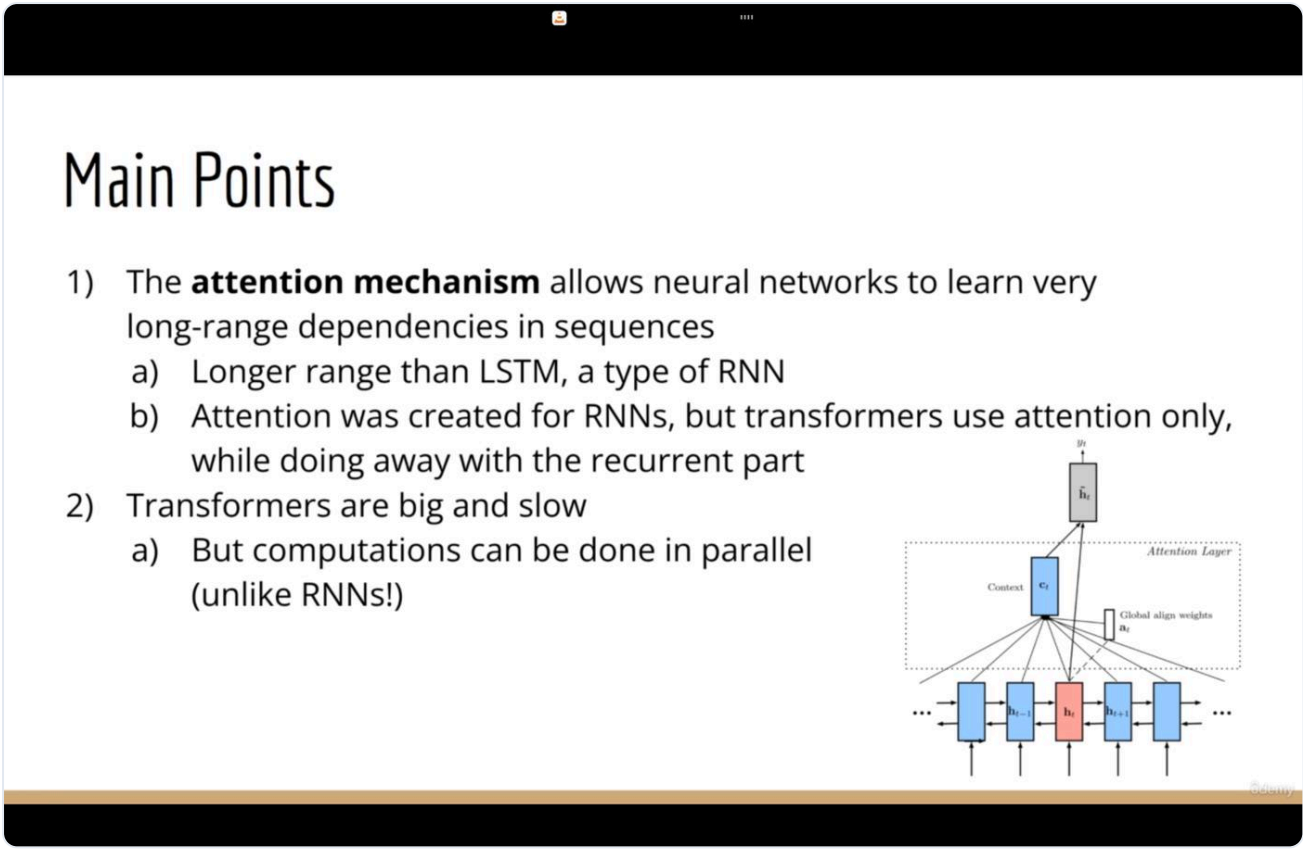

FIGURE 3.14 | Main Points

However, the type of computation it does can be done in parallel. Whereas this is not the case for RNNs. RNN computations must be done sequentially, and there is no way around this.

Now, again, since this lecture is optional, there is no need to go through the rest. So if you just want to remember these main points, please feel free to skip the remaining parts. Okay. So to begin our story, we'll start with RNN.

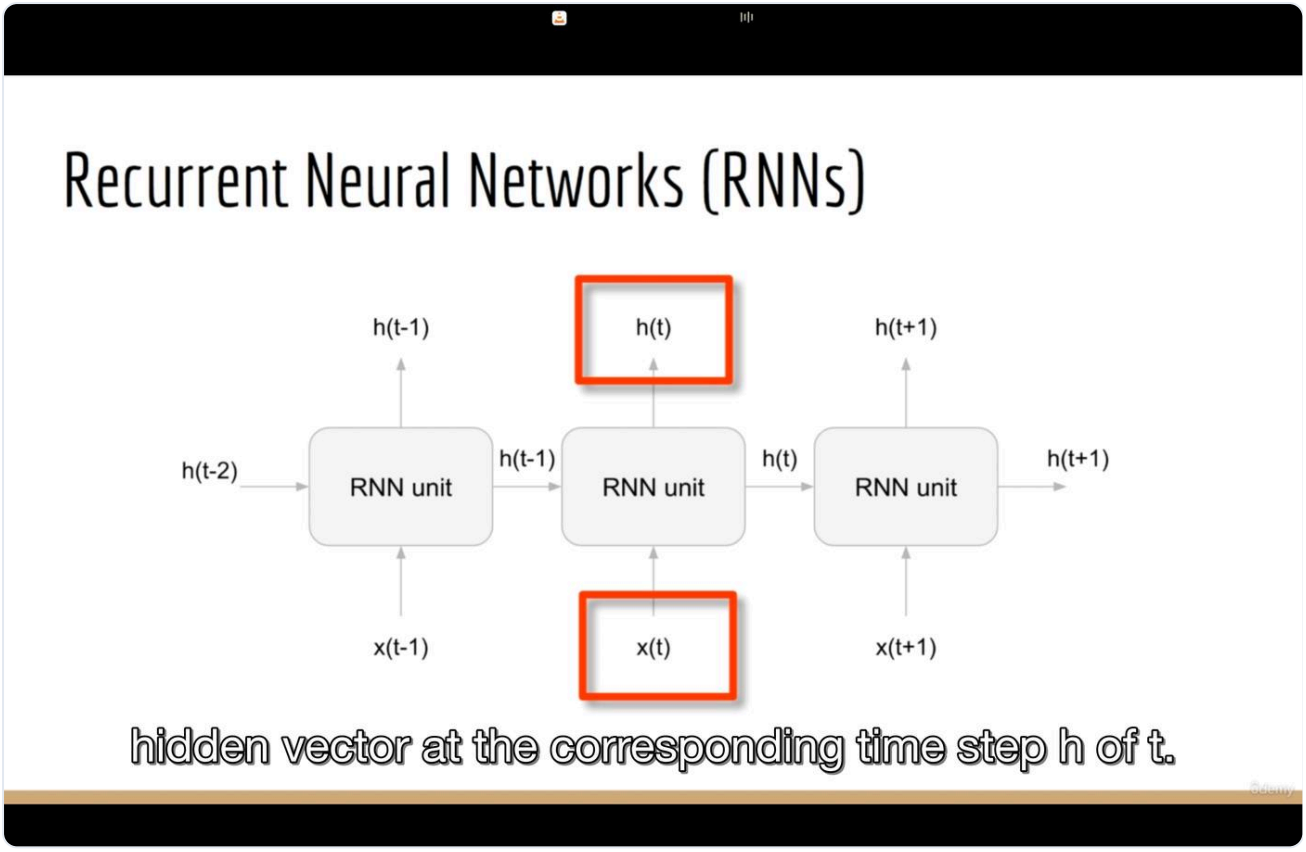

FIGURE 3.15 | Recurrent Neural Networks (RNNs)

As you recall, RNNs are used for sequences for each input x of t, an RNN unit will produce a hidden vector at the corresponding time step h of t. This hidden vector can be considered the output of the RNN unit, but also as a kind of state.

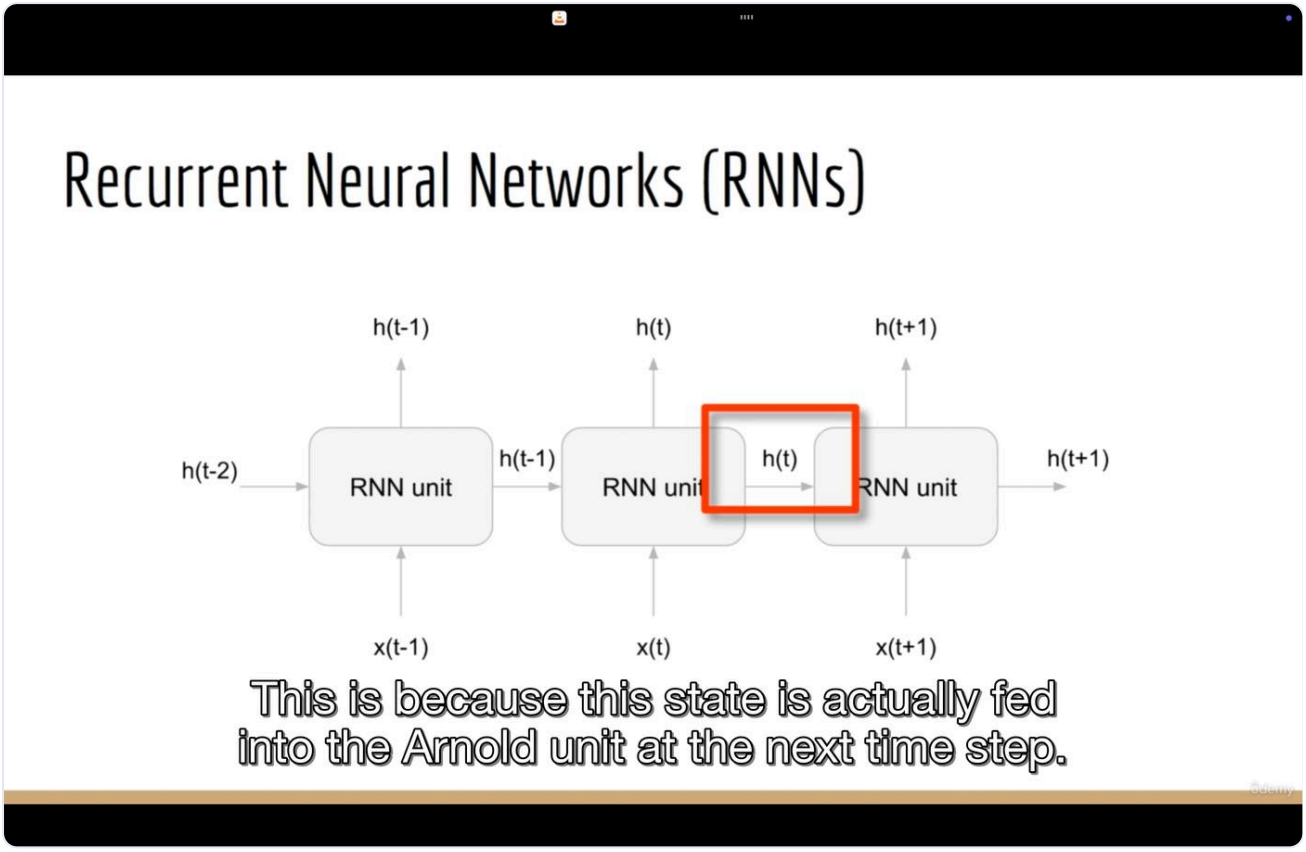

FIGURE 3.16 | Recurrent Neural Networks (RNNs)

This is because this state is actually fed into the RNN unit at the next time step.

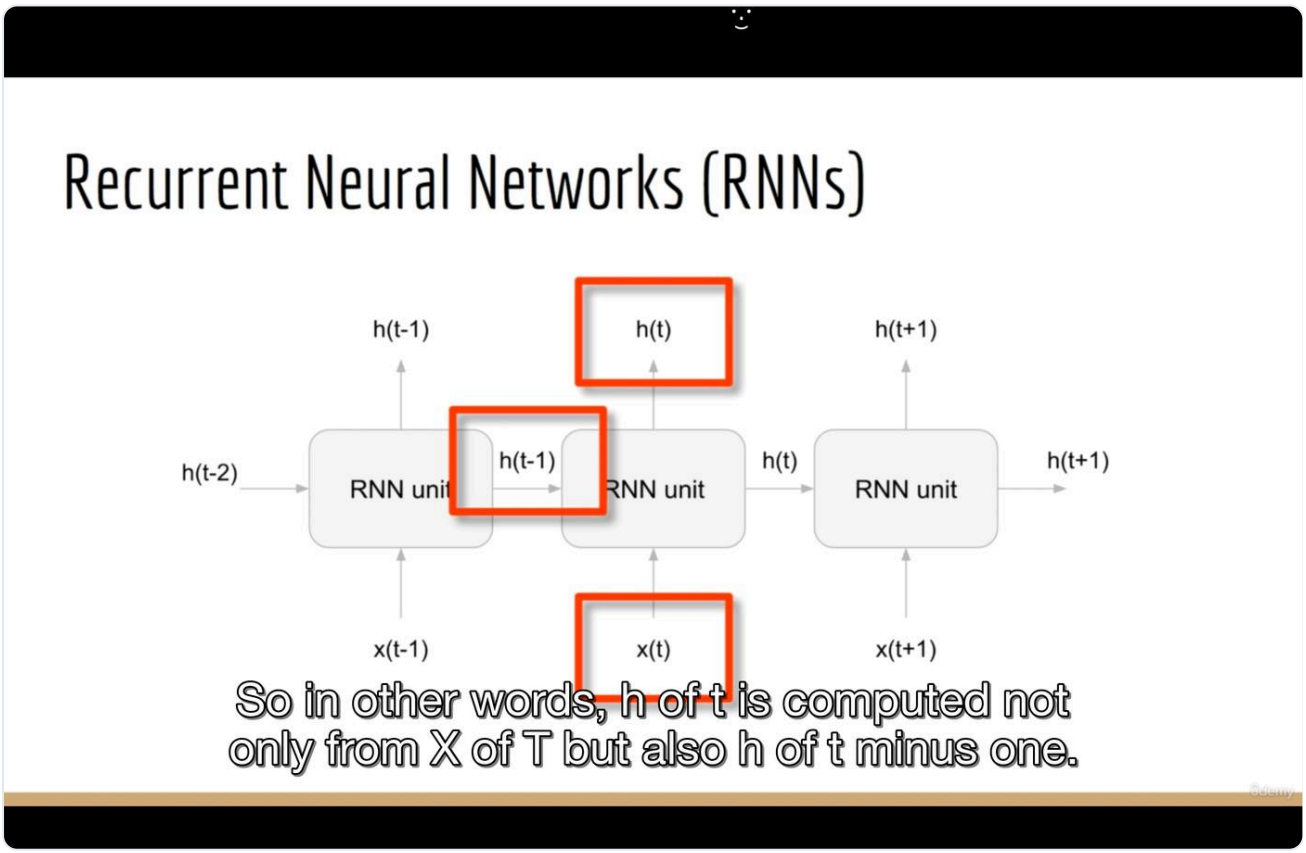

FIGURE 3.17 | Recurrent Neural Networks (RNNs)

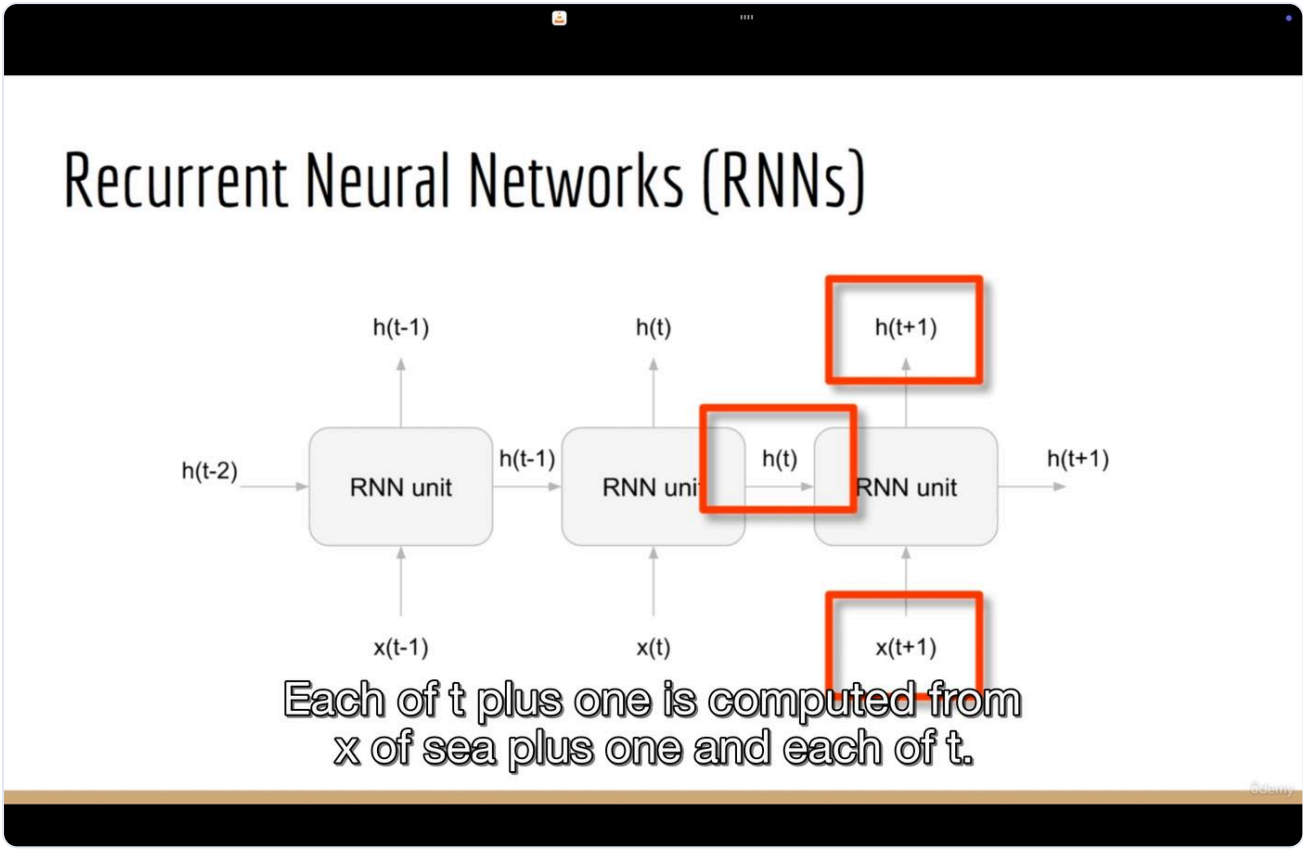

FIGURE 3.18 | Recurrent Neural Networks (RNNs)

So in other words, h of t is computed not only from X of T but also h of t minus one. h of t plus one is computed from x of t plus one and h of t.

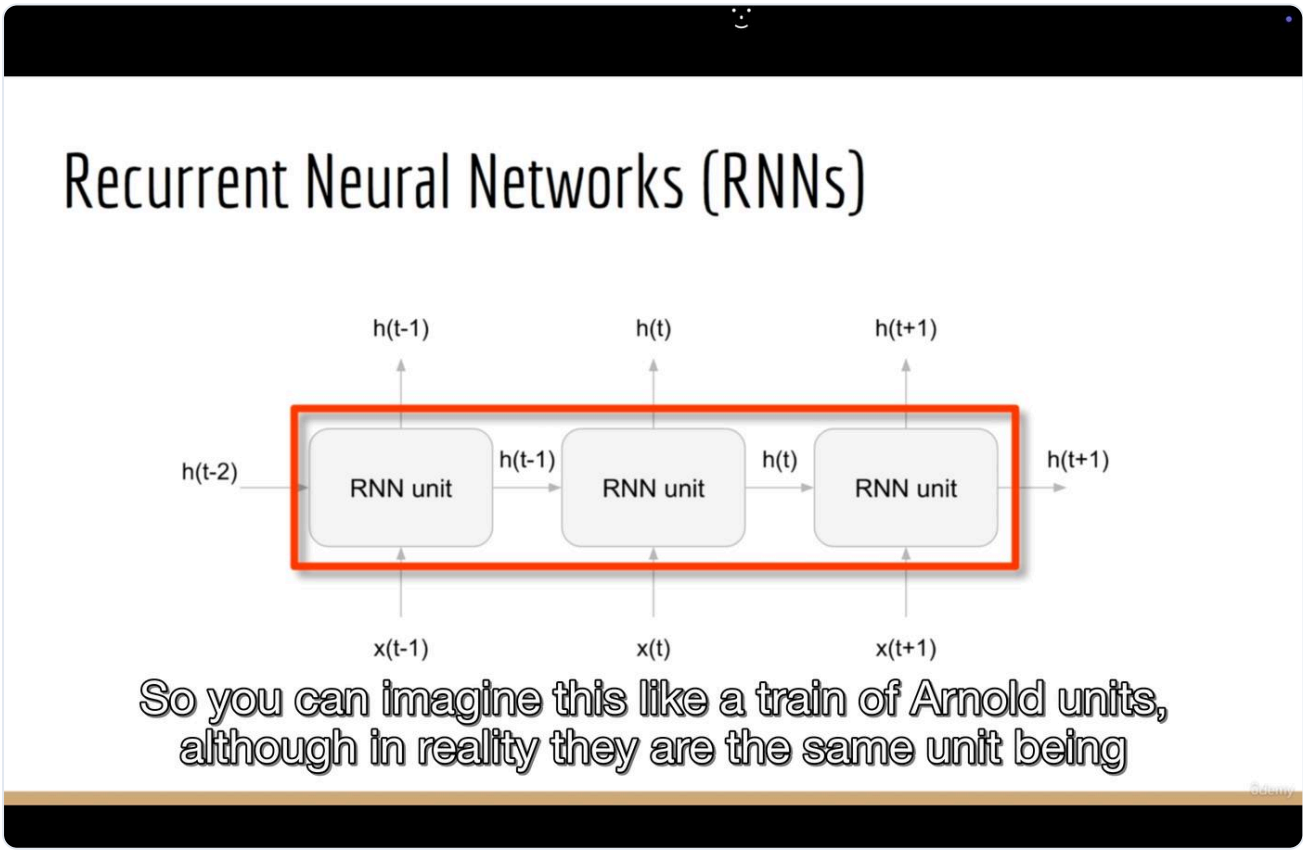

FIGURE 3.19 | Recurrent Neural Networks (RNNs)

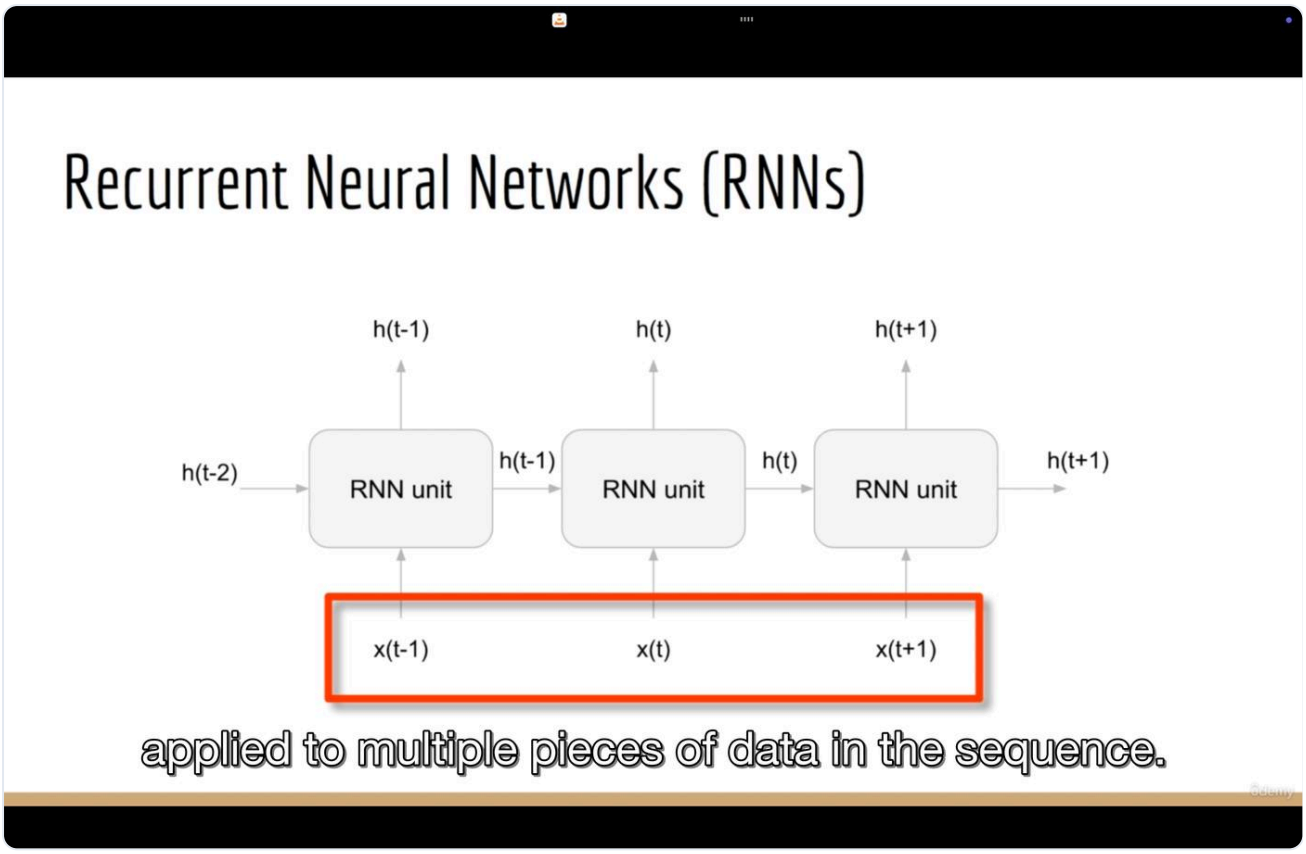

FIGURE 3.20 | Recurrent Neural Networks (RNNs)

So you can imagine this like a train of RNN units, although in reality they are the same unit being applied to multiple pieces of data in the sequence.

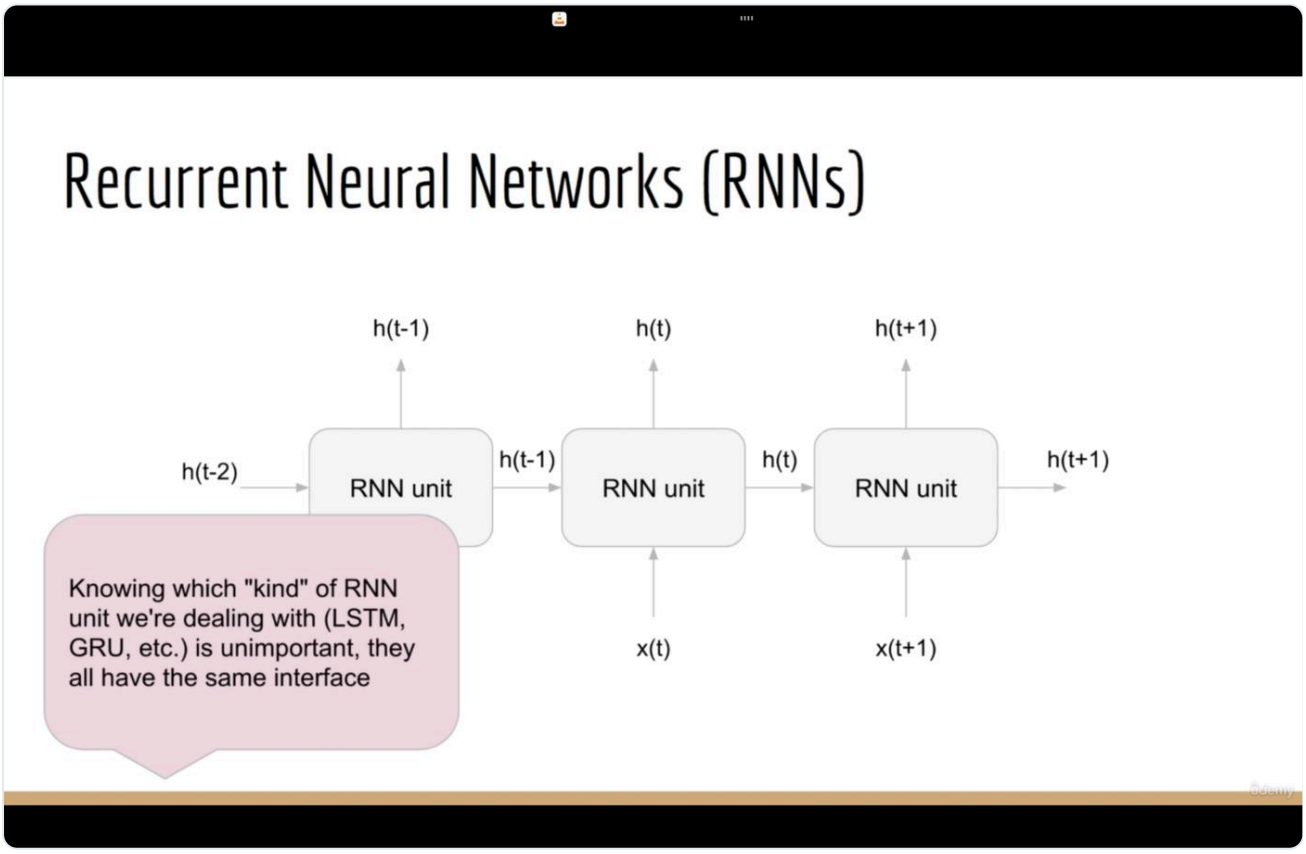

FIGURE 3.21 | Recurrent Neural Networks (RNNs)

And as you recall, there are multiple kinds of RNN units such as the LSTM or G, r u in this lecture, knowing which kind of unit we're dealing with is not required.

So as you recall, RNN units are used for multiple kinds of tasks which all use slightly different architectures.

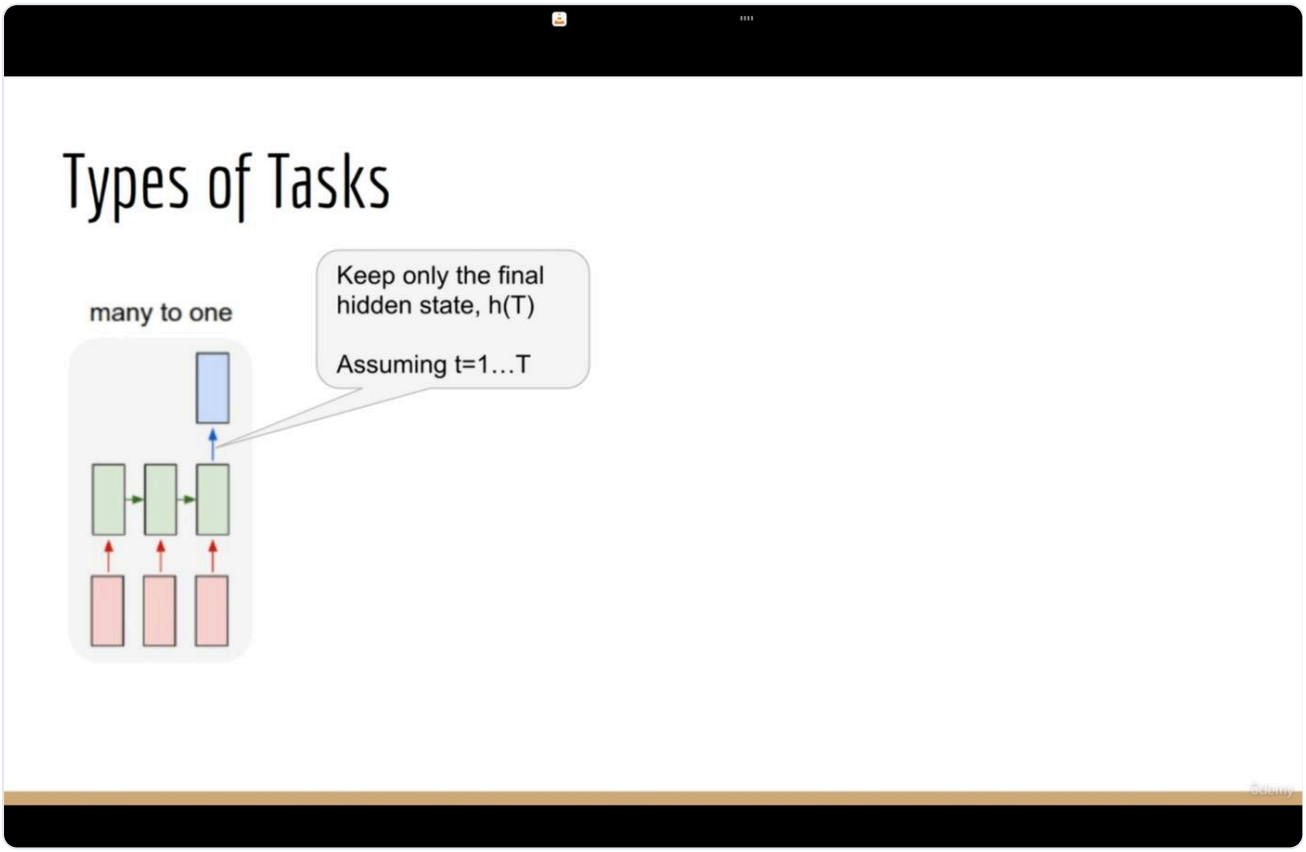

FIGURE 3.22 | Types of Tasks

For instance, we might be doing a many-to-one task, such as spam detection. In this case, we only have a single output from our RNN, which is typically computed from the final hidden state in the simplest case.

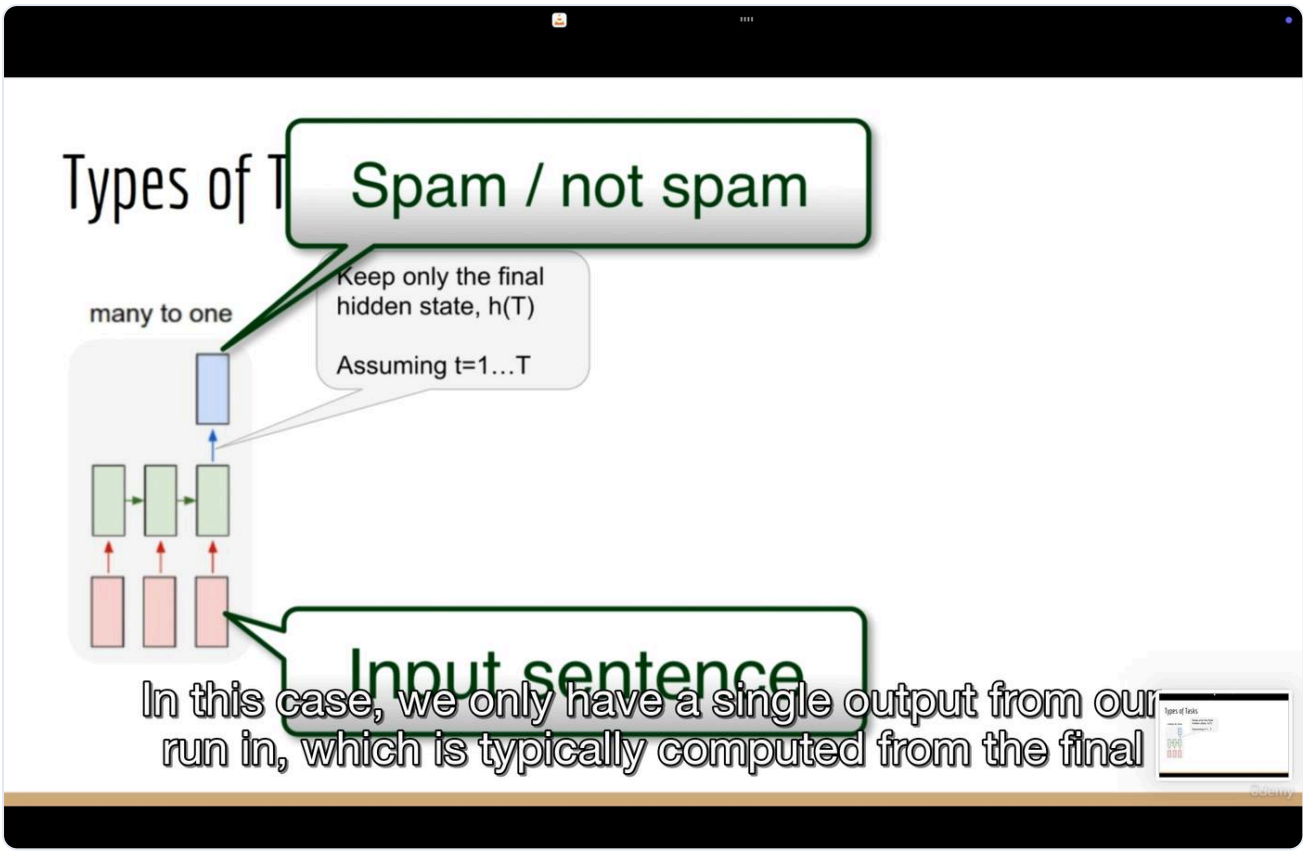

FIGURE 3.23 | Types of

This final hidden state, h of big T, is passed through a simple, dense layer which then produces an output probability. However, there is another kind of task known as a many-to-many task where we have multiple outputs.

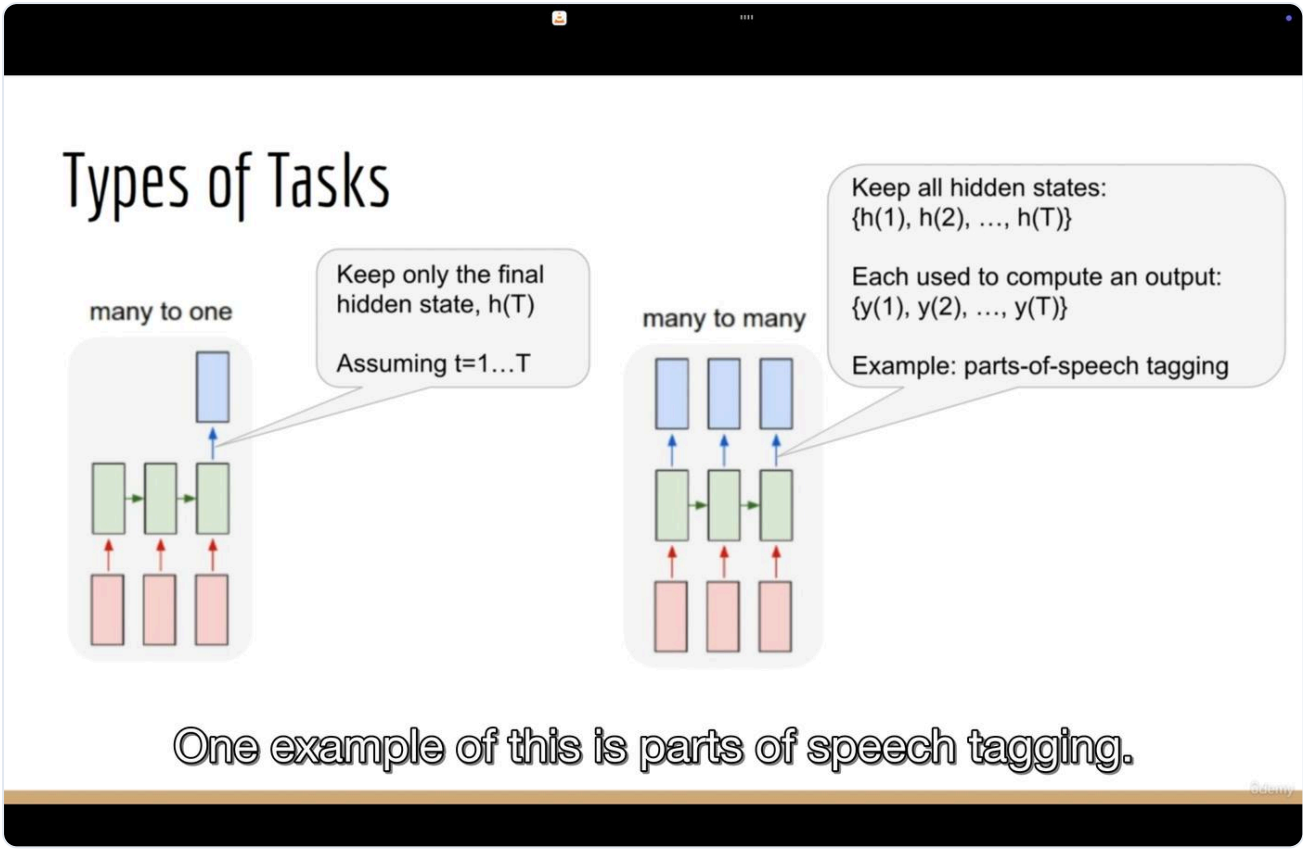

FIGURE 3.24 | Types of Tasks

One example of this is parts of speech tagging. In this case, we have one output for every word in the input sequence.

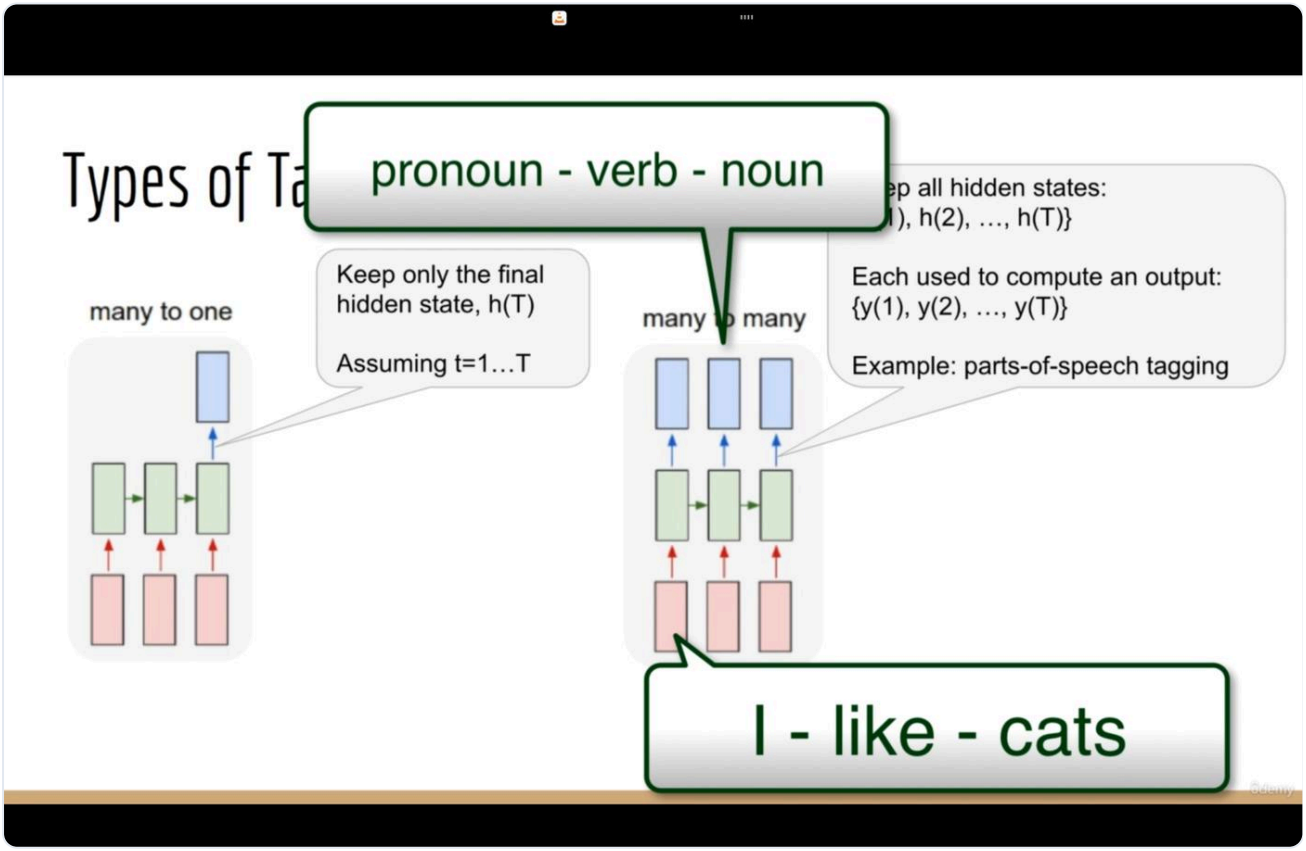

FIGURE 3.25 | Types of

In other words, every word gets a parts of speech tag.

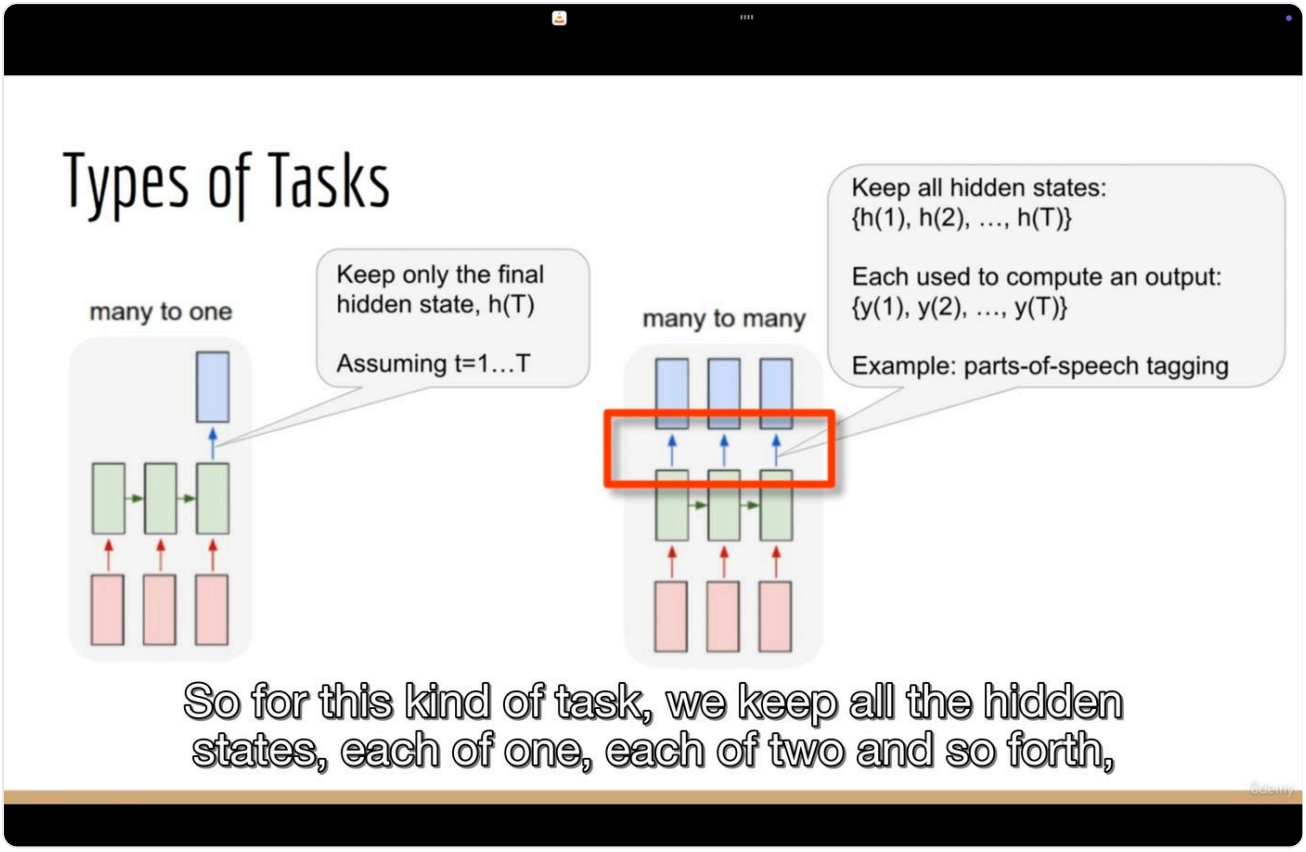

FIGURE 3.26

So for this kind of task, we keep all the hidden states, h of one, h of two and so forth, up to h of big T.

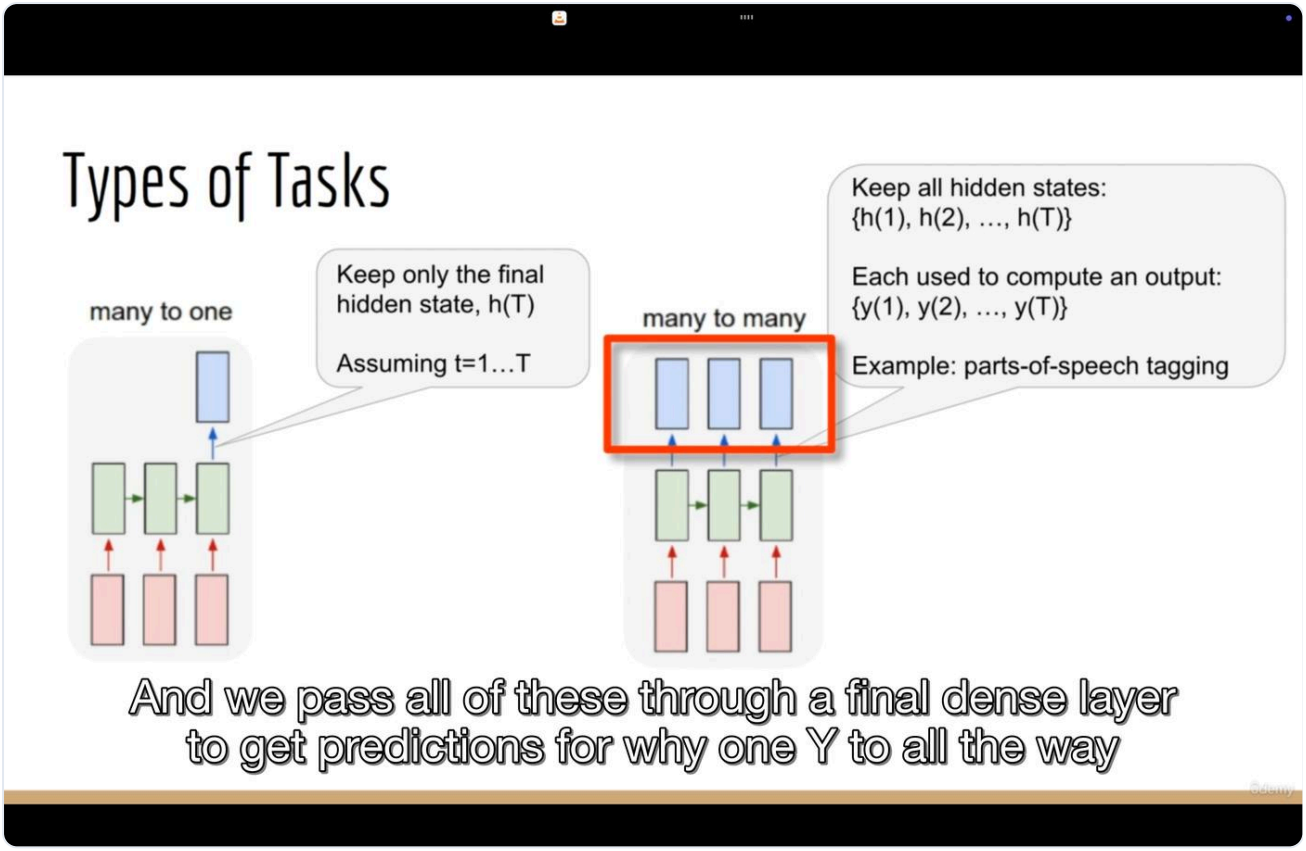

FIGURE 3.27

And we pass all of these through a final dense layer to get predictions for y one, y two, all the way up to y big T. Now, here's a scenario where neither of these previous two methods will work.

Suppose we're doing language translation where the input is a sequence of words in the target is also a sequence of words, but in a different language. As you may know, if you speak different languages, this is not as straightforward as simply translating one word at a time due to every language having different rules and different numbers of tokens or words to represent the same concept.

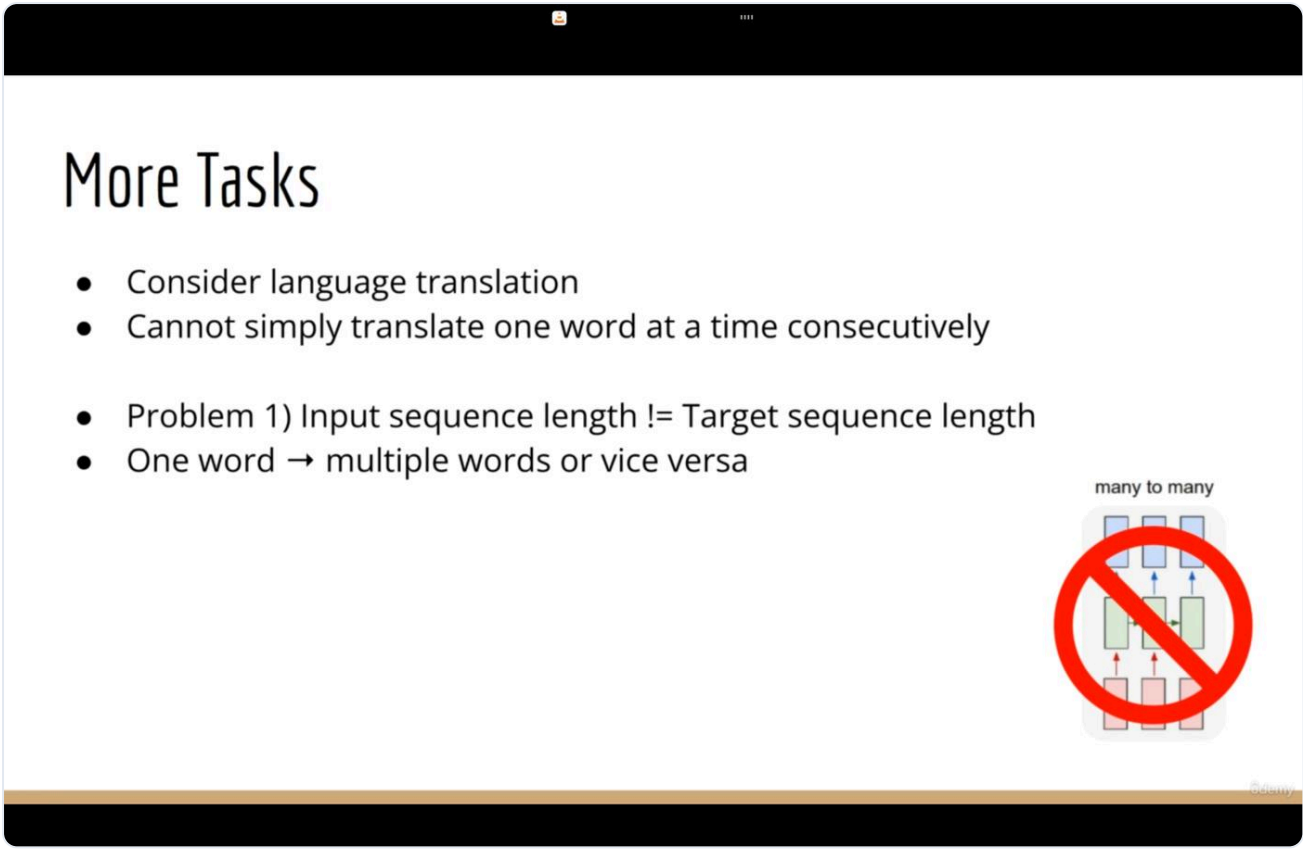

FIGURE 3.28 | More Tasks

Put more simply, there are two problems. Problem number one is that the input sequence length is not necessarily equal to the target sequence length.

For instance, one word from the input might map to multiple words in the target or vice versa.

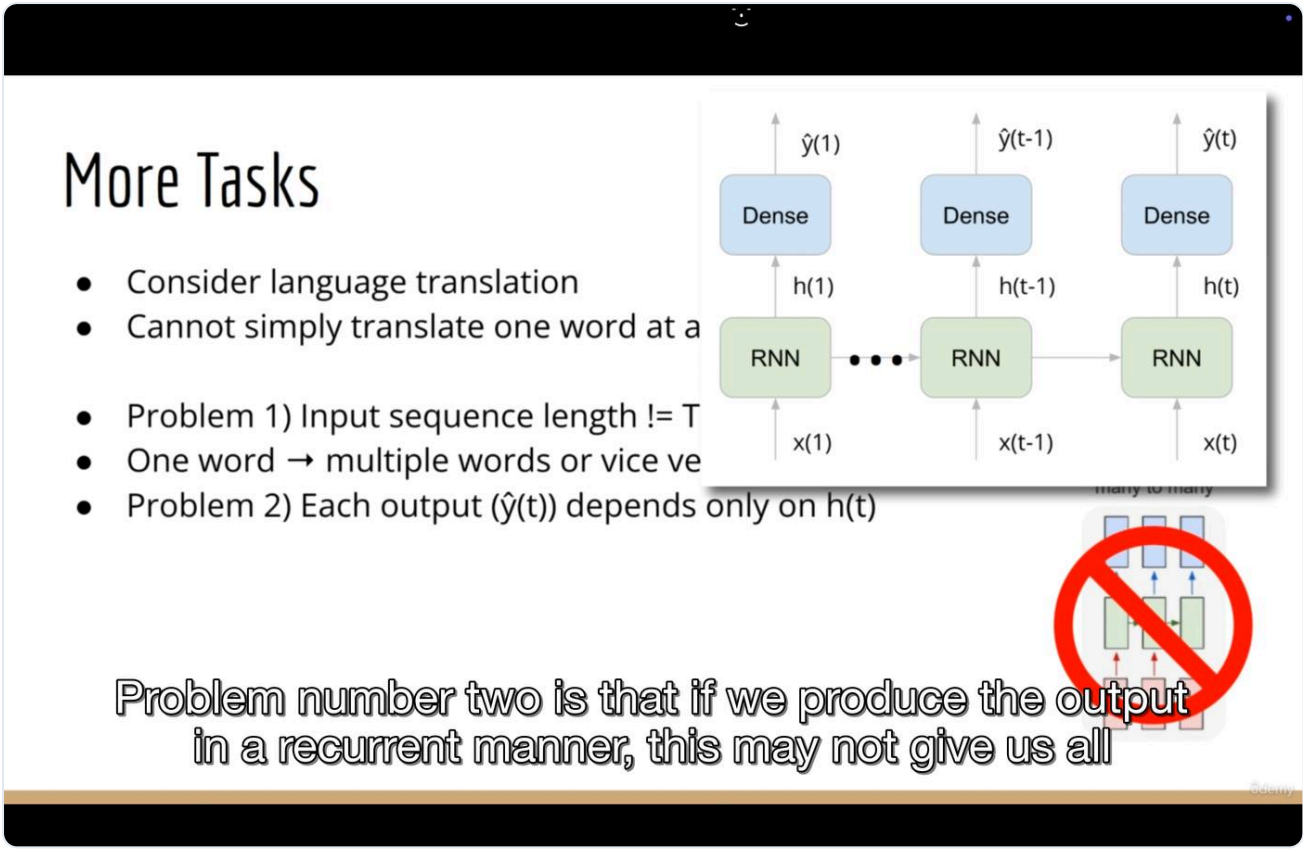

FIGURE 3.29 | More Tasks

Problem number two is that if we produce the output in a recurrent manner, this may not give us all the information we need to compute an accurate prediction.

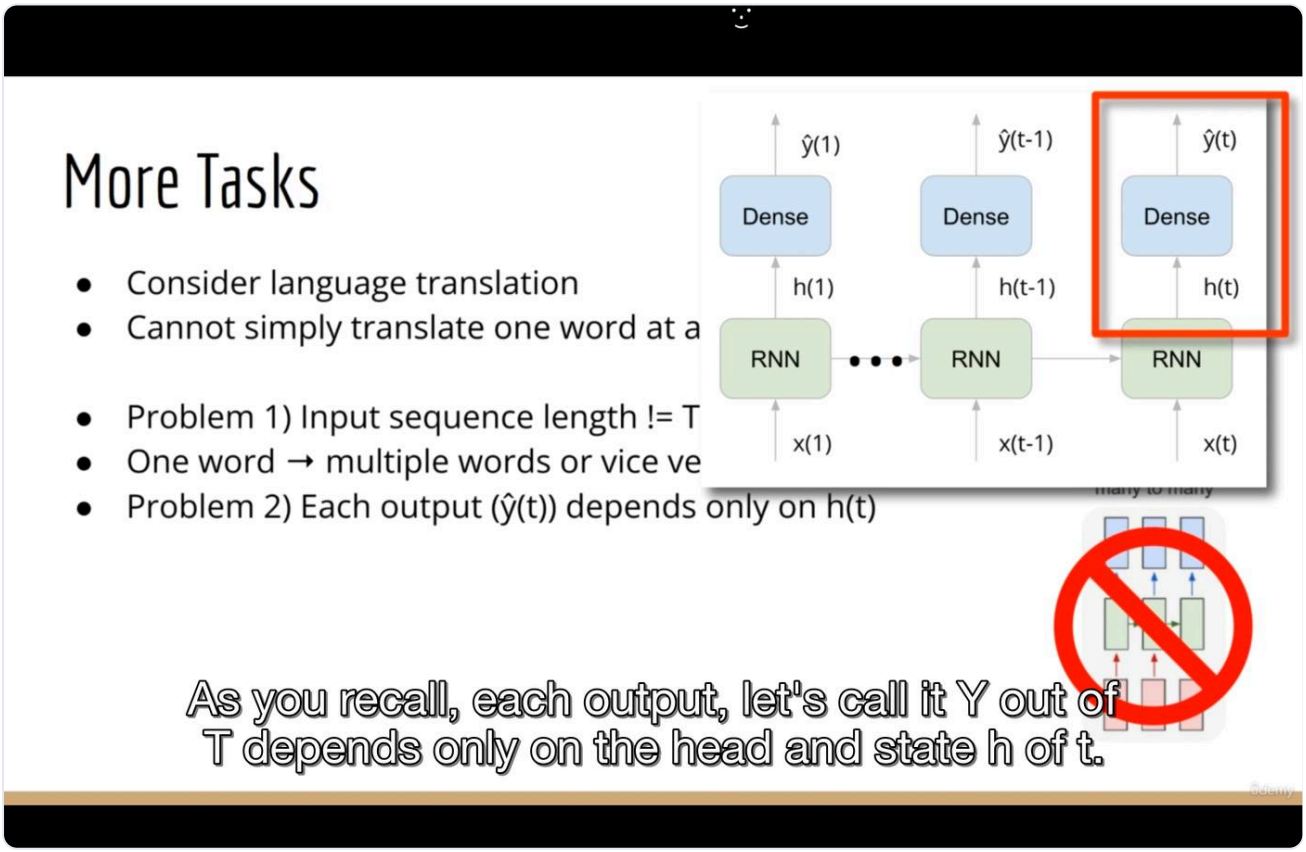

FIGURE 3.30 | More Tasks

As you recall, each output, let's call it y hat of t, depends only on the hidden state h of t. But what does h of t depend on?

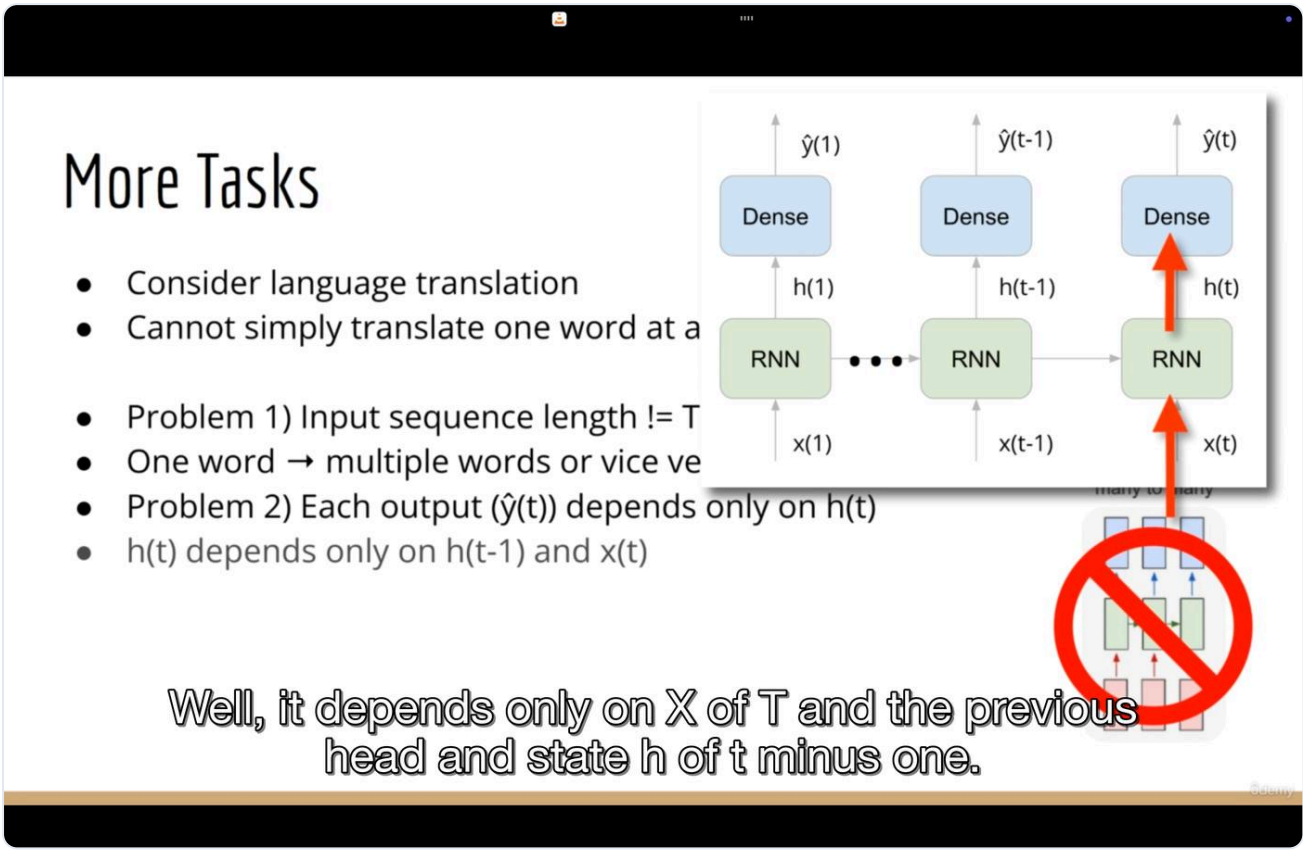

FIGURE 3.31 | More Tasks

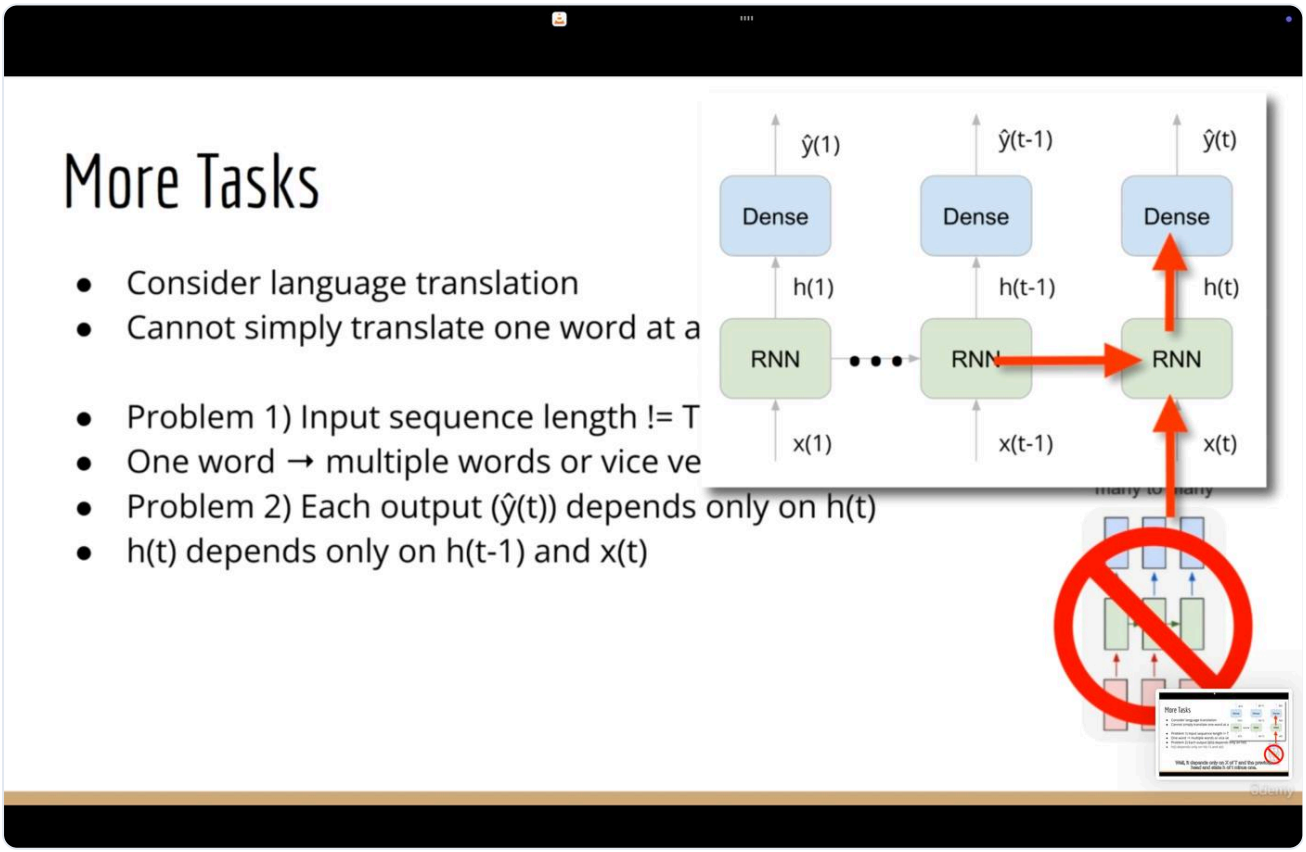

FIGURE 3.32

Well, it depends only on X of T and the previous hidden state h of t minus one.

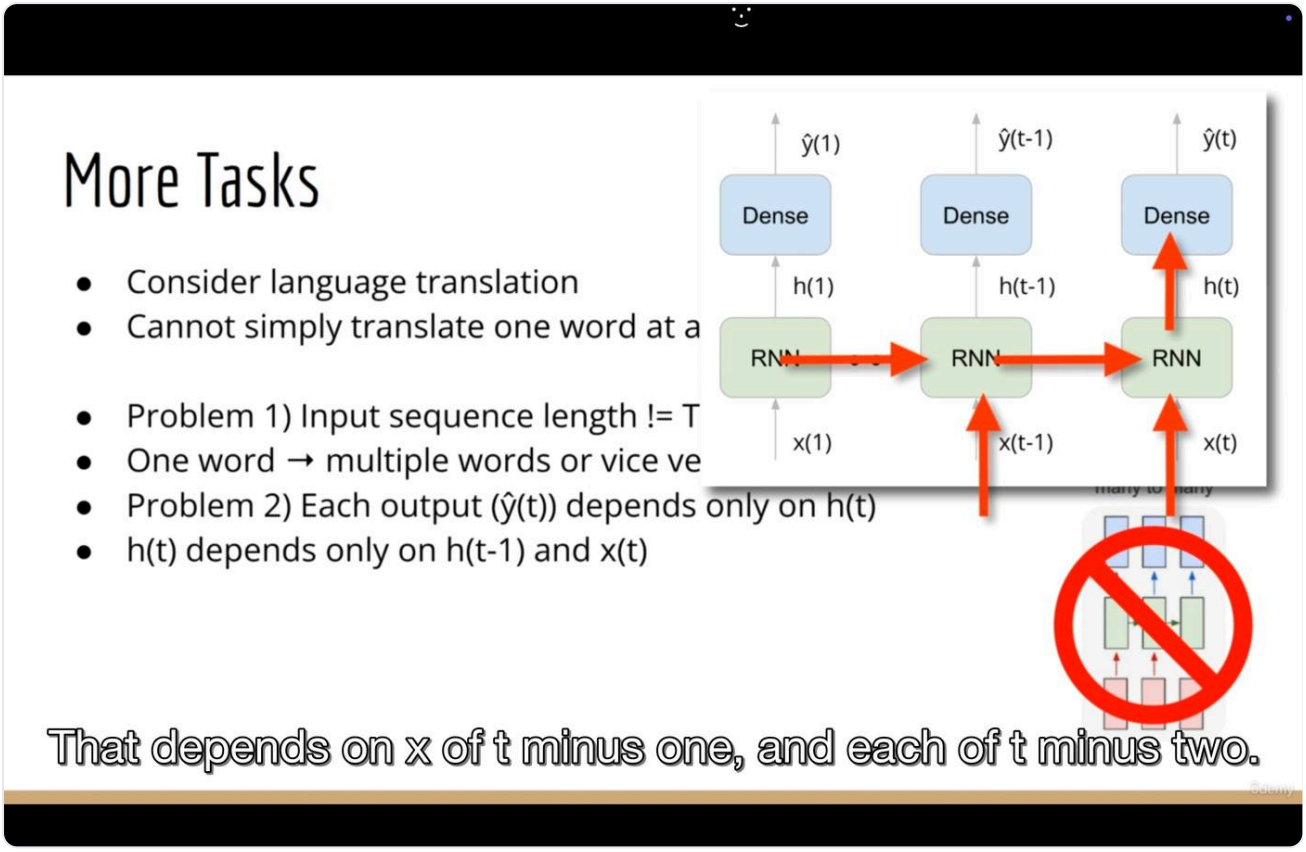

FIGURE 3.33 | More Tasks

In turn, h of t minus one depends on x of t minus one and h of t minus two.

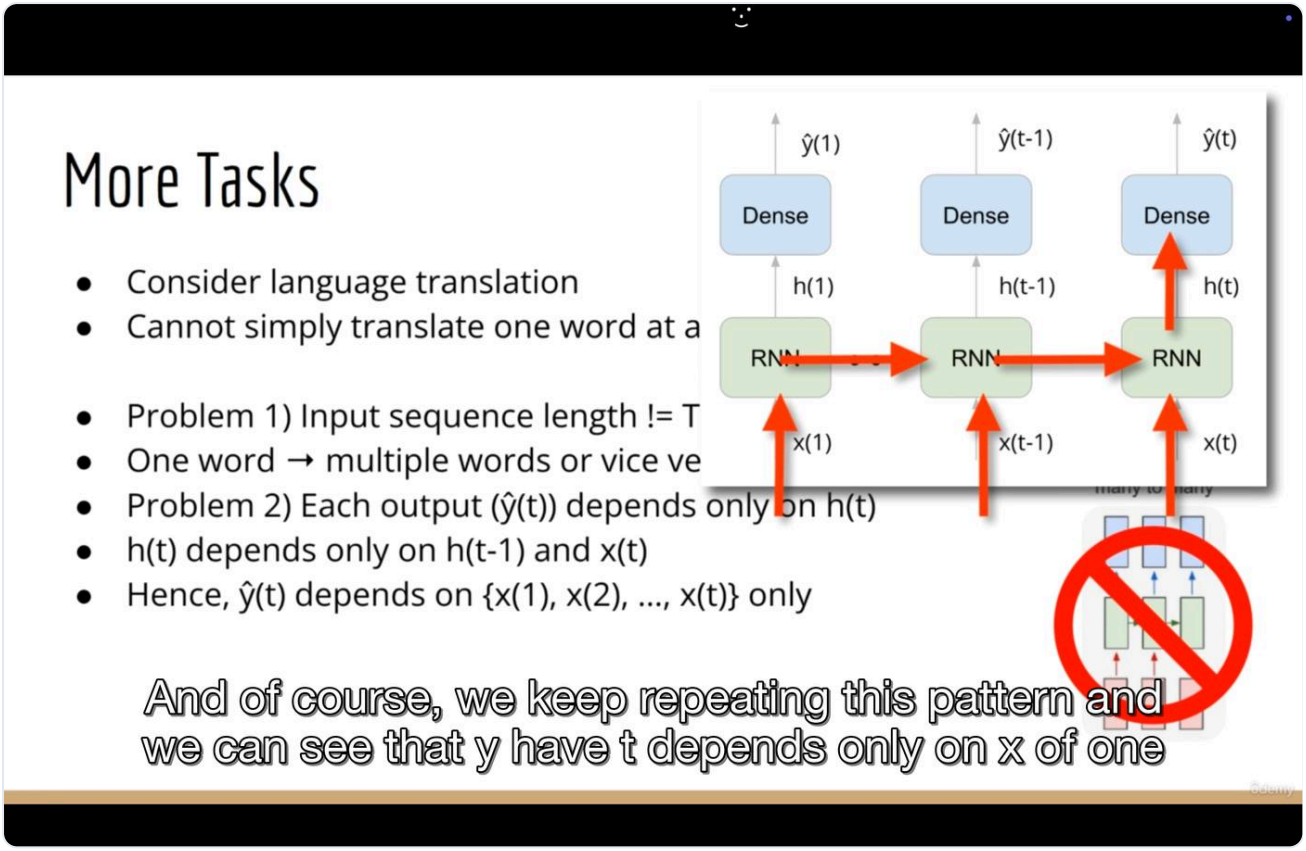

FIGURE 3.34 | More Tasks

And of course, we keep repeating this pattern, and we can see that y hat of t depends only on x of one up to x of t. In other words, the prediction at time t depends only on the inputs up to this time.

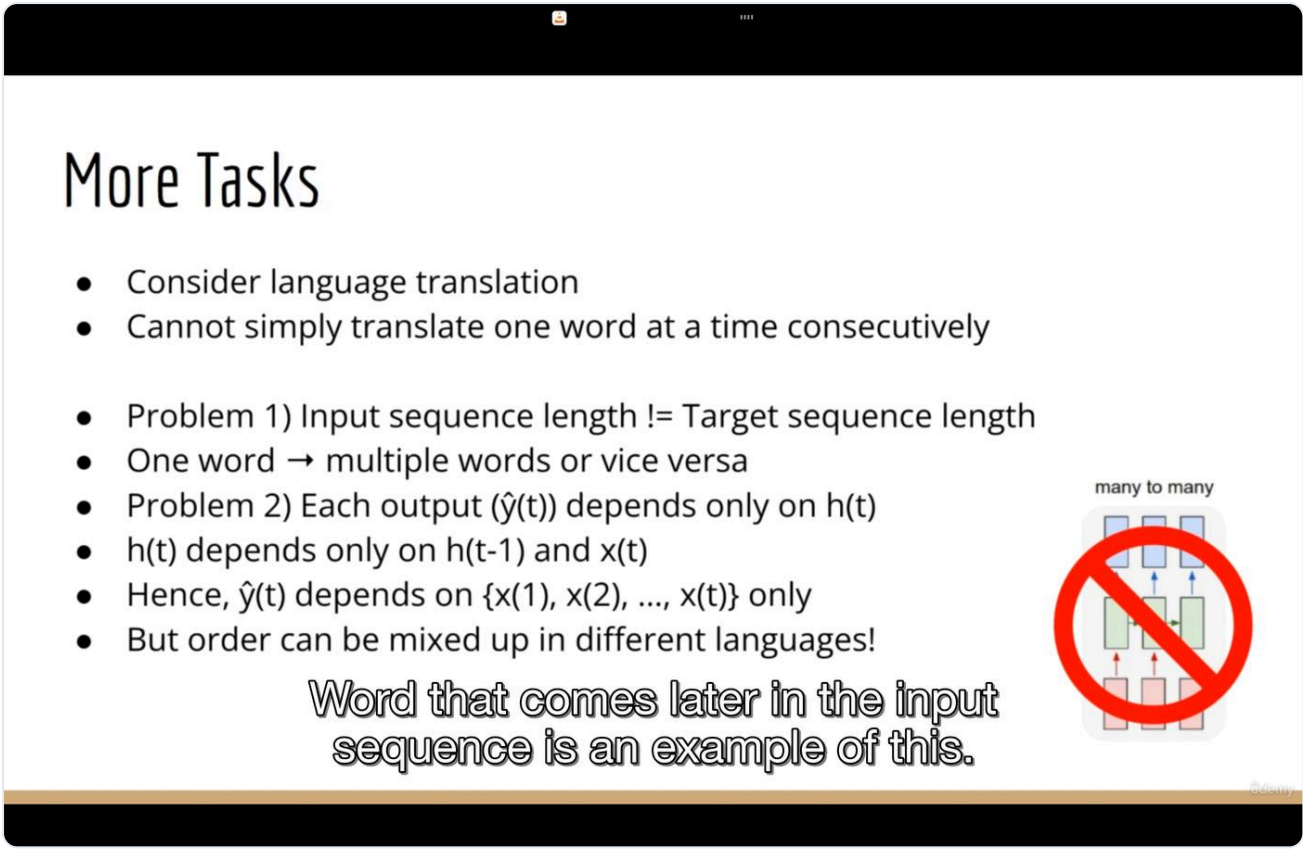

FIGURE 3.35 | More Tasks

However, this doesn't make any sense for language translation because there can be scenarios where the translated word depends on an input word that comes later in the input sequence. As an example of this,

we can note that in French adjectives come after nouns, which is not the case in English.

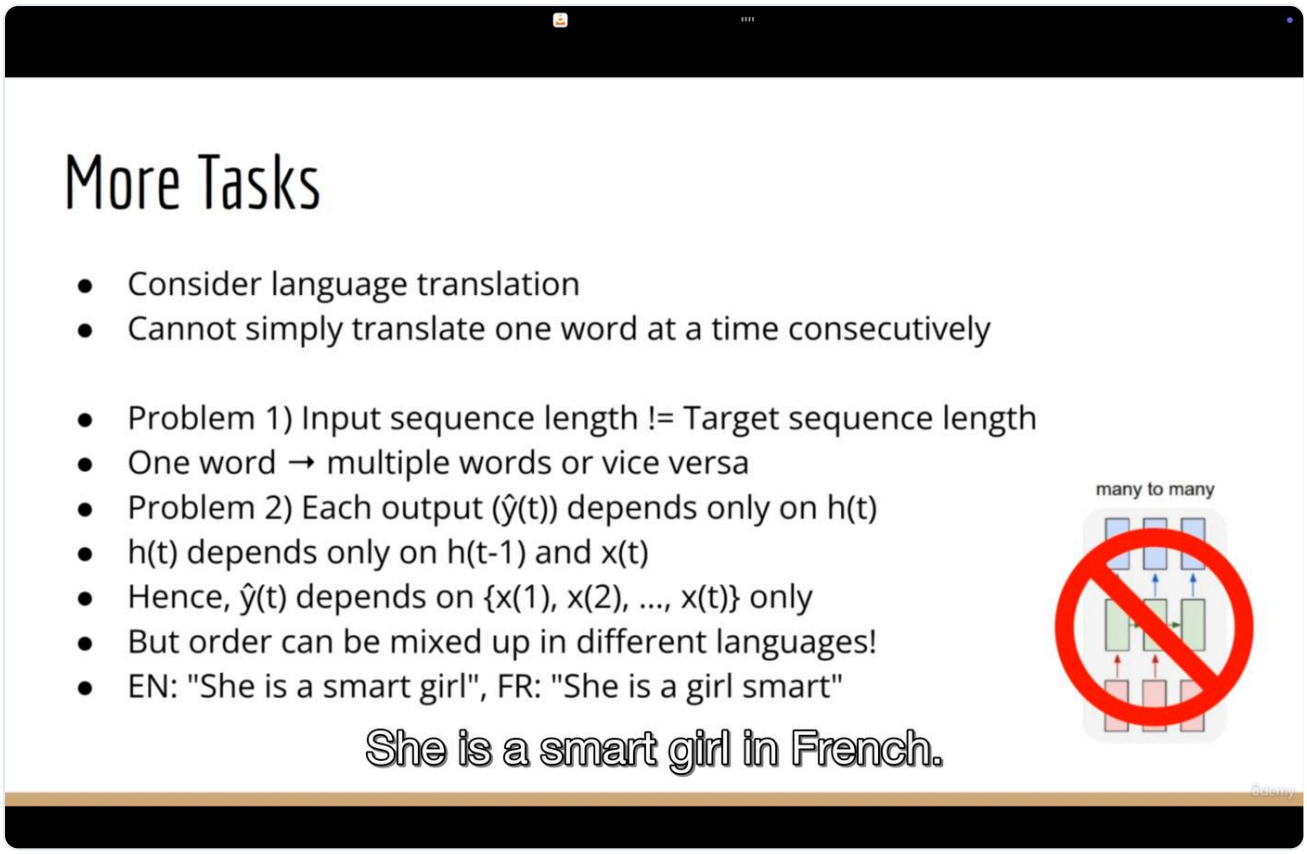

FIGURE 3.36 | More Tasks

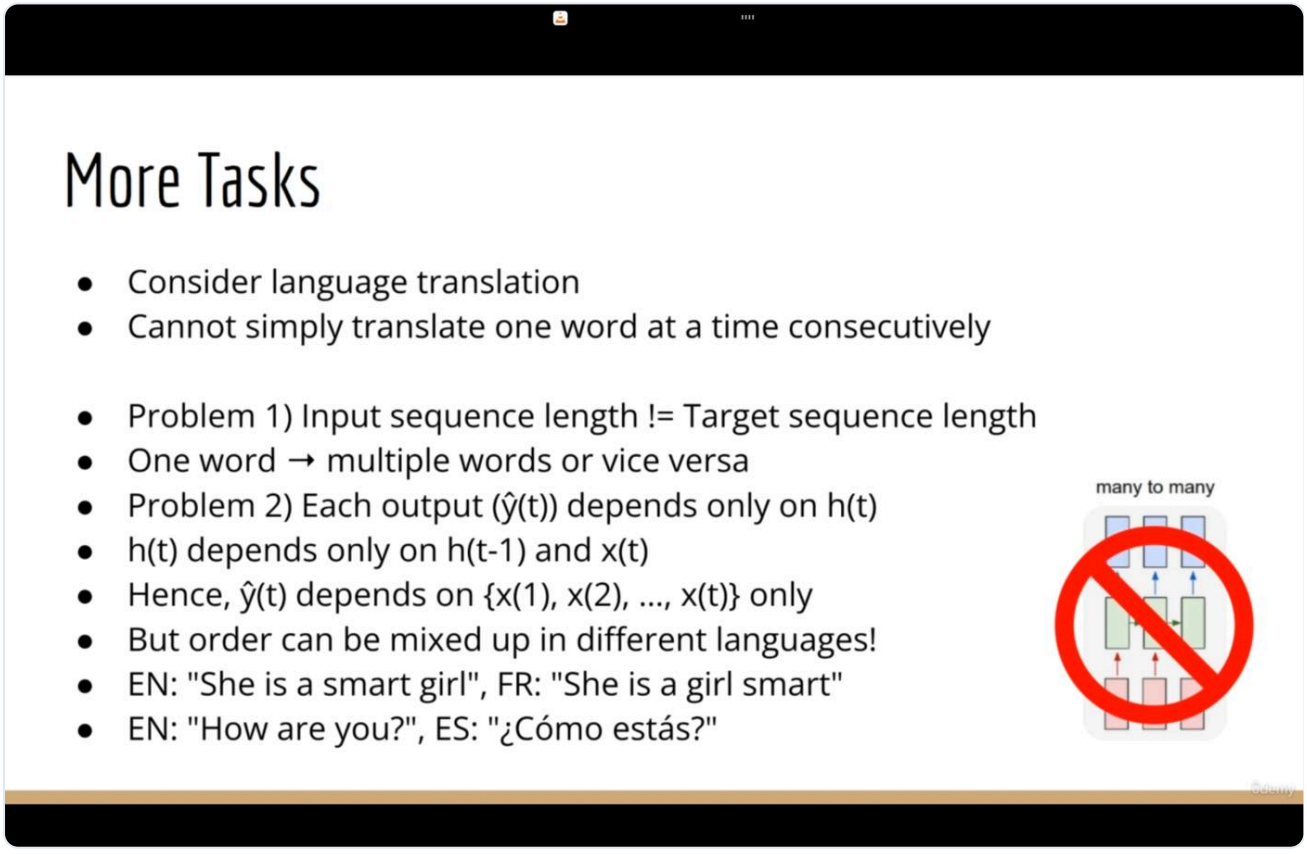

FIGURE 3.37 | More Tasks

For instance, take the English sentence "She is a smart girl". In French, The equivalent ordering would be she is a girl smart.

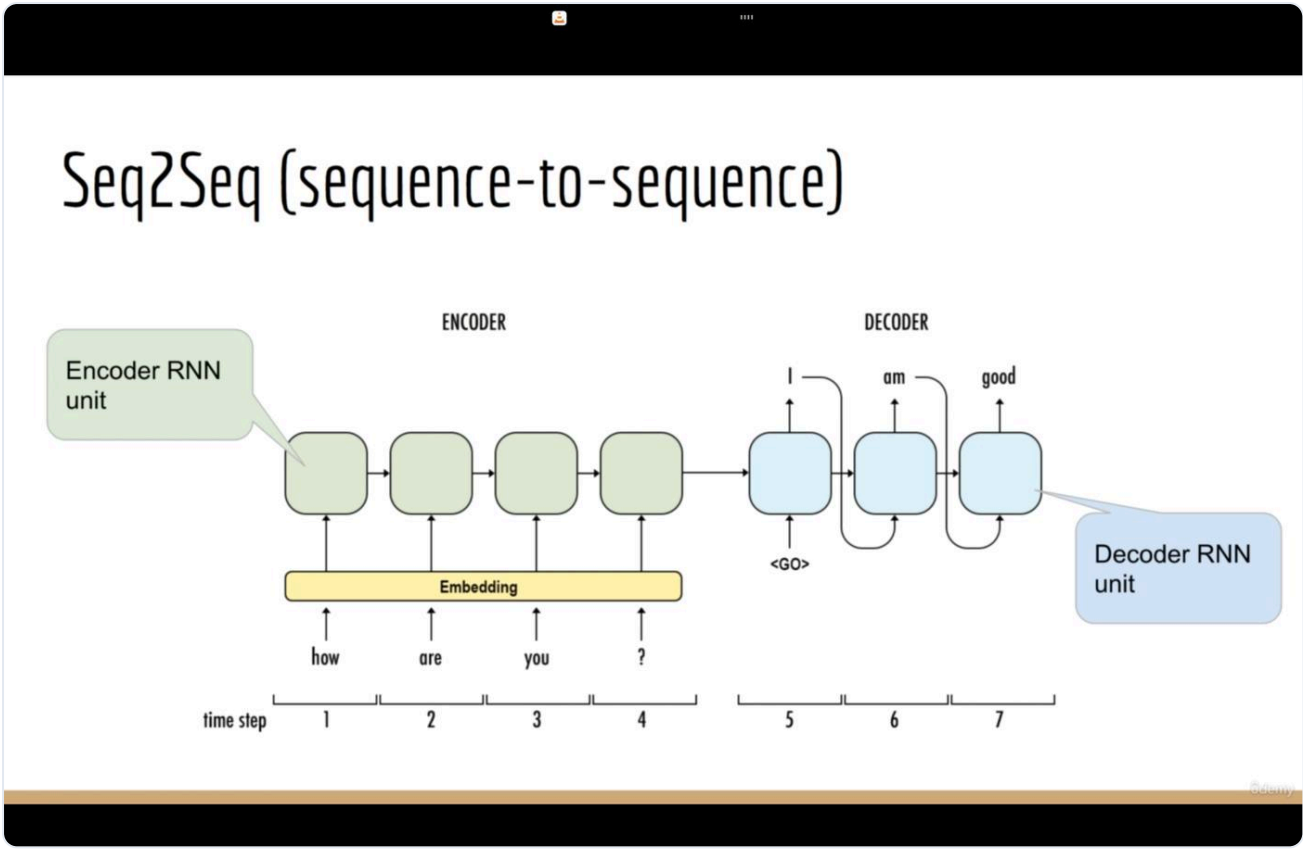

FIGURE 3.38

Another example is that in Spanish questions not only ends with a question mark, but they also begin with an inverted question mark and thus there are two tokens. If we're translating a question into Spanish that would be used to denote that our input was a question, but that corresponds to only one token in English, which appears at the end of the sentence.

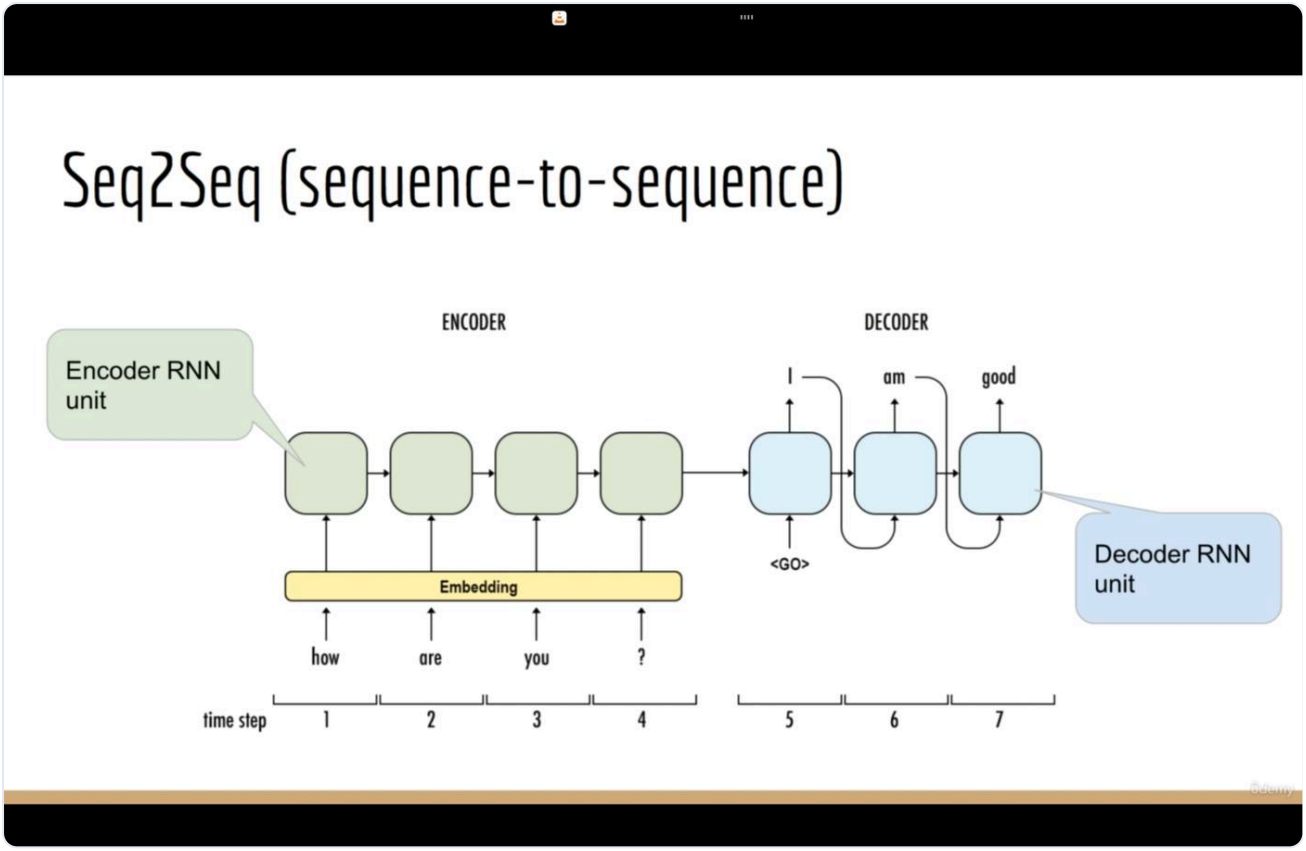

FIGURE 3.39

So what is the solution to this problem? Well, the initial solution to this problem was called seq2seq, short for sequence to sequence. This introduces the concept of having two RNNs, one called the encoder and one called the decoder.

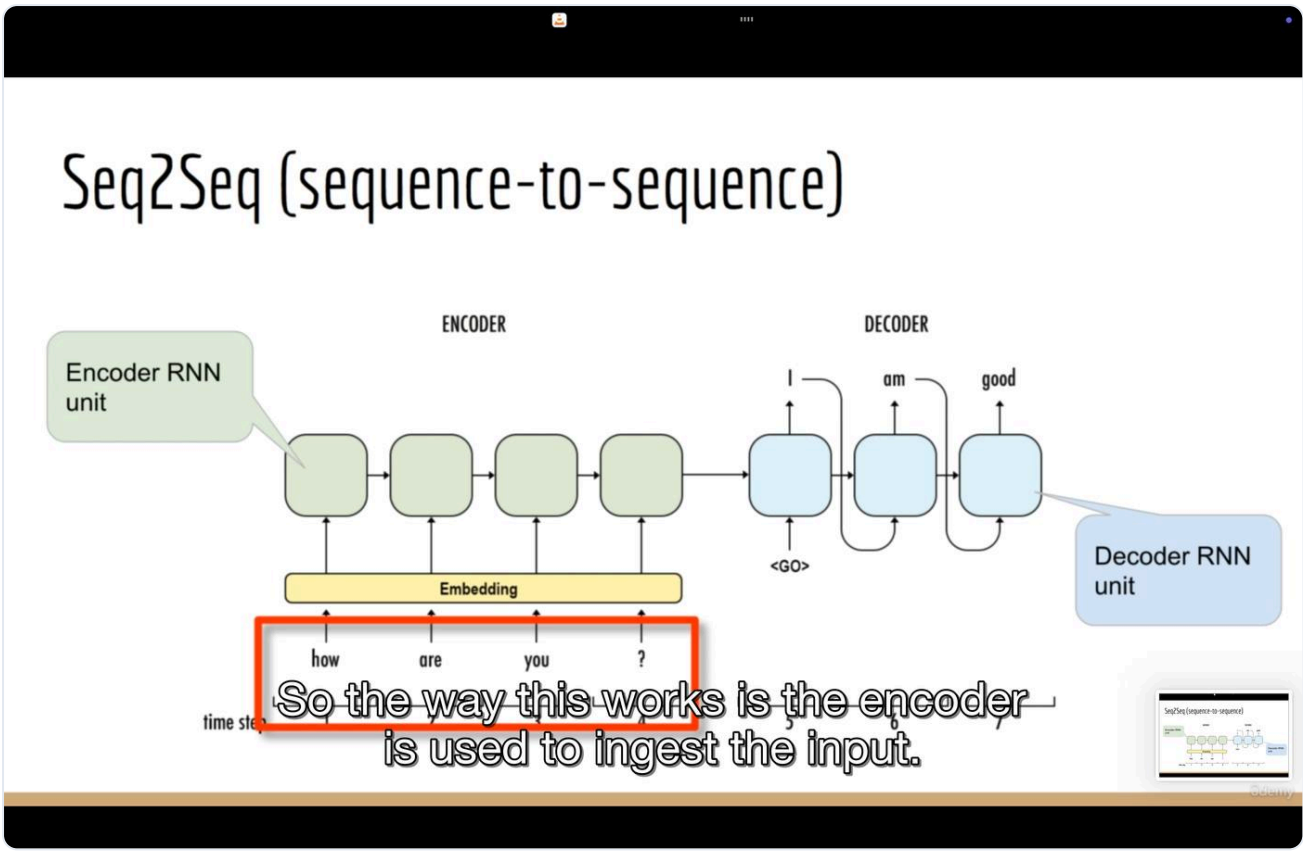

FIGURE 3.40

This is helpful in understanding transformers as well, since some transformers also use an encoder decoder system. So the way this works is the encoder is used to ingest the input.

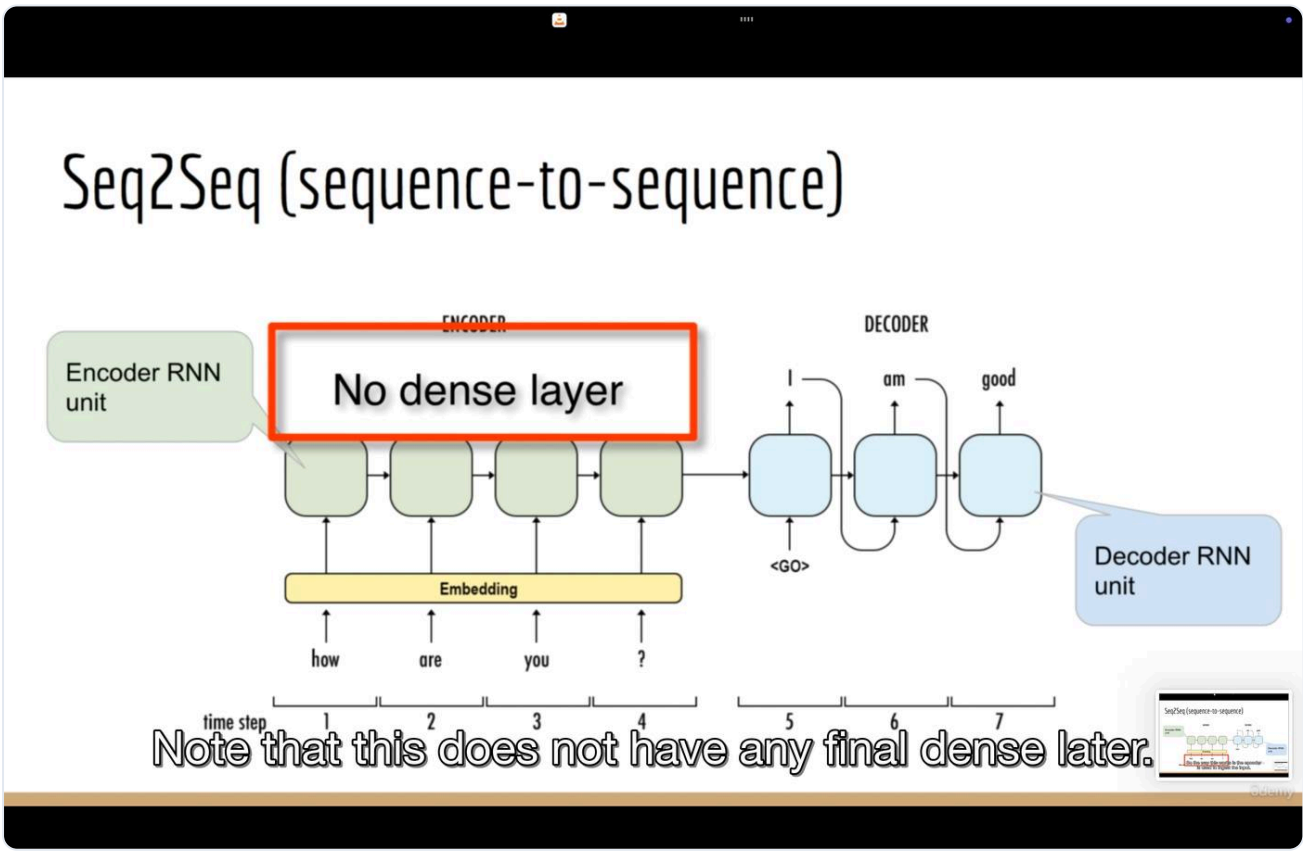

FIGURE 3.41 | Encoder RNN Am 00

Note that this does not have any final dense layer.

We simply compute all the h of t's at every time step of the input.

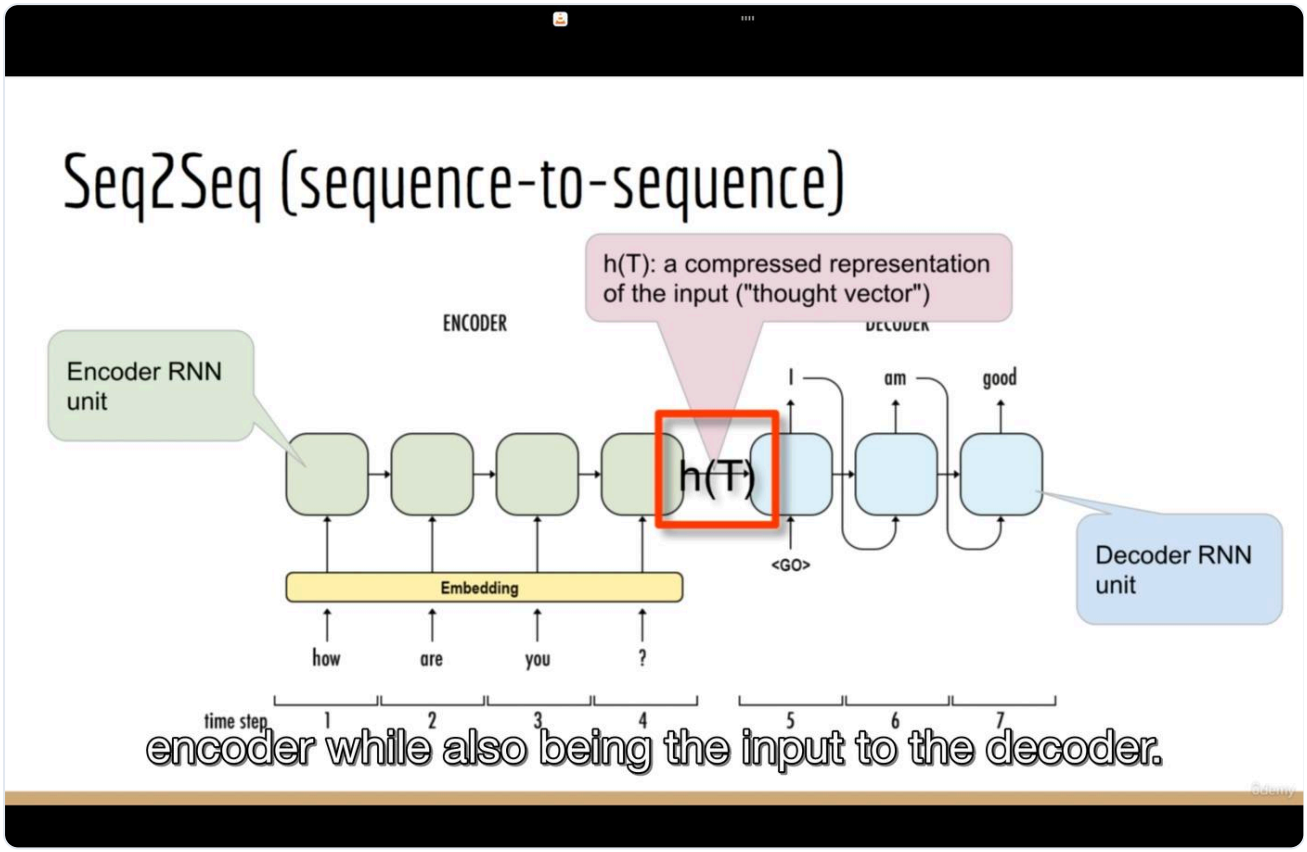

FIGURE 3.42

This will give us one final hidden state vector, which we'll call h of big T, assuming that the input sequence has length big T. h of big T will be considered the output of the encoder while also being the input to the decoder. You can think of h of big T as a vector, which is like a compressed representation of the input sequence, hence why we call it an encoder.

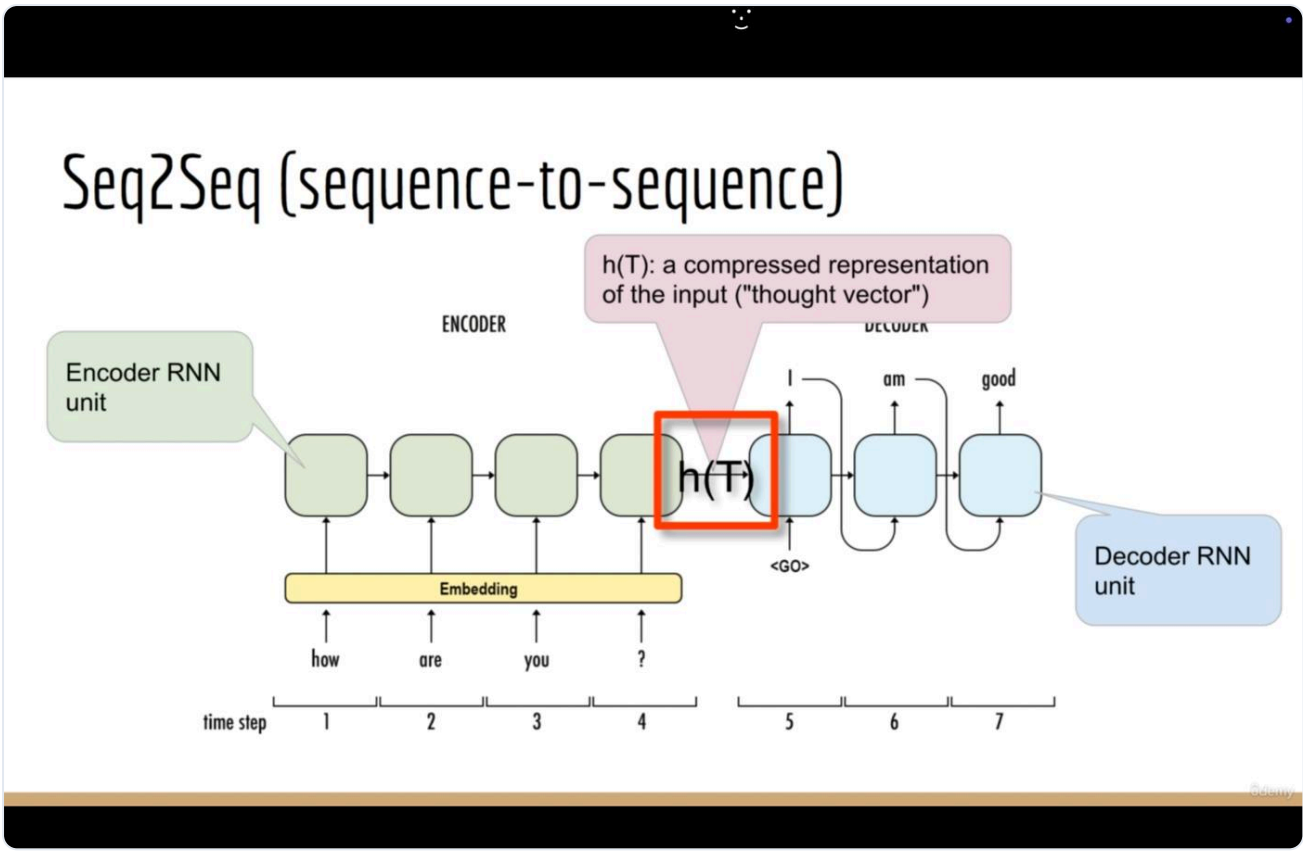

FIGURE 3.43

So the decoder is also an RNN, but a different RNN from the encoder.

Now, as you recall, every RNN unit has two sources of input data, the first being the actual input and the second being the previous hidden state.

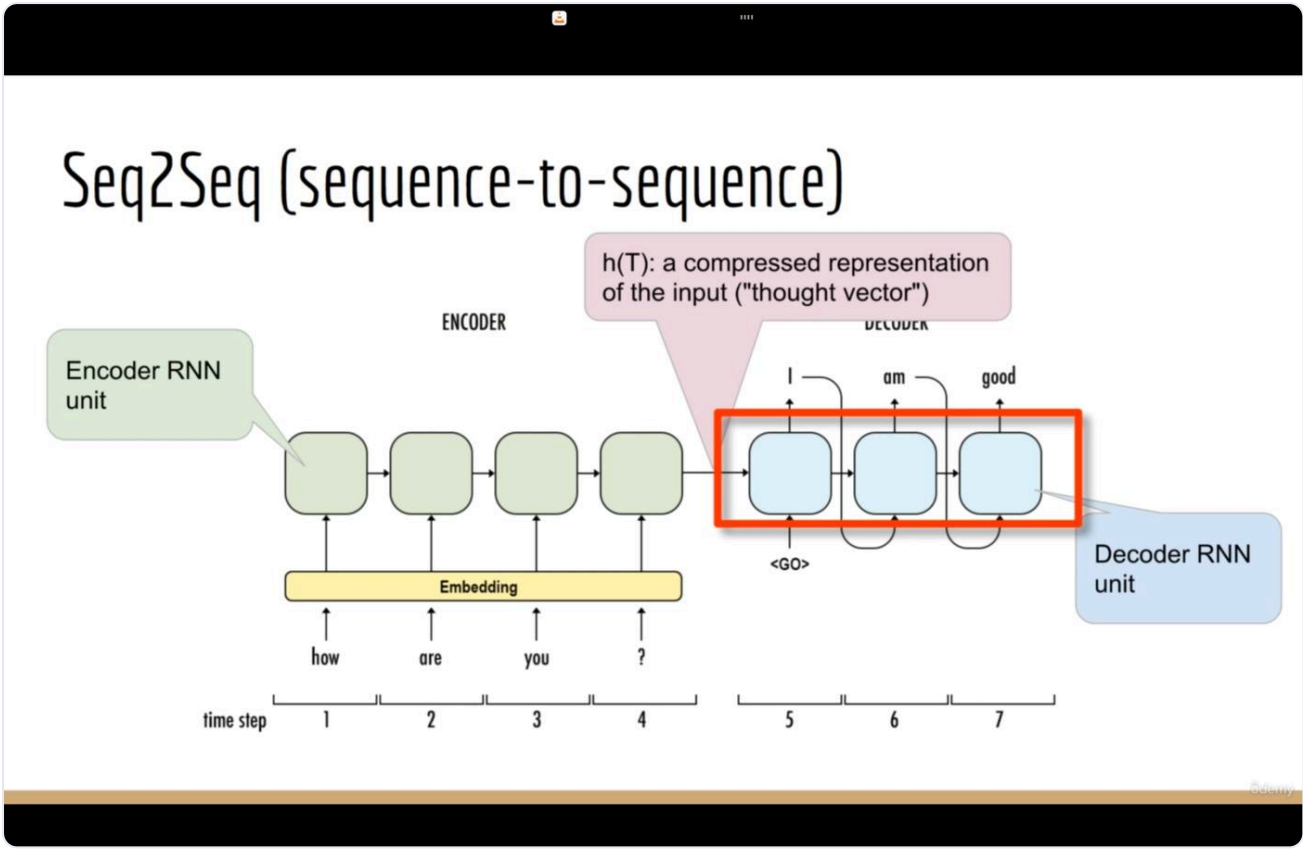

FIGURE 3.44

You can see that in this diagram. The inputs go along the bottom of the RNN unit while the previous hidden state comes through the left.

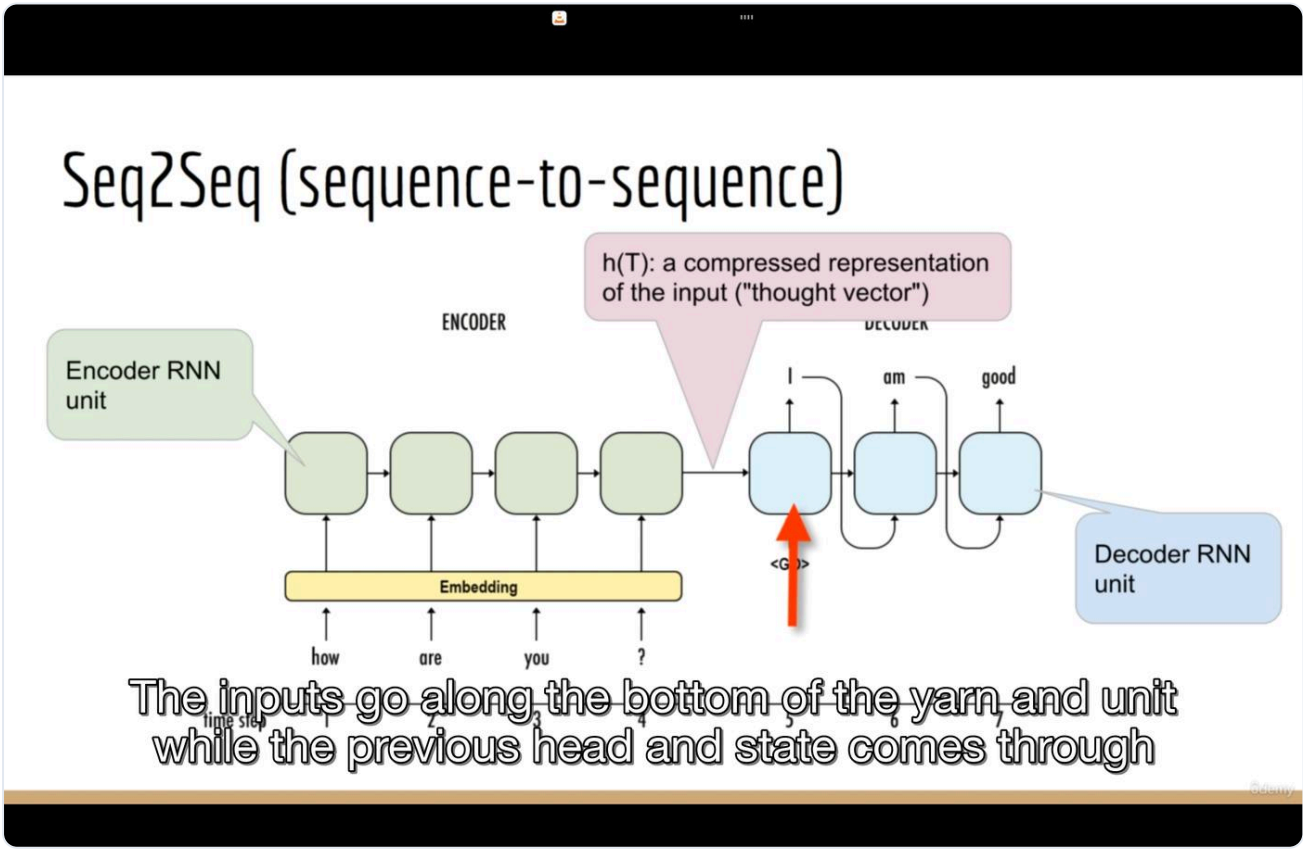

FIGURE 3.45

Now, for a normal RNN the initial hidden state is usually just a vector of zeros, or maybe some trainable but fixed vector.

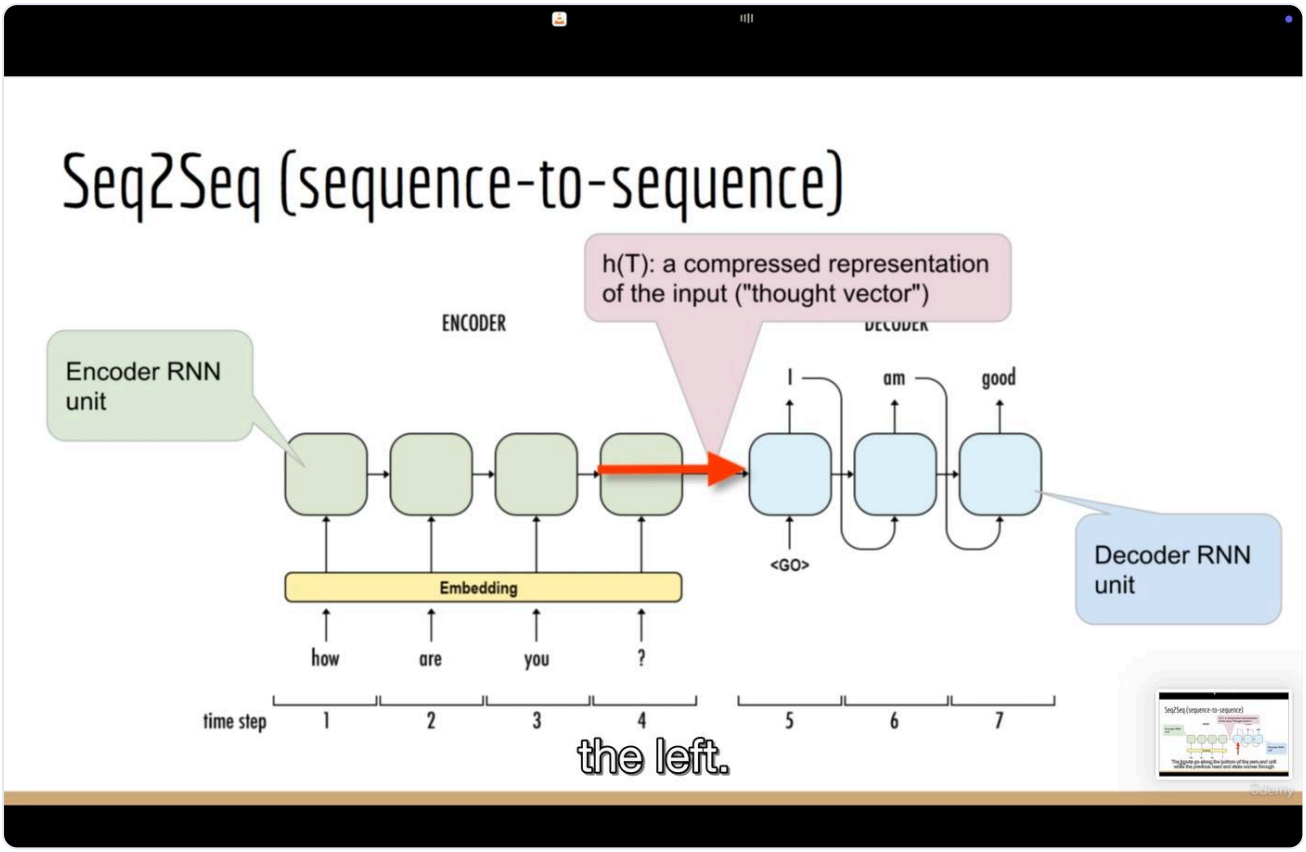

FIGURE 3.46

In this case, for the decoder, you can see that the initial hidden state actually comes from the encoders output.

Thus, the job of the decoder is to decompress the compressed representation of the input.

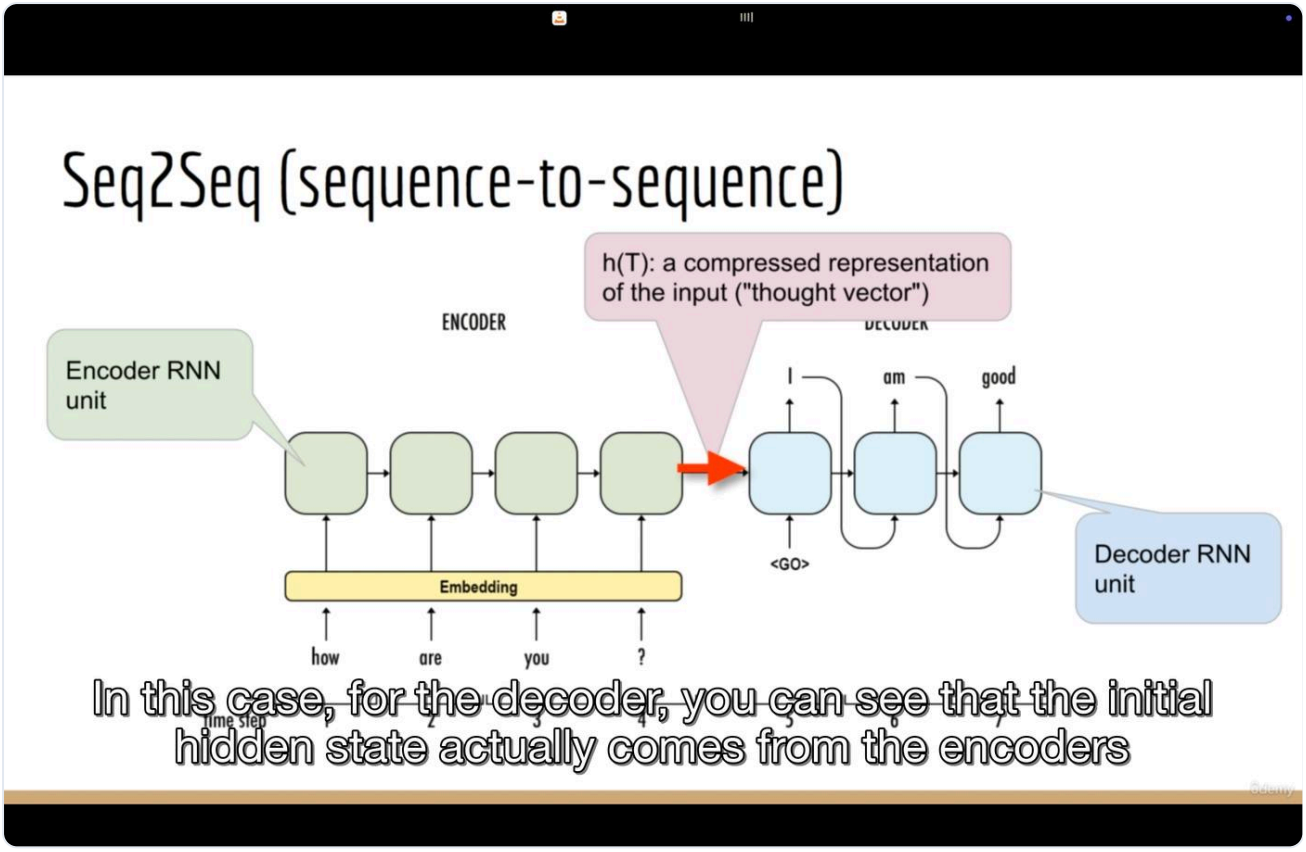

FIGURE 3.47

In the past, some people have called this hidden state vector a thought vector, since every sequence of words represents a thought and we're essentially converting that thought into a sequence of numerical components. So that's one way to think of the intermediate representation between the encoder and decoder.

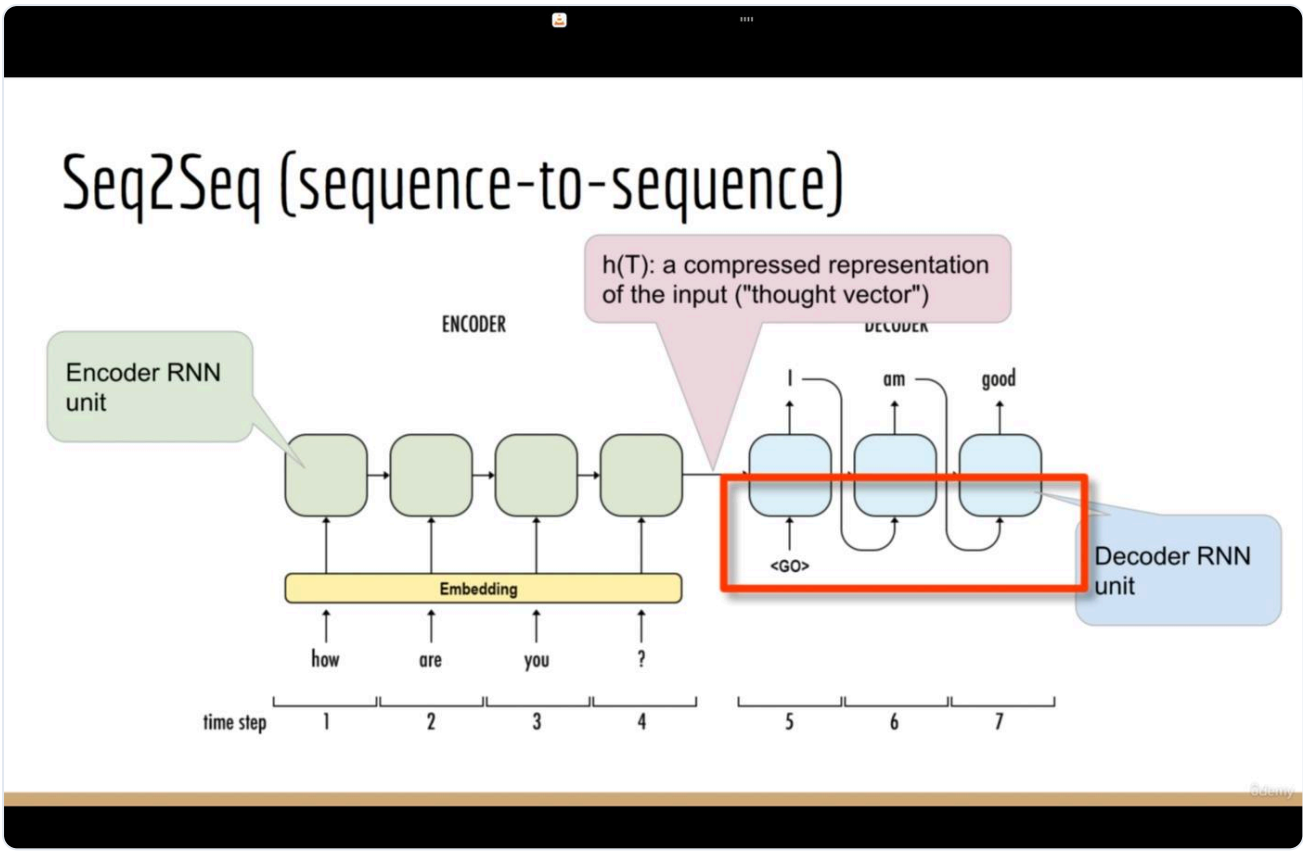

FIGURE 3.48

It's essentially a thought but represented in numerically.

Now the next question we have to answer is what goes into the bottom of the decoder RNN unit?

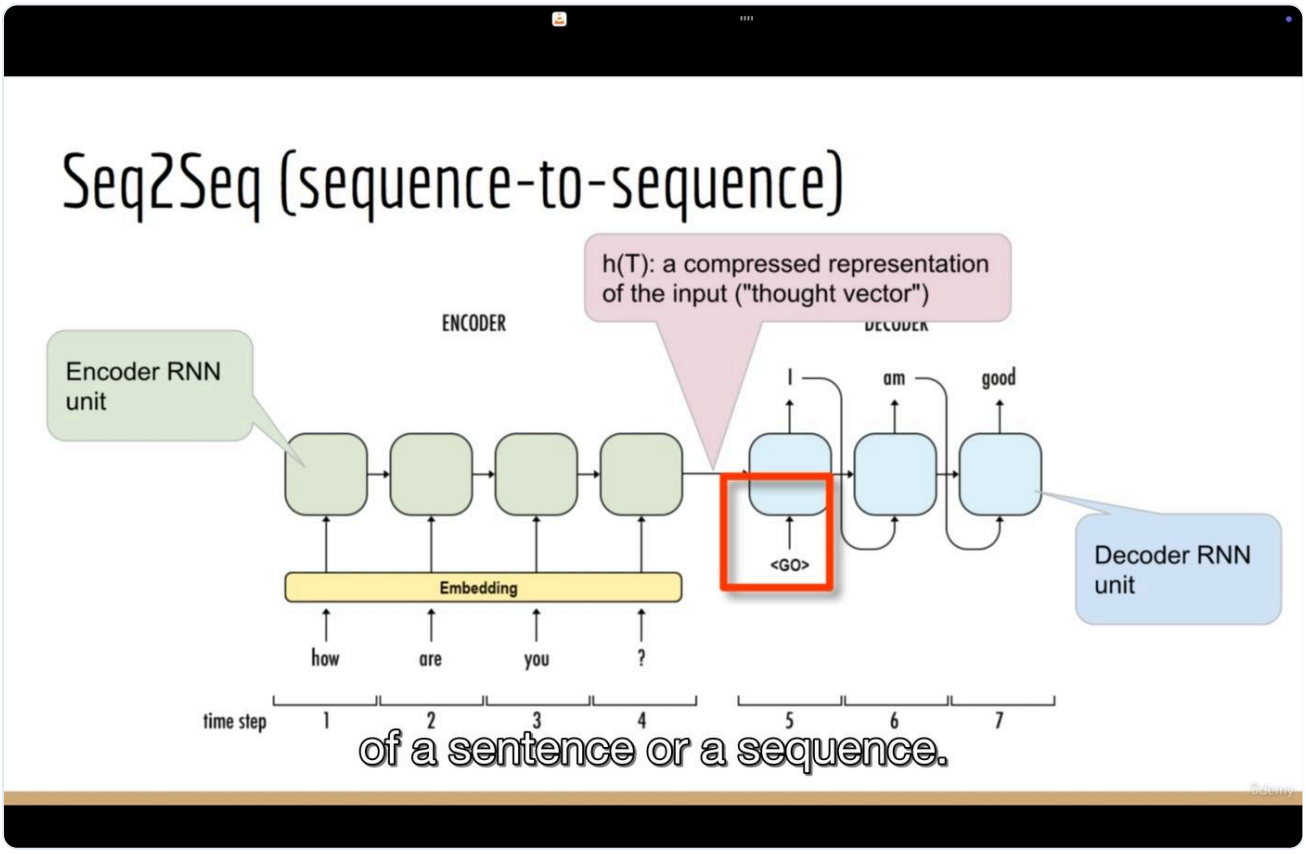

FIGURE 3.49

Initially, this is just a special token called a start token, which obviously signifies the start of a sentence or a sequence. From here, the decoder RNN is essentially a text generation network. As you recall, this is done in an auto regressive manner similar to how we forecast a time series.

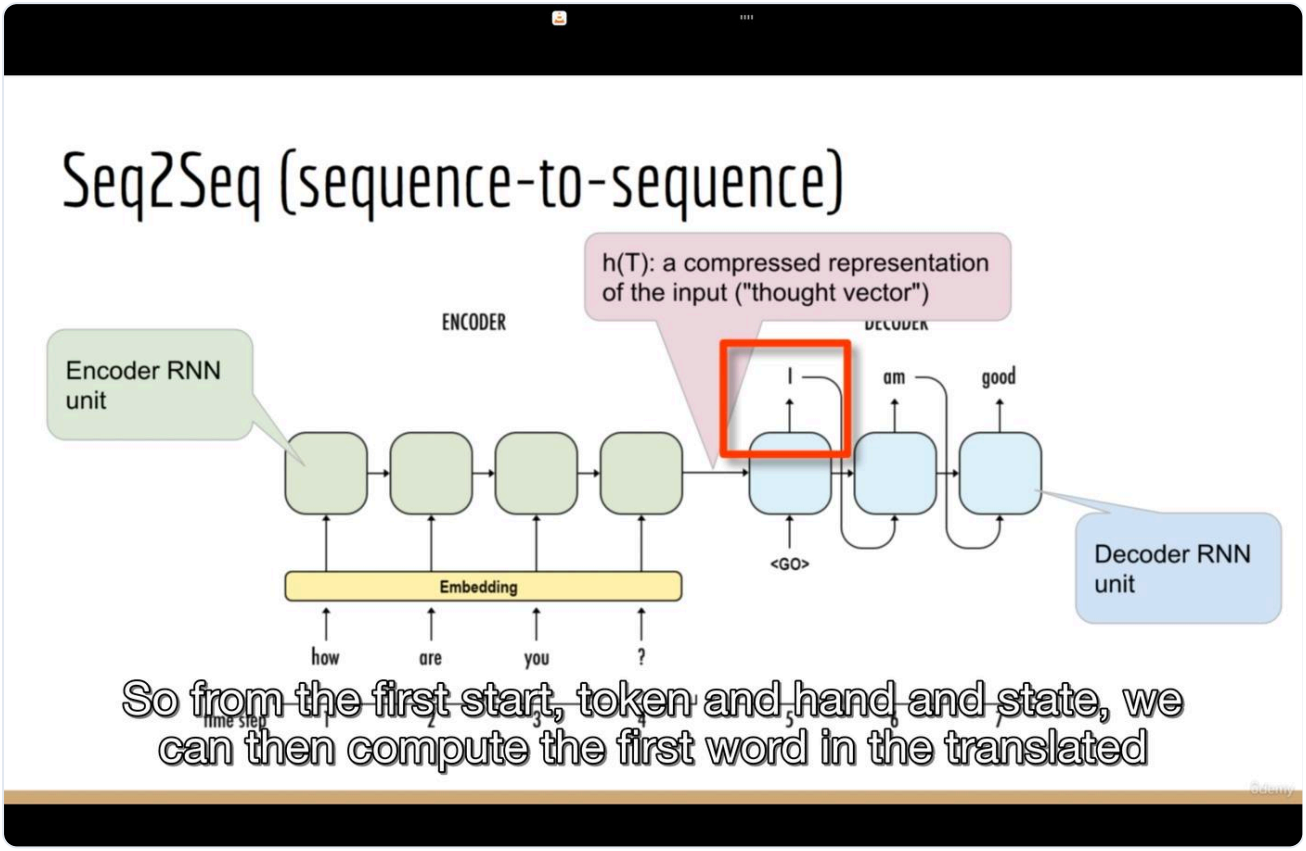

FIGURE 3.50

So from the first start token and hidden state, we can then compute the first word in the translated sequence.

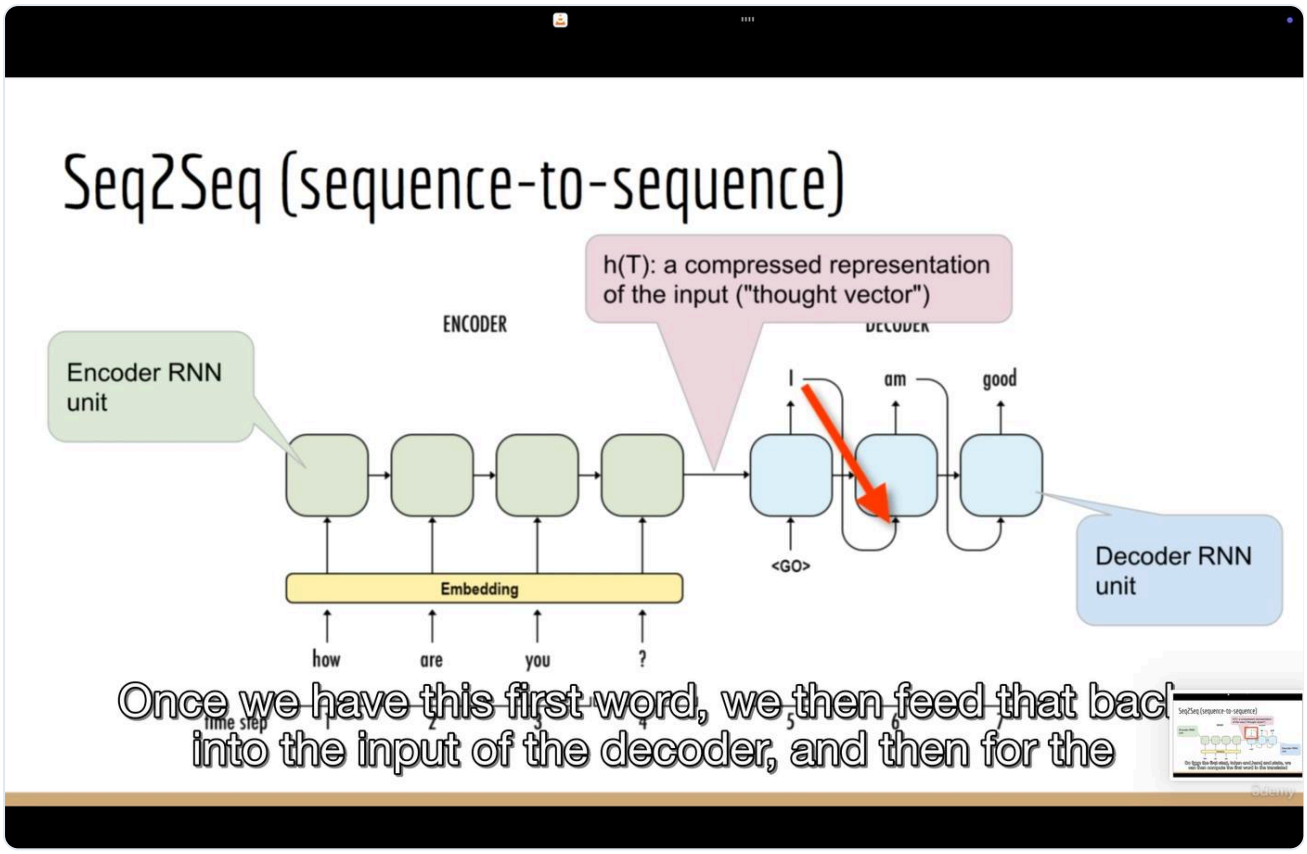

FIGURE 3.51

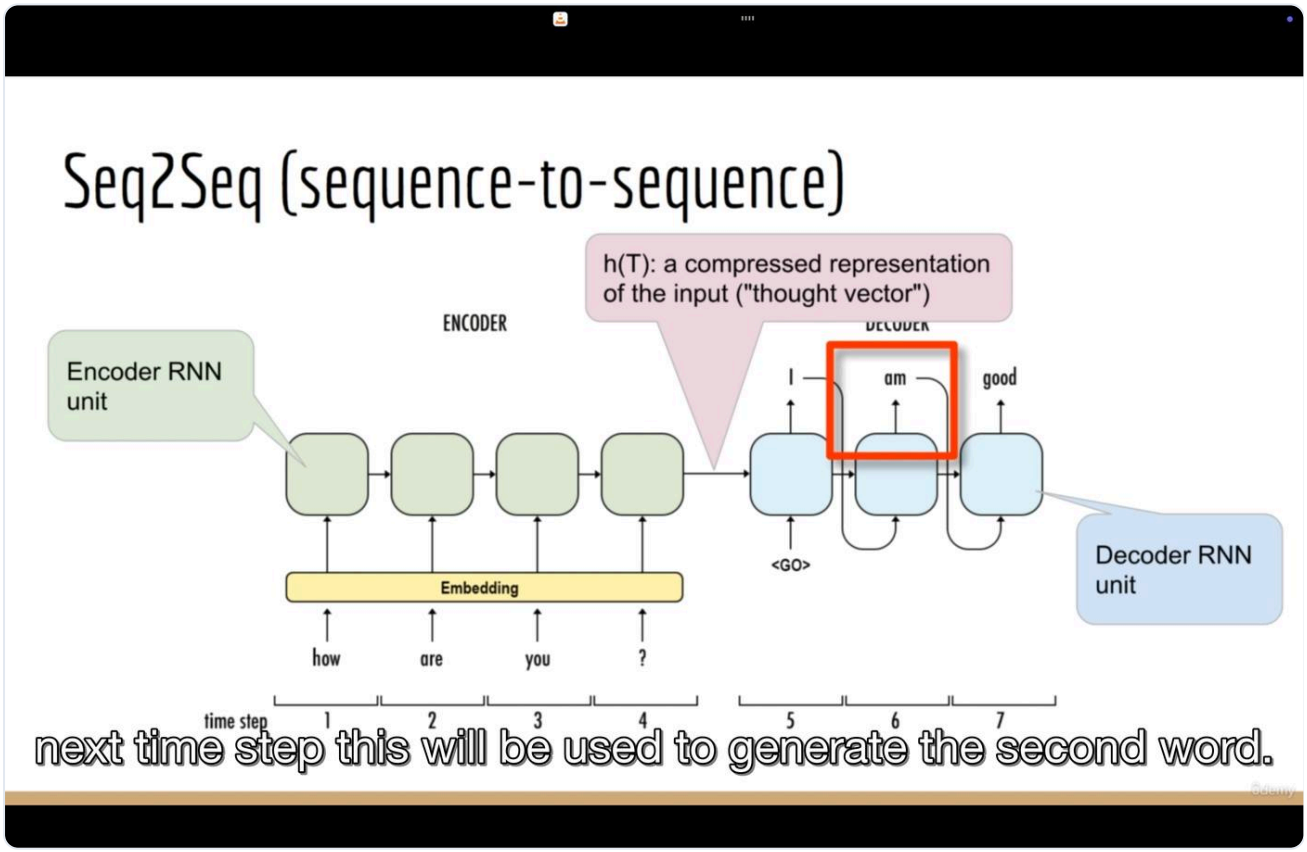

FIGURE 3.52

Once we have this first word, we then feed that back into the input of the decoder, and then for the next time step this will be used to generate the second word.

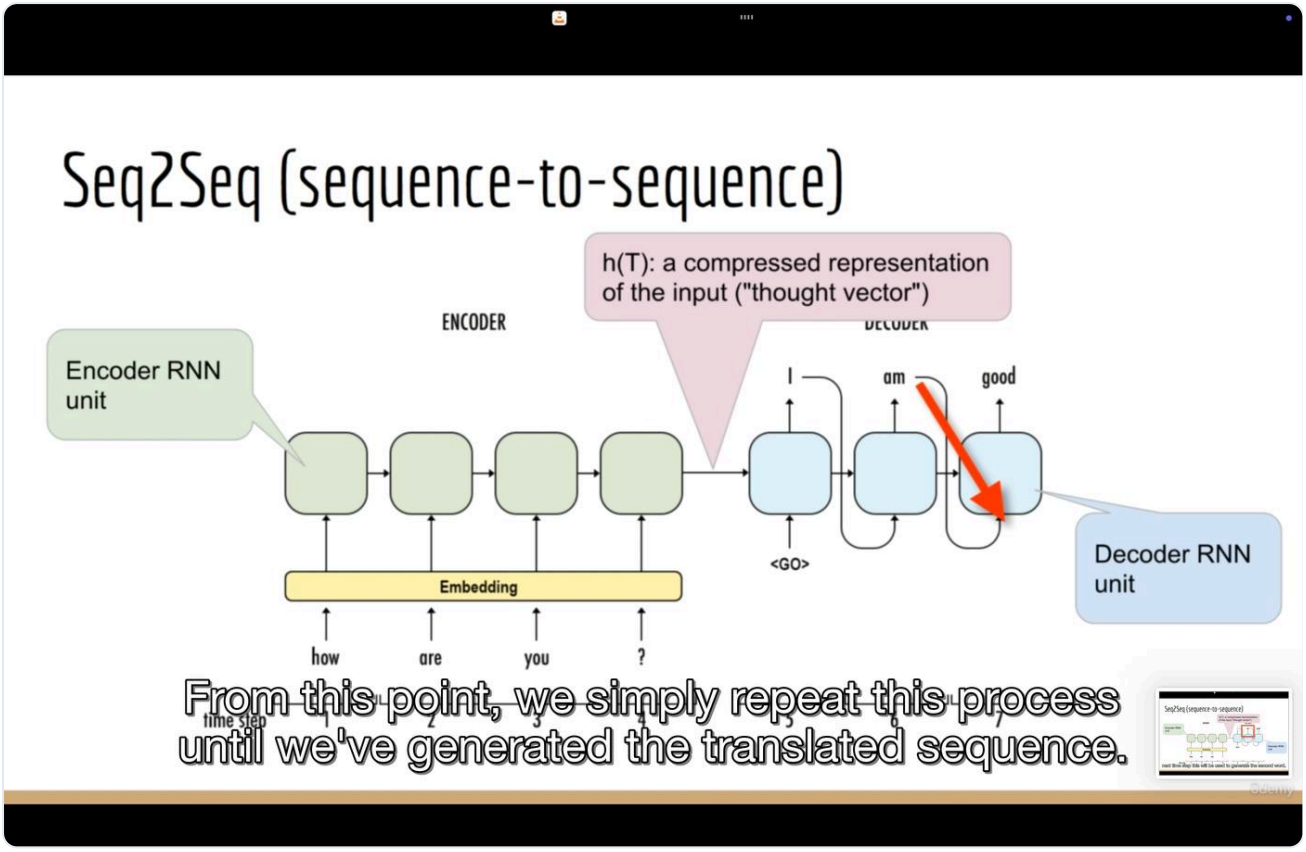

FIGURE 3.53

From this point, we simply repeat this process until we've generated the translated sequence.

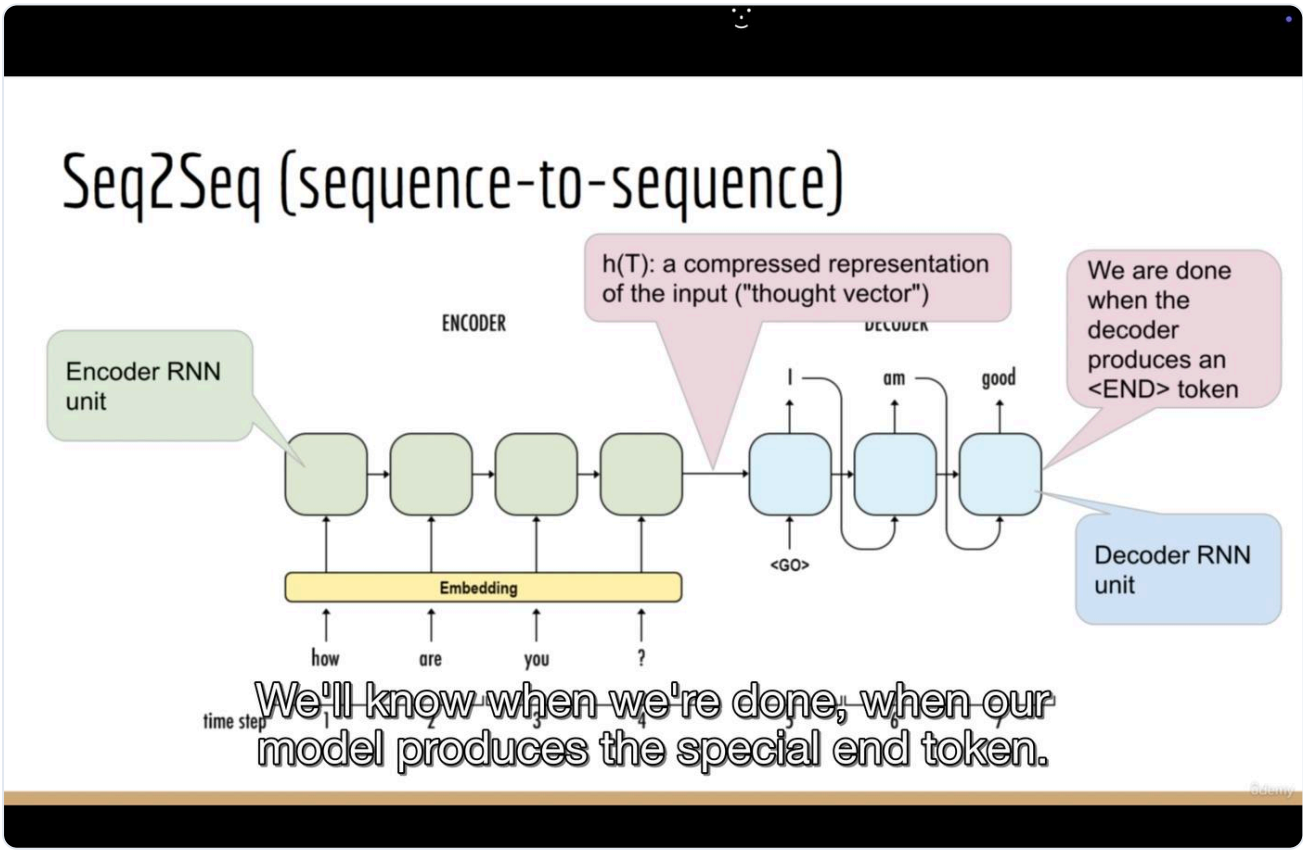

FIGURE 3.54 | We Are Done

We'll know when we're done, when our model produces the special end token. So that's how we know when our neural network should stop producing more outputs. So seq2seq turned out to work pretty well. But there was one small issue.

As you recall, every input sequence is converted using an encoder RNN into a single vector.

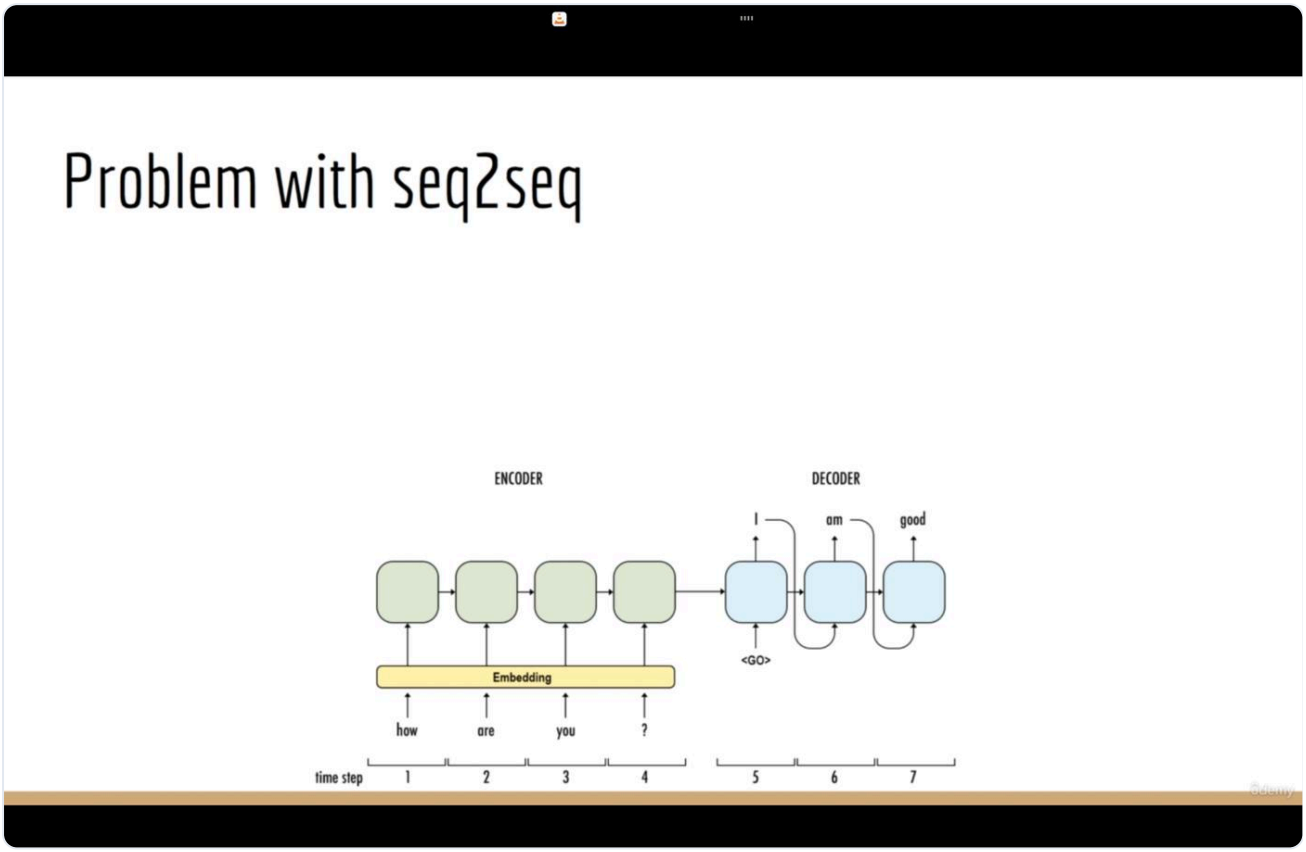

FIGURE 3.55

Here's the problem. What if your input sentence is very long? No matter the length of the input, the final thought vector still has the same size. This is not really very intuitive. We know that longer sequences naturally contain more information.

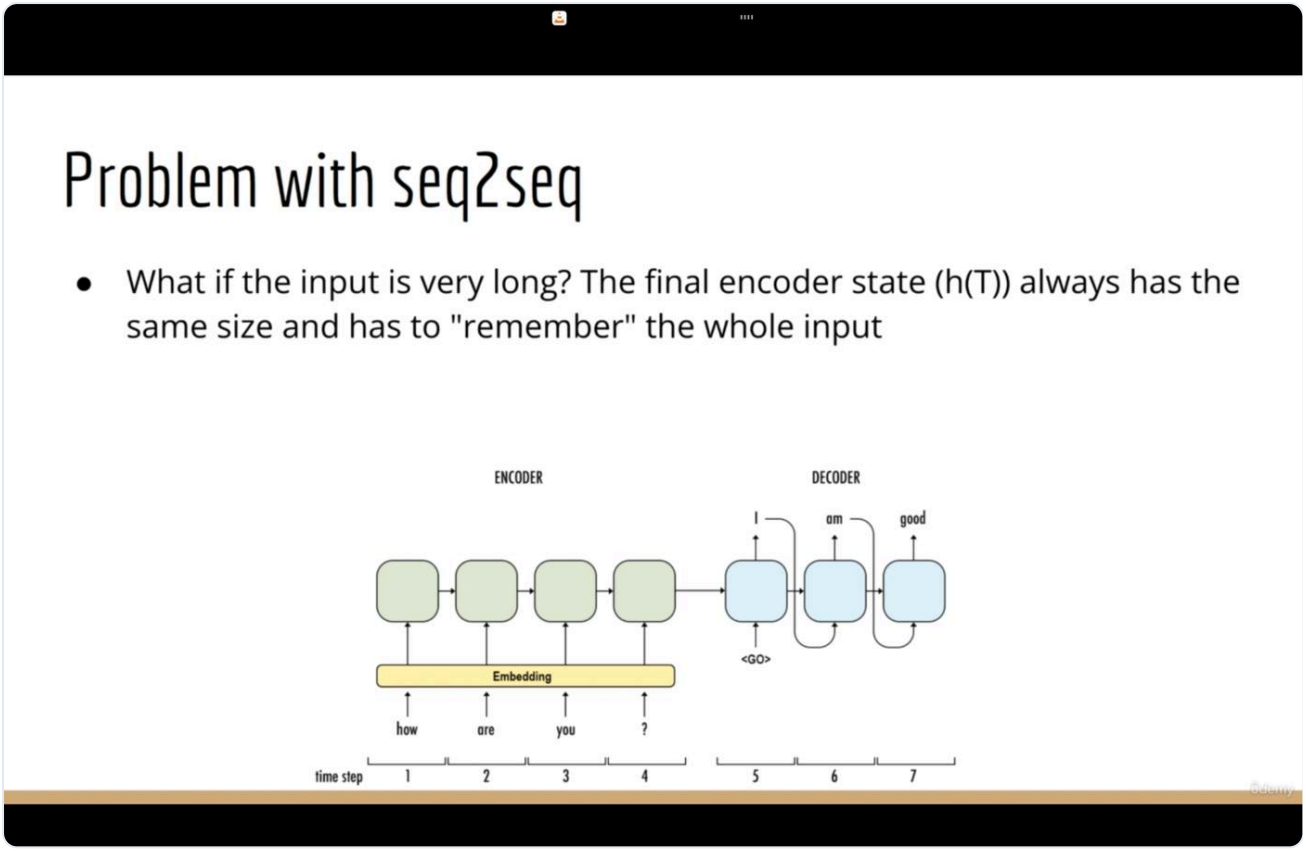

FIGURE 3.56

In addition, we know that RNNs are not good at remembering things in very long sequences. Furthermore, this is not really how we as humans translate sequences.

We don't memorize a whole passage and then try to generate the translation one word at a time from that memorized concept. Instead, we're more likely to read a few words at a time and produce the translation while focusing on localized areas of the input. Sometimes we might have to go a few words forward or a few words back, but the key is there. We might want to scan the input multiple times to figure out which parts to pay attention to and translate next.

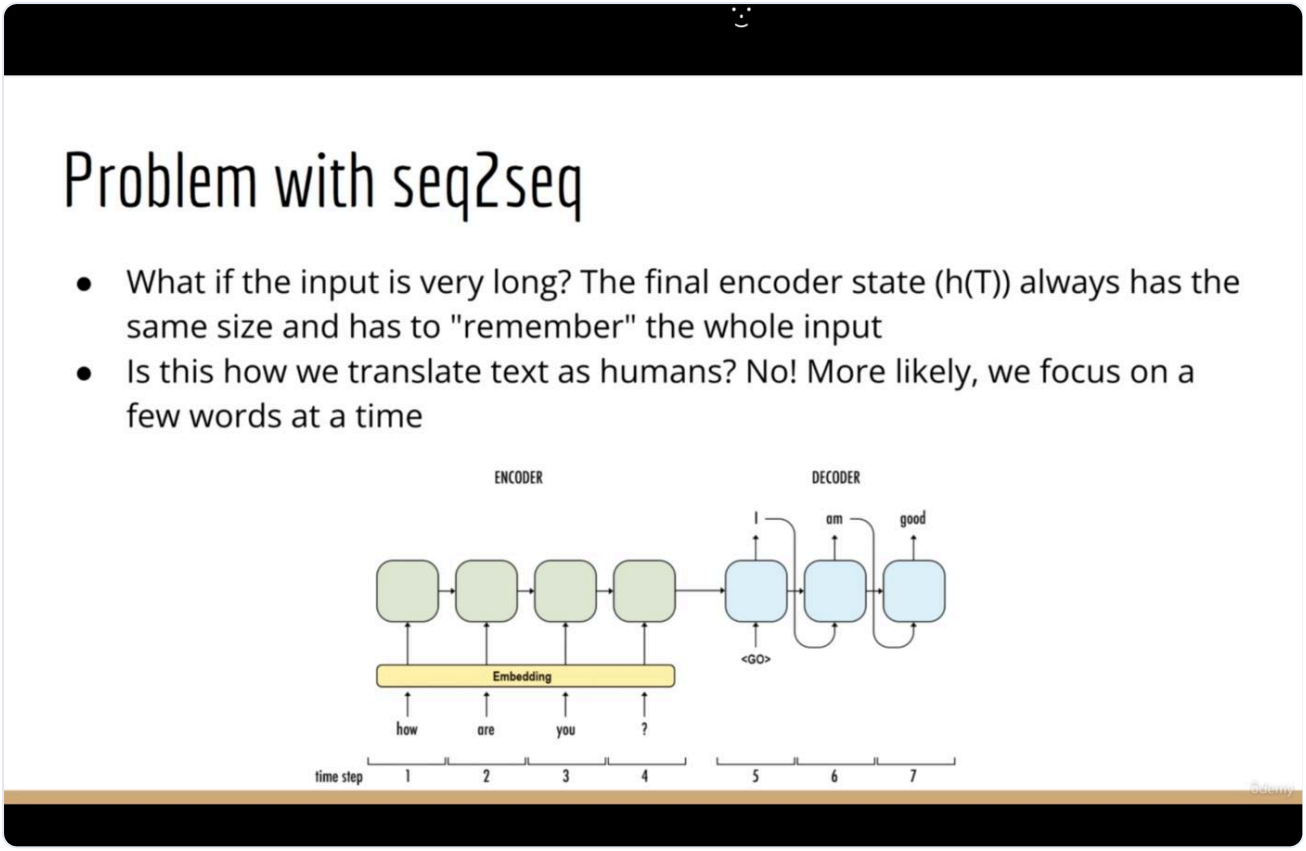

FIGURE 3.57

So this is the main concept behind attention, which was originally developed for Encoder Decoder RNNs. Now the details get pretty complex and the diagrams get pretty messy. So for this lecture we'll be focusing mainly on intuition. In addition, note that if your goal is to learn about Transformers, the way attention works there is a bit different from how it works for seq2seq. So there isn't really a good reason at this point to go into detail.

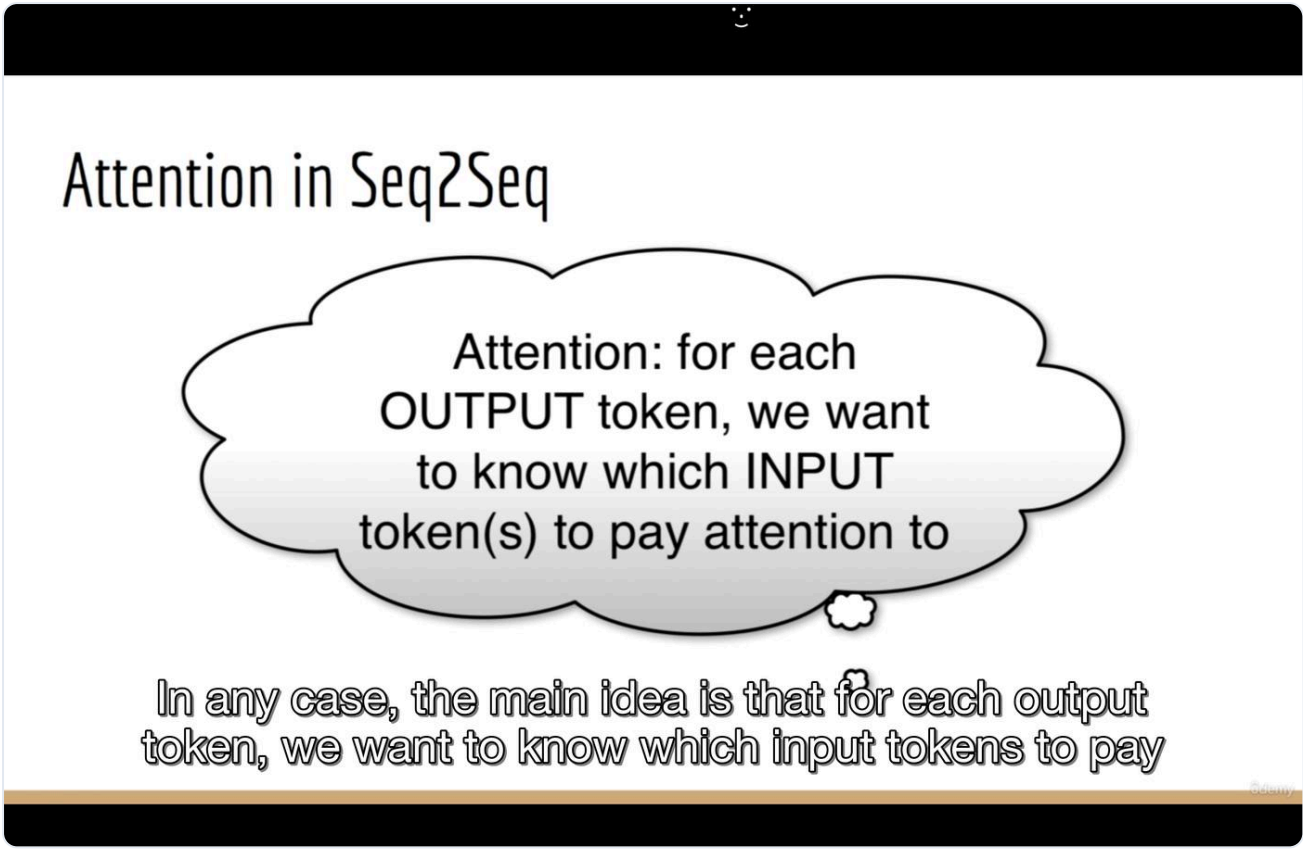

FIGURE 3.58 | Attention

In any case, the main idea is that for each output token, we want to know which input tokens to pay attention to.

So why don't we discuss attention?

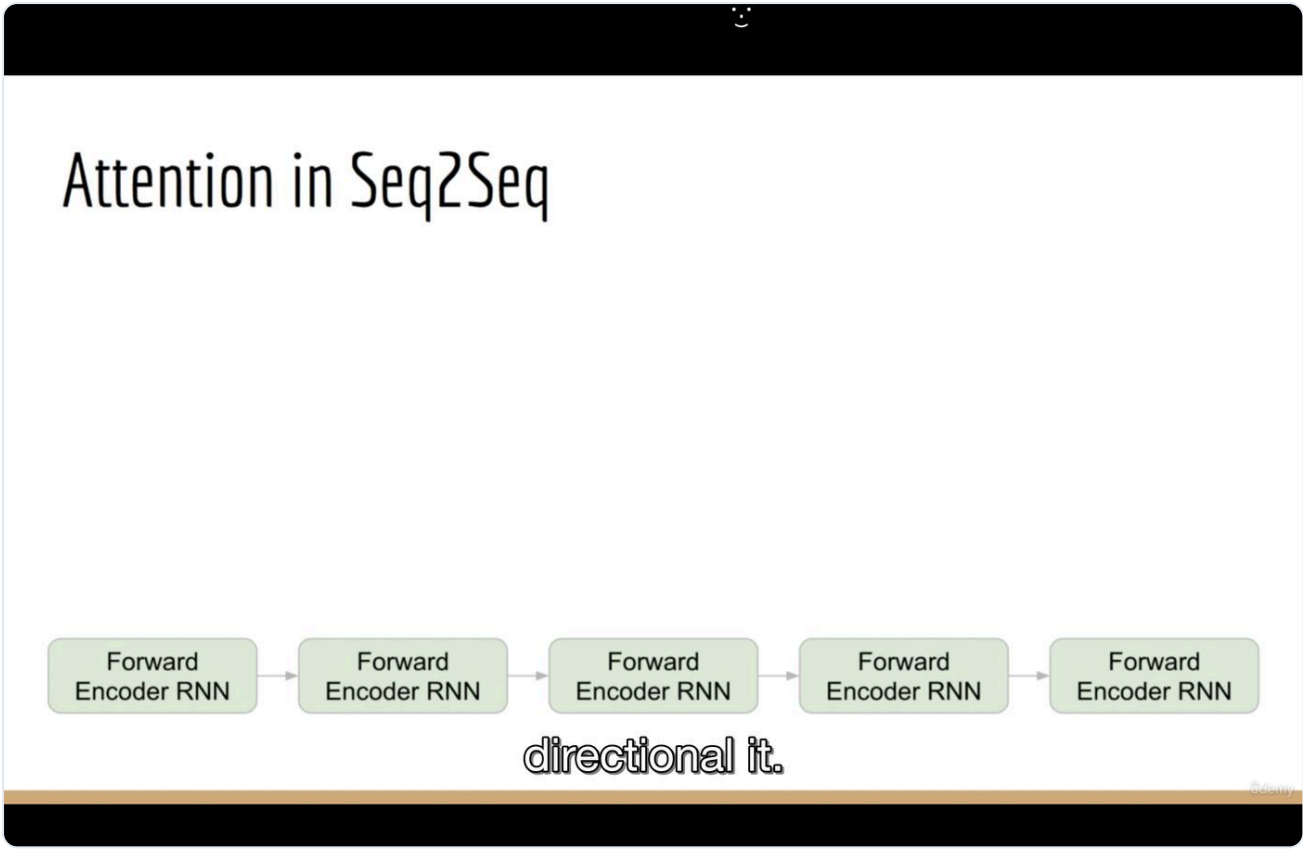

FIGURE 3.59

It's helpful to draw a seq2seq diagram a bit differently in particular. Instead of drawing the encoder and decoder side by side, we draw the decoder on the top row while the encoder goes in the bottom row. One important detail is that for the encoder, instead of just using a regular RNN we use a bidirectional one.

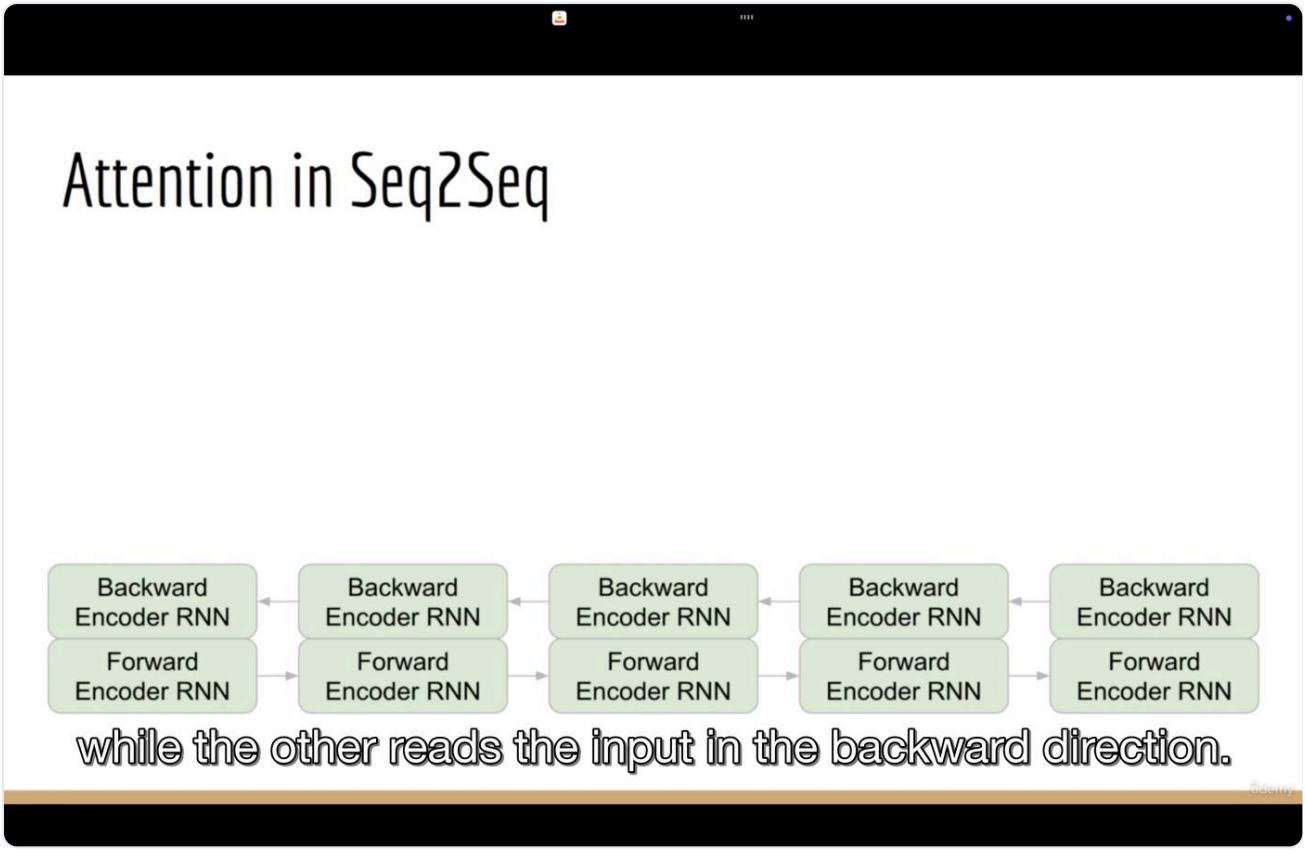

FIGURE 3.60 | Encoder RNN Encoder RNN Encoder RNN

Basically, you can think of this like two stacked RNNs where one reads the input in the forward direction while the other reads the input in the backward direction.

In this way, every hidden state will now take into account both the context of the previous words and the context of the following words. For our diagram, this ends up looking pretty much the same.

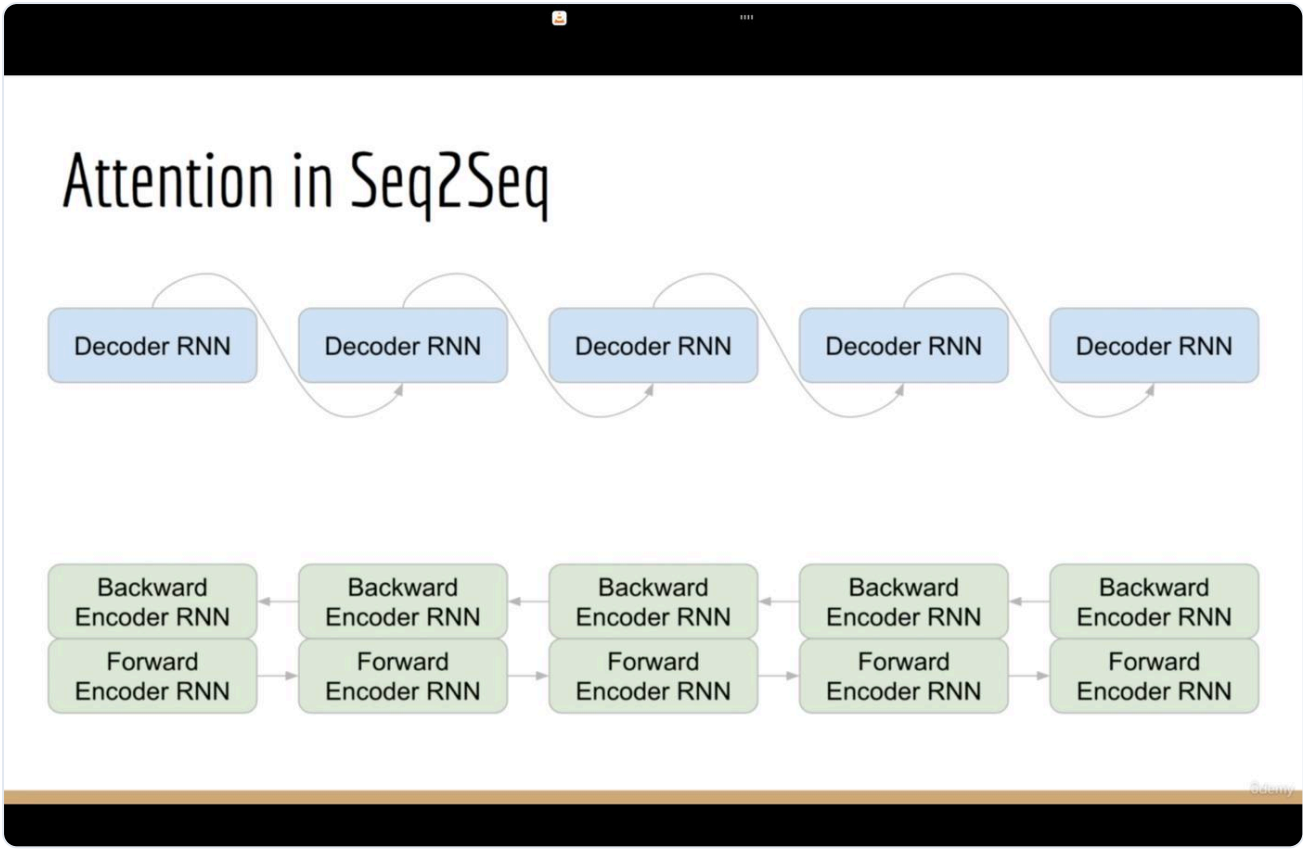

FIGURE 3.61 | Attention in Seq2Seq

For the decoder. We still generate the output one word at a time.

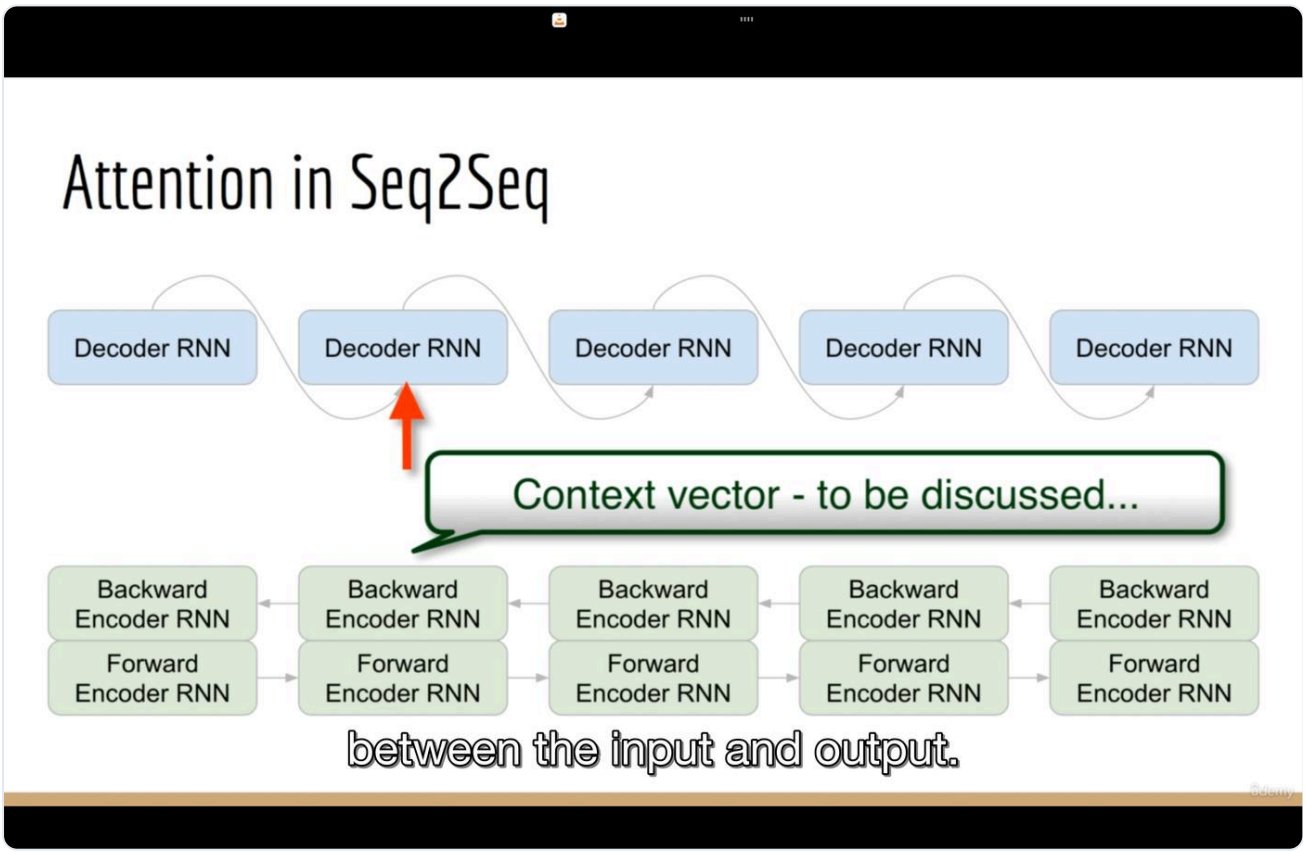

FIGURE 3.62

Using the password will be generated as an input to the next time step. So this is not a bi directional RNN. However, the important part is we now also take in a context vector as input as well to help us differentiate between the input and output.

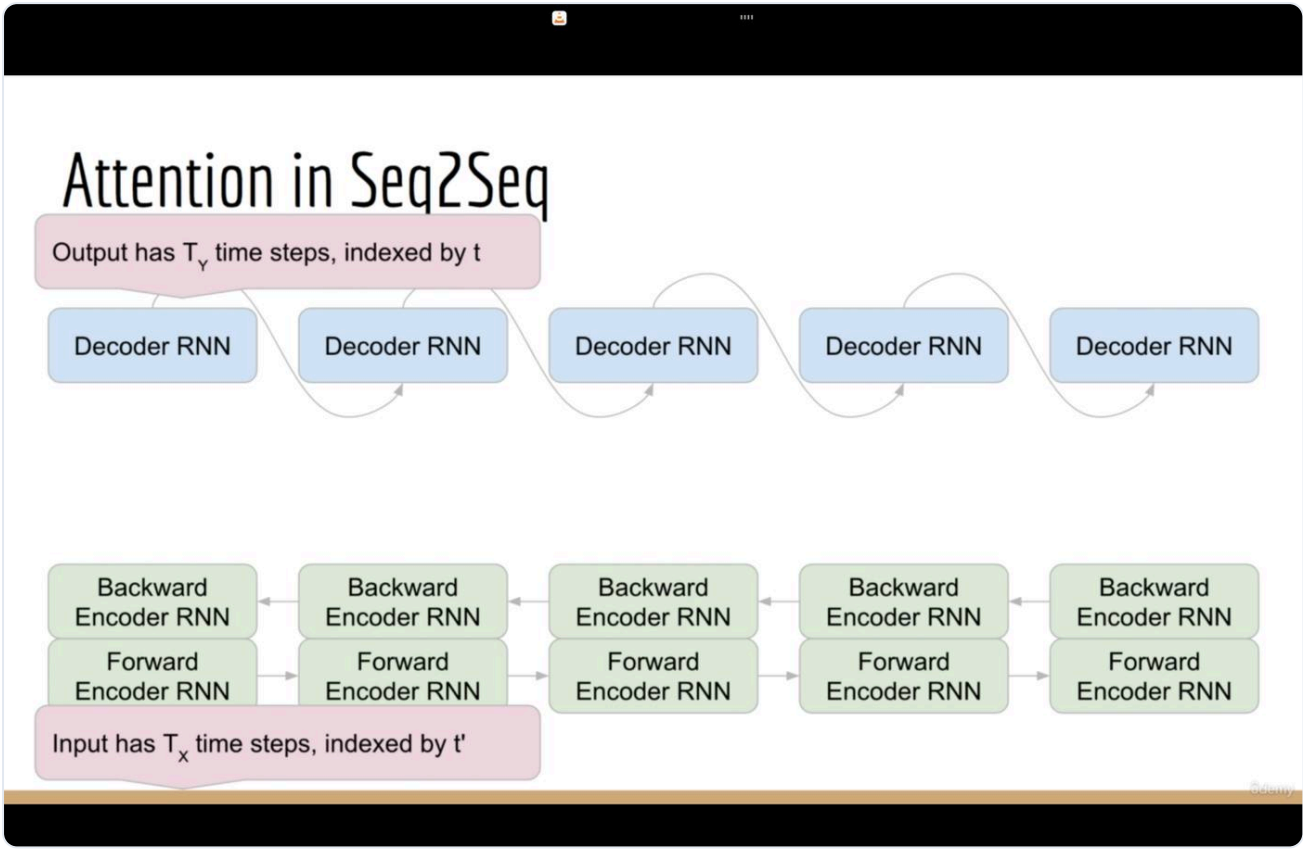

FIGURE 3.63 | Output Has T, Time Steps, Indexed by T

We'll say that the input has T x time steps, while the output has T y time steps. We'll index the input time step using the letter t prime and we'll index the output time step using the letter t.

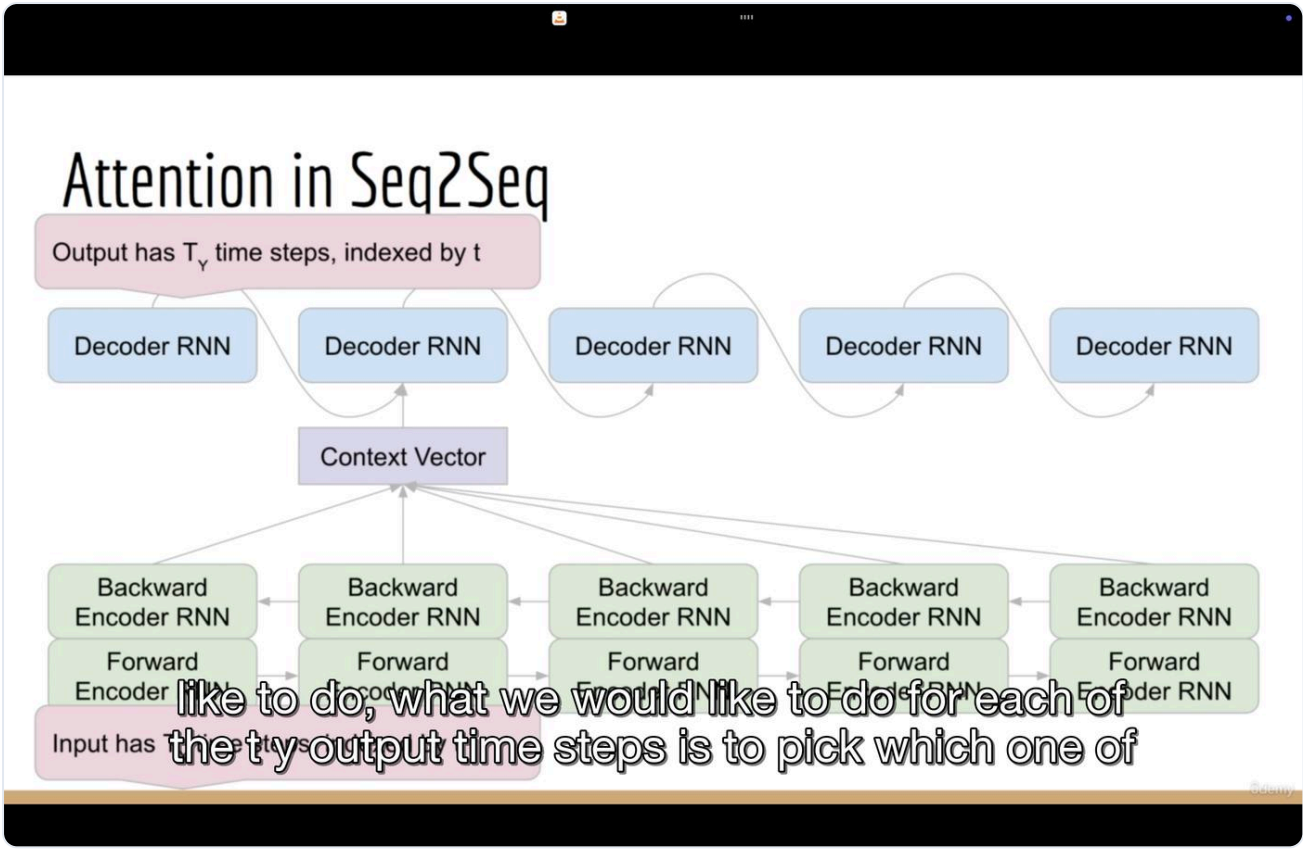

FIGURE 3.64 | Output Has T, Time Steps, Indexed by T

Now, before we get into the details behind attention, let's discuss the intuition for what we would like to do, what we would like to do for each of the t y output time steps is to pick which one of the t x input time steps to pay attention to.

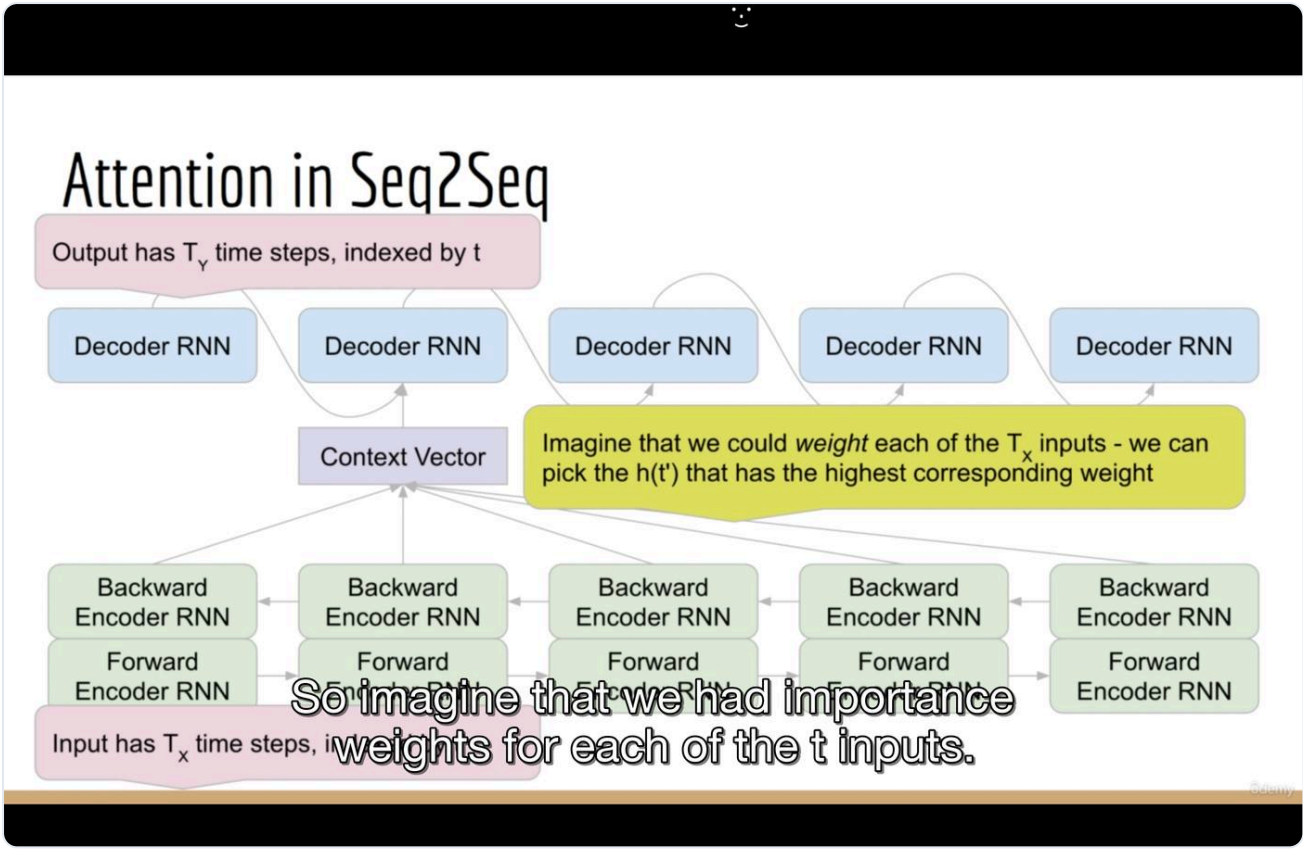

FIGURE 3.65 | Attention in Seq2Seq

So imagine that we had importance weights for each of the t inputs.

Our goal is to pick the H of T prime that has the corresponding maximum importance. In this simple scheme, we simply set the context vector to be this h of t prime and it goes into the decoder as a regular input.

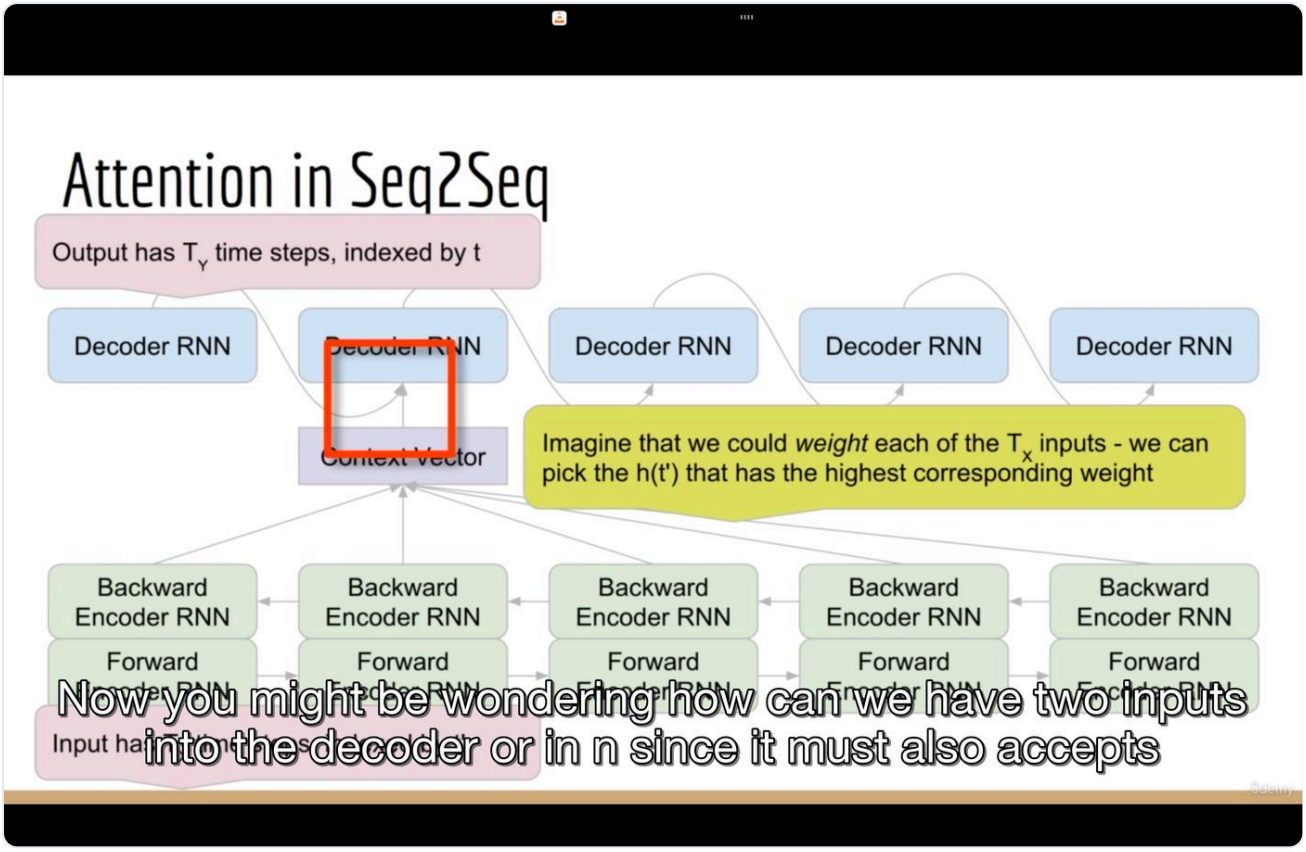

FIGURE 3.66 | Output Has T, Time Steps, Indexed by T

Now you might be wondering how can we have two inputs into the decoder RNN, since it must also accepts the previously generated word. The answer, to put it simply, is to just concatenate it with the context so that we get a single vector.

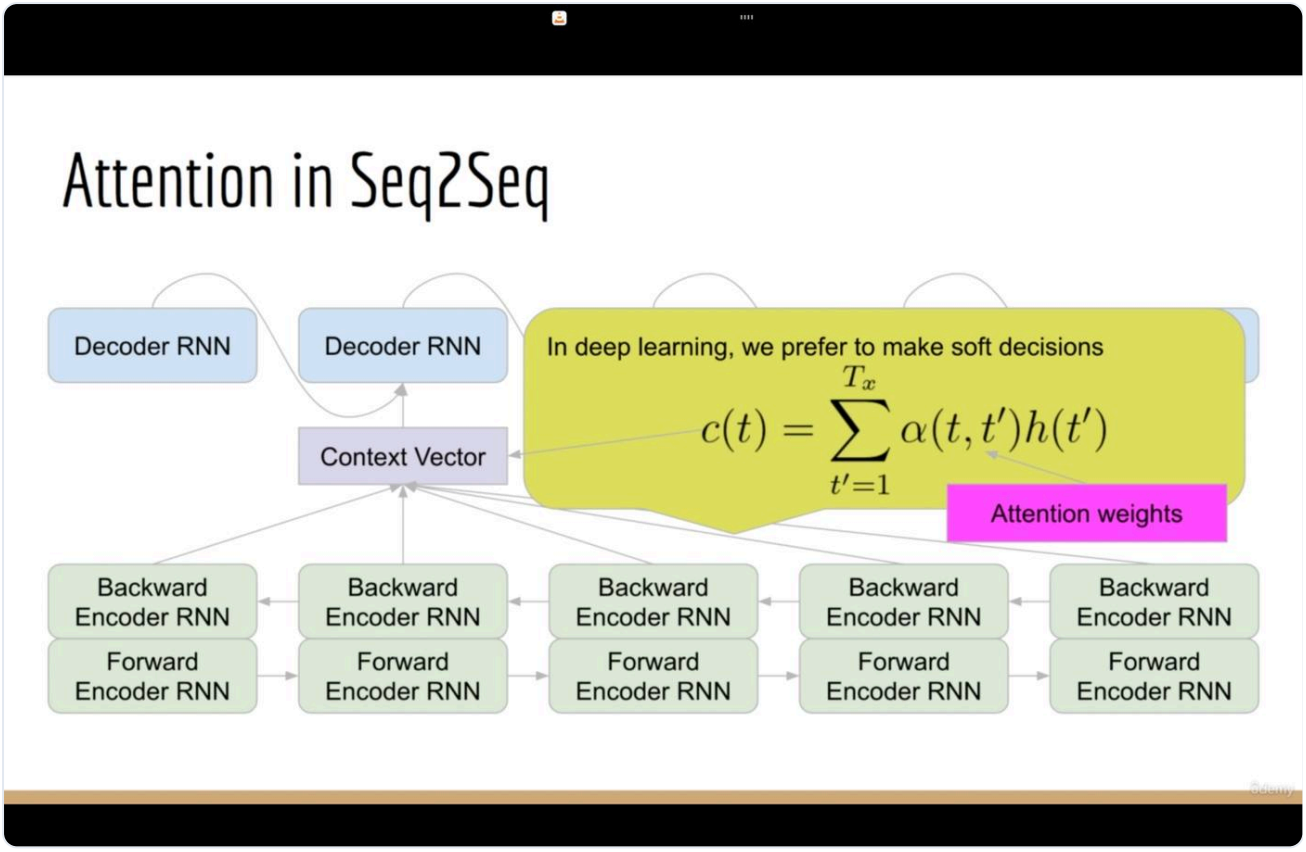

FIGURE 3.67

Now in deep learning, we know that instead of making hard decisions like picking the hidden state with maximum importance, we usually make soft decisions instead. In other words, instead of picking just a single hidden state, h of t prime, we'll take a weighted sum of all hidden states from h of one up to h of T x. The key is every hidden state from the encoder will influence every input to the decoder. Through weights we call attention weights, we'll call the attention weights

alpha of t and t prime, which is a number that represents how much the output at time step t should pay attention to the input at time step t prime.

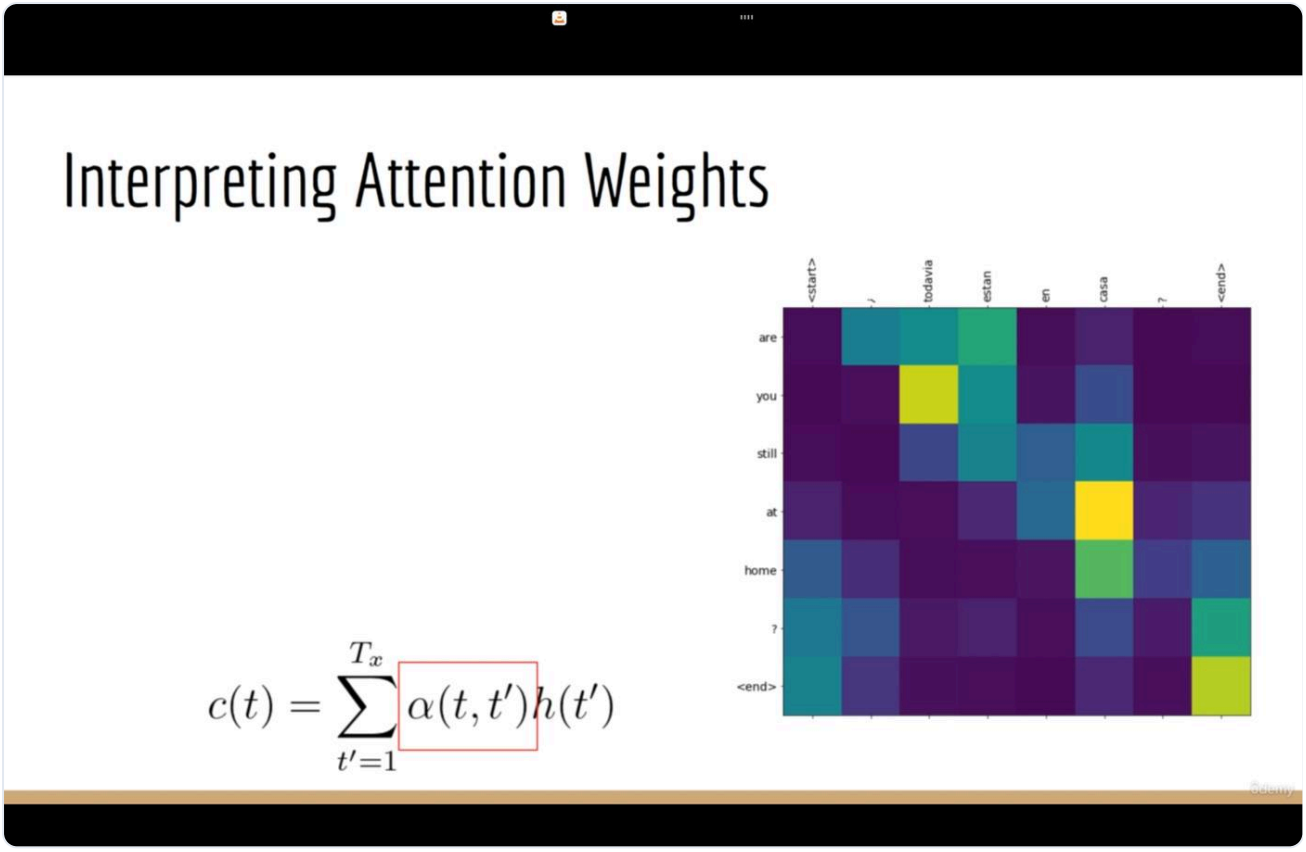

FIGURE 3.68

In this case, the context vector is simply the weighted sum of ages where the weights are the alphas. So this is the basic idea behind attention. We have weights from every input to every output, which tells us how much each output should pay attention to each input. So it's worth thinking about how these attention weights can be interpreted.

Firstly, let's consider how many such weights we should have.

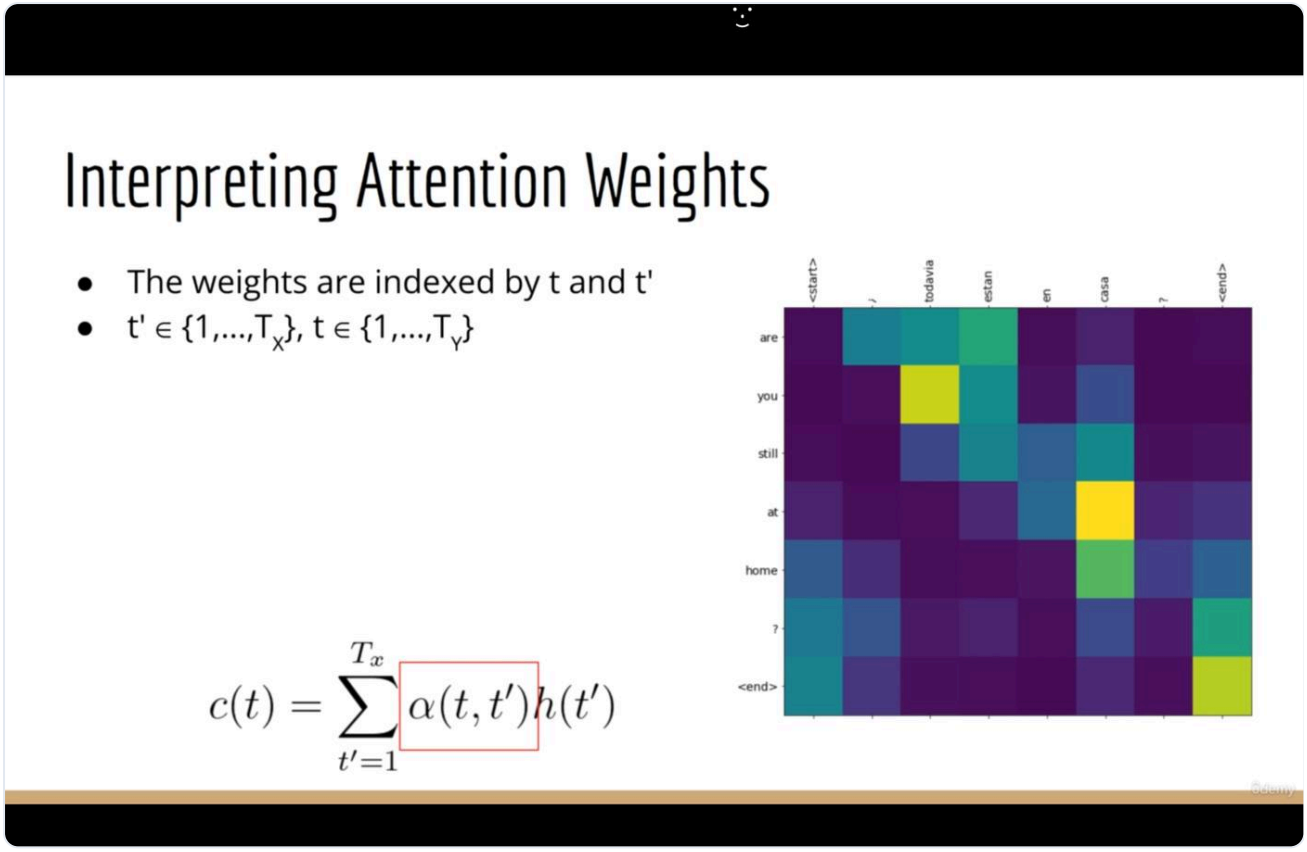

FIGURE 3.69

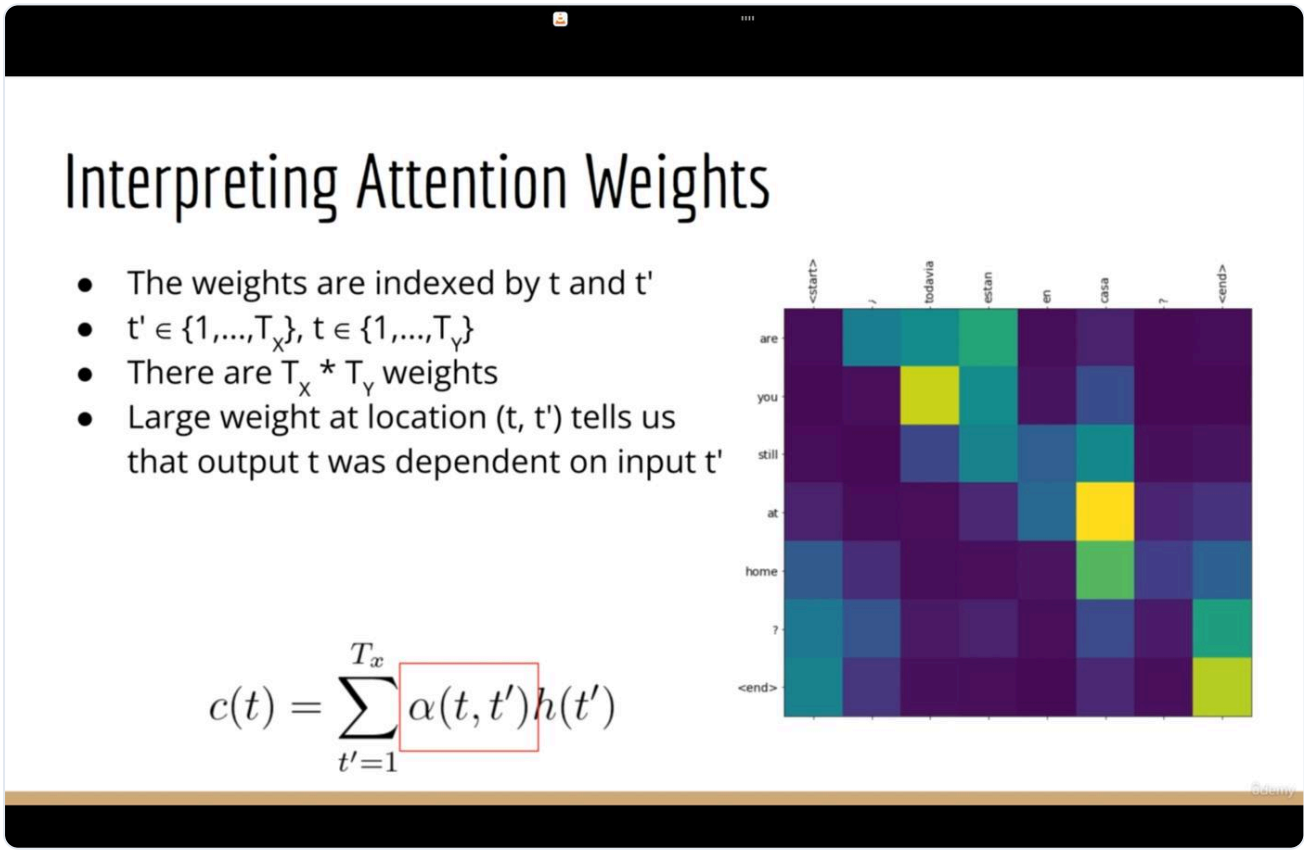

FIGURE 3.70 | The Weights Are Indexed by T and T

As you can see, alpha is indexed by t and t prime. t goes from one up to T y, while t prime goes from one up to T x. Therefore we have T y times T x attention weights, one for every input and output pair. Note that it's common to plot the attention weights to help us interpret why our model might produce a certain output. In the case of translation, this allows us to directly see how each translated word was found, or in other words, which input it was paying attention to when it produced each output.

Now the story of attention doesn't end here. Some years after the introduction of attention for seq2seq, there was a paper titled Attention is All You Need, which introduced the Transformers. Essentially, this says forget about the RNN and only use attention. Hence, attention is all you need. You can see that by doing this, we still get the most important feature of our translation model, which is that every output knows which input to pay attention to.

So we can still do complex tasks like machine translation, but without any of the disadvantages of RNNs. Specifically, there are two main drawbacks to using RNNs. Firstly, RNNs are slow because every output must be computed sequentially from the previous output.

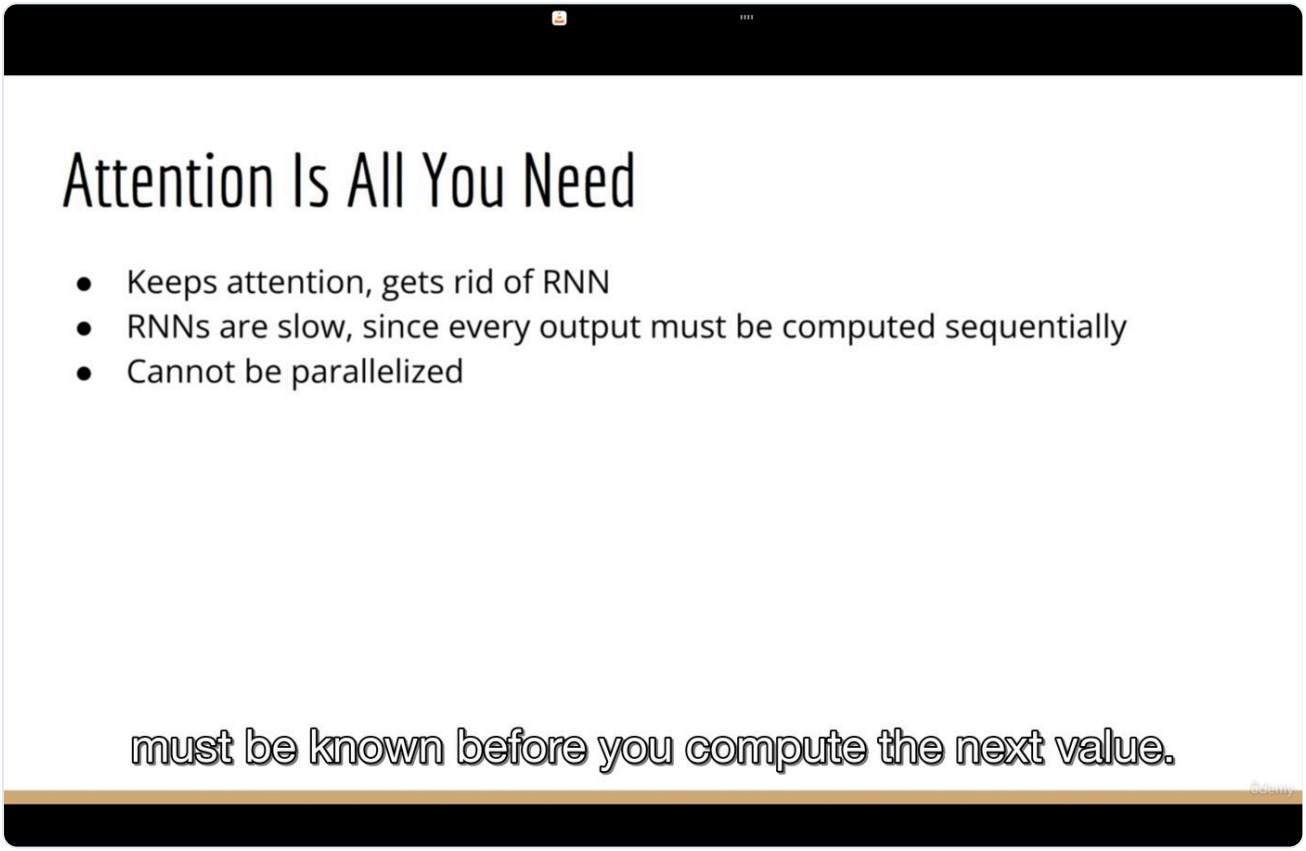

FIGURE 3.71 | Attention Is All You Need

There is simply no way around this and you cannot parallelize this operation since the previous value must be known before you compute the next value. The second drawback to RNNs is the concept of the vanishing gradient.

Modern RNN units like the LSTM and the GRU do an okay job of preventing this problem, but only to a certain point.

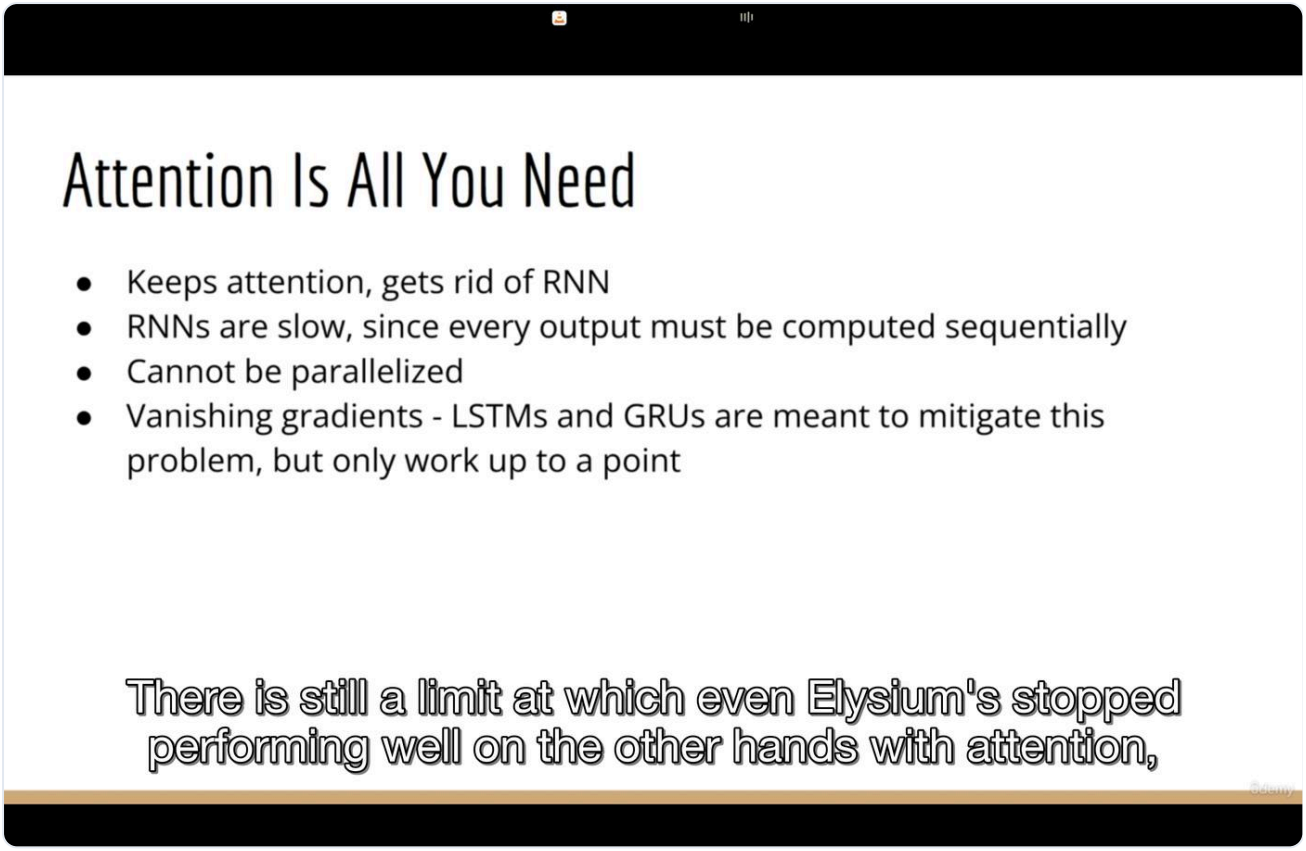

FIGURE 3.72 | Attention Is All You Need

There is still a limit at which even LSTMs stopped performing well On the other hand, with attention, even with very long sequences, There is a direct connection between every input and every output. Therefore, we don't have to worry about vanishing gradients.

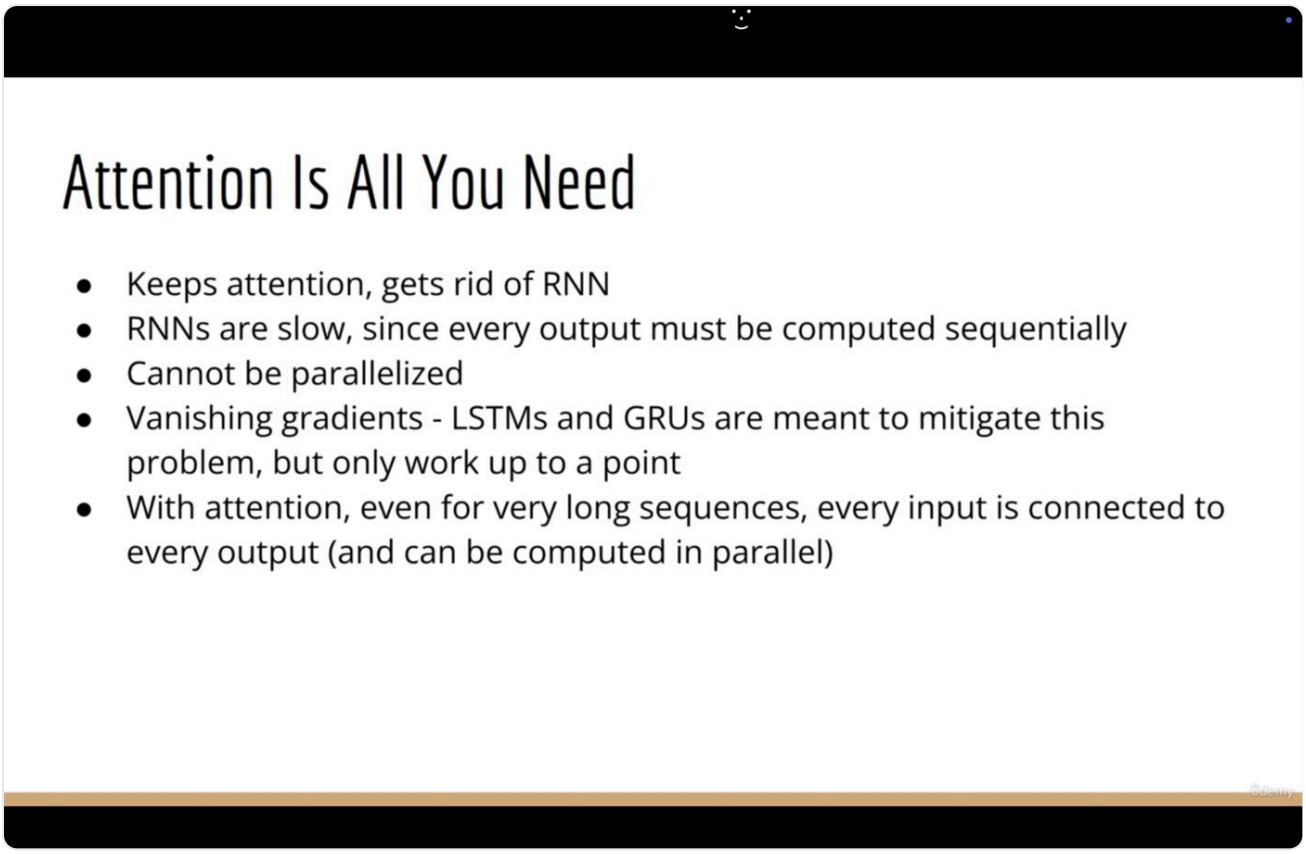

FIGURE 3.73 | Attention Is All You Need

Furthermore, since the computations don't have to be performed sequentially, there is an opportunity to speed things up by doing computations in parallel.

The only downside is that the attention mechanism is inherently quadratic. We say that it's O of n squared because if we have n inputs and outputs, we need to compute n squared attention weights.

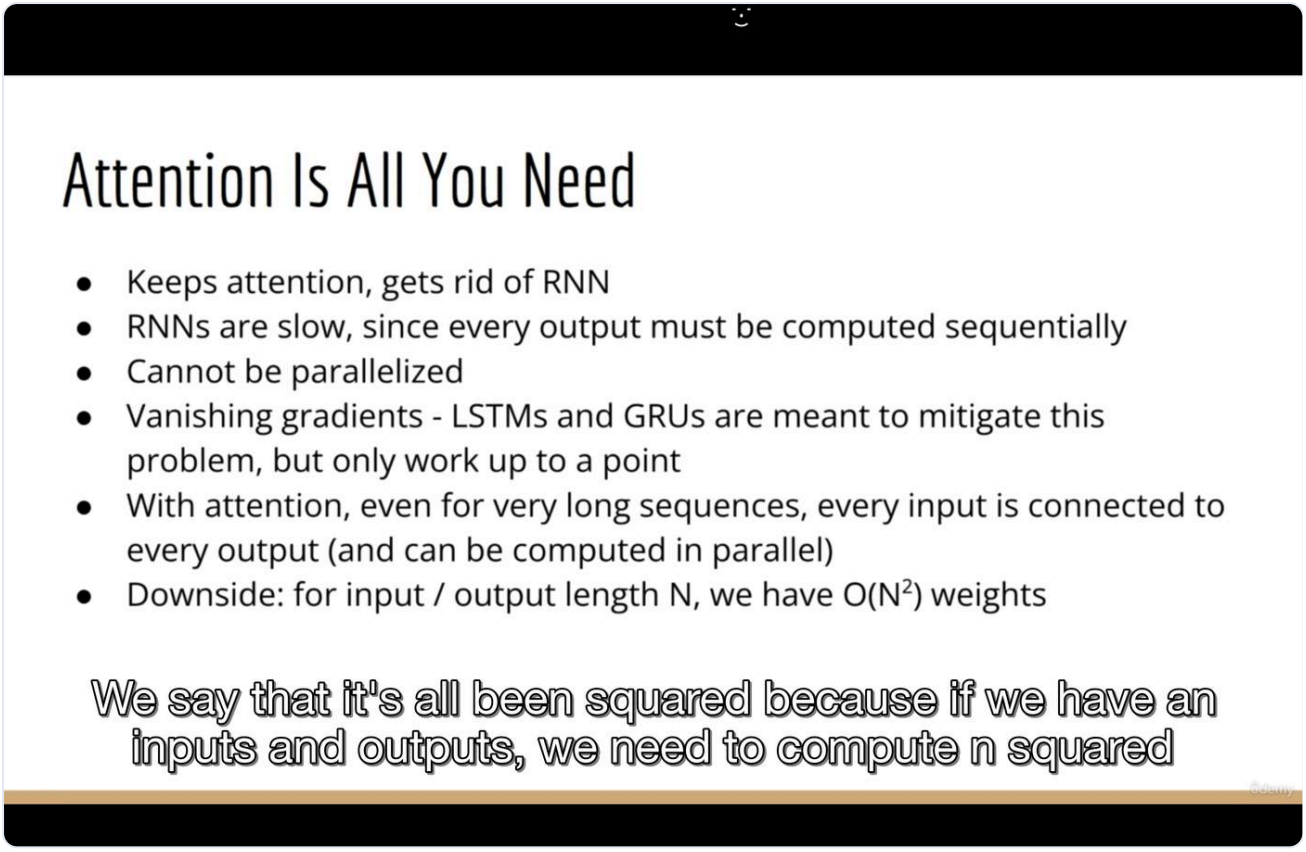

FIGURE 3.74 | Attention Is All You Need

So what is a transformer? At a very high level.

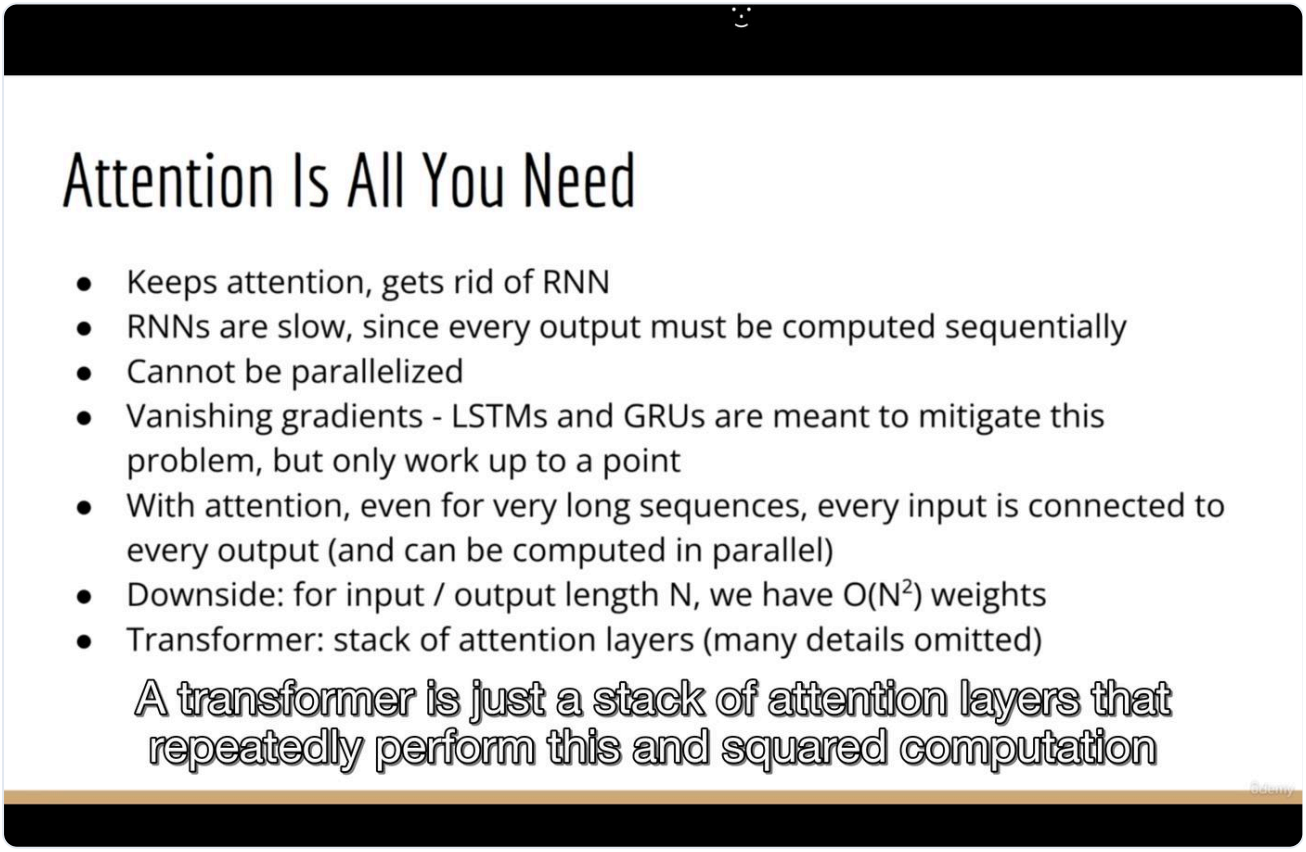

FIGURE 3.75 | Attention Is All You Need

A transformer is just a stack of attention layers that repeatedly perform this n squared computation to figure out which parts of the input to pay attention to. And this is the intuition behind Transformers.

3 · Sentiment Analysis

So in this lecture, we are going to look at our first task of this section, which is sentiment analysis. Now, at this point, I'm going to assume that you know what sentiment analysis is and why it's useful.

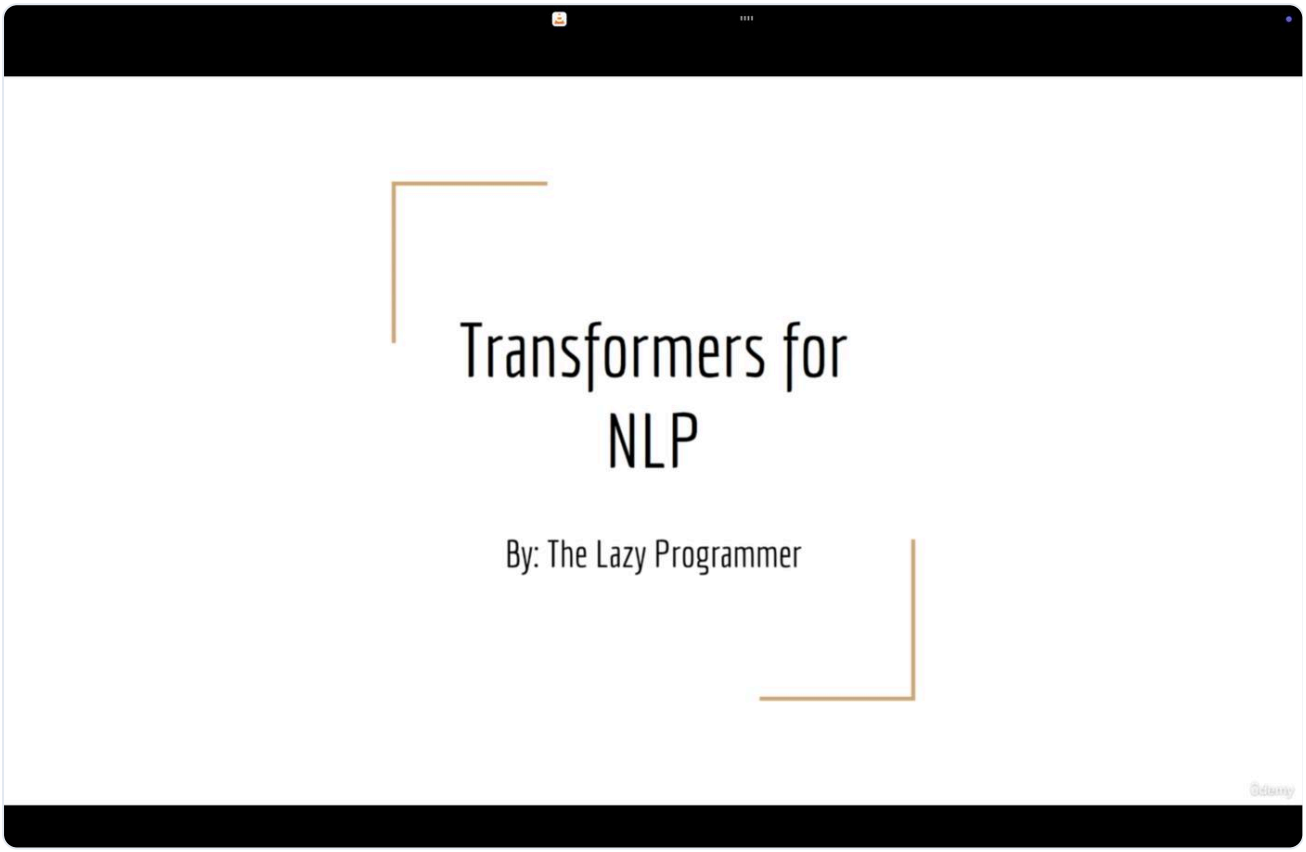

FIGURE 3.76 | Transformers for

So we're only going to review the main points. The main focus of this lecture, or the thing which will likely be new for you, is the code.

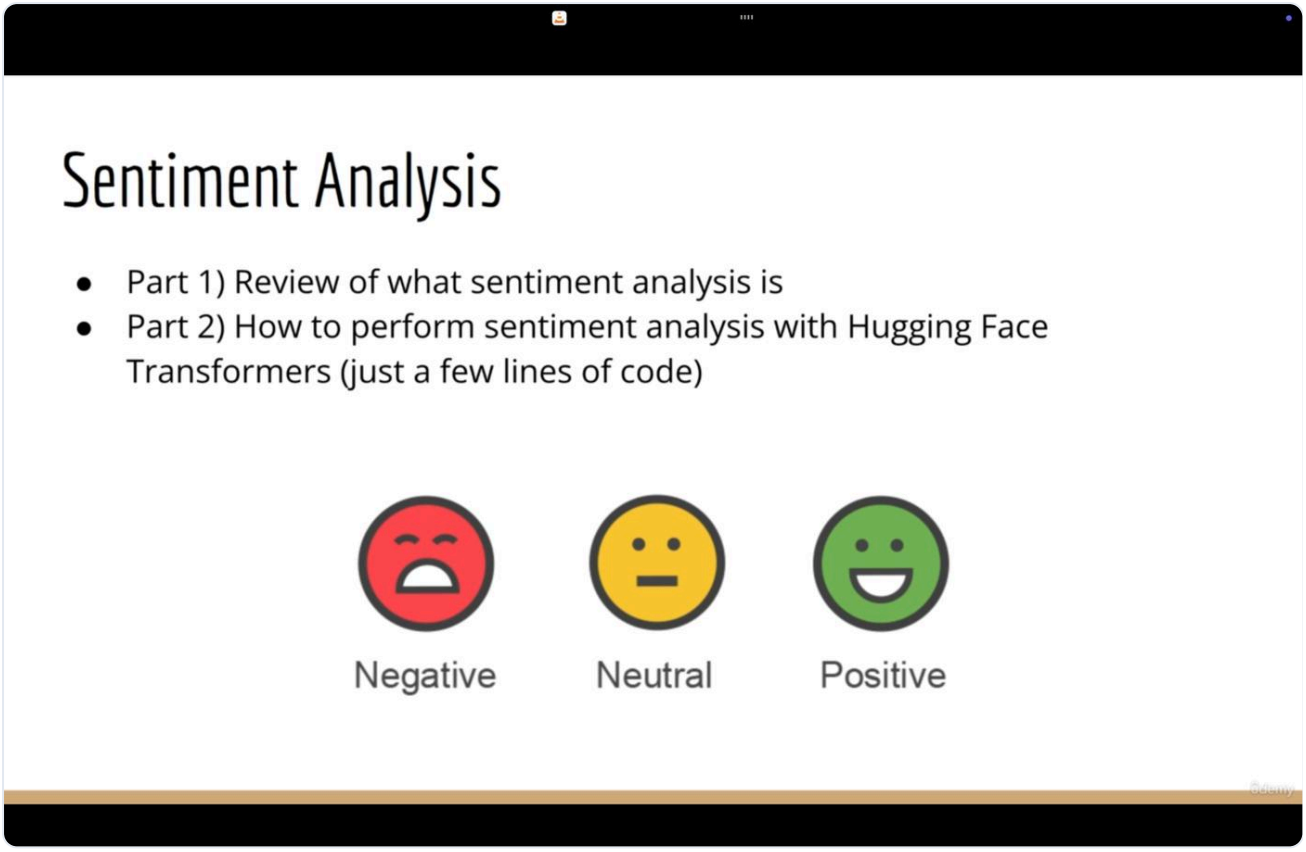

FIGURE 3.77 | Sentiment Analysis

In this section, we'll be using a very high level interface for Transformers, which is the Hugging Face pipeline.

Using this interface, you'll see that we can perform any of the tasks in the section with just one or two lines of code. So this should be useful for those of you who get intimidated when you see code or math or both.

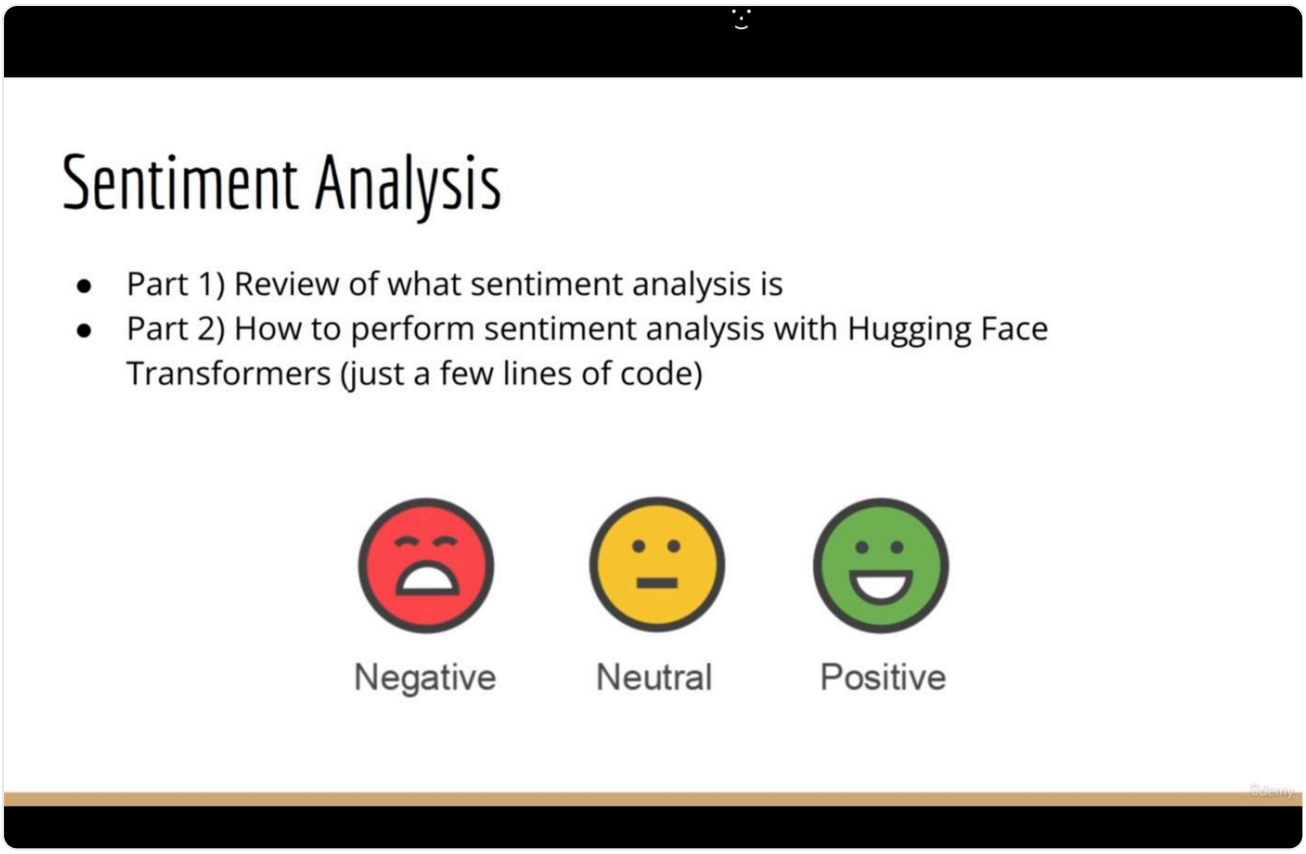

FIGURE 3.78 | Sentiment Analysis

So what is sentiment analysis? Consider the following sentences.

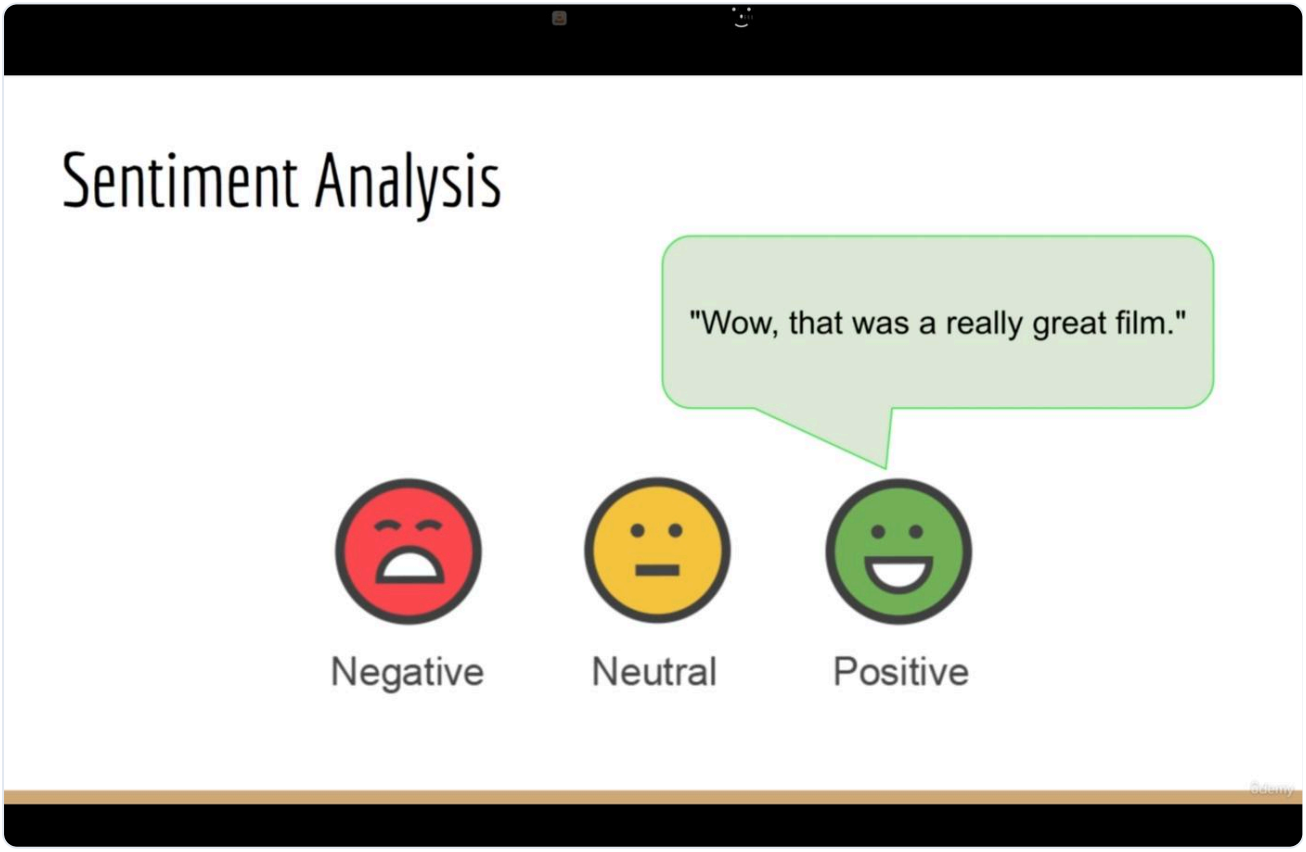

FIGURE 3.79 | Sentiment Analysis

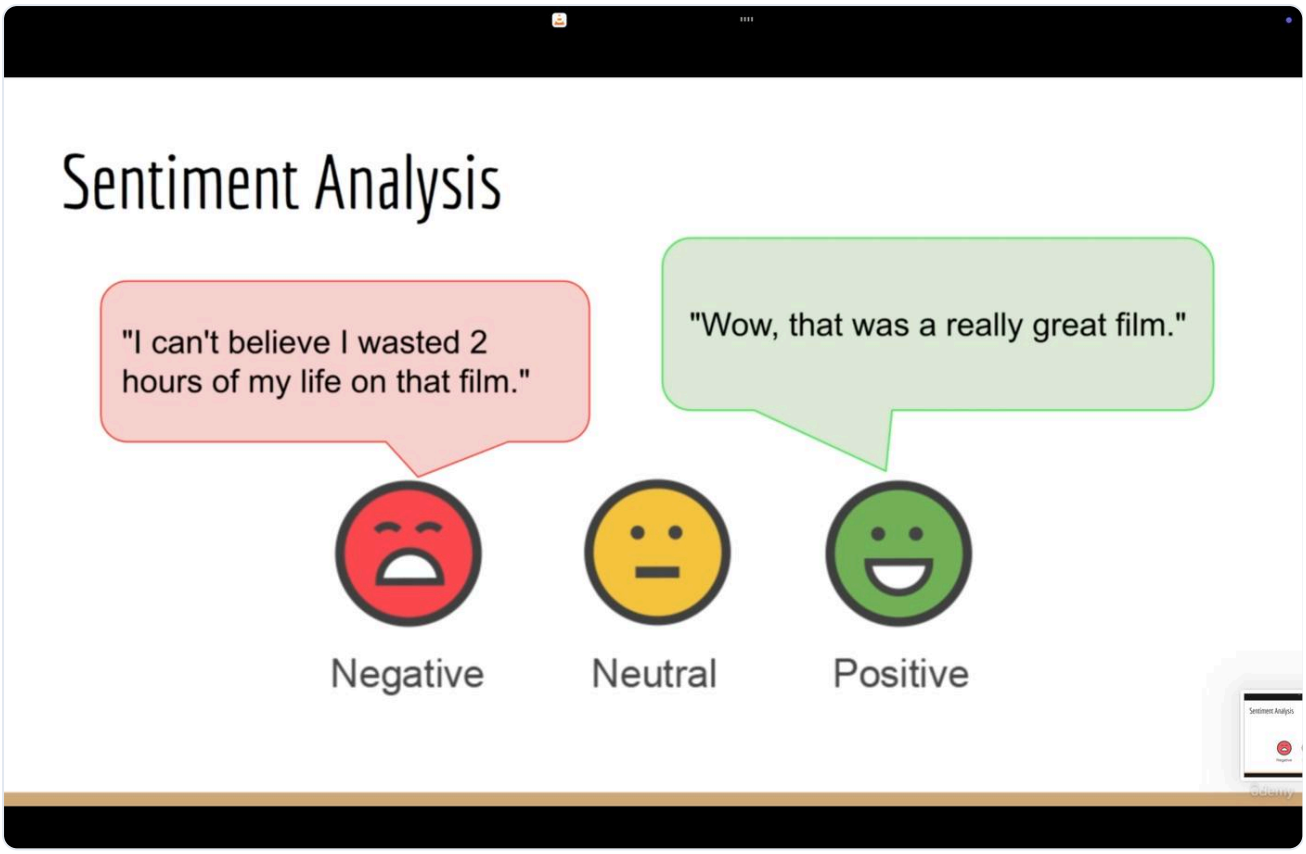

FIGURE 3.80 | Sentiment Analysis

Wow, that was a really great film. And now consider this sentence. I can't believe I wasted 2 hours of my life on that film. So what is the difference between these two sentences?

Well, it's probably very clear that one of these sentences is a very positive reaction to a film, while the other is very negative.

FIGURE 3.81 | Sentiment Analysis as a Classification Task

This is the concept of sentiment. Typically, we categorize sentiment as being either positive or negative or somewhere in between. Note that sentiment analysis is typically thought of as a classification task. Sometimes we simply call it sentiment classification, which is, in my opinion, a bit more clear now because sentiment can be thought of as being either positive or negative.

We often build binary classifiers to discriminate between these two classes.

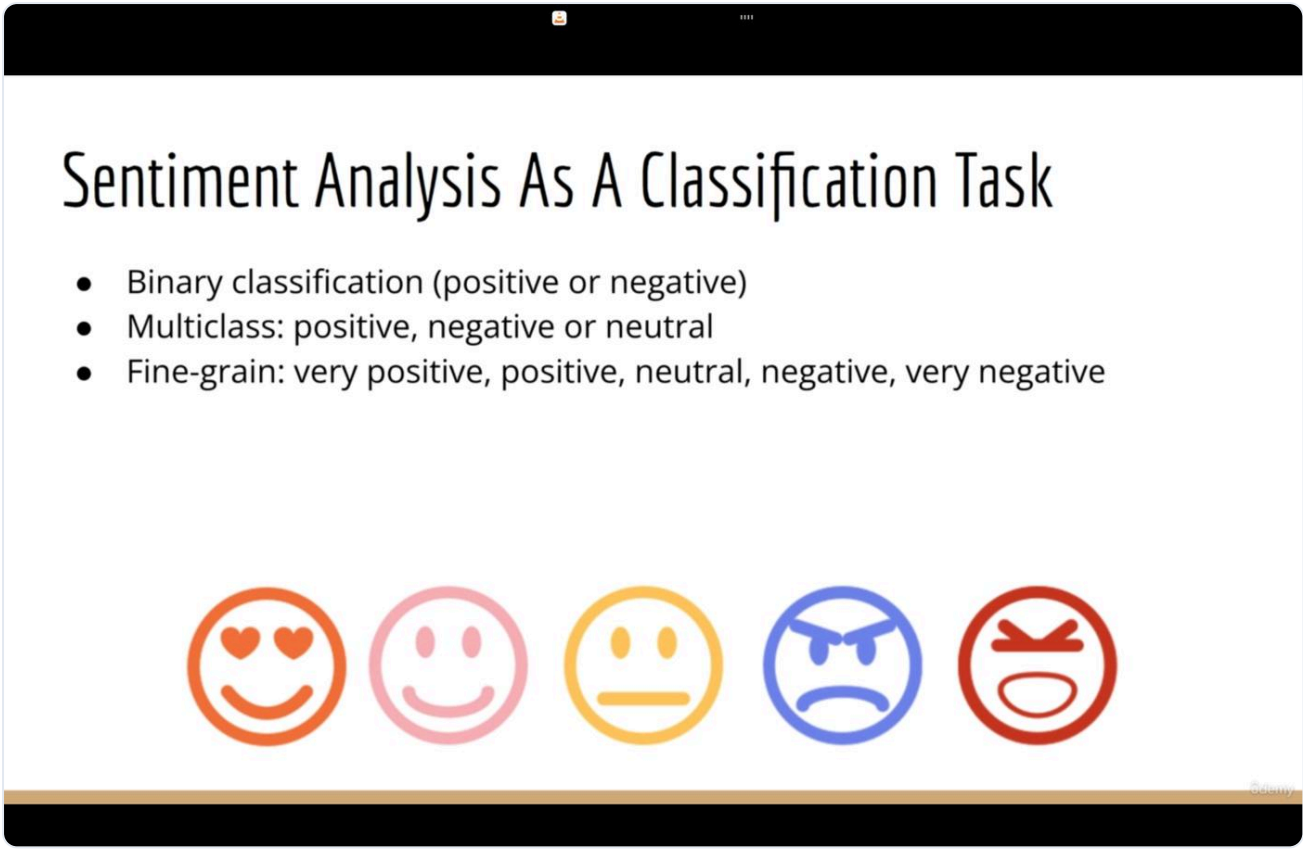

FIGURE 3.82 | Sentiment Analysis as a Classification Task

However, you will find instances where there might be three categories or even five categories. For example, in some datasets the categories are positive, negative and neutral in the other datasets. Perhaps we'd like to take a more fine-grained view. So we have positive, very positive, negative, very negative and neutral. So there are some different possibilities when we want to categorize sentiment.

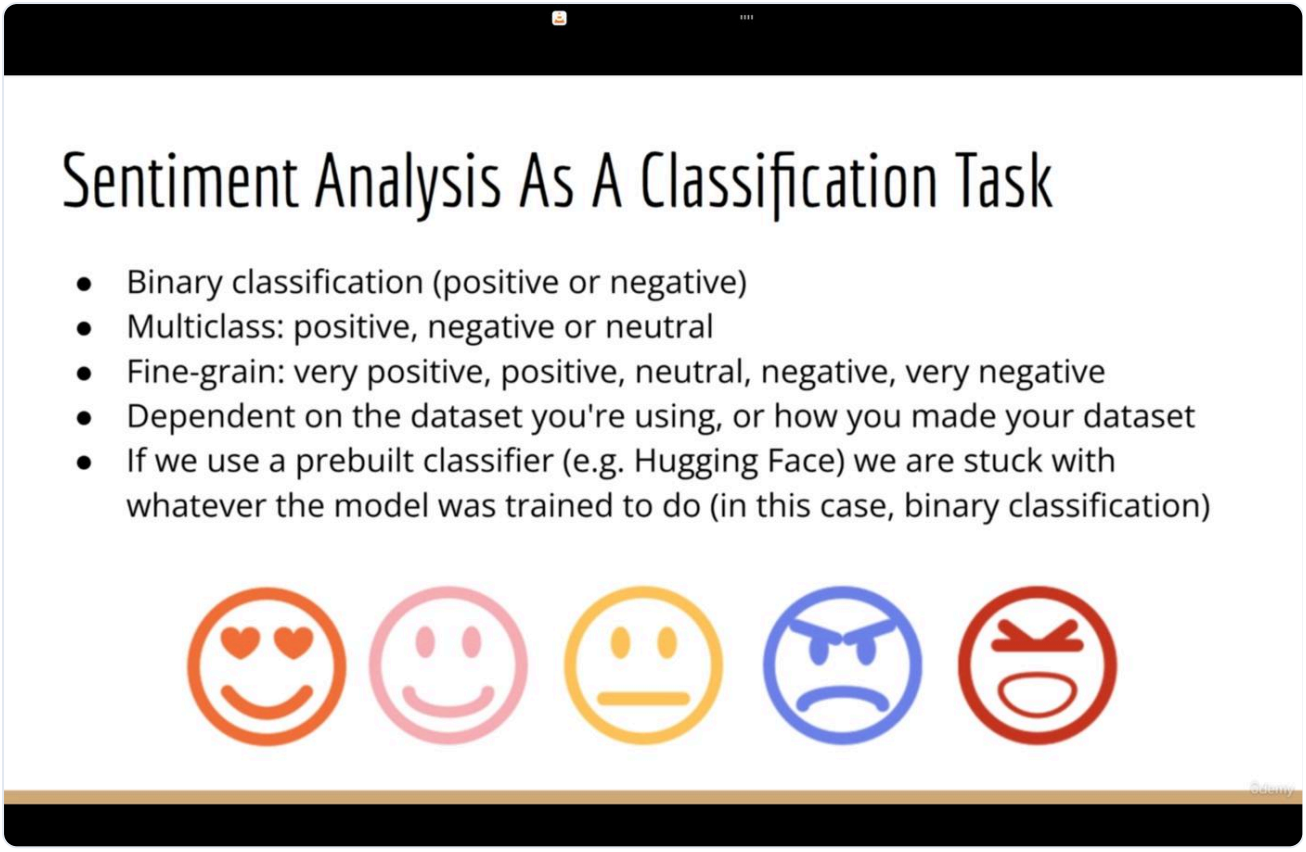

FIGURE 3.83 | Sentiment Analysis as a Classification Task

Now, this is really dependent on the dataset you are using, or if you created the dataset yourself, then it's dependent on how you decided to label it. But because we will be using a prebuilt classifier, this means that we will only be able to use the classes that this pre-trained model has been built to use. In this case, it will be binary. Thus, the only possible classes for us will be either positive or negative. So the next topic of this lecture will be to try and understand why sentiment analysis is useful in the real world.

As always, I like to think of this in terms of money because most students understand money, and most students understand that either as a business or as an individual. Your goal is to make more money. So how can you use sentiment analysis to make more money for your business or yourself?

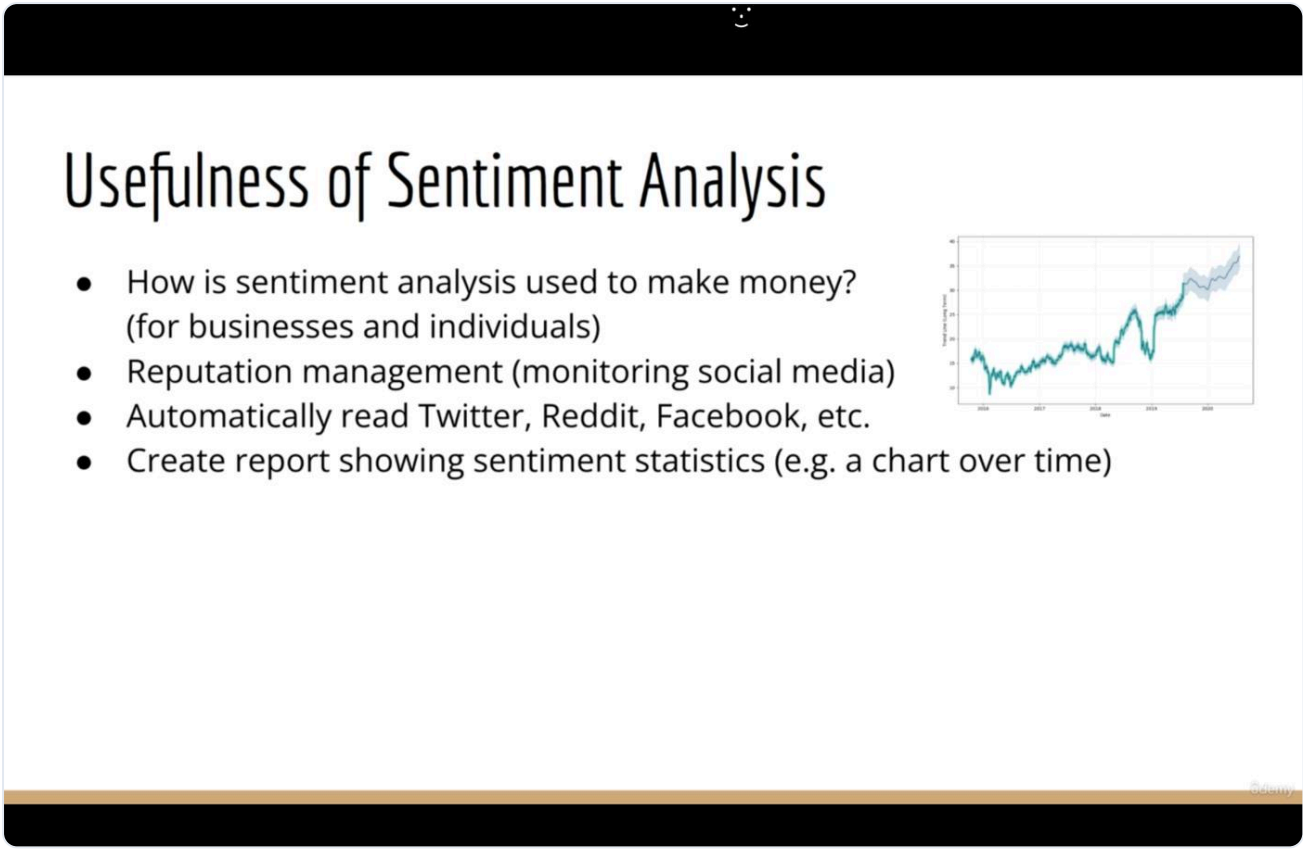

FIGURE 3.84 | Usefulness of Sentiment Analysis

In order to understand this. It's helpful to look at some examples. One example is reputation management, which involves monitoring of social media. Basically, the idea is you want to automatically look at tweets and posts on Reddit, Facebook and so forth that users are making about your company.

You can then pass these through your sentiment analyzer and create reports showing user sentiment statistics. Note that this can all be done programmatically so no one has to go in and read tweets themselves, which saves your company time and money. You can then take action based on what your customers have said about your company. Furthermore, note that this is not limited to your own company.

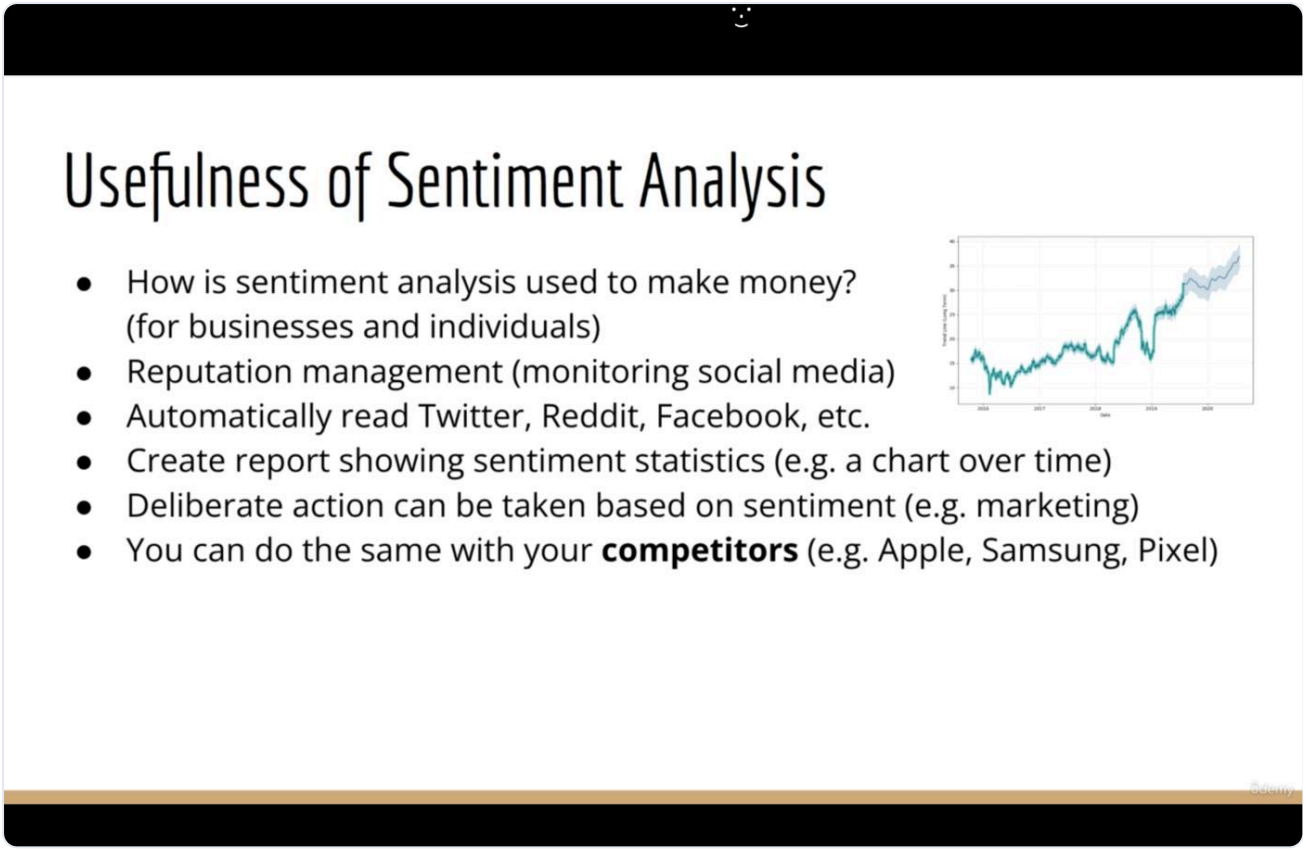

FIGURE 3.85 | Usefulness of Sentiment Analysis

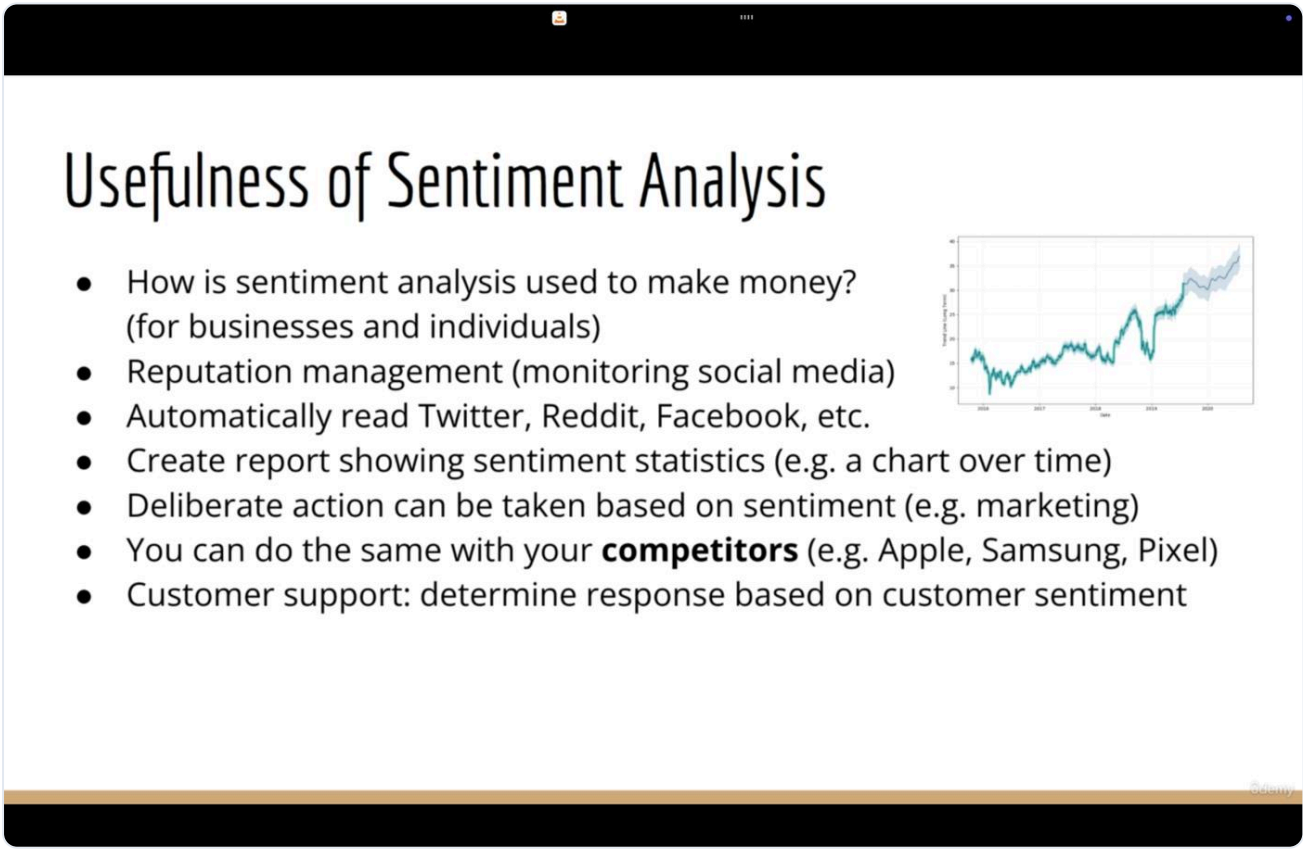

FIGURE 3.86 | Usefulness of Sentiment Analysis

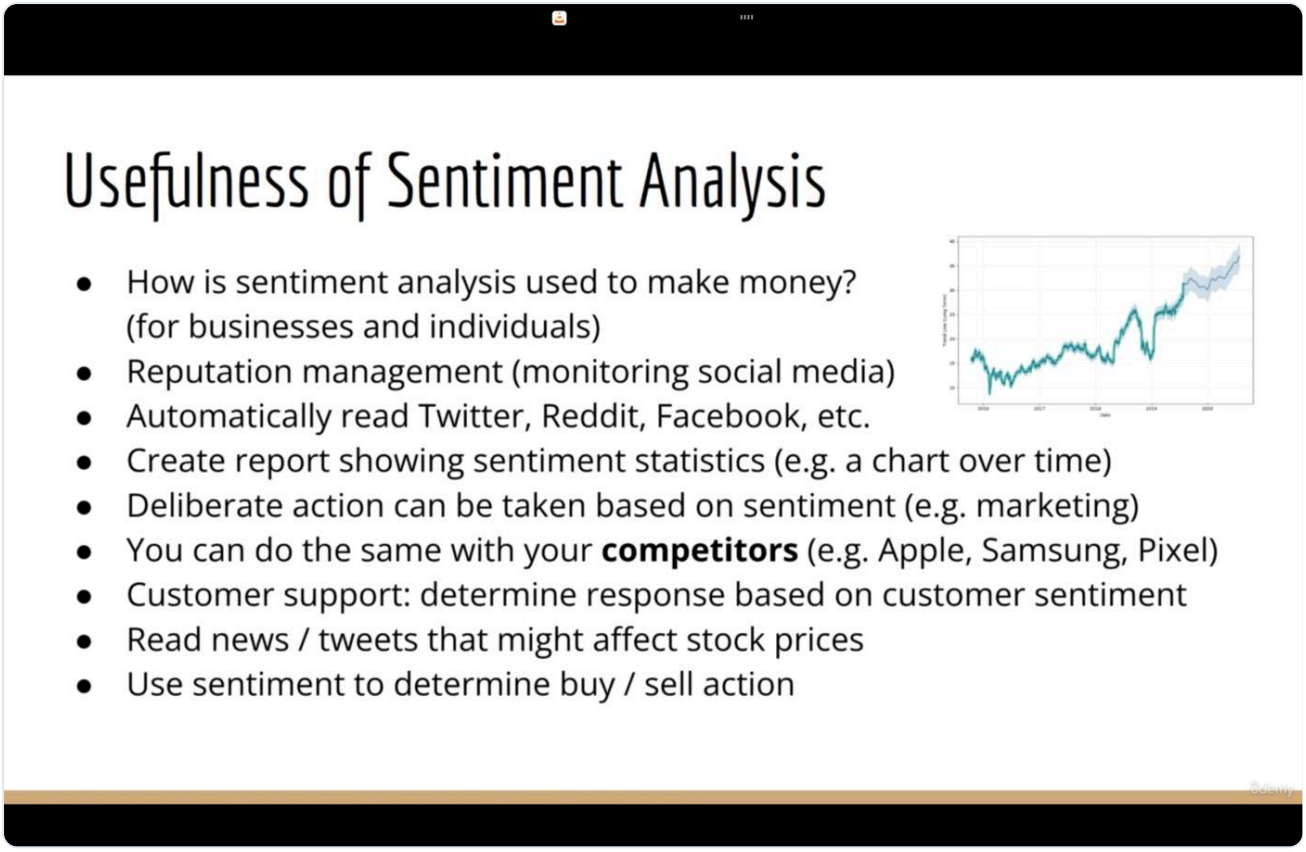

FIGURE 3.87 | Usefulness of Sentiment Analysis

You can do the same with your competitors as well. So if you're Apple, you might be interested in the sentiment behind the Google Pixel or the Samsung Galaxy.

Another example is to predict the sentiment of customers who are using your customer support. Perhaps the action taken by your customer support team is different based on the sentiment of the customer. Another popular example is to use sentiment analysis on news articles, tweets and other media concerning companies and industries whose stock prices might be affected by the news. So if your model predicts that the sentiment of an article is negative, you might want to consider selling your stock and vice versa if the sentiment of an article is positive.

By making more intelligent trades, you can profit by buying and selling stocks at the right times. Now you've probably seen that simple bag of words. Models can perform pretty well on certain data sets when it comes to the task of sentiment analysis. However, there are some tricky corner cases that require more advanced models. As you recall, a bag of words does not take order into account, and thus it is unlikely to be able to understand the relationships between words.

For example, consider the word interesting. We know that this has positive sentiment. For example, this movie was very interesting.

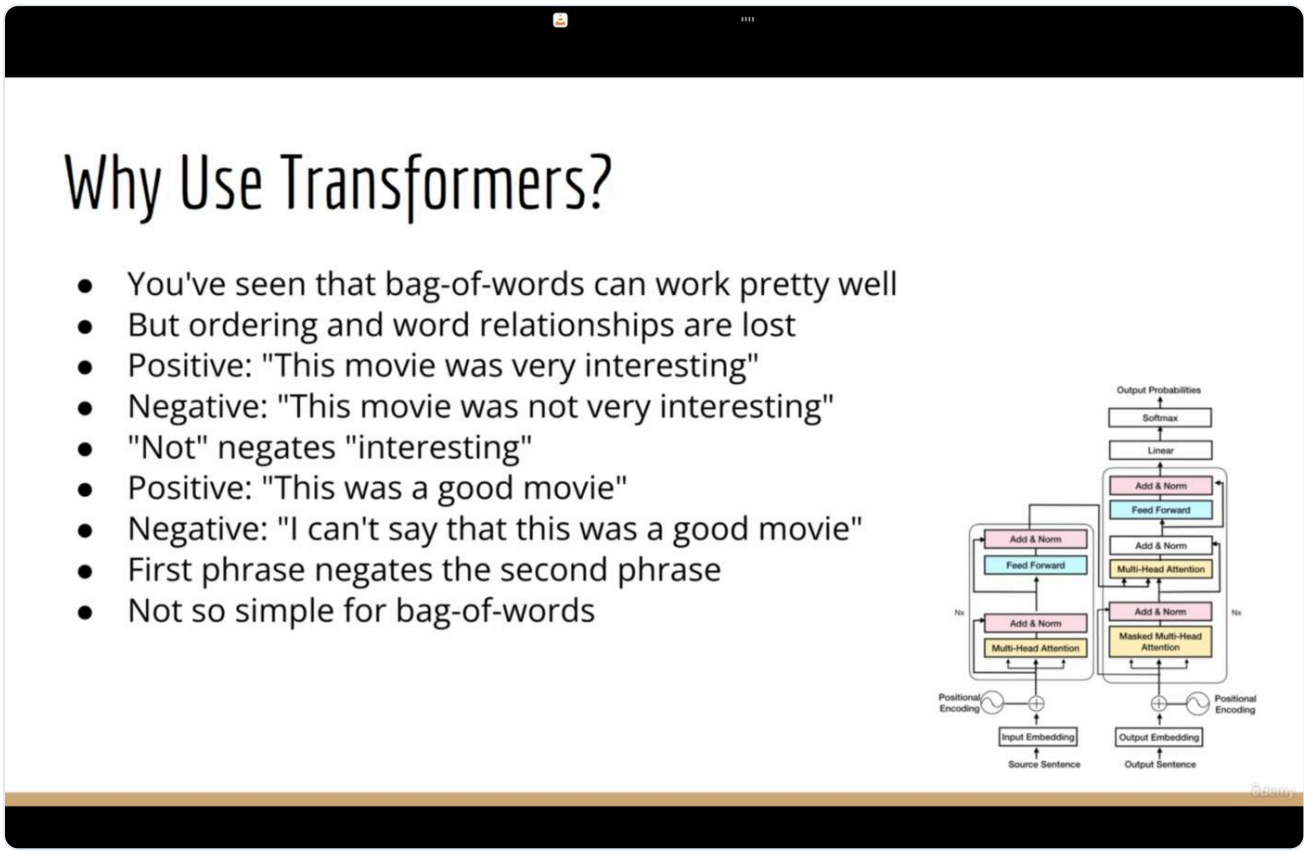

FIGURE 3.88 | Why Use Transformers?

But what if we say something like this movie was not very interesting? Now all of a sudden it has negative sentiment. The simple bag of words approach does not have an easy time figuring out that the word not negates the polarity of the word. Interesting. Or consider the sentence. This was a good movie. Clearly this has positive sentiment.

But now consider the sentence. I can't say that this was a good movie. And so in this case, it's not just a single word that negates another word, but a whole phrase that negates another phrase. This would be very difficult for a bag of words model, which would effectively just add up the sentiment of each individual word. So if we want to learn how the sentiment changes with the relationships between words, we need a more advanced approach. Luckily, sequential models like RNNs and CNNs can help.

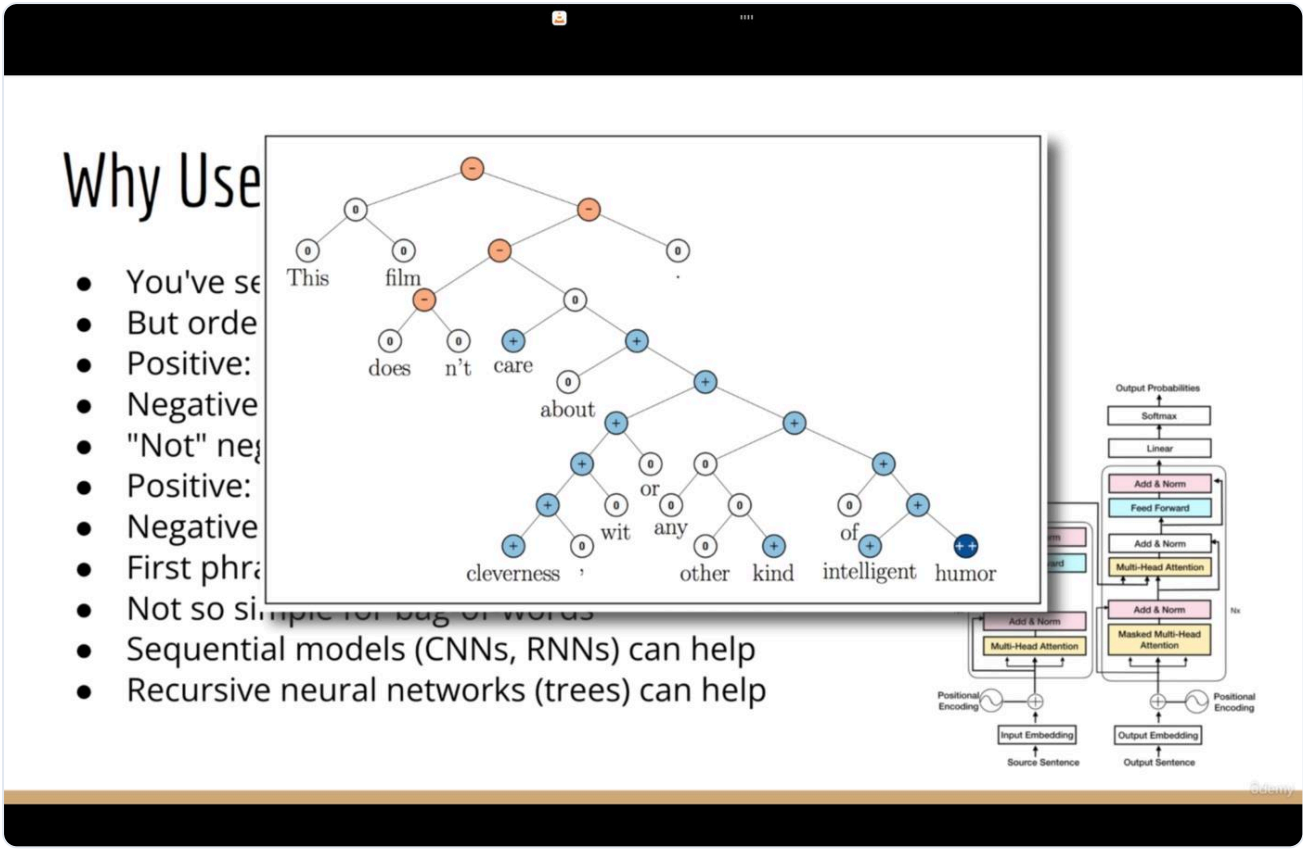

FIGURE 3.89 | Why Use

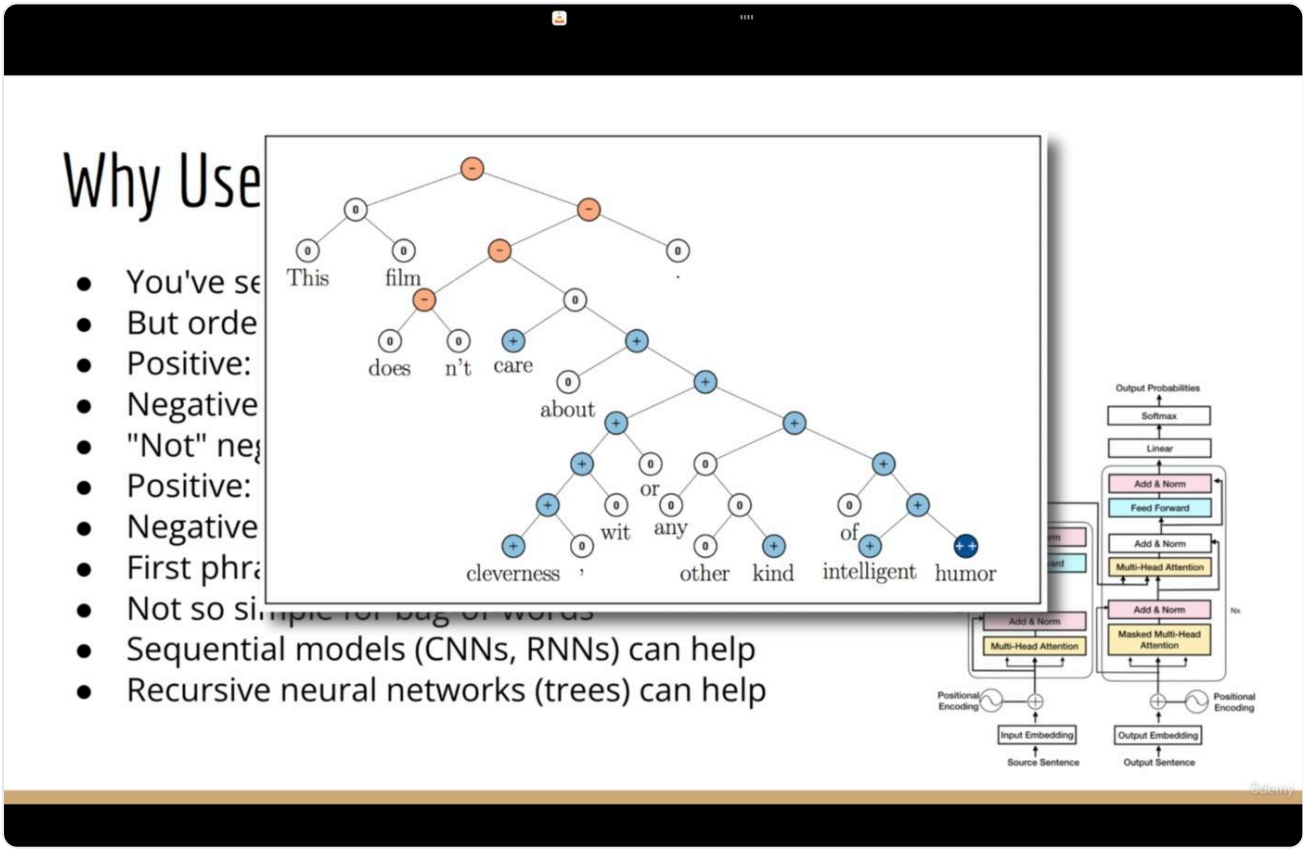

FIGURE 3.90 | Why Use

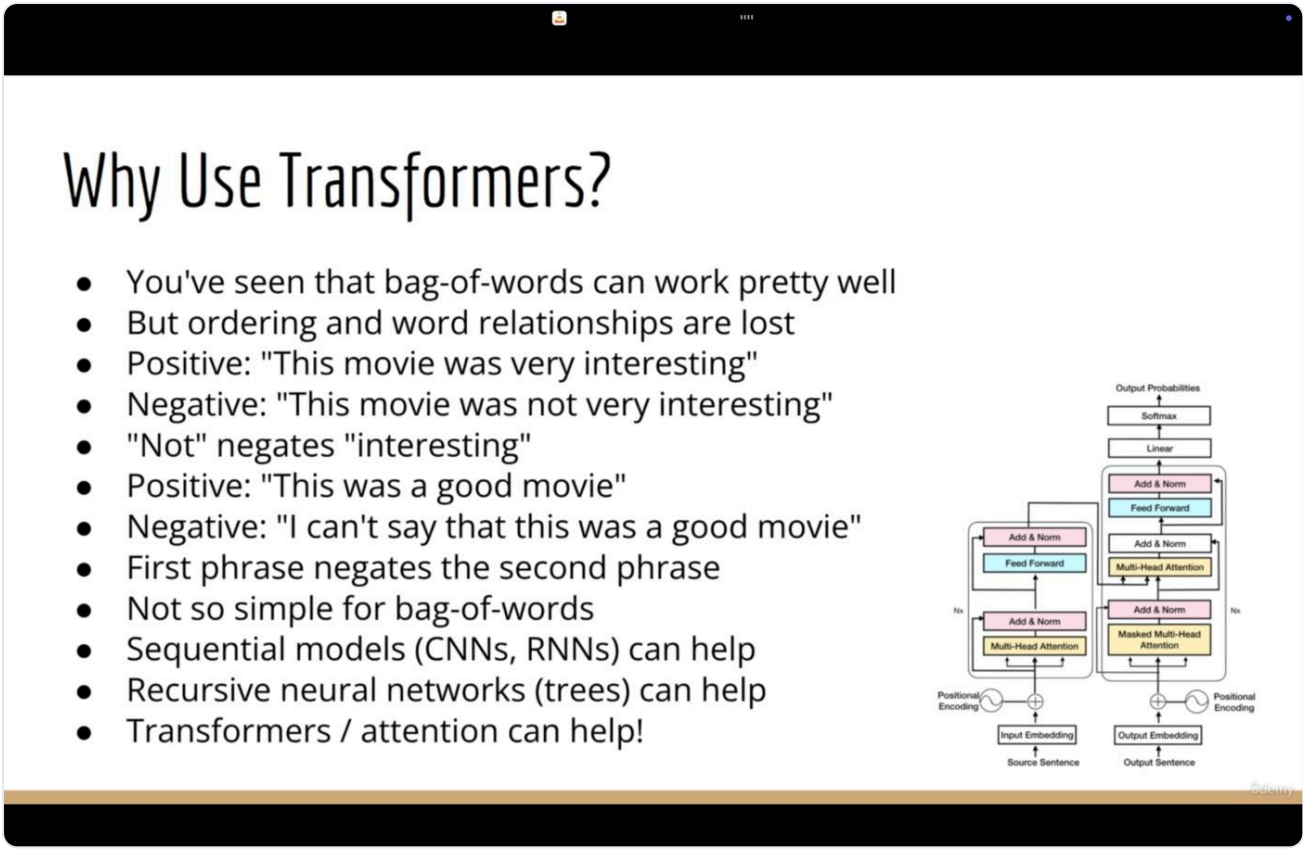

FIGURE 3.91 | Why Use Transformers?

In the past, I've discussed recursive neural networks, which are neural networks with a tree structure that also do very well with negation and even negating whole phrases since they just become nodes of the tree. In the next lecture, we'll see that transformers, which are sequence models with attention, also have this capability. Okay. So now that you know what sentiment analysis is and why it's useful, let's look at some code that can do sentiment analysis using Hugging Face.

The key function we need is a high level abstraction called a pipeline.

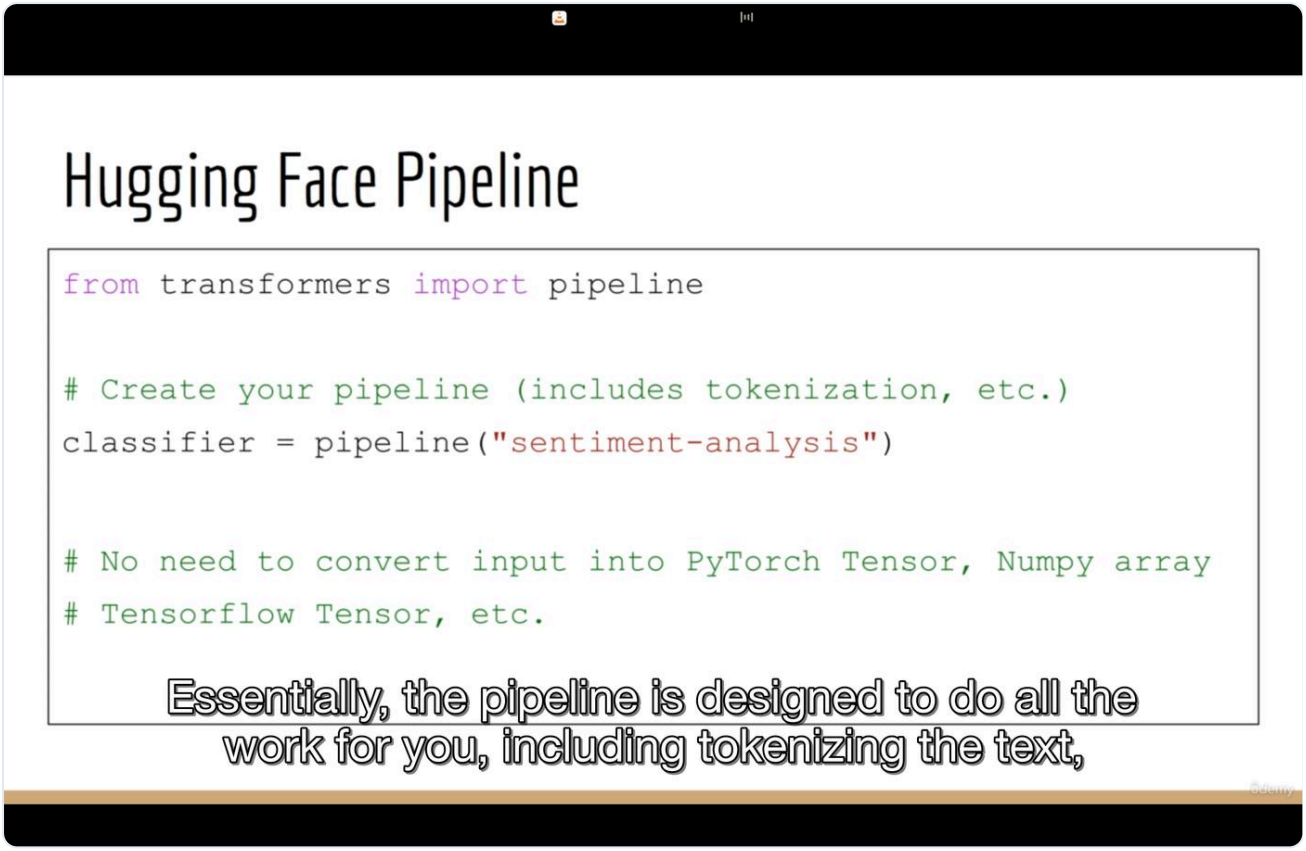

FIGURE 3.92 | Hugging Face Pipeline

Essentially, the pipeline is designed to do all the work for you, including tokenizing the text, converting the text into integers, and passing those integers through the model to get the output. So all the work you may have learned to do in the past, whether that was in TensorFlow, PyTorch or any other library, all of that is done inside this one interface. So to begin, we import pipeline from the Transformers library.

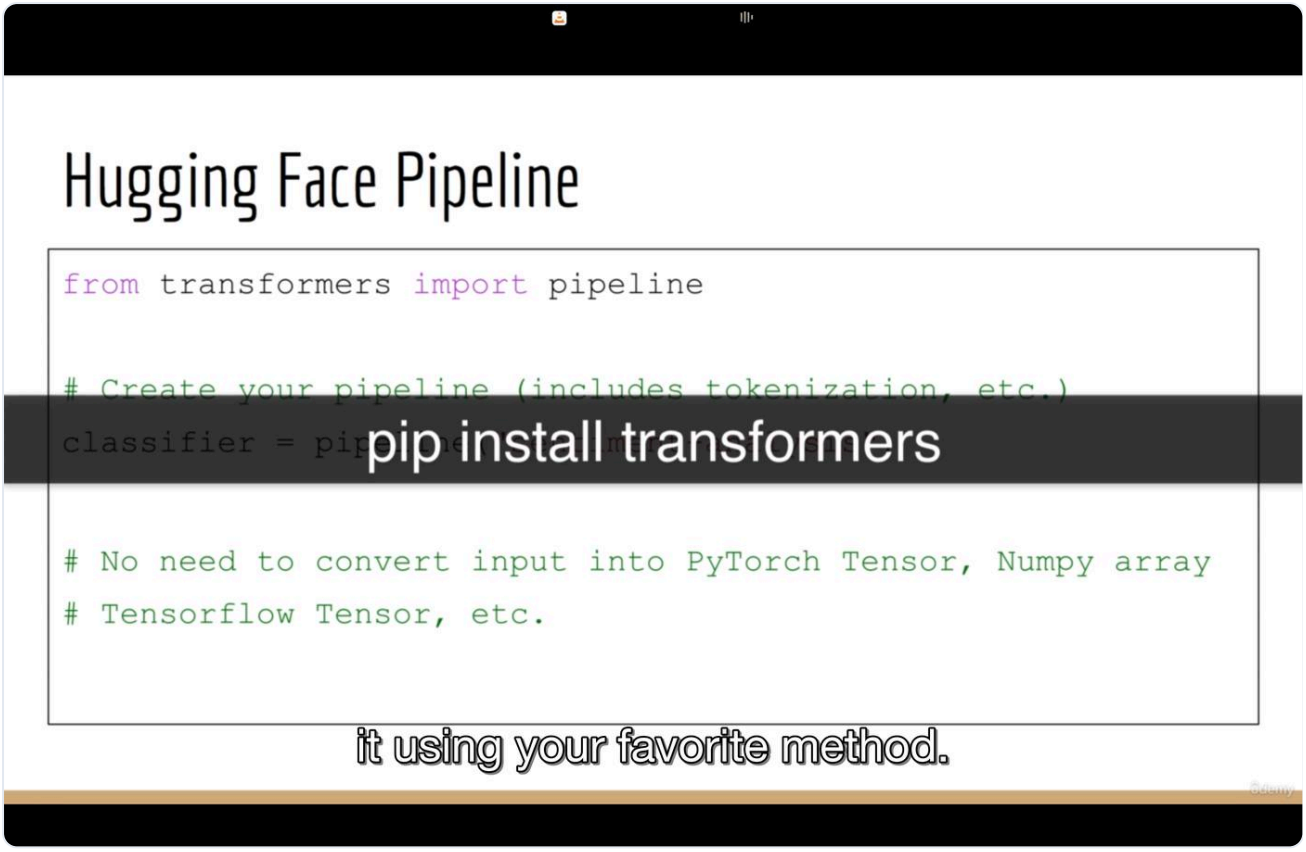

FIGURE 3.93 | Hugging Face Pipeline

And as usual, if you do not have this library installed already, it's simply a matter of installing it using your favorite method. For example, using pip install transformers. Once you've imported the pipeline function, you can then call the pipeline function specifying the task you want to perform as an argument. In this case, we pass in the string sentiment dash analysis. Now you might be wondering where can I get a list of all possible tasks that can be done with the pipeline?

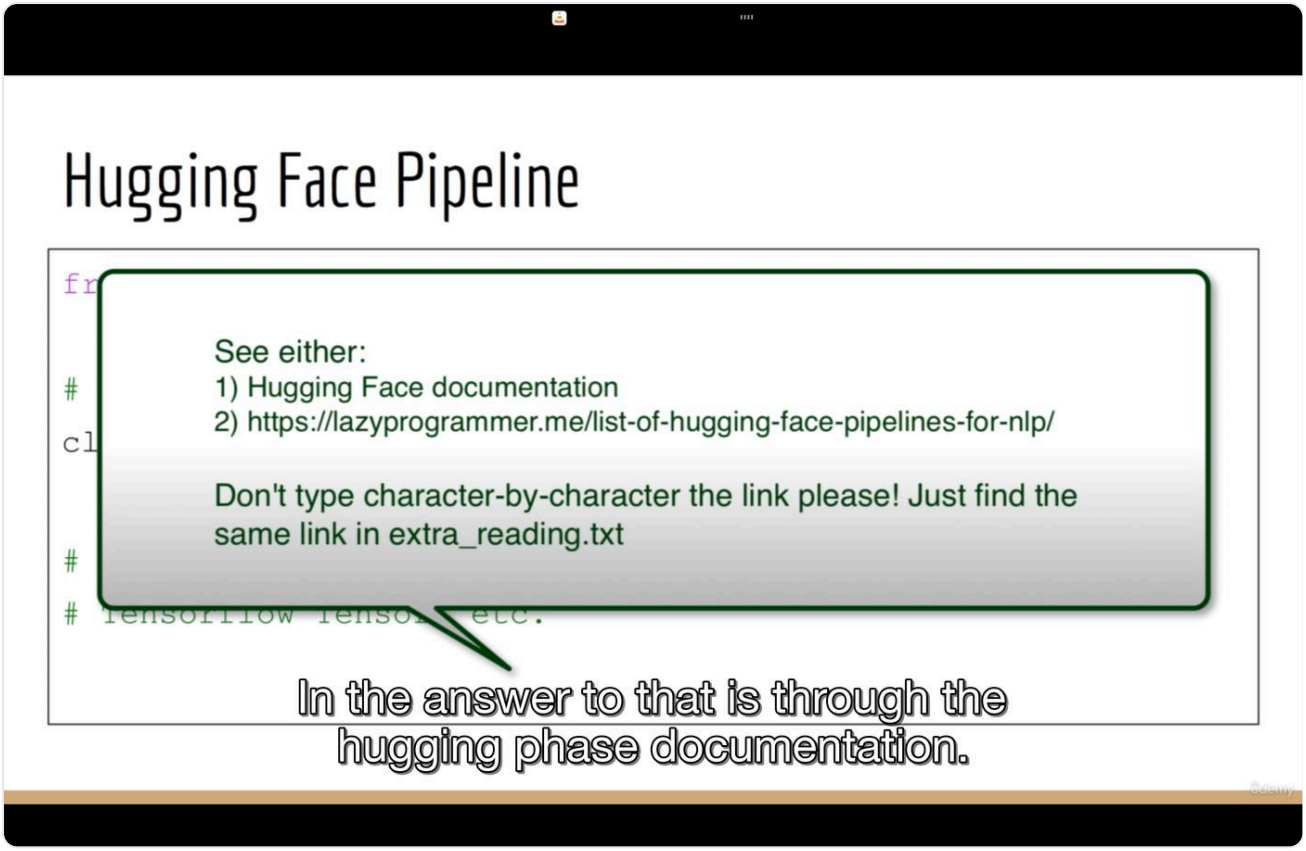

FIGURE 3.94 | Hugging Face Pipeline

And the answer to that is through the Hugging Face documentation. Either copy the link you see here or go to my website, which is lazyprogrammer.me. Although do note that we will be going through most of these tasks in this section anyway. So when you call the pipeline function, this basically returns a model which you can then use to make predictions. However, as mentioned, this is a special kind of model because it does all the work for you.

This is unlike a PyTorch model, for example, where you'd have to convert your data into PyTorch tensors before actually using it. Or in the case of TensorFlow, TensorFlow tensors and so forth. In this case we can simply pass in raw text. So for example, I can pass in the string.

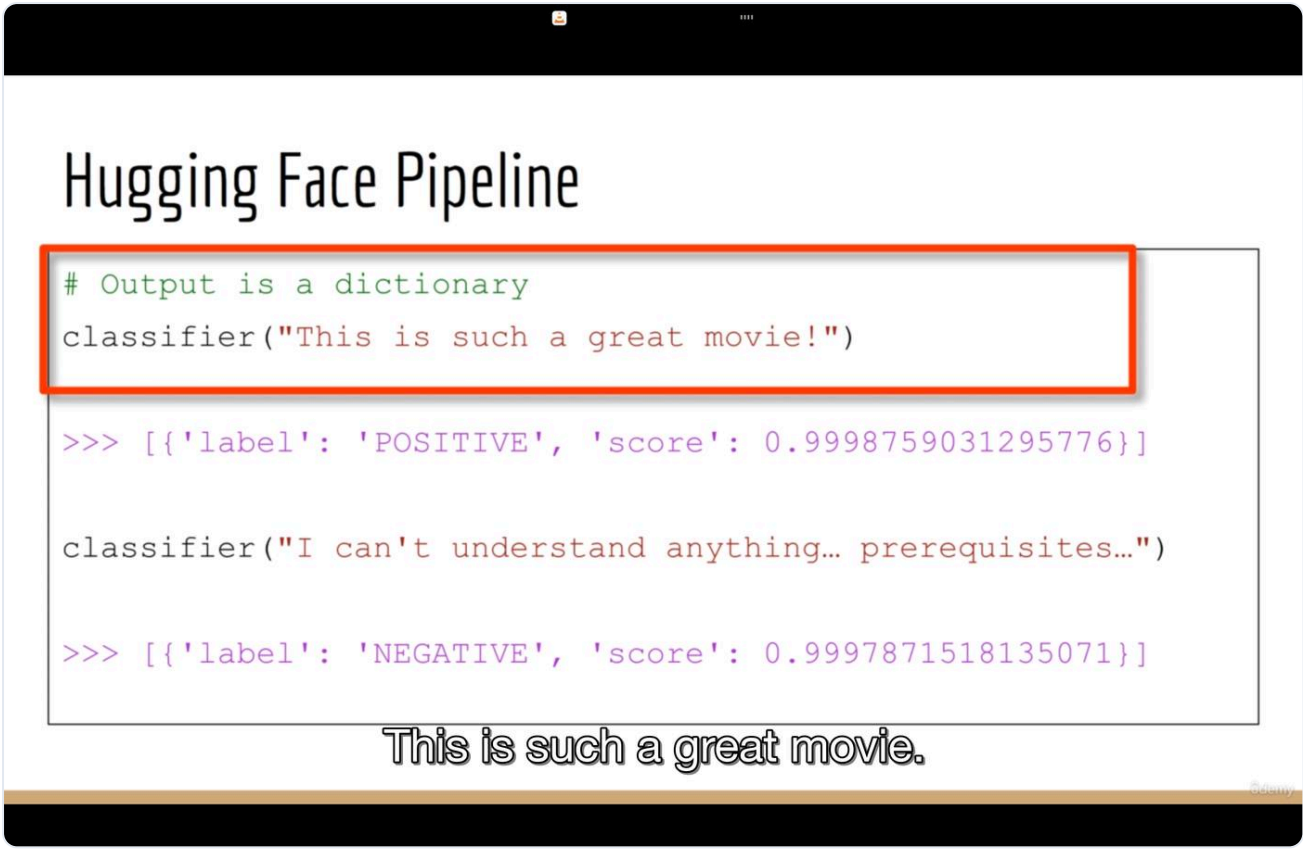

FIGURE 3.95

This is such a great movie. This will return a dictionary with the corresponding prediction. In particular, this dictionary contains two items a label and a score. Note that the score corresponds with the label.

For example, if the label is positive and the score is 99%. You can interpret that as the posterior probability for the text.

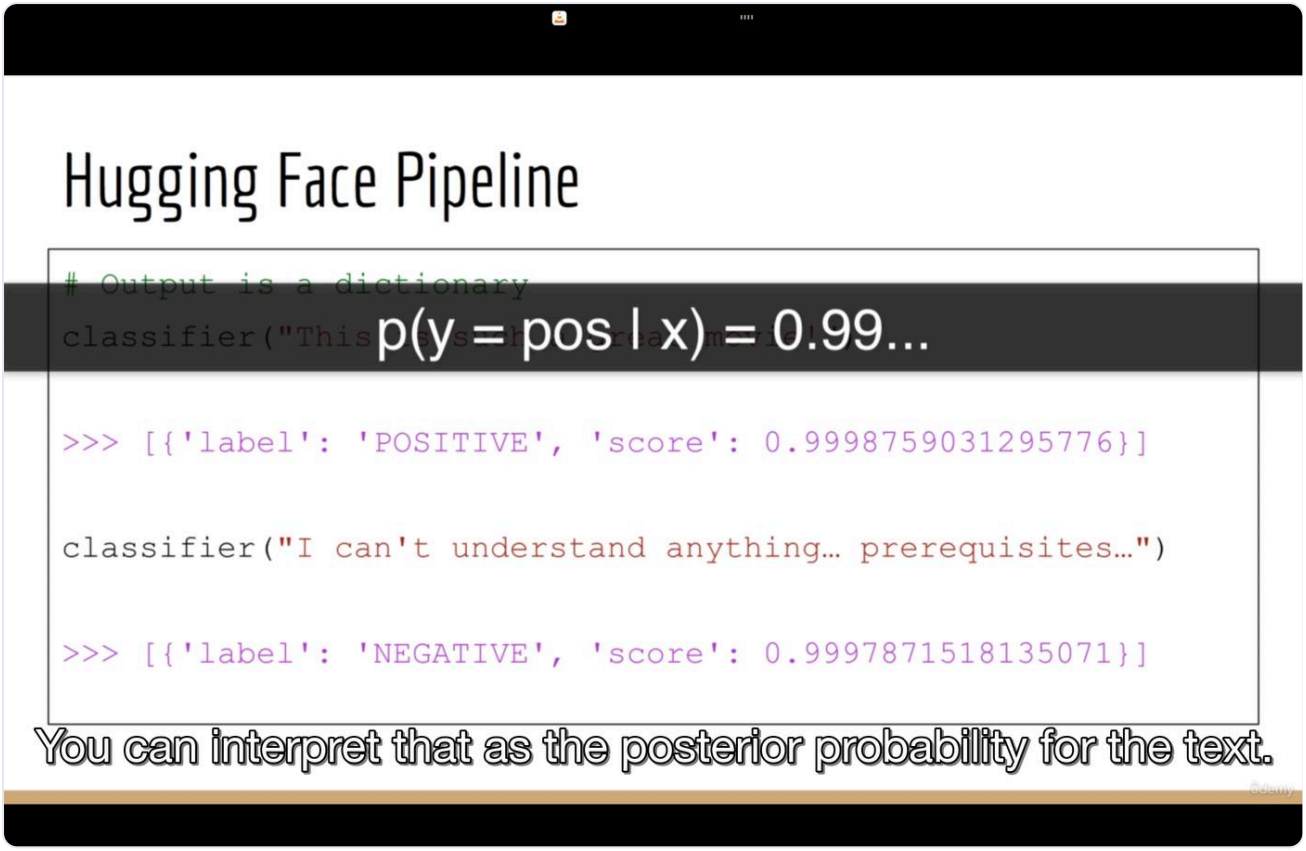

FIGURE 3.96 | Hugging Face Pipeline

Being positive is 99%. But note that this will not return 1% if the label is negative. In the case where the prediction is negative, then under the key label it will be negative and the score might be something like 99%. Again, meaning that there is a 99% posterior probability that the input text was negative. So for example, if I pass in the string, I can't understand any of this.

The instructor kept telling me to meet the prerequisites.

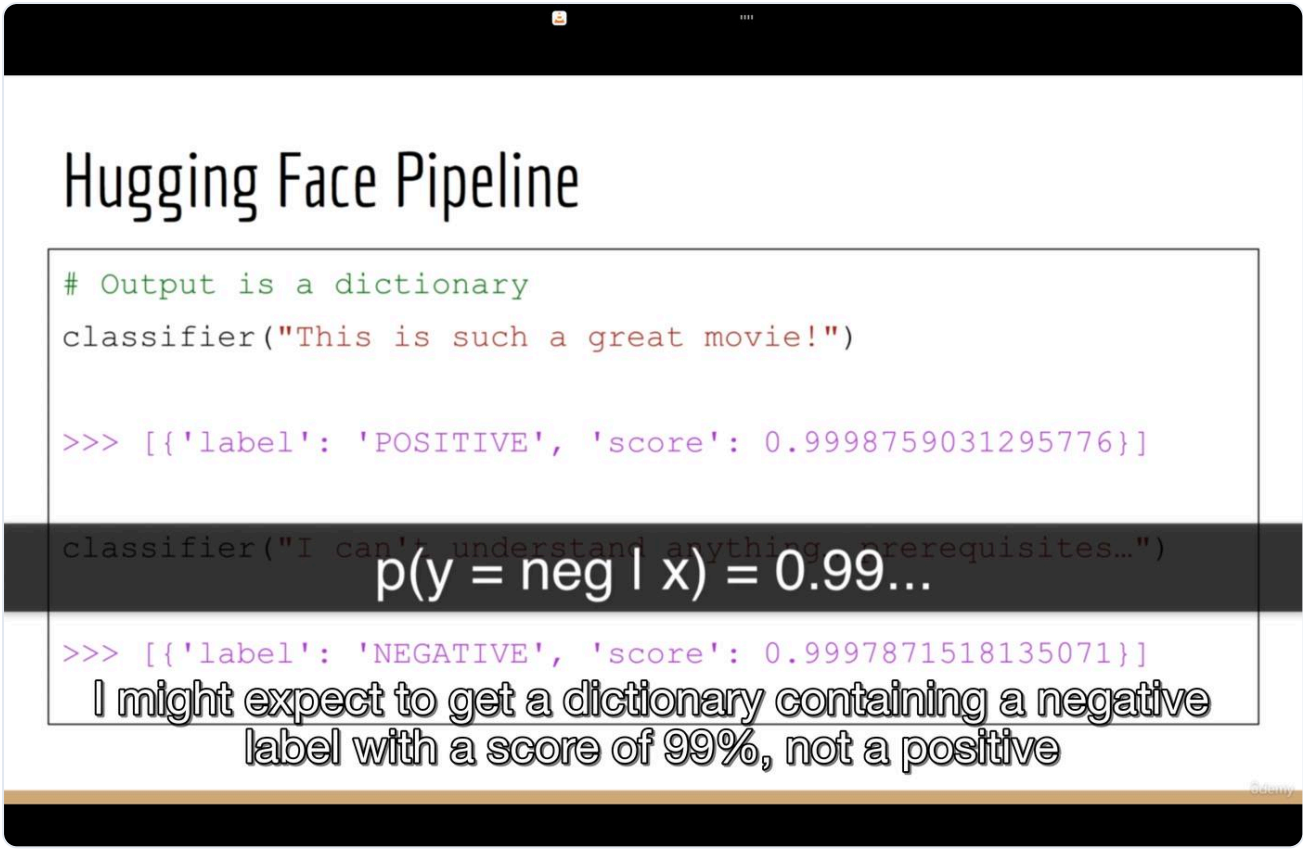

FIGURE 3.97 | Hugging Face Pipeline

I might expect to get a dictionary containing a negative label with a score of 99%, not a positive label with 1%.

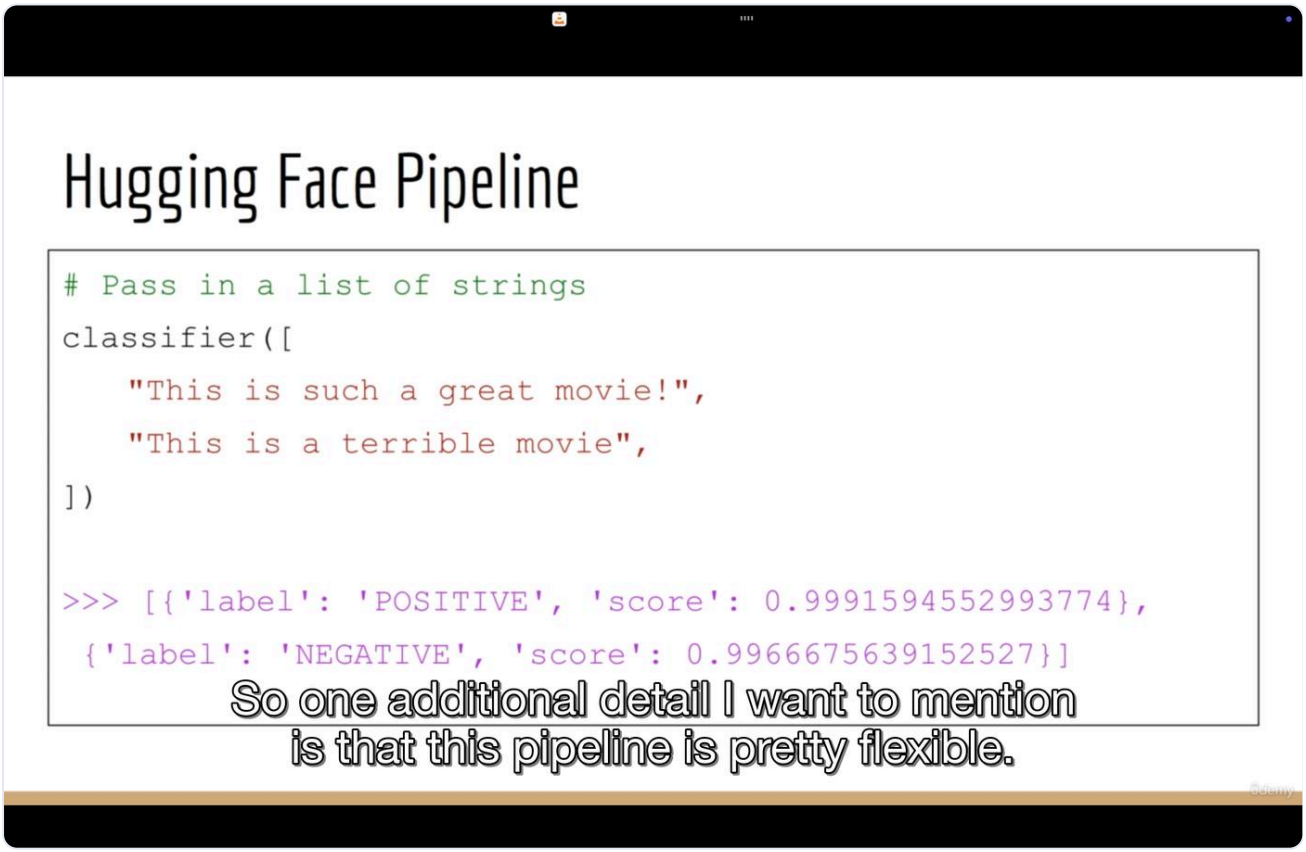

FIGURE 3.98 | Hugging Face Pipeline

So one additional detail I want to mention is that this pipeline is pretty flexible. So if you want to pass in more than one sample, you don't need to do it in a loop. Simply pass in a list containing all the text documents you want to classify. In return, you'll get a list containing dictionaries of the same form as above.

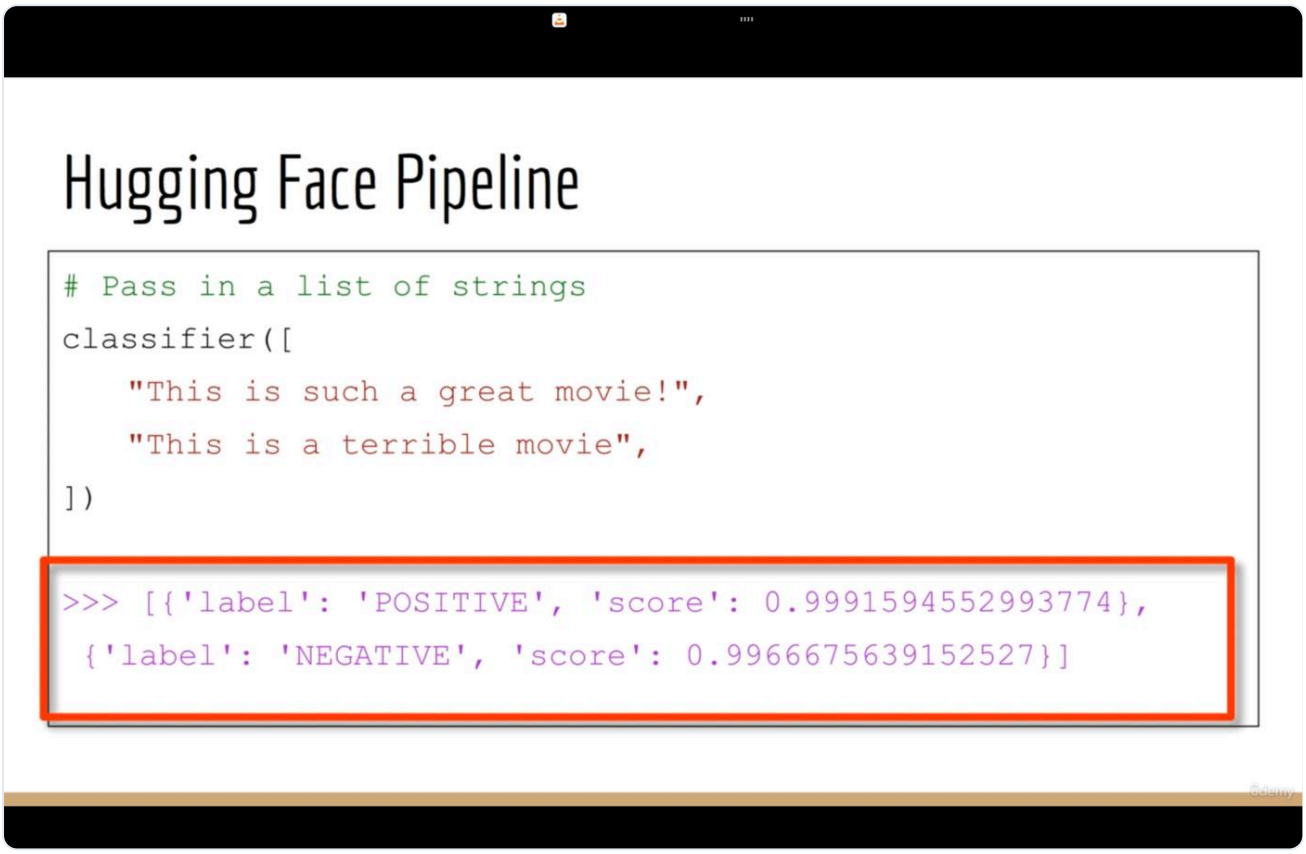

FIGURE 3.99 | Hugging Face Pipeline

Okay. So that's pretty much it for sentiment analysis. As promised, it can all be done in just one line of code. To recap, the steps are number one to import the pipeline function.

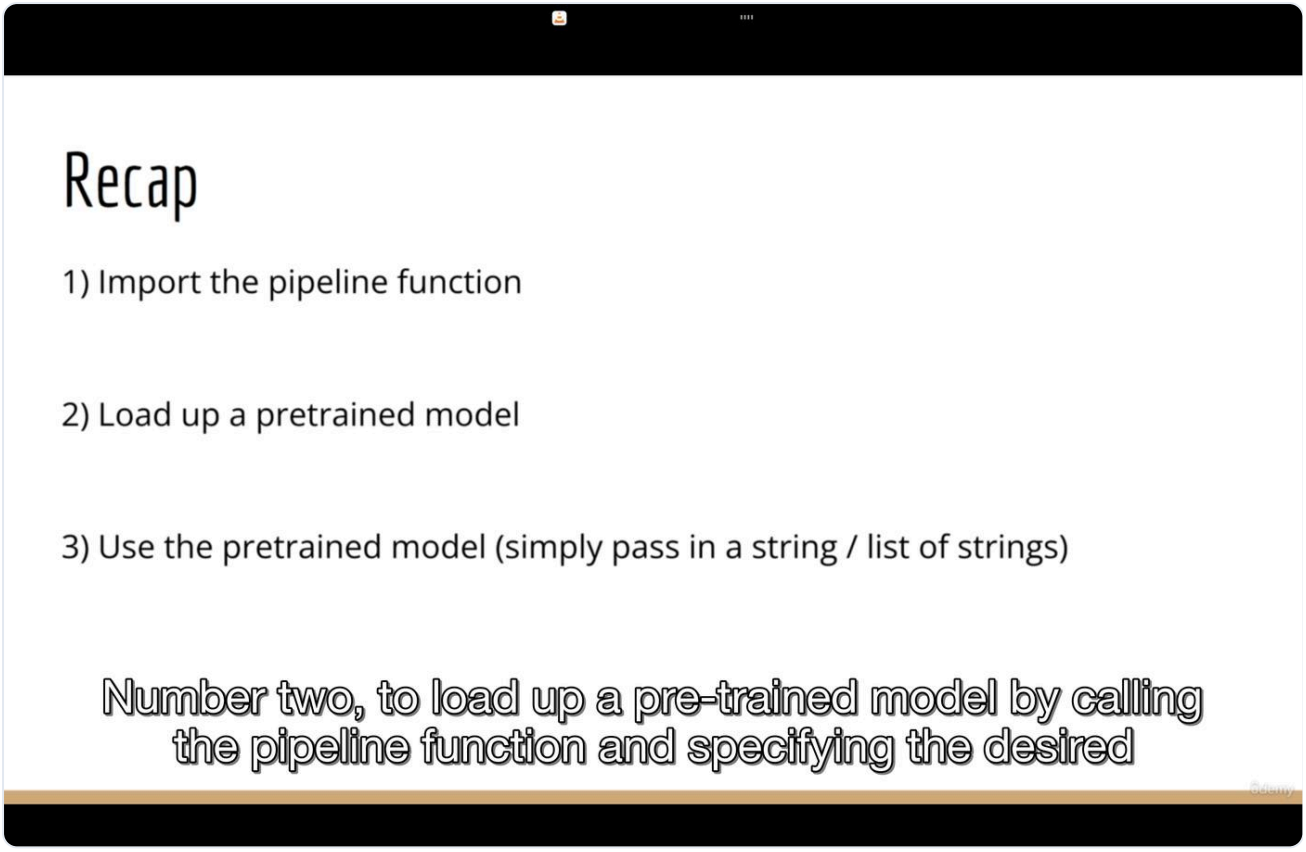

FIGURE 3.100

Number two, to load up a pre-trained model by calling the pipeline function and specifying the desired task. And number three, use the pre-trained model by passing in a string or list of strings.

4 · Sentiment Analysis in Python

See the notebook here.

So in this lecture, we'll be looking at the notebook to do sentiment analysis with Transformers. The first thing we'll do in this notebook is install Transformers since it does not come with CoLab currently. The next step is to download our data sets. This data set is a set of tweets which have been labeled with their corresponding sentiment. Note that the dataset is a three class dataset, including the neutral class, which is not supported by the model will be using.

Thus, at some point in the script, we'll need to filter out any tweets labeled neutral. The next step is to import everything we need in this script. You've probably seen this all before, so if there's anything you're unsure of, just use the Q&A to ask. Obviously, we'll need to import pipeline from Transformers, since that is the main purpose of this script. The next step is to demonstrate basic usage of the Hugging Face pipeline will begin by calling the pipeline function and passing in sentiment analysis so that the model we get back will be appropriate for this task.

Now notice how when you run this, some information gets printed out. Firstly, it says no model was supplied, defaulted to distilbert-base-uncased, fine tuned and so on and so forth. So this is just telling us what model is being used. Of course there are many transformers, including GPT,

BERT, BERT, T5 and so forth. So there are typically multiple options for any particular task. In this section, of course. Since we haven't yet learned anything about any specific transformer model, it suffices to choose the default.

Notice that this also automatically downloads the model in the case where it does not yet exist locally. Okay. So the next step is to check the type of the classifier variable we just got back to see what it is. Okay. So we can see that it is an object of type text classification pipeline. So this makes sense. Note that it is not a raw PyTorch model or TensorFlow model, even though that is what is used underneath. As you recall, the pipeline contains many more steps, including the model, but also including the tokenizer.

The next step is to use our classifier object to make a prediction while passing the string. This is such a great movie. As you can see, this has been predicted as positive with the very high score as you may have expected. Okay. So the previous example was pretty easy. Let's try a more difficult example, which requires that the model knows how to negate. So the sentences. This show was not interesting. As you can see, the model correctly predicts that this has negative sentiment.

Now, just for completion sake, let's try the opposite sentence, which is this show was interesting. And as expected, the model predicts that this has positive sentiment. Okay. So now let's try a pretty tricky sentence. This show was not bad at all. Now, clearly, you and I can understand that this is a positive comment. However, think about how it might be interpreted by a simple bag of words model like TF-IDF. In that case, it might see the word bad and presume that it is negative.

The word not also has some negative connotation. For example, I went to the store, but I did not find what I needed. So the word not might also contribute to a negative sentiment prediction in a more naive model. So let's see how our transformer performs. So as you can see, the model correctly predicts that the sentence is positive. Let's now try another difficult example in which the sentence is made up of two phrases, and the first phrase, it negates the second.

I can't say that this was a good movie. The second phrase, which is this was a good movie, is positive, but the sentence as a whole is negative. Okay. So our model correctly predicts that this is a negative sentence. Note that the pipeline can be used to make a prediction for multiple data points at once, which we can pass in as a list. For this example, we'll pass in two strings. The first string is. This course is just what I needed. We might expect that this is positive.

The second string is I can't understand any of this. The instructor kept telling me to meet the prerequisites. What are prerequisites? Why does he keep saying that? And obviously this is just my own caricature of reviews I've seen in the past. For the second review, we might expect that it will be negative. Okay. So as expected, the first document is classified as positive with a very high score, and the second document is classified as negative also with a very high score.

Note that the score corresponds to the label being predicted. So if it's negative, you're not going to get 0.001. As you might expect with the raw neural network. Now by default, pipelines do not exist on the GPU, which is what we'll need if we want to use these on a full dataset. In fact, you're

encouraged to try this script both with CPU and GPU to see the difference in speed for yourself to get our GPU device will begin by importing torch. The next step is to check whether or not CUDA is available.

As you can see, this returns true, meaning the GPU is available. The next step is to check the current CUDA device. As you can see, this returns zero so we can use zero to refer to the GPU. The next step is to create a new pipeline object where we still pass in sentiment analysis, but we also specified device equals zero corresponding to the GPU as we just established. The next step is to read in our CSV using pd.read_csv. The next step is to call the head function to see what's inside our data frame.

Note that this dataframe contains many columns, but the ones we are interested in are the tweets and the corresponding labels. You can feel free to look through these yourself, but note that they will not be useful since we are using a pre-trained model. The next step is to filter out our dataframe by selecting only the columns we need. The next step is to plot a histogram of our labels. This will tell us whether or not our data is imbalanced and what classes we have in our data set.

Okay. So as you can see, we have three classes which does not match the two classes available in the Hugging Face pipeline. Also, notice that the classes are imbalanced as we have many more negative tweets. The next step is to filter out any rows which have been labeled as neutral because there's no way our model can predict this class anyway. The next step is to map the labels to integers, which may be useful later on. We'll begin by creating a dictionary which maps the string positive to one and negative to zero.

We then call the map function on the airline sentiment column. Passing in the dictionary mapping we just created will assign the result to a new column called Target. The next step is to call the head function again to see what our data frame looks like now. As you can see, we have our new Target column corresponding to whether the sentiment was positive or negative. The next step is to check how many samples we have in our data frame. I found this useful to get a sense for how much time this model took to make a prediction for each sample.

Note that because transformers are such large models, they take much longer than what you might be used to. As you can see, we have about 11,000 samples, which is quite modest. The next step is to actually use our pre-trained classifier to get predictions. We'll begin by grabbing the text column of our dataframe and then calling the two list function to turn it into a list. Well, then passing this list of tweets into the pipeline object and we'll call the result predictions.

Now because this is an edited video, you won't get to see how long this actually took. So you're encouraged to run this for yourself, to get a feel for how long it takes. You should find that it takes about a minute, which is a lot longer than anything else we've done previously, which might be surprising. And remember that this is not even training. This is just making predictions. The next step is to simply print out the predictions to see what we got back.

Okay. So as you can see, we get a very long list of dictionaries of the same form we previously saw. Note again how there is a mix of positive and negative predictions. Now because their data is imbalanced. We like to compute metrics such as the AUC, which requires probabilities. However,

these probabilities must all be of the same kind, which ours are not. What I mean by this is when our prediction is positive, the score is the probability for the positive class.

But when the prediction is negative, the score is the probability for the negative class in order to compute the AUC correctly. We must have all the probabilities be for the positive class in order to do this. We're going to loop through our predictions and map these to a list of probabilities only for the positive class. So in this loop you can see that we take the RAS score, but only if the label starts with P, meaning that it is positive. Otherwise we set it to one minus the score, which converts the negative probability to a positive one.

The next step is to print out probabilities to see what we got back. Okay. So as you can see, we now have some values which are very small, meaning very close to zero, which is not what we had before. The next step is to convert our predictions into an array which will help us compute the accuracy more easily. To do this, we'll have a loop similar to the above, except that if the label starts with P, we return one. Otherwise we return zero. The next step is to cast the list we just created into a NumPy array.

The next step is to compute the accuracy, which is just the mean of the targets equals equals the predictions. As you can see, we get almost 90%, which is pretty good, considering that this model has never seen any tweets before. However, we do have to keep in mind that this is an imbalance dataset and thus the accuracy could be misleading. The next step is to compute the confusion matrix will set normalize the true so that each row sums to one. Okay.

So as you can see, we get large numbers along the diagonal, which is encouraging. However, it's best to visualize the confusion matrix in a more presentable manner. So the next step is to define a function and to plot the confusion matrix. If you're curious about why I've included this function instead of just using the one at scikit-learn note that I've made a comment about this in The Notebook. In particular, we know that essentially all of the libraries in the numpy stack are constantly being updated with breaking changes.

It turns out that currently the confusion matrix plot is one of those breaking changes. Unfortunately, the latest version of scikit-learn is not included in CoLab yet, so for now we'll be using this custom function, which works just as well. If you want to know what this function does, it's pretty simple. Essentially, we're going to create a heat map and we're going to use a pen as dataframe. Since that allows us to give names to both the rows and columns.

So we'll begin by defining a function called plot_cm, which takes in a confusion matrix called cm. We then put our list of classes into a variable called classes. The next step is to create a data frame for our confusion matrix. Note that the index which corresponds to the rows, has been set to classes and the columns have been set to the same lists. So both the rows and columns are the list of classes. The next step is to use Seaborn to plot a heat map passing in the dataframe we just created.

We then set the X and Y labels so that we know which axis corresponds to the predictions and which corresponds to the targets. Once we've defined our function, we then apply it to our confusion matrix above. As a side note, recall that we do not have any train-test split because we

haven't done any training in this notebook. Effectively, our entire dataset is a test set. Okay. So this is our confusion matrix, which looks about what you would expect. We can see that we get the highest accuracy on the negative class, although the positive class is not that far off.

The next step is to check the F1 score. As you can see, this returns about 76%, which is okay. However, note that the F1 score is not symmetric since it depends on what we consider the positive class. As you've learnt, the F1 score is the harmonic mean of the precision and recall, which are easiest to understand in terms of document retrieval. So it really depends on what we are trying to detect. Or in other words, retrieve. To demonstrate this, we can simply invert both the targets and predictions by subtracting them from one.

If the value was one, we get one minus one, which becomes zero. And if the value was zero, we get one -0. And it becomes one. So now we can see that the new F1 score is about 93% and thus it really depends on which label we assign to zero and which label we assign as one. The next step is to check the AUC, which is not sensitive to the meaning of the labels. Note that this uses probabilities since the ROC and hence the AUC is generated by setting different thresholds for detection.

Okay. So for the AUC, we get about 95%, which is decent. And just to demonstrate that the AUC doesn't depend on the label assignment, we're going to compute the AUC on the inverted labels as well. As you can see, we get the same answer as promised.

5 · Text Generation

So when this lecture, we will be discussing our next task, which is how to generate text. To begin this lecture, we'll start with something that I think is a bit more intuitive, and this comes from my courses where I've taught time series. So one of the main goals of building a time series model is to predict the future. The reason why we would want to do this should be pretty obvious. If it's not obvious to you and none of this makes sense.

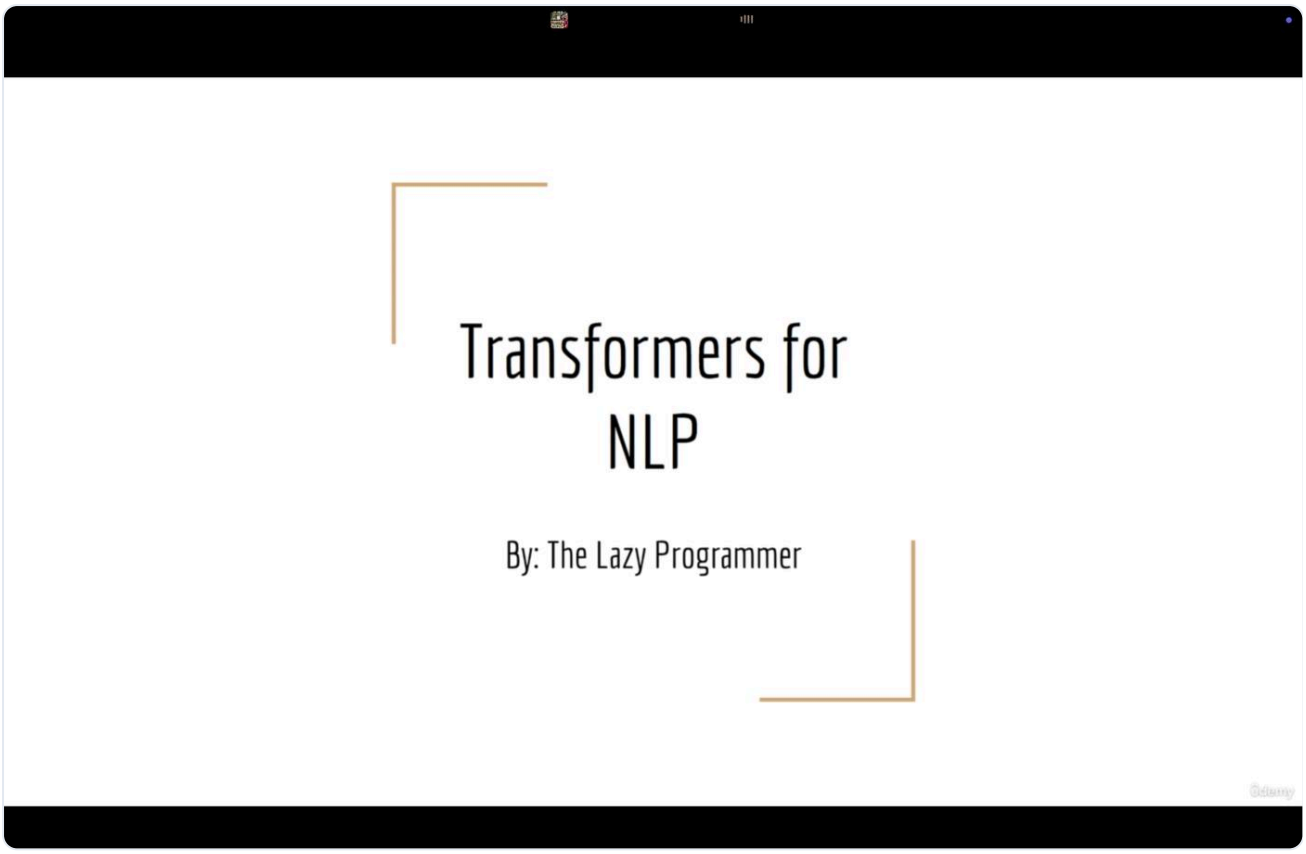

FIGURE 3.101 | Transformers for

Please let me know on the Q&A so that we can get that sorted.

Okay. So why would we want to predict the future? Here are some simple but powerful examples. Suppose that we could predict how much the price of a stock will go up or down.

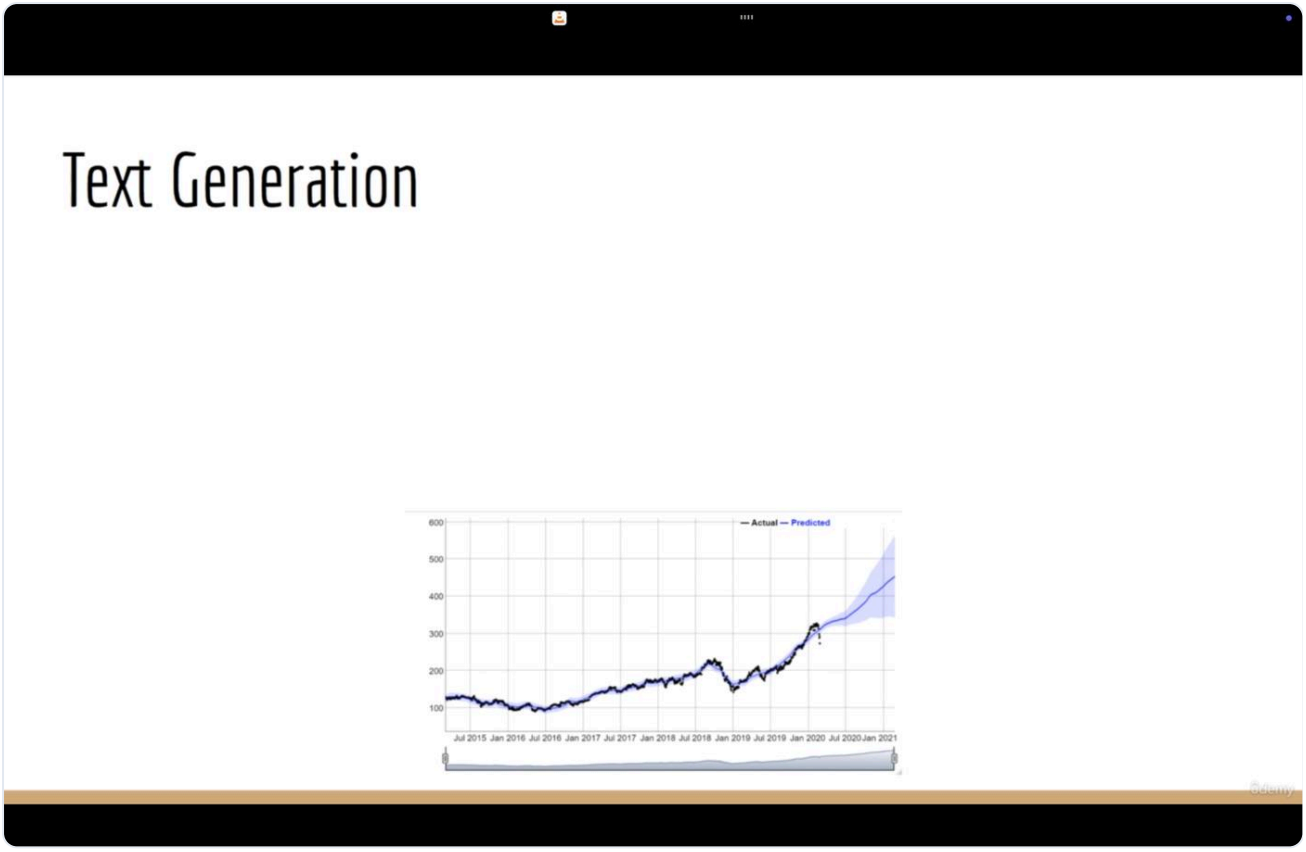

FIGURE 3.102 | Text Generation

As a side note, just in case this is an obvious, the price of a stock over time forms a time series. Okay, so what is the advantage to us? If we could predict how much a stock's price will go up or down? Well, the answer is we could potentially make lots of money. If we predict that the price of a stock will go up, then we should buy that stock and sell it later for more money, hence making a profit.

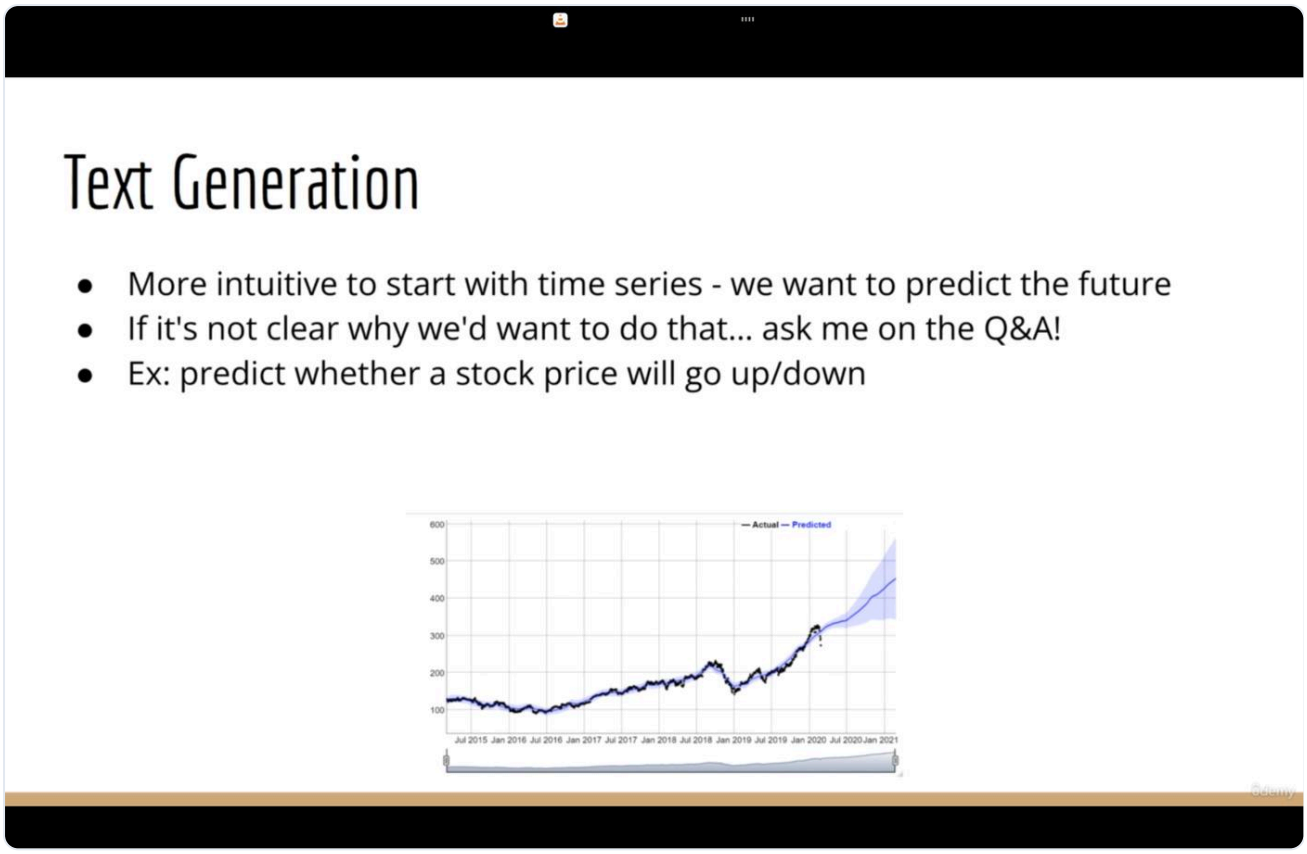

FIGURE 3.103 | Text Generation

If we predict that the price of a stock will go down, then we might want to sell it now to avoid incurring losses in the future. So that's one obvious example. Another example is with sales. If we can predict the demand for a product, we can ensure that we have sufficient inventory to meet this demand. This type of prediction is especially useful in the field of operations research.

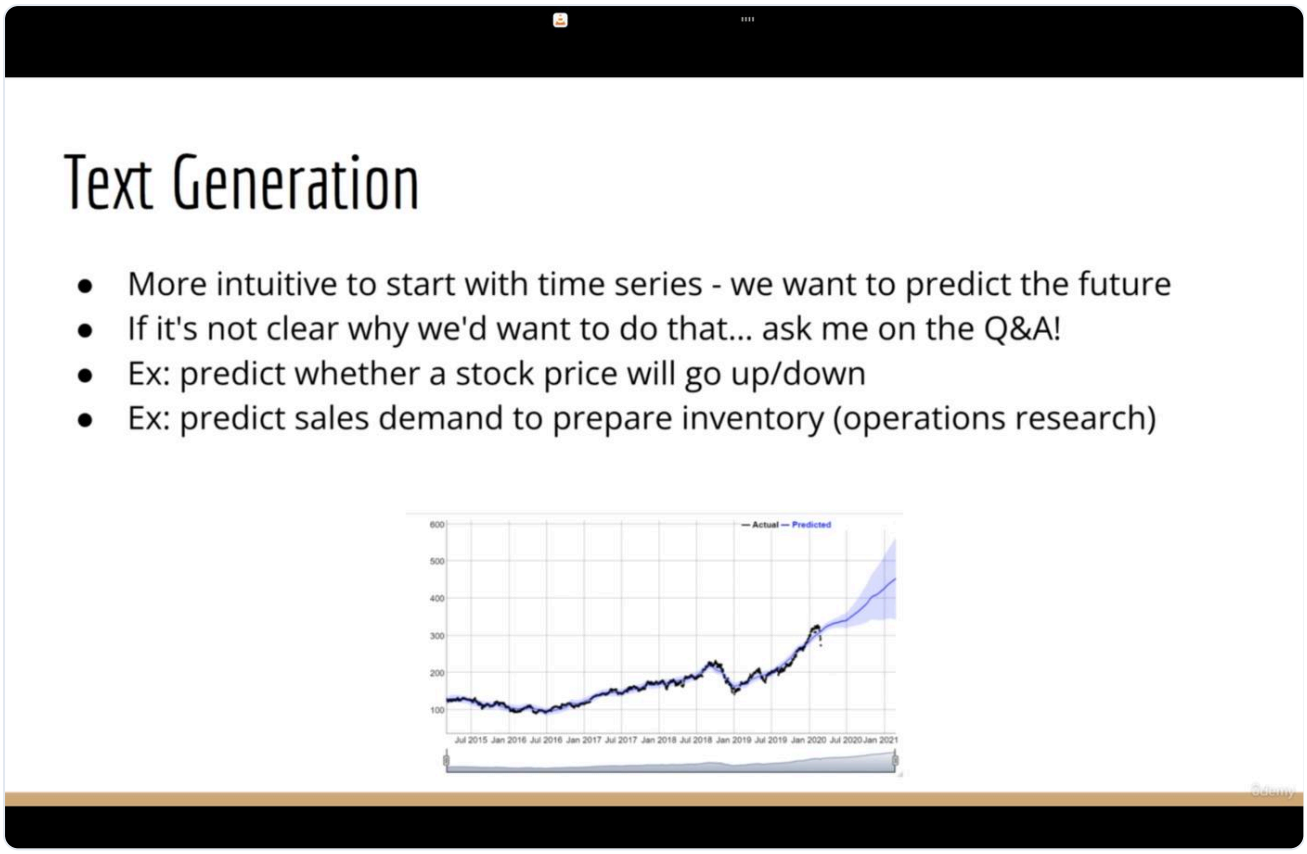

FIGURE 3.104 | Text Generation

So if you work in business, this should be another obvious example. Here's another obvious example that I think everyone should understand, and that is predicting counts related to COVID.

For example, the count of new cases per day or the count of new hospital admissions per day. And again, this is obviously useful so that our health care providers can ensure that we have sufficient resources to handle your health care needs. Okay.

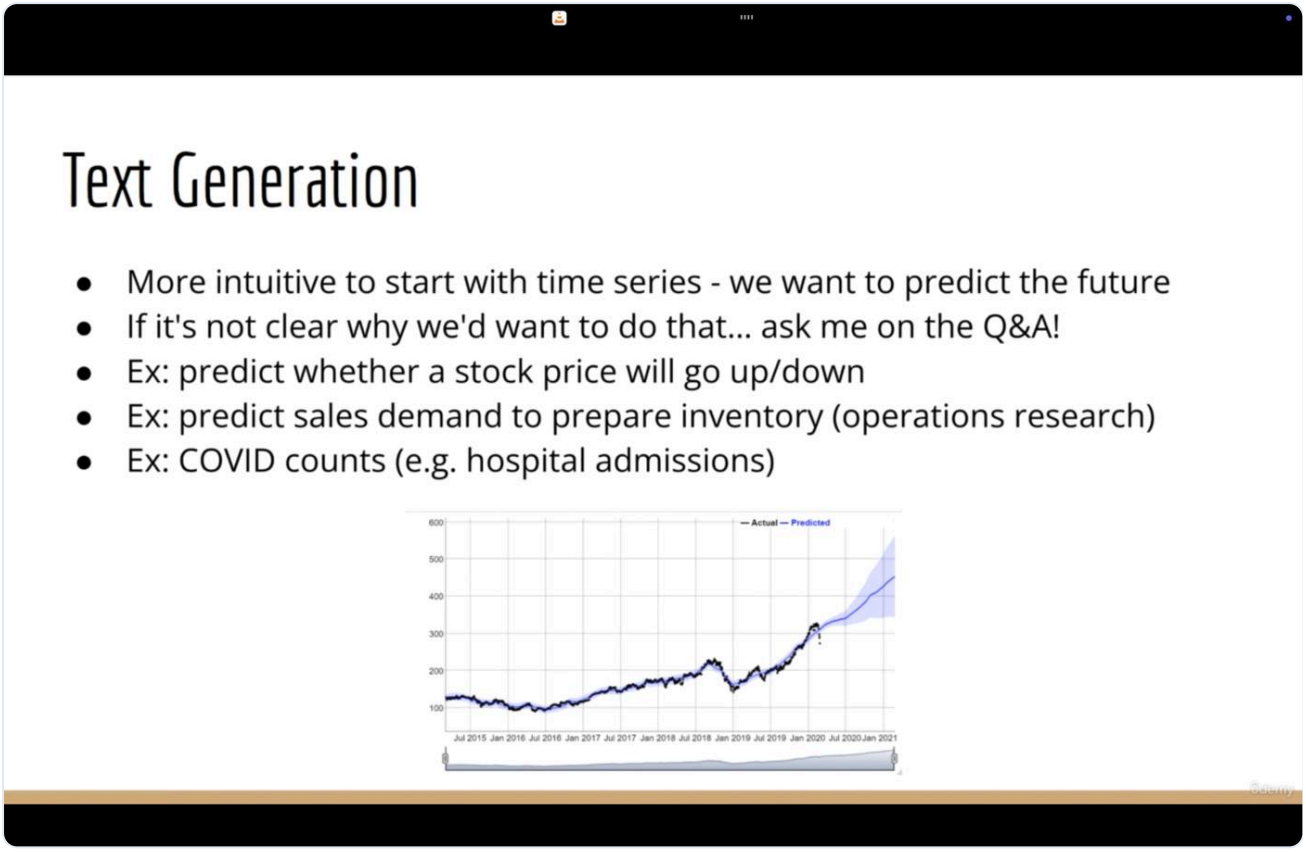

FIGURE 3.105 | Text Generation

So how do we build models to predict the future? The most Common Time series model is auto regressive. Now, in this context, regressive really is a double entendre. We're doing regression because we're predicting a number, but it's also regressive in the sense that we're looking into the past.

Auto as usual, means self.

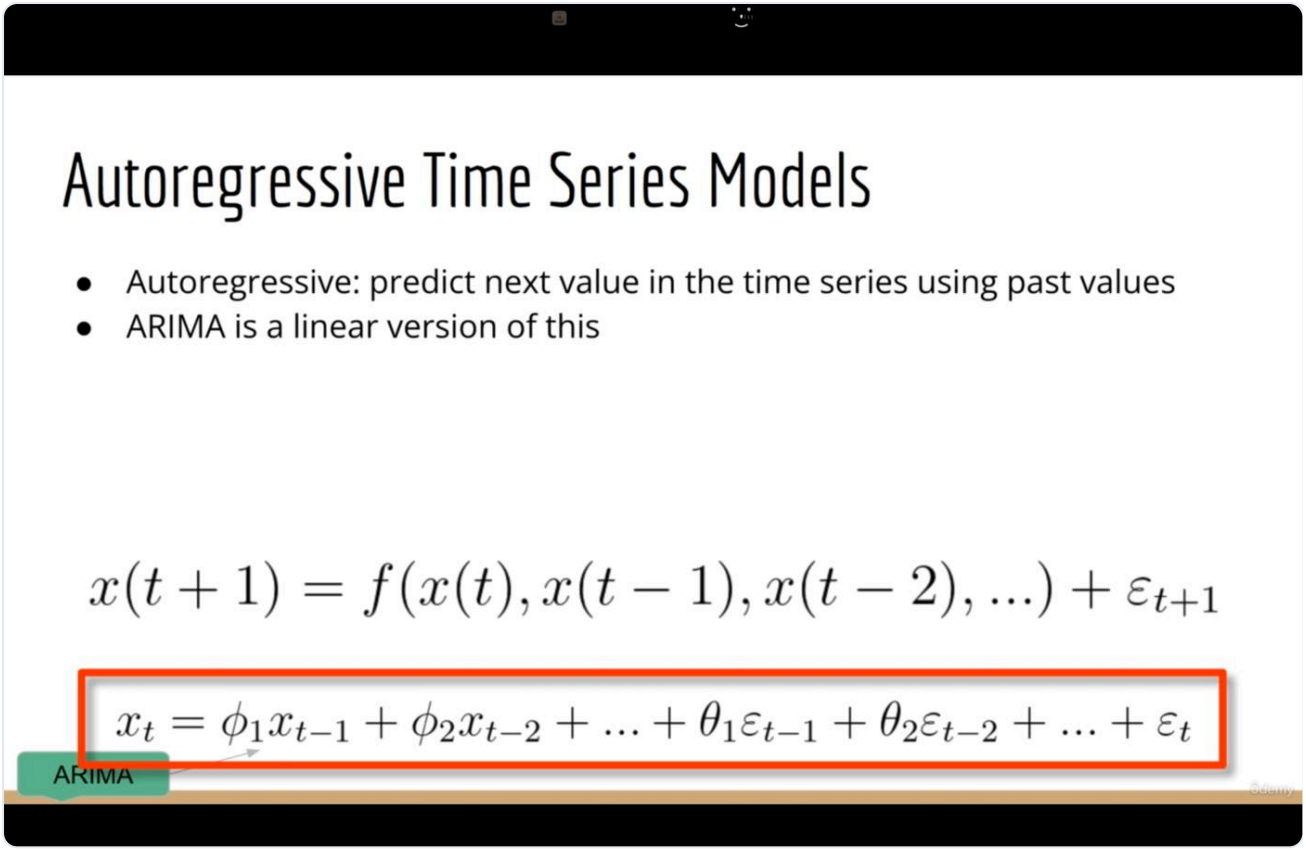

FIGURE 3.106 | Autoregressive Time Series Models

So in other words, our auto regressive model predicts the next value in a time series using previous values in the same time series. For example, I can predict X of t plus one using x of t, x of t minus one, x of t minus two and so forth. The classic form of this model is a linear model known as ARIMA. In this case, the next value in a time series is a linear function of the previous values and also the previous errors. Now, please don't worry if you find math intimidating, as it's not necessary to understand for this lecture, this is just for intuition only.

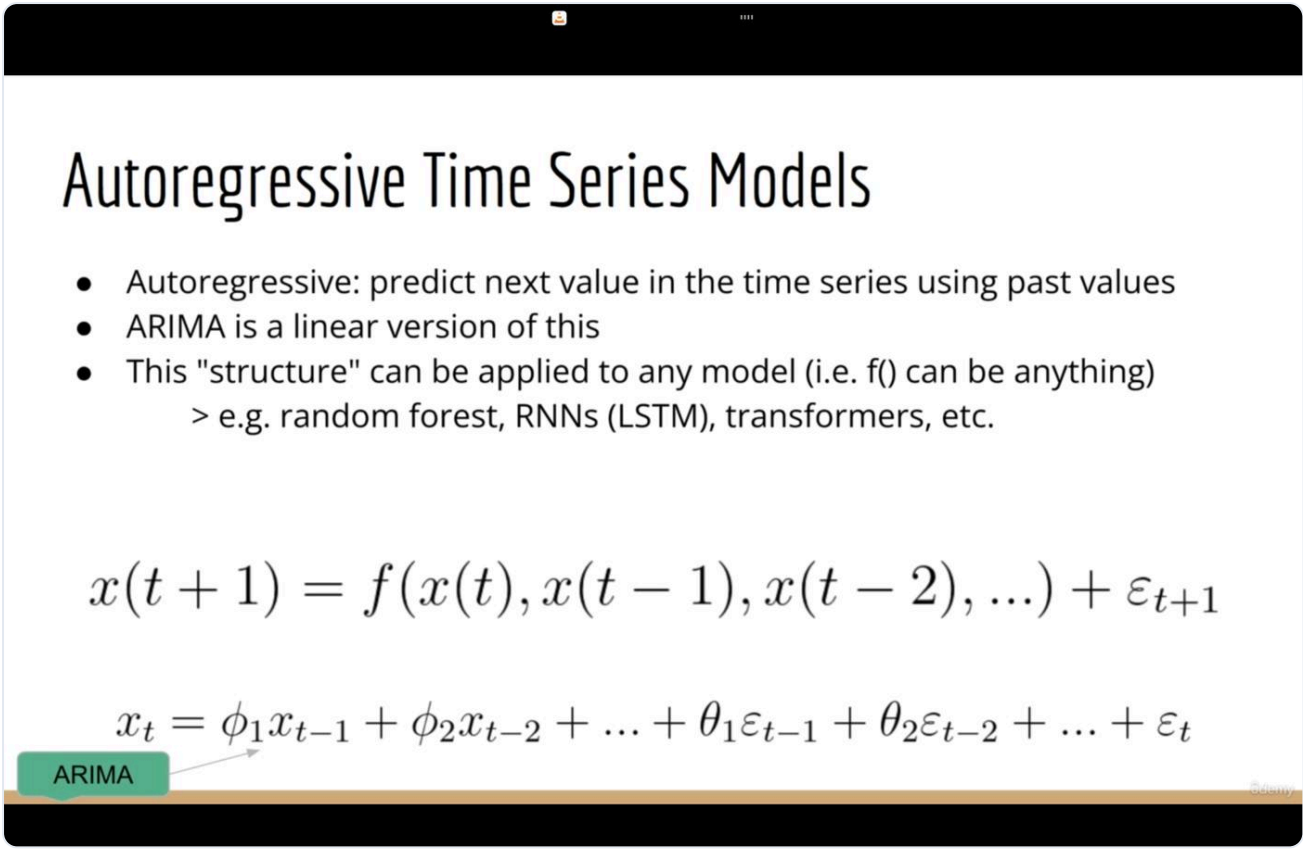

FIGURE 3.107 | Autoregressive Time Series Models

The point is that this kind of structure can be applied to other models as well, including the random forest. recurrent neural networks like LSTMs and Transformers. So really any kind of model can use past values to predict future values. And this is one of the main tasks of interest when studying time series analysis. Okay. So that makes sense for a time series.

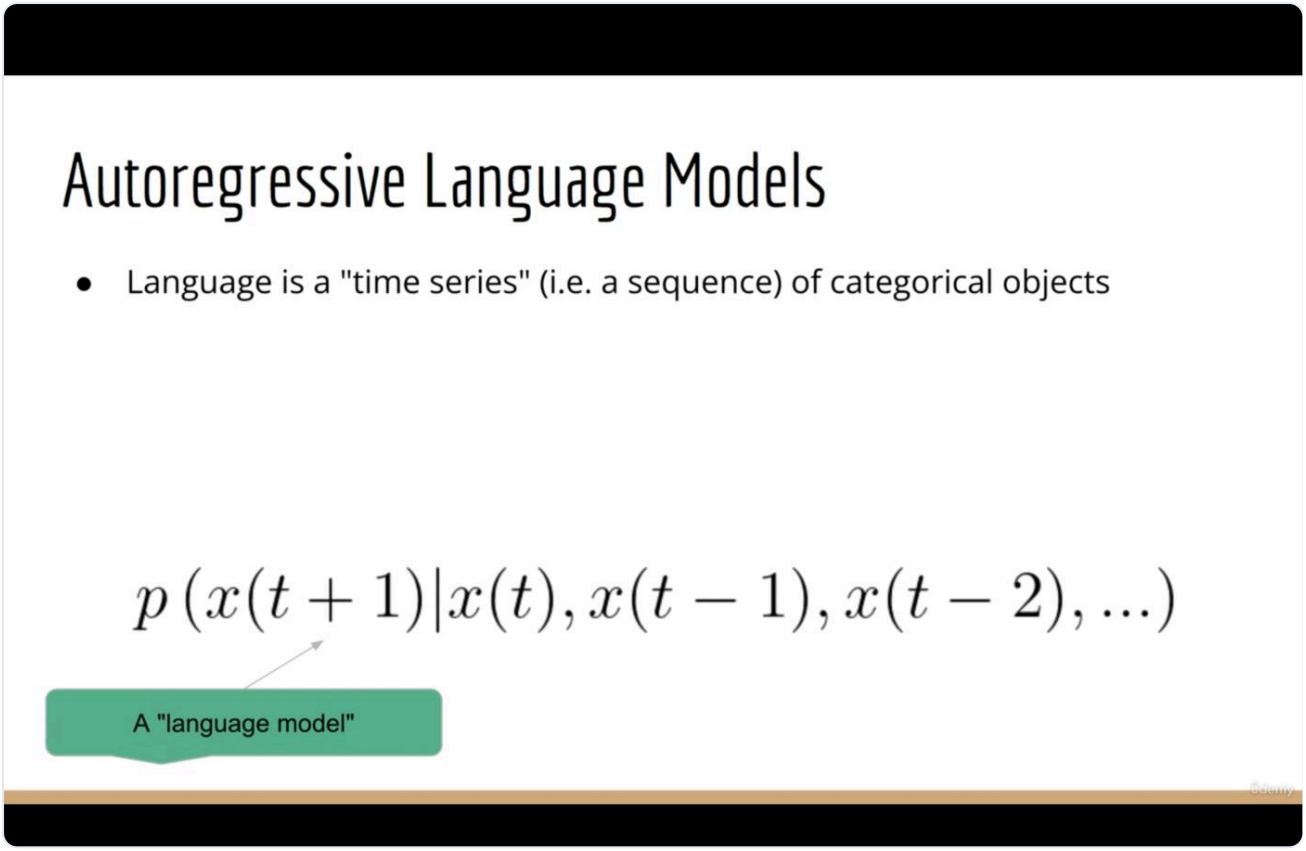

FIGURE 3.108

But what about for language? Well, language is simply a time series of categorical objects.

So it's not typically what we would think of as a time series, but it is a sequence and time series are sequences as well. Thus, we might expect that the same kinds of approaches might work well. In fact, this is the case.

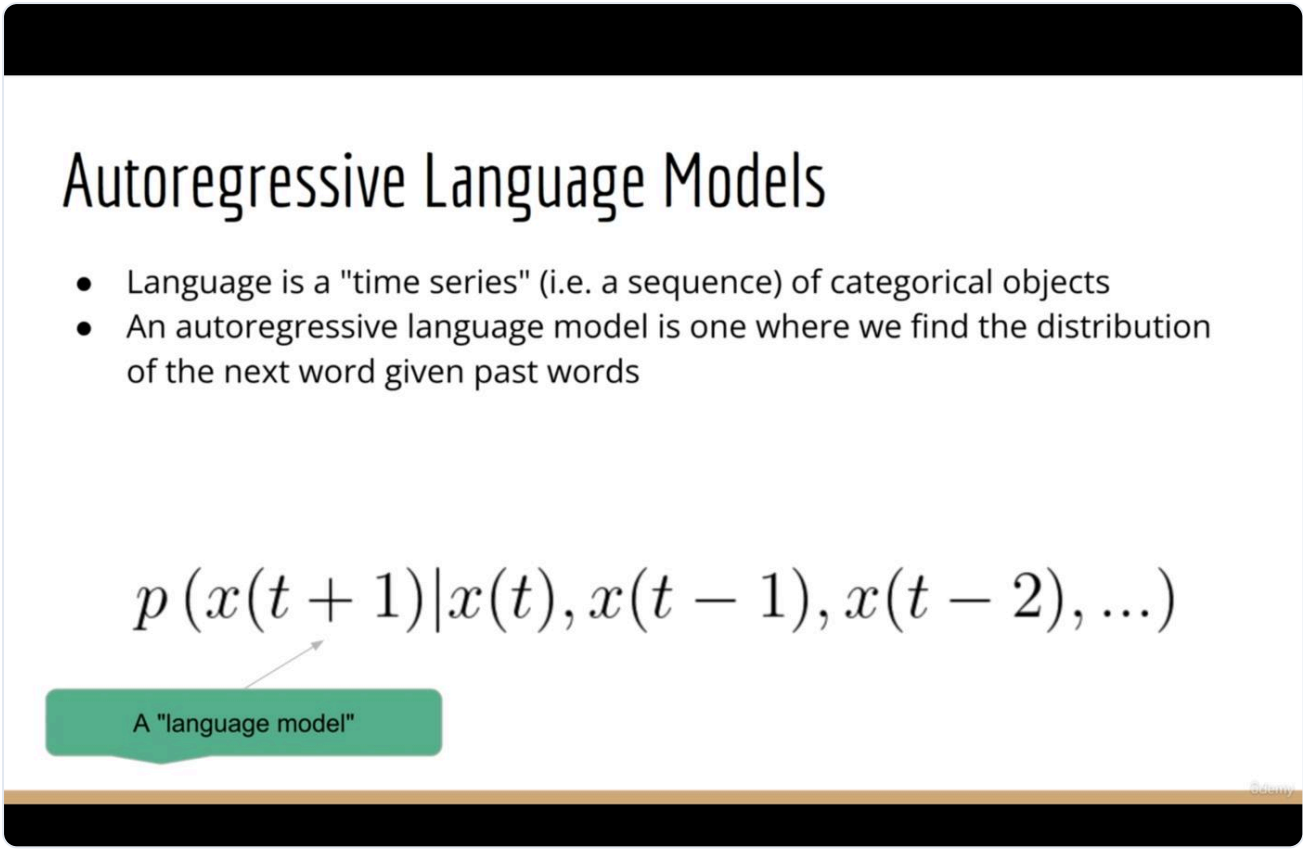

FIGURE 3.109

In particular, when we do this for language, we call it a language model. Our goal is to find a conditional distribution for the next word. Given the past words in a sequence, mathematically, we could write this says P of X of t plus one given x of t, x of t minus one, x of t minus two and so forth.

As you recall, such language modeling has a history going back over 100 years. Initially, Andrei Markov used Markov chains to describe language. In this framework, we use the Markov assumption, which is that each word or token depends only on the immediate past.

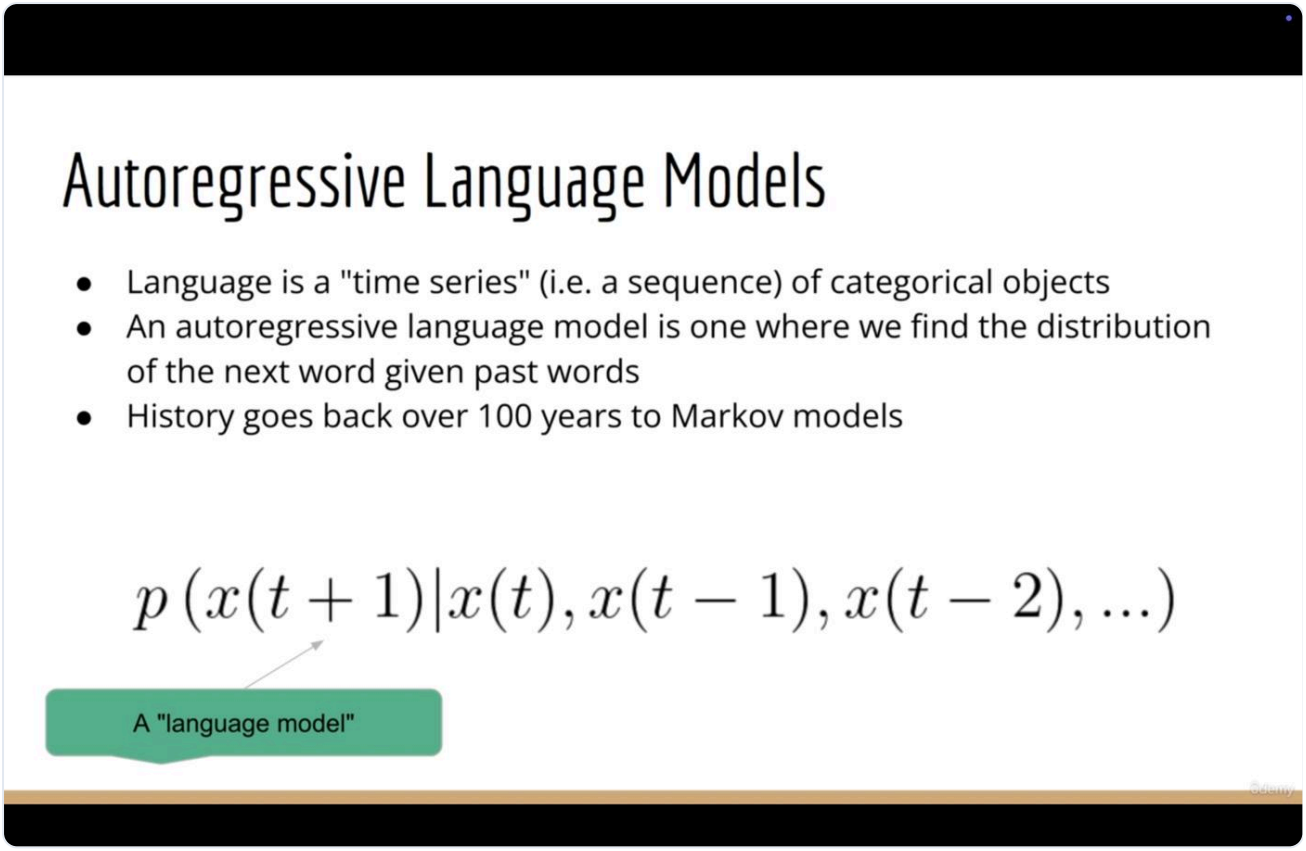

FIGURE 3.110

Mathematically, this means that X of T plus one depends only on x of t. In turn, x of t depends only on x of t minus one. This is mathematically convenient, but it's also such a strong assumption that the resulting models are not very proficient at more advanced tasks like generating coherent text.

For example, take the word great, as in that this was a great movie. So the word movie is a plausible continuation of a sentence containing the word great, but now take the sentence.

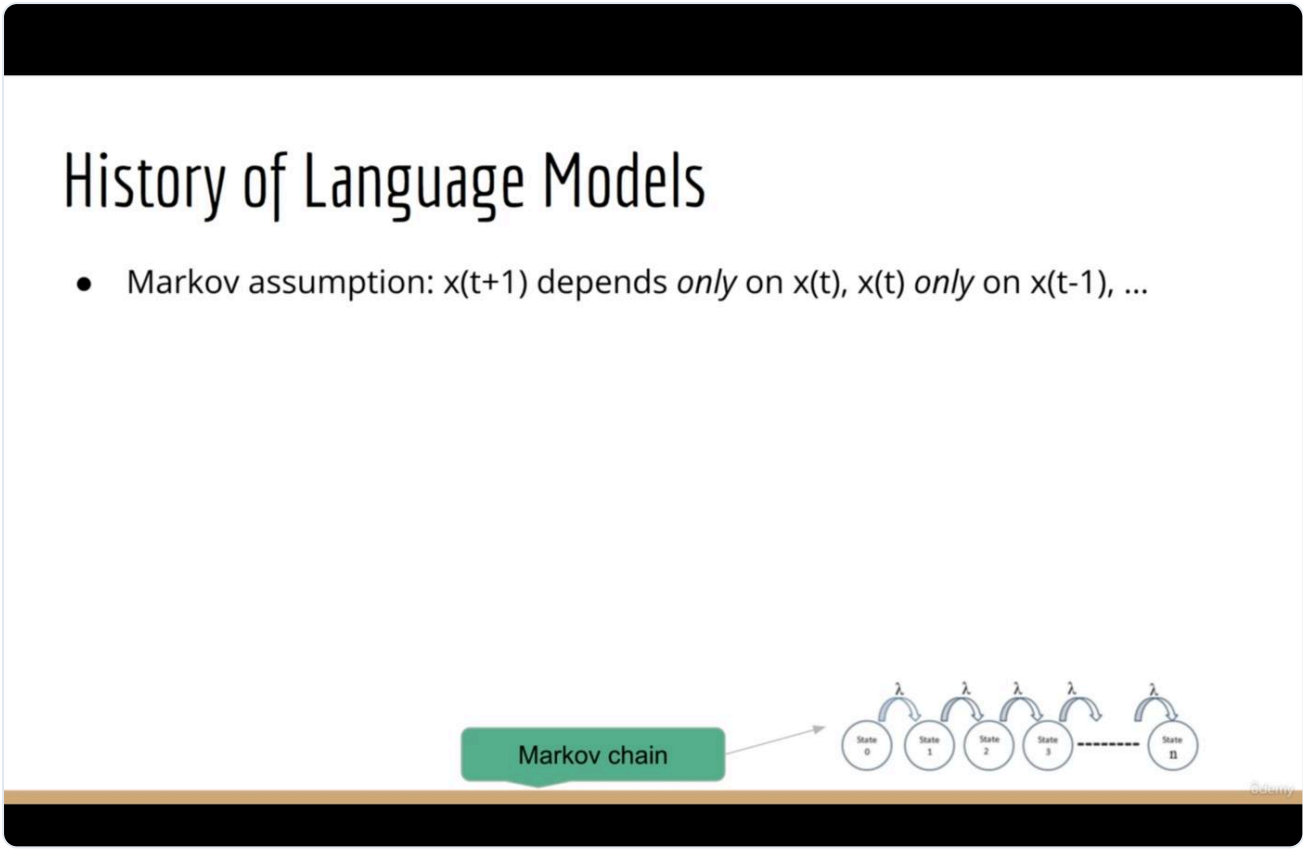

FIGURE 3.111 | History of Language Models

Today in school, we learned about Alexander the Great. Suppose that we had just generated the sentence and we were to use our Markov model to generate the next word. But our Markov model ignores all the past words and only considers the final word great. Because of this, the word movie is still a plausible continuation of this sentence.

So our sentence would end up looking like.

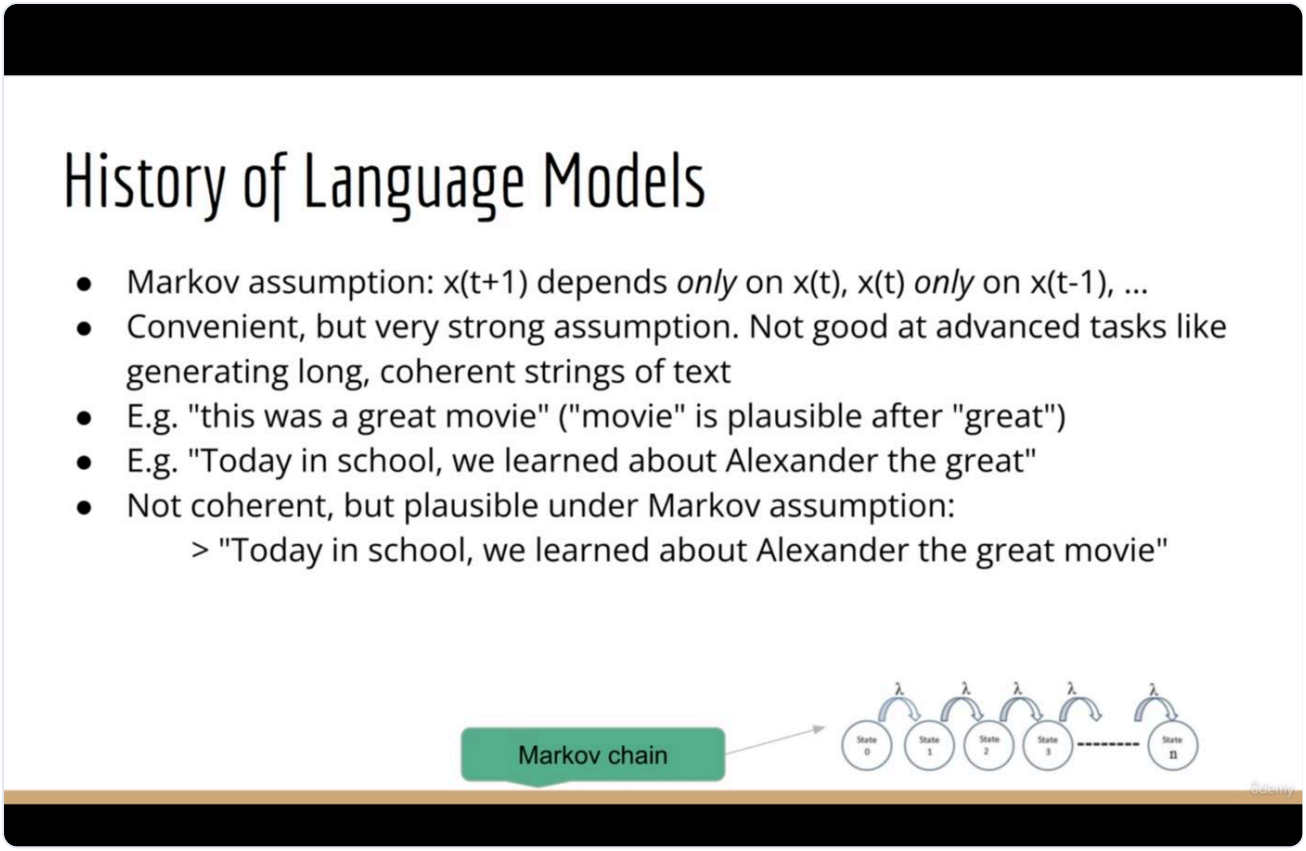

FIGURE 3.112 | History of Language Models

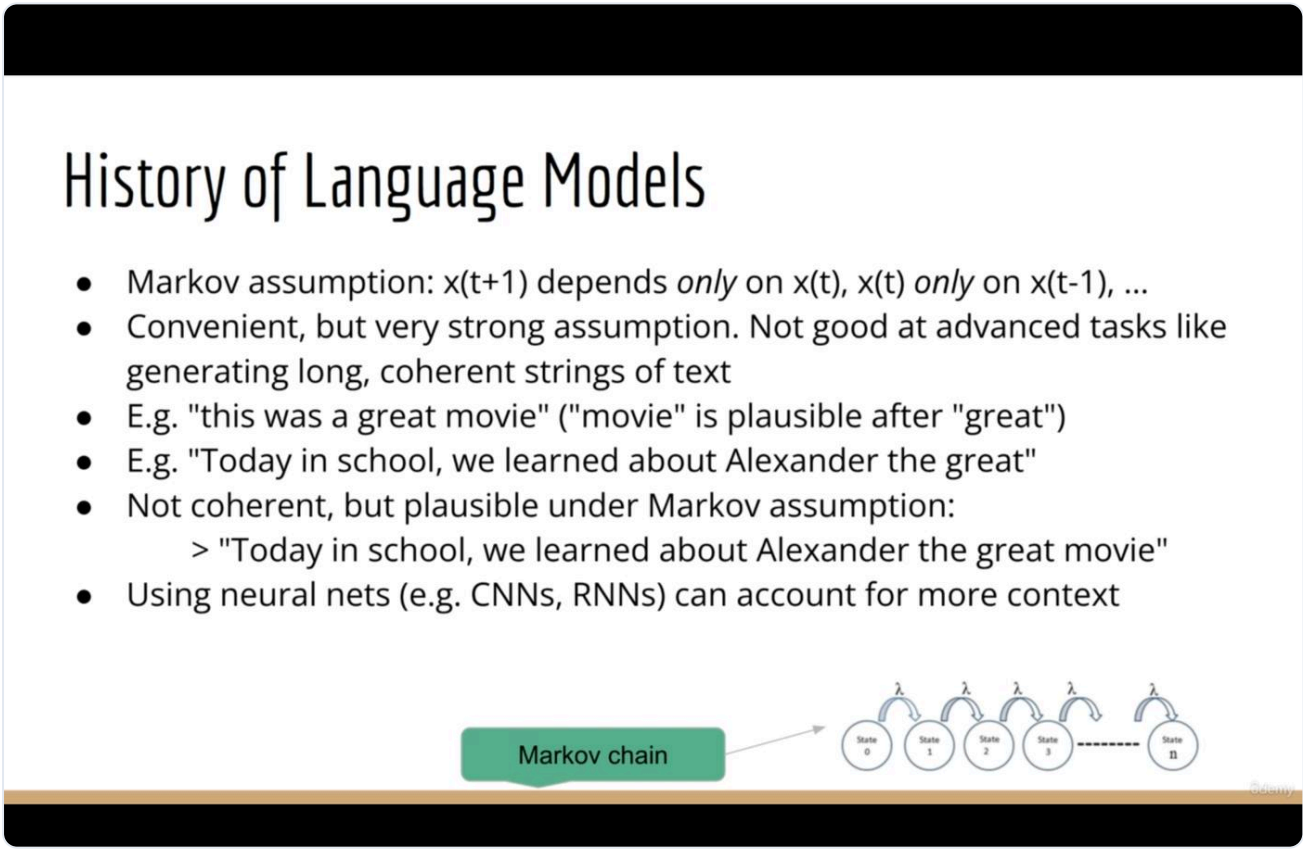

FIGURE 3.113 | History of Language Models

Today in school, we learned about Alexander the great movie, which obviously makes no sense. However, if we were to account for more of the past, then we might not make such a mistake. As you recall, recurrent neural networks in convolutional neural networks are two ways to include more context in a scalable manner. The reason why Markov models are not scalable is due to the curse of dimensionality.

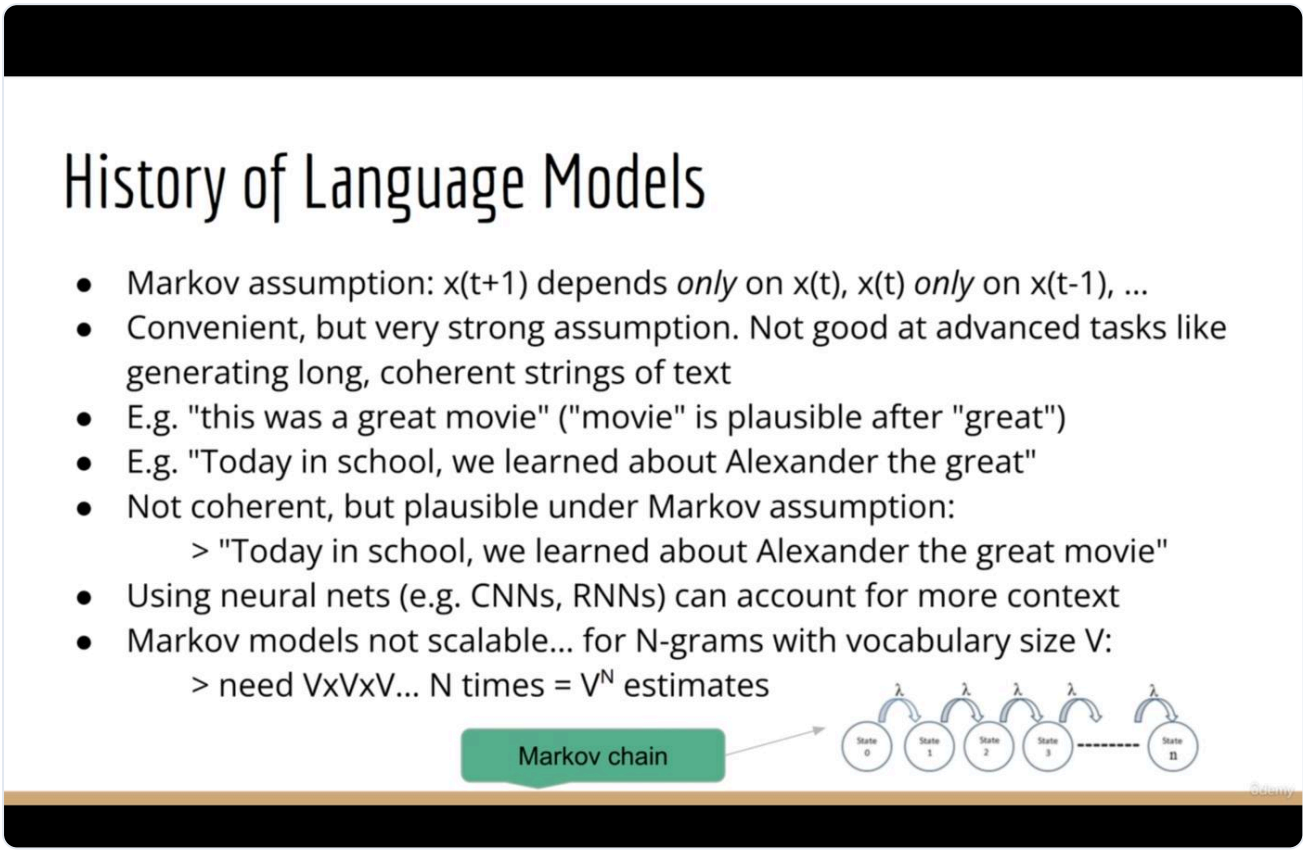

FIGURE 3.114 | History of Language Models

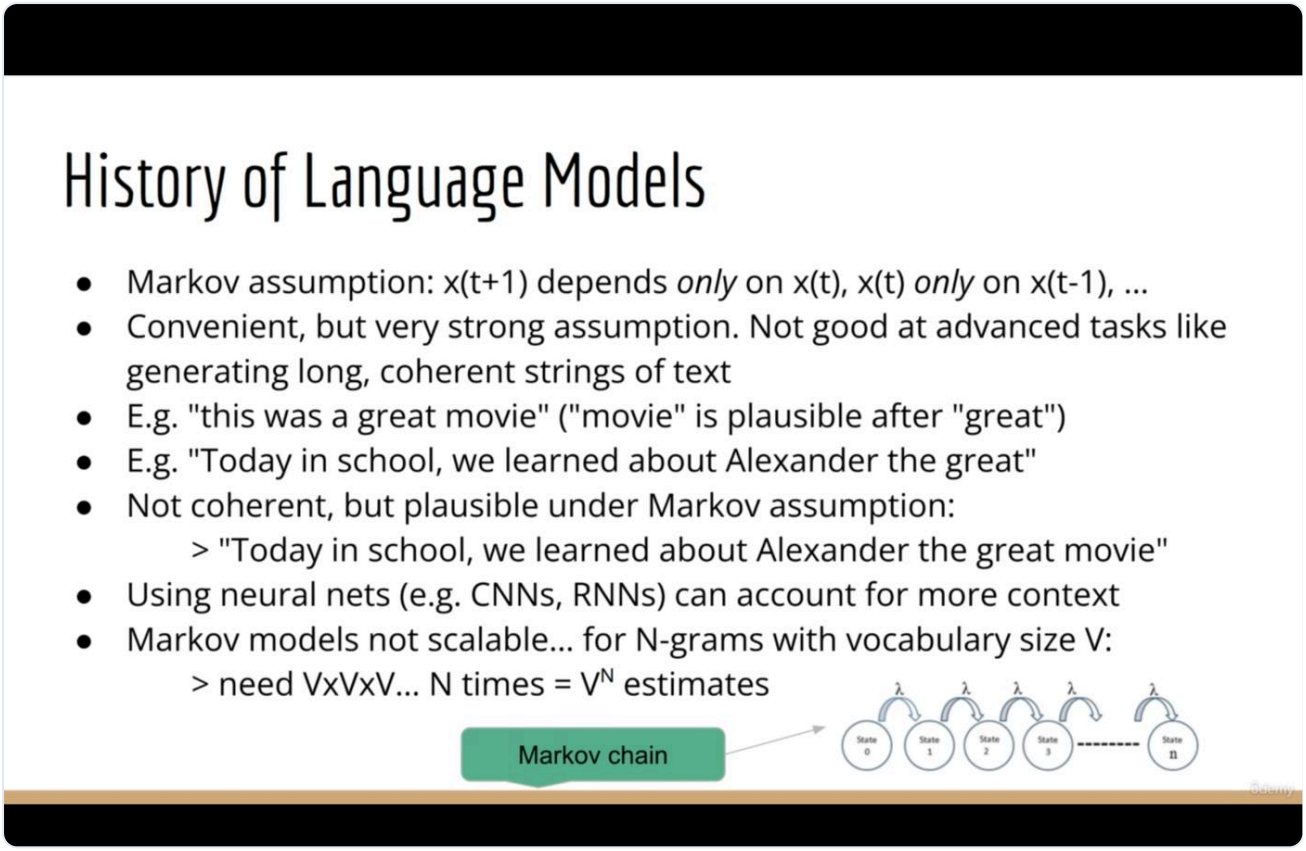

FIGURE 3.115 | History of Language Models

If we want to build a model for n-grams and we have a vocabulary size V, then the size of our distribution would be V, times v, times v and so forth and times.

In other words, v to the power n. Clearly, this is exponential, which, as you recall, is intractable. But by using neural networks, we can find more efficient ways to model such distributions. So in the past we've used these auto regressive language models to generate poetry. Usually this works okay, since poetry tends to be more accepting of strange or unexpected phrases. On the other hand, it's much more difficult to generate coherent text. In fact, even today, we cannot generate fully coherent text.

We're getting much closer than we used to be. But ultimately, our language models still make silly mistakes. That being said, that's not to say we shouldn't try. In fact, this still remains to this day one of the most important tasks in NLP. For example, the largest and most popular model today called GPT-3 is pre-trained using this method. So although it doesn't fully quote unquote work, you shouldn't take this as a sign that it's not something worth pursuing.

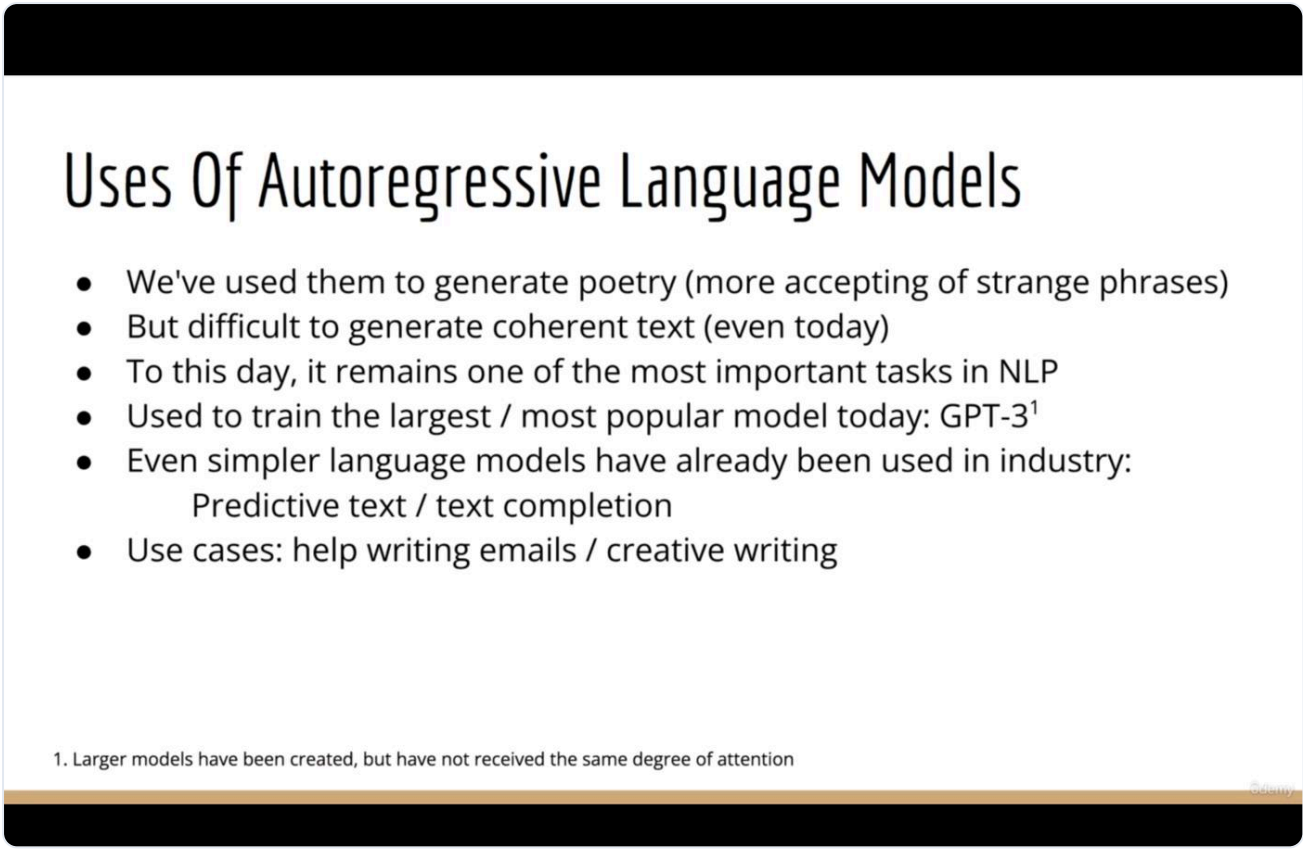

FIGURE 3.116 | Uses of Autoregressive Language Models

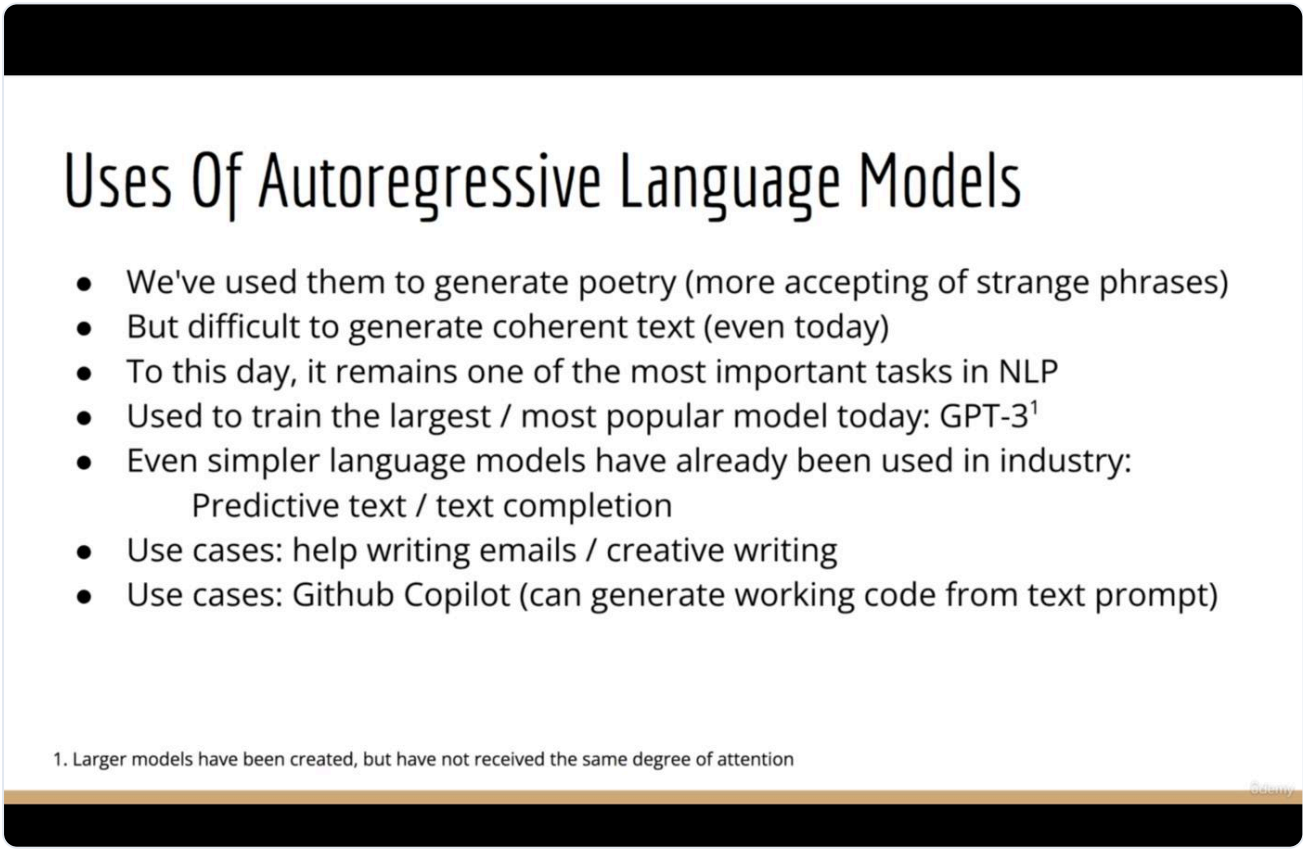

FIGURE 3.117 | Uses of Autoregressive Language Models

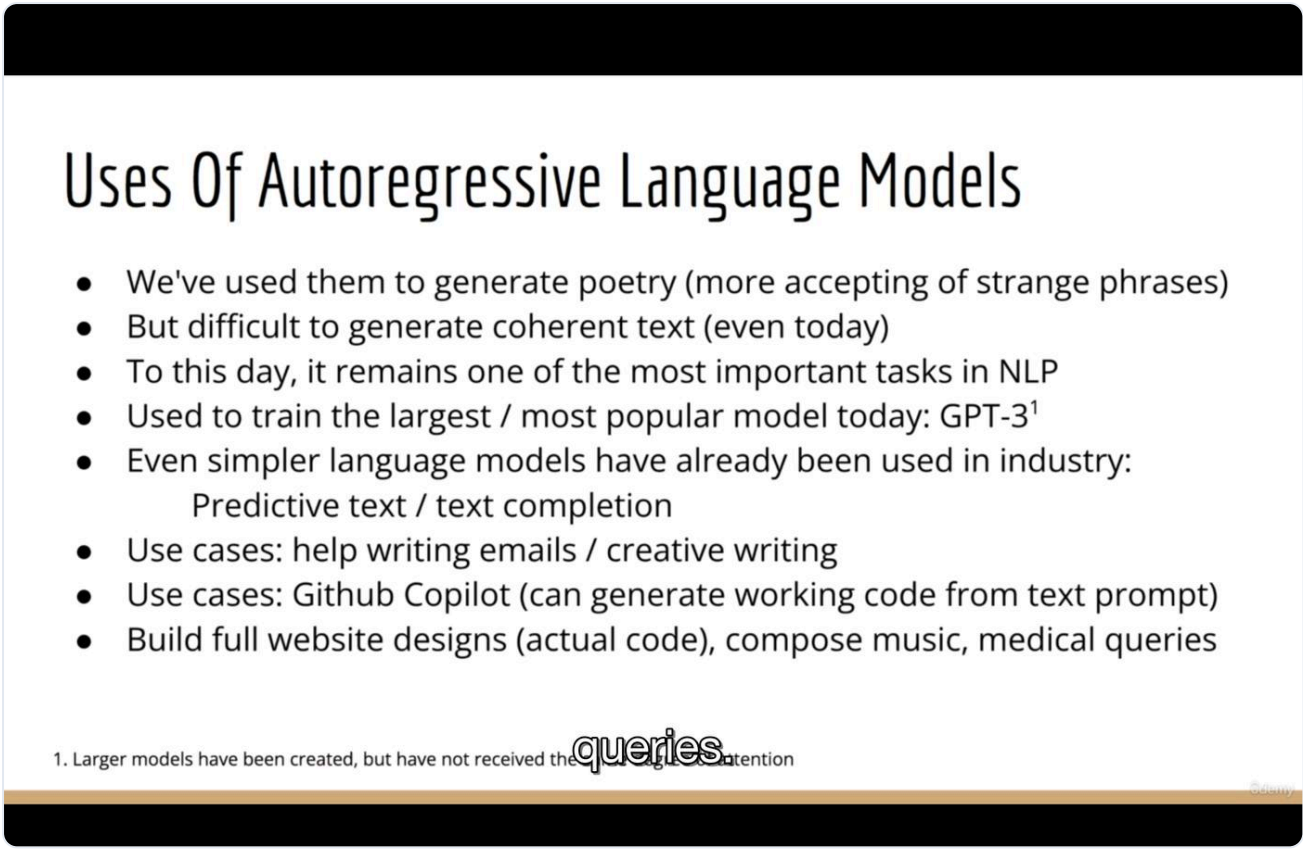

FIGURE 3.118 | Uses of Autoregressive Language Models

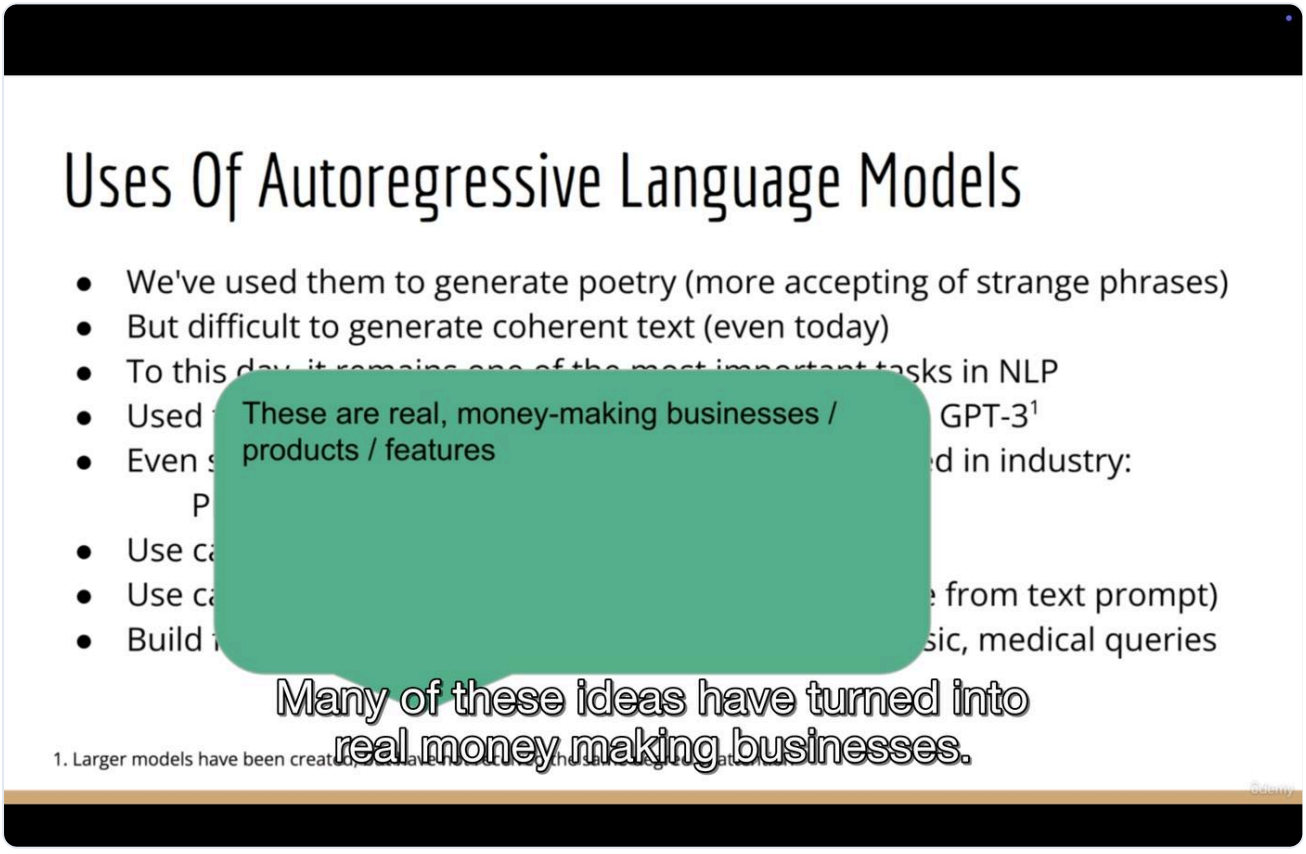

FIGURE 3.119 | Uses of Autoregressive Language Models

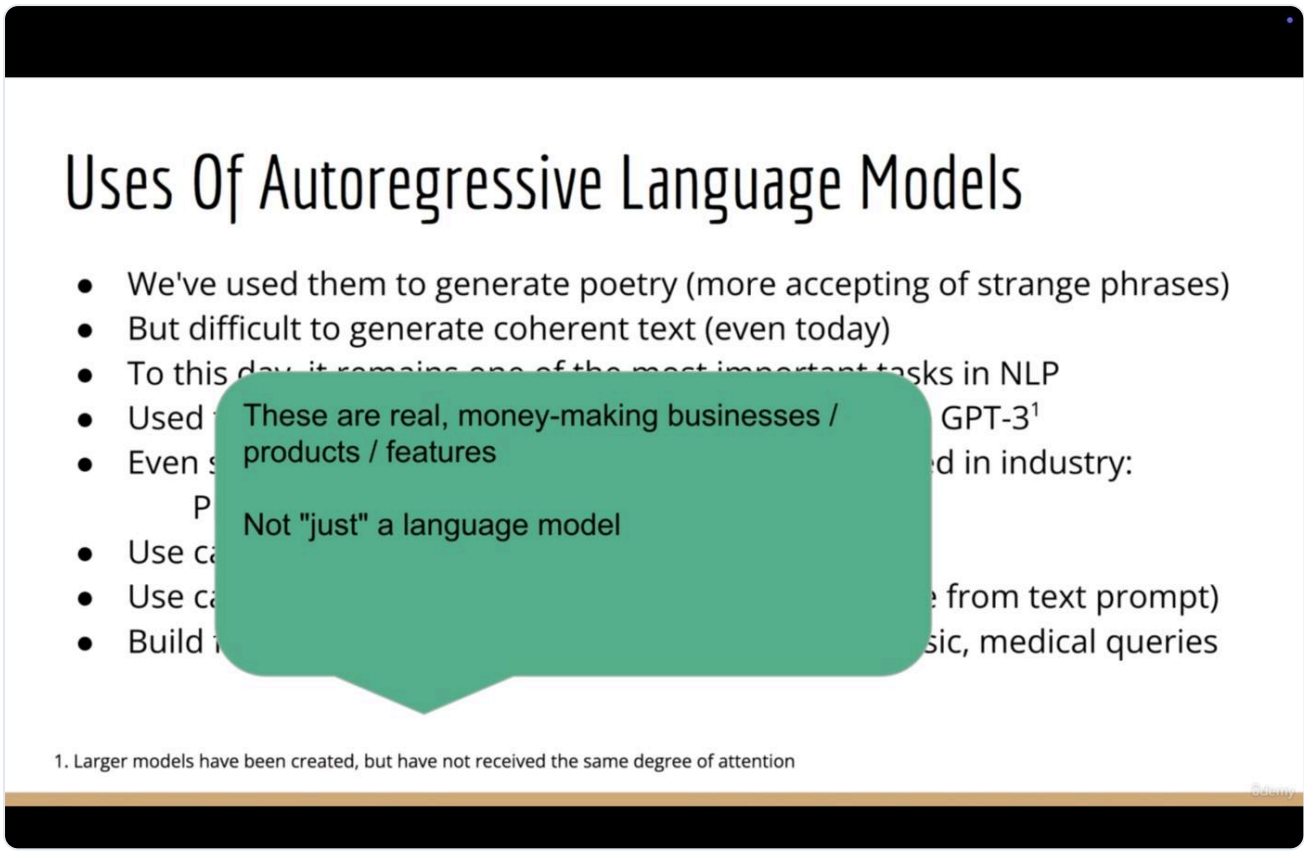

FIGURE 3.120 | Uses of Autoregressive Language Models

In fact, even simpler language models have been quite successful in a variety of applications, including text completion. So imagine you were writing an email and your email service predicts the next word as you type. I'm sure you've all seen this before. This is an example of a language model predicting the next word. Another example is with creative writing. If you're a writer, perhaps you sometimes have some trouble thinking of what to say. A language model can assist without any effort.

Thus, this is a way to augment your human capabilities. Impressively, GPT-3 goes even further. Some examples include a GitHub copilot which can automatically generate working code. So if you're a software engineer, this is again an opportunity to augment your capabilities with the machine. It's been shown that we can even build full website designs, compose music and even answer medical queries. Many of these ideas have turned into real money making businesses.

So if you think a language model is just a language model, now is the time to expand your horizons and to really see the full array of possibilities.

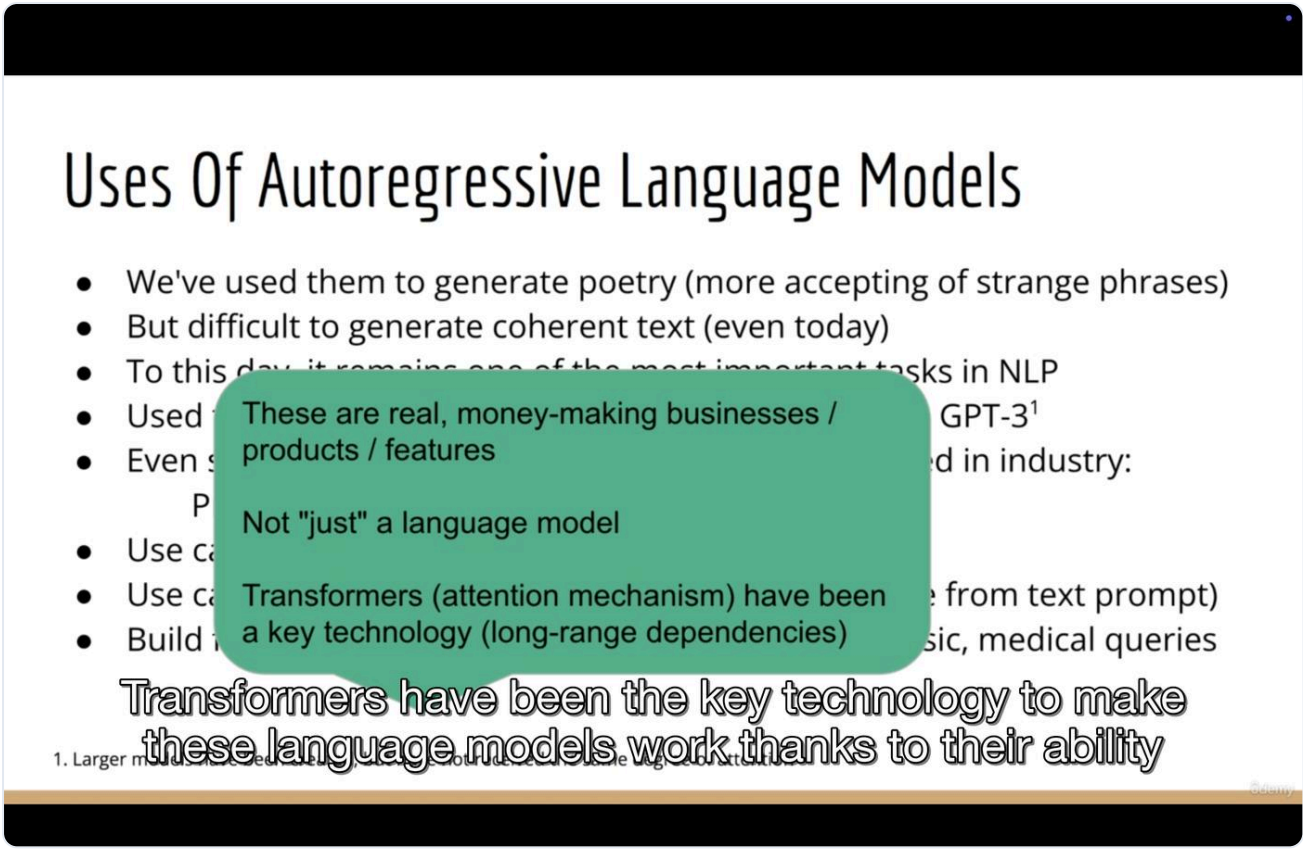

FIGURE 3.121 | Uses of Autoregressive Language Models

Transformers have been the key technology to make these language models work thanks to their ability to capture long range dependencies. As you recall, this is due to the attention mechanism where each output token does a full scan of all the input tokens to figure out which to attend to. Now there is one downside to this, which I think is quite amusing.

When the more advanced GPT models were invented, OpenAI was hesitant to share their pre-trained models. There were probably several reasons for that, but there is one interesting consequence.

Interesting Thought

- Recall: OpenAI was hesitant to share their pretrained GPT models
- One interesting "use-case": unethical website owners / marketers may use these models to fill their websites with incoherent, machine-generated text
- But language models are trained from text on the Internet
- This makes a loop (will we require a new training objective in order to "improve"?)

FIGURE 3.122 | Interesting Thought

Code Preparation

```
from transformers import pipeline
gen = pipeline("text-generation") # uses GPT-2
prompt = "Neural networks with attention have been used with
          great success"
gen(prompt)
>>> [{'generated_text': 'Two roads diverged in a yellow
wood, some of the highways carrying cars had lights that lit
up'}]
```

hugging face as usual.

FIGURE 3.123 | Code Preparation

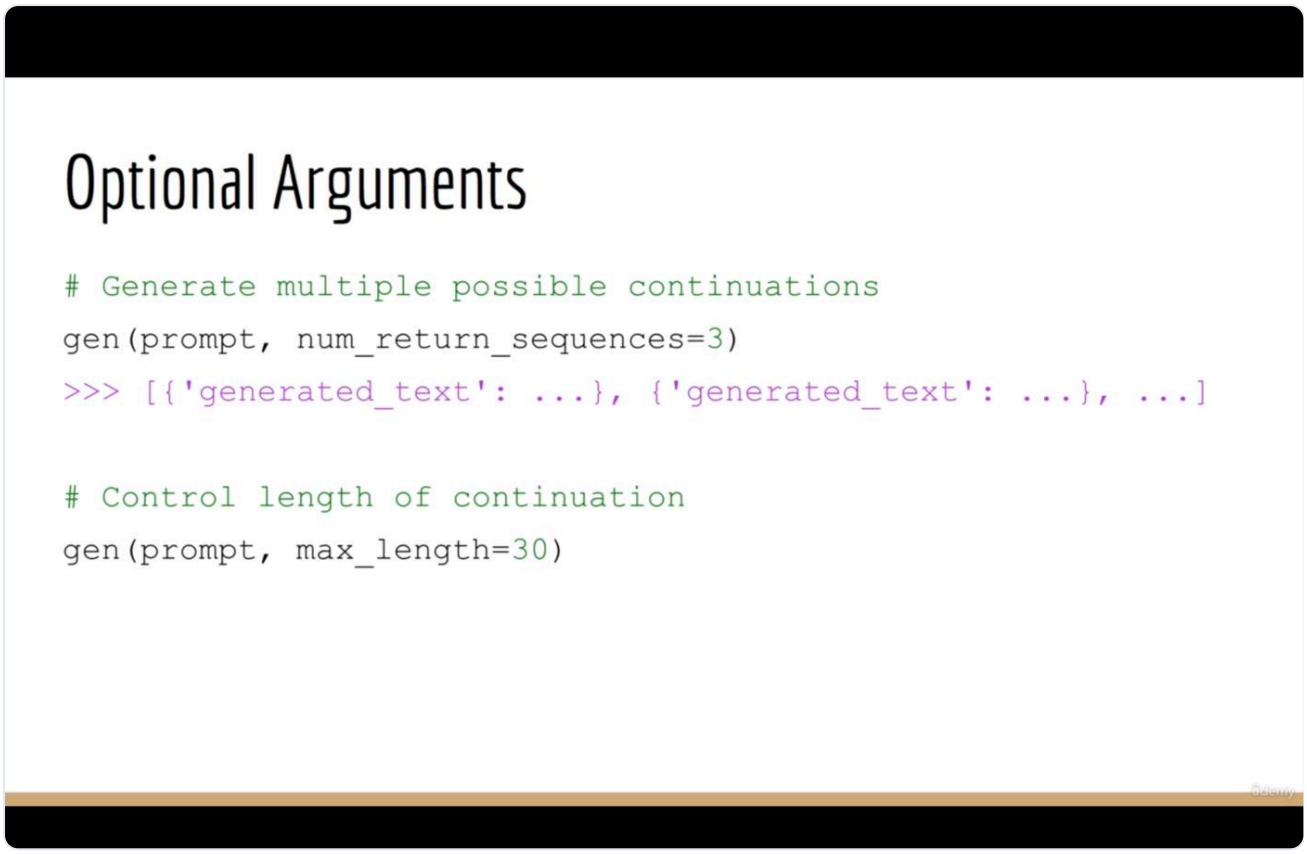

FIGURE 3.124 | Optional Arguments

Inevitably, website owners and marketers who perhaps lack some ethics will use these models to generate content full of incoherent machine generated text. The problem is these GPT models are trained on text from the internet. So what you end up having is this loop where machines are used to put text on the internet, but machines are trained on text from the internet.

So for at least some portion of the training data, the machines are just copying older versions of themselves. Whether or not that's actually an issue remains to be seen. Okay. So for the final portion of this lecture, we'll be looking at the code to do text generation using Hugging Face as usual. The first step is to call the pipeline function passing in an argument to specify the type of model we want. In this case, the string to pass in is text generation.

Note that by default this uses GPT-2. Currently GPT-3 is only available using an API. So once you have your pipeline, the next step is to feed it with the prompt, which is the text to condition on. After passing in this text, the model will auto-regressively generate more text. The format for the return value is a dictionary containing just one key called generated_text. Note that the pipeline accepts some optional arguments, including num_return_sequences and max length for num_return_sequences.

This will generate multiple possible continuations of your text, so you might want to set this number larger than one. In the case, you want to review multiple options. In this case, there will be multiple return dictionaries stored within a list. For max length. This controls the maximum number of tokens in your final text. So obviously you make this smaller if you want less text and make this larger if you want more text.

6 · Text Generation in Python

See the notebook here.

So in this lecture we will be looking at the notebook to do text generation and code for this lecture. I think it would be instructive to do poetry generation, as we have done in the past, to contrast these results from what you may have seen before. They should also help you understand what the model has learned and how that will affect what it will produce. So to begin, we're going to download a text file containing Robert Frost poems. The next step is to install Transformers, as we normally do.

The next step is to import the libraries we'll need in this notebook. The next step is to cat the Robert Frost text file to remind ourselves what's in it. Note that some lines are empty, which we might possibly want to account for. The next step is to load in our text file by reading each line and stripping off the whitespace. Now, since we found some empty lines, we're going to remove those lines by checking whether or not the length of the line is bigger than zero.

The next step is to call a pipeline function to get our text generation pipeline. As you can see, this will use GPT-2. The next step is to set the seed for the Transformers library. But feel free to turn this off for your own use. The next step is to print the first line in our list of lines, which is what we'll be using as a prompt. So it says two roads diverged in a yellow wood. The next step is to pass this line into our pipeline to see what we get back.

Now, unfortunately, this returns a lot of text that goes off the screen. So the next step is to use the P print function, which will print the above in a more readable format. So this says two roads diverged in a yellow wood, while the road in a white puddle was a complete blue on account of the rain coming in. It had been raining outside for some time, so it hadn't taken long for people on both sides of. So these results are interesting and reflect what we see later as well.

Basically, this does not seem like poetry, which makes sense because this model was trained on text from the internet rather than text from poetry. So the model is not a model of poetry, but just generic text. Note that, as one might expect from a language model, the results are grammatically correct, but it obviously doesn't make a whole lot of sense. For example, it says that there is a road in a white puddle, which does not make sense. You'll also notice that the previous text was quite long, which is not really expected if you're writing a poem.

So for the next example, we're going to set max length to 20. Okay. So this time the result is two roads diverged in a yellow word. Some of the highways carrying cars had lights that lit up. Again, the results are pretty much grammatically correct, but not sensible. Also, notice how the sentences just end arbitrarily. So using a model like this would still require a human to assess how the text should be used in which parts to discard. So you'll notice that for the previous two examples, even though we passed in the same prompt, we got back a different result.

As you recall, text generation is stochastic since we're sampling from a probability distribution. One thing you may want to do in the real world is to generate multiple samples from the same prompt and to choose the best one. To do that, we simply need to pass in the argument

num_return_sequences. So you can see that we've set this to three. Okay. So as expected, we have three results. Two roads diverged in a yellow word where someone threw a stone at the front passenger door.

Two roads diverged in a yellow wood with three different ways to cross the bridge, including three. Two roads diverged in a yellow wood, but the main lines were cleared and no one in there. Okay, so again, all these three results make sense, but are definitely not what we'd consider poetry. The next step in the script will be to try to write a multi-line poem, even though it's not likely to look like much of a poem. Obviously, you could still apply this kind of approach to any kind of text so the code wouldn't be different.

For this example is the principles which are more important to pay attention to. So we'll begin by writing a helper function to wrap our text. Inside this function, we simply call the textwrap.fill function, which has some additional arguments to help with formatting the text that we want to print. The next step is to generate some text based on the first line of our Robert Frost file. The basic idea is we're going to use every other line in this file.

So lines one, three, five and so forth will come from a real Robert Frost poem. Every other line, meaning lines two, four, six and so forth will be generated by GPT-2. Note that have set the max length to 30, which is a bit longer than we might want. But remember that we can simply keep the useful parts and discard the rest. So the result is two roads diverged in a yellow wood, including one that blocks the road leading to another intersection in the middle of the city.

The cars that were. Okay. So since the last sentence is incomplete, will discard that part. In the next block of code, you can see that I've created a string called Previous which contains everything we've generated so far, including the actual Robert Frost line and the line generated by GPT-2 as input to our pipeline. We're also going to add the next line of Robert Frost, which is the line at index two. This time we'll set the max length to 60. Okay.

So the new part of our poem is and B, one traveler long I stood in front of the burning wreckage. That's what I had written on Twitter a few minutes ago. The city's fire. Note that the original Robert Frost part is just and b one traveler long. I stood, and the rest was generated by GPT-2. So for this will keep everything up to a few minutes ago. Okay. So for the next block, we'll keep repeating the same pattern where we include everything that we consider part of our poem so far.

Again, we'll append the next line of the poem, which is at index four and will increase the max length to 90. Okay. So the new parts are as follows. So where it bent in the undergrowth, to the deep valley, to inanimate non-human corpses of people in a state of collapse and others whose bodies. Okay, so pretty grim poetry by GPT-2. Now obviously this is not the kind of model you'd want to use to generate poetry unless you did some further fine tuning on actual poetry.

So that what the model generates looks more like poetry and less like generic text. That being said, it should be quite easy to think of some creative applications of this model as is. As an exercise, I encourage you to think about how this model could be used in the real world. I don't want to poison your thoughts by giving you ideas, since that might restrict you to only what I've given you. Instead, I encourage you to be creative and really think about how to apply this.

For the next example, I want to show you a more typical way that GPT is used. We've already seen that it's not too great at poetry, but this is simply because it wasn't trained on poetry. But I feel that this does a disservice to GPT. Since you don't really get to see its full capabilities, the more common way to use GPT is to pass in a prompt about some subject and then to observe that it's able to generate very coherent text with very long range dependencies.

Basically, we can generate a whole document where the end is still clearly has some relation to the beginning. So for this example, I've set the prompt to neural networks with attention have been used with great success in natural language processing. Now let's see what GPT-2 comes up with. Okay. So this is very impressive. Neural networks with attention have been used with great success in natural language processing as the prompt. On a recent project in which we worked with Kim Sanchez, we discovered that neural networks can improve the accuracy with which they are linked using multiple visual cues.

Here we show that we can apply neural networks to a natural language in order to apply attention and to identify the semantic structure of language with minimal effort. Furthermore, we demonstrate that it is possible to translate simple sentences into sentences using this approach. This is the first study to investigate semantic content using neural networks. And then it lists a bunch of authors. And then it says synthesis and characterization of semantic content using a small, caffeine based neural network.

Cerebral cortex. So this is meant to be a paper title in journal name, most likely. So basically the model interpreted our prompt as being the start of perhaps a paper abstract. Now the text doesn't perfectly make sense. For example, it says Translate simple sentences into sentences. Which is weird. And it says that neural networks can improve the accuracy with which they are linked using multiple visual cues, which makes absolutely no sense, but sounds fancy.

But grammatically, everything seems to be perfect. It's also far better than anything we've done in the past. Which is why this was so exciting when it first came out. It's also pretty impressive that the model understands the structure of a paper abstract, with author names being initials followed by last names. And then the paper title and journal name. Also, notice that the whole thing is coherent from start to end. So the first tokens are neural networks, while the final tokens are cerebral cortex.

So there's definitely a connection between all of the tokens in this document. And this is far better than what we've been able to do with sequential models like LSTMs.

7 · Masked Language Modeling (Article Spinner)

In this lecture, we'll be looking at another task very closely related to the previous one on generating text. To recap, the previous lecture looked at an auto regressive language model, also known as a causal language model. This is because our model learns a distribution of tokens conditioned on the past.

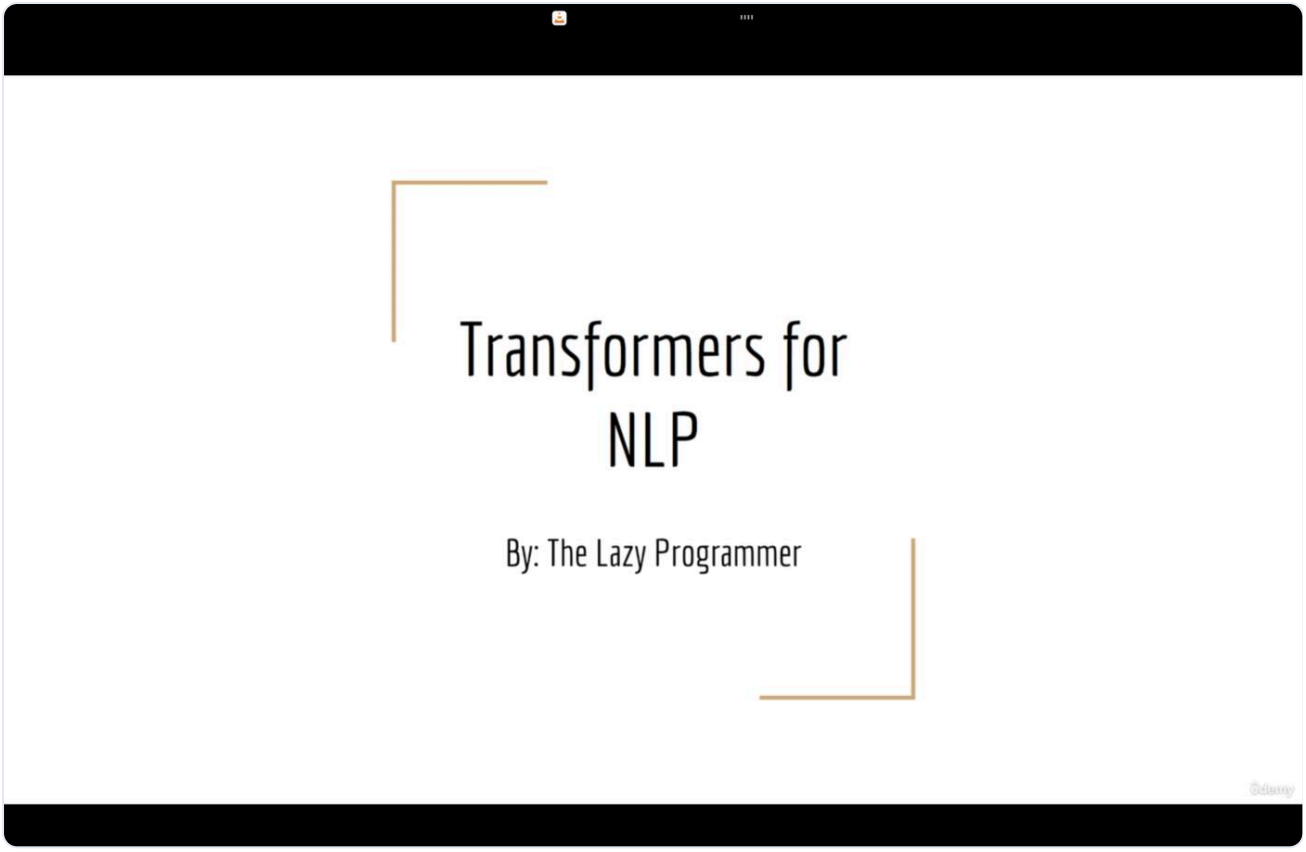

FIGURE 3.125 | Transformers for

So we want to predict X of T plus one given X of T, X of T minus one and so forth. Importantly, this only goes in one direction, which is forward.

In this lecture, we'll consider a slightly different model, one that goes in both directions.

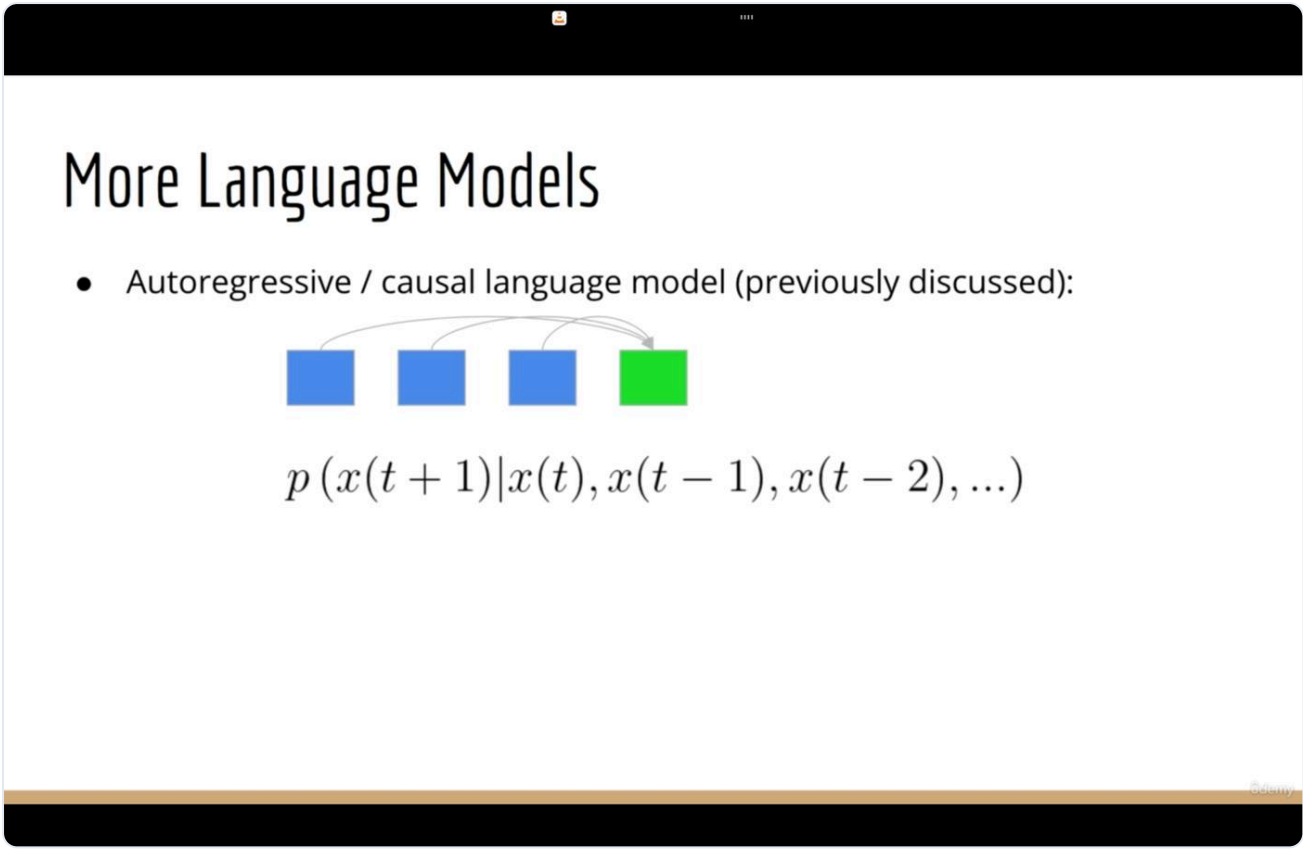

FIGURE 3.126 | More Language Models

In particular, suppose I build a distribution to predict X of T given the past and also given the future. So, for example, I conditioned on X of T minus one, x of t minus two and so forth, but also x of t plus one x of T plus two and so forth. This is called a masked language model for reasons which will become obvious shortly.

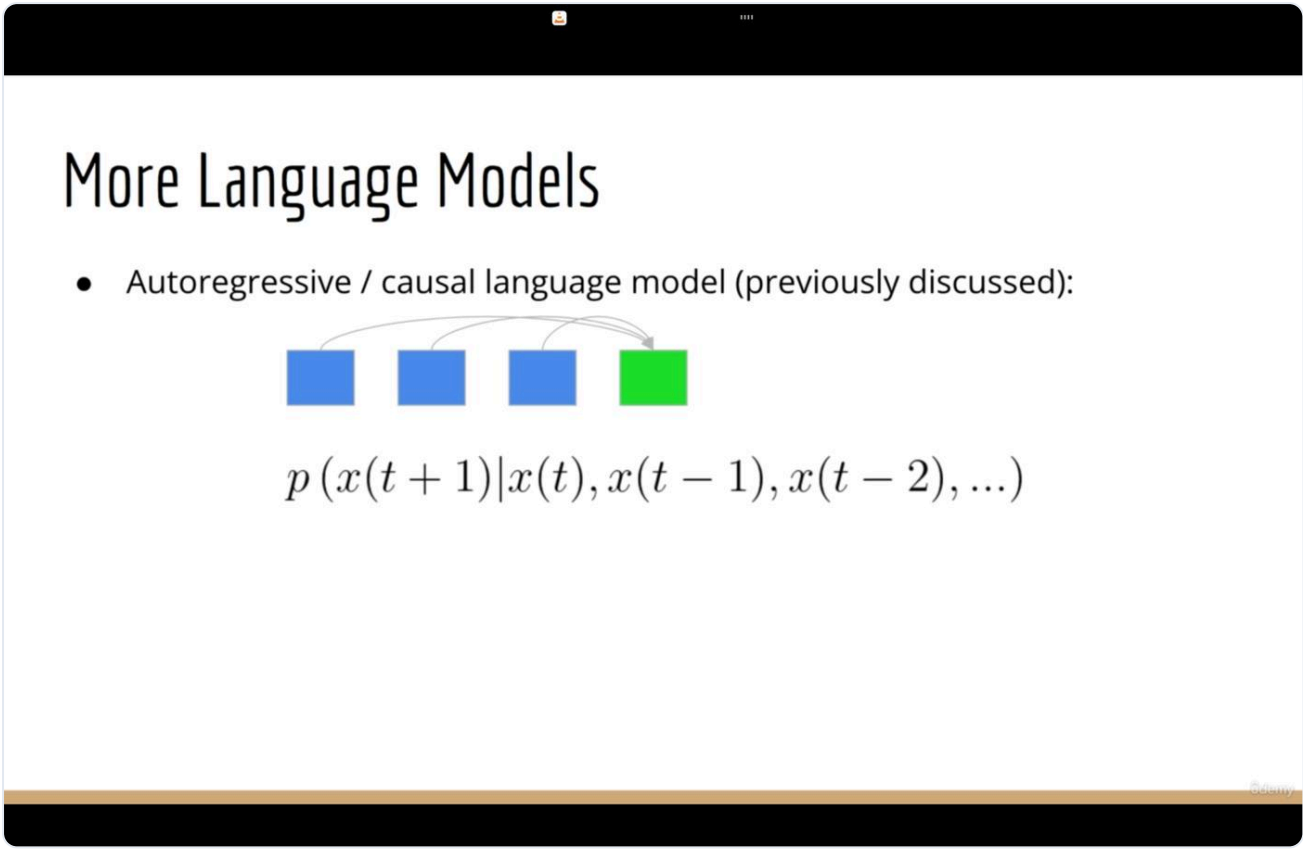

FIGURE 3.127 | More Language Models

So at this point it should be mentioned how these methods of pre training language models are related to actual specific models.

We use in particular at the auto regressive language model is the method used by GPT. That's why, for example, the default chosen in the Hugging Face pipeline was GPT-2.

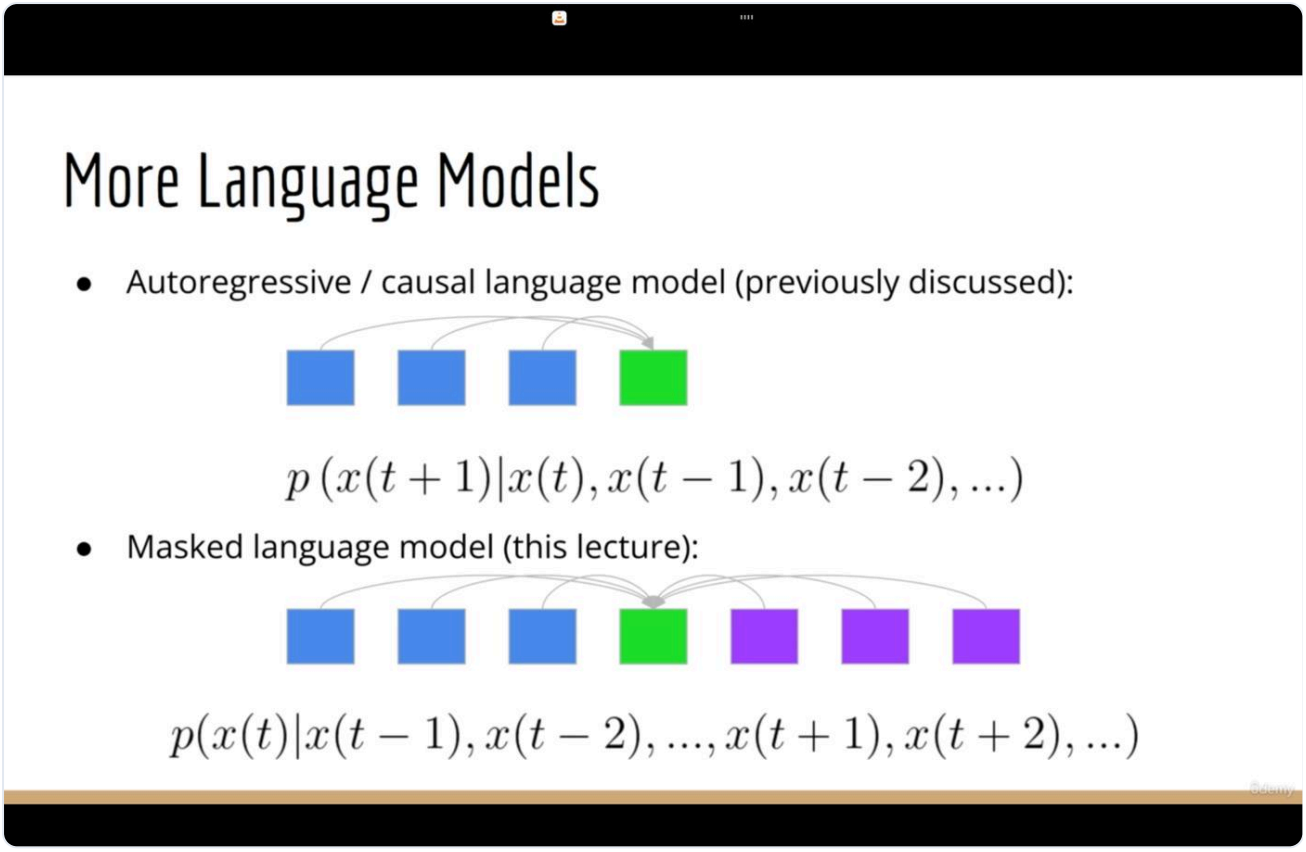

FIGURE 3.128 | More Language Models

On the other hand, the masked language model is the method used by BERT. BERT stands for bidirectional encoder representations from Transformers. The key word here is bidirectional, meaning that the model learns how to predict words from both the past and the future.

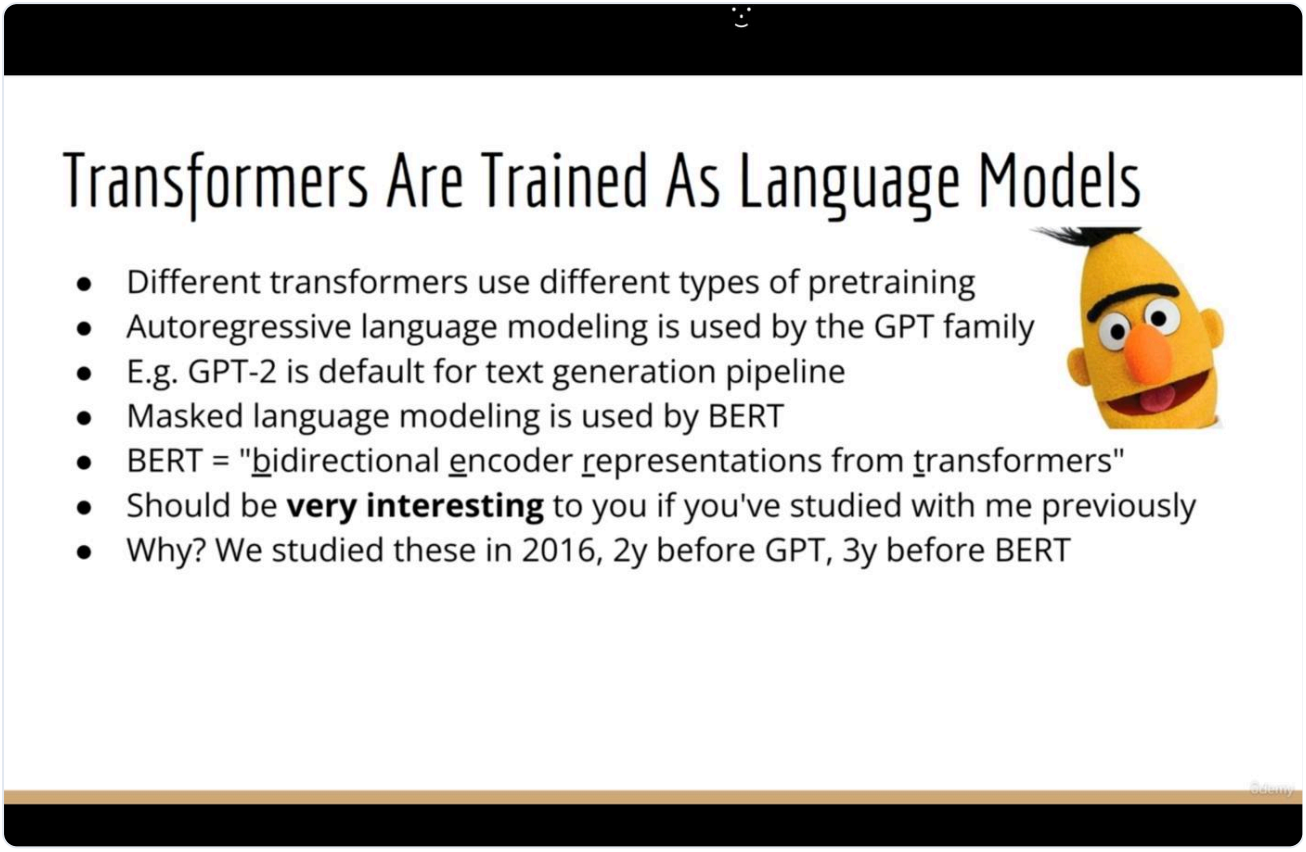

FIGURE 3.129 | Transformers Are Trained as Language Models

Now, this should be very interesting to you.

If you've studied my courses in the past, in particular, we've been doing this kind of training and building these kinds of language models since 2016, which was two years before GPT and three years before BERT. So although these tasks seem a bit simplistic and perhaps if you were just beginning machine learning total nonsense, they are in fact very powerful.

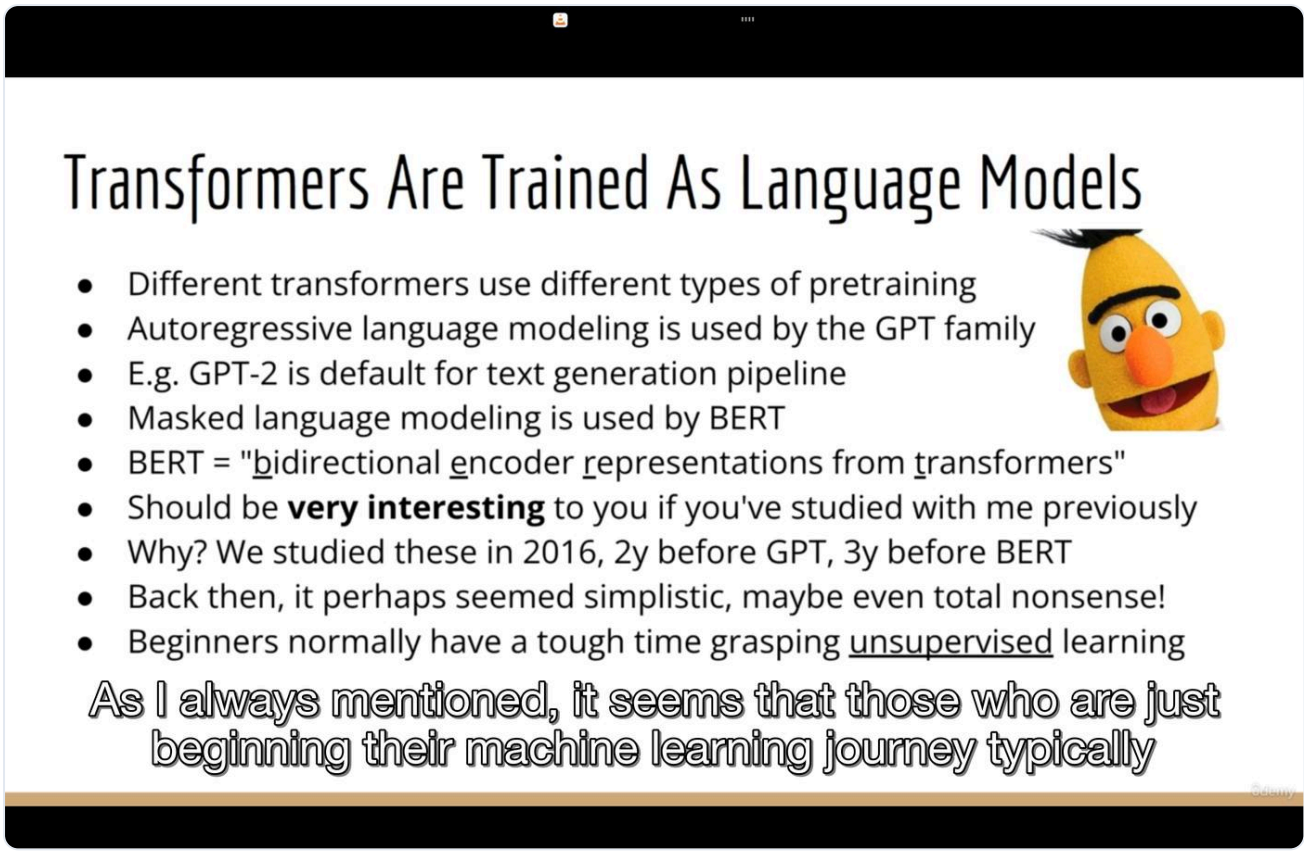

FIGURE 3.130 | Transformers Are Trained as Language Models

As I always mentioned, it seems that those who are just beginning their machine learning journey typically have a pretty tough time understanding why unsupervised learning is so useful and powerful.

Mostly it's because they lack patience and are looking for a quick fix. Supervised learning gives you that quick fix because you immediately know what the task is and hence what the application is. But to move beyond the beginner level, you have to be more patient and more creative and more open minded about the possibilities, even if what you're learning just seems like a bunch of useless math.

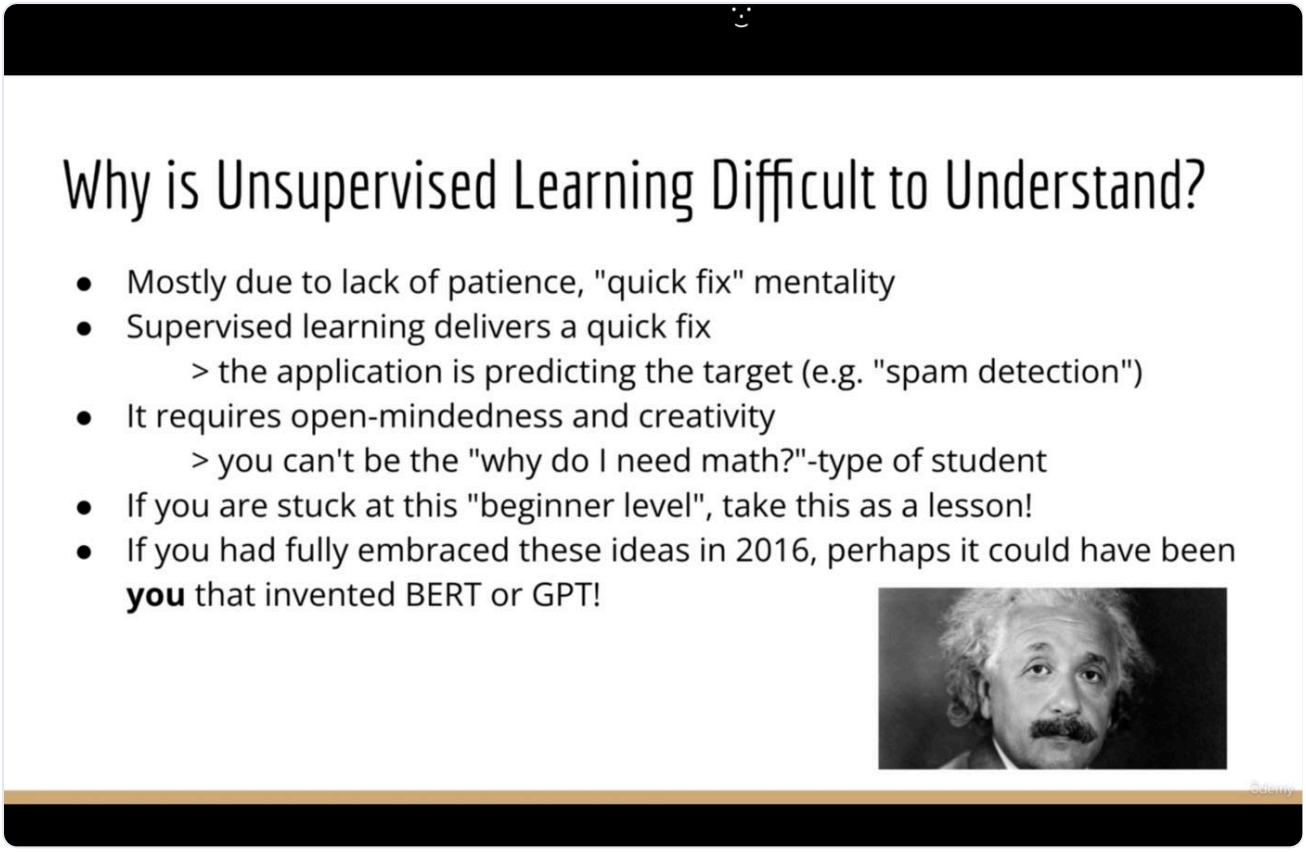

FIGURE 3.131 | Why Is Unsupervised Learning Difficult to Understand?

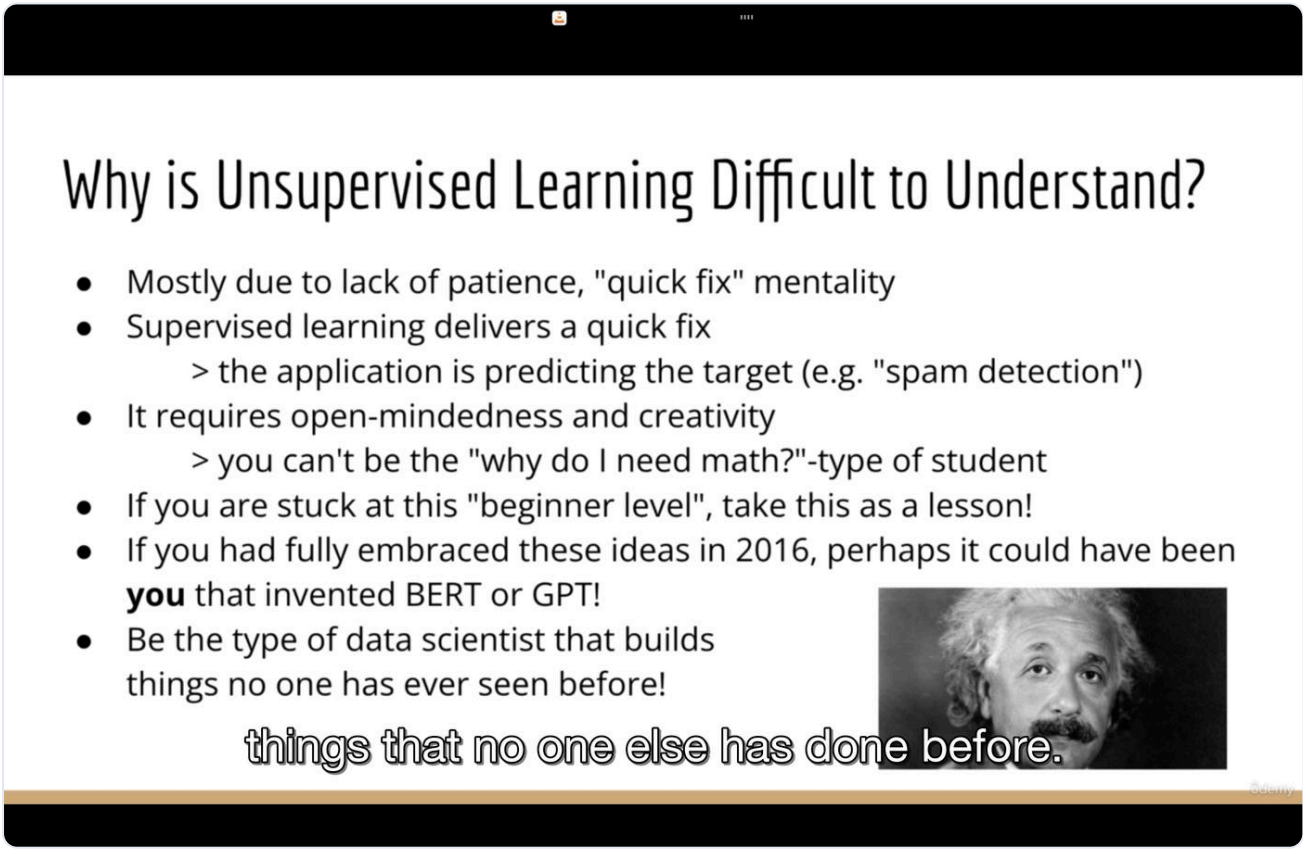

FIGURE 3.132 | Why Is Unsupervised Learning Difficult to Understand?

So if you were one of those students stuck at this beginner level, take this as a lesson about how to approach machine learning, even if it requires some patience on your part.

Perhaps if you had taken these ideas to heart in 2016, it could have been you that invented BERT or GPT. In other words, by opening your mind to these possibilities, it empowers you to create new and useful things that no one else has done before. Okay. So let's talk about a very useful application of the masked language model, which is article spinning. This is what I've called it in the past. Since it's closely related to a very widely used tool by blackhat marketers and SEOs in particular.

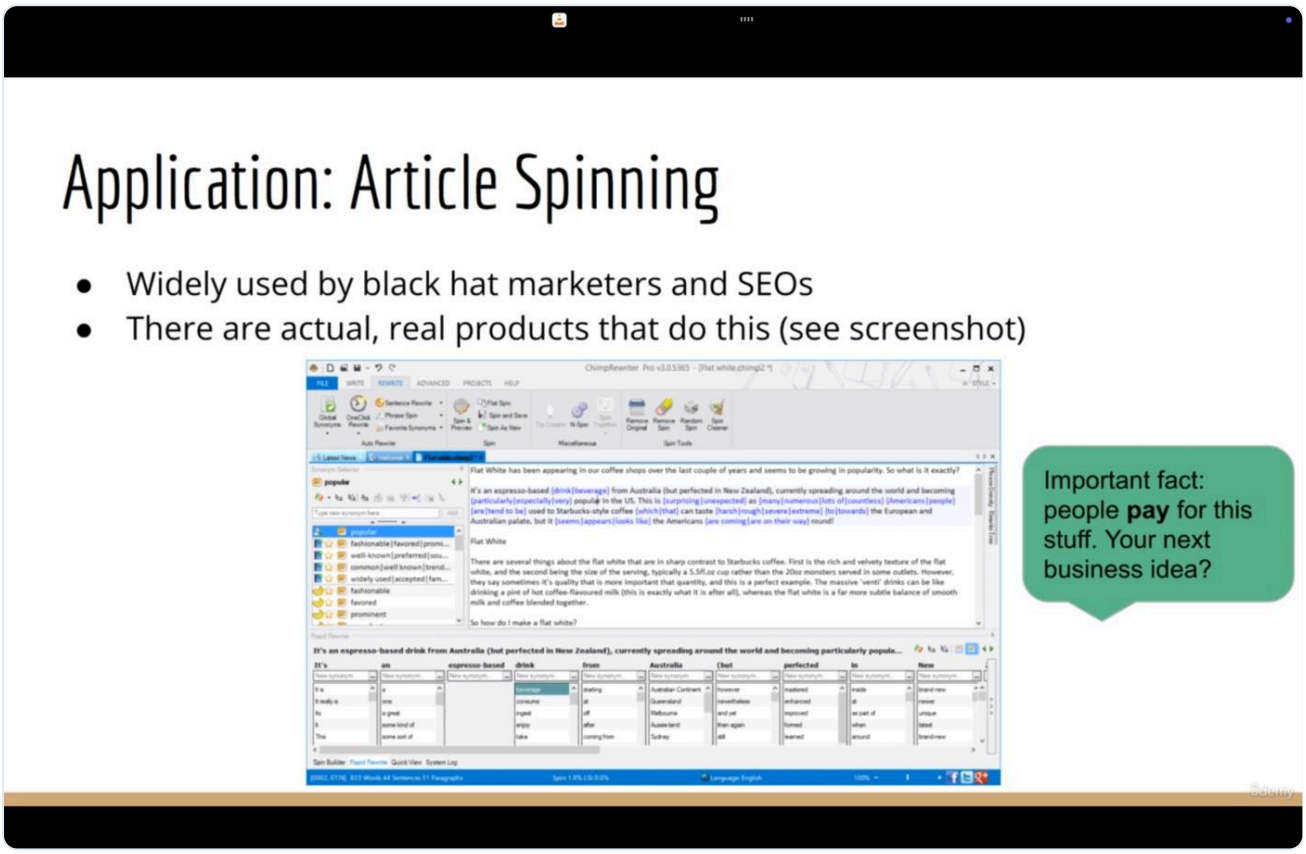

FIGURE 3.133 | Application: Article Spinning

There are actual real products like the one shown here that do exactly what we've been describing. Essentially, you take random words in a document and replace them with a machine generated word such that the sentence still makes sense and hopefully retains its meaning. Now, just to give you a bit of background, why is this useful? So this part is not about machine learning, but it helps to understand the big picture. Now, the best way to understand this is if you've had personal experience building a website or a blog or an online business.

As you know, it's pretty difficult to get visitors to your site. That's why online advertising is such a big industry. However, there is a free method to get visitors, which is through search engines such as Google and DuckDuckGo. The techniques which people apply to improve their search engine rankings are collectively known as SEO.

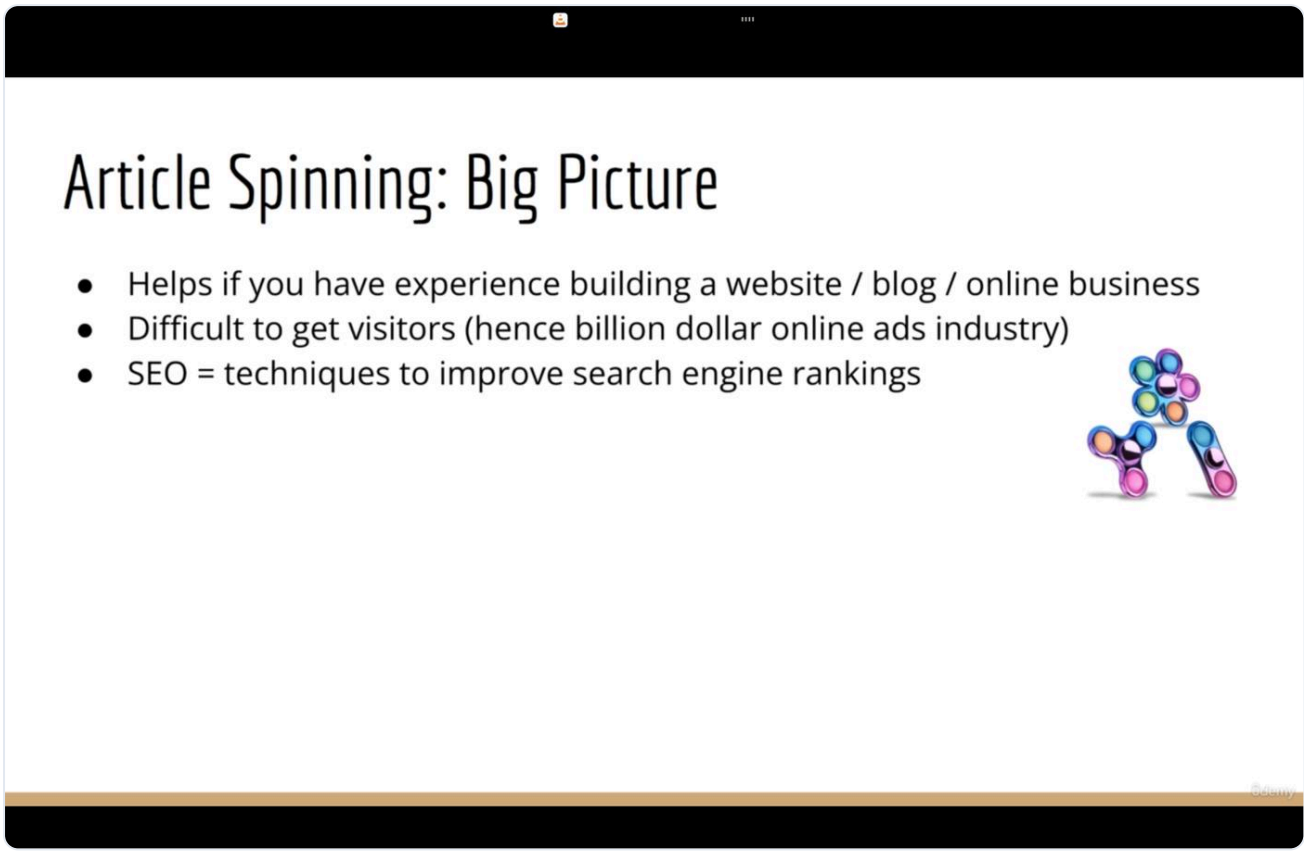

FIGURE 3.134 | Article Spinning: Big Picture

Okay, so SEO is a set of techniques that people use to improve their search engine rankings. But what does this have to do with article spinning?

Well, one of these techniques is to simply create lots of content with keywords that might match a user's query. For example, if you're a target audience is interested in running shoes, then your goal might be to write lots of articles about running shoes and other aspects of running. Now, as anyone who's had experience writing should know, writing is not easy and takes quite a bit of time and effort. Thus, if you're the type of person who perhaps lacks ethics, you might decide to simply copy or steal articles written by others.

But doing that by itself is not enough. We know that Google is pretty smart, so if they notice that you've simply copied someone else, your rankings will end up being penalized, not improved. So what black hat marketers end up doing is they still want to steal articles from others, but they try to change random words such that the result still makes sense but is no longer a good match to the articles they copied from.

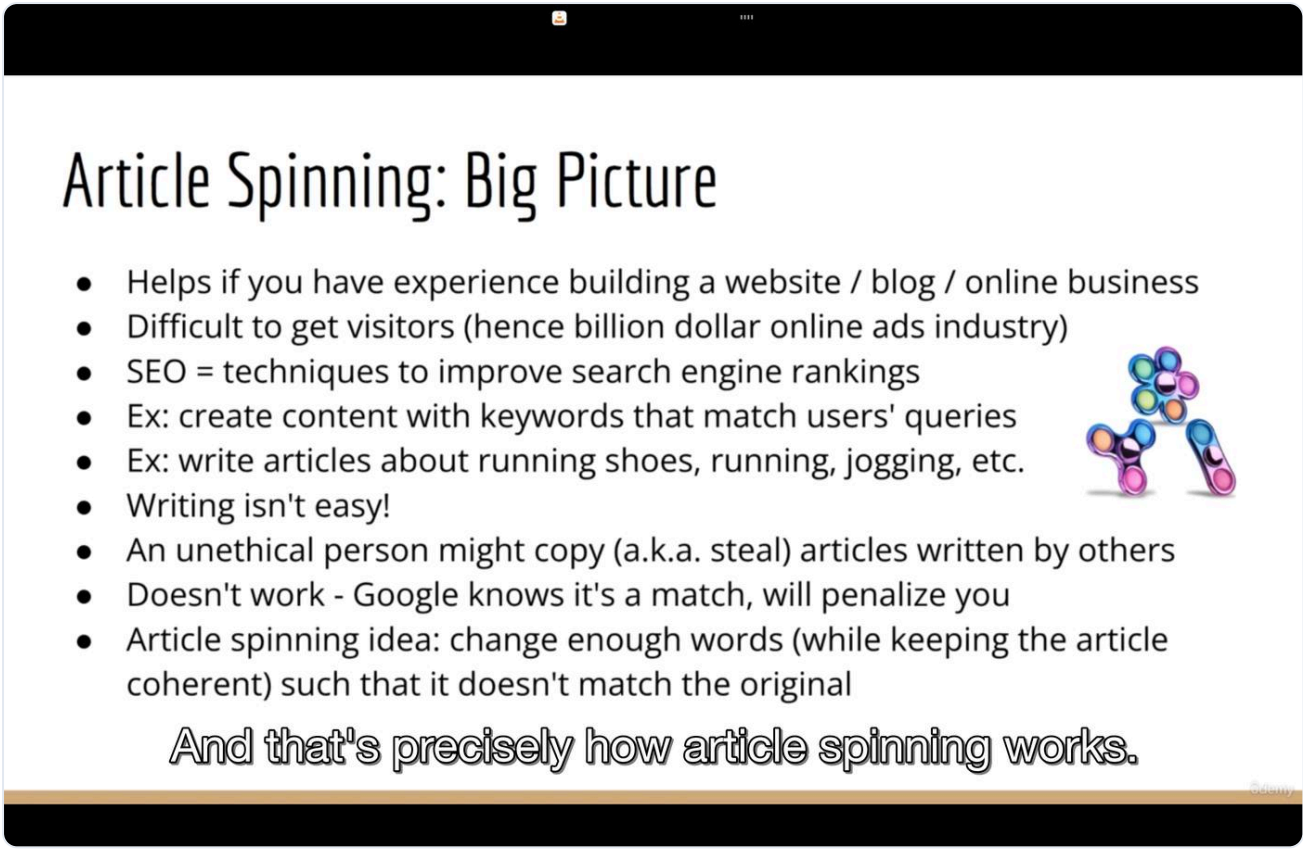

FIGURE 3.135 | Article Spinning: Big Picture

And that's precisely how article spinning works.

So to be a bit more detailed, suppose we take the sentence.

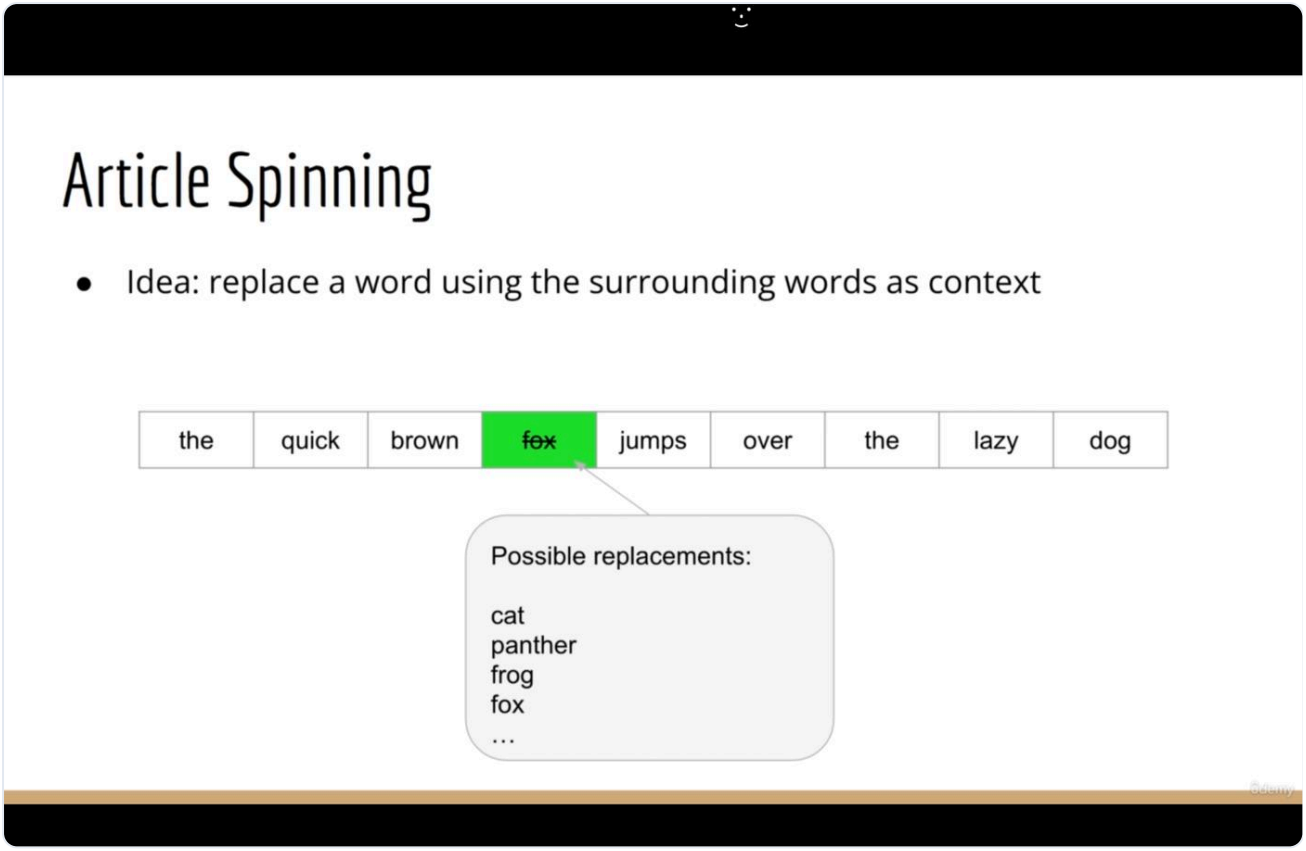

FIGURE 3.136 | Article Spinning

The quick brown fox jumps over the lazy dog. What do we want to do in article spinning is to take one of these words such as Fox and replace it with something else. Now, obviously the goal is to reduce human effort. So although you could come up with a fine substitution yourself, you'd rather be helped by a computer. And that's where our language model enters the picture. Clearly, what I want to do is ask my model to generate a word, given that the previous words were the quick brown and the future words are jumps over the lazy dog.

And this is exactly what models like BERT do. Now I realize that some people may be skeptical that this is a real world application.

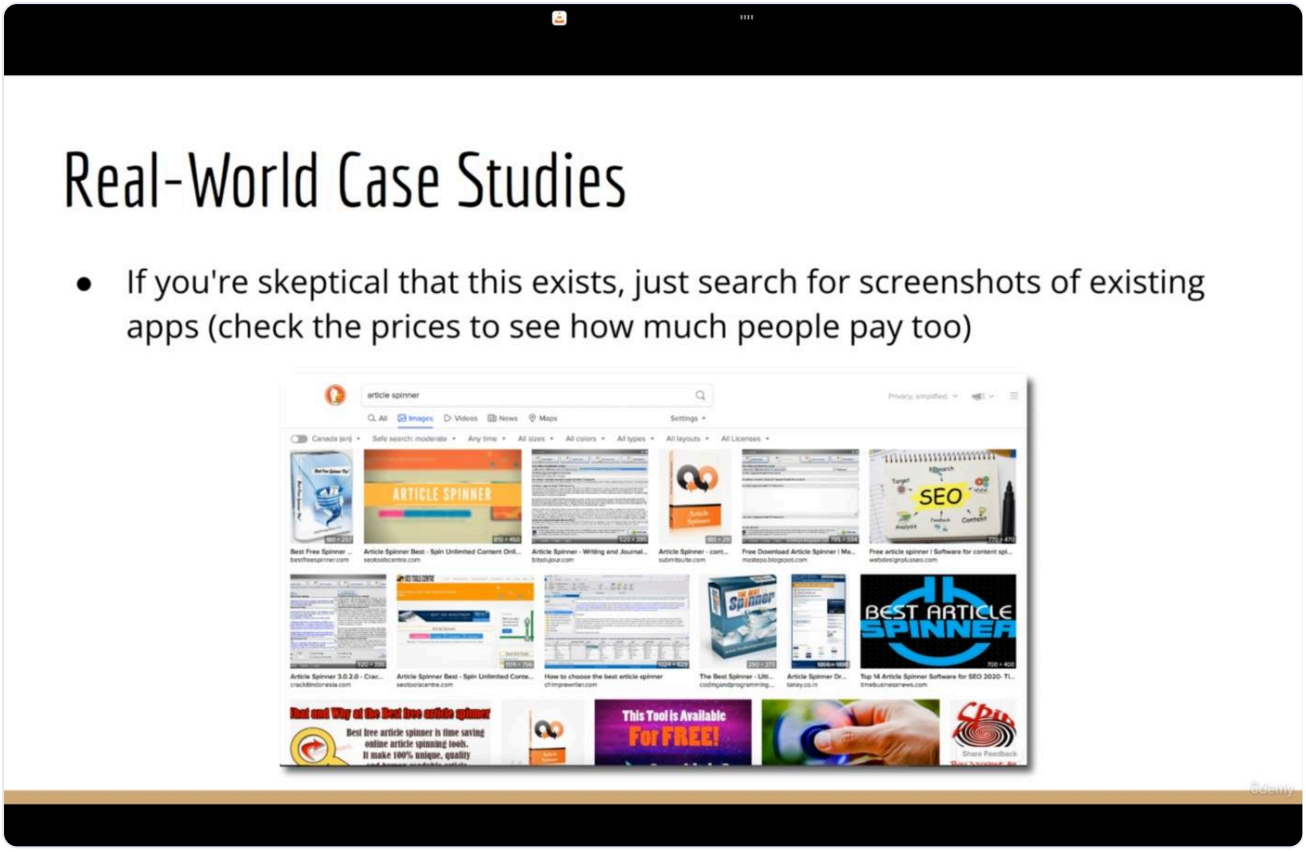

FIGURE 3.137

However, it's quite easy to simply do a search and find screenshots of real computer programs that do exactly what I've described. So the screenshots I've been showing you in this lecture are real programs. People actually pay money to buy and use these programs. Now, there's one pretty recent example where a popular YouTuber tried to use this method but failed.

I won't name any names, but if you watch machine learning videos on YouTube, you may have come across this story. The summary is this YouTuber, although he was very popular in the area of machine learning, all of his videos used code that was stolen from GitHub without any attribution. That wouldn't have been a problem if he gave credit to the people he stole from. But he always named his videos like Watch Me Build Such and such, implying that he was the one who wrote the code, which was obviously false.

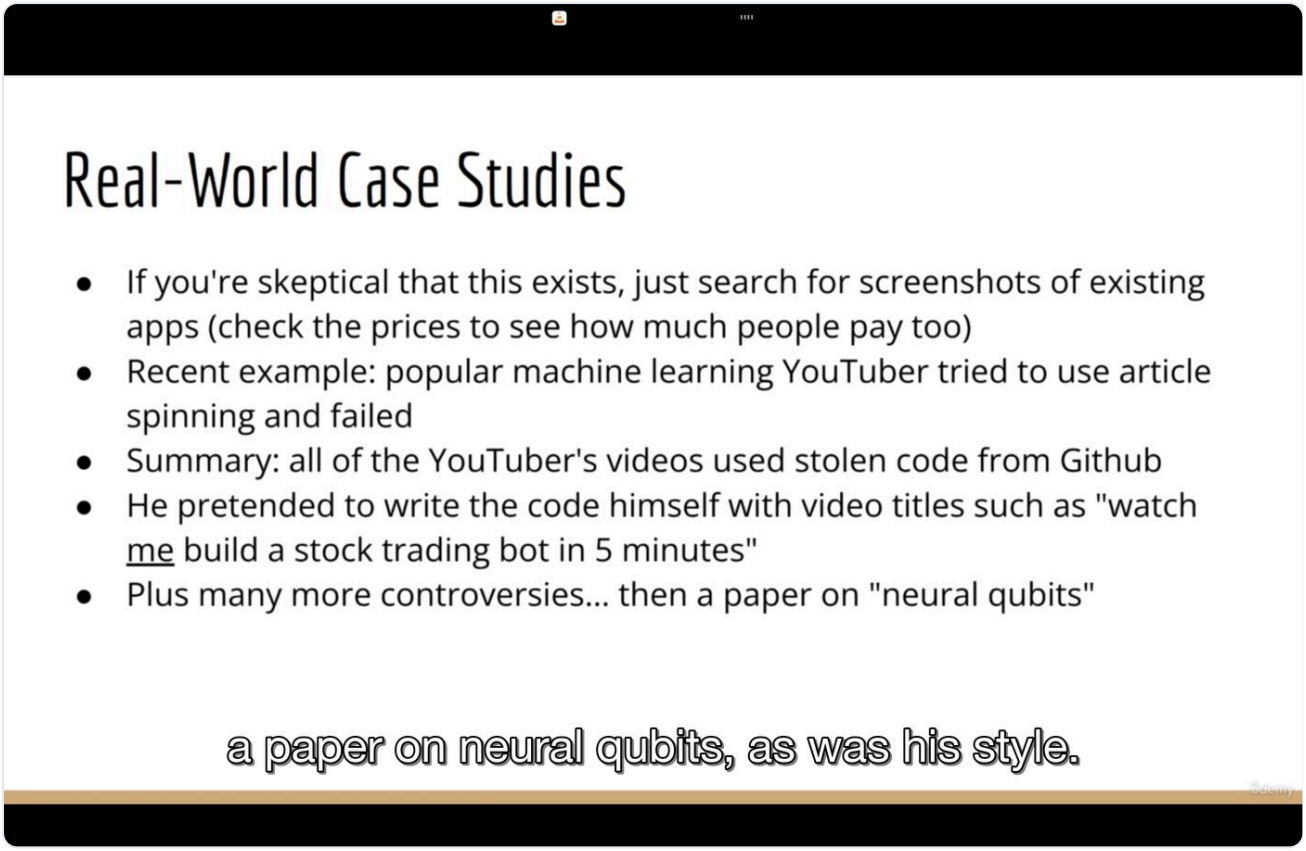

FIGURE 3.138

After a few more controversies which you can read up on if you like, he ended up pretending to write a paper on neural qubits, as was his style.

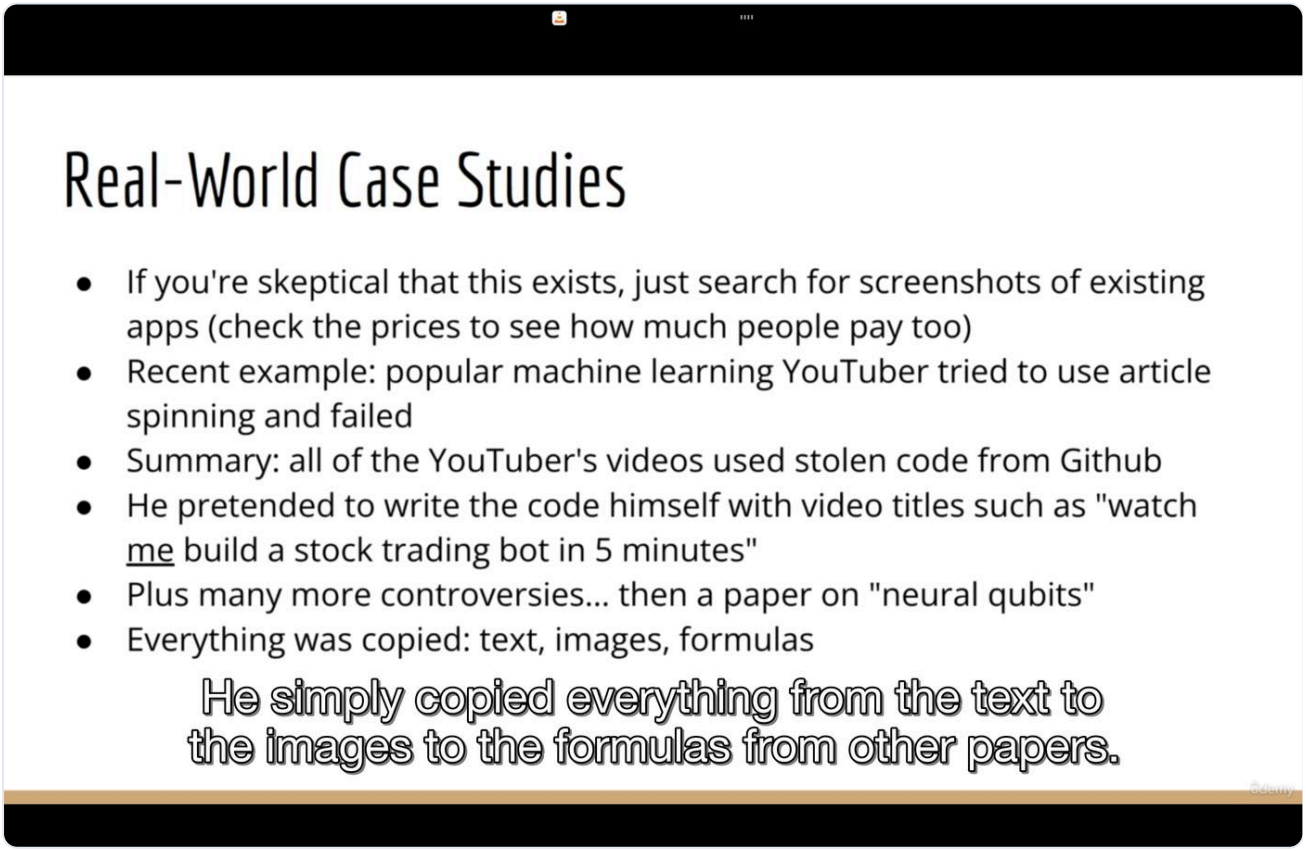

FIGURE 3.139

He simply copied everything from the text to the images to the formulas from other papers. So in order to disguise the text, he used the tool for article spinning. Except that it wasn't very good. In particular, it changed the complex Hilbert space into complicated Hilbert space. Obviously this doesn't make sense, although complicated is a synonym for complex in one context.

Complex in this context refers to complex numbers. So it's not a synonym for complicated in this context. Another example is changing quantum gates to quantum doors again in the right context. Gate can be a synonym to door. But clearly this is not that context.

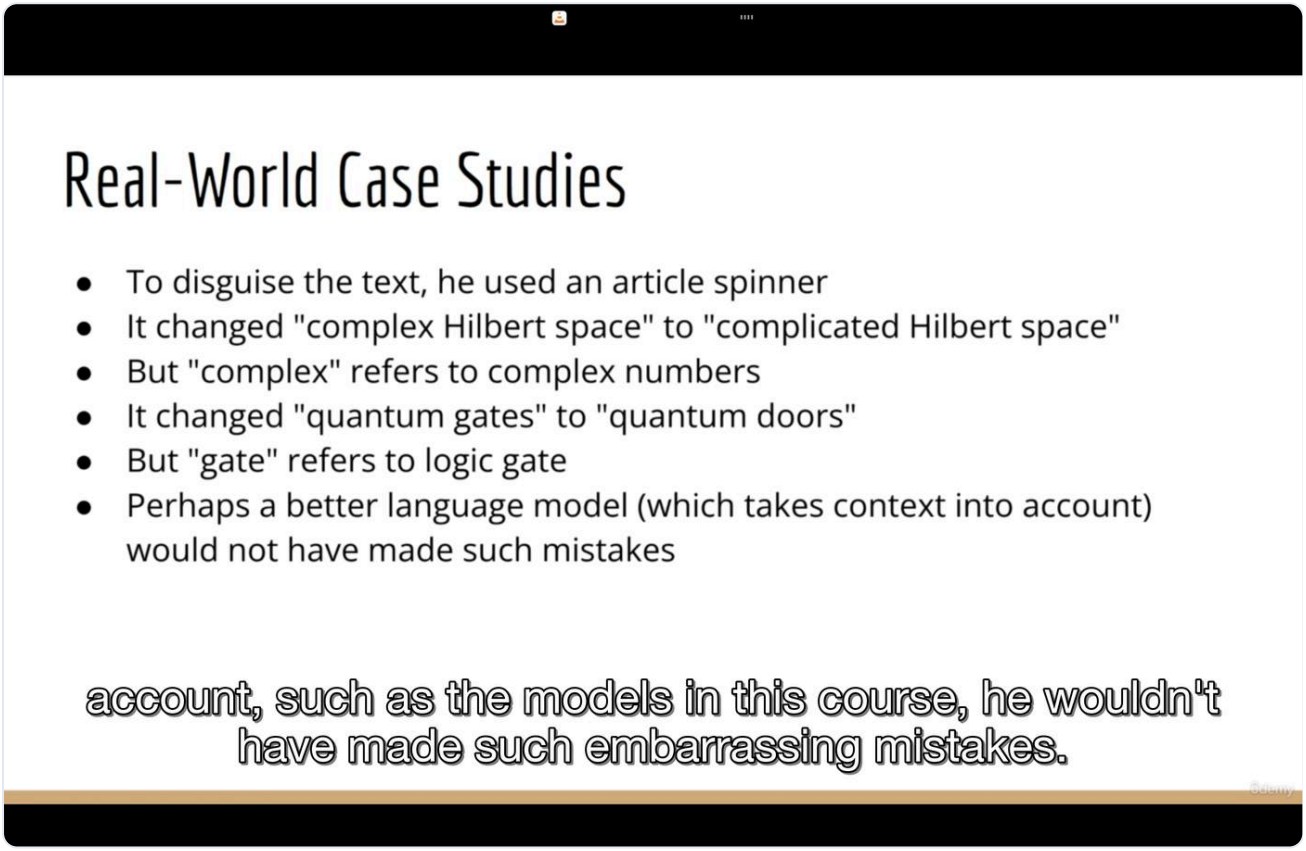

FIGURE 3.140 | To Disguise the Text, He Used an Article Spinner

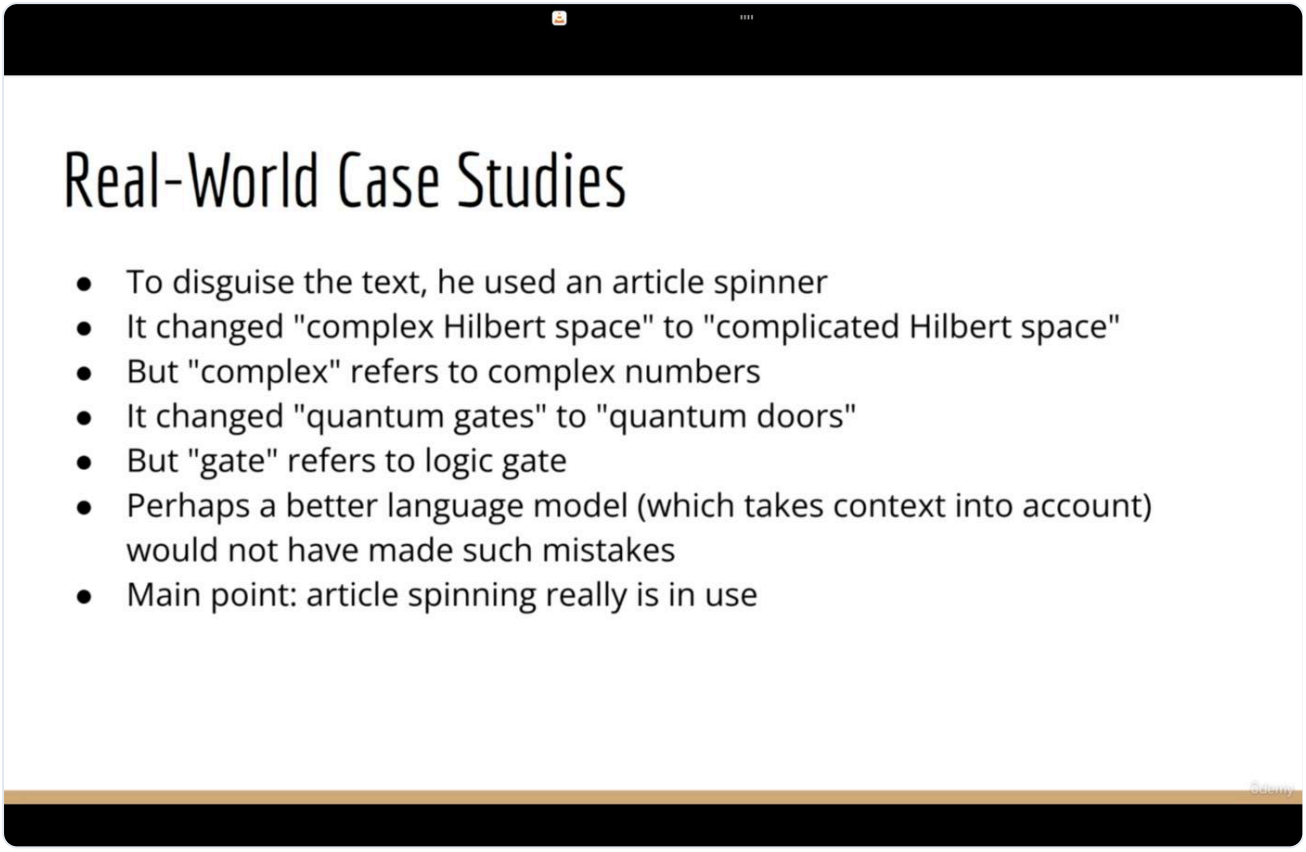

FIGURE 3.141 | To Disguise the Text, He Used an Article Spinner

So it may be said that if this YouTuber had used a better language model that can take context into account, such as the models in this course, he wouldn't have made such embarrassing mistakes. At the same time, it is still likely he would have been caught since people still read the other parts of the text, recognize the images and so forth.

But the point of this is this concept of article spinning really is used in the real world, typically for nefarious purposes. Okay. So now that you understand the masked language model and you're convinced of its real world application, let's discuss how we do this with Transformers. Again, it's very simple with just one or two lines of code.

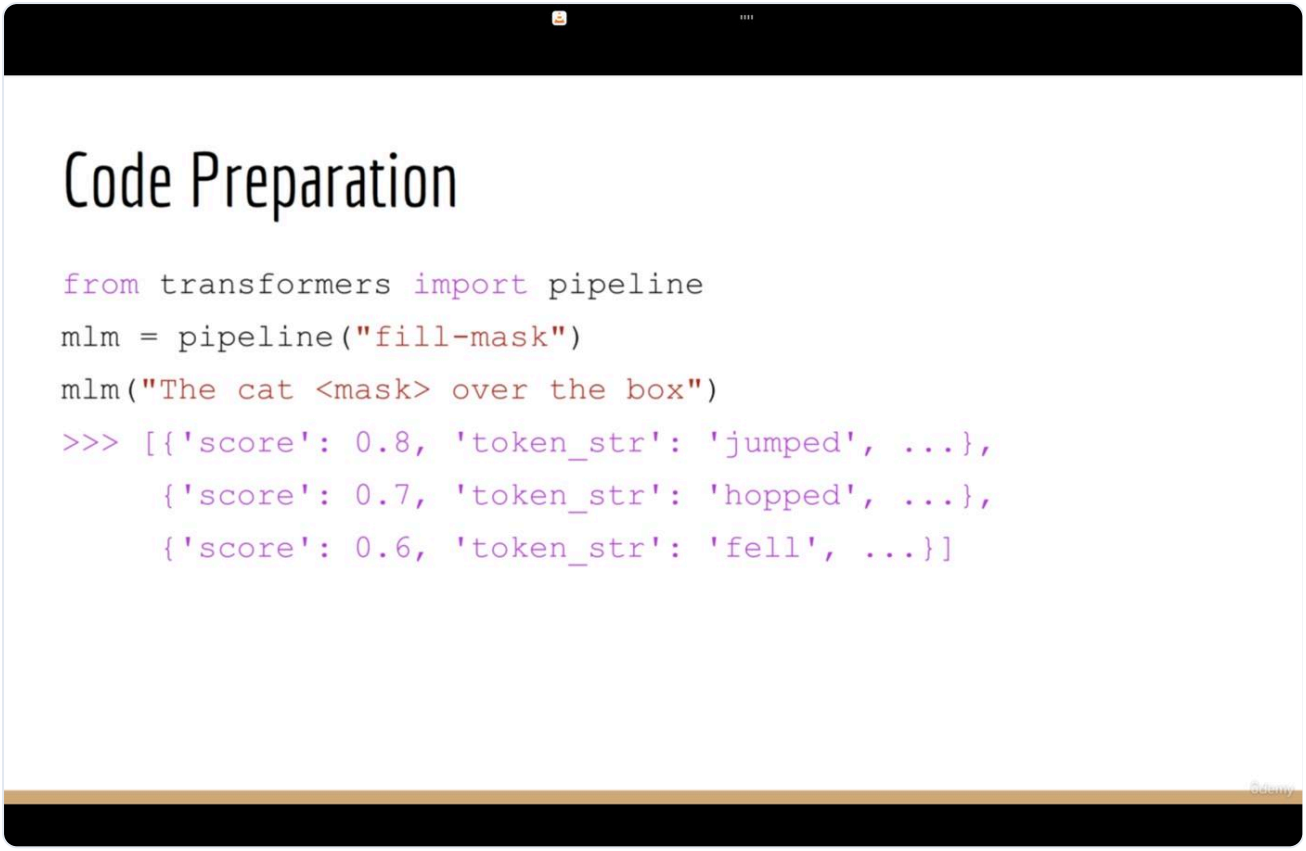

FIGURE 3.142 | Code Preparation

We begin by calling the pipeline function, passing in the string fill-mask as before. Once we've done this, we can immediately use the pipeline simply by passing in some text.

The format of this text should be such that it includes a special mask token denoting the token you want to fill. Note that the mask token is surrounded by angled brackets. As output, you will get multiple suggestions stored in a list.

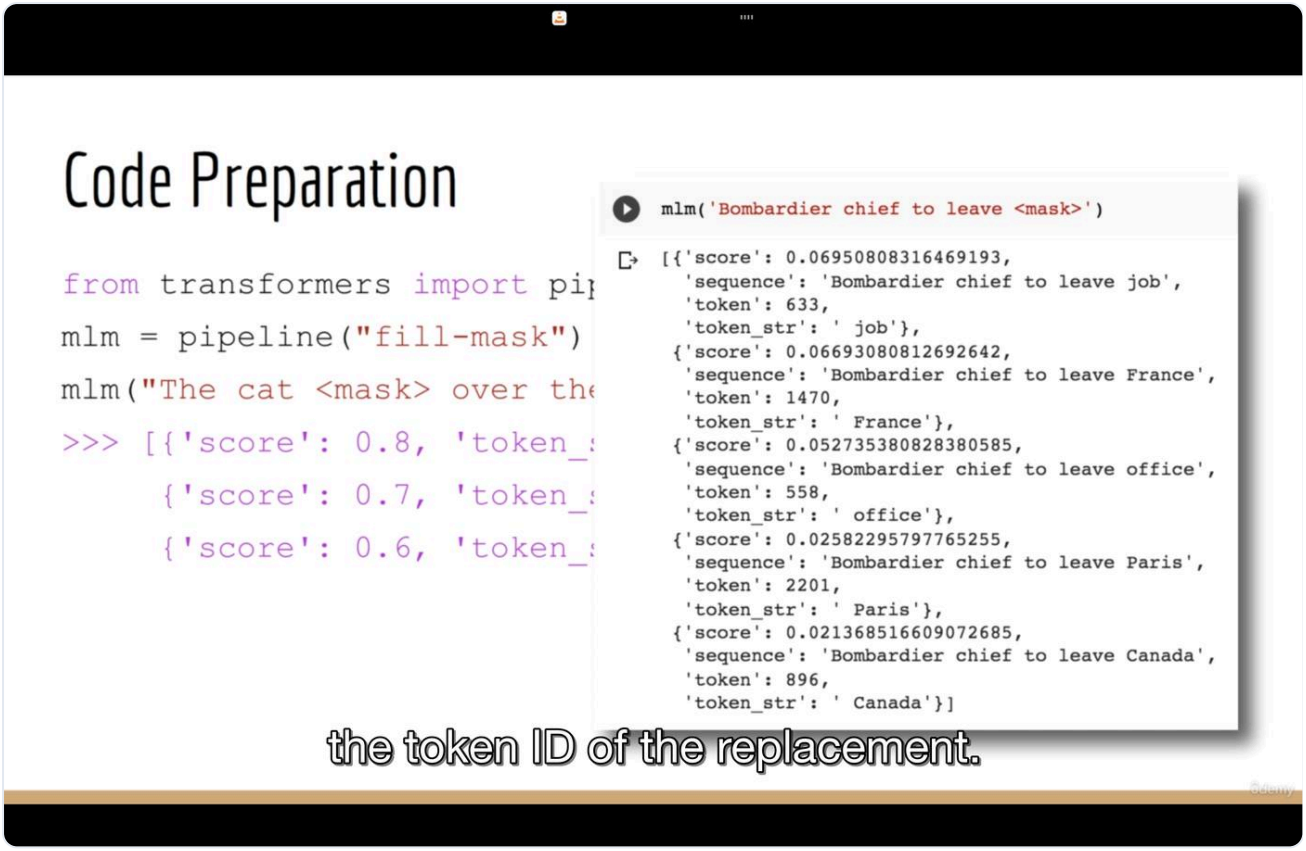

FIGURE 3.143 | Code Preparation

Each item is a dictionary with the completed sequence, the replaced token corresponding score and even the token ID of the replacement. So as before, it's pretty easy to use. The next thing I want to do in this lecture is give you some insight on why we sometimes call this an auto encoding language model.

The best way to understand this is if you've had experience with unsupervised, deep learning. Although if you don't, then intuition should be enough.

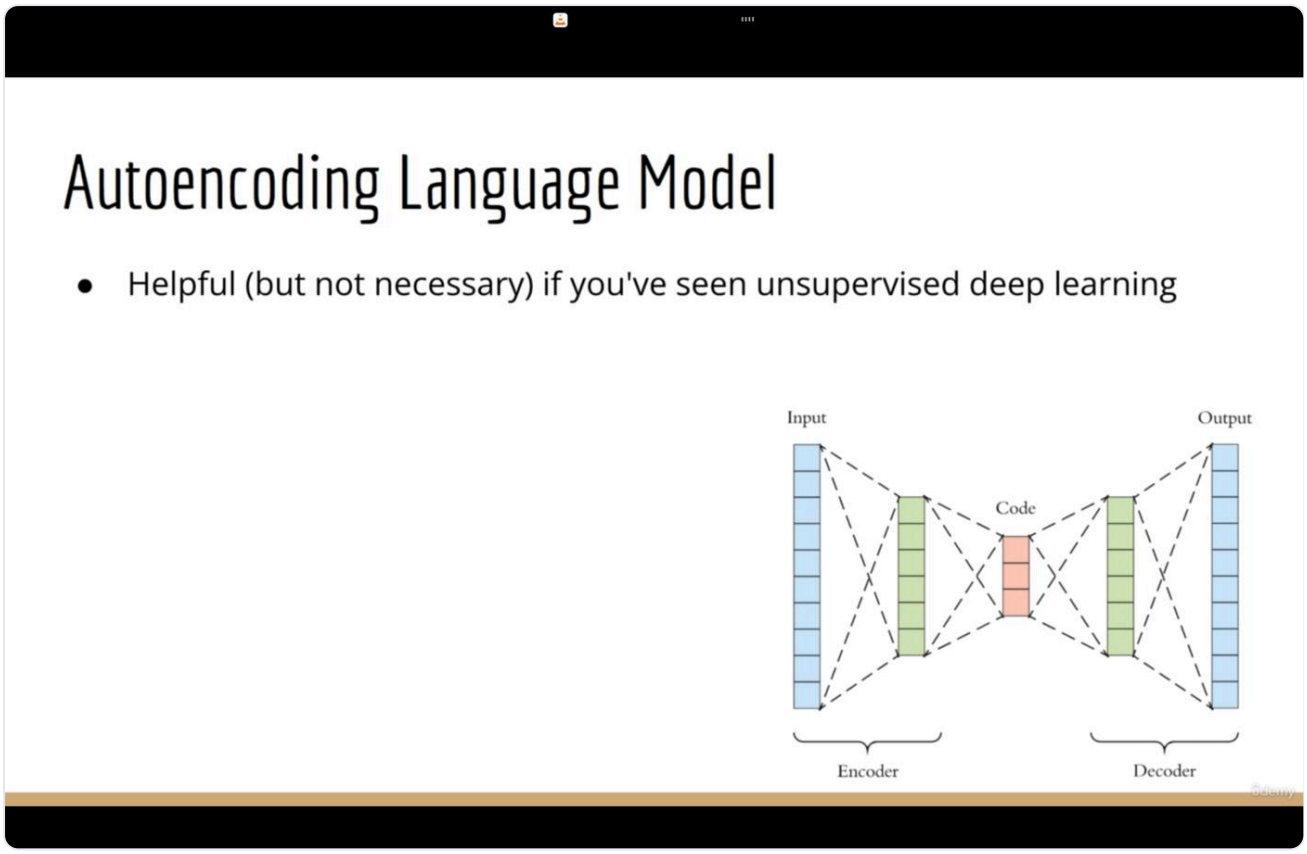

FIGURE 3.144 | Input Output

Okay, so the intuition comes from auto encoders. Auto encoders are a type of neural network that taken some input and simply tried to produce the same thing as output. That is, it tries to simply replicate the input. Now at first that might seem pretty useless, but in fact there have been quite a few applications which I've discussed in the past, such as recommender systems and pre-training, like what we do in this course.

So if you would like to know more, simply ask me on the Q&A. One variation of the basic auto encoder is the denoising autoencoder.

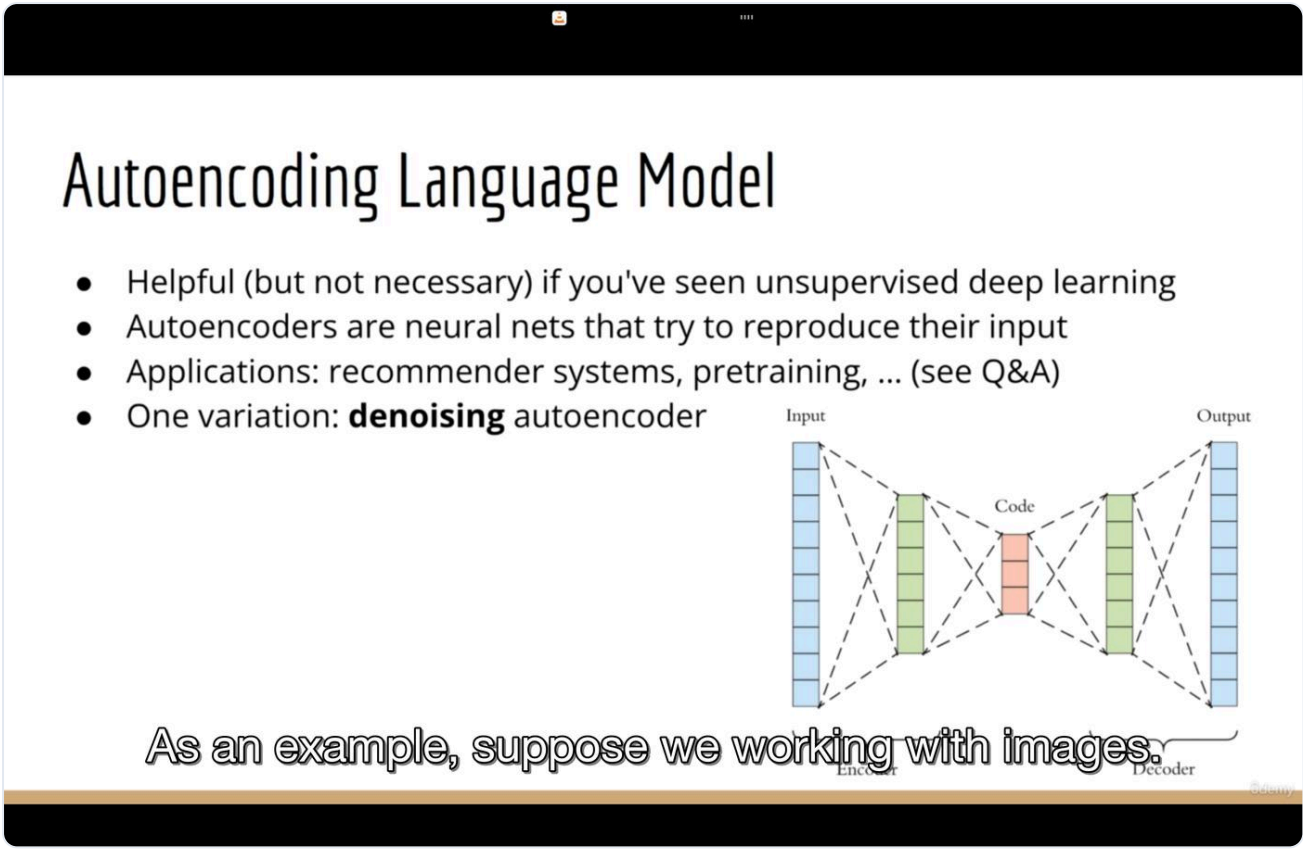

FIGURE 3.145 | Applications: Recommender Systems, Pretraining, ... (See Q&A)

As an example, suppose we working with images.

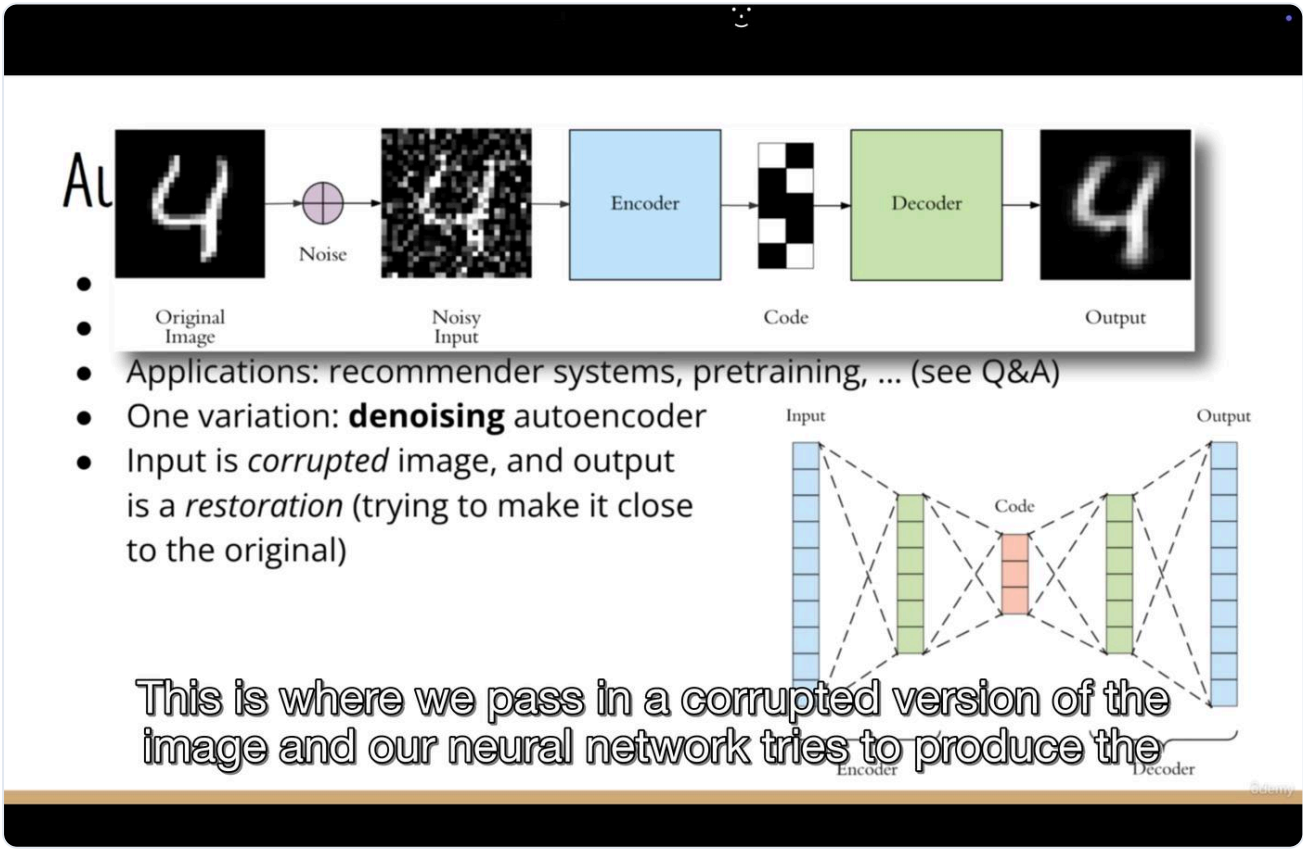

FIGURE 3.146 | Original Noisy Code Output

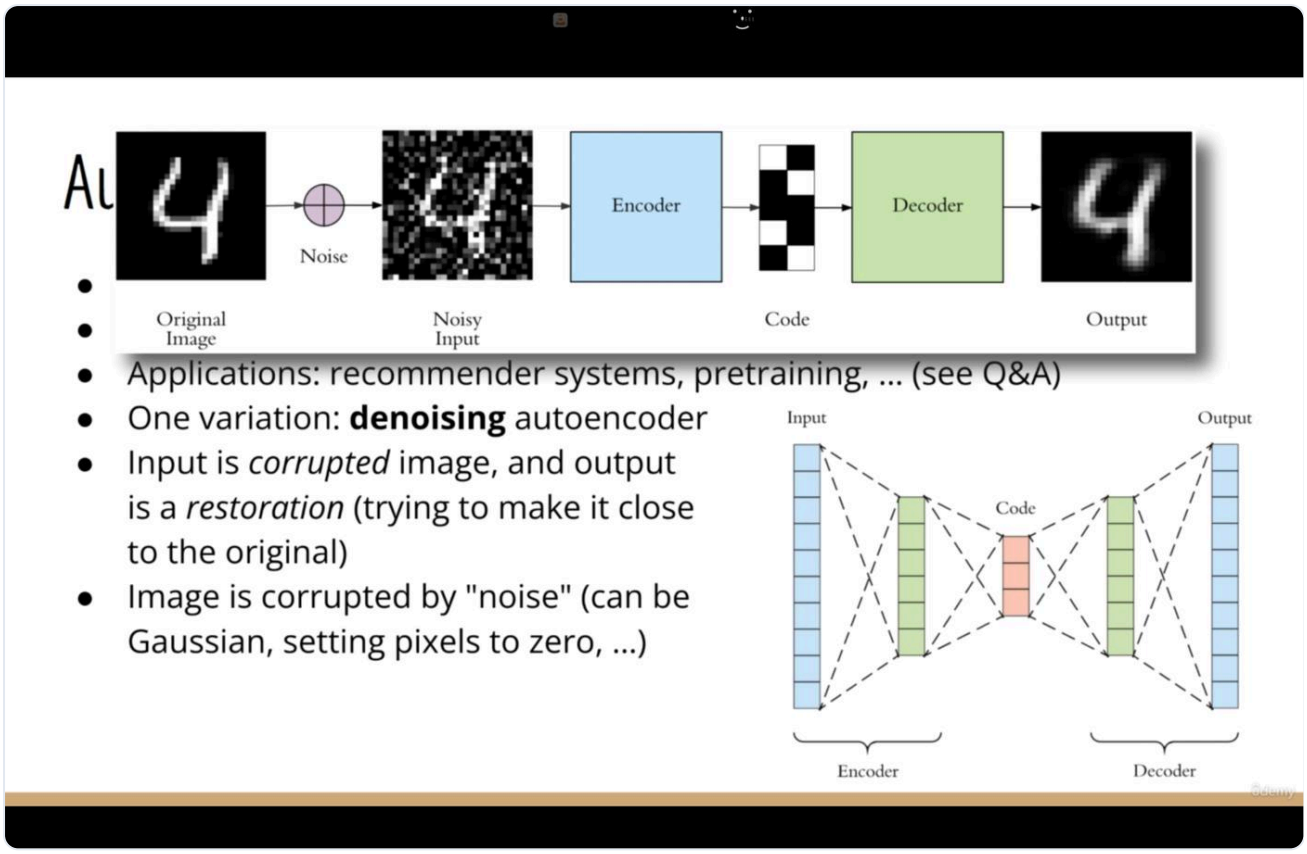

FIGURE 3.147

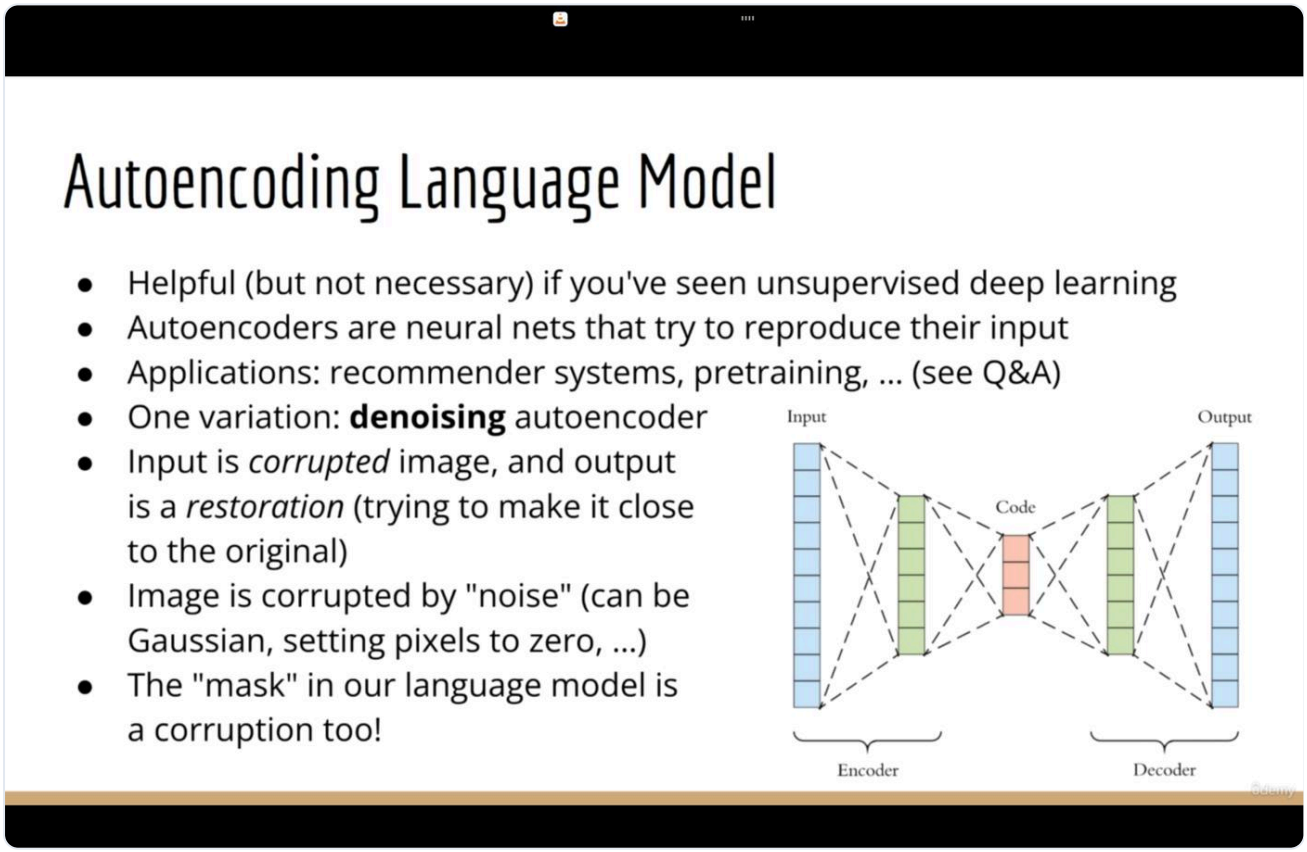

FIGURE 3.148

This is where we pass in a corrupted version of the image and our neural network tries to produce the uncorrupted image. That is, it tries to restore the image and remove the noise. This noise can be of any kind, such as Gaussian noise or simply setting a pixel value to zero.

But this should remind you of what this lecture is about. Effectively, what we're doing when we mask a word in a sentence is we are adding noise. Our goal is to build a neural network that will find the missing word and hence restore the sentence to its original state. So this is another way to think of the masked language model. It's simply a noisy auto encoder. But for text, it's also another way to think of how this relates to the causal language model we previously studied.

As you recall, that was auto regressive. So that was auto regressive and this is auto encoding.

8 · Masked Language Modeling (Article Spinner) in Python

See the notebook here.

So in this lecture, we will look at some code to demonstrate how to use the masked language model in Hugging Face. To begin, we're going to download the BBC News dataset. The next step is to install Transformers, as we normally do. The next step is to import everything we need for this notebook. The next step is to load in our data using pd.read_csv. The next step is to call the FDA ahead to remind ourselves what our data frame looks like. As you can see, this dataset consists of two columns, which are the labels and the text.

Thus this is a supervised data set, but for this notebook we won't be making use of the labels. The next step is to print out the unique set of labels to see what exists. As you can see, we have a set of five labels for various categories of news. The next step is to choose one of the labels which we want to make use of in the script. I've chosen business since this seems like it would have good articles to spin. The next step is to filter out any rows which do not match or label, and to grab just the column containing the text.

The next step is to set a random seed so that you can replicate these results for yourself. The next step is to randomly choose a document using np.random.choice will call the result doc. The next step is to print our chosen article to see what it looks like. Okay. So the title is Bombardier Chief to Leave Company. So this is the article we are going to spin. The next step is to instantiate our pipeline object passing in fill mask to specify our task.

Note that the default model is DistilRoBERTa, which is a derivative of BERT. The next step is to test their model passing in the title of our chosen article. Okay. So as you can see, this returns multiple substitutions. The first suggestion is Bombardier chief to leave job, which is essentially equivalent to the original, which was Bombardier chief to leave company. We can also see that a few of the suggestions are places including France, Paris and Canada.

This kind of makes sense since this is a company with its headquarters in Canada. But it's not clear if it has any relation to France. However, this appears to be what our model has learned. So perhaps it works. Note that as usual, these all make grammatical sense, even though they change the meaning of the sentence. The next step is to try your model again. But with the next sentence in the article, note that we've applied the mask token to the word train.

Okay. So as you can see, Airbus is the top scoring replacement. This makes grammatical sense, but it doesn't make sense semantically because Airbus is simply another company. In other words, this sentence suggests that Bombardier makes Airbus, which doesn't make sense. The same thing goes for the second suggestion, which is Boeing, which is another company in the same space. Interestingly, the other replacements also don't make a lot of sense. Bombardier doesn't make aerospace, nor does it make airlines, nor does it make aviation.

So perhaps this model simply doesn't recognize the relationship between a plane making a bombardier and the term we want to replace. The next step is to mask a different word in the same sentence. Note that with this interface it's only possible to mask one word at a time. So if you want to replace multiple words in the same sentence, they have to be done in separate calls. So in this case, we're going to replace the word departure, since the original sentence contain the phrase following the departure of its chief executive.

Okay. So these results make a lot of sense. The first replacement is resignation, which is essentially equivalent to a departure. The second replacement is departure, which is the original word. The third replacement is the plural of departure, which actually makes more sense grammatically compared to the original. As you can see, there actually are multiple departures because it's the chief executive and two members of the board. The next replacement is dismissal, which also makes sense, and the next replacement is firing, which again makes sense.

So any of these would have been great substitutions. The next step is to use the same sentence again. But this time we're going to replace the word executive. Okay. So not unsurprisingly, the first suggestion is the actual word executive, which makes sense. The second suggestion is the plural of executive, which also makes sense. The next suggestion is economist, which makes a little less sense, but still makes sense. The fourth suggestion is officer, which kind of makes sense.

The final suggestion is engineer, which also still makes sense. So, again, pretty good substitutions. Okay. So again, we're going to use the same sentence, but this time we're going to mask the word members. Okay. So the first replacement is just members, which is actually encouraging because it means our model really has learned what to expect. Note that this has a much higher score compared to the other replacements. The next replacement is thirds, as in two thirds, which also makes sense but changes the meaning.

The third replacement is directors, which also make sense. The fourth replacement is chairs, which kind of makes sense. As far as I know, there's usually only one chair, so perhaps it would be unconventional for this to be plural. The final replacement is chiefs, which again pretty much makes sense. Now you'll notice that in this script we chose the words to replace manually. In the real world, this may be okay, or you may want to automatically choose which words to replace.

So this is your exercise for this lecture. Your job is to write a function that will automatically choose which words to replace and then replace them using the same method as above. You can feel free to choose these words however you like. One method you might use is to choose words with a high TF-IDF, since these tend to be more important in the document.

9 · Named Entity Recognition (NER)

So in this lecture we are going to look at how to do named entity recognition, also known as NER with transformers.

FIGURE 3.149 | Deep Learning

So what is NER?

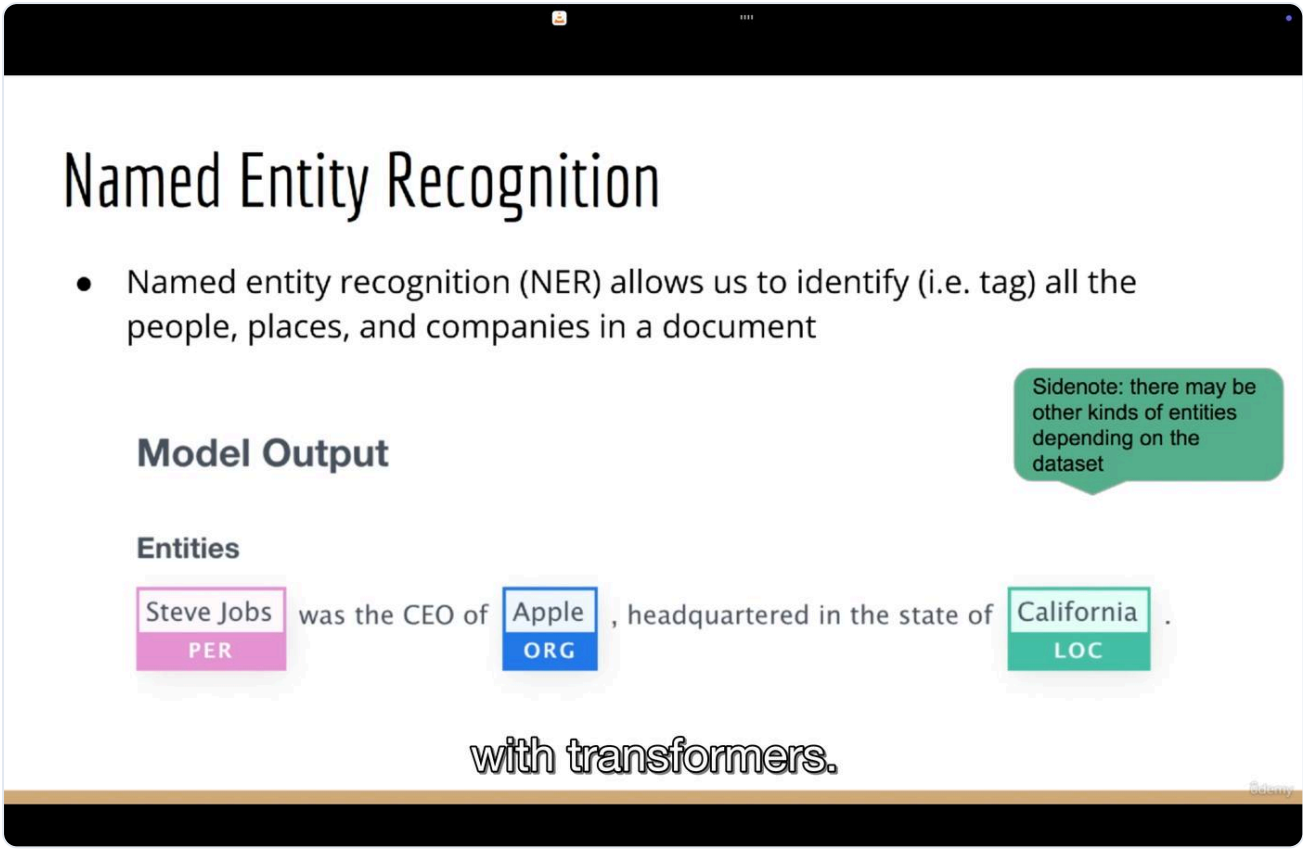

FIGURE 3.150 | Named Entity Recognition

Well, basically the idea is you have a document and you want to identify all the people, places and companies in that document. This data might be used as a preprocessing step in a larger piece of code. So, for example, if I see Apple, I would mark that as a company. If I see Steve Jobs. I would mark that as a person. And if I see California, I would mark that as a place.

In any case, suppose you have some use for it. How does this actually work? Well, it turns out that NER is essentially exactly the same as parts of speech tagging, which you already know how to do. As you recall, that was a many-to-many task where we assign each word to a tag and we do the same thing with NER. So every place has the tag LOC. Every person has the tag PER. And every company has the tag ORG. Now, one challenging aspect of NER is that some entities span multiple tokens.

So, for example, consider Steve Jobs. This has two tokens, but both of them are part of a name.

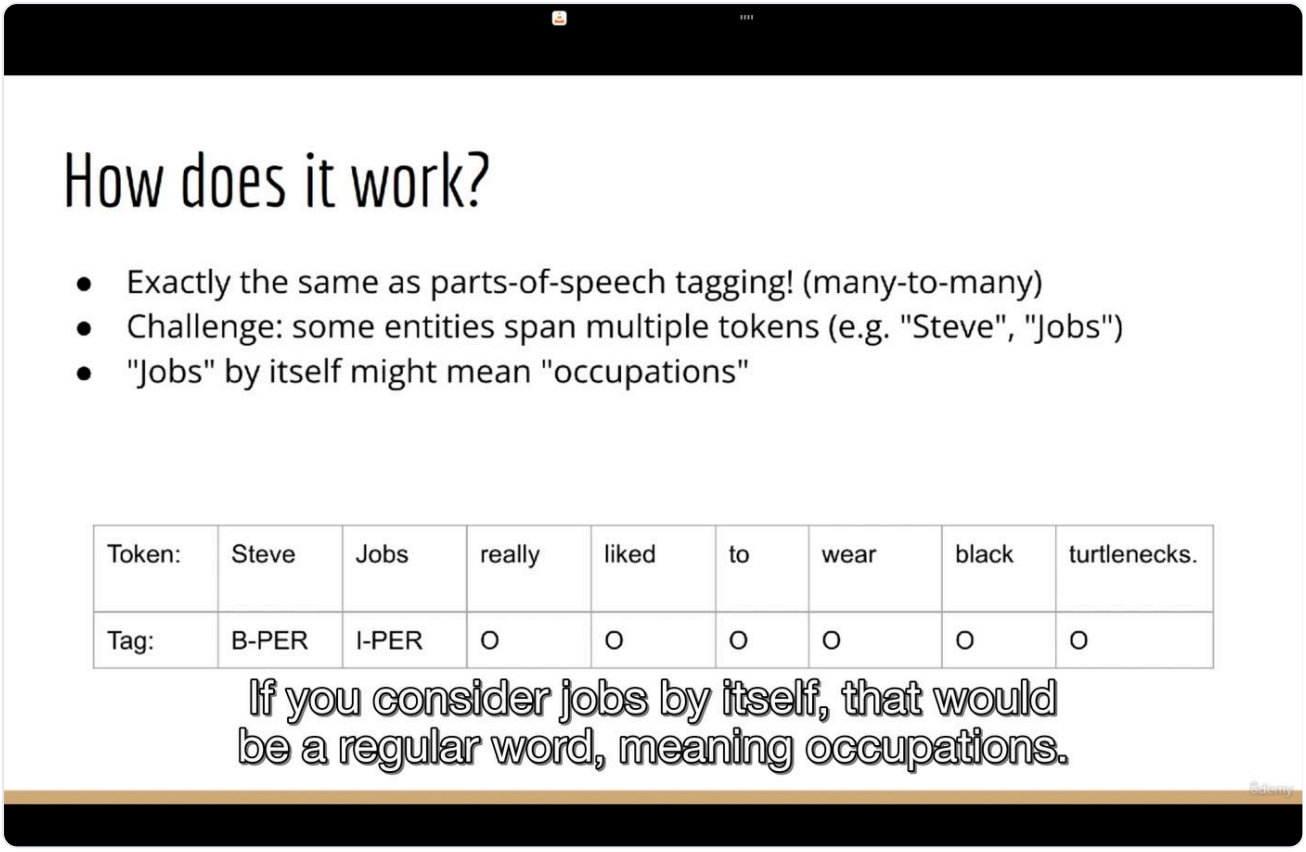

FIGURE 3.151 | How Does It Work?

If you consider jobs by itself, that would be a regular word, meaning occupations. Another challenging aspect of NER is that this data is highly imbalanced. You can see that most words just get the tag O, which means that it's not a place, person or company. Consider this sentence from our dataset.

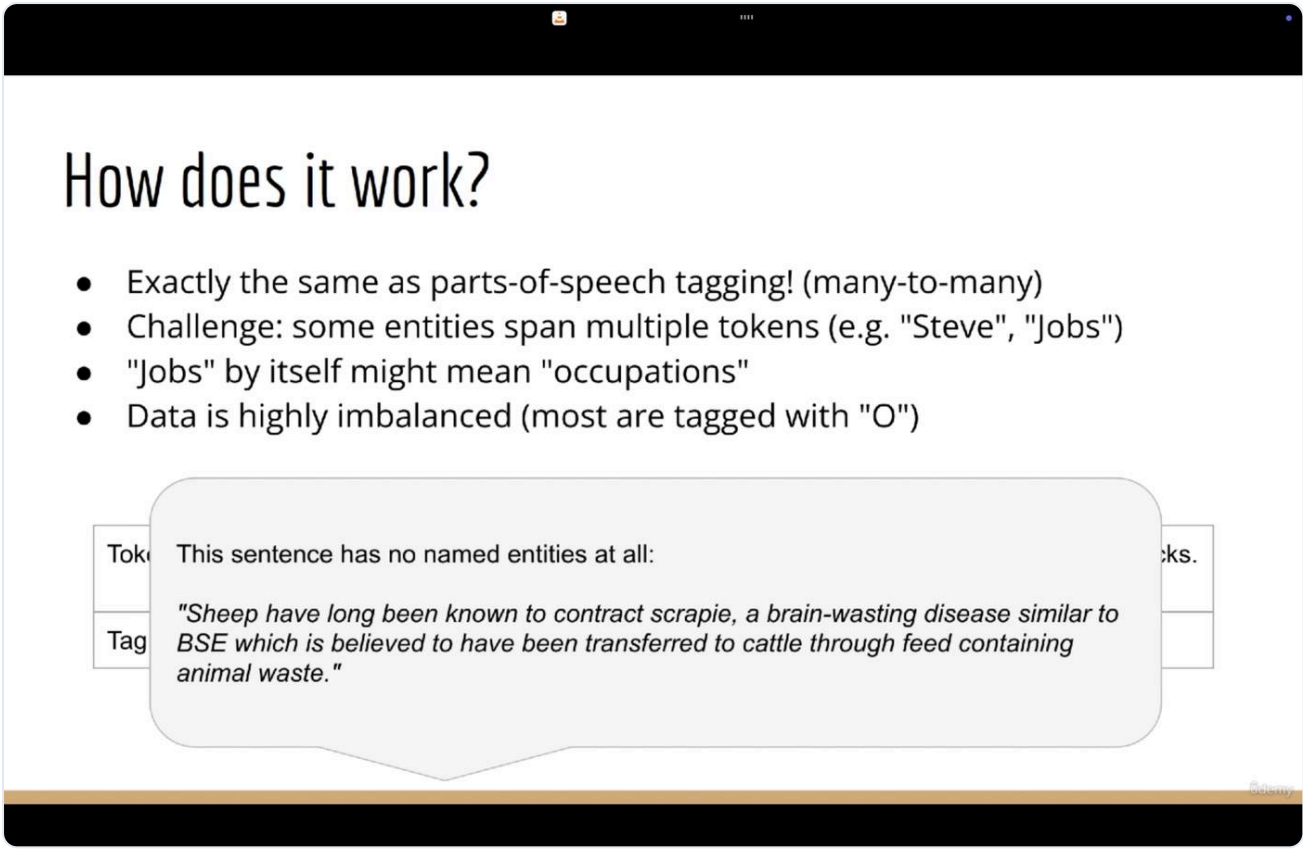

FIGURE 3.152 | How Does It Work?

Sheep have been long known to contract scrapie, a brain wasting disease similar to BSE, which is believed to have been transferred to cattle through feed containing animal waste.

This is a very long sentence and it has no named entities. All the labels are just. Oh. Let's briefly discuss the format of NER tags, since you're probably wondering what all these A's and B's signify.

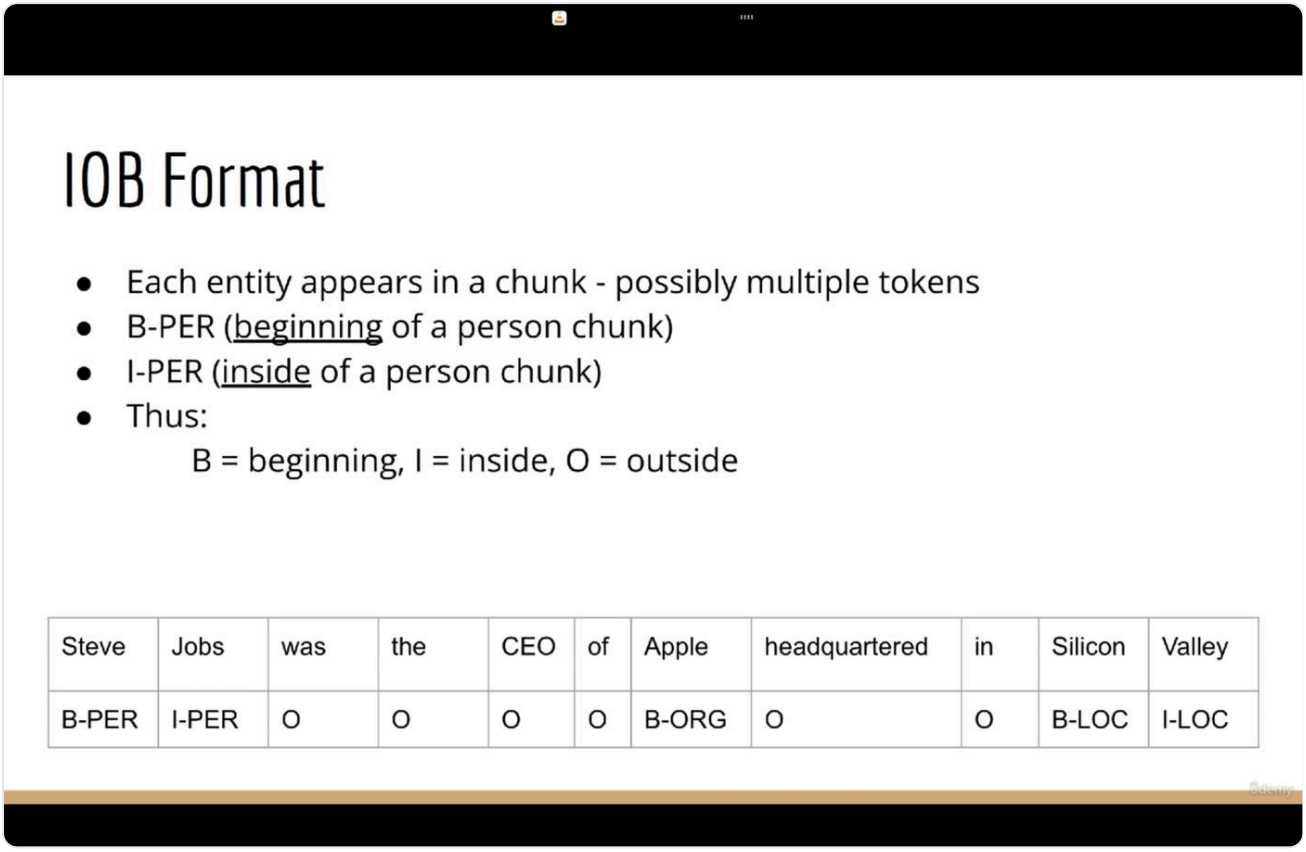

FIGURE 3.153

This is called IOB format, since obviously we use the letters IO and B in our tags. The idea is that each entity potentially appears as a chunk. A chunk is a sequence of multiple tokens. An example of that is Steve Jobs. In this case, we would use B-PER to represent the fact that Steve is a person tag and it's the beginning of a chunk.

For jobs, we would use I-PER to represent the fact that Jobs is also a person tag and it's inside a chunk.

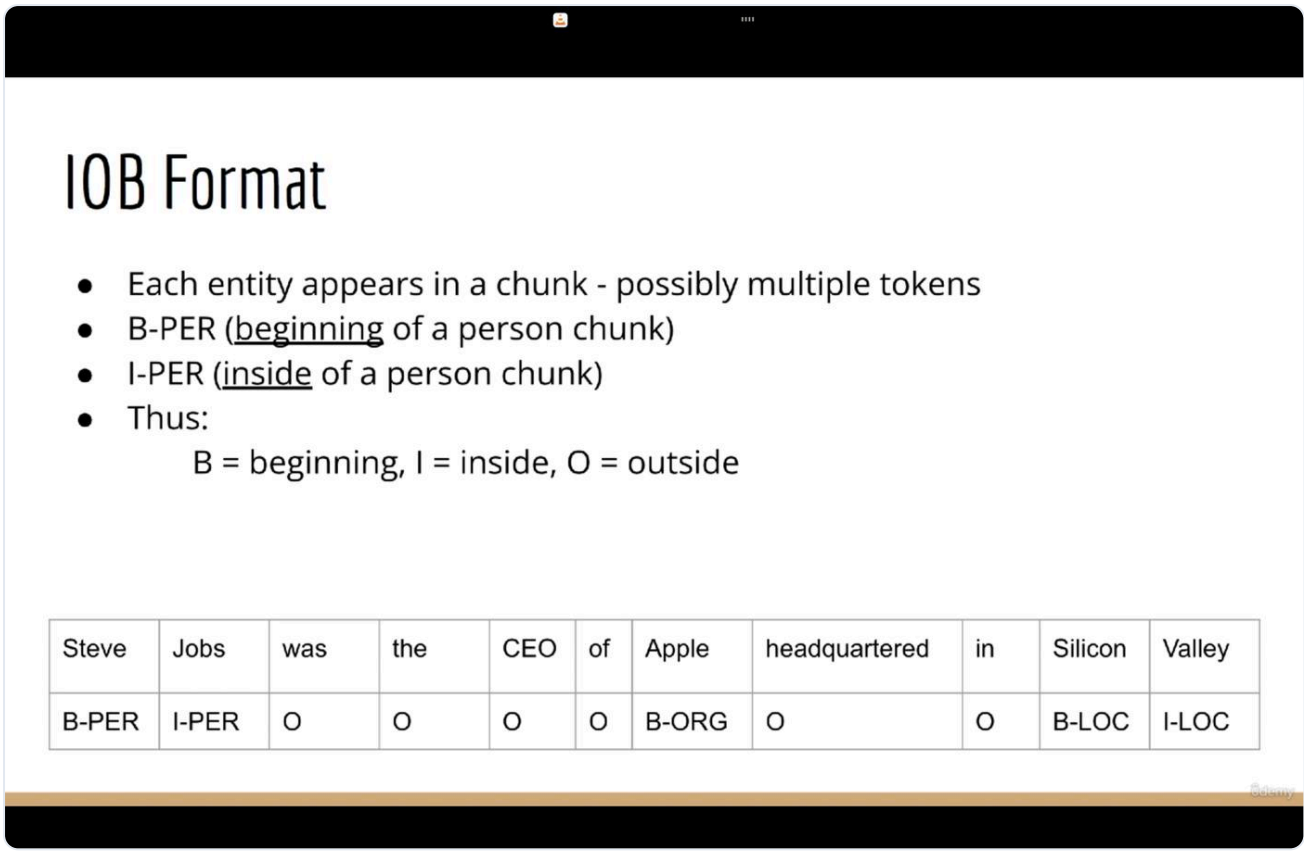

FIGURE 3.154

So B stands for beginning. I stands for inside, and O stands for outside meaning outside of any chunk. Luckily, these details don't really concern us, since ultimately our dataset looks exactly like POS tagging from the previous lecture. We have a sequence of words and each word is map to a tag. Okay.

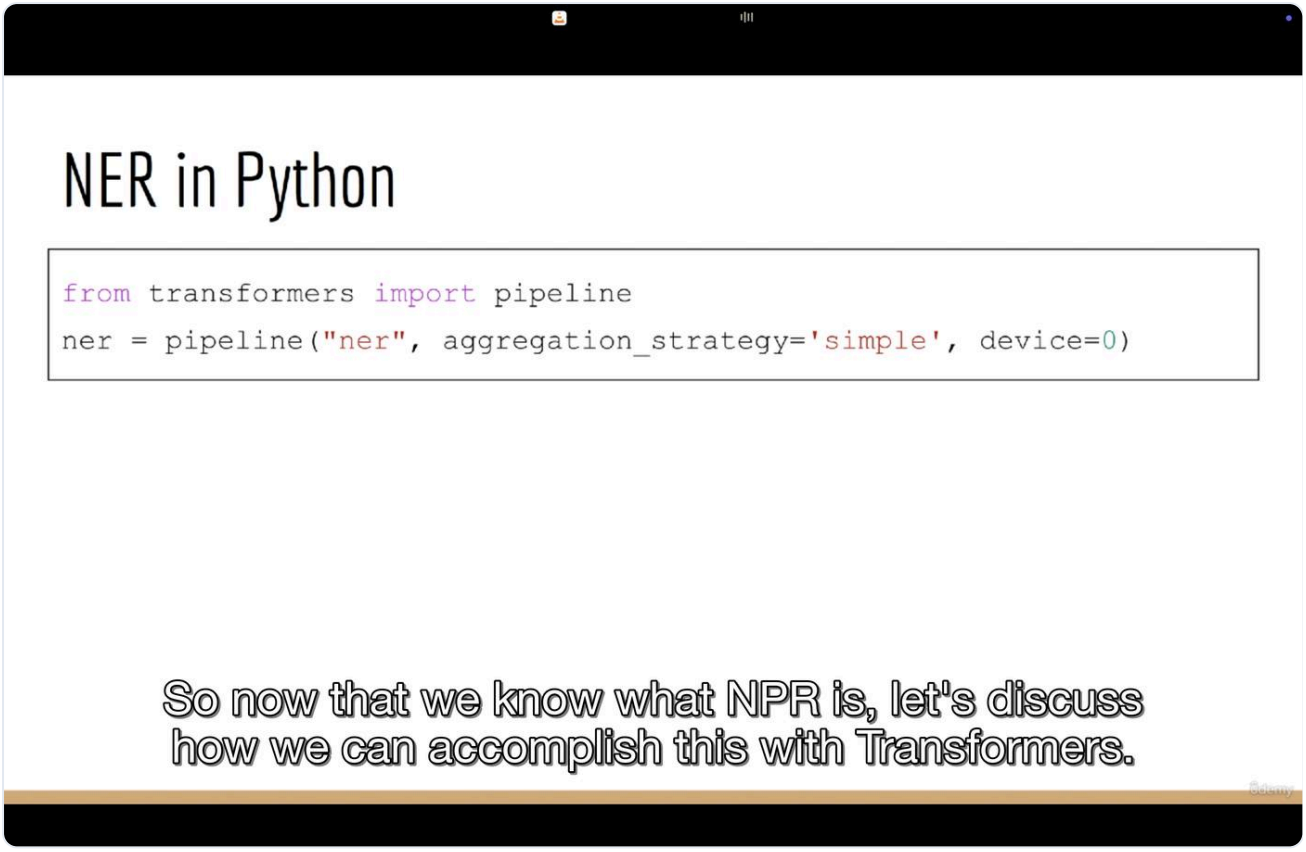

FIGURE 3.155 | NER in Python

So now that we know what NER is, let's discuss how we can accomplish this with Transformers.

As usual, we create a pipeline object and we specify the task as NER. Note that we also pass in an argument for aggregation strategy, which in my opinion, is best set to simple. As you recall, some entities can span multiple tokens. Using this method ensures that instead of returning the same entity for each token separately, it will put them all together. Unfortunately, the Hugging Face documentation is not that great.

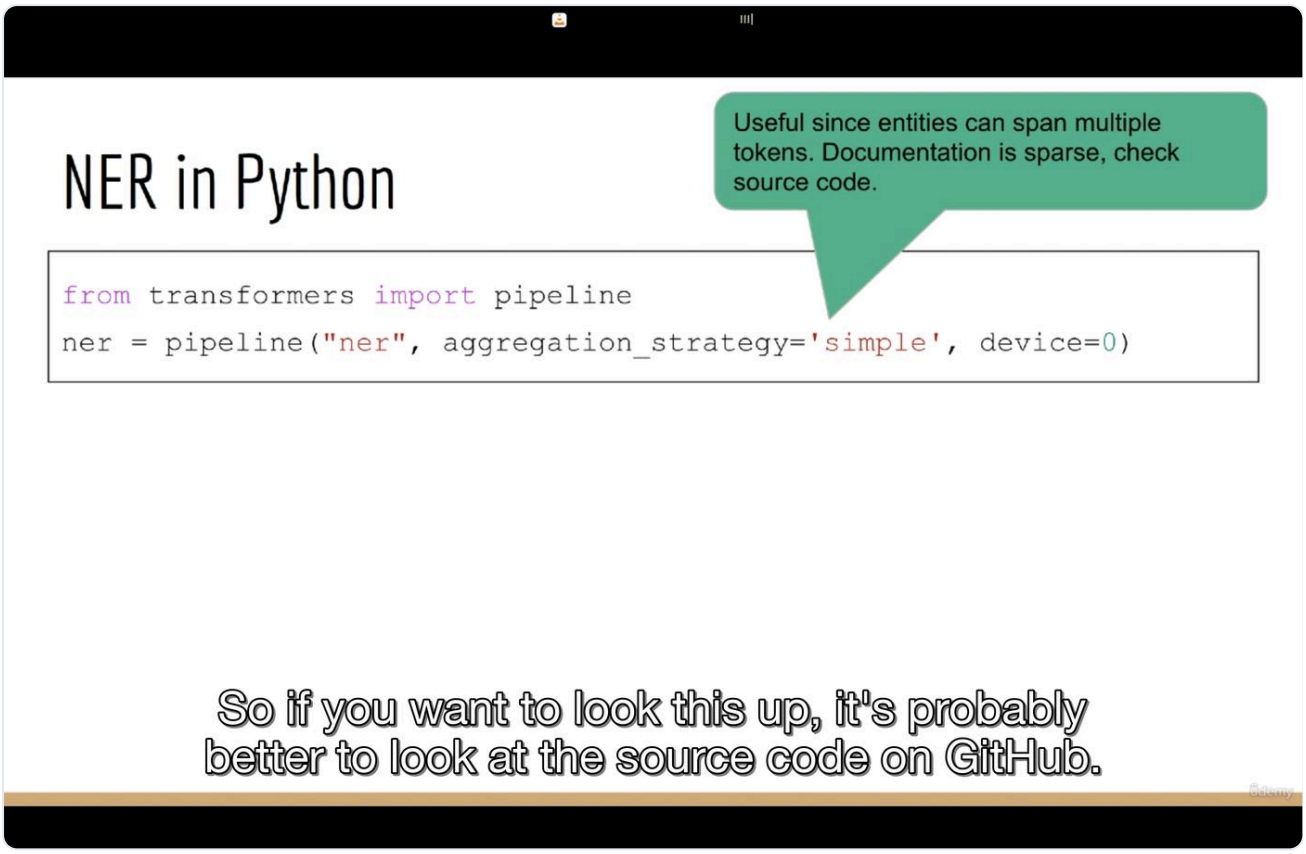

FIGURE 3.156 | Useful Since Entities Can Span Multiple

So if you want to look this up, it's probably better to look at the source code on GitHub.

In any case, when you do this, the output will look as follows.

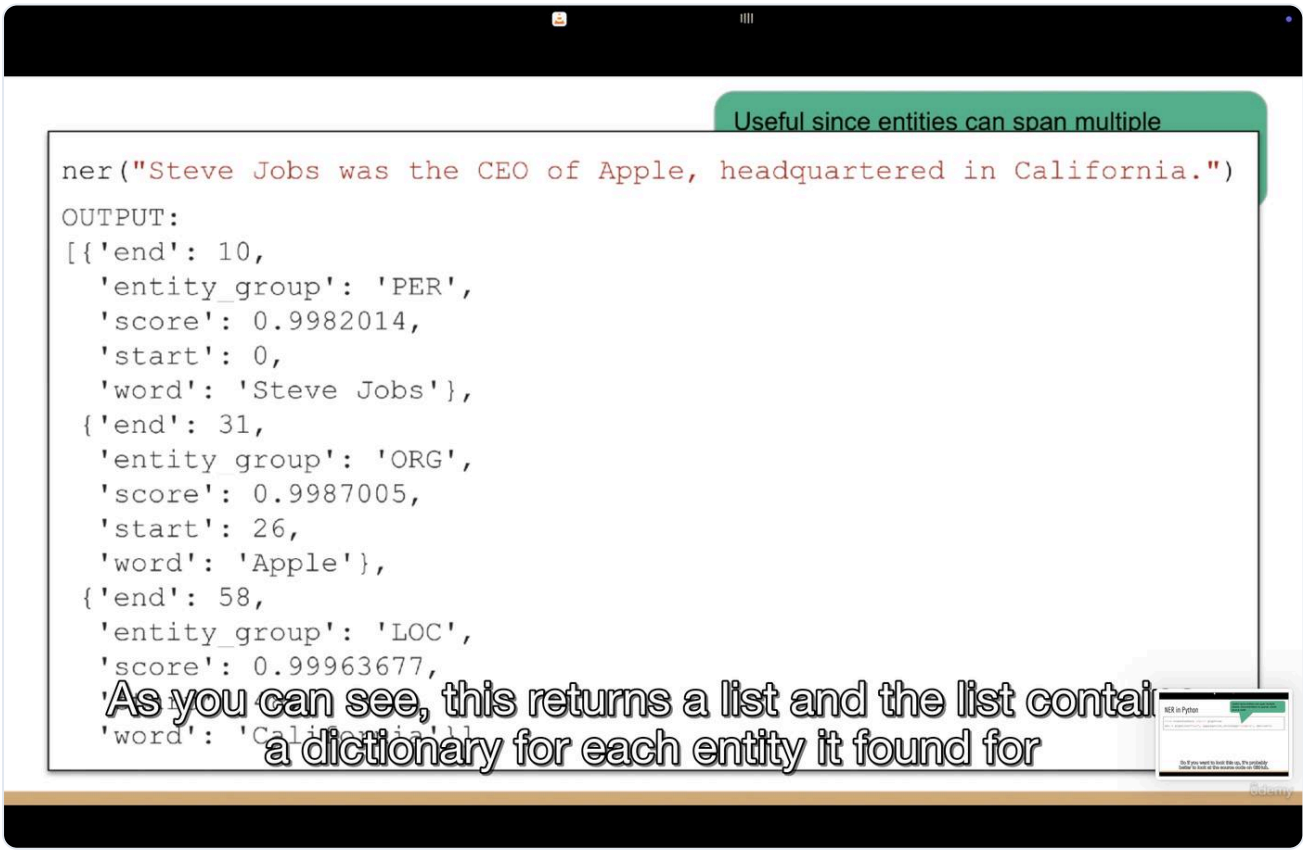

FIGURE 3.157

As you can see, this returns a list and the list contains a dictionary for each entity it found for each entity.

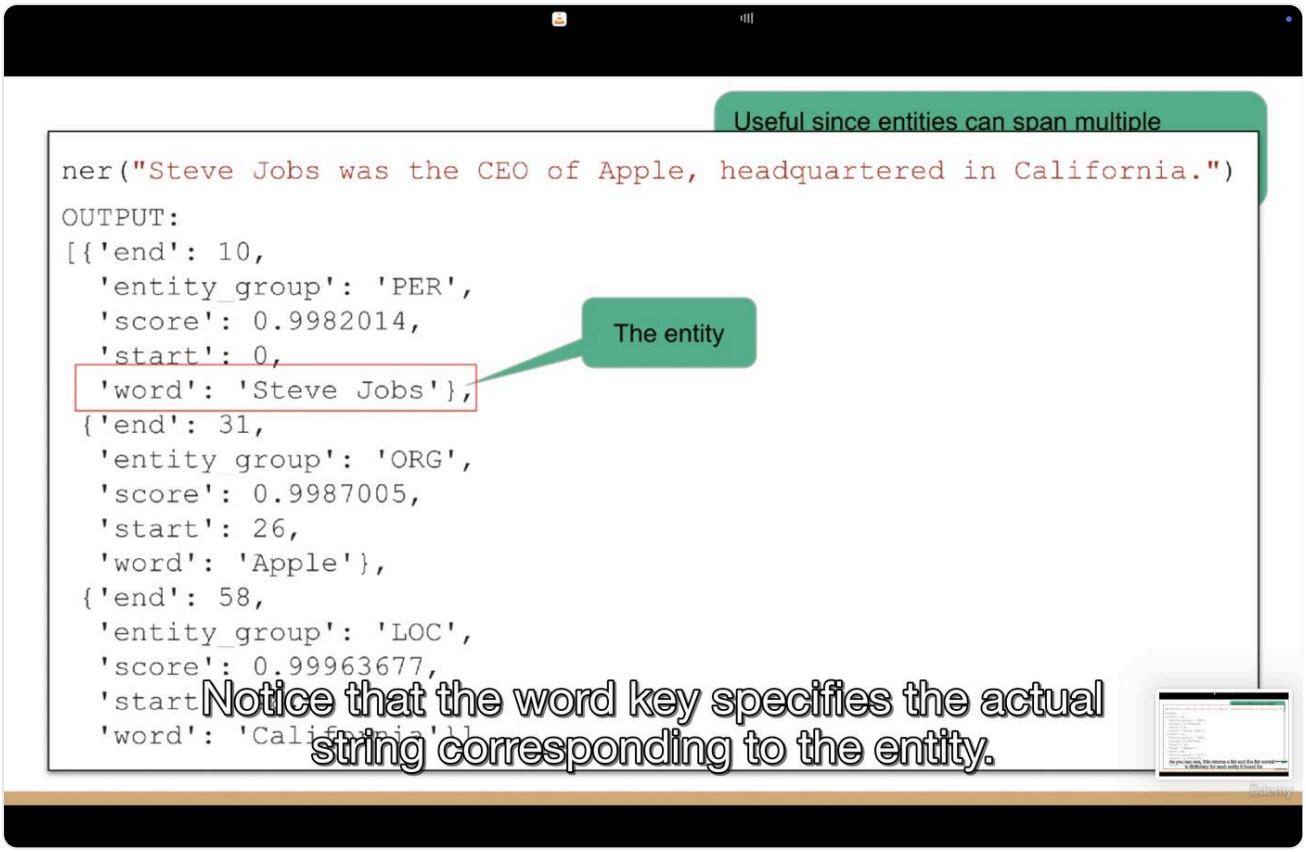

FIGURE 3.158

Notice that the word key specifies the actual string corresponding to the entity. Sometimes this will contain multiple actual words. Note that the key entity group specifies the type of entity.

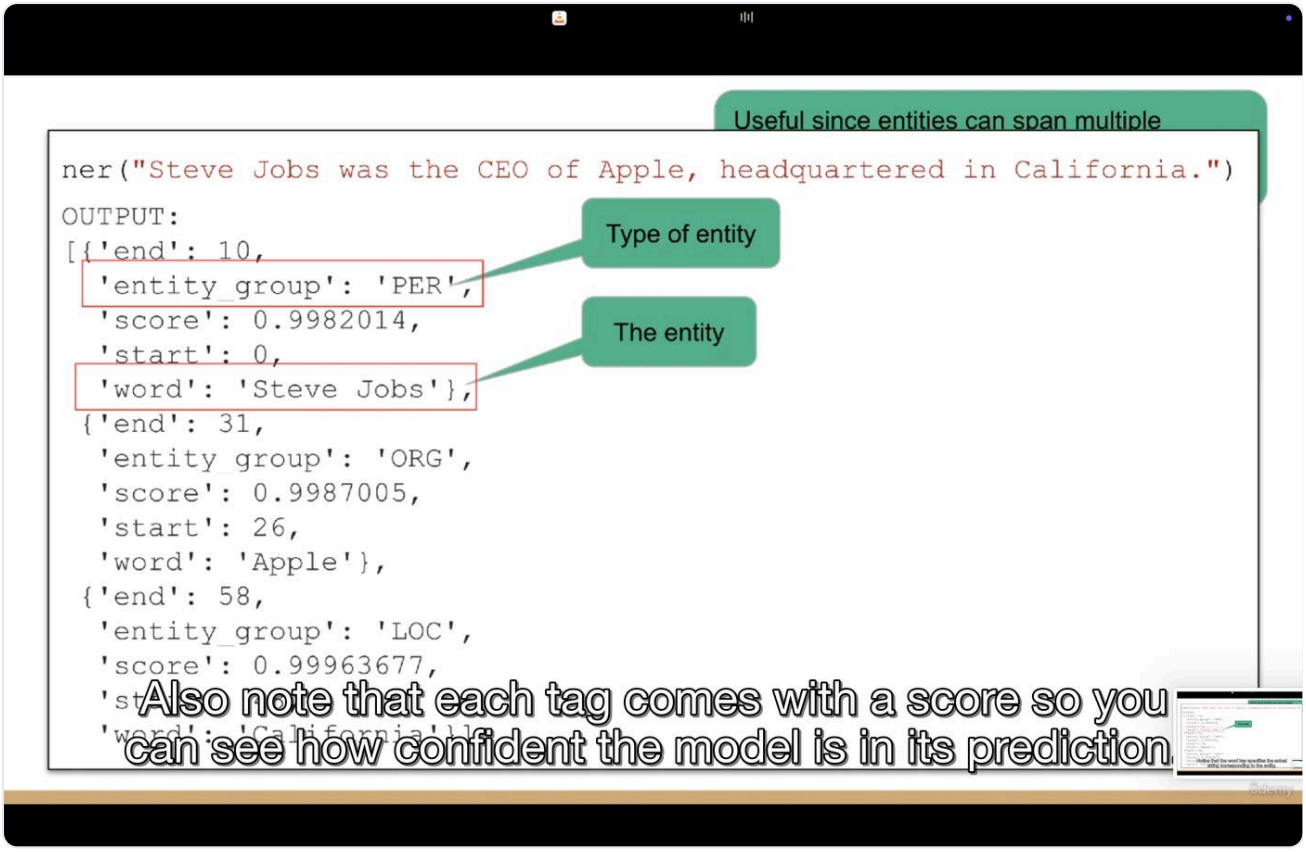

FIGURE 3.159

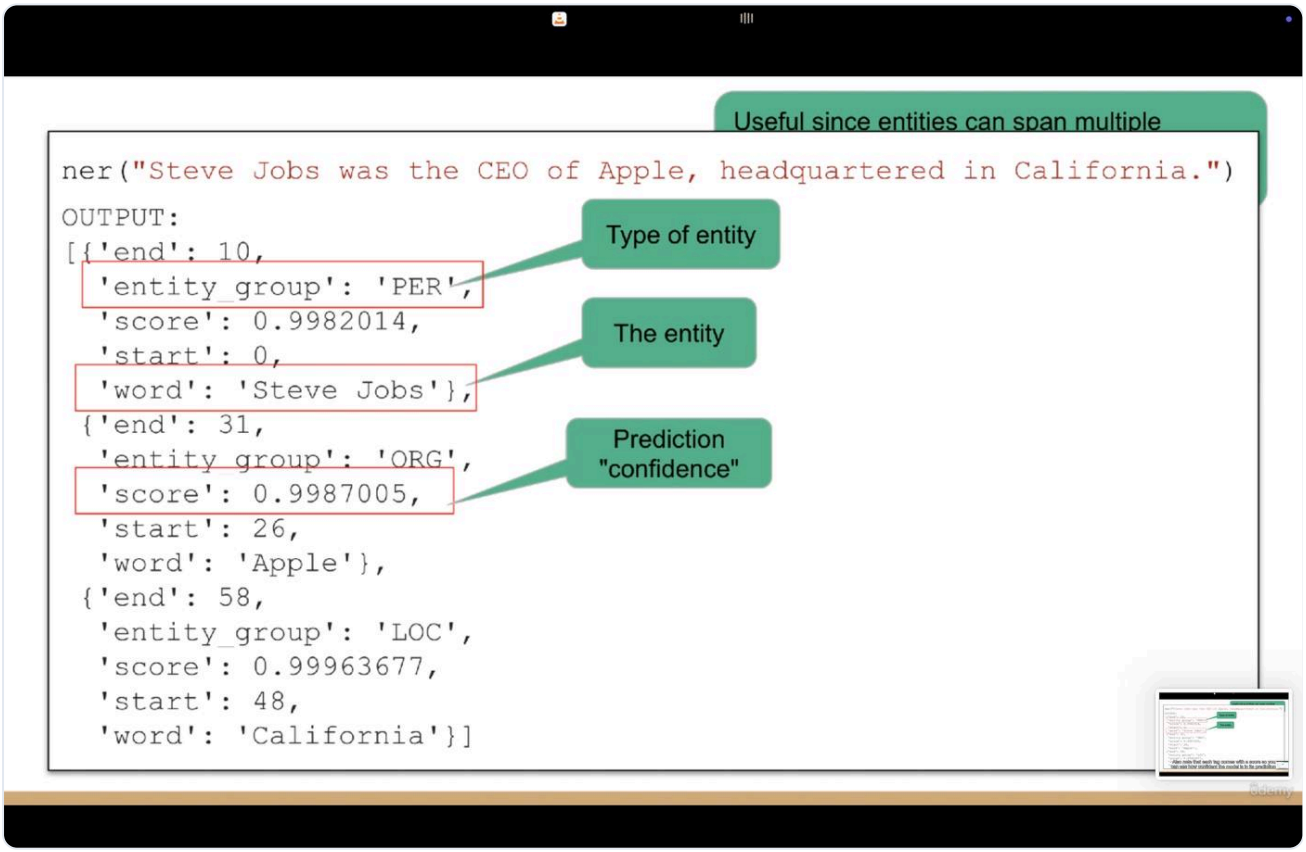

FIGURE 3.160

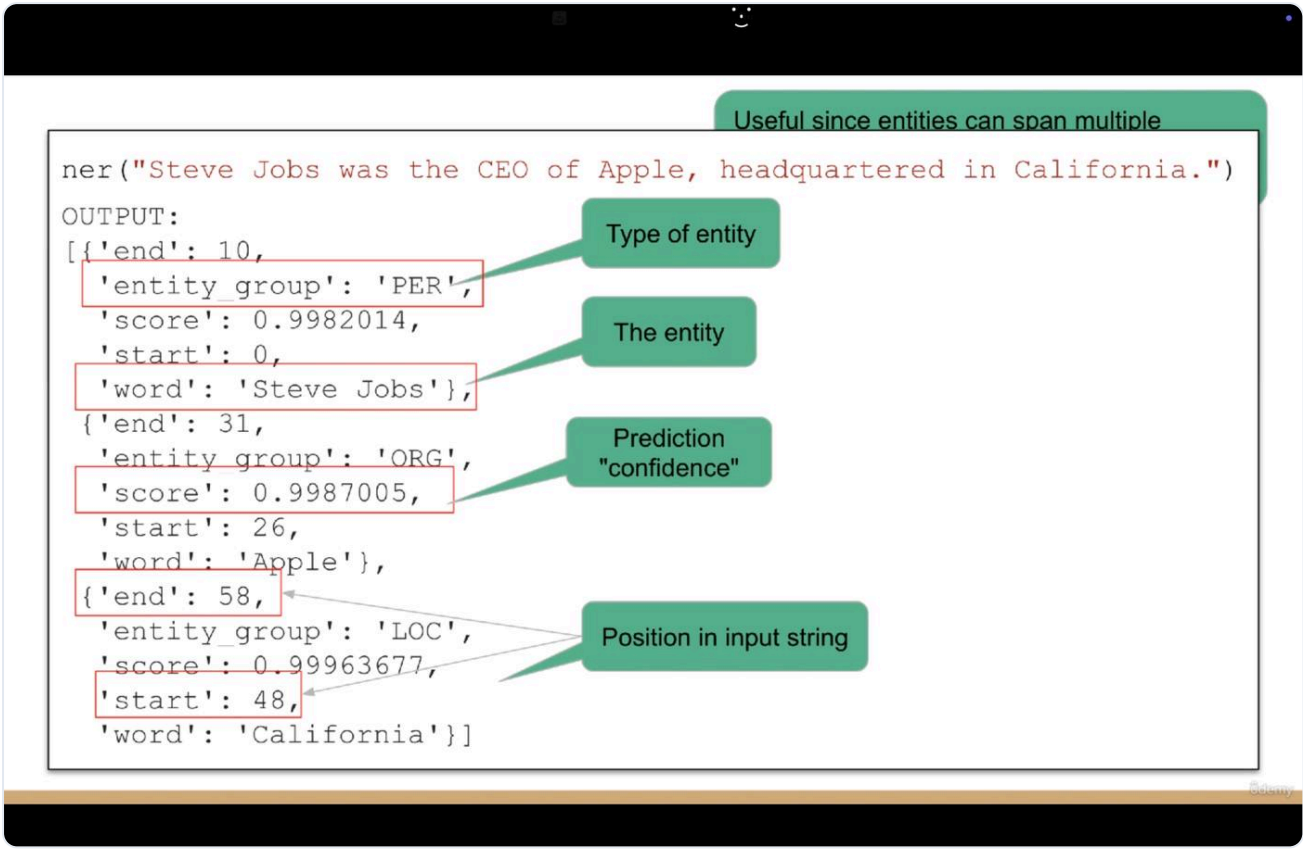

FIGURE 3.161

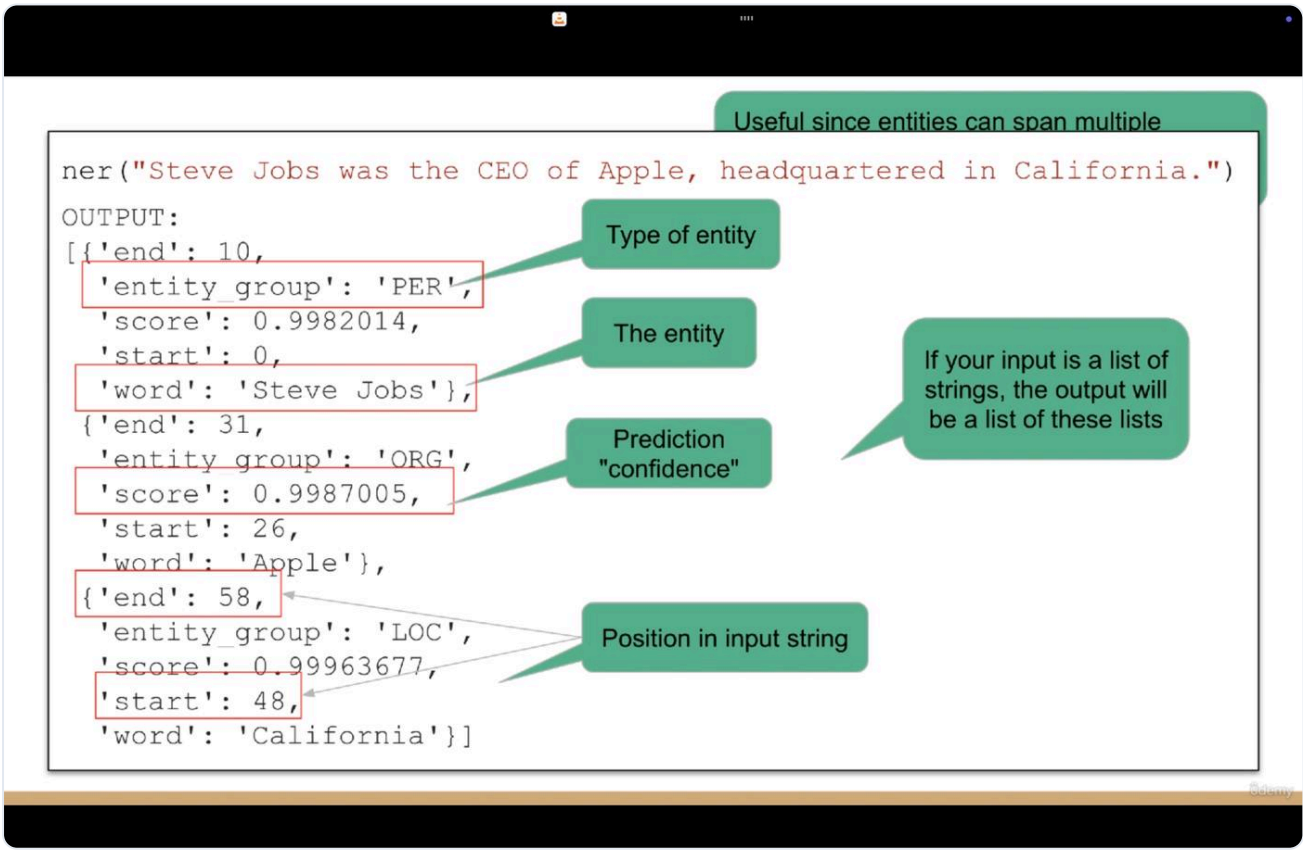

FIGURE 3.162

Also note that each tag comes with a score so you can see how confident the model is in its prediction.

And finally, note that the result includes the start and end position of the entity with respect to the input. This will be useful in the coming notebook and as usual, note that if you pass in multiple strings in a list, the above result would be a nested list inside another list.

10 · Named Entity Recognition (NER) in Python

See the notebook here.

So in this lecture we'll be looking at how to do named entity recognition with Transformers. We'll begin by installing Transformers. The next step is to import pipeline as we normally do. The next step is to create an NER pipeline. Note that if passed in an argument for aggregation strategy, which is set to simple. The next step is to import pickle, since we'll need this to load in our data set. The next step is to download our dataset called CoNLL 2003, which is currently the most popular NER dataset.

Now one thing to be aware of is that if you look up above to see what model we downloaded, it was actually fine tuned on this exact dataset. So it shouldn't be surprising if our model performs very well. As an exercise, you might want to try a different NER dataset and use that instead. The next step is to load in the train and test data using pickle. So the reason these are pickled objects is because the original data wasn't in a very useful format. So I did some work to make the format easier to work with for the purpose of this course.

As an exercise, you're welcome to download the original and write the code to parse it yourself. The next step is to just see what the data looks like. So as you can see, we have a list. And inside that list we have more lists. Inside the nested lists, we have tuples containing words and corresponding tags. Now it would be useful to separate the tokens and tags. So that's what this next block does. You don't have to worry about how this code works, but feel free to read it yourself as it's not that complex.

Note that for this notebook we will only be looking at the test set. As you recall, transformers are quite slow and as such, this notebook would take too long if we looked at the train setup or both the train and test sets. As an exercise, you should try that by yourself. The next step is to just look at one of the inputs as an example. So as you can see, we've successfully separated the inputs and targets so that each input is now just a sequence of words.

The next step is to import the TreebankWordDetokenizer. So why do we need to do this? Well, as you recall, the Transformers pipeline works with raw text, not tokenized text. Therefore, when we pass in text into the pipeline, it needs to be joined into a single string. Now, you might think, why can't we just use the regular string join? As you recall, this is just a basic function in Python. Well, this won't work, since not all tokens should be joined the same way.

For example, if we have a period, there should be a space after, but not before. For regular words, they should be separated by one space. But for hyphens, there should be no spaces. As such, we need a function that knows these rules to construct our sentences correctly. So the next step is to just test the detokenize method. Okay. So the results look correct. The next step is to look at the corresponding targets just as a reference. The next step is to test our model on the sentence we just looked at.

So these results look like they match the target, which means our model works. You should confirm to yourself that this is the case before moving on to the rest of the notebook. The next step is probably the most advanced part of this section, so please feel free to skip this if you like, as it's not necessary to understand what's going on. Basically, the reason we need this function is because the model only outputs the positions of the entity with respect to a single string input.

But we need to know the predictions for each word. As you recall, this is the format of our data set. For each input sentence, we have a list of words and each of these words have a tag. This is not the format of the transformer output. However, the transformer output is arguably more useful in the real world depending on what your code looks like. In any case, the only reason we need this is because I wanted to compute the accuracy of this model on the test set.

So if you don't care about that, then the rest of this notebook can be skipped. Now, in my opinion, probably a majority of you will not be interested in this code, so I'm not going to explain it in this lecture. However, you can feel free to let me know on the Q&A if you think differently. Basically, what this function will do is it will convert the results returned from Hugging Face back into a list of tags in the same format as their targets. This is so that they can be directly compared in the later steps.

The next step is to test our function above. So we begin by first tokenizing the input. The next step is to pass this input into our initial pipeline to get NER results. The next step is to call our function to get a list of predicted tags, which is called P tags. Okay. So at the very least, it runs without any errors. The next step is to import some functions from scikit-learn to help us compute some metrics. The next step is to compute the accuracy of our prediction.

As you can see, we get a perfect score. The next step is to check what our predictions actually look like in comparison to the targets. So as you can see, they are exactly the same as expected. Okay. So now that we know that this works, the next step is to do this for the whole test set. The first step will be to tokenize each sentence. The next step is to pass in our tokenized inputs into the pipeline. The next step is to use the function we wrote above to convert the model outputs into lists of tags.

Now, as you recall, each sentence has a different length. So what we have now is a jagged list of lists. Unfortunately, these don't work well with typical data science functions. So we'll need to flatten these lists. Note that this function was just copied from Stack Overflow. The next step is to flatten both the predictions and targets. The next step is to compute the accuracy. As you can see, we get over 99%, which is pretty good. The next step is to compute the F1.

As you can see, we get over 95%, which is pretty good as usual. F1 is a bit less optimistic. As an exercise, you should compare these results with other models you've studied, such as RNN.

11 · Text Summarization

So in this lecture we will do a brief introduction to text summarization and how to accomplish this task with Transformers. Now, as always, we're going to answer the question Why is this practical?

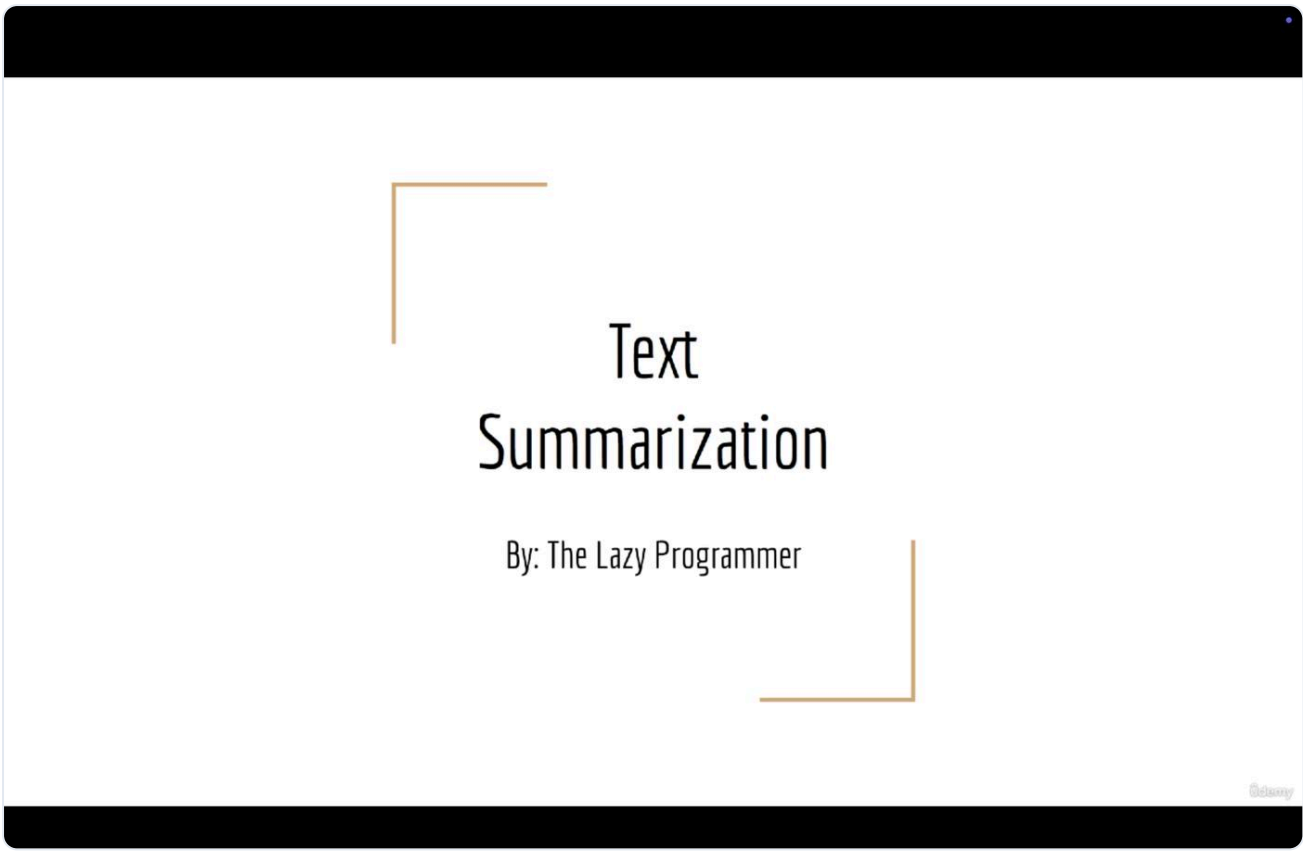

FIGURE 3.163 | By

And I hope that this is another one of those tasks where the answer is very obvious. So why should we summarize text?

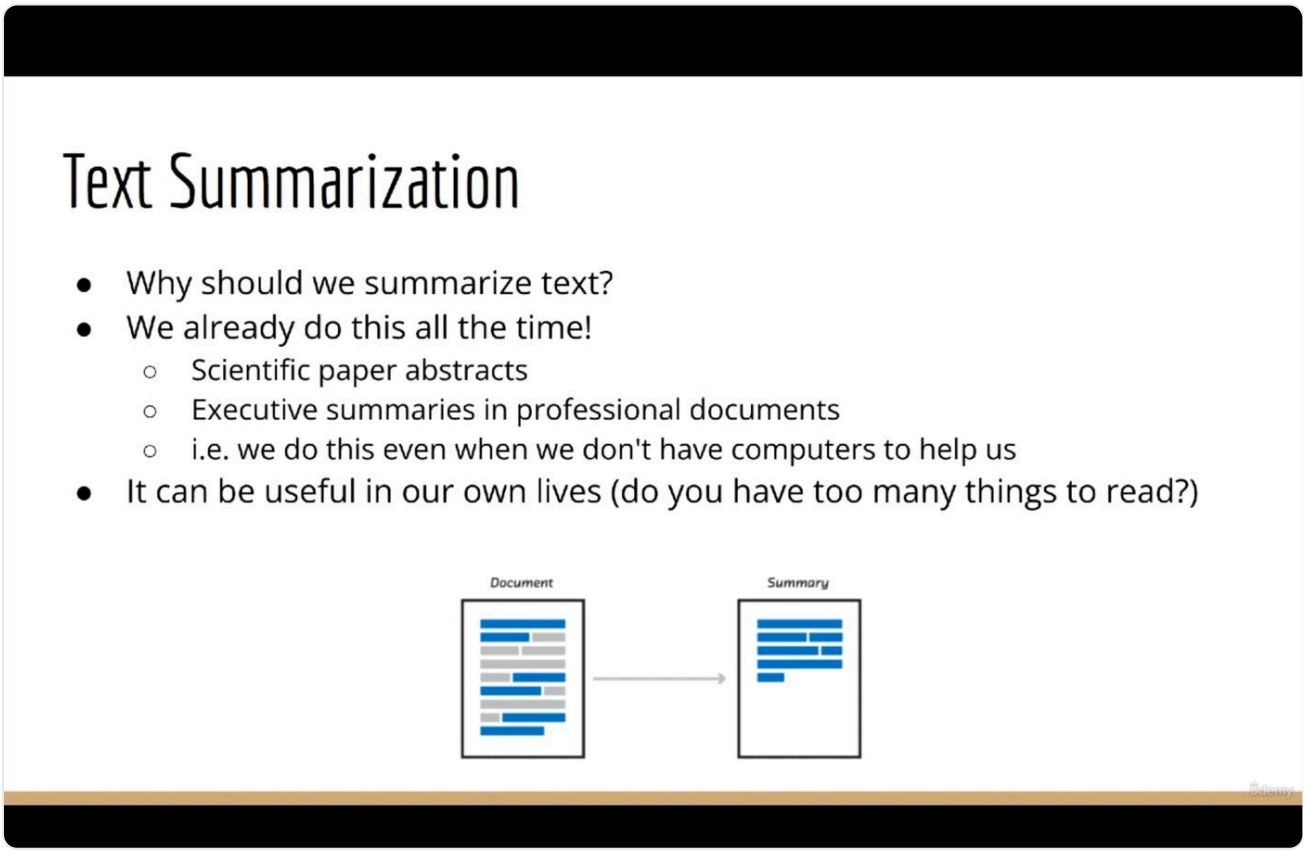

FIGURE 3.164 | Text Summarization

Well, notice how we already do this all the time. For example, when you read a scientific paper, it always comes with an abstract, and abstract is a summary of that paper.

So you don't need to read the whole paper if you just want to understand the basic idea. You can get away with just reading the abstract and then read the paper in full. If you want more details. Professional documents also have a similar concept, although instead of calling it an abstract, it's sometimes called an executive summary. So clearly this is a useful concept in the real world. Since we do this already, even without the help of computers and obviously we can extend this concept to our own lives.

I know that for myself, my reading list is longer than the time I have to spend reading. So it would be very useful if I could reach the same outcome or reach the same conclusions by reading only the most important parts of each document that I have to read. Another reason we might want to build text summarization systems is for pushing forward the field of artificial intelligence. Now, you might think this seems kind of trivial. build AI algorithms to develop the field of AI.

But that's not what I mean. What I mean is that text summarization is kind of a critical aspect of learning. For instance, think of everyday conversations that you have with other people. Sometimes when people explain things to you, you will paraphrase or summarize what they've told you in response. Then the other person might say, Yes, you're correct or No, you're not correct.

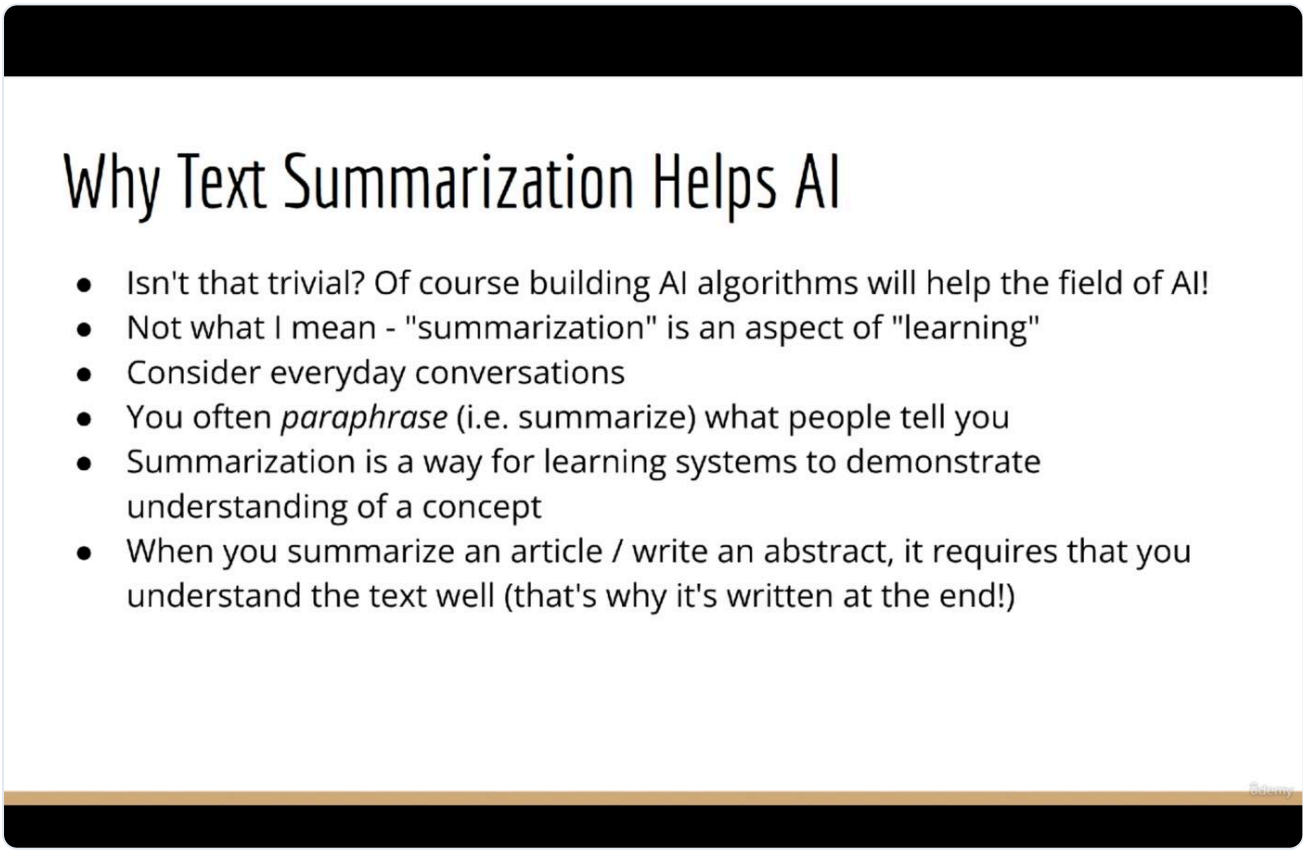

FIGURE 3.165 | Why Text Summarization Helps AI

So summarization is a way for learning systems, whether that's yourself or a machine, to demonstrate their understanding of a concept.

If you think about it further, you'll realize that whenever you summarize an article or write an abstract, it really requires you to have a good understanding of what you are trying to summarize. In other words, it seems to require a true intelligence. Here's another application for building summaries out of text.

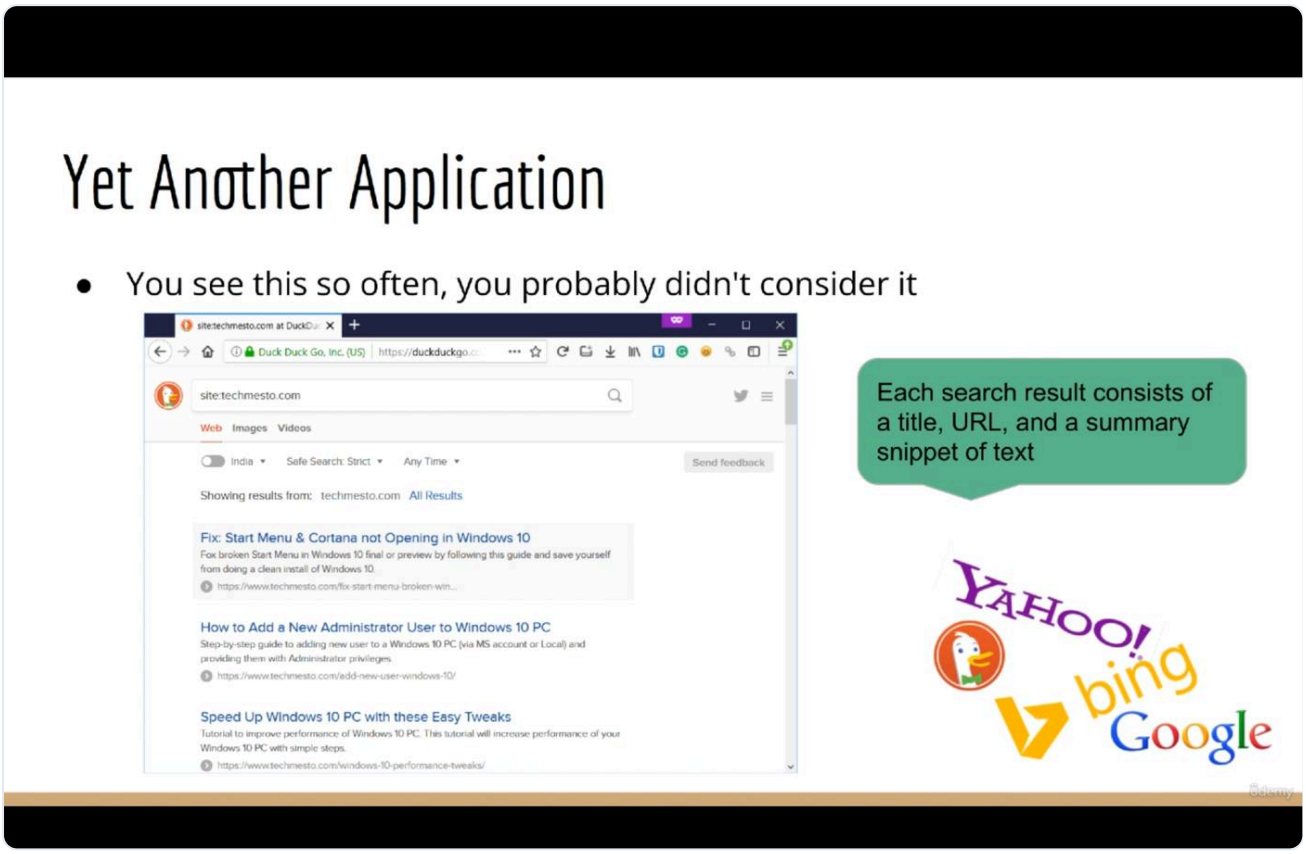

FIGURE 3.166 | Yet Another Application

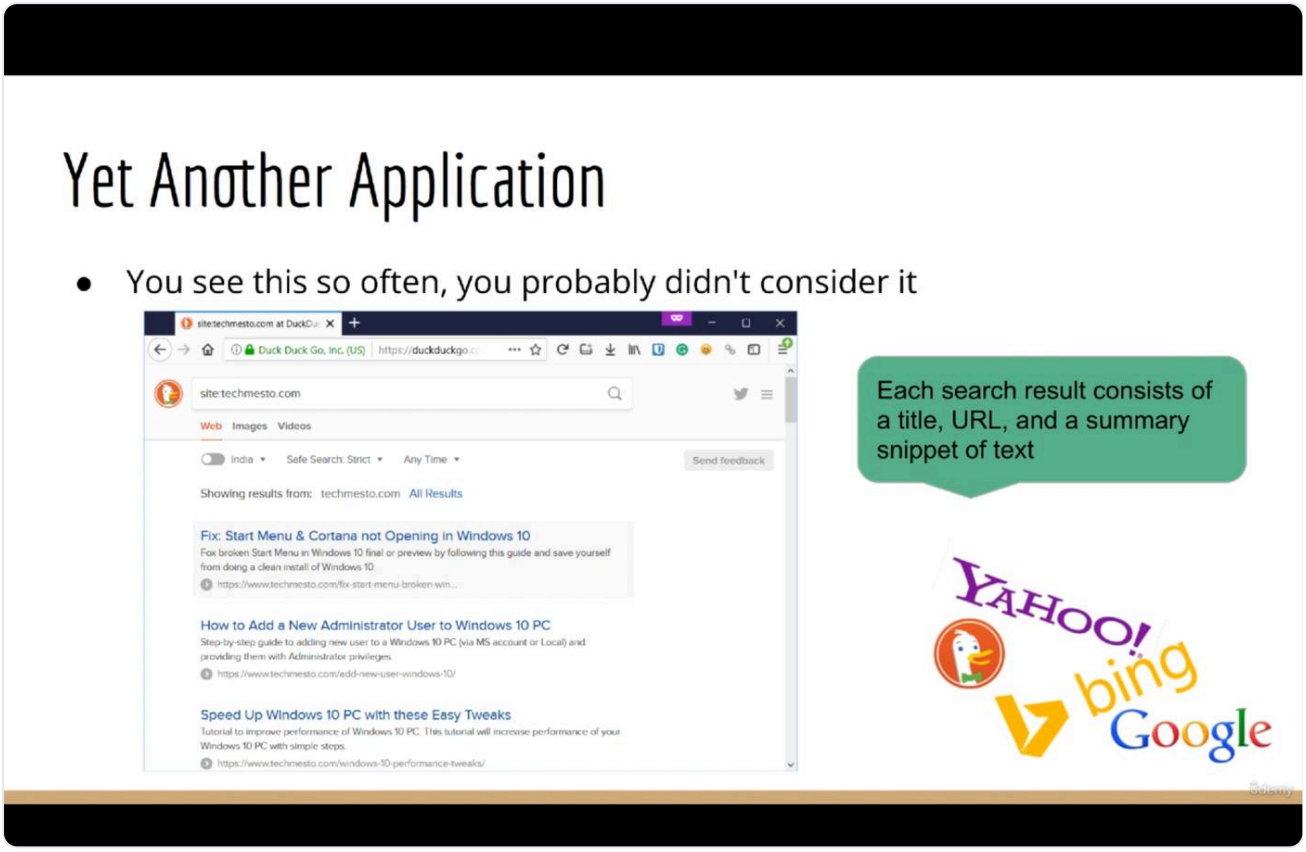

FIGURE 3.167 | Yet Another Application

In fact, you see this so often, you probably don't even realize that it happens. As you recall, our sort of window to the Internet is search engines such as Google, Bing, DuckDuckGo and so forth.

Now, without looking. Think about what you see after you type in a query. And while the answer is you see a bunch of search results and how are these search results presented? Well, typically, you have a Web page title, a link to the Web page and a corresponding summary, which is these days usually customized to your query. So this is another very important application. So now that you know why text summarization is useful, let's discuss one detail before we outline this section.

This detail is that we can actually categorize different methods of summarizing text. These categories are extractive and abstractive.

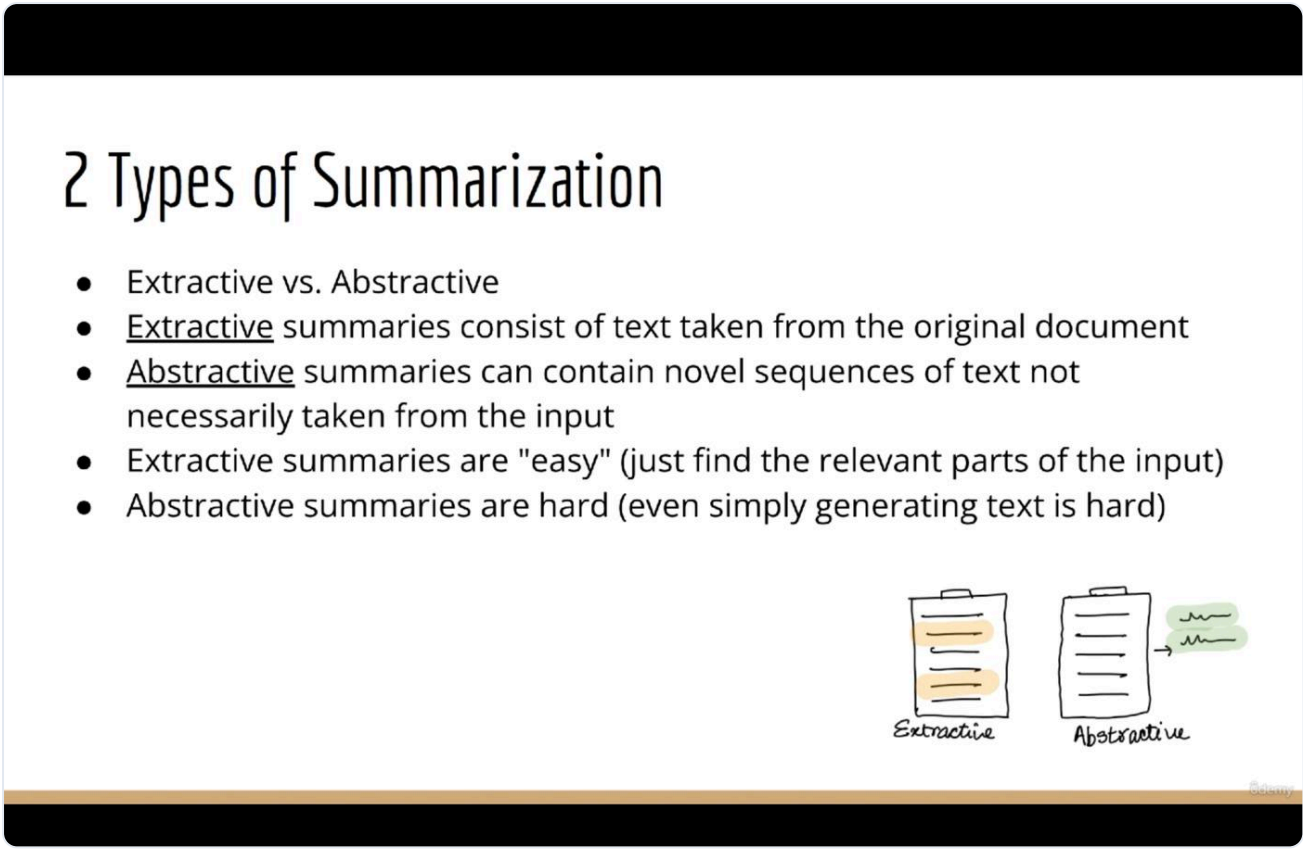

FIGURE 3.168 | 2 Types of Summarization

So what do these mean? Well, extractive summaries are summaries that are built out of the document itself. So the summary is a subset of the document that was given. In other words, the summary was extracted from the document. abstractive summaries, on the other hand, are novel sequences of text that are not just partial copies of the given document.

As you can imagine, extractive summaries are much easier to generate compared with abstractive summaries. With extractive summaries, we only need to identify the most relevant parts of the input. But with abstractive summaries, we need to develop a deeper understanding of the input, and we also need to convert that understanding into coherent text. Now using more basic techniques, extractive summaries are usually the only choice in order to build abstractive summaries.

We need to have models that not only understand text but can also generate new text.

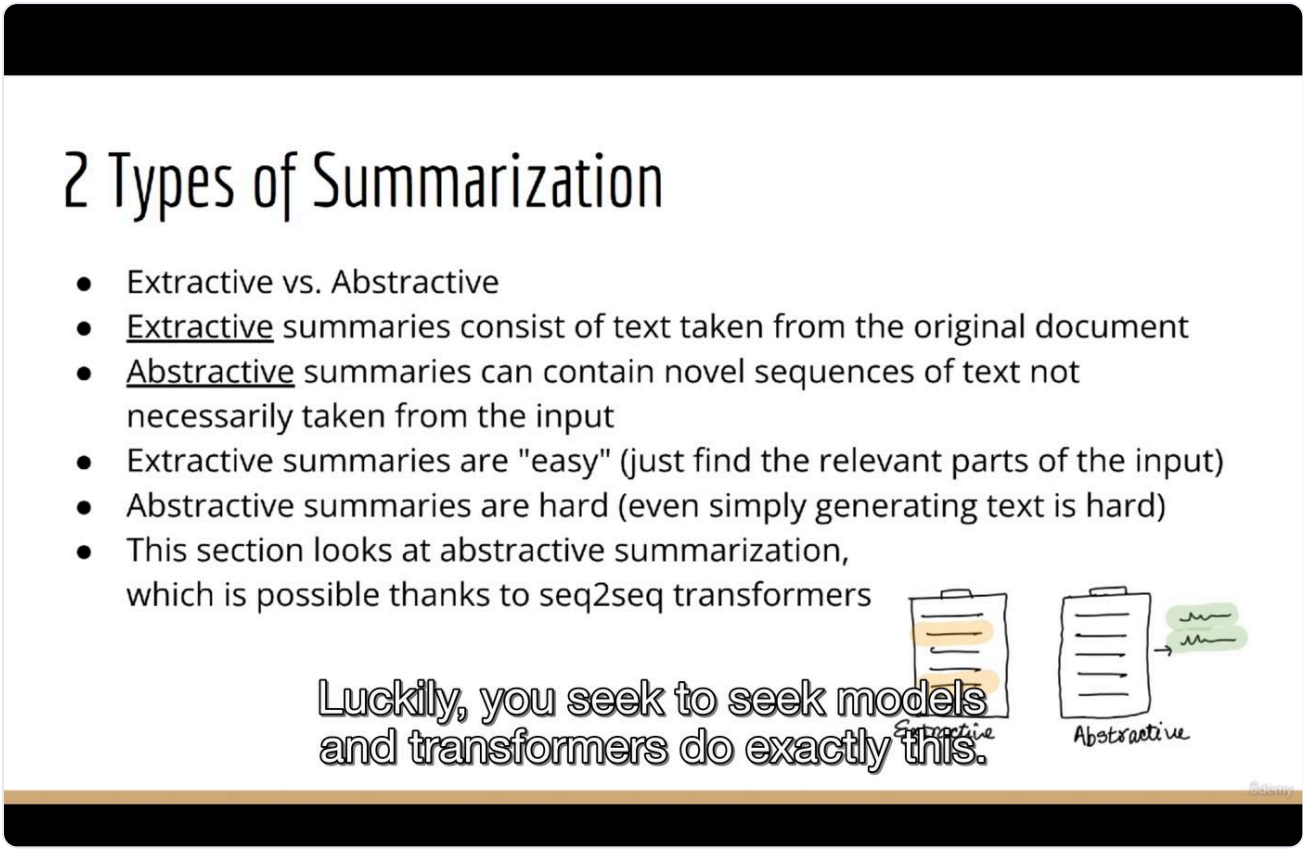

FIGURE 3.169 | 2 Types of Summarization

Luckily, you seq2seq models and transformers do exactly this. Okay. So now that you understand what text summarization is all about, let's look at how we do it in Python. As usual, we begin by creating a pipeline object, passing in the string summarization as the one and only argument.

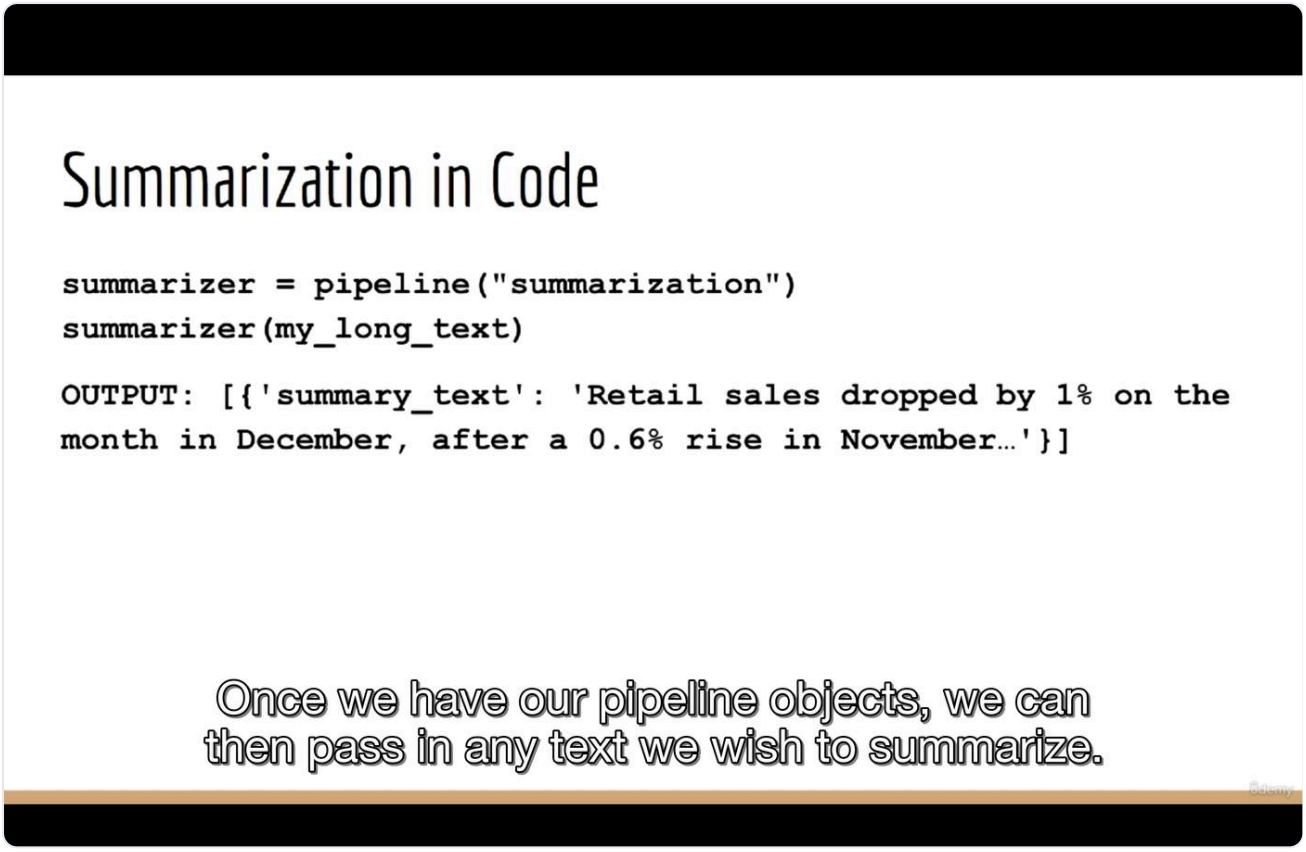

FIGURE 3.170 | Summarization in Code

Once we have our pipeline objects, we can then pass in any text we wish to summarize.

The output will be a list containing one or more dictionaries, each of which contains a key called summary text. So very simple. As per usual.

12 · Text Summarization in Python

See the notebook here

So in this lecture, we'll be looking at how to do text summarization with transformers. We'll begin by downloading our dataset, which is the BBC news dataset. This contains nice, long articles that would be good candidates for the summarization task. The next step will be to install the Transformers library. Once again. The next step will be to import everything we need for this notebook. The next step is to read in our data using pd.read_csv. The next step is to call df.head to remind ourselves what this data looks like.

So it's a two column dataframe containing both text and the corresponding label. Note that for this lecture we will only be interested in the text. The next step is to pick an article at random. Note that for this first example, well, choose an article from the business class. Also, note that we set the random seed to 42 so that you can replicate these results. The next step is to define a text wrapping function that will be convenient since the text is so long that it will go off the page.

The next step is to simply print the news article we selected. Okay. So the article title is Christmas Sales Worst Since 1981. So it's basically an article about how sales around the holiday season were bad. The next step is to instantiate a summarization pipeline object. Okay. So notice

that by default this uses a model called DistilBART as a little preview of what you will learn if you study Transformers further. BART is an example of a sequence to sequence model.

You may recall that such models are useful for tasks like language translation, where the output sequence length is not necessarily equal to the input sequence length. In these architectures, there is both an encoder and a decoder, meaning that the model will do some kind of encoding of the input, and then the decoder will take that encoding and convert it into a new sequence of text. In other words, this is not the simple case of extractive summarization.

Instead, this is abstractive since the model has to generate a summary based on a representation of the input. The next step is to pass our article into the summariser we just created. Note that I've split the article by the first newline and kept only the second part. This is because the first line of every article is the title, which in a sense can be thought of as a one line, a summary of that article. As such, we do not want to pass the title into the summariser, since that would kind of be like cheating.

Okay. So note that this returns a list of one item, and that item is a dictionary with one key, which is summary text. The next step is to define a function that will take in a document and compute the summary like we did above and print out the result using the wrapping method we wrote earlier. Okay. So let's try your function. Okay. So the summary is retail sales dropped by 1% on the month in December after a 0.6% rise in November. Clothing retailers and non specialty stores were the worst hit, with only Internet retailers showing any significant growth.

The last time retailers endured a tougher Christmas was 23 years ago, when sales plunged 1.7%. So what should we notice about this summary? Firstly, it seems to be a good summary. It tells us the essence of the article, plus a few relevant details. Also note that these sentences do not exactly appear in the input. If you cross-reference the summary with the inputs, you'll see that the first two sentences are just parts of sentences that came from the input.

The final sentence is nearly the same as the input, except that the original said it 23 years previously, while this one says 23 years ago now, it's important to note that this summary is still very close to being extracted. It's not as if the neural network generated something completely different. The next step is to select another random article from the entertainment class. Okay. So the title of this article is Good REM Wins Top Female MTV Prize. The text of the article mentions other musicians and which awards they won.

Okay. So now let's summarize this document. So the summary is the 21 year old singer won the award for Best Female Artists. Australian Idol runner up Shannon Noll took the title of Best Male at the ceremony. Other winners included Green Day, The Black Eyed Peas, Missy Higgins and Green Day. The VH1 first Music Award went to Cher, honoring her achievements within the music industry. Okay. So notice that the first two sentences in the summary are actually both from the same sentence in the input.

It seems that our summariser decided to split that sentence into two. Notice that the third sentence is very strange. It says Green Day, the Black Eyed Peas, Missy Higgins and Green Day again, which is obviously a mistake. So notice how when you use that abstract method, all of a sudden more errors become possible.

13 · Neural Machine Translation

So in this lecture we will be discussing the next task in this section, which is on machine translation. As with many of our other tasks, I'm sure this needs no introduction, and most, if not all students will already be aware of the applications.

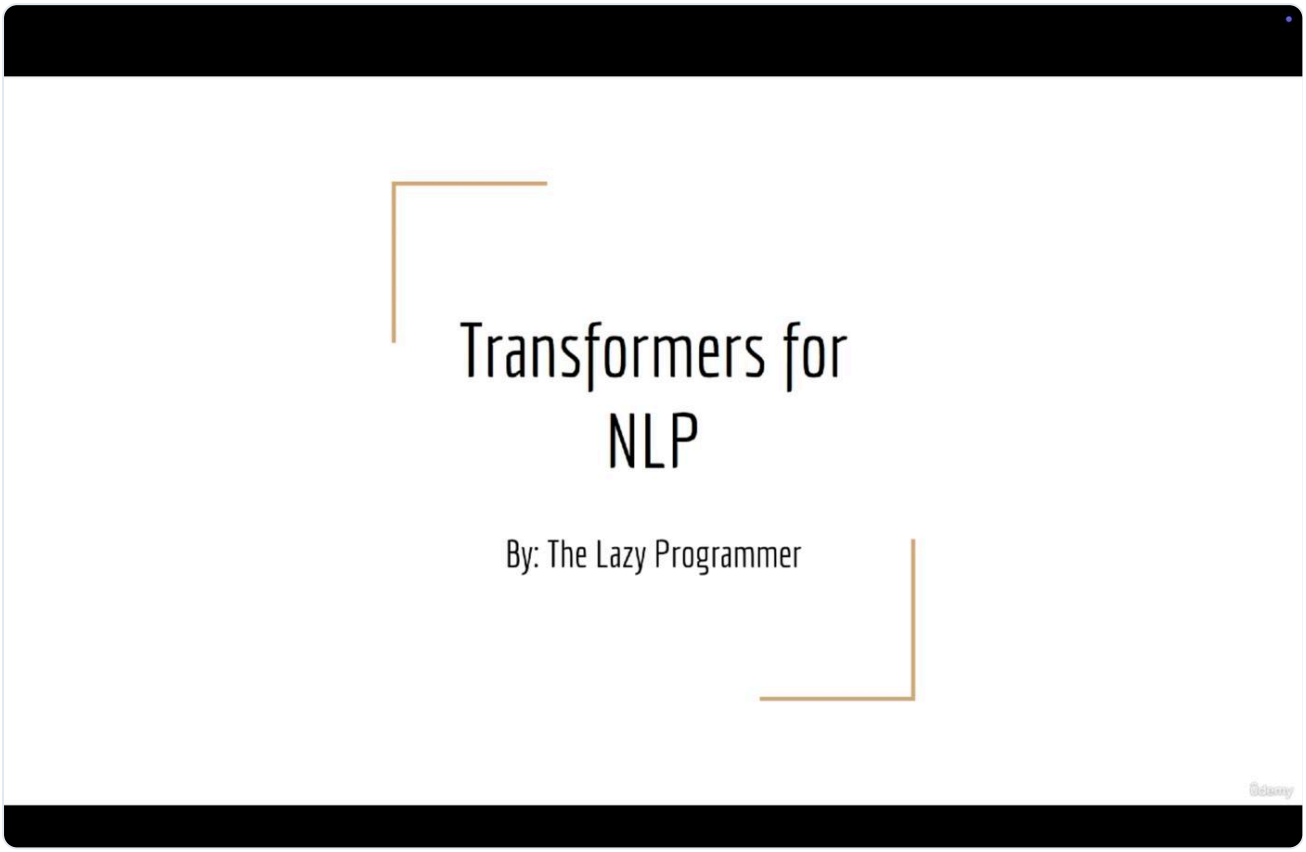

FIGURE 3.171 | Transformers for

But just in case, let's briefly go over why this is a useful task.

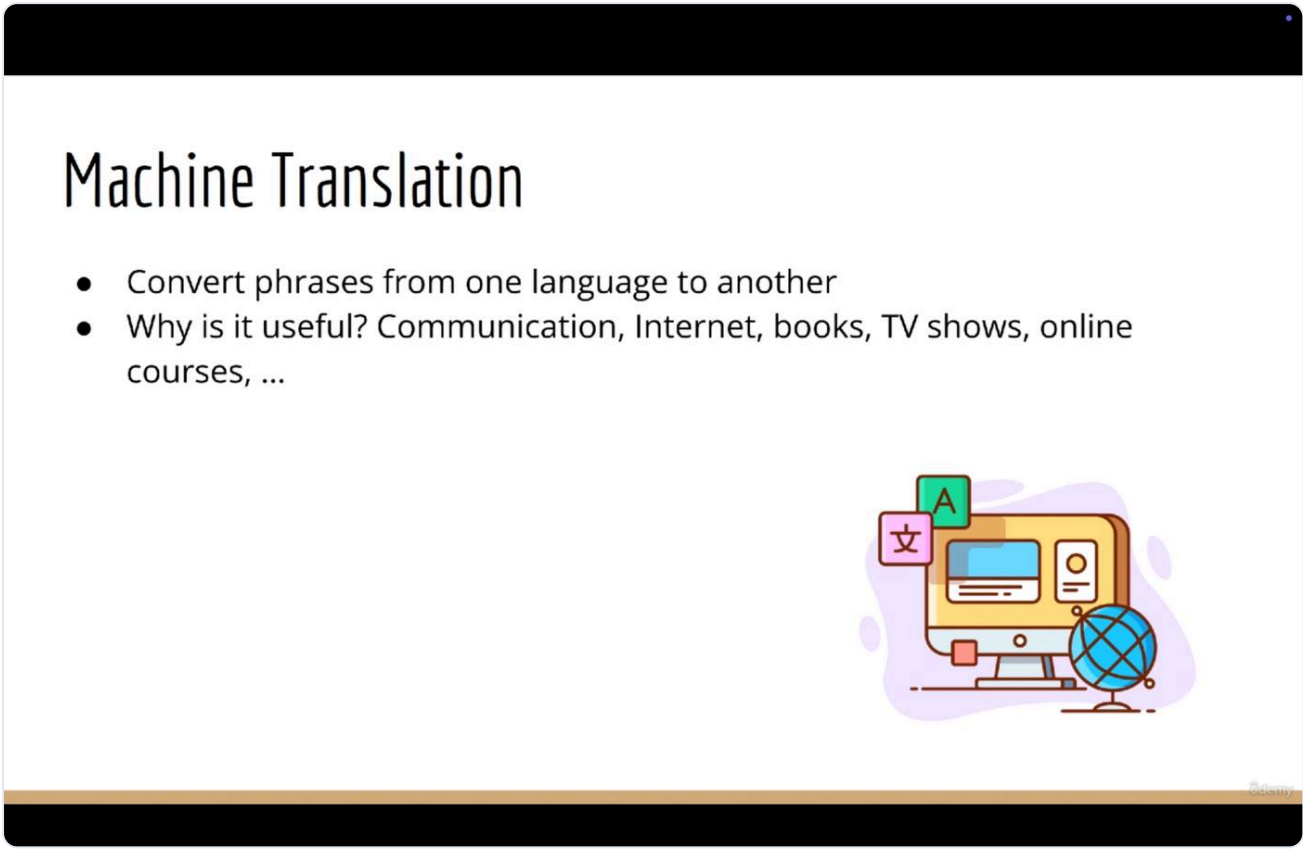

FIGURE 3.172 | Machine Translation

Firstly, to be clear, machine translation is a task where we convert phrases from one language into another language, for example, from English to French or Spanish to Portuguese.

Now, why is this useful? Well, if you only know one language, then you can only communicate with other people who know that language. On the Internet, you can only understand Internet content in your language. If you're reading a book, you can only understand books in your language. But with machine translation, a lot more content suddenly becomes available. Now you can watch TV shows from a different country by translating the subtitles. You can meet new people in foreign partnerships with businesses from overseas.

So clearly, machine translation opens up many opportunities that you otherwise wouldn't have had. One important fact to note is that like text summarization, machine translation is also an example of a sequence to sequence task, and thus it uses sequence to sequence architectures such as those that can be implemented with RNNs and transformers.

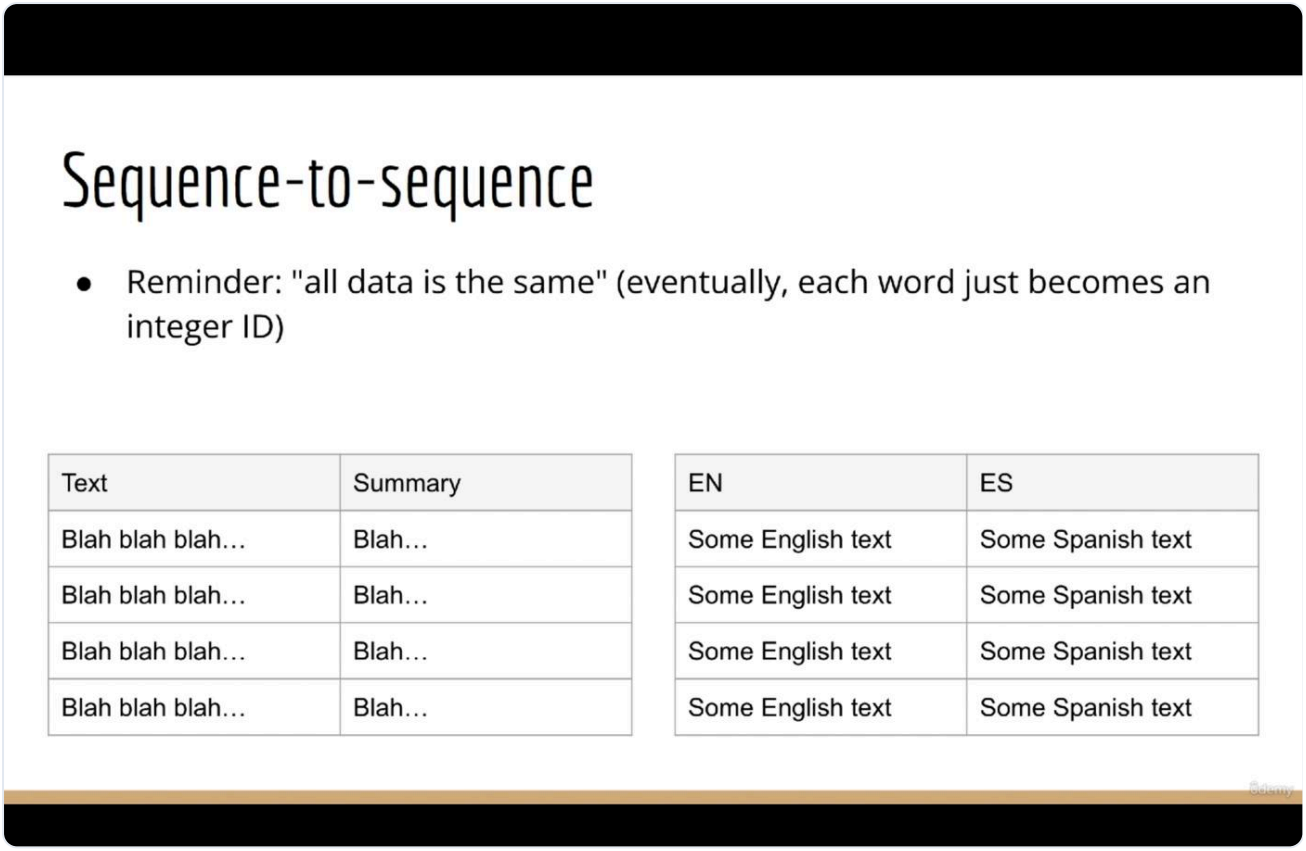

FIGURE 3.173 | Text Summary EN ES

In fact, if you apply my rule, all data is the same. You will understand that these two tasks are not really so different.

Consider what a dataset for these tasks might look like. In both cases, you essentially have two columns of text. The first column of texts contains the input sequence, and the second column of text contains the target sequence. As usual, the type of data makes no difference to a neural network because the neural network just sees numbers. All of these words are tokens will simply be converted into integer IDs.

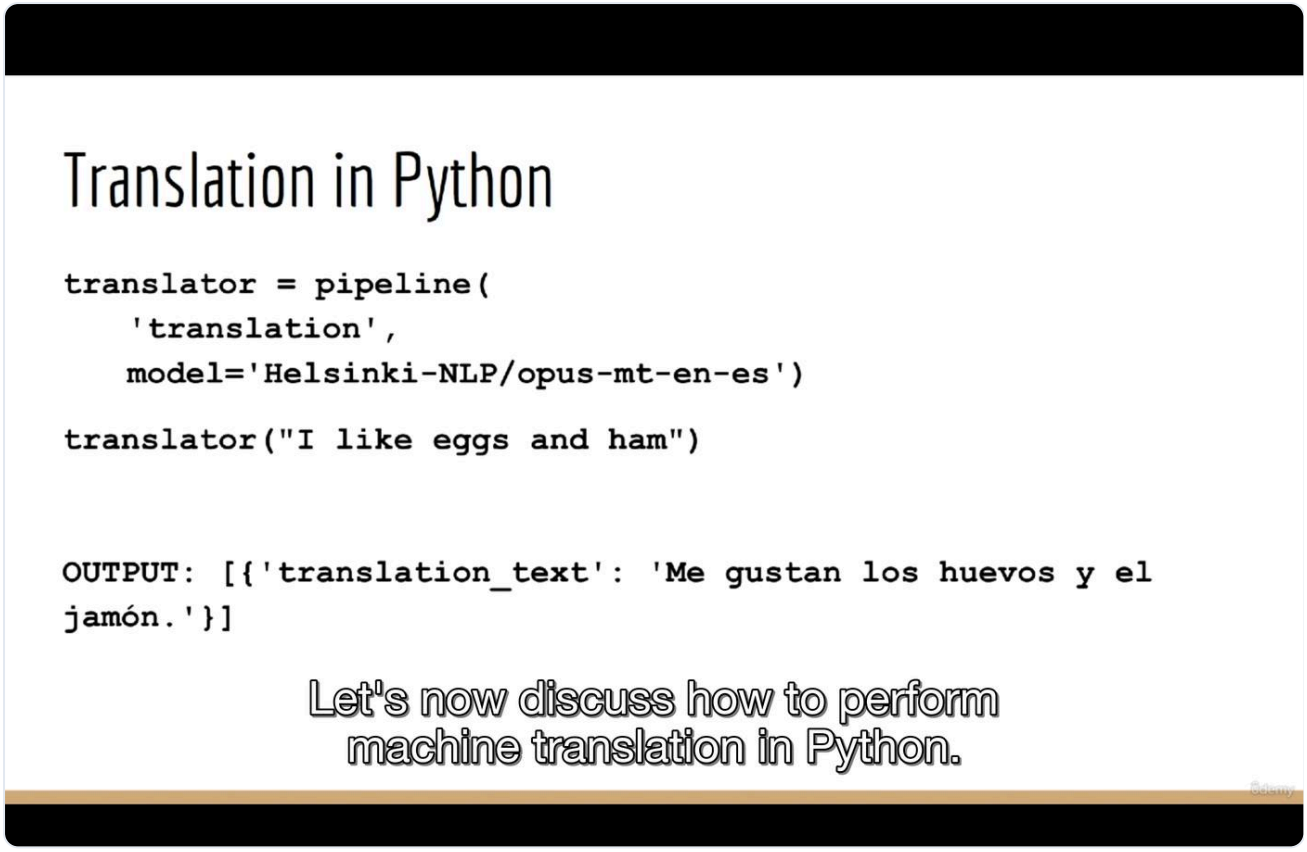

FIGURE 3.174

Let's now discuss how to perform machine translation in Python.

Again, this turns out to be very simple. Thanks to the pipeline API, we begin by creating a pipeline object passing in the string translation. Now, unlike the other task we've learned about, this is not all we need to do in particular. This is because this is not enough information for us to know which language we want to translate from, in which language we want to translate to. So for this task, you'll also need to specify a model which has been built specifically for the language pair you want.

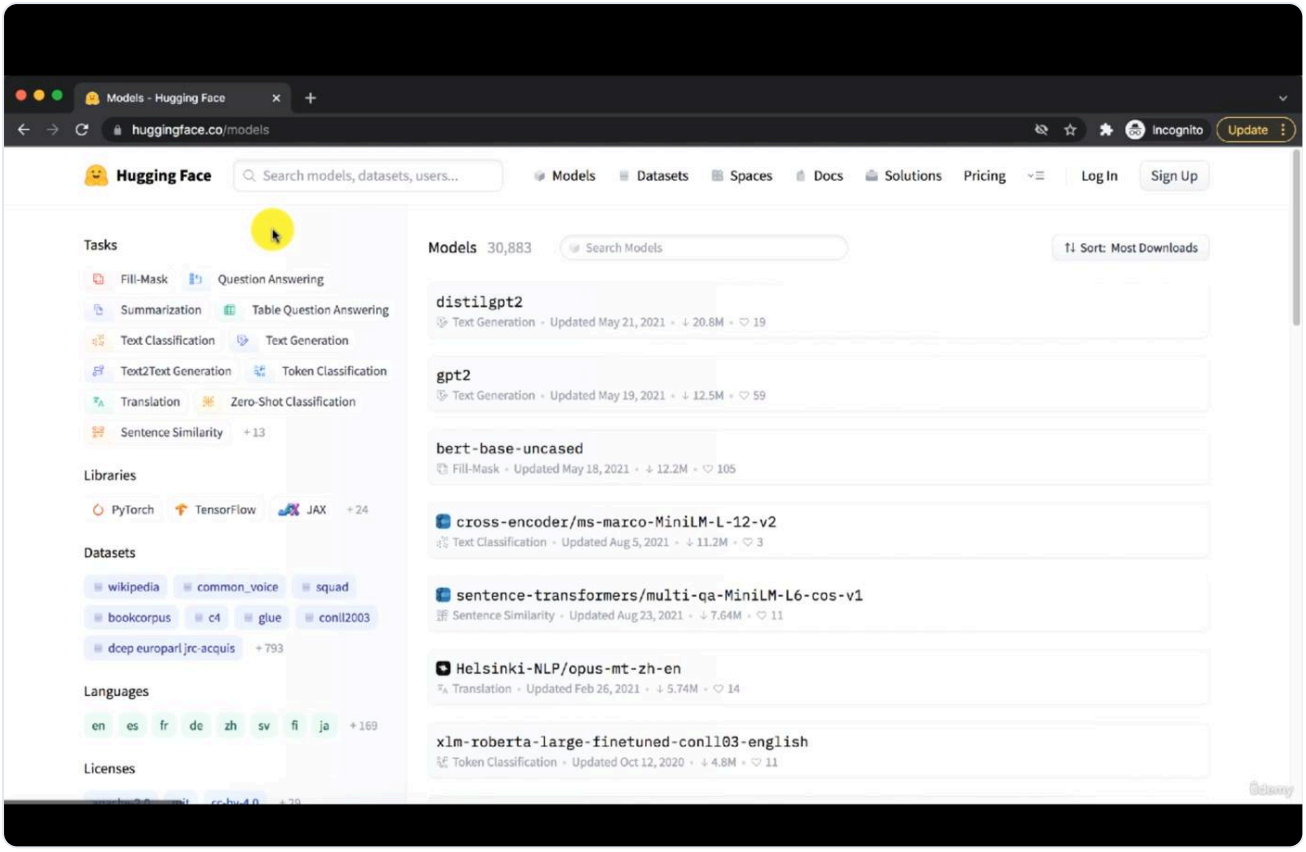

FIGURE 3.175

As usual, you can use the pipeline by passing in a string or list of strings, and this will translate your input into the target language. The output will be a list of one or more dictionaries, and each of these dictionaries will contain one key called the translation text. Now you might be wondering, where do you even find these models? The answer is to use the Hugging Face model hub. Basically, you go to Hugging Face, dot, CEO slash models and you'll see what is basically a search engine for different model types.

On the left side, you can filter models by task and language. So for the task you would want to select translation.

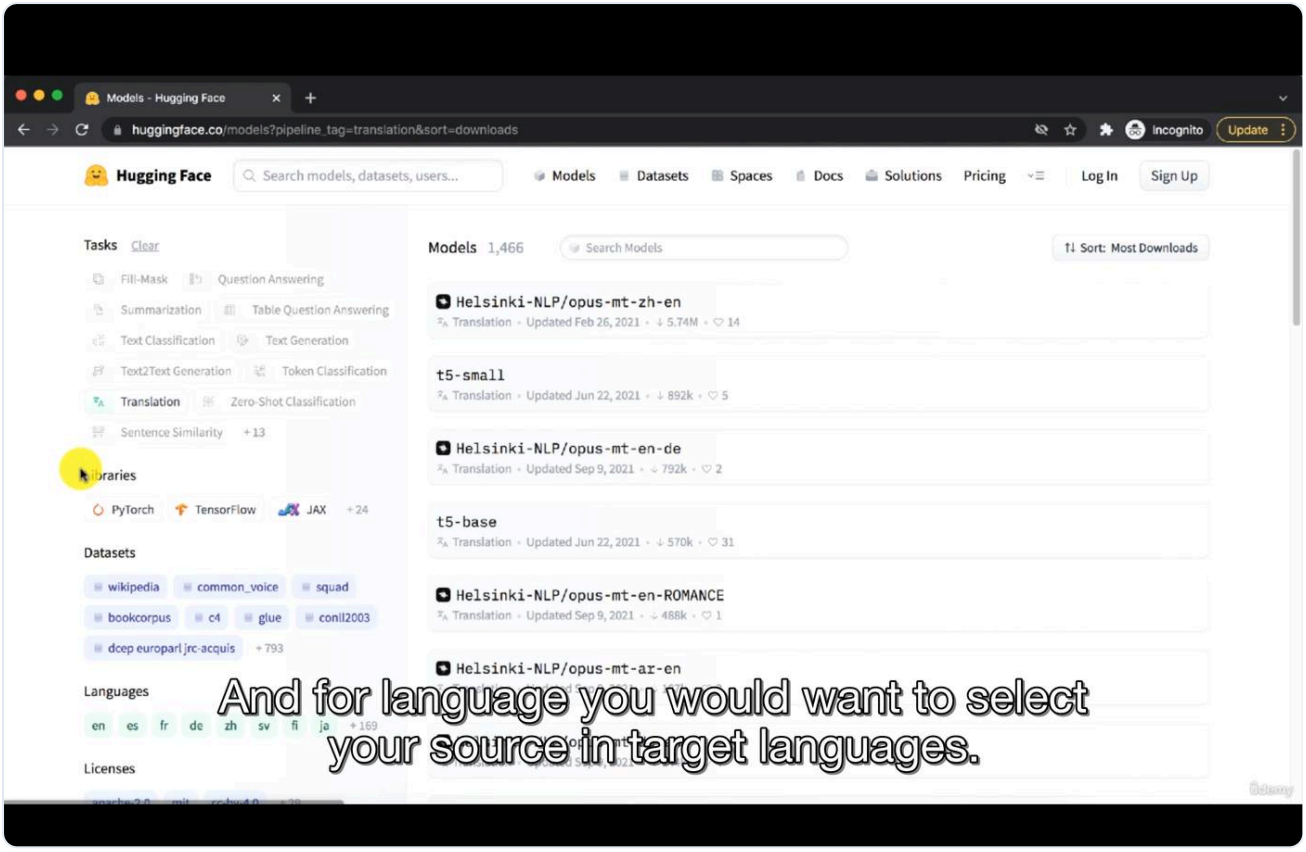

FIGURE 3.176 | Tasks Models 11 Sort

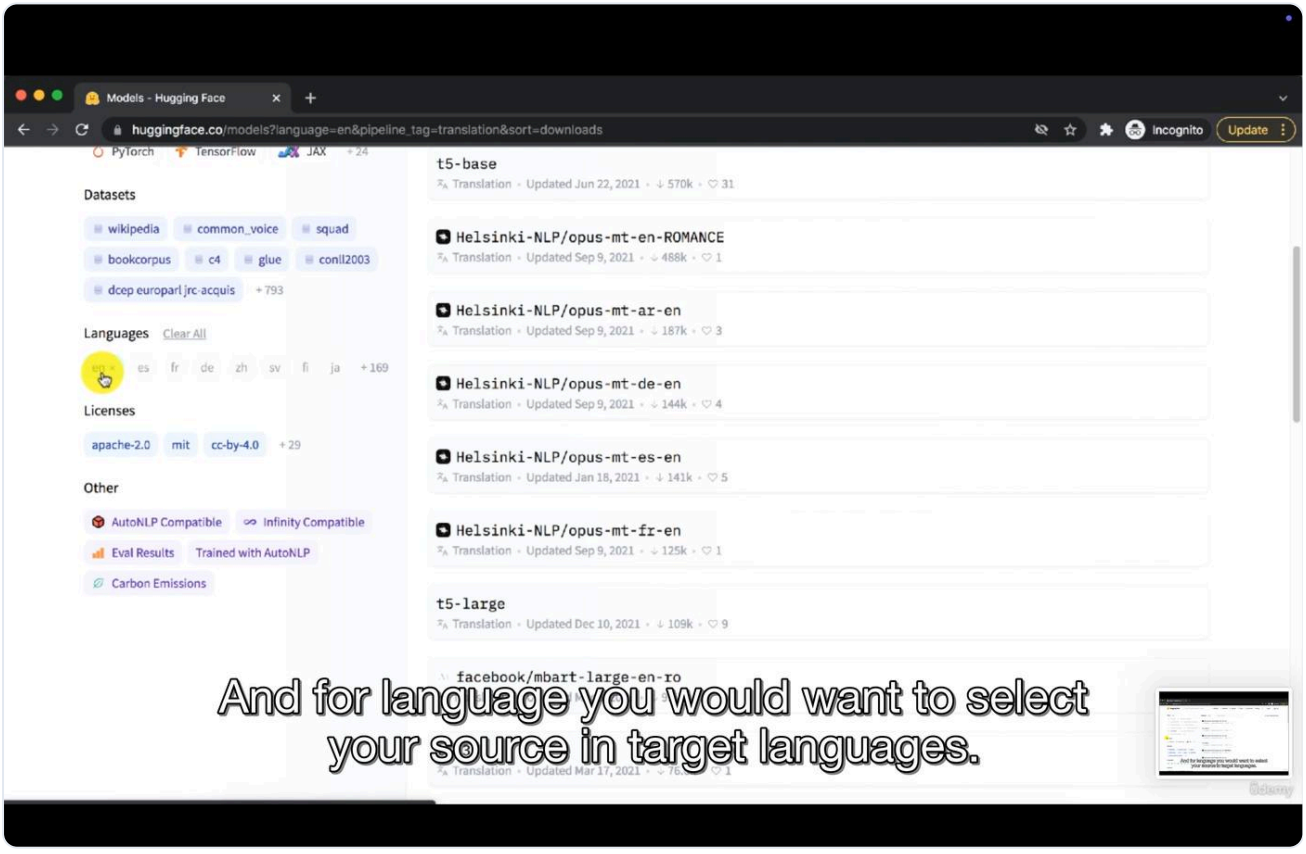

FIGURE 3.177 | Pytorch ensorFiow M JAK

And for language you would want to select your source in target languages.

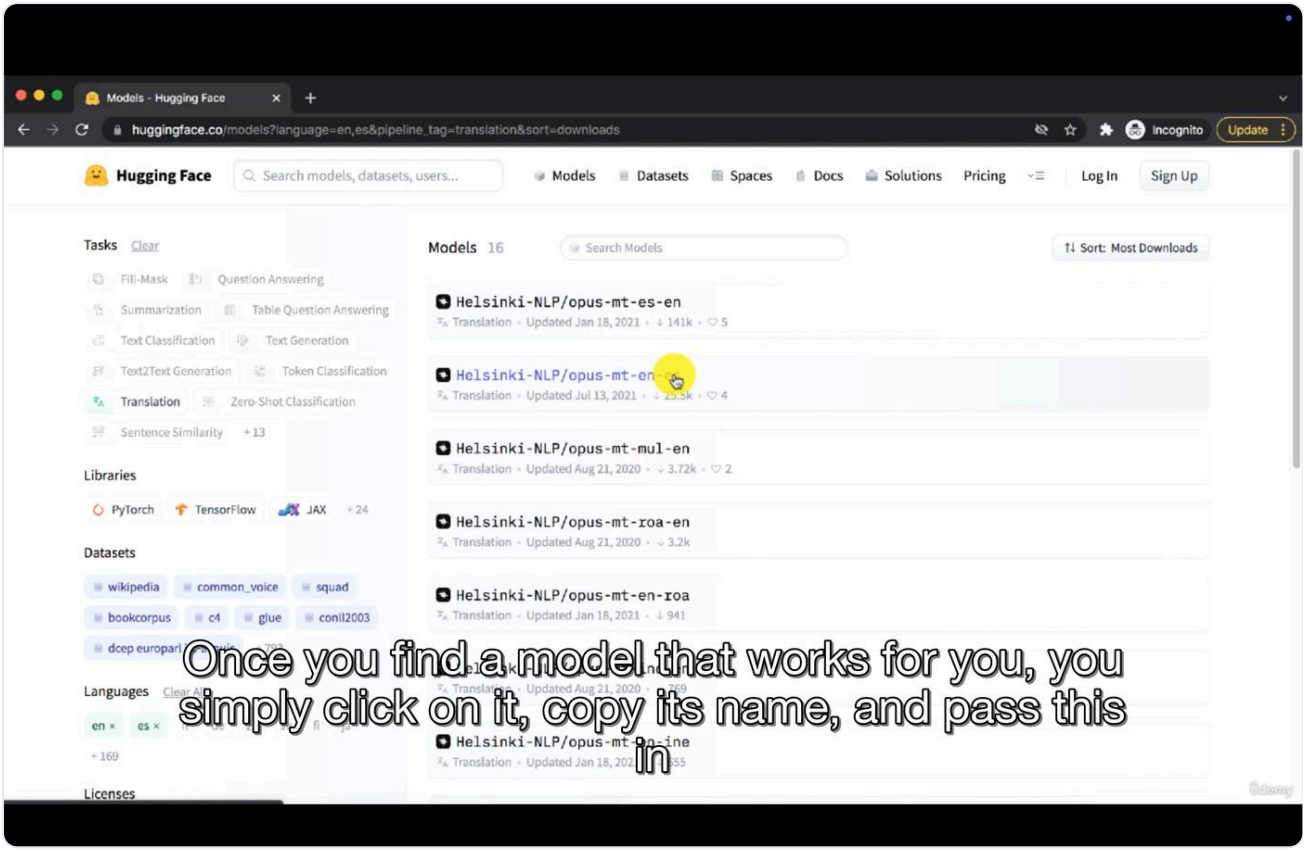

FIGURE 3.178

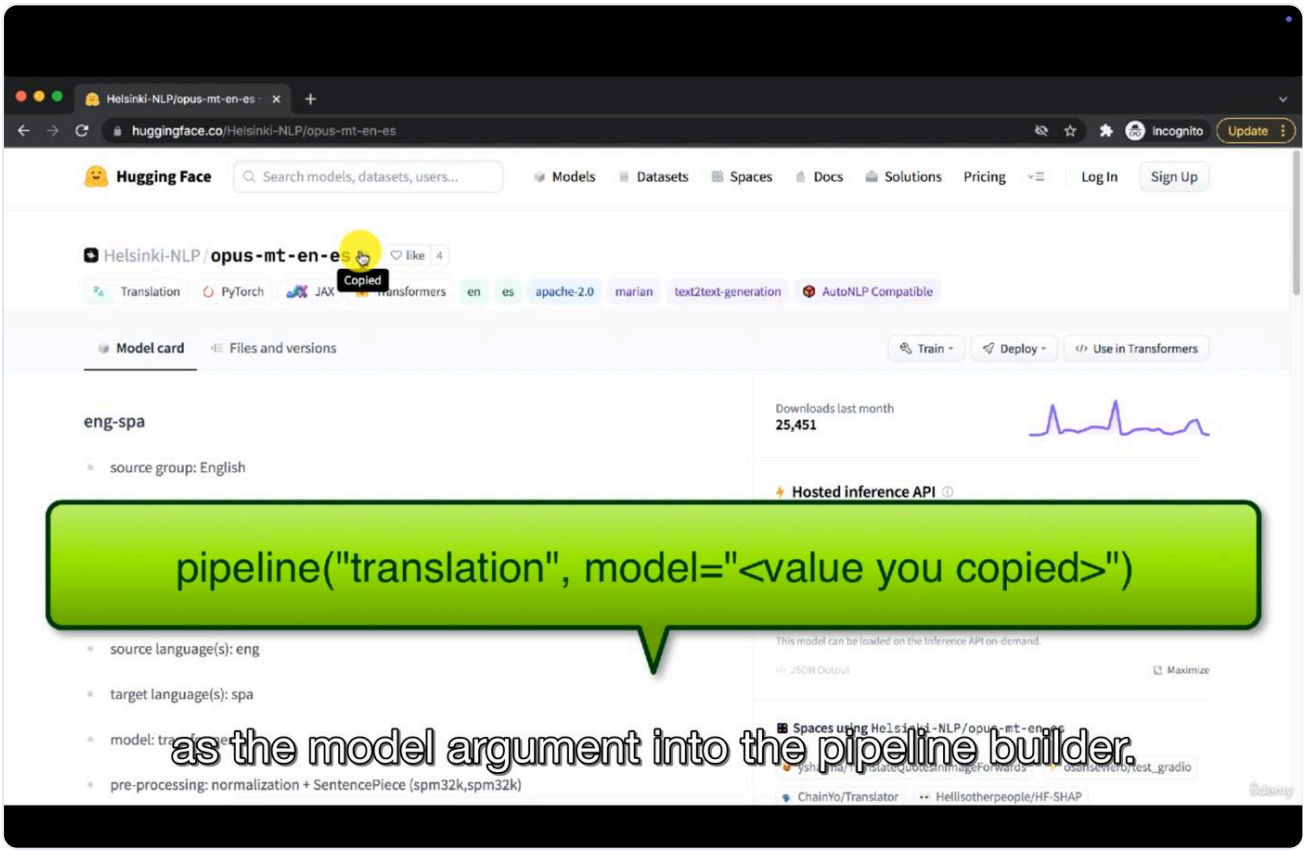

FIGURE 3.179

Once you find a model that works for you, you simply click on it, copy its name, and pass this in as the model argument into the pipeline builder. So as one final topic in this lecture, I want to briefly discuss how we evaluate translation models.

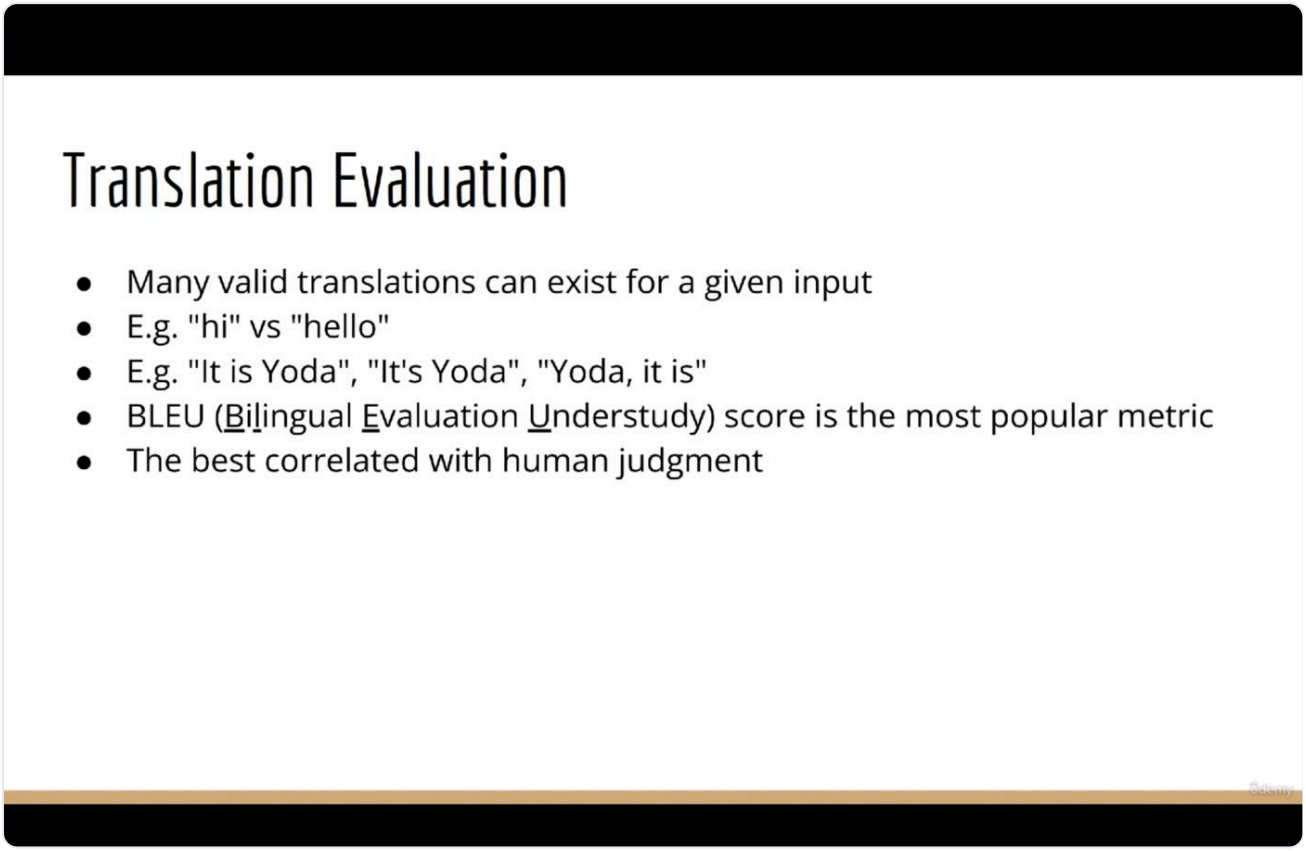

FIGURE 3.180 | Translation Evaluation

This is actually quite complicated since many valid translations can exist for a given input.

So you can't simply compare each output token to each target token. Furthermore, the output and target may not even be the same length. As an example, consider hi and hello. Both of these would be valid translations. But if you consider hi to be correct, then for hello. You would mark that as incorrect even though it's still valid. Consider also that there can be many ways to say the same thing. For example, it's Yoda. It is Yoda. And Yoda. It is. So clearly we should have some way of accounting for these differences and to still return a high score, even when the output doesn't exactly match the target.

Now, I don't want to get into the details of this point, since this is just the beginners corner. But the most popular metric for this task is the BLEU score. Out of many possible metrics. The BLEU score is the metric that has been said to be the most correlated with human judgment. And that's what matters, since the purpose of text is for humans to read it. But note that it's not perfect and still has a few disadvantages. Nonetheless, it is what is commonly used.

Basically, what you need to know is three facts. Number one, the BLEU score compares your prediction with multiple reference texts. So it allows for multiple possible answers. Number two, the BLEU score returns a number between zero and one, where zero is the worst and one is the best. So it's intuitively on the same scale as other metrics we use.

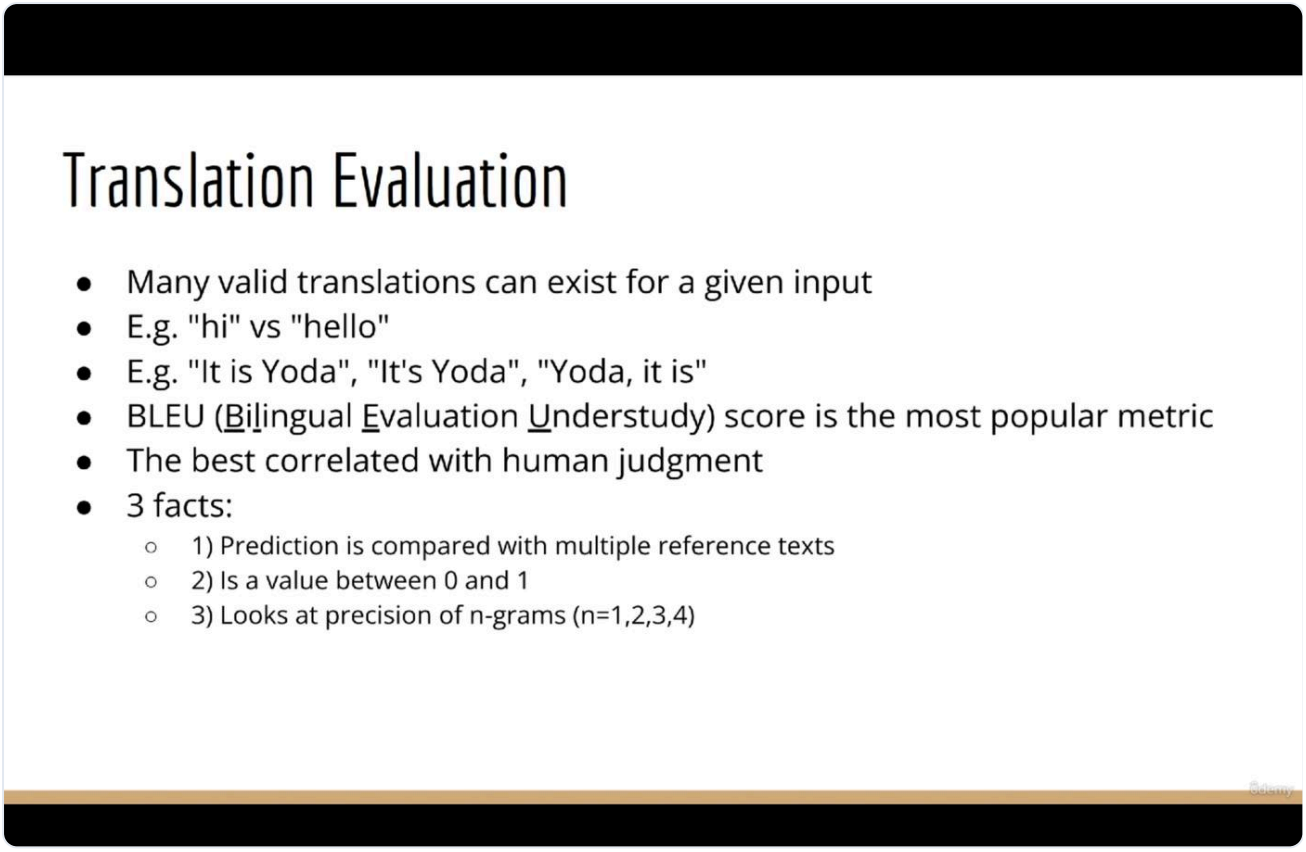

FIGURE 3.181 | Translation Evaluation

And number three, the BLEU score looks at the precision of n-grams for multiple values of n. So for example, unigrams, bigrams, trigrams and so forth.

So for each of the n-grams we find in the prediction, we're asking, can we also find this n-gram in one of the targets? And if the answer is yes, that's good.

14 · Neural Machine Translation in Python

See the notebook here.

So in this lecture we'll be looking at how to do language translation with Transformers. Now we already know how to translate text, which is just one line of code. So that's not very interesting in this lecture. What we're going to do is we're going to download a whole data set and see how well this model performs on that data set. This will give us a sense for how well the model is doing. Even if you're like me and you don't know Spanish. We'll begin by downloading a data set for English to Spanish text.

The next step is to unzip the zip file we just downloaded. The next step is to run the ls command to see what we got. So it appears to be a folder. The next step is to check what's inside this folder. As you can see, our data is probably in the file spa.txt. So let's run the head command to see what this looks like. As you can see, this is a tab separated file with the English text on the left and the Spanish text on the right. Importantly, notice that there are possibly multiple acceptable translations for each input.

The next step is to load in our file and store the data in a data structure suitable for our needs. In this instance, it would appear that a dictionary would be best, since the key can be the input phrase, while the value can be a list of possible target phrases. The next step is to print out a dictionary. As you can see, it has the format we expect. Some inputs only have one translation, while others have many. The next step is to import a function from NLTK to compute the BLEU score.

The next step is to import a special tokenizer since the BLEU score function works with tokens rather than raw text. This is a simple tokenizer that removes punctuation just to keep things simple. The next step is to test our tokenizer. As you can see, we get back three tokens with punctuation removed. The next step is to test our BLEU score function by just passing in the tokens we previously computed. Note that the first argument is a list. Since, as recall, there are many possible valid translations.

So the first argument is a list of acceptable translations, and the second argument would be the model output. Okay. So as you can see, the result is one which makes sense. But we get a warning. As you recall, the BLEU score computes the scores for grams up to n equals four, since there are no four grams in a three token sentence. This will result in a warning note that it suggests we could use a smoothing function, which is why we imported that above.

Now, personally, I don't think this is that important at this stage. To demonstrate this, consider the BLEU score for a single token, which is just hi. As you can see, we get a warning about two grams because the input only has one token. Now let's try the BLEU score again. But this time we'll pass in a smoothing function. As you can see, the score is now 0.32, which does not make a lot of intuitive sense. We can plainly see that the input is a perfect match.

Now, just as a side note, note that this function works on tokens of anything so we can even pass in integers. As you can see, we do not get a warning since the input has four tokens. The next step is to pre tokenizer targets for later use. We could have done this earlier, but it makes sense to do this now, since now you know why this needs to be done. The next step is to install transformers along with another package called sentencepiece, which is required by the translation model.

The next step is to instantiate our pipeline object. The next step is to test our translator. So hopefully one of, you know, Spanish and can let the rest of us know if this is correct. The next step is to check how many English phrases we have so that we know how many samples we have to translate. So we get about 100,000 samples. Now, unfortunately, this many samples is way too much for our compute capabilities to handle, and as such we'll select just 1000.

Note that I've selected 1000 starting at the index 20,000 since the phrases are ordered in terms of length, because we know that short phrases will get warnings when we compute the BLEU score. Starting at this index will give us longer phrases and will avoid most of those warnings. Obviously you feel free to select whatever subset you like. The next step is to translate the phrases we selected. Note that even with just 1000 phrases, this takes a bit of time.

So as an exercise, compute how long it would take to translate the whole dataset. The next step is to print out the first translation to check the format. As you can see, each translation is a dictionary with a single entry. The next step is to compute the BLEU score for each translation we got

back. This just uses the same functions we defined above. The next step is to plot a histogram of our scores just to see how they are distributed. So as you can see from most phrases, we get a perfect score of one.

Occasionally it's less, and sometimes it's even zero. As an exercise, if you know Spanish, you might want to check these samples and see whether the model really is incorrect. The next step is to compute the average score. So as you can see, the average score is about 80%. The next step is to print a few sample translations. So for those of you who know Spanish, you can check whether or not these make sense. So this function just prints the input, the output and the true matches.

So let's try this out. Okay. So as you can see, our translation is perfect in this case. Let's try another. Again. Our translation exactly matches the input. Let's try another. So this time the first word in the translation does not match either of the targets. However, the second word does notice how the model is able to insert the inverted question mark at the beginning. Even though the English sentence only has a question mark at the end, our model has learned that when the input is a question, the translation should begin within an inverted question mark and end with the regular question mark.

If we looked at the attention plot, we would probably see that this first token of the output pays attention to the final token of the input.

15 · Question Answering

So in this lecture we will be discussing the next task in this section, which is on question answering. Now, question answering is a pretty broad task, so it helps to think of all the different flavors that may exist. In a general sense, our ultimate goal is to really have a system that can answer any question. For example, if this were possible, you can imagine that in a I could replace a doctor or a teacher in some instances.

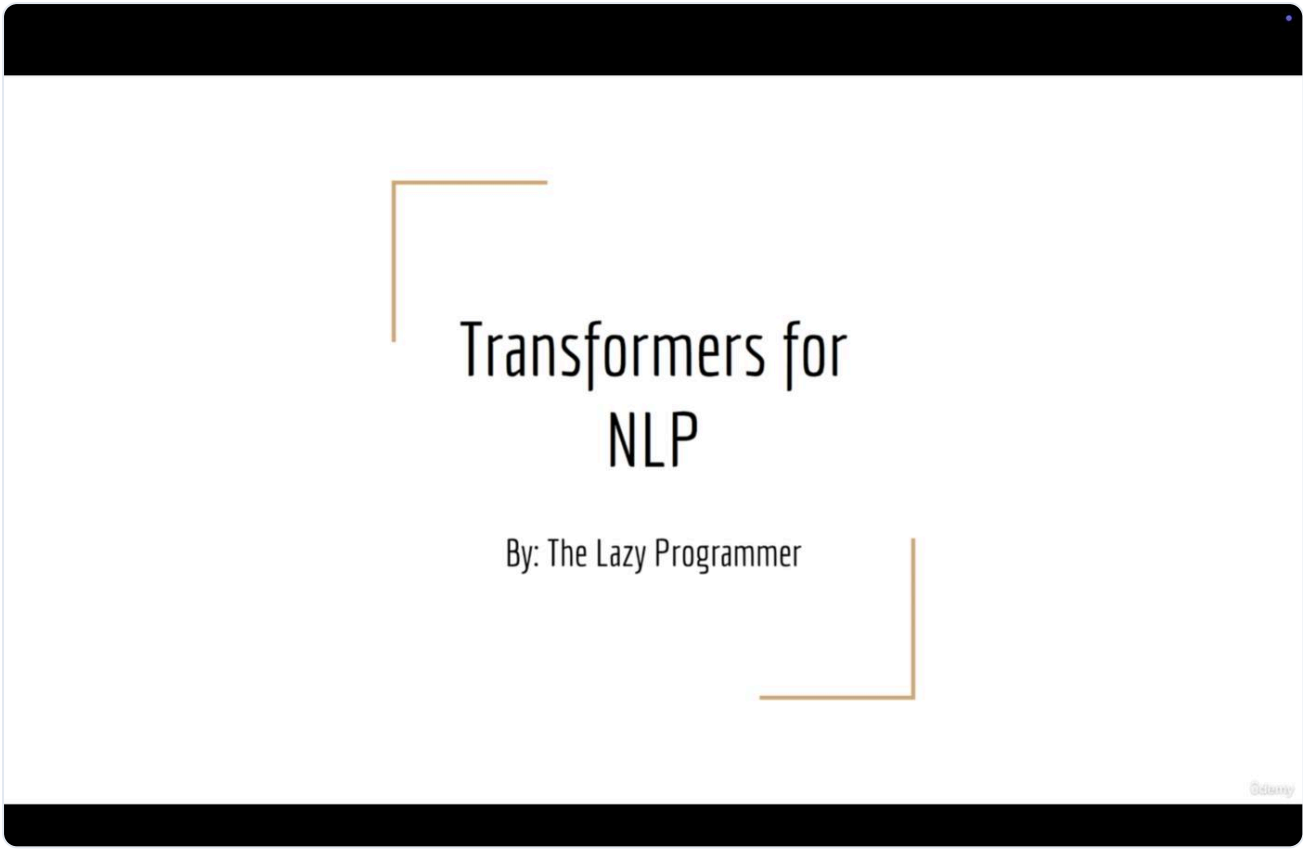

FIGURE 3.182 | Transformers for

In this ideal scenario, the system would have some database of knowledge, or perhaps that knowledge could be stored in neural network weights.

When the user asks the question, the neural network would simply understand the question related to its stored knowledge and provide the appropriate answer, whatever that may be. Now we're going to end up making a number of simplifications to this ideal scenario in practice. Note that some systems such as Apple's Siri already do this to some extent. For example, you can ask Siri for the weather and it will tell you the weather.

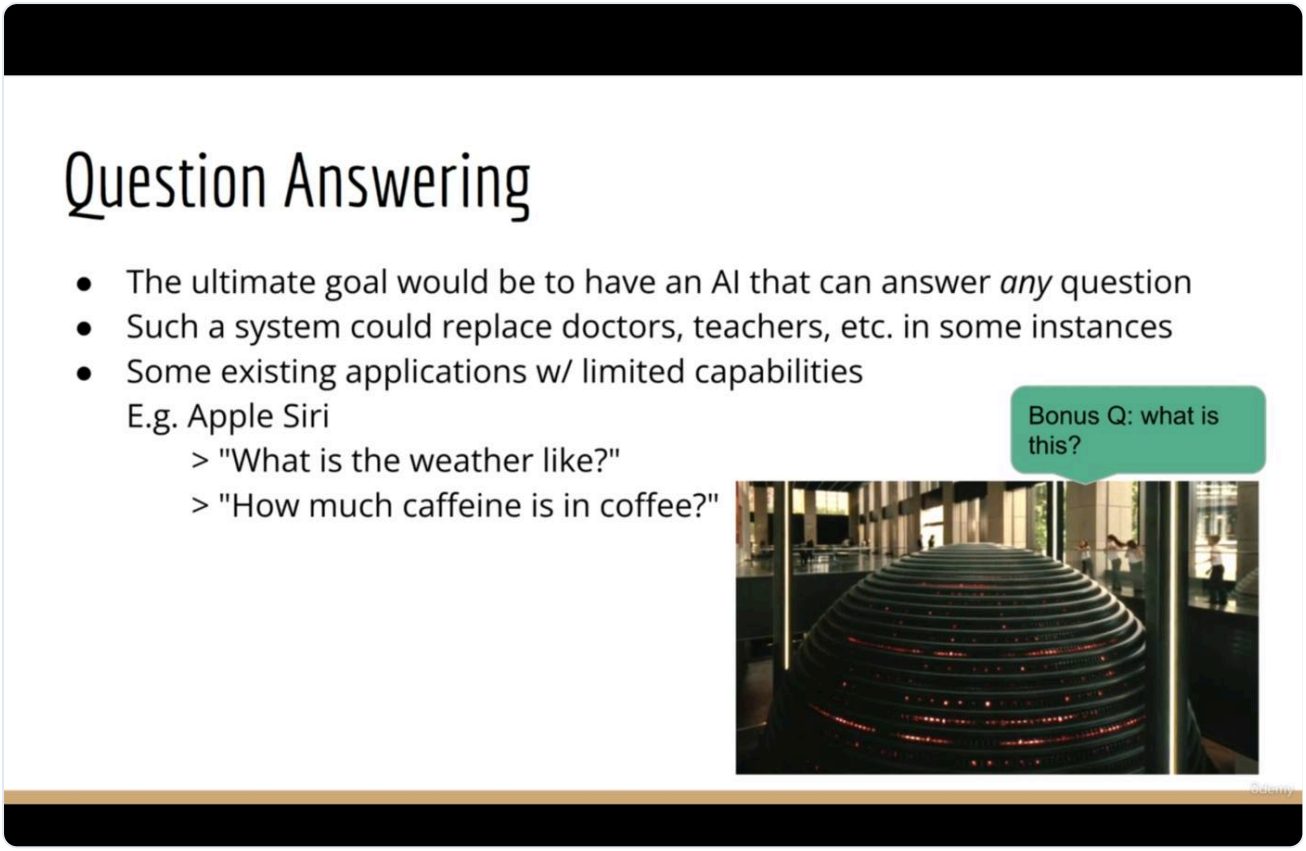

FIGURE 3.183 | Question Answering

You can even ask Siri for nutritional facts like how much caffeine is in coffee.

But this is not general enough since it eventually ends up as a database lookup. In practice, what we would like to do is more akin to natural language understanding. So one of the standard question answering benchmark data sets in this modern NLP era is SQuAD. SQuAD stands for Stanford question answering data set.

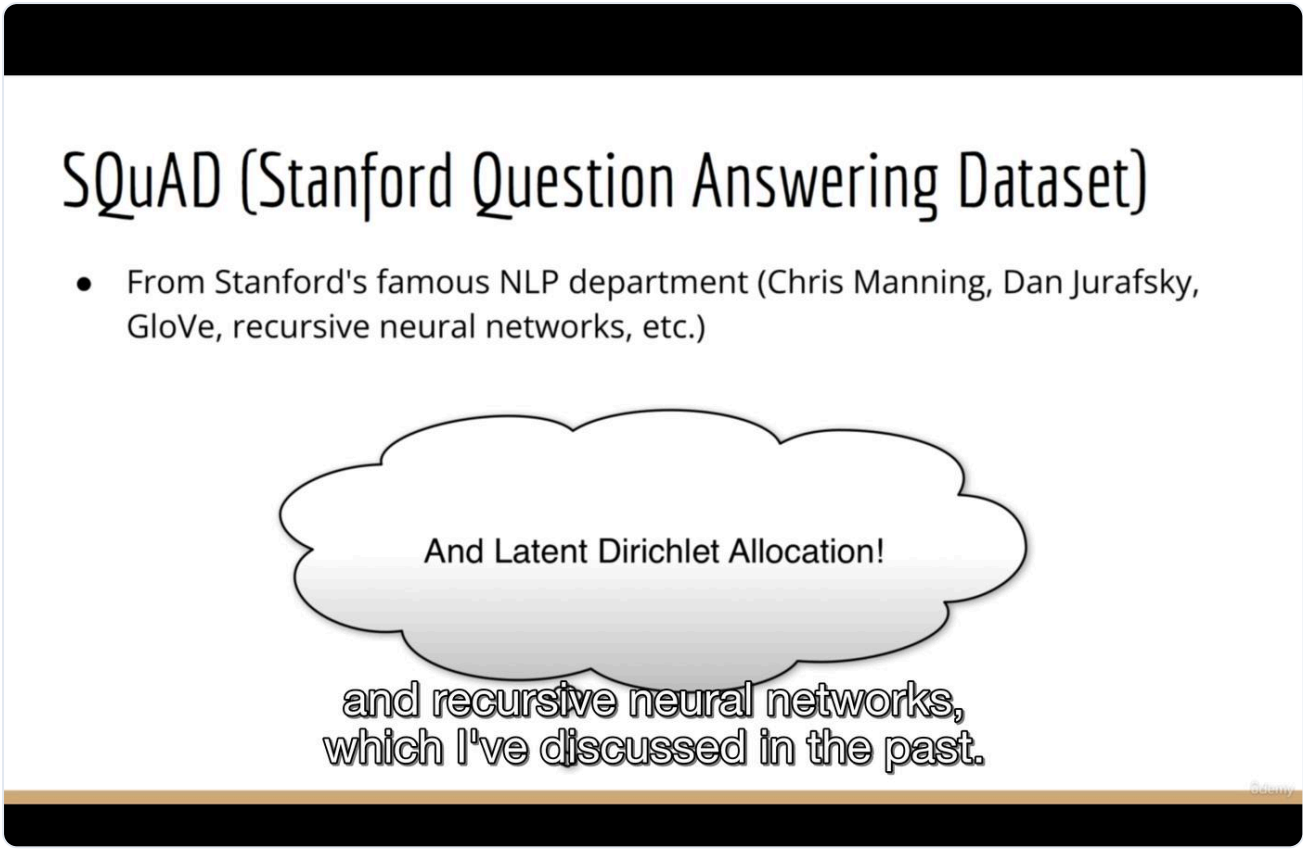

FIGURE 3.184

As you recall, Stanford is a pretty famous NLP department, which hosts famous NLP researchers such as Christopher Manning and Dan Jurafsky They've also developed some widely used tools, such as GloVe and recursive neural networks, which I've discussed in the past.

SQuAD is an example of an extractive question answering dataset, and it aims to test a reading comprehension.

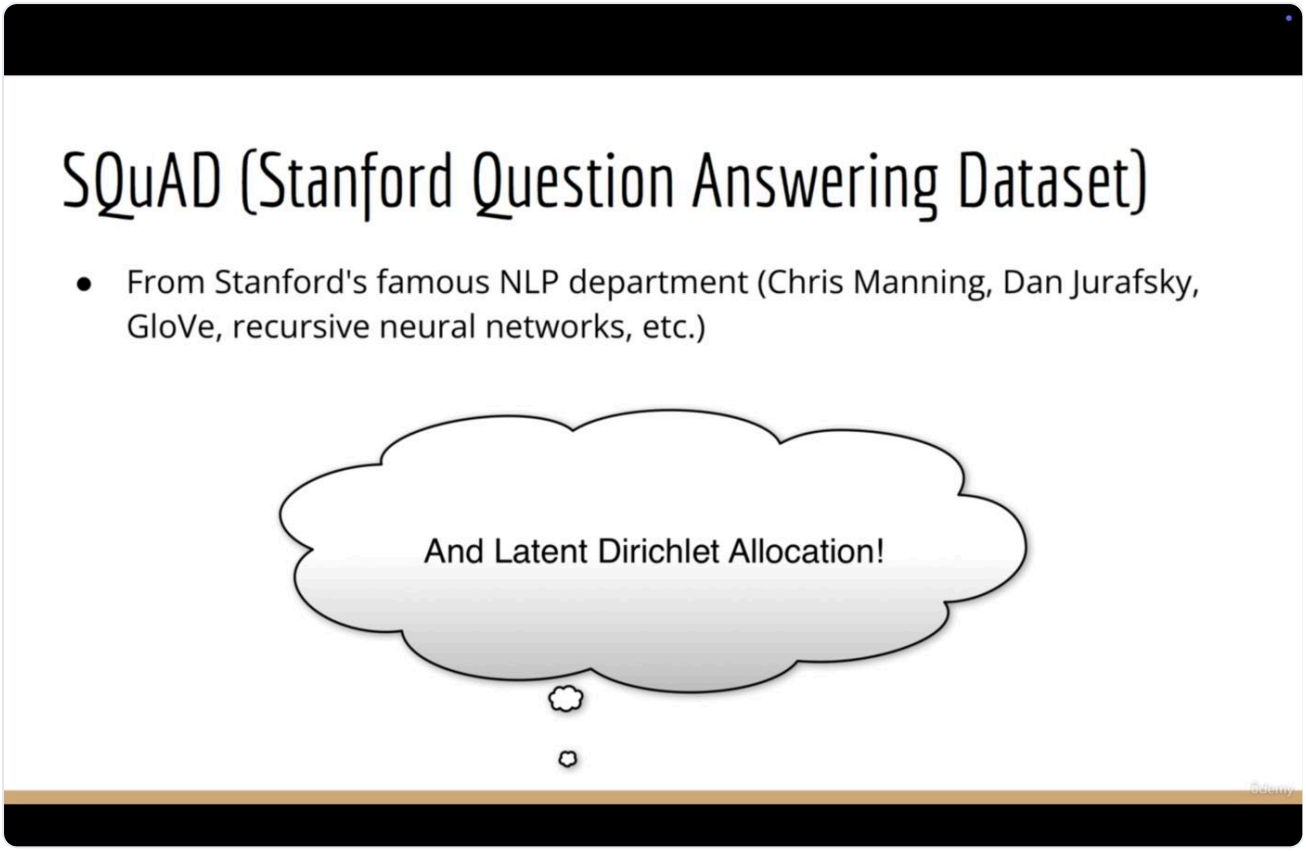

FIGURE 3.185

So what do we mean by extractive? As mentioned in practice, we place some restrictions on the ideal scenario of having a robot answer any question we ask.

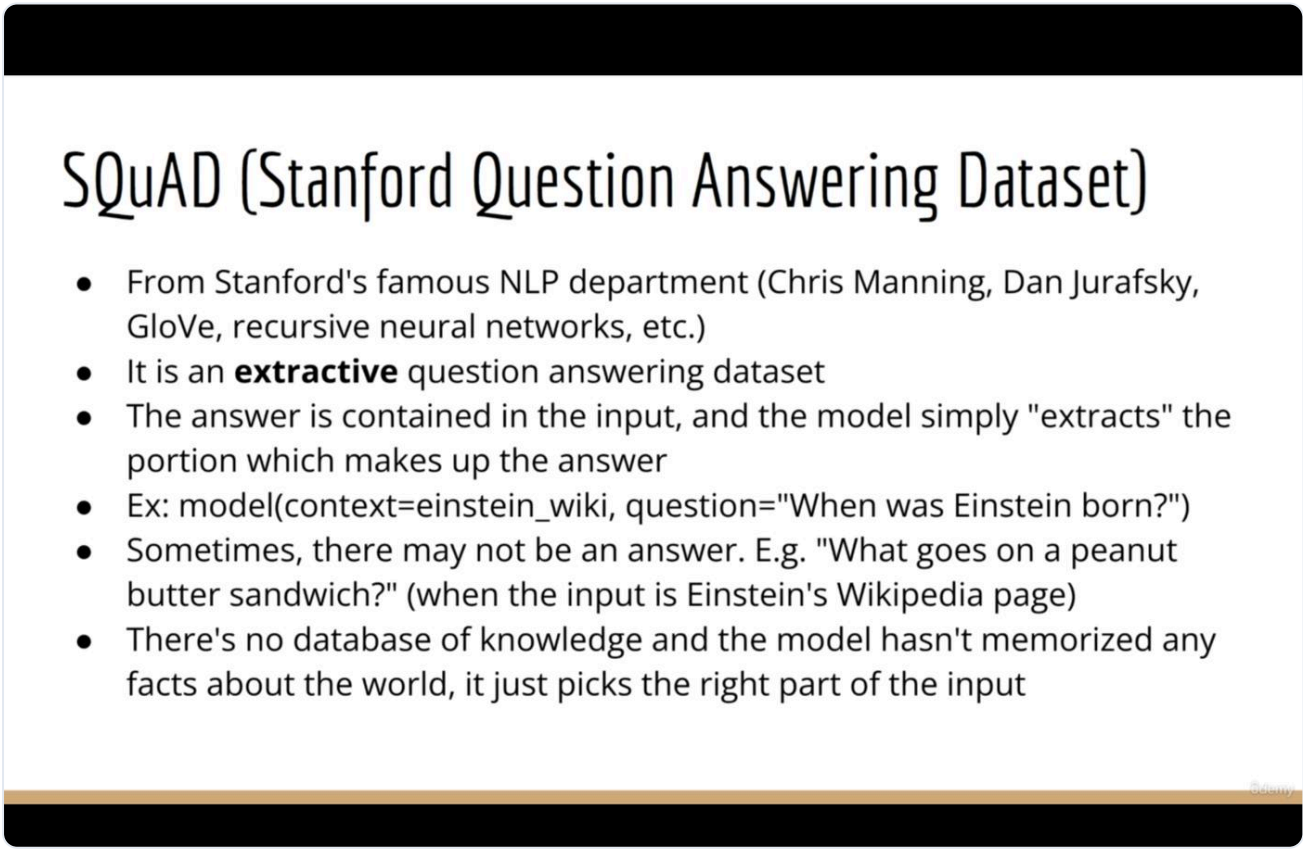

FIGURE 3.186 | It Is an Extractive Question Answering Dataset

Extractive means that your input actually contains both the question and the answer, and all the model has to do is choose the right part of the input. In other words, extract the answer. So, for example, the input might be a whole Wikipedia page about Albert Einstein.

You might then ask, When was Albert Einstein born? And the model would have to choose the right part of the Wikipedia page, which contains the answer to when Albert Einstein was born. What this means is that there are some scenarios where it's not possible to have an answer. For example, if my input contains information about Albert Einstein and I ask What goes on a peanut butter sandwich, this would not be an answerable question. So in this case, the model isn't learning how to draw from a database of knowledge, but rather it's learning how to read and comprehend and then to answer questions about what it has just read.

In other words, there's a lack of memory. Now, don't let this fool you, since, as you'll see, the results are quite impressive. So one thing that's interesting about this task is that it's very different from the previous task we've discussed. In particular, the previous tasks have all been simple cases, like many-to-one of many-to-many, sequence to sequence, language modeling and variations thereof. Now this technically could be structured as a kind of sequence to sequence task, but instead the solution is simpler.

As mentioned, the answer is always contained in the input. Furthermore, the answer is always a contiguous sequence and thus a simpler way to represent. The answer is by providing just the start and end indices from the input.

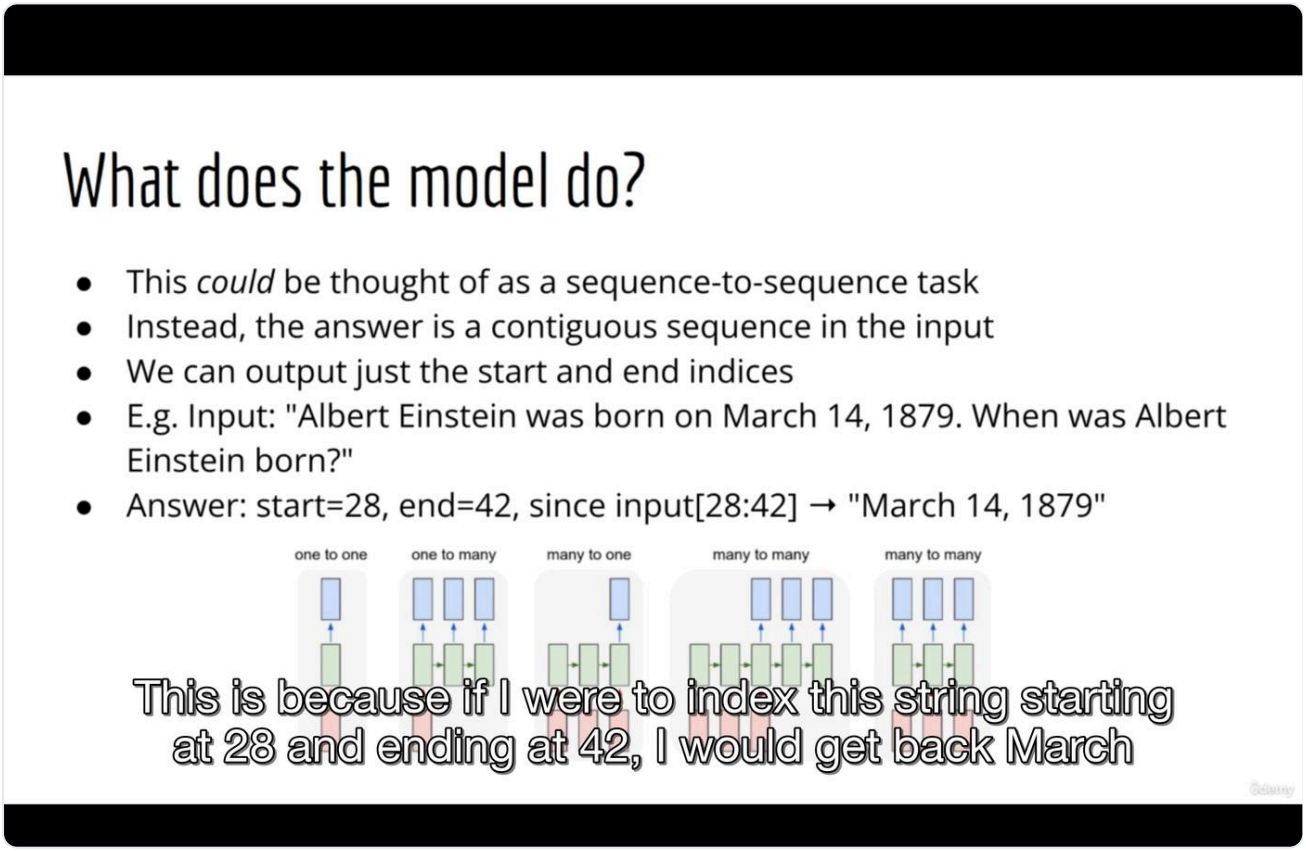

FIGURE 3.187 | What Does the Model Do?

For example, if my input string was Albert Einstein was born on March 14, 1879. When was Albert Einstein born? The answer to that would simply be start equals 28 and end equals 42. This is because if I were to index this string starting at 28 and ending at 42, I would get back March 14, 1879.

And if you have experience with neural networks, it would be helpful to consider how you might build a neural network to output such information. So that's one thing that might be different from what you're used to. Something else that might be different is the input. Typically, the input to our neural network is usually just one piece of information. For example, one document we want to translate or one document we want to classify. As you recall, large language models are pre-trained using very simple inputs and outputs, like trying to predict the next word in a sentence.

So during this pre-training stage, the neural network never sees anything terribly complex, such as a context in question pairing.

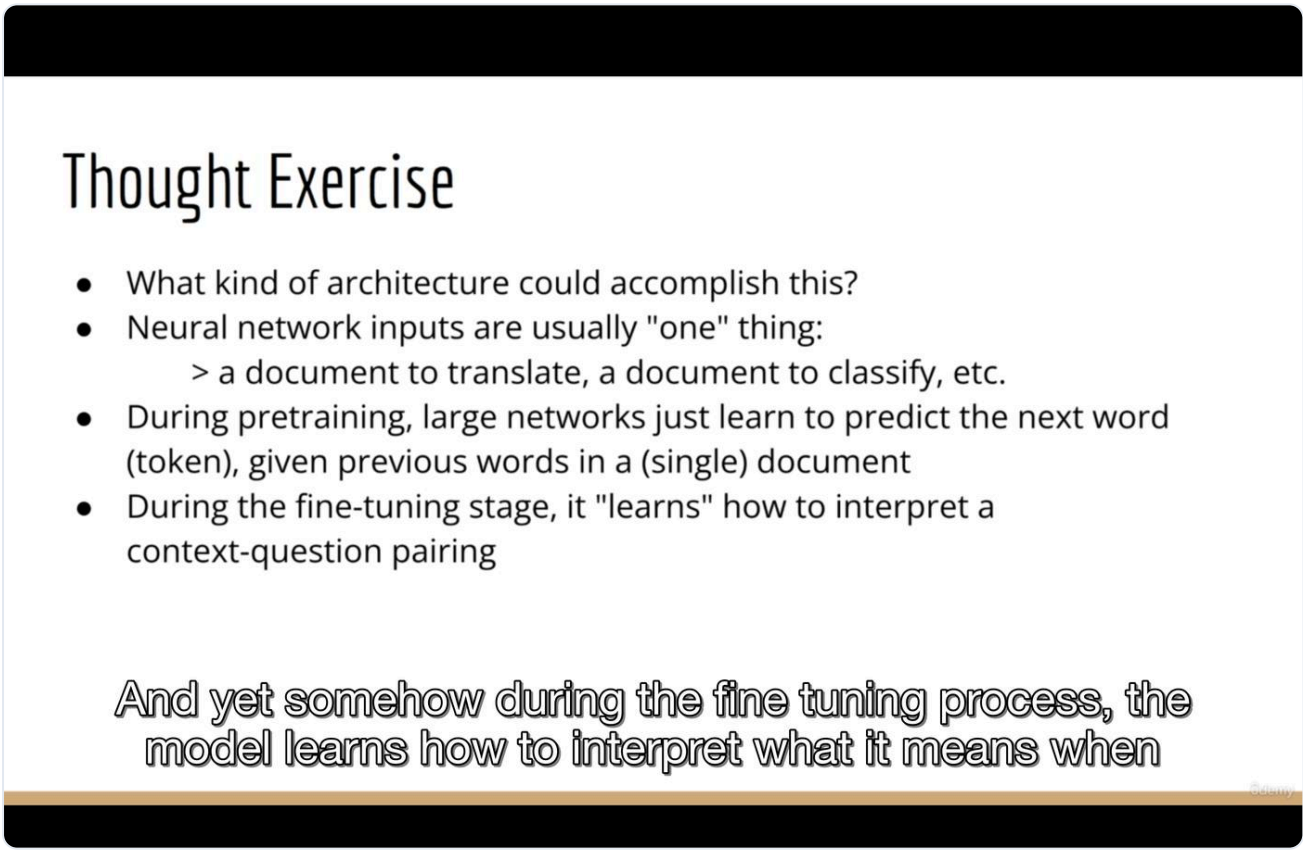

FIGURE 3.188 | Thought Exercise

And yet somehow during the fine tuning process, the model learns how to interpret what it means when you concatenate a context and question together, in particular as input to our model, we create a string with the following format. We start with a special class token followed by the question, followed by a special separator, then the context, and then another separator.

The context is the part which contains the answer. And so not only does the model need to understand the question and how to find the answer in the context, it also has to learn what it means to have an input formatted this way, it learns that when you pass into sequences like this, its job is

to answer the question based on the information in the context. So the interesting part about this is even though we have two things to pass into the input, we are still managing to represent it as a single piece of text.

It's interesting because of the fact that the neural network was not originally designed to understand what this means. And the reason this is powerful is because it will let us do even crazier things, such as few shot learning. Okay. So now that you understand what question answering is and why it's useful, let's look at some code. As usual, it's extremely simple. Using the pipeline from Hugging Face to create a question. Answering pipeline we call the pipeline function passing in the string question answering.

From there, we simply use the pipeline as a function which takes in two arguments context in question. And both of these are strings. As mentioned, the context should be text containing the answer to the question. For example, the context might be Today I made a peanut butter sandwich, and the question might be, what did I put in my sandwich? And the answer to this question is provided as a dictionary as usual. In the key answer we have the actual answer represented as text.

We also have start and end keys, which are indices into the context where the answer can be found. Finally, we have a score representing how confident our model is in its answer. So perhaps you might want to throw away or double check any answers with the low score.

16 · Question Answering in Python

See the notebook here

So in this lecture we will be looking at a demo that shows us how to do question answering in code. One interesting fact about this demo is that there is no data set. We are simply going to make up our own questions and observe that this pre-trained model works quite well. So let's begin by installing Transformers, as we normally do. The next step is to input the pipeline function as we normally do. The next step is to create a pipeline by calling the pipeline function passing in the spring and question answering.

The next step is to define a context. And the question. The context is.

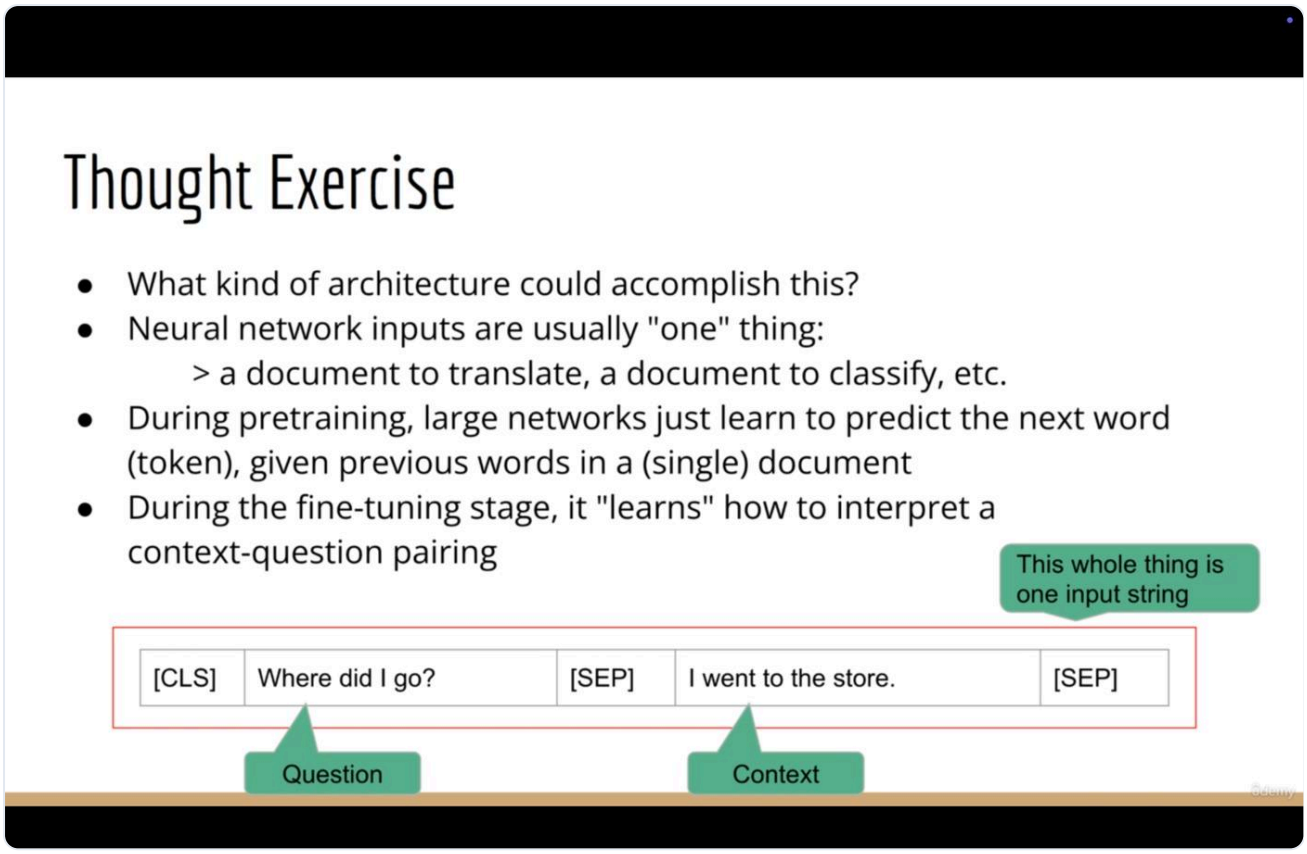

FIGURE 3.189 | Thought Exercise

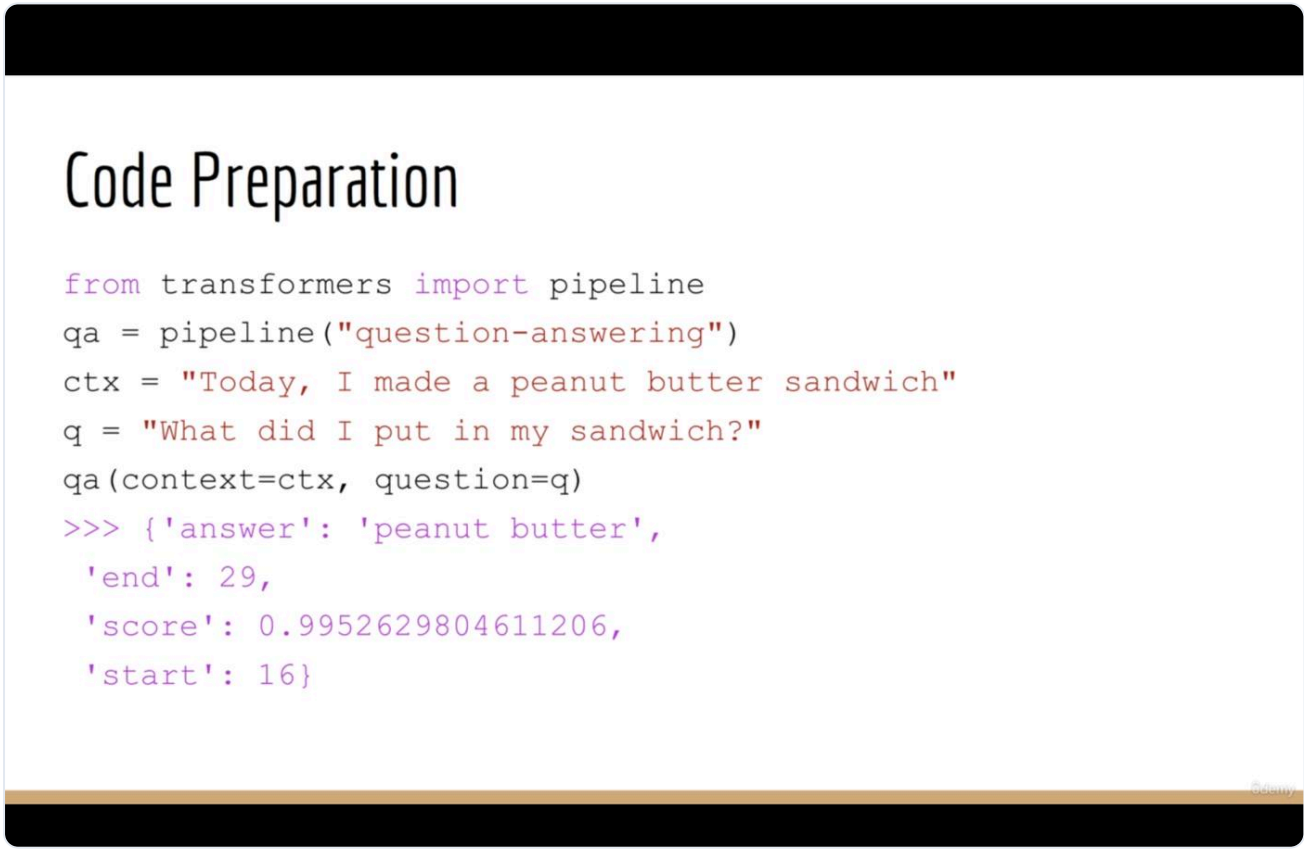

FIGURE 3.190 | Code Preparation

Today I went to the store to purchase a carton of milk. The question is, what did I buy? Now I phrased my question this way on purpose in the context I use the word purchase, but in the question I used the word buy. What I would like our model to do is to understand that both of these words mean the same thing. So let's run our model on this question. And as you can see, the model correctly identifies that a carton of milk is the answer.

Note that we can also index the context directly using the start and end indices. So as you can see, this returns the portion of text corresponding to the answer. So for the next experiment, the context is out of all the colours. I like blue the best. And the question is, what is my favorite color? So again, you can see how I am testing the model here with something non-trivial. In particular, we want the model to know that what I say I like best is synonymous with what is my favorite.

Okay. So as you can see, the answer is blue, as we expect. For the next portion of this script, we're going to use a snippet from Wikipedia's page on Albert Einstein. So the context is simply some text I've pasted from that page. So the first question we'll ask is, when was Albert Einstein born? As you can see, the model correctly identifies the date March 14, 1879. So this is interesting because the context actually contains two dates. And furthermore, it doesn't explicitly say this was when Albert Einstein was born.

Instead, it's simply two dates surrounded by parentheses and separated by a dash. What this means is that the model has learned our syntax for representing birth and death dates. The next question will ask is, what was Albert Einstein's occupation? And the answer is theoretical physicist, which is correct. As you recall, the context starts with Albert Einstein. 14th March 1870 918th April 1955 was a German born a theoretical physicist. The next question will ask is What is Albert Einstein best known for?

And the answer is developing the theory of relativity, which is correct as well. As you recall, the context contains the phrase Einstein is best known for developing the theory of relativity. The next question we'll ask is What else has Albert Einstein contributed to? This might be a bit more difficult, since it's plausible that it might just say relativity again. Impressively, the answer is theory of quantum mechanics, which is correct. To recap the context, says Einstein is best known for developing the theory of relativity, but he also made important contributions to the development of the theory of quantum mechanics.

Let's now ask a question from the next sentence. What are the two pillars of modern physics? And the answer is relativity in quantum mechanics as expected. As you recall, the context says relativity and quantum mechanics are together the two pillars of modern physics. Let's now try to ask a more difficult question, which is where was Albert Einstein born? Now, please note that the context actually does not contain the answer in an extractive sense. The answer is Germany, but the context does not contain the string Germany.

However, it does say that Albert Einstein is German born. Thus it would be interesting to see what our model comes up with. Okay. So the answer is actually as correct as possible, which is German. This is because the model cannot return Germany since that doesn't exist in the context. For the last question, we're going to ask something which the model cannot answer just to see what we get. The question is, what is peanut butter made of? Obviously, the answer is peanuts.

Of course, it's not possible to answer this question from the existing context, since that is about Albert Einstein. So the answer is Albert Einstein, which of course does not make sense. However, note that this has a low score, so that would be a sign that the answer may not be trustworthy.

17 · Zero-Shot Classification

So in this lecture, we will be discussing another new and interesting application called zero-shot Classification. This is one of those fancy new terms of the deep learning era. So let's consider what this means.

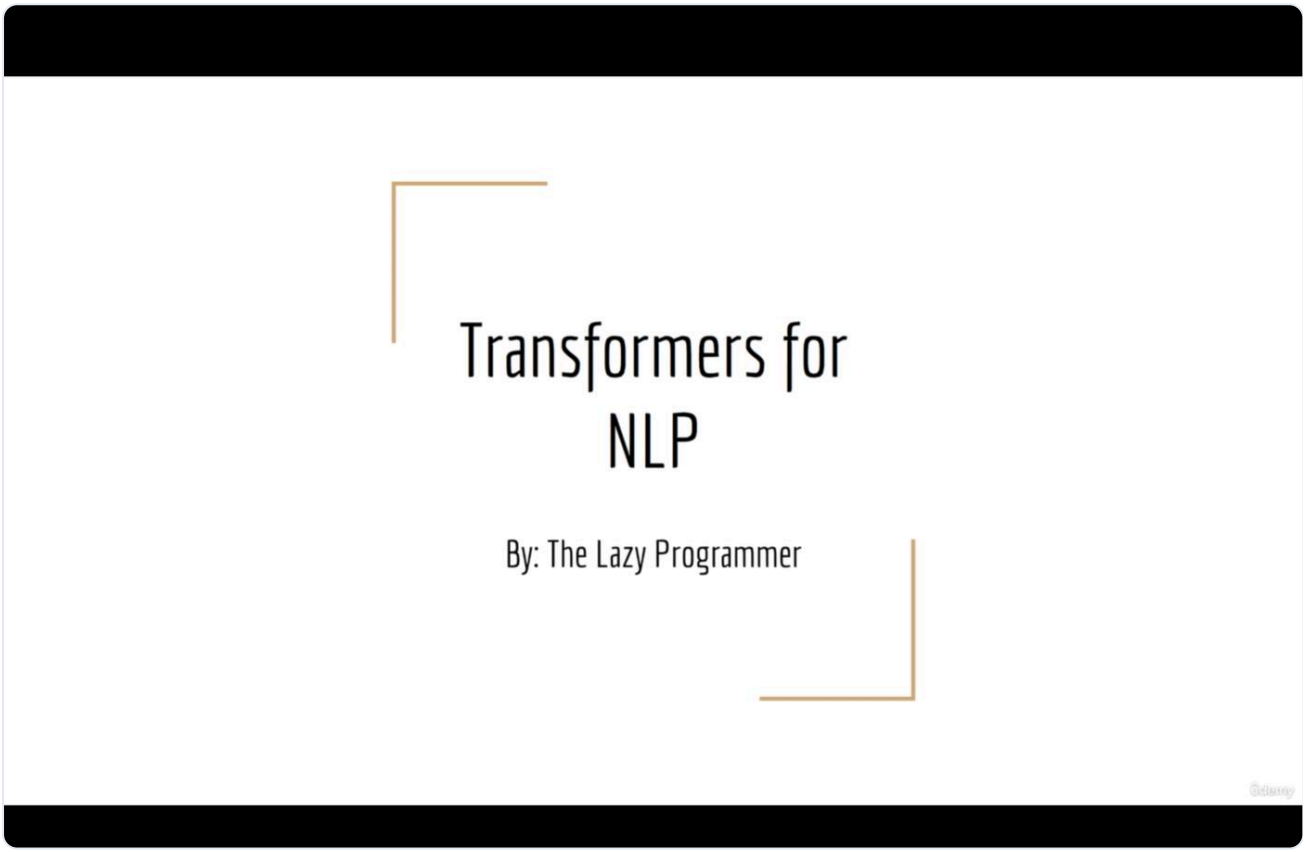

FIGURE 3.191 | Transformers for

As you know, regular classification is where you train a model based on a labeled dataset. The labels are discrete categories, and each input sample is assigned at one of these labels. For example, we might train an image classifier to differentiate between people with masks and people without masks.

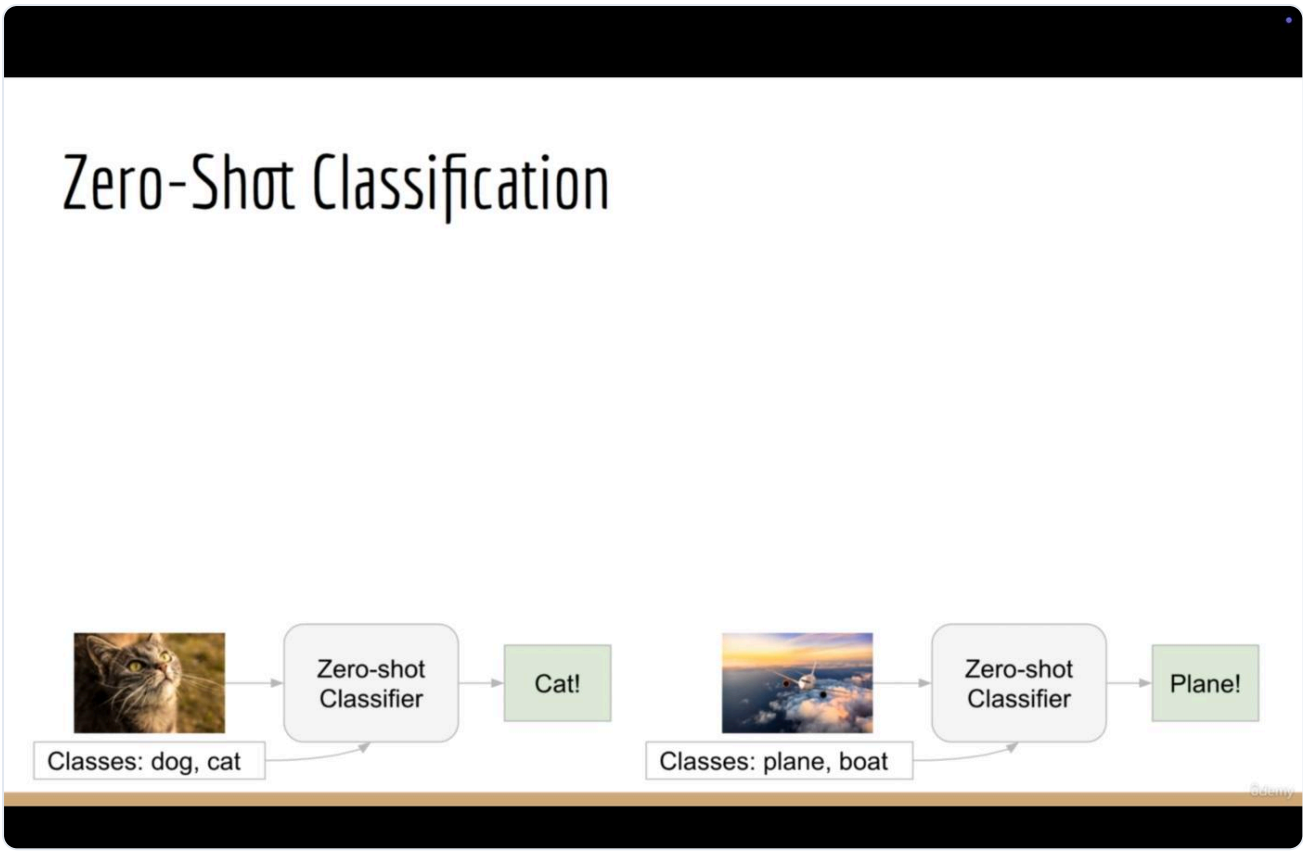

FIGURE 3.192

After training such a model, we can then use it to make a new predictions on new images. The caveat is that this model can do nothing else. It can only be used to predict whether or not a person is wearing a mask.

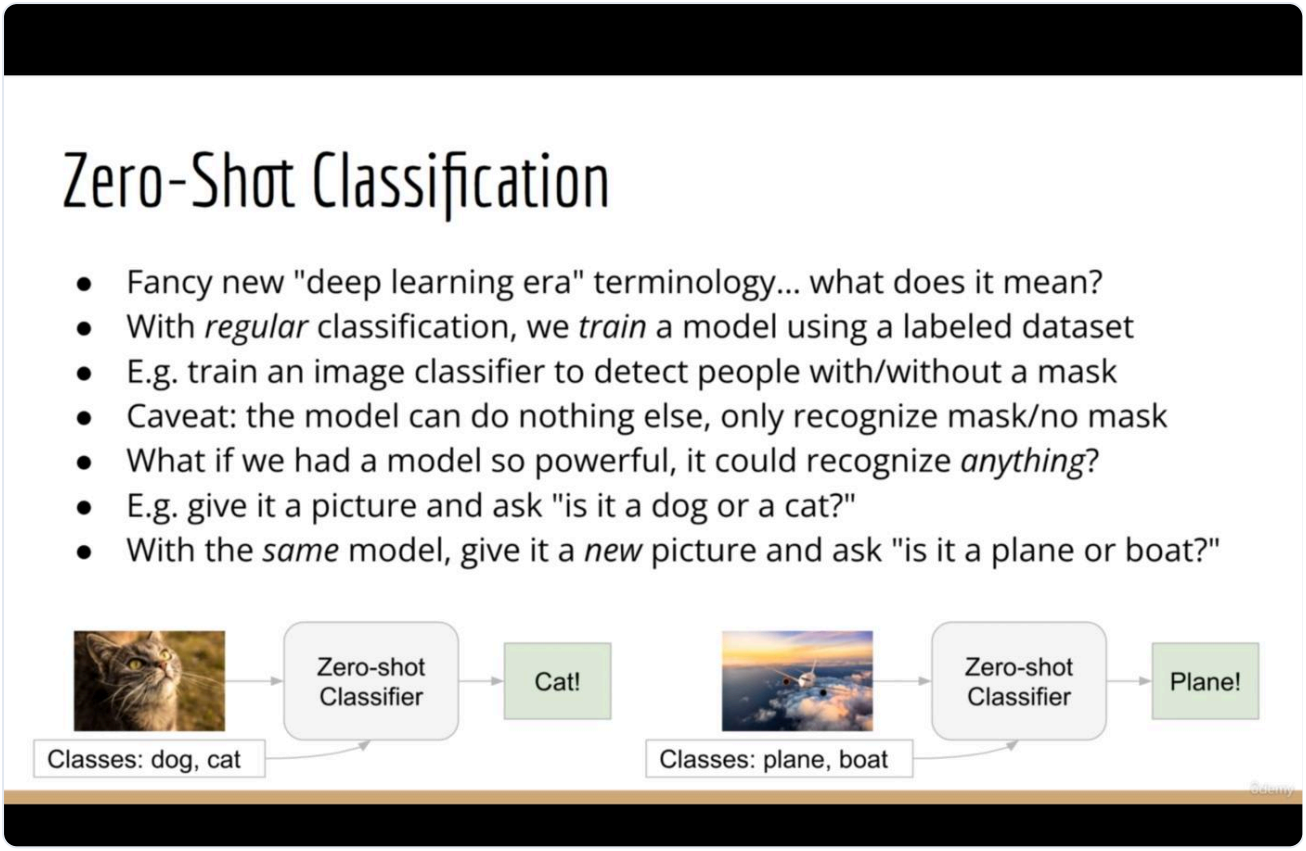

FIGURE 3.193

But suppose that you built a model so powerful that it could recognize anything. For instance, suppose I had some arbitrary model and I asked it. Is this a picture of a dog or a cat? And suppose this model could tell me this is a picture of a cat.

Now suppose I could use that same model and ask it. Is this a picture of a plane or a boat? You might think there's no way a model could be that flexible, when in fact this is precisely what zero shot classification is. Now since this is a course about NLP, let's consider some text based examples. Although do note that this type of paradigm works with images too. Suppose my input document is a wikipedia page on Albert Einstein and I want to know is Albert Einstein a scientist or a painter?

So I can ask my NLP model to classify this document given the classes scientist and painter. Obviously the correct answer is scientist and in fact the NLP model would likely give us this answer. Now, with this same model, I can input a completely different document and I can specify a completely different set of classes. For example, I can pass in a document about the mitochondria and I can ask, Which subject does this belong to? Biology, math, or psychology?

And impressively, the model will likely output biology. And again, much like the question answering task, if you have some experience with building neural networks, it would be useful to think about how would you build a model to do this? Keep in mind that this does not work like a typical softmax neural network. As you recall, for a typical classification task, we would have one final dense layer with a number of outputs equal to the number of possible classes.

Each output corresponds to a different class, but of course, zero-shot classification does not work this way.

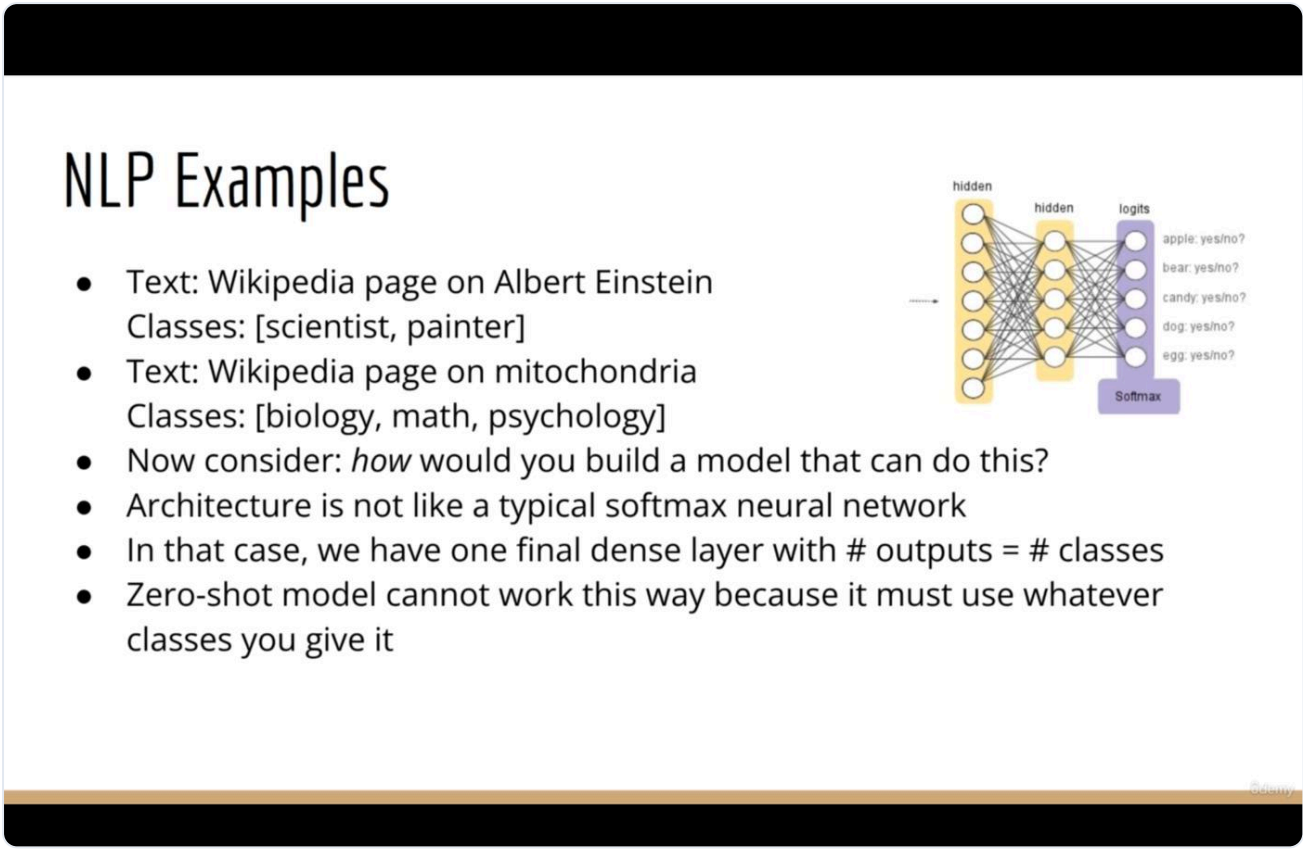

FIGURE 3.194

And this is clear because it can handle any set of classes you give it. So think about how you might accomplish this. So I hope it's pretty clear why zero shot classification is useful and relevant.

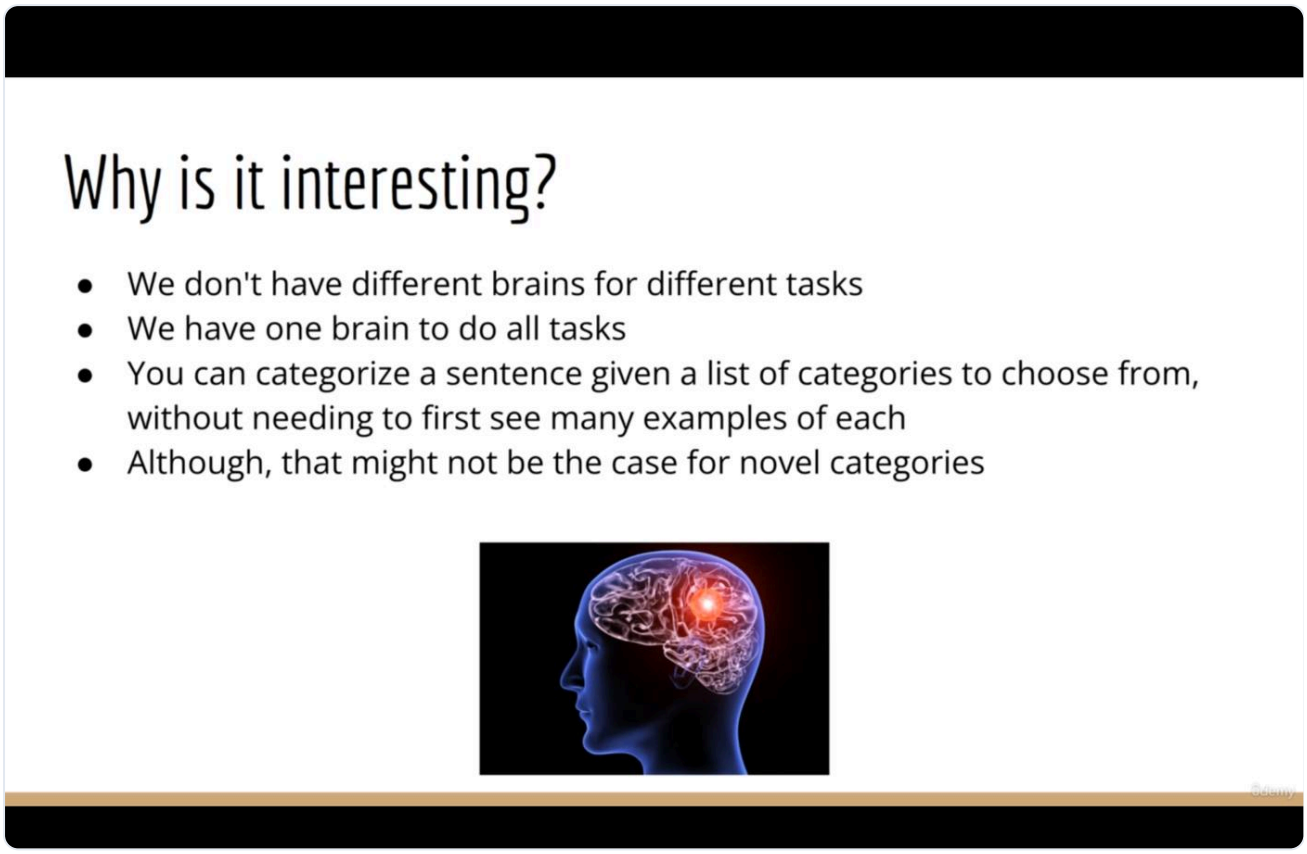

FIGURE 3.195 | Why Is It Interesting?

We don't have different brains for different tasks. We have one brain which can do all tasks. So if I give you a sentence and I ask you to categorize that sentence, you would be able to do so without needing to see many examples of each category.

Now, that might not be the case if it's something completely novel to you. But for everyday sentences, you should do pretty well. So that's why zero-shot classification is so intriguing. So now that you understand what zero-shot classification is and why it's relevant, let's discuss how to do it in code. As usual, we begin by calling the pipeline function and this time we use the string zero-shot classification. Note that this is quite slow, so you'll want to use the GPU.

The next step is to use the model by passing in the text you want to classify and the labels you want to classify the text into.

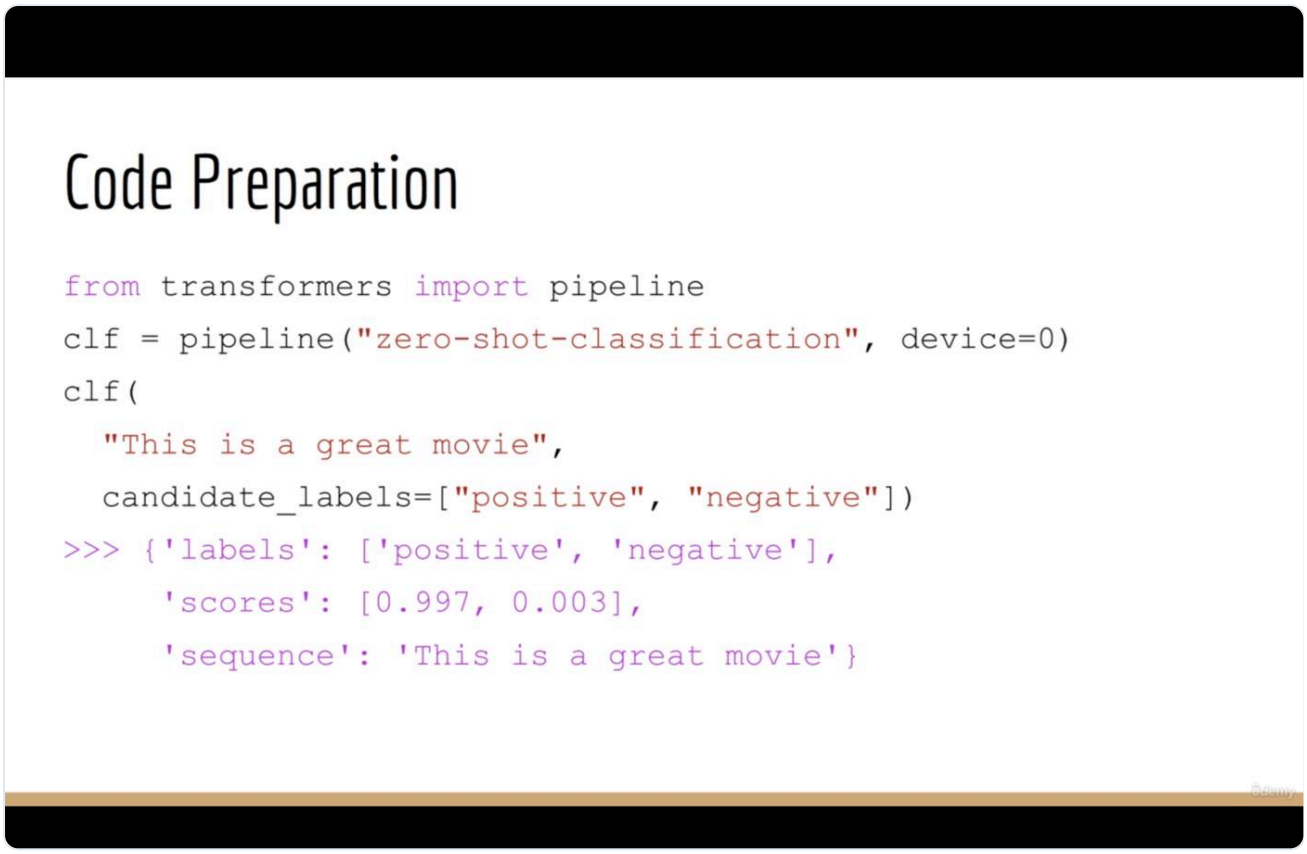

FIGURE 3.196 | Code Preparation

The labels must be passed in using the argument candidate labels while the text is simply the first argument. As usual, you may also pass on a list of strings, in which case each of them will be classified according to the provided labels. The output of the model will be a dictionary which requires a bit of explanation. So there will be three keys in this dictionary.

The first key is sequence, which is simply the text you passed in. The second key is labels, which is an ordered list of the labels you passed in. Note that they are ordered in descending order according to their score. As usual, you can interpret the score as a posterior probability. The third key is scores, which contains the corresponding scores, and as per the labels, they will be in descending order. Thus, you have to be careful that you treat the scores correctly.

In particular, you can't assume that the scores in each position will correspond to the same labels for each sample, which is what we normally do in deep learning.

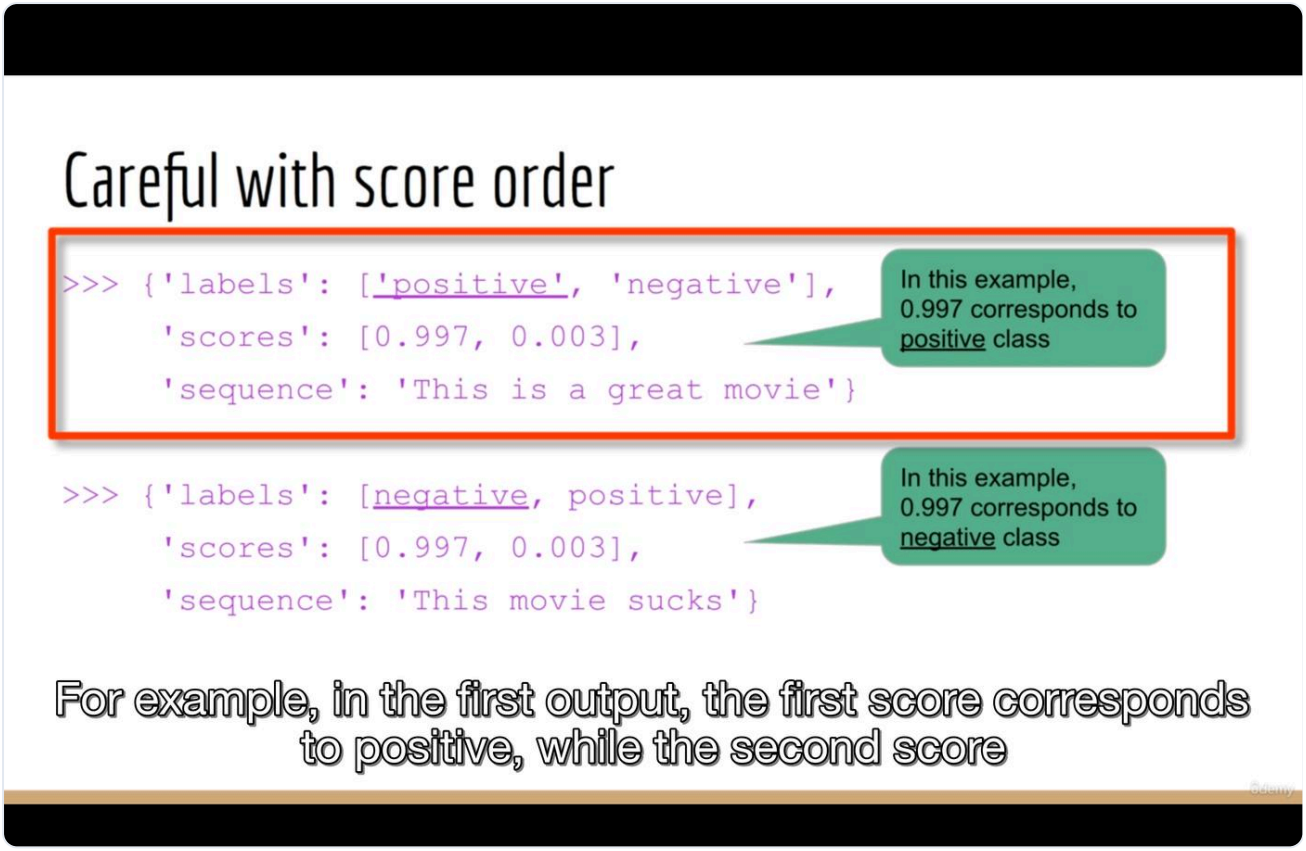

FIGURE 3.197 | Careful with Score Order

For example, in the first output, the first score corresponds to positive, while the second score corresponds to negative. This is because the model prediction is positive and this score is the highest. However, in the second output, the first score corresponds to negative, while the second score corresponds to positive.

This is because the model prediction is negative and this score is the highest.

18 · Zero-Shot Classification in Python

See the notebook here.

So in this lecture we will look at the code to do zero shot classification. For this notebook, we'll be using the BBC News dataset. So let's download the CSV. The next step is to install Transformers, as we normally do. The next step is to import everything we need for this script. The next step is to instantiate a model by calling the pipeline function. Note that we are using the GPU since this will be much too slow for the CPU. You are encouraged to give that a try just to see how long it will take.

The next step is to just test their model with a simple sentiment task. So the input is this is a great movie and the labels are positive and negative. Clearly, the correct prediction is positive. Okay. So as you can see, the model predicts the correct class with a very high confidence note that the ordering of the labels corresponds with the descending scores. The next step is to test our model again, but this time with a longer string of text. This text is from a Wikipedia page on Ampk, which probably means nothing to you if you haven't studied biology.

Luckily, all you need to know for this lecture is that it has something to do with biology. I've provided the Wikipedia link here in case you want to read about it yourself. Note that as candidate labels, we're going to choose from biology, math and geology. So you can imagine how this would be useful in the real world if you have a bunch of documents to classify, but perhaps no time to label the training set yourself. You can simply use a model like this which can classify given any set of labels.

Okay. So as you can see, our model correctly predicts that the classes biology, math and geology have relatively low scores. And again, notice how the labels are ordered according to the descending scores. Thus, the predicted label will always be in the first position. Okay. So the next step is to use our model on the full BBC News dataset. So we'll begin by loading in our dataset using pd.read_csv. The next step is to check the length of our data set.

As you can see, we only have about 2000 samples, which is quite modest. However, you'll see that the scripts will take quite a long time. Note that our documents are quite long since they are full news articles. And as you recall, attention layers have quadratic complexity. The next step is to look at some samples in our data frame just to remind ourselves of what's in it. So as you can see, we have two columns, one for the text and one for the labels.

So in order to get a list of labels, we're going to cast the labels column to a set. As you recall, the elements of a set are unique. So this will return the set of all unique labels. However, the input to our model must be in the form of a list. So after casting the labels to a set, we then further cast it into a list. Okay. So as you can see, we have five labels tech, business, politics, entertainment and sport. The next step is to print out a random document which I've arbitrarily chosen.

As you can see, it is quite a long document. Let's now check the label. As you can see, this article is an article about politics. So let's test our model to see if it works okay on this document. Okay. So as you can see, it does correctly predict that the document is about politics, but note that the score is not that high. This may indicate that the model is not as good at discriminating between these labels. Or perhaps this is just a difficult example.

The next step is to convert our text column into a list and then run that through our model to get a full set of predictions. Note that this will take quite a long time, so you may want to find something else to do if you're running this yourself. Okay. So as you can see, that took about an hour, which is significantly longer than the other tasks we've looked at. The next step is to simply print our predictions to remind ourselves of the format. Again.

The predicted label is located at the first position in the list of labels in each dictionary. However, because the labels and scores are ordered, the probabilities are not in a consistent order from sample to sample. Thus, if we want to create a big matrix of probabilities to compute a metric like the AUC, we'll need to reorder them to be consistent. The next step is to generate a simple list of predicted labels in the string format. To do this, we'll use a list comprehension to loop through each dictionary in the result.

For each dictionary, we'll grab the key labels and we'll index the resulting list at index zero. As you recall, this corresponds to the label with the top score. The next step is to assign our list of predicted labels back to our data frame. This will make it easier to compute the accuracy. The

next step is to compute the accuracy, which is quite easy since both the targets and predictions are in the data frame. Okay. So as you can see, our model doesn't perform exceptionally well.

However, do note that because we have five classes, the expected accuracy, if we were to make random predictions would be 20%. Thus, this is still much higher than the naive benchmark. The next step is a bit of a challenging coding task, which is to compute our matrix of probabilities. As you recall, this is not straightforward since the scores in each dictionary are not in the same order. We'll begin by defining N and K, which are the number of samples and number of classes respectively.

The next step is to define a mapping from label to index. Basically, we're going to use our existing list of labels as the official ordering. So that's politics, sport and so forth. You can double check that above if you like. So why do we need this? As you recall in our results, we only have string labels and scores, so we know which scores map to which string labels, but we don't know which scores map to which column of this probability matrix. Therefore, we need a mapping from string label to index by always using this mapping.

We can ensure that our scores are always in a consistent order. Okay, so the next step is to instantiate our probability matrix, which will be a matrix of size N by K. The next step is to look for each result going from zero up to N. Note that we need the index i, since this will correspond to the row of our probability matrix. Inside the loop we first grab the prediction dictionary at index I, which will call D. The next step is to loop through the labels and scores in D.

As you recall, the labels in scores are both in corresponding order and thus we'll use the zip function to loop for both of them concurrently. So this will give us one label and the corresponding score for each iteration of this loop. Inside the loop we use our label to index mapping where we pass in the label and we get back the index K as recall. This corresponds to the column of our matrix. Therefore we now know where to store the score we stored at row i column k.

Now, as you recall, if you want to compute the AUC, we'll need to have the labels as integers. This will also be useful for the next step, which is to compute the confusion matrix. So to compute the integer labels, we simply loop through all the labels and map them using the label2index dictionary we defined above. The next step is to compute the confusion matrix, as mentioned. Now, as you recall, I said we would be using integer labels and predictions.

Thus, before we call the confusion matrix function well, first take the argmax of our probabilities to get our predictions into integers as well. So let's talk about why we want our inputs to be integers. Well, as you recall, a confusion matrix will list each label along the rows and each label along the columns. However, this is just a matrix. In other words, a numpy array. We don't automatically know which index will be assigned to which label, since our targets and our predictions would otherwise simply be strings.

Therefore, in order to ensure that we know which position goes with which class, we'll use integers instead. So the first element will be for class zero. The second will be for class one and so forth. So the next step will be to plot our confusion matrix. As you recall, we've defined our own plotting function since scikit-learn is currently in a transitory period. However, it's quite easy to use scikit-learn's interface. So please try that if you like.

Note that this is the time for us to apply our string labels so that we can easily see which index corresponds to which label. Okay. So as you can see, our model does perform better than chance for all classes, although performance varies between each class. The next step is to check the F1 score. Now, as you recall, the F1 score is more naturally defined for binary classes. Thus, to use it, we need an extra argument for average, which we've set to micro.

Okay. So as you can see, the F1 score reflects the accuracy. Since the classes are not imbalanced. It's not great, but it's acceptable. The next step is to check the AUC. Again, this is more naturally defined for the binary case, but there are ways to modify it for the Multi-Class case. Thus we've passed in multiclass equals OVO, which stands for one versus one. Notice again that this requires integer labels. Since we need to know which column in our probability matrix corresponds to which label.

Okay. So as usual, the AUC is a bit more optimistic than the F1 score. So I hope you found this example quite interesting in that we were able to get a model to classify a set of documents using classes it has never been trained on. Although the model didn't perform as well as a classifier that was specifically trained for these classes, the results were still surprisingly good. In fact, it's interesting that something like this is even possible now as a final exercise for this example.

You're going to try the same thing, but with the airline tweets, sentiment, dataset. Feel free to filter out the neutral class or keep it if you like. If you want to have three classes, otherwise, just use positive and negative. Note that this will be a binary classifier, so you'll have to change some of the code above accordingly. Once you've seen the results, you should compare them with the results from the actual sentiment analyzer that we saw earlier in the section.

What you should find is that the results are pretty surprising, so please try it yourself and see if you can figure out what I'm referring to.

19 · Beginner's Corner Section Summary

So now that you've seen plenty of examples of how transformers can be used, it's now time to close off this section.

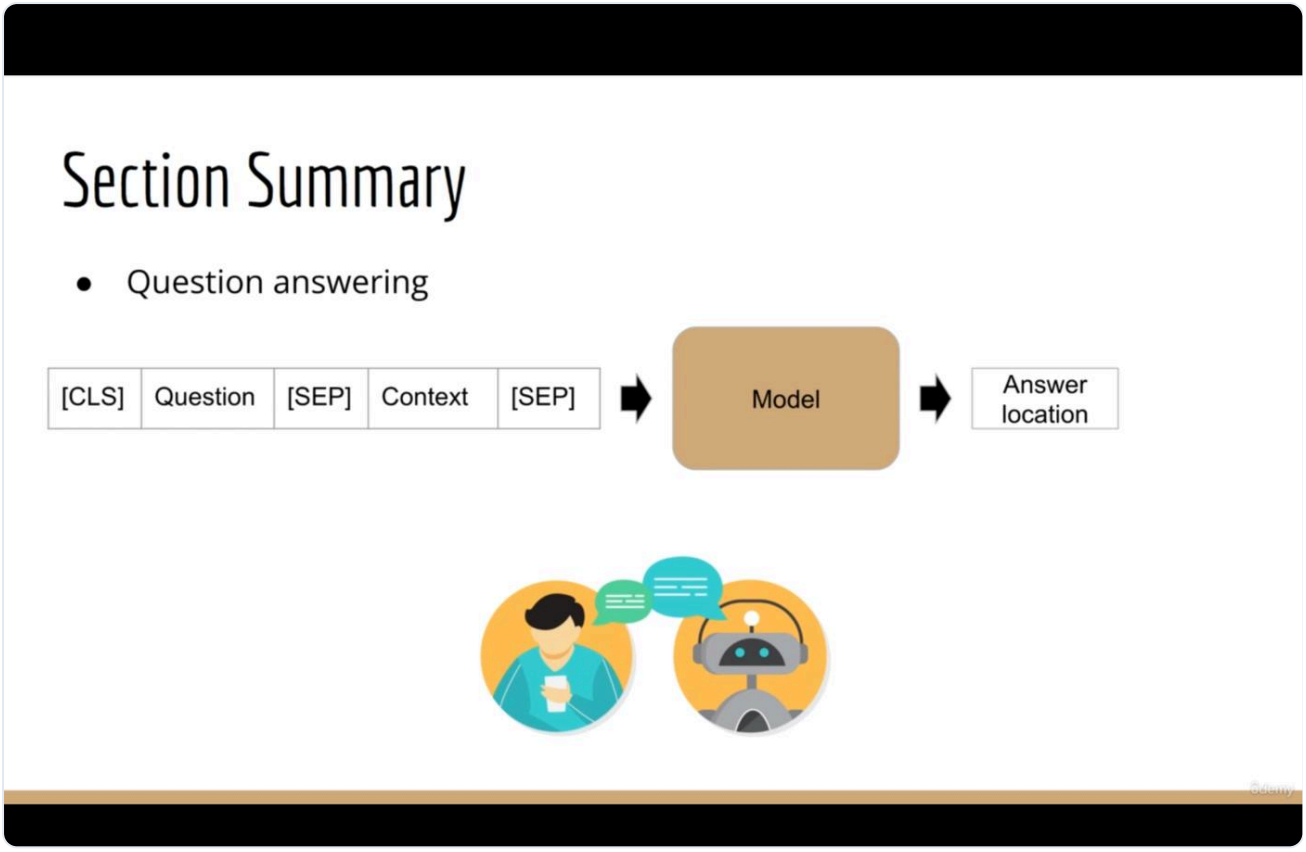

FIGURE 3.198 | Section Summary

This lecture will recap each of the applications we looked at, and we'll also review the sort of big picture of why they are relevant for this course.

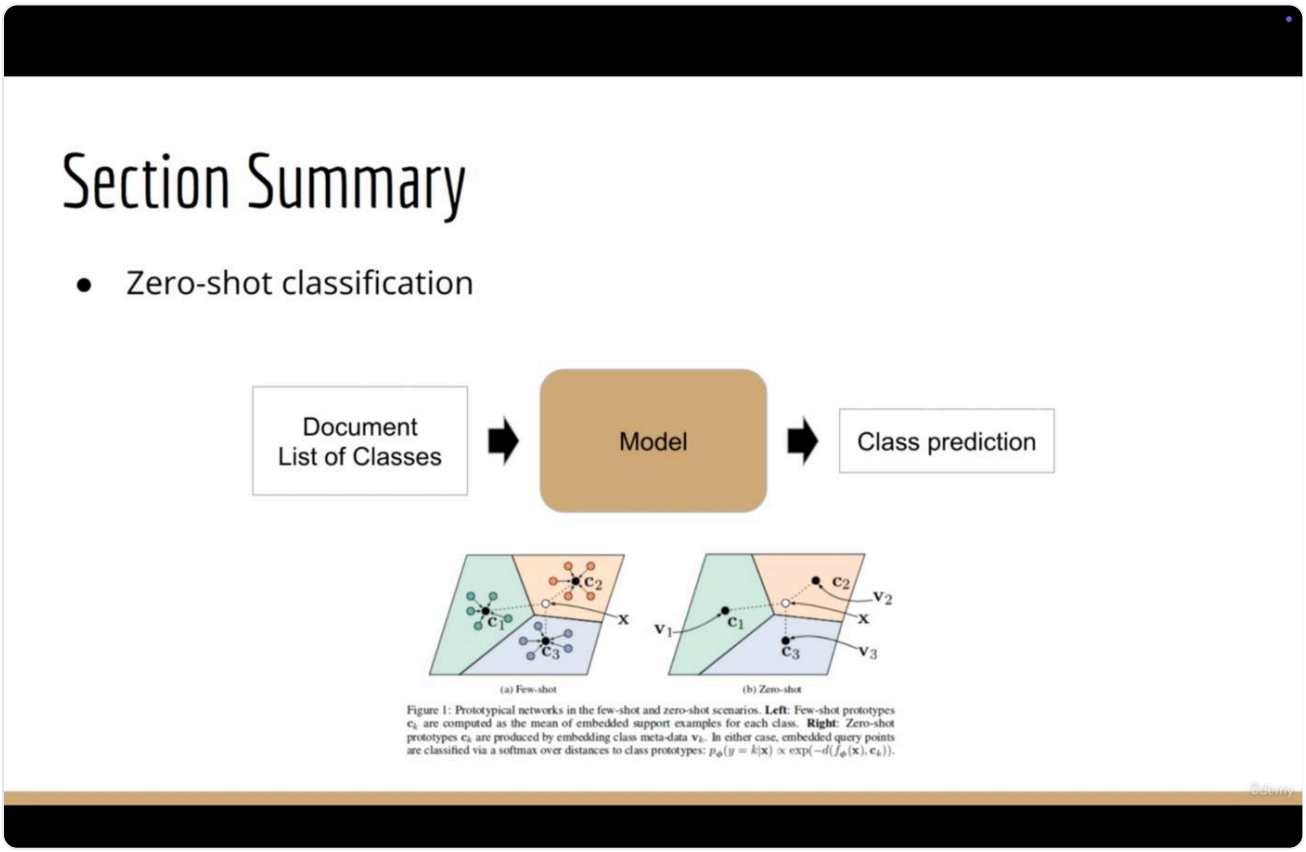

FIGURE 3.199 | Section Summary

So we began by looking at sentiment analysis, which is a simple to understand task, but also non-trivial in some ways. As you recall, simple models that use bag of words can obtain pretty good performance, but they fail to account for relationships between words due to the fact that order is not taken into account.

FIGURE 3.200 | Section Summary

For example, seeing this show was interesting versus this show was not interesting has opposite meanings.

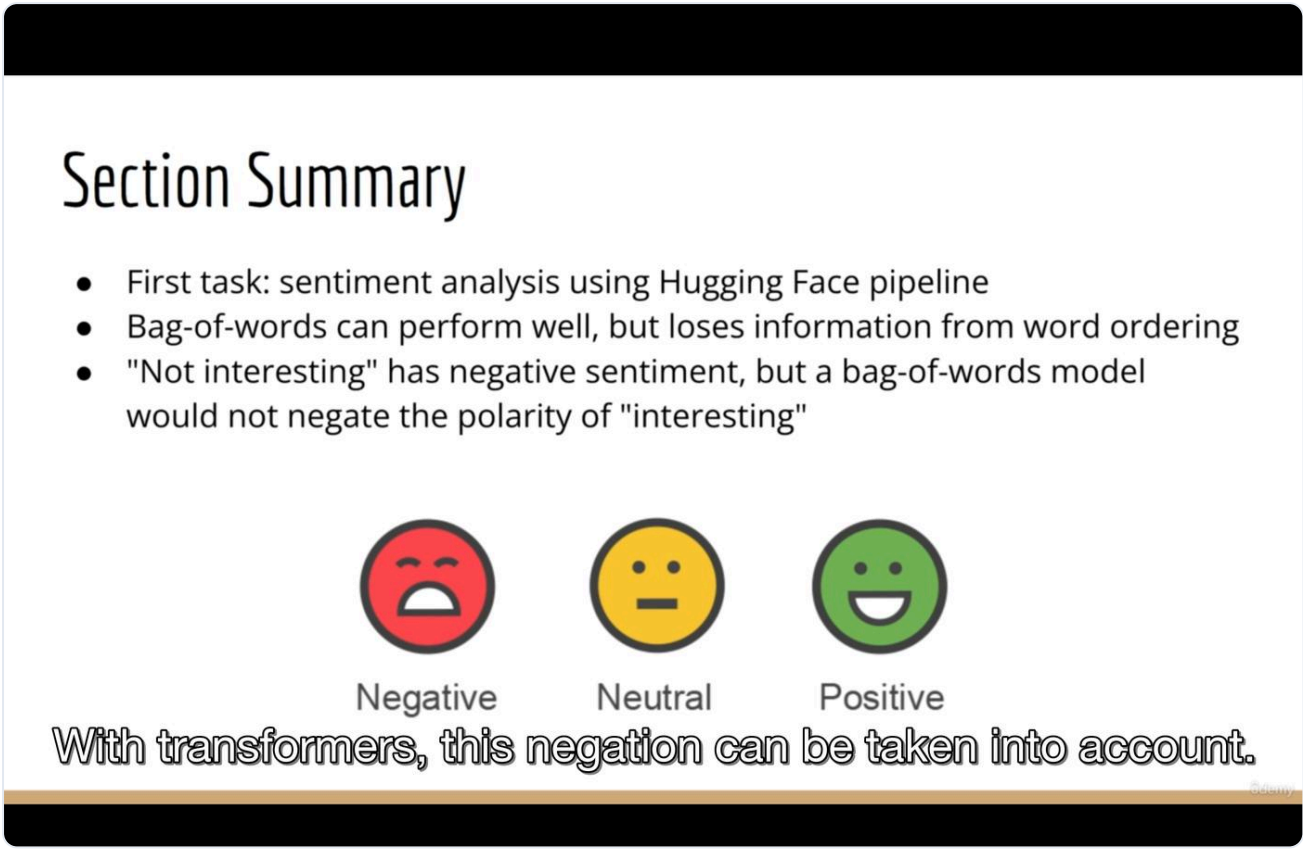

FIGURE 3.201 | Section Summary

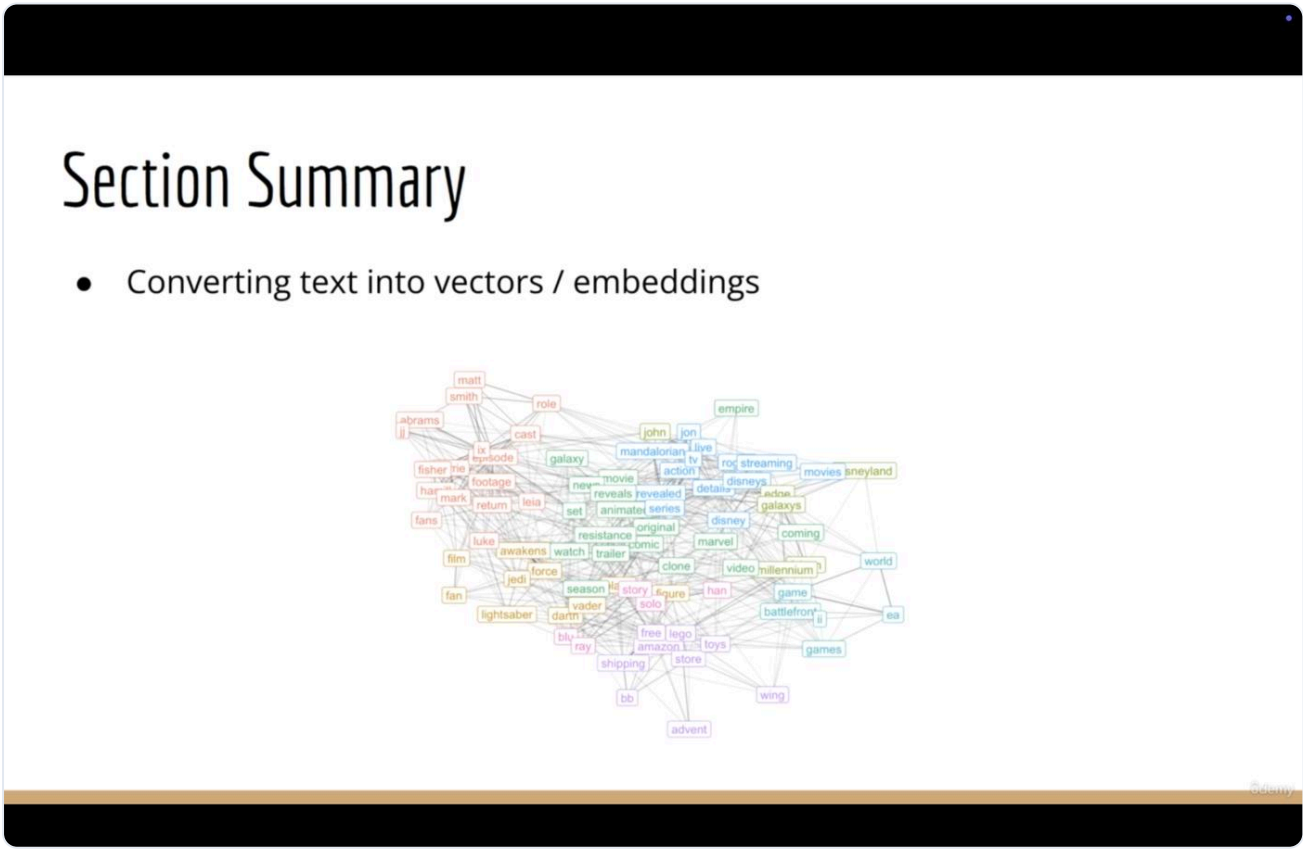

FIGURE 3.202 | Section Summary

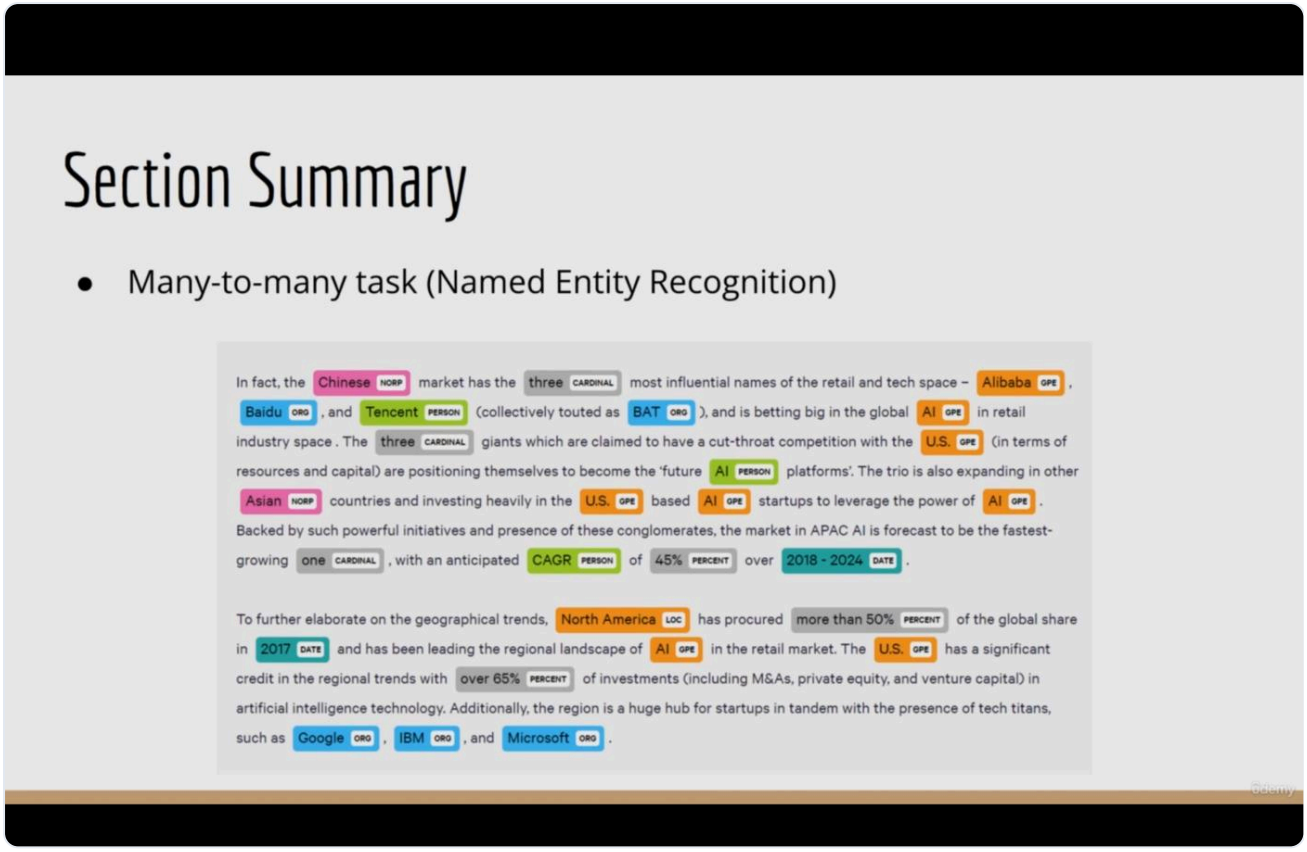

FIGURE 3.203 | Section Summary

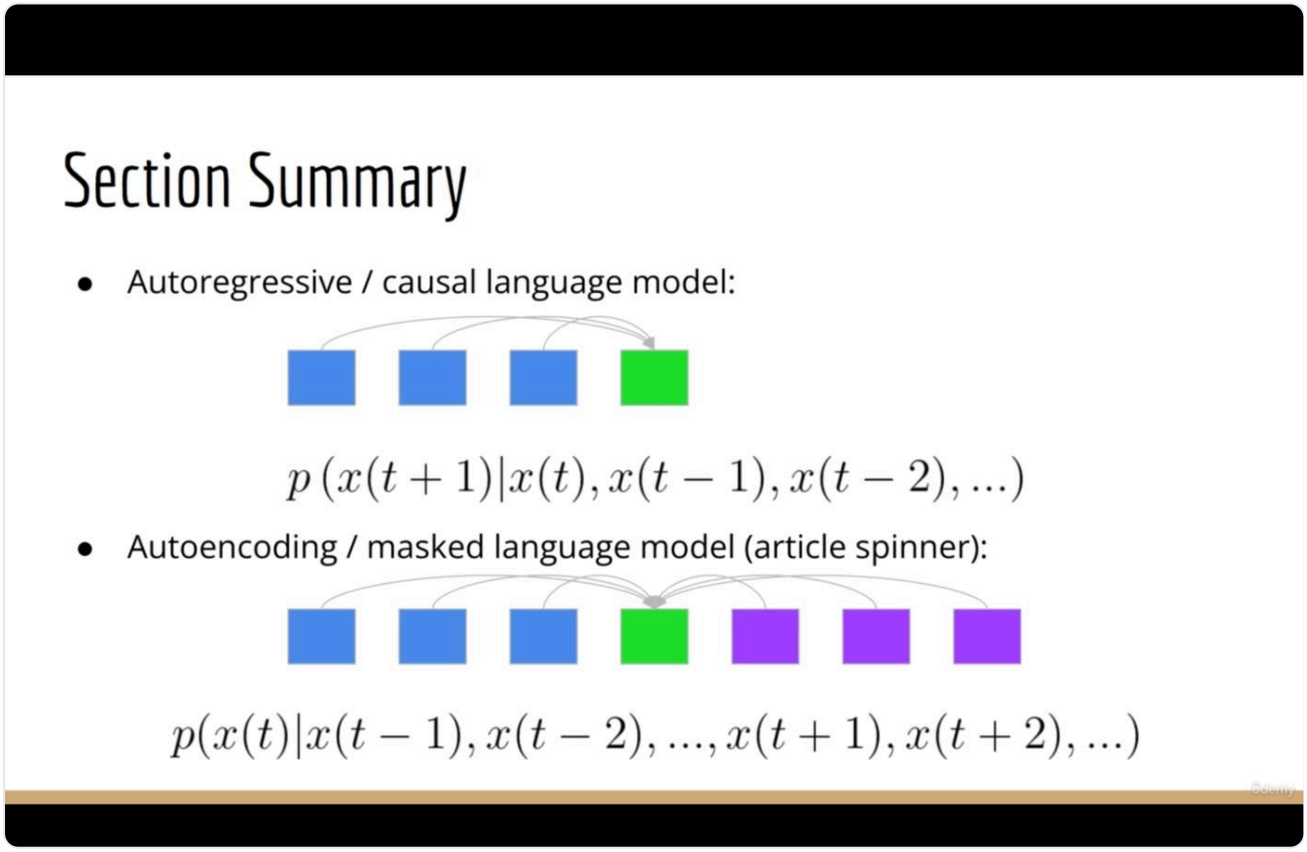

FIGURE 3.204 | Section Summary

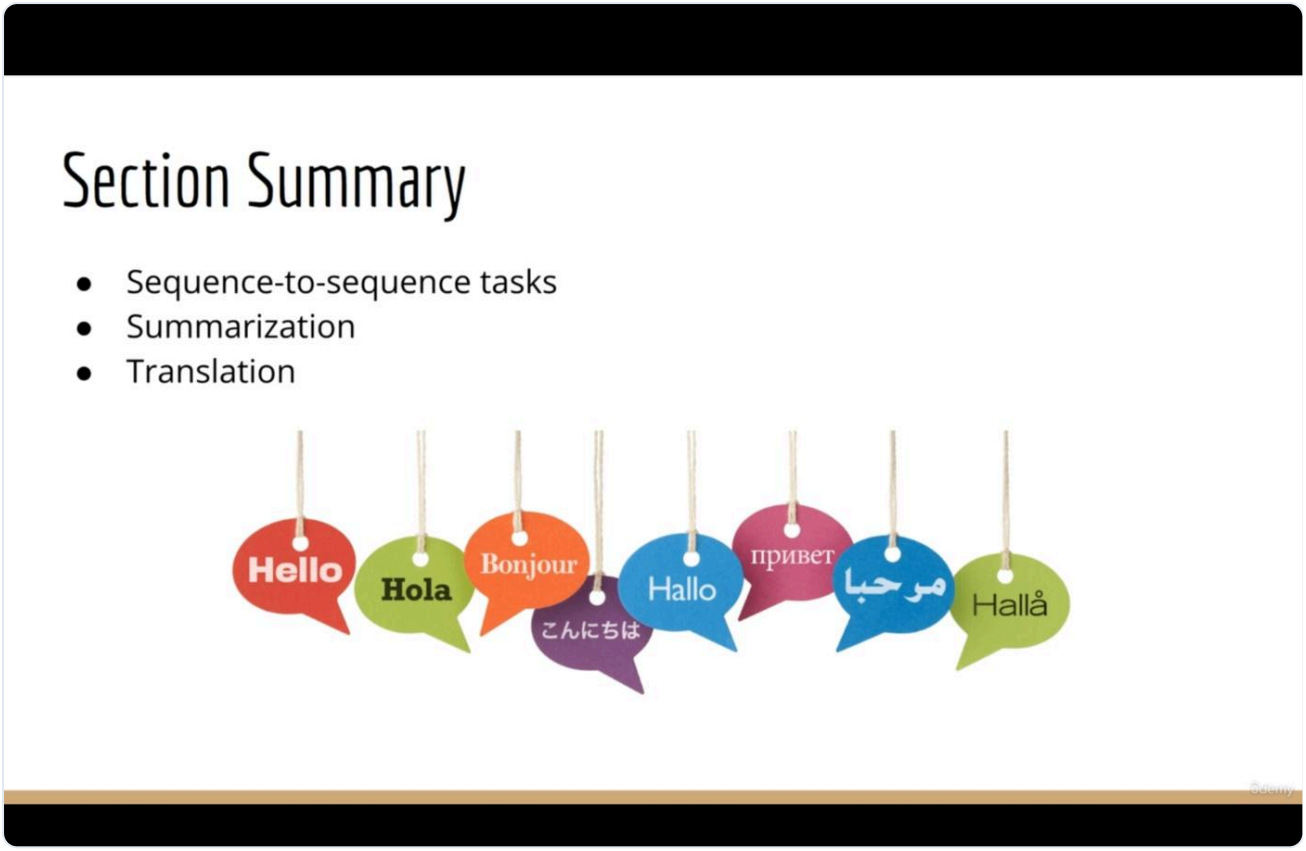

FIGURE 3.205 | Section Summary

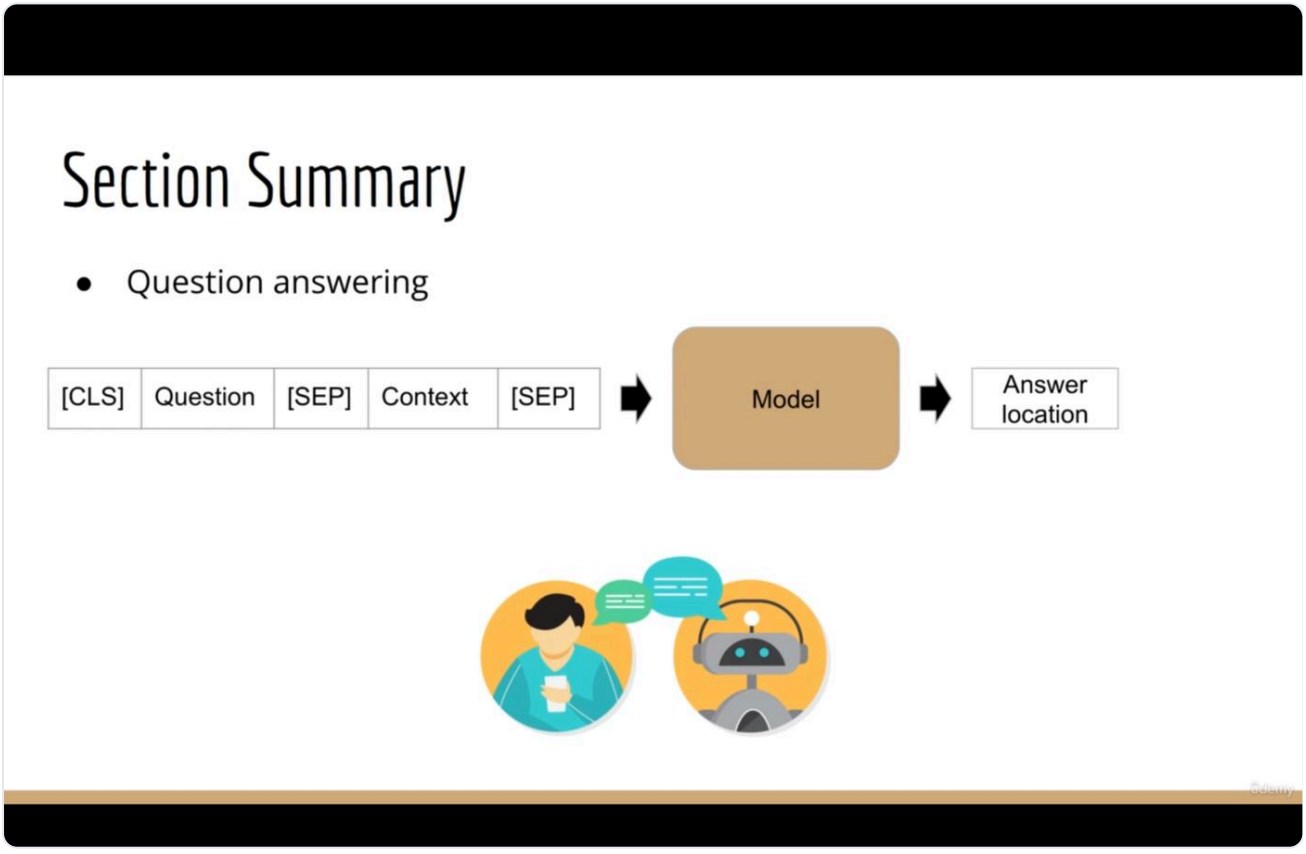

FIGURE 3.206 | Section Summary

We know that not interesting has negative sentiment, but a bag of words model might only know that interesting is positive and the word not does not negate its polarity. With transformers, this negation can be taken into account. The next task we looked at was how to convert documents into vectors or embeddings. This is similar to what we do with images where we pre-trained a neural network and then we take the embedding vector from the second last layer.

These turn out to map the inputs into a useful embedding space where we can do things like face recognition for documents. This is useful for things like similarity search. We then looked at a many-to-many task, namely named entity recognition. This is a classic task in NLP which can be handled by sequential models like RNNs as well. The next task we looked at was how to generate text by training a language model. This is a completely natural thing for a transformer to do because this task is actually what we use to train our transformers.

We learned that the GPT family is trained as an auto regressive or causal language model, meaning that each word is predicted from the past. We observed firsthand the impressive capability for these models to generate very long and coherent documents. We then looked at a different kind of language model, which was the auto encoding or masked language model. This is essentially the article spinner, which I've described in the past, and I showed you how many people have been making use of such technologies.

Importantly, these are people who are willing to pay money for such tools. We then turned our attention back to supervised training, where we looked at some sequence to sequence tasks. Now, although parts of speech tagging and named entity recognition are also sequence to sequence tasks, there is a big difference between those and these other examples, namely text summarization and language translation, in particular with summarization and translation.

The output does not have the same length as the input. This is unlike tasks like named entity recognition where we have a 1 to 1 mapping from each input token to each prediction. The next task we looked at was a completely new task known as question answering. This is an interesting task because unlike the others, it requires more understanding of the relationships between the words and the input. Firstly, we noted that the input actually consists of two items.

The question in the context, which is a document that contains the answer to the question. So this task not only requires the model to understand the relationships between the words and the document, it must also know that the question is something to be answered and that the answer lies somewhere in the context. Personally, I find these results very impressive. The next task we looked at was another new task known as zero-shot classification. This is another very interesting task because it is unlike typical machine learning where we must train a model on a labeled dataset if we want it to be able to classify inputs from a certain set of classes.

In other words, it used to be the case that we had to train a model specifically for a task if we wanted to do that task. But this is different using the zero shot method. We can use the model to classify inputs from among any set of classes, which means that we are asking the model to do something it has never done before. To me, this is very impressive as well. And thus these two final tasks are a good way to end the session by demonstrating the true potential of transformers.

20 · Suggestion Box

So in this short lecture, I want to tell you about my suggestion box. The idea behind this is analogous to the kind of suggestion box you would find at a restaurant. You can write down your thoughts and leave them for the owner to look at, to give them feedback so that they can improve your dining experience in order to share your thoughts with me in my suggestion box. Please go to lazyprogrammer.me/suggestions. Here you will find a simple form where you can tell me anything you like.

I have a few suggested fields, but none of them are required. However, they would be helpful for me to know. For example, tell me your background. I think someone coming from a background in marketing is going to have very different feedback compared to someone coming from a Ph.D. in physics. Tell me what course you are taking. So I have some idea of what you are talking about. Obviously, the more specific you are, the better I can help you. Tell me how difficult you thought the course was.

Was it too easy? Was it too hard? Tell me if there was something I neglected to explain. For example, was there some word that I used that you have never heard of? Was there Python code that you haven't seen before? Tell me what you thought was missing from the course, even if it was just something small, like you missed explaining this line of code. Were you looking for an algorithm that wasn't included? This is helpful for me, but it's also helpful for you because many times students are looking for an algorithm or an explanation in the wrong course.

So I can say, actually, I teach that algorithm or I explain that thing in such and such course and this isn't the right course. So have a look over there. Are there any topics you want to request that were not included in this course? For example, maybe you signed up for a deep learning course and you were looking for a science, but the course didn't have CNN's in it. Let me know. Finally, let me know your suggestions for future courses. If there's a particular topic you want me to teach that I don't yet have a course about, such as gradient boosting or transformers or even quantum mechanics.

Let me know. Finally, there is a big text box for any comment you want to write. You can write anything in this box and make it as long as you want. In fact, the longer the better. If you think I talk to you slower, I talk too fast. Let me know if you think my prerequisite instructions aren't fair. Let me know. I want you to be as specific and as detailed as possible. A lot of the time people give me comments that I can't really act on, which is unfortunate, and this is often because they are too generic and not specific enough.

Tell me exactly what lecture and exactly what time stamp you're referring to. If you want to refer to a question on the Q&A, include a link. Be very specific and give me concrete examples so that I know exactly how to act on the information you've given me. Thanks for listening and I'll see you in the next lecture.

SECTION 4

Fine-Tuning (Intermediate)

1 · Fine-Tuning Section Introduction

So in this lecture we will be introducing the next section of the course, which is all about how to fine tune a Transformer model. So let's think about why this is different from the previous section of the course. The previous section of the course was easy because it only involved using models that someone else has trained.

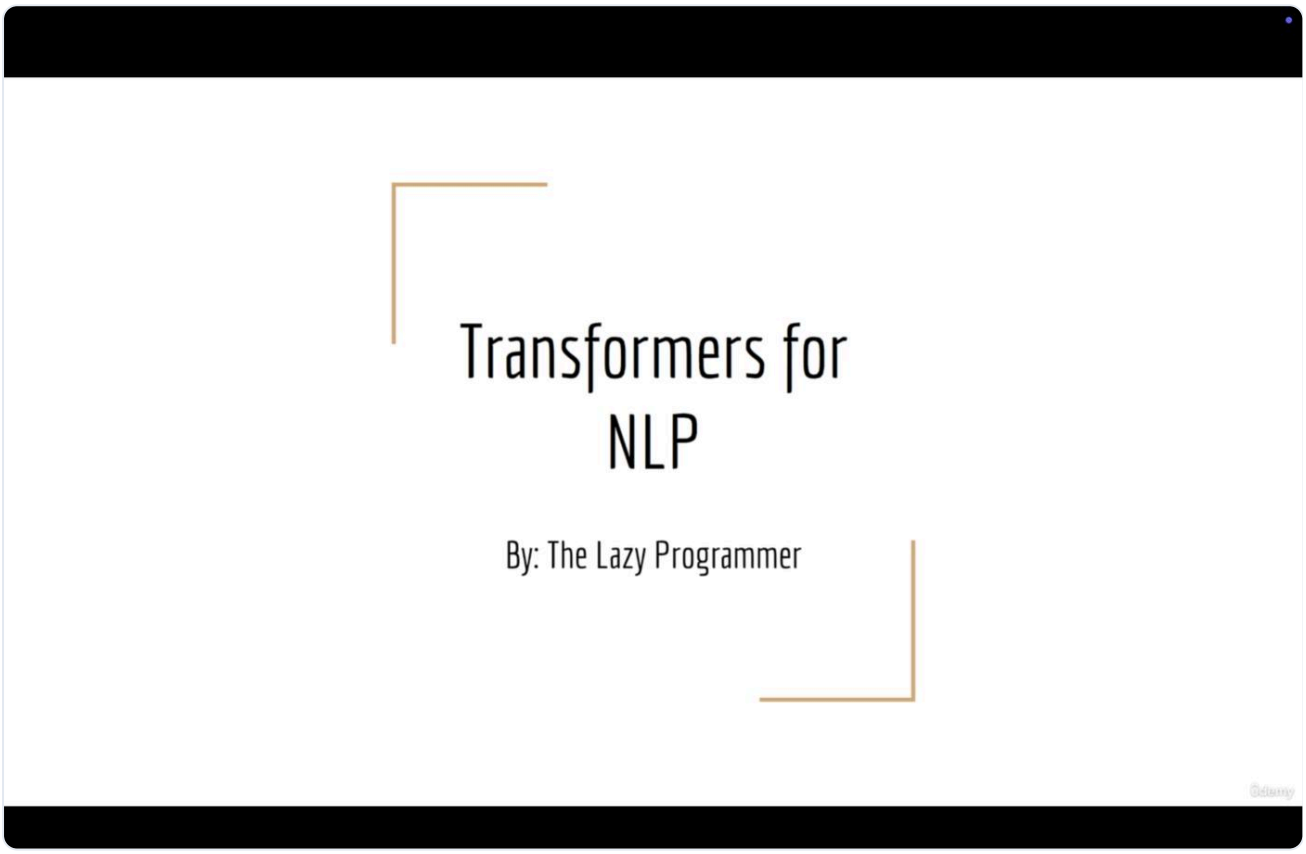

FIGURE 4.1 | Transformers for

We were just using those models in inference mode, also known as predictor mode. The code was very simple because the library we're using provides a nice universal interface for different types of models.

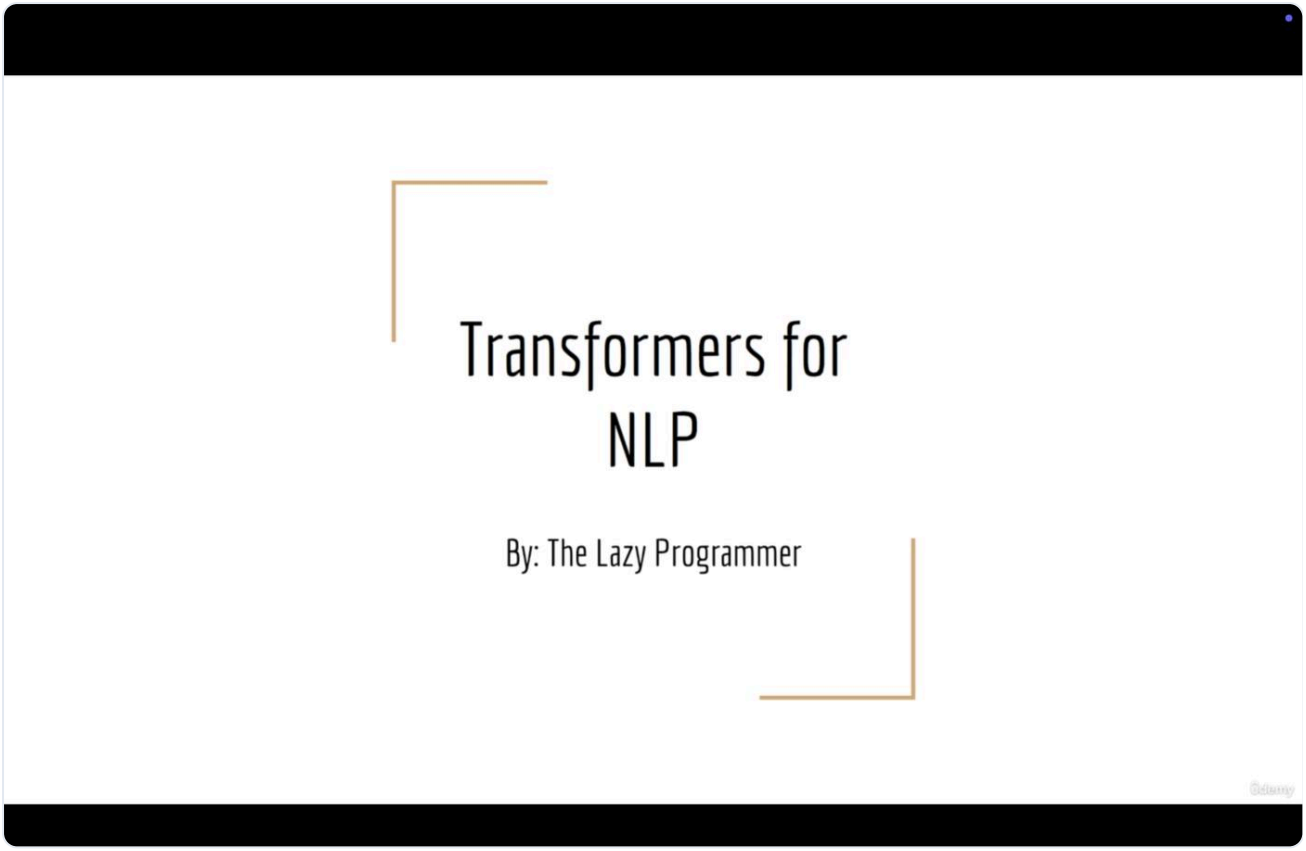

FIGURE 4.2 | Transformers for

So we have the same interface for different kinds of tasks. In this section, we won't just be making predictions, but training our own models as well. This will make things much more complex than the previous section in terms of the code we have to write.

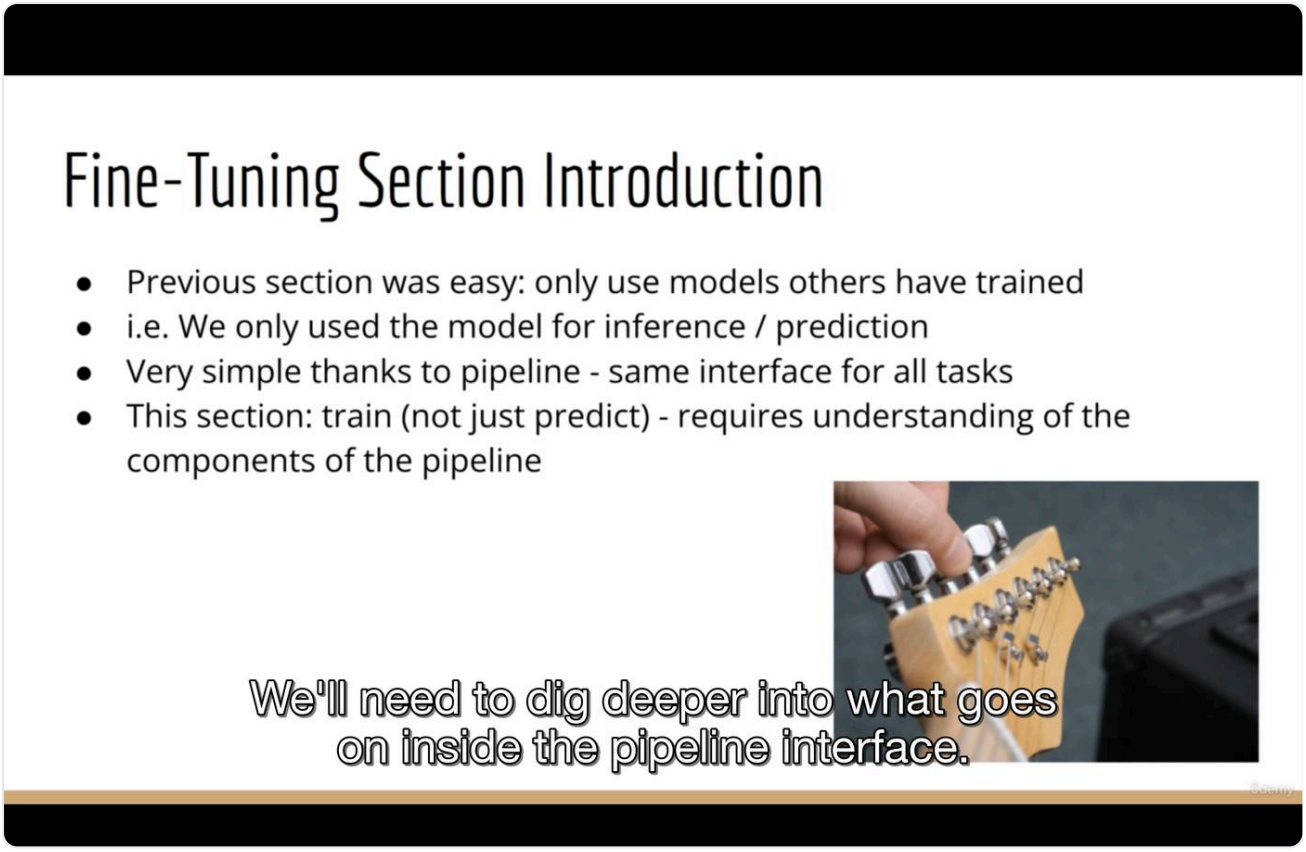

FIGURE 4.3

We'll need to dig deeper into what goes on inside the pipeline interface. So let's do a quick outline of this section so you understand what's coming next. The first thing we'll do is review how text processing works.

As you know, machine learning models work with numbers, not text. Therefore, we'll need a strong grasp of how we convert text into numbers so that our machine learning models can work. This involves a process called tokenization, which splits a single text document into a list of tokens. It also involves mapping tokens into integers as well as padding so that our model inputs all have the same length.

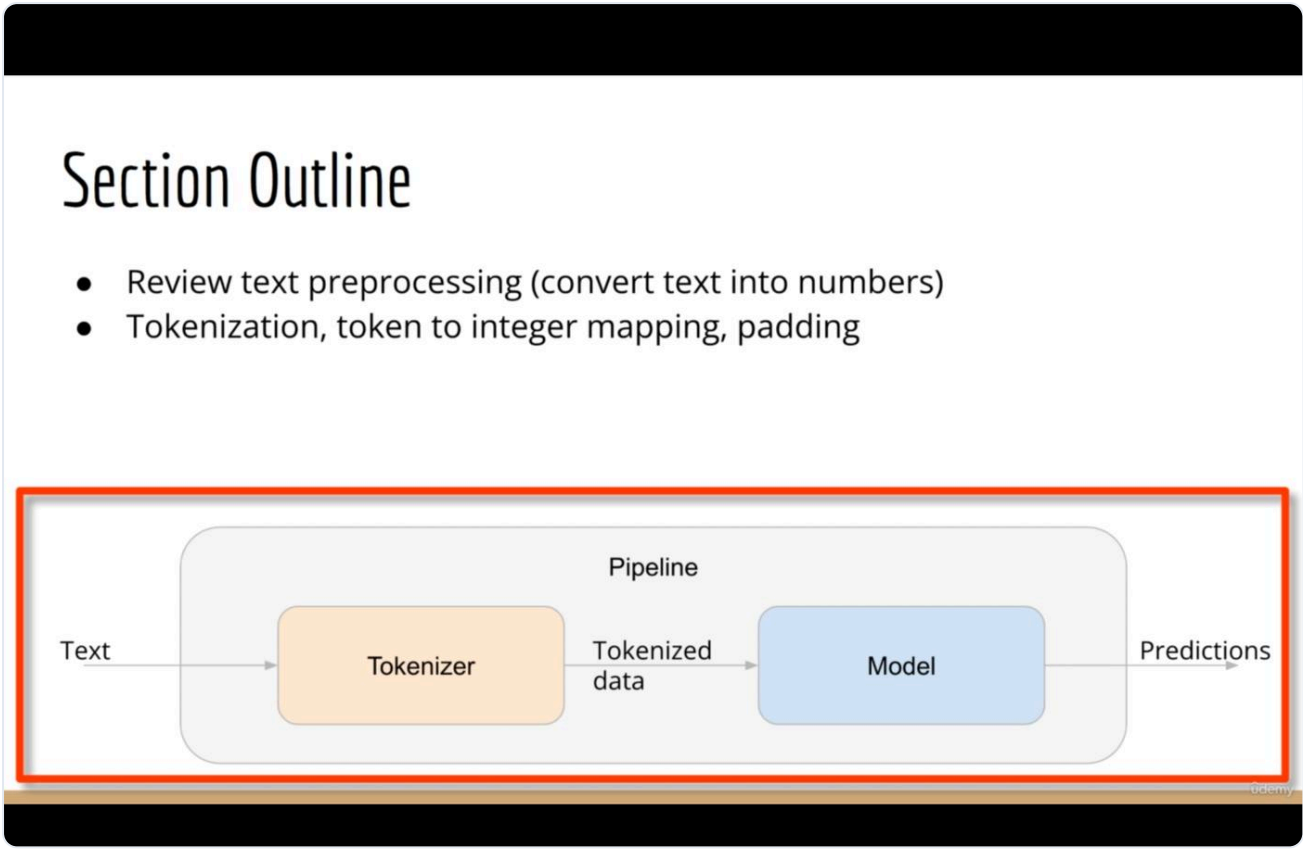

FIGURE 4.4 | Section Outline

This brings us to the next point, which is what exactly goes on when you use a pipeline.

We know that we can just pass in text and magically get an answer. But I just told you how machine learning models don't work with text directly. Well, the key is that the pipelines are actually composed of two parts the tokenizer and the model itself. In the next few lectures, we'll look at how we can use tokenized and models directly with Hugging Face instead of using a pipeline. This will be necessary in order to proceed to the next step, which is how to actually train a transformer.

This is where one of the key innovations in NLP has been made in the past few years.

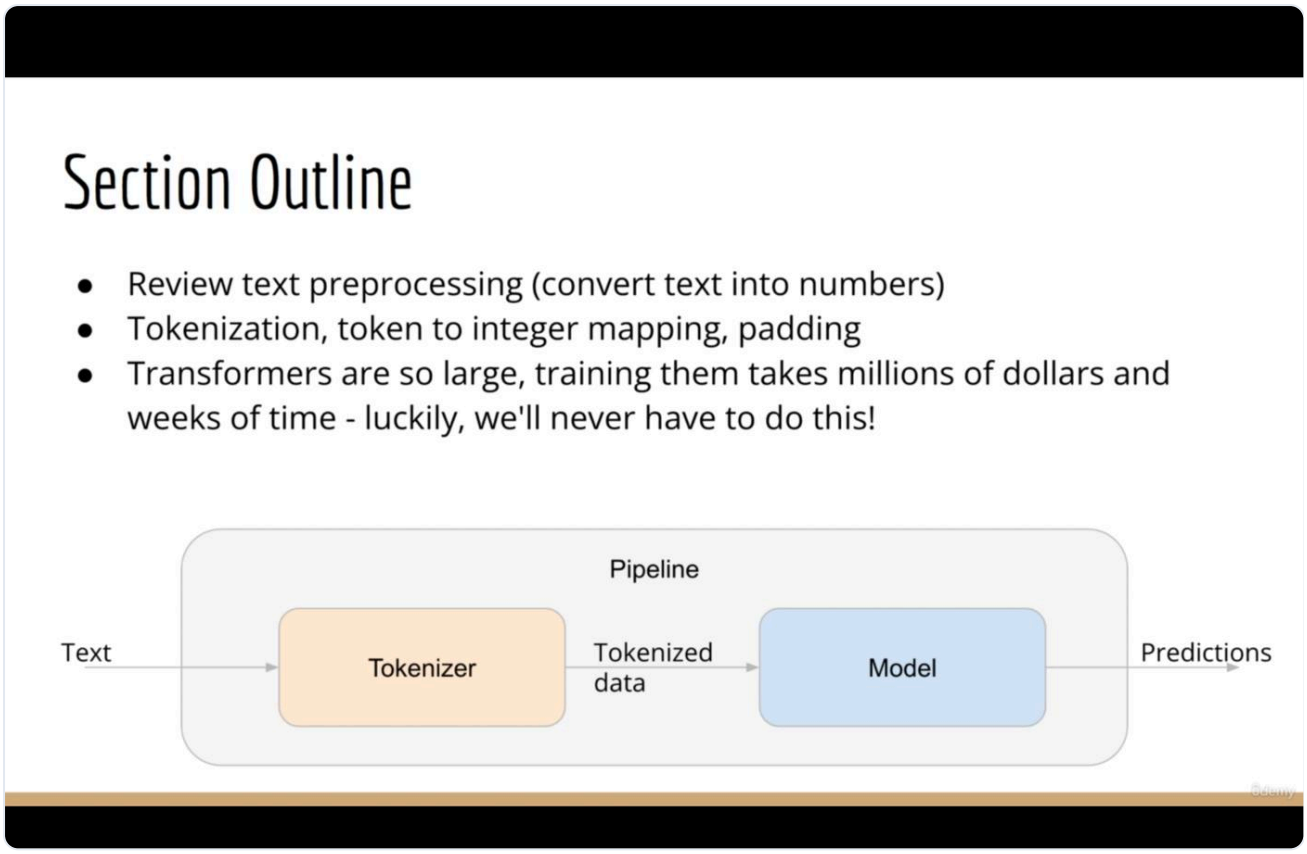

FIGURE 4.5 | Section Outline

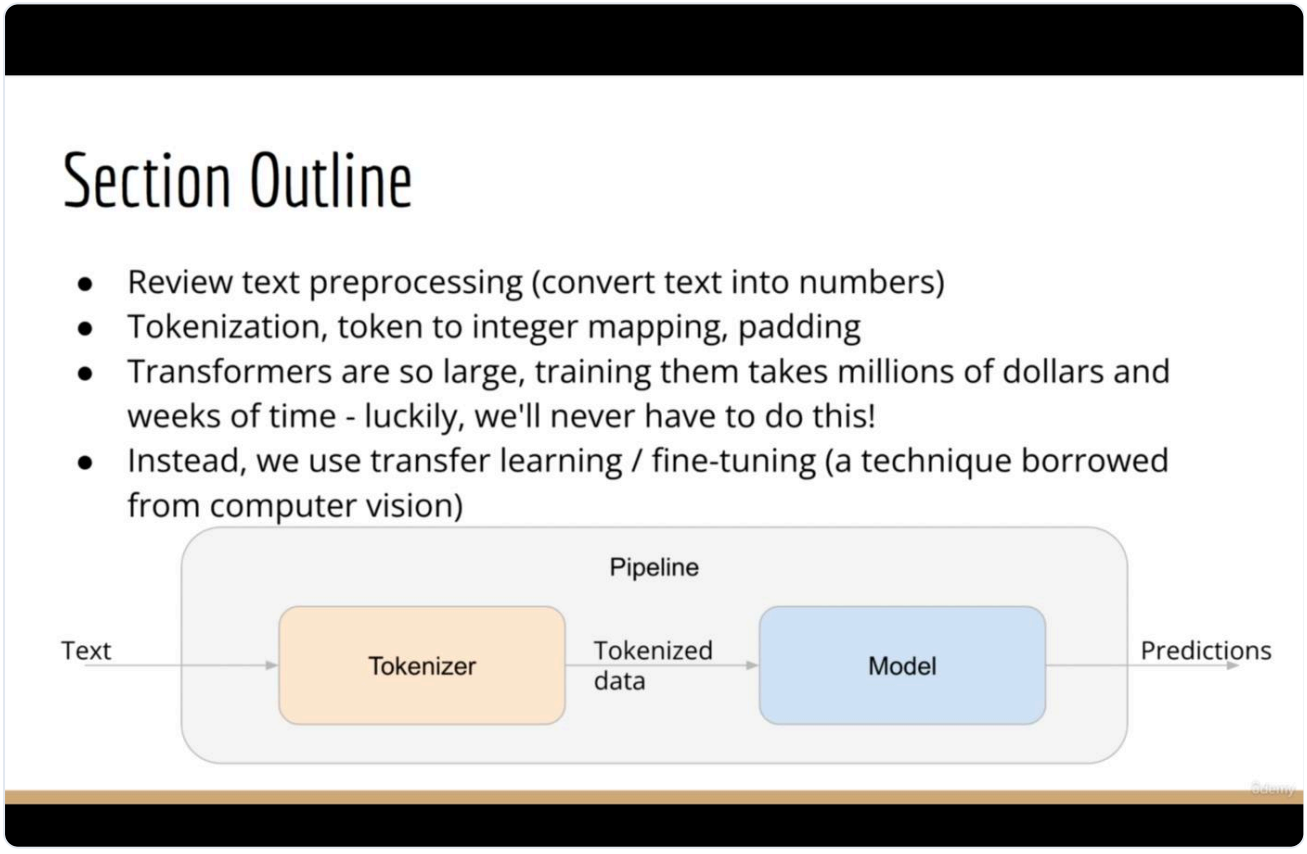

FIGURE 4.6 | Section Outline

You may have heard that transformer models have become so large that they take millions of dollars and weeks of time to train. This is true. Luckily, we'll never have to train a transformer from scratch in this course, which most of us would not have the resources to do. Instead, NLP has borrowed a technique from computer vision, which is to use the pre-trained weights of a large and powerful model as the backbone of a new neural network built for a new task.

NLP uses a technique known as fine tuning, which we'll review in this section.

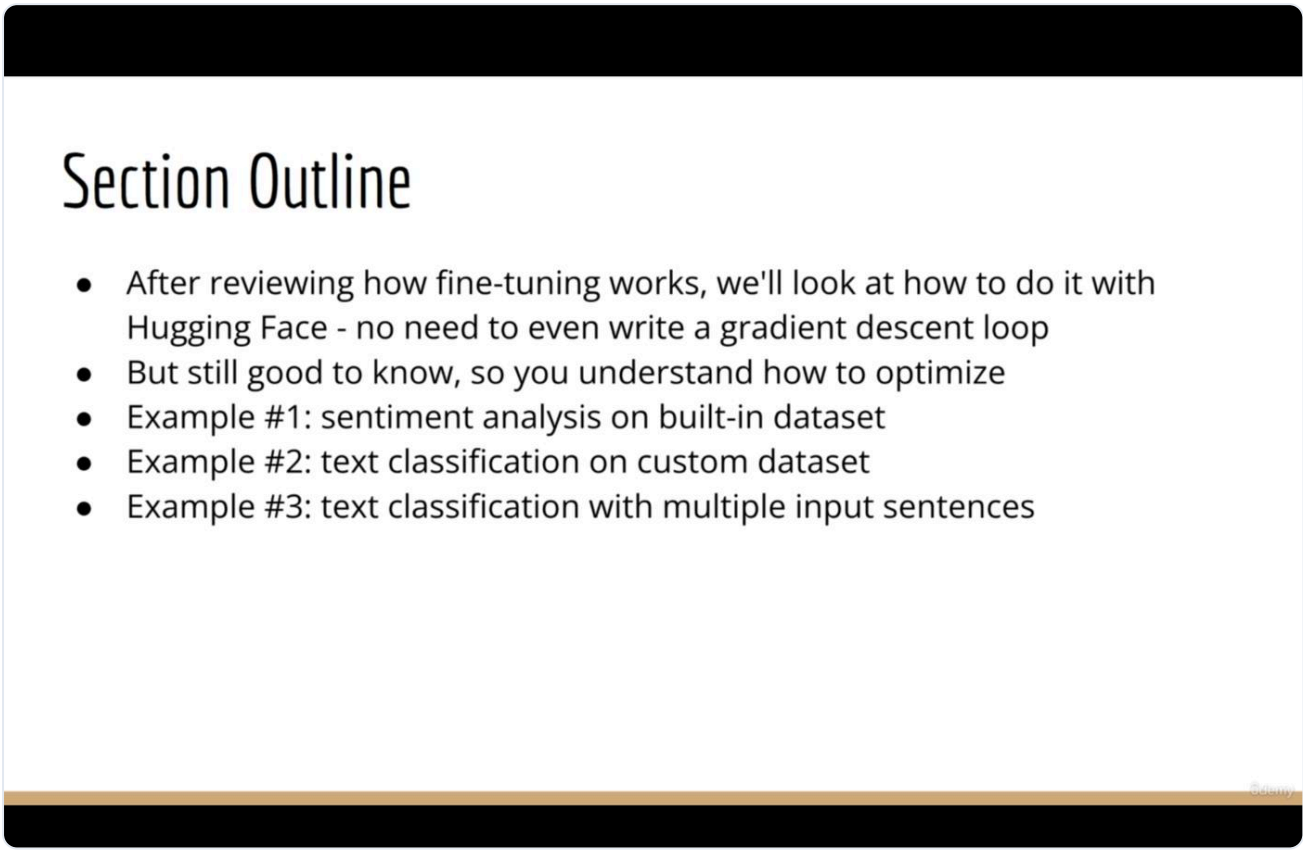

FIGURE 4.7 | Section Outline

After reviewing how fine tuning works. Well, then look at how to accomplish this in Hugging Face. In fact, Hugging Face makes this very easy as well. You don't even really have to know how to write your own gradient descent loop since the library will do all that work for us. Although do note that you might want to have some idea of how it works since the hyper parameters are still free for you to choose.

If you don't want to use the defaults. Now this first example will be a very simple example where we do sentiment analysis on an important benchmark dataset included with the Hugging Face suite of packages. But it's so simple that you might be left wondering, How do I apply this on my own dataset? That's the next thing we'll cover. This will probably be one of the most practical things we do in this course. Since text classification is a very common NLP task in in the real world, you'll most likely have your own dataset, which is not part of the Hugging Face Datasets package.

To end this section. We'll do one more example, which will teach you a bit more about how Transformers work. In this example, we'll look at a classifier that can process not one, but two input sentences at the same time. You'll see how transformers are very flexible in this regard. This

will be the first step in learning how Transformers can handle all kinds of different input formats. Thanks for listening and I'll see you in the next lecture.

2 · Text Preprocessing and Tokenization Review

So in this lecture we are going to review how to preprocess text for use in a neural network. We'll begin by enumerating all the steps that need to be done, and then we'll dig a little deeper into each step and the reasons why they are needed. So the steps are as follows Given a document which is represented as a string of text, we always begin by tokenizing the text. Note that there isn't just one way to do this. This can be as simple as calling string that split, splitting the documents into words, splitting the documents into characters, or as we'll see in this course, splitting the documents into sub words.

The key point is our first task is to split up our text into a sequence of tokens. Once we have our text corpus represented as sequences of tokens, we then need to come up with an integer representation that is, instead of a sentence saying, I like cats.

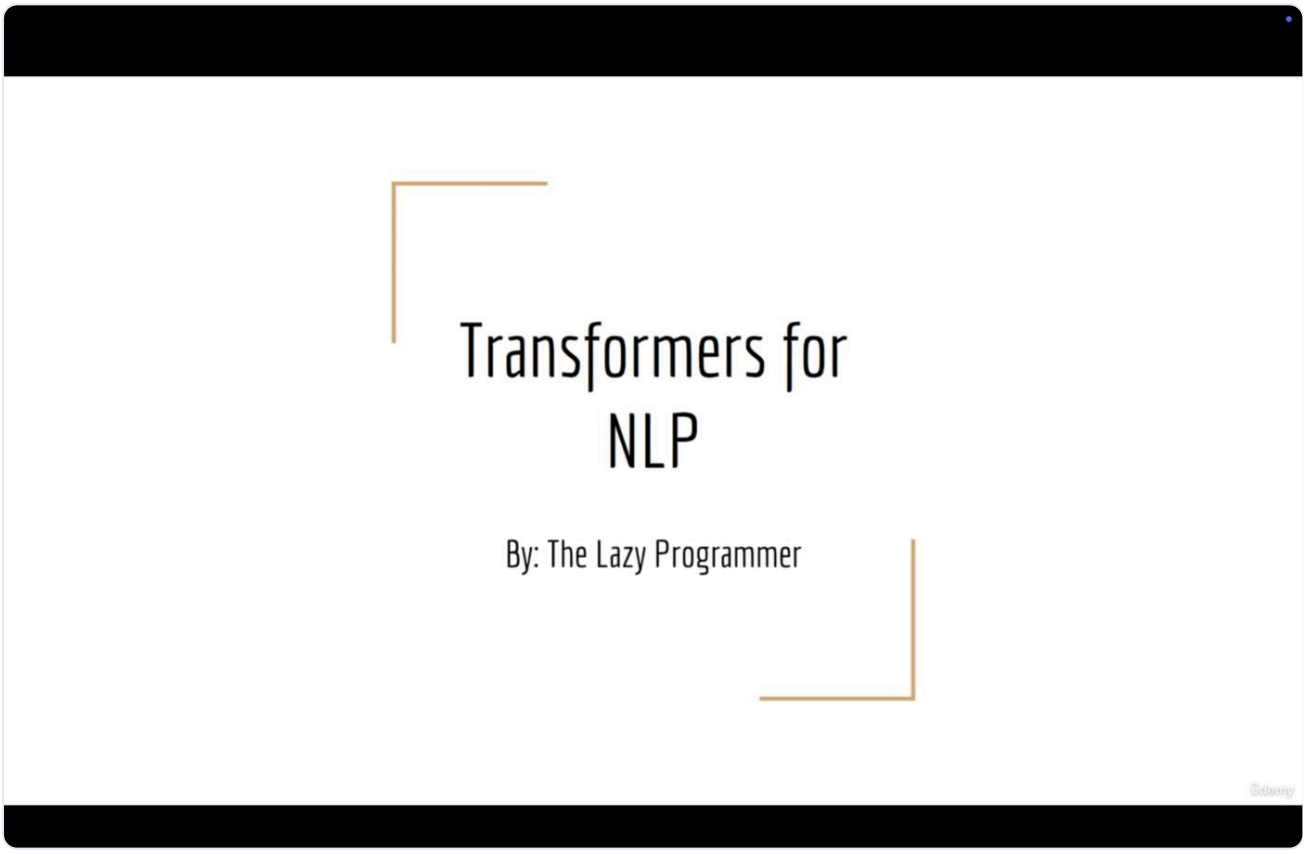

FIGURE 4.8 | Transformers for

Well, turn this into a sequence of token IDs like four, 26, 50 and one or three. One obvious thing we need to figure out is which tokens map to which integers and how to keep track of that information.

Once our data is in the form of sequences of integers, it is theoretically possible to process them using a neural network. However, we often want to process more than one sequence at once. As you recall, in deep learning, we normally process batches of data, for example, when we are do-

ing mini-batch gradient descent. Unfortunately, neural networks are not capable of directly processing sequences of varying length. Instead, what we need to do is pad the sequences so that they all have the same length.

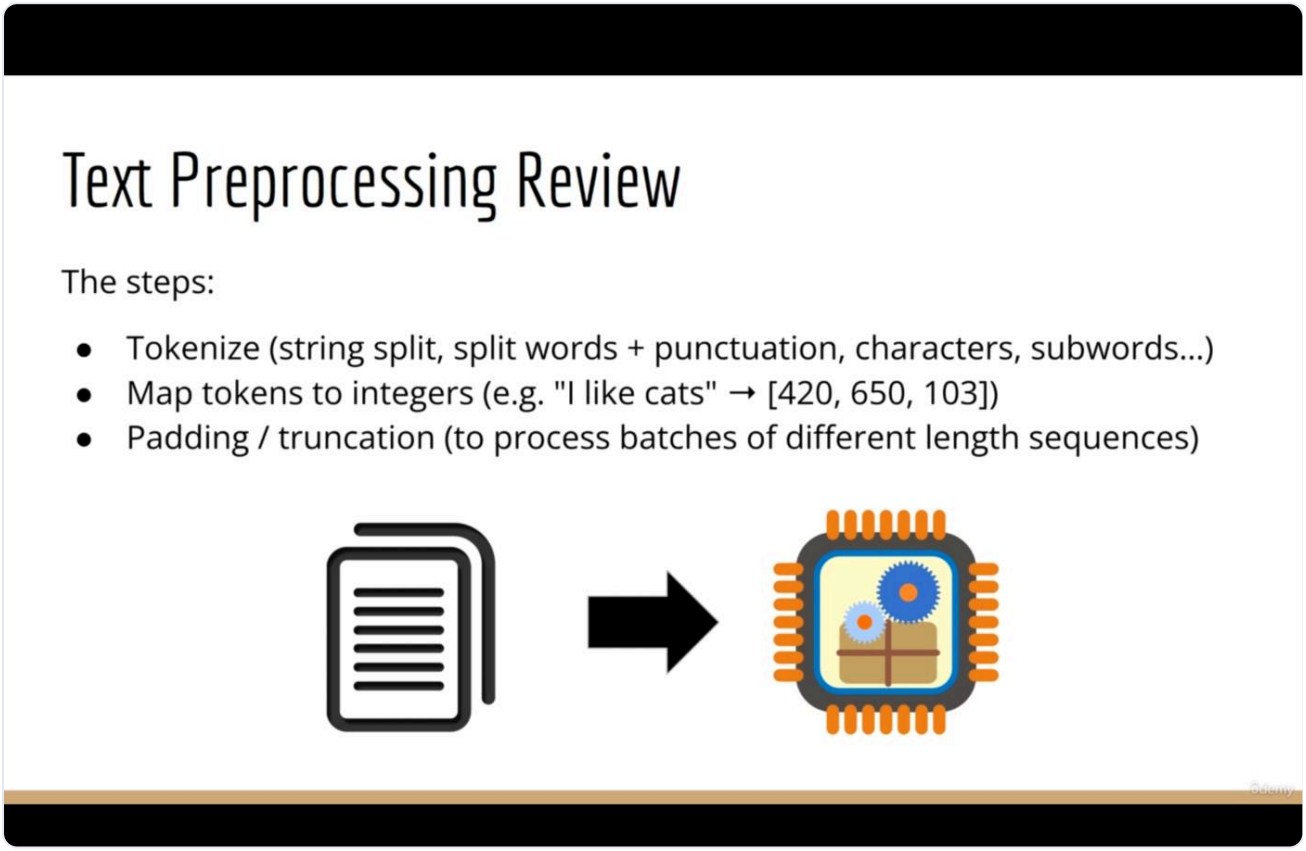

FIGURE 4.9 | Text Preprocessing Review

So we'll have a look at padding and related concepts like truncation. Okay. So now let's take a more in-depth look at each of the steps we just discussed. The first and probably most important step is to tokenize the text. If you've studied other machine learning and deep learning techniques for NLP, you're probably already familiar with tokenizing text into words. In this case, we treat each word as a token. Sometimes this can even be as simple as calling string that split.

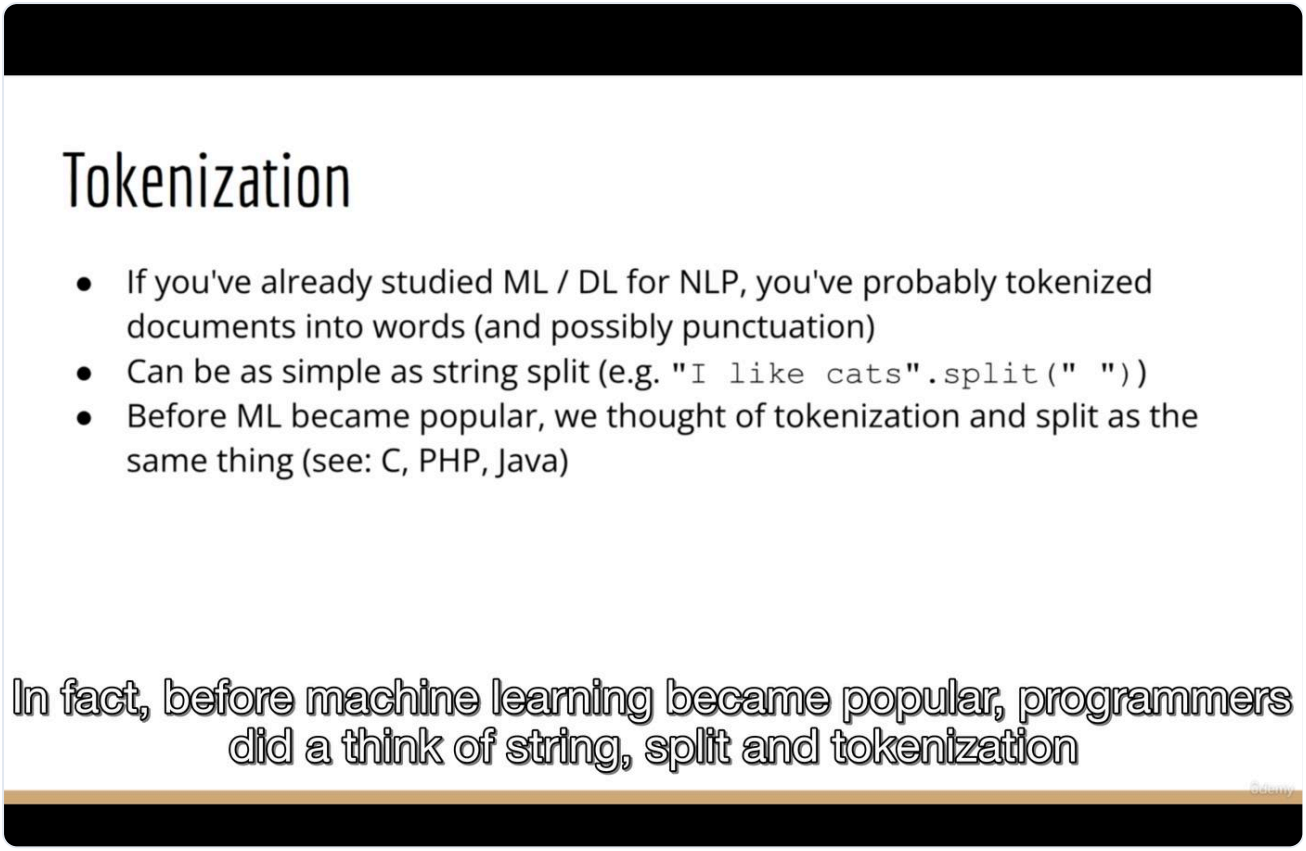

FIGURE 4.10

In fact, before machine learning became popular, programmers did think of string split and tokenization as the same thing.

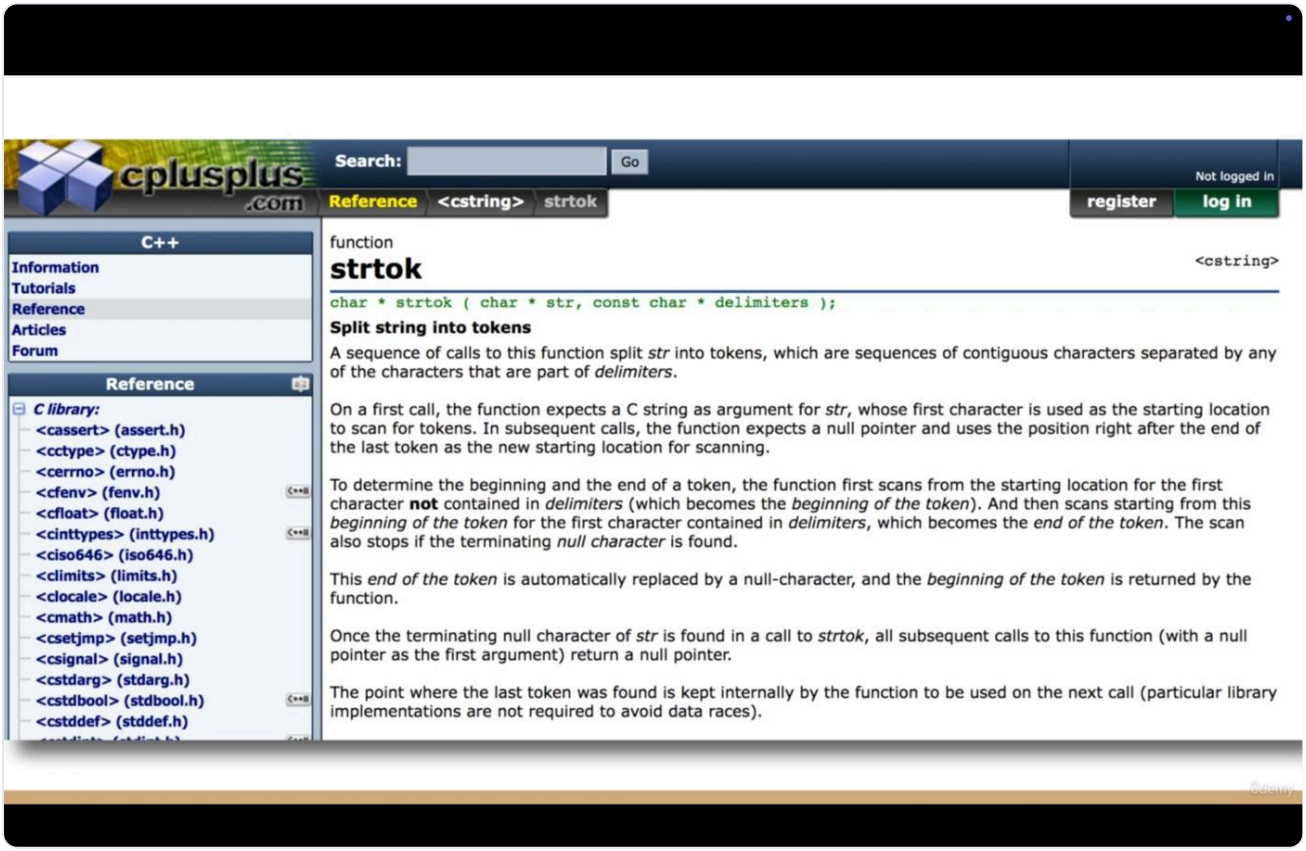

FIGURE 4.11

As an example, check out the strtok function in C and PHP.

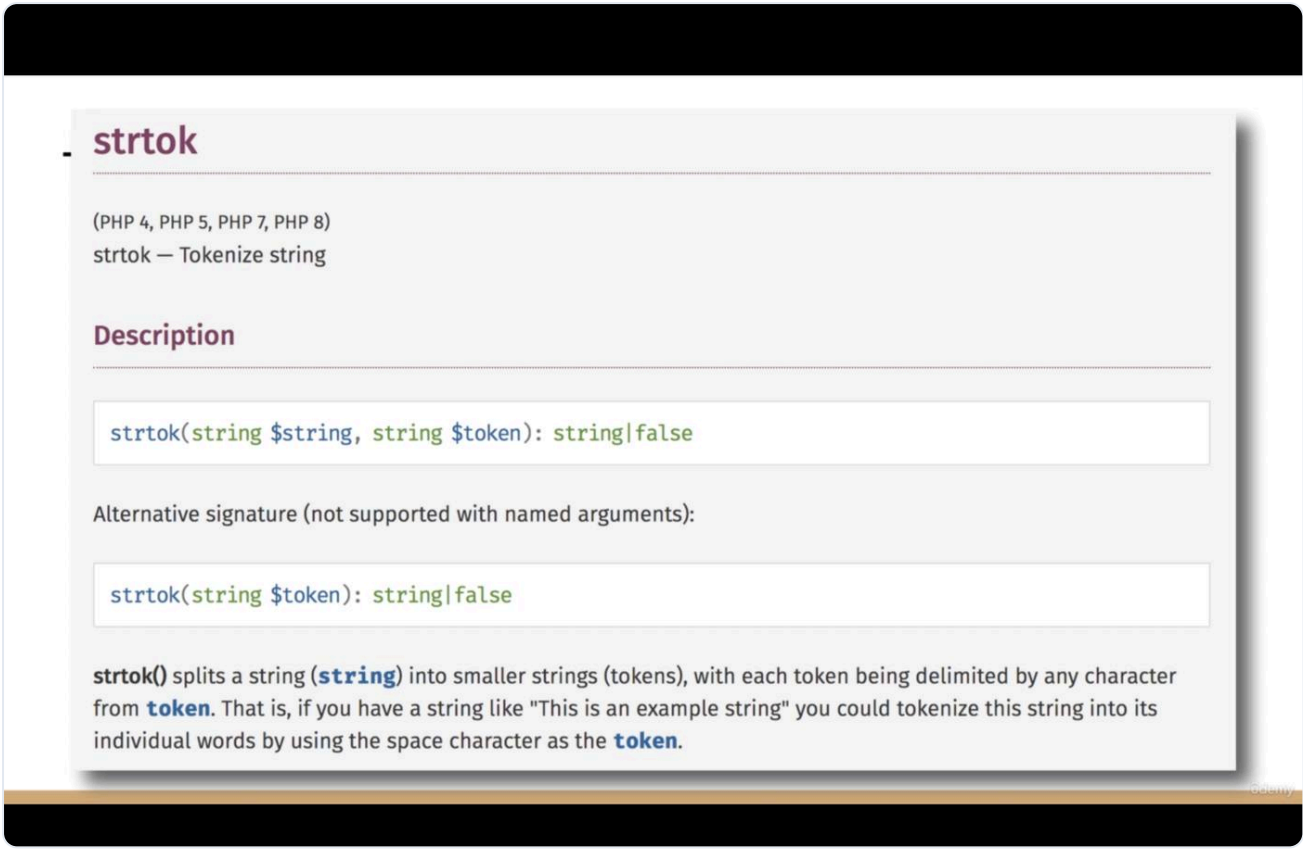

FIGURE 4.12

This function takes in a string and a delimiter as input and will return an array containing the tokens which result from splitting the string by the delimiter.

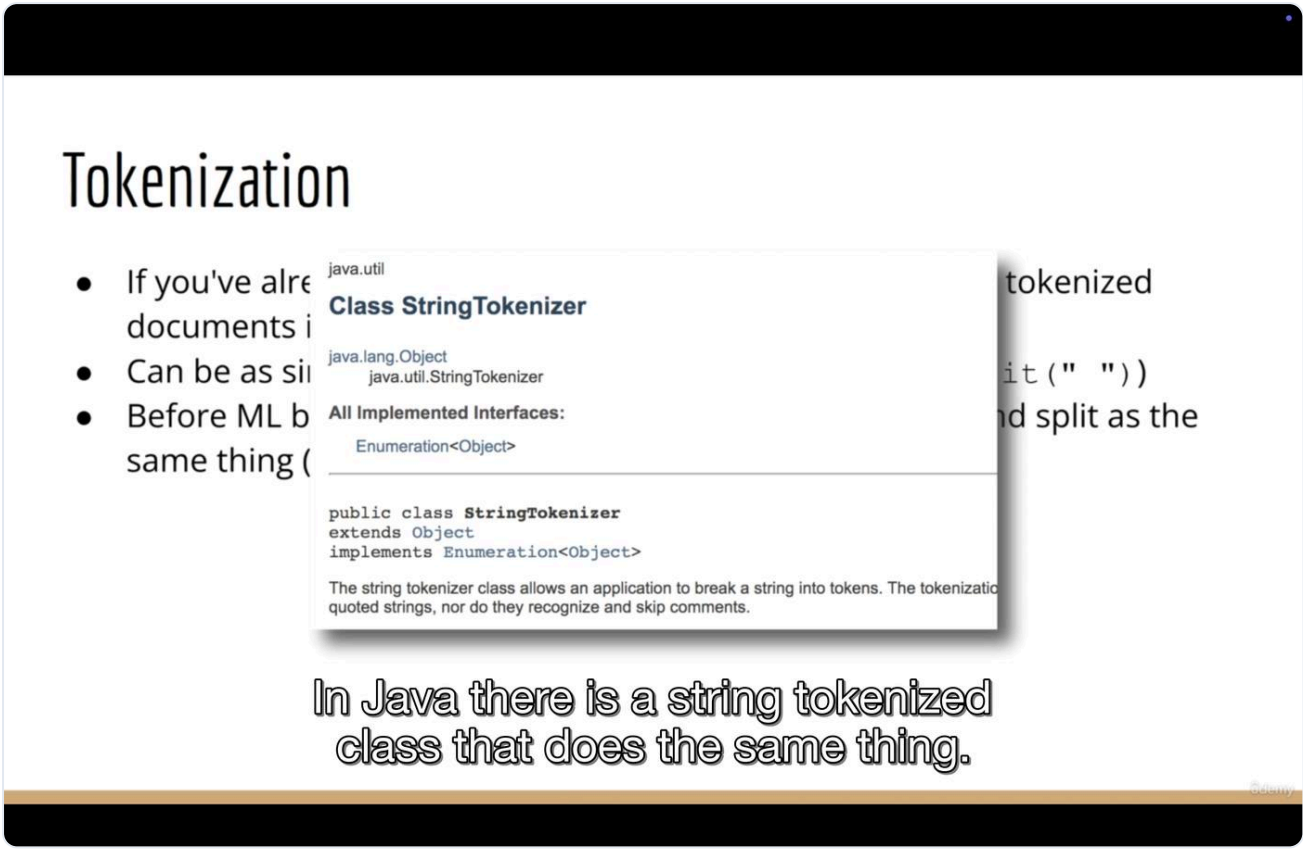

FIGURE 4.13 | Before ML B

In Java there is a StringTokenizer class that does the same thing. So this is all to point out the fact that the vernacular has changed over time.

In the era of machine learning, tokenization now normally means something more than just a string split. Consider, for example, punctuation.

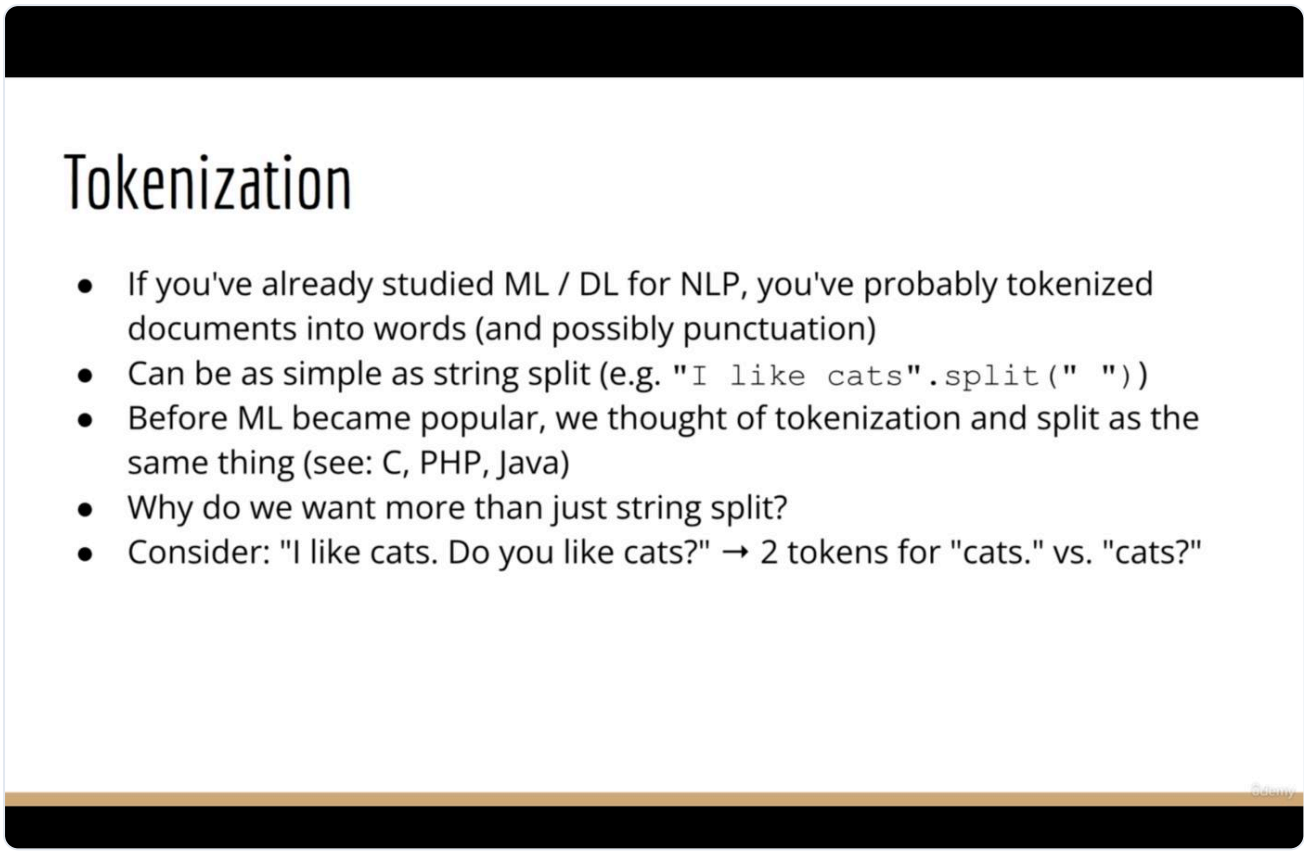

FIGURE 4.14

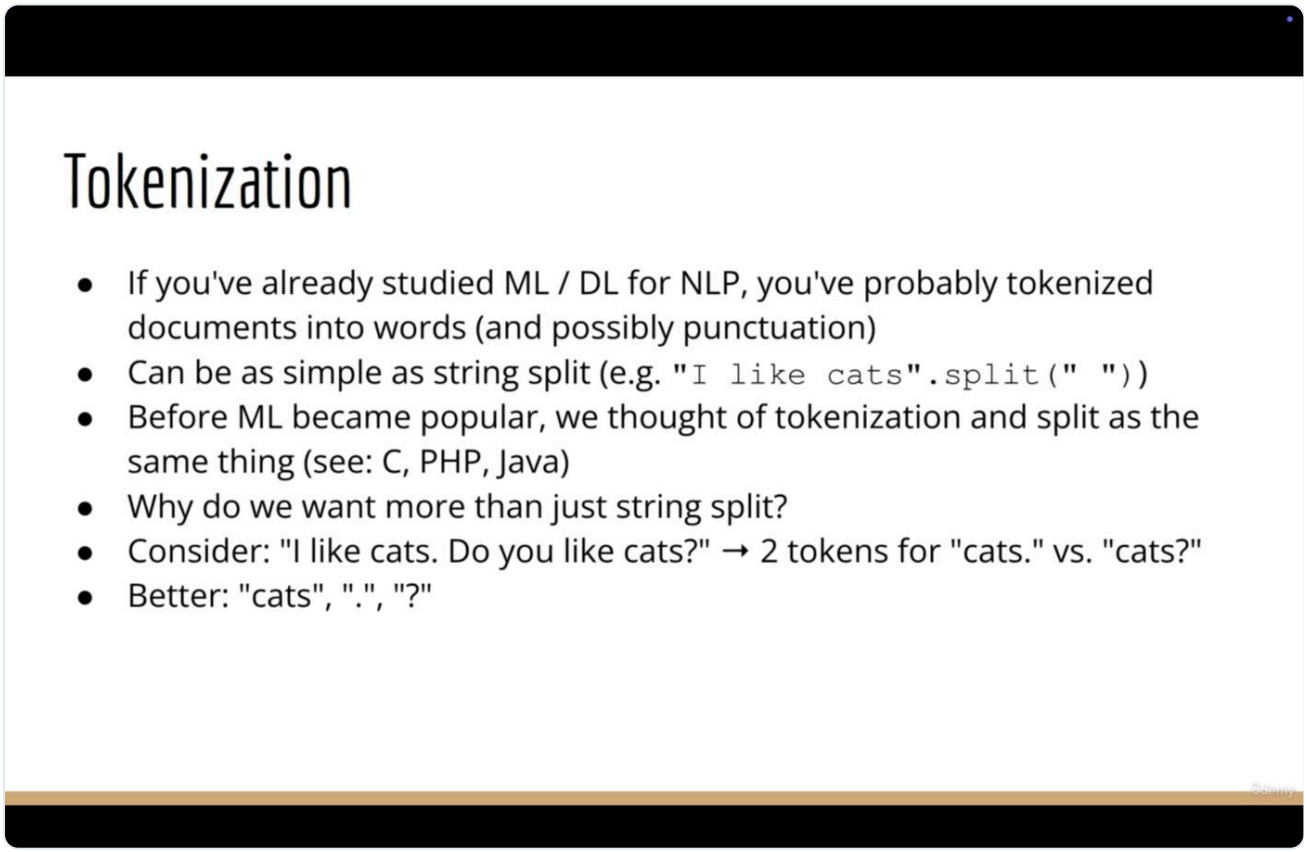

FIGURE 4.15

Let's say you have an input document which says, I like cats. Do you like cats? If we just did a string split, you can see that the first occurrence of the word cats would be joined with a period. The second would be joined with a question mark. In effect, these would be treated like two totally distinct tokens, which is probably not what you want.

Instead, what you probably want to do is to split the punctuation as well, so that the period and question mark are themselves separate tokens. Another option for tokenization you may have seen in the past is character level tokenization using this type of tokenization. You can build a character level language model. One example of that might be building a name generator.

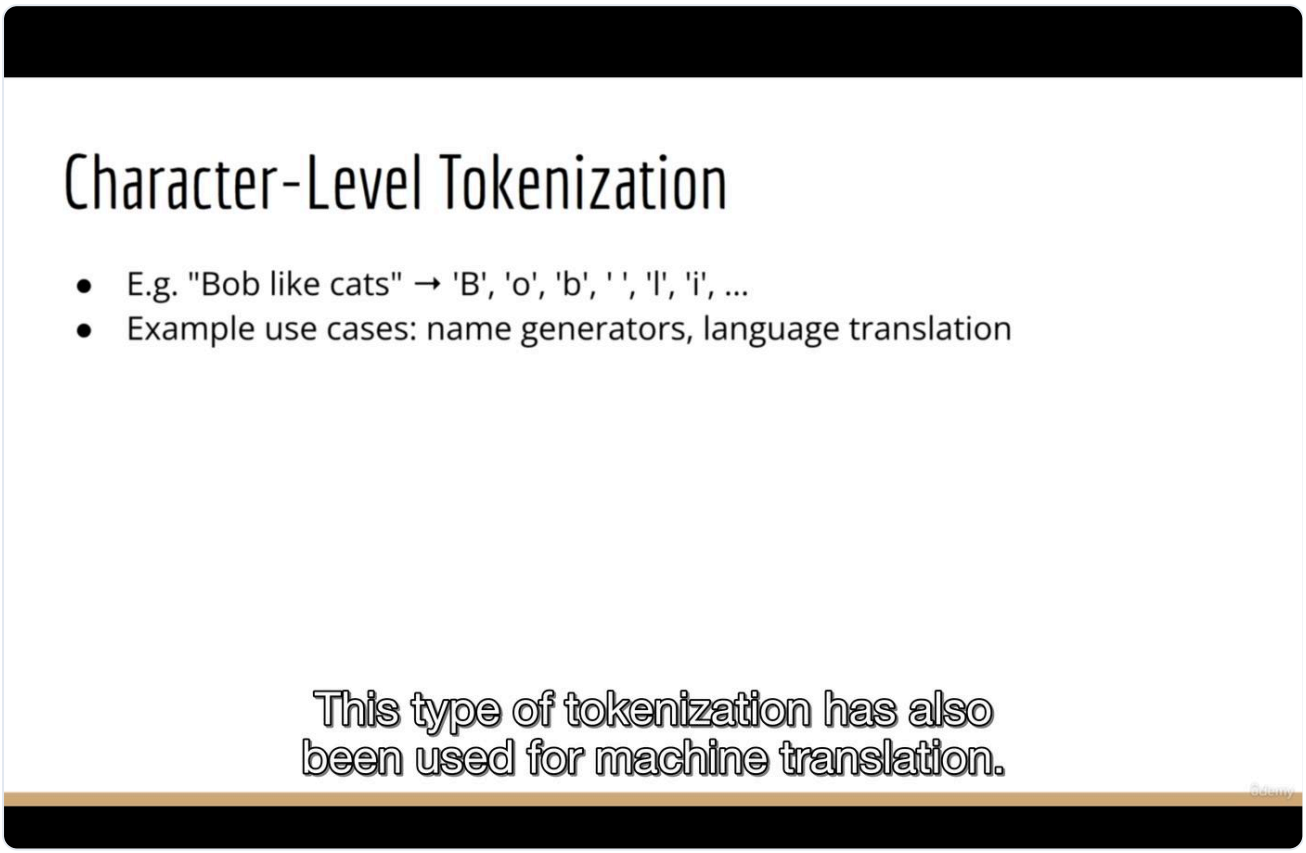

FIGURE 4.16

This type of tokenization has also been used for machine translation. But the type of tokenization we tend to use for transformers is even more complex.

In the era of Transformers, it has been customary to use sub word tokenization. That means we end up sometimes splitting words into multiple tokens. This makes sense. For example, consider the words run running runs and so forth.

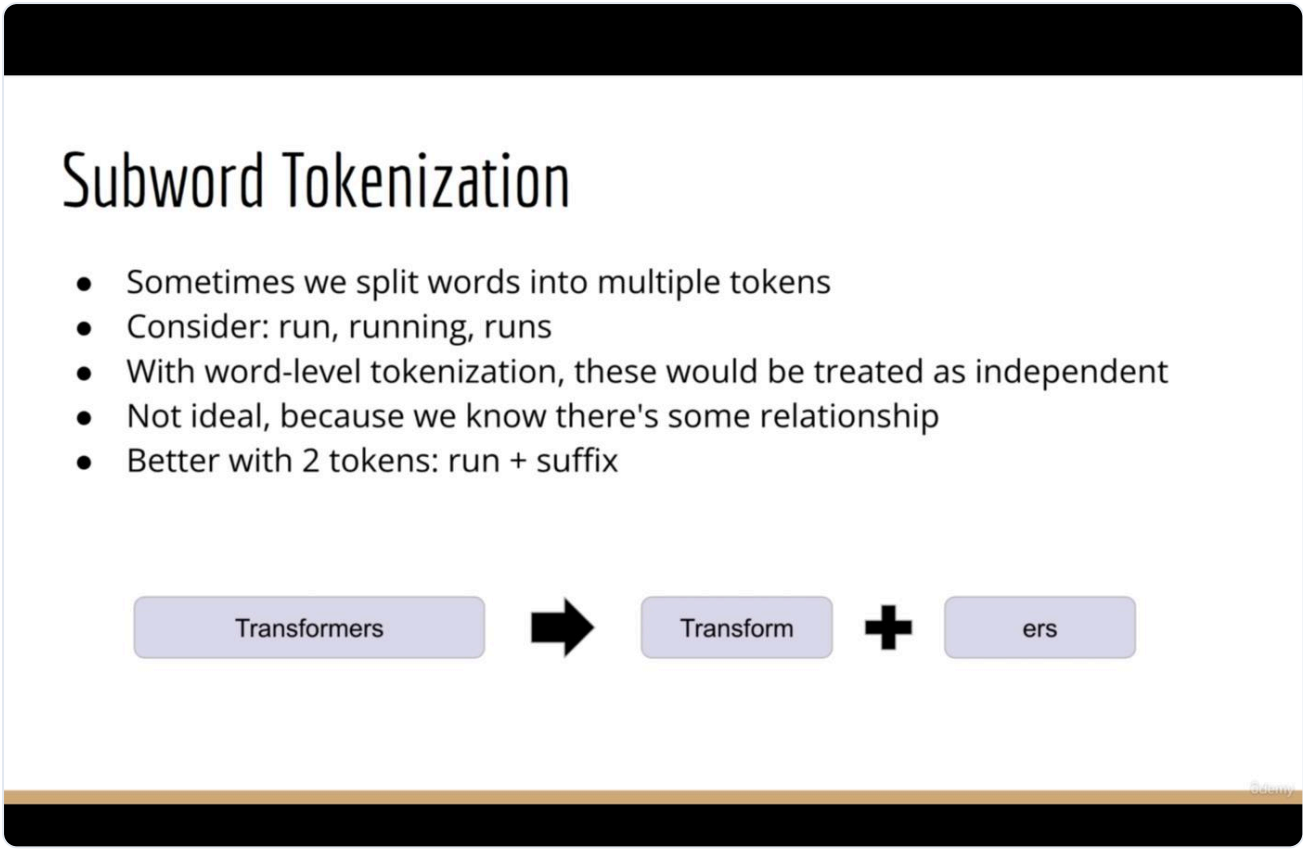

FIGURE 4.17 | Subword Tokenization

If we only did word level tokenization, these would all be treated as separate tokens and we would have to learn their meaning separately from the rest. That's not ideal because we know that all of these words have the same root word, and so there is some relation between them.

But suppose that we did subword tokenization. In this case, we would recognize that they all contain the token run, but each with a different suffix that slightly changes their meaning. Now it's important to note that although we often use sub word tokenization with transformers, they are still separate concepts. So for example, suppose you want to build an RNN model for some NLP task. You can still choose to use subword tokenization for that task, even though you are not using transformers.

In other words, although it may seem like subword tokenization is tied to transformers, this is not the case.

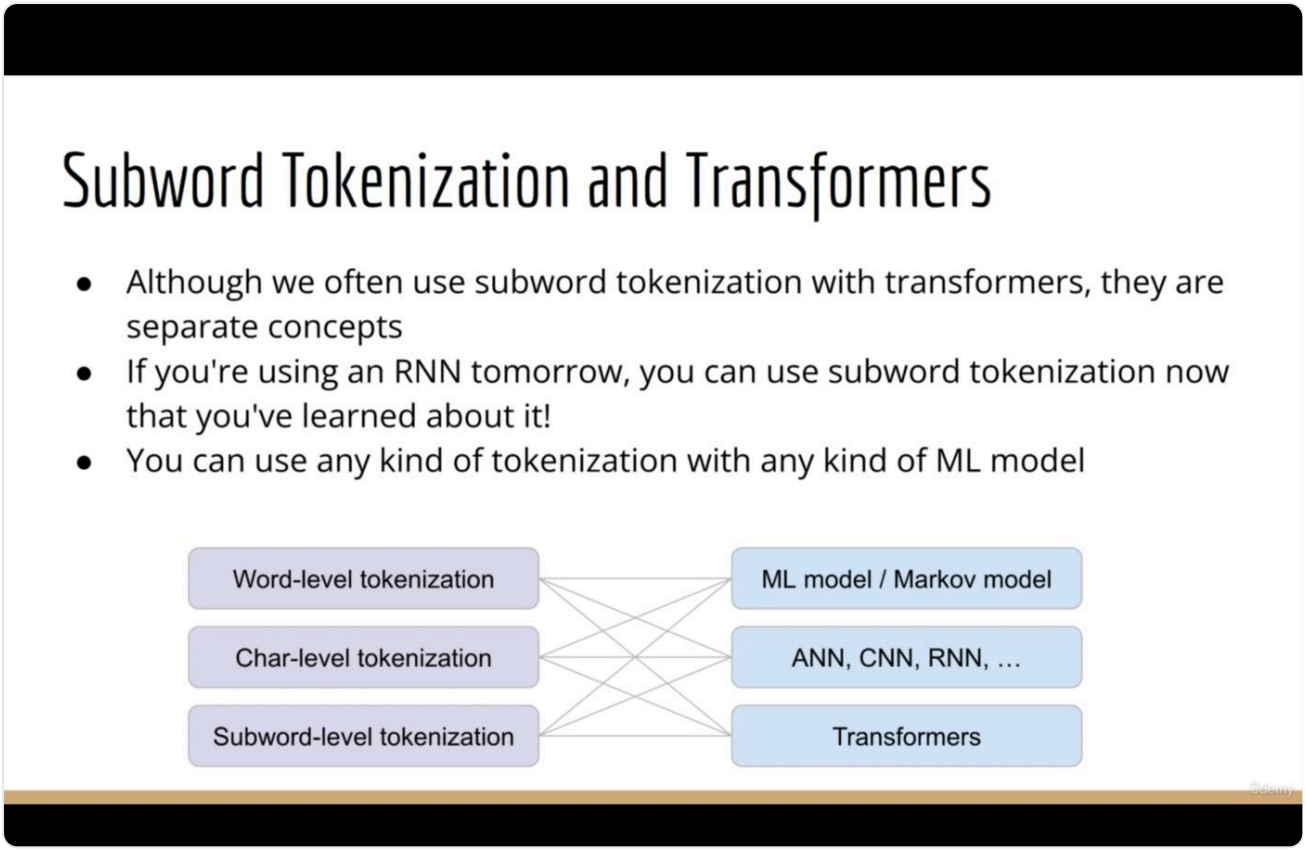

FIGURE 4.18 | Subword Tokenization and Transformers

We can have Transformers with any kind of tokenization and we can use any kind of tokenization with any kind of machine learning model. Another important question to consider when it comes to subword tokenization is how do we choose the subword boundaries? It's possible that we could just have a predefined set of rules like we do in NLTK, for instance.

If we see a word like wouldn't we know that this is actually a contraction of would not. This could be determined just by having a preset list of rules. But when it comes to subword tokenization, we can be smarter. In fact, we can train a model to learn the best subword tokenization based on information-theoretic concepts. As you recall, information theory is the field of study that shows us how to come up with algorithms for compressing data like images, video and text.

These same algorithms can also be applied to find the best way to tokenize text. We'll have more to say about this in the in-depth sections.

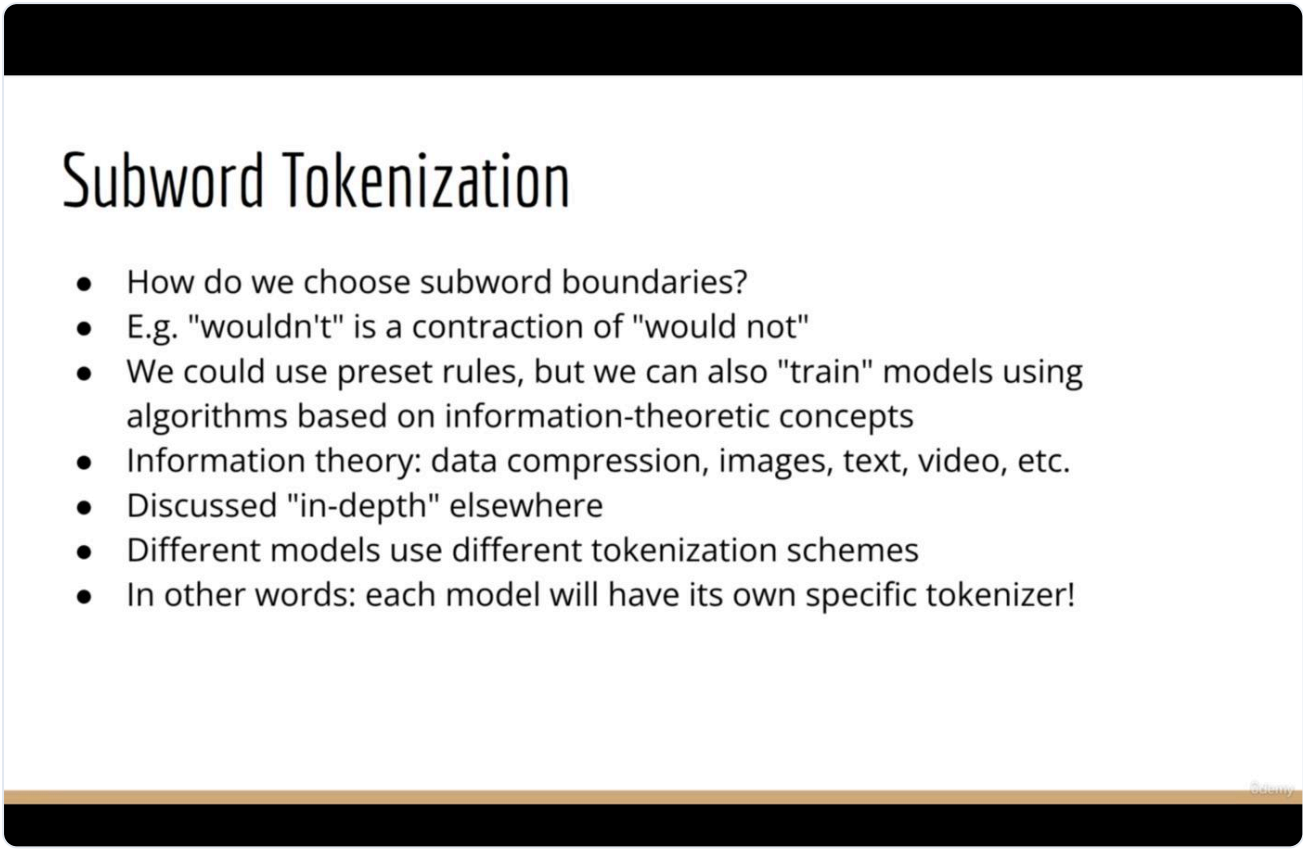

FIGURE 4.19 | Subword Tokenization

Finally, one more important point to consider is that different models use different tokenization schemes because tokenization can essentially be trained on a data set. We can expect that the tokenizer we end up with will not be the same for every model. Since this is the case, we always have to make sure that when we use a specific model, we also use that model's corresponding tokenizer.

Okay. So now we are finally ready to move on to the next step. Now that we have tokens, our next task is to map each token into an integer.

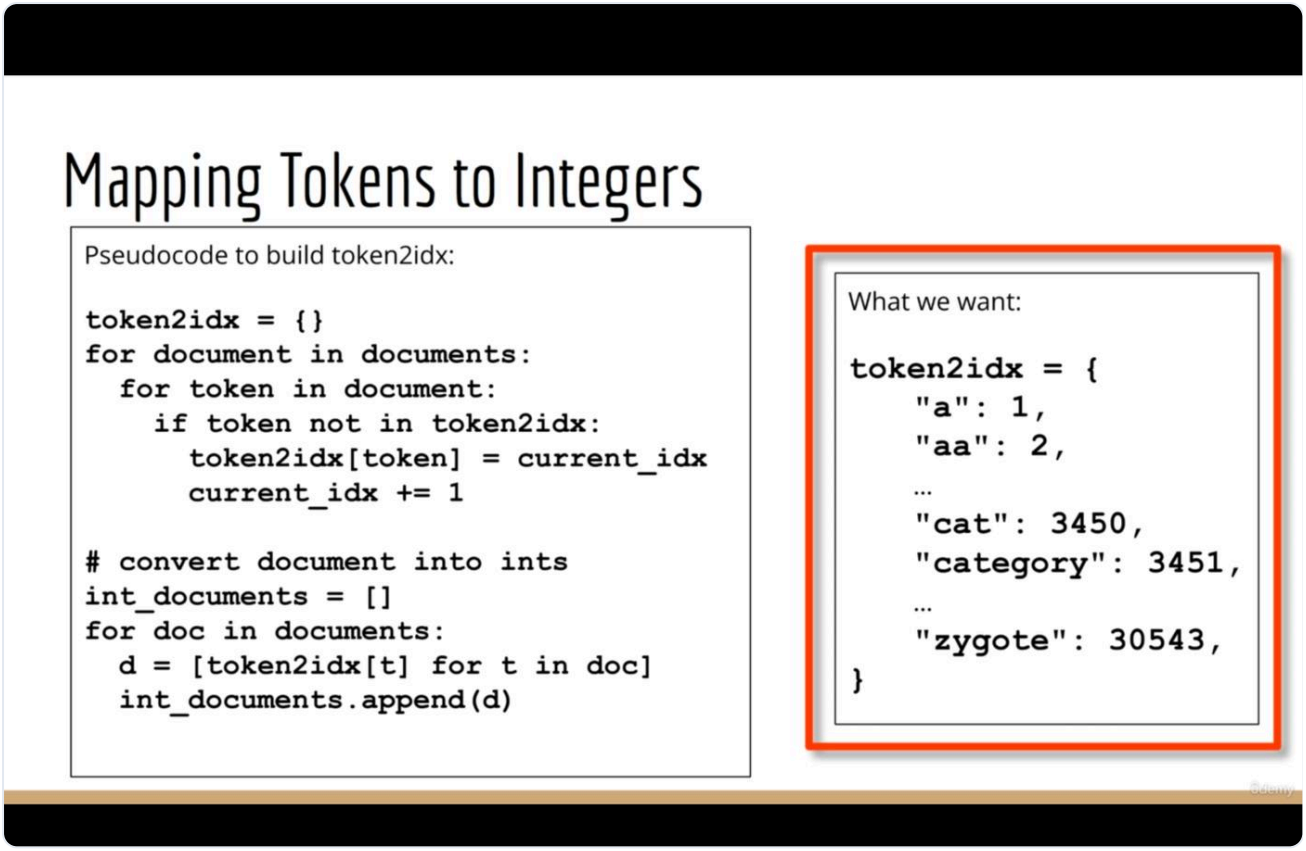

FIGURE 4.20 | Mapping Tokens to Integers

As you recall, in Python, we represent a mapping with a dictionary. We call this a token2index mapping because the key to the dictionary is a token and the value is its corresponding unique integer. On this slide, I've shared with you some pseudocode that demonstrates how one might generate such a mapping, given a data set that has been tokenized.

This is not because you'll need to code this yourself in this course, but because it will give you a better idea of what a token to index mapping is and where it comes from. The algorithm is very simple. All we do is loop through our entire data set of tokens every time we encounter a new token. We assign that a new ID and then we increment the next ID so that every ID is unique.

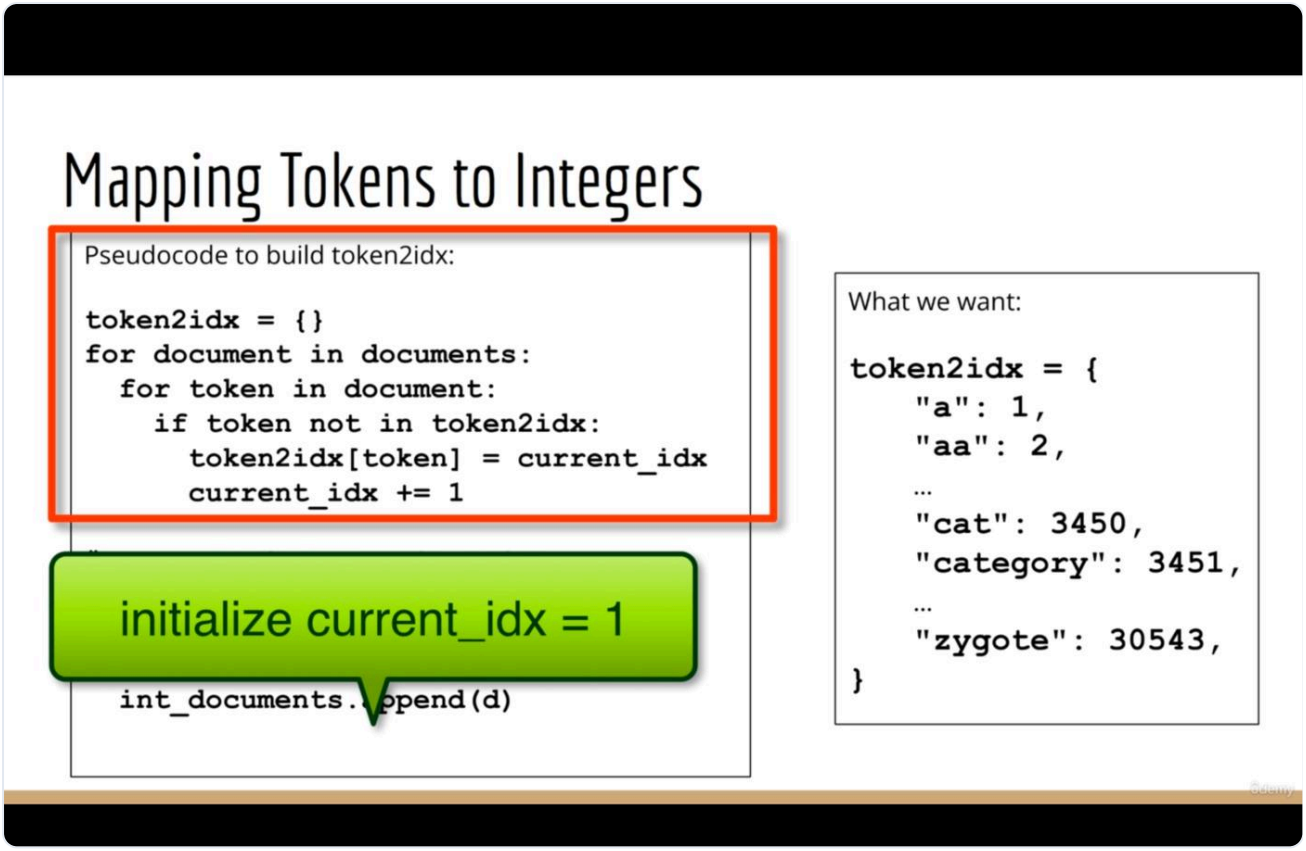

FIGURE 4.21 | Mapping Tokens to Integers

Note that there are actually two steps to this algorithm. The first step is to simply build the token to index mapping, which gives us our dictionary.

The second step is to actually convert our text dataset into sequences of integers. This will be necessary if we want to pass our data into a neural network, because neural networks don't know how to deal with strings, they only process numbers. So the second loop here simply goes through each input document, which is a sequence of tokens, and then uses that mapping to get the corresponding sequence of token IDs. Additionally, we also often make use of the reverse mapping, which is a mapping from index to token.

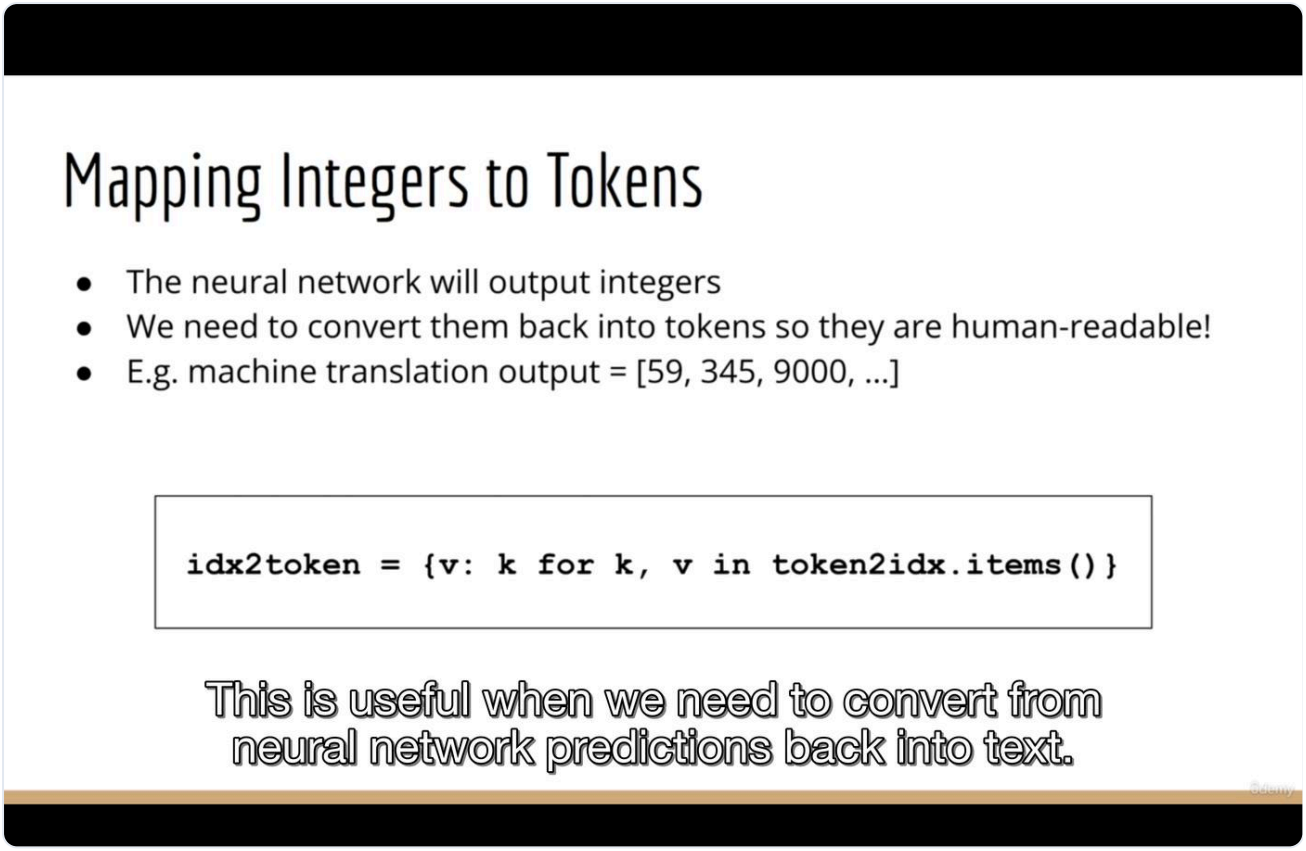

FIGURE 4.22 | Mapping Integers to Tokens

This is useful when we need to convert from neural network predictions back into text. So for example, if our task is machine translation, the neural network will predict a series of token IDs. But in order to represent the sequence of token IDs as an actual sentence we can read. We need to convert them back into tokens and then join these tokens back into full words. The final step we want to discuss in this lecture is the concept of padding. We just converted our input documents into sequences of integers, but of course every document will have a different length.

In deep learning, we often want to process data in batches, which means multiple input documents at the same time.

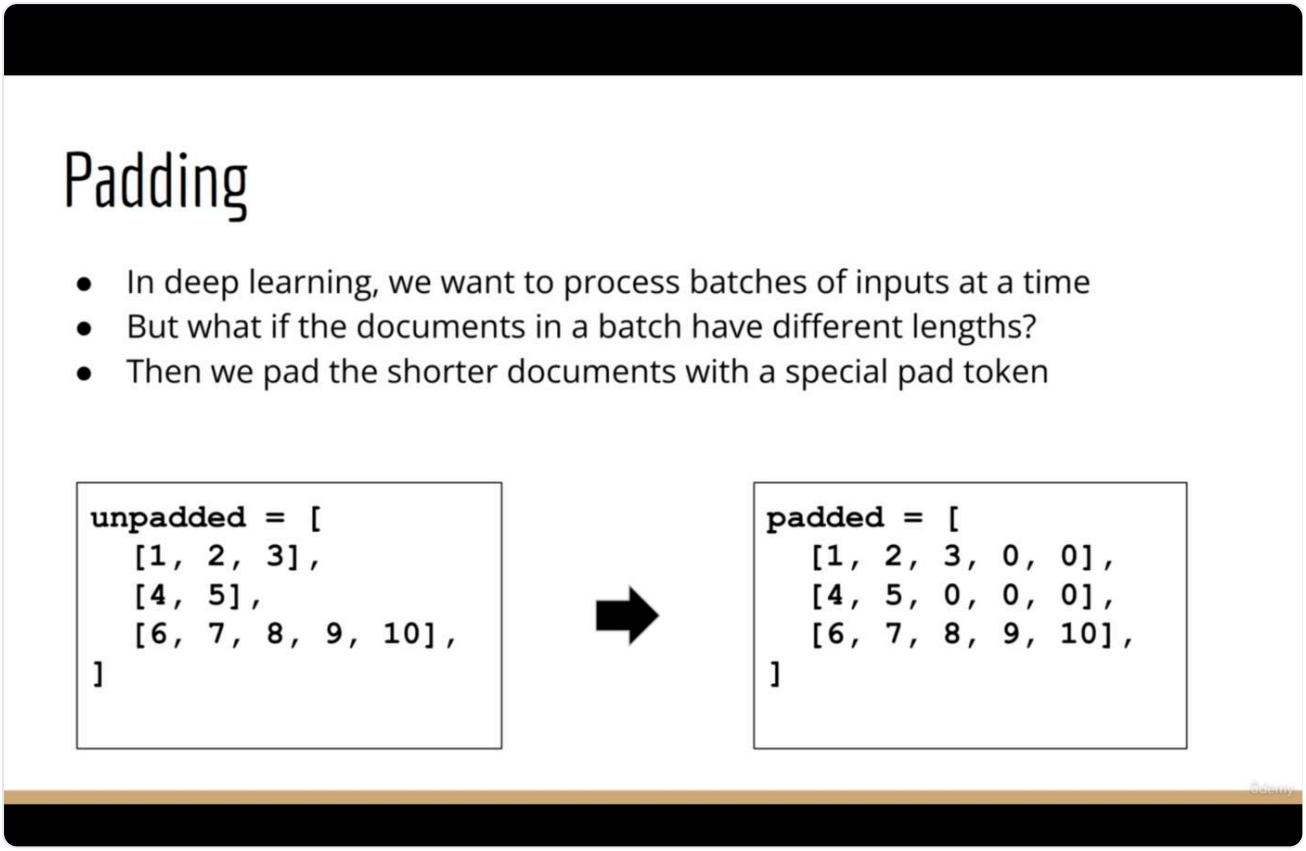

FIGURE 4.23

However, we can't process multiple inputs of different lengths simultaneously. This is because neural networks work with matrices and vectors and other high dimensional objects of constant size in each dimension. So what we need to do is called padding. Essentially, this means that we'll add a special pad token to all of our sequences so that they all have the same length.

Since the pad token is technically a token, it will also have a corresponding integer ID. One thing to note is that different tokenizers will use different IDs for the pad token. So you can't, for instance, assume that it's always zero.

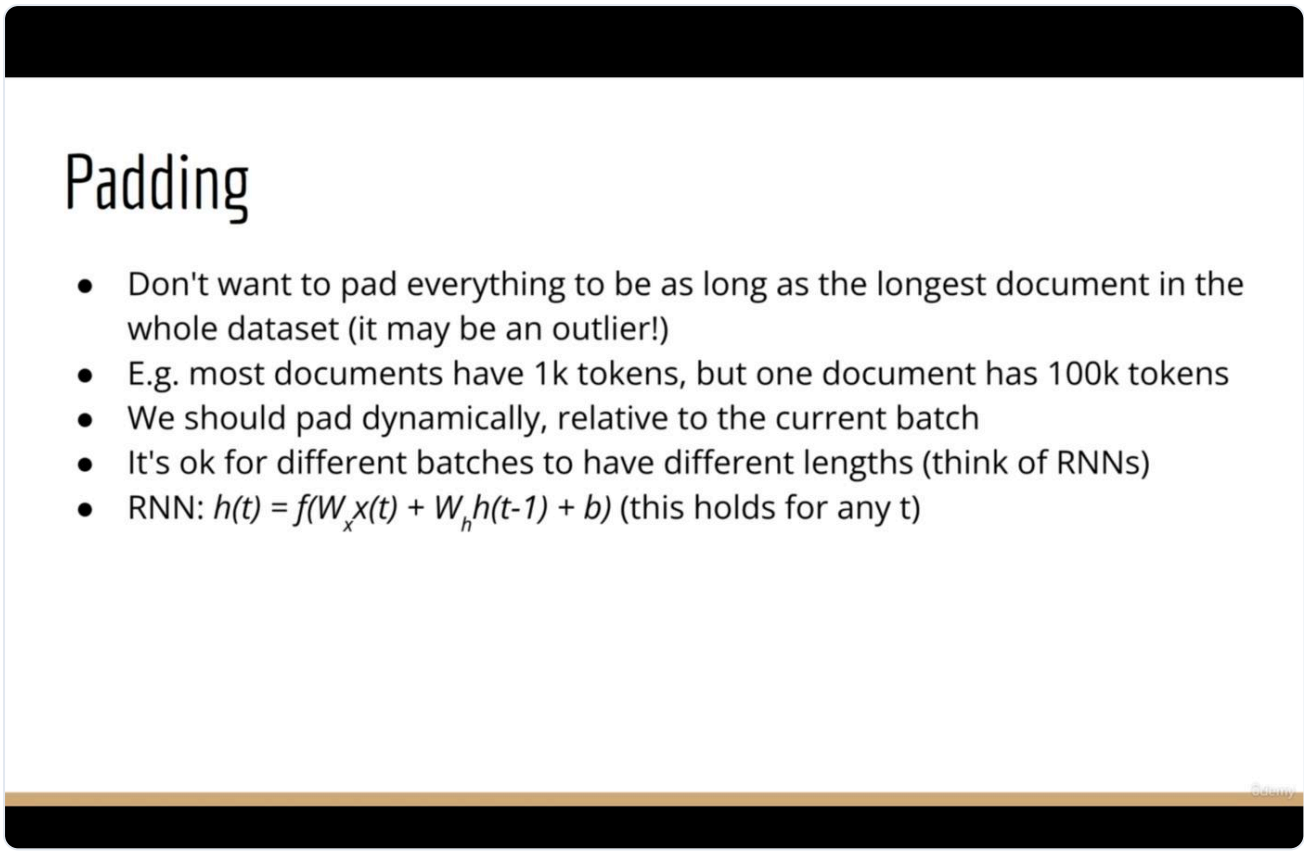

FIGURE 4.24

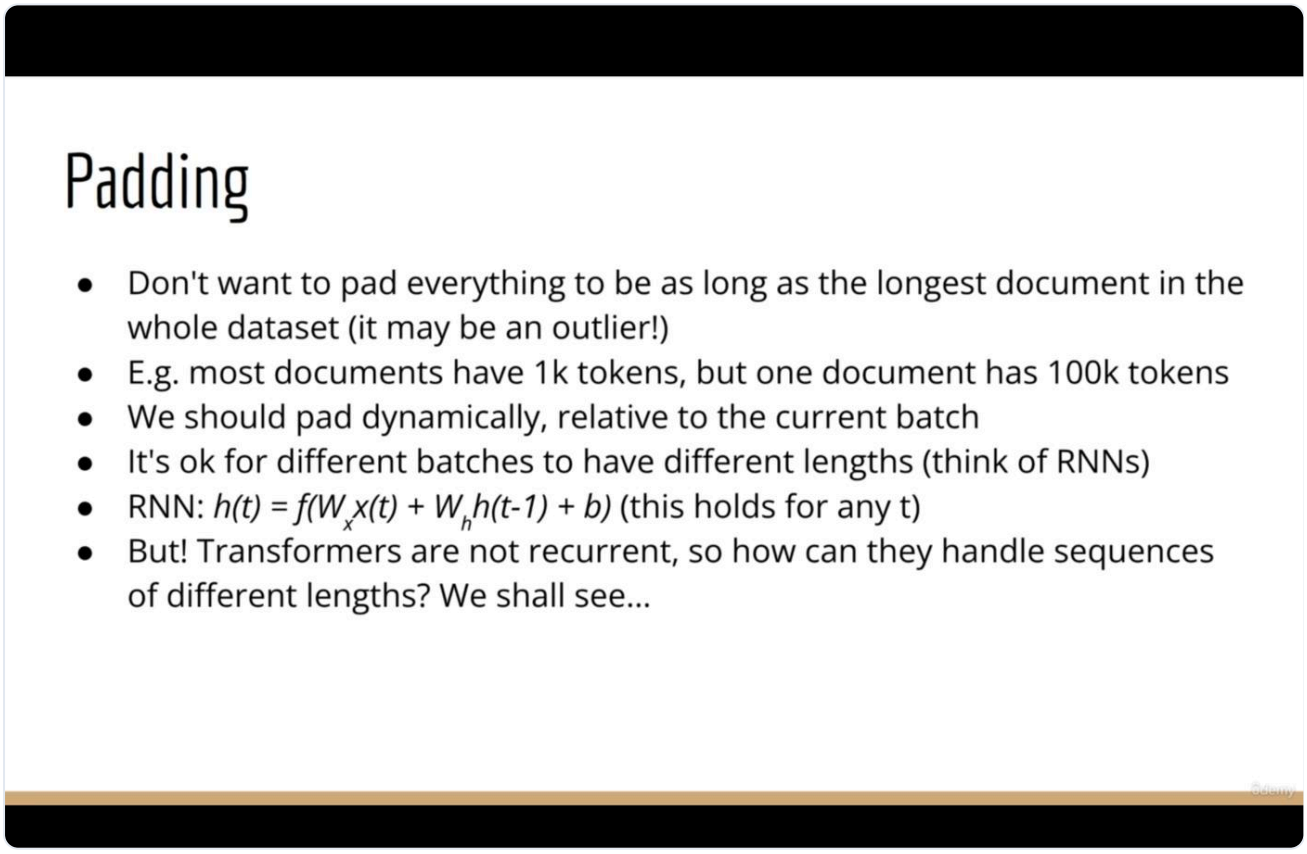

FIGURE 4.25

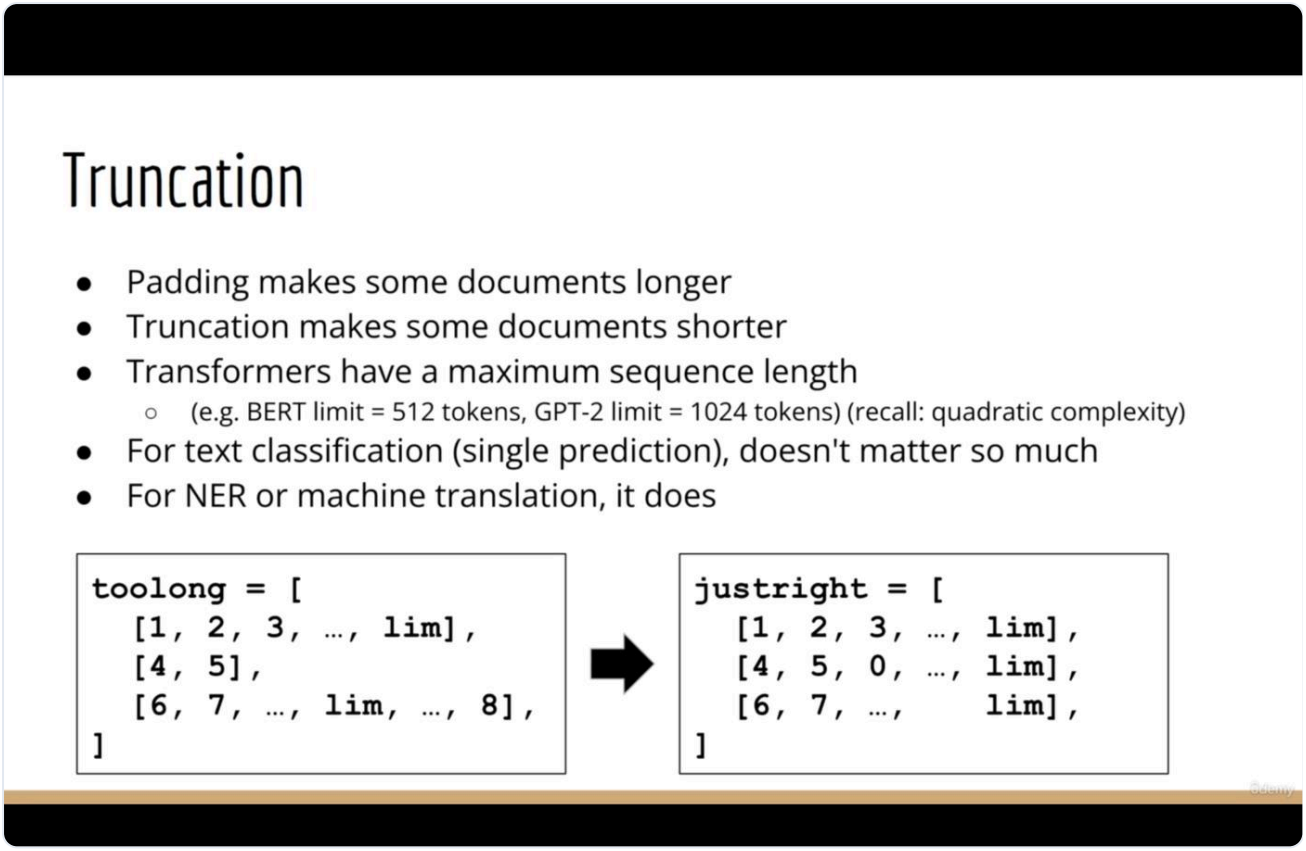

FIGURE 4.26 | Padding Makes Some Documents Longer

So in practice it's important to note that we don't pad all of our input sequences relative to the whole dataset, meaning we don't want to pad everything to be as long as the longest input document. This is because the longest input document may be an outlier in terms of size.

For instance, perhaps is the case that most of your documents are 1000 tokens long, but you have one document that's 100,000 tokens long. You don't want to end up padding all the 1000 token documents to be 100 times their size, since that would be a waste of space in computational effort. Instead, we should pad our sequences dynamically so that we only pad it relative to the current batch. As you recall, this is also the case with RNNs. Because of how RNNs work, they can handle inputs of any sequence length.

As you recall, the equation for an RNN is just W_x times x of t plus W_h times h of t minus one, plus b and this is the case for any T, so t can be arbitrarily large. The only constraint is that if you pass in multiple sequences at once, they must have the same length. Since X of T can represent a whole batch of data. But here's a question to consider. We know that transformers and attention are not recurrent. They do not work like RNNs as such.

How is it even possible for transformers to handle sequences of different lengths? In order to understand this, we'll really need to understand the theory of how self attention and transformers really work in terms of the math. So stay tuned for that. Another concept to keep in mind is the concept of truncation. Truncation is kind of the opposite of padding with padding. We want our documents to be longer with truncation. We want our documents to be shorter.

Unlike with RNNs, transformers do have a limit on the sequence length of the input. For example, BERT has a limit of 512 tokens. GPT-2 has a limit of 1024 tokens. As you recall, attention has quadratic complexity. So the longer your input sentence, the longer it takes to process in a quadratic sense. Therefore, you will quickly reach computational limits if your input sequences are too long. With CNNs, RNNs and transformers. A common technique is to truncate the input sequence.

If it is too long, that means cut off the beginning or cut off the ends so that the result does not exceed the limit. For tests like sentiment analysis and spam detection. This doesn't matter since the prediction is only a single value. The answer can likely be determined by the part that you didn't remove. This is a problem. However, if you're doing a many-to-many task like a named entity recognition or machine translation, if you're doing named entity recognition, you have a prediction for every word.

So if you cut off any words, you won't have predictions for those words. Similarly, for machine translation, if your document consists of multiple sentences and you cut off a few sentences, then those sentences won't be translated. One solution is to simply break up those inputs into multiple inputs and then translate them in parts.

3 · Models and Tokenizers

So in this lecture we will begin looking at concepts specific to the Hugging Face library, specifically what components make up the pipeline and how that's related to the text preprocessing steps we just discussed. Essentially, we can picture the pipeline as containing three distinct steps. The first step is the text preprocessing. This takes in raw text and converts it into a numerical format that can be handled by a neural network. The second step is the model.

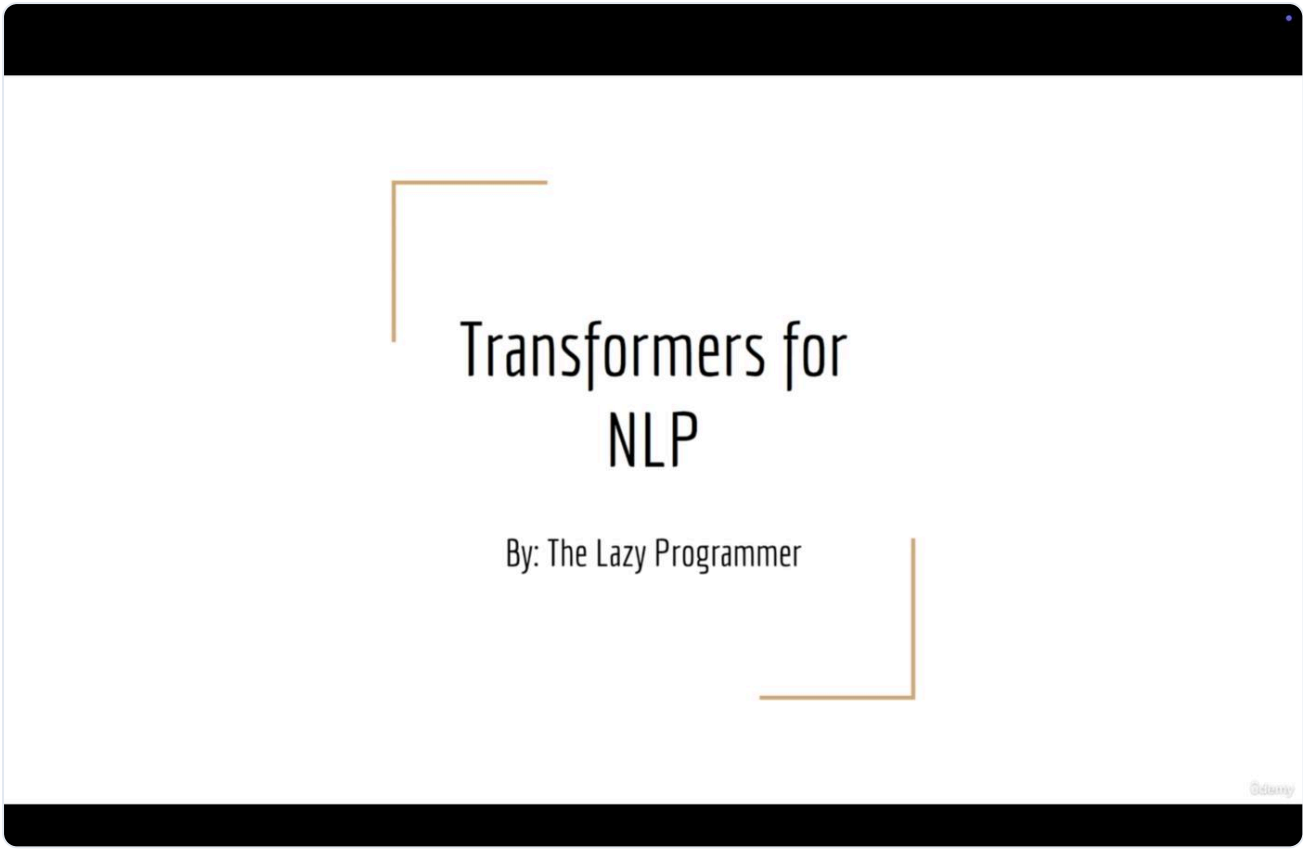

FIGURE 4.27 | Transformers for

This is the actual neural network that crunches the numbers. The third step is post-processing. This takes the numbers output by the neural network and converts them back into a human readable format. In this section, the will mainly be focused on the first two steps. The third step is not really handled by a specific thing, since it will be dependent on the kind of task.

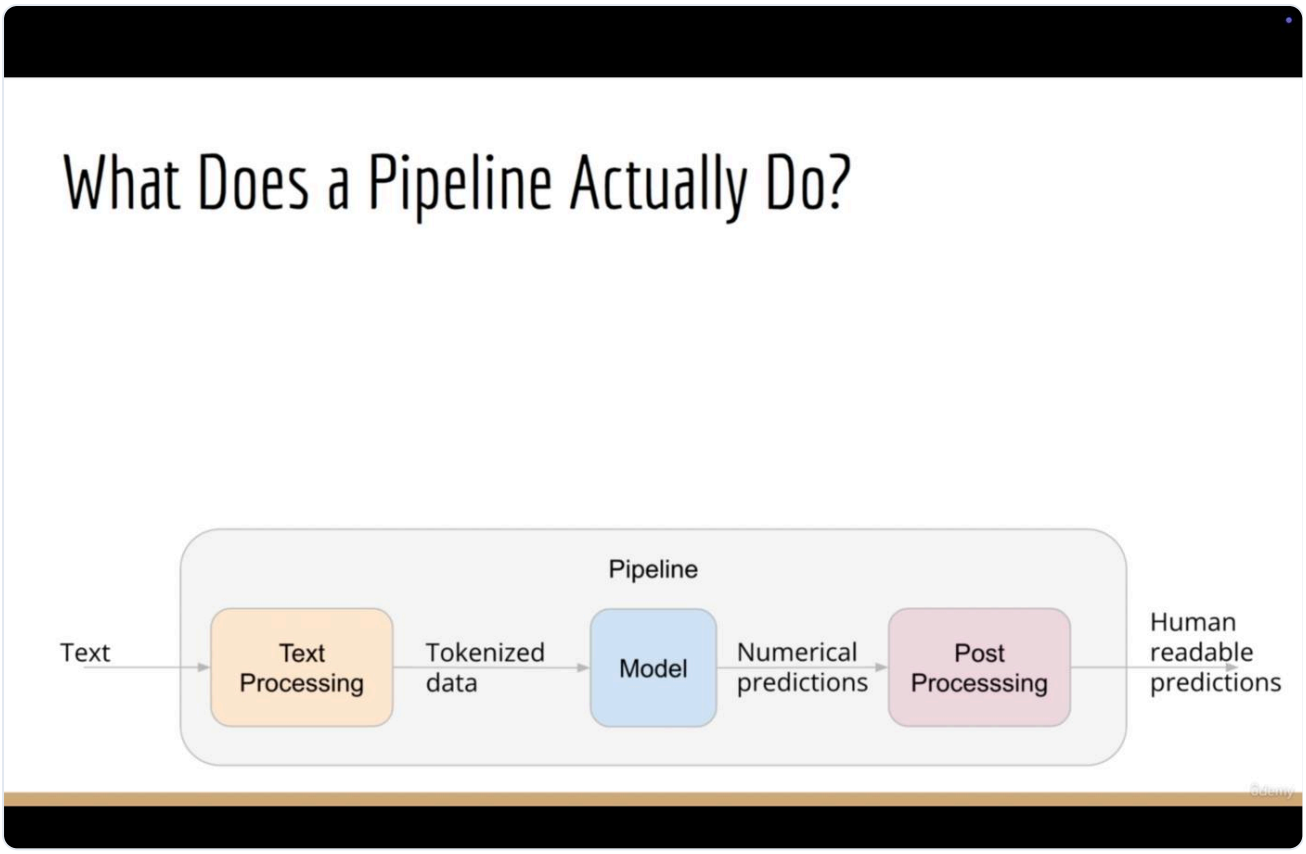

FIGURE 4.28 | What Does a Pipeline Actually Do?

For instance, if it's just classification, then a prediction will be an integer class ID and we just need to map that back into a class label.

If it's machine translation, then our prediction will be a sequence of token IDs which will need to convert back into tokens and then back into a full document. Luckily, that can be done by the tokenizer. One hard example is named entity recognition, since entities correspond to full words well for transformers, tokens can be sub words. In this scenario you'll have to write the code yourself.

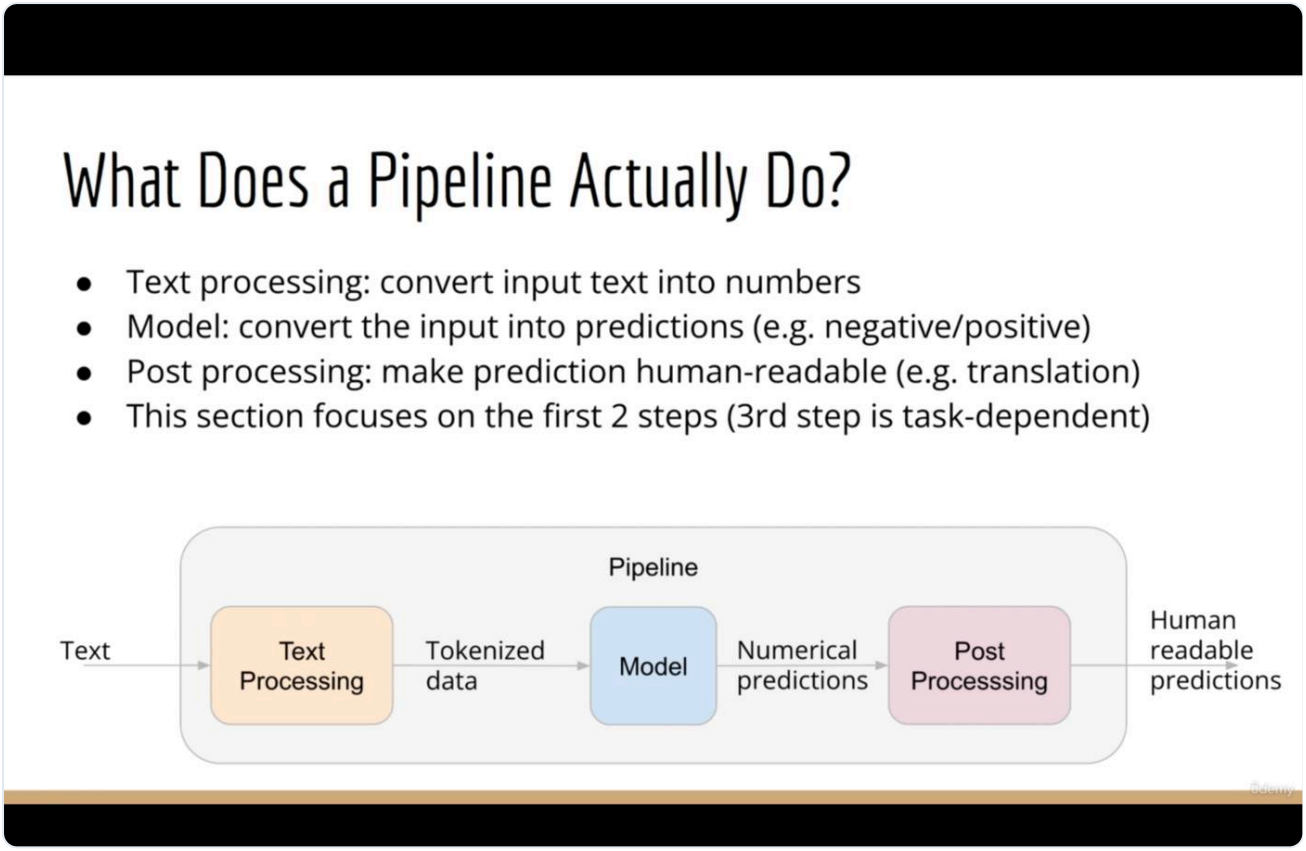

FIGURE 4.29 | What Does a Pipeline Actually Do?

So that will be outside the scope of this lecture and we'll just deal with any issues as they arise.

The first step, which is text preprocessing, is done by what we call a tokenizer in Hugging Face. You may have seen similar examples of tokenizers in other libraries, such as Keras. The reason why this might be a bit confusing is because while this is called a tokenizer, it does not only do tokenization. It actually does all the text preprocessing steps we previously discussed.

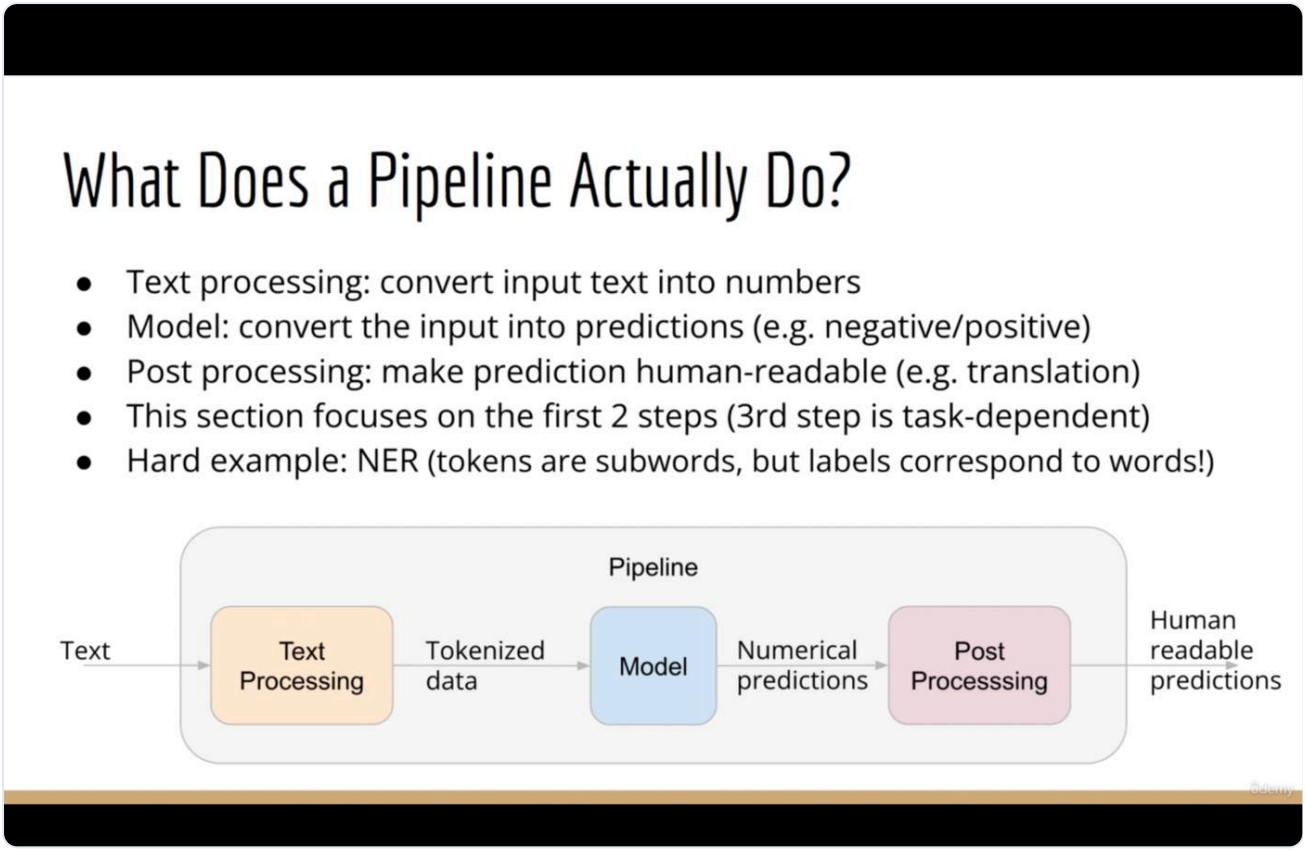

FIGURE 4.30 | What Does a Pipeline Actually Do?

But in Hugging Face, we call them tokenizers. As you recall, there are different tokenizers for every kind of model, so there's a BERT tokenizer, a DistilBERT tokenizer, a GPT-2 tokenizer and so forth.

However, we'll see that there is a nice universal interface to all of these tokenizers that will let us make our code more generic. The second step is the model. The model is essentially the neural network, which is just like the model object you've worked with in TensorFlow and PyTorch in the past.

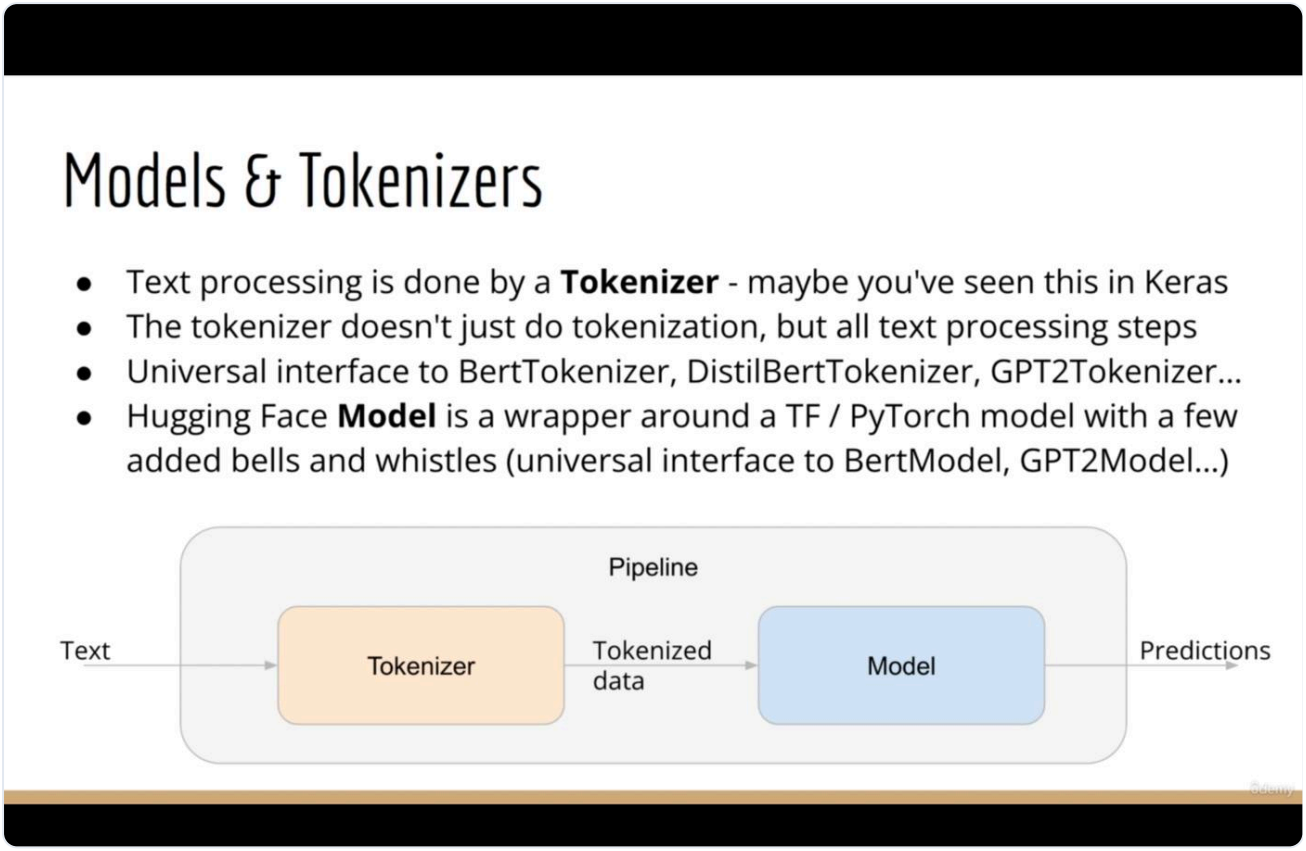

FIGURE 4.31 | Models & Tokenizers

The Hugging Face model is basically a wrapper around either a PyTorch or TensorFlow model, which adds a few bells and whistles on top of what they already do.

Like the tokenizer, there is a universal interface to all types of models, so you never have to explicitly specify the BERT model that should be easy to model and so forth unless you really want to. So now that you understand the main steps, let's see what the code actually looks like. We'll begin by looking at the tokenizer. Now, as mentioned, there are multiple tokenizers that are model specific. But there is a universal interface so that you don't have to worry about picking the right class.

Specifically, there was a class called AutoTokenizer. This class has a method called from_pretrained, where you can pass in a model checkpoints just like we did with pipelines. This will automatically give you back the correct tokenized objects. So for example, if your model checkpoint is based on BERT, you will get back a tokenized object that has all the right stuff in it.

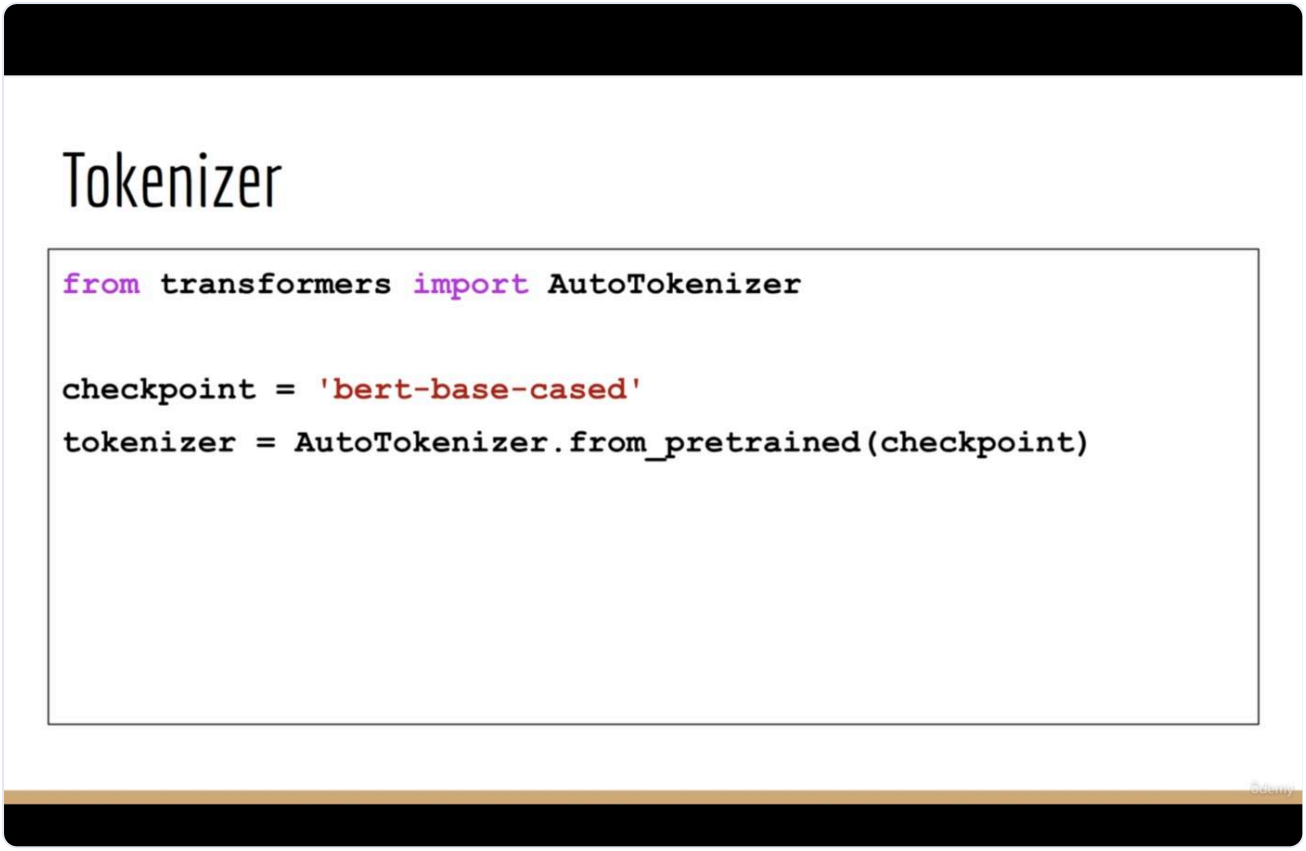

FIGURE 4.32

Okay. So once you have your tokenizer, how do you use it? Well, it's actually just as simple as using a pipeline.

You can pass in your text or list of multiple texts, and out comes the model inputs formatted specially for the corresponding model. Note that this is a dictionary with several keys. The key input IDs corresponds to the integer IDs of the tokens that you would have gotten back from tokenizing your input. Note that this key will always be present because you will always want to convert your text into token IDs. The other keys are attention, mask and token type IDs.

We'll discuss what these mean later on in the course, but note that these keys can be specific to the type of model.

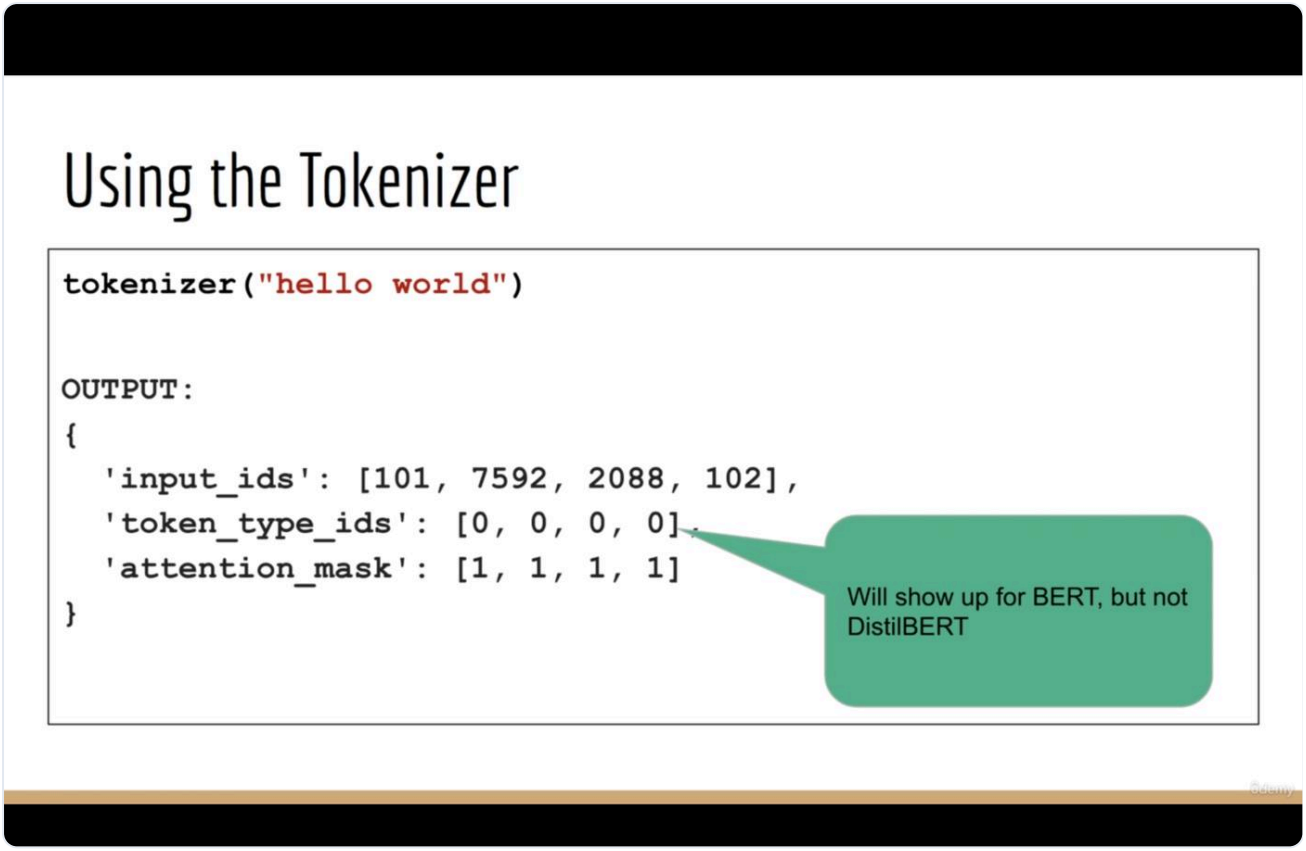

FIGURE 4.33 | Using the Tokenizer

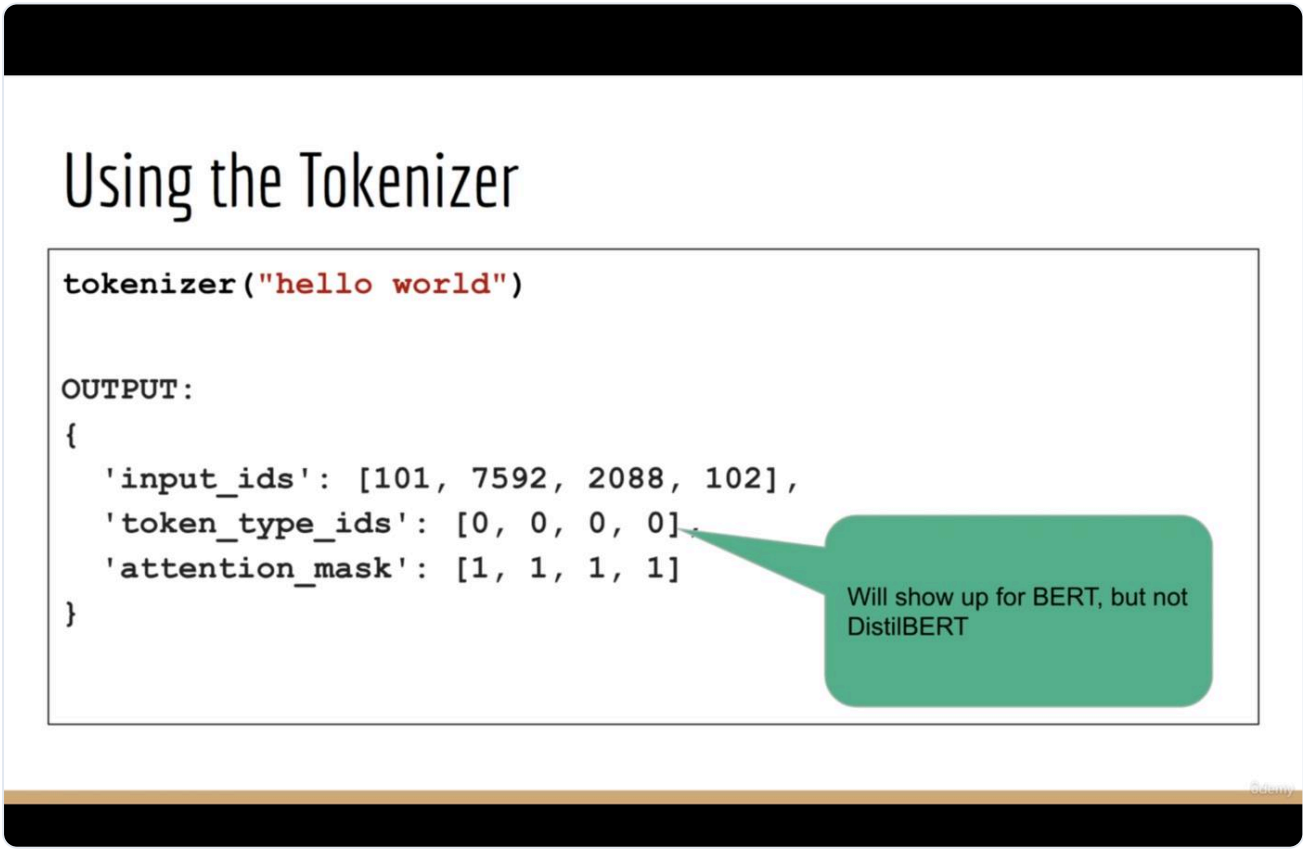

FIGURE 4.34 | Using the Tokenizer

For instance, token type IDs will show up for BERT, but not to DistilBERT. In any case, if all you want to do is plug in some code, you don't really need to worry about what these things mean, since you can just pass these model inputs into the model without really thinking about what's inside. But the next few slides will look at what's happening more in-depth.

Currently, there are still some alternative ways to produce the same transformations using the tokenizer object. Note that this is optional because we already saw code that does all the steps.

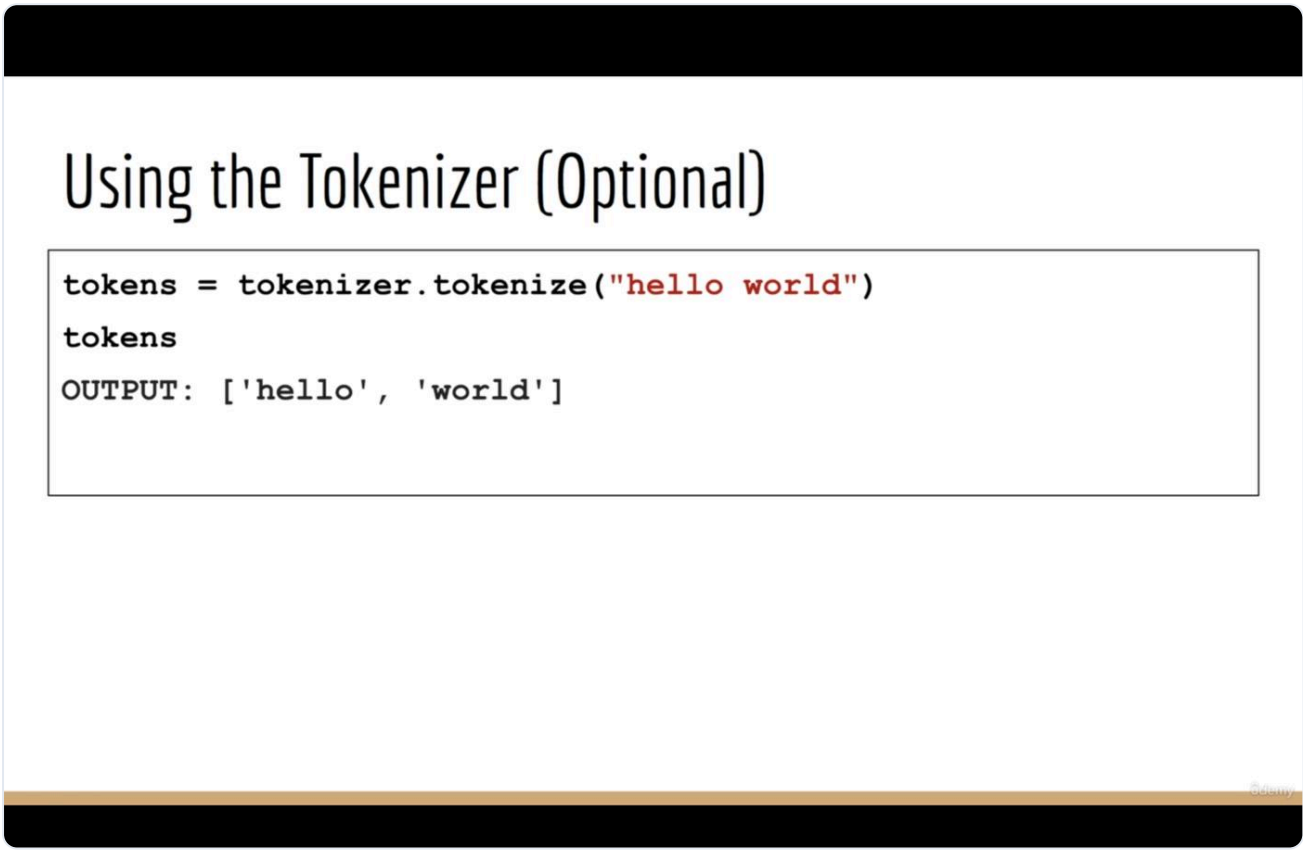

FIGURE 4.35 | Using the Tokenizer (Optional)

This code is meant to just give you some more insight about what goes on. The first method we will look at is the tokenize method. This method, it takes in some text and it only does the tokenization step. That is, the output of this will be a list of tokens. The next step is to convert these tokens into token IDs.

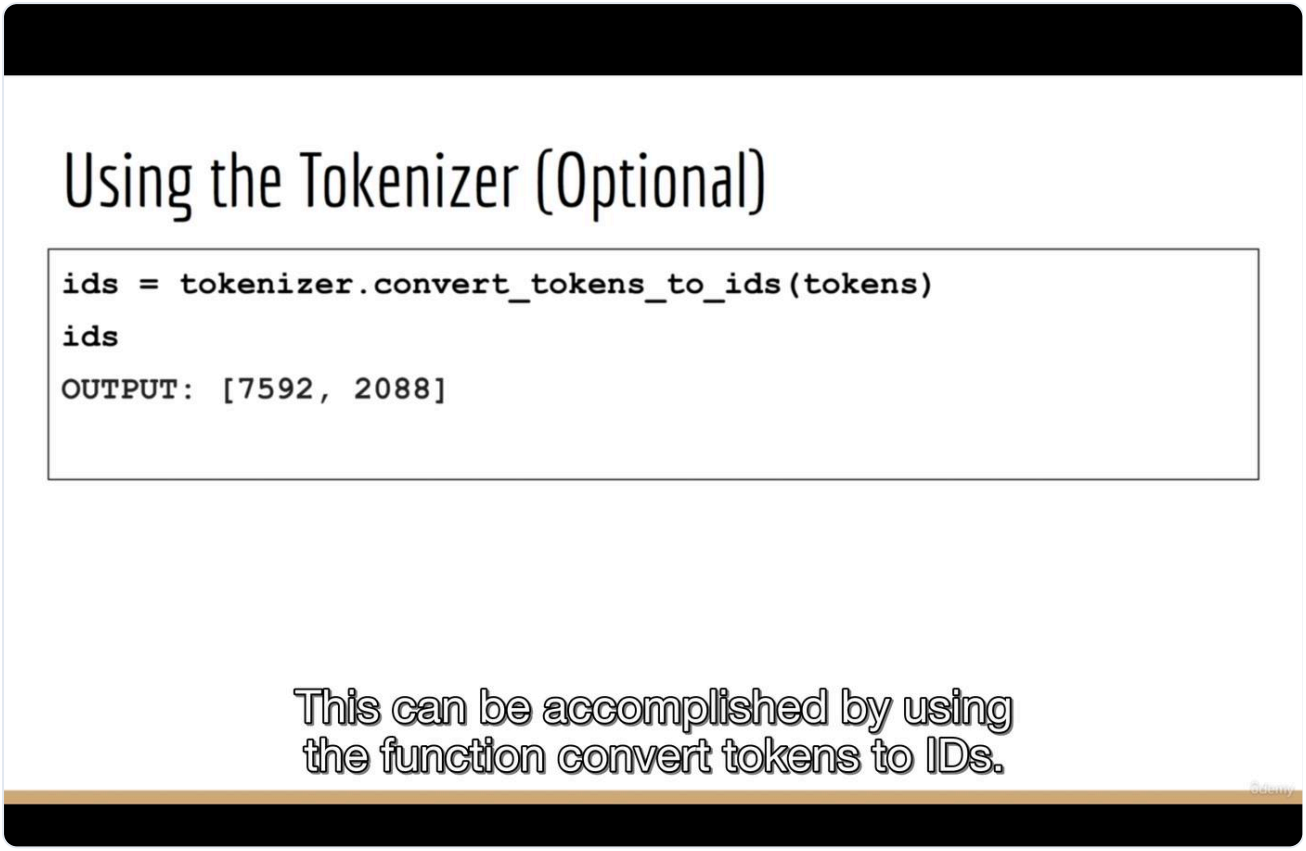

FIGURE 4.36 | Using the Tokenizer (Optional)

This can be accomplished by using the function convert tokens to IDs. As you can see, this gives us back a list of integer IDs.

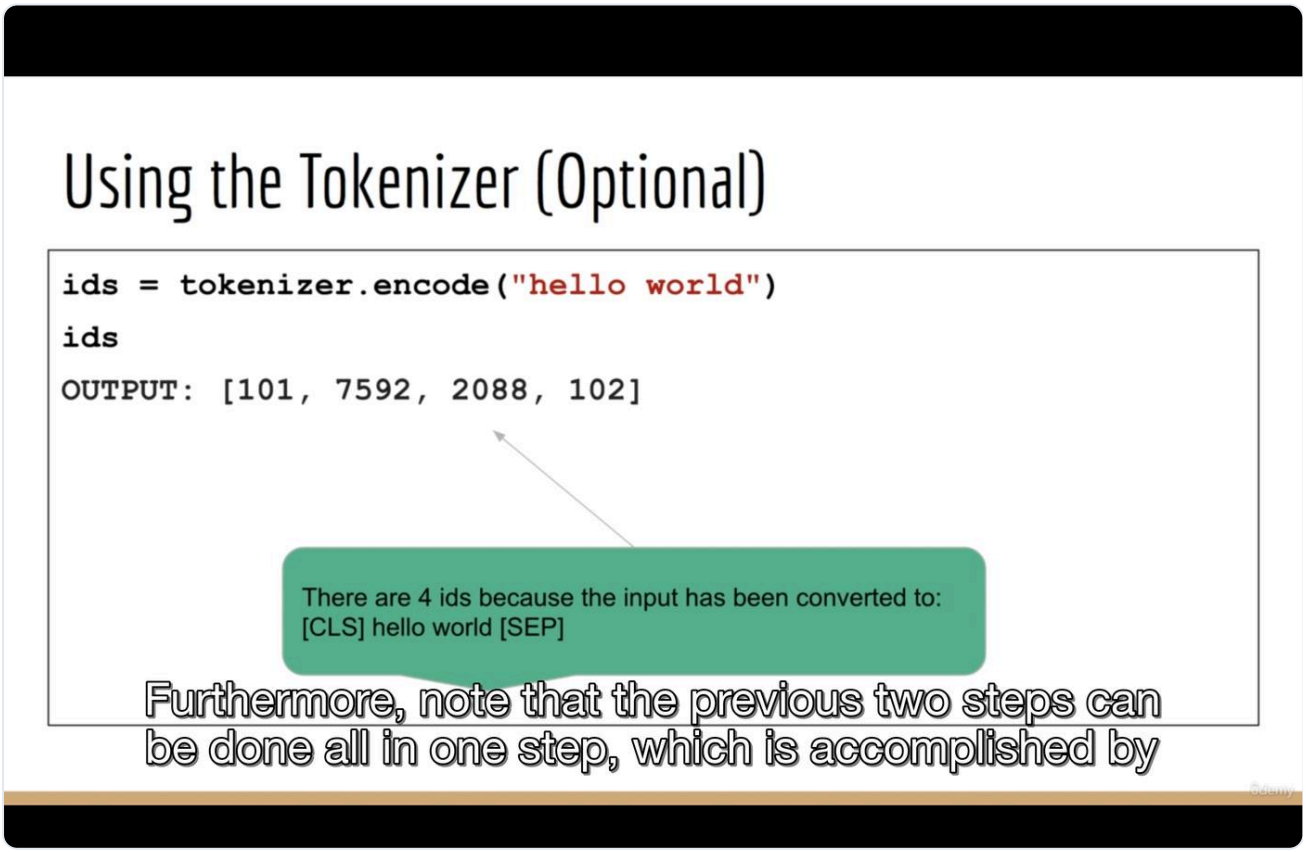

FIGURE 4.37 | Using the Tokenizer (Optional)

Furthermore, note that the previous two steps can be done all in one step, which is accomplished by calling the encode method. However, note that this method also does an additional step, which is to add a special BERT specific tokens called CLS and SEP. We'll understand more about what these are for as we go through the course.

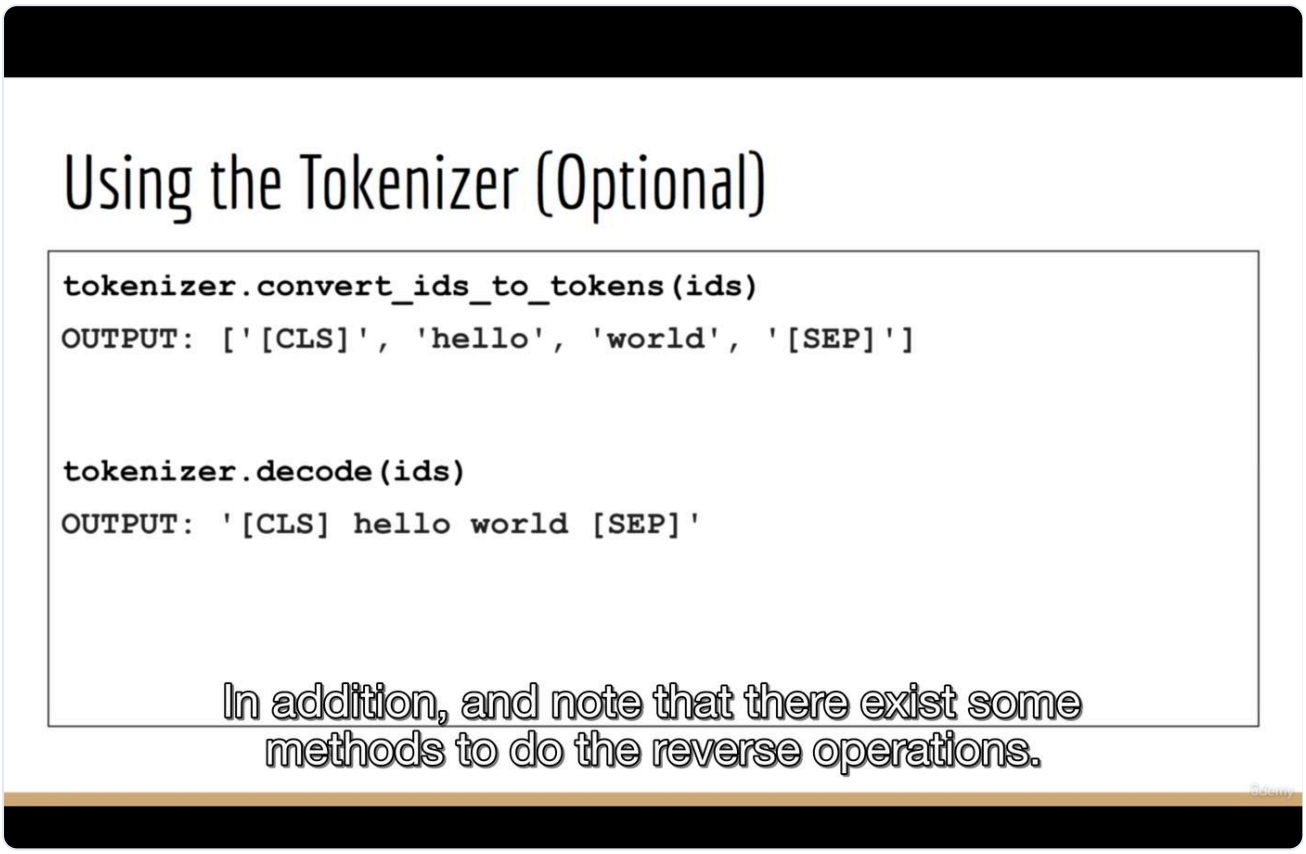

FIGURE 4.38 | Using the Tokenizer (Optional)

In addition, and note that there exist some methods to do the reverse operations. For instance, if we have IDs, we can convert these back into tokens by calling the function convert_ids_to_tokens. As you can see, this will give us back a list of tokens. Furthermore, we can pass in the IDs into a function called Decode and this will give us back a single string with the tokens that join back together. So this does the opposite of tokenize as well.

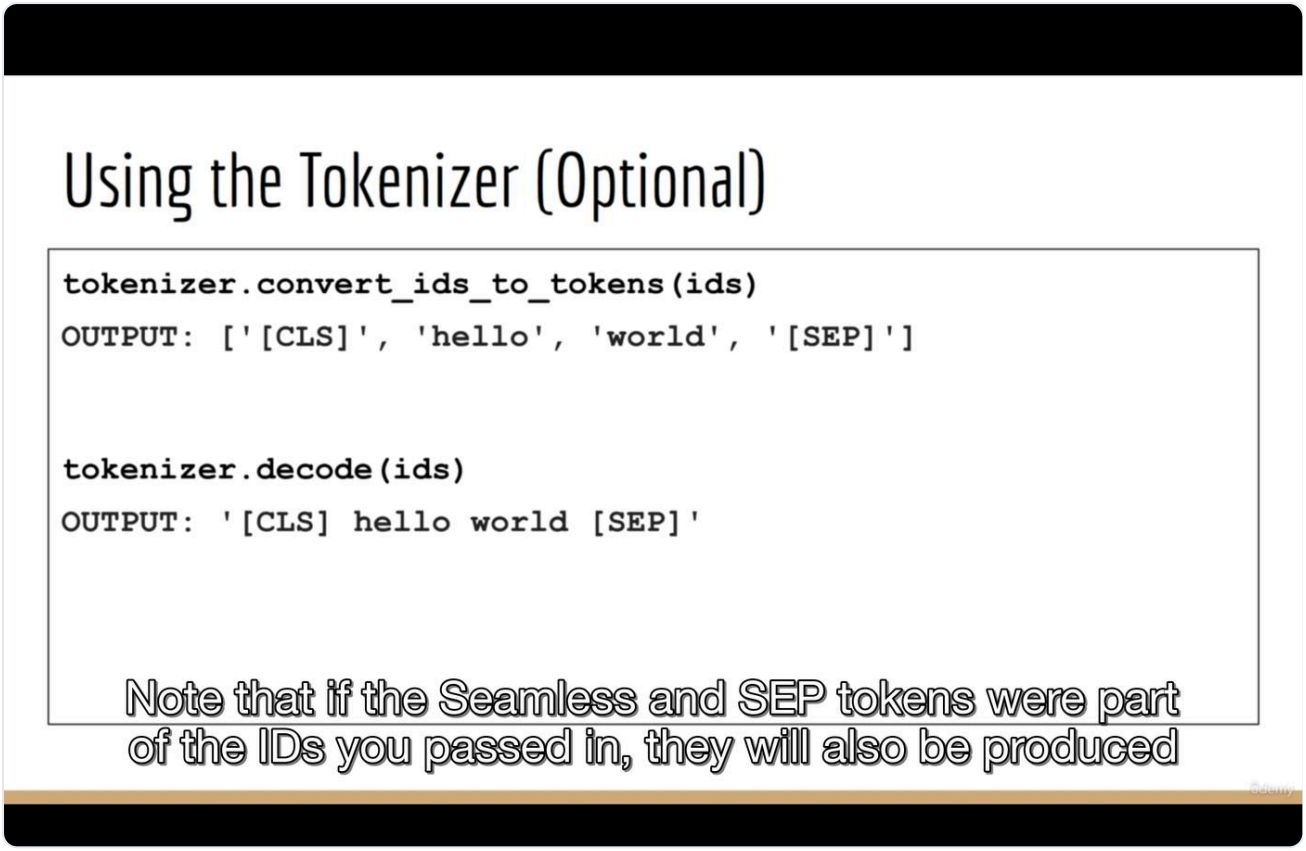

FIGURE 4.39 | Using the Tokenizer (Optional)

Note that if the CLS and SEP tokens were part of the IDs you passed in, they will also be produced in the decode output.

Now you might be wondering why do all these methods exist when you can just call the tokenize and get everything all at once, including the extra inputs like the attention mask? Well, it's important to remember that this is a new library. Things are always changing. In fact, I found that a few related methods which were not discussed in this lecture still work but are now deprecated. So just keep in mind that things are constantly evolving. And don't be surprised if the syntax changes overnight.

And as a corollary to that, don't worry when it does.

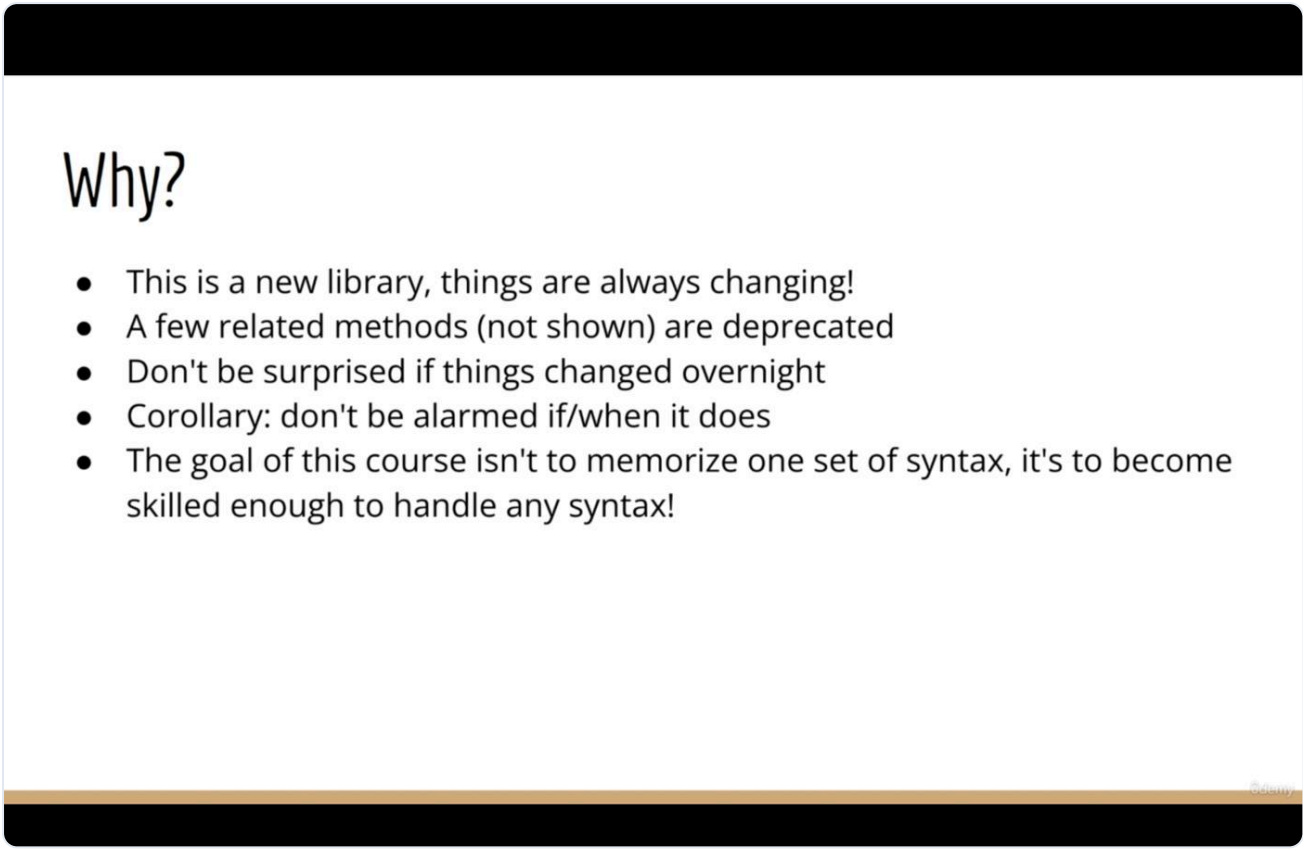

FIGURE 4.40 | A Few Related Methods (Not Shown) Are Deprecated

Remember that the goal of these courses isn't to get you to memorize one set of syntax. It's about turning you into a kind of programmer that can handle any syntax. So the next thing to note is that what we've done so far won't just work with our model. As you recall, the previous output was a dictionary containing values, which were all lists. But PyTorch doesn't take this as input. Instead, PyTorch models process torch tensors in order to get the values back as torch tensors.

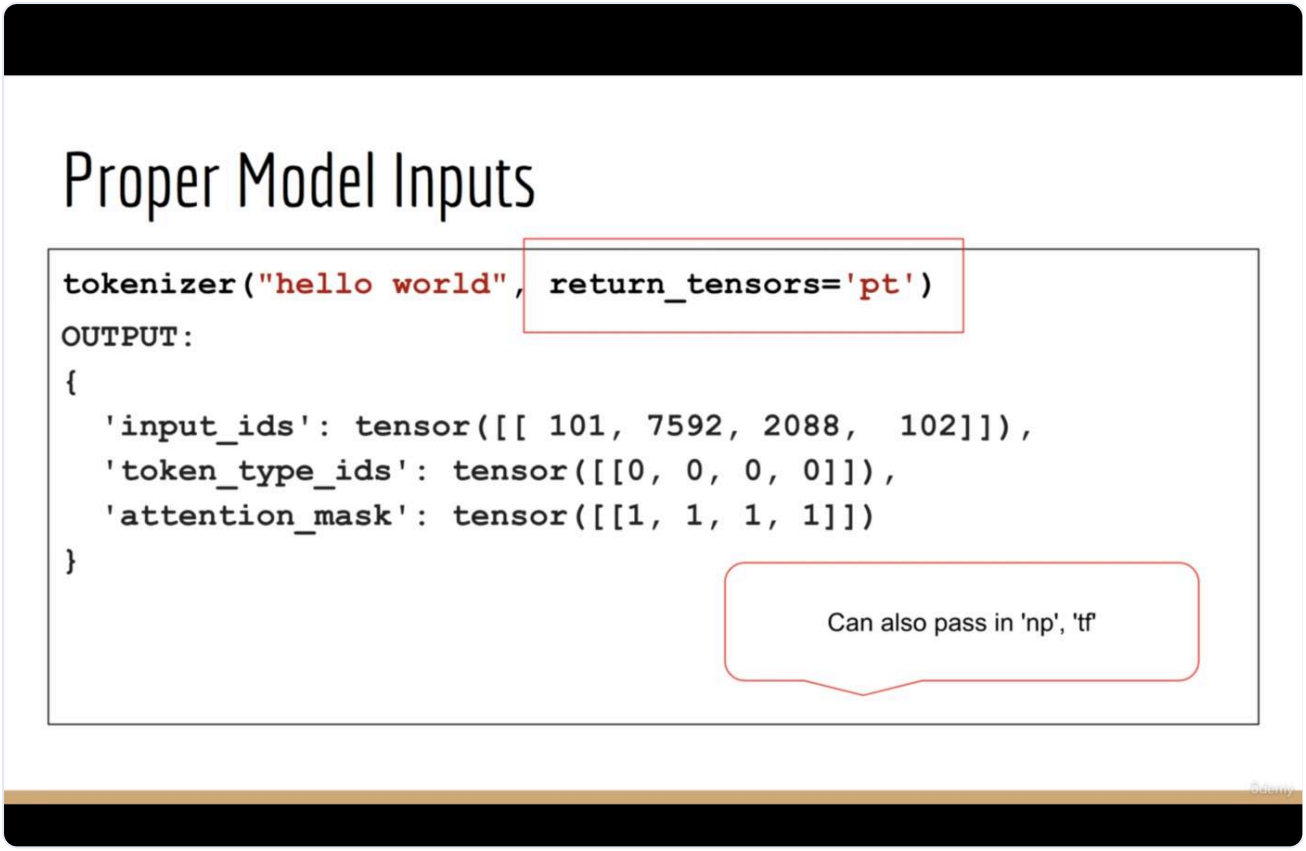

FIGURE 4.41 | Proper Model Inputs

We set the argument return_tensors to the string. Note that there are other possible options. For instance, if you're using TensorFlow, you can use the string 'tf' or if you just want NumPy arrays, you can pass an NP. In this course we'll be using pytorch models and Hugging Face since they are the most flexible. Okay.

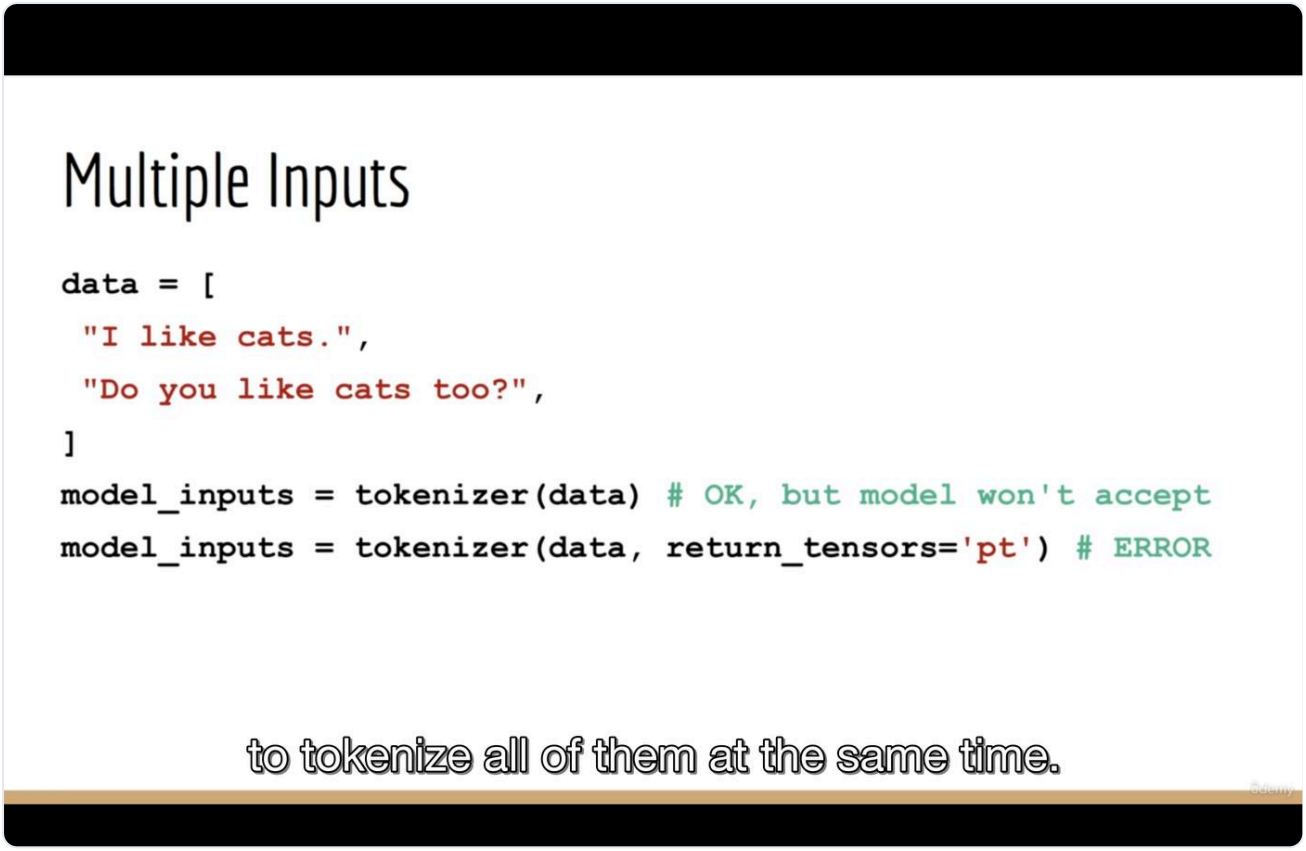

FIGURE 4.42 | Multiple Inputs

So the next step is to consider what happens when we have multiple strings or documents and we want to tokenize all of them at the same time.

You'll notice that if you try this naively, you're likely going to get an error message if you also specify that you want PyTorch tensors. So why does this happen? Well, remember that in deep learning we don't like things to have variable length. If we're going to process them in batches. There's no such thing as a jagged pytorch tensor.

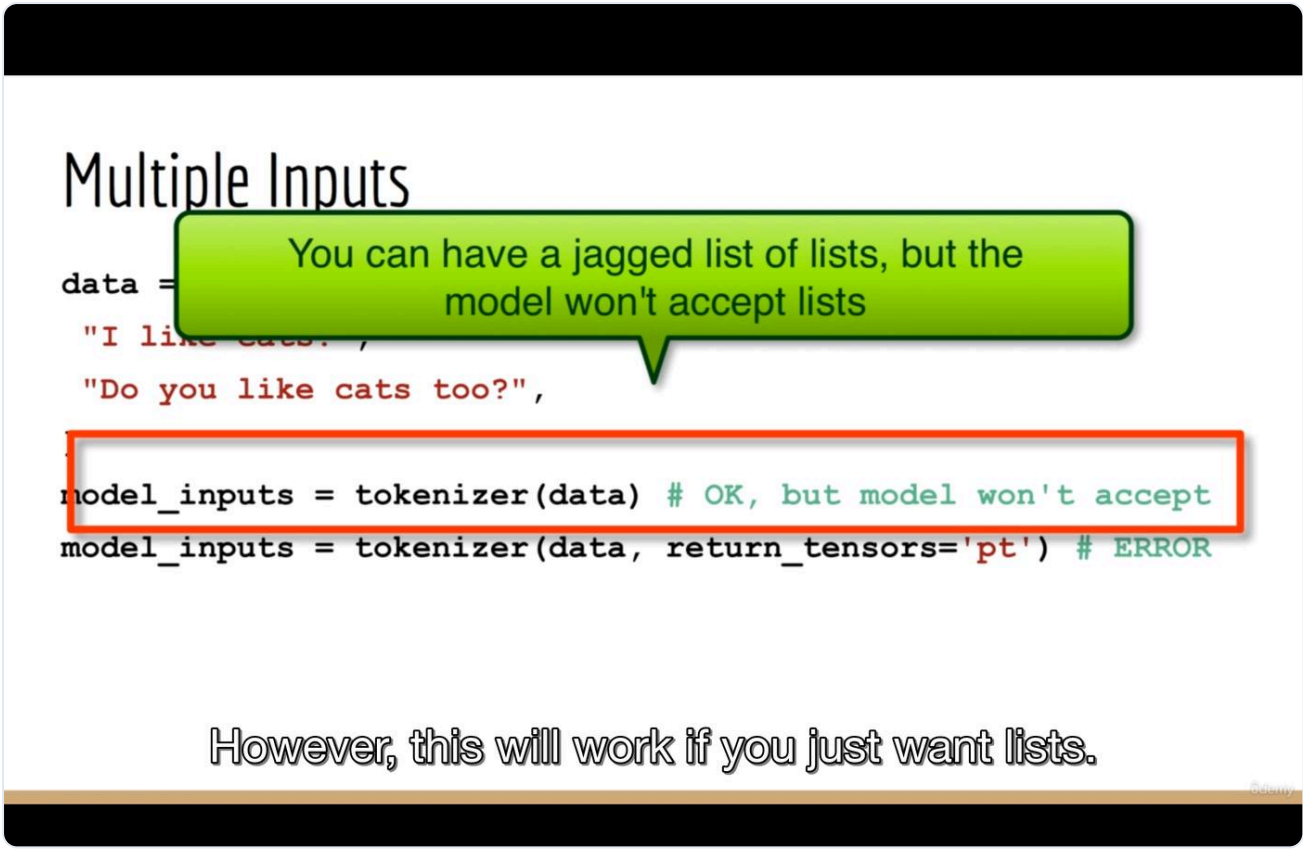

FIGURE 4.43 | Multiple Inputs

However, this will work if you just want lists. At the same time, those lists cannot be used as inputs to the model. So you don't want to do that anyway.

What we need to do is to add two additional arguments, which is padding and truncation. In particular, we should set these to true. It is unclear why these are the defaults.

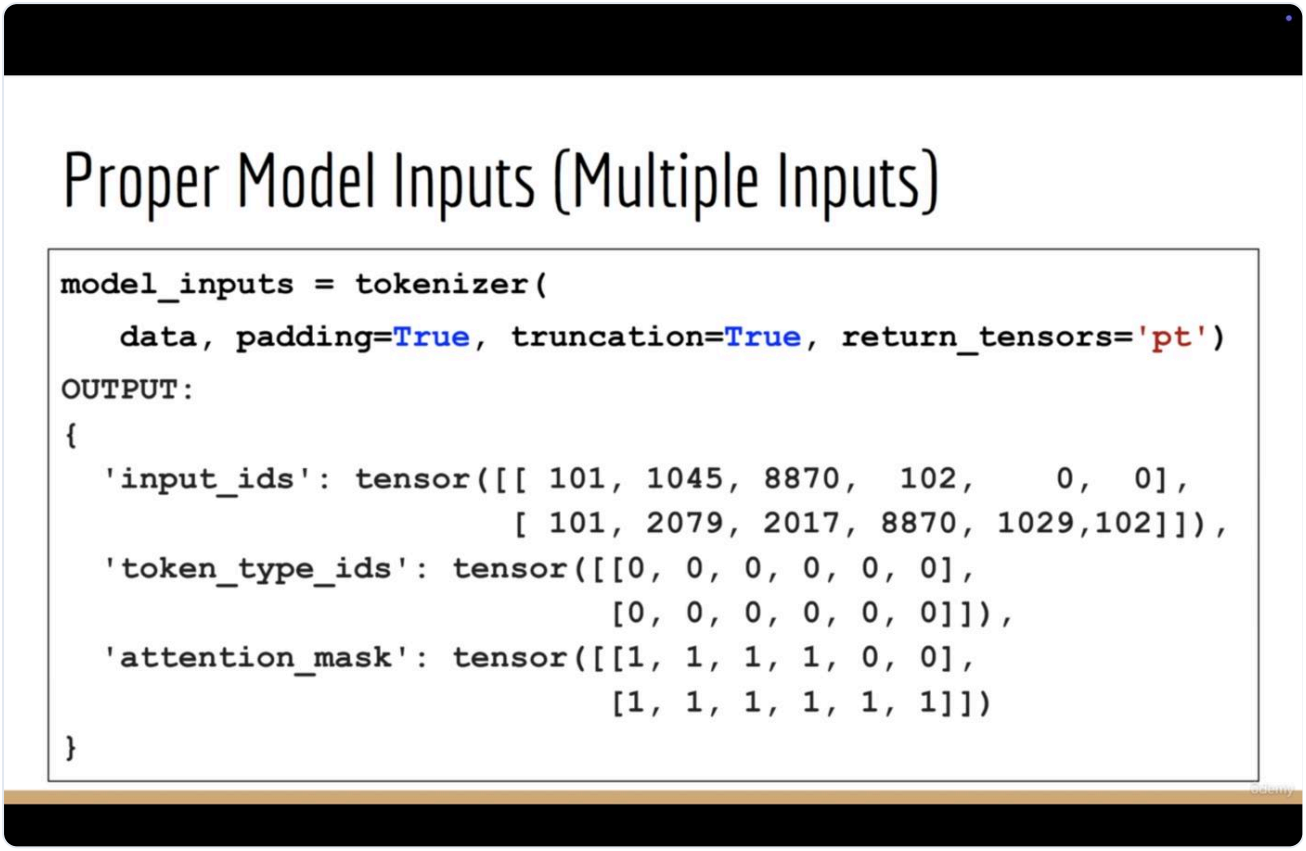

FIGURE 4.44 | Proper Model Inputs (Multiple Inputs)

But again, remember that this is a constantly evolving and new library. So that may be the case in the future, as you can see. Whereas previously each value in our dictionary was a list. Now each value is a list of lists where each inner list corresponds to each input string. Importantly, notice how each of these inner lists now has the same length.

The shorter sequences have been made artificially longer by adding a pad token at the end. This is also a good time to mention the purpose of the attention mask.

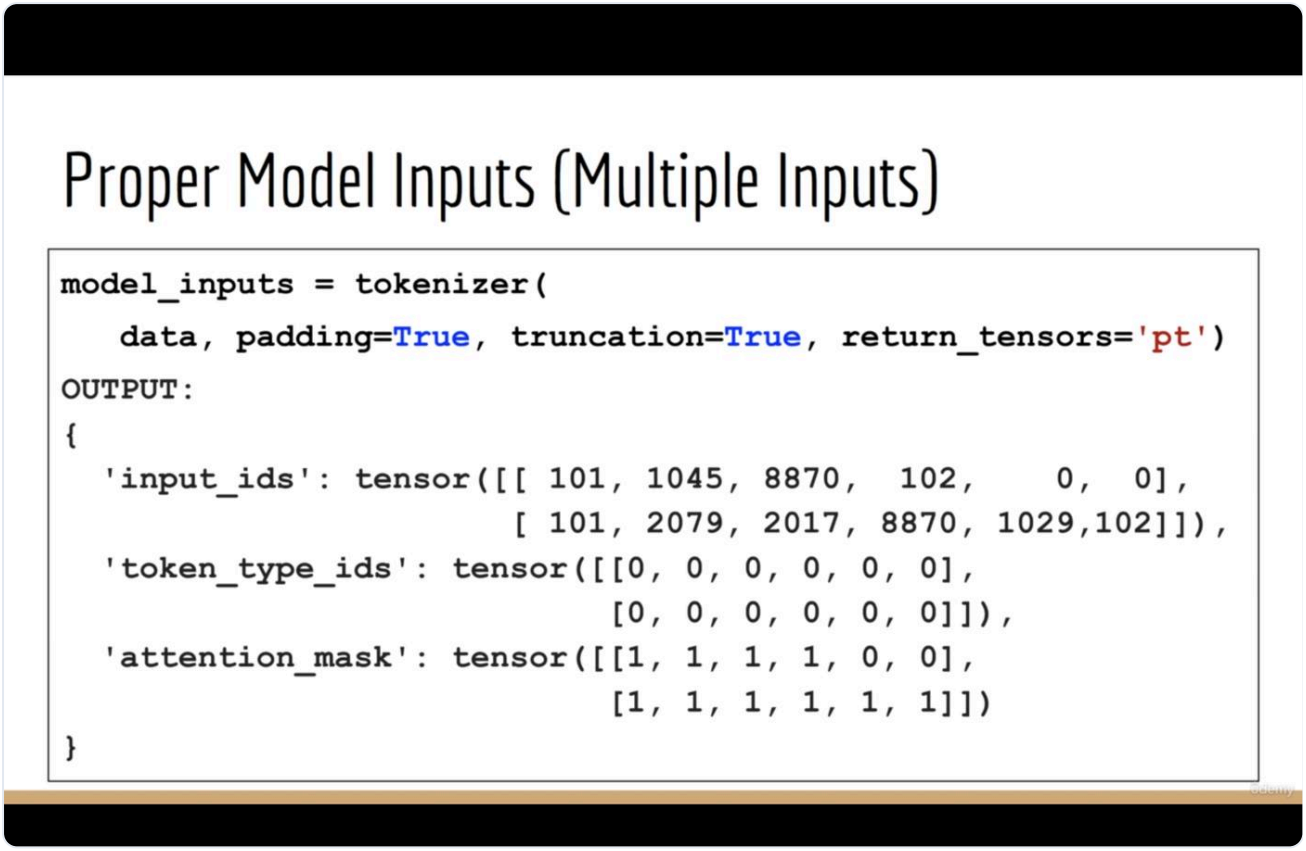

FIGURE 4.45 | Proper Model Inputs (Multiple Inputs)

Essentially, this tells the model where it should bother to pay attention to for any tokens where the attention mask is zero. Our model will ignore those tokens and it will not be possible to use them to compute the model output, which is of course what we want. So in total to ensure that we get a batch of data that can be used as input into our PyTorch model, which is the default for Hugging Face and currently the most flexible.

We need to specify the padding argument, the truncation argument and the return_tensors argument. Okay. So at this point, we are ready to pass in our absolutely names of model inputs into the model.

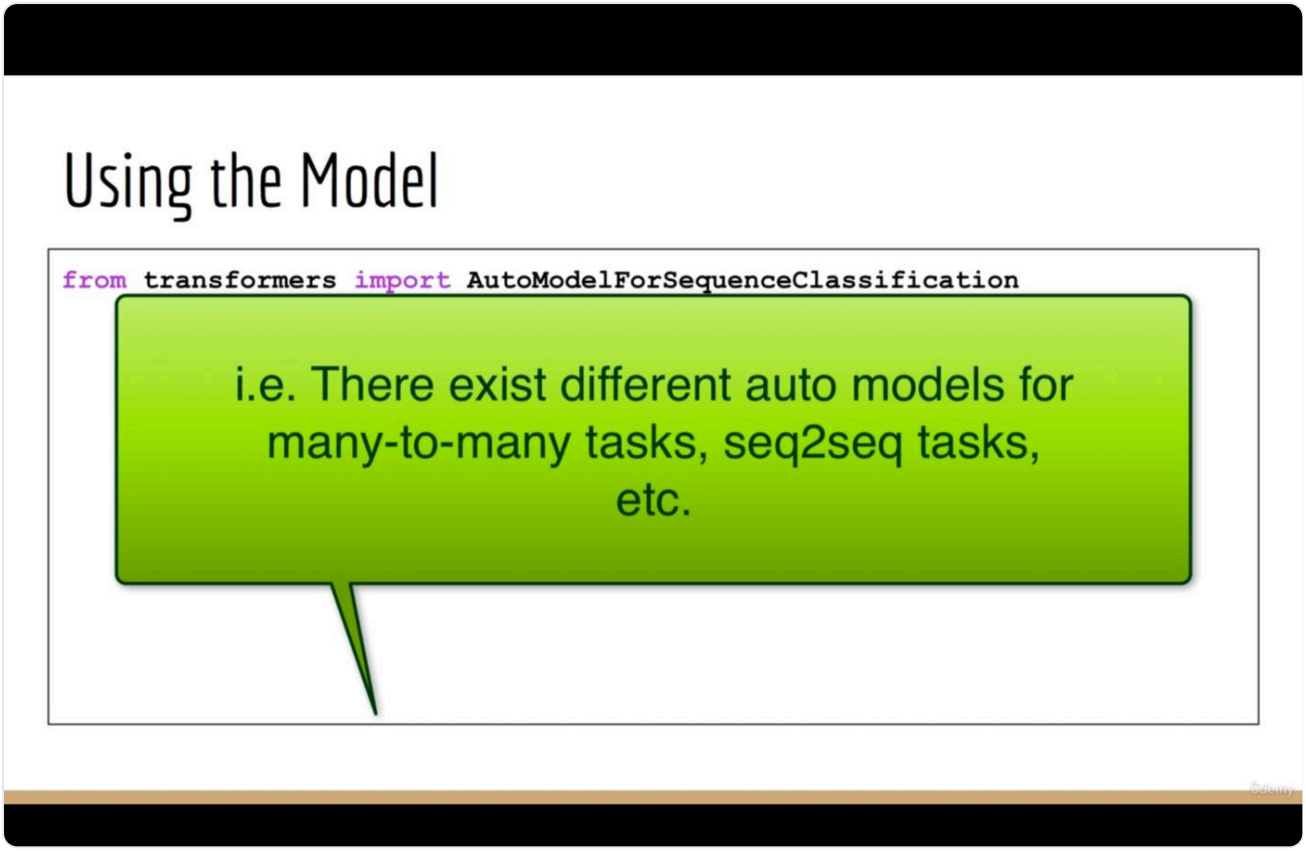

FIGURE 4.46 | Using the Model

So first, we'll need to look at how to create a model. Now, although we could take a completely pre-trained model like the ones we looked at for pipelines in this section, we will focus on a fine tuning for text classification. So suppose that we want to fine tune a BERT model for text classification like the tokenize.

Or we could create a BERT-specific object, but will instead use an auto model for more flexibility. In particular, we'll use the class auto model for sequence classification. Note that there are similarly named classes for other tasks which we will encounter in a later section. In order to load a pre-trained BERT model, we simply call the function from_pretrained.

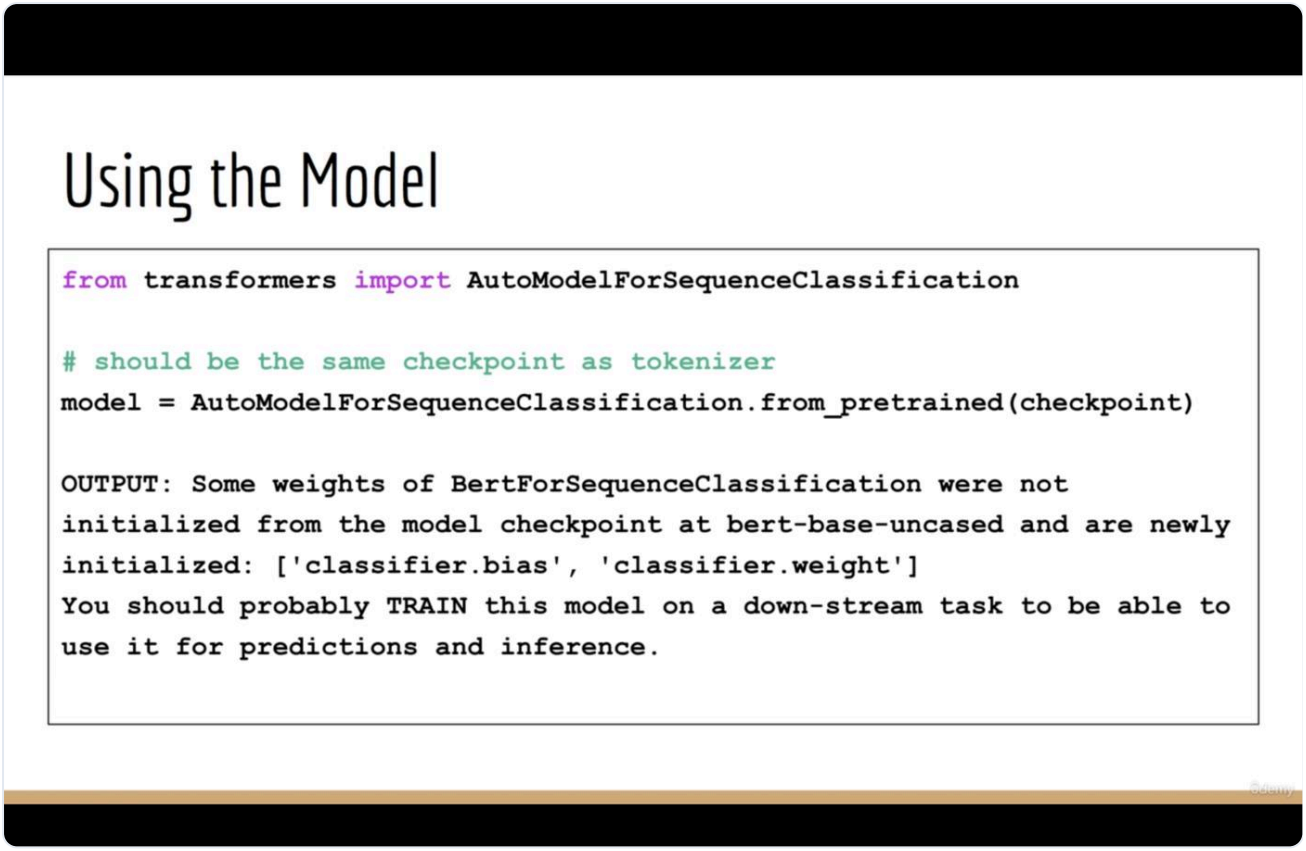

FIGURE 4.47 | Using the Model

Just like we did with the tokenizer. Note that the checkpoint we pass in must match the checkpoint we passed in for the tokenizer so that we get the right tokenize for the model.

Now you'll notice that when you run this, you will get a warning stating that some of the layers have not been trained. This is exactly what we want. What this means is that when we call this method, it does all the work of taking the pre-trained BERT body, cutting off the head, which was meant for another task, and adding a new head for our new task. Because this is classification, our final layer is just a dense layer with K outputs.

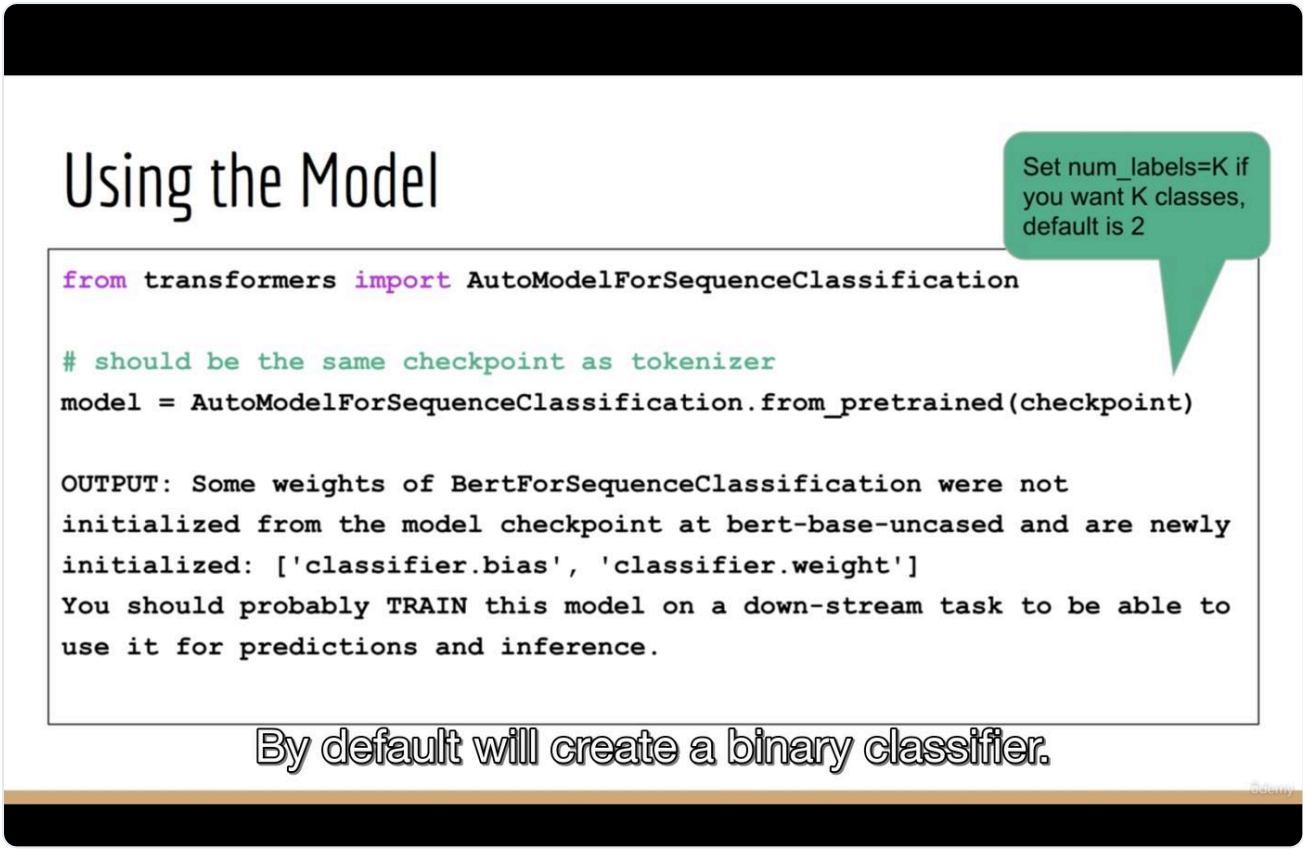

FIGURE 4.48

By default will create a binary classifier.

But you can change this by specifying the argument num labels. Okay. So the next step is to actually pass in some data into your model.

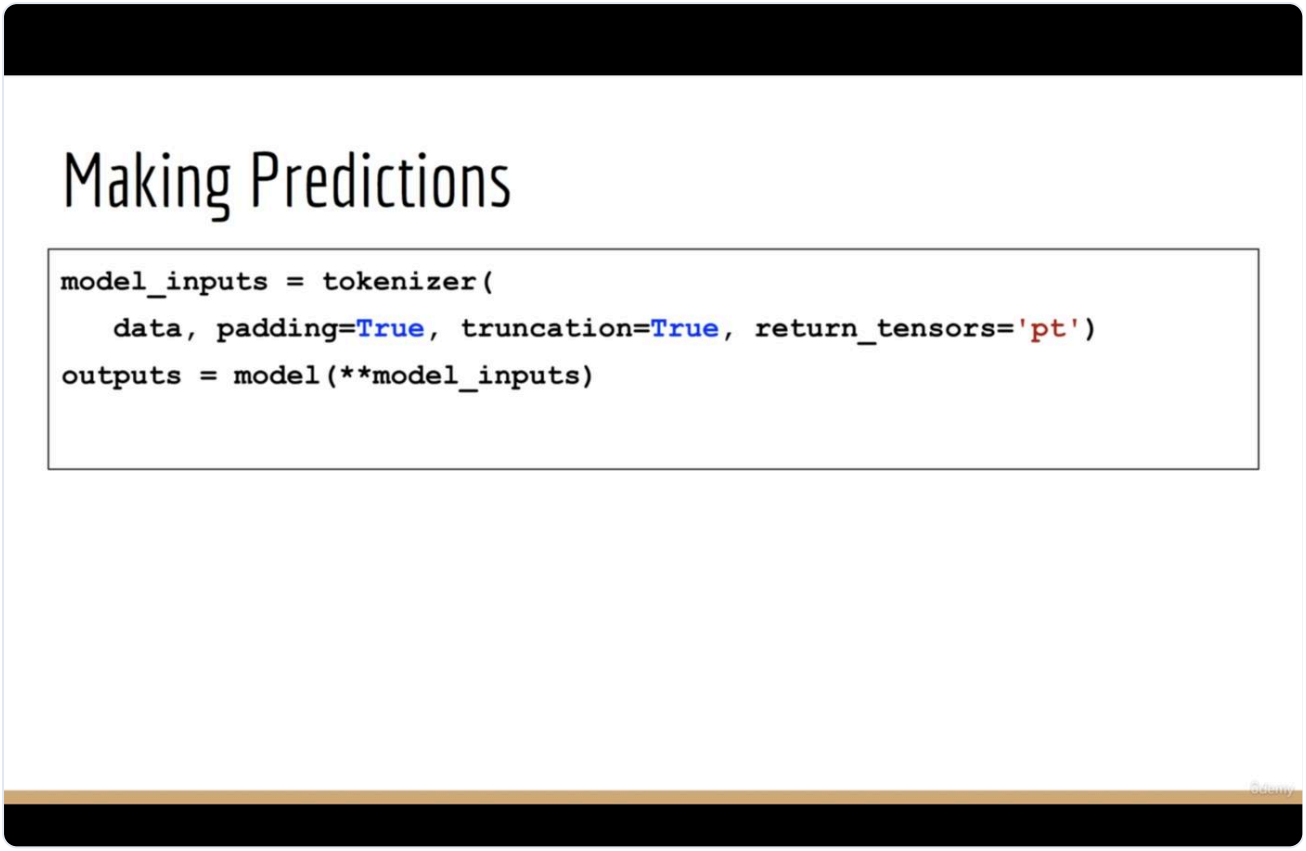

FIGURE 4.49 | Making Predictions

At this point, we're not going to discuss how to train a model, since that will come shortly. This is just how to make predictions with the model, which of course is a prerequisite of being able to train it. To do this, we take the model inputs dictionary that we got back from the tokenisation and we pass them into the model with a double asterisk in front.

Now you might be wondering what's with the double asterisk?

FIGURE 4.50 | Double Asterisk

You recall that in Python at the double asterisk converts a dictionary into named arguments. As an example, suppose I have a function, a create user that takes in a name, email and password. I could call this function like a normal function call where I just pass in the arguments in order. Another option is to specify the argument names explicitly, and yet another option is to have a dictionary with the method parameters and to use the double asterisk.

So how does this relate to our model?

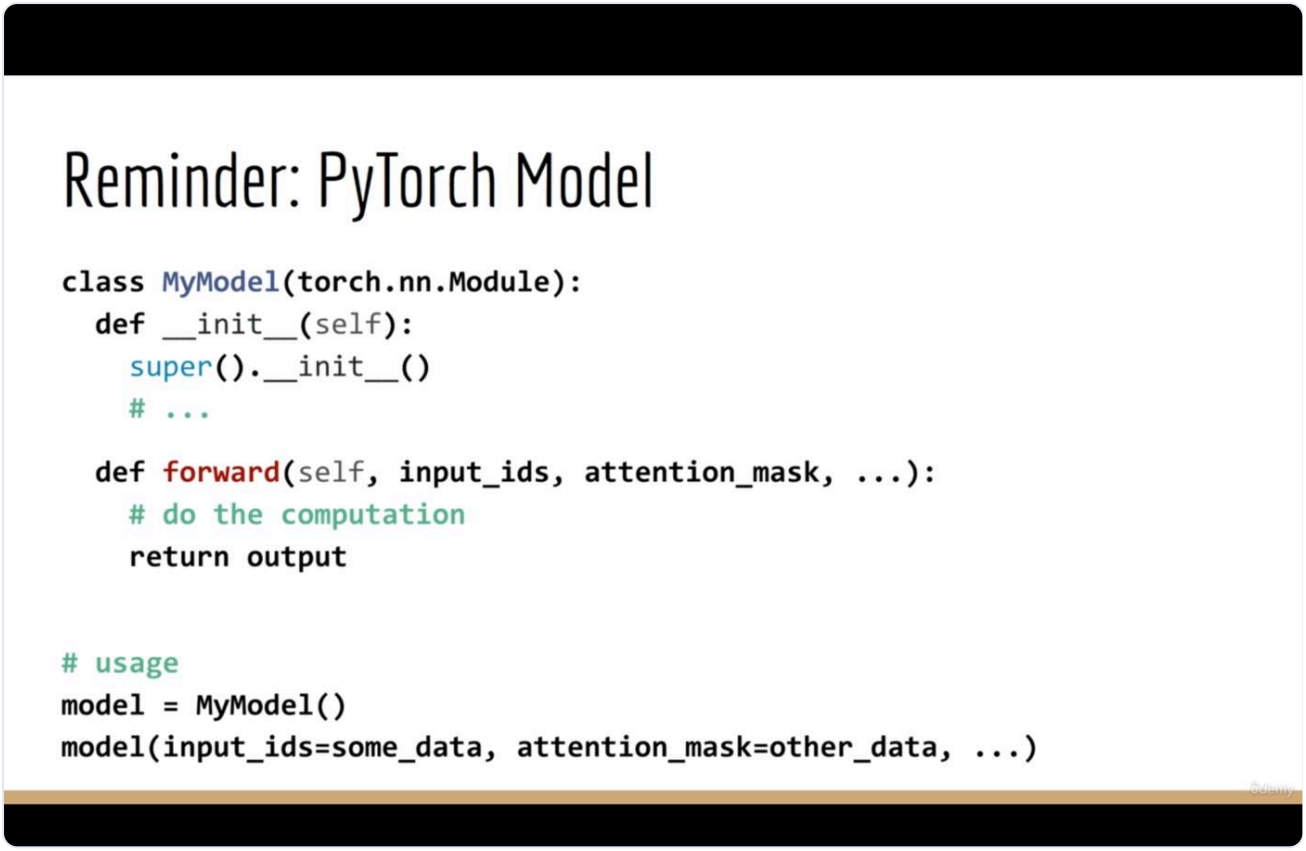

FIGURE 4.51

Well, you'll recall that in PyTorch when we create a model or a module, we create a forward method where we specify what operations the model should actually do to produce an output. We can then call that module as if it were a function or a method. Of course, this method has arguments which have names, and that's what we're passing in when we pass in our dictionary of model inputs. Finally, let's talk about what kind of outputs you should expect to get back from a model.

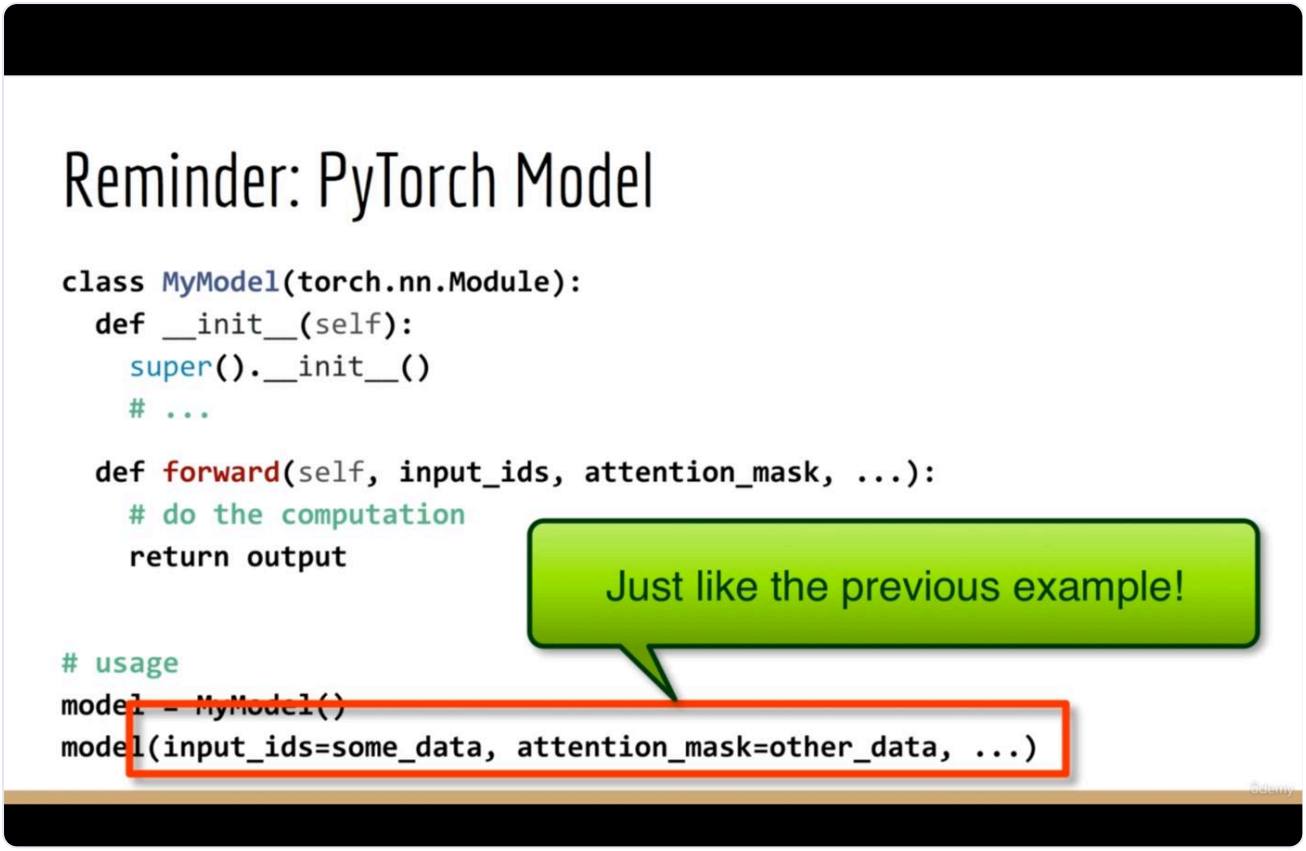

FIGURE 4.52

Since we're going to be working with PyTorch and since this is a classifier with K classes, we're going to get back a PyTorch tensor with K predictions for each input sample. If we passed in and documents, output will be of size n by k. If we pass in only a single document without a sample dimension that is a 1D tensor of input ids, then our output will just be of size k. Our interpretation of the outputs is that they are logits, which means these are values we have before we apply the softmax function.

If you want probabilities, you can always apply the softmax yourself.

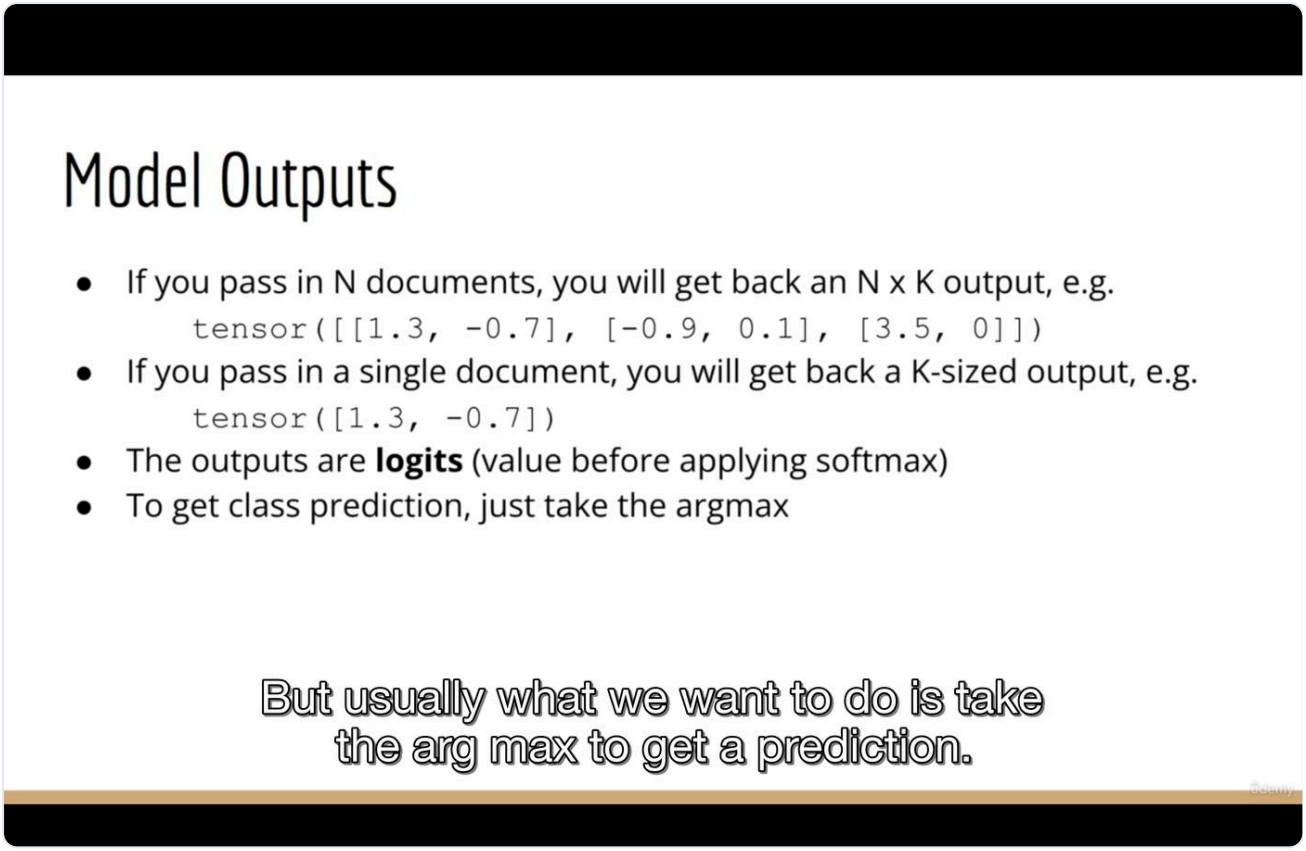

FIGURE 4.53 | Model Outputs

But usually what we want to do is take the argmax to get a prediction.

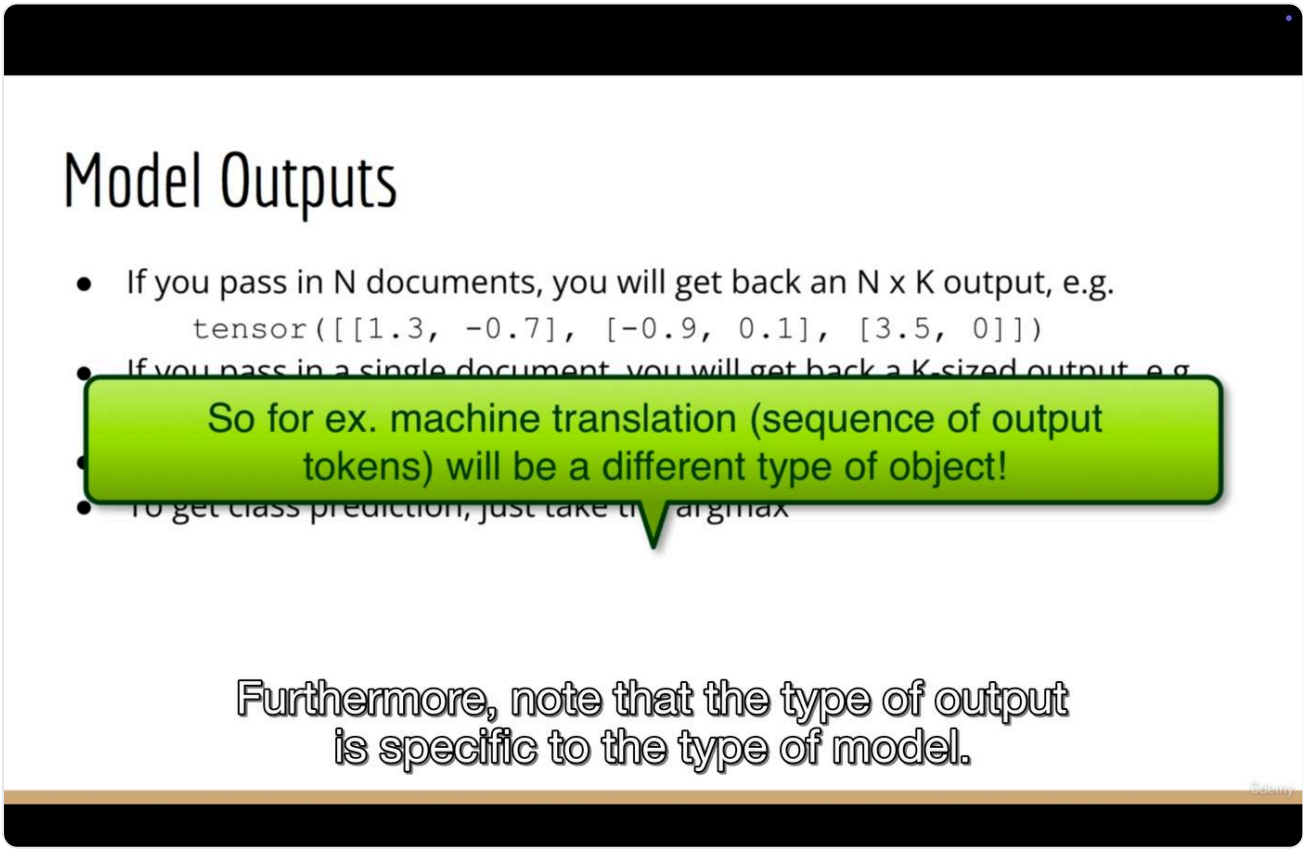

FIGURE 4.54 | Model Outputs

Furthermore, note that the type of output is specific to the type of model. Since we built a sequence classifier.

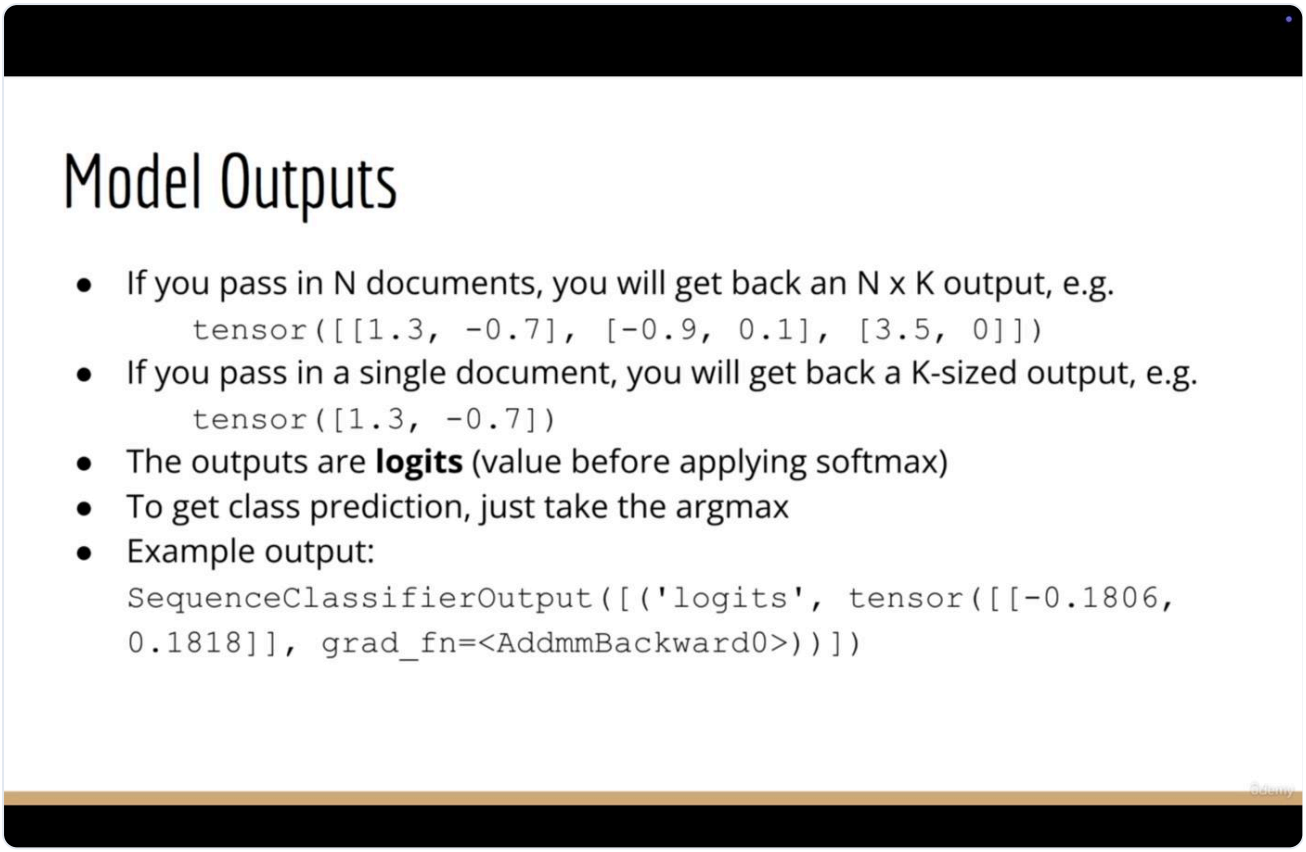

FIGURE 4.55 | Model Outputs

The output will be an object of type sequence classifier output. You'll see that this acts like a named tuple in Python, which is kind of like a dictionary and kind of like an object and kind of like a tuple all at the same time.

There is one key called logits, and to access the logits you can use the dot notation like an object.

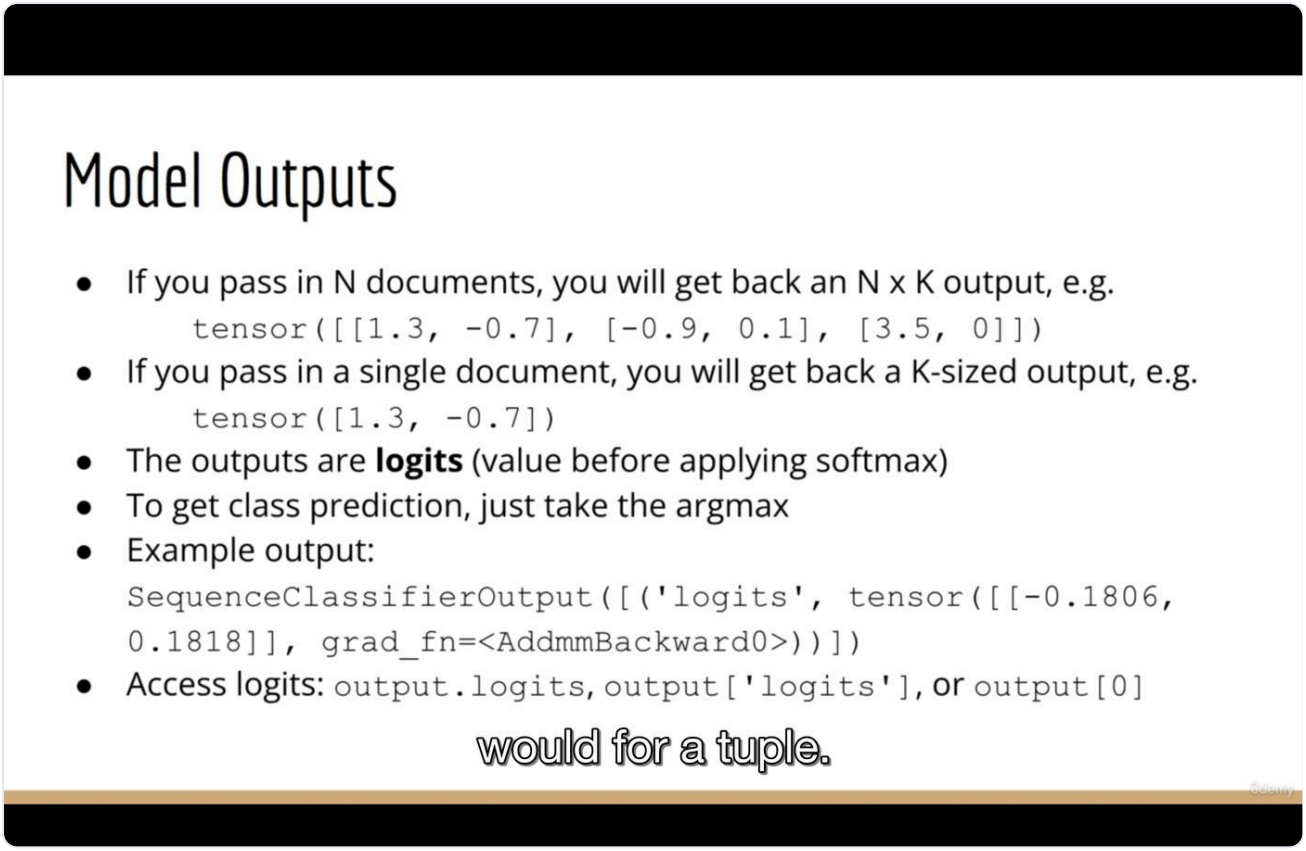

FIGURE 4.56 | Model Outputs

You can use dictionary notation, or you can specify the position of the item you want, just like you would for a tuple.

4 · Models and Tokenizers in Python

See the notebook here.

So in this lecture we are going to do a little demo to show you models and tokenizers in action. We'll begin by installing Transformers as we normally do. So the next step is to import the AutoTokenizer class. The next step is to define our desired checkpoint, which for this lecture will be bert-base-cased. We'll also create our tokenizer by calling the method from_pretrained and passing in our checkpoint. The next step is to simply print out our tokenizing objects.

So as you can see, we get an object of type PreTrainedTokenizerFast. Note that some useful information gets printed along with this, including the name vocab size. Maximum sequence length is fast, which tells us whether or not the tokenize has been optimized for performance. Which side the pattern goes on, which side we truncate from, and finally, how special tokens are represented in the string format. The next step is to test our tokenizer with a simple string which says Hello World.

As you can see, this returns a dictionary with three keys, input IDs, token type IDs and attention mask. As you recall, the input IDs are the integer representation of each token. While the attention mask tells our model which tokens should enter into our attention computations token type.

These have yet to be discussed, but it's important to note that these won't show up for every model we use. The next step is to try more granular methods on the objects.

This will help us understand what the tokenize actually does behind the scenes. We'll begin by calling tokenize passing in the same string as before. So as you can see, this returns a list with two tokens, hello and world. The next step is to convert these tokens into integer IDs by calling convert tokens to IDs. Let's now check out the IDs. Okay. So these are the IDs for hello and world. The next step is to convert the IDs back into tokens. As you can see, we get back a list containing hello and world as expected.

The next step is to call the decode function, which not only converts the IDs back into tokens, but also joins the tokens back into a string. As you can see, we get back the string hello world as expected. Let's now try the sort of opposite function to decode, which is encode. As you can see, this gives us back a list of token IDs. But interestingly, this has four IDs, whereas the previous steps above only gave us two IDs. To investigate why this might be.

Let's convert the IDs back into tokens. As you can see, the reason we had two extra tokens is because the encoding method adds the special BERT tokens, CLS and SEP. Now let's see what happens if we try to decode the output of ENCODE directly. As you can see, we get back to the original input with the CLS and SEP tokens as expected. So essentially, this is the true input into the BERT model, even though we technically only will pass in the hello world.

While these extra tokens might seem superfluous, now you'll see why they are important later on when we consider different kinds of tasks, especially ones where you need to pass in multiple sentences into the model. For example, if you want to build a model that can tell you whether two sentences mean the same thing or if you want to, passing a question along with multiple possible answers. The next step is to tokenize the hello world string. One last time to obtain model inputs that we can use to pass into our model.

Now before we go on to actually using a model. Let's do one last example where we look at how to tokenize multiple sentences at the same time. As you can see, this works without any problems. The next step is to import the class auto model for sequence classification. The next step is to call the from_pretrained method passing in our checkpoint, same as we did with the tokenizer. Okay. So this is the warning that we saw in the previous lecture, which tells us that the model needs to be trained specifically because the model head has random weights.

The next step is to pass in our model inputs into our model. Okay. So as you can see, we get an error. Please note that this is intentional. So do not be alarmed. What we have learned here is that the model doesn't just take in any kind of input since the default models and Hugging Face are pytorch models. Our model accepts torch tensors. We just passed in this note that it's possible to also create TensorFlow based models which would require TensorFlow tensors.

The syntax is essentially the same. The model just uses a different class. In any case, we're not going to worry about that right now because we are using PyTorch. So the next step is to create our model inputs again. But this time we're going to explicitly specify that we want to get back torch

tensors. So now let's try to pass the inputs into our model again. Okay. So this time everything works as expected. We get back an object of type sequence classifier output which contains the logits which have been output by our model.

Now, remember, these logits are meaningless because the top layers of our model have not yet been trained. In addition and note that by default the library assumes that we wanted a binary classifier. The next step is to create another model, but this time with three outputs instead of two. For this we specify the argument num labels. So again, we get a warning telling us that we need to train the final layers of the model. The next step is to pass our inputs into this new model to see what we get.

As you can see, our logits now have three values instead of two, which is what we expect. The next step is to demonstrate a few ways we can access these logits. The first way is to treat the return object as if it were an object that had an attribute called the logits. As you can see, this gives us back the torch tensor representing the logits which we could then use to make a prediction. The next step is to show you a different way of doing the same thing, which is to treat the output like a dictionary.

So this time we're accessing the key called logits. As you can see, this returns the same thing we got before. Yet another method. If you know the ordering of the values is to treat the return objects like a tuple since there was only one thing in the output object. Of course the index should be zero. Again, you can see that we get the same thing. Note that this method is not recommended since it's better to be explicit about what value we want. Now, just as a little review.

Suppose that we wanted to convert the logits into a NumPy array, which would be useful if we wanted to compute metrics like accuracy, AUC and F1 score which are all part of scikit-learn to do this. We need to do a few steps. First we call Detach because the tensor above has a gradient. We then call CPU to move the tensor from GPU back to CPU. This is if your model computations are happening on the GPU. And finally we call the numpy method to convert the torture tensor into a NumPy array.

Okay. So as you can see, we get back a numpy array as expected. Okay. So the next step in this notebook is to go back to demonstrating how to process multiple strings at once. We begin by creating a data list with two strings. We then call the tokenizer like we did above. So as you recall, this worked before. Okay. So note that we get an error. Again, note that this was intentional to remind you of why this won't work. As an exercise, you might want to think about what we did wrong before we move on to the next step.

Okay. So hopefully you remember that there are two important arguments that we need to pass in. The first is to set padding to true. And the second is to set truncation to true. This will ensure that all of our outputs will have the same length and we do not exceed the maximum length allowable by the model. Okay. So as you can see, this works without any errors. Let's now check out the input IDs. Okay. So both inputs have the same length, which makes sense because a torch tensor cannot be a jagged array.

Note that the first sentence now has zeros at the end, which implies that the padding token ID for BERT is zero. Next will print out the attention mask. So as you can see, the attention mask corresponds to where we have real tokens and where we have padding tokens. So we have one where there are real tokens in zero, where we have padding tokens by passing this into the model. It will avoid it using the padding tokens to do any computation of the output.

Of course, we don't want them to influence our prediction. The next step is to demonstrate that we can pass in these model inputs just the same as we did before. So as you can see, we get back a two by three tensor, which makes sense because we passed in two documents and the number of classes is three.

5 · Transfer Learning & Fine-Tuning (pt 1)

So in this lecture, we will be discussing another important concept related to transformers known as transfer learning. Now it's important to recognize that transfer learning is not related to the architecture of transformers, nor is it a concept that is specific to Transformers.

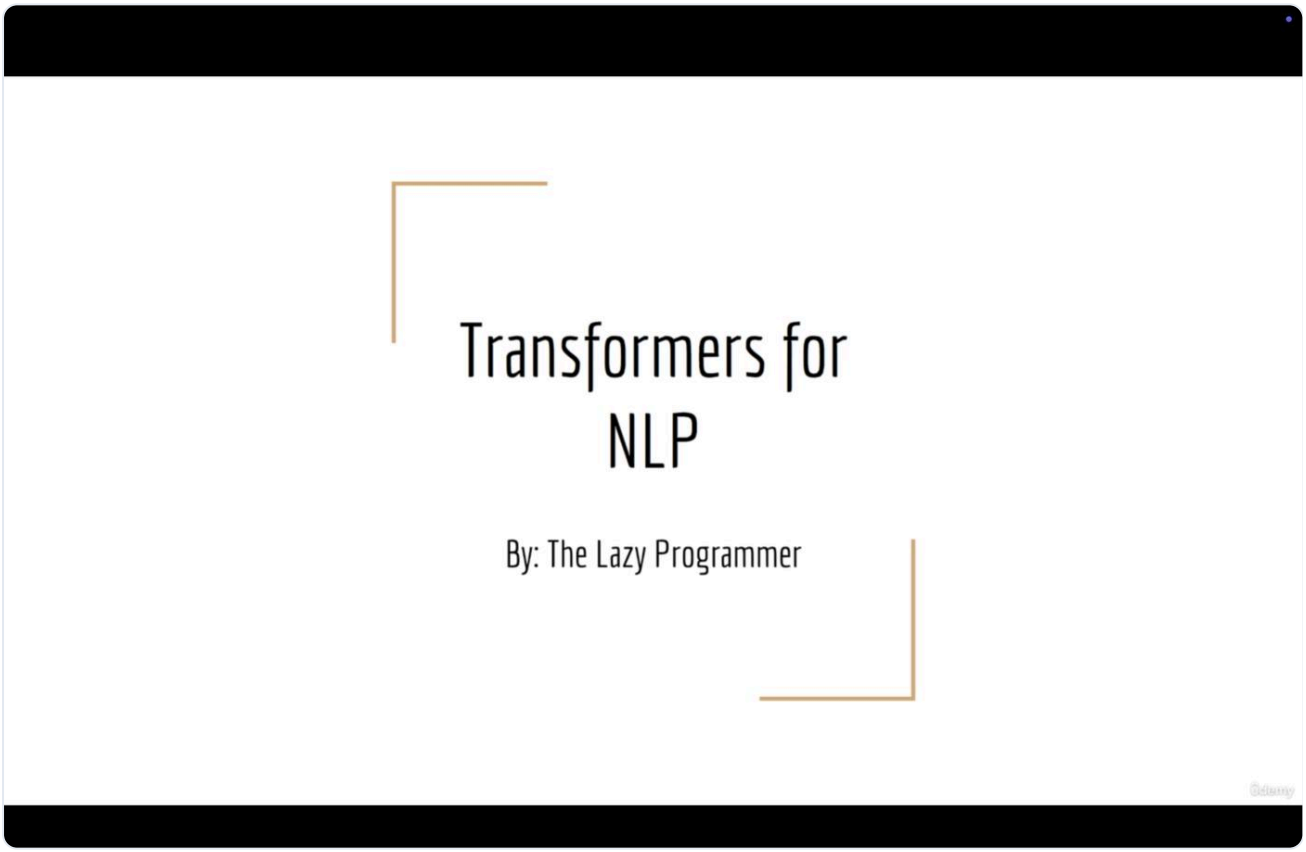

FIGURE 4.57 | Transformers for

In fact, transfer learning is a general machine learning technique that's been applied to Transformers. One way to think of this is transfer.

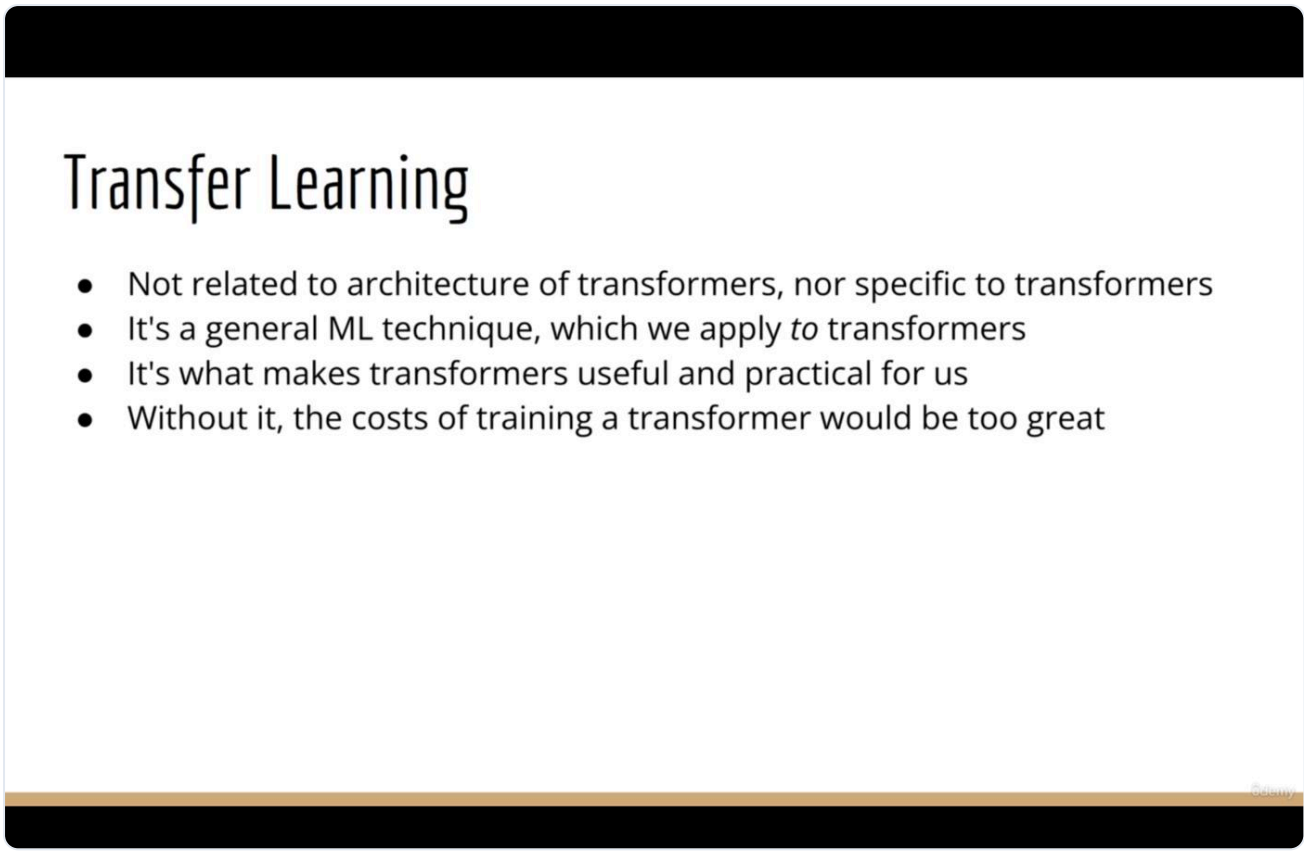

FIGURE 4.58 | Transfer Learning

Learning is what makes Transformers useful and practical for practitioners in this field.

Without transfer learning, the cost to train a transformer network would simply be too great. So transfer learning is what allows us to make building intelligent networks faster and cheaper so as to be practical for everyday use. Okay. So what is transfer learning all about? The best way to understand this is through its history. In particular, transfer learning comes from the field of computer vision. So as you know, one of the remarkable results we have observes from training CNNs is that each layer seems to learn progressively more complex feature transformations of the input.

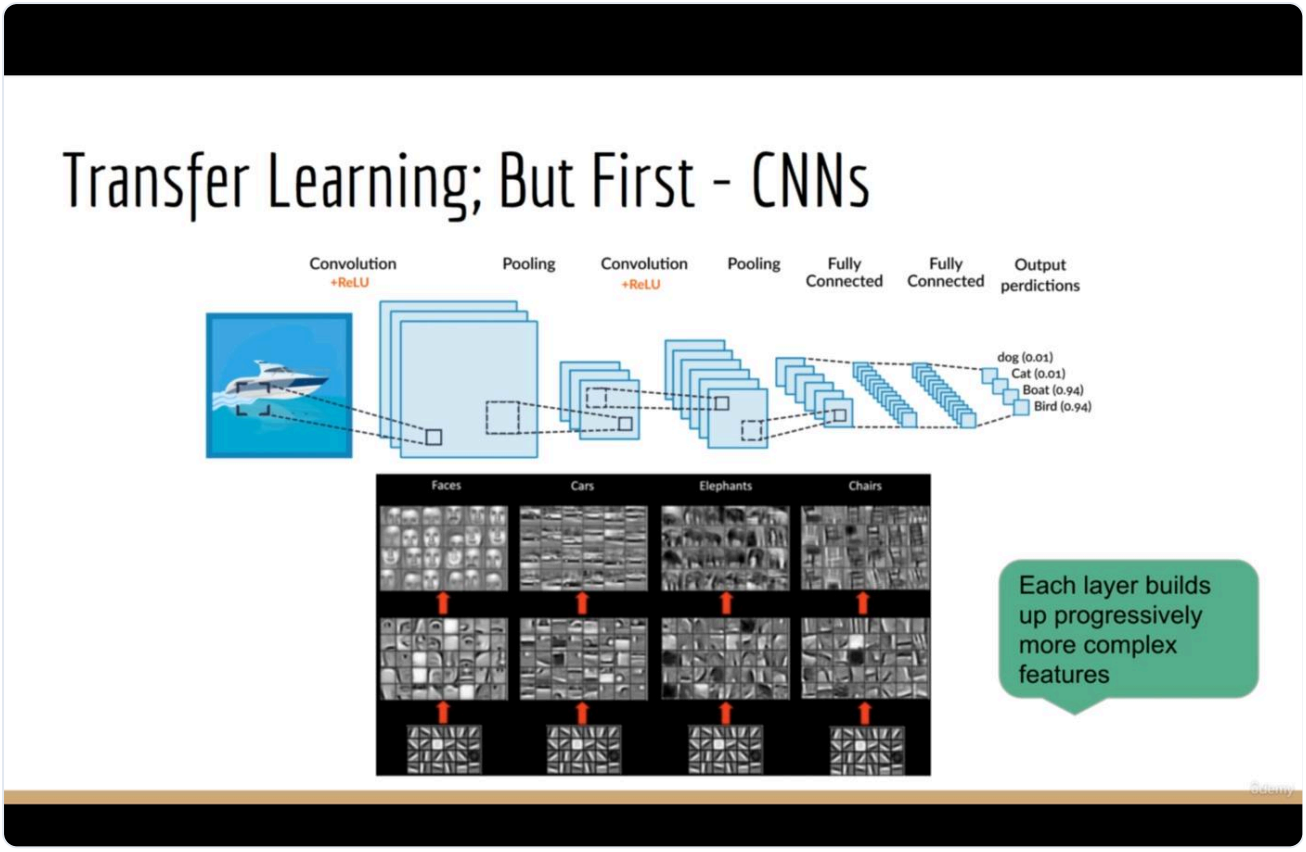

FIGURE 4.59

For example, in a face recognition network, the first layer might learn basic lines and strokes, while the second layer might learn individual facial features such as noses, eyes and lips. The third layer might learn about different whole faces, so each layer builds up a more complex representation based on the simpler features from the previous layers. In effect, what we can say is that every layer seems to output something useful which can then be made use of by adding further layers.

So initially, one of the main tasks of interest in computer vision was classifying images. To this day, the standard benchmark is the ImageNet dataset, which contains millions of images and 1000 classes. Now, because of the large size and variety of this dataset, these CNNs learn to recognize many things. So many things, in fact, that you could pass in a completely random image, not in the ImageNet dataset.

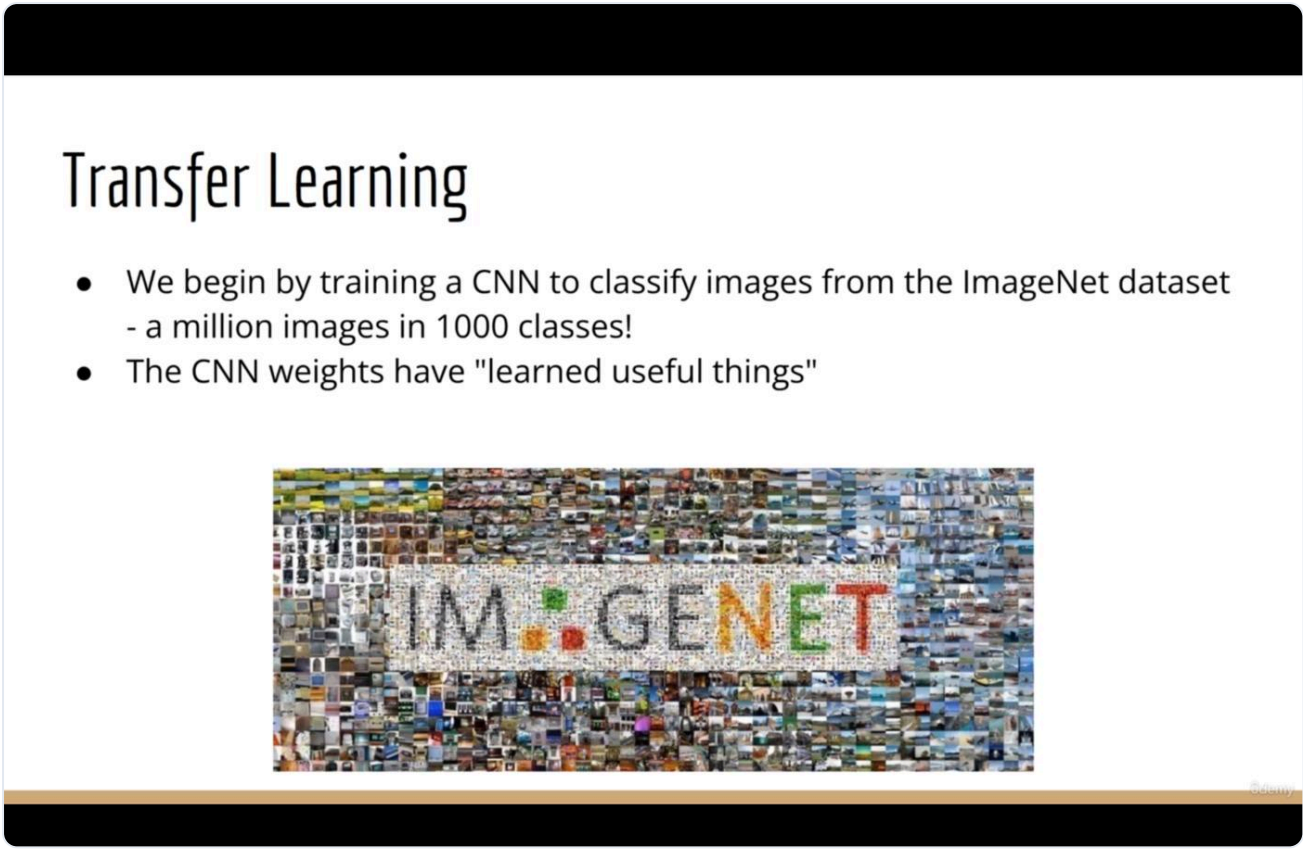

FIGURE 4.60 | Transfer Learning

And chances are you would be able to recognize whatever is in that image.

The key point for transfer learning is that because the features learned by these networks are so useful in a generic sense, there is no longer any need to train them. So how does transfer learning work? Firstly, we should recognise that transfer learning is dependent on large organisations and research groups being generous. Thanks to the openness and generosity of researchers in this field, the weights of very powerful neural networks have been uploaded to the internet.

Free for you to access. That is, you can download a neural network that another research group has worked maybe weeks or months to train and use it instantly with much less effort. We call these pre-trained networks.

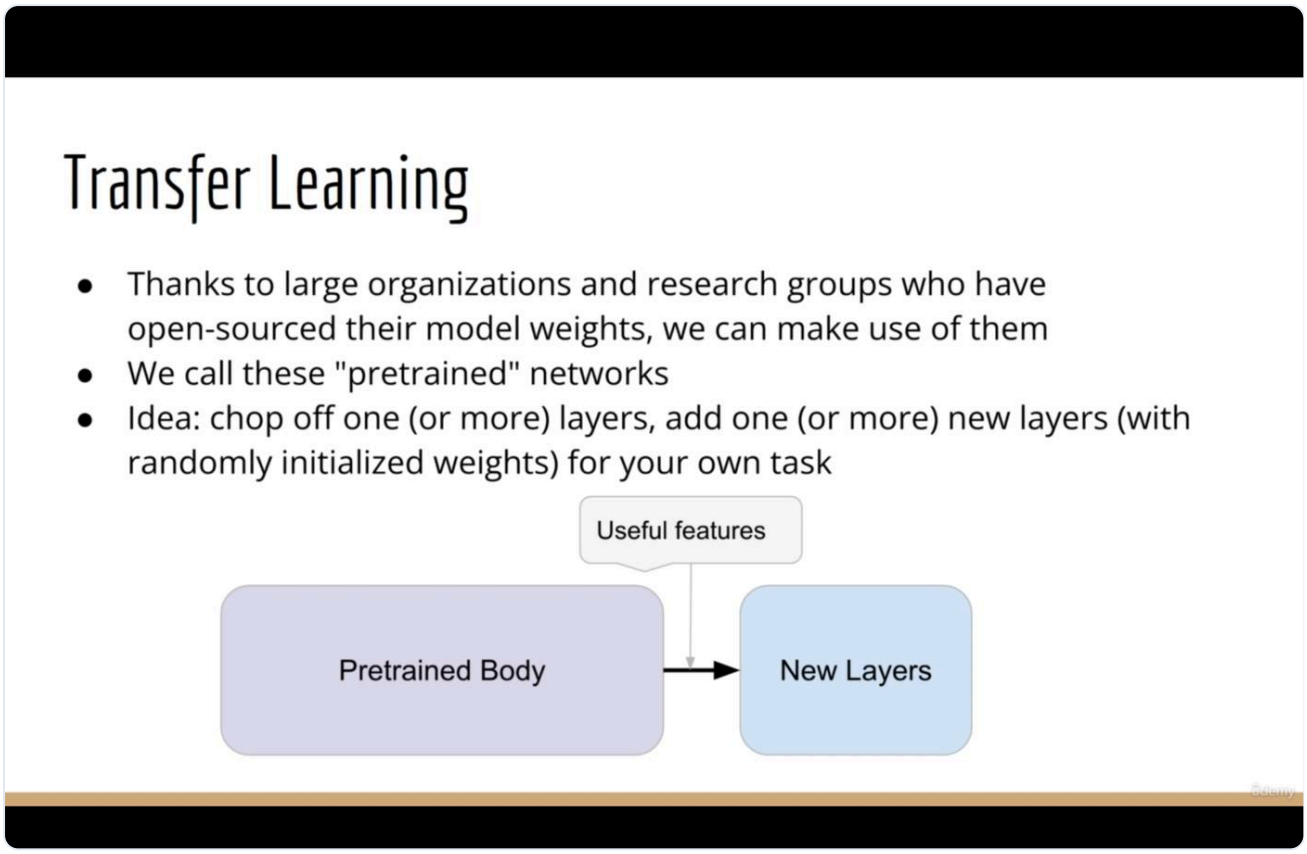

FIGURE 4.61 | Transfer Learning

So what do you do with a pre-trained network? Well, you simply chop off the head, meaning the final dense layer, and you add your own a final dense layer with an output size that activation appropriate for your task. Note that on some occasions you might chop off multiple final layers and you might also add multiple final layers.

Once you've added your new layers, you can then train the network on your own and data sets. Crucially, in computer vision, it is typical to only train part of the network, specifically only the layers you added. The earlier layers in the network which come from the pre-trained network, usually do not need to be trained further. In other words, what you are effectively doing is training a very tiny neural network, or even just a tiny linear model using the pre-trained network as a feature extractor.

And the reason why we can do this is because, as you recall, this pre-trained network already outputs very useful features.

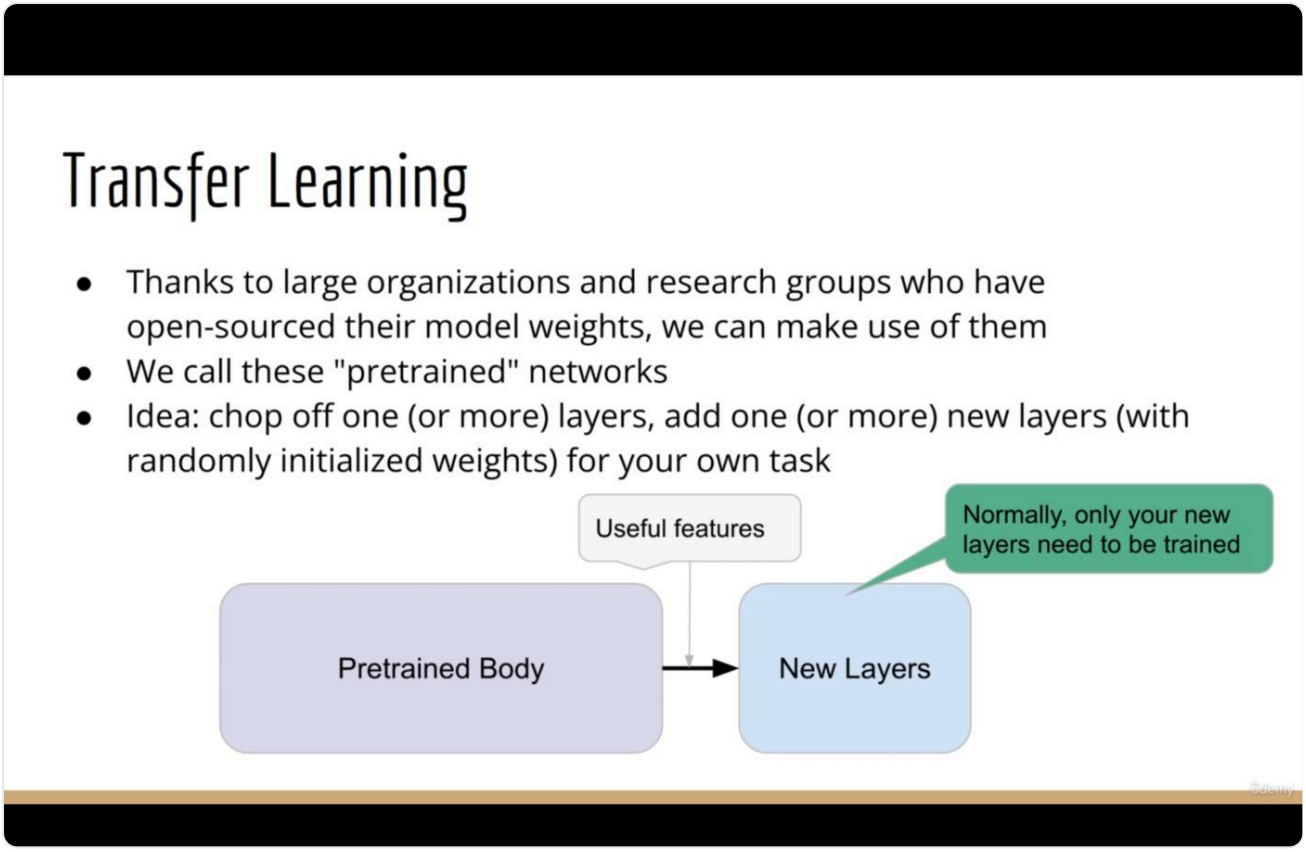

FIGURE 4.62 | Transfer Learning

Having been trained on a very large dataset, importantly, the kind of data you want to train on should be similar to the kind of data the pre-trained network was trained on. For example, if the network was pre trained on RGB images, then your dataset should also consist of RGB images. But you would not want to do something like take a network pre-trained on English text and then use transfer learning on Spanish text.

So one way to think about transfer learning is that the pre-trained network is not part of the model per se. As you recall, in machine learning, we train our models on input features. Those input features can be of any kind, such as raw tensor data, or maybe features we engineered specifically for our domain. One example of this is TF-IDF features which we use in NLP. So in general, the process of training a machine learning model can be visualized by starting with a simple CSV file containing our input features and their corresponding targets.

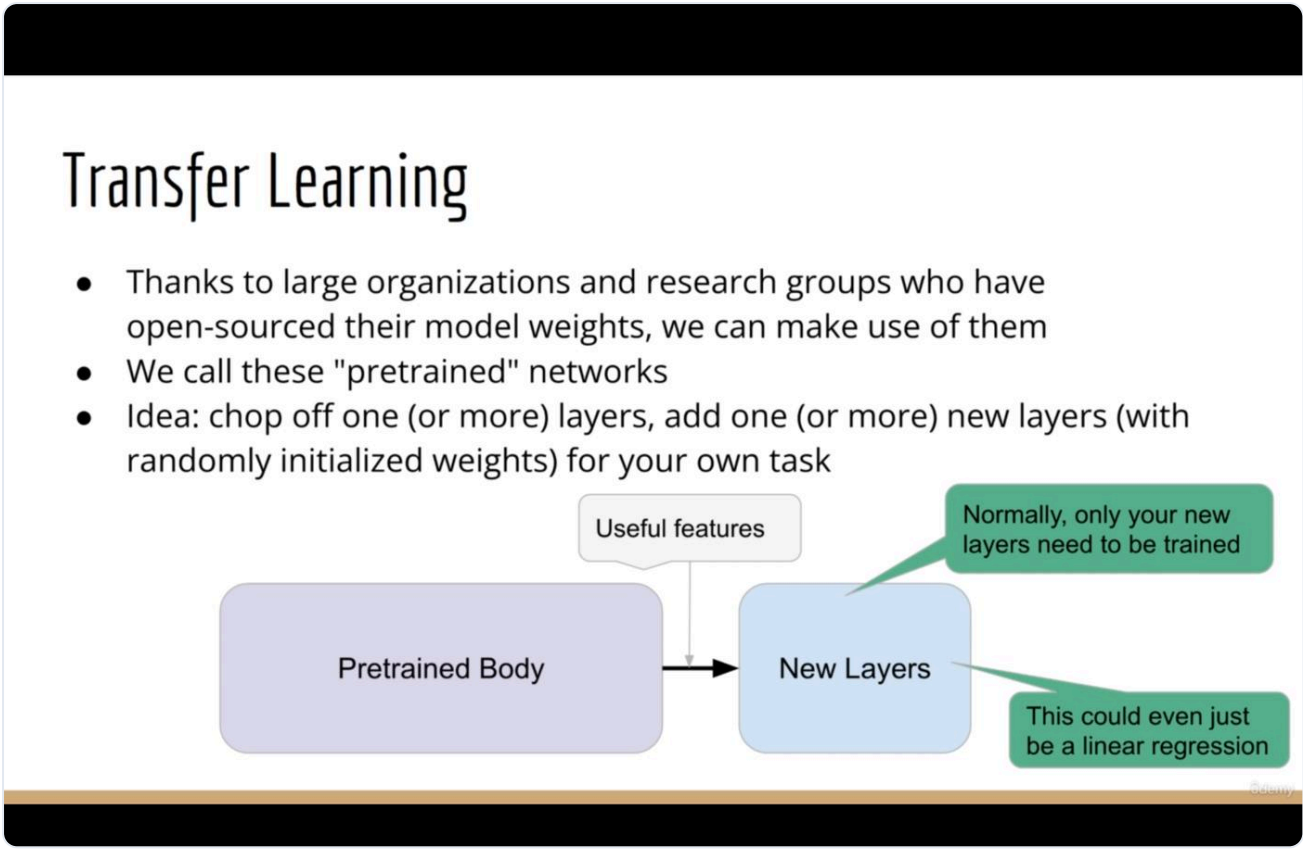

FIGURE 4.63 | Transfer Learning

We then go into Python, instantiate a model like a random forest and we call the fit function on our dataset. Now, where does this CSV file come from? Well, a typical scenario is that we download it from Kaggle or if this is in your workplace, perhaps it was produced by the data engineering team at your company. But one way to think about transfer learning is that this CSV is simply produced by the pre-trained network without its final layer. As you recall, this pre-trained network takes in an image and produces a feature vector.

Thus what we could do is simply pass in our whole data set, compute the corresponding feature vectors and save them as a CSV for later use with other models such as those in scikit-learn. So this is another way to think of transfer learning.

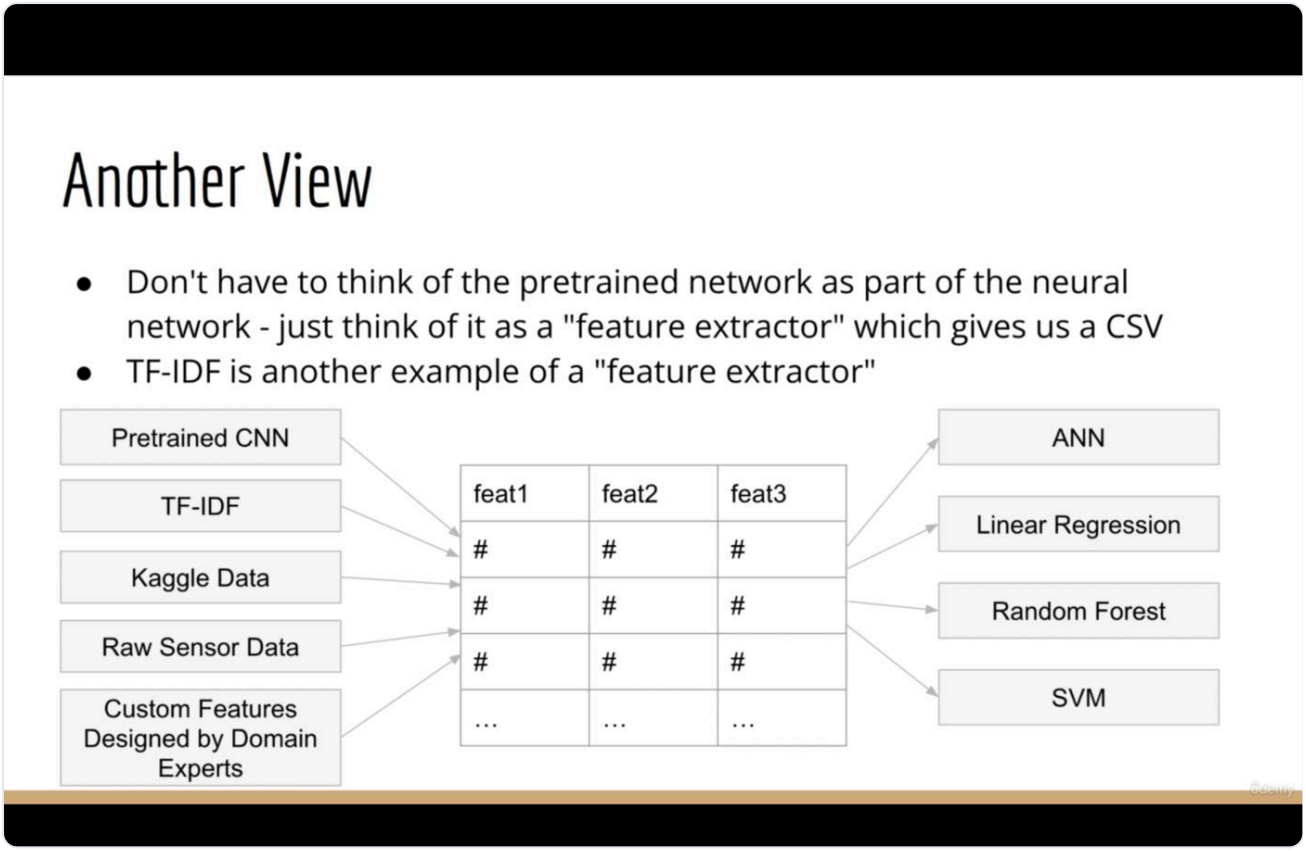

FIGURE 4.64 | Another View

In fact, this exact process is demonstrated in both my TensorFlow two and PyTorch courses. So if you've taken those, then you've seen the actual code to do this. Note that it's also much faster since you don't have to keep passing your data through the pre-trained network.

On each step of gradient descent. That can be slow, especially if the pre-trained network is large, which is the case for many modern neural networks. One important thing to note about transfer learning is that you can really do any kind of task. The only requirement is that your final layers have the right number of outputs, the right activations, and that you use the right loss function so the same requirements with any other neural network. The reason I mention this is to make it clear that we are not limited to just classifying images, which is what the pre-trained network was trained to do.

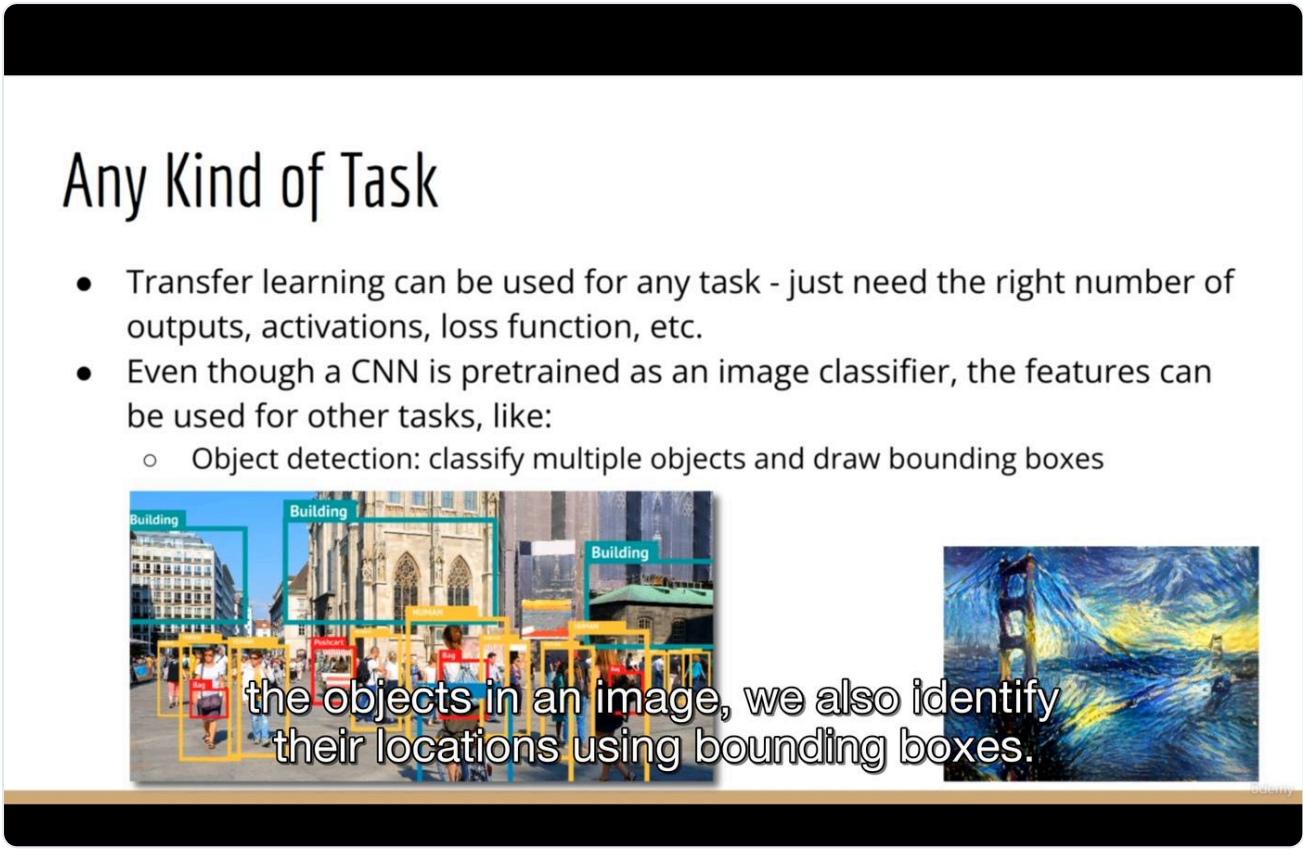

FIGURE 4.65 | Any Kind of Task

Some examples of what we can do with image CNNs include object detection, where not only do we classify the objects in an image, we also identify their locations using bounding boxes. Another task is style transfer, where we take in one image called the content image and another image called the style image.

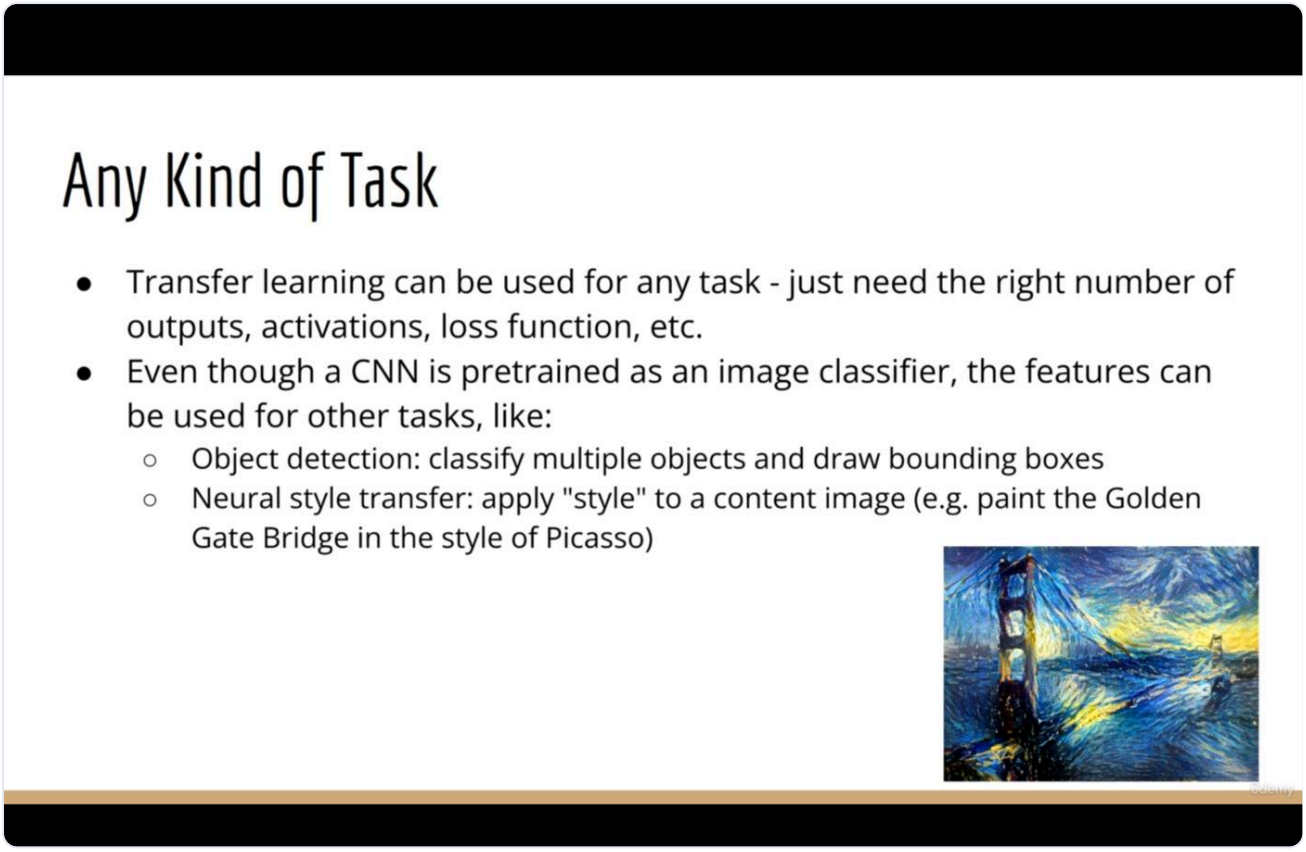

FIGURE 4.66 | Any Kind of Task

We then apply the style from the style image to the content image to produce a new image that has the same global structure as the content image, but with the textures and colors of the style image.

So these are two tasks which go far beyond just classifying images. But using a network that has been pre-trained to classify images turns out to be very useful.

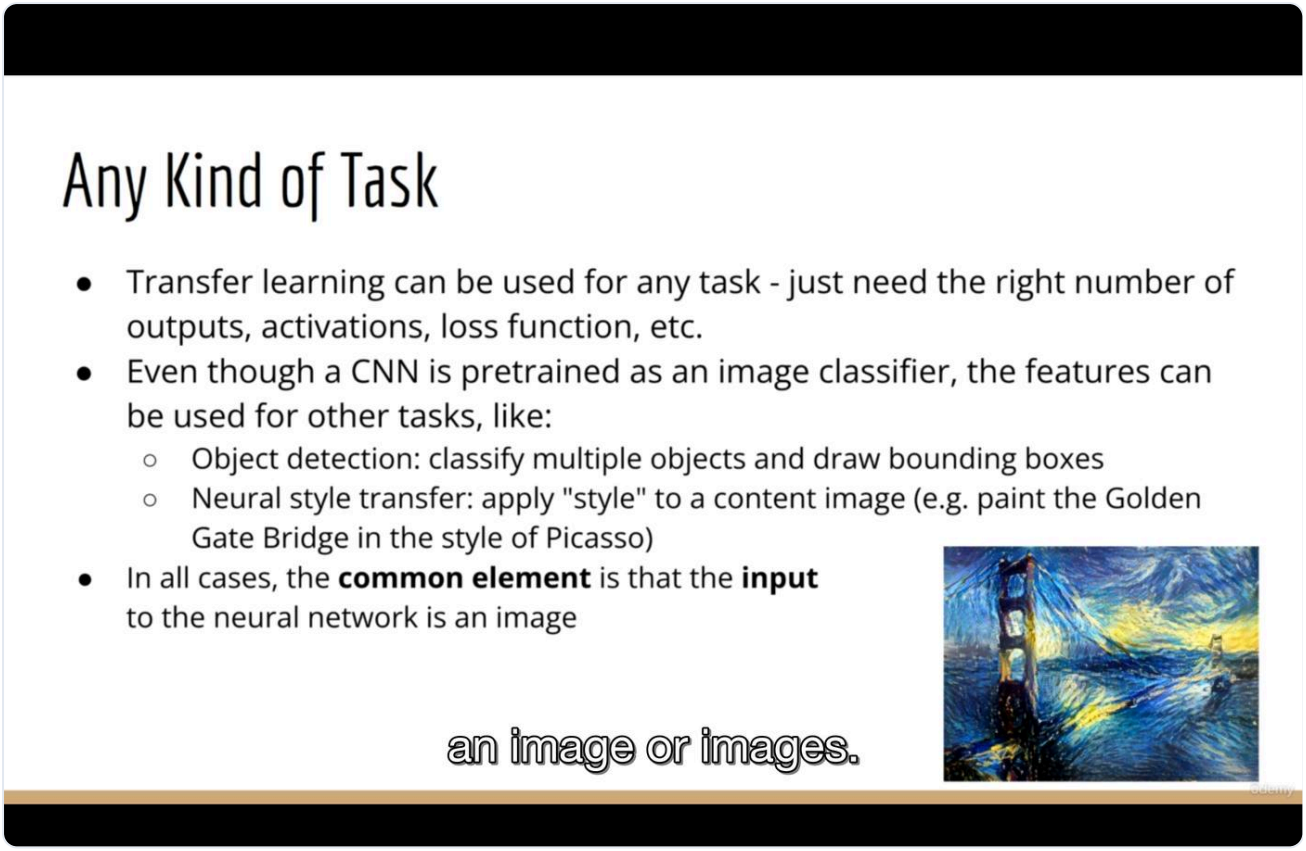

FIGURE 4.67 | Any Kind of Task

In all cases, the common element between all these tasks is that the input to the neural network is an image or images. Okay. So Transfer learning was first used in the field of computer vision. But an obvious question to ask is can this method be applied to language as well? Now it's worth thinking about why these two particular types of data are so important.

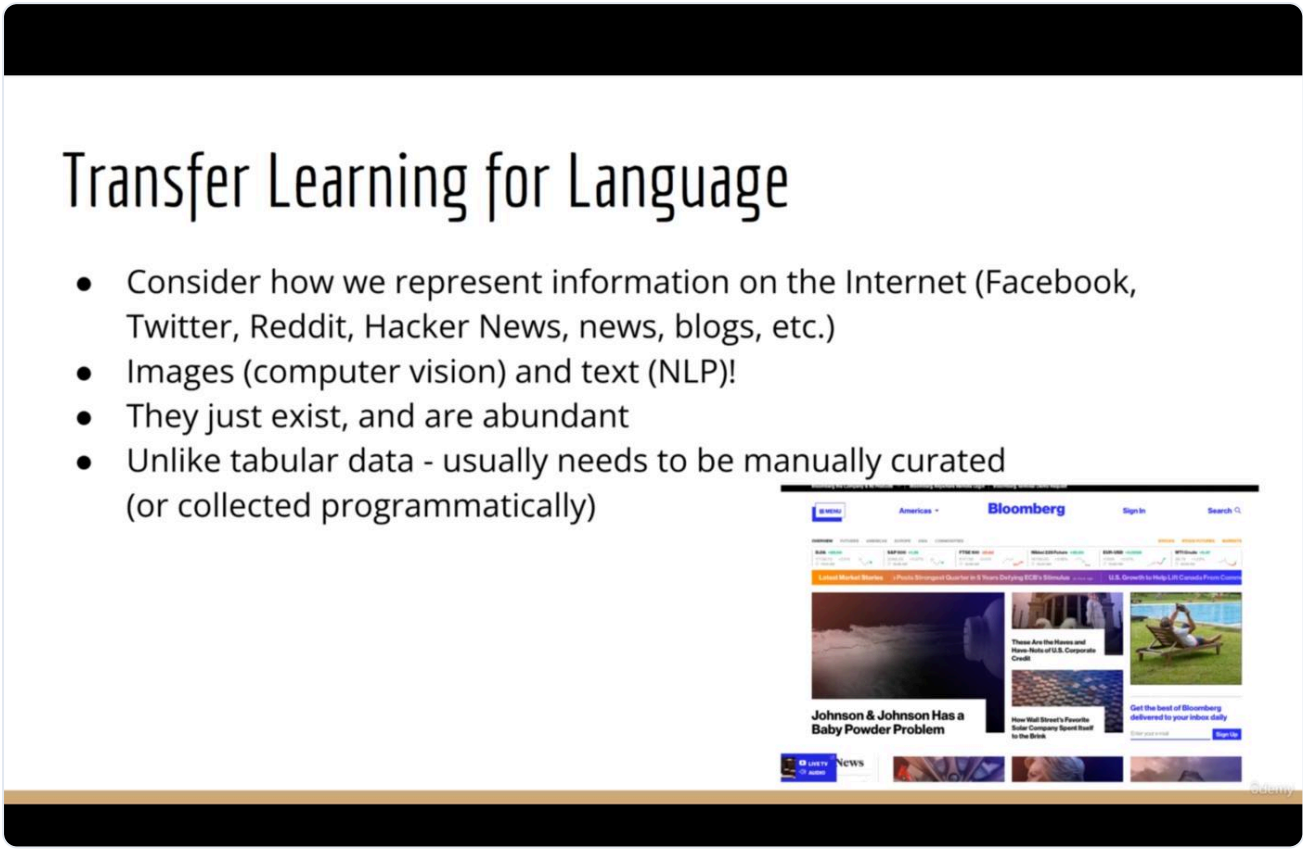

FIGURE 4.68 | Transfer Learning for Language

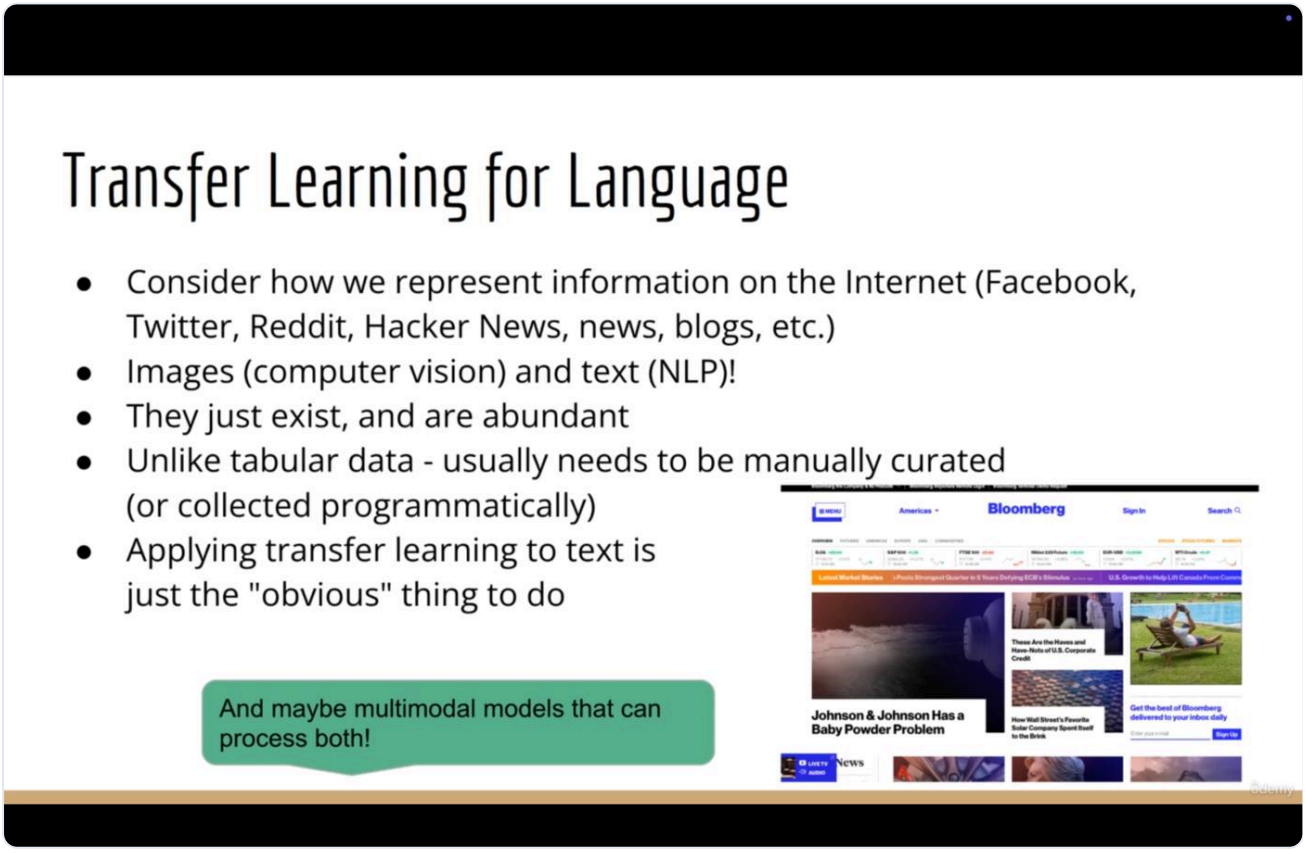

FIGURE 4.69 | Transfer Learning for Language

Consider how we represent information on the Internet. Whether you're on Facebook, Twitter, Reddit, Hacker News, or any other kind of news blogs and so forth, all of this information comes in the form of either images or text. Thus, these are two very important modes of communication. Images and text are how we as humans represent information and how we transfer information from one human to another. These are a kind of natural data because they just exist in the world and they are abundant.

This is unlike tabular data, which is typically curated by a human. We might unnaturally organize it into columns of feature vectors that we've measured or recorded, but images and text are natural. They exist in abundance. And it just so happens that computer vision is the field that deals with images and natural language processing is the field that deals with text. So the point of this is with such an abundant source of data and knowing that this method already works with images, trying to do transfer learning on text is simply an obvious thing to do.

6 · Transfer Learning & Fine-Tuning (pt 2)

So in this lecture we are going to continue our discussion on transfer, learning and fine tuning. The previous lecture we discussed transfer learning and how it's been applied to the field of computer vision and how it might be applied to NLP. But there's another aspect to transfer learning known as Fine Tuning.

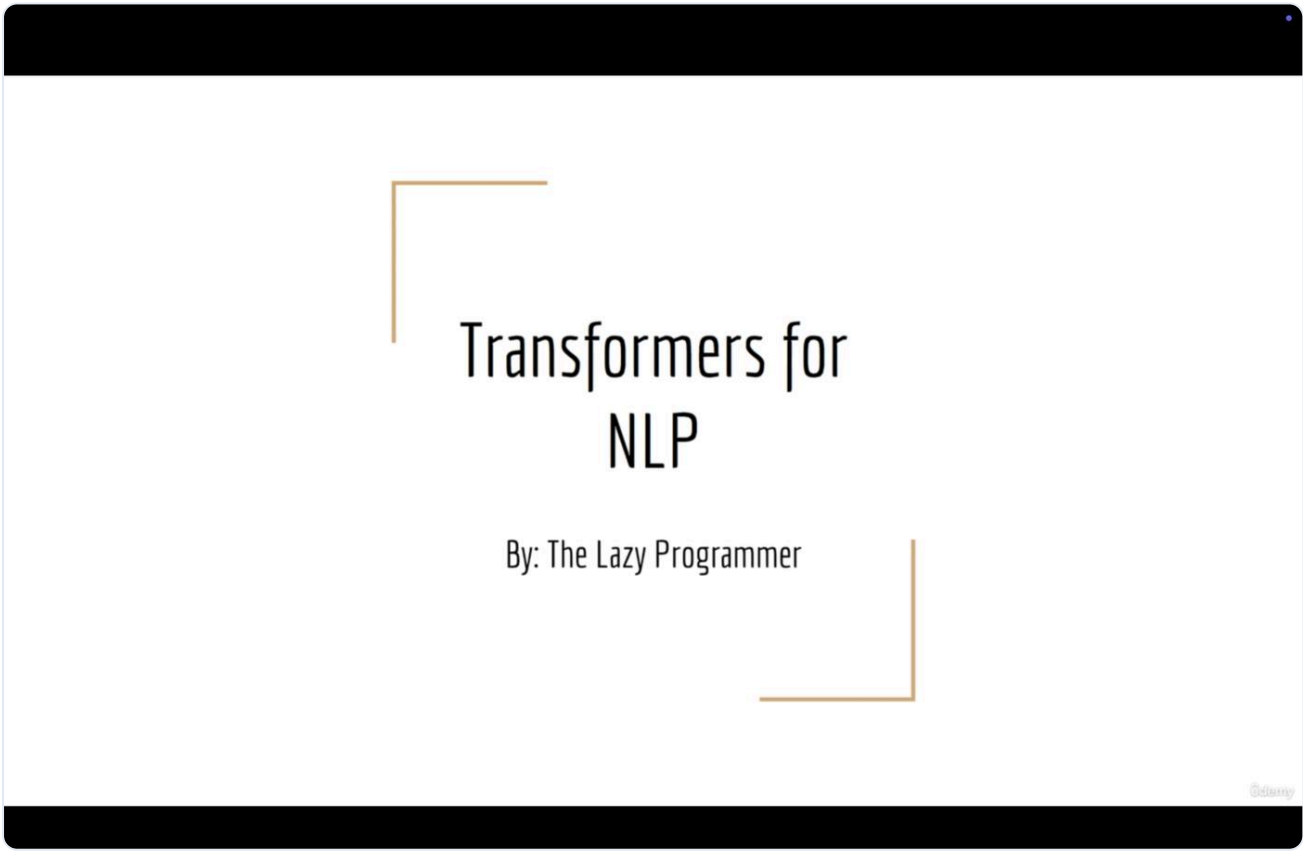

FIGURE 4.70 | Transformers for

Now there's a bit of confusion around the difference between these terms, and I hope that this lecture will clear up any potential confusion you might have.

Transfer learning is a more broad general term, meaning that we take parameters, learns for one task, and we use those same parameters in a some capacity on another task.

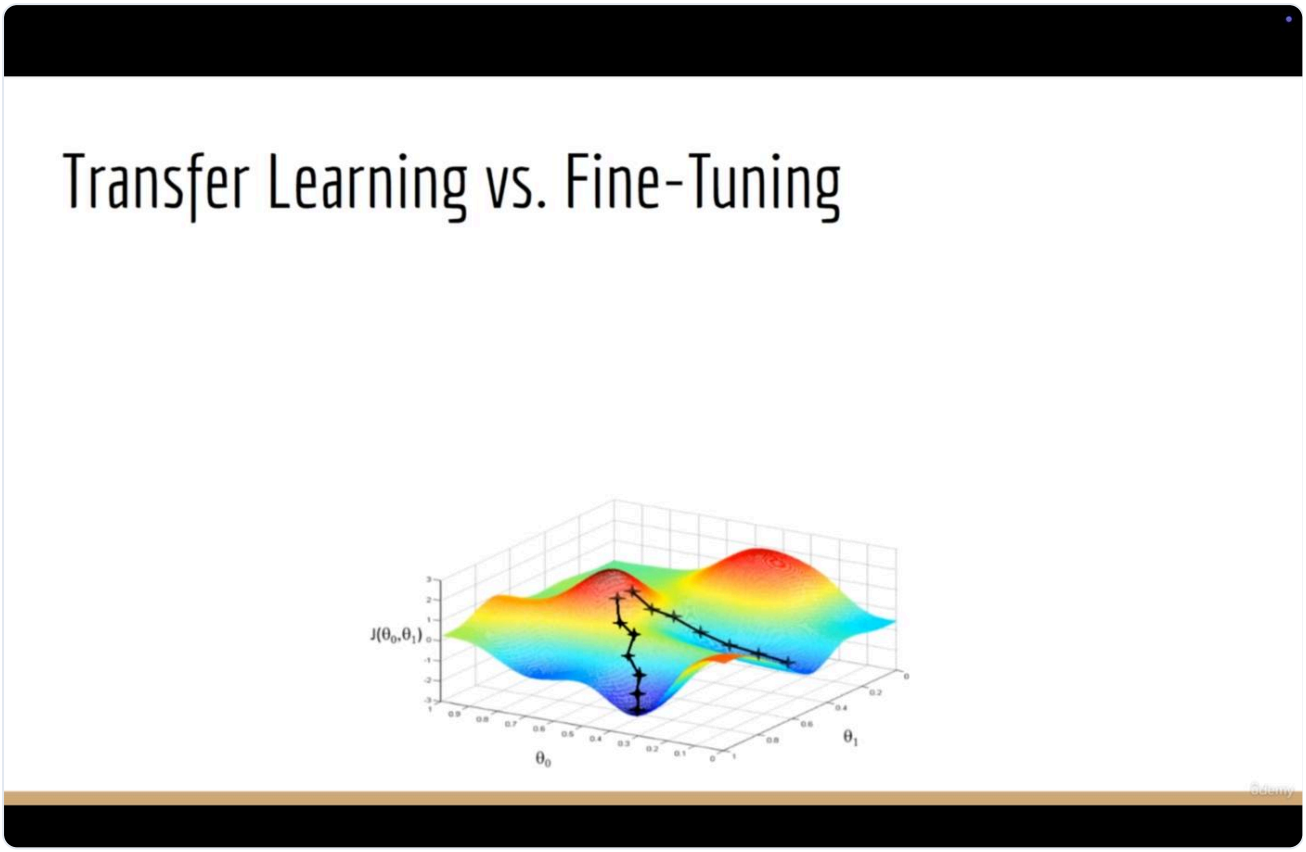

FIGURE 4.71 | Transfer Learning vs. Fine-Tuning

Fine tuning, on the other hand, is a more specific term that describes a particular mechanism one can apply during the process of transfer learning. Specifically, we know that the process of learning is called a gradient descent. We can picture the loss surface in our parameter space and the pre-trained parameters already being in a pretty good spot.

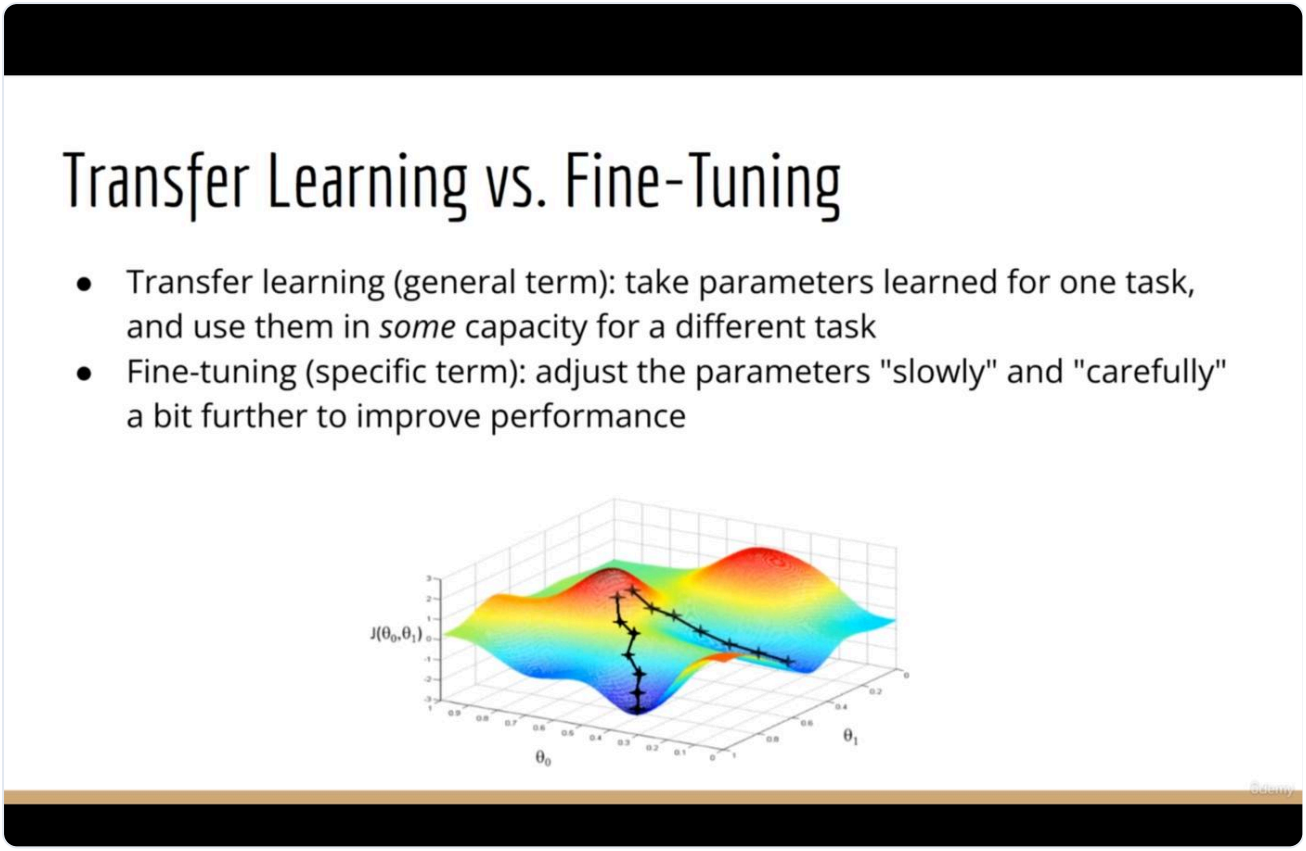

FIGURE 4.72 | Transfer Learning vs. Fine-Tuning

The fine tuning step would be to slowly and carefully tune these parameters a tiny bit further to improve performance on our task. Now, don't worry if that sounds a bit too abstract, since we'll use actual examples in this lecture. Just remember that transfer learning is what we call it. Any time we use the weights from one task on another task, whether those weights will be fine tuned or not. Okay. So let's describe the process of fine tuning as it applies in the field of computer vision.

In computer vision, we start with a big CNN, which is trained on an image classification dataset such as ImageNet. Using this big CNN. We then chop off the final dense layers and we add new, untrained, final dense layers for our new task. The next step is to train these new layers on our own dataset.

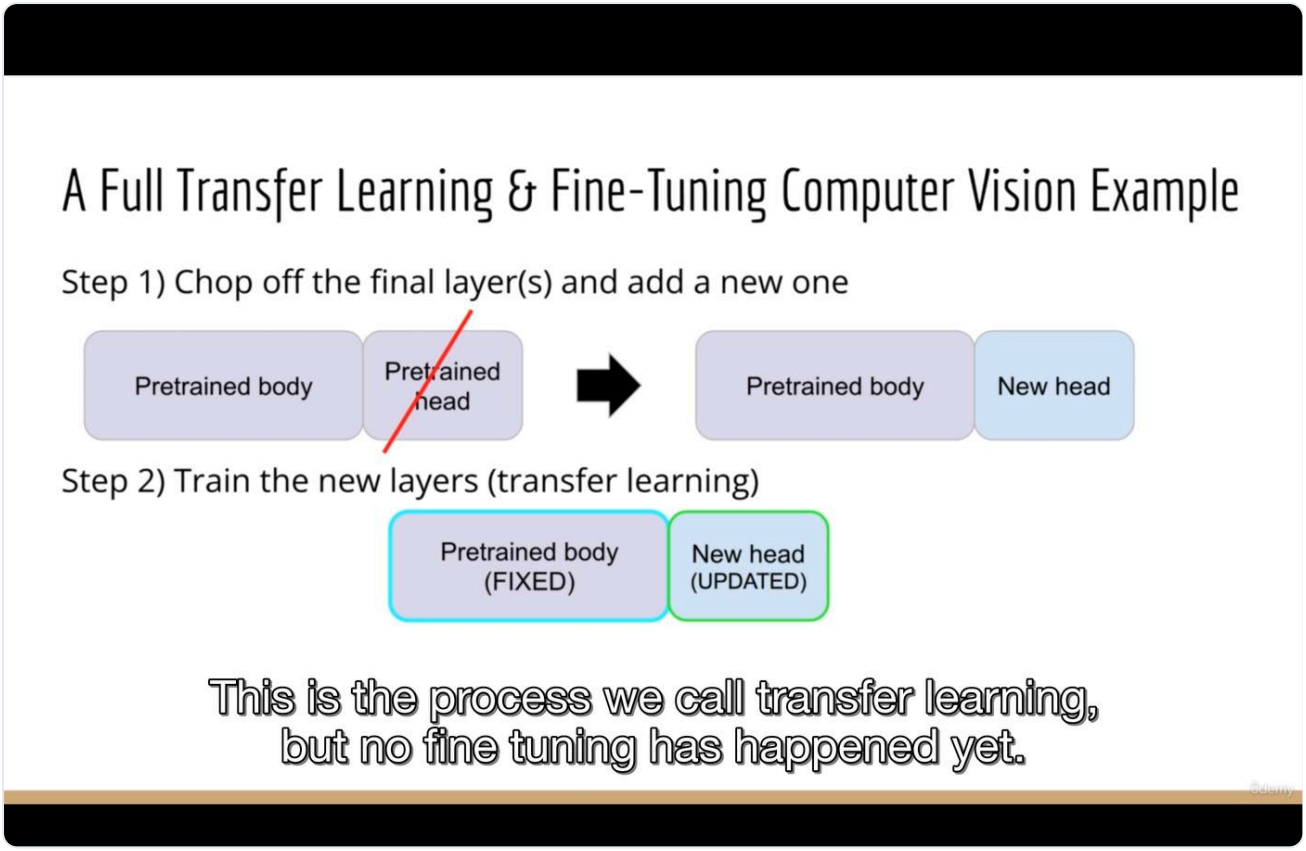

FIGURE 4.73 | A Full Transfer Learning & Fine-Tuning Computer Vision Example

This is the process we call transfer learning, but no fine tuning has happened yet. Note that during this process only the new layers were trained, the pre-trained weights remained fixed.

But after this there is an optional next step, which is fine tuning. During this fine tuning step, we unfreeze the whole network and we train all the layers, typically with a very small learning rate. By small, I mean small in comparison to the learning rate we were using before.

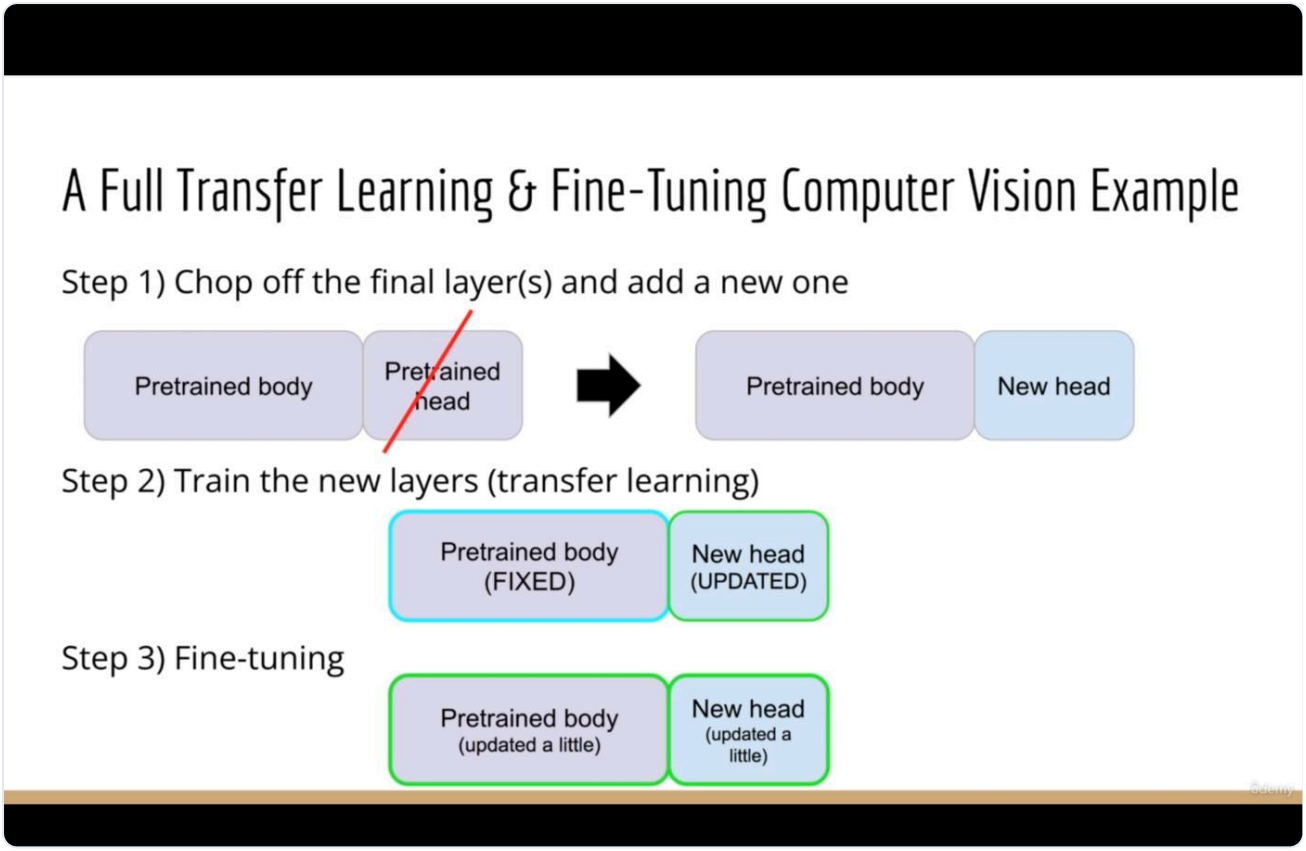

FIGURE 4.74 | A Full Transfer Learning & Fine-Tuning Computer Vision Example

Remember that the weights are already in a pretty good place, so we only want to take very tiny steps to increase performance by a little bit. If our steps are too large, we might end up in the totally wrong place, which is not good.

Another potential concern is that too much fine tuning could lead your model to over fit. Now please note that there are many options for this fine tuning step.

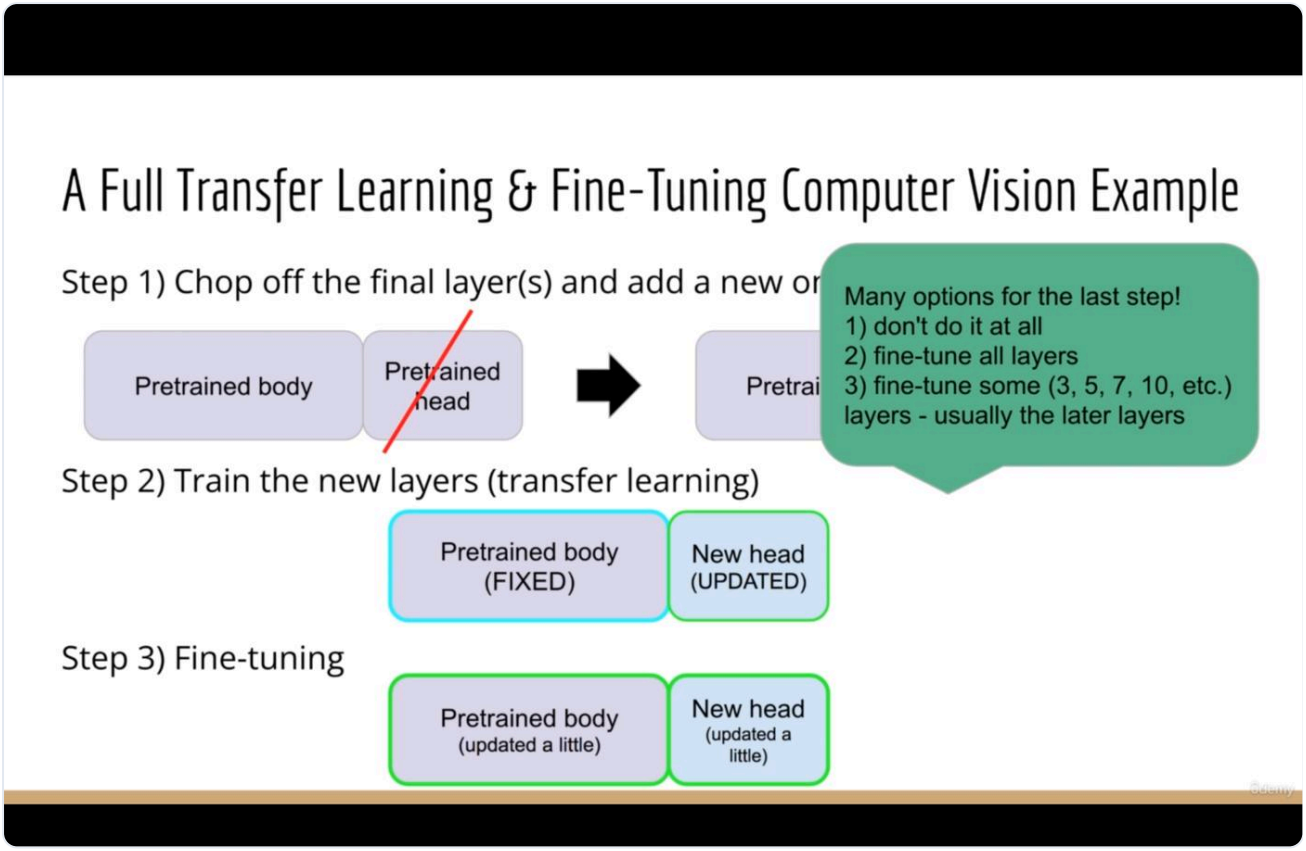

FIGURE 4.75 | A Full Transfer Learning & Fine-Tuning Computer Vision Example

It's really up to you to experiment and determine what is right for you. For some data sets, you may only need to fine tune the last few layers maybe five layers, seven layers or ten layers. Usually the first few layers don't need to change them much, if at all, since they represent very generic features like edges and blobs.

Now, the process I just described for fine tuning is, in my opinion, a bit too specific. The best way to really understand deep learning and neural networks is to understand one of the first uses of fine tuning, and that was with a method called greedy layer wise pre-training.

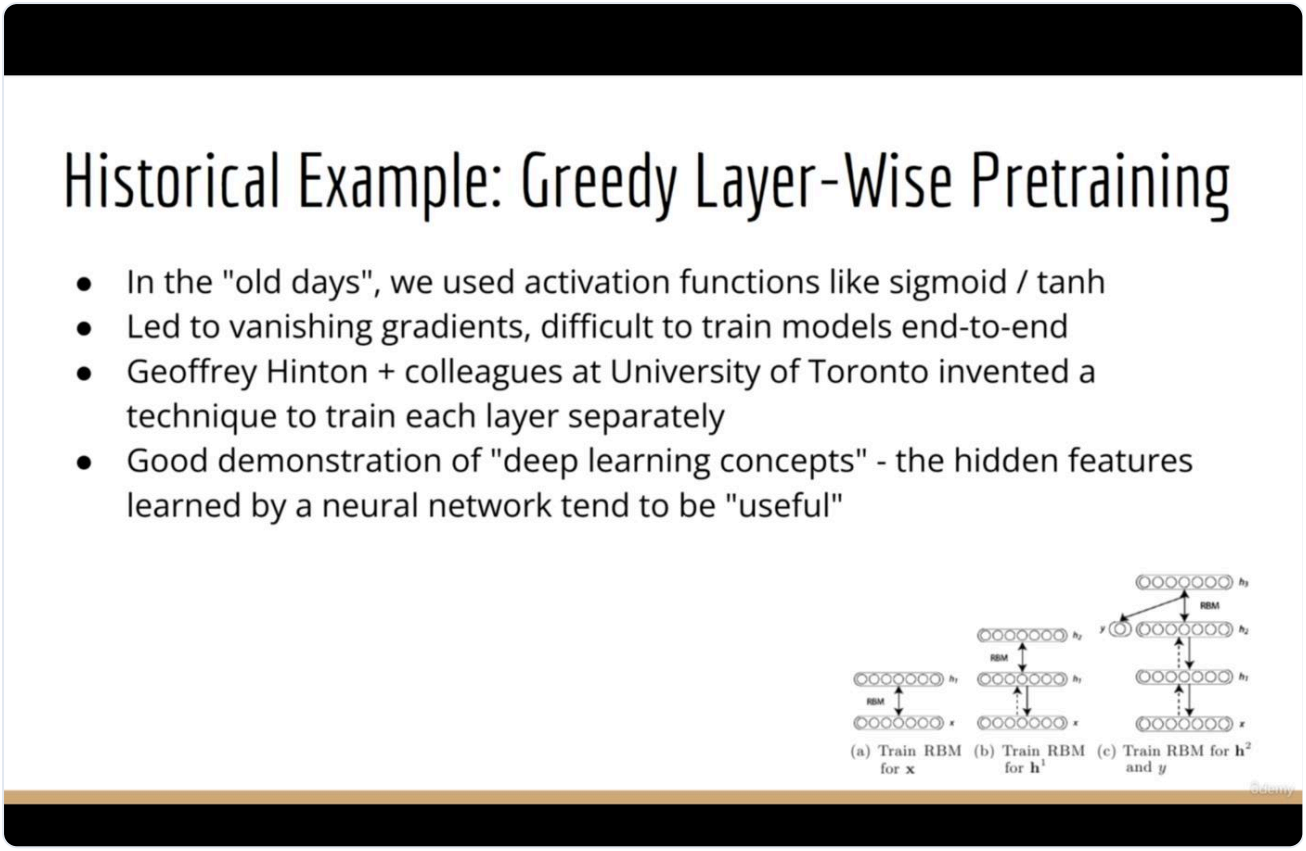

FIGURE 4.76 | Led to Vanishing Gradients, Difficult to Train Models End-to-end

In the old days, we use activation functions like the sigmoid and the hyperbolic tangent which led to vanishing gradients, and as such it was difficult to train neural networks in an end to end fashion like we do today.

One technique invented by Geoffrey Hinton and his students at the University of Toronto was to train each layer separately, one after another.

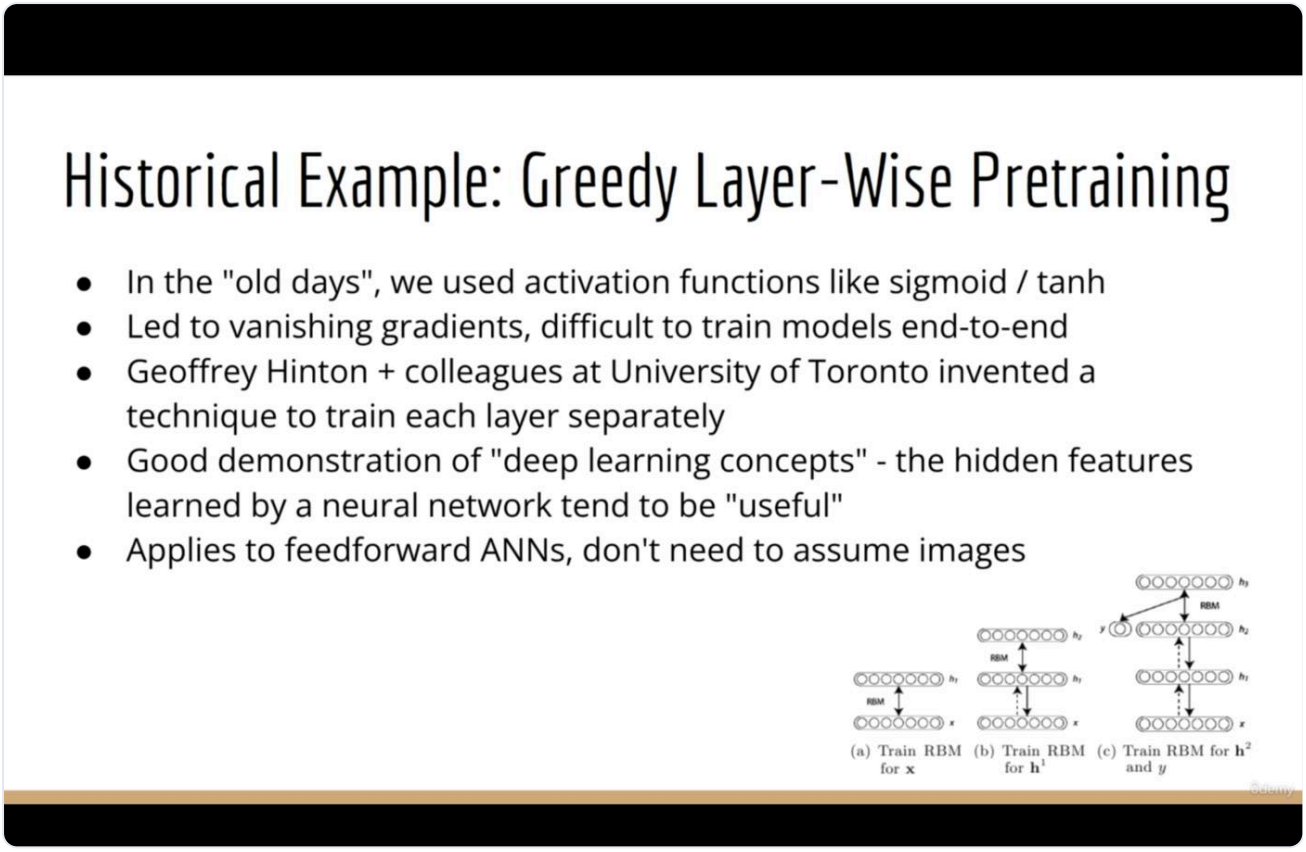

FIGURE 4.77

This is a really great way to understand how neural networks work, because it demonstrates how the hidden features learned by a neural network always tend to be useful by training a neural network to do some task, whatever that task may be. Somehow, it always magically turns out that the intermediate representations learned by the neural network have some useful property.

This method is also nice because it applies to regular feedforward NNs. So we don't have to assume we're working with images. In any case, the process works like this. Suppose that we have some data set and we train an auto encoder on it. As you recall, an auto encoder is a neural network that simply learns to output its own input. But as mentioned, neural networks tend to learn the useful intermediate representations.

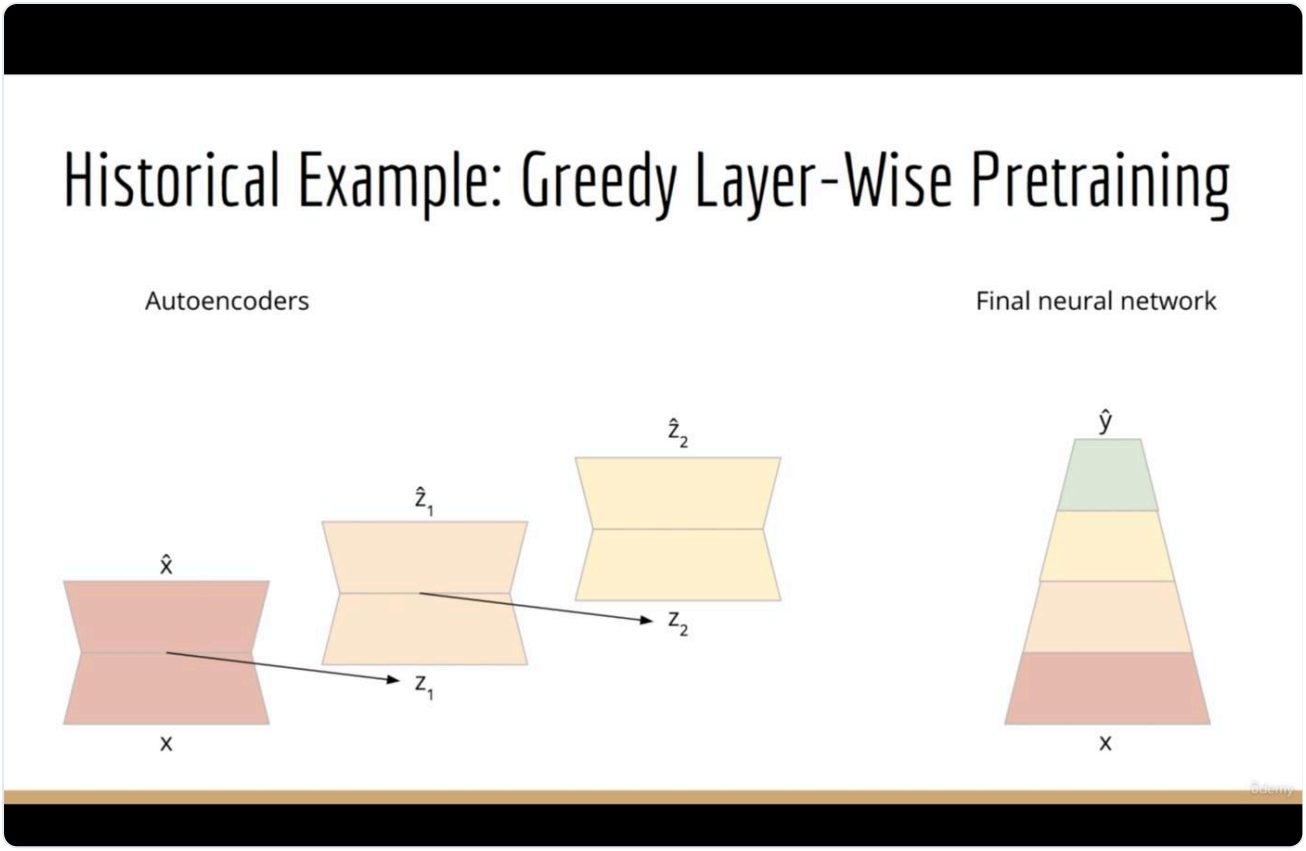

FIGURE 4.78 | Autoencoders Final Neural Network

So suppose that we take the intermediate representation of this first auto encoder, call it Z one, and throw away the second half of the network.

What we then do is train another auto encoder, but this time on Z1 instead of the original inputs x. Let's call the intermediate representation of the second auto encoder Z2. Again, we throw away the second half of the auto encoder and repeat the process training a new auto encoder on Z2. What we're effectively doing is we're transforming the input again and again into new higher level representations each time, since each transformation is just a neural network layer.

The whole chain just gives us a big multi-layer neural network. So this multi-layer neural network is made up of the first halves of each of the auto encoders. Eventually we can stop adding new auto encoders and add one final dense layer specific to our task.

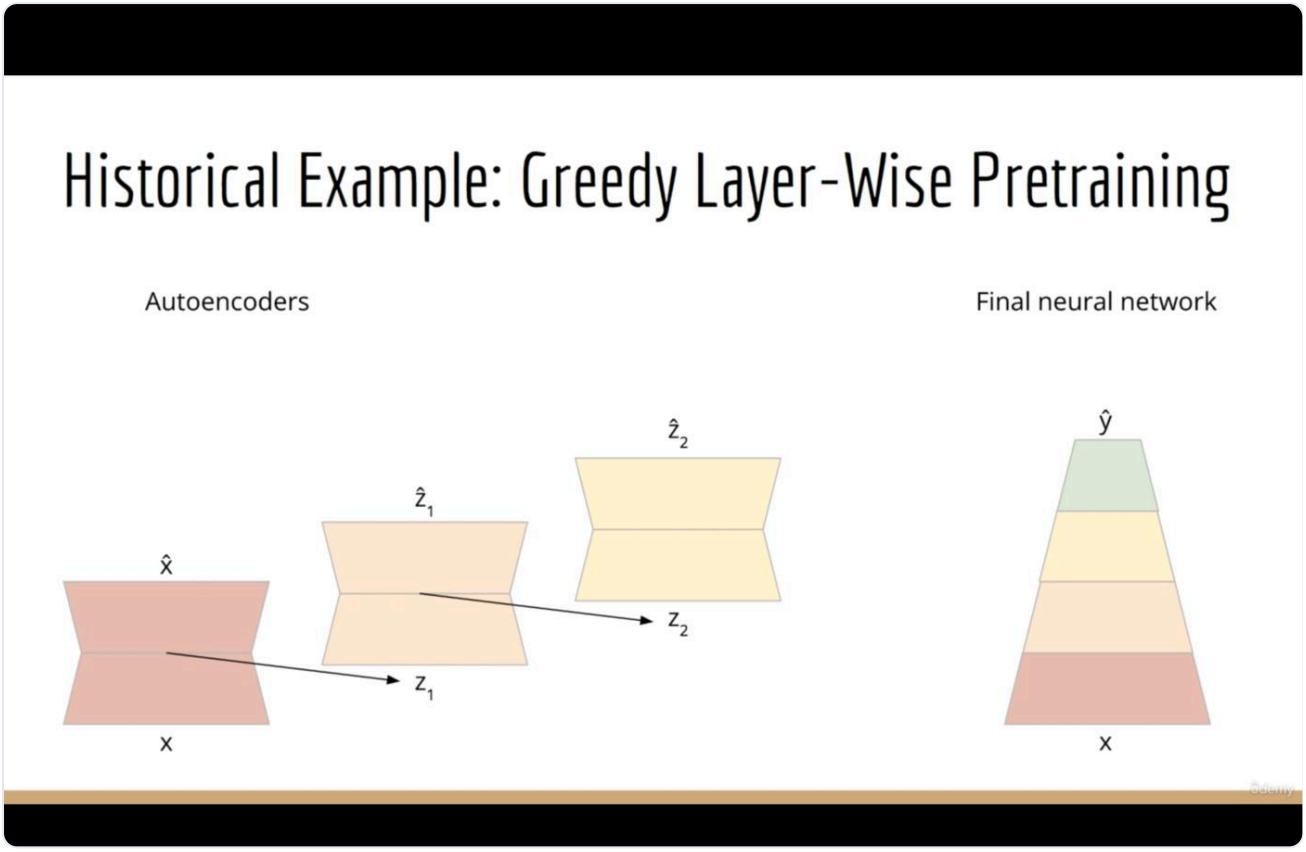

FIGURE 4.79 | Autoencoders Final Neural Network

We then train this final dense layer and all the previous auto encoders, plus the final dense layer, make up our full neural network. The key result was that this neural network performed better than a neural network that was trained end to end.

This process is called greedy, layer wise pre-training, which hopefully makes sense now that you understand the process. So where does fine tuning come into play? Well, it works just like how I described earlier in this lecture. After the greedy, layer wise pre-training process, which only trained one layer at a time, we then take the whole neural network and train the whole thing all at once. But we do this with a very small learning rate, since our parameters are already in a good spot.

We just want to make them slightly better.

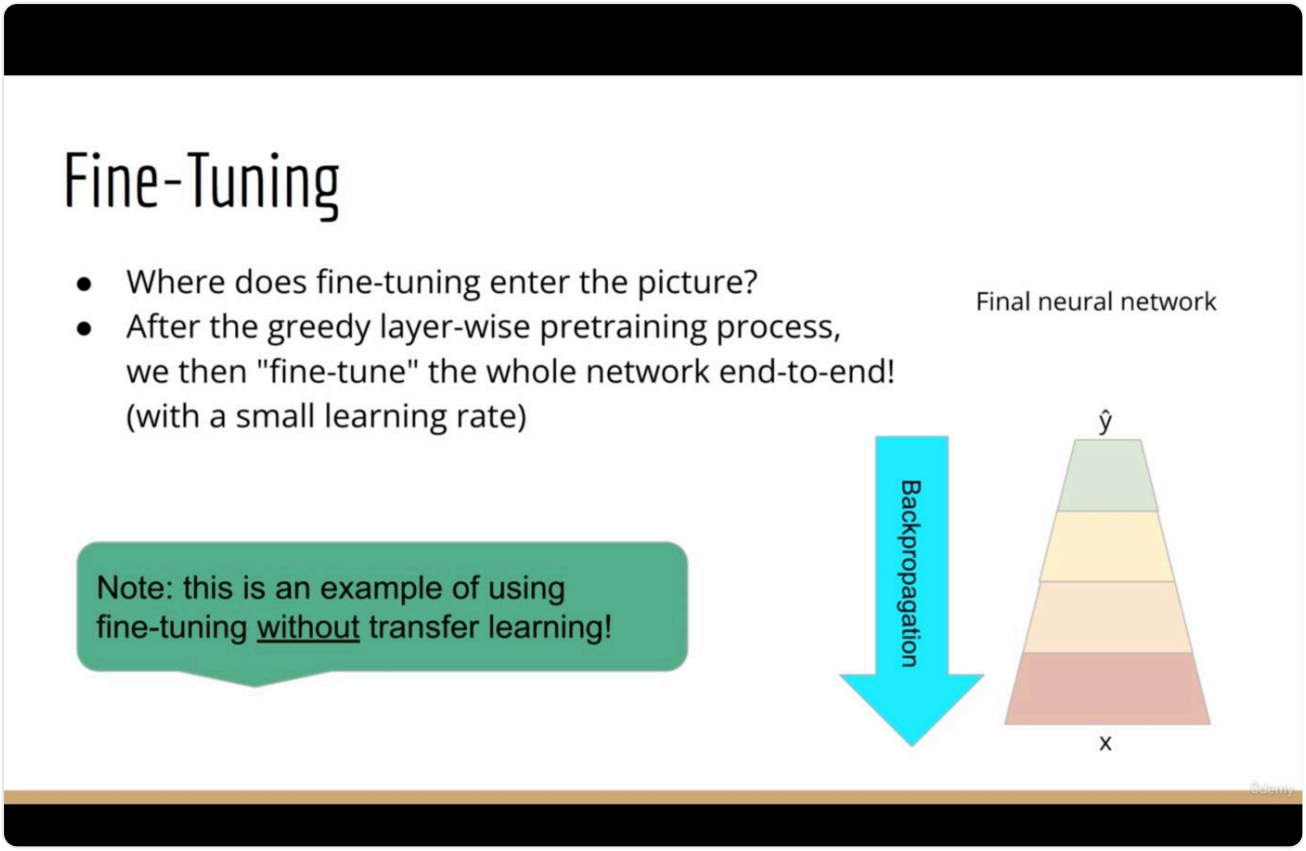

FIGURE 4.80 | Where Does Fine-tuning Enter the Picture?

Now, to be clear about why we are discussing this example, this is an example of where we can have fine tuning but without any use of transfer learning. So while what we are doing in this course might make it seem like these two concepts are conflated, they are in fact distinct. So one purpose of the previous examples was to demonstrate that there are multiple scenarios where one might apply fine tuning. Note that one scenario involves transfer learning while the other did not.

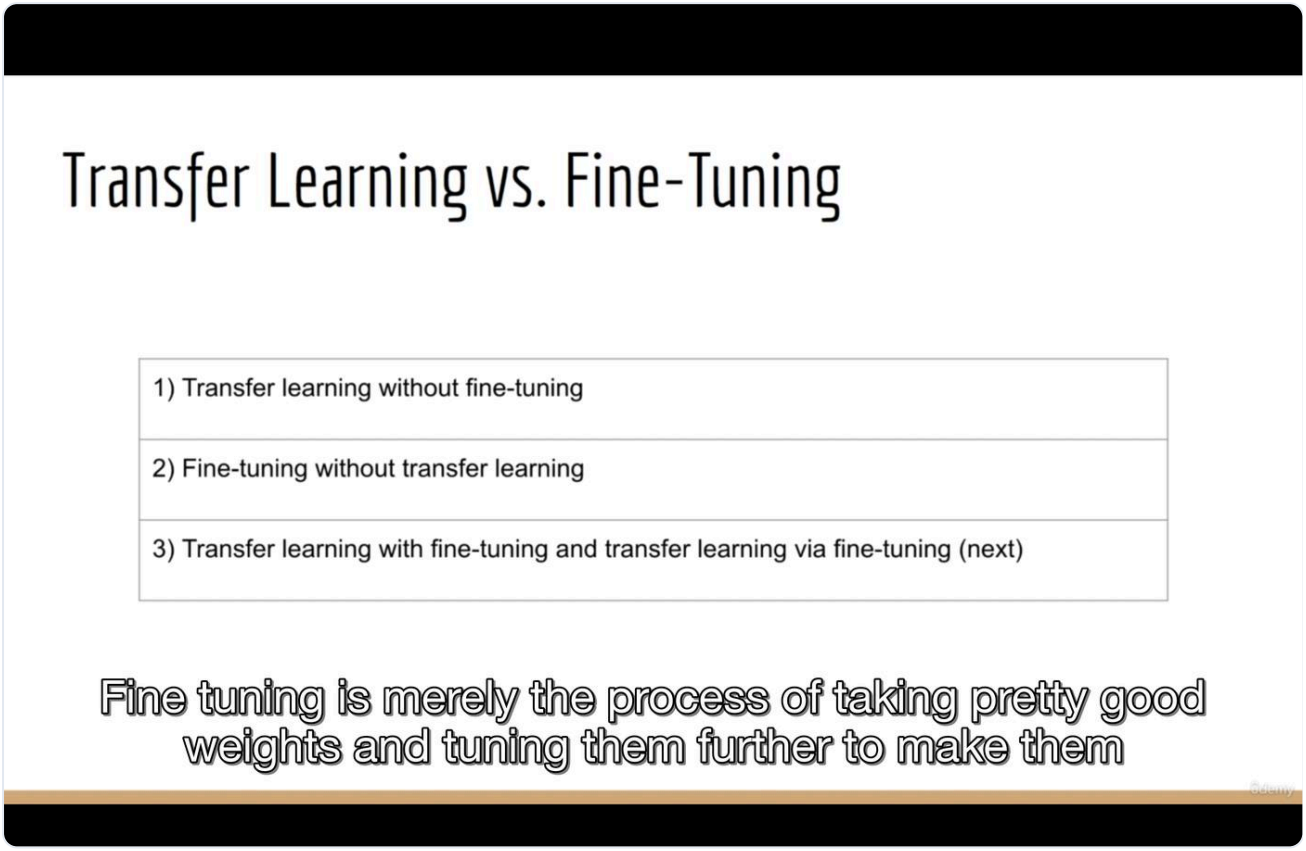

FIGURE 4.81 | Transfer Learning vs. Fine-Tuning

Fine tuning is merely the process of taking pretty good weights and tuning them further to make them even better. Understanding this, we can now discuss what fine tuning will look like for Transformers. So for transformers in the field of NLP, the first part of the process looks essentially the same. We begin by training a transformer on some NLP task, which is to be discussed. We then chop off the last layers and add new final layers for our new task.

At this point, the body has trained weights, but the head does not. But instead of how we do things in computer vision where we train only the head first, the library will be using simply trains, all the weights at the same time.

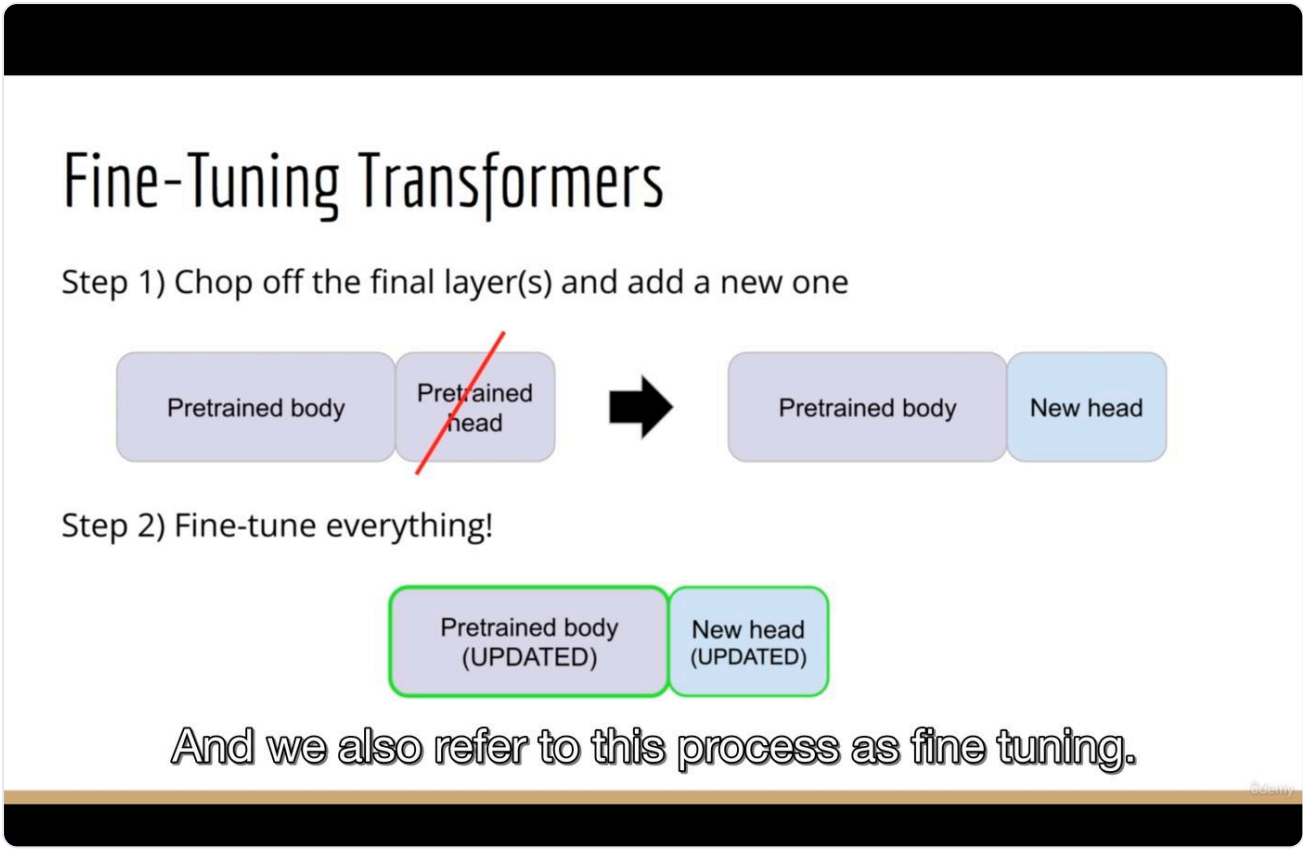

FIGURE 4.82

And we also refer to this process as fine tuning. Note that this somewhat contradicts our earlier definition of fine tuning. Technically, not all the weights are in a pretty good place when we start this process, because the new layers have totally random weights.

However, this method seems to work well enough. Interestingly, training only the head doesn't seem to work as well. You're encouraged to try it yourself, to confirm.

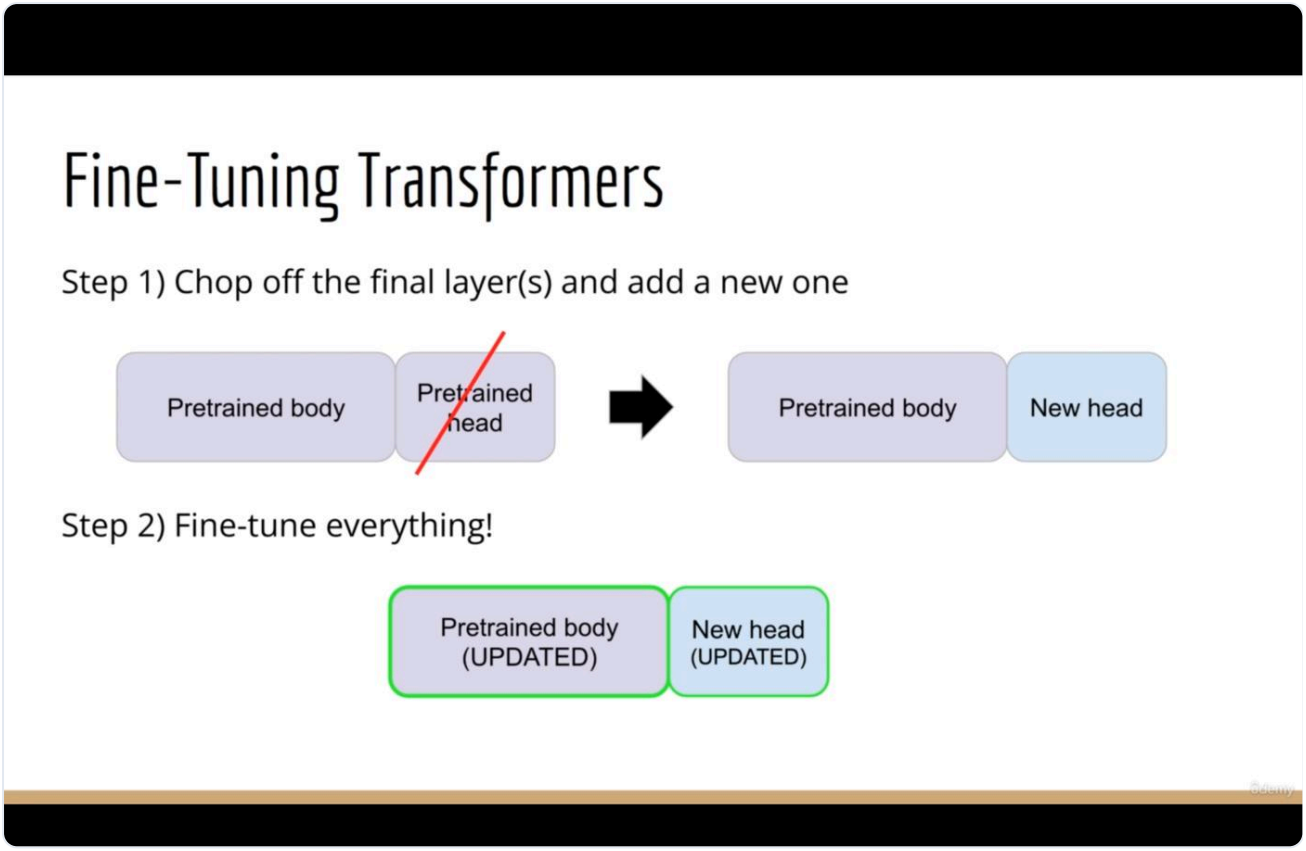

FIGURE 4.83

Finally, it's good to be aware that none of this is set in stone. There are no rules which state that you have to do it this way or you have to do it that way. Perhaps it's the case that freezing the first few layers works well. Perhaps it's the case that freezing the whole pre-trained network works well.

Perhaps it's the case that first training the new layers and then fine tuning as we do in computer vision works well.

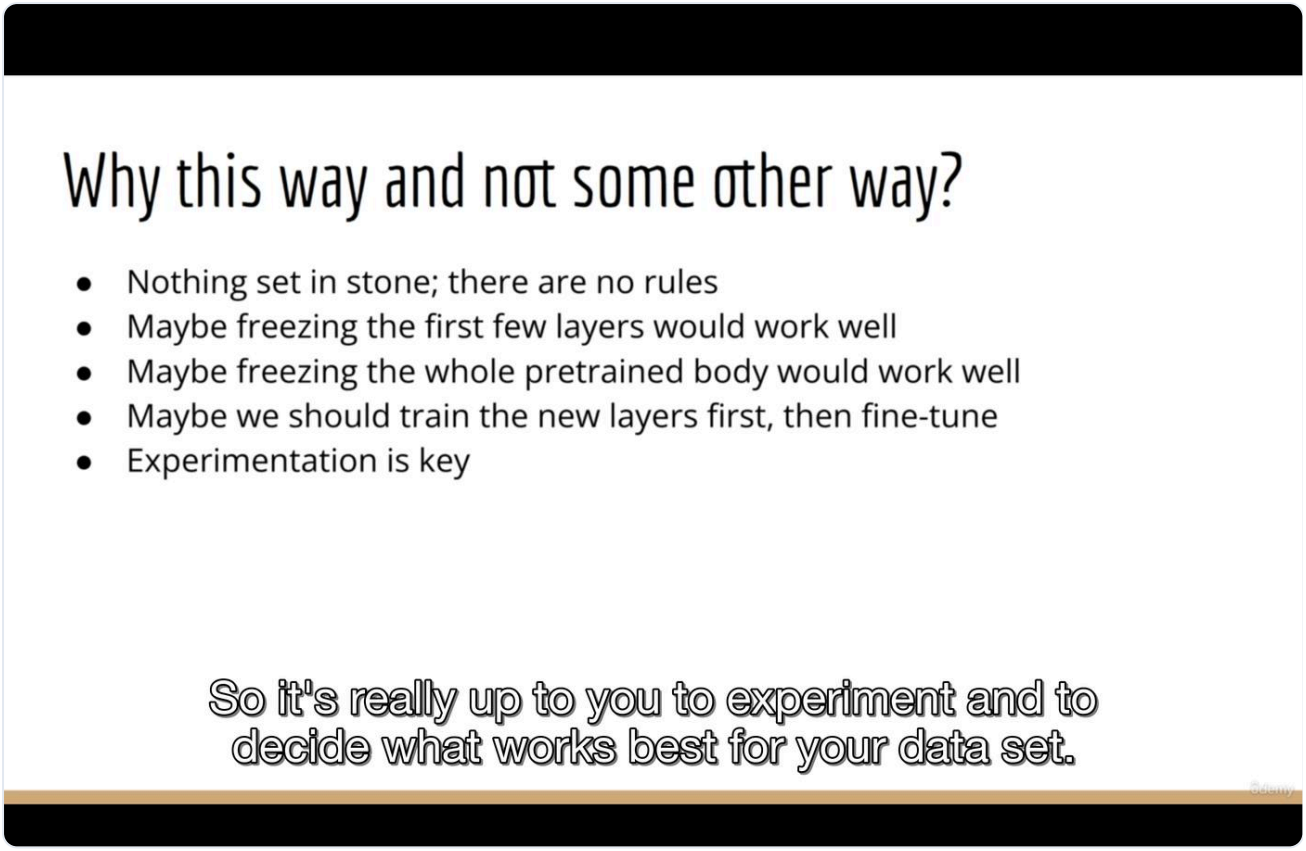

FIGURE 4.84 | Why This Way and Not Some Other Way?

So it's really up to you to experiment and to decide what works best for your data set.

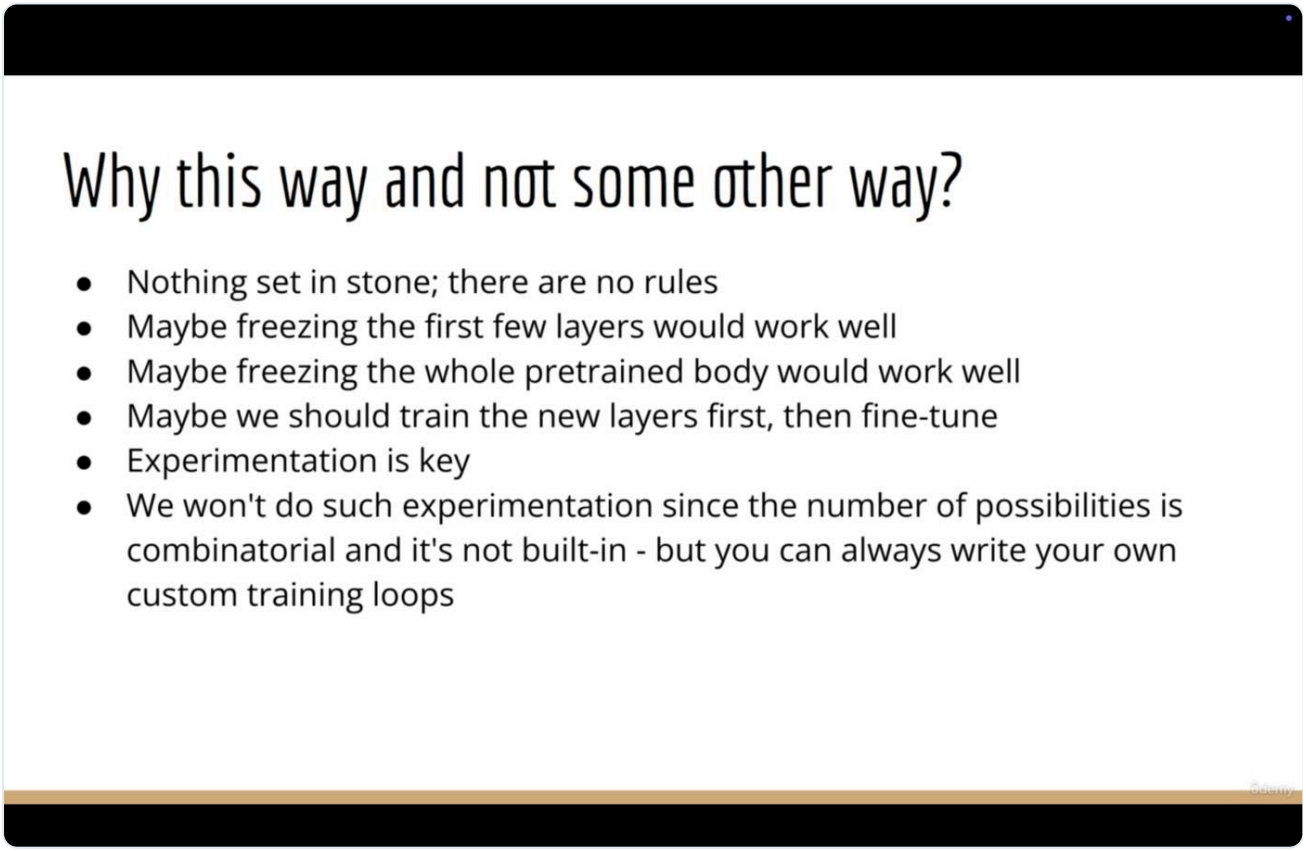

FIGURE 4.85 | Why This Way and Not Some Other Way?

For this course, we won't do this experimentation because the number of possibilities is combinatorial and none of these alternative options are built in. But if you have something you want to try, you are strongly encouraged to do so. Now you don't often get updates to a course before the course is even released, but this is a rare example.

After I made this lecture, I came across a recent journal article that actually came out while I was making this course. As I always mentioned in these courses, get used to the fact that things move fast.

FIGURE 4.86

The paper is titled BitFit: Simple Parameter-efficient Fine-tuning for Transformer-based Masked Language-models. You can find it in extra_reading.txt as usual. So this paper was last revised only a week prior to me making this update.

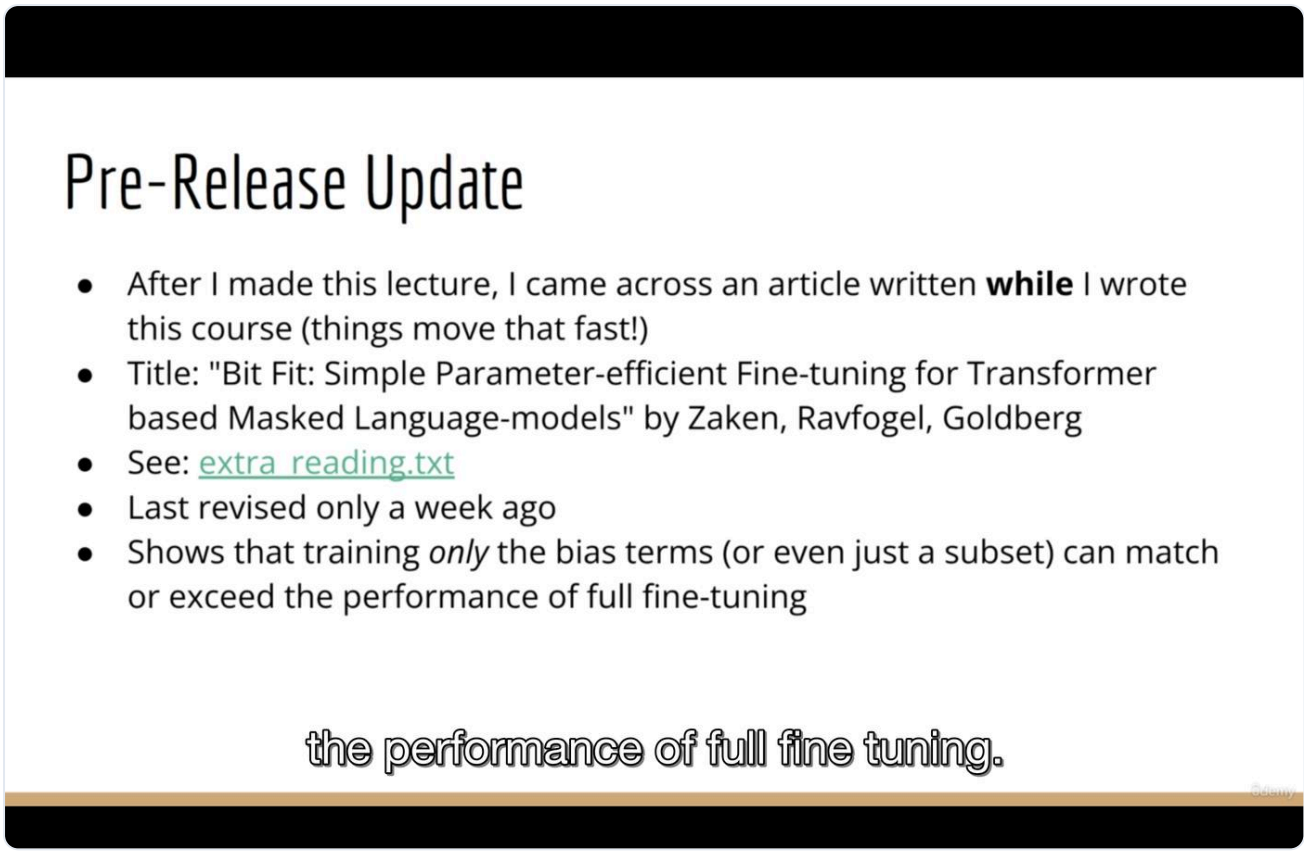

FIGURE 4.87

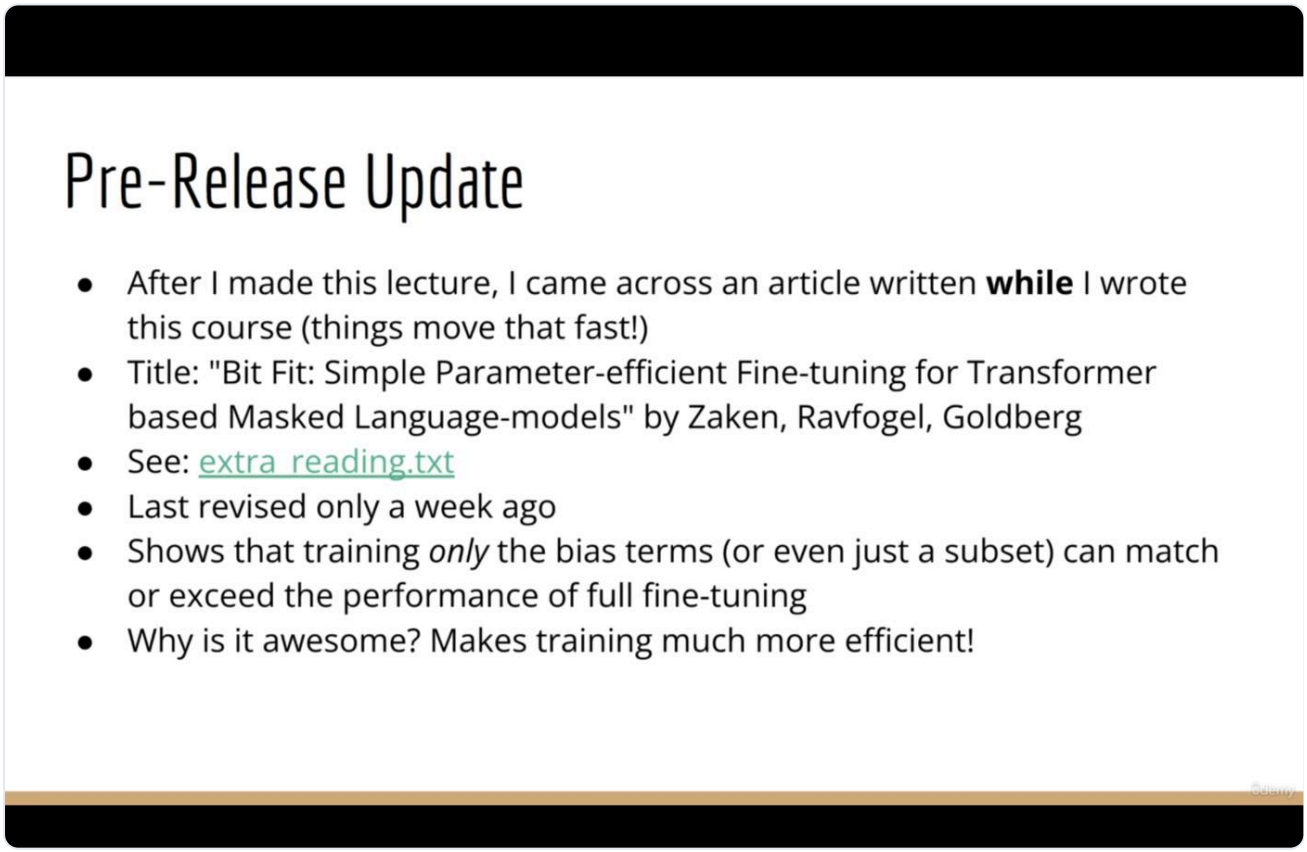

FIGURE 4.88

The paper shows that training only the bias terms or even just a subset of the bias terms can match the performance of full fine tuning.

And in some cases, performance can even be improved. It should go without saying why this is an awesome development. The bias terms make up only a small portion of the total number of parameters. This lets us avoid having to make expensive matrix computations, which will make training much more efficient. So this is a great example for those who just want to memorize the rules of thumb and never explore an experiment by themselves. If you want to make great discoveries like this, you must explore and you must experiment.

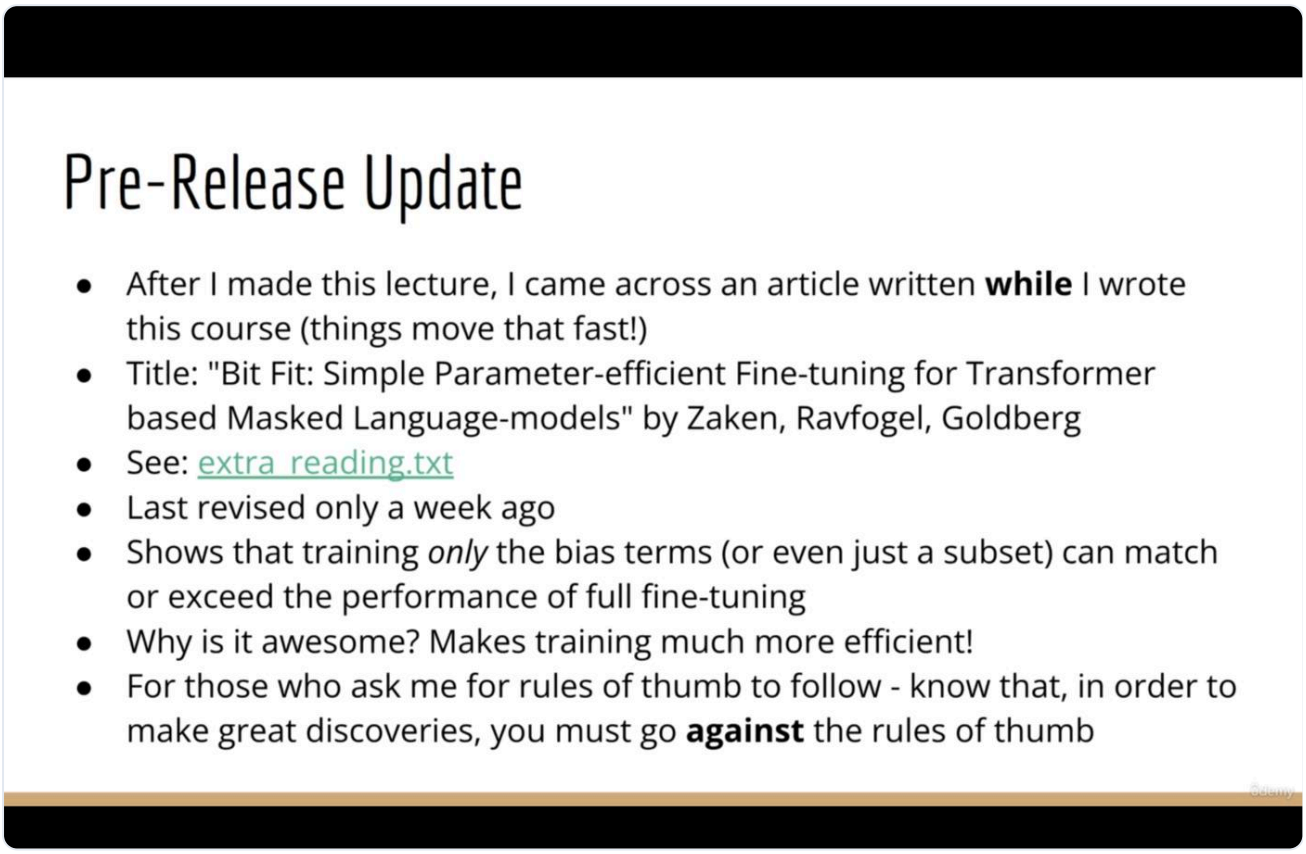

FIGURE 4.89

And you must go against the rules of thumb.

7 · Transfer Learning & Fine-Tuning (pt 3)

So with computer vision, the way we pre-trained our neural networks is by classifying images. Recall that this is supervised learning because it requires us to provide label data for each image. We must also specify which class that image belongs to.

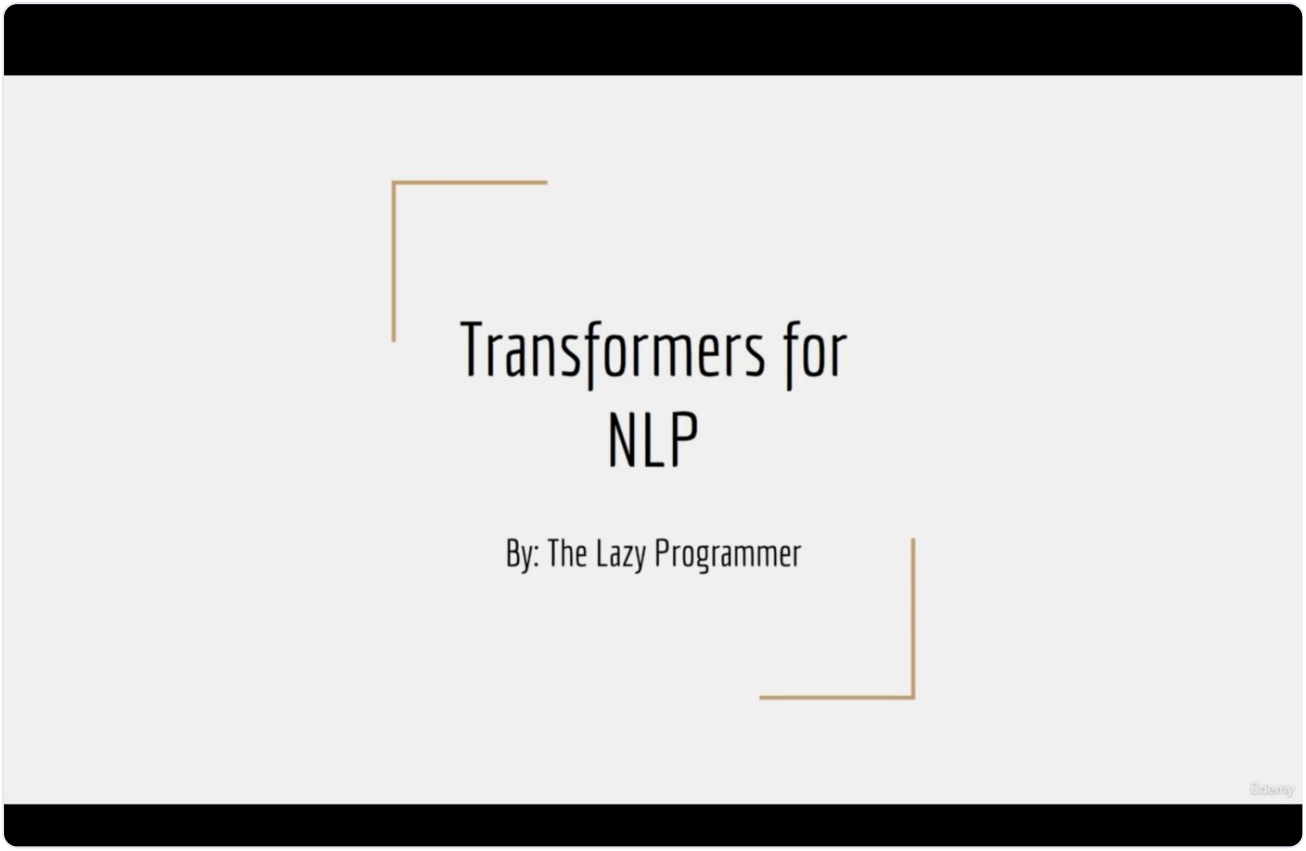

FIGURE 4.90 | Transformers for

The question is what kind of task should we use for text? Well, it turns out that supervised learning is not an ideal choice.

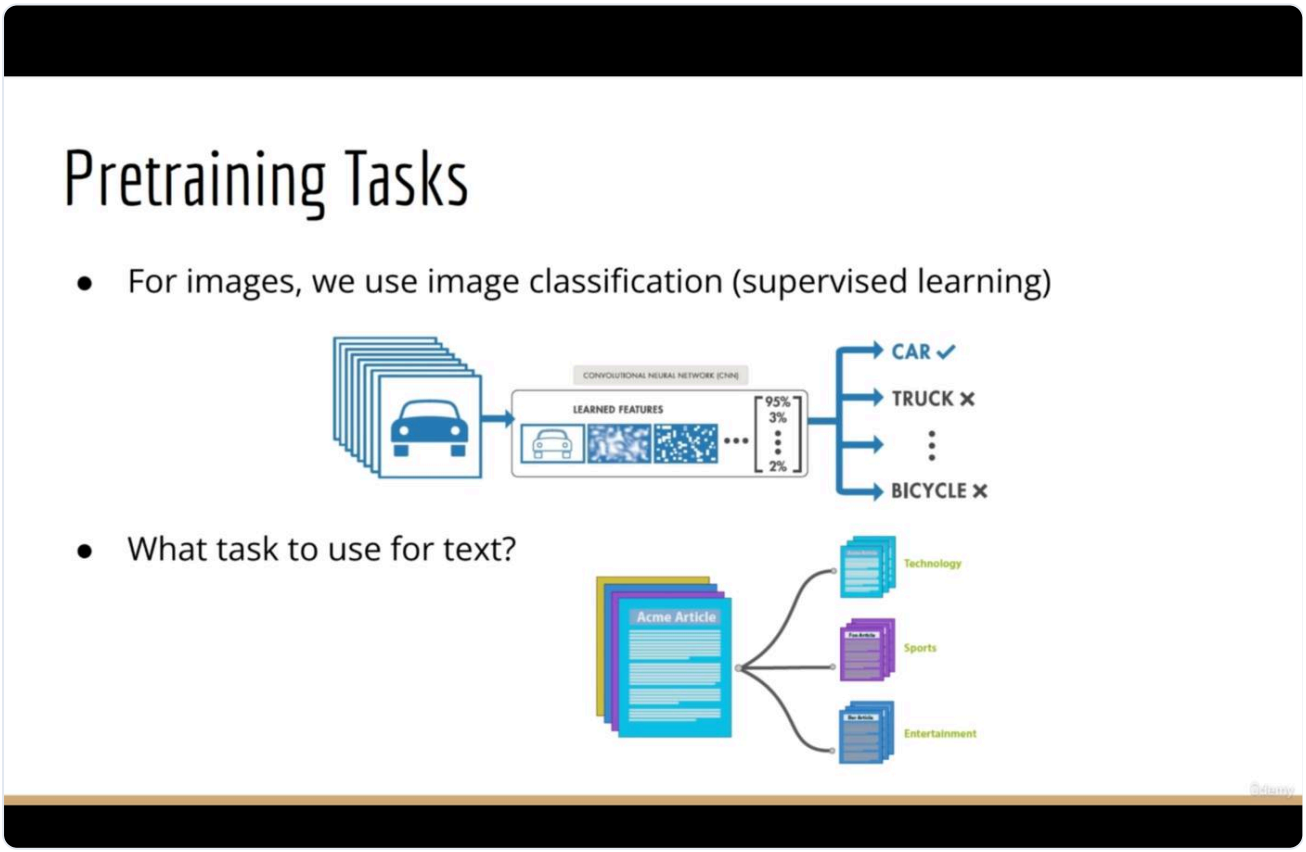

FIGURE 4.91

Firstly, we'd have to consider what we would use as labels. We'd have to make sure that these labels made sense and actually helps the neural network learn the structure of language.

But this is actually not necessary for classifying documents.

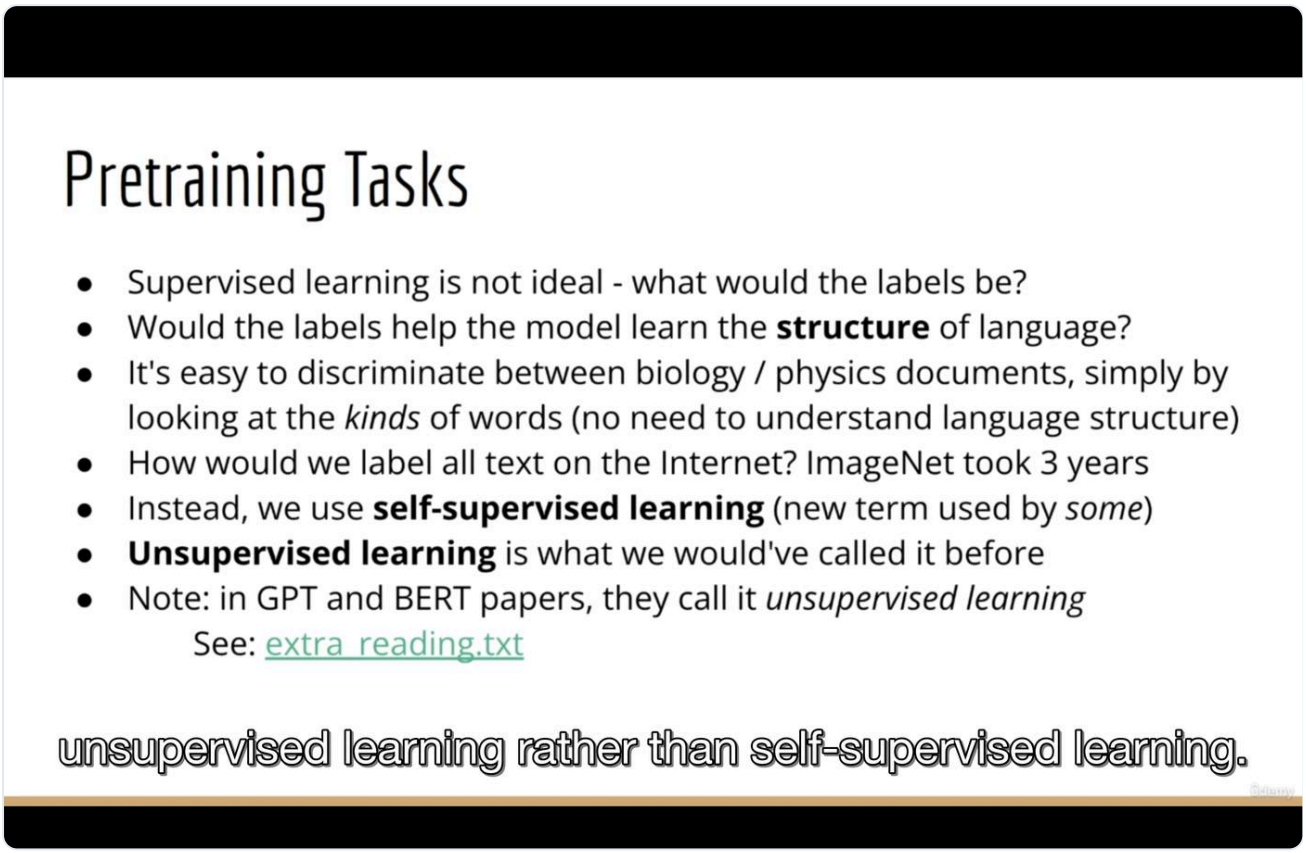

FIGURE 4.92

For example, it's easy to tell whether a document is about biology or physics simply by looking at the kinds of words in the documents. There is no need to learn the structure of language. In other words, merely classifying documents as a pre-training task might not help the neural network learn the structure of language as well as we would like. Secondly, with so much text available on the Internet, it would be difficult and costly to consistently and correctly label every piece of text as an example of why this would be undesirable.

Consider that. Imagine it took three years to label. So the answer is that we don't use supervised learning, but rather what is now known as self-supervised learning. This is a newer term, despite the fact that these techniques have existed for many years. Another way to think of it is unsupervised learning, which is what we would have probably called it many years ago. Note that in the both the original GPT and BERT papers, they referred to the Pre-Training process as unsupervised learning rather than self-supervised learning.

In my opinion, the idea is fine, but I like to use unsupervised learning since there is really no need to make up a new name for something that already had a name. So how does it work? Well, there are multiple ways to pre train, but one method used by large models such as GPT builds what is called a language model, which you may have heard of. So what is a language model to review?

Language Model

- A language model in its usual form models the probability of the next token in a sentence given the previous tokens

  E.g. $p(x_t \mid x_{t-1}, x_{t-2}, \ldots)$
- Recall, the Markov model: $p(x_t \mid x_{t-1}, x_{t-2}, \ldots) = p(x_t \mid x_{t-1})$

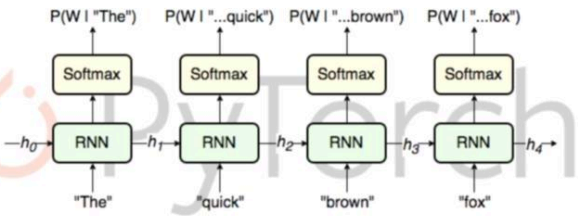

FIGURE 4.93 | Language Model

Language Model

- A language model in its usual form models the probability of the next token in a sentence given the previous tokens

  E.g. $p(x_t \mid x_{t-1}, x_{t-2}, \ldots)$
- Recall, the Markov model: $p(x_t \mid x_{t-1}, x_{t-2}, \ldots) = p(x_t \mid x_{t-1})$
- How to build one? A simple many-to-many RNN will do the trick!

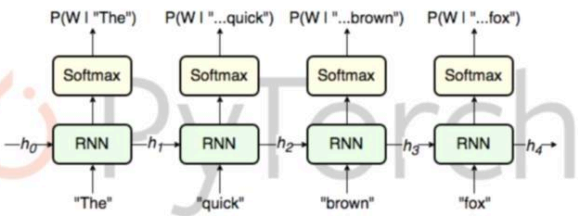

FIGURE 4.94 | Language Model

This is where we model the probability of the next token in a sentence given the previous tokens.

Mathematically, we would write this says P of x of T, given x of t minus one, x of t minus two and so forth. As you recall, when we use Markov models, these probabilities are reduced to one step transitions. So we just have P of x of T given x of t minus one. With neural networks, we don't have to make the Markov assumption, and thus we can account for more context. So how do we build such a neural network? One example is with RNNs what we take as input the previous words in the sentence, and then as output we try to predict the next word.

And note that because of the structure, this kind of model is what allows us to generate text as well. So if you train a language model in this fashion, you can use it to generate poems, stories, social media comments and even computer code. Now you recall that I said that this method has actually existed for many years. In fact, many decades ago we began doing the same thing with Time series. It turns out that this is how we train at time series models such as A.

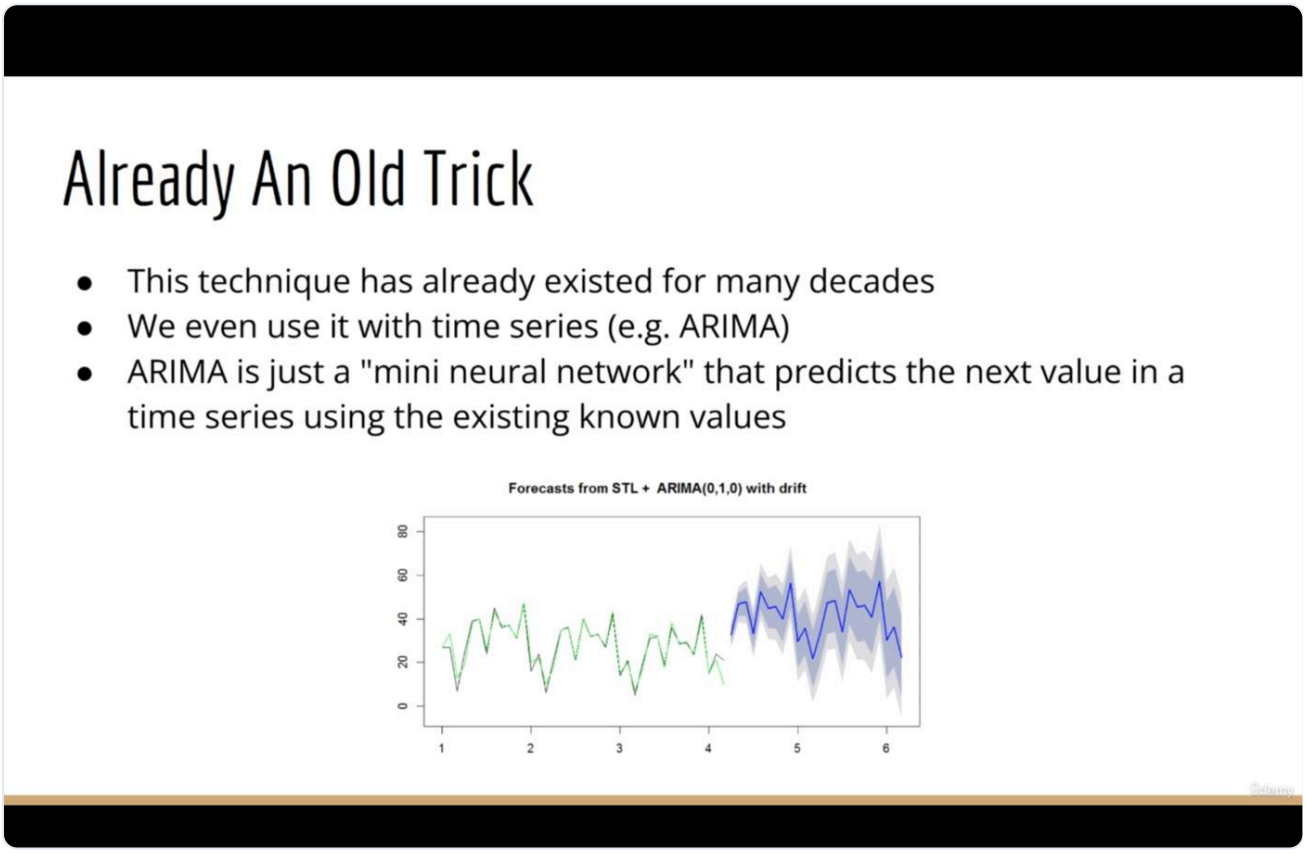

FIGURE 4.95 | Already an Old Trick

We essentially build a mini neural network that takes as input the previous data points in the Time series and tries to predict the next value in the Time series. So this method is not new by any means. It just so happens that this same method turns out to work pretty well these days on new models such as transformers. Now, the previous Pre-Training objective we just discussed is not the only Pre-Training objective that is possible. The previous example described an auto re-gressive language model because we use past tokens to predict the next token.

But there is another kind of language model called the masked language model.

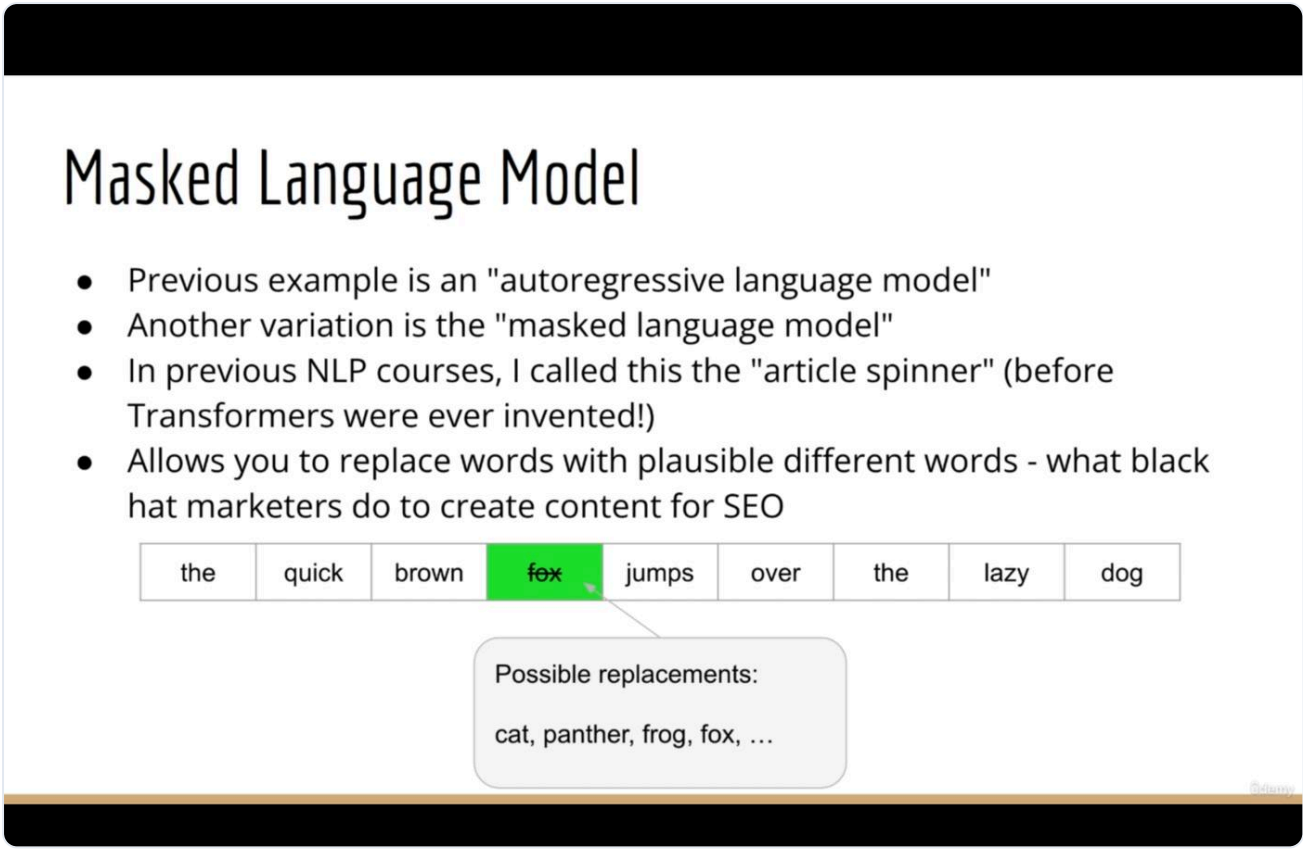

FIGURE 4.96 | Masked Language Model

In previous courses, I've called this the article spinner because it allows you to replace words in a sentence with plausible replacements, which is what blackhat marketers do when they want to spin articles to generate content for SEO. In contrast to the previous Pre-Training objective, we will no longer be using the past tokens to predict the next token. Instead, the masked language model uses all surrounding tokens to predict the middle token.

That is, the tokens we conditioned on. It can come from past tokens or future tokens. For example, consider the sentence the quick brown fox jumps over the lazy dog. Imagine that we remove the word fox, replace it with what we call a masked token, and then we pass this through our neural network. The job of our neural network is to predict what the masked word should be. As such, it will learn a probability distribution to predict the missing word. Note that the token we remove can come from anywhere in the sentence.

So it could be Fox jumps over Lazy Dog and so forth. To recap, we just looked at two different Pre-Training objectives for training transformers. The first method is used for training GPT and GPT like models, and it's called a causal language model or an auto regressive language model. This is because they are just like auto regressive time series models where we use past data points in the Time series to predict the next data point.

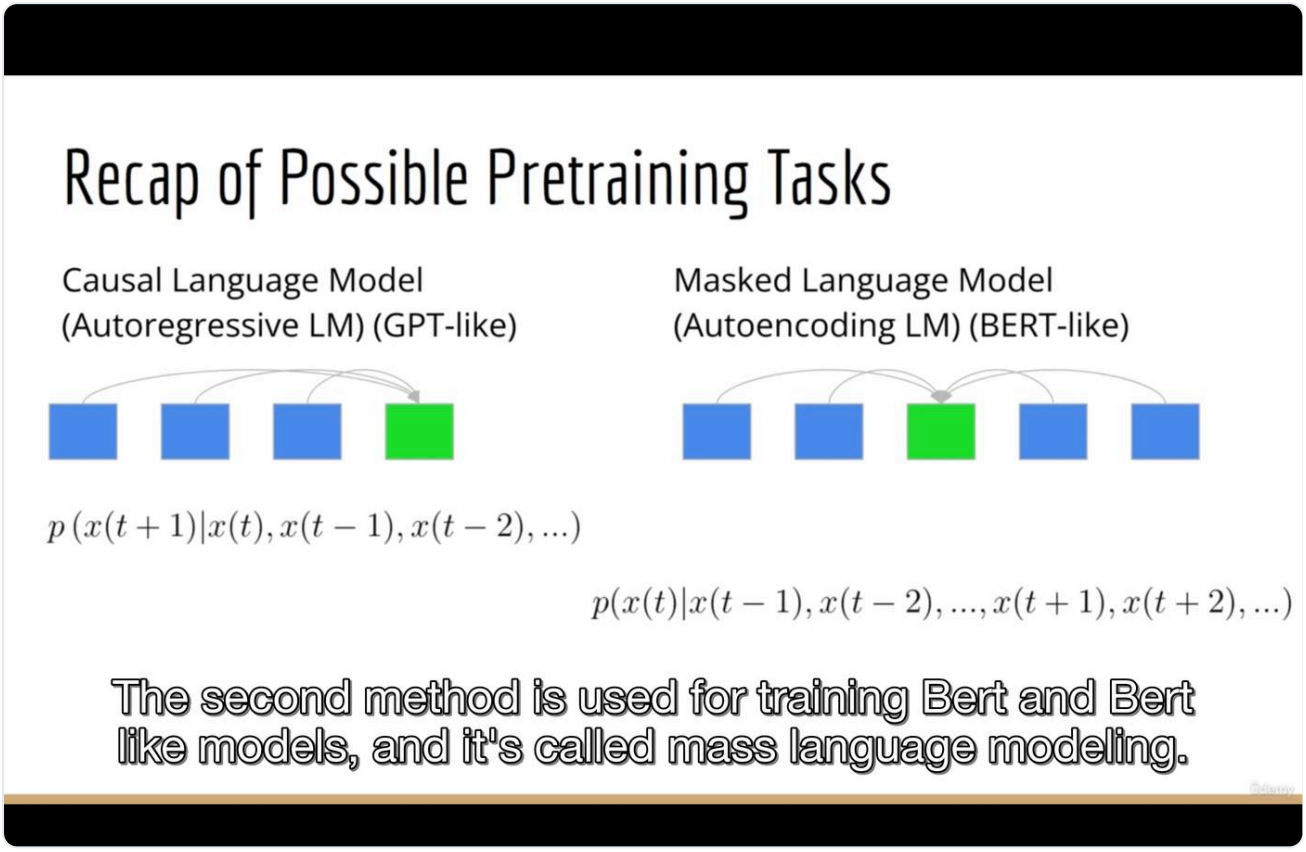

FIGURE 4.97 | Recap of Possible Pretraining Tasks

The second method is used for training BERT and BERT like models, and it's called masked language modeling.

Sometimes these models are called auto encoding language models. This is because they are similar in principle to auto encoders, which you've learned about if you study deep learning with me in the past. One kind of auto encoder is the noisy auto encoder where we pass in a feature vector, which could be an image, but more generally could be any kind of feature vector.

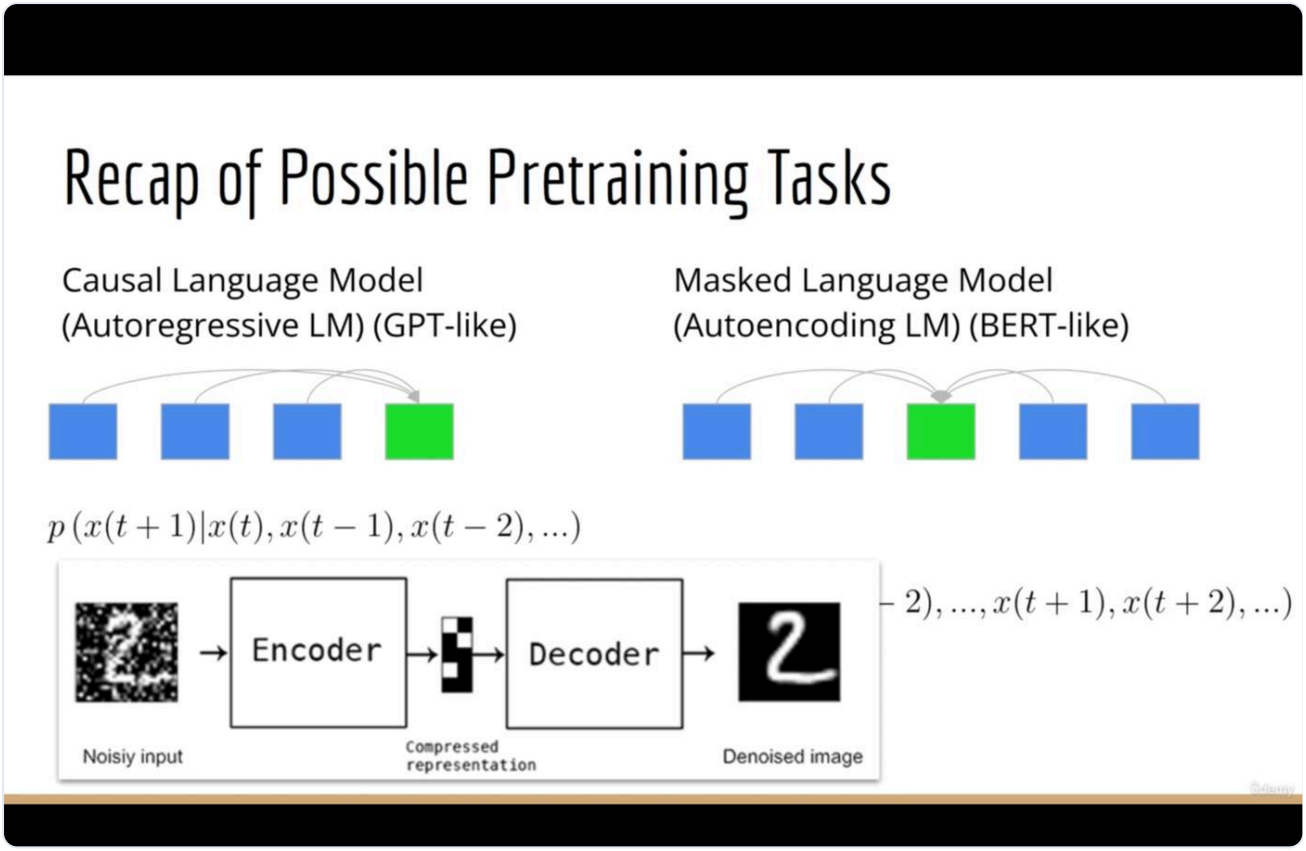

FIGURE 4.98 | Recap of Possible Pretraining Tasks

We then add noise to the input feature vector, which for images might simply mean setting those pixel values to zero.

The job of the auto encoder is to recreate the original image without this noise. As you can see, this is really what we are doing when we remove a word from a sentence and ask the neural network to predict that word. We are asking the model to recreate the original sentence, which has been corrupted with noise. Now, just as a side note, there is actually one additional test that BERT uses for unsupervised pre-training, which is next sentence prediction.

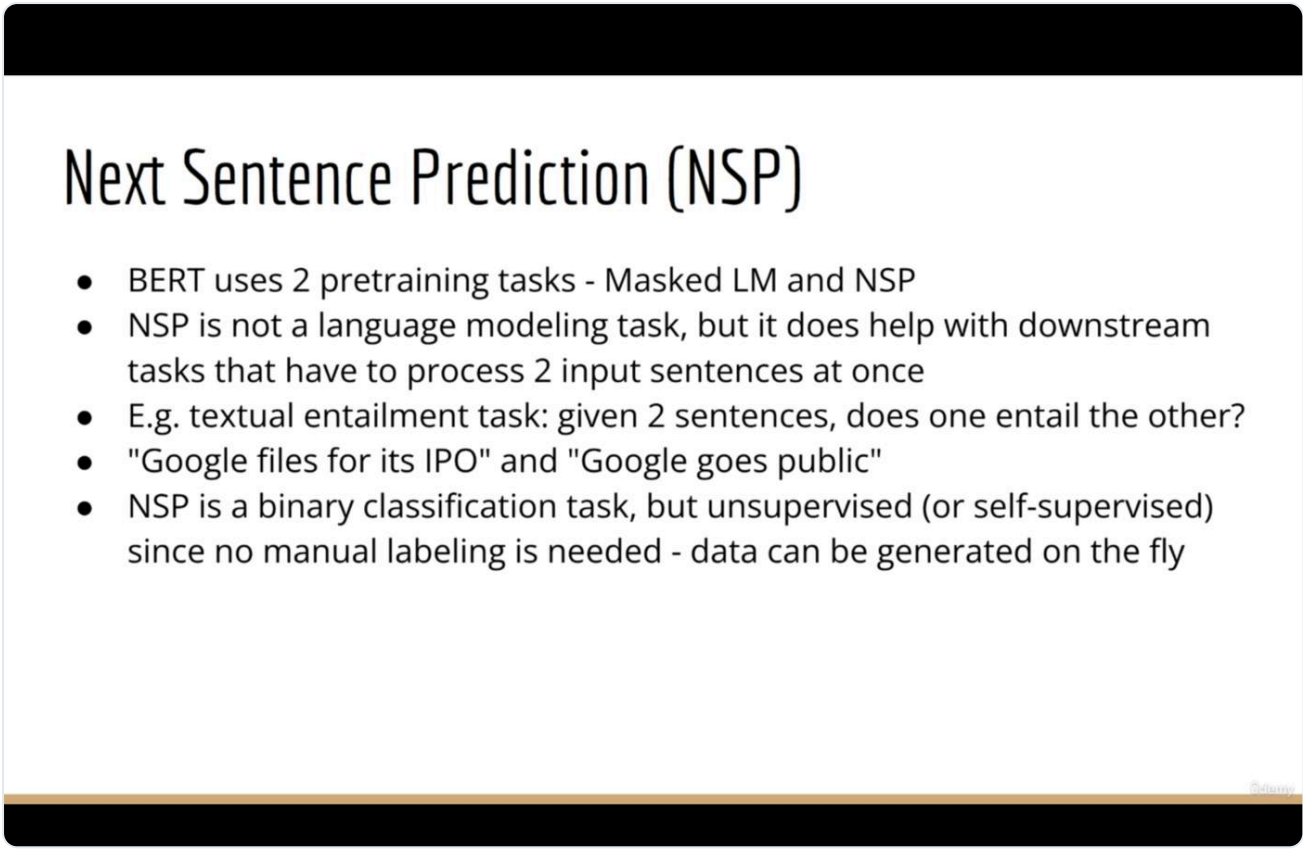

FIGURE 4.99 | Next Sentence Prediction (NSP)

This is not a language modeling task, but it does help with downstream tasks which have to deal with two input sentences at once. One example of such a downstream task is called textual entailment, where we want to know given two sentences whether or not they mean the same thing. An example of that would be Google Files for its IPO and Google goes public, which both mean the same thing in order to pre-trained BERT so that it can understand the relation between the multiple input sentences.

We use next sentence prediction for this task. We simply input two sentences and the model tries to predict whether or not the second sentence followed the first in the training data. Note that while this is a binary classification task, it is also an example of unsupervised or self-supervised learning. Since we don't need to manually label the data, this data can be easily created or generated on the fly from a corpus of text. Now there's one small issue with Pre-Training language models, which is that they take lots of time and lots of compute.

Luckily, there's some parallelization that can be done, which does reduce the time but does not reduce the compute. Compute translates to cost. And thus one of the major barriers to entry is the amount of money you have to spend. To give you an example of this. Let's consider one of the more recent and well known language models called GPT-3. This language model contained 175 billion parameters for comparison of LeNet, which was one of the first popular CNN architectures, had just 60,000 parameters, as you recall, in deep learning.

We found that one of the most powerful ways to improve model performance is to simply make the model bigger and to add more data. So GPT-3 is a bigger version of GPT-2, which is in turn a bigger version of the original GPT. As a side note, GPT stands for generative pre-trained trans-

former, which should make complete sense to you at this point.

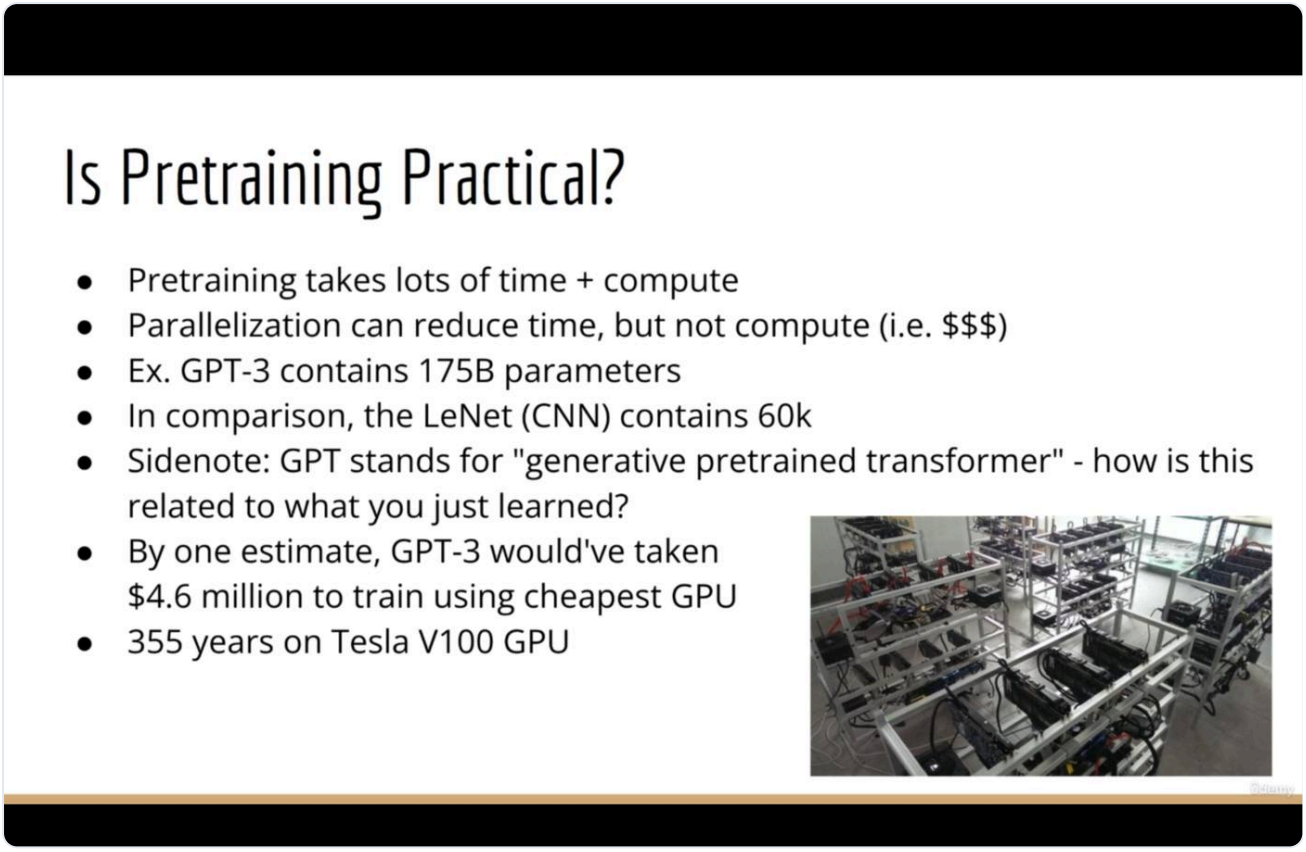

FIGURE 4.100 | Pretraining Takes Lots of Time + Compute

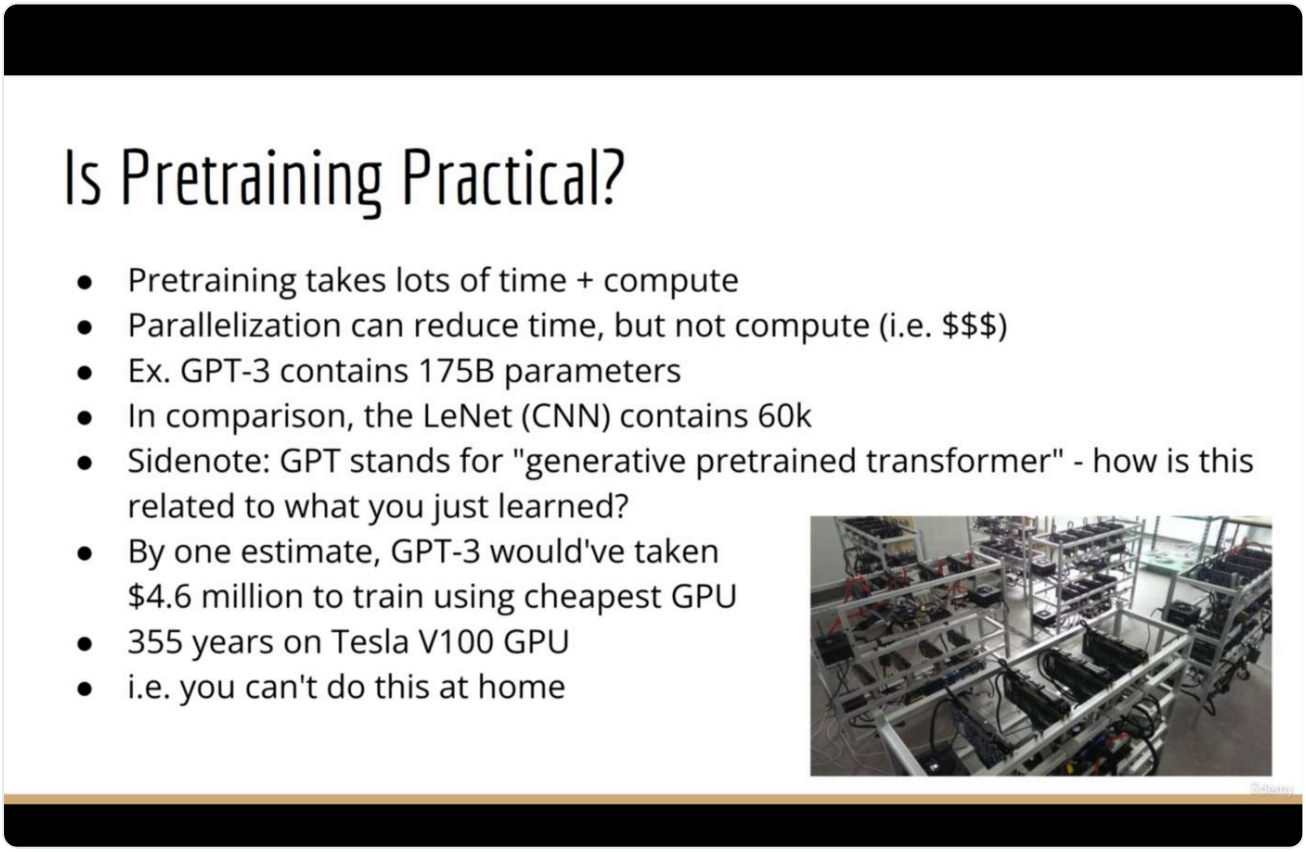

FIGURE 4.101 | Pretraining Takes Lots of Time + Compute

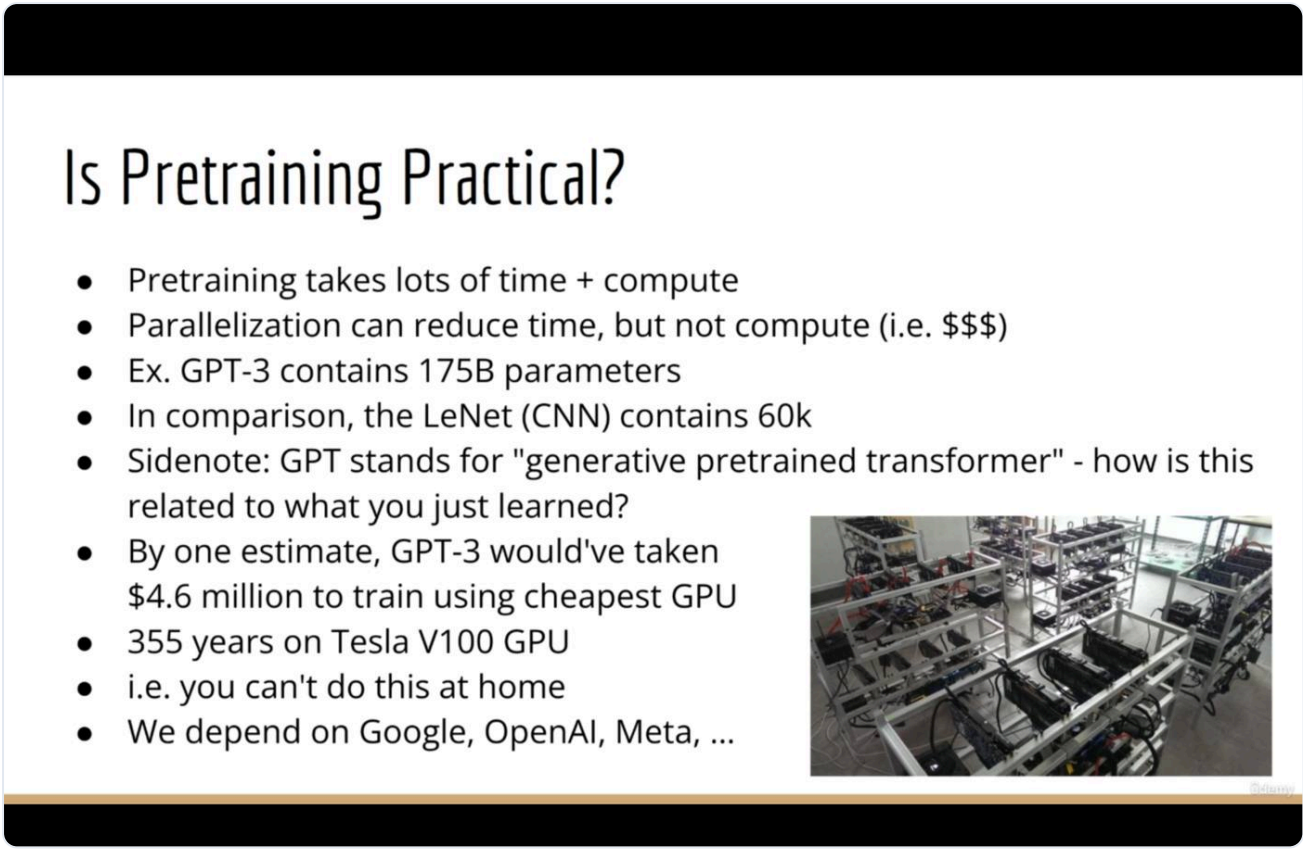

FIGURE 4.102 | Pretraining Takes Lots of Time + Compute

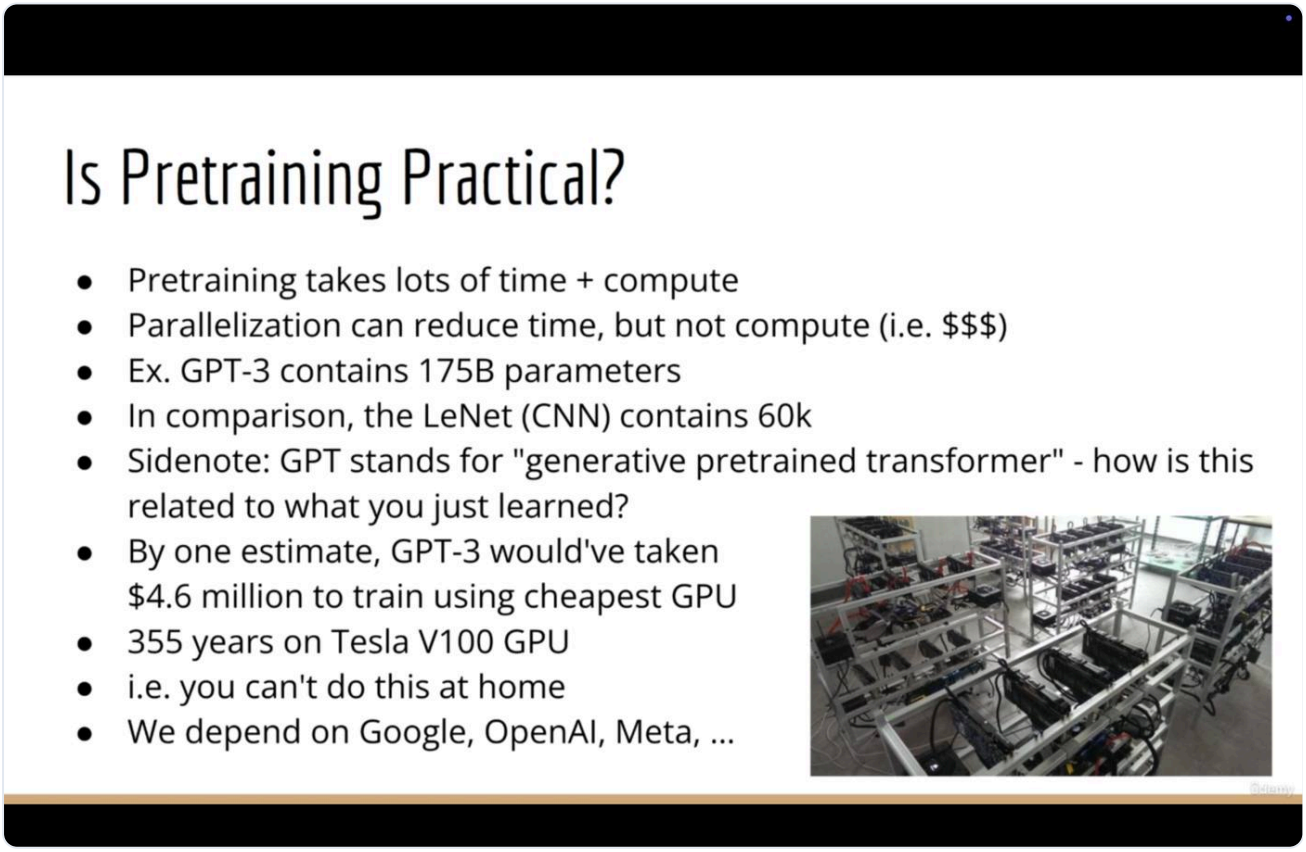

FIGURE 4.103 | Pretraining Takes Lots of Time + Compute

Well, it turns out that based on one estimate, GPT-3 would have taken over $4.6 million to train using the cheapest GPU cloud on the market using a single Tesla v 100 GPU.

This would take 355 years to train. Note that this calculation is theoretical since the model is actually too large to fit on a single GPU. In any case, it's definitely not something you're going to do at home. Now, what's the point of mentioning all of this? Well, the point is to highlight the absolute necessity of transfer learning. It's not simply a matter of being a cool new trick. There is just no other way we would be able to ever make use of such large models.

We depend on companies like Google, Meta and OpenAI to open source these weights so that we can make use of them.

8 · Fine-Tuning Sentiment Analysis and the GLUE Benchmark

So at this point, we know about tokenizer and we know about models. We even know about how to take a pre-trained model, chop off its head and add a new one for text classification. What we do not yet know is how to actually fine tune this model on a data set. So that's what we'll be discussing in this lecture for this code preparation lecture.

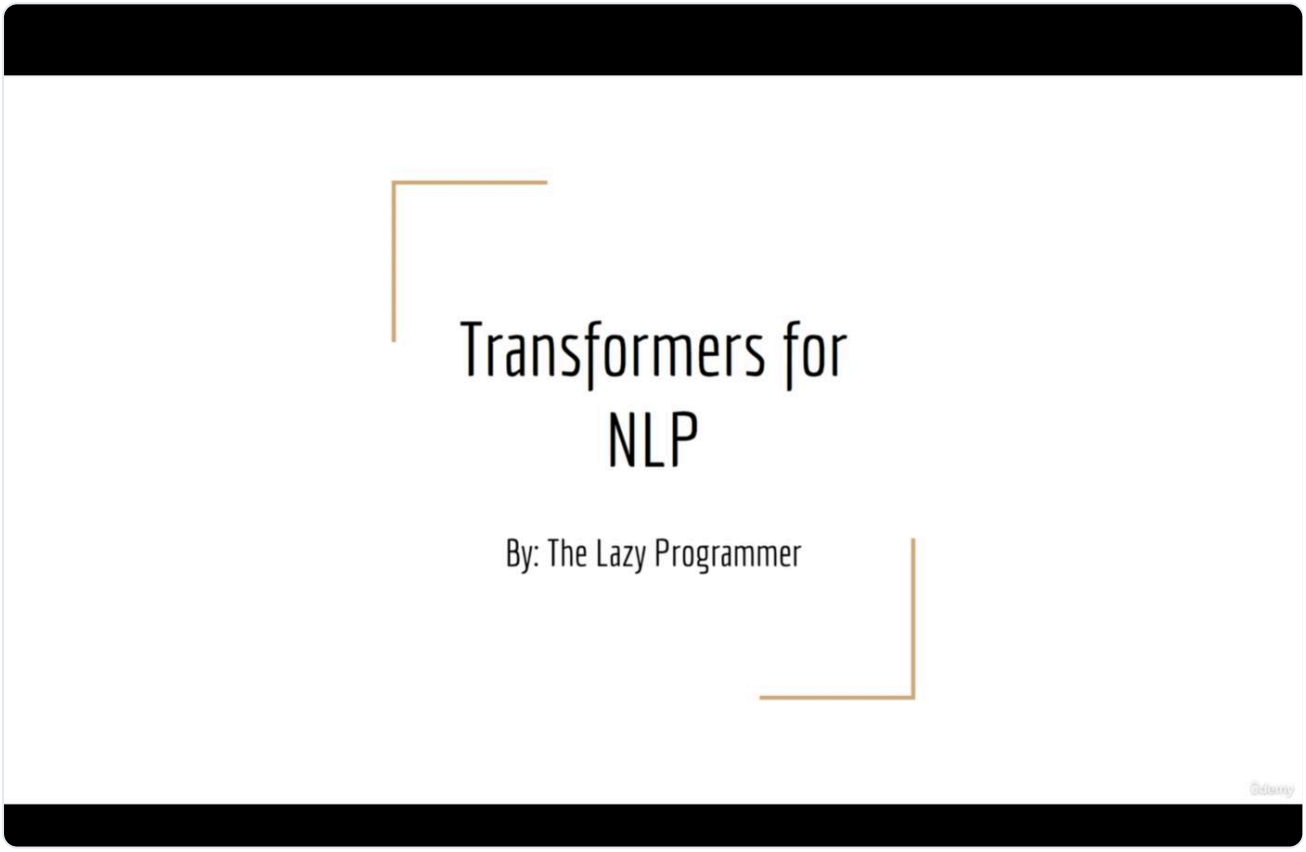

FIGURE 4.104 | Transformers for

We'll begin by discussing how data sets are represented in the Hugging Face suite of packages. Well, then look at the trainer objects and the trainer arguments objects which allows us to specify a few training parameters.

Finally, we'll look at how we can compute metrics during the training process. Which Hugging Face has their own way of doing?

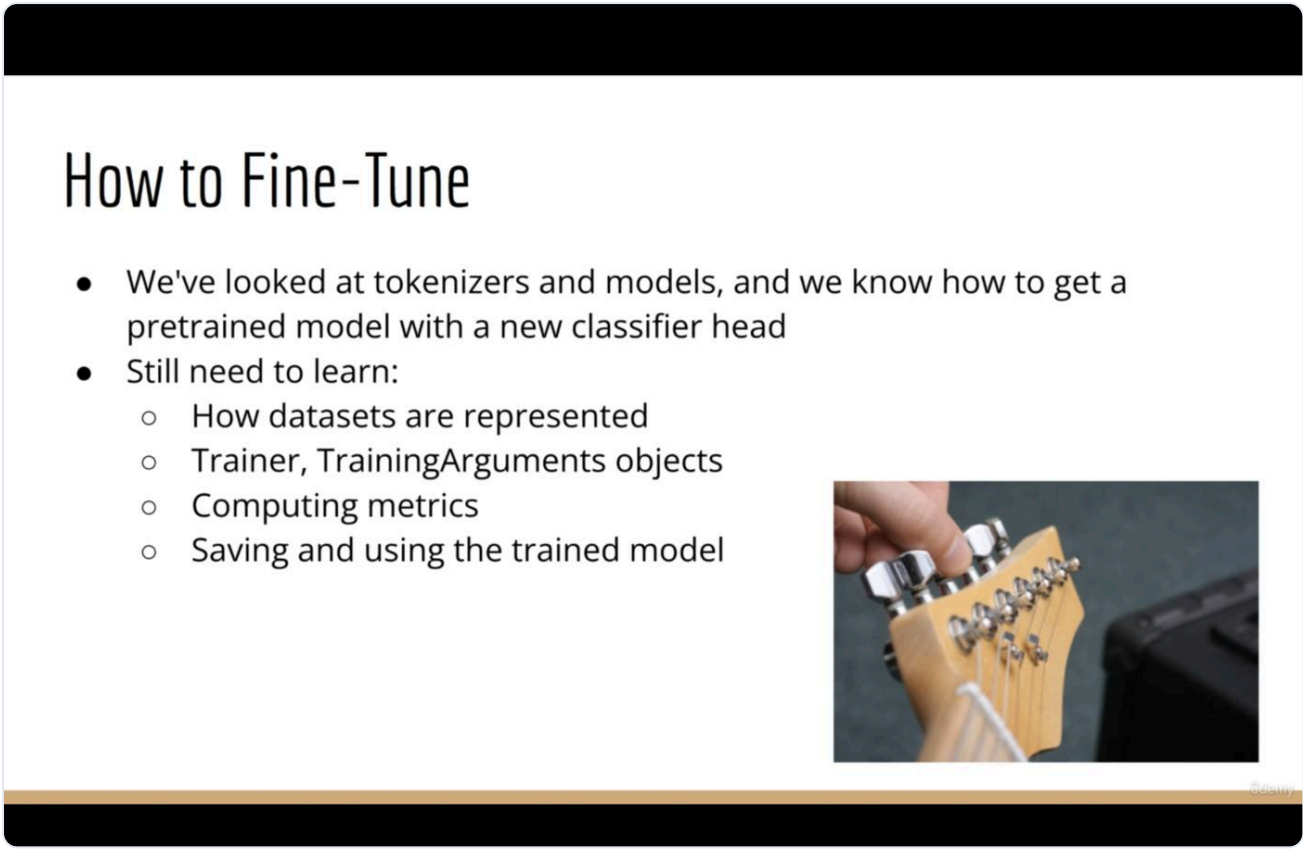

FIGURE 4.105

After training a model, you'll probably want to know how to save it for later use. So we'll discuss that as well. Okay. So the first thing to mention about data sets is that there is actually a separate library from Hugging Face called datasets.

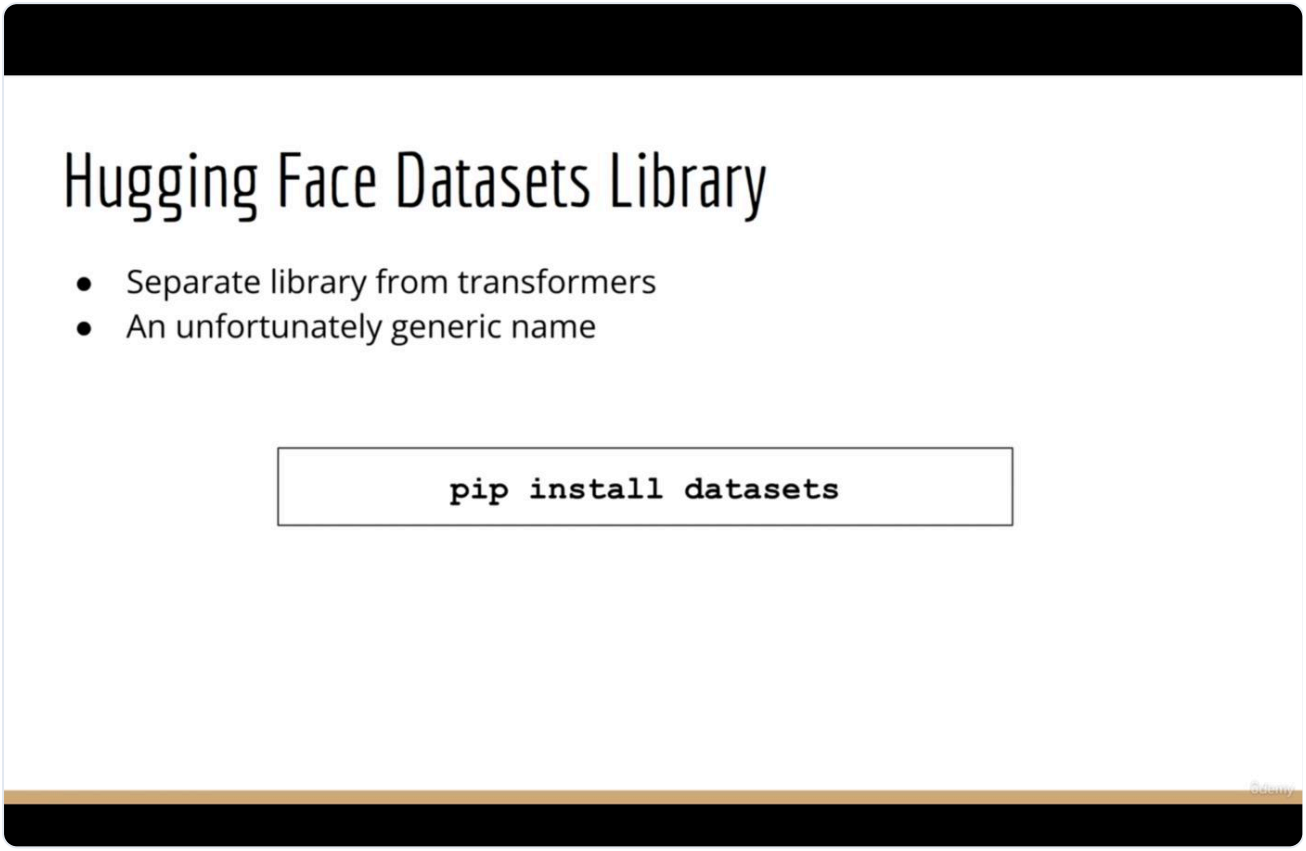

FIGURE 4.106 | Hugging Face Datasets Library

This is not part of the Transformers module. As such, it can be installed in the usual way by calling pip install datasets.

Somewhat of an unfortunately generic name. In this course.

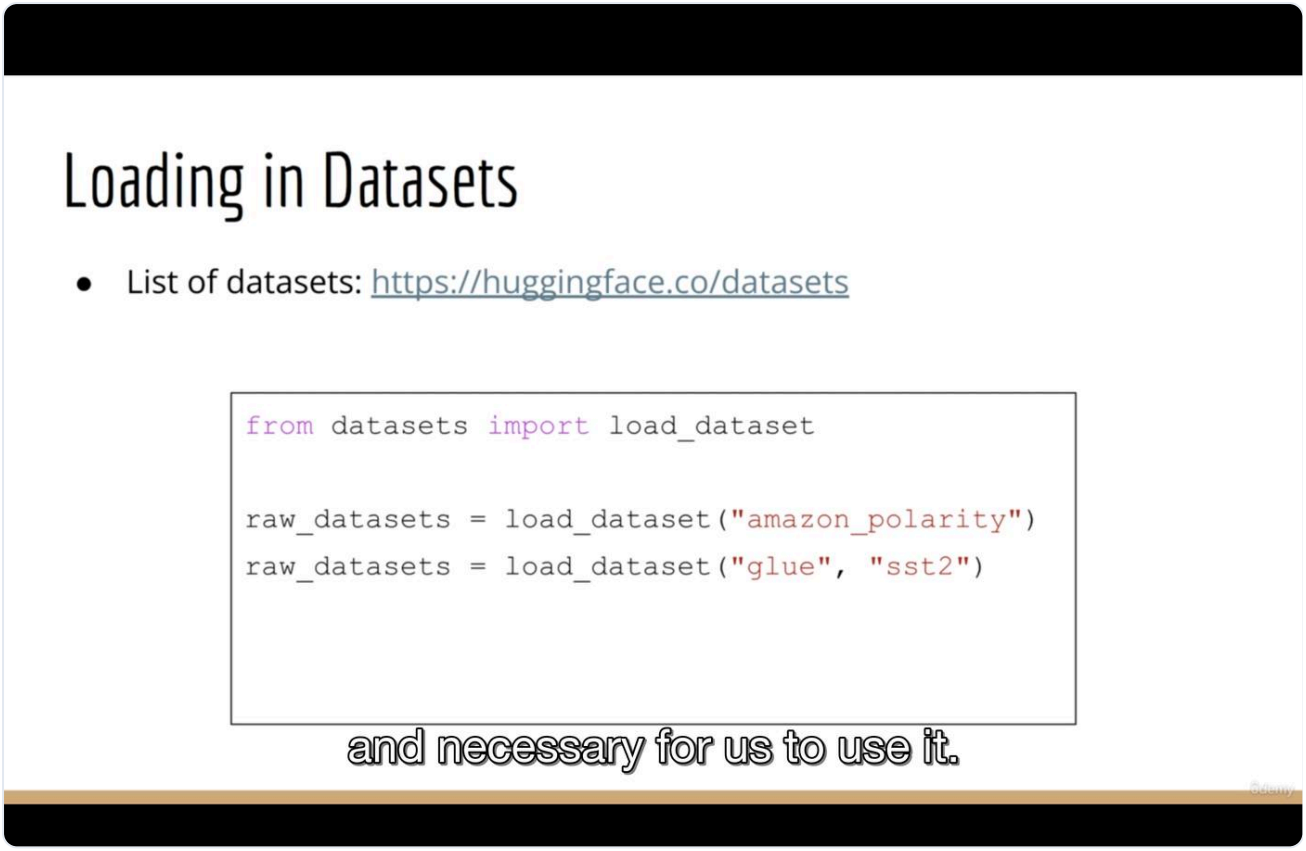

FIGURE 4.107 | Loading in Datasets

We'll begin by looking at data sets that are included with the data sets package that is will be able to load in the data set using a single line of code which will do all the formatting and processing and necessary for us to use it. If you're concerned about how you might use your own data from your own files, don't worry as that will be coming next. For a full list of datasets you can go to.

huggingface.co/datasets. Once you find the name of a dataset that you want to use, you can simply pass it into a function called the load_dataset, which will return the appropriate data sets. So for example, there's a data set called Amazon Polarity, which is a data set for sentiment analysis. The data set that we'll be using in the next lecture is called SST-2, which is part of the GLUE benchmark. The syntax for this looks kind of weird since we actually passed in the two arguments into the load_dataset function.

The first argument is GLUE, which specifies the data set. And the second argument is SST-2, which specifies the specific task. Although technically this is a data set itself and the GLUE benchmark consists of multiple such datasets.

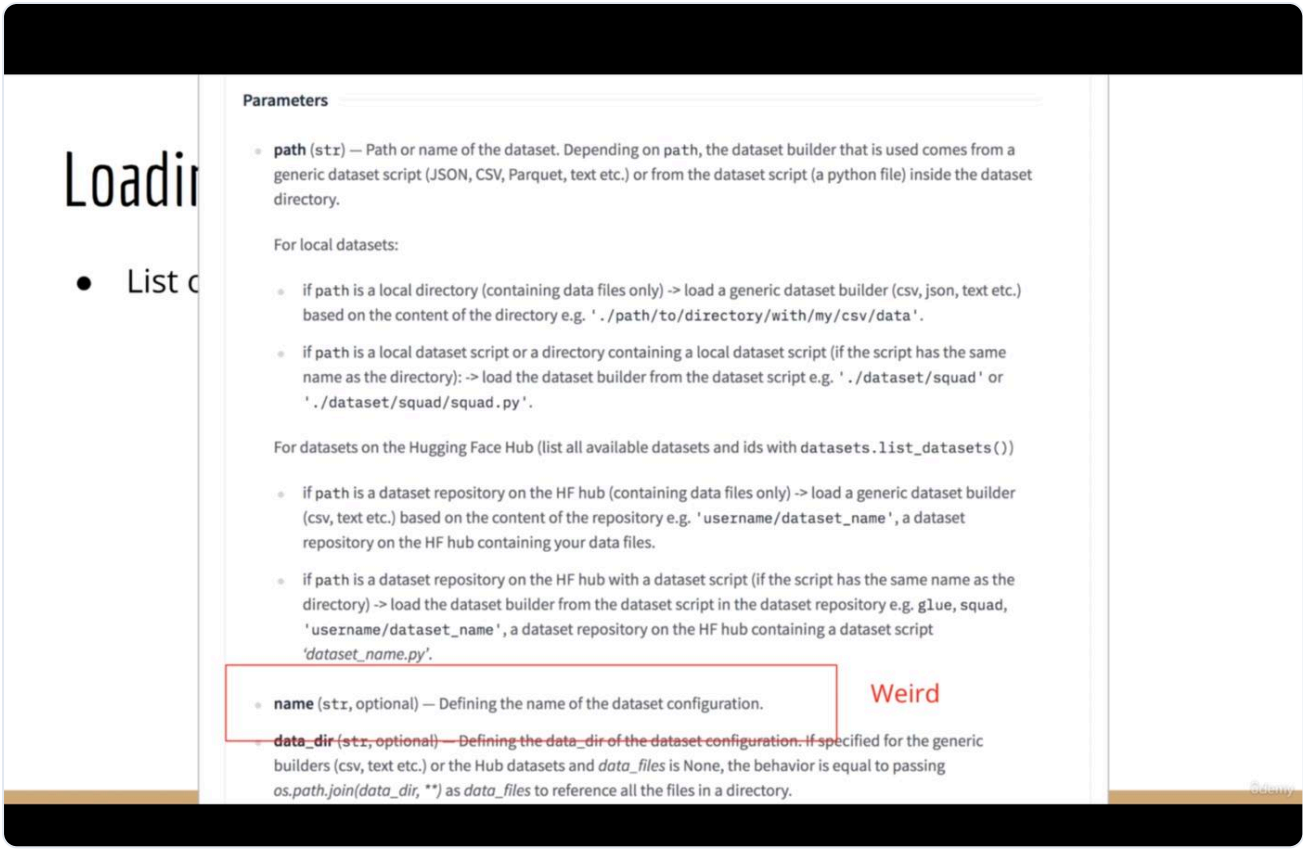

FIGURE 4.108

Officially, the second argument is described as the name of the data set configuration. However, this is a bit confusing in light of the above. So you might be wondering what is the GLUE benchmark? As you recall, researchers used benchmarks to test their model's performance so that they are comparable to other models made by other researchers.

An example of this is the MNIST benchmark and the ImageNet benchmark in computer vision.

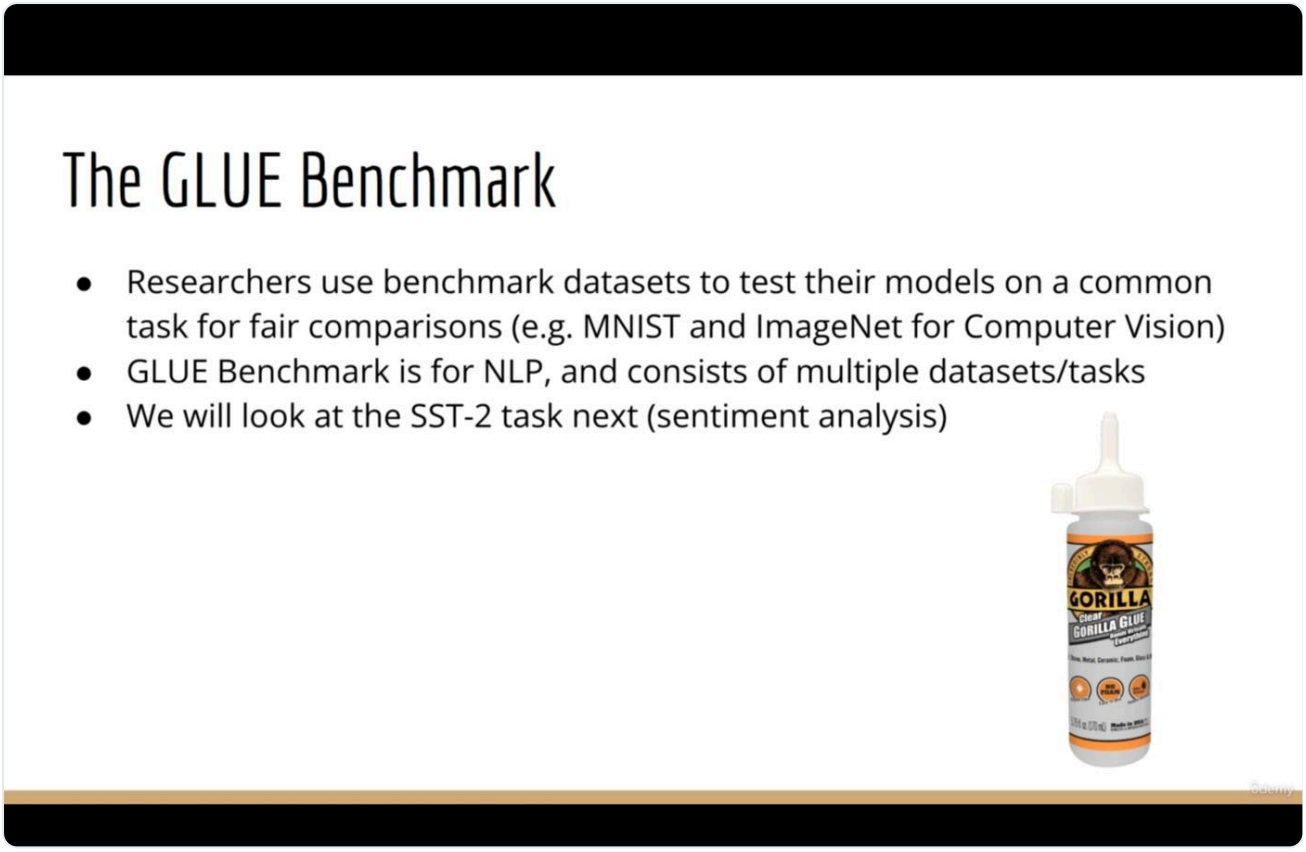

FIGURE 4.109 | The GLUE Benchmark

The GLUE benchmark is a common benchmark that has been used in modern NLP. The GLUE benchmark actually consists of multiple tasks and multiple data sets. The particular task will be looking at shortly is the SST-2 task, which is for sentiment analysis as we go through the course and you learn about more complex techniques. You'll see other tasks. Okay. So when you call the load_dataset function, this will actually return a data set dict object, which is basically like a dictionary that contains multiple data sets.

FIGURE 4.110

So for example, it might contain a train dataset, validation dataset and test dataset which would be treated as separate data set objects from here. Things are pretty flexible because of the variety of tasks that we can do in NLP. If we print out the data set object for SST-2, we'll see that it shows us two attributes which are its features and the number of rows it has. As such, you can think of a data set objects like a table. The features are really two columns of the table.

FIGURE 4.111

For us, the relevant so-called features are the sentence and the label. The sentence is the model input, and the label tells us whether or not that sentence is positive or negative. Note that this is exactly the same format that we've worked with in the past. So you can picture this in your mind like a CSV containing input documents and the corresponding labels. Now here's something a bit weird about the Hugging Face API, at least as it is currently implemented.

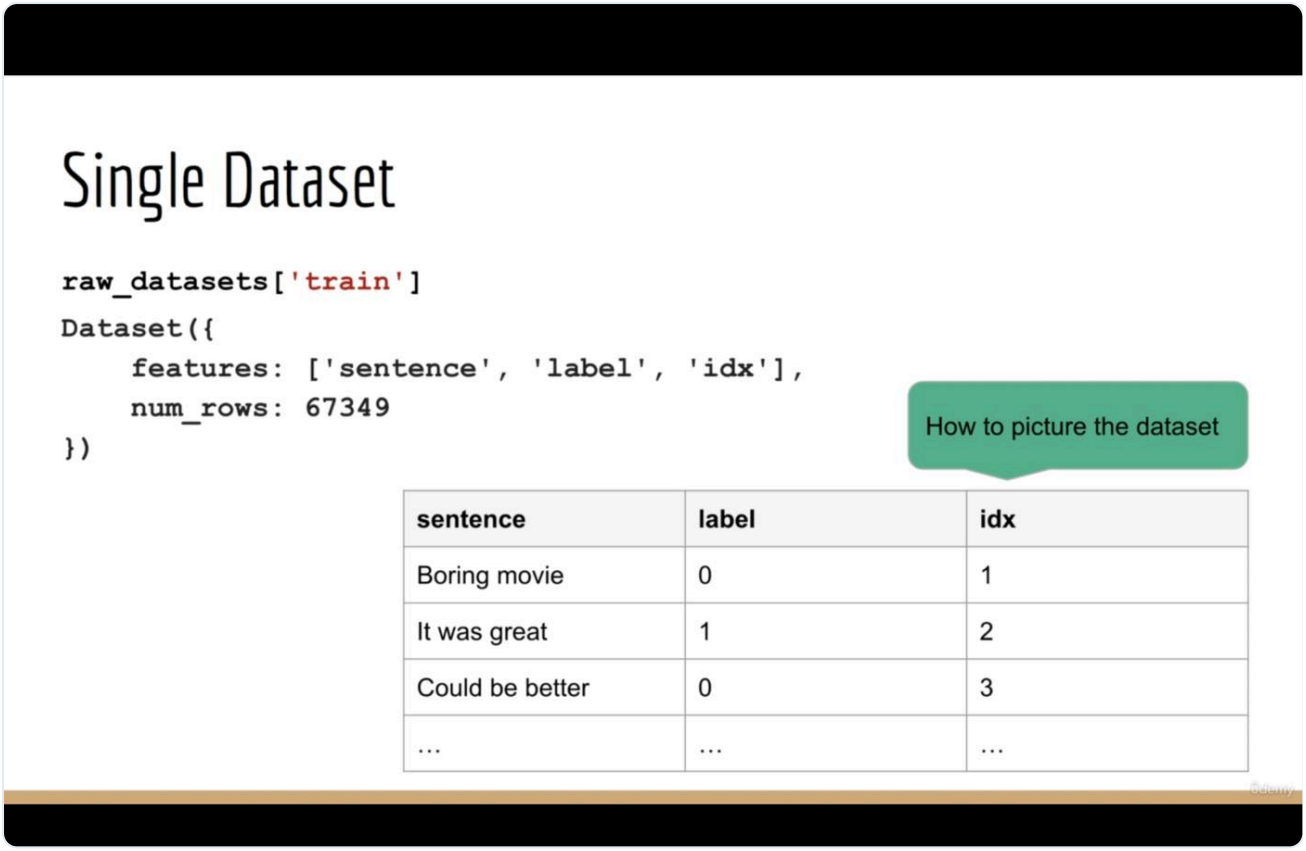

FIGURE 4.112 | Single Dataset

Recall that previously we discussed the need for truncation padding and also to convert our tokenized data into torch tensors. In the next notebook, we are actually not going to do all of these steps because they will be handled for us behind the scenes. In practice, what we are actually going to do is to partially apply the tokenizer to the data set dict object. We just load it in by calling the map method. The map method is analogous to the usual map method in Python, which you can call when you want to apply the same function to every element in a sequence.

What makes this map method nice is that behind the scenes it does the mapping in batches to improve performance by calling the map method and passing in a tokenized function.

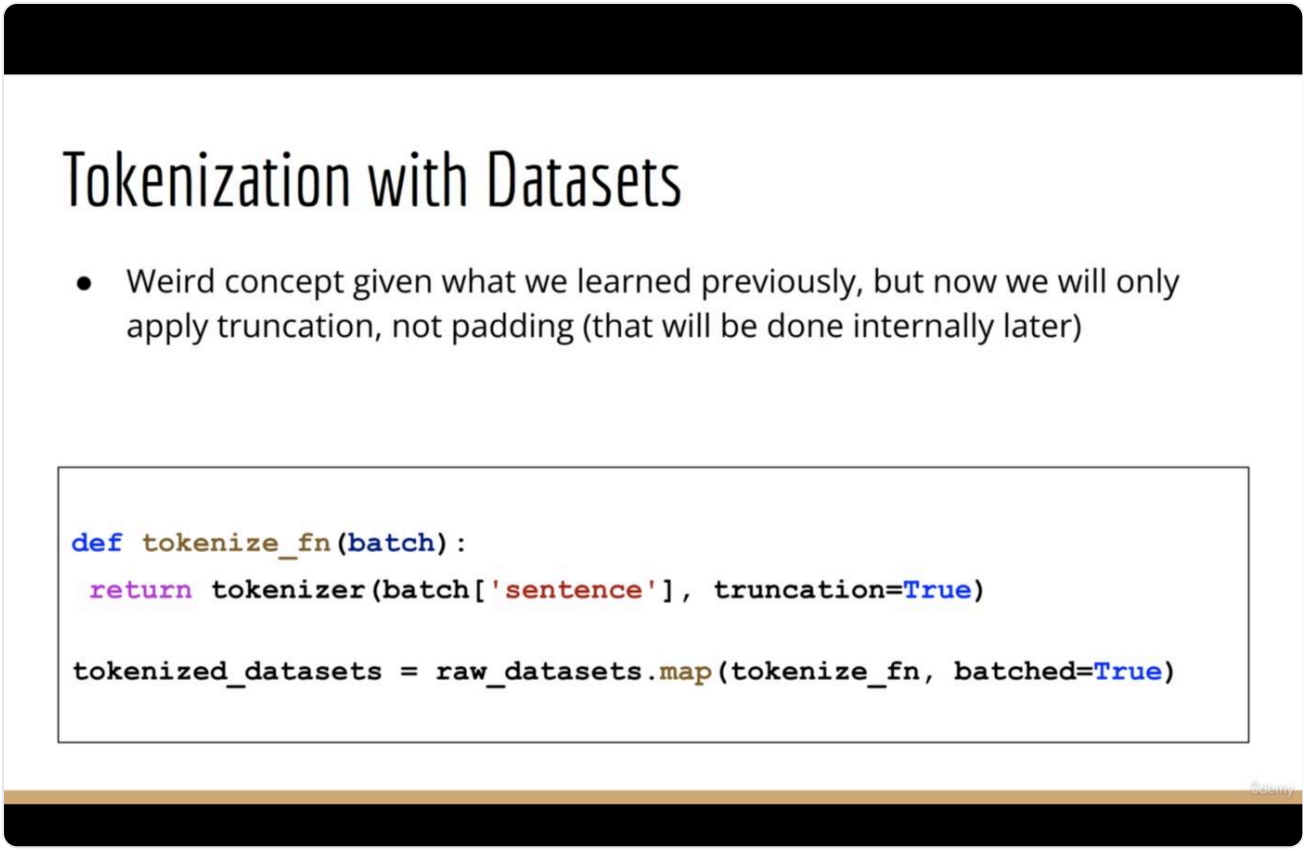

FIGURE 4.113 | Tokenization with Datasets

The data sets library automatically knows it should apply the same function to every data set that is train, validation and test. Note that in this example we apply only truncation but not padding or conversion into torch tensors. This will be handled behind the scenes by the trainer object we create later.

Now, if you find the map method a bit confusing, that's perfectly understandable. If you're having trouble wrapping your head around it, keep in mind that there's a simple Python analogy.

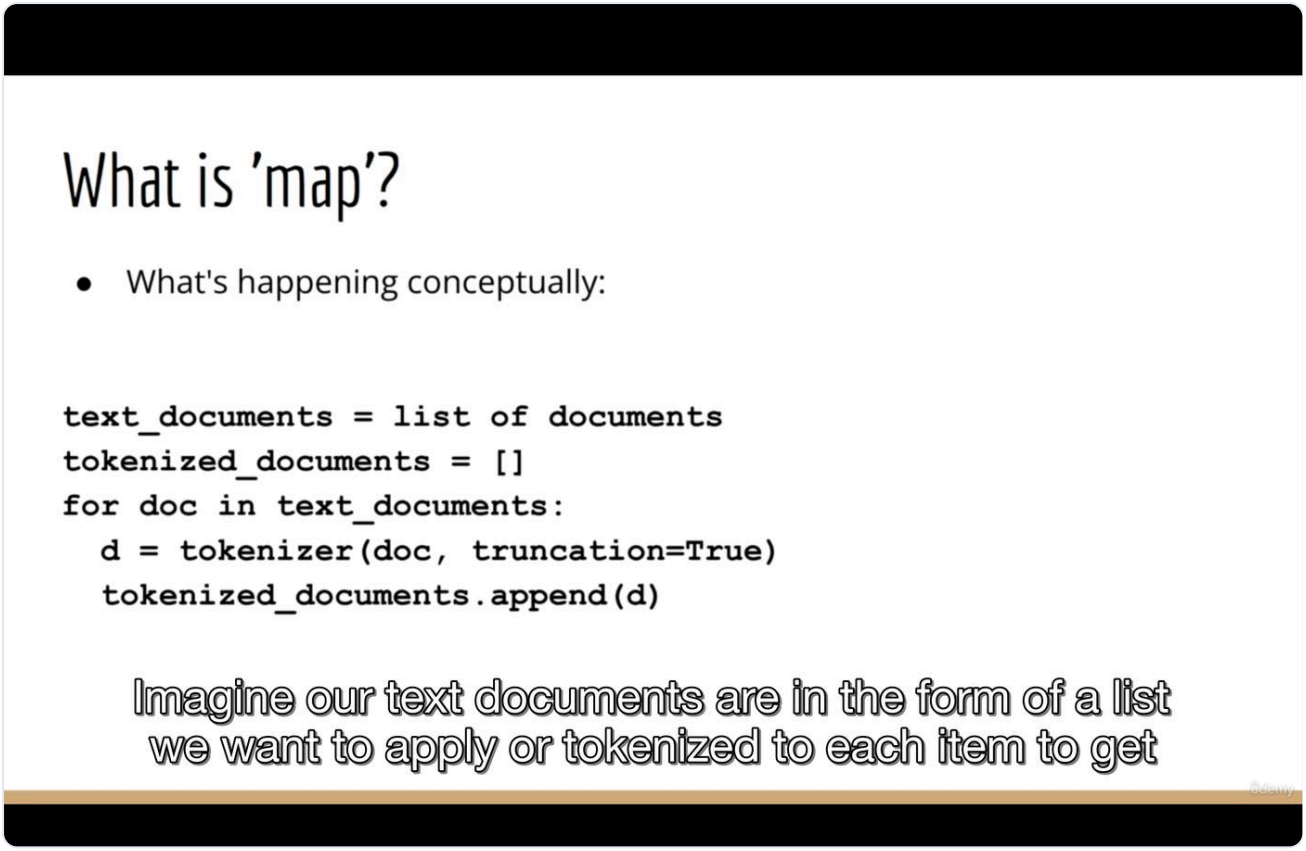

FIGURE 4.114

Imagine our text documents are in the form of a list we want to apply our tokenizer to each item to get a sequence of integer IDs for each document. In order to do that. We might loop through each element in the list, apply the tokenizer, and save the output to a new list.

That's what the method is doing, but it's doing it more efficiently. The next item to consider is the training arguments class. Basically, you can think of this like a training configuration.

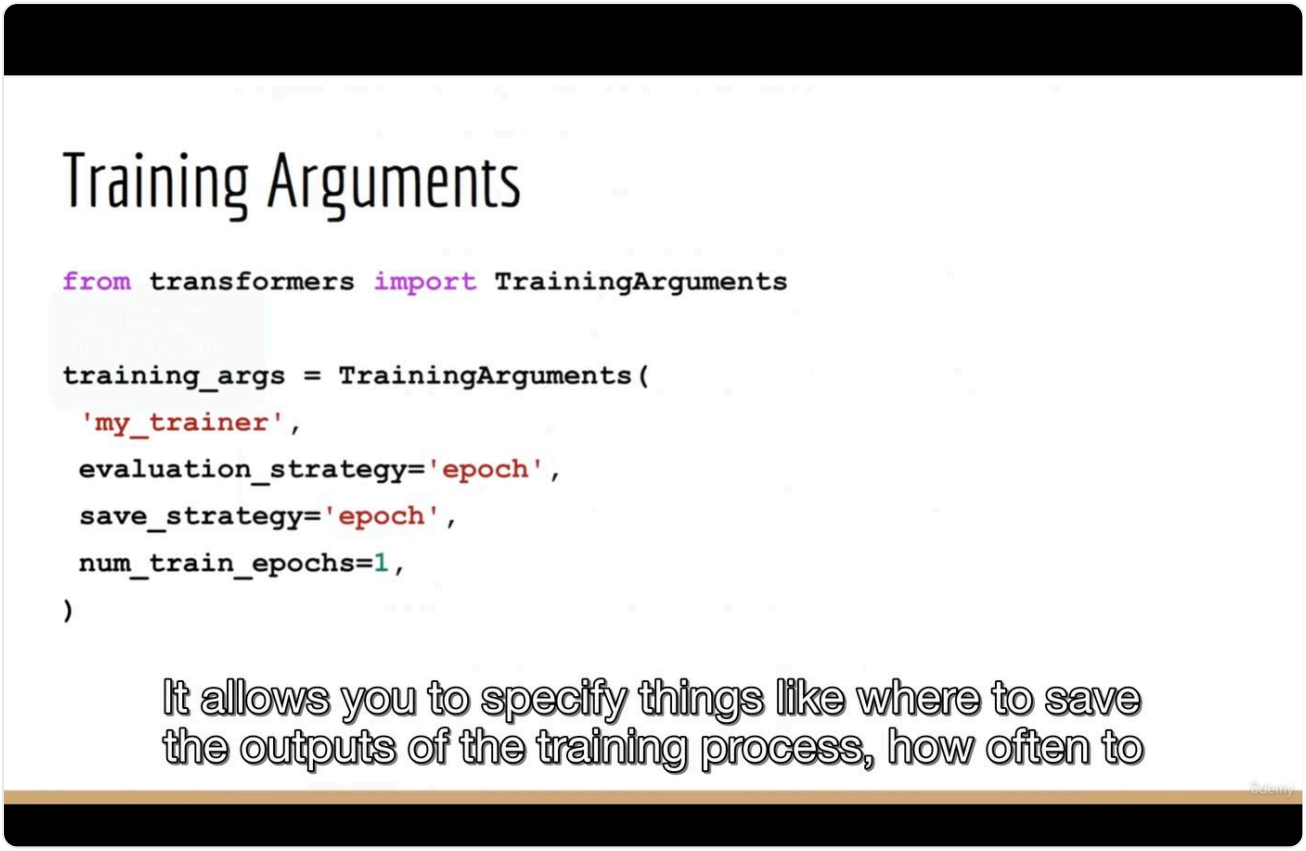

FIGURE 4.115 | Training Arguments

It allows you to specify things like where to save the outputs of the training process, how often to compute metrics, how many epochs to train for the learning rate, and many more. There are tons of arguments into the constructor for this class.

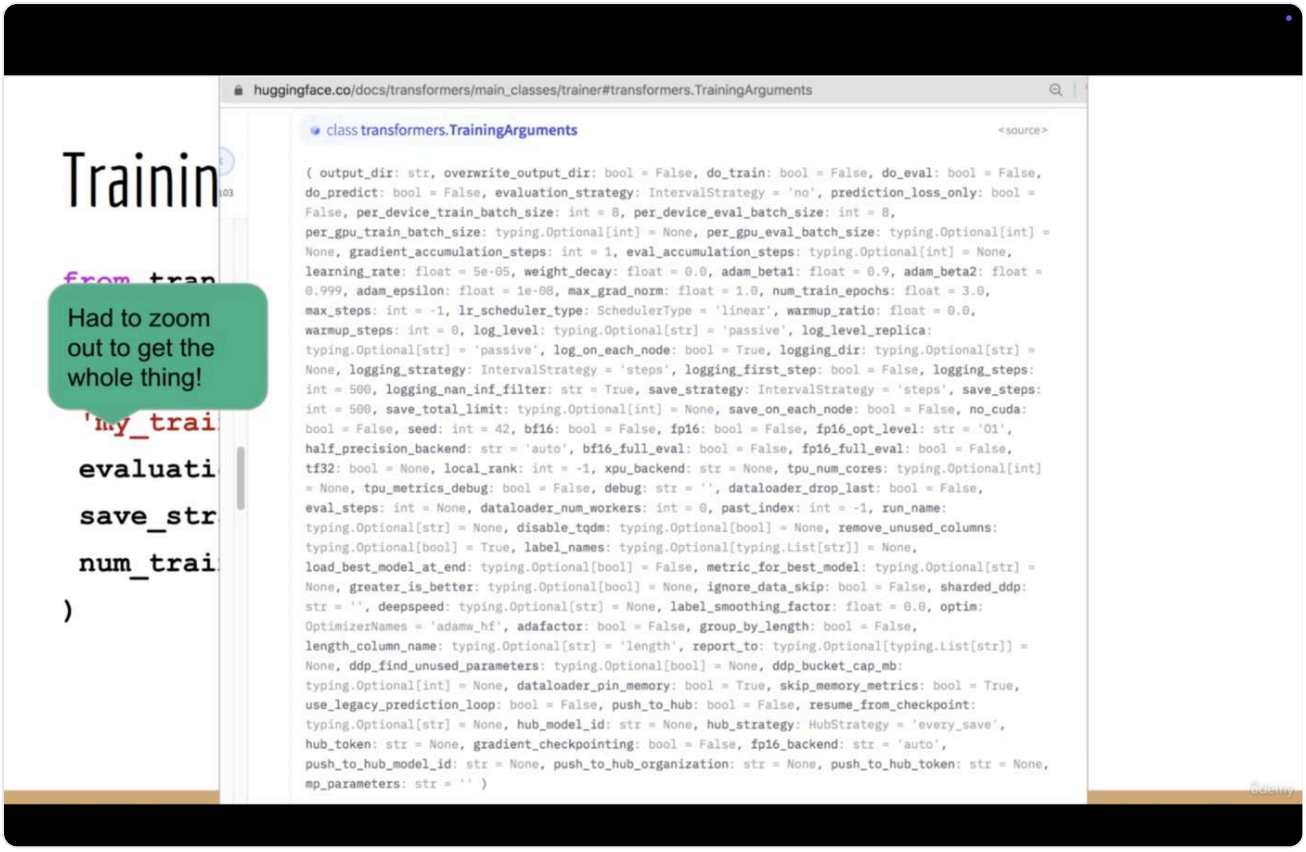

FIGURE 4.116

So check out the documentation if you want to see them all.

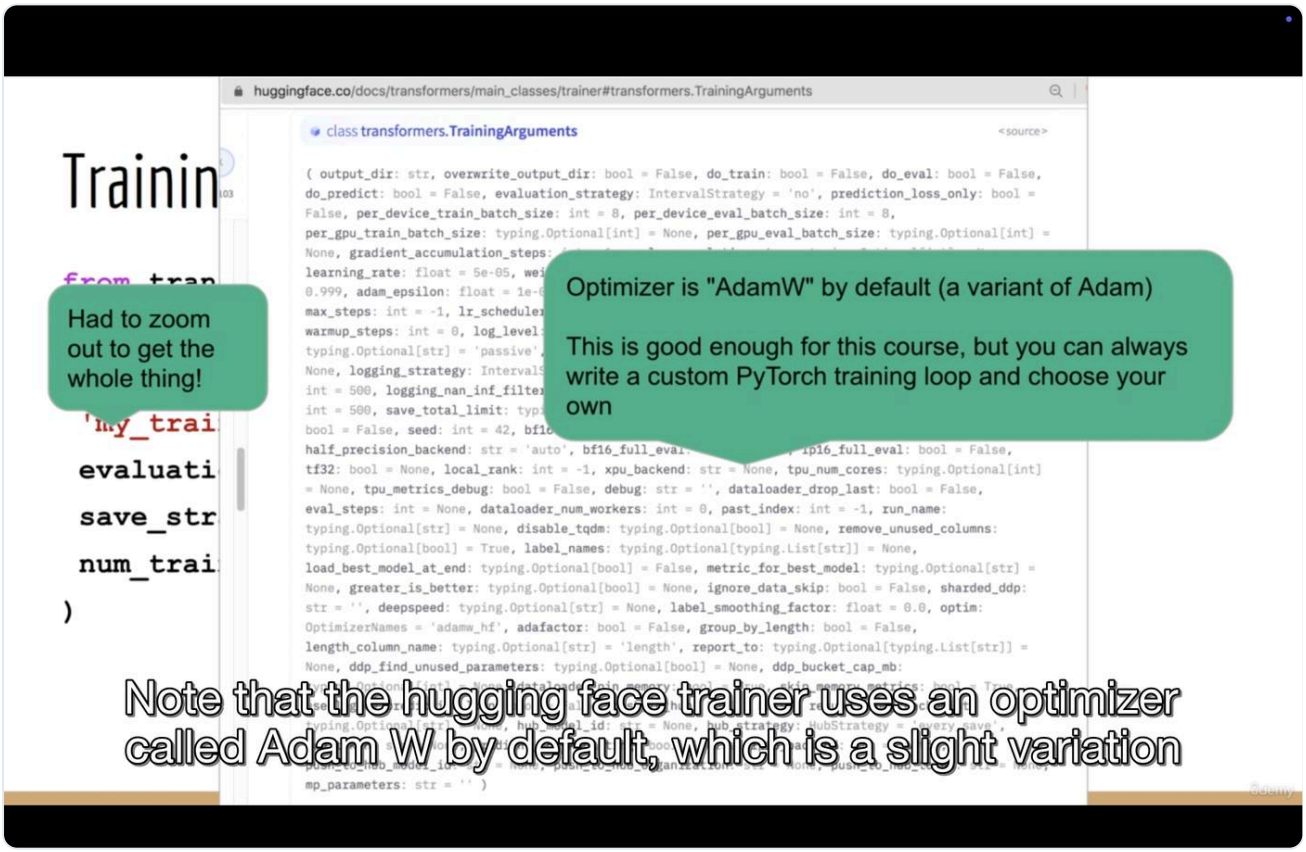

FIGURE 4.117 | Had to Zoom

Note that the Hugging Face trainer uses an optimizer called AdamW by default, which is a slight variation of Adam. This is essentially built in and it's good enough for our purposes so we won't try to mess around with it in this course. If you want to use a different optimizer, you can always write your own custom training loop in plain pytorch syntax as well. The next thing to discuss is the actual trainer object, which is what we will eventually use to run the training process.

The arguments to the constructor are quite simple. We pass in the model, the trainer arguments, objects we just discussed. The training data set, the validation data set and the tokenizer. As you recall, we discussed how to create the model and tokenizer in previous lectures. Note that the data set we pass in should be the tokenized version of the dataset, which is what we got back from the map method we called earlier where we passed in the tokenizer.

So why do we need to pass in the tokenizer again? Unfortunately, it's still needed for further computation. Personally, I don't think this is a great design, but that's the way it currently is. In any case, once you've created your trainer object training the model is very simple. Just call trainer dot train. Now, if you call the train method, as we just described it, you won't get much information other than the loss. But suppose you care about other metrics like accuracy, F1 score and so forth.

Well, discuss later on how to compute custom metrics, but for now we'll look at the Hugging Face way of doing things, which is to use a set of metrics that have been predefined for the data we are using. Specifically, there is a function called the load metric, also part of the data sets library. The arguments to this happens to be the same as the arguments into load_dataset for this particular example. More generally, however, you can pass in an argument to specify a specific metric like the BLEU score or a python file where you've defined your own computations.

So how does this work? Unfortunately, we cannot pass this metric into the trainer directly, which is another oddity about this library. Essentially, we need to define another function to do some extra work. Firstly, the metric we get back from load metric is an object which has a method called compute, which will return a dictionary containing the computed metrics for a given task. This compute method it takes in two arrays, the predictions and the targets, called references.

For some reason, this will give us back a dictionary with the value of the metric or metrics with their names as keys. However, as stated, we cannot just use this function directly. We need to write a separate function that takes in a tuple containing the output, logits and targets. So here have shown you how to do that. We begin by writing a function called compute metrics.

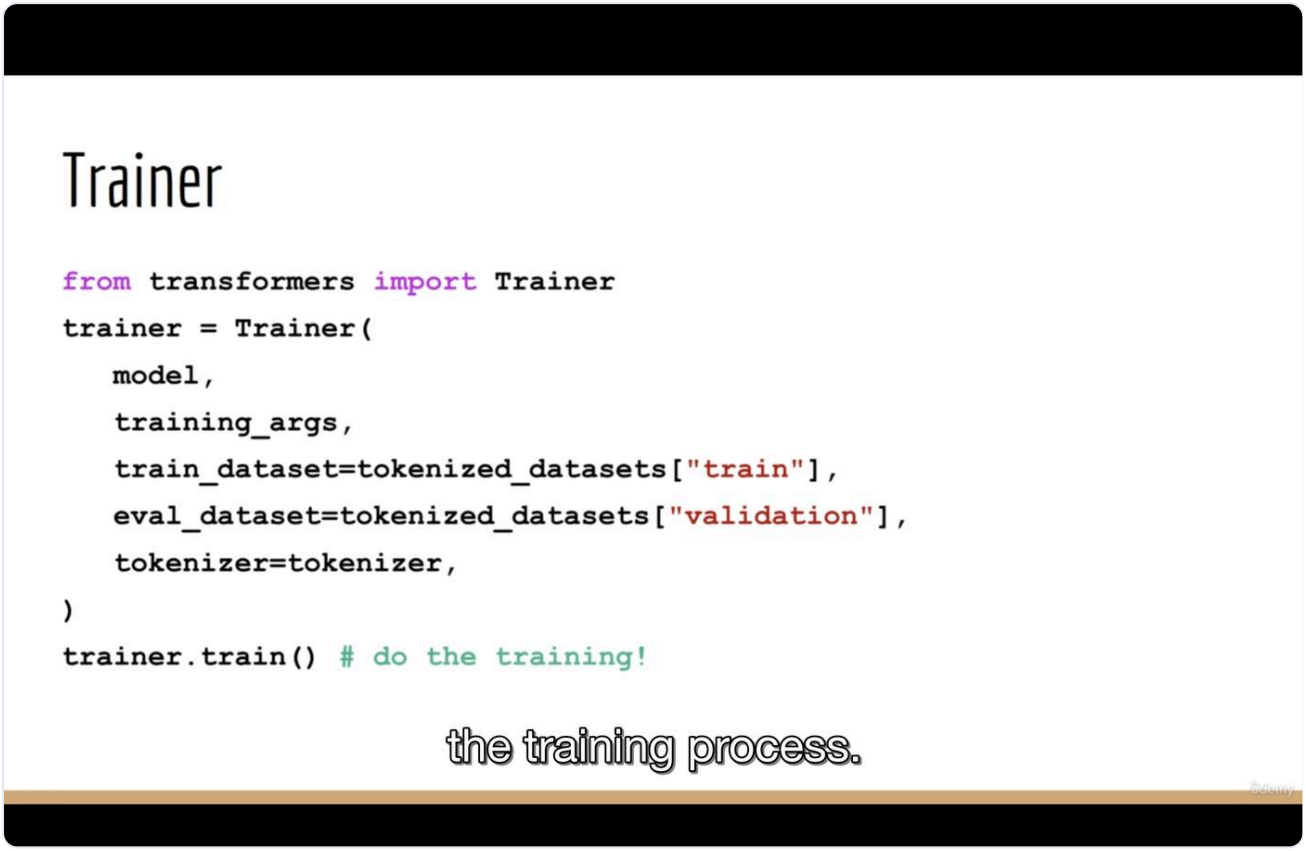

FIGURE 4.118

Although you can call this whatever you like. This takes in a tuple of logits and labels, which I've simply named logits and labels inside the compute metrics function we call load metric.

Although please note that it would be more efficient to put this outside. The next step is to unpack the logits and the labels to be separate variables. Once we've done that, we can call the argmax function on the logits to get actual predictions. Once we have the predictions, then we can compute and return the metrics. Okay. So once we have this compute metrics function, using it with the trainer is simple. We just pass into one additional argument into the constructor called appropriately enough, compute metrics.

After doing this and calling Trainer Dot Train as shown earlier, you will see the additional metrics being printed out at the frequency you specified with the trainer arguments. Okay. So the final thing to discuss in this lecture is what you do with the model. After you've trained it, you probably will want to save the model and use it on data you encounter in the future. Saving the model is simple. We just call trainer.save_model passing in the path where you want to save the model.

This will output several files which will store the model weights configuration, tokenizer data and so forth. Once you've saved the model, using it is even easier. You recall that pipelines are a universal interface that can be used to apply any trained model directly to text. Of course, we can use pipelines with our own trained models as well. To do this will create a pipeline by first specifying the task, which in this case is text classification.

The model argument should be set to the path where we saved our model, and from here the pipeline can be used just like we learned earlier. You pass in text and you get a result. Now there are a few subtleties which we haven't yet discussed, but we will see them in the next lecture.

9 · Fine-Tuning Sentiment Analysis in Python

See the notebook here.

So in this lecture we will be looking at the code to implement fine tuning for sentiment analysis. We'll begin by installing not just the Transformers library, but the Datasets library as well. The next step is to import the load_dataset function as well as numpy. The next step is just a comment, noting that you could also try running the script with the Amazon polarity dataset, which is another dataset for sentiment analysis. However, this data set is larger and would take more computation.

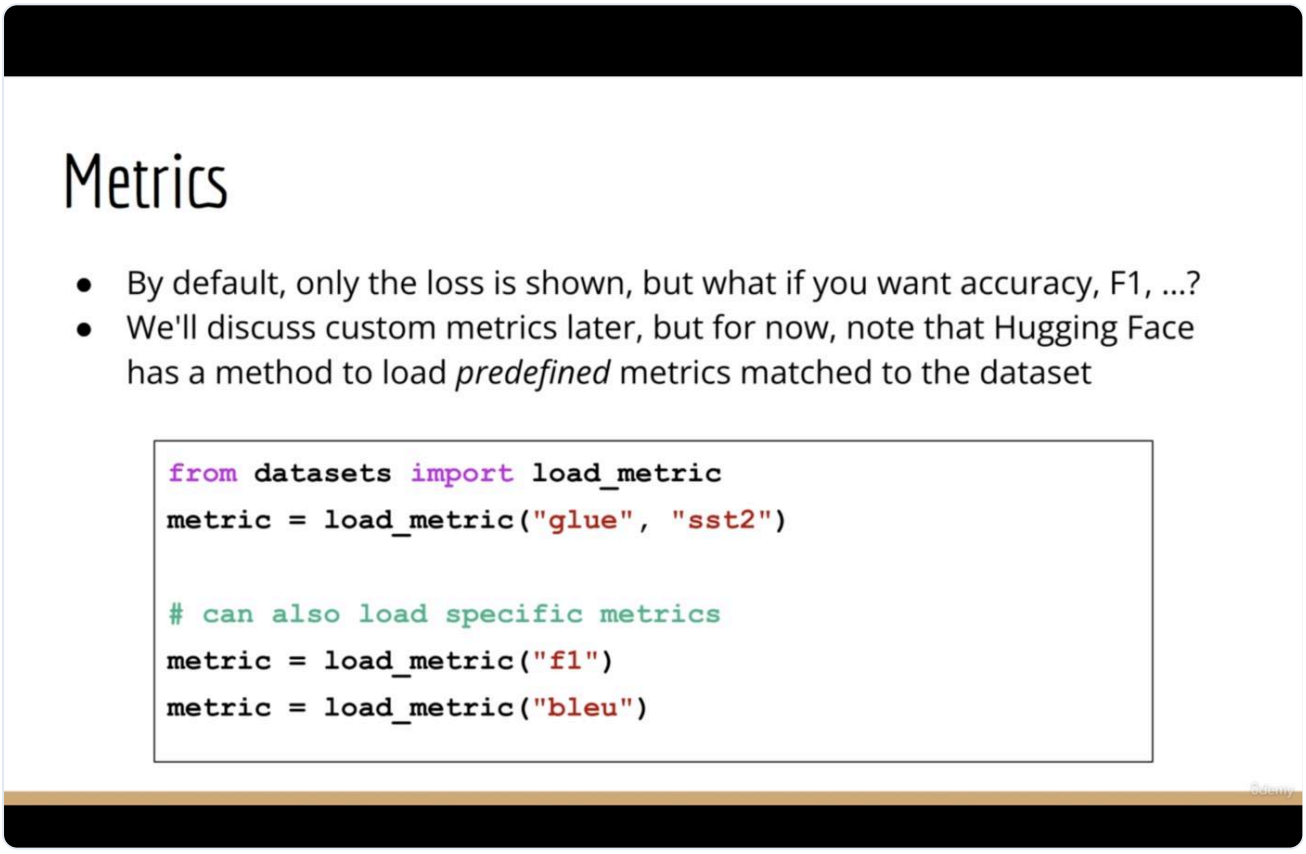

FIGURE 4.119

The next step is to call the load_dataset function, specifying that we want the GLUE dataset and specifically the sub task SST-2. The next step is to just print out the variable. We just got back to see what it is. As you can see, we get back a data set dict object which contains dataset objects. There are three such data set objects one for train, one for validation and one for test. The next step is to demonstrate how we can select one of the data sets, since our variable behaves like a dictionary.

We should be able to index it using the above keys. In this instance, we'll look at the train dataset. So as you can see, this tells us which columns exist for this dataset and how many examples it contains. The next step is to run the dir function on the data set object. This is useful when you want to see what attributes and methods that the object has. So feel free to look through these on your own as you may find something useful. Some interesting things are that we can save the data set to disk count the number of rows and columns, shuffle the examples converted to CSV, convert it to JSON and do a train to split.

The next step is to simply check the type of objects we have, which tells us where this class lives in the data, says library. So this might be interesting if you ever want to look up the documentation. The next step is to check the data attribute, which was one of the items above. So this is interesting because it tells us the type of each column along with showing us some examples. Note that we can also index the data set object as if it were a list or an array.

So as you can see, the first example is what we also saw. But notice that this contains in both the model input, which is the sentence along with the corresponding target. So here's another example showing that we can also index the data set with a range of indices. As expected, we see three sentences along with three corresponding labels. Although do note that we get a dictionary of lists rather than a list of dictionaries. The next step is to check the features attribute, which will show us some useful information.

So this tells us not just the type of each column in the dataset. But notice for the label, it also tells us the names of the classes. This is important because recall that for PyTorch models, the class predictions just come out as zero one, two and so forth. At some point we'll want to know what those class integer IDs actually mean. This shows us that the zero class is negative, whereas the one class is positive.

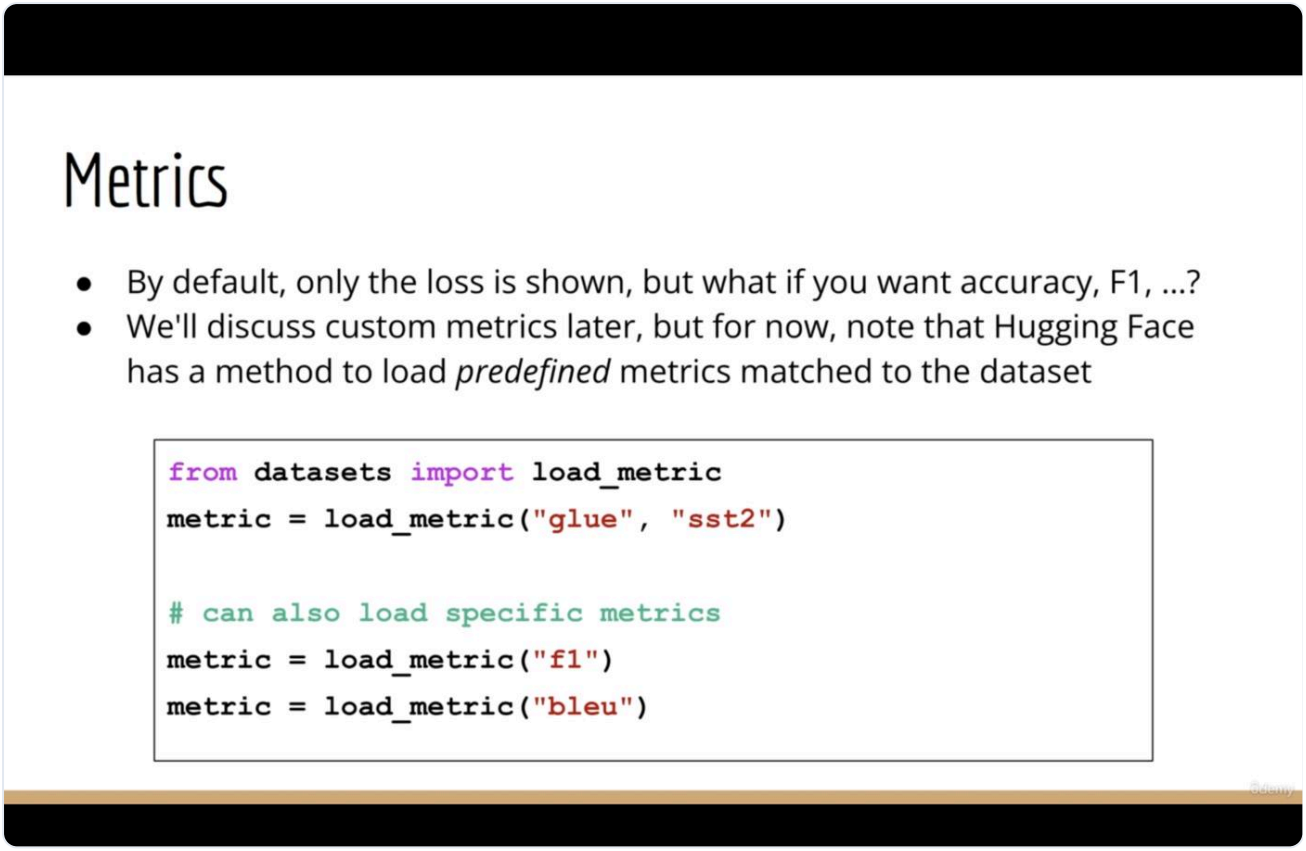

FIGURE 4.120

The next step is to import the AutoTokenizer class.

The next step is to define the model checkpoint we want to use and load up the pre-trained tokenizer for this model. I've chosen to DistilBERT instead of BERT since it trains faster. Please feel free to try other model checkpoints if you like. The next step is to test our tokenizer on a subsample of our sentences. So as you can see, this returns a dictionary containing the attention masks and the input IDs. Note that there were no token type IDs because that's only for BERT and not the DistilBERT.

The next step is to wrap our tokenizer with a new function. The point of this is so that we can pass in the argument. Truncation equals true. The next step is to call the map function and to map this tokenizer function to each of the samples in all of our datasets. We'll call the result tokenized datasets. So this is what will be passed into the trainer later on. The next step is to import the training arguments class. The next step is to define a training arguments object.

Note that this model will take quite some time to train and it will begin to over fit with just a few epochs which you might want to confirm yourself as such will set the number of epochs to one. In fact, for pretty much every example in this course, we're going to train for a very few epochs. Note also that I've set evaluation strategy to epoch as well as save strategy to epoch. Evaluation strategy is needed if you want to actually evaluate your model on the validation set during training.

The default is not to do this, which is not desirable. In our case. We also set save strategy to epoch, which saves the model at the end of every epoch by default. So this will save the model on each training step, which is way too much, and will result in you running out of space on CoLab.

The next step is to import the class auto model for sequence classification. The next step is to load a pre-trained model specifying the checkpoints, as well as explicitly specifying the number of labels for good measure.

So as shown earlier, we get a warning telling us that we need to train the final weights because they have been initialized randomly. This is to be expected since that is what we intend to do. The next step is to check the type of our model object. So as you can see, this returns a specific class, which is the DistilBERT for sequence classification. The auto model is able to return many different types of objects, and the type of objects that is returned will be based on the check point you passed in earlier.

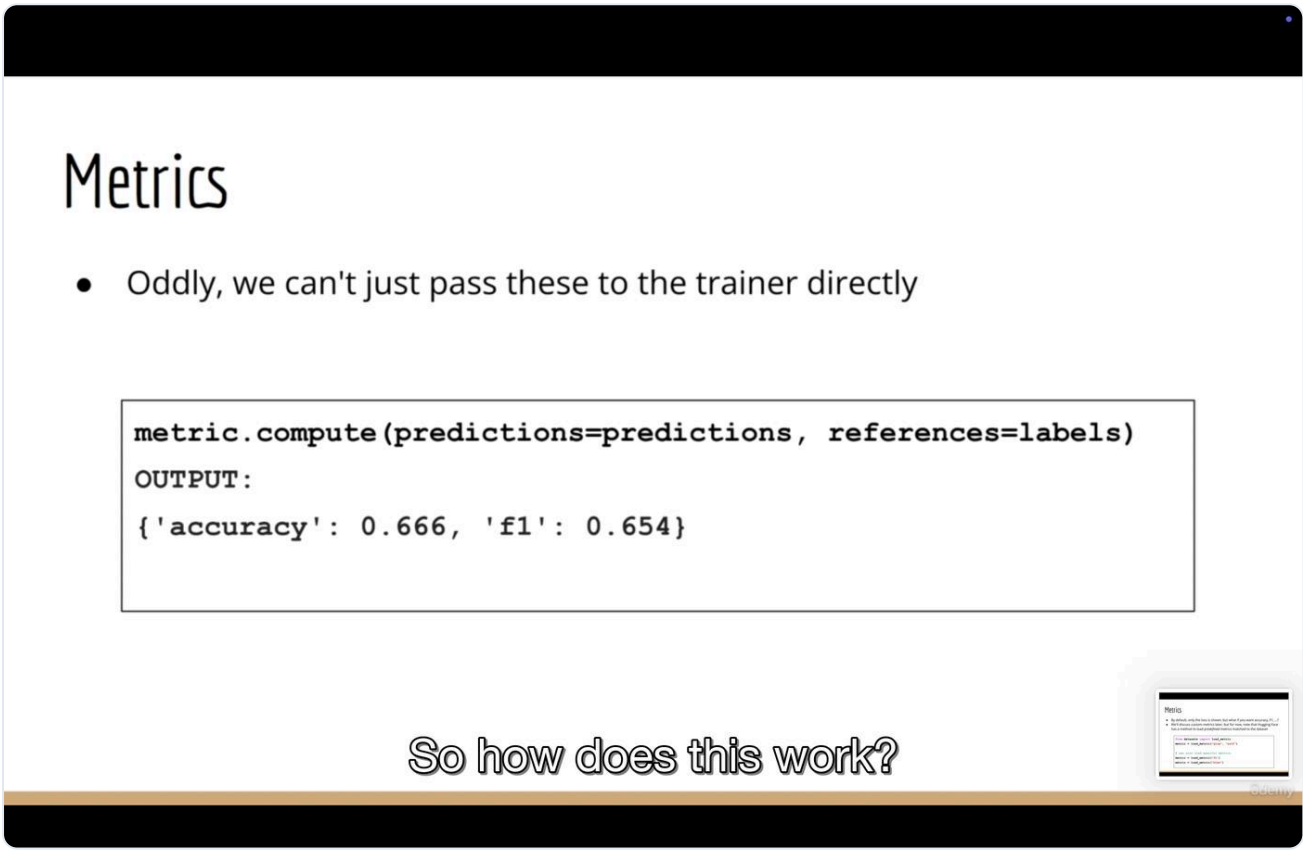

FIGURE 4.121

If you've ever studied C++, this should remind you of the auto variable type which allows you to use the word auto when you define a variable. Instead of explicitly defining the type yourself. The next step is to just print out the model object itself. So as you can see, this returns a kind of summary of the model telling you some details about each layer that's involved along with the input and output dimensions of the built in layers. Also useful is the summary function which is included in a package called torchinfo.

This is an install on Colab automatically, so we're going to install it now. So the next step is to import the summary function and call it on our model. So you'll notice that the summary function does not print out the data shape at each layer output by default, like what usually happens with TensorFlow. This is unfortunate because that information is really quite useful. So this is one area where TensorFlow beats PyTorch. Also unfortunate is that this library does not work well with Hugging Face models.

In particular, you'll notice that some code is commented out. Contrary to what I said earlier, the summary function can in fact show the output shape from each layer. But the caveat is that you also have to specify the input shape of the model. You can see that I've done so in the comments. Unfortunately, it doesn't work as advertised and you need to specify a few more arguments to actually get it to work. Furthermore, even with the right arguments, it still won't work if your Colab instance is running with a GPU, which it is right now.

It will only work if everything is on CPU. So if you are running this yourself and you want to see the output shapes by uncommenting this line, you'll have to change the runtime type to CPU. Now there's one thing I wanted to do as a sanity check. Recall I said earlier that when we fine tune transformer models, we actually end up training all the weights in the neural network, not just the new untrained weights. In order to verify that this is true, we are going to save all the model parameters before we begin the training process.

These are stored inside a list called params before. The next step is to import the trainer class. The next step is to import the load metric function. The next step is to call the load metric function passing in our task. Our goal right now is just to play around with this and see what it does. So in this block of code, we've passed in some dummy lists just to see what we get back. So as you can see, for the SST-2 task, the only official metric is accuracy.

Note that this function returns a dictionary.

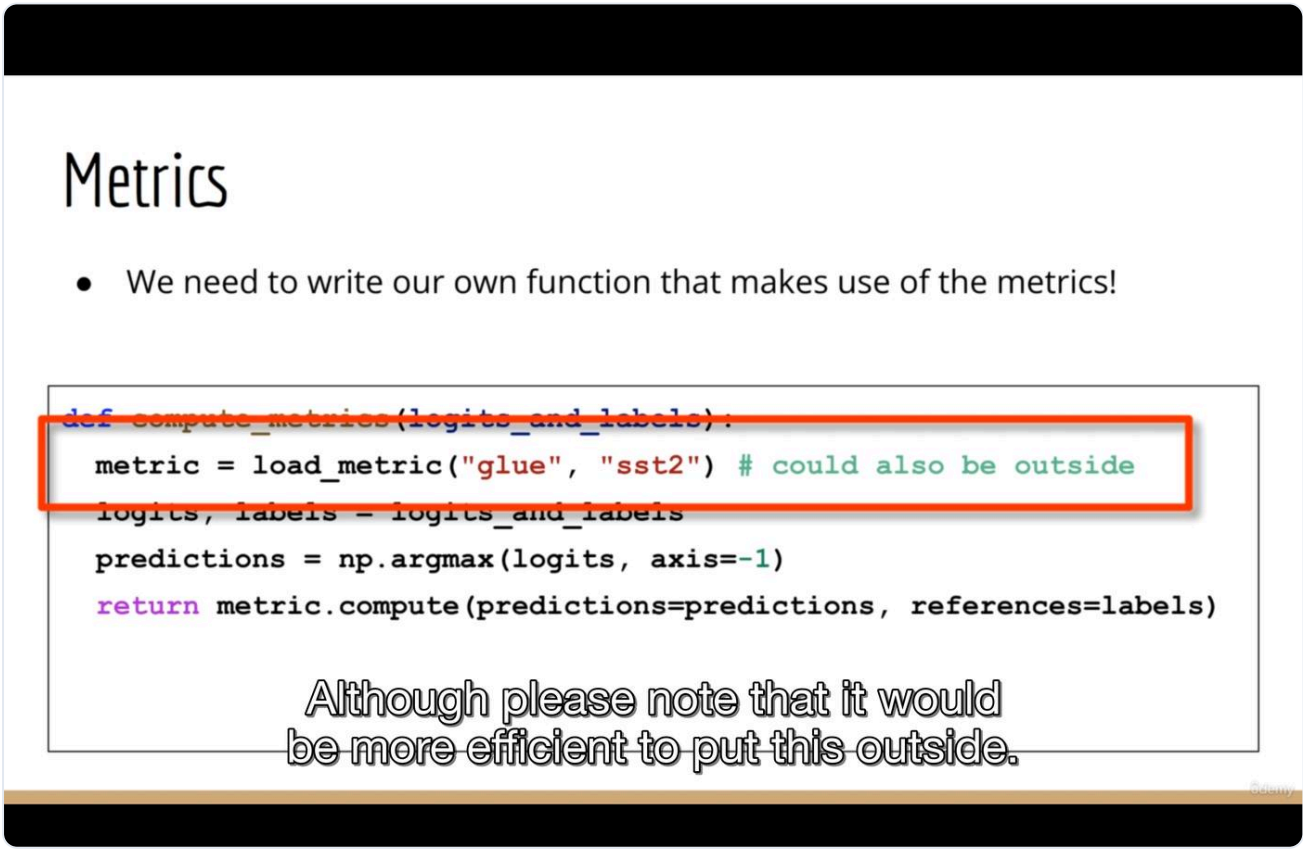

FIGURE 4.122

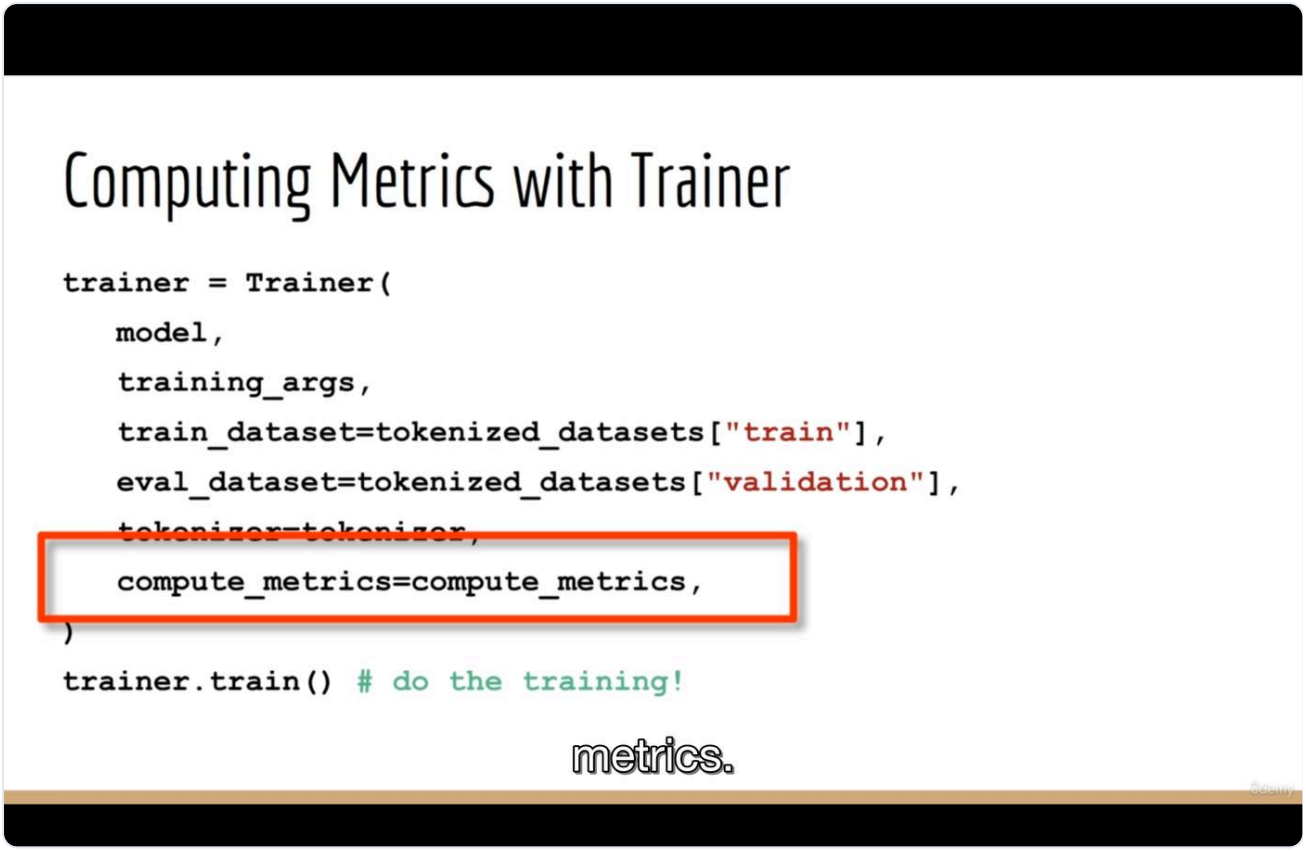

FIGURE 4.123 | Computing Metrics with Trainer

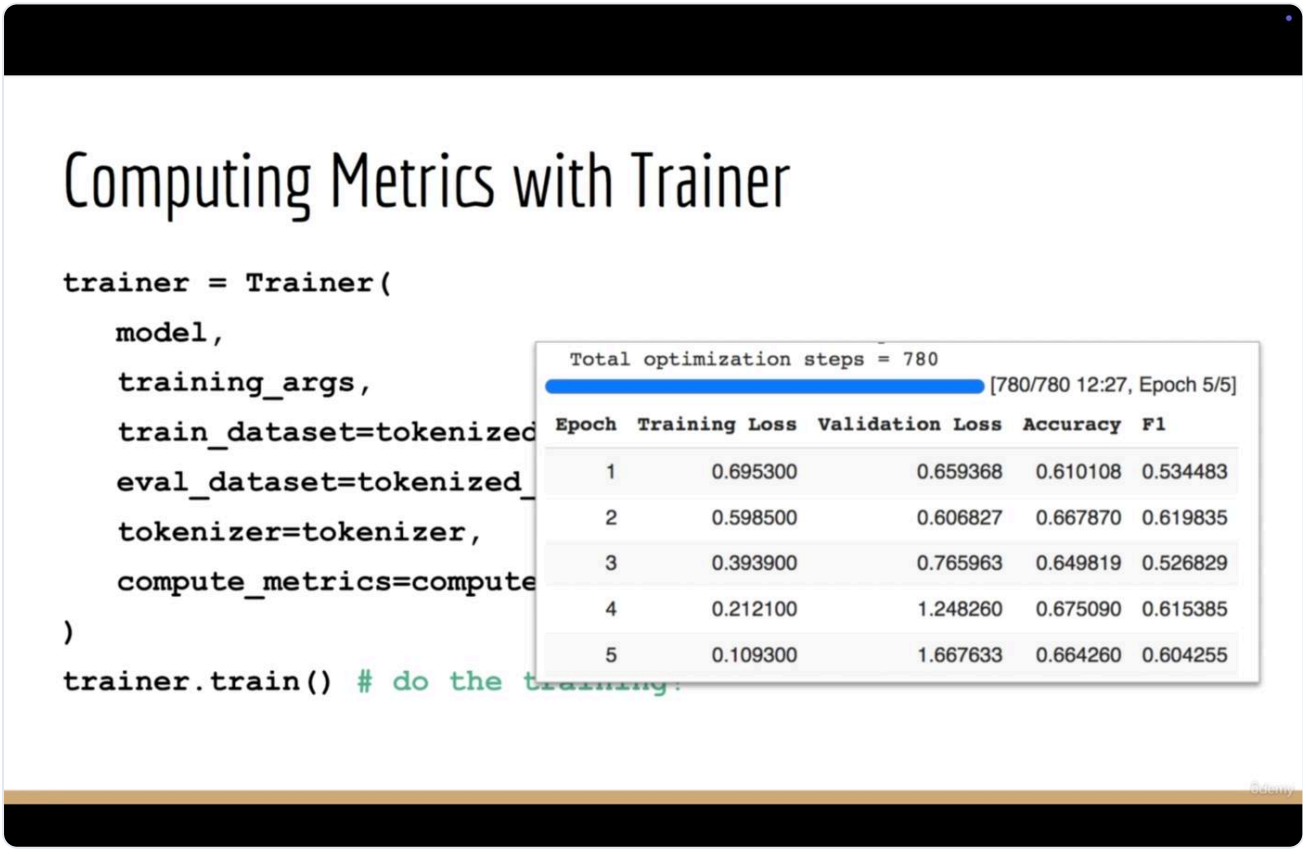

FIGURE 4.124 | Computing Metrics with Trainer

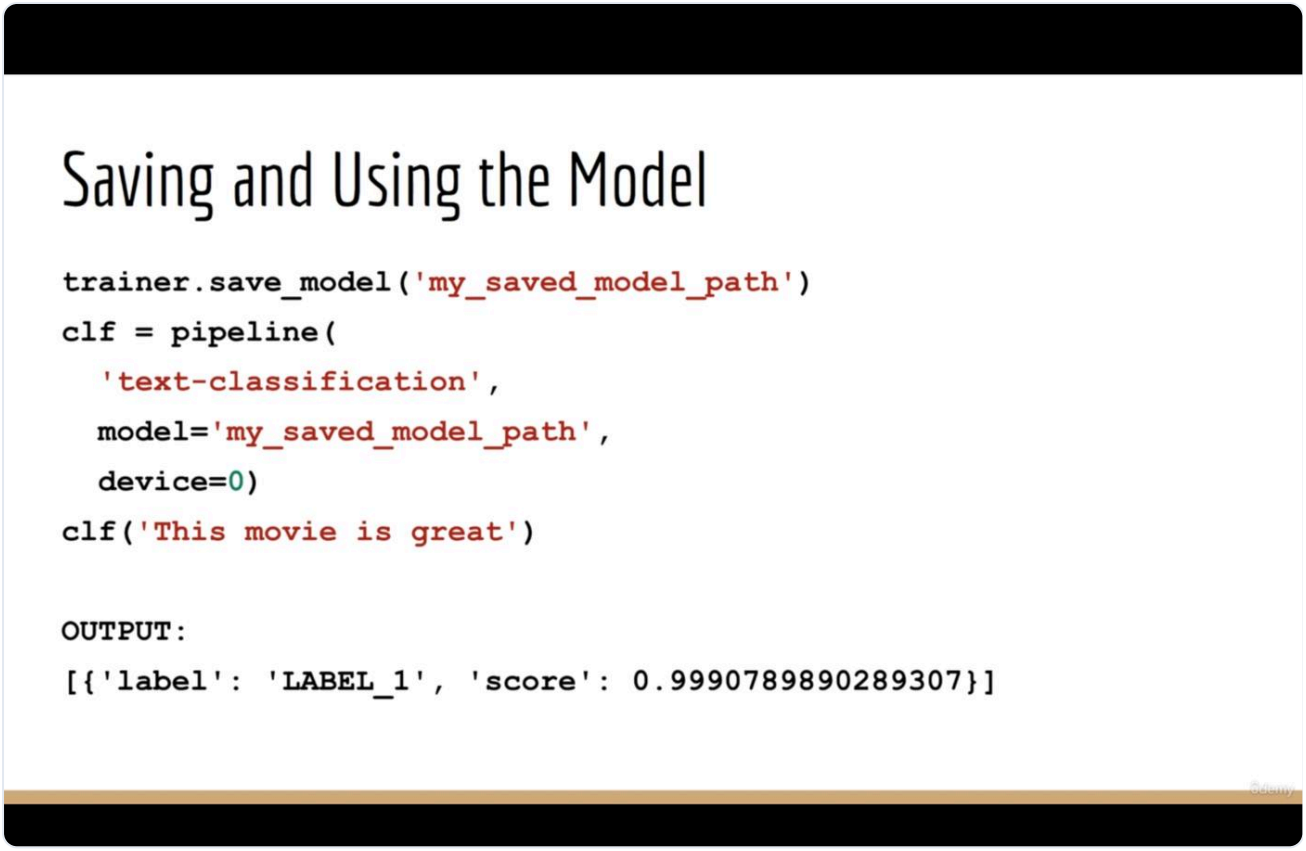

FIGURE 4.125 | Saving and Using the Model

So if we wanted to write our own function instead of using the built in ones provided by Hugging Face, we would also want to create a dictionary with the same format. The next step is to define our own compute metrics function a since as you recall, the API for metrics compute above oddly does not match with what is expected by the trainer. So in this function we'll take in a tuple containing the logits and the labels inside the function.

We begin by separating the logits and the labels. We then call the argmax on the logits in order to get class predictions. Finally we call metric.compute passing in the predictions and the labels. So at long last, it is time to finally create our trainer object. We pass in the model tokenizer training, our compute metrics function and the tokenized data sets we defined above. The next step is to call trainer.train to begin the training process.

Note that this takes about 7 minutes to finish. Depending on the GPU you get. This should take about 14 minutes. Note that it will take approximately double the time if you use BERT instead of DistilBERT. Okay. So in total, the training took about 15 minutes. So once you've trained your model, you'll likely want to save it. So that's what we're going to do next. So as you can see, we are going to save it into a directory called My Saved Model. The next step is to run the ls command to see that our saved directory exists.

The next step is to run the ls command again. inside the saved directory. As you can see, these are the files that make up our model pipeline, which include and not just the model itself, but the tokenizer data as well. So the next step is to demonstrate how to actually use our trained model.

This brings us back to the first section of this course on pipelines. Since we can use pipelines for any saved Hugging Face model, there is no need to worry about explicitly defining a tokenizer model.

We'll begin by importing the pipeline function. The next step is to create our pipeline object. We specify the task as text classification, the model as the directory where we saved our model and the device as the GPU. Okay. So since this model is a sentiment analyzer, we can test it with generic sentences like This movie is great. As you can see, our model predicts label one, which corresponds to the positive class. Now let's try a negative example. This movie sucks.

So as you can see, our model correctly predicts label zero, which corresponds with the negative class. Now you might be wondering why are the labels generically named? This is probably not desired behavior. Technically, we passed on the label names into the trainer objects earlier, so it should have been able to save those label names when we saved our trained model. Unfortunately, it does not. If you really need to have the correct label names output by your pipeline.

One simple method I came across was to simply edit the config file yourself. Here, I've run the cat command on the config json to show you what it looks like. Importantly, it does not contain any information about label names. The next step is to import the JSON library. So in this block of code we are going to edit the config file ourselves. Specifically, we start by loading in the file and using the JSON library to convert it into a dictionary. We then add a new key called id2label, which contains the mappings from label id2label name.

Finally, we save the JSON back to the same file path. So note that if we cat the config file again, you will see the new information we just added. So as you can see, it shows up right here. The next step is to load up our pipeline again using the same code as before. We can now run our model again to see whether or not it will give us better label names. And as you can see, we get a more informative label name. Now there's one thing left to do in this script, which is to check whether or not all of the weights in our neural network were changed during training.

We're going to run the same code from earlier to get all of the model parameters as NumPy arrays. We'll save this to a list called the params after. The last step is to compute the sum of the absolute differences of each parameter. So as you can see, all of these sums are non-zero, which confirms that in fact all of the model weights were updated during the training process.

10 · Fine-Tuning Transformers with Custom Dataset

See the notebook here

So in this lecture we will be looking at how to do text classification on your own custom dataset. The previous lecture looked at how to use data sets built into the Hugging Face platform. But this lecture will look at how you can do the same thing for any dataset, which is much more powerful. Let's begin by installing the transformers and data sets libraries as we did before. The next step will be to download our dataset, which is a dataset of airline tweets.

Again, this is a sentiment analysis dataset. The next step is to import the various libraries and library methods will be using in this script. The next step is to load in our data using pd.read_csv. The next step is to call the head method to remind ourselves what this dataset looks like. So you recall that we're interested in the columns, airline sentiment and text. So this is the airline sentiment, which is the label. And here we have the text, which is the tweet.

The next step is to select only the columns we need. The next step is to call the head method again to see what our new data frame looks like. So as you can see, it has the two columns we selected. The next step is to check a histogram of the labels. So as you recall, this is an imbalance data set since the negative class outweighs the other two. We'll see whether or not our model can handle this or if it's biased towards a particular class. The next step is to map the labels to integers, which is required for neural networks.

The next step is to save our data frame to a new CSV file, since surprisingly we'll be able to make use of the load_dataset method which you saw before. However, our CSV should have a special format. In particular, you'll notice that I've created a new dataframe called df2, which is a data frame of the text and the new target column. Importantly, notice that I am renaming the columns. The first column corresponding to the text is not that important because the inputs are pretty flexible.

What is surprising, and also not well documented, is that your targets must have the column named label I would recommend as an exercise. Not doing this renaming just to see what happens. The next step is to run the head commands on our new CSV. So as you can see, we have a two column CSV with the text in the first column and the label in the second. The next step is to import the load_dataset method. Once we've done that, we can use it to load in our CSV as a Hugging Face dataset.

Note the syntax for this. The first argument is simply a string that says CSV. While the second argument for data files specifies the path to our new CSV. The next step is to print out our data set object just to see what we got back. Okay. So notice that the format is similar to what we had before. However, note that we only have a single dataset called Train. This is because we haven't yet split our data into train and test sets. We'll see how to do that shortly.

So the next step is to split our data into train and test sets. Recall that the data set has an instance method called train test split. We can make use of this method to split our dataset. Notice that the arguments are similar to scikit-learn's method of the same name. Let's now print out the split object just to see what we got back. Okay. So notice that we get another dataset dict object with both train and test sets. This has the same format as what we saw in the previous lecture.

Now just for completion sake. I've included some comments to show you some other ways that you can load in your own data. Firstly, suppose that you have multiple CSV files. If this is the case, then you can just pass in a list containing the paths to these files. Furthermore, note that these paths can also be URLs. So another thing you can do is if your data is already split into train and test to pass in a dictionary specifying the train and test data separately.

Note again that these can either be lists of paths or just a single path. And furthermore, note again that they can be URLs. The next step is to define the checkpoint for the script, which will be the DistilBERT in this case. You're welcome to try regular BERT as well. As you recall, DistilBERT is like a lightweight version of BERT. The next step is to import the AutoTokenizer class. The next step is to call from_pretrained passing in our checkpoint to get our tokenized object.

The next step is to define a tokenize function like we did before. Note again that this does truncation but not padding, since padding will be handled automatically by the trainer. The next step is to apply the tokenize function to our train test split by using the map method. The next step is to import the auto model class as well as the trainer class and the training arguments class. The next step is to load up our model from the above checkpoints. Note that the number of labels is three.

The next step is to install torchinfo so that we can call the summary method on our model. The next step is to import the summary method. The next step is to run the summary method on our model. What should be interesting about this is the number of parameters. Okay. So our model has about 66 million parameters. Oh. And notice how they are all trainable. The next step is to create our training arguments object. For this example, we are going to explicitly specify the batch size for both training and evaluation.

The next step is to define our compute_metrics function. Remember that we used preloaded metrics which corresponded to the built in dataset we were using. But this time there is no such metric because our data set is custom. However, we can use the same approach because we know what this function should return. As you recall, it returns a dictionary with the name of each metric as the key and the value of those metrics as the value. We begin by first splitting the logits and the labels.

Next, we grab the predictions by using the argmax function on the logits. The next step is to compute the accuracy and then we compute the F1 score. Finally, we return a dictionary containing both the accuracy and the F1 score. The next step is to build our trainer object, passing in the same arguments you saw previously. The next step is to call trainer.train, which will begin the training process. Okay. So this is going to take about 5 minutes. Okay.

So notice how our models seem to over fit a bit on the third epoch. However, this doesn't mean that we have to start all over again. Since the model was saved at the end of each epoch, we can simply take the best model from one of the intermediate steps. So the next step is to run the ls command in the training dir, which is where we save our training data. So as you can see, there are three checkpoints named after the number of training steps that were taken up to that point.

Since we want the model from the second checkpoint because it had the highest validation performance. We want checkpoint 128 to. The next step is to import the pipeline function. The next step is to instantiate a pipeline. Again, the task is text classification. This time the model is in a model we saved explicitly, but the path to a model saved after the second epoch. So the next step is to remind ourselves how to access the test set. The next step is to actually run the test set through our saved model in order to get the test predictions.

Of course, in the real world you would be using this on data you haven't yet seen. So the next step is to print out the predictions, to remind ourselves of the format. As you recall, the pipeline that doesn't just return class labels or probabilities. So as you can see, the label appears as LABEL_ and then the label ID. So we can compute any metrics directly on these results. The next step is to define a function called get_label, which will convert the dictionary items above into an integer label will then call this function for every item in our predictions list to convert our predictions into a list of integer labels.

The next step is to check the accuracy of our predictions. Note that this should match our results from the second epoch above. The next step is to check the F1 score as well. Again, this should match the F1 score from above. The next step is to plot the confusion matrix for our train model on the test sets. Recall that this is an imbalanced dataset, so we would like to see if it is performing better on one class compared to the others. Also recall that scikit-learn changed their API for the confusion matrix plot.

So in order to avoid this code not working for people with different versions, we are going to make our own. You can feel free to change this to scikit-learn's confusion matrix plot if you like. Now. Since the F1 score was less than the accuracy, we can be pretty certain that there is some imbalance in terms of how well our model performs for each class. Notice how when the target is negative, we do very well predicting that it is negative. On the other hand, when the target is neutral.

We don't do that well in predicting that it is neutral. Instead, we incorrectly predict that a lot of those tweets were negative. When our target is positive our incorrect predictions seem to be evenly split among the negative and neutral. Okay. So as a final exercise for this lecture, here's what you should do. You recall that we looked at this dataset with a pre-trained pipeline. However, we only looked at the positive and negative classes. So as an exercise, remove the neutral class from this example and check the performance compared with the previous results to see which performs better.

The Fine Tune model or the pre-trained model?

11 · Hugging Face AutoConfig

See the notebook here

So in this lecture, we'll be investigating something that came up earlier, which is why the fine tune model outputs only generic label names. As you recall in the previous lecture, we solved this in kind of a hacky way, which was to modify the config file after calling the save method. If you're a programmer, this might make you recoil in horror. Luckily, there is a slightly better way. Unfortunately, what you would hope existed doesn't actually exist.

Specifically, it would be nice if you could just pass in specific label names into the from_pretrained method, just as you can specify the number of labels. This would be ideal and in my opinion makes the most sense. But unfortunately, this currently isn't possible. However, there is a way to achieve a similar effect, which is what we'll look at in this lecture in particular. Hugging face also has config objects. We'll pass in this config object into the from_pretrained.

So it pretty much works like our ideal scenario. Note that these config objects are model specific like tokenisation. So you can have a BERT config, a GPT-2 config and so forth. As you might expect, there is also an auto config which automatically chooses the right config object based on the checkpoint you give it. Just as we can have AutoTokenizer and auto models. We also have auto configs. Now please note that most of this notebook is the same as the previous one.

So we'll skip to the relevant parts. So we'll begin by importing auto config along with the auto model and the trainer class. So recall that earlier in this notebook we've loaded in the data sets, converted them into the correct format and so forth. So the next step is to load up a config by calling from pre-trained passing in our checkpoint. The next step is to print out our config just to see what it looks like. So as you can see, it's sort of like a dictionary with keys and values.

Importantly, notice how there's nothing here about label names. Now, if you check the attributes of the config object, you'll see that there are two relevant attributes corresponding to labels, id2label and label2id. So let's look at id2label. As you can see, this is a dictionary mapping an integer label ID to the corresponding label name. As you may have expected. The next step is to look at the label2id. So as you can see, we get the reverse mapping, which again, you may have expected.

So it should be evident that what we need to do is overwrite these id2label in label2id attributes. Now the API for this isn't too great. In my opinion, there should be a function for doing this so that you don't have to manually overwrite attributes yourself. For example, you can just pass in gibberish and it would break your config. But since no such method exists, we'll just stick with what we can get. So you can see here that we're basically assigning these the targeted map from earlier in this notebook.

As you recall, the targeted map had our desired label names mapped to corresponding integer IDs. The next step is to call from_pretrained with their auto model to get back a model object. The difference between what we did before and what we are doing now is that we are now passing in the config object we just looked at. Okay. So essentially all of the remaining steps are the same as the previous notebook, so I won't bother to explain them again. Okay.

So at this point we fine tune our model, saved it and loaded it back in as a pipeline object. At this point, we can just pass in some strings and get back predictions. Okay. So our first input is JetBlue. Thank you. Predictably, the prediction is positive and the label shows up as the string positive instead of something like label one. So passing in the config object was a success. We no longer had to manually modify the config file.

12 · Fine-Tuning with Multiple Inputs (Textual Entailment)

So in this lecture, we are going to look at another example of text classification, which is slightly more complicated. In this example, we are going to build a text classifier where the input consists of not just one but two sentences. This is very practical and we can think of examples where we might want to have a model that knows how to process multiple input sentences at the same time. One example is answering multiple choice questions as input.

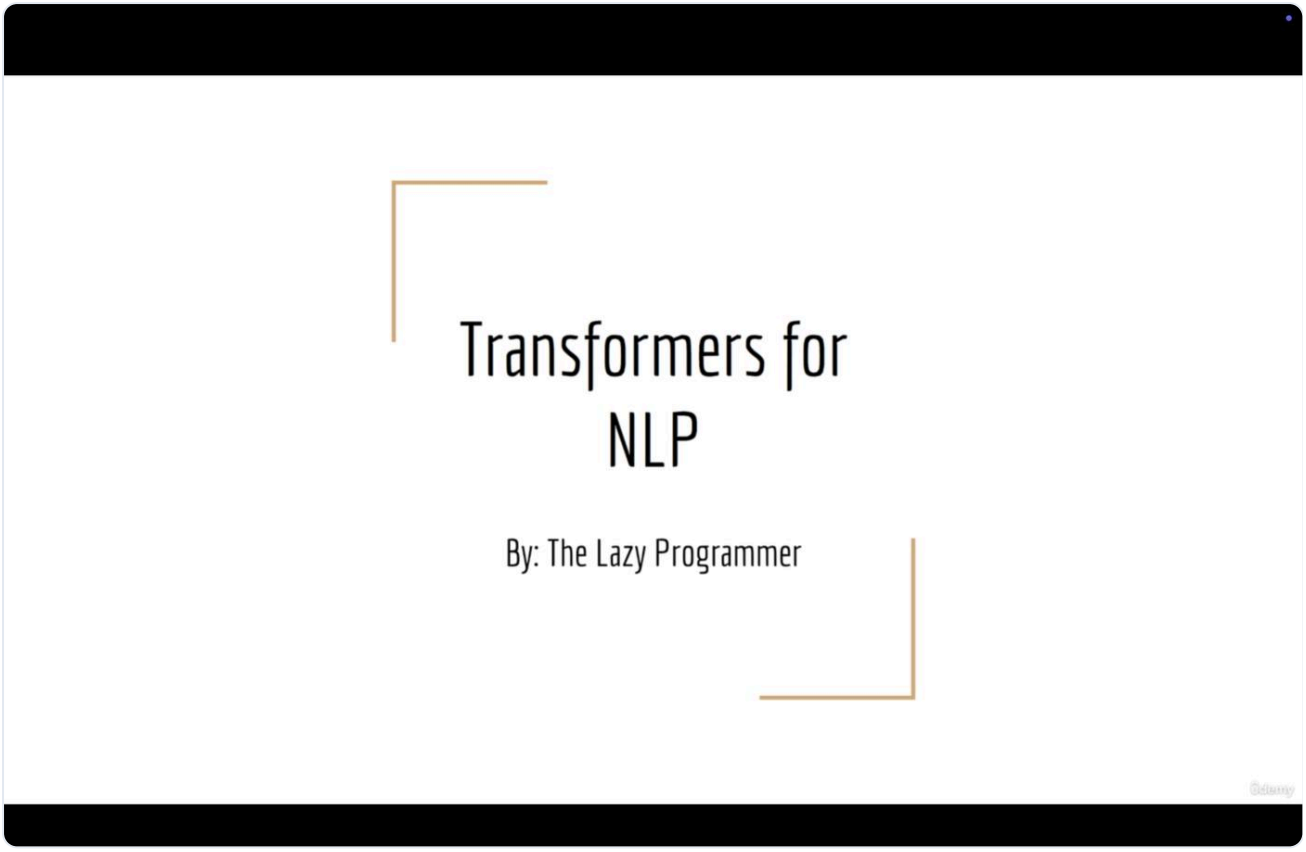

FIGURE 4.126 | Transformers for

We might want to pass in the question along with all the possible answers, and we want our model to select the right answer.

Another example is with chatbots. With earlier chatbots, a simple way to train them was with single prompt and response pairs. So you would give the chatbot a prompt and the chatbot would memorise appropriate responses. But this doesn't take into account the history of the conversation, which might make the conversation awkward.

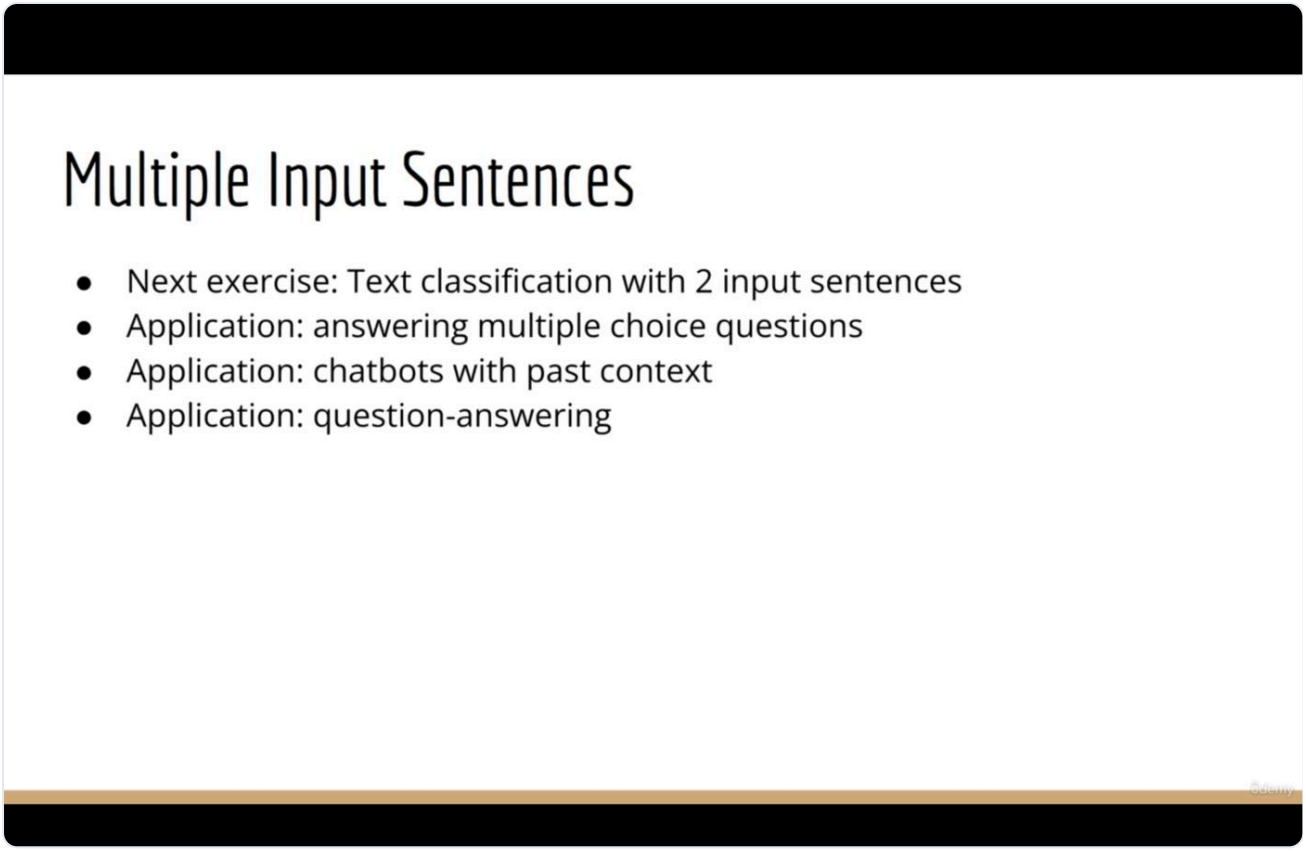

FIGURE 4.127 | Multiple Input Sentences

Another example is question answering where we pass in a question along with a passage of text which contains the answer.

We would then like the model to select the portion of the passage where the answer resides. Now you might be wondering how are we going to build a transformer that can handle multiple sentences as inputs? If you have experience with machine learning or neural networks, you might have some sense that this would be a difficult task.

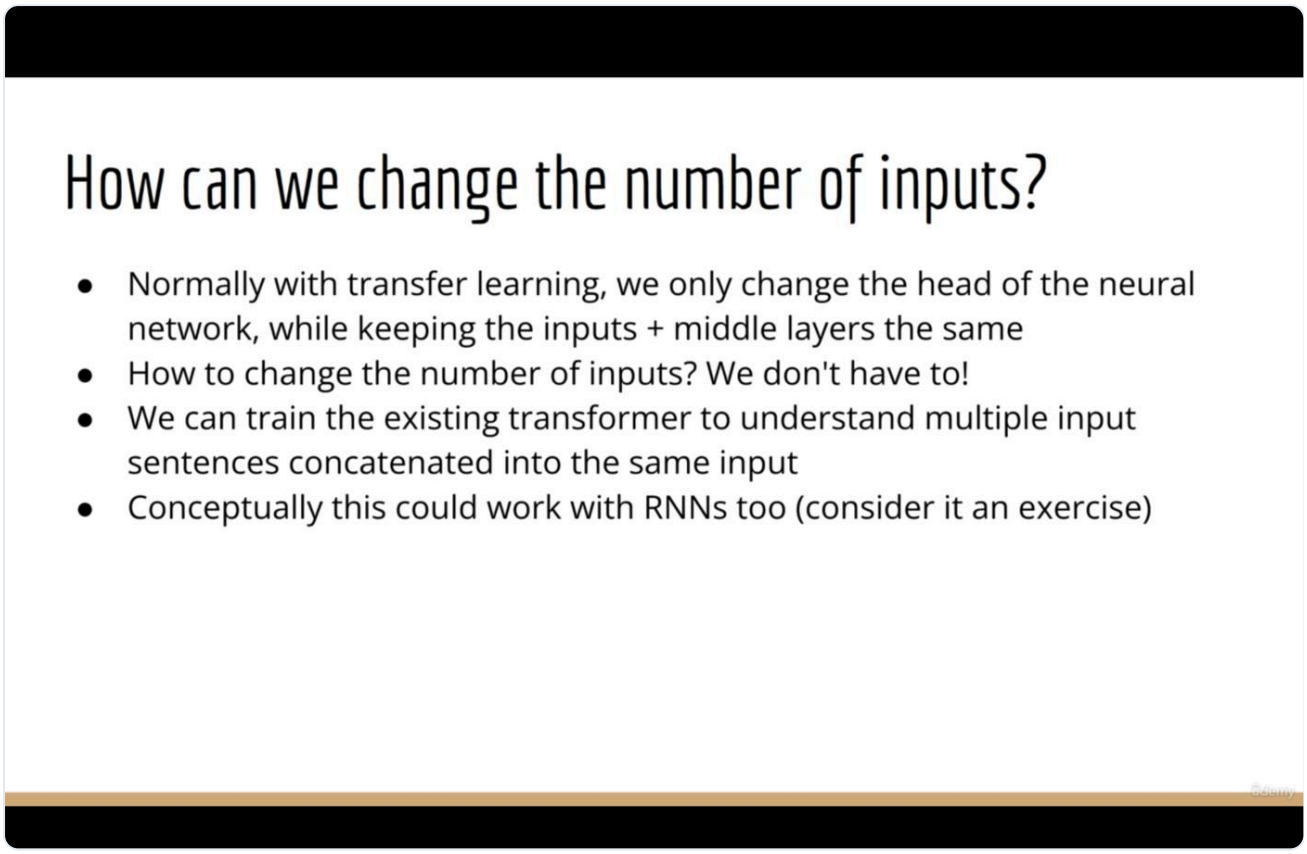

FIGURE 4.128 | How Can We Change the Number of Inputs?

Normally, with transfer learning, we only have to change the head of the pre-trained network while keeping the inputs in the middle layers of the same.

But how can we change the number of inputs? The answer is we don't have to. It turns out that with the single input already in place, we can train the transformer to understand having multiple sentences concatenated into the same inputs. In fact, conceptually, this could work with RNNs as well. I'll leave it as an exercise for you to think about how that might work. You'll recall that with BERT, which is the main model we are using in this section, is pre-trained on unsupervised NLP tasks.

Well, it turns out that one of these tasks involves having multiple sentences in the same input. So it makes sense that BERT should be able to handle multiple sentences. In particular, the task is called next sentence prediction. To build the data for this task, we take two sentences from our training corpus and the label is whether or not the second sentence follows the first. In other words, we build a binary classifier. For BERT, we use special formatting to help the model understand where the first and second sentences are located.

In particular, we always start with the class token. We then follow it with the first sentence. We then add the SEP token. We then follow that with the second sentence.

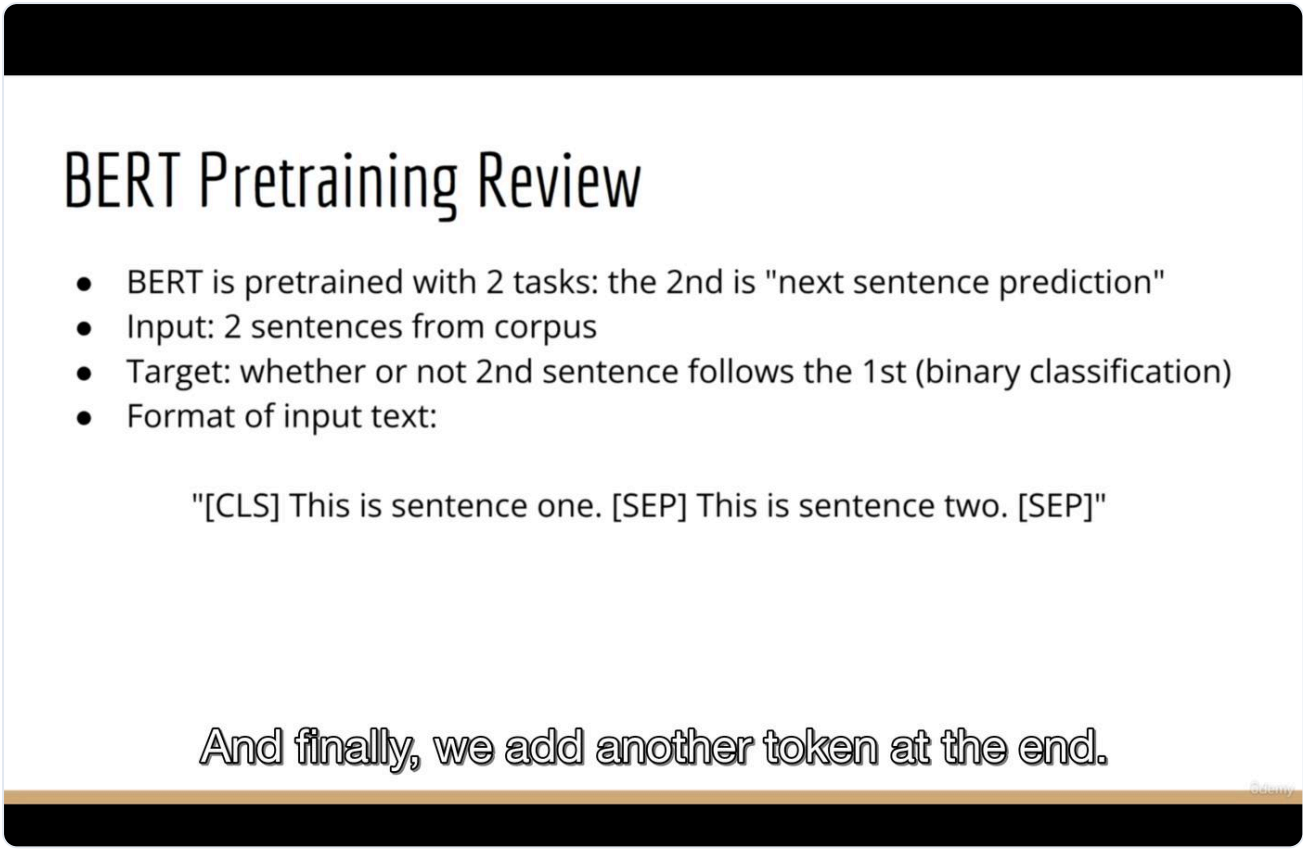

FIGURE 4.129 | Input: 2 Sentences from Corpus

And finally, we add another token at the end. So hopefully you now understand that the utility of these special tokens, when we only had one sentence as input, they may have seemed superfluous, but now they actually have practical utility. The task we'll be looking at next is known as textual entailment.

This might sound complicated, but in fact it is quite simple. Thanks to my rule, all data is the same. It is no different from the next sentence prediction task I just described. In particular, we're still going to have to input sentences and the target will still be binary. The only difference is the meaning of the task. In this case, we want to know whether one sentence entails another. For example, consider the input sentence pair. Bob buys a car and Bob owns a car.

This is an example of where the first sentence entails the second.

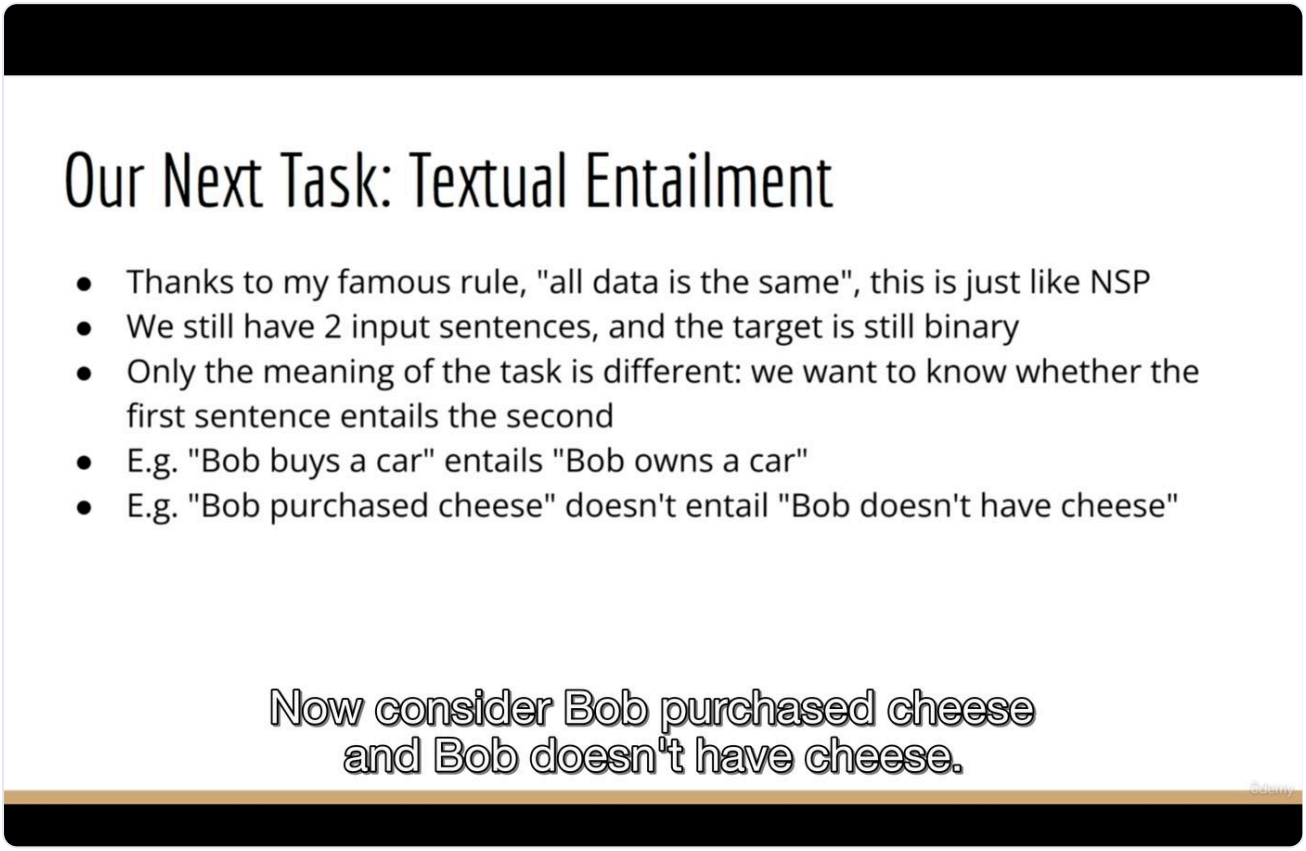

FIGURE 4.130 | Our Next Task: Textual Entailment

Now consider Bob purchased cheese and Bob doesn't have cheese. This is an example of where there is no entailment. Okay. So now let's discuss what this will look like in code. If you think about it, we know that the model has to be the same. What changes are the inputs? The input is still text and the output is still a binary prediction. What changes are the inputs? In other words, we should pay attention to the data set and tokenizer.

So the data set will look something like this.

FIGURE 4.131 | The Code: Dataset

Recall that you can think of it like a tabular data set with different columns for different things, just like a csv previously. We only had one input sentence and a label. This time we'll have two sentences and a label in our dataset. These are simply called sentence1 and sentence2. Luckily, the tokenizer from Hugging Face is well equipped to handle two sentences at once. You simply need to pass them in a separate arguments.

This will conceptually convert the inputs into a string formatted as we described earlier, where we have the class token in the first sentence. The SEP token. The second sentence. And another SEP token.

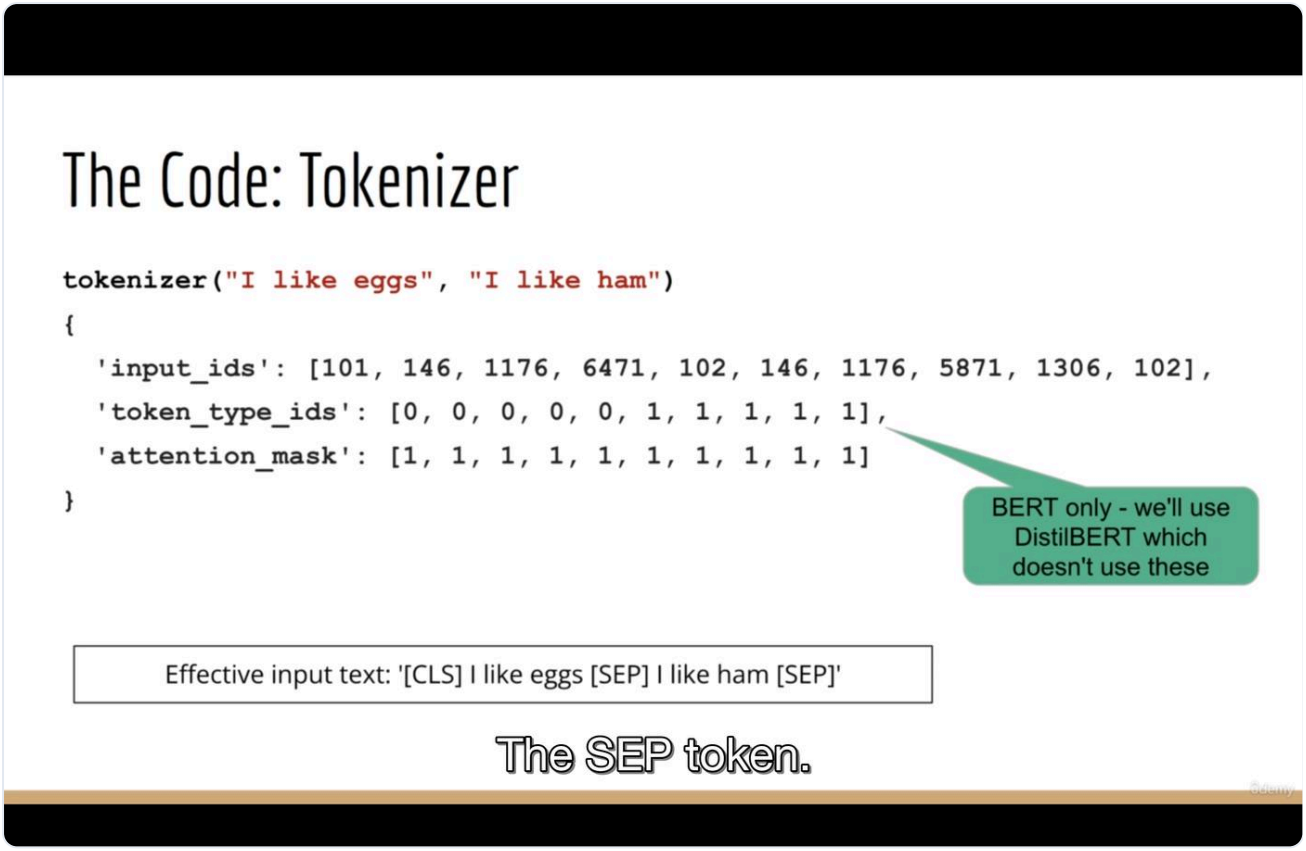

FIGURE 4.132 | The Code: Tokenizer

In reality, these of course, will be converted into tokens and then token IDs. Now there's one important note to make at this point. You'll recall that for the BERT tokenizer, we generate this input called token type IDs. Having two input sentences will help us understand that the role of this input when we use BERT and we have two sentences as input, the BERT tokenizer will use zeros for the positions of the first sentence and ones for the positions of the second sentence.

So now you know that this input actually has a purpose and isn't totally superfluous. However, this is an interesting point, and the next example will be using a slightly different model called DistilBERT. For this model, there is no input called token type IDs. It is simply not necessary as an exercise. After seeing the following notebook, you should double check that this is the case. when we use BERT, there will be an input called token type IDs, but when we use DistilBERT, there won't be.

Also compare the results to see which model performs better along with their speed of training.

FIGURE 4.133 | The Code: Tokenized Dataset

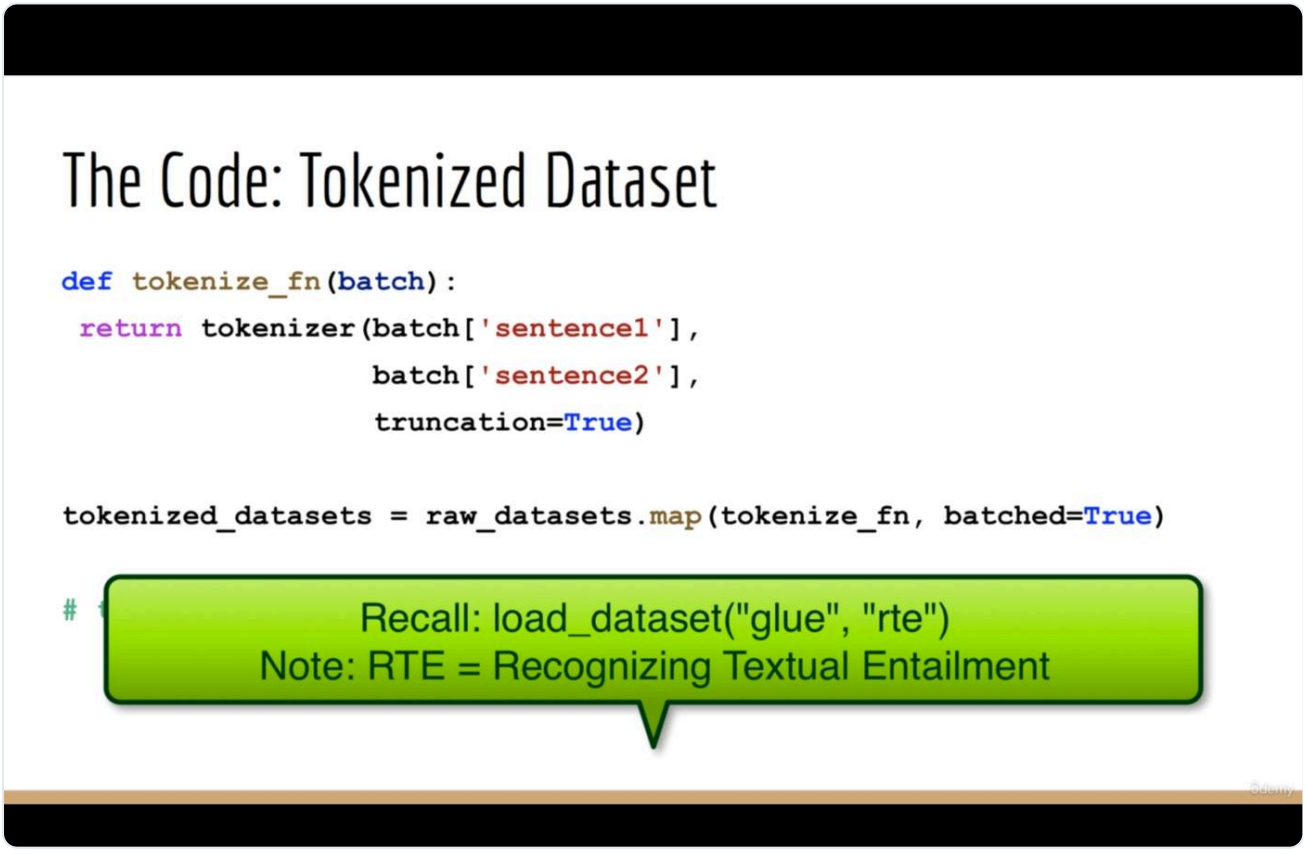

FIGURE 4.134 | The Code: Tokenized Dataset

As you recall, we would like to apply that tokenisation to every sample in our data set object. So we write a tokenized function that we can pass into the map method. This will also take care of truncation. The next step is to call the map method passing in the tokenize function, as we did before. This gives us back our tokenized data. Of course, at this point, the format of our data is exactly the same as it was in the previous lectures, which means there is no more work to do.

As an exercise. Since this is quite simple. You might want to complete the following notebook on your own. Before looking at the next lecture. We'll be using the RTE dataset, which is also part of the GLUE benchmark. So good luck and I'll see you in the next lecture.

13 · Fine-Tuning Transformers with Multiple Inputs in Python

See the notebook here.

So in this lecture, we'll be looking at how to do fine tuning for text classification when we have more than one input sentence. We'll begin by installing transformers and data sets, as we normally do. The next step is to import load_dataset as well as numpy. The next step is to load in our dataset, which is the RTE dataset. This is part of the GLUE benchmark. Like the earlier example on sentiment analysis. Note that I've pasted a description of the dataset here which explains that the data has been processed to be for binary classification.

Other options are possible, for example, having three classes where the third class is neutral, meaning that the two sentences neither entail nor contradict. The next step is to print out the data set object. We got back to see what it contains. So as you can see, we have train, validation and test sets. Each data set object contains two sentence columns along with the label and index. The next step is to check out the features attributes. So as you can see, this tells us that we have two classes where the first classes entailment and the second is not_entailment.

The next step is to simply print out some sentences just to see what we're dealing with. So feel free to check these out, if you like. The next step is to define our checkpoint. As mentioned, will be using DistilBERT for this notebook. But feel free to try it as well as an exercise. The next step is to import everything we need from the Transformers library, including the tokenized model trainer and training arguments. The next step is to load up our tokenizer from the checkpoint.

The next step is to test their tokenizer on the first pair of sentences in our dataset. The next step is to check the keys of the dictionary. We got back. So as you can see, the key for token type IDs is not present. Recall that one exercise for this notebook is to use BERT instead and to check out the format of the token type. It is. The next step is to decode the input IDs. So as you can see, our input text is effectively the two sentences concatenated into a single string.

We use the set token to separate the two sentences. The next step is to load up our pre-trained model from the checkpoint. The next step is to create the training arguments object. Note that there is one new argument here which is logging steps. The reason we need this is because this data set doesn't have a lot of samples. However, the default value for logging steps is very large, resulting in this operation not occurring. The result is that without setting this argument, we would see no log appearing under the train laws.

When we train our model instead of the loss, which is nice to know. So try commenting out this argument to see that for yourself. The next step is to import the load metric function. The next step is to get the metric for our current task. The next step is to test our metric just to see what we will get back. So unfortunately we only get accuracy, which is kind of boring. Instead we're going to include the F1 score, so we're going to import f1_score from scikit-learn.

The next step is to define our compute metrics function. We begin by splitting up the logits and the labels. The next step is to take the arguments of the logits in order to get the predictions. Once we have the predictions, we can then compute the accuracy and the F1. Finally, we return a dictionary containing our desired metrics. The next step is to define our tokenizer function as usual. The input to this function is a batch of data from our datasets.

We can pass in both sentence1 and sentence2 as separate arguments and also truncation equals true. The next step is to create our tokenized datasets. The next step is to create our trainer object. The next step is to call trainer.train. Okay. So notice that five epochs is probably too much since the validation loss seems to be creeping up. So as a final exercise for this lecture, don't forget to try BERT with this notebook as well. Compare the performance of the two models as well as the training time.

Consider if this were a real world project, which model you would prefer to use? As another exercise. Please also compute the metrics on our test set as well.

14 · Fine-Tuning Section Summary

So in this lecture, we will be summarizing everything we learned in this section.

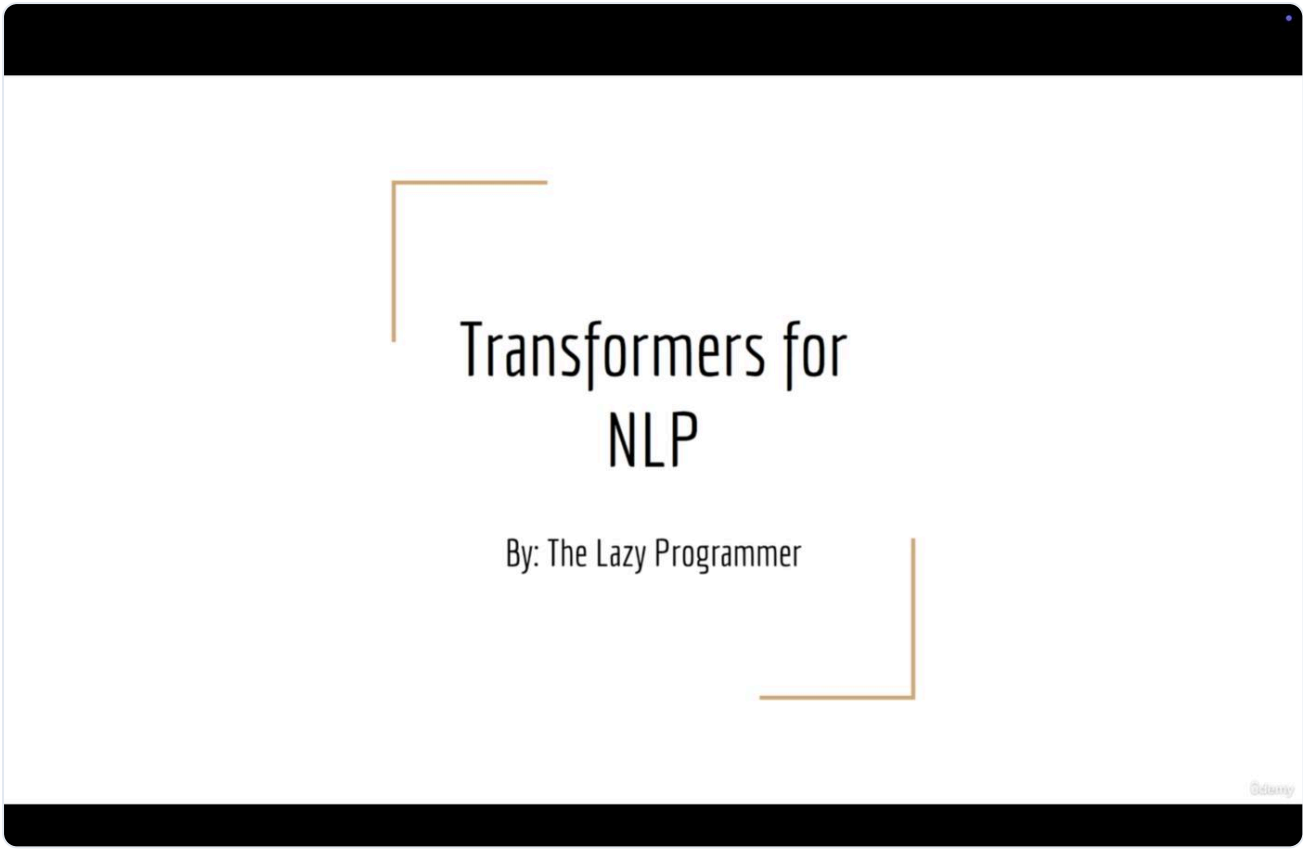

FIGURE 4.135 | Transformers for

This section was all about fine tuning a pre-trained transformer so that you can get better performance on your own data sets or even create models for different tasks.

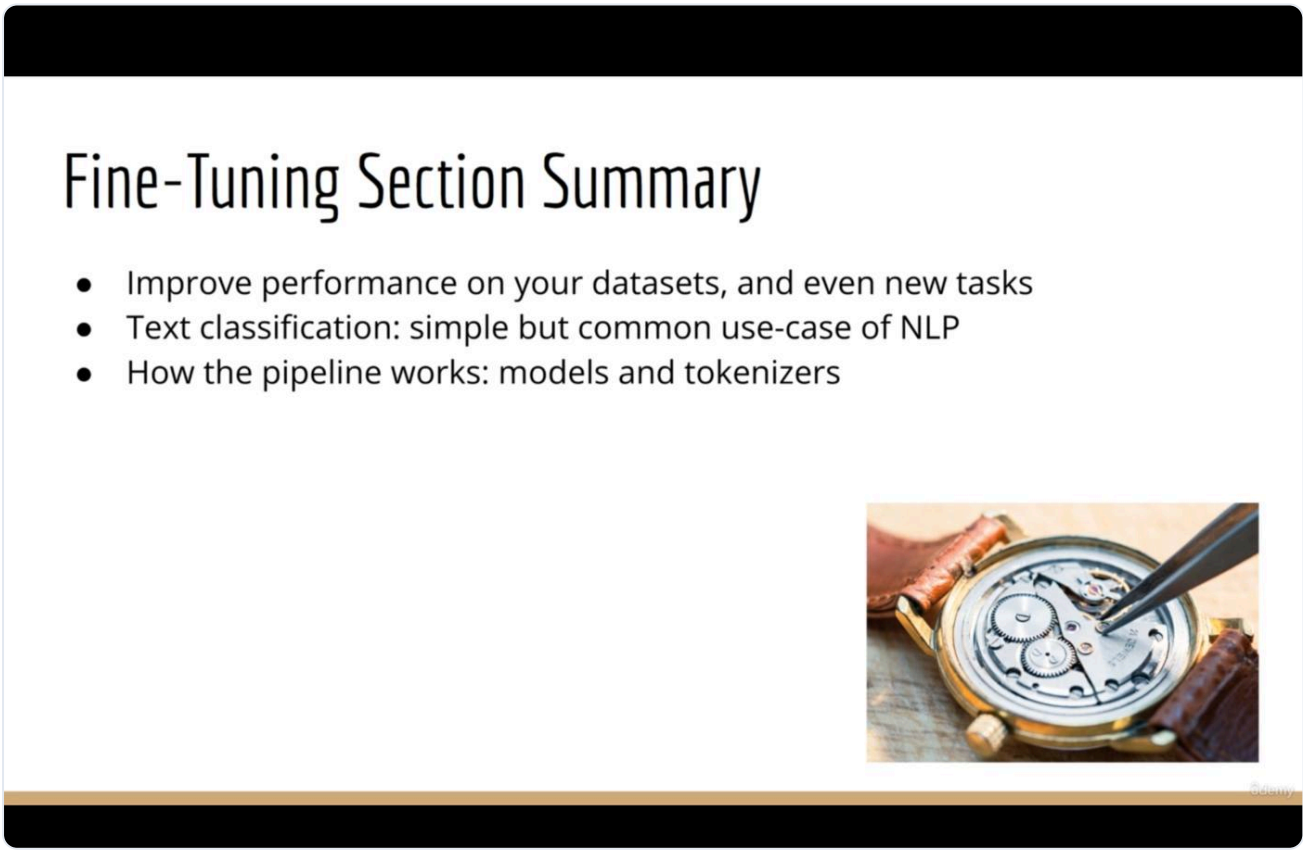

FIGURE 4.136

This section focused on the task of text classification, which is a simple but very common use case of NLP. It's also a good introduction to what goes on behind the scenes when you use a pipeline. In particular, we looked at the model and the tokenizer.

Although this does require more coding, this is a necessary step if we want to train transformers and not just use them. We started this section with an optional review of two important concepts text, preprocessing and fine tuning. Additionally, we also reviewed Transfer Learning, which is sometimes associated with fine tuning but doesn't have to be. Text preprocessing involves many steps, although we encapsulate them in a single class called a tokenizer.

We noted that the naming is a bit weird because tokenization technically involves only the splitting step while the tokenizer object does much more. This includes converting the tokens into integers, padding and truncation. We then turned our attention to transfer learning and fine tuning. Transfer learning is an important concept, first applied in the field of computer vision, but now also popular in NLP. This is when we train a model on one task and then we use some of those weights for a related task.

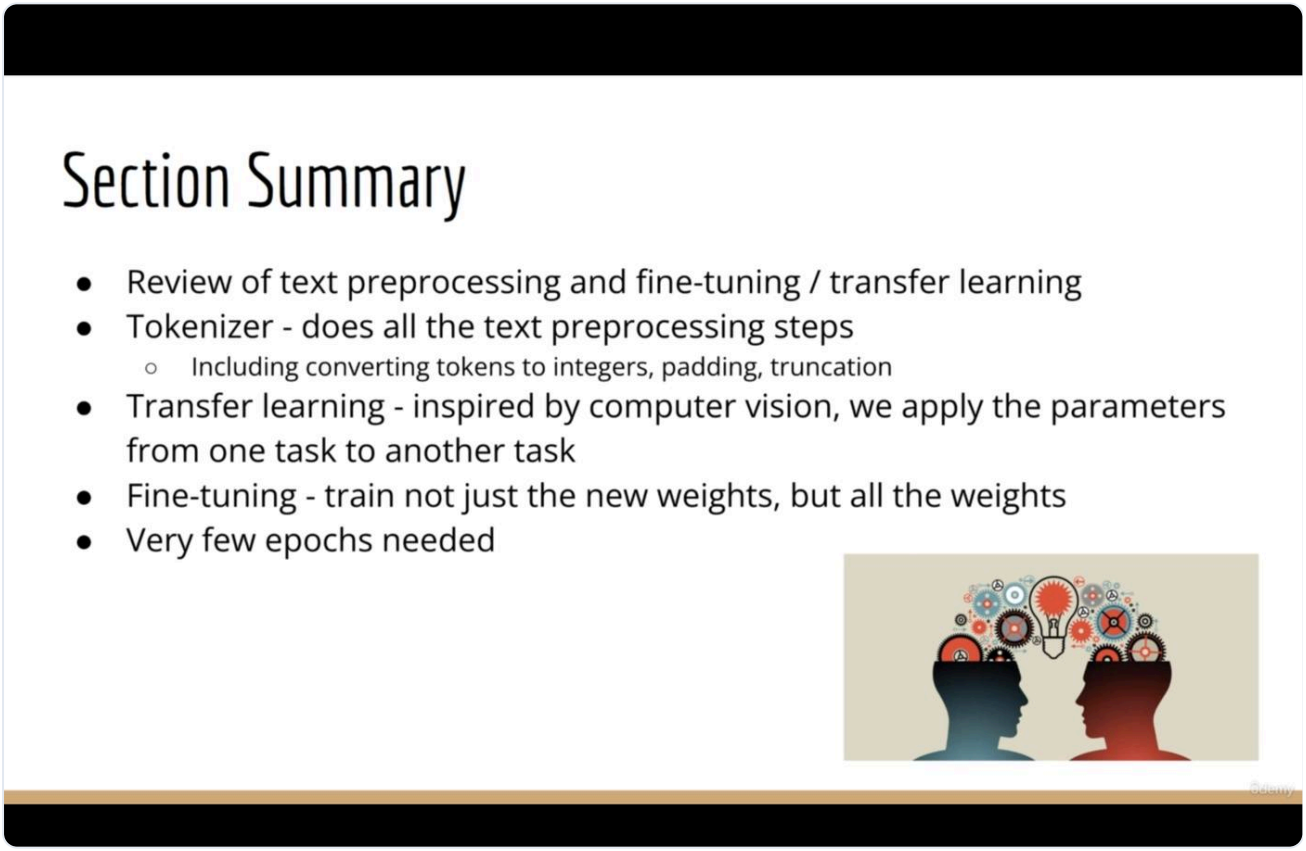

FIGURE 4.137 | Section Summary

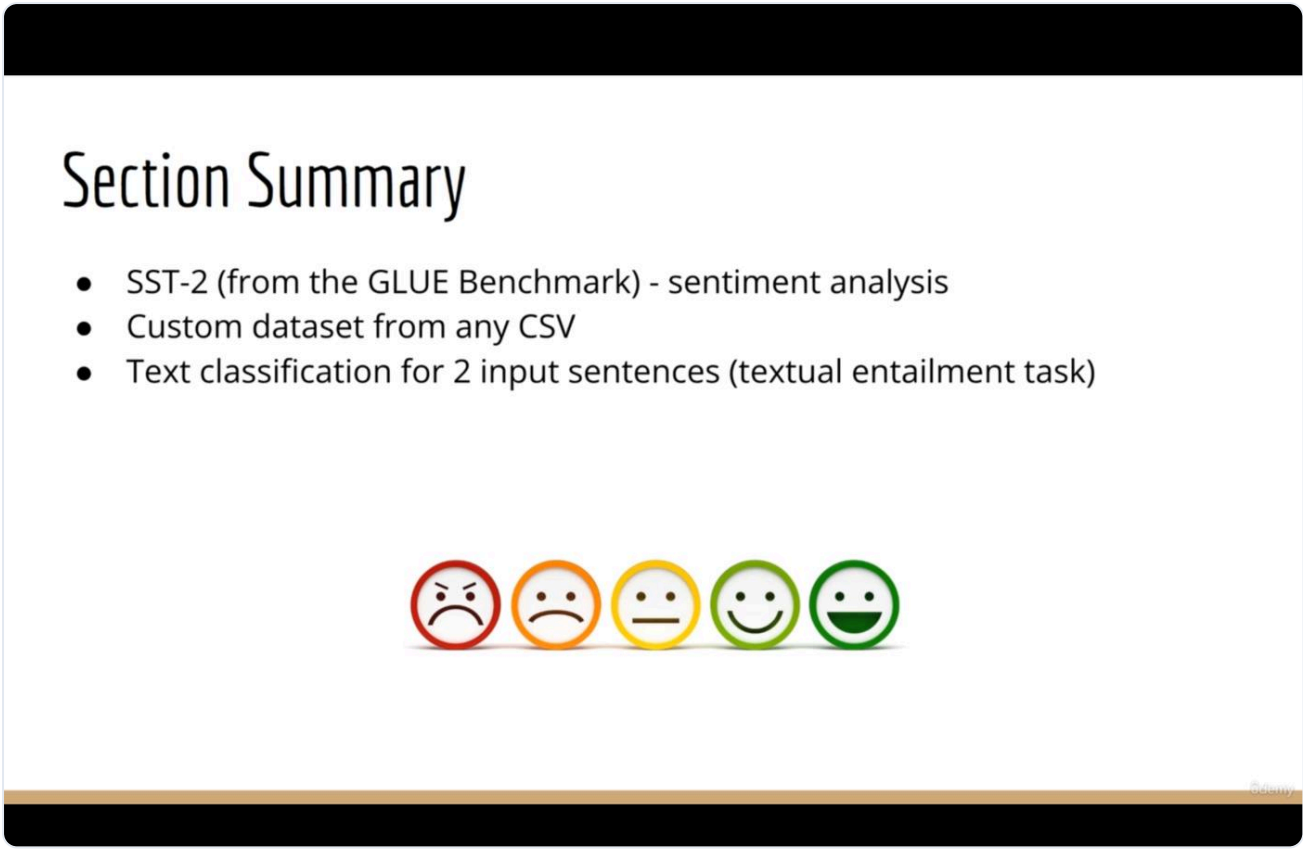

FIGURE 4.138 | Section Summary

For NLP, we also make use of the concept of fine tuning, which involves not just training the new weights but all of the weights simultaneously. We saw that at least for the tasks in this section. It doesn't take many epochs for the model to start overfitting. In this section, we looked at the following tasks. First we looked at the SST-2 task, which is part of the GLUE benchmark. This is a sentiment analysis task which is simple, easy to apply in the real world and easy for everyone to understand.

This also introduced us to the GLUE benchmark, which is itself an important concept in NLP. Since this is what transformers are now tested against. The next task we looked at was also very practical how to train a text classifier on your own custom dataset. We learned that as long as you can convert your data into a simple CSV format, which shouldn't be difficult for anyone taking this course, you can continue to use the Hugging Face Datasets package.

The third task we looked at was also of practical importance and introduced a new concept. In particular, it showed us how to train a model where we have not just one input sentence but two. For this example, we looked at the textual entailment task, which is another task in the GLUE benchmark.

SECTION 5

Named Entity Recognition (NER) and POS Tagging (Intermediate)

1 · Token Classification Section Introduction

So in this lecture we will be introducing the next section of this course, which will be a slightly more advanced version of what you just learned. In the previous section, we looked at how to classify whole documents. This meant that each document belonged to a class, and we wanted to predict what that class was. This section is different, but it extends what we previously did. In particular, this section covers how to create models that make predictions on a per token basis.

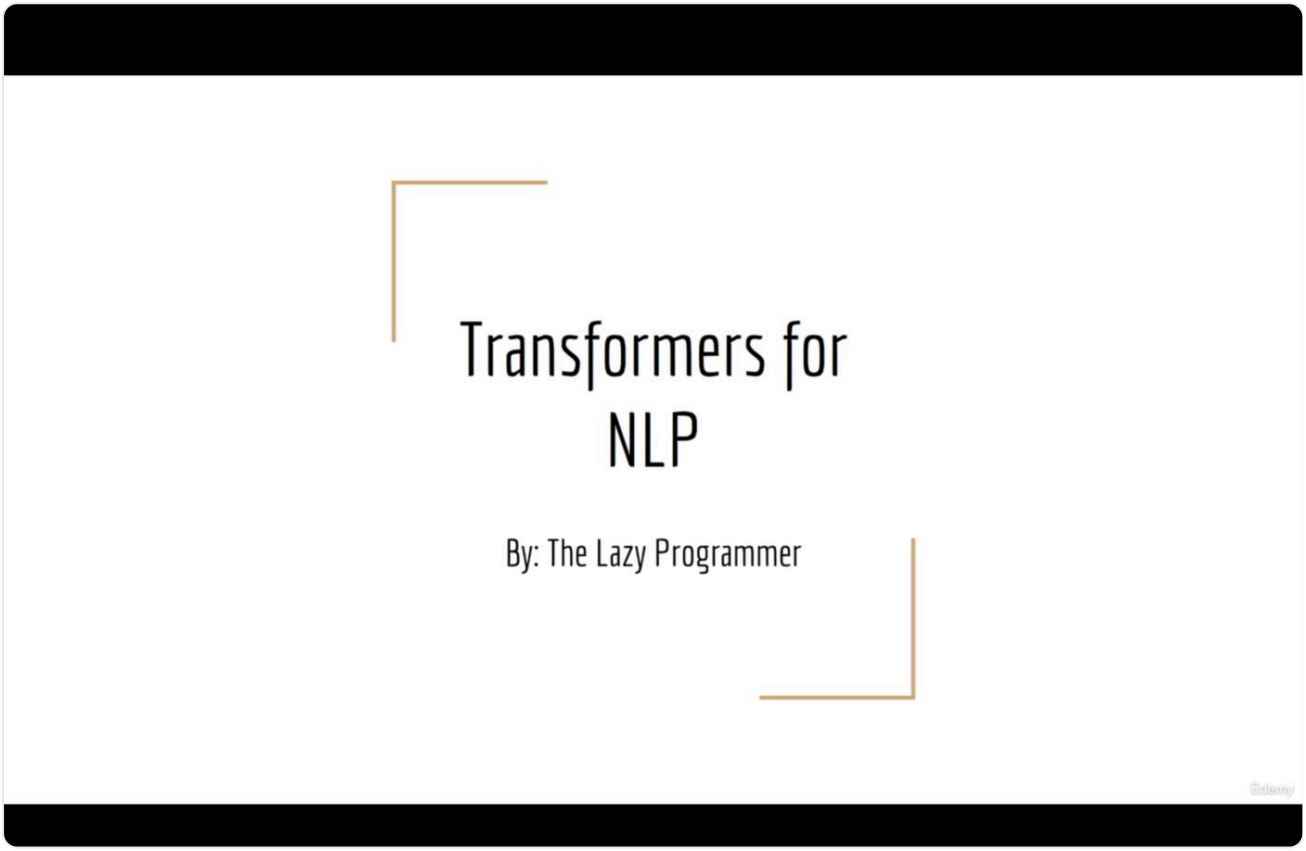

FIGURE 5.1 | Transformers for

This is a type of many-to-many task, since the input is a sequence of many tokens and the output is a sequence of many labels. So instead of predicting just one label, we will be predicting a label for every token in the input. Luckily, we've already seen one example of how this works and how it can be applied. Specifically, we looked at the named entity recognition task where a goal was to find all the named entities in a document. They could be either people, places, companies or miscellaneous entities.

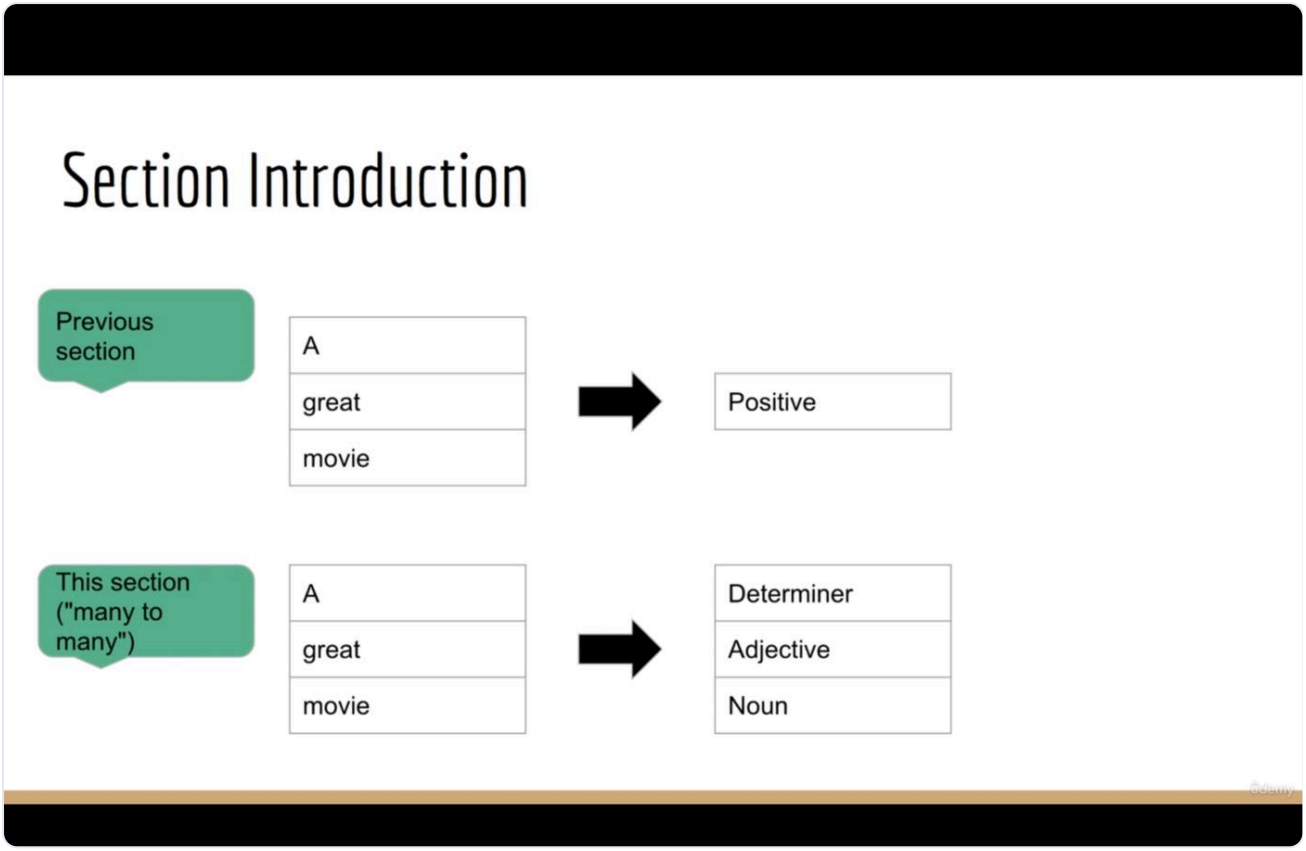

FIGURE 5.2 | Section Introduction

Otherwise, they were assigned to a class of no entity. Another example of this kind of task is parts of speech tagging. For this task, we would like to return whether each word in our document is a noun, verb, adjective and so forth. Thanks to my rule, all data is the same.

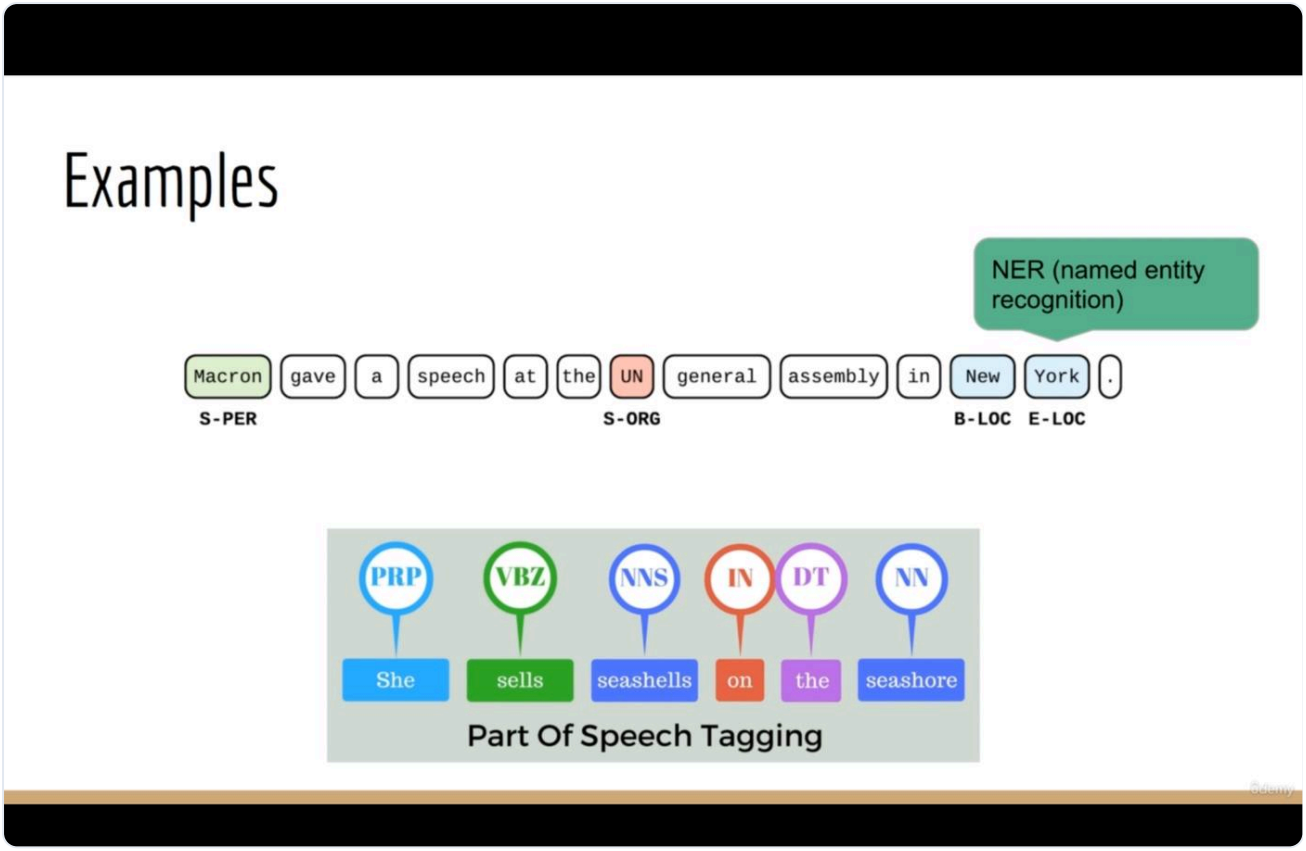

FIGURE 5.3

These two tasks are effectively equivalent. Mechanically speaking. Again, we'll continue using the Hugging Face library, which makes fine tuning convenient now because we're using a specific library, we should get familiar with their terminology.

Specifically, they call this type of task a token classification task. This makes sense because we're assigning each token to its own class, and so we need to make a prediction for every token in the input.

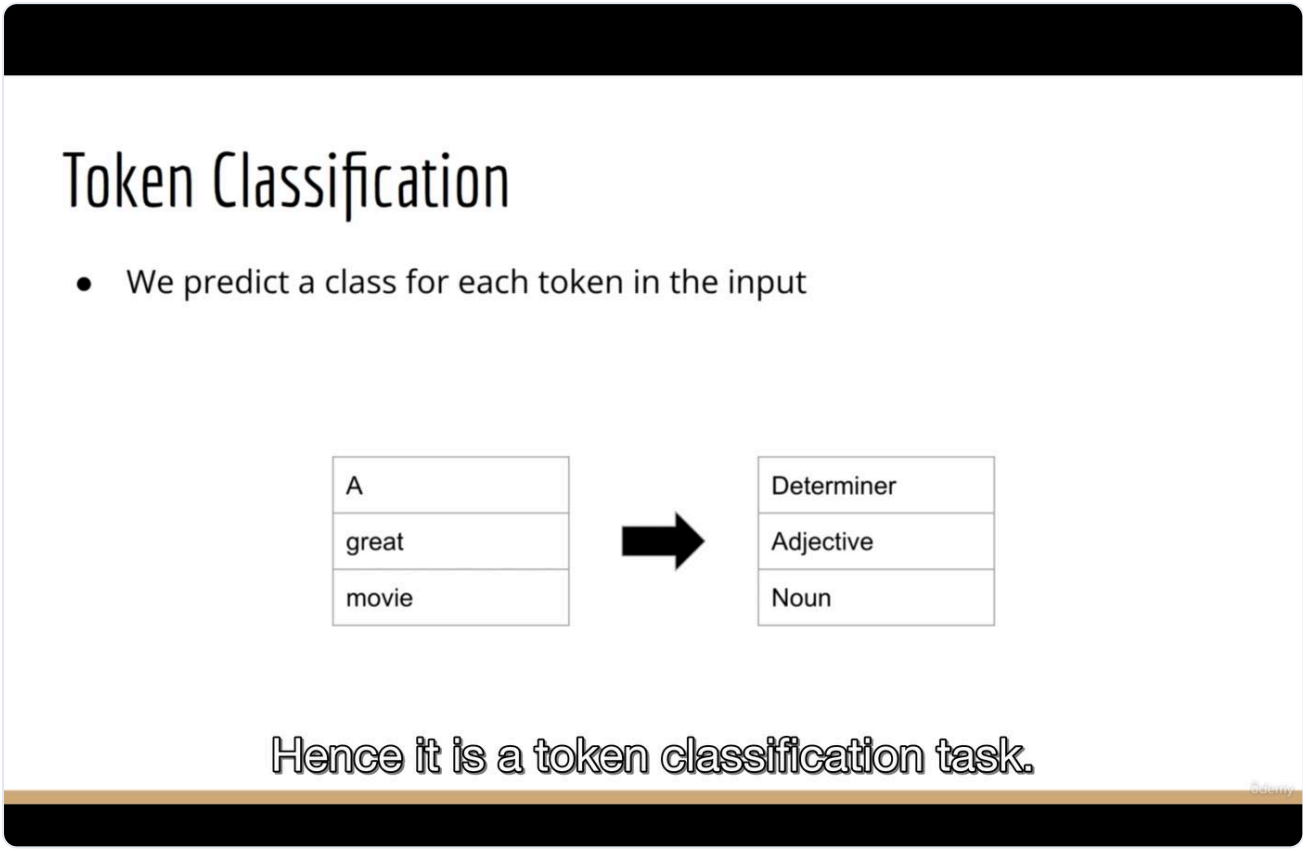

FIGURE 5.4 | Token Classification

Hence it is a token classification task. Okay. So now that you know what this section is all about, let's look at an outline for the remaining lectures. We'll begin by looking at how to load in and process the data, because this is now a many-to-many task.

Our data will be structured a bit differently. We will no longer have a single class as the target, but a whole sequence of classes. And there's another challenging aspect to this. We know that transformers tokenize documents into sub words, but for tasks like NER and parts of speech tagging. The targets are provided on a per word basis. That is to say, after tokenizing the document into sub words, the input tokens and per word targets will no longer be aligned.

The number of targets will always be less than or equal to the number of tokens, whereas we would like to have them be exactly equal. We'll need to fix this somehow. In addition, the data set will be using has already split the document into words. Previously we saw how to use the tokenizer to split a single string into tokens, but we'll need to figure out how to tokenize a document which has already been partially tokenize. So to summarize this, well, first need to figure out what the data is even going to look like.

Since now we have a sequence of targets instead of just a single target, we need to figure out how to properly tokenize the data since the data is already split into words. And finally, we'll need to figure out what to do with the targets.

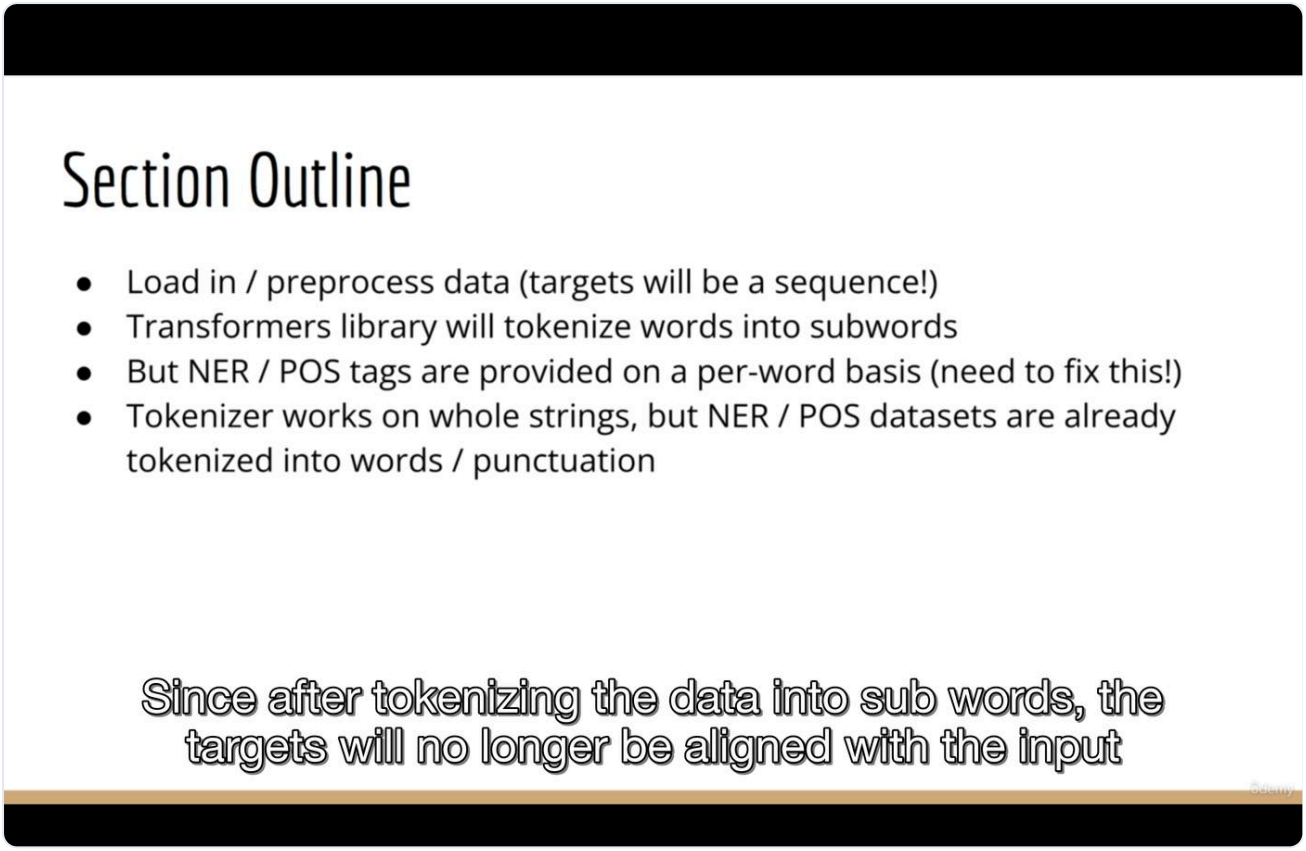

FIGURE 5.5 | Section Outline

Since after tokenizing the data into sub words, the targets will no longer be aligned with the input tokens. Now there is one more data processing step that we need to learn about, which is the data collator. Although we could have looked at this in the previous section, we didn't because it didn't make a difference in the results.

In fact, it was already being applied internally. In this section, however, it will be required, as you recall, when we pad our data.

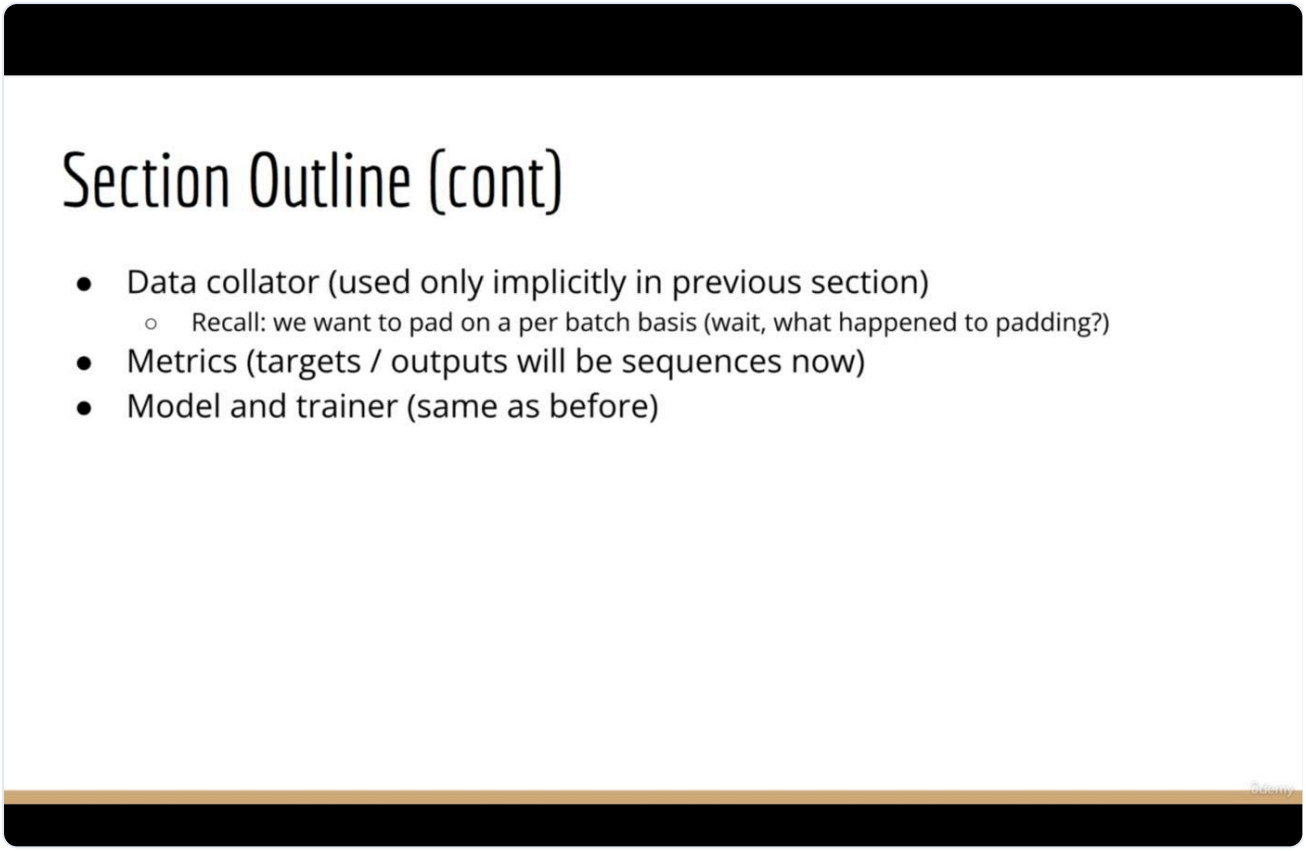

FIGURE 5.6

We want to pad it on a per batch basis. We don't want to pad everything to be as long as the longest document, which would be a waste of space and time. The data collator is what allows us to do that. Once we've discussed that, we'll talk about metrics since will now have a whole sequence of targets and predictions. The way we compute metrics will be different.

Finally, we'll look at the model in the trainer, which, unlike the previous steps, will be relatively simple and will pretty much follow what you learned in the last section. So most of the complicated stuff in the section has to do with the fact that the targets are now sequences and are on a per word basis instead of a per token basis. Now it's worth thinking about where we are in this course from a bird's eye point of view. One question worth considering is Why do we even need this section?

Why couldn't we just apply what we learned in the previous section to a new kind of task? For instance, when we learn about how to build RNNs with TensorFlow or PyTorch, we can just as easily classify a whole document with a single label or classify each token separately. The amount of new work you need to do is not that significant. But for what we are doing now, we have to keep in mind that we don't really know how to build a transformer in TensorFlow or PyTorch just yet.

We are still relying on a library to do the work for us. This library defines specific APIs and classes for each kind of task so we can no longer use auto model for sequence classification because we are no longer trying to classify a whole sequence. But this is what happens when you use a library to do the work. Every time you want to change something or do something new, you can't really figure it out on a level of first principles. If I'm writing a neural network in TensorFlow, I can do anything I want because I know how to build things on a fundamental level.

But if you're using a library like Hugging Face, you cannot do anything you want because you do not have access to the building blocks. It's like going to a restaurant. You can look at the menu and order different items, but you can't give the chef your own recipe.

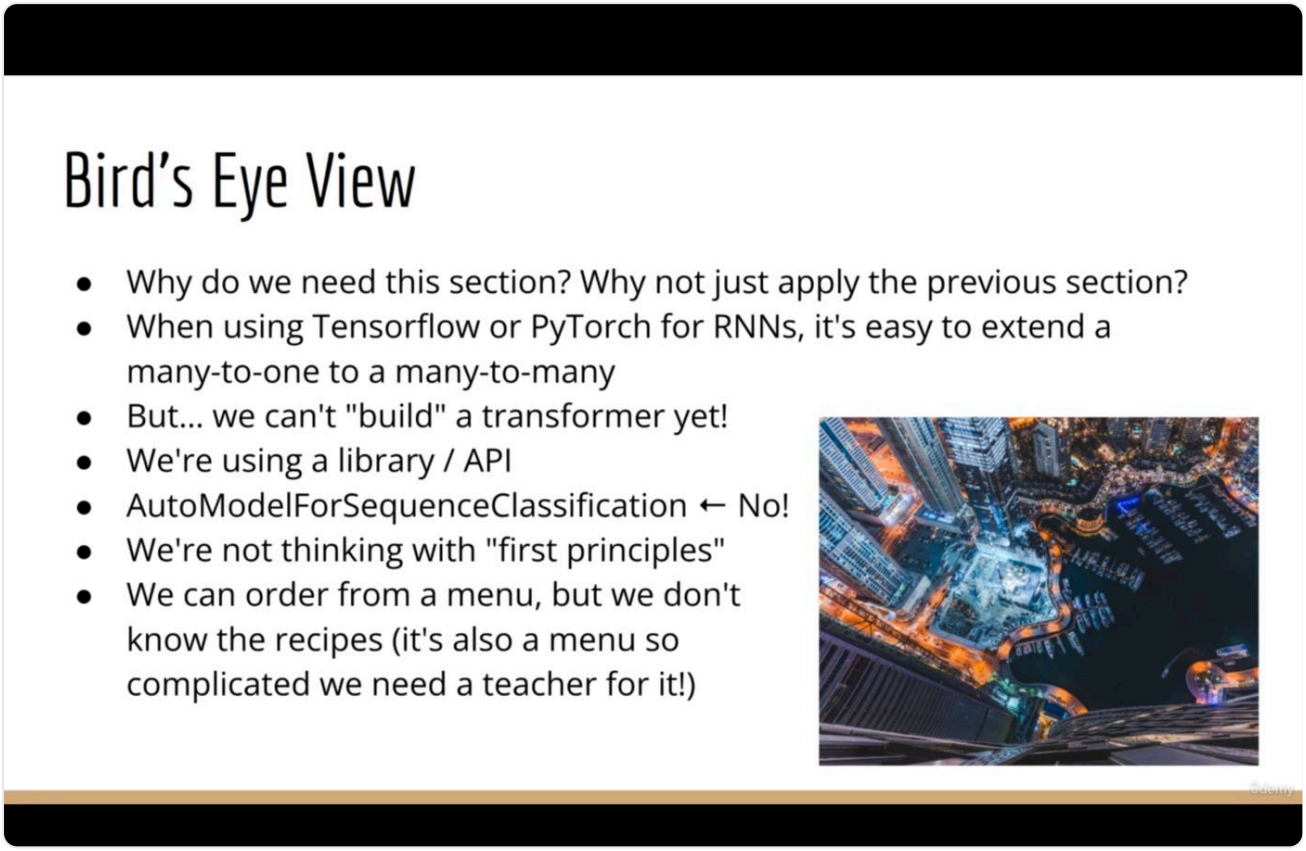

FIGURE 5.7

And what's worse is that this is a very complicated menu, so complicated that it's pretty much required for someone else to teach you how to use it. Also suboptimal is that this isn't fundamental, deep learning knowledge per se.

It's more of an exercise in reading documentation and gluing stuff together. At the same time, this is actually very useful because it makes Transformers more easily accessible to everyone. As always, there are pros and cons to both sides.

2 · Data & Tokenizer (Code Preparation)

So in this lecture, our main goal will be to discuss what our data looks like and how we will process it. We'll begin by discussing what our data will look like on a conceptual level.

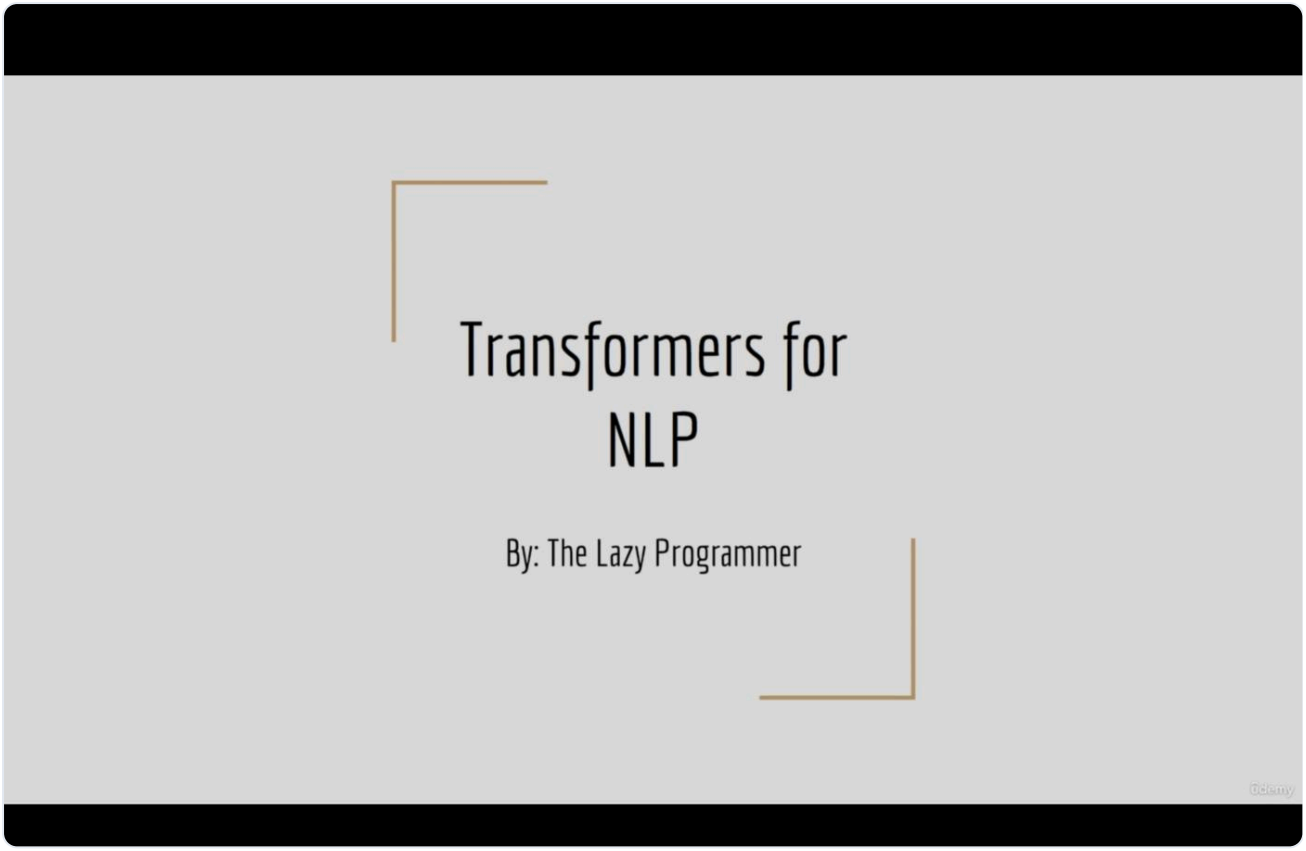

FIGURE 5.8 | Transformers for

As you know, we'll be looking at the named entity recognition task. So our data will be in the form of input sentences and target tags. Here's an example.

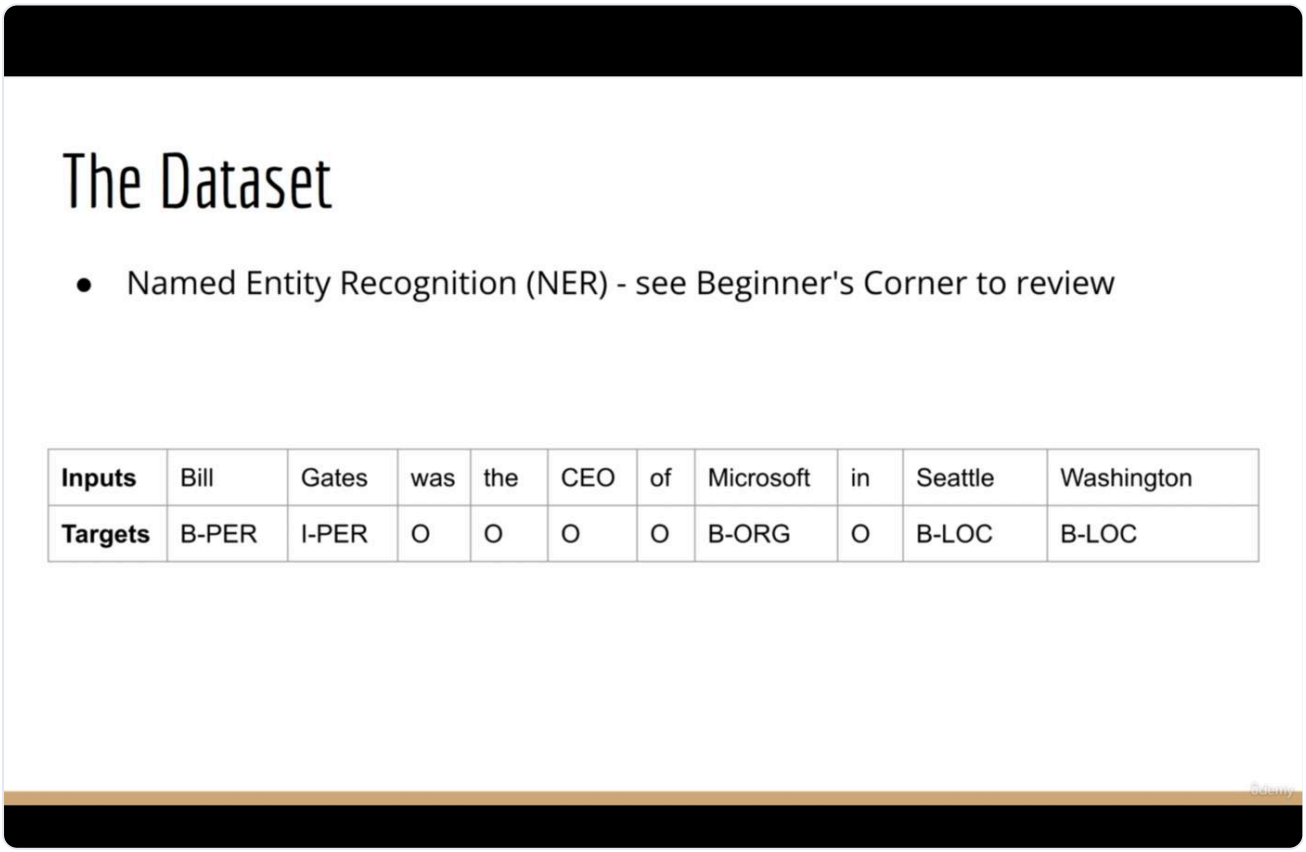

FIGURE 5.9 | The Dataset

Our input sentence is Bill Gates was the CEO of Microsoft in Seattle, Washington. Our target tags are beeper hyper 000.

Be org oh block and b look. Note that in this instance, we are treating Seattle in Washington as two distinct place entities. Although whether or not you really want to do that would be up to you and you would label your data set how you like. So our dataset will be a whole bunch of these sentence and tag sequence pairs. Now, if you need a review of how NER labels work, like what do the B, the I and the O mean, please check out the beginner's corner section of this course.

So now that you understand how the data will look on a conceptual level, let's look at how we'll be working with our data set in the coming notebook.

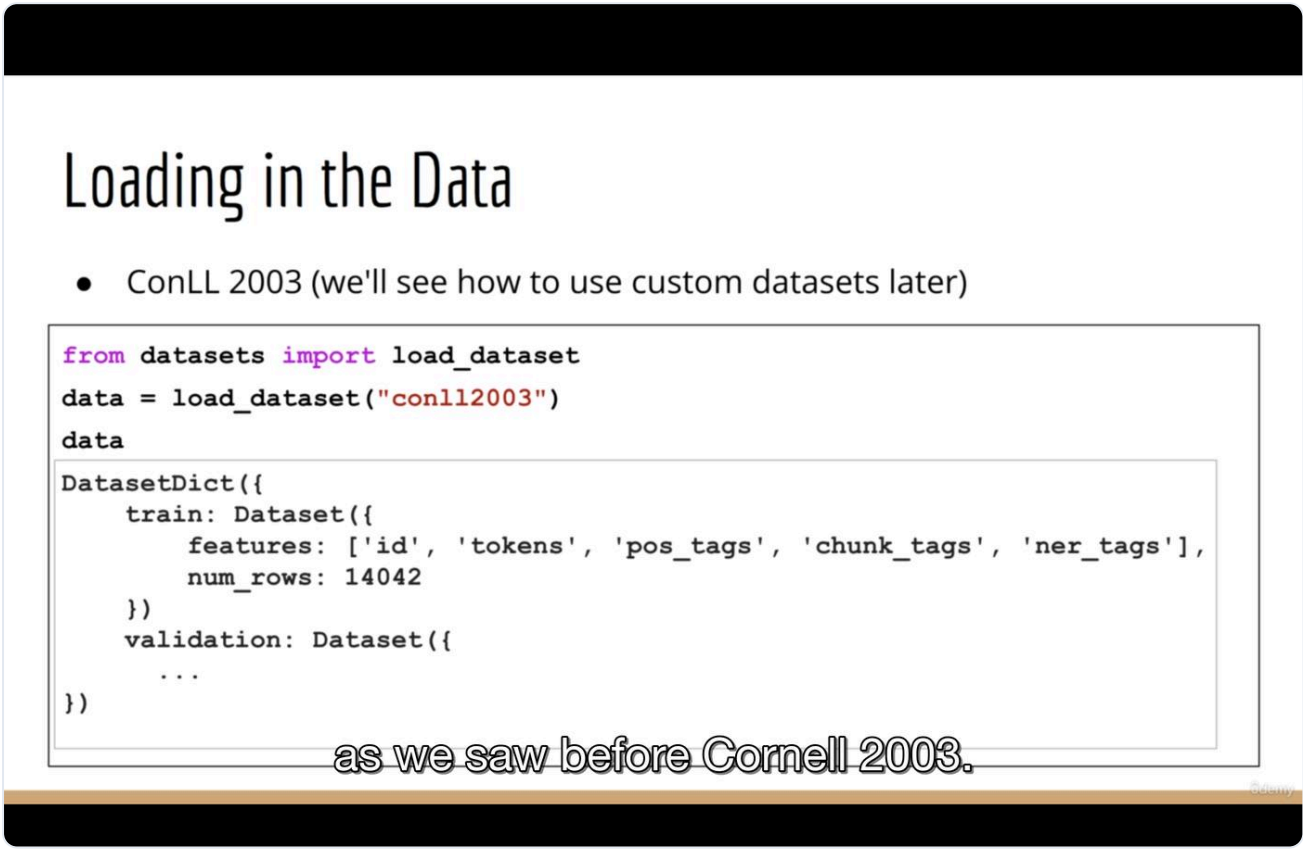

FIGURE 5.10 | Loading in the Data

For this example, we'll start again by using a built in data set, which happens to be the same one as we saw before CoNLL-2003. Later in this section, we'll see how you can use your own custom data set as well. As usual, we can load up this data set by importing the data sets package and calling the load_dataset function and passing in the string conll2003.

This will give us back a data set dict object which will contain at train validation and test data sets as before. You can think of this like a tabular dataset with a number of rows are the number of samples and the number of columns are various attributes. For this data set, there are actually three possible tasks you can do corresponding to the different columns. The column for the input sentence is called tokens, which is a bit confusing since these are actually full words and not the separate tokens we'll be seeing later.

But for some reason, the person who uploaded this dataset to the Hugging Face library decided to call them tokens.

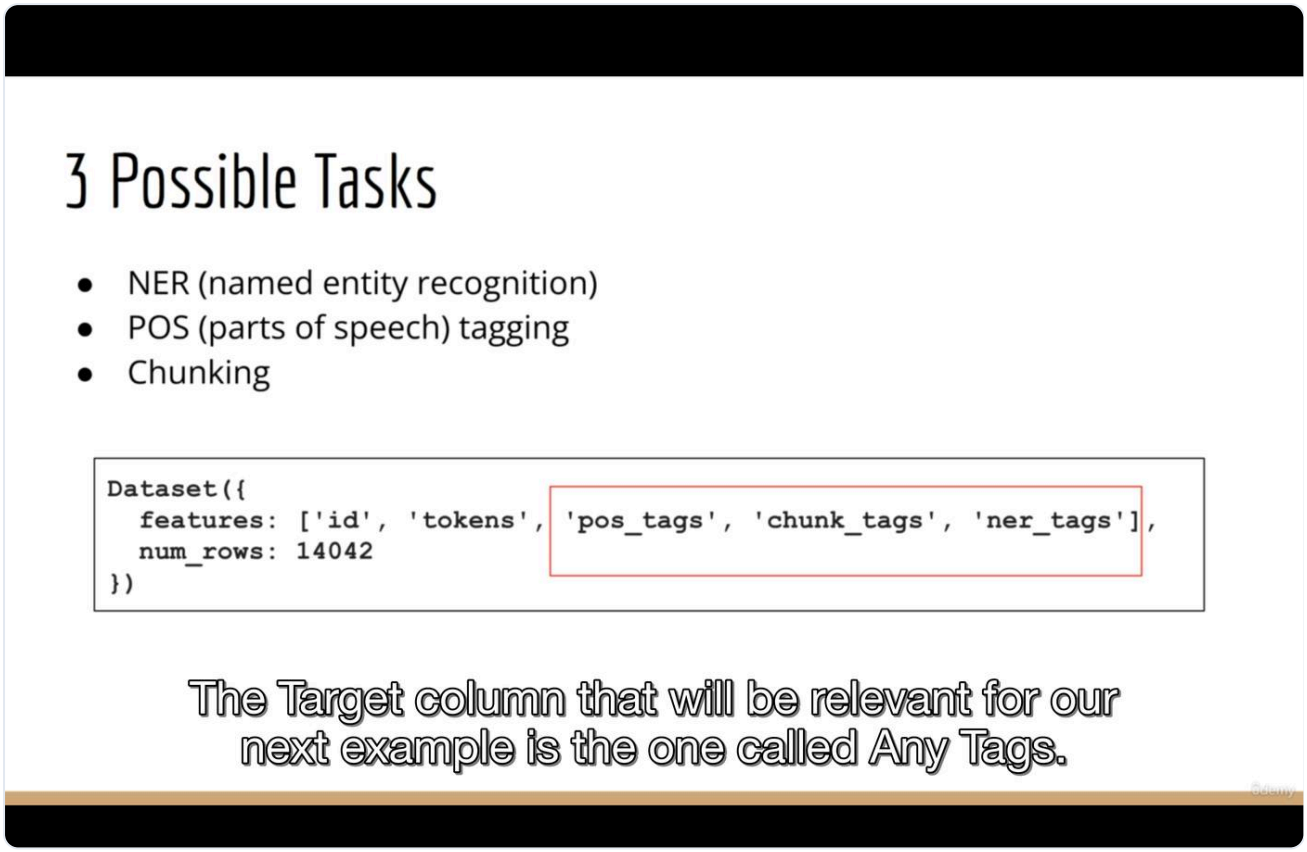

FIGURE 5.11 | 5 Possible Tasks

The Target column that will be relevant for our next example is the one called ner_tags. Since this contains the tags for the NER task, you can also use this dataset for parts of speech tagging for which you would use the column called pos_tags. There's also a column called chunk_tags where you try to predict which words should be chunked together as part of the same entity.

For example, Bill Gates would be a single chunk because the two words together corresponded to a person. But note that when we do NER, we are also effectively doing chunking because the tags will tell us whether we are beginning a chunk, continuing a previous chunk, or we are outside a chunk.

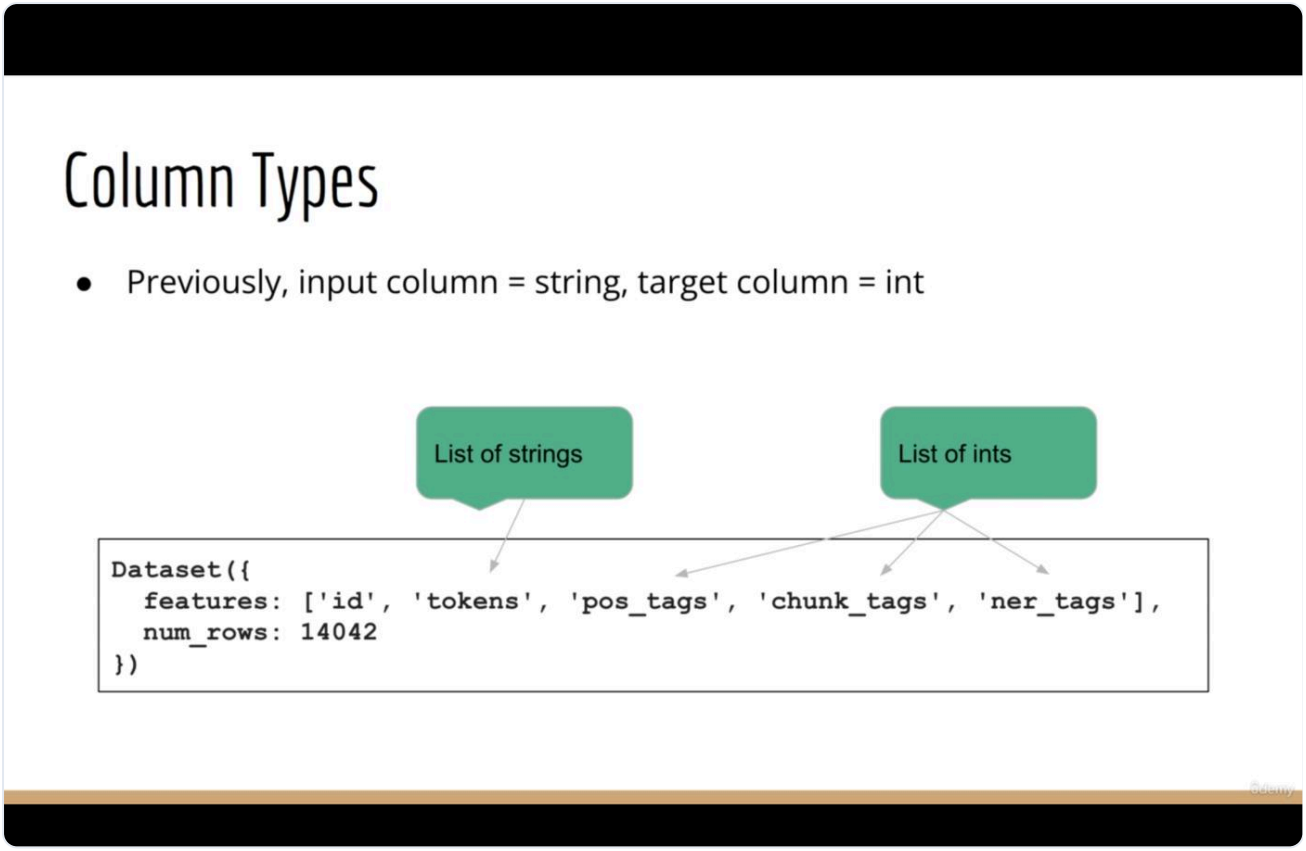

FIGURE 5.12 | Column Types

Now, one thing to notice at this point is how the data set object is quite flexible. Although I said you can think of it like a table. There's a wide variety of different types of values that can be stored.

In the previous section, when we wanted to classify documents, the input column was a string, while the target column was an integer. But in this section it's different.

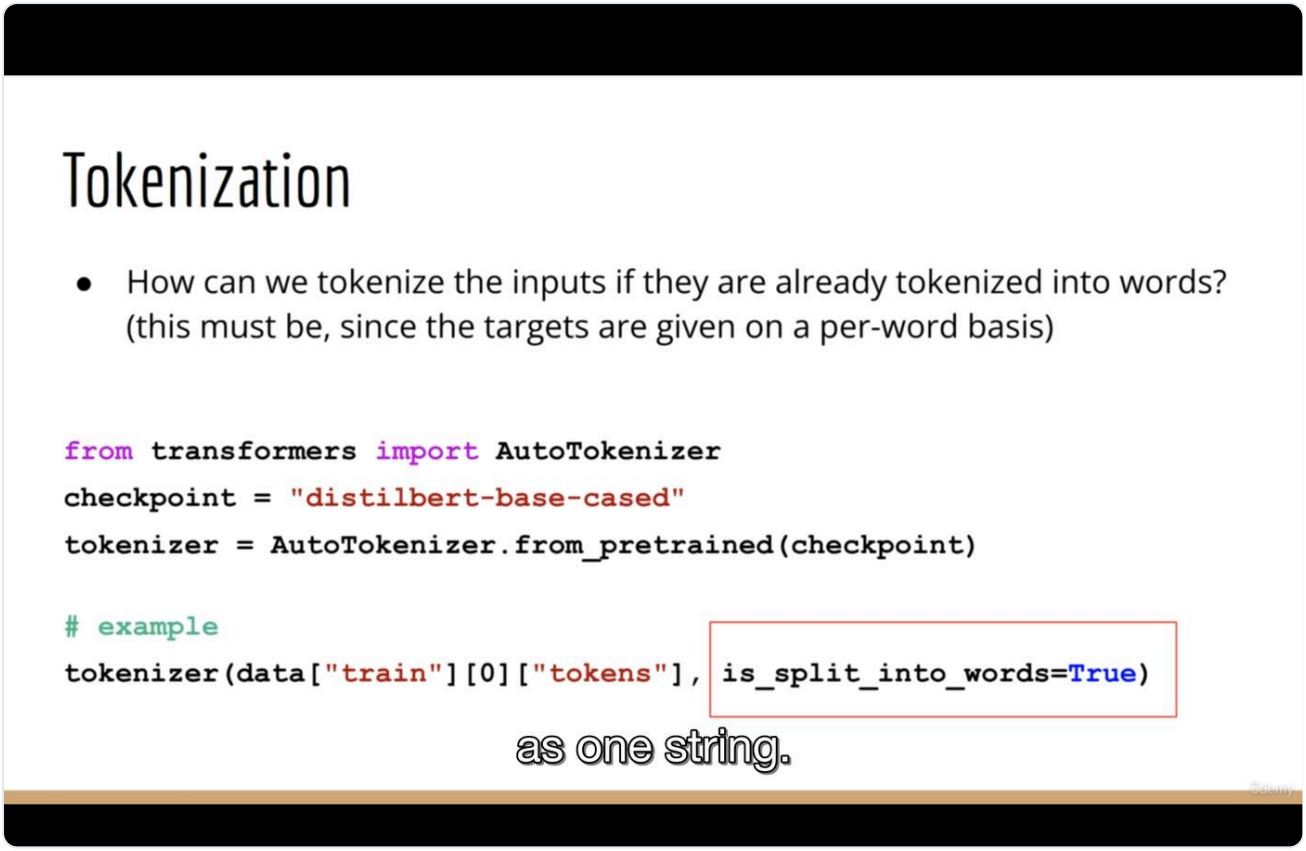

FIGURE 5.13

As you'll see, the inputs are already tokenized into words, and as such they will not be strings, but each element will be a list of strings. Similarly, for the targets, we won't just have an integer, but a list of integers. So as you've seen, once we've loaded in our data, we would like to tokenize it.

The problem is that we've only seen our tokenization do its tokenization steps on documents represented as one string. With this new dataset, our document is already split into words, some of which could possibly be tokenized into sub words. Luckily, the tokenized classes are built to handle this situation. We simply need to pass in. the argument is_split_into_words equals True. When we do this, we can pass in a list of words as input and it will be tokenized as if it were a single string.

For the coming example, we'll continue using the DistilBERT checkpoints, which you can also replace with the BERT checkpoints, if you like. But at this point, you might be wondering why don't we use a different model such as GPT-2?

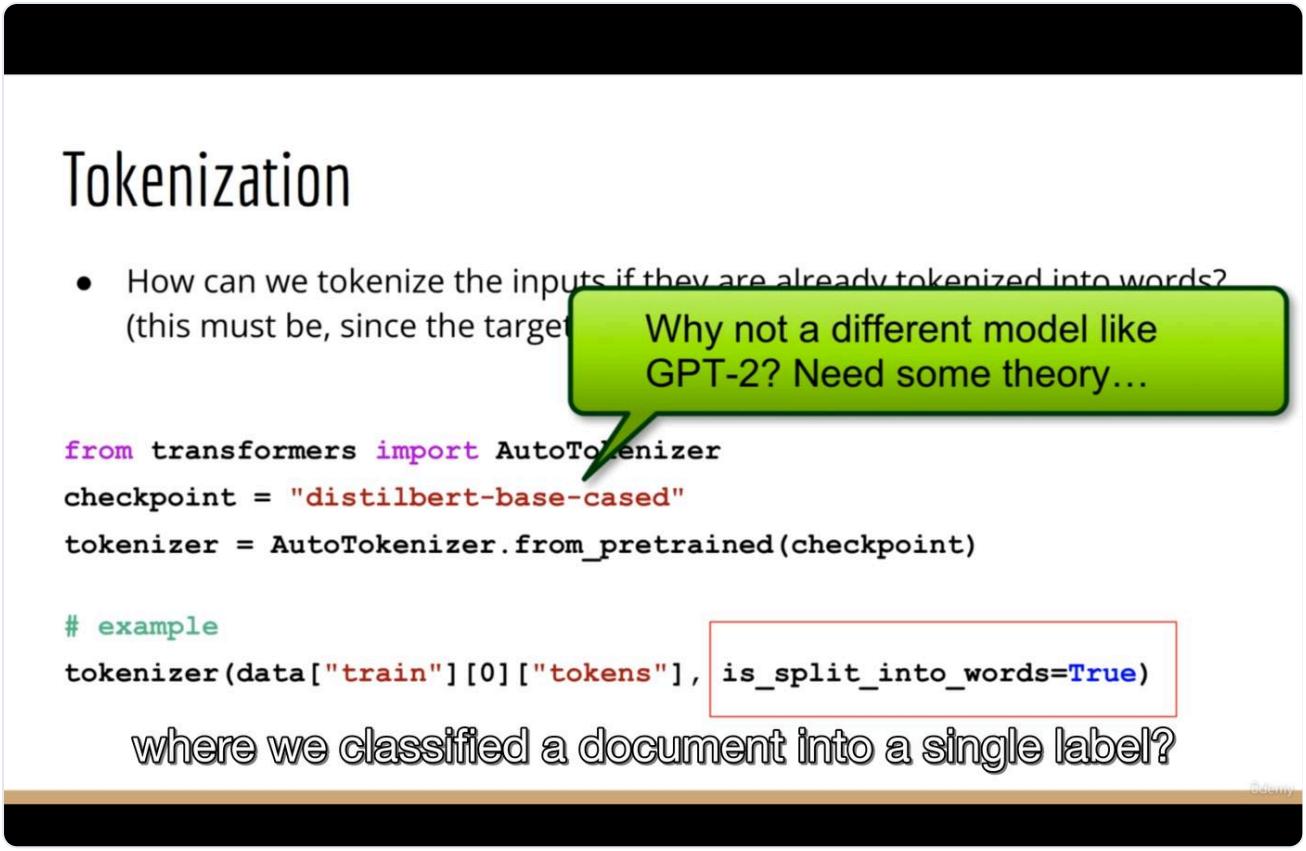

FIGURE 5.14 | GPT-2? Need Some Theory

And furthermore, how can the same BERT model be appropriate for both this task and the previous task where we classified a document into a single label? Hopefully you've been thinking about these questions. They will be answered when we finally discuss the theory behind Transformers.

3 · Data & Tokenizer (Code)

See the notebook here.

So in this lecture we will be looking at the code to load up and play around with our data set as well as set up the tokenizer object. Note that because this is such a long notebook, we'll be going through it in parts. So first you'll learn a few new concepts. Then we'll go through parts of the notebook relevant to that concept. Then we'll go on to the next concept and so forth. So we'll begin by installing transformers and data sets, as we normally do.

So the next step is to import the load_dataset function. The next step is to call load_dataset passing in the string conll2003. Let's now check our data object. So as you can see, it's just like we saw before where we have the data set dict which contains three data set objects, one for train, one for validation and one for test. Note the meaning of the columns. First, we have tokens which are actually words and not the tokens we will eventually end up with after we pass this through our tokenizer.

Next, we have the three types of tags POS tags, chunk tags and NER tags. So you can actually use the same dataset for three different tasks, but we are just going to be doing one. The next step is to check out the first sample in the train set. So as you can see, our tags come in the form of lists of integers while our tokens come in the form of a list of words. Note that punctuation is included in the words have not been lowercased. Next, let's have a look at the features attribute of the train set.

Okay. So as you can see, this tells us what kind of data is stored in each column. Chunk tags and NER tags and POS tags are all sequences. These sequences contain a single feature called Class Label. We can see that chunk tags have 23 classes. While NER tags have nine classes while POS tags have 47 classes. Notice that the tokens are also a sequence, but the feature is a value and not a class label. The d-type of this value is string. As you may have expected.

Okay. So recall that what we just got back was a dictionary. So let's try to access the tags. As you can see, this gives us the sequence object we saw above. Now, as you recall, Hugging Face objects tend to work like named tuples. So we can probably access the features attribute, which would then give us a class label object. The class label objects, it looks like it has an attribute called names. So we call that feature dot names. As you can see, this gives us back a list containing the names of each class label so we can use this when we want to map label IDs back to label names.

The next thing to do is save the output we just looked at into a variable called label names, which we will be using later in this notebook. Okay. So that's pretty much the extent of what we want to do with our data set by itself. The next step is to actually tokenize this data set. We'll import the AutoTokenizer once again and we'll load up a tokenizer from the DistilBERT checkpoint. Note that this is a cased DistilBERT since casing will probably help our model do NER better.

For instance, the string bill could be a name as in Bill Gates. Or it could be a word as in dollar bill. As usual. Feel free to switch this to BERT instead of DistilBERT. I've chosen DistilBERT because it trains faster. Now at this point, you may also be wondering why do we keep using BERT and DistilBERT, which is presumably similar to BERT? Why not a model like GPT-2? The answer has to do with the architecture of these models and how they relate to the task we are trying to do.

If you've studied RNNs, you should be familiar with this concept. For instance, you can use a similar kind of RNN for classifying text or NER as we've done with Transformers. The only difference is that we would have to save all of the hidden states instead of just a single hidden state. However, for a task like machine translation, we would need a different RNN architecture. Specifically seq2seq. The same rules apply for transformers.

In any case, we'll learn more about this as we progress throughout the course. Okay. So as mentioned, our data is already split into a list of words. But we saw in the previous section that this tokenizer will tokenize whole strings. As you can see, it's very easy to get the same tokenizer to tokenize, a document which has already been split into a list of words we simply need to pass in the argument is_split_into_words equals true. So as you can see, our data has been successfully tokenized.

Now you should note that although the value we just got back prints out as if it were a dictionary, it's not actually a dictionary. In fact, it is an object. So let's check out the type of this object. As you can see, it's an object of type BatchEncoding. Note that this object has some useful instance methods. For instance, we can call tokens and this will return the tokens represented in the string format instead of integers. So as you can see, we have the usual special BERT tokens, CLS and SEP.

We can also see that the word lamb has been tokenized into two sub words. Now you'll recall that in the previous section, the next step would be to create a tokenize function that wraps the tokenizer so that we can apply truncation. We then call the map function to map the tokenizer to our data sets. However, in this section, we can't move on to that step just yet. In this section, the tokenized function will be a bit more complicated because not only are the inputs the sequence, the targets are also a sequence.

Importantly, because words may be split up into multiple sub words, the input sequence may end up being longer than the target sequence, which won't work. At this point. We've seen all we need to see for this portion of the notebook. So in the following lecture, we'll continue on with the next topic.

4 · Target Alignment (Code Preparation)

So in this lecture, we are going to tackle one of the tricky issues with token classification, which is that after we tokenize our data into sub words, the tokens will no longer align with the targets. In fact, we've already seen how this can happen.

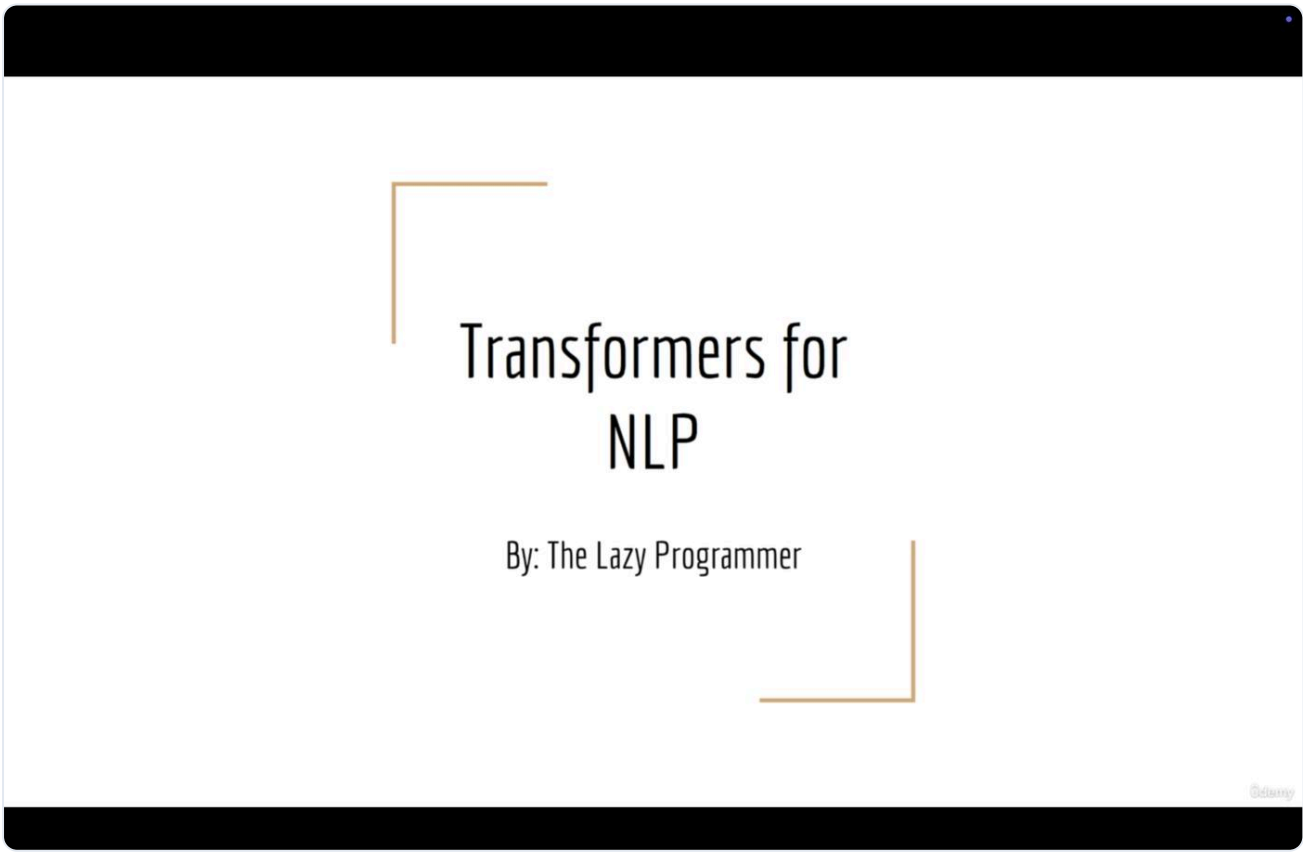

FIGURE 5.15 | Transformers for

In the previous lecture, the word lamb got split into two sub words, but in our dataset we would only be provided with one target for the whole word. This lecture is all about how we are going to fix this issue.

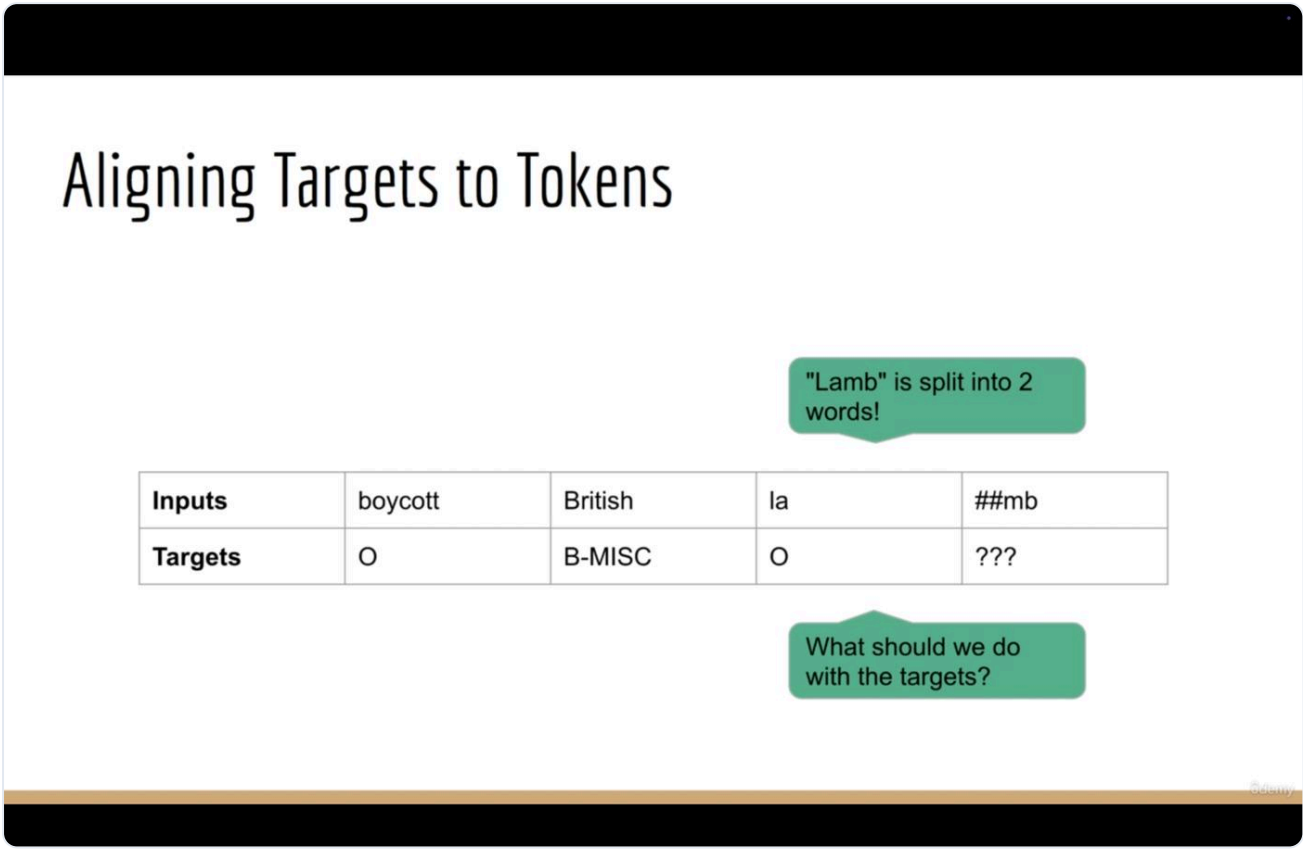

FIGURE 5.16 | Aligning Targets to Tokens

So essentially our strategy will be to realign and expand the targets such that they correspond with the tokenized inputs. Now, what precisely do I mean by that?

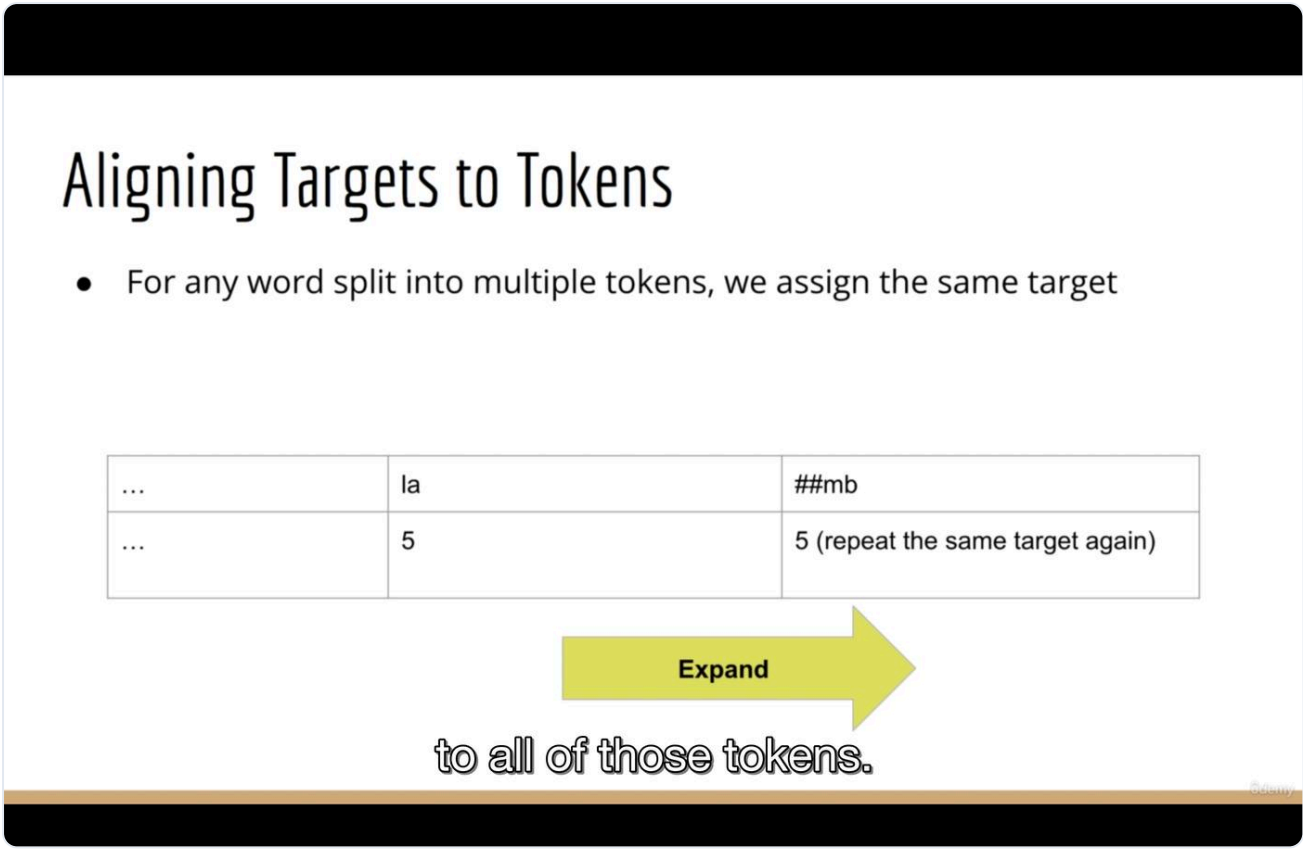

FIGURE 5.17 | Aligning Targets to Tokens

Well, for any input word that is split up into multiple tokens, we are going to assign the same target to all of those tokens. That's what I mean by expand. So if the word lamb has the target five and it's split into two sub words, this simply means that the Target five will repeat twice in the positions corresponding to those sub words.

Effectively by the end of this process, the length of the sequence of targets will be equal to the length of the sequence of the input tokens. Now there is another issue we have to consider, which is that not only do some words get split into sub words, we also have two additional special tokens created, especially for BERT. Some examples are the CLS and SEP tokens which you have seen before, since these are legit tokens that will go into the model.

They will need to be accounted for in the target sequence so that the target sequence has the same length as the input sequence.

But Wait!

- What about [CLS] and [SEP] tokens?
- We need to create targets for these too
- Why? Consider an RNN for the same task (x(t) → h(t) → y(t), for all t)

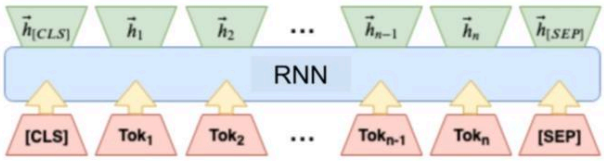

FIGURE 5.18

But Wait!

- What about [CLS] and [SEP] tokens?
- We need to create targets for these too
- Why? Consider an RNN for the same task (x(t) → h(t) → y(t), for all t)
- What value to set? Set them to -100
- Why? Because that's the value the library uses to know to ignore
- Good exercise: think about how you'd create a custom loss function to ignore locations where target is -100

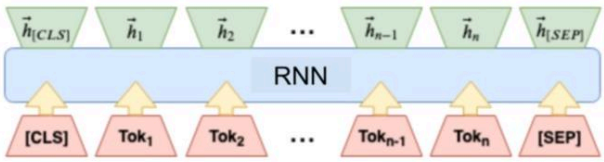

FIGURE 5.19 | We Need to Create Targets for These Too

If you need some intuition for why this is the case, consider how we would train a many-to-many RNN for the same task. We would have a hidden state for every input token and every hidden state would get mapped to a prediction. So we'll have as many predictions as we do input tokens. Therefore, we must have the same number of targets as well.

Again, the theory behind Transformers will come in due time, but for now just relating it back to RNNs should be sufficient. In any case, this is one of those times where the knowledge you need isn't fundamental deep learning knowledge, but rather just documentation familiarity. Specifically, the Hugging Face library uses a special integer value of -100 for outputs and targets that should be ignored. So for CLS and SEP, the corresponding targets should be -100.

Now again, if you want to relate this back to the RNNs you've implemented in the past, that would be a good idea if you were to implement this from scratch. What you would want to do is effectively build a custom loss function where the loss for the special tokens is a constant such as zero. By doing this, the gradient will also be zero and hence those outputs would not contribute to any weight updates. Okay. So now the big question is how do we actually accomplish what I just described?

Unfortunately for some of you, this is where you are going to have to employ your coding abilities. Specifically, we're going to have to write our own algorithm to do this conversion to realign the targets. Now, the typical beginner might ask, But isn't there a library for this? The answer is that we are already using a library. It's called the Hugging Face. But as you know, this is still a new library. So for some tasks, you still do have to write code yourself.

This will be the case for the coming sections as well. At the same time, this is an excellent exercise. So after watching this lecture, I would strongly encourage you to implement what you've seen by yourself.

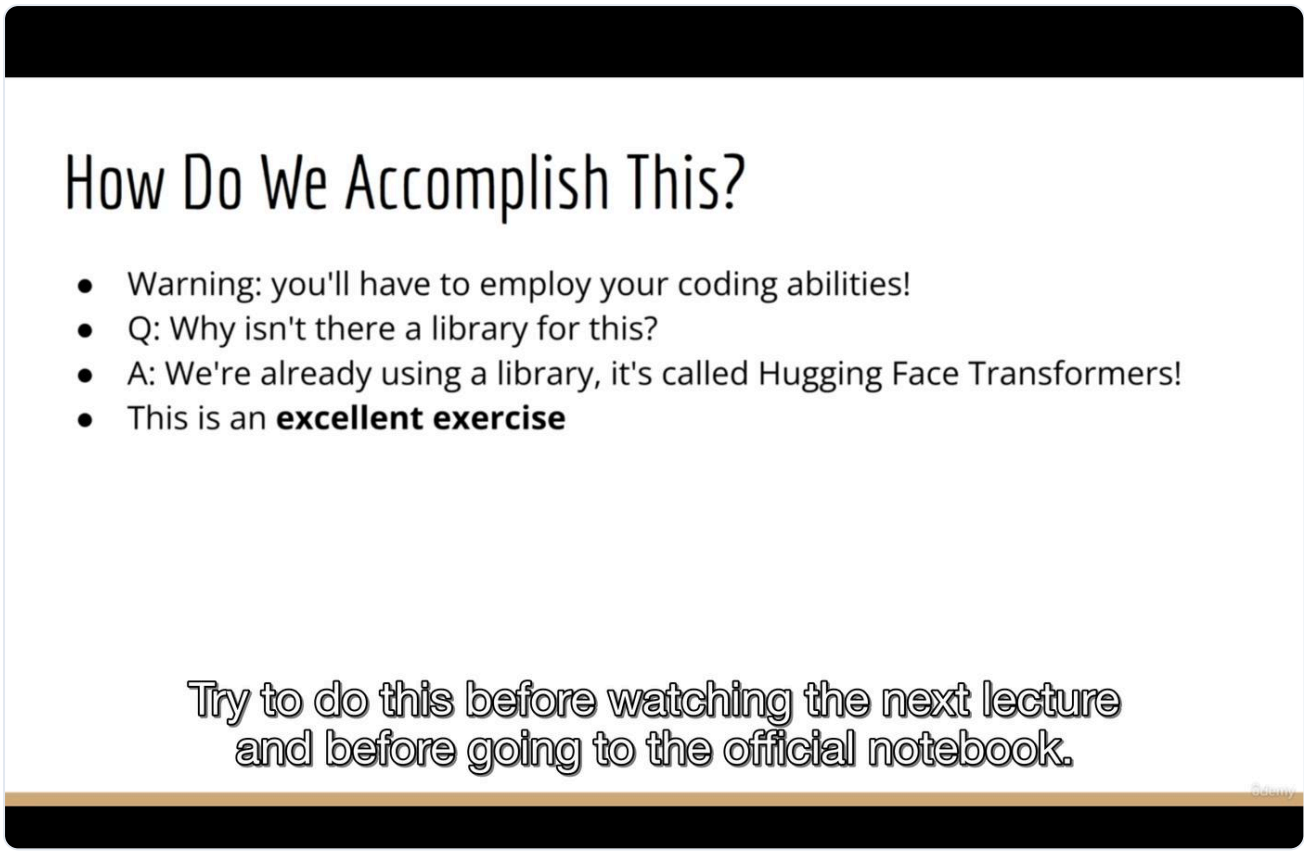

FIGURE 5.20 | How Do We Accomplish This?

Try to do this before watching the next lecture and before going to the official notebook. Although I am going to show you the code in the next few slides. Okay. So the key piece of data we need in order to do this conversion is the word IDs list, which comes from the output of the tokenizer object.

As you recall, the tokenizer output is a BatchEncoding object which has instance methods. Basically, the format of this list is as follows. This list has the same length as the input tokens, so that includes the CLS and SEP tokens along with all the sub words. if any words were split. For CLS and SEP, the word ID is None. Otherwise, the word ID starts from zero and counts up incrementally by one. So the first word in the original input has ID zero, the second word has ID one and so forth.

The important part is that if one of the original words got split up, that word ID will repeat. So consider this made up example. This lamb is little. Since we have four words, the word IDs will be zero, one, two and three.

FIGURE 5.21 | Word IDs List

Now again, this is a made up example, but suppose that lamb is split up into two tokens and little is split up into three tokens. Just for the sake of this lecture. In this case, the word IDs from our tokenizer will be None for CLS, zero for this, one, one for lamb,

two for is and three, three, three for little and one final None for SEP. So hopefully this is sufficient to understand how this works. The next step is to think about a sort of outline of our algorithm. The basic idea is this We're going to loop through our word IDs. From here, we have three scenarios. If we see that the word ID is none, then we should add -100 to our target list. If we see an integer word ID, then there are two scenarios.

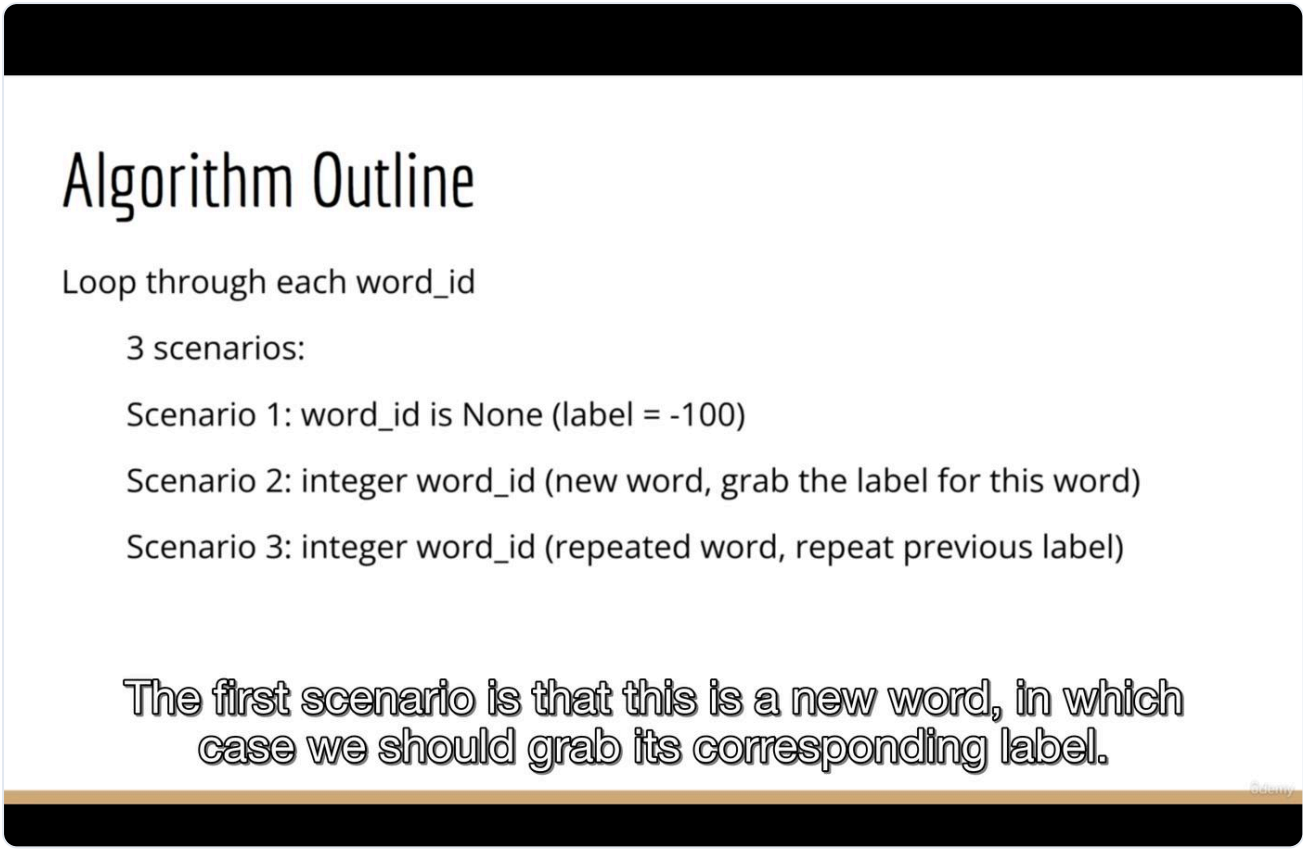

FIGURE 5.22 | Algorithm Outline

The first scenario is that this is a new word, in which case we should grab its corresponding label.

The second scenario is that this is not a new word, and this token belongs to a sub word that we've already seen part of. In that case, we should repeat the previous label. So that's kind of a high level overview of what we want to do. However, note that because of how NER tags work, there is a bit of a complication. As you recall, the tags use IOB format where B is for the first token of an entity. While I means we are inside an entity. In other words, not the first token.

So contrary to what I said previously, we cannot simply repeat the same label twice. For instance, again, suppose for the sake of this lecture that the name Steve gets split into two tokens [unclear]. The Steve word would have the label B-PER because it's the beginning of a person tag. But when we split this into two tokens, we do not want to copy B-PER twice.

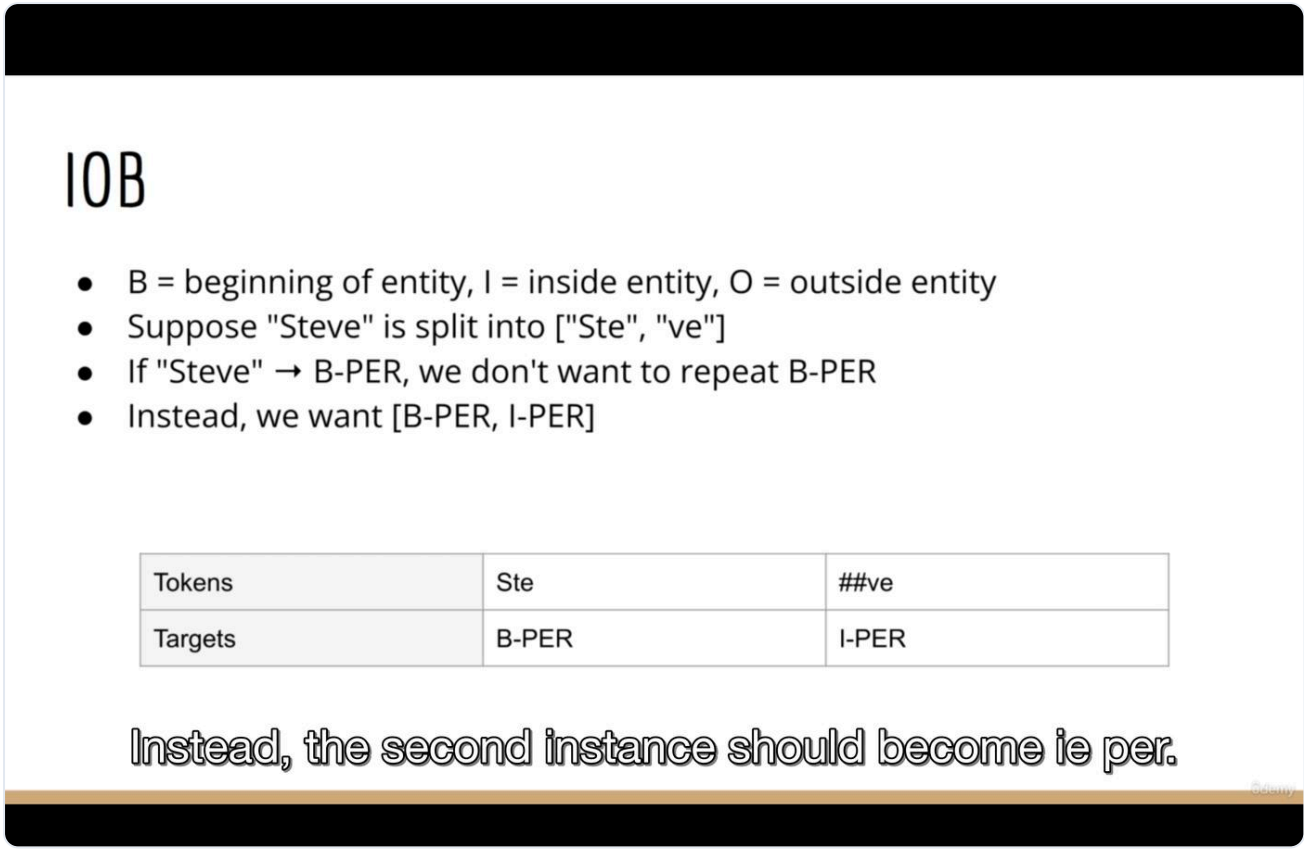

FIGURE 5.23

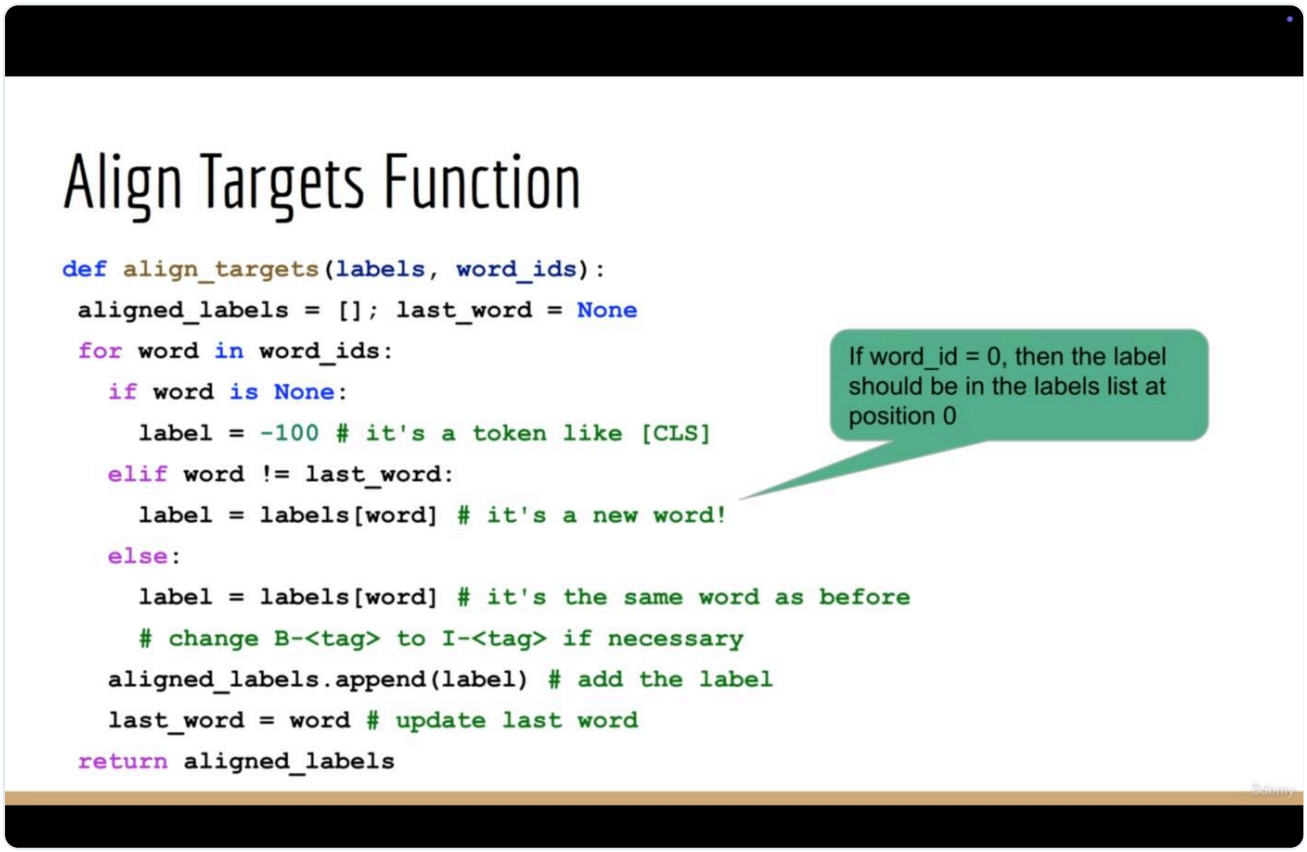

FIGURE 5.24 | Align Targets Function

Instead, the second instance should become I-PER. So at this point, you have all the information you need to understand our token alignment method.

In this lecture, I'm going to show you a full code. But as mentioned, after you've seen this, you are strongly encouraged to go and implement this yourself without looking. So we begin by defining a function called align_targets, which takes as input two lists, the labels and the word IDs. Keep in mind the length of these two lists, the word IDs have the same length as the tokenized input, but the labels are just the original labels, so they have the same length as the original input.

In other words, if there are any subwords present, it will be shorter. Our goal is to create a new list called Aligned Labels, which will be the same length as the tokenized inputs. We'll initialize that to be an empty list. We'll also initialize a variable called last_word, which we will set to none. The next step is to loop through each of the word IDs. Inside this loop, we just have to check the three scenarios we outlined before. First, we check whether or not the word ID is none.

If it is, then we know it's a special BERT token, so the label should be -100. The next thing we check is whether or not the current word ID is equal to the previous word. If that is not the case, then this is a new word, at which point we can just index our labels by the word ID. So why does that make sense? Well, remember that the word IDs are just zero, one, two and so forth. Therefore, they also serve as indices into the labels list. So if I index the labels at the index word ID, that will be the correct label for this word.

The else block covers the third scenario, which is the tricky one. We'll begin by assuming that the label should be the same as the labels list, indexed by the word ID, which is the same as the previous label. However, we should have some way of checking this label. If it's a label that starts with a B. We'll need to replace it with the corresponding label that starts with an I. For instance, B-PER becomes I-PER and B-ORG becomes I-ORG. At the same time, This is more complicated than simply checking whether the label starts with a B, because as you recall, the labels are stored as integers and not strings.

The details of this will be left until the actual notebook, but please try to figure it out yourself before we move on. Finally, once we are done checking all three scenarios and finding the correct label, we can append it to our list of aligned labels. We should also update our last word variable to be the current word. Once we exit the loop, we can return our aligned labels as they will have been populated.

5 · Create Tokenized Dataset (Code Preparation)

Okay. So after we've written our method for aligning targets, there is still more work to be done. In particular, recall that in the previous section, after creating the tokenizer, we wrapped it in another function to add the truncation step. From there we called the map function on our data sets to apply the tokenizer to every sample in batches. However, we weren't able to do that step previously because our targets were not yet aligned with the inputs.

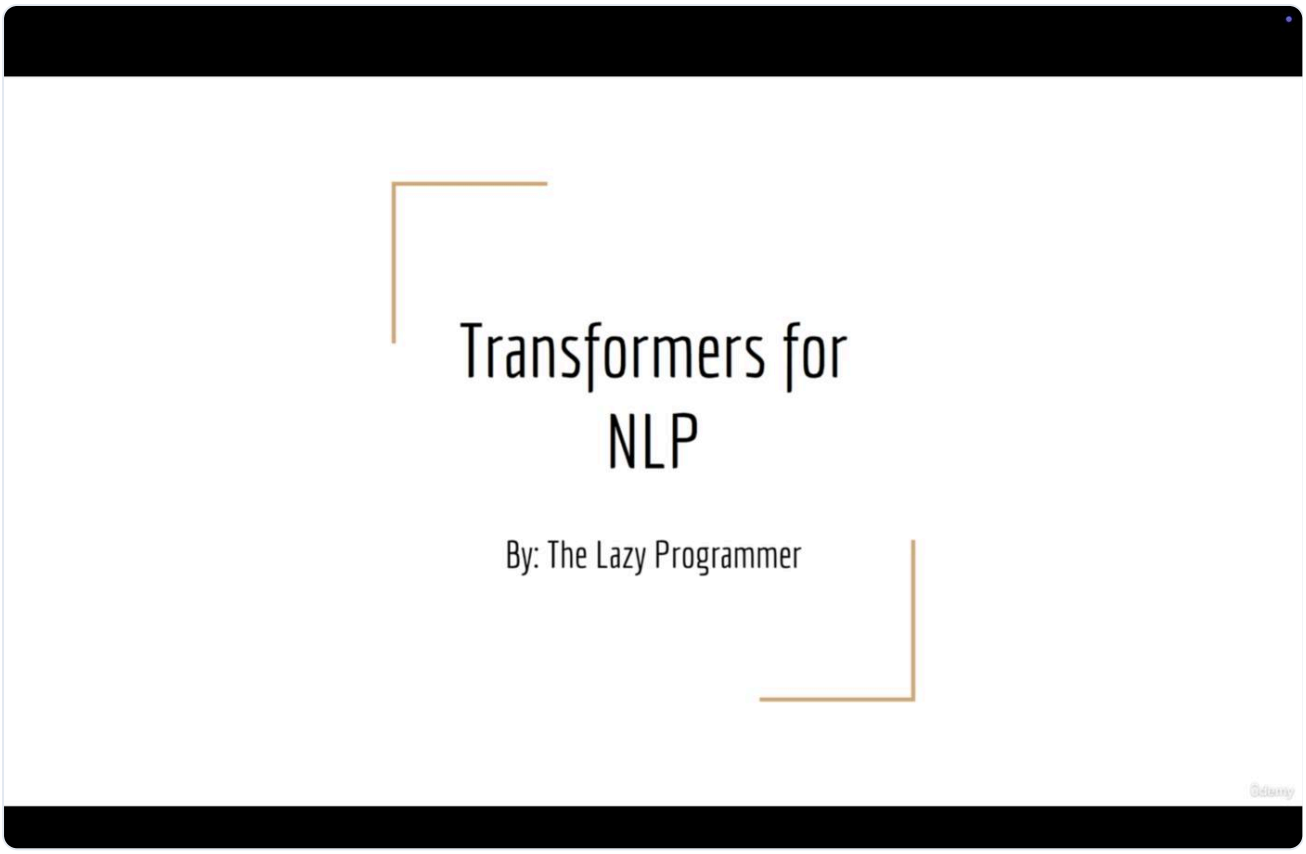

FIGURE 5.25 | Transformers for

Now we have a function that can do this, so we are ready to move on to this step. As you recall, our data set objects for BERT should have several columns. Specifically, they are input IDs, attention mask, token type IDs and labels. Token type IDs will be present for BERT, but not the DistilBERT. But you don't have to worry about that since the tokenizer will populate those fields for us.

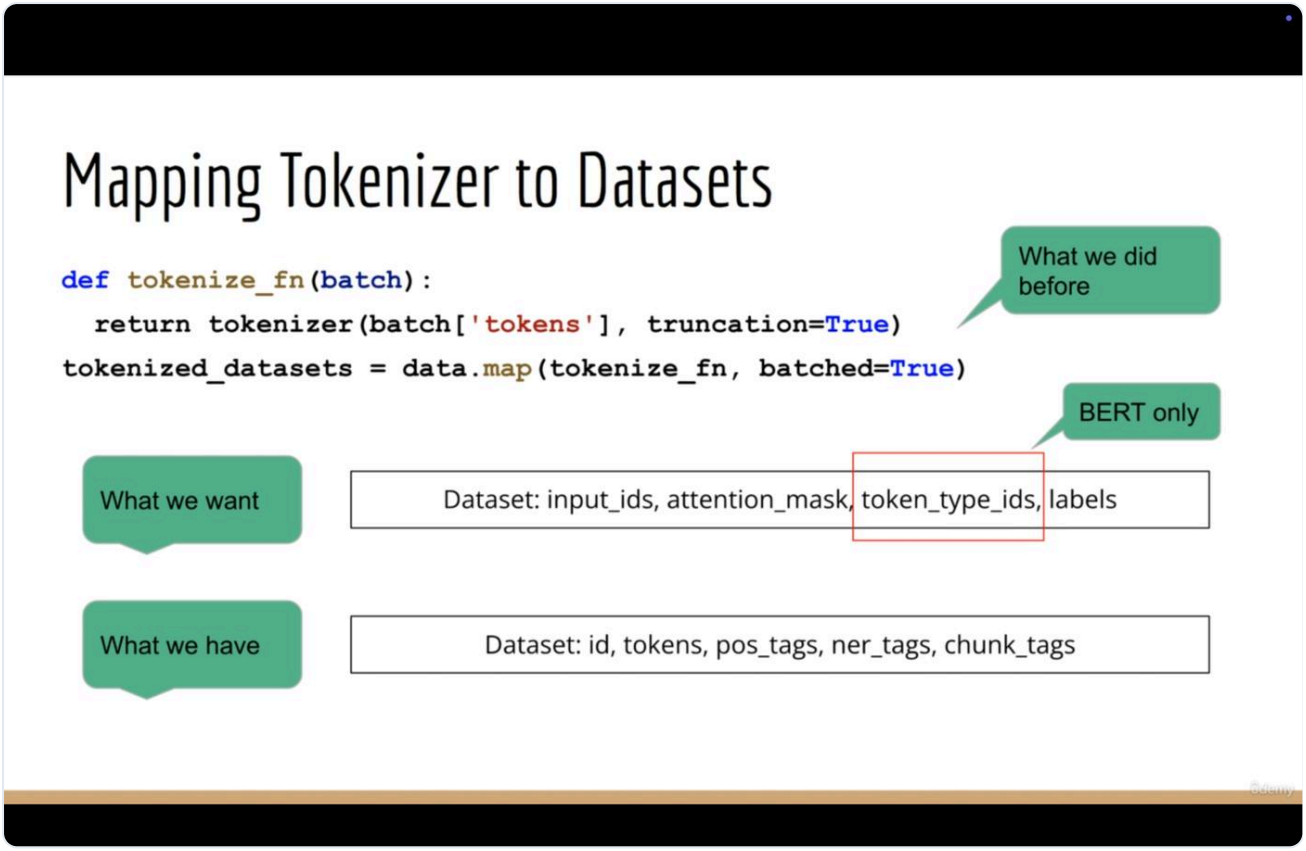

FIGURE 5.26 | Mapping Tokenizer to Datasets

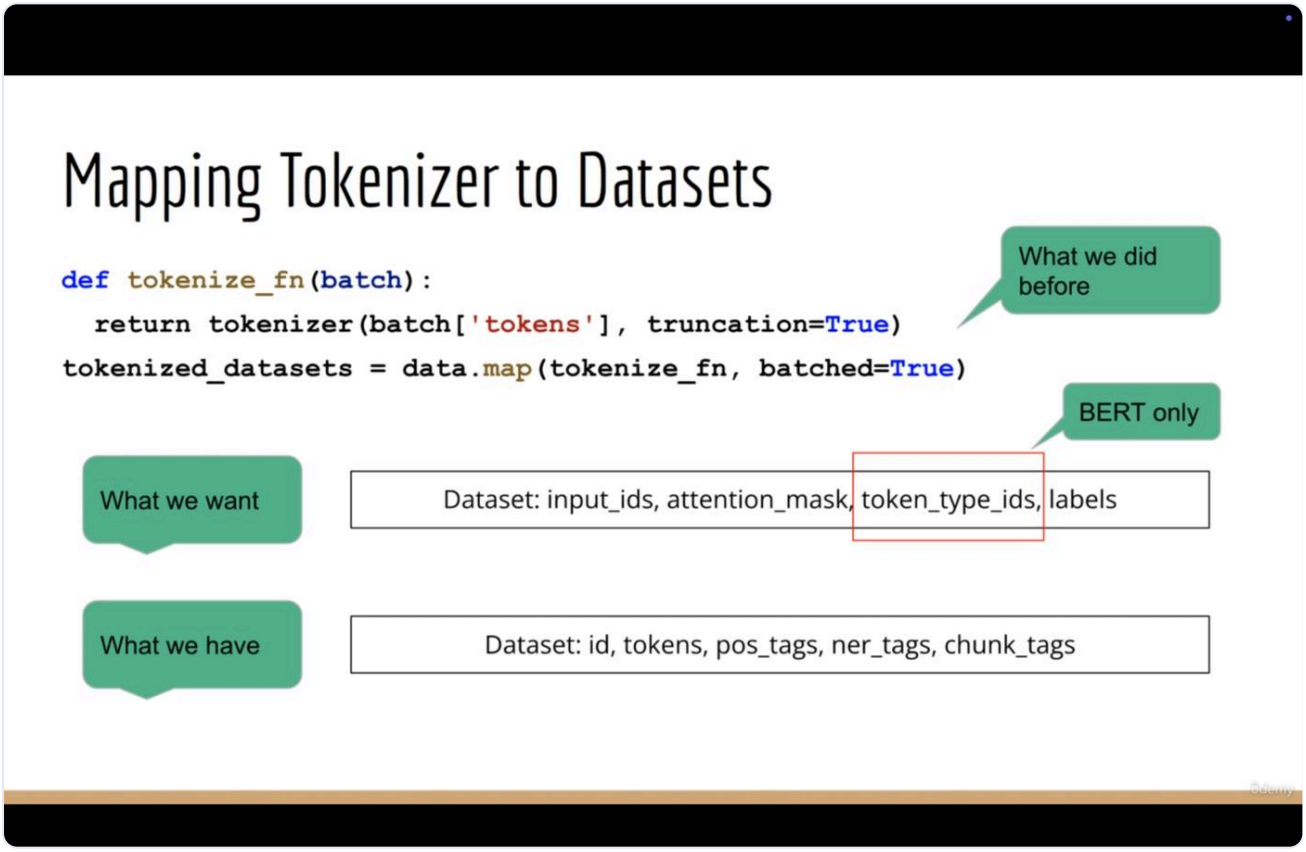

FIGURE 5.27 | Mapping Tokenizer to Datasets

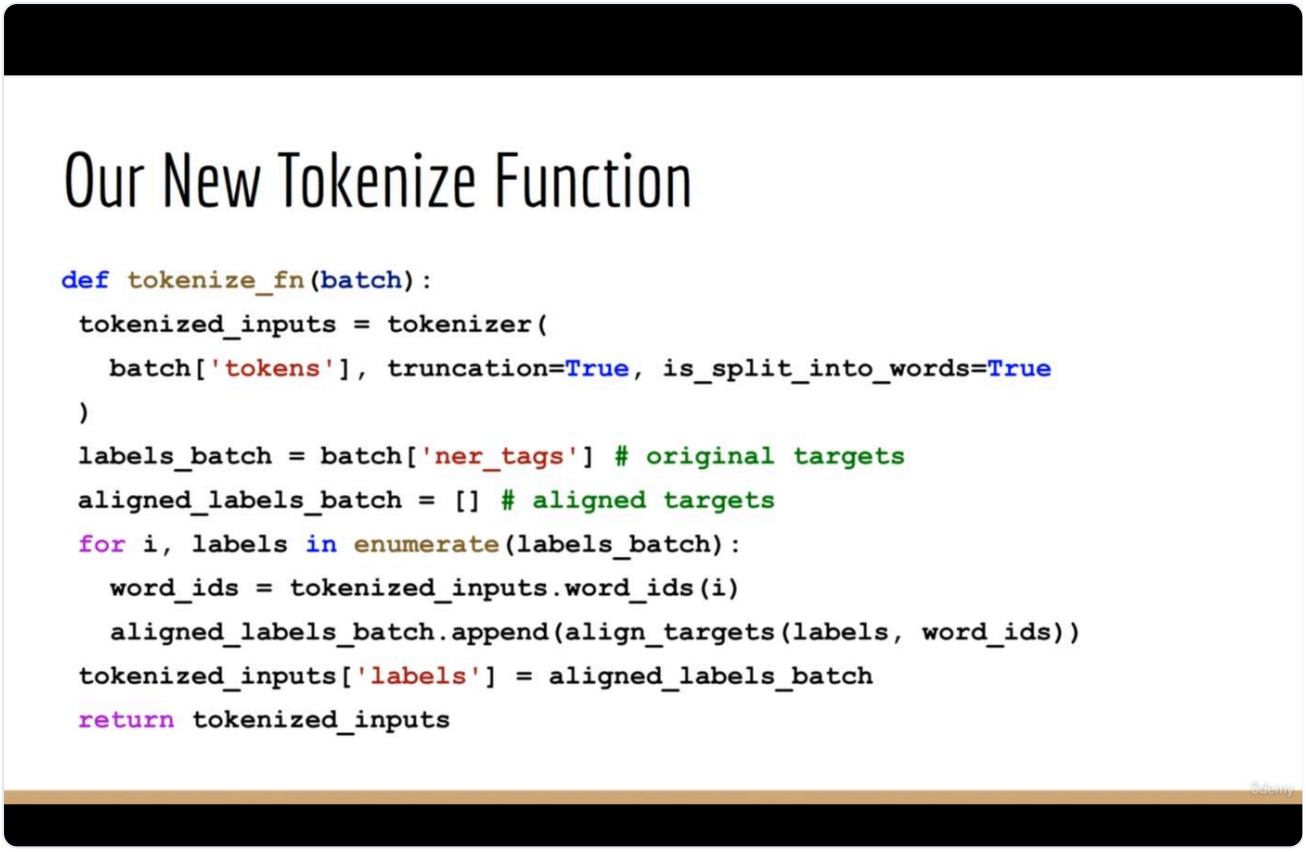

FIGURE 5.28 | Our New Tokenize Function

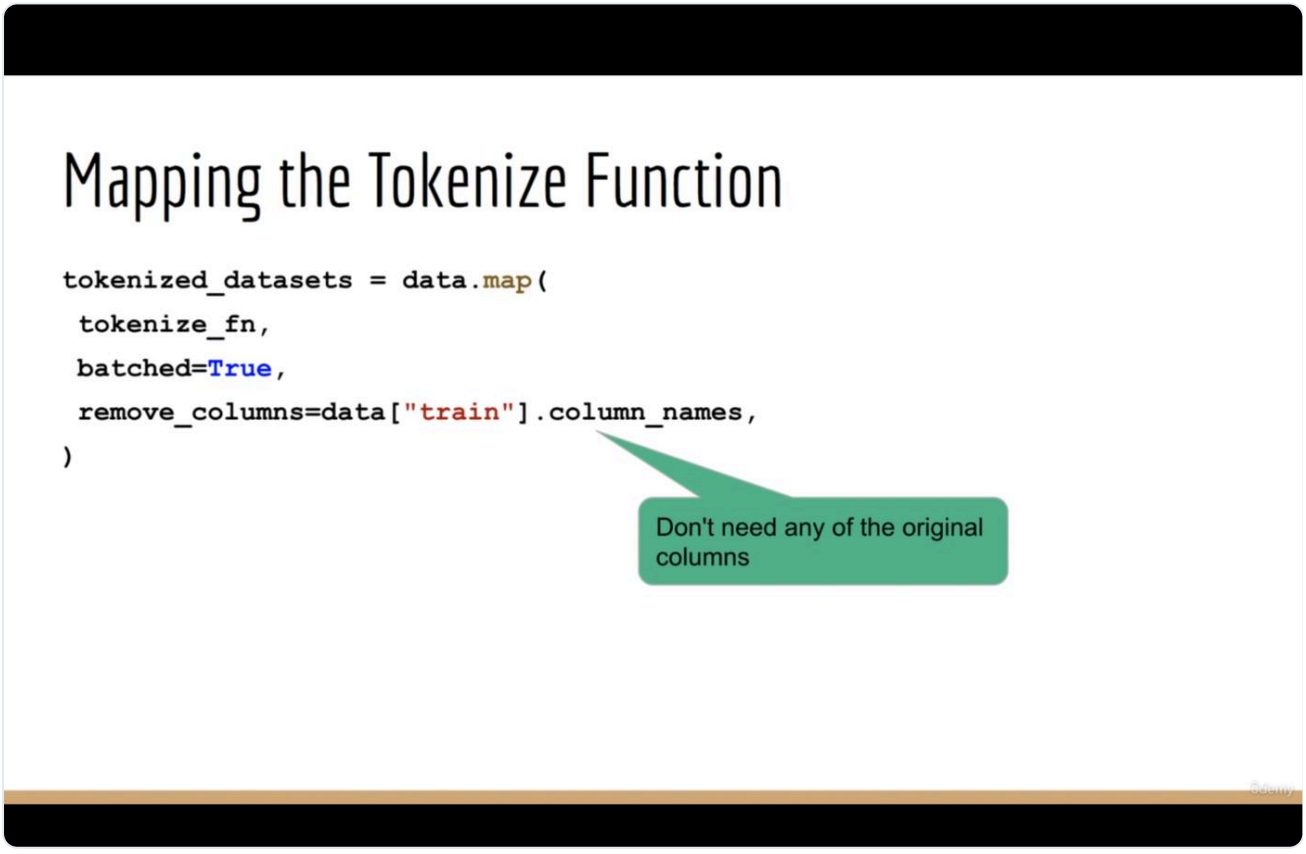

FIGURE 5.29 | Mapping the Tokenize Function

We've seen that the data set objects we loaded in currently have the columns, tokens, POS tags, NER tags and chunk tags.

So basically we need to write a tokenize function that takes these input columns and gives us back the columns we need. So again, I'm going to show you some code, but ideally you should try to recreate this yourself before moving on to the notebook. We'll begin by defining a new tokenize function that takes in a batch of data as input. Inside the function, we can immediately call our tokenizer on the tokens column of the batch. These are the inputs. We add truncation equals true as usual, and we also add is_split_into_words equals true.

Since our data is split into words, this will give us back tokenized inputs. In the previous section, this was all that we needed. In this section, we need to do more because we still need to populate the labels. We'll begin by grabbing the tags, which are the original targets. We know that we can't use this because they are not aligned with the tokenized inputs. We'll create a new empty list where we will store the aligned targets. The next step is to loop through each label sequence in the batch.

We use enumerate because this will give us the index which we will use inside the loop. Speaking of which, the next step is to grab the word IDs for the sample in the batch. Luckily, the word IDs method accepts an index as inputs. Once we have a word IDs, we can call our align_targets function, which we looked at in the previous lecture. This will give us back our aligned labels, which we can add to our list. Once that loop is over, we'll have all of our aligned label sequences and we can add that to a column called labels in the tokenized inputs. When we call the map method later,

This will add all of these new columns to the dataset object. So as you know, the next step is to pass our tokenize function into the map function of the data set dict, and this will apply that function to each sample. One extra detail which is useful to know is how to remove irrelevant columns. As you recall, the tokenize function will add new columns that we need for the model such as input IDs, attention mask labels and so forth. But we don't need the original columns like tokens and NER tags and POS tags.

In this case, there is an extra argument called remove columns that we can use. We can pass in the column names of the existing dataset objects. Since we don't make use of any of the original columns.

6 · Target Alignment (Code)

See the notebook here.

So in this lecture we will be looking at the code to do label alignment, which is a continuation of the same notebook we previously saw. We'll begin by reviewing our sample sentence. It states EU rejects German call to boycott British lamb. Importantly, the word lamb has been split up into two sub words. Our goal is to expand the targets so that the same target will apply to both sub words. The next step is to check out the word IDs for a sentence. So as you can see, the second last word ID, which is seven repeats twice as expected.

Note that eight corresponds to the period. So the next step is to implement our align_targets method. One detail we didn't look at in the theory lecture is how we will map begin tags to inside tags for words that get split up. To do this, we need to know the corresponding IDs. In the com-

ment, I've pasted the label names from above which will help us understand how to map the IDs. As you can see, the B version always precedes the AI version. So we have B-PER then I-PER.

We have B-ORG and then we have I-ORG. And as you know, the indices of these label names correspond to the label IDs. In other words, label one should map to label two, label three should map to label four and so forth. This takes care of all four types of entities. So the next step is to look at our align_targets method. As input, we'll take in a sequence of labels and a sequence of word IDs. Inside the function, we'll begin by initializing the aligned labels to be an empty list.

We'll also set last_word to None, although note that this technically is the last word ID, not the actual word. The next step is to loop through all the word IDs inside the loop. We first check if the word IDis none. Recall that this corresponds to special tokens like CLS. If this is the case, then our label should be -100. The next case we check is if the current word is not equal to the last word. That means we are looking at a new word. Since we are looking at a new word.

We can be sure that B tags don't have to be converted into I tags. So we can just take the current label as is. Now you might wonder why is the label equal to the labels list indexed at the word ID? Well, recall that the word IDs are sequential, starting from zero, then one, then two, and so forth. So clearly these are indices into the labels as well. For instance, the label for the zeroth word is just the labels list at position zero. The label at position one corresponds to the word with word ID one.

So hopefully that makes sense. The else block handles the third case where our word ID is the same as the previous word ID. This means that we are going through the subword IDs corresponding to the same word and this is the second or later sub word. In this case, we'll start by assuming that the label is just the label for the given word ID. The next step is to check whether or not this is the actual correct label. We'll know if it's not if it corresponds to any of the labels that start with B.

Since the current sub word is not the beginning of a word, this corresponds to any of the label IDs which are keys in the dictionary we called begin2inside, which we just defined. So first we check whether or not this label ID is in this dictionary and if it is, we grab the corresponding value, which is the I tag. So after we've checked all of the three cases, we have our label, which can be appended to our list of aligned labels. The final step in the loop is to update the last word ID to be the current word ID.

Lastly, we return the list of aligned labels. The next step is to test our new function on the sentence from above. We begin by getting the labels, which are the NER tags. We then grab the word IDs and then call our function to get the aligned targets. So as you can see, this returns a list which starts and ends with -100. But it's kind of difficult to tell whether or not this is correct, just based on integers. So in the next step will map our target list back into their corresponding label names and print them alongside the original tokens.

So as you can see, the labels appear to be correct. Our aligned labels list is equal in length to the sequence of tokens. And the same target applies to both sub words of the word lamb. Now. Unfortunately, LAMB is not the best word to test on our function because it's just an O tag, which

is not that interesting. Instead, it would be better if we could test a tag that's an actual entity because we want to see whether or not B tags get correctly transformed into I tags.

So to that end, I'm going to create a fake sentence with fake word IDs. We know that German should not be split into subwords, but for the sake of this example, we are pretending that it is. I've also put in the word Microsoft so that we have an org label as well. Note the word IDs, the word id zero repeats twice because that corresponds to German, while the word ID four repeats twice because that corresponds to Microsoft. The labels are 7, 0, 0, 0, 3, where seven represents B-MISC and three represents B-ORG.

After that, we run the same code as above. So as you can see, our function works as expected. The second token for German gets converted into I-MISC and the second token from Microsoft gets converted into I-ORG. So the next step in our notebook is to write our tokenize function wrapper, which does a bit more than it did in the previous section of the course. In particular, it not only tokenizes each input sentence in the batch, it also aligns the labels as before.

The input into this function is a batch of samples from a data set. We begin by calling the tokenizer with truncation set to true as before. We also set is_split_into_words equals true since our data is already split into words. This gives us the input IDs, attention mask and so forth. The next step is to align the targets for every sample in the batch. So we begin by grabbing the labels for the whole batch which are stored in the column ner_tags. We then create an empty list where we will store our aligned labels.

The next step is to loop through each label in our batch of labels. Note that we also use enumerate since we will be using the index of the sample. The next step is to call the word_ids method passing in the index i to get the corresponding word IDs for the current sample. The next step is to call the align_targets function, which we wrote above passing in the current labels and the current word IDs. We append the result to our list of aligned labels.

When the loop is complete. We assign the list of aligned labels to a column called labels. As you recall, for this library, the target must be stored with this name. The final step is to return our tokenized input object, which contains both the input IDs, attention mask and so forth, but also the aligned labels. So the next step is to remember that our dataset has a whole bunch of columns which we don't need when we call the map method. Later, the new columns will be added to the existing ones.

In this block of code. We're just going to print out the existing column names. So as you can see, we don't need any of these columns to train or use our model. Our model will be using the input, IDs, attention, mask and labels instead. The next step is to call the map method passing in our tokenize function from above. As before we set batched equal to true so that the operation is more efficient. In addition, we are going to remove the existing columns since they are no longer needed.

So the next step is to just print out the result. So as you can see, we now have a new dataset dict object, but the columns in the data sets are what we need for the model, specifically the input IDs, attention mask and the labels.

7 · Data Collator (Code Preparation)

So in this lecture we will be discussing the concept of the data collator, which we avoided discussing earlier in the course. To understand the role of the data collator Consider our lesson on text preprocessing. As you recall, there were several required steps before we could pass our tokenized input into the model.

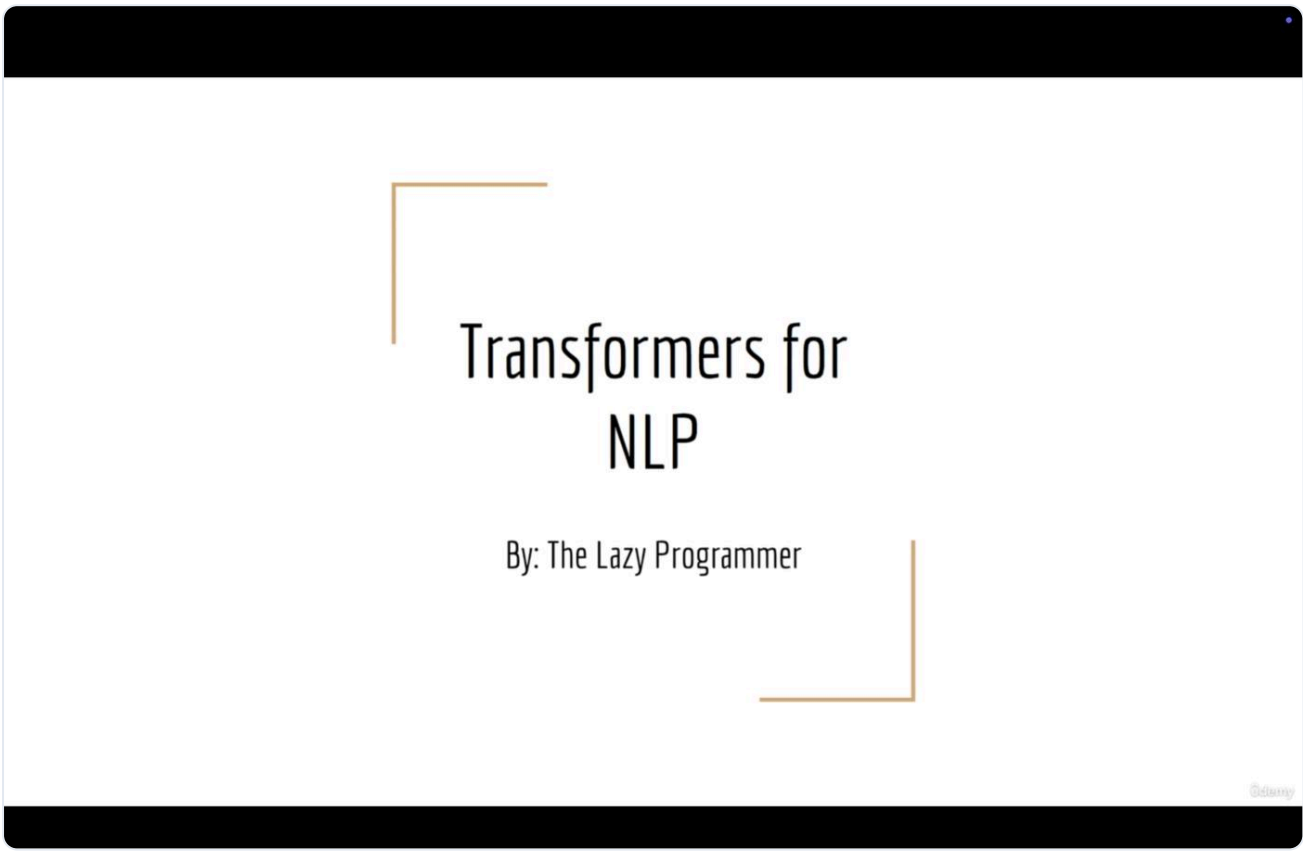

FIGURE 5.30 | Transformers for

Specifically, we had to pad the tokens, truncate the tokens, and finally convert them into PyTorch tensors. But you've seen that when we use our tokenize function, we don't do most of these steps.

This begs the question when do these other steps happen?

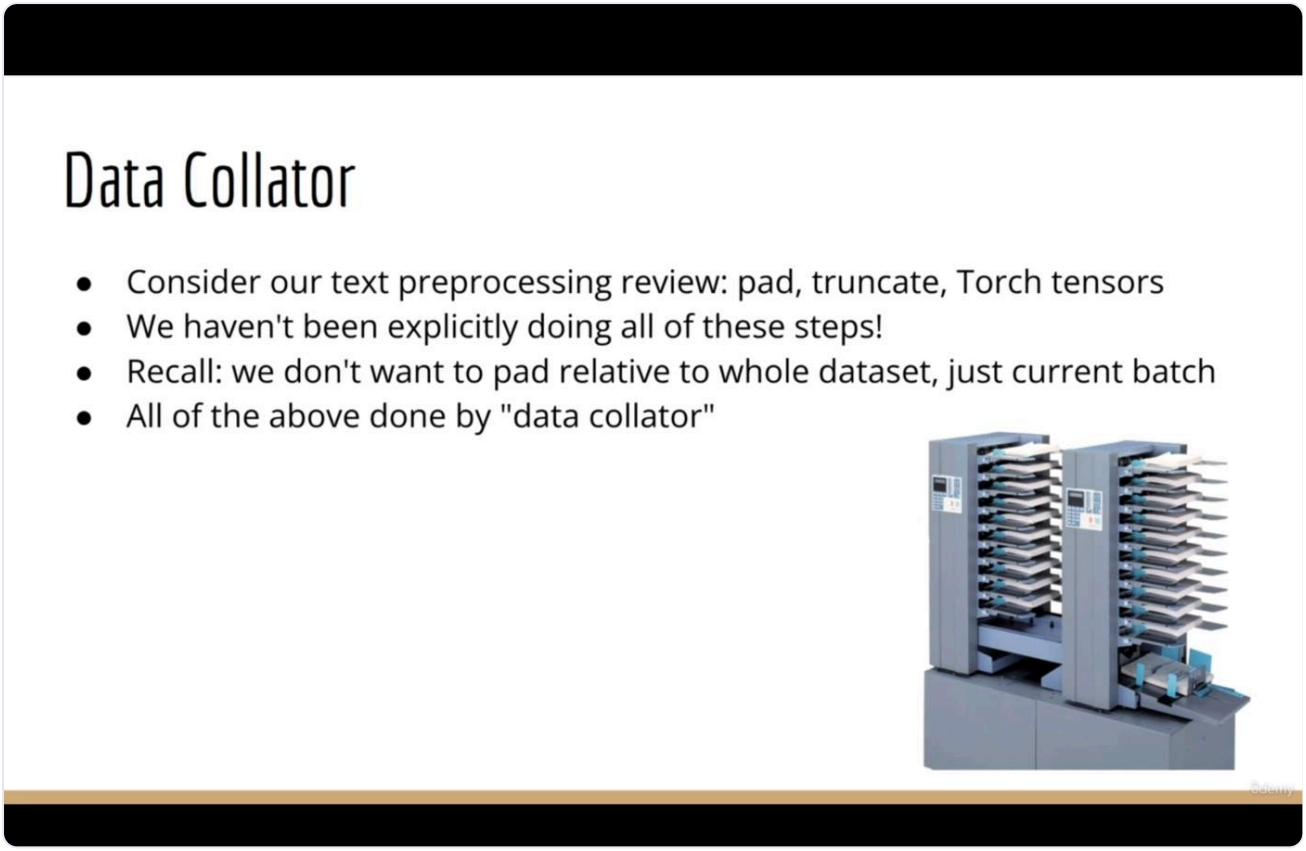

FIGURE 5.31 | Data Collator

Furthermore, I mentioned earlier that when we pad our tokens, we don't want to pad everything to be the same size relative to the whole data set, since the longest sequence may be much longer than everything else. As such, we would rather do padding on a per batch basis. The answer is that all of this is done by the data collator. So instead of the tokenizer doing the padding and conversion into torch tensors,

These are now done by the data collator.

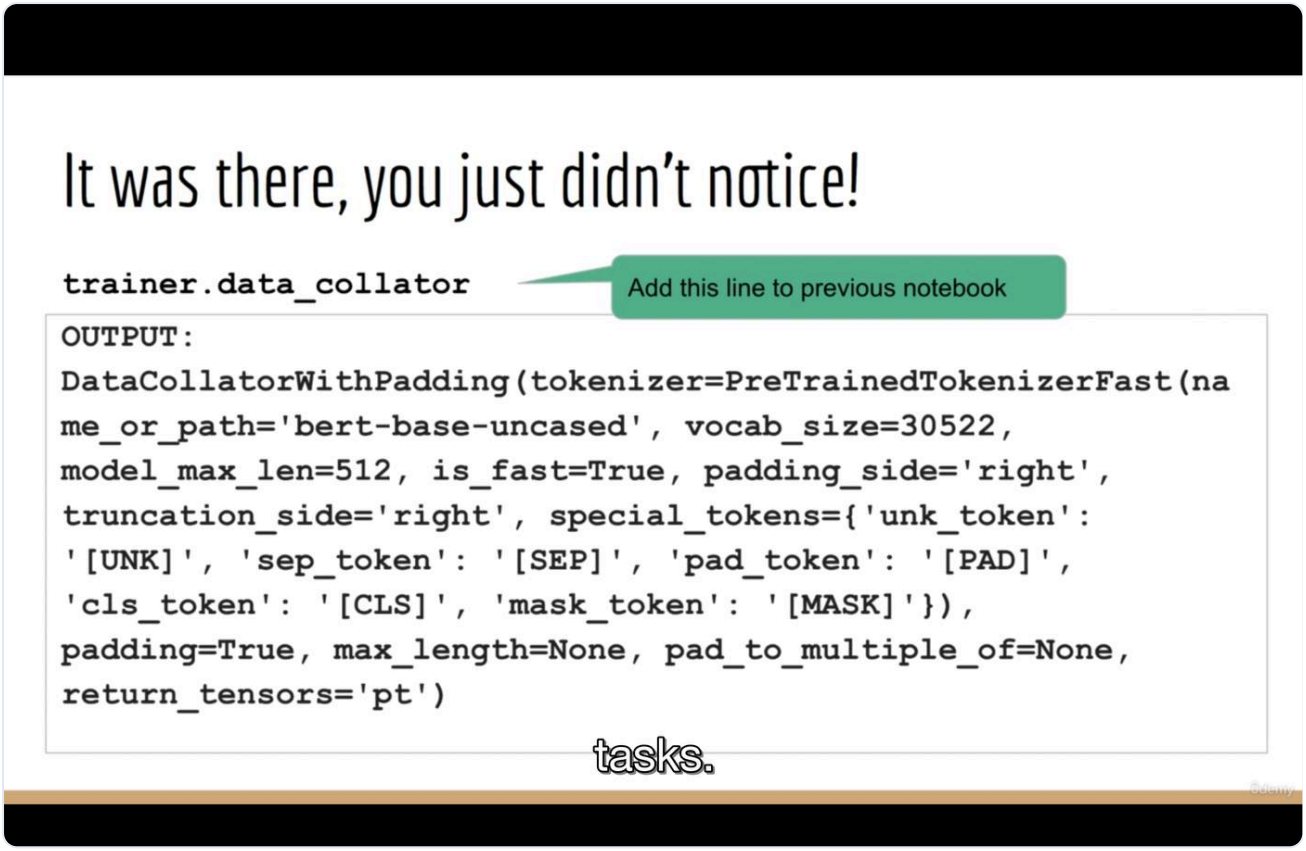

FIGURE 5.32

Furthermore, these are done only on the current batch rather than on the whole data set at once. In reality, the data collator has been working behind the scenes this entire time in the previous section. For some reason, the data collator is built into the trainer and works for sequence classification tasks. You can check this yourself to confirm that this is true. If you go back to the notebook from the previous section, you will see that the trainer object has an attribute called data collator.

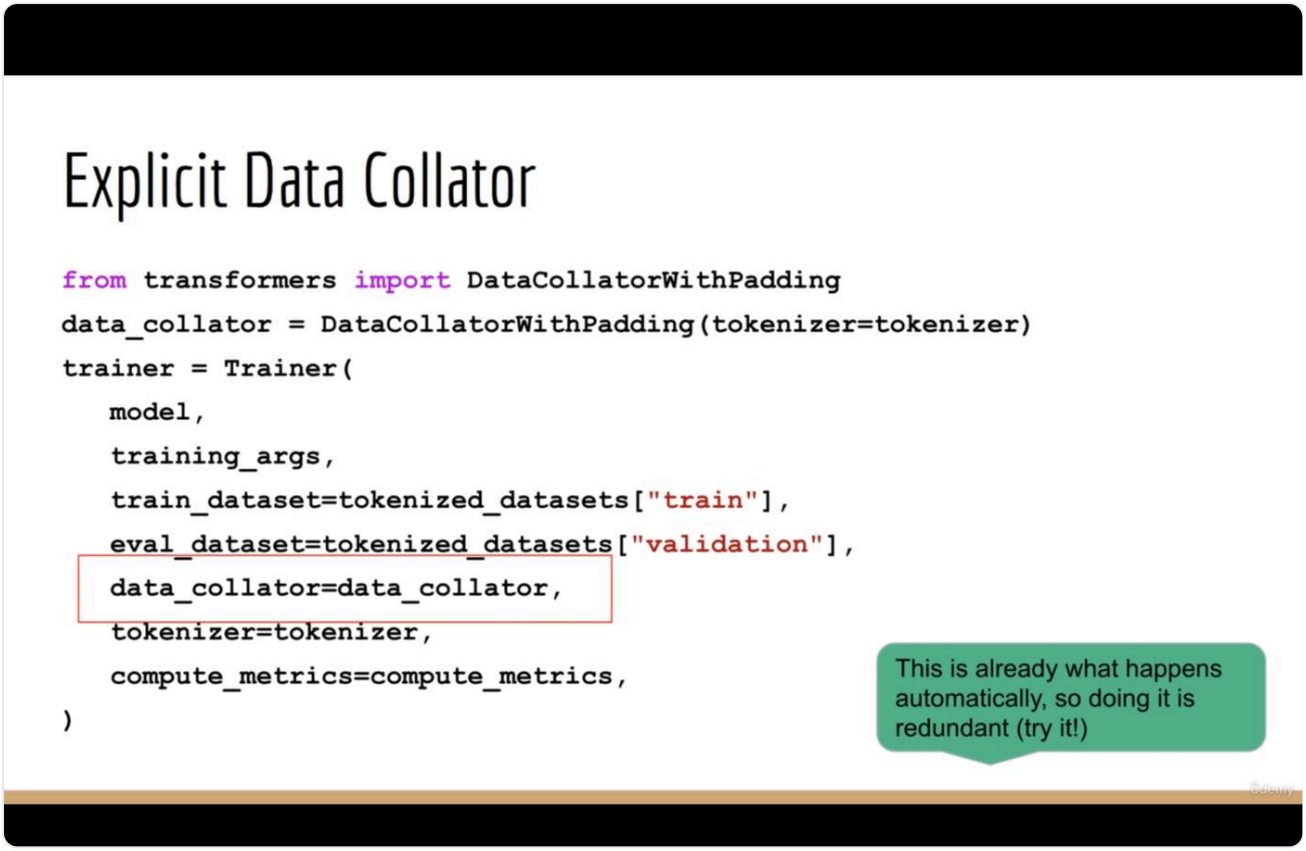

FIGURE 5.33 | Explicit Data Collator

If you print out a trainer data collator, you will see that this is an object of type DataCollatorWithPadding. This is precisely the data collator we would need in the previous section if we wanted to define one ourselves, but that wouldn't have been useful since it would be the same as what we already got automatically. If you want to try this yourself, here's the code that you would use. First you would import the class DataCollatorWithPadding from transformers.

Then you would create a data collator object passing in the tokenizer we created.

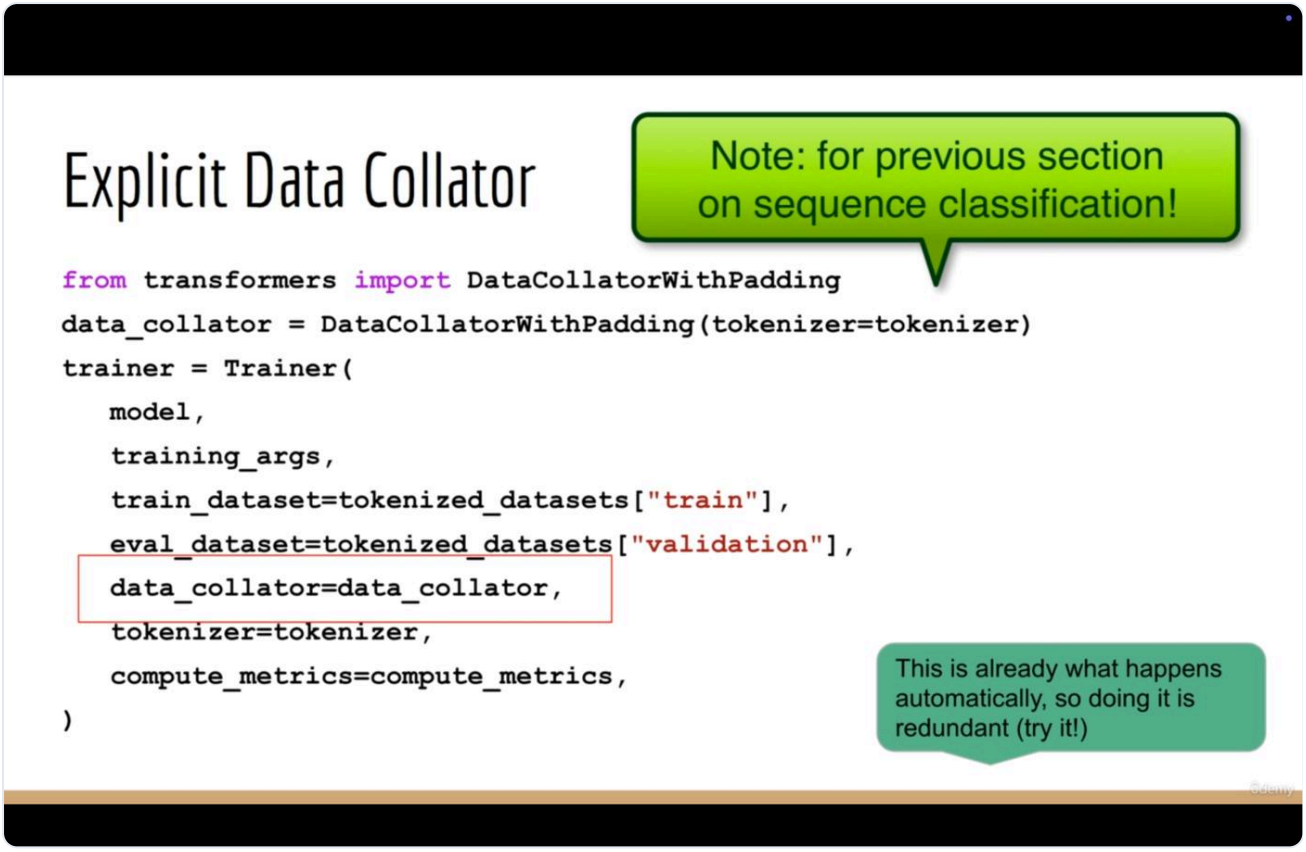

FIGURE 5.34 | Note

Finally, when you create your trainer object, you would pass in the data collator explicitly. You will see that this has no effect and the result will be the same as what we already got. And it will also take the same time to train. Since as I stated, this data collator is already instantiated behind the scenes automatically. On the other hand, for this section, the default data collator does not work for our task and we need to define one manually.

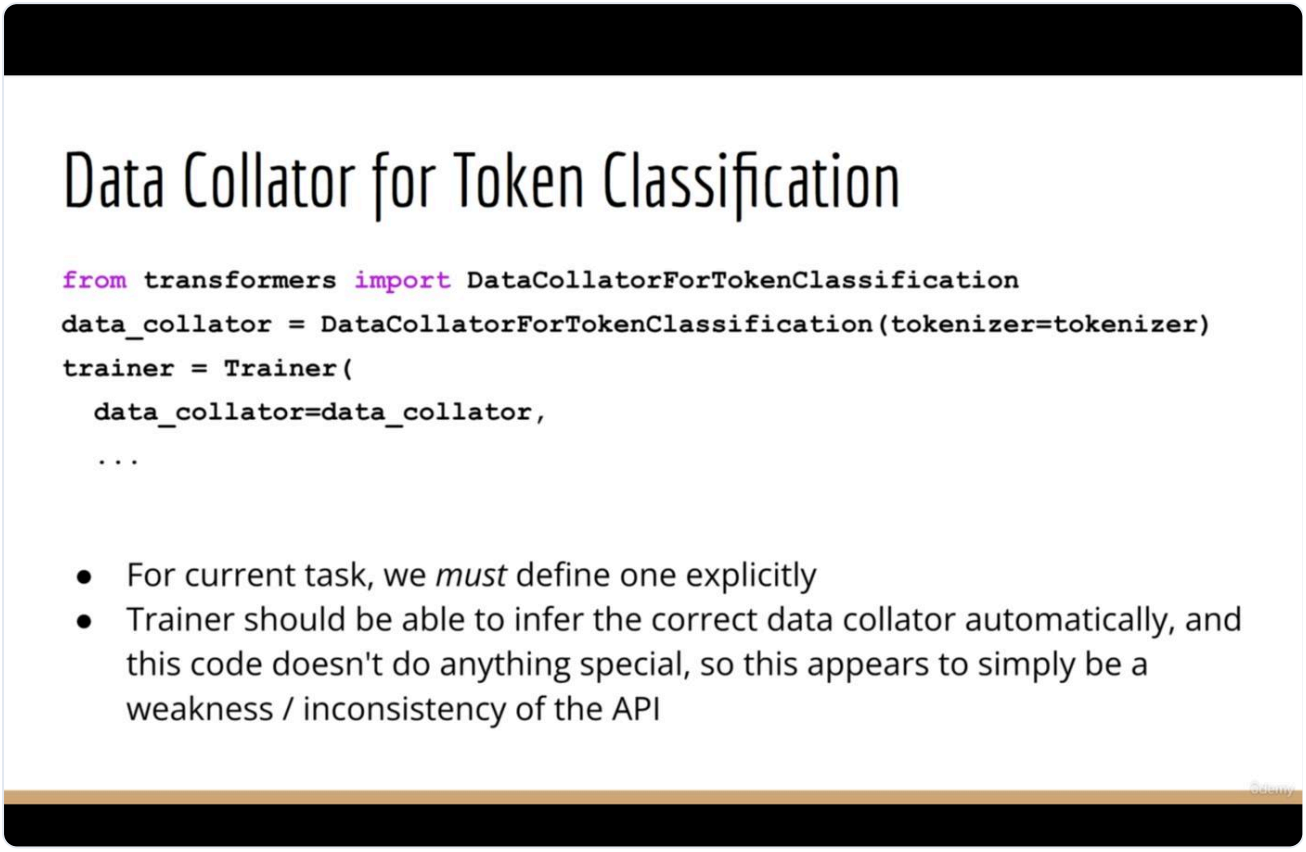

FIGURE 5.35 | Data Collator for Token Classification

In my opinion, this is just a weakness of the API, since the trainer object should be able to infer what type of task we are doing based on the model and create the correct type of data collator based on the task. Unfortunately, at this time this is not the case and so we still need to define our own data collator objects. Specifically in this section, we'll create an object of type DataCollatorForTokenClassification. But although the object type is different, the steps are exactly the same.

We create an object by passing in our tokenizer. Then later, when we create our trainer, we will pass in this data collator object along with the tokenizer, which is kind of redundant.

8 · Data Collator (Code)

See the notebook here.

So in this very short lecture, we are going to continue our notebook and look at the steps to create and use the data collator. We'll begin by importing the class DataCollatorForTokenClassification. We'll also create an object passing in the tokenizer that we created above. Now, although we will not normally have to use the data collator manually, it might be nice to see how it works. As you recall, our data set object is quite flexible in the way it can be indexed.

One method, if we want to grab multiple samples at once, is to use a range index. So for instance, if I want to grab at the first two samples, I can index it using zero colon two. As you can see, this returns a dictionary of lists of lists. This is opposed to a list of dictionaries where each dictionary

is a separate sample. That should remind you of the JSON we created earlier. Instead, we just get one dictionary which holds all the samples. Unfortunately for some reason, this is not a format that the data collator can work with.

Instead, what we need to do is grab a dictionary for each sample separately and store them in a list. So you can see here that I'm looping through the integers from zero up to two, and for each integer I'm grabbing that sample from our tokenized data sets. As you can see, this returns a different format, which is a list containing dictionaries, and each dictionary is a separate individual sample. The next step is to pass in what we had above into our data collator.

We'll print out the labels to sample what we got back. Although you're encouraged to check out the other attributes as well. So as you can see, we get back a PyTorch tensor. That's one thing to notice. Another thing to notice is that these labels have been expanded. As we described earlier in the section. We have -100 for all the tokens where we don't care about the output. For instance, the CLS and SEP tokens. We can also see that -100 applies to the pad tokens as well.

Since our second sentence is much longer than the first. This makes sense because we don't want any pad tokens to contribute to the loss. If you've taken courses with me before where I taught you how to build RNNs for many-to-many tasks, you've seen that if you do include pad tokens in the loss, this will lead to poor performance. Your model will just end up predicting pad token all the time, which makes the accuracy look good, but it will perform poorly on the labels you actually care about.

9 · Metrics (Code Preparation)

So in this lecture we will look at how to compute metrics for token classification tasks. The main issue, which is the reason why we need to discuss this, is because computing metrics isn't as straightforward as it is when you only have a single target per sample.

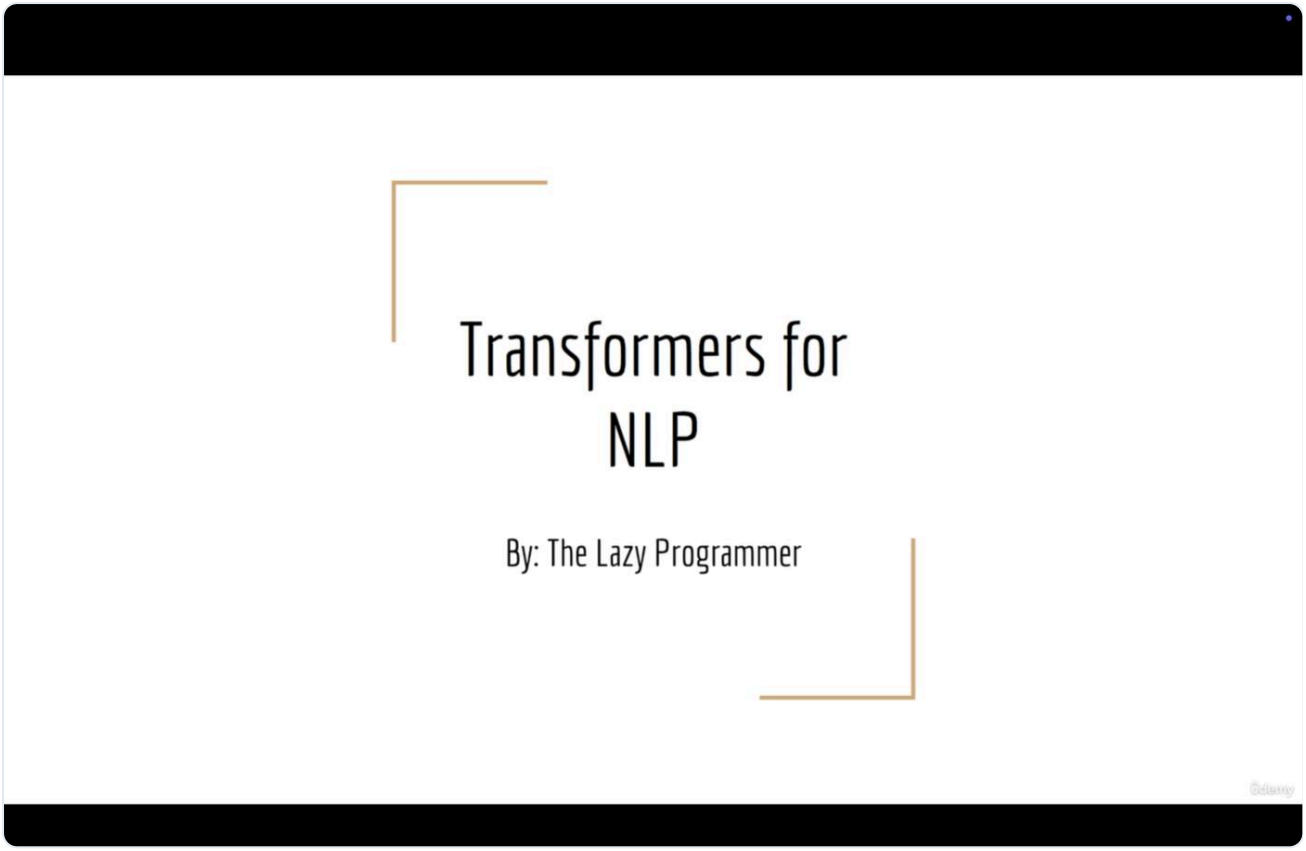

FIGURE 5.36 | Transformers for

For instance, consider accuracy. When we compute accuracy, we count up the number of samples whose target we guessed correctly and divide that by the total number of samples. But what if there are multiple targets per sample as there are now?

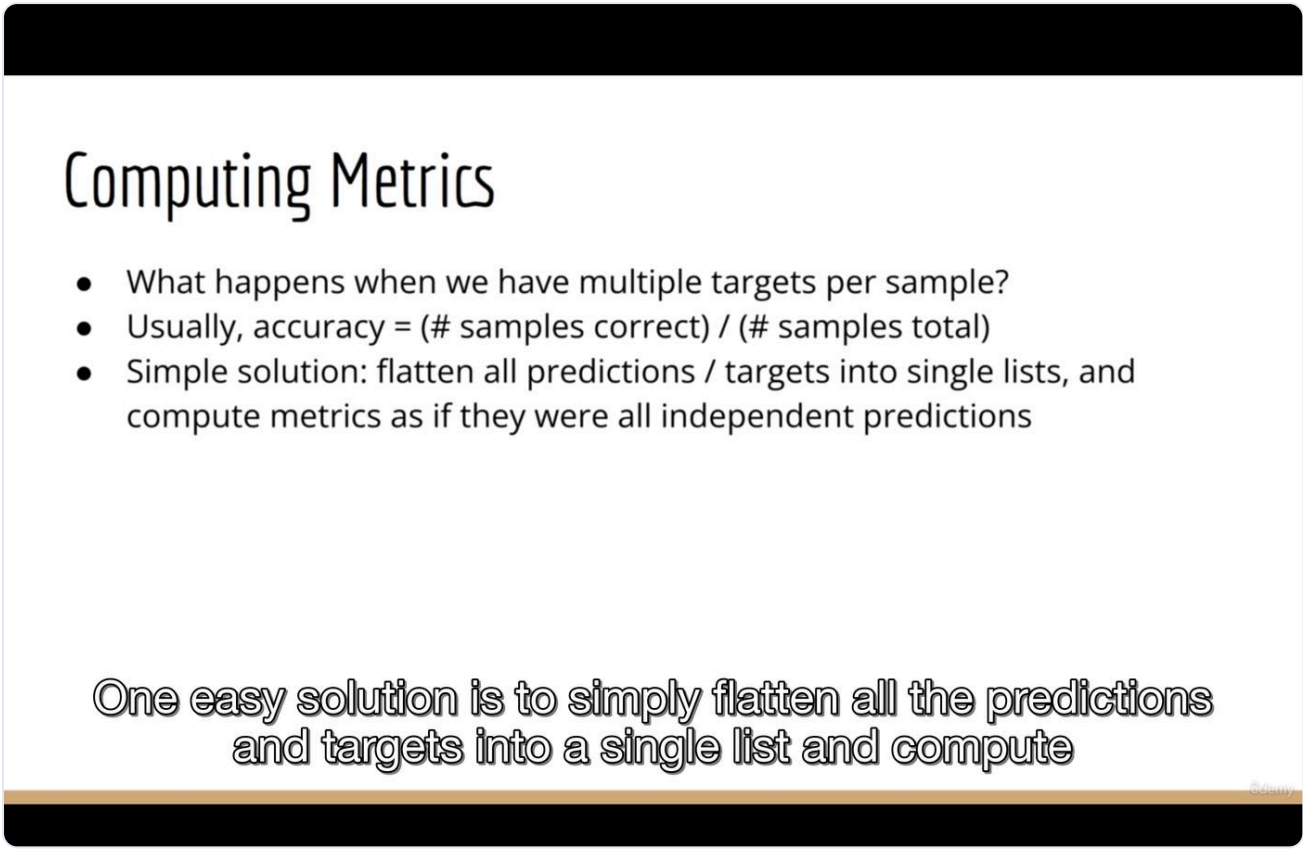

FIGURE 5.37 | Computing Metrics

One easy solution is to simply flatten all the predictions and targets into a single list and compute metrics like you normally would. Now, although you could do such a computation yourself, the standard way in the Hugging Face library is to use another library called seqeval.

FIGURE 5.38

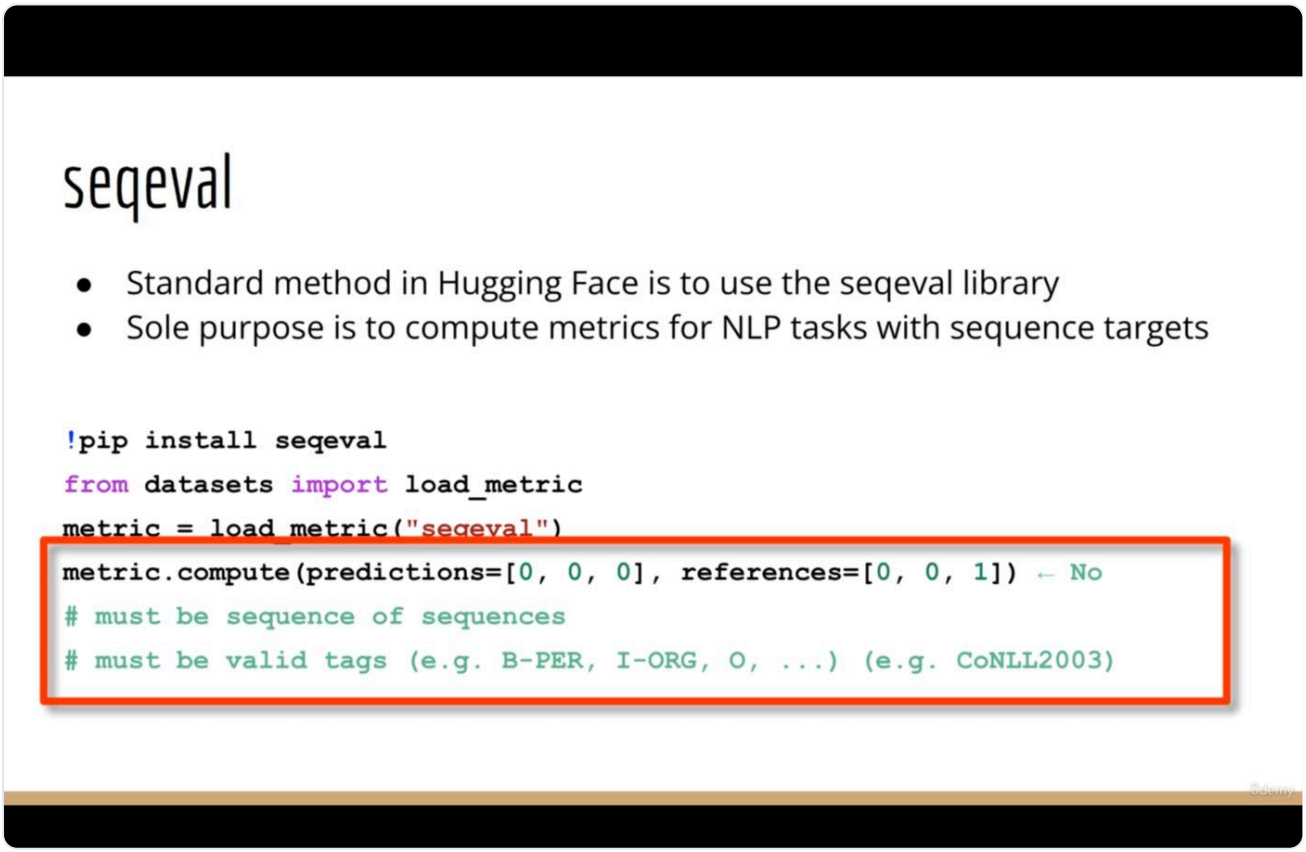

FIGURE 5.39

This is a library whose sole purpose is to compute metrics for NLP tasks where your targets are sequences. The reason we want to use this is because there are a few more details we haven't considered but that are built into this package.

So from a practical standpoint, we don't have to worry about these details and we can just use a standard library for computing metrics. Using this library is very easy. First, we need to install the seqeval package by using pip install as usual. Once we've done that, we can pass in the string seqeval when we call load metric.

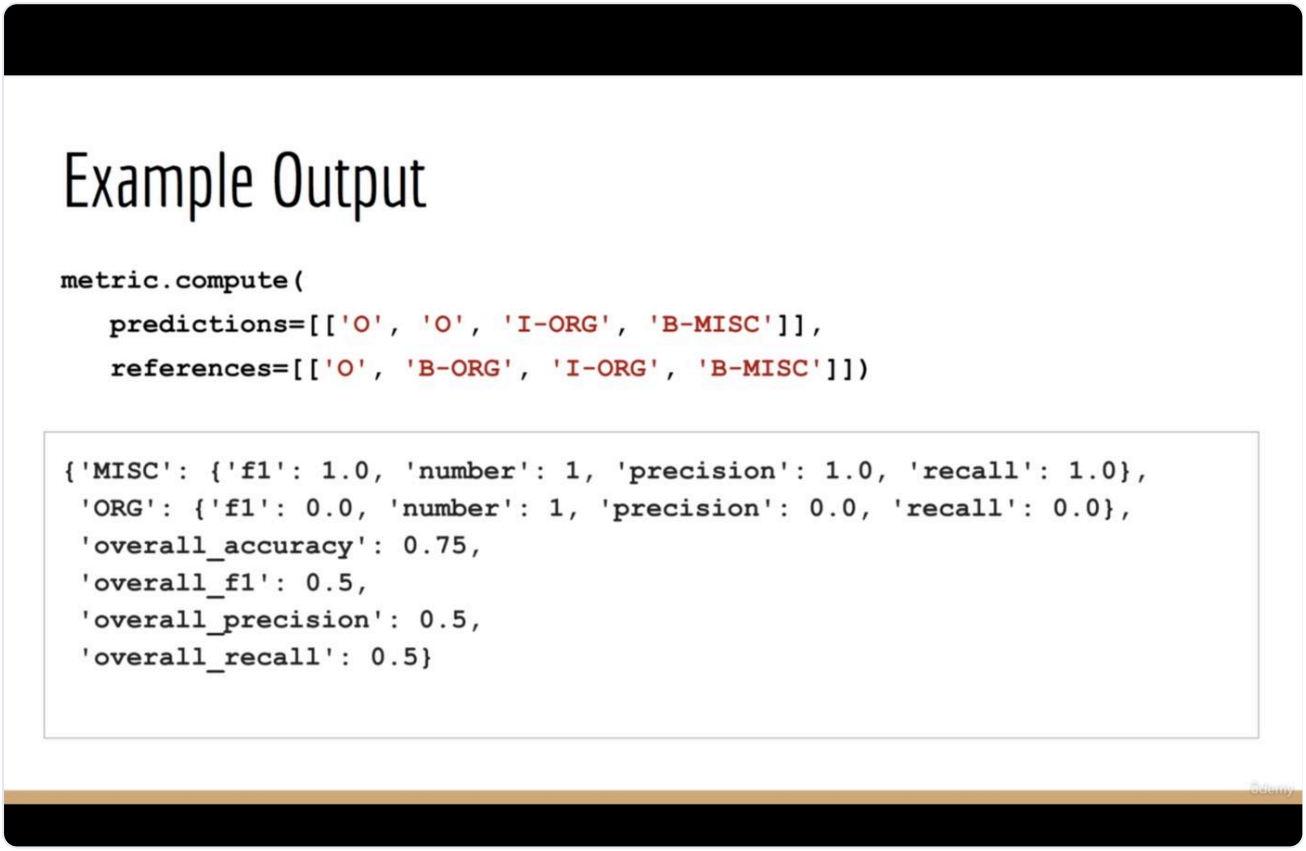

FIGURE 5.40 | Example Output

Once we have a metric object, we can call metric dot compute as before. But one thing that's important to know is that the inputs to compute metric are very specific.

For instance, if I want to test this metric by passing in some dummy data like 000 for the predictions and 001 for the targets, this will not work. There are two requirements that need to be met. Firstly, the targets and predictions should be sequences of sequences like lists of lists. This would make sense because normally we want to compute the metrics for multiple samples at once.

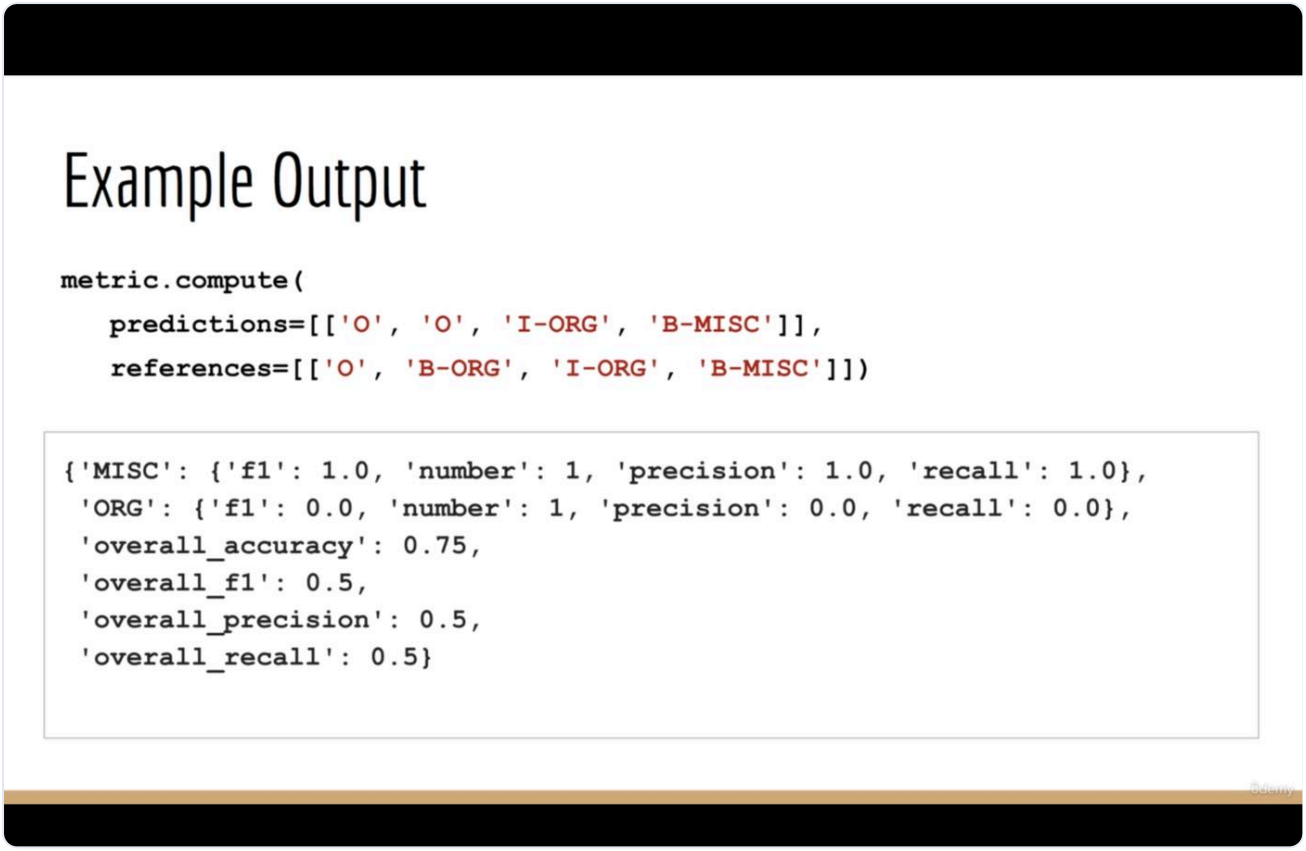

FIGURE 5.41 | Example Output

So we would be passing in a sequence of targets. And each of those targets is itself a sequence.

For instance, a sequence of NER tags. That brings us to our second point, which is that the actual targets and predictions themselves must consist of valid tags represented as strings, like what you would see in the CoNLL-2003 data set. This is because this library was built with the CoNLL data set in mind. And how it works is based on the rules of the IOB format.

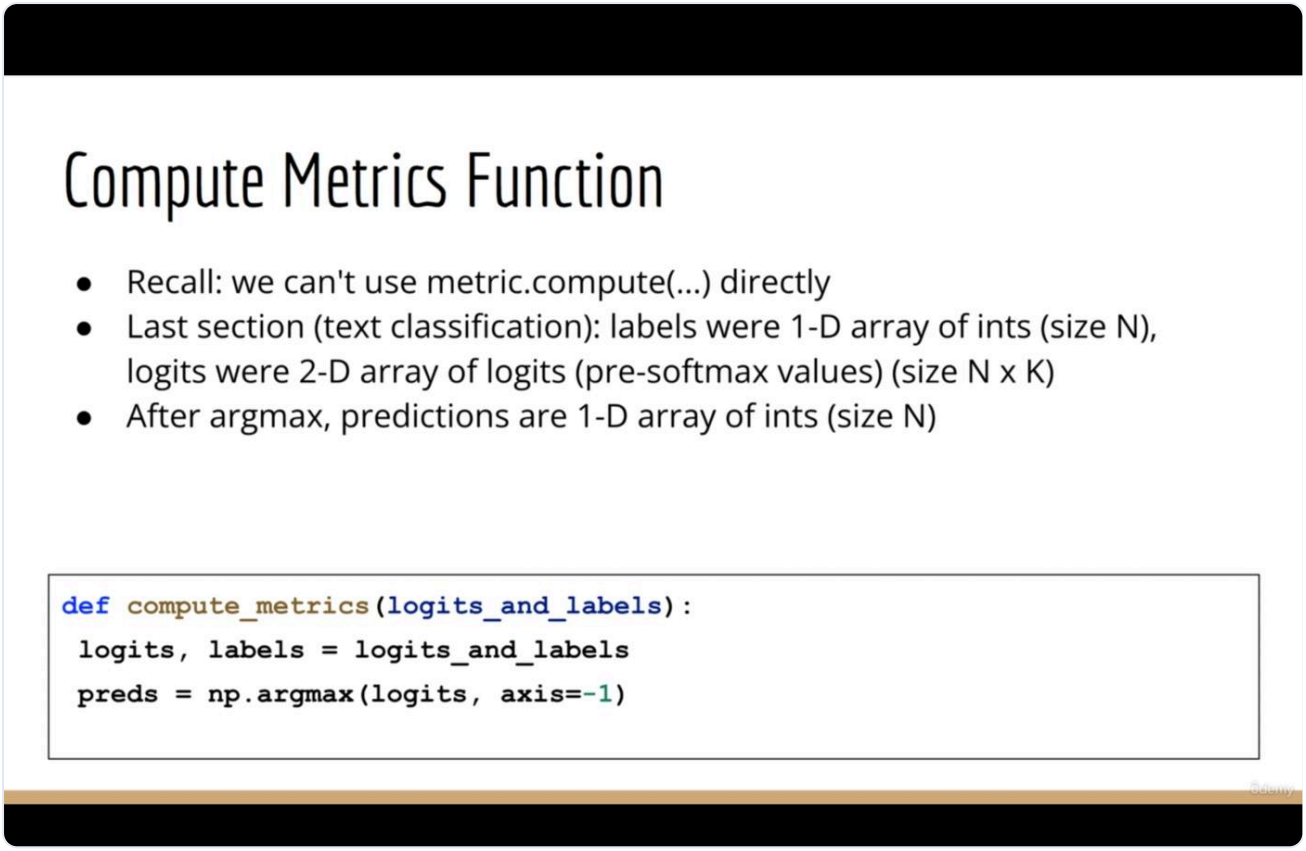

FIGURE 5.42

In other words, if you try to pass in random symbols like ABCD, this would not work because it doesn't follow the IOB format.

Finally, note that this metric will return typical NLP metrics such as accuracy, precision, recall and F1 score. Note that per label metrics are also reported, so if your input contain the tag PER, you'll get the F1 precision and recall just for this tag alone. If your input contains the tag ORG, you'll get another item for this tag as well. You will not get any individual per label metrics for tags which did not appear in the input. Now you recall that once we have a metric object, we can't just pass it into the trainer directly.

Instead, we need to write a function that accepts as input, a tuple of logits and labels, and returns a dictionary containing the metrics of interest.

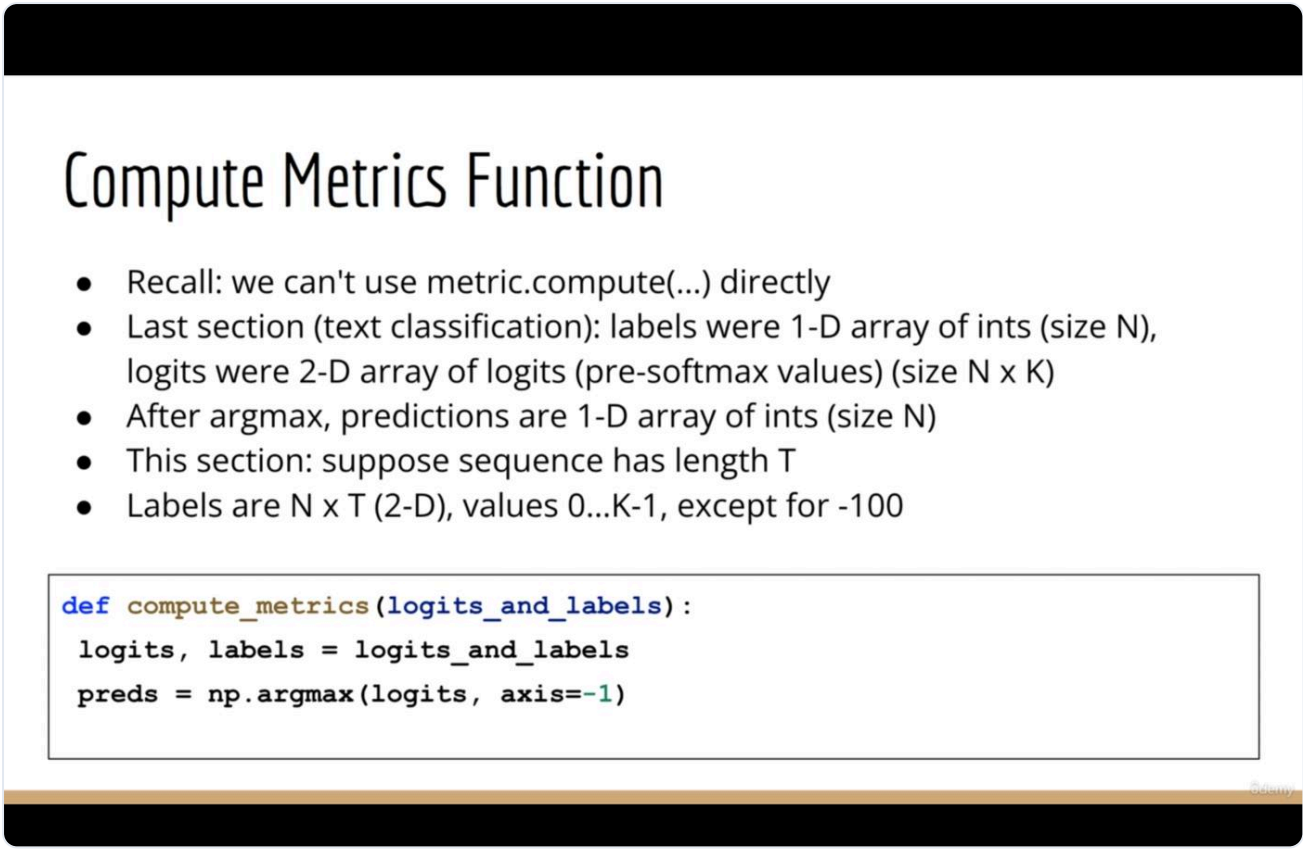

FIGURE 5.43

To that end, we're now going to discuss what such a function will look like. In this section, it's important to realize that the shapes of the logits and labels will not be the same as the previous section. In the previous section, the labels were a one dimensional sequence of integers, while the logits were a two dimensional array of logits.

That is the value before applying the softmax. So for instance, the labels would be an N length vector and the logits would be an N by K matrix. After taking the argmax of the logits, we would get a 1D sequence of predicted labels.

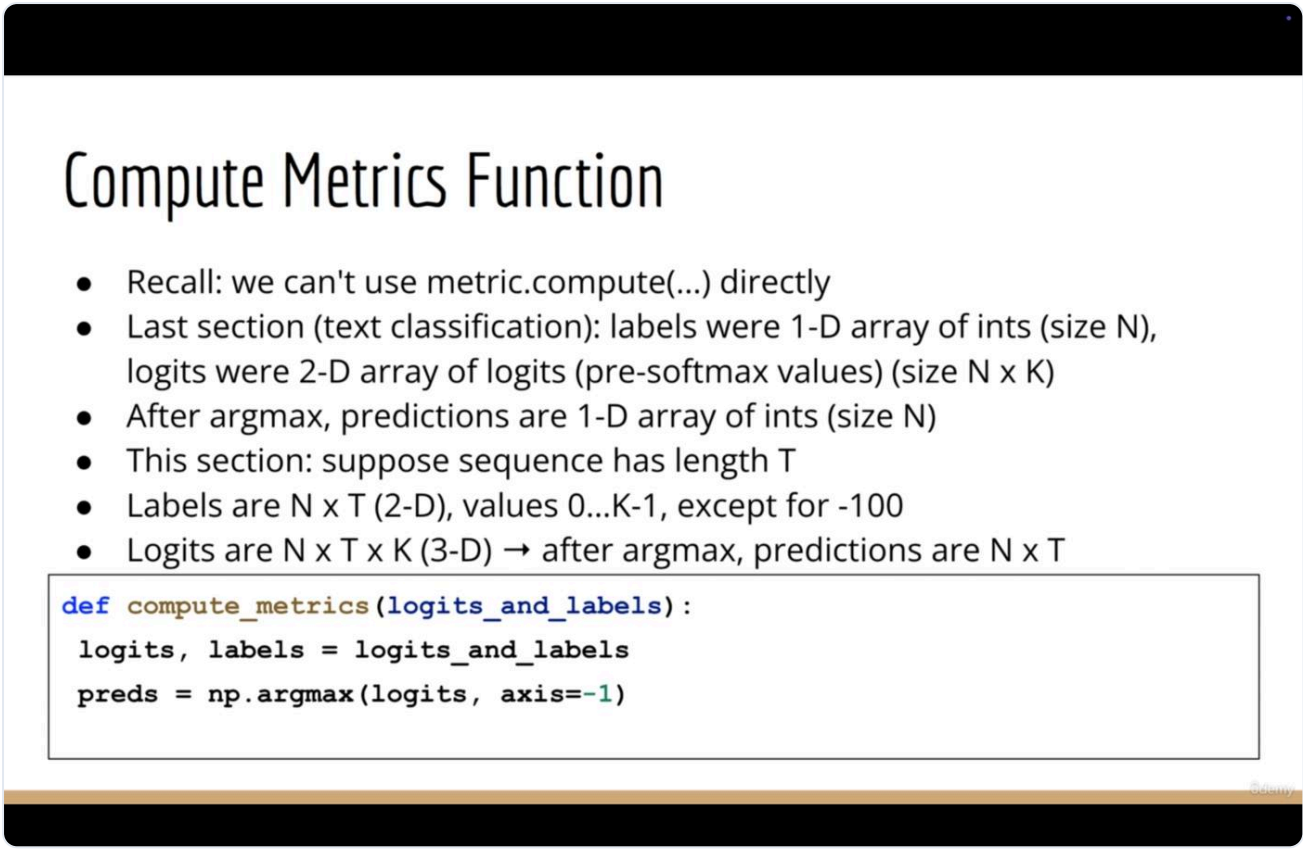

FIGURE 5.44

In this section, we add a dimension to both the logits and the labels in particular. Suppose all sequences in the batch have length t, then the labels would be a 2D array of size n by t, and each elements would contain an integer from zero up to k minus one.

Except if it corresponds to items we want to ignore, in which case it would be -100. The logits will now be a 3D array of size N by T by K. After taking the argmax in the last dimension, we'll get back an N by T array of predicted labels, which will again be integers from zero up to k minus one. Note that from the logits alone we do not know which predictions to ignore.

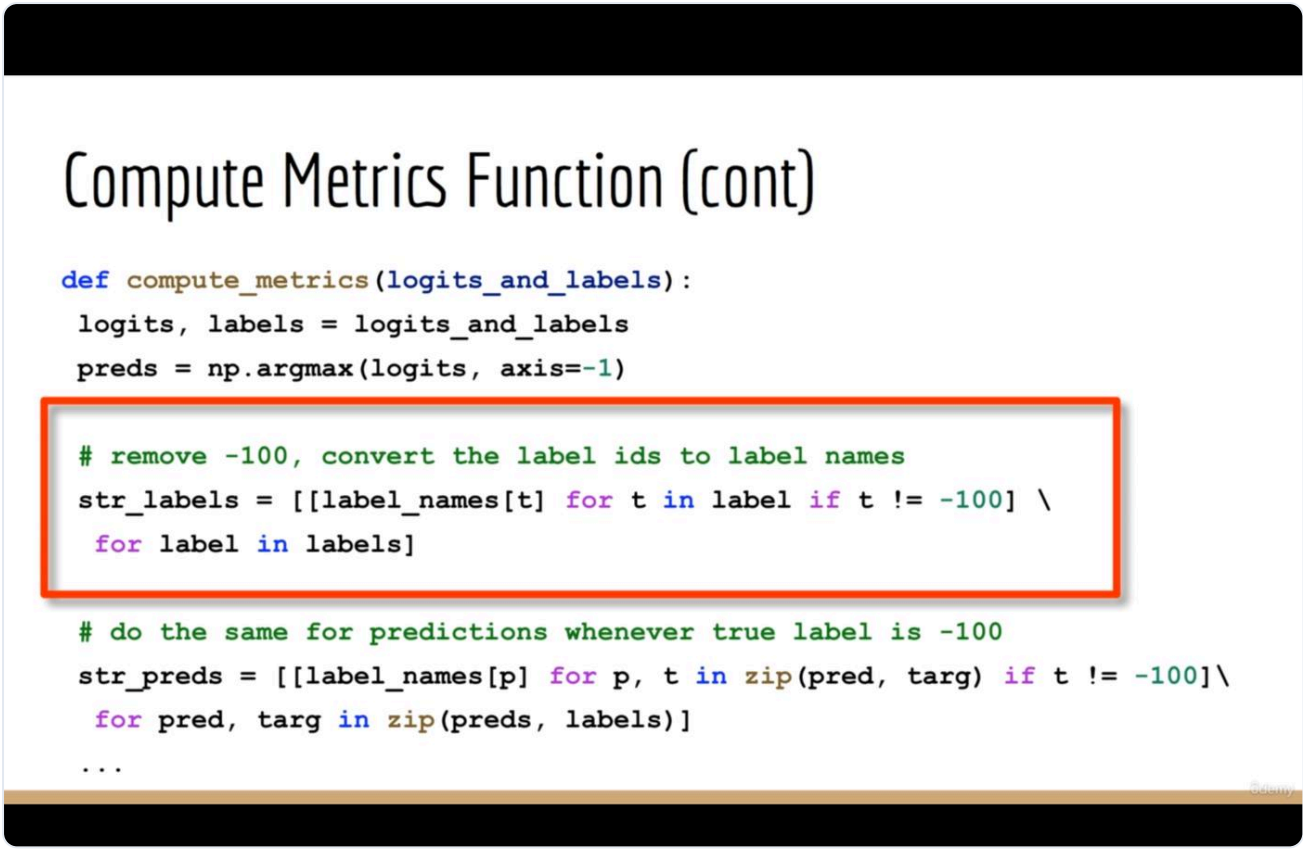

FIGURE 5.45 | Compute Metrics Function (Cont)

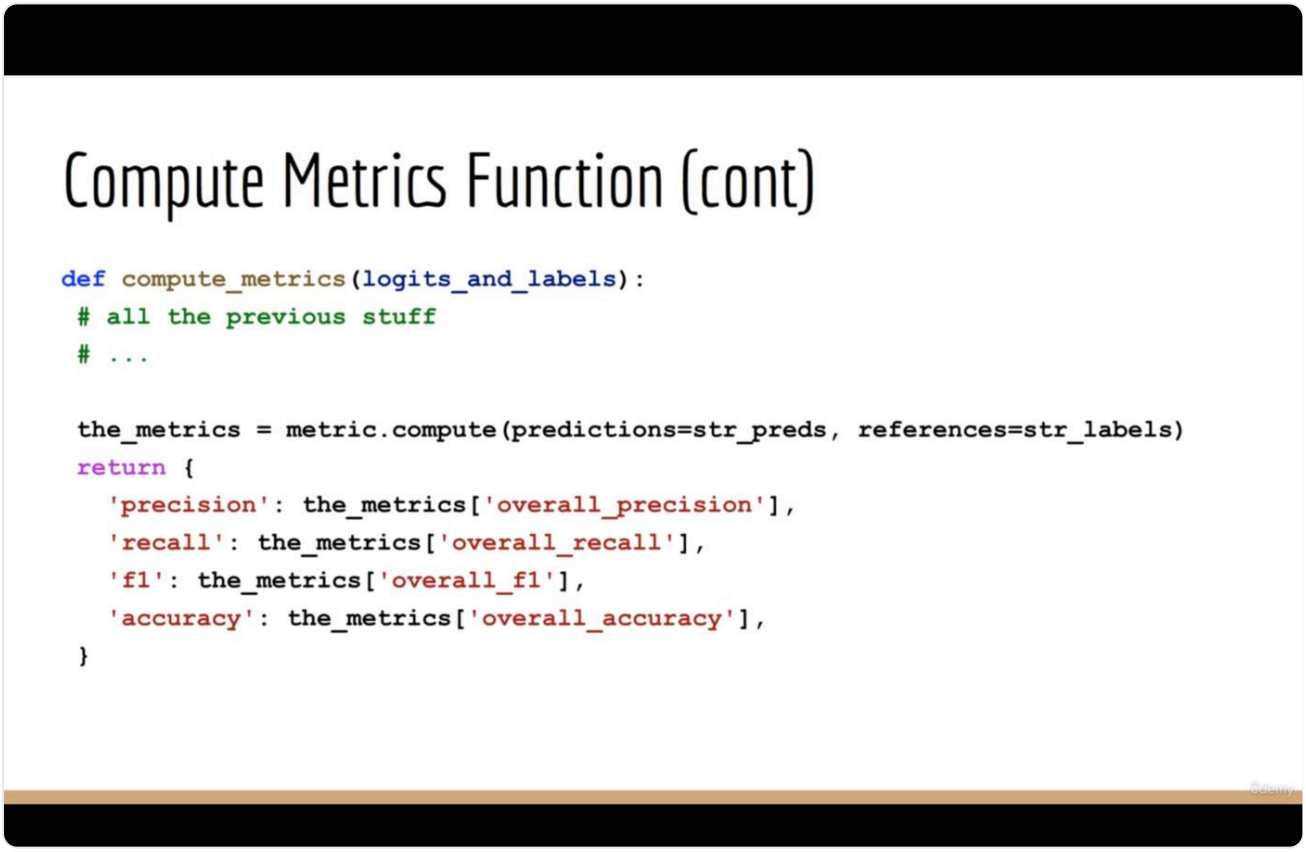

FIGURE 5.46 | Compute Metrics Function (Cont)

We need to see where the corresponding true label is. -100. To that end, let's look at the code which will convert our labels and predictions into the required format.

Remember that our goal is to get a list of lists of true labels and a list of lists of predicted labels. Note that at this point we want to remove padding and other special tokens because we don't want to include them when we compute metrics. So our list of lists will become a jagged data structure, which is okay. So the first line of code here essentially does two things. Firstly, it uses a double list comprehension to loop through each sample and each label within the sequence of labels for each sample.

During this process, it filters out any labels which are set to -100. The second thing it does is it maps the integer label back to the corresponding string label. As you recall, we save this in a variable earlier in the notebook. So this line of code will give us back a jagged list of string labels. The second line of code does something similar, but for the predictions instead of the targets in this loop, we are again using a double list comprehension.

But this time we need to loop through the predictions and the true labels in corresponding order. As you recall, we need to do this because we need to know when the target is -100 because those predictions need to be removed. Again, we map these back to string label names. The final step in our function is to call metric compute passing in the string labels and string predictions we just found. From here, we format the results into a new dictionary which gives the metrics better names and also removes the per label metrics which we don't need.

10 · Metrics (Code)

See the notebook here.

So in this lecture, we will be looking at how to evaluate our model, which is different from the previous notebooks, since our targets are now sequences of class labels. As mentioned, we'll need to first install the library seqeval. The next step is to import the load metric function and then call it passing in the string seqeval. The next step is to test the metric by calling metric.compute. We'll begin by simply trying to pass in a single sequence of label IDs.

Okay. So as expected, it fails. As you recall, this metric assumes that we are working with batches of sequences and therefore we cannot just pass in a single list. We should instead pass in a list of lists. The next step is to try metric compute again, but this time passing in a list of lists. Note that the values are the same as what we tried above. Okay. So this time it seems we get a bunch of warnings. Basically, they're telling us that the labels we passed in are not proper tags.

As you recall, this is because this seqeval library was built specifically for CoNLL tasks. As such, they won't work with just any set of labels. Moreover, the labels must be represented as strings and not integers. However, do note that there is a result that gets printed out. The next step is to test metric.compute again, but this time we're going to pass in a sequence of string labels instead of integer labels. Okay. So as you can see, this again gives us the same warning, because even with string labels, the library knows that they are not the proper strings corresponding to tags in the CoNLL

data sets. The next step is to test metric.compute one last time, but this time we're going to pass in real labels from the set of any tags. Okay. So as expected, it works. But note that in the results we not only get overall metrics for accuracy, F1, precision and recall, but we also get metrics on a

per entity basis. So we see precision recall in F1 for both ORG and MISC, but we do not have any metric for the O tag since those are not entities.

Now one interesting thing to consider is the precision and recall for the org tag. The precision, recall and F1 are zero, even though out of the two org tags we do get one of them correct. This may seem odd, but remember that this library is doing special computations specifically based on the IOB format. The next step is to write our compute metrics function. We'll begin by importing numpy since will be using it below. The next step is to define the compute metrics function as we normally do, which takes in a tuple of logits and labels as input inside the function.

We first split up the logits and labels. We then apply the argmax on the logits to get the predicted labels. Note that at this point, both the labels and the predictions are arrays of shape n by t where n is the batch size and t is the sequence length. The next step is to remove the -100 values from the labels, since we don't want them to factor into our metrics. Furthermore, they don't have any corresponding label names, which is another thing we do in this step.

So all the labels will be converted into their string representation. The next step is to do the same thing for the predictions. Note that for the predictions we need to use the zip function so that we can loop through the predictions and targets in corresponding order. This is because the targets will be -100. For any predictions we want to ignore. Finally we call metric compute, which will return a dictionary of metrics. We then return the overall metrics in the new dictionary while also giving more concise names.

So at this point we've done all the groundwork to load in our data, set, tokenize the inputs, realign the targets and compute metrics. In the next few lectures, we'll create our model and then pass everything into the trainer where we can then fine tune the model on our data set.

11 · Model and Trainer (Code Preparation)

So in this lecture, we will be moving on to the final step of our notebook, which is to load up a model from a checkpoint and fine tune it on our dataset relative to the previous parts of this section. This will be quite brief as before will be instantiating an auto model, but in the previous section we use the class auto model for sequence classification, which is not appropriate for the current task. In this section we'll be using the class auto model for token classification, which is built specifically for our current task. As before, we'll pass in a checkpoint such as BERT or DistilBERT.

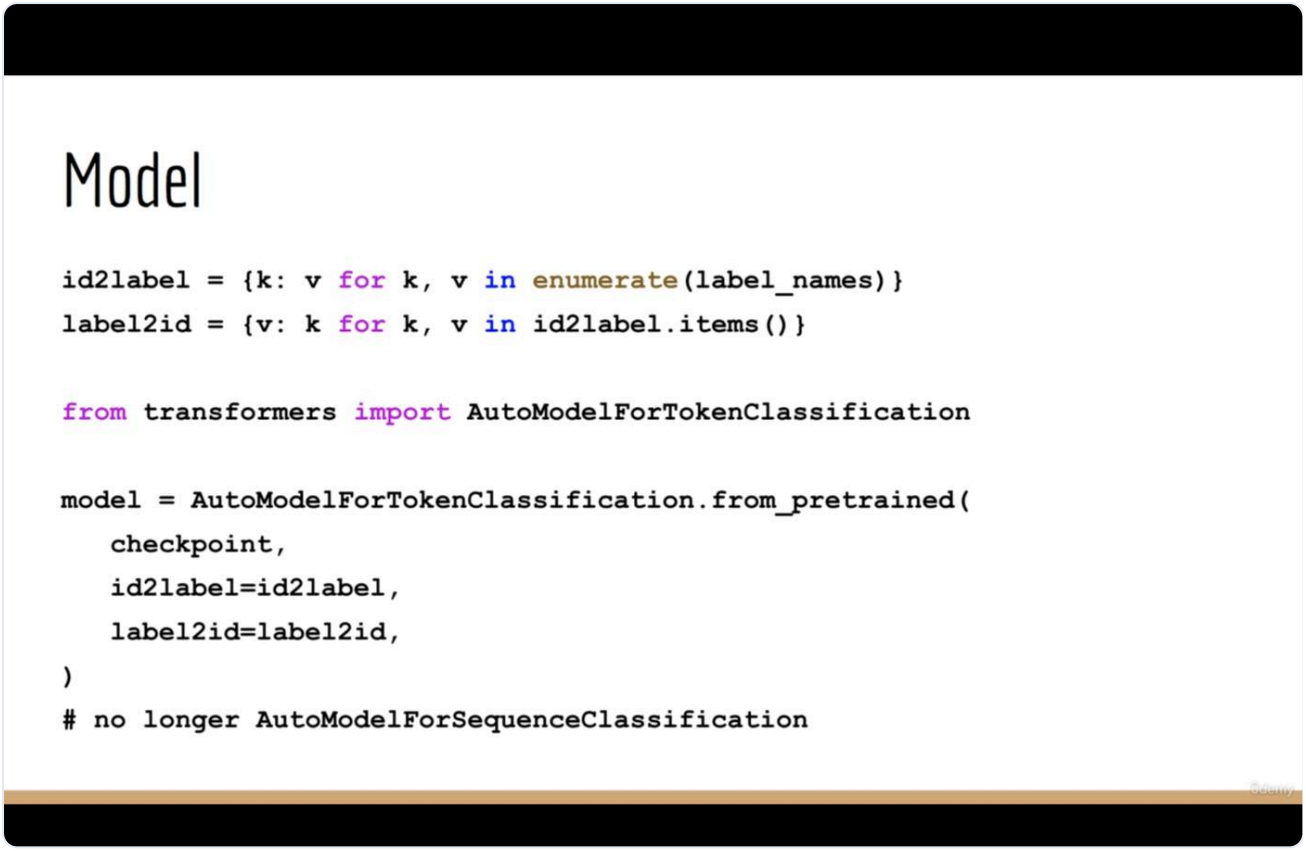

FIGURE 5.47

I've also shown you an alternative way of specifying the labels, since, as you recall, if you don't do this, they'll just come out to be label zero, label one and so forth. Specifically, we're going to pass in the arguments id2label and label2id. Note that these can be built using a list of label names which we saved earlier in the notebook. So this is an alternative method of specifying the label names instead of using a whole auto config object or modifying the config file directly.

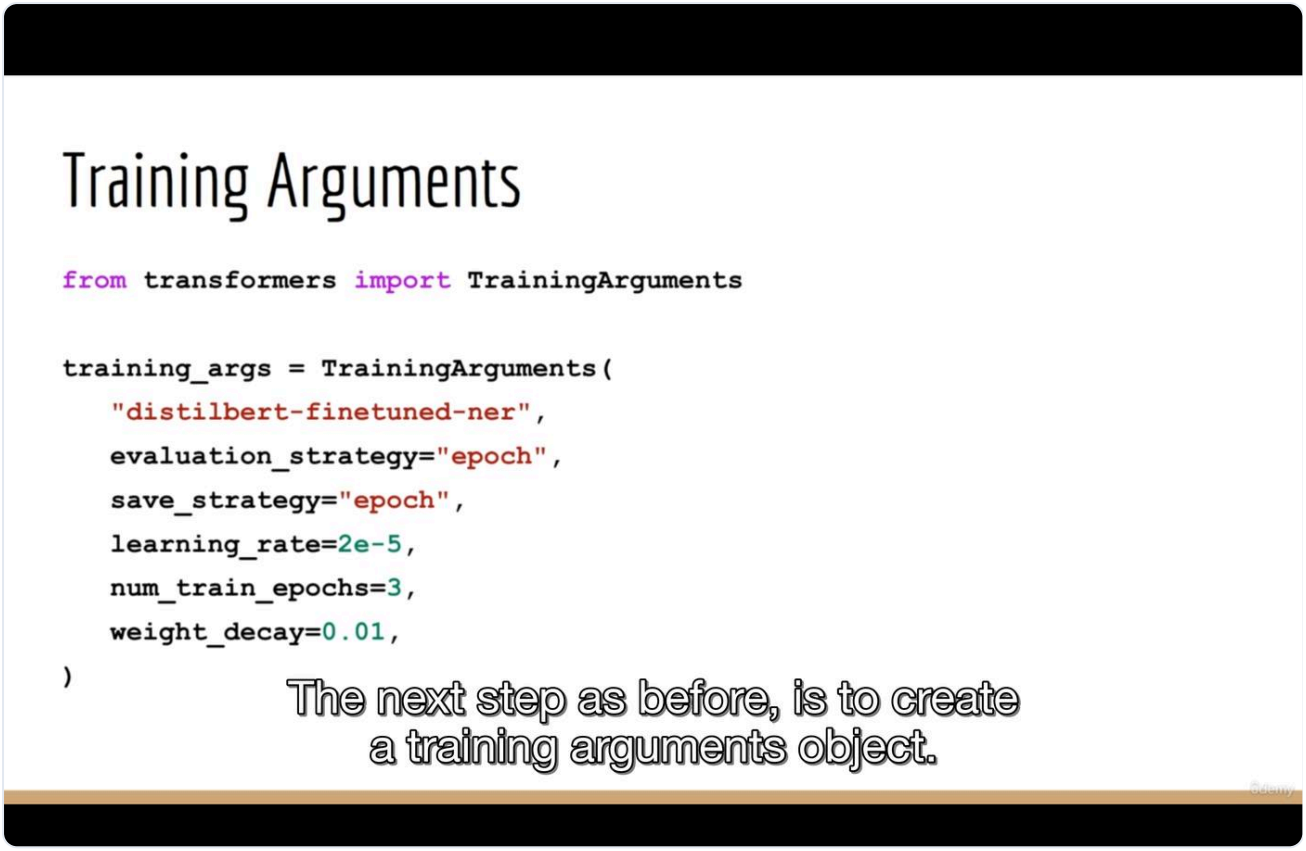

FIGURE 5.48 | Training Arguments

The next step as before, is to create a training arguments object. This works exactly the same as the previous section, so nothing more needs to be discussed.

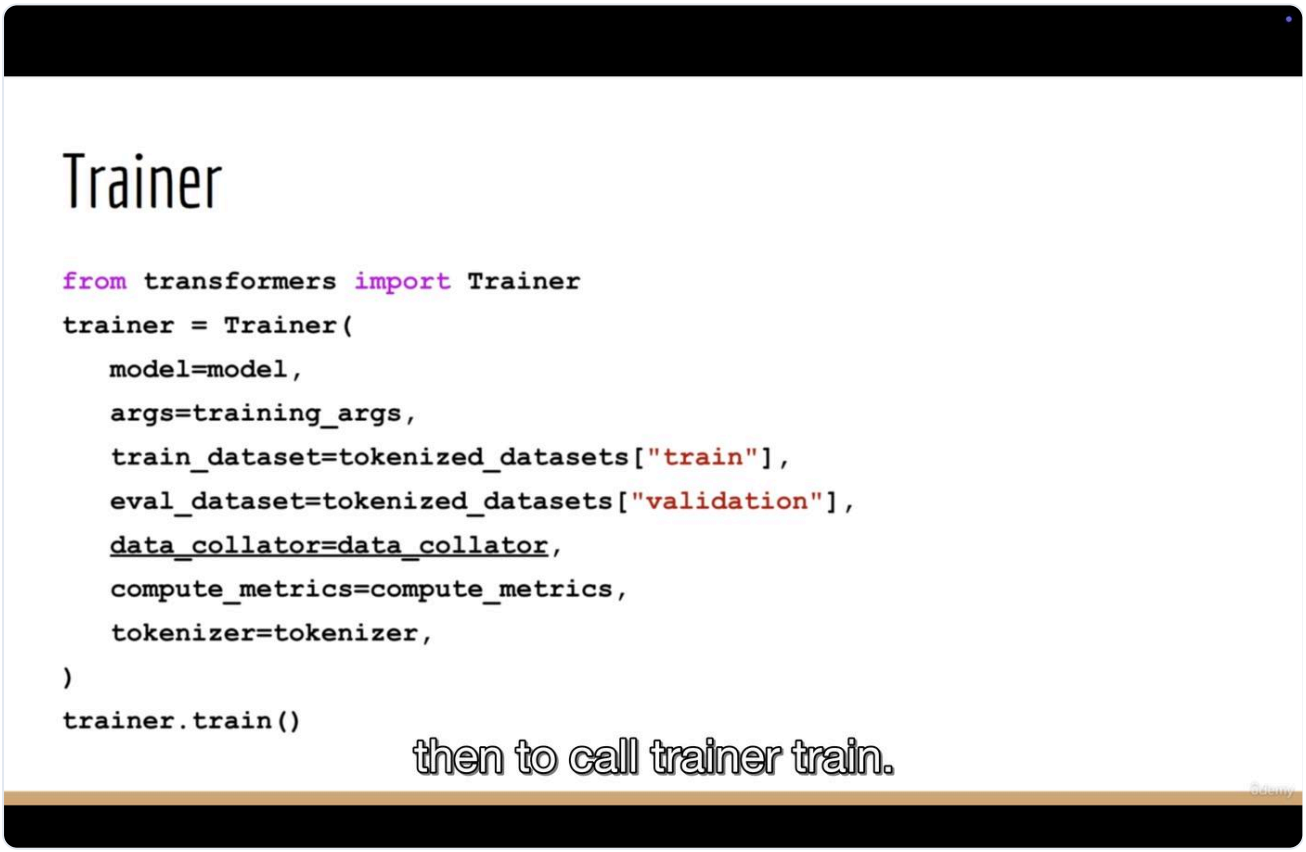

FIGURE 5.49

Once we have our training arguments, we have all the pieces we need to create a trainer object and then to call trainer.train. Note that the main difference between this section and the previous section is we are explicitly specifying the data collator. Okay. So once we've trained our model, recall that we can call trainer.save_model to save the model to a local path.

After we've saved the model, we can load it up using a pipeline. Specifically, the token classification task, as you recall, from the Beginners Corner section. We also specify the aggregation strategy as simple so that the entities returned by the model are presented in a convenient format as before. We can use the pipeline simply by passing in a string or list of strings.

12 · Model and Trainer (Code)

See the notebook here.

So in this lecture, we'll be continuing our notebook in order to load up the model. Fine tune it, save it, and then test it out as a pipeline. So we'll begin by creating the id2label and label2id dictionaries. As you recall, our label names variable is a list which already stores a mapping from id2label. The next step is to import the class auto model for token classification, which is the type of model we need for the current task. We then call from_pretrained passing in our checkpoint to get a pre-trained model.

Note that we also pass in id2label and label2id so that our model uses our specified label names. The next step is to import the training arguments class and then to create an object of this type. Since the arguments are the same as the previous section, there's not much to discuss. Note that we've also used weight decay, which you may have seen if you've studied deep learning with me in the past. The next step is to import the trainer class and to create an object of this type.

As you know, everything we pass in here are objects we created earlier in this notebook. Furthermore, everything is exactly the same as it was in the previous section except for the data collator since currently the Hugging Face library does not use the correct version by default. We also called trainer.train at this point to begin the training process. Okay. So now that we've trained our model, we can save it to disk. The next step is to import pipeline and to create a pipeline from the model we just saved.

For this type of task, the correct string to pass in is token classification and we also set aggregation strategy to simple. The next step is to test our pipeline on a simple sentence.

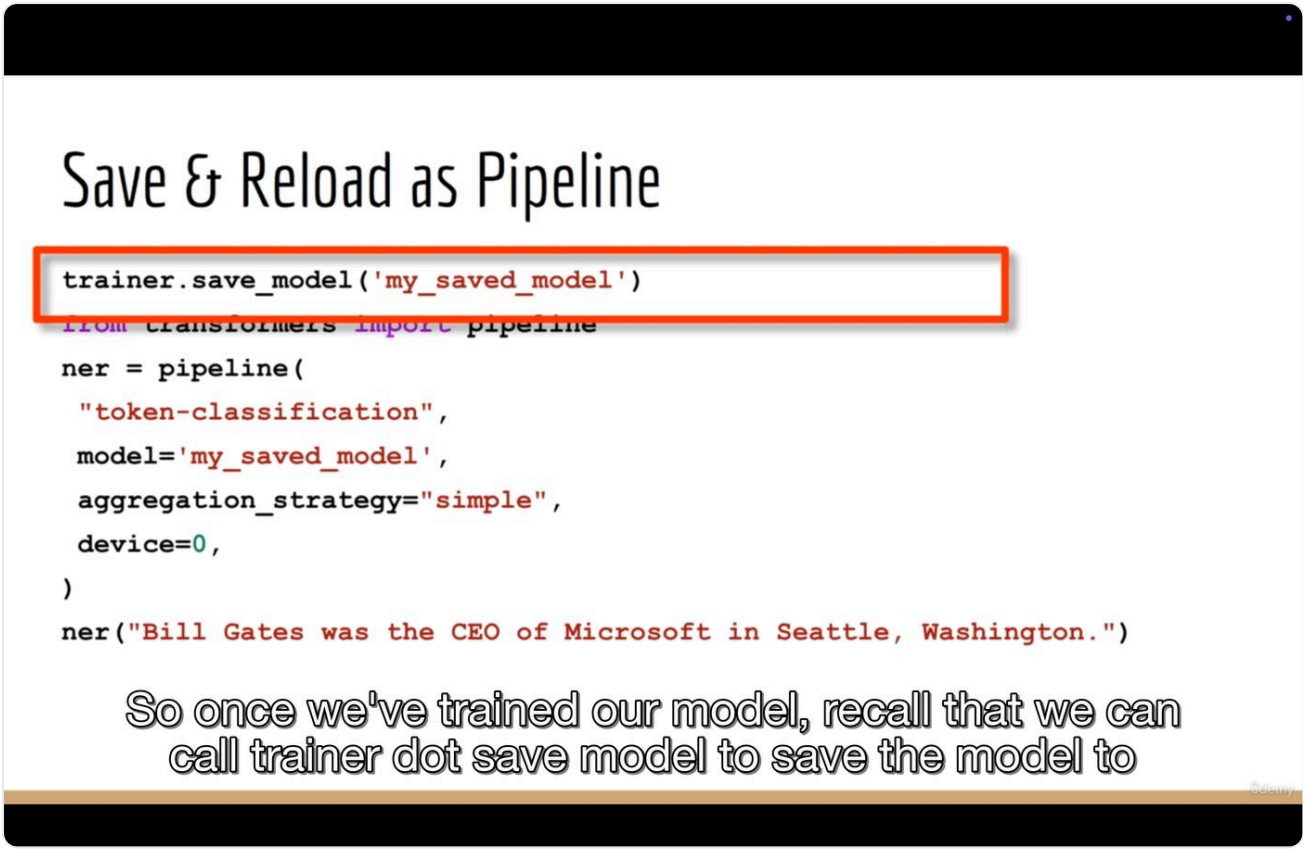

FIGURE 5.50 | Save & Reload as Pipeline

Bill Gates was the CEO of Microsoft in Seattle, Washington. We know that Bill Gates is a person. Microsoft is a company. And Seattle and Washington are both locations. Okay. So as you can see, our model correctly finds that Bill Gates is a person entity, Microsoft is an org entity, and Seattle and Washington are both location entities.

13 · POS Tagging & Custom Datasets (Exercise Prompt)

So in this lecture, we are going to discuss a practical question, which will also become your next exercise for this course.

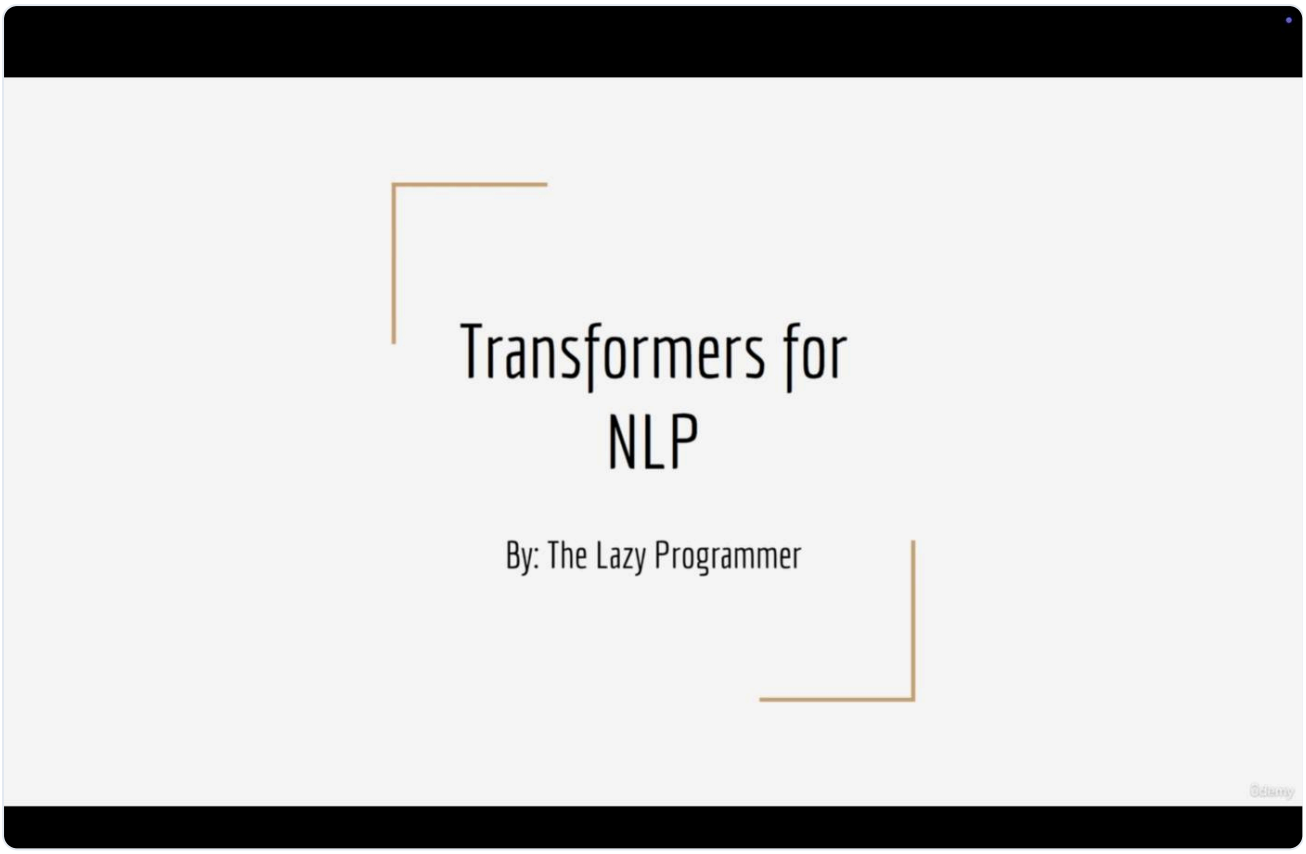

FIGURE 5.51 | Transformers for

In particular, the question is how can we do token classification with a custom data set?

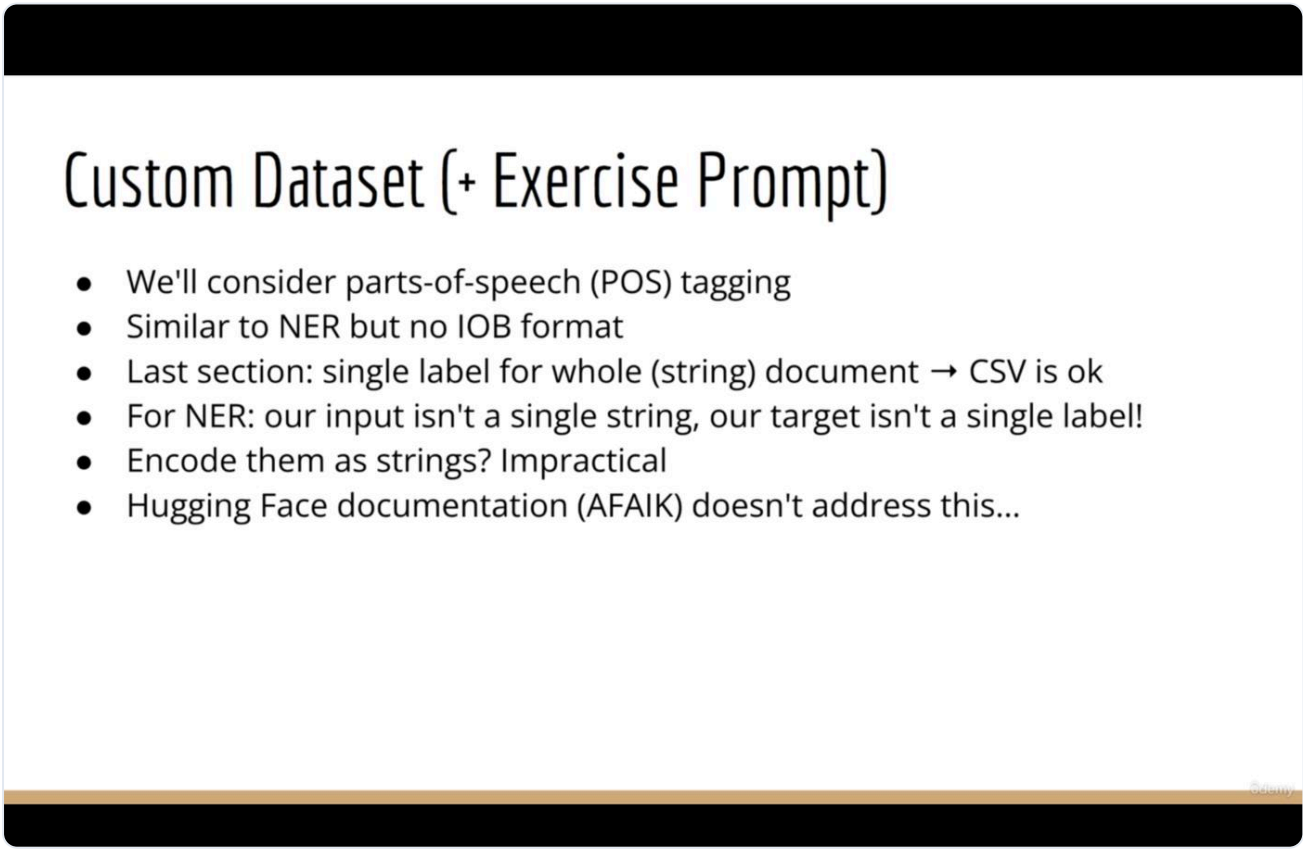

FIGURE 5.52 | Custom Dataset (+ Exercise Prompt)

For the purpose of this lecture, we'll consider the task of parts of speech tagging, which is similar to NER, but simpler since it does not require the IOB format. The main issue is this in the previous section where we looked at predicting a single label for a whole document we used CSVs when we wanted to provide our own custom dataset.

But let's consider how our data was formatted for NER. In this case, our input document wasn't just a single string, and our target wasn't just a single label. In theory, we could find some way to encode both of these strings and then put them into a CSV, but that doesn't seem practical. In fact, for this type of task, the Hugging Face documentation, as far as I know, does not address this issue. However, there is a solution which is to use a different format other than CSV.

Specifically, JSON Lines.

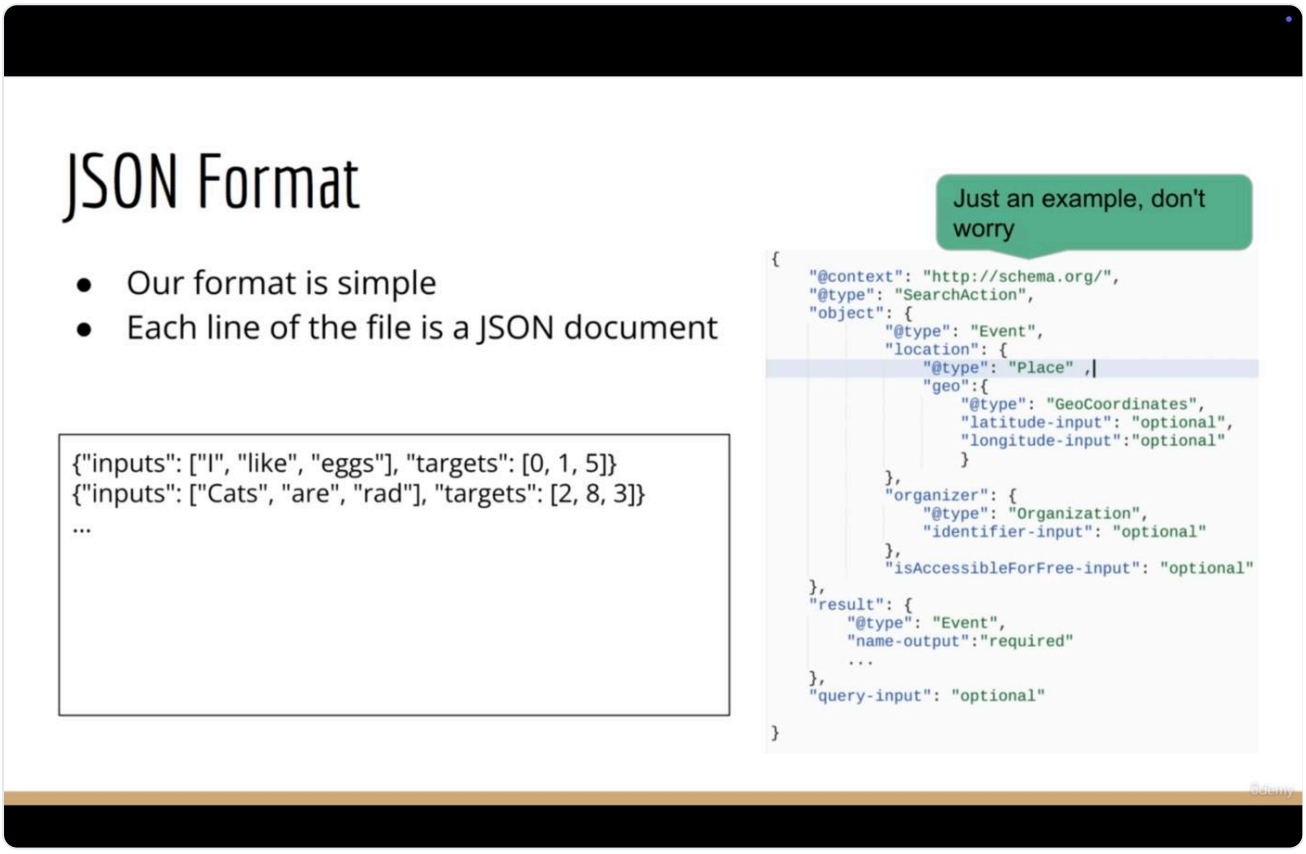

FIGURE 5.53

In particular, we'll represent our data in a text file. Each line is a separate JSON. Each JSON object represents a single sample. This is pretty flexible and it works for our use case. Since as you may know, JSON keys must be strings, but the values can be many things such as integers, strings, booleans, lists of any of the above and even another JSON objects. In our case, the format will be pretty intuitive. Each JSON will have two keys, inputs and targets.

The inputs will be a list of words, and the targets will be the corresponding list of tags. Once you have a JSON file in the format I just described, it's very easy to load into Hugging Face as we saw before.

FIGURE 5.54 | Load Dataset

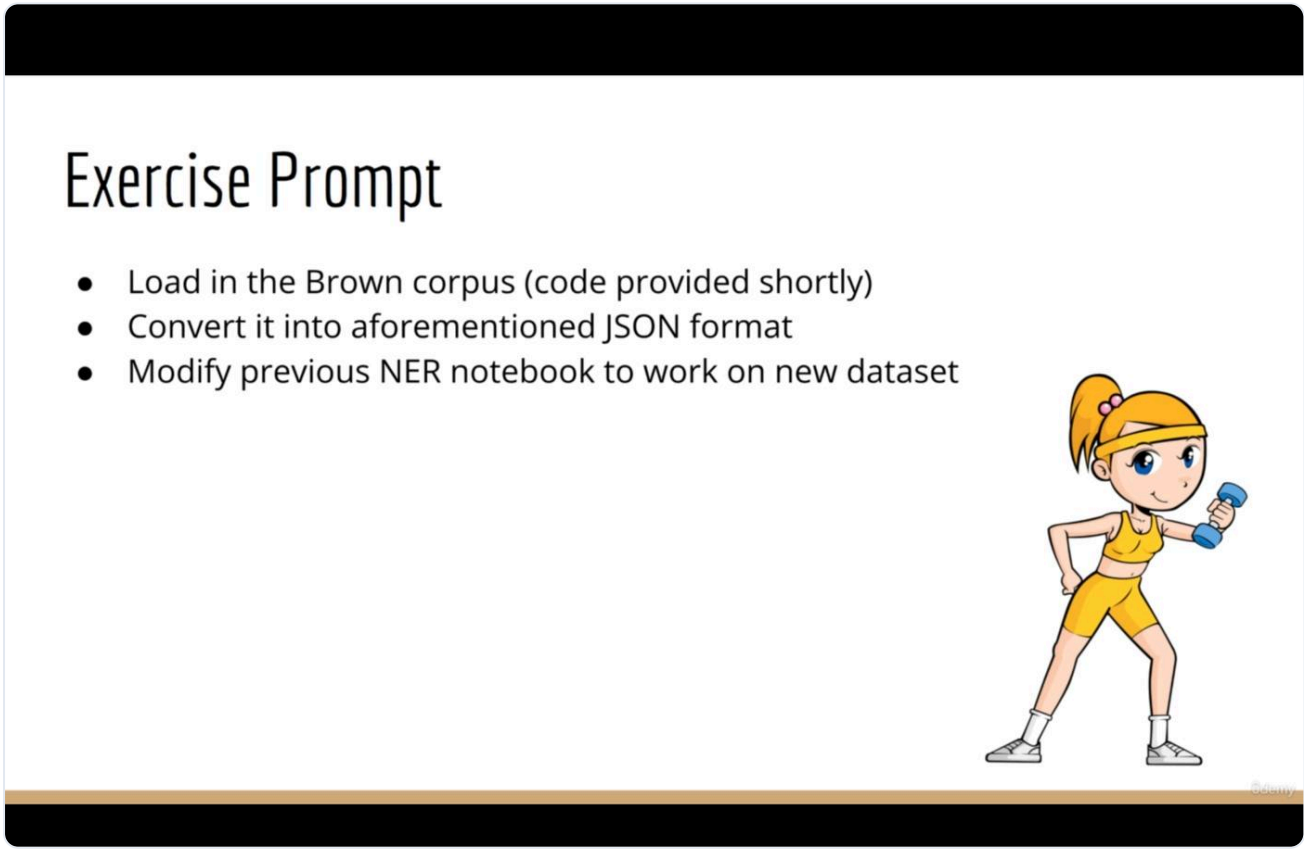

FIGURE 5.55 | Exercise Prompt

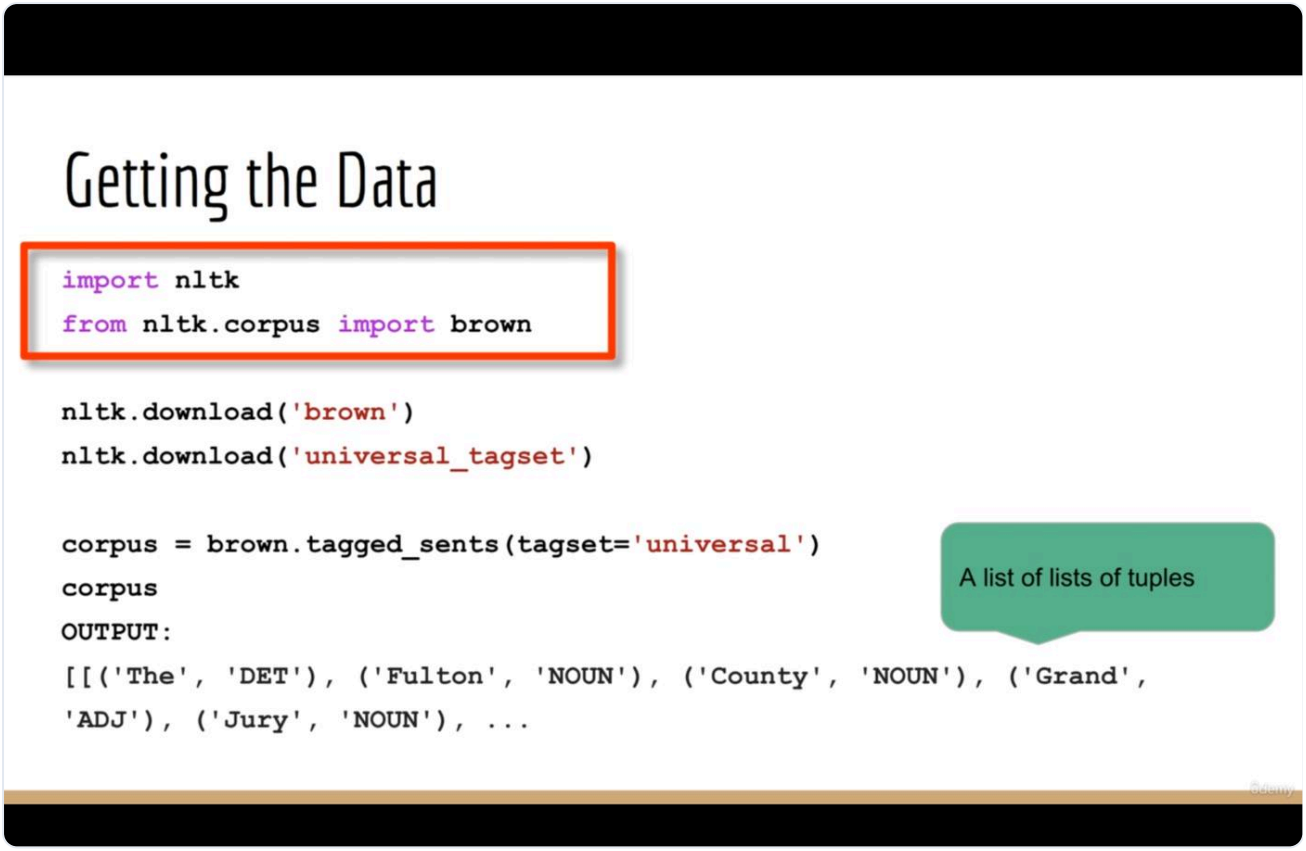

FIGURE 5.56 | Getting the Data

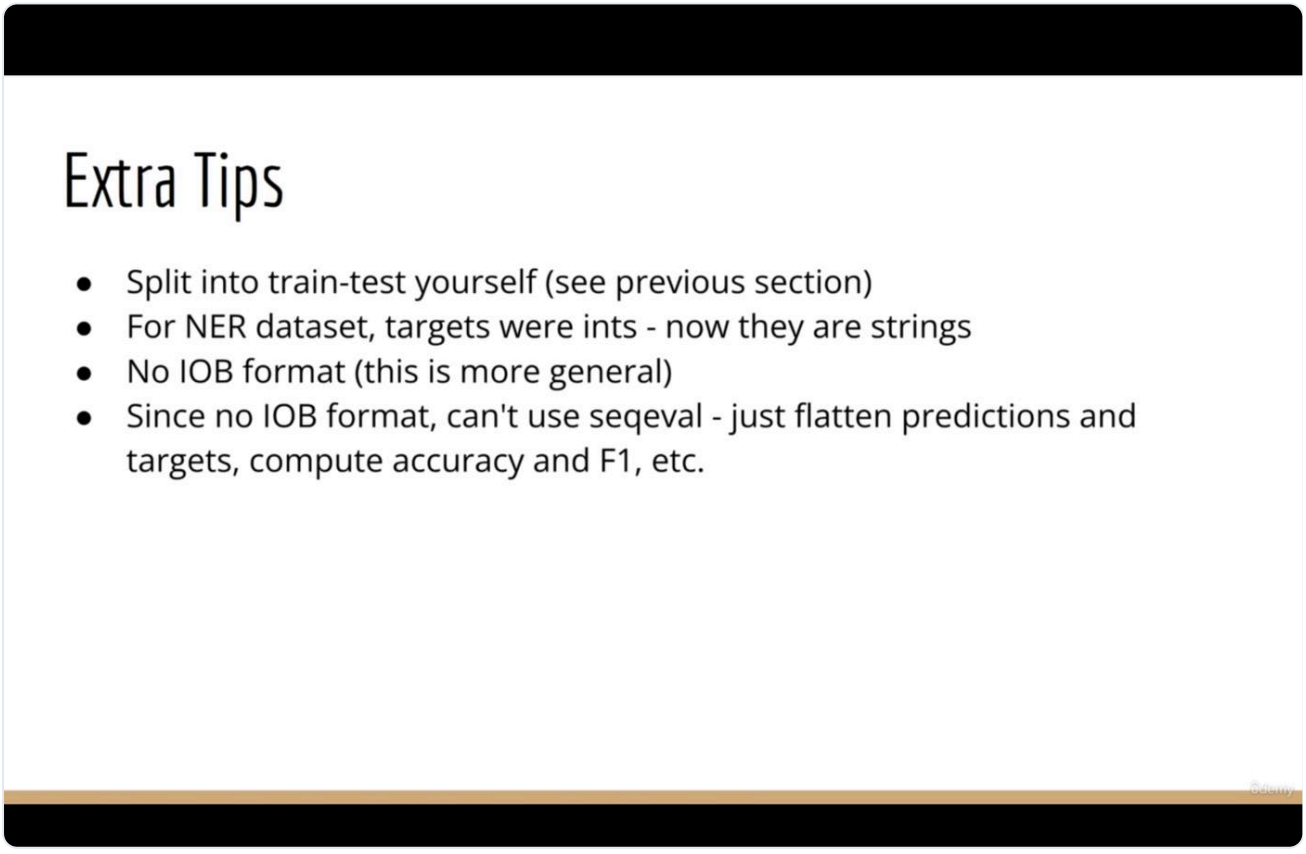

FIGURE 5.57 | Extra Tips

We'll continue to use the load_dataset function, except the first argument will now be JSON instead of CSV. We continue to pass in the data files using the same format as we saw before. So the next exercise, at least conceptually, is very simple.

You're going to load in the Brown Corpus, which I'll show you how to do shortly. Then you're going to convert this dataset into a JSON file like I just described. Then you're going to take our previous NER notebook and modify it such that it will work for our new data sets. So hopefully you agree that your job is pretty clear cut. Now let's discuss some useful details. Firstly, how do you get the data set? The data set will be using comes from NLTK and it's called the Brown Corpus.

You want to import NLTK as well as the brown module from nltk.corpus. You'll need to download a few files using NLTK. Specifically the Brown Corpus itself, as well as the tag set for this task. Finally you can obtain the data set by calling brown.tagged_sents passing in the argument tagset equals universal. This will give you a list of data in pretty much the same format as CoNLL. Specifically, each element of this big list is another list. Each of these inner lists represents a sentence.

Each of these sentences is represented as a list of tuples in each tuple. We have two items the word and the corresponding POS tag. Of course, there will be some processing you need to do in order to convert it into a JSON like we just discussed. That is part of your exercise. Here are some other tips to consider. Firstly, since the data set isn't split into train and test, you should do that yourself. There's an example of that in the previous section of the course.

Secondly, you'll recall that in the NER data set, the targets were already represented as integers when we loaded in the data. But since this dataset isn't built into Hugging Face, that won't be the case. And this example, the targets will be the actual POS tags represented as strings. So you'll have to modify the code as appropriate. The third tip to consider is that because there is no IOB format. You can delete any code related to this. This also makes the task much more general as it can represent any token classification task.

Whereas IOB is quite specific. Fourth tip because the targets are not in IOB format, we cannot use the seqeval metric which requires it. Instead, a simple solution is to take the jagged arrays of targets and predictions and flatten them into single lists. Once you have those lists, compute the accuracy and F1 as you normally would. Make sure to test your compute metrics function to ensure it behaves as you expect.

14 · POS Tagging & Custom Datasets (Solution)

See the notebook here.

So in this lecture we will be looking at the solution to the previous exercise, which was to implement parts of speech tagging using a custom dataset. We'll begin by installing transformers and data sets, as we normally do. The next step is to import NLTK as well as the Brown module. The next step is to download the necessary data we need for this data set. The next step is to load in our corpus using brown.tagged_sents, specifying that we want to use the universal tagset.

Okay. So note that our dataset is currently in the form of a list of lists of tuples. The outside list contains a bunch of sentences. Each sentence is represented as a list of tuples, and each tuple contains a word with a corresponding tag. Importantly, we noticed that the data set is not in the form. We need to represent it as a Hugging Face dataset. To take a step towards that, we are going to begin by separating the inputs and targets. So the inputs will become a list of lists of words and the target will become a list of lists of tags.

We'll begin by initializing an empty list to store the inputs and targets. Next, we enter a loop through the corpus. Each element of this list is called sentence tag pairs for obvious reasons. Inside the loop we initialize an empty list called tokens and an empty list called targets. These are where we will store the words and tags respectively. The next step is to loop through the sentence itself. Since each element is a tuple containing the word in the tag, we can say for token comma tag in a sentence tag pairs.

Inside the inner loop, we append the token in the tag to their respective lists. Note that there is a more efficient way to do this, but this is, in my opinion, the more intuitive. Once we've completed the inner loop, we then append the list of words and list of targets to our list of inputs and targets. So the next step is to save our data into JSON formats so that we can load it in as a Hugging Face dataset in the next step. In the real world, you would write your own code to build this JSON.

As you can see, we simply loop through the inputs and targets in corresponding order. And for each of these pairs we create a dictionary called J. We then convert J into a JSON string and then we write it out to a file called Data dot JSON. Now again, we could have done this much more efficiently by combining this with the previous step, but I've decided to leave it like this so you understand the thought process. The next step is to import the load_dataset function.

The next step is to call load_dataset specifying the type as JSON and the data file as data JSON. The next step is to print out what we just got back. So as expected, we get a DatasetDict containing only a train set. As you can see, our train set has two columns which are inputs and targets. And this corresponds to how we built our JSON file above. Also make note that we have about 60,000 samples. Now for the purpose of this lecture, 60,000 samples is a bit too much, and it's simply going to make the notebook take longer without any benefit.

So the next step is to take a subset of our data sets. In order to do this. I'll begin by shuffling the data set and then selecting the first 20,000 samples. In effect, we're selecting 20,000 random samples of the dataset. We'll call the results small. So as you can see, we get back another dataset with only 20,000 samples. The next step is to do a train test split. The next step is to check out the first sample of our dataset to make sure it looks like what we expect.

So as you can see, we get a dictionary containing both the inputs and targets which are lists. The input is a list of words and punctuation, while the target is a list of tags. Note again that the tags are strings and not integers. So you would have had to account for that if you copied the previous notebook. The next step is to check out the features attribute of our dataset. So as you can see, both inputs and targets are sequences, each with a value type of string.

The next step is to determine the full set of targets so that we can map them to integers. We'll do this by using a set data structure. Since all elements of a set must be unique. So what we are doing here is looping through each target sequence in our data set and adding everything we see to the same set. The next step is to create our id2label and label2id variables. To do this, we'll begin by converting our target set into a target list, which is better since it has a specific ordering.

From this we create id2label and label2id as we normally do. The next step is to create our AutoTokenizer. Note that for this notebook I've again chosen DistilBERT. But feel free to change this to BERT and compare the results. The next step is to test our tokenizer on the first sample of our dataset. As you recall, we have to specify is split into words as true. Okay. So at this point, the results shouldn't be surprising. The next step is to remind you that the output of the tokenizer is not a dictionary, although it does look like one.

It is in fact an object of type BatchEncoding. So as you recall, the batch encoding object has a tokens method. So this is the tokenized sentence. You're encouraged to try other sentences to see which words will be split into sub words. The next step is to recall how we can get the word IDs of a tokenized sentence. The next step is to define our Align Targets method. This is a bit simpler than for NER, since there's no need to account for the IOB format.

Again, we take in a two inputs which are a list of original labels and the word IDs. Inside the function, we initialize an empty list where we will store the aligned labels. The next step is to loop through each of the word IDs. So because there's no need to account for IOB format in this example, we only have two scenarios. The first scenario is where the word ID is none, which means it's a special BERT token. The second scenario is where the word ID is not None, which means it's a real word when the word I.D.

is none. The label should be -100, and when the word ID is an integer, we should grab the label for that word. Now recall that our labels lists will be a list of strings, but for neural networks we need these to be integers. Therefore, after we get the label as a string, we use it to index the label2id dictionary we defined above. This will give us the label ID which the neural network can work with rather than the string label. Once we have our label, we add it to our list of aligned labels, and once our loop is complete, we return the aligned labels.

The next step is to test our function on the same sample from our data set as above. So as you can see, we get a list of integers which isn't that intuitive to look at. The next step is to print out the aligned labels along with the tokenized input in corresponding order. So as you can see, this shows us that the targets have been aligned correctly. The next step is to write our tokenize function, which we're going to apply to our datasets in the next step as before.

We begin by using the tokenizer to tokenize the inputs. We then grab the targets of the whole batch. We then initialize a list where we will store the aligned targets. The next step is to loop through the targets in the batch for each sample at index i, we grab the word IDs for the i-th sample. We then align the targets to those word IDs, and then we add those to our list of aligned targets. When we've completed the loop, we add the aligned targets to the key labels, which is where Hugging Face looks for the labels.

Finally, we return the tokenized inputs. The next step is to remind ourselves of the column names in the original data. So as you recall, these are no longer needed, so we can remove them in the next step. On this step, we map the tokenize function to each sample in our dataset. We use batching to make it more efficient and we remove the original columns. The next step is to check the output. So as you can see, we now have a data set dict that has been split into train and test.

Each dataset has the columns we need, including input, IDs, attention, mask and labels. The next step is to create our data collator as we did before. The next step is to define a function that will flatten a list of lists. You don't have to worry about how this function works since it's just copied from StackOverflow. But you should understand why we are using it. As you recall, in the previous notebook, we used the seqeval library to compute metrics.

This is built specifically for sequence labels in IOB format, but our data is not in this format and therefore it does not apply. Instead, we are going to take a simpler approach. As you recall, our targets are now a list of lists because the target for a single sample is a list of tags. A simple way to evaluate our model would be to flatten both the predictions and targets into a single list and compare them as if they were individual independent predictions.

Then we can use scikit-learn to compute the accuracy and the F1 as we normally do. So the next step is to write our compute metrics function. We'll begin by importing numpy and our desired metrics from scikit-learn. Please feel free to use any other metrics you feel like our compute metrics function begins as it did before. We take as input, a tuple of logits and labels, and we split them into separate variables. We then take the argmax to get the predicted labels.

The next step is to remove all the -100 values from the labels, which shouldn't factor into the metrics. Note that unlike the previous example, there is now no longer any need to convert the labels back into strings, which is nice. I've called the result labels jagged since after removing all the special tokens like padding, each list will have a different length. The next step is to do the same thing for the predictions. Removing any prediction where the corresponding label is -100.

The next step is to flatten both the labels and the predictions. The next step is to compute the accuracy and the F1. Finally, we return the accuracy and F1 in the dictionary. The next step is to test our compute metrics function, which is a good idea since it's very different from what we had before. Before I run this, try to figure out what the result should be as an exercise. As you recall, the first and last predictions are irrelevant. Okay. So hopefully you've written down what you expected the result to be.

Since there are five labels and one of the predictions is incorrect. Our accuracy is 0.8. The F1 is a bit harder to compute, so don't worry if you didn't do that. The next step is to load up our pretrained model. The next step is to create our training, arguments, objects. The next step is to create our trainer object, passing in all the objects we define above. We then call trainer dot train to begin the fine tuning process. So we get a final accuracy of about 98.5% and a final F1 score of about 95%.

The next step is to save our model to disk. The next step is to load up our model as a pipeline object. Note that have not specified any aggregation strategy since this is just general many-to-many task. In general, you would want to see the label for every token in the input sequence. The

next step is to test the model on some data. Okay. So please have a look at the results yourself and make sure that they make sense. We see that bill is a noun, which is correct.

Gates is a noun, which is also correct. Was is a verb, which is correct. There is a determiner, which is correct.

15 · Token Classification Section Summary

So in this lecture, we will be summarizing everything we learned in this section. In this section, we extended what we learned in the previous section by learning how to not just predict a single label for a whole document, but to predict a label for every token in the document. As you saw, this has several important applications in NLP, including NER, parts of speech tagging and chunking. This section builds off what we learned in the previous section.

So many of the steps remain the same. We learned how to load up a Hugging Face dataset, how to apply tokenized to the dataset, how to download and fine tune a pre-trained model, and how to save it and reload it as a convenient pipeline object. This section added a few new steps, including the data collator and target realignment.

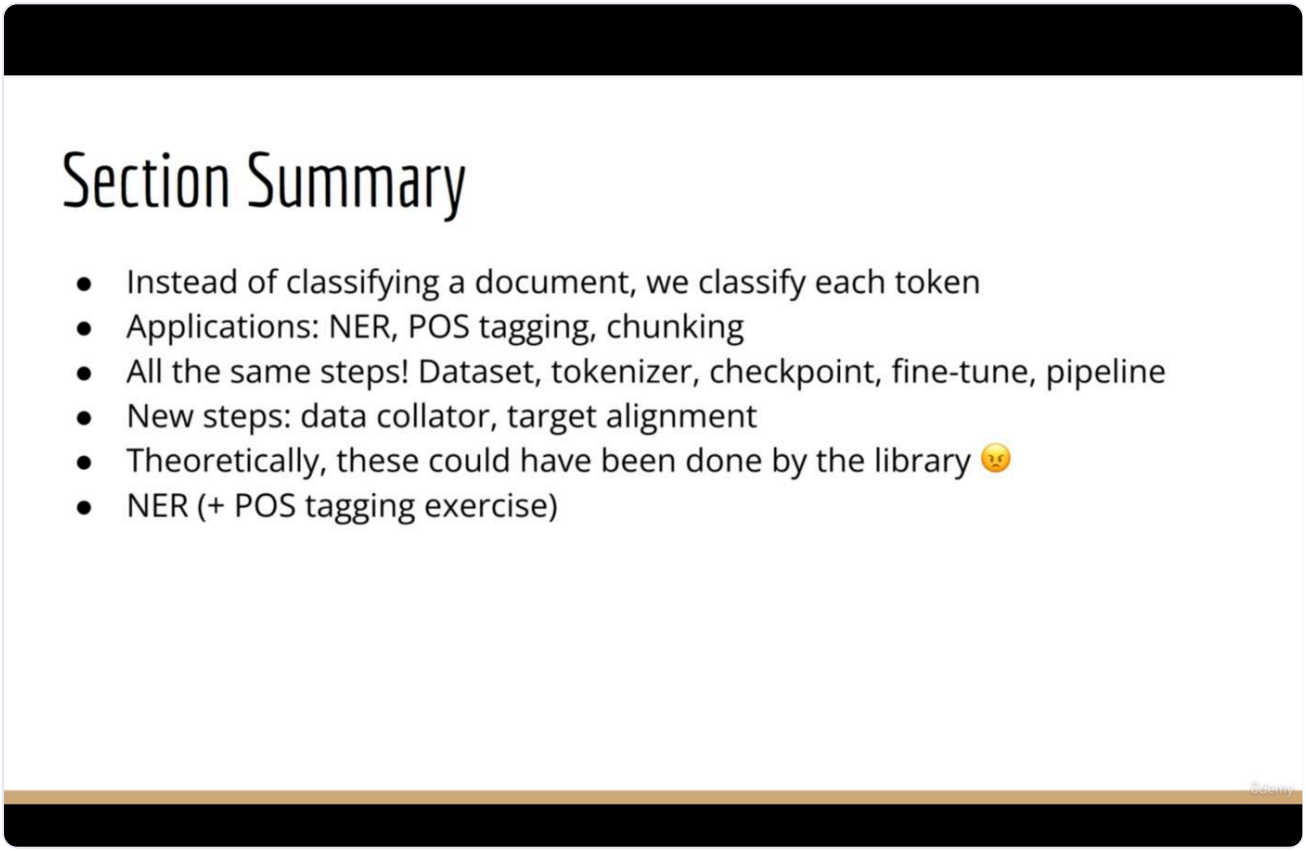

FIGURE 5.58 | Section Summary

We noted that, theoretically speaking, some of these things could have been done automatically by the library, but currently that is not the case.

This section required us to write some of our own algorithms, which is okay since we got to learn a bit more about how things work. This will be the case in the subsequent sections of the course as well. For this section we looked at the named entity recognition task in parts of speech tagging, which was an exercise for you to complete by yourself. Now, after completing this section, you might be wondering. This section only considers tasks where each token is mapped to a tag.

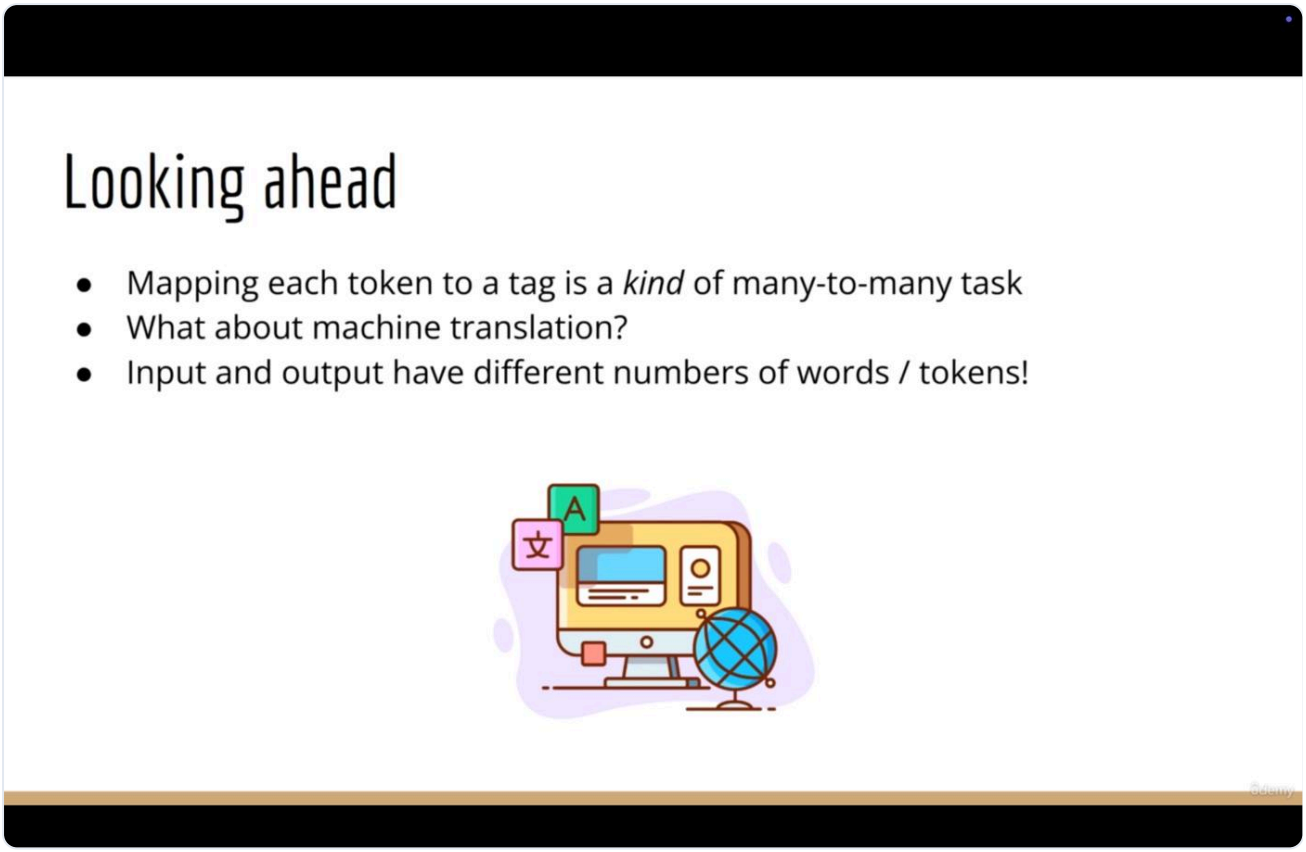

FIGURE 5.59 | Looking Ahead

It's a kind of many-to-many task, but there are other kinds of many-to-many tasks where this is not the case. For example, machine translation where the input and the target can have different numbers of words as you'll soon find out. This requires a whole new type of model, and so this will be discussed later in the course.

SECTION 6

Seq2Seq and Neural Machine Translation (Intermediate)

1 · Translation Section Introduction

So in this lecture we will be introducing the next section of this course, which is on how to fine tune a transformer for neural machine translation. This is a continuation of the part of this course that focuses on how to fine tune transformers to improve performance. So let's talk about this section from a bird's eye point of view and why it's similar, but also different from the previous section of the course. In one sense, translation is very similar to what we just did with NER in parts of speech.

Both of these are many-to-many. The input is a sequence of many tokens, and the output is a sequence of many labels. The difference is that in the previous section, the inputs and labels were 1 to 1. That is, for every input. There was a corresponding target for translation. This is not the case. A short sentence in English might turn into a long sentence in Chinese, or vice versa. In addition, this also makes evaluating the model more difficult with NER or parts of speech.

We can just check at each position whether or not our prediction matches the target. But with translation, we cannot do this since there are many possible ways to express the same thought, all of which can be valid. As such, this section will require new tools to handle these challenges. Speaking of new tools, again, this section really amounts to learning about which are the right objects and classes to use. That is to say it's more about learning about the API as opposed to learning fundamental, deep learning knowledge, as you learned in the previous section.

Different tasks require different objects. One example of that is the model instead of using auto model for a sequence or token classification. Now we'll use AutoModelForSeq2SeqLM. Another example is the data collator. Instead of using DataCollatorWithPadding, we'll use DataCollatorForSeq2Seq. Importantly, this section even requires us to use a new trainer. So instead of the regular training arguments class, we use the class Seq2SeqTrainingArguments and instead of the Trainer class, we'll use Seq2SeqTrainer.

Of course, you can expect these to be similar to their non-seq2seq counterparts. Now, at this point, you must be wondering what is seq2seq? For those of you who have studied seq2seq RNNs with me in the past. You're in luck. If you understand seq2seq for RNNs, you pretty much already understand seq2seq for transformers. The concept is exactly the same, except that you replace the RNN unit with a self attention unit. Details of that are to be discussed, but at a high level it should make perfect sense.

So seq2seq would be a useful but not explicitly required background for this course. At this point, it's sufficient to understand that at a high level, we're going to be using a new model architecture. And what that means is we're finally moving away from pure BERTs. So let's now outline the topics in this section. We'll begin by looking at the format of our dataset as well as how to use

the tokenizer on it. You should already be thinking about how this might be different since we're now working with two languages instead of just one in this section, both the inputs and the targets need to be tokenized.

The next step will be to look at how to prepare the data so that it can be passed into the model for fine tuning. Well, then look at the model itself, which will be the same as what we saw in the beginners corner. Well, simultaneously look at the data collator, which in this section has a dependency on the model. At this point, we'll also talk a little bit about how seq2seq works, since it'll help us understand the API a bit better. Our goal in this portion of the course is to discuss theory as little as possible.

So this will be just enough to understand why your model inputs look the way they do. Well, then look at metrics. Although we already discussed the BLEU score earlier in this course, we're also going to look at another interesting metric, which is the BERTScore. Finally, we'll look at how to train and evaluate our fine tune model as well as load it back in as a pipeline object.

2 · Data & Tokenizer (Code Preparation)

So in this lecture we will be looking at the data set will be using in the section along with how to apply the tokenizer to it. We'll be using a dataset called KDE4, which, like the other data sets we've seen that can be loaded in automatically by using the load_dataset function. This dataset has a ton of language pairs, so if you're interested in trying this code on a different language pair, I've left a link both in the notebook and extra_reading.txt, which lists the pairs in the following notebook.

We'll be translating English to French. In this case, we want to pass in the arguments lang1 and lang2, which we set to en and fr. When you do this you'll see that we get a data set which has not yet been split into train of validation and test. The data set has over 200,000 examples. Now it turns out that 200,000 examples is way too many for a notebook to finish in a feasible amount of time. You'll notice that our model is so large that it even reaches the limits of what the free version of CoLab can handle.

So there will be a few things we need to do in order to address this issue. In terms of the data set, we're going to draw a subsample of only 1000 pairs. This is less than 0.5% of the full dataset. Doing this will result in the training process taking just under 5 minutes. So you can extrapolate the time it will take if you want to use more data. As you've seen, it's also pretty simple to create our own train test split. So as you saw, our dataset contains two columns which are translation and ID. ID is not really part of the data.

So translation is really what we want to look at. If you print out a sample, you'll see that the value stored in translation is a dictionary which contains two keys en and fr, both of which point to their respective sentences in those languages. Obviously the keys will be different depending on which language codes you choose. As a side note, please note that these are standard two letter language codes. So if you're curious about what the code is for some particular language, it should be easy to look up.

So the data is pretty simple. What we really need to know is how to process this data. One thing that should immediately come to mind is how do we tokenize the target? Clearly we cannot use an English tokenizer because that wouldn't work with languages like Chinese for example. So do we need two separate tokenizers? The answer is yes and no. Yes, because there are two separate tokenizer models internally, but no, because they are both stored in the same tokenizer object.

So the answer really is to choose an appropriate checkpoint for your data set. This is because the checkpoint is what defines what tokenizer you will get when you call AutoTokenizer.from_pretrained. Luckily, as with sequence and token classifiers, there are already checkpoints for many language pairs. In fact, it turns out that we can use the same checkpoint we used in the beginners corner. Obviously with the correct language pairs selected. Once you have your tokenizer, there are two ways to use it.

If you just use the tokenizer like we did before by calling it like a function. This will tokenize the model inputs and this will assume the language specified for the input of the check point. If you want to tokenize the targets, you need to use what is called a context manager. This should look familiar if you've ever worked with files in Python. The syntax is with tokenizer.as_target_tokenizer() followed by the usual tokenizer syntax. This ensures that the correct tokenizer will be used for the target language.

Now, just as a side note, it might be worth knowing that it's possible that the input tokenizer might work, or at least won't crash if you use the input tokenizer by mistake. This is especially true if the input and target languages are similar, like English and French or Spanish. However, the tokenized output will be suboptimal. One thing to be aware of, since we are now no longer using BERT, is that there are different special tokens, as you recall.

BERT uses special tokens like CLS and SEP, but it's more important to keep in mind that these are BERT specific. They don't exist for other models, and other models may have their own set of special tokens and conventions for the model will be using. There will be no CLS and SEP. Instead, every input sentence well ends with an end tag which looks like an HTML or XML tag. In addition, you'll recall that when BERT tokenizes words into sub words the second and later sub words start with two hash symbols.

With our new model, this won't be the case. Instead, the convention is that each new word will start with a big underscore and subsequent sub words won't have any prefix. This is not important for you in terms of a code you will write, but it's good to know so you won't be surprised by what you see.

3 · Things Move Fast

So this is just a short lecture to remind all of you that this field moves fast. Things will change and break. This is to be expected. Case in point, in the translation lectures, you'll see that we use a context manager called as_target_tokenizer when we tokenize the translation text. This was the case while I was making this course. Today, as I am still in the midst of making this course, this is no longer the case. Literally, one minute this is working fine.

And when I ran the same code a few minutes later, I was getting a warning telling me that this will change. So the new way of doing this is instead of using as_target_tokenizer you simply pass in your text using a special argument called text_target. In my opinion, this looks nicer since it's kind of weird that a variable you define inside the scope of a context can still be used outside of it. In any case, the following lecture is will use as_target_tokenizer, but you may need to update this when writing your own code.

And just remember, guys, when we are hiring for jobs, we don't care that you've memorized one library. We care that you have the skill to learn any version of any library. And that includes being able to handle changes like this with relatively little efforts, even by yourself. So if I see any of you freak out over this, just remember that this is not what employers want to see. This is one of the many examples I have that things in this field move super fast, and if you're not prepared to handle it well, then you simply won't do as well as those who can.

4 · Data & Tokenizer (Code)

So in this lecture we'll begin looking at the notebook where we will fine tune a translation model. We'll begin by installing transformers, datasets and SentencePiece. Note that SentencePiece is required for the tokenizer to work. The next step is to load up our data set, which is a data set with English and French language pairs. So as you can see, we have over 200,000 samples. Unfortunately, this many samples is way too much for this notebook to complete in any reasonable amount of time.

As such, we'll begin by randomly selecting just 1000 samples from this dataset. We'll call the results small. Note that this is less than 0.5% of the total. Using that, you can make an educated guess about how long this would take with the full dataset. The next step is to split the data set into train and test. The next step is to print out one of the samples from our dataset. As you can see, the samples are kind of weird. The next step is to load up our tokenizer object.

As you can see, we are using the same checkpoint from the beginners corner. This might be surprising since that was not the case in the last two sections. You might be wondering what should you do if you want to train a translation model completely from scratch? Well, given the amount of time and memory that this notebook takes to train, you can imagine that doing it from scratch would require even more resources. The next step is to print out an example sentence pair which we will subsequently tokenize.

The next step is to tokenize the input sentence, which can be done with the usual syntax. So as you can see, the result looks as we expect. The next step is to tokenize the target sentence, which requires us to use the as_target_tokenizer context manager. Again, we note that the result has the expected format. Note that this also comes with the attention mask, which isn't actually required for the target. The next step is to look at the tokens for the target.

So we're going to convert the IDs back into tokens. The goal of this will be to demonstrate what will happen if you do this correctly compared to when you do this incorrectly. So as you can see, for this particular example, we happened to get one token per word. Note that the beginning of

each word is denoted by the large underscore. Another thing to note is that the convention here is clearly different from BERT, which we've been using in the course up to this point.

What this teaches us is that each model has their own conventions for how to process the input. Just like computer vision. In this case, there is no special start token, but each sentence ends with a special end of sentence tag. The next step is to check out what will happen if we use the tokenizer on the target sentence incorrectly. We'll convert the IDs back into tokens to see what we would get. So as you can see, this looks a lot different from what we just saw.

There are a few important things to note. Firstly, note that this does not completely fail. If you've studied tokenizers used for deep learning before for example, in TensorFlow, this might surprise you. Usually tokenizers use some form of splitting on whitespace and punctuation and perhaps follow a set of built in rules to further split certain words. Each token is stored in a dictionary, which we call the vocabulary. The vocabulary is the set of all known tokens.

Now, usually what happens if we try to tokenize words that did not appear in the train set is they would simply be replaced by a special unknown token, or they would be ignored altogether. Clearly, that's not what happened in this case. The English tokenizer should not be able to recognise French words, but it is incomplete and tries to tokenize the French words anyway. This is another hint that tokenizers in the era of Transformers work a bit differently than what you have seen in the past.

However, although we didn't get any errors, the result is suboptimal. The English tokenizer doesn't know about French words, and so it splits them up at illogical points. As you can see, tokens which are not the start of a word, do not have any underscore prefix. This is different from the BERT tokenizer, where new words have no prefix and subsequent tokens have the double hash.

5 · Aside Seq2Seq Basics (Optional)

So in this optional lecture, we are going to go off on a slight tangent and talk about how sequence to sequence models work. This will help give you a stronger footing for understanding why the API is the way it is in the notebook. Note that this section does not require any knowledge of transformers, but knowing about RNNs would be very useful. Effectively, anywhere you see an RNN block in this lecture, you can simply replace it with a transformer block.

The inputs and outputs won't change. To help you understand the structure of this lecture, let's start with an outline. We'll begin by discussing language models and how they are trained and used. This might seem kind of off topic at this point, but it's exactly what we need. The language model actually forms the decoder component of the seq2seq architecture. Once we understand the decoder, we can then discuss the encoder and how we can combine both the encoder and decoder to make a full seq2seq model.

The encoder is much simpler to understand and it's basically what we've been using already. The RNN analogy would just be a basic RNN for text classification or parts of speech. Note that these were the two types of tasks we just looked at earlier in the course. Okay. So as per our outline,

let's begin by discussing language models at a high level. A language model is a model that predicts a distribution for the next word in a sentence given the previous words in a sentence.

Training an RNN to do this works just like parts of speech tagging, except with different targets. Specifically, we have a target for every input token, but our targets are really just the sentence we want to learn how to generate, and the inputs are a shifted version of that same sentence. As such, this is a many-to-many token classification task, just like NER. It makes more sense with a picture. Consider the sentence. I like cats. As you can see, the first input is just a generic start tag.

The first target is the first word in the sentence. In this case, it's I. On the second step, the input is the first word in the sentence, which is I. Now our model has to try and predict the word like. In other words, it must learn that like comes after I and the start tag. On the third step, The input is the second word in the sentence, which is like. The third target is the final word in the sentence, which is cats. As such, a model is learning how to predict the word cats given the previous token's start tag, I and like.

Finally, the fourth input might be cats and the corresponding target might be an end of sentence token. So it should be pretty intuitive that by training a model in this way, we are training it to predict the next word in a sentence. Given all the previous words, this is a language model. Now, as you recall, language models can also be used to generate text completely from scratch. That is, we don't even need to be given a whole set of previous words.

They can simply generate whole sequences of text by themselves. So how does this work? Well, you'll recall that for the inputs, they always start the same way with the special start tag. From here, the model can really generate any word that would be appropriate for the start of a sentence. It's kind of like building a GAN to generate images. You might train it on MNIST, but with a normal vanilla GAN, you can't control whether you're going to get a 0, 1, 2, 3 and so forth.

It just depends on the random samples you draw. In any case, let's get back to our RNN. Once we generate the first word, you might wonder what can we do to generate the second word? Usually the input for the second time step would be the first word in the target sentence. However, in this case, recall that we are now generating text. That means the second word should be an appropriate follow up to the first word we just generated. Suppose that word was the.

As such, our model is now giving us the distribution of p of x2 given x1 equals the. To obtain this distribution, we feed our first output back into the input of the RNN at the next time step. Now suppose that the next word we generate is cat. Again, we pass this word into the input for the third time step. Now our model is giving us the distribution of x3, given x2 equals cat and x1 equals the. Now suppose that we continue this process until we generate the sequence.

The cat sat on the mat. You might be wondering how does our model know when to stop? If we keep feeding words back into the input, we might just keep generating words forever. Of course, we also taught our model how to generate end of sentence tokens since these were part of the targets. So if our model ever generates this special token, then we know we should stop the process and not feed it back into the input. Okay. So we just discussed how language modeling works.

As mentioned, the language model forms the decoder portion of our seq2seq network. The other part is the encoder. The encoder. If we're talking about RNNs, it's really just a text classification RNN, where we simply do not compute any outputs from the hidden states. Normally, as you recall, we would take the hidden states and pass them through a final dense layer in order to get a prediction. If we're classifying a whole document into a single label, then we would typically take the final hidden state.

But if we were doing a token classification task, then we would do this for every hidden state. As an encoder, we do not use any final dense layer, but instead we simply save the final hidden state. We can think of this like an encoding of the input sentence. Hence why we call this an encoder. It encodes the input sentence into a single vector, which is like an efficient, compressed version of the sentence. Now that we have both our encoder and decoder, we can combine them to form a seq2seq network.

As you recall, every RNN unit has a prior hidden state and outputs a new hidden state. That's why we call it recurrent. It's a network that loops back into itself. In a decoder only language model, the initial hidden state would be some fixed vector, like a vector of zeros. In that scenario, the initial hidden state is kind of irrelevant. And when we generate the first word in such a language model. Recall that what we generate is just random, like an image we would generate from a GAN.

What we generate is not conditioned on anything. We simply generate something from the training sets distribution. But in a seq2seq model, we can condition on the input sentence by having the initial hidden state to the decoder come from the output. Hidden state of the encoder. As you recall, the encoder converts the input sentence into a vector. We can think of this like a concept vector or a thought vector, which lives in a vector space which you can imagine, like the space of all possible concepts or thoughts.

So now our decoder will not just generate a random sentence, but it will be conditioned on this information it gets from the encoder. Specifically, it will try to decode this concept or this thought back into a sentence in the target language. That's why this vector space of thoughts is really a vector space of thoughts. Given the fact that the same thought can be expressed in at least two different languages. The thought really is in fact, language agnostic.

It's like a numerical representation of a thought. Our decoder simply learns how to express this thought using words. However, the key point is that in a decoder only model, the sentence we generate is not conditioned on anything. So what we generate can be totally random with the encoder decoder combination. The decoder is conditioned on an input sentence, and the loss function is such that it learns to translate the input sentence into a new language.

As a final topic for this lecture, let's consider the inputs and targets for training a seq2seq network like we just described. At a high level, it's pretty simple. We just need pairs of input sentences in one language and corresponding target sentences in the target language. After we tokenize these sentences and convert the tokens into integer IDs, they can be used as inputs and targets for the network. But consider how the decoder, which is a language model, has to be trained.

The decoder for the purpose of this lecture is an RNN. It has both inputs and outputs. We know that the outputs should be mapped to the targets and the loss function should be some comparison between the two. But what should the inputs be? Remember that these inputs here are different from the inputs into the model as a whole. The actual model inputs will be the tokenized version of the input sentence that goes into the encoder. But now we have to consider what the inputs to the decoder should be.

In fact, it's very similar to training a language model with no encoder. The decoder input is simply a shifted version of the target. So at every time step the decoder receives the true past tokens. In effect, the decoder language model is learning to predict the next word in a sentence conditioned not only on the original input sentence, but also the previous words in its own sentence. But just like the decoder only language model, when we use the seq2seq model in inference mode to actually generate text, we do not have any true targets.

This is when we want to use our model to translate arbitrary text. In this scenario, the decoder will generate one token at a time, and for every token it generates we feed it back into the input of the decoder for the next time step. This is just like how we use the decoder only language model to generate text. The main difference now is the same as what we saw during training is that the decoder output is conditioned on the input sentence.

6 · Model Inputs (Code Preparation)

So now that we've discussed how to tokenize the inputs and targets, we are ready to think about how to prepare the data to be input into the model for training to understand the next step. Let's relate it back to what we did earlier in the course when we wanted to classify documents, we created a tokenize function that simply called the tokenizer with truncation. As you recall, this fills in the fields such as input, IDs, attention, mask and so forth.

When we wanted to classify each token, we created a new function that not only called the tokenizer but also populated the labels field. As you recall, the labels field is where Hugging Face looks for the targets, and we had to realign the targets manually so that we could set this field properly. In this section, we have a similar situation. We need to populate fields for both the inputs and the labels. In fact, this section is a bit easier because we already know that the inputs and targets can have different sequence lengths.

As such, there is no need to take any special steps to align the targets. So here's what our tokenization function will look like. Essentially, we just need to tokenize the inputs, tokenize the targets, and assign the targets to a field called labels. So we begin our function by first splitting up the inputs and targets into separate lists. We can do this by looping through the translation field in our batch of data and grabbing the sentence corresponding to each language.

The next step is to call the tokenizer for both the inputs and targets, which you've already seen how to do. However, you can see that we've added a few new arguments. Specifically, we added truncation, which you should expect by now, and we also add a parameter to specify the maxi-

mum sequence length In the notebook, we'll choose some number that covers most of the sentences in the dataset. The final step is to take the input IDs from the tokenized targets and assign these to a field called labels in the tokenized inputs.

This is because the input IDs store the token IDs for the targets, but we want the targets to be available in a field called labels, as they were for the rest of the models we studied. So everything is stored in the objects that we're calling tokenized inputs. We then return the tokenized inputs object. As usual. The next step is to call the map function to map the tokenize function to every sample in our dataset. Again, we remove the original columns since they are no longer needed.

The next step is to instantiate our pre-trained model. Now, this might seem a bit out of order compared to the previous notebooks we studied, but there is a reason for this, which you'll see shortly. For now, all you have to know is that just like with token classification, there is a special type of model we need for this task. Specifically, we need a model called AutoModelForSeq2SeqLM. If you've studied seq2seq in NLP with me in the past, this name should make complete sense and in fact it should give you a sense of comfort because it's all stuff you've seen before.

In particular, we know that the type of model we need for translation is seq2seq. seq2seq models are made up of two components the encoder and the decoder. The decoder is a language model because it's built to generate a sentence one token at a time in an auto regressive manner. This is just like, say, generating poetry, except that instead of starting with a completely random token, we conditioned on the hidden state output from the encoder, which is a vector representation of the input sentence.

In any case, we take the usual steps of calling from_pretrained and passing in our checkpoints. Once we have our model, we can then create our data collator. Again, because this is a new task, we're going to have a new data collator, called appropriately DataCollatorForSeq2Seq. Now you might think we've done the steps out of order because we usually do all our data processing before we create the model. But the reason we created the model in the previous step was because we're actually going to pass it into the data collator constructor.

Basically, this is going to be used to populate the decoder input IDs. So let's review what the data collator needs to do. Although do note that this is optional since you can just copy the code without understanding what's happening. This is for those of you who want to have some idea of what the data collator does. Okay, so we know that both the inputs and targets will be variable length sequences of token IDs. Therefore, they will need to be padded as well as converted into torch tensors.

That much we have in common with the previous sections of the course. But there is a new step since the decoder is itself a neural network. Again, we can relate this back to RNNs. We know that the decoder is an RNN and it makes its computations based on two vectors the input vector and the output of the encoder. We can imagine the input coming from the bottom and the encoder output coming in from the side. Transformers are structured in the exact same way.

The decoder is just an auto regressive model that takes as input the previous words in a sentence and tries to predict the next word in a translation system. The decoder is also conditioned on the input sentence we are trying to translate, which gives us the encoder hidden state. As such, the decoder input IDs are really just a shifted version of the targets. Conceptually, the first input will just be a generic start token from which we try to predict the first output.

Once we have the first output, we then want to generate the second output. The second output takes as input. The true first token of the translated sentence. So we're always trying to output the next token given the previous tokens in the sentence. Just like a language model, except that it's also conditioned on the input sentence we want to translate. In fact, let's zoom into this image and explain this in a more step by step fashion. Note that this image is for a chatbot, but the concept is exactly the same.

As you recall, this is just a manifestation of my rule. All data is the same. The exact same model that can be used for translation can also be used for chatbot. In any case, the first step is to pass the input sentence into our encoder. The encoder then processes the whole input sentence and produces one final hidden state. This hidden state then becomes the initial hidden state of the decoder. As you can see, coming in from the left. But the decoder is also an RNN, meaning.

Not only does it have a previous hidden state, but it also must have some input. Well, the input during training is simply a shifted version of the targets. Recall that this is exactly how a language model works. At the beginning we simply start with a generic go tag. From this and the initial hidden state, it must learn to generate the first word, which is "it". Note that "it" is also the input into the model at the next time step. From this the model must learn to produce

the word "is" and as you can see, that is also the input at the next time step. And this pattern keeps repeating such that the decoder input IDs are simply a shifted version of the targets. And this is so that the decoder learns to predict the next word in a sentence, given the previous words.

7 · Model Inputs (Code)

So in this lecture, we will be continuing our notebook. We'll begin by drawing a histogram of the sequence lengths of the inputs and targets. The reason we want to do this is because we want to specify a maximum sequence length for truncation. To do this, we'll first grab the translation column of the train set. Then we'll use a list comprehension to collect the lengths of each English sentence. So as you can see, most of the samples in the train set fall in the shorter end.

We would be justified in setting the maximum input length to be somewhere between 100 and 200. The next step is to draw a similar histogram, but for the targets instead of the inputs. So these are the French language sentences. So as you can see, the sequence lengths are pretty similar to the English sentences. So again, we would be justified in setting the maximum target length to be somewhere between 100 and 200. The next step is to write our tokenize function.

We'll begin by setting the maximum sequence length for both the inputs and targets to be 128. Then we'll define our tokenize function, which takes a batch of data as input. Inside the function, we begin by splitting up the samples into input and target lists. The next step is to tokenize

the inputs. This is similar to what we saw before, except that we also specify the max length. The next step is to tokenize the targets. Again, we have the same code, except we use as_target_tokenizer.

Now, as you know, the Hugging Face API expects the targets to be in a field called labels. So that is where we will place the token IDs from the targets. Note that the attention mask is not needed, although the tokenizer does produce them. Finally, we return the Tokenized Inputs object, which contains both the input fields and the targets or labels field. The next step, as usual, is to map our tokenize function to the DatasetDict, as well as remove the original columns which we no longer need.

The next step is to load up our pre-trained model. This time the correct class to use is called AutoModelForSeq2SeqLM. After learning a bit about seq2seq, the reason for this name should be clear. seq2seq is the type of model required for translation and LM means language model, which describes how the decoder works. The next step is to create our data collator. As with token classification, we need to explicitly define a data collator and in this case it also accepts a model as input.

This model is used for the purpose of generating the decoder input IDs. The next step is to test out our data collator. We'll begin by reviewing the necessary input format. As you recall, the data collator expects as input a list of samples where each sample is a dict. This is opposed to what we would get if we index the dataset with a range of indices which would give us a dict of lists. So as you can see, each sample consists of three fields which are input, IDs, attention, mask and labels as expected.

The next step is to test our data collator on the above inputs. We'll also print out the keys of the result to see if we get back anything new. Recall that in the past we simply use the data collator to pad the sequences and to convert them into torch tensors. Okay. So as you can see, the data collator seems to have done one extra step, which was to create another field called decoder input IDs. As you recall, these are needed for the training process. The next step is to print out the labels of the data collated batch.

So as you can see, they are now torch tensors. Furthermore, we continue to use -100 for the targets that should be ignored. So that hasn't changed. The next step is to look at this new field, which is called decoder input IDs. So as you can see, we get back another torch tensor. Now we know that the first sentence was the shorter one, as per the above results. We can see that the decoder input IDs have been padded with the ID 59513. But interestingly, this ID also appears at the beginning of both sentences.

So it seems to have a dual function of being both a start token and a pad token for the decoder inputs. Remember that the decoder input IDs are just a shifted version of the targets. So in order to get a better idea of what this special token is, we're going to convert the IDs back into tokens for the decoder input IDs. So as you can see, the mystery token is in fact a pad. Curiously, this is also used as the start of sentence tag. The next step is to convert the targets back into tokens as well, just to confirm that the decoder inputs really are just a shifted version of the targets.

So as you can see, the decoder inputs really are just a shifted version of the targets. Interestingly, for the targets, we use the unknown tag for any target beyond the end of sentence tag. In fact, this is just because the -100 does not appear as one of the token IDs in the vocabulary. And just for completion sake, let's just print out the sample itself to see the real input and target sentences. So as you can see, it's just KDE. As some trivia, note that KDE is the group behind the Kubuntu Linux distribution.

8 · Translation Metrics (BLEU Score & BERT Score) (Code Preparation)

So in this lecture, we are going to discuss the metrics that will be using in our notebook. Just as a brief review, let's remind ourselves why we need different metrics for the translation task. The main point is that little differences here and there are unlikely to change the meaning. For example, the cat chased the mouse and the cat ran after the mouse. Furthermore, different sentences can mean the same thing, and as such there can be more than one valid translation of an input.

As such, the BLEU score looks at n-grams, which are subsequences of tokens. The BLEU score is then a weighted average of n-grams for different sizes of n. It also chooses the best score over each reference. The final BLEU score is a score between zero and 100, but is sometimes expressed to be between a zero and one. In the beginners corner of this course. We already looked at the BLEU score. But that was through NLTK. In this section, we'll continue to use the BLEU score, but we use a new library that claims to compute the BLEU score more consistently than other packages.

We can install this library by using the command pip install sacrebleu. We can then use the metric in the usual way. We import the function load metric and then we call this function passing in the string sacrebleu. When we call metric dot compute, the predictions must be a list of strings and the targets must be a list of lists of strings. As you recall, this is because there can be more than one target per translated sentence. In the following lecture, we'll also be using another metric, which I think is more interesting and makes use of our knowledge of deep learning.

In particular, we'll be looking at a metric called the BERTScore. The BERTScore also handles this issue of slightly different sentences, still having the same meaning. It does this by internally using a pre-trained BERT model and computing contextual embeddings for both the predicted translation and the targets. Since these embeddings are just vectors, they can be compared by using cosine similarity. If we had a sentence like I ran to the store and compared it with I rushed to the store.

We would consider these to mean the same idea. The BLEU score marks the word ran as completely incorrect, but the BERT model knows that these two words essentially mean the same thing in this context. And this is what deep learning really is all about. It's about mapping inputs to a useful vector space so that we can infer relationships between those inputs. To use the BERTScore, we proceed as usual. We first need to install the BERTScore package by using pip install bert-score.

Then we can load in the corresponding metric by calling load metric passing in the string bert_score. Note that when you install the library it has a dash, but when you use it in code it has an underscore. Finally, we can use the metric by calling metric compute passing in a list of predictions and a list of lists of references, again using the same format as the BLEU score. Note that we do need one additional argument, which is the language. Again, this is specified by the two character language code for the BERTScore.

Note that it returns precision, recall and F1 individually for each prediction you give it. As such, if you want the overall score for all samples, you can take the mean.

9 · Translation Metrics (BLEU Score & BERT Score) (Code)

So in this lecture, we will continue looking at our notebook. Moving on to metrics. We'll begin by installing two libraries which implement the metrics we need the BLEU score and the BERTScore. The next step is to import the load_metric function and then to load up our two metrics. The next step is to test our metrics on a few examples to demonstrate how they behave. In this course, we're using the BLEU metric because it's pretty standard, but we're also using the BERTScore metric since it overcomes drawbacks of the BLEU metric.

We'll begin with a simple sentence, which is I love cats. Note that the prediction is exactly the same as the target. So as you can see, the score turns out to be zero because the BLEU score doesn't really work with short sentences. Now let's try a longer sentence. Note that Marian is the framework which was used to train the model from this notebook. Again, the prediction is the same as the target. So as you can see, the score is 100. So the maximum BLEU score is 100 instead of just one.

Now we know that the BLEU score metric compares n-grams. Either they match or they don't. This means that it's not capable of detecting similar words or words with the same meaning. This is a real downside of this metric because for translation, what we really care about is that the meaning is correct, not that a specific set of words were used. This is also why the BLEU Score accepts multiple references to begin with. Although, of course, it would be very difficult to cover all possibilities.

For the next example, we're going to compare to similar but different sentences. I love cats and I like cats. We know that these sentences are very similar in the meaning, but only two out of three words are correct. Note that we must also specify the language code which is en for English. So notice how the first time you call metric.compute. This will actually download some version of BERT. As you recall, this is because internally the metric converts the words into contextual embeddings and then computes the cosine similarity between them.

Thus, the model is required to make these computations. So as you can see, we get an F1 of nearly 99%, which is a good sign that this metric understands meaning and doesn't just try to match words. Now as you recall, we cannot use metric.compute directly. We need to define our own compute metrics function. We'll begin by importing numpy as we normally do. The next step is to define a function which takes as input a tuple of predictions and labels.

Note that the API is inconsistent here and we actually get token IDs and not just logits. As you recall in the previous sections we got logits and labels, not predictions and labels. And this is because getting the predictions from the logits is not as easy as simply taking the argmax as we did earlier in the course. Instead, machine translation models use a special algorithm called beam search in order to find the most likely sequence of words. As such, it would be much simpler to just give us the token IDs instead of just giving us the logits and requiring us to call the beam search function ourselves.

Now we saw already that the metrics above take strings as input instead of tokens or token IDs. As such, the next step is to decode the predictions back into sentences. Note that this also makes use of the tokenizer, since it has all the information needed to convert strings into token IDs and vice versa. Interestingly, it somehow already knows that this is for the target language and not the input language. Now, for some reason, the tokenizer doesn't recognize the -100 values, so in the next step we're going to manually replace all the minus one hundreds in the labels with the pad token ID we saw before.

The next step is to then decode the labels into sentences as well. The next step is to get rid of extraneous whitespace and also to put the targets into their own lists as required by the metrics. The next step is to call metric compute for both the BLEU metric and the BERTScore metric. Finally, we return a dictionary containing both the BLEU score and the BERTScore. Recall that the BERTScore is returned per sample. So we take the mean.

10 · Train & Evaluate (Code Preparation)

So in this lecture, we'll be looking at how to do the final steps of our notebook, which will be to train and evaluate our model. Furthermore, we'll look at how to save the fine tune model and load it back in as a pipeline object. As you've seen in previous lectures, this basically involves creating a training arguments object, a trainer object, and then calling trainer.train. This section will follow the same process, but with a few crucial modifications.

We'll begin by creating the training arguments object, which now comes from the Seq2SeqTrainingArguments class. As before, the first argument is simply the path to save the data we generate as we train. In this example, we're going to set evaluation strategy to no because as you see, this is quite a time consuming process. Instead, we'll evaluate the model once before training and once after training to observe that our metrics improved. Another argument worth looking at is predict_with_generate.

This makes it so that when we evaluate our model will generate the predictions token by token, the same way we would do it if we were to generate a real translation. This is opposed to using the true decoder input IDs, which will be more optimistic because those are only one step predictions. When you use your model in practice, the true target will of course not be available. If you study time series analysis, you can relate this to that topic as well.

Using the true decoder input IDs would be equivalent to doing one step forecasts. So the model only ever has to predict one step ahead, which always looks pretty good, but not using the decoder input IDs would be equivalent to doing a multistep forecast. So if you want to predict the

forecast for ten days into the future, you'll first predict one day ahead and then use that prediction to predict two days ahead and then use both those predictions to predict three days ahead and so forth.

In other words, any errors you generate in the preceding steps will propagate to later predictions, making it worse in general compared to the one step forecast. Similarly, it's much easier for the translation model to predict just one next word than it is to predict the whole sentence. The final argument I want to draw your attention to is FP16, which will be set to true. Normally we use 32 bits to store floating point numbers, so we call FP16 half precision.

This means that we're using half the space to store each number, which will speed up computation and take up less space at the cost of having less precise weights. For this task. This will really be necessary because our model is very large and even with this setting, you might run out of GPU memory every now and then. Note that in order to use this setting, you must be using the GPU. Once we create the training arguments, the next step is to create the trainer.

As with the training arguments, we'll now have a special trainer called the Seq2SeqTrainer. Luckily, the arguments are pretty much the same, so there's nothing more to discuss. As usual, we call trainer.train, which will start the training process. So as mentioned during the training process, we will not evaluate the model because it takes a long time. So instead we will manually evaluate the model on the test set by calling trainer.evaluate passing in the maximum target length.

We'll do this once before training and once after training to see if our metrics have improved. Once we've trained our model, we can save it using the usual function, trainer.save_model passing in the path where we want to save the model. We can then load it back in as a pipeline, just as you saw in the beginners corner by specifying the task as translation. Finally, we can translate any arbitrary text by passing in a string or list of strings.

11 · Train & Evaluate (Code)

So in this lecture, we will be looking at the next portion of our notebook where we train and evaluate our transformer. We'll begin by creating an object of type Seq2SeqTrainingArguments. Note that this is a special training arguments class which we haven't used before. Specially made for seq2seq. One seq2seq-specific argument we'll be using is predict_with_generate which has been set to true. As you recall, we need to set this to true so that during evaluation the model will be evaluated without using the true targets as input into the decoder.

Instead, the decoder will generate the translation by itself in an auto regressive manner, just like a language model. In other words, it will generate one token at a time, and each token it generates will be fed back into the input for the next time step. Now, because even making predictions with a large model like this is so computationally intensive, we're going to set evaluation strategy to no. Instead, we'll run evaluation manually before and after training to check whether or not our model has improved.

For the same reason, we also set FP16 to true, which will cause the model to use floating point numbers with 16 bits instead of 32 bits. The next step is to instantiate a Seq2SeqTrainer object. Note that the arguments into the constructor are the same as what we've seen before, so there's not much to discuss. However, do note that like the training arguments, we have a special trainer class specifically made for seq2seq. The next step is to evaluate our model before we begin the training process.

As you recall, our pre-trained model has already been trained to translate English into French. As such, its performance shouldn't be too bad. So as you can see, BERT is being downloaded once again. This time it's a multilingual version of BERT, since, as you recall, our BERTScore metric needs to compute a score based on French sentences. Earlier we were evaluating English sentences, which used a different version of BERT. So our model does pretty good. About 86% BERTScore and about 38.9 BLEU score.

The next step is to call trainer.train to launch the training process. Note that sometimes the GPU runs out of memory, which means that even with this very small sample size and half precision floating points, we still reach the limits of CoLab. One easy solution is to simply restart the runtime and run everything again. But in practice, you might want to get a more powerful GPU. So now that our model has been fine tuned, we're going to evaluate it one more time.

Okay. So now the BERTScore is about 88% and the BLEU score has gone up to 43.4. So our model has definitely improved. The next step is to save our model by calling trainer.save_model. The next step is to load in our saved model as a pipeline object. The next step is to test our translator on an English sentence. It says, I hope this course has helped you on your data science journey. So if you speak French, please let us know whether or not this translation is any good.

12 · Translation Section Summary

So in this lecture, we will be summarizing everything we learned in this section. This section focused on the task of neural machine translation, which is a kind of sequence to sequence task. Although the previous section looked at a task where our outputs and targets were sequences. This is different because the target sequence doesn't have the same length as the inputs. In the case of translation, each prediction does not correspond to each input. Unlike a task like NER. Computationally, this kind of task is quite a bit more difficult and even requires a whole new kind of model called seq2seq.

This involves both an encoder block and a decoder block, whereas the previous models we looked at were only encoder blocks. This section follow the same usual steps of our fine tuning process in Hugging Face. We began by looking at the dataset and tokenizer. The tokenizer in this section was a bit different because not only did we have to tokenize the input, we also had to tokenize the target. We then looked at how to prepare the model inputs, including the data collator in this section.

The data collator introduces a new kind of input, specifically the decoder input in order to understand why this was needed. We had to discuss a little bit about how seq2seq works in this course. The goal is really only to use the least amount of theory necessary to get through the Hugging

Face notebooks. The next step was to discuss metrics for evaluating our translator outputs. We already learned about the BLEU score in the beginners corner, but in this section we also looked at the BERTScore, which is, in my opinion, much more interesting.

Instead of trying to match exact tokens, it instead maps each token into a contextual embedding using a BERT model such that tokens could be compared using a vector similarity instead of just match or no match. Finally we looked at how to train, evaluate and save our model for later use. We noted that with models as large as these special steps need to be taken for the training process to be feasible. For instance, using 16 bit floating point numbers.

At this stage we are reaching and even surpassing the limits of the CoLab platform.

1 · Question-Answering Section Introduction

So in this lecture we will be introducing the next section of this course, which is all about a very interesting topic known as question answering. Specifically, we will look at how to fine tune a transformer for this task. As you recall, this is the task where we give the transformer a piece of text and ask it a question based on that text. Currently, the state of the art allows us to do extractive question answering, meaning that the correct answer is simply a substring of the given text.

As a little exercise, it's worth thinking about how a neural network would be able to do that. Consider whether this is a problem of classification or regression and what the outputs and loss function might look like. In order to prime your mind to learn the content of this section, you may find it beneficial to review the beginner's corner where we looked at the question answering pipeline, which applies a pre-trained model with just one line of code.

So let's discuss a brief outline for this section of the course. As with the other sections of the course on fine tuning, we will follow the same high level steps. As usual, we begin by tokenizing the inputs and processing the inputs so that they can be passed into the model. In this section, this will be our most laborious task. We'll then look at how to compute metrics which, like the previous steps, will require a large amount of work. This will look very different from the previous sections, since we'll need to do quite a bit of work to convert the model outputs into an actual string of text.

After these preliminary steps, we can finally move on to training the model and evaluating the model after training is complete. As usual, this portion will be brief. So as a general theme for this section, basically it involves a little more API hunting, which means figuring out the right functions to call in what they do, but also a lot more in terms of getting down into the weeds, much more so than the previous sections of the course. This is primarily due to two reasons.

Reason number one is that, as you recall, inputs for question answering come in the form of context and question pairs. You can imagine a context as something like a Wikipedia page on some topic because of this context can be very long and we'll need some way to handle this, along with any complications that arise from how we choose to do that. The number two issue is that it's going to take quite a bit of work to convert our model outputs into an actual answer represented as text.

At a high level, this is because the neural networks output numbers while what we want is text. In my opinion, issue number one may simply be due to the fact that this is still a new library and the developers haven't yet had a chance to encapsulate these steps into a more convenient API. In any case, what this does mean is that like some of the previous sections, you will need to put on your programming hat and write actual code. This is not basic code like you'd see in a typical Udemy course, but real code that will require you to think algorithmically.

Now, just as a heads up, there is one quirk with the Hugging Face API that will become apparent in this section and this is that they tend to call everything an ID, so you'll have sequence IDs, example IDs, token type IDs, token IDs, all kinds of IDs. This becomes very annoying to keep track of since sequence and example are such generic words. I've done my best to give variables less insane looking names. But do keep in mind that one of the challenging things in this section is keeping track of what each variable actually is.

To be honest with you, when I first encountered this code, I found it to be quite boring and overwhelming. But in fact the code is quite interesting, so I encourage you to stick with it. Put an honest amount of effort into understanding each step, and it will become an interesting problem to solve if you are the type of person that likes to code. Plus, it's a very cool application of NLP, so if you want to train your own question answering system on a custom data set, this is something you'll have to know how to do.

2 · Exploring the Dataset (SQuAD)

So in this lecture we will discuss the SQuAD data set, which is the dataset we'll be using for this task. SQuAD simply stands for Stanford Question Answering Dataset. Now, as you already know, the task of question answering is still constrained in a few ways. We can't yet give a neural network a big database of knowledge and just ask it any question we want. Instead, we do what is called extractive question answering. What this means is that we're going to give the network a pair of texts, namely the question and the context which contains the answer to the question.

In that way, the answer is always a substring of the context. Note also that because of this, the network never has to actually generate any text, so we don't require an encoder decoder setup. Now there are some details that will become important when you want to actually write the code. Firstly, let's look at how we will load in the data. As you can see, we just call the standard function load_dataset passing in the string squad. The data set comes with five columns which are ID title, context, question and answers.

Note that the title is pretty much irrelevant. Interestingly, ID is not irrelevant, which may seem strange since we've ignored it up until this point. We'll discuss this more when the time comes. So let's look at some examples. Here's a context. It says, Architecturally, the school has a Catholic character. Atop the main building's gold dome, is a golden statue of the Virgin Mary, and so on and so forth. The question is what is in front of the Notre Dame main building?

And the corresponding answer is a copper statue of Christ. Note that the answer seems to have a funny format. Firstly, the text is stored in a list. We'll see why that makes sense shortly. Furthermore, we see that in addition to the text, we also get the position of the start of the an-

swer in terms of characters. As you recall, a string is simply an array of characters. So if you think of the context as an array of characters, this would be the index of the start of the answer.

So for this example, the corresponding title happens to be University of Notre Dame. As you can see, this is irrelevant for finding the answer to the question. What should strike you as interesting is that the answer column is plural. This implies that there can potentially be multiple answers to the same question. This also explains why the answer data is stored in lists. Now, how can this be? Well, consider the question where did Super Bowl 50 take place?

One possible answer is Santa Clara, California. This is a true fact. But another possible answer is Levi's Stadium. This is also a true fact. So how can one question have multiple answers? Well, here's the context where this came from. It says The game was played on February seven, 2016 at Levi's Stadium in the San Francisco Bay area at Santa Clara, California. Depending on how you interpret this question, both answers would be valid. So this is an example of where the same question can have multiple valid answers.

Now, oddly, this data set is built such that for some questions, the exact same answer can appear multiple times. I'm not sure why that is. Finally, note that this only happens for the validation set for the train set. Although the column is called Answers. There is only one answer per sample. Since our neural networks loss function is only built for one target per input, this is a good thing since it means we don't have to do any extra work to split up multiple answers into separate training samples.

3 · Exploring the Dataset (SQuAD) in Python

In this lecture, we will begin looking at our question answering notebook. As usual, we begin by installing transformers and data sets. The next step is to import the function load_dataset. We then call this function passing in the string squad. We'll call the result raw_datasets. So note that the data set already becomes split into two parts one for train and one for validation. The train set has about 88,000 samples, while the validation set has about 10,000.

Note that each data set it comes with the columns ID title, context, question and answers. So far in this course, we haven't had much a need for the ID column, but in this section it will be crucial. On the other hand, we won't be using the title. Let's look at one of the titles anyway just to see what it is. As you can see, it says University of Notre Dame. Now let's look at the context for the same sample. As you can see, it's the same context I showed you in the previous lecture.

Now let's look at the corresponding question. So it says what is in front of the Notre Dame main building. Finally, let's look at the answers. So the answer is a copper statue of Christ. Now, as you've seen, the answers are stored in a list and the column is the plural answers indicating that there can be multiple answers per input. I stated that although this is always possible, it is not the case for the train set. So in this line we simply check how many samples in the train set.

Have a list of answers with any length, not equal to one. So as you can see, after we apply this filter, we find that zero samples have an answer list with length and not equal to one. This means that for the train set, every input has precisely one answer. The next step is to check out one of

the answers from the validation set. As you can see, this particular sample has three answers. So for the validation set, it is possible for one question to have multiple answers.

Let's now check the corresponding context to see why this question may have multiple answers. So we find the text. The game was played on February seven, 2016 at Levi's Stadium in the San Francisco Bay area at Santa Clara, California. This explains the three possible answers we saw above. In fact, there are probably even more valid answers. For instance, we could say San Francisco Bay Area or simply Bay Area. Although these are not in the data set. Now just for completion sake.

Let's check out the question. As you can see, the question is, as expected, where did Super Bowl 50 take place? Now, one weird aspect of this data set is that for some cases where there are multiple answers, the answers are actually all the same. Let's check the first sample to see an example. So as you can see, we get Denver Broncos three times. Now, you might think that they could be different answers if the same string shows up multiple times in the context.

But in this case, we see that the answer_start index is the same in all three cases.

4 · Using the Tokenizer

So in this lecture we will move on to the next step of our question answering notebook. Now, as you recall, typically after we load in the data, we apply the tokenizer to convert the text into a numerical format for the neural network. At a high level. That seems simple, but as you've seen many times already, the devil is in the details. Firstly, let's note that the checkpoint will be using for this notebook will be either BERT or DistilBERT. This might surprise you since question answering seems like a pretty complicated task.

So you think it might require an encoder decoder setup? But this is not the case. This will be easier to understand once we discuss the model outputs and perhaps some theory about how transformers actually work. For now, it suffices to know that BERT and BERT-like models are the correct choice for the task. This means that we'll be using the same tokenizer that we used earlier in the course. As you know, this tokenizer can already handle two input sentences such as you saw in the textual entailment task.

Okay. So just to review what happens when we use the tokenizer to tokenize two pieces of text and how do we do it? Well, you'll recall that this works by simply passing in the two pieces of text as two separate arguments by convention will pass in the question first and then the context. If we decode the outputs from the tokenizer, meaning turn the token IDs back into words, we will get a big, long string containing both the question and the context

concatenated together. In particular, this always starts with the special CLS token, followed by the first sentence followed by the SEP token followed by the second sentence. And finally, one more SEP token. Now, please note that I'm using the word sentence loosely. In fact, the context may contain more than one sentence. In fact, it likely will most, if not all of the time. Now, one challenging aspect of question answering is that the context part of the input can be really long.

This is unlike the textual entailment or next sentence prediction examples, since for those the two inputs are just two actual sentences. In this case, the context will contain many sentences. Now, as you recall, BERT can only handle a limited number of tokens, but it wouldn't be a good idea to truncate the context, since the part we leave out may contain the answer to the question. And it also wouldn't be a good idea to truncate the question. Since then, we wouldn't know the question.

So what is the solution to this? The solution is to split the context into multiple windows. As a result, one data sample will turn into multiple data samples, some of which will contain the answer and some of which will not. So by doing this, we introduce a case where no answer can be found. Now you might be concerned for those of you very keen students, you might notice that we have a problem when the answer crosses the boundary between these windows.

If half of the answer is at the end of one context window and the other half of the answer is at the beginning of the next context window, then no answer will be valid and the model won't really be learning the answers to the questions. We solve this problem by using overlapping windows instead. In the Hugging Face library, the amount of overlap is called the stride. So the full tokenizer call looks like this. As before, we pass in the context and the question.

The next argument is max_length where we specify the maximum length of the entire input. This includes the question and the context and the special tokens. The next step is to specify truncation equals only_second. This means, since the context is the second input text, only truncate this but do not truncate the question which is the first input text. The next argument is the stride, which defines how much overlap there is between the context windows when they are split up.

And finally we set return_overflowing_tokens to true. A better name for this would probably be overlapping tokens since this corresponds to the tokens that overlap when we set the stride to a positive number. Importantly, note that when we call the tokenizer like this, it expands the data. If we have one question in context pair, this might be converted into multiple input samples depending on how long the context is. One thing that will become important later on is how we know which sample from the tokenized output corresponds to which sample from the tokenized input.

Imagine for instance, that we enumerated the original input samples. So we have sample zero, sample one, sample two and so forth. At the output, we might have 0, 0, 0, 1, 1, 2, 3, 3, 3 and so forth. This means that Sample zero got expanded into three separate model inputs. Sample one got expanded into two model inputs and so forth. Luckily we do get access to this data when we call the tokenizer as described above. This will return a new key we haven't seen before called overflow_to_sample_mapping.

It contains exactly these integers, specifically the original input sample indices. Now there's one more important argument into the tokenizer that I've left out until now. This is the argument return_offsets_mapping. We will be setting this true now like the previous overflow_to_sample_mapping. This probably seems very random and unnecessary, but don't worry, I feel the same way. It will all come together when we look at our next task. But this is just to set up the preliminaries, so let's just assume that it's useful.

Basically, when we set this argument to true, we will get back an additional tokenizer output in addition to the usual input IDs, attention mask and so forth. This will be called the offset mapping. What this does is for each model input, it's going to give us a list of tuples. Each of these tuples corresponds to a token in the input sequence. Specifically, each tuple will contain the start and end character positions of that token. So as an example, suppose the question is where is Bob?

And the context is Bob is at home. The first tuple is zero zero, which corresponds to the special CLS token because technically this doesn't take up any space. The second tuple goes from 0 to 5 because the word where contains five letters. The next tuple goes from 6 to 8 because the word is contains two letters and so forth. Note that after the question is complete, we have another zero zero tuple which corresponds to the SEP token which does not take up any space.

After this we have the tuples for the context which importantly start counting from zero. Now again, it might be totally unclear why we would even need this information. To give you some intuition for why we might need this. Consider that when we finally want to represent the answer to our question, the answer will be given as start and end positions of each token. So, for example, tokens 2 to 4, which corresponds to the phrase at home, but in order to convert this back into a string to present to the user, we must know where these tokens begin and end.

That is what is the first character and what is the last. So hopefully that gives you some idea of why this information is needed.

5 · Using the Tokenizer in Python

So in this lecture, we will be continuing our notebook from earlier in the session. As you know, the next step will be to load up our tokenizer. In this notebook, we'll be using DistilBERT as our checkpoint, since it trains much faster than BERT. But if you're doing this at home by yourself, I would recommend trying BERT instead. [unclear] As usual we call AutoTokenizer.from_pretrained passing in the checkpoint. The next step is to test our tokenizer on a question and context pair.

We'll choose the sample at index one again. In order to understand the results, we'll take the input IDs and call tokenizer decode to get the result back in English. So as expected, we get both the question and context. Concatenate it into one string. Note that by convention we'll always put the question first. As you can see, the string starts with the special CLS token as usual. Then we have the question, which is what is in front of the Notre Dame main building?

Then we have the SEP token, then we have the context, and then it ends with another SEP. So as you've just seen, the context can be quite long. So long, in fact, that the combination of the context and question might be too long for the model to handle. The accepted strategy for this is to split the context into multiple samples. In order to do this, we need to specify a few additional arguments when the tokenizer is called. Specifically, we set max length to 100, specifying that the number of input tokens for any given sample should be at most 100 tokens long.

We set truncation to only_second, which means that we will only truncate the context but not the question. As you recall, the context is the second input string. We wouldn't want to truncate the question since then we wouldn't know what the question is. And because we split the context into multiple windows, we don't want to accidentally cut off the answer. Meaning we don't want part of the answer to be in one window and the other part of the answer to be in another window.

So in order to avoid that situation, we set the stride which in this case has been set to 50. Note that 100 and 50 are just toy values for this experiment. Later on, we'll be using different values. Finally we set return_overflowing_tokens to true, meaning we want to return the tokens in the overlapping parts of the context windows. So once we've called the tokenizer, note that it will give us multiple samples. This is because the context is split into multiple windows and it's combined with the question each time.

So after we've called the tokenizer, we're going to loop through each of the resulting samples and print the decoded input IDs. So as you can see, we get four new samples. This means that the context was split into four different parts. Note that the question has not been truncated, which is what we specified. Instead, the context in this case is just a part of the full context which we saw above. And again, note that we have the CLS and SEP tokens in their usual places.

Now you know that when we call the tokenizer, we don't just get back the token IDs. So let's check the keys of the result to see what we actually got. So in addition to the input IDs and attention mask, which you've already seen, we get another item called overflow_to_sample_mapping. So let's print this out just to see what it is. So it's just a list of zeroes, which is not that informative. What I'm going to do is tell you right now what this means and then do a more meaningful demonstration in the next step.

So what does this mean? Well, as you recall, when we split up the context, we get back multiple samples. However, we may want to know what is the original sample index it came from? That's what this tells us. Since we only passed in one original sample, this is going to be zero each time because they all came from the zeroth original sample. So just to get a better feel for this overflow_to_sample_mapping, we're going to call the tokenizer once again, but this time on the first three samples instead of just one, we're also going to add one additional argument return_offsets_mapping, which has been set to true.

Again, we print out the overflow_to_sample_mapping results. So hopefully these results make more sense. As you can see, all of our input samples have been split into four parts. This tells us that the first four outputs correspond to the original sample at index zero. The next four outputs correspond to the original sample at index one, and the final four outputs correspond to the original sample at index two. Now to really confirm that this is the case, we can also decode the input IDs.

So as you can see, this confirms that each of the first three samples really was split into four parts. This is why we see the same three questions repeated four times. So for the next step, we're going to go back to looking at a single context question pair with the same arguments. As you recall, we just added a new argument called return_offsets_mapping. This will return something that will be very useful throughout this notebook. We'll follow the tokenizer call by checking the keys of the output once again.

So as you can see, we get one new output, which is called offset mapping. So let's take a look at what this offset mapping actually is. So as you can see, it appears to be a list of lists of tuples. Note that I've also made a note in the comments describing what you see. So check that out as well if you need. Basically these tuples show us the character positions of each token in the model input. As you recall, the model input can really be conceptualized as a string, and a string is really an array of characters.

So if we think of the input as an array of characters, we can describe where each token is located by its positions in that array. Note that special tokens like CLS and SEP always have the tuples zero zero. Otherwise, consider the question for this input. Recall that it's what is in front of the Notre Dame main building. The first tuple other than CLS is zero four. This makes sense because the first token of the question starts at position zero and it's four characters long, as in the word what has four characters in it.

The next tuple is five seven. This makes sense because the next word which is is starts at the next position and is two characters long. The next tuple is 8, 10, which also makes sense. The next word is in which is two characters long. The next tuple is 11, 16, which again makes sense. This corresponds to the word front, which is five characters long. So hopefully you get the idea. Please feel free to go through the rest of the words in the question to make sure that this makes sense to you.

Note that the final token is just one character which corresponds to the question mark. Another important detail to notice is that when we look at the context, the indices start back at zero. This will be useful to know for any computations you want to do later on. Now, as you recall, although we only have one question and context pairs input, the tokenizer still produced multiple samples because the context was split up. So let's have a look at the offset mapping corresponding to the next sample.

So here we can see that it goes to the next sample because a new list starts. Now, as you can see, it starts at the same. We have zero zero for CLS. Then we have other tuples for the question, which again is what is in front of the Notre-Dame main building question mark. We again have zero zero for SEP. But then we have something interesting. The first context tuple starts with 174 180. Importantly, it does not start from zero. What this means is that these tuples tell us the positions of the characters in the original full context.

Again, these facts may seem kind of random, but you'll need to know them in order to properly understand the code. Later on in this notebook. Now, just to make things more concrete, let's check the size of the offset mapping list. So as you can see, it contains four elements because the original question context pair was split into four different model inputs. What may also be useful is checking the length of the elements inside the parent list. As you can see, it contains 100 tuples since we truncated each input to be size 100.

Now you should note that not all of the inputs will be of this size. As an exercise, try to find one that isn't and think about why.

6 · Aligning the Targets

So in this lecture, we are going to continue on to the next step of our notebook. The next step is a very important step, and it will help us understand why we needed the information we retrieved from the tokenization. Since it will be required in order to get this job done, this lecture will start off with more preliminary, seemingly random steps. So it'll be good to just talk about what we are trying to accomplish. As you recall, we just learn how to expand our inputs into multiple context windows with some maximum size.

This is to ensure the inputs actually fit into the transformer we want to use and that the answer can actually be found in at least one of these context windows. The problem is this, as you recall, for any given answer, it has some starting position in the context given the data set. But after we split the context up into multiple parts, this is no longer valid. As such, we need to recompute where the answer actually is after expanding the context. So that's what we are trying to do at a high level.

Remember that if we do end up splitting the context into multiple windows, we will get multiple training samples out of one context. Question and answer triple. Of course, if we change the context by doing this splitting, we will also change the position of the answer, which is essentially the neural network target. As such, this lecture is really all about recomputing the targets after we split the context. Recall that for some of these context windows, the answer will simply not exist or may only exist in parts.

Now again, I want to mention a one more time that this lecture will begin with some seemingly random preliminary steps. Just assume that they are useful as we go along and wait until the end to see how it all comes together. Now, before we get into the details of this lecture, I want to give you another high level picture of what we are actually trying to do. This will help you understand how we actually train the neural network to perform this task.

Basically, you can imagine a neural network with a bunch of output nodes, one for each time step. Suppose that we treat question answering as a classification task. So all we really want to do is pick the correct time step for the answer. So we'll have one class per time step. If we have T time steps, then we have T classes. Since we need to choose a whole range of values, it will be convenient to simply find the start and end. Therefore, in fact, we will have two sets of output nodes for each time step.

Our target class will simply select a time step. In other words, if the answer exists between token five and token ten, then our target will be five for the first set of nodes and ten for the second set of nodes. In other words, our goal in this lecture is to determine the start index and end index of the answer, where both of these will be some integer between one and T, where T is the sequence length. Note that in code we start counting from zero, but for simplicity we don't show this.

So to get back to the details. Well, first note that there is an important function. You can call on the tokenizer outputs called sequence IDs. This takes in an integer corresponding to the input sample. As usual, we count from zero, one, two and so forth, just like array and list indices. The

output is a list which looks as follows. Firstly, there are nones wherever there is a special token in the tokenized input. There are zeros which correspond to the tokens from the first sentence, and there are ones which correspond to the tokens from the second sentence.

Now, you might be wondering, isn't this pretty much the same as the token type IDs that we saw earlier in the course? The answer is yes. However, recall that token type IDs are not present in all types of models such as DistilBERT. Therefore, we can't depend on it. Instead, this sequence ID function can always be called no matter which model we use. Another thing you should notice about this is that this is a great example of the weird naming scheme.

Unfortunately, the phrase sequence IDs is so generic and so ambiguous that it's impossible to understand what this function does by its name alone. I mentioned this in the intro and you are now seeing this firsthand. Too many things in this library are called ID and too many things have very generic and ambiguous names. However, it's important to recognize it and be aware of it so that you can work around it. Now using this list of sequence IDs, we know where the context is contained.

However, this is only conceptual. Suppose that in the code we actually need the positions of where the context starts and ends. That is to say, we need the index of the first one and the index of the last one. This is a nice coding exercise, so if you'd like to pause the video and try it yourself before moving on. I would recommend you do. Otherwise, let's continue. So in order to find the first one, there is actually a handy method in Python called index which belongs to the list object.

This takes in an argument which is the element you want to find and returns the index of the first occurrence of that element. So this will give us the start of the context. In order to find the final one, we can use the same method, but we need to do some math. Firstly, recall that we can reverse a list using this weird syntax, the double colon followed by a negative one. Now we can use the index method again, which will return the position of the final one.

But for the reverse list, in order to get the corresponding position in the reversed list, we need to subtract it from the length of the list. We also need to subtract another one at the end. So this operation will give us the index of the final one, which corresponds to the end position of the context. The next question to consider is this Given the context window as a string, how do we determine the position of the start and end characters corresponding to the answer?

Furthermore, how do we determine whether or not the context window even contains the answer to begin with? This is where the offset mapping from earlier comes into play. Firstly, let's suppose we have the full context. In this case, we know the position of the start character since that is given in the data set. We don't know the position of the end character, but we can compute it simply by adding the length of the answer to the start position. This will give us the start and end if the context is not split up.

Now suppose that we do not have the full context, but only context windows. Let's break this problem up into parts and consider how we might determine that the answer is not contained in the context window. Corresponding to each context window. We have two important pieces of in-

formation, specifically the offset mapping and the token positions of the context start and end. Recall that the offset mapping is a list of tuples, one for each token containing the character positions of those tokens.

From calling sequence IDs. We also have the start token and end token so we know where the context starts and ends. Note that these line up. If we index the offset mapping at the context start, we will get the tuple containing the start and end positions of the first token in the context. The first element of this tuple will give us the first character of the context. Now suppose we index the offset mapping at the context end. If we do this, we will get the tuple containing the start and end positions of the last token in the context.

The second element of this tuple will tell us the last character of the context. So in summary, we're getting the beginning character of the beginning word and the last character of the last word. Now all we need to do is check these against the answer start character and the answer end character. So if the context window start character position is larger than the answer start character position, then the answer is not fully contained in the context.

This means that the answer starts before the context. The other situation we have to consider is if the answer goes beyond the end of the context. In this case, the context window end character position will be less than the answer end character position. So if either of these two cases are true, then the context does not fully contain the answer. In this case, we tell the neural network that the start position and end position of the answer are both zero.

This is a special code we will use when the answer does not exist. The other case we need to handle is when the answer is contained within the context. In this case, we need to actually find the start and end positions of the answer. Note that these are shifted now because the context window can start from anywhere with respect to the full context. Now, one thing to keep in mind is that this gets pretty confusing because we are essentially dealing with two different types of units at the same time.

Specifically, we have two types of indices. We have indices that index tokens and we have indices that index characters for the neural network target. We need to find the index of the start and end tokens because the neural network works with tokens. It doesn't know about characters. However, the answer in the data set is given by the position in characters. As such, we need to convert from character index to token index. To do this, we will simply loop through every tuple in the offset mapping.

This is essentially a naive linear search through every tuple to find where the character positions match and then return the corresponding indices. Before we begin, we initialize a counter variable i to context_start. Note that we do not want to start from zero because we already know where the context begins. Furthermore, we don't want to accidentally get one of the tokens from the question. Then we enter a loop that loops through every tuple in the offset mapping, starting at the context.

Inside this loop, we begin by splitting the tuple into the start and end character of the current token. Then we check whether this start character is equal to the start character in the answer. If it is, then we know that this token is the start token for the answer. So we set start_idx to i. So we

just went from start character to start token. Once we found this, all that remains is to find the end token as well. To do this. We simply have another if statement that checks whether the end character of the current token is equal to the end character of the answer.

If this is the case, then we know that this token is the end token for the answer. So we set end_idx to i. Once we found this, we can break out of the loop. Since we know that both start_idx and end_idx have been found. At the end of the loop, we increment I so that I always contains the current position in the list. Alternatively, you could use enumerate. So once we've written this code, it will be useful to check that it actually works. In order to do this.

What we can do is index the input IDs by start_idx up to end_idx plus one. This will give us the token IDs of the answer. In order to convert this back into English text, we simply call tokenizer decode. Now the next little baby step will be to put our above code into a loop. As you recall, at a high level, our eventual goal is to tokenize whole data sets. But in order to do that, we need to split the input samples into multiple samples each. if their context needs to be split into multiple windows. However, this will necessitate recomputing the targets for each of the expanded samples.

In this block of code, we'll perform the above code again, but this time for every context window instead of just one. We begin by initializing lists to store the start and end values for each sample. Recall that we'll have one pair of indices for each context window. The next step is to loop through each offset mapping. We'll use enumerate so that we can get the loop index as well. Inside the loop, we begin by getting the sequence IDs for the sample. Now, if you're like me, you've already forgotten what sequence IDs even means.

Recall that this is a list like the token type IDs, which contains zeros in the positions that correspond to the question and ones in the positions that correspond to the context. The next step is to search through the sequence IDs in order to get the context, start and end indices. The next step is to find start_idx and end_idx. Suppose we encapsulated everything we wrote before into its own function called find_answer_token_idx. After we obtain start_idx and end_idx, we just append these to their corresponding lists.

7 · Aligning the Targets in Python

So in this lecture, we'll be continuing our notebook and putting together the pieces we discovered in the previous steps. The next sort of arbitrary step is to check out this function called sequence IDs. This takes in an argument which specifies which input we'd like to look at. As you recall, there are four since the context was split up into four parts. So as you can see, this gives us a list which is reminiscent of the token type IDs we saw earlier in the course.

Importantly, we cannot use the token type IDs because they are not present in all models such as DistilBERT. To describe what we are seeing. This is a list that has the same length as the tokenized input. There are nones which corresponds with the special tokens like CLS and SEP. Zeros denote the tokens which are part of the question and ones denote the tokens which are part of the context. We'll be making use of this information later on in the notebook.

So just as a quick reminder, let's print out an example answer and understand why this is an issue. As you can see, the answer is represented with two pieces of information. First, we have the start index, and second, we have the actual text. The problem is this when we train our neural network, we need to have a target for each input. This target should point out the position of the answer within the context. The problem is that this representation of the answer doesn't work because it's relative to the original full context, not the actual model inputs where the context has been split up.

Furthermore, there may be some instances where the answer is not present. And finally, note that this answer is presented in terms of character positions and not token positions as required by the neural network. As such, the goal of this lecture is to basically figure out how to take the information we've gathered and convert it into proper neural network targets. Now, in order to do the next step, it'll be useful to know the type of the sequence IDs that we got back above.

So as you can see, it is a python list. So now that we know that the sequence IDs are represented as a list, we can use this to do the next step. We'll first begin by setting the sequence IDs from the first sample to its own variable, simply called sequence IDs. Our goal in this block of code is to figure out the indices of where the context starts and where the context ends. As you recall, the sequence IDs are formatted such that there are ones for every token.

That is part of the context. Therefore, the start of the context is simply the index of the first one. And similarly, the end of the context is the index of the last one. The challenge is how to find these indices. Luckily, since we know that the sequence_ids variable is a list, we can make use of python list functions such as index. This will tell us the location of the first one, which is the start of the context. So we'll call this variable ctx_start.

In order to find the final one, we can still make use of the index function, but we need to do a little math. Intuitively, we just need to flip the sequence IDs, find the first one in the flipped version and subtract the results from the length of the list. In detail, we first use the double colon minus one syntax to reverse the sequence IDs. Then we call index passing in one to find the first occurrence of a one. But in the flipped sequence IDs, we then subtract this from the length of the list and also subtract one.

This will give us the index of the final context token. Okay. So as you can see, the result is 13 and 98. This makes sense since the final token at position 99 is just a SEP. So the next thing we're going to do is the most challenging part of this lecture. Currently, we know the character positions of the answer within the full context. But in order to get the neural network targets, we want to know the token positions of the answer within the current context window.

So there are really two transformations happening here. First, we are going from character positions to token positions. And second, we are going from full context to partial context of the current input. As you recall, we're currently just looking at the first input sample for simplicity, but later we'll put this in a loop so that we can do the same thing for every sample. So let's begin by grabbing the answer start character and the answer end character.

As you recall, we've currently stored the answer data in a variable called answer. For the start character, this is simply stored in the key answer_start. We also index the result at zero since the actual index is stored in a list. For the end character, we simply need to add the size of the string to the start. In order to get the size of the string, we use the len function. Again, the answer itself is stored in the text key and we index the result at zero since it is stored in a list.

The next step is to grab the corresponding offset mapping, which we will simply call offset. Note that we grab the offset mapping at index zero. Since as you recall, we are currently only looking at the first input sample. The next step is to initialize the two variables we eventually want to return, which are start_idx and end_idx. We'll set these to zero since this is the default value for the target when no answer exists. The next step is to check whether or not the answer is fully contained within the context.

If it's not, then we can set start_idx and end_idx to zero, so we just return zero zero. So how do we actually check whether this is the case? Basically, we need to check whether or not the answer goes out of bounds, the bounds being the start and end characters of the context. There are two cases that would mean we are out of bounds. Case number one is that the answer start comes before the context start. And case number two is that the answer end comes after the context end.

And remember, we don't have the whole context. We only have a context window since we truncated the context and split it into multiple samples. Luckily we know how to find the context start and end positions since they are located in the offset mapping. As you recall, the offset mapping stores tuples containing the start and end characters of each token. Luckily, we also know which token is the start of the context and which token is the end of the context.

Since we stored these in ctx_start and ctx_end, the start of the context is the first character of the first token. In order to get this, we index the offset mapping at ctx_start, which returns a tuple telling us the start and end of the first token. Indexing this tuple at position zero will return the start. We do a similar operation for ctx_end, except that we check the tuple at position one, since that returns the end of that token. And finally, note that if this expression returns true, then the answer is out of bounds.

So there is nothing to do since start_idx and end_idx are already set to the correct values which are zero and zero. The else block covers the more complicated case where we have to actually find the position of the answer. Remember that we know where the answer is already in terms of character positions, but we do not know where the answer is in terms of token positions. The real trick here is keeping track of which variables are in units of tokens and which variables are in units of characters.

So in order to find these token positions, we're basically going to do a linear search through the offset mappings. Since the offset mappings contain the character positions, we just need to find where they match with the answer character positions we obtained above. We begin by setting a counter variable called i to ctx_start. Note that we shouldn't start looping from zero, since as you recall, the input contains both the question and the context and the counter starts at zero for both.

So we don't want to accidentally choose a position in the question by mistake. The next step is to loop through each tuple in the offset mapping again starting from ctx_start. Inside the loop we separate the tuple into start and end. Remember that these are the start and end positions of the current token. Then we simply need to check whether either of these match the answer's start and end. We begin by first checking the start. If the start of this token is equal to the start of the answer, then we know that this token is the start token of the answer, and thus we set start_idx to be the current token index, which is i.

At this point, you may want to break out of the loop since we found what we were looking for, but remember that we still need to find the end token as well. So in the next if statement we check the end position. If the end of this token is equal to the end of the answer, then we know that this token is the end token of the answer, and thus we set end_idx to be the current token index, which is i. At this point we can break out of the loop since the end token must come after the start so we can assume that the start has already been found.

Finally we increment i by one so that i always points to the position of the current token. And after we exit the loop, we print out start_idx and end_idx. So as you can see, we get 53 and 57, which is not too informative. The next step is to actually check whether or not our calculations are correct. To do this, we'll get the input IDs for the sample at index zero. Then we'll index these input IDs at the start_idx and end_idx we just found. So again, it's just a bunch of numbers.

Luckily, these are numbers that we can convert into text by calling tokenizer.decode. As you can see, this ends up giving us the correct answer, which is a copper statue of Christ. The next step in this notebook is simply for convenience. We're going to take all the code we just wrote and put it into a function so that we can call it later. This function is called find_answer_token_idx, which hopefully makes sense. As input, it takes in ctx_start, ctx_end, start_char and end_char and the offset mapping.

So it assumes we've already computed or retrieved these values. We won't describe what's inside this function since it's the same as above, except that when the target is zero zero, we won't print anything since we don't want this to be printed out later. So the final step in this lecture is to do what we just did again, but for all input samples instead of just one. As you recall, the context was split up into multiple parts. So even with just one context and question pair, we have multiple samples to operate on.

We begin by creating lists which will be used to store all the start_idxs and end_idxs we find. The next step is to loop through every offset mapping. We also use enumerate so that we get the index as well. Inside this loop, we grabbed the sequence IDs. Recall that the sequence IDs are like the token type IDs where they contain zero for the positions of the first input string and one for the positions of the second input string. In any case, this code is just a repeat of the above.

Same as above. We find ctx_start and ctx_end using the same code. The next step is to call the function find_answer_token_idx. This will give us back start_idx and end_idx. We then append start_idx and end_idx to their respective lists. Once we're done, we print out these lists, which, as a reminder, represent the targets for our neural network. So hopefully these results make sense. Notice how the answer appears at 53 to 57 for the first window and at 17 to 21 for the second window.

This makes sense because recall that there is a stride when we build the context windows, so there will be some overlap. And it turns out that for this example, the answer is positioned such that it appears in both context windows. However, for the third and fourth context windows, the answer does not appear, so the target is zero zero in these cases. Now just as a fun thought exercise. Here's something to consider. It is possible for the answer to show up in three context windows.

You might think this is impossible because our window size is 100 and our stride is 50, meaning that there is a 50% overlap between windows. And if that is the case, then the third window should not overlap with the first at all. Think about why this logic does not hold and what the mistake is.

8 · Applying the Tokenizer

So at this point, we have everything we need in order to create our tokenized function, which we will then apply to each sample in our data set using map to get the model inputs. As you recall, this is the next step if we follow the same pattern as the previous sections in the course. For this notebook, we will end up writing two separate functions to process the train and validation sets. The reason for this is when we evaluate our model, we're going to evaluate it based on the string version of the answers.

But the string versions of the answers are already given in the data set, so there is no further processing to do. We'll see exactly how that works later on. So firstly, we'll be setting max length to 384 and stride to 128. You might be wondering why we would set max length to 384 if BERT can handle more tokens than this. The answer is that this is what Google used in their experiments. So we will follow what they did. Inside the tokenize function. We'll begin by noting that some questions in the data set are poorly formatted with leading and trailing whitespace.

So the first thing we'll do is strip the whitespace from each question. The next step is to call our tokenizer on our batch. Note that we'll be adding one extra argument which sets padding to the string max_length. In other words, we'll be padding all input samples to have the same length as the maximum sequence length. This is okay in our case, since we'll be truncating every input sample and splitting up large contexts. So we don't necessarily need to pad per batch and hence we won't need the data collator since many samples will reach this maximum length.

The next step in our function will be to pop the offset mapping column and the overflow_to_sample_mapping column, since these won't be needed by the model. Note that a better name for overflow_to_sample_mapping is simply original sample indices, since that's what they are. Hopefully this will help the code be more readable. We also store the answer in a variable called answers, and we create empty lists to store the start indices and end indices for each expanded input sample.

Once we've done this, the rest of the function is essentially a copy of what we already saw. In any case, it'll be helpful to review it once again. We begin by looping through each sample's offset mapping. Remember that our goal now is to find the start token index and the end of token index

for each sample. These are used as targets for our network. We need to compute these manually because the data set only provides the answers in terms of character positions and the actual answer text.

So inside the loop, we begin by grabbing the original sample index. This part is new and it's important to understand. As you recall, after passing the data through the tokenizer, we expanded the number of samples. So multiple output samples correspond to the same input sample. This is important because they all point to the same answer, but the answer is stored in our unexpanded original inputs, so we need some way to map back to those. Luckily, we can do that because we've stored the original sample indices by indexing orig_sample_idxs at position i, we get back the original sample index.

From here we can index the answers list at this sample index to get the correct answer. And from here we can grab the answer start character position and the answer end character position as we did before. We also grab the sequence IDs as we did before. Note that this uses the index i because it looks at the tokenized output. Again, recall that the sequence IDs are just a list of zeros and ones where zeros represent the question and one represents the context.

From this, we can find the start and end token positions of the context. After this, we call our function find_answer_token_idx to get the start token index and the end token index for the target. Once we have these, we append them to their corresponding lists. Once we finish populating these lists, we then assign them to new columns called start_positions and end_positions. Note that we must use these names since they will be used by the trainer later on.

Lastly, we return the tokenized inputs objects. Okay. So once we have our tokenized function for the train set, we can pass it through the map function as usual. We can also take this opportunity to remove the original columns since they are no longer needed. The next step is to build a tokenized function for the validation set as well. Note that this will be a bit simpler because there is no need to compute any targets. Instead, when we evaluate our model, the metrics will be computed on string answers, which we already have from the data set itself.

One important step, which is not obvious at this point, is to understand the role of the ID field. We haven't needed this in the course so far, but if you print out one of the samples, you will see that this ID field is present. It's basically just a unique string ID that identifies each sample. Intuitively it makes sense why we would need this, since if the tokenization expands the inputs into multiple samples, the model inputs will no longer align with the original answers.

So we need some way to match up the true answers with each of the predicted answers. The first few steps of the tokenize function for the validation set are the same as for the train set. We begin by stripping white space from the questions. The next step is to call the tokenizer, as we did before, by padding everything to the max length. We will also pop the overflow_to_sample_mapping, but we will not remove the offset mapping like we did before. This will actually be kept because we're going to make use of it when we compute our metrics.

The next step will be to create an empty list, which will be used to store the ID of each sample. Recall that these are the IDs that we looked at on the previous slide. Unfortunately, this is yet another instance where we use the term ID in this section, so hopefully you're keeping track of

which ID is which. The IDs we are looking at now are unique strings that identify each sample. So in this case, ID is more appropriate. So the next step is to loop through the range of inputs.

We don't actually need the inputs themselves, just the index. Inside this loop, we first grab the original sample index. We then index the original sample and grab its ID. Now we know the original sample ID for the sample at position I in the tokenized output. We append this to our list of sample IDs. Additionally, we are also going to modify the offset mapping field since as you recall, I just said that we will be making use of this when we compute metrics.

We begin by grabbing the sequence IDs at position i. Again, just as a reminder, this is a list which contains zeros at the positions for the question and one at the positions for the answer. The next step is to grab the offset mapping at position i. We then loop through each offset mapping and replace any item that does not belong to the context to none. Now that probably sounds pretty complicated, so it helps to visualize what we are doing. In fact, you've probably forgotten what offset mapping even is, since I've done that myself many times.

Recall that offset mapping is a list containing the tuples of the start and end character positions of each token. Our goal right now is to modify this list to replace any tuple that does not pertain to a context token with none. Again, this might seem like a completely random operation, but it will be useful when we compute metrics later on. Finally, once this loop is complete, we assign the sample IDs to a new column called sample_id. Then we return the tokenized model inputs.

As with the train set, once we finish defining the tokenized function for the validation set, we will apply it to every sample in the validation set by using the map function. So to summarize this lecture, we just implemented the tokenized functions for both train and validation, which we will subsequently map to each sample in our data sets.

9 · Applying the Tokenizer in Python

So at this point in our notebook, you may have realized we are still not yet done working with the tokenizer object. We've done some experiments to demonstrate what we want to do, but we have yet to apply it to all of the samples in our data sets. In this lecture, we are finally ready to define our tokenized functions and actually run them to process the data to prepare them for the model. We'll begin by demonstrating that some questions are formatted poorly and have leading or trailing whitespace.

As a test, we'll look through the first 1000 samples and if the stripped version of the question is not equal to the question itself, then we know that there's extra whitespace on the ends. So as you can see, we find six questions that have leading or trailing whitespace for some reason, all involving Beyonce. So at this point we are finally ready to create our full tokenized function for the training data. Note that we'll be treating the train and validation data differently since we won't be evaluating the model in terms of its ability to classify the start and end positions correctly.

This would probably give us a misleading perspective of how good the model is. For instance, if we're one token away from the true target, this could be just as bad as being 50 tokens from the true target. Furthermore, recall that there can be multiple answers in different positions. In any

case, we'll discuss more about how we evaluate the model later. But for now, it suffices to know that we only need to do all the prior computations on the train set.

We'll begin by setting the actual maximum sequence length and stride to 384 and 128. The next step is to define our function called tokenize_fn_train, which takes in a batch of raw data. Inside the function we strip away the whitespace from the questions. We then call our tokenizer using the same arguments as we had above. However, we also add one new argument which is for padding, which we set to max length. In previous sections, we've used the data collator to pad the data dynamically for each batch.

But for this example, that's less important, since the contexts will be relatively long. The next step is to grab the offset mappings and the original sample indices from the processed data set. As you recall, overflow_to_sample_mapping is a pretty terrible name, so hopefully original sample indices makes more sense. Note that we pop these values, which means not only do we retrieve them, we also remove them from the inputs object and we want to remove them since they are no longer needed.

The next step is to grab the answers, which come from the original raw data set. We also initialize start_idxs and end_idxs to be empty lists. The next step is to enter a loop to loop through each of the offset mappings for each input sample. Note that this loop is essentially the same as what we wrote above, so we'll go through this quickly. Inside the loop. And this part is new. We begin by grabbing the original sample index, which we will call sample index.

So why do we need this? Well, recall that the answers come from the original raw data set, not the expanded data set from the tokenizer. Therefore, in order to grab the correct answer, we need to know the index of the answer from the original dataset, which is what the original sample index will tell us. Therefore, we index the answers object at sample index. The next step is to find the answer start character index and the answer end character index as we did above.

The next step is to retrieve the sequence IDs for the i-th sample, as we did above. The next step is to find the context start and the context end, as we did above. The next step is to call our function find_answer_token_idx as we did above. This gives us back start_idx and end_idx, which are the token positions of the answer. These are the neural network targets. The next step is to append these to our lists of start_idxs and end_idxs. Once we finish the loop, we will have populated start_idxs and end_idxs.

So the final step is to add them to our inputs objects which we subsequently return. Note that they must have the keys start_positions and end_positions, since this is what will be used by the trainer later on. So now that we've finally written our tokenize function, we can apply it to our train data set by calling map. As usual, we pass in batched equal to true and remove the original columns. After we do this, we print out the length of the raw data set and the outputs.

So as you can see, the output data set is larger than the raw data set. Since we expanded the contexts that were too long. The next step is to write the tokenize function for the validation sets. But before we do that, let's look at an example input. So one thing we haven't yet looked at is the ID, which we'll be using shortly. As you can see, it's a string which contains alphanumeric characters. This will uniquely identify which original sample the tokenizer output comes from.

Since, as you recall, the tokenizer can generate multiple samples per input when the context is split up. So basically the two points to remember for the validation tokenize function are as follows. Firstly, we don't need to generate targets. And second, we're going to modify offset mapping with some useful information. Unfortunately, as with most things we've done in this notebook, it won't be obvious why this is useful until later on. So at this point, just keep this in the back of your mind.

We begin by defining a function called tokenize_fn_validation, which again takes in a batch of data. Inside the function, we again strip whitespace from the questions. The next step is to call the tokenizer using the same arguments as above. The next step is to pop the overflow_to_sample_mapping once again. And once again, we'll give this a better name, which is original sample indices. Now, you'll notice that this time we are not popping the offset mapping. This is because we'll be keeping it.

But as mentioned above, we'll be making some changes to it. At this point, we'll also create an empty list to store the sample IDs we encounter. The next step is to loop through the index of each sample. We'll call the loop variable i. Inside the loop we index the original sample indices at position i to get the original sample index. We'll use this to index the original raw data set to get the corresponding ID. Remember, that's the alphanumeric string we saw above.

We append this to our list of sample IDs. The next step is to retrieve the sequence IDs for the i-th sample. And again, just in case you forgot, because of its terrible name, the sequence IDs are like the token type IDs. They have zeros in the positions that correspond to the question and ones in the positions that correspond to the context. Basically, they help us find where the question is and where the context is. The next step is to grab the corresponding offset mapping for the i-th sample.

Essentially what we want to do is modify the offset mapping by replacing all the non context tokens with none. As you recall, the offset mapping is just a list of tuples where each tuple contains the start and end character positions of each token. For reasons that are not clear yet. We want to format this such that only the context tuples remain. So that's what this loop is doing. Notice how we use the sequence IDs because that tells us where the context is.

Anywhere we find that the sequence ID is one, we keep the tuple, otherwise we replace it with none. Once the loop is complete, we assign the sample ID to a new column called sample_id. And finally we return the tokenized output. So the final step is to apply our validation tokenize function to the validation set. And again, just for our own curiosity, we'll print the length of the data set before and after. So as you can see, for the validation set, only a few hundred extra samples were created.

10 · Question-Answering Metrics

So in this lecture we will be looking at how to compute metrics for the question answering task as usual. Since this is a standard data set for NLP benchmarks, it should be expected that there is also an associated metric for this task. As you're pretty used to by now, we will load in the metric

by calling load metric passing in the string squad. So what does this metric do? As you recall, I mentioned earlier in this section that the metric will be using does not involve any low level quantities like the position of the answer within the context.

Instead, the metric uses the actual answers themselves in the form of strings. So if the answer is Albert Einstein, then our prediction should also be Albert Einstein in order to be considered correct. Of course, this begs the question how do we convert our model predictions back into strings? That will represent the answer. This remains to be seen. For now, we'll just focus on how to call metric compute and the results we will get back. In terms of the results we get back, metric compute will return two items, the exact match score and the F1 score.

You can think of exact match like accuracy, which will return the percentage of results where the prediction exactly matched the true answer. On the other hand, F1 is more granular and looks at each token in each answer. So if the answer was general relativity, but your prediction was special relativity, you would at least receive partial credit. Also recall that for this data set there can be multiple possible true answers to a question. Therefore, the metric also takes this into account.

The next question to consider is how do we actually call metric compute? What inputs does it expect? As usual, we have two arguments to pass in which are predictions and references. Each of these should be lists of dictionaries. The prediction dictionaries will have a different format than the reference dictionaries. In particular, the reference or target dictionaries will be similar to what we have in the data set. Each of the target dictionaries will have an ID denoting which sample it pertains to.

They will also have an answers item which stores another dictionary, which is the exact same dictionary that we get from the data set. As you recall, that dictionary has two items which are text and answer start. The text contains the actual string answers, while answer_start contains the initial character index of the answer. Recall that these are both lists since each question context pairing can have multiple possible answers. The predicted answers are a bit simpler.

In this case, we only need to provide the ID and a key called prediction_text which contains a single string as the value. Note that these keys must be named as shown here since metric compute expects them to be this way. Furthermore, note that all of the items shown here are required and you will get an error if they are missing, even though they are not part of the answer. So for example, if you leave out the ID, that won't work. If you leave out answer_start,

that also won't work.

11 · Question-Answering Metrics in Python

So in this lecture we will be continuing our notebook and looking at the SQuAD metric. As usual, we'll begin by importing load metric. We then obtain our metric by calling load metric, passing in the string squad. The next step is to just see how this metric works. With some dummy predictions and targets for the predicted answers, we'll have Albert Einstein, physicist and general relativity. Make note of how these predictions are formatted. Each prediction is a dictionary and they all go in a list.

Each dictionary contains two items. Each dictionary has not just the prediction text, but also a corresponding ID. This will allow the metric compute function to cross reference. Which predictions go with which answers. For the target answers. Note that this is also a list of dictionaries. However, these dictionaries contain different items. What they do have in common are the IDs. The new field is the answers field, which contains a nested dictionary inside this dictionary.

It basically contains the same information as the raw data set. We have a text field which points to a list of possible answers in text format. We also have the answer_start field, which tells us the character start index of each answer, and that is relative to the full context. Now, oddly, this value is required if you want the function to work, even though it doesn't need to be there. Since this is just dummy data, it doesn't really matter what values I use.

As you can see, I've just used 100 for every sample. Once we've made up our predictions and targets, we can call metric compute. As usual, we set the arguments, predictions and references. So as you can see, we get two thirds for exact match and 5/6 for the F1 score. Exact match makes immediate sense because we got two out of three answers. Exactly right. F1 is a bit more complex, so feel free to work that out if you like.

12 · From Logits to Answers

So in this lecture, we are going to move on to a very important task. The task is to convert our model outputs back into a string answer so that we can call metric compute. And obviously this is not just so we can call metric compute, but so we can actually read the answers as well. It wouldn't make sense to have a question answering system but then not be able to read the answers. This is our second major task for this section, the first being how to process the model inputs.

Now in order to test our code and see some actual results immediately. We're not going to wait until we've trained our model. Our approach will be as follows We're going to download a pre-trained question answering model and use that to get model predictions that are not just random. Well then use the predictions of this pre-trained model to learn how to convert the results back into answer text. Once we figured out how to do this for a small subset of our data set, we can then apply it to the full data set after our model has been fine tuned.

Okay. So this is a preview of the code we'll be using. We'll begin by grabbing just the first 100 samples of our validation sets. We'll then define a new checkpoint for this trained model. Since this is a different model, we'll need a different tokenizer. However, our functions above use the tokenizer variable as a global variable, and as such, we'll need to overwrite the tokenizer object temporarily and change it back later so we save our current tokenizer in a variable called old_tokenizer.

We then assign the variable tokenizer2 to the existing variable tokenizer. Remember, this is because the function we wrote will access a global variable with this name. We then map the tokenized validation function to the mini data set and we'll call this small_validation_processed.

Lastly, we reassign old_tokenizer to tokenizer, so we're changing it back. The next step will be to get our model outputs, whatever they may be. First we import torch and of course, we'll need a model class specifically built for this task.

It's called AutoModelForQuestionAnswering. The next step is to remove columns that the model doesn't use as input, such as sample ID and offset mapping. The next step is to get the GPU device, since this all must now be done manually. Now, as you recall, the data set acts like a dictionary. So the next step is to loop through each of the columns of the tokenized data set that we still have left and move them to the GPU. This will return another dictionary, but this time with all the data on the GPU.

The next step is to download our trained model and to also move that to the GPU. Finally, we call our trained model passing in the inputs to get the model outputs. We use torch.no_grad since we won't be using the gradients. Now, if you print out the model output, you'll see that it's an object of type question answering model outputs. This object has two attributes which are start_logits and end logits, both of which point to PyTorch tensors. So this should give you some idea of how this model works.

Suppose that we have a batch of N samples, each with a padded sequence length of t. Then we would expect both of these logits to be of size N by T. So what's the meaning of this? For simplicity. Suppose we only have one sample that is one question, one context, and one answer. So effectively the start logits will be a vector of size T and the end logits will also be a vector of size t. If we took the softmax over the time dimension, we would get the probabilities for each time step.

What we do with these probabilities will be described very shortly, but it would be a good idea to start thinking about this now. Now in order to work with these logits directly in code, it's more convenient to have them as numpy arrays. As you recall, in pytorch, we can get NumPy arrays from tensors by first moving them back to the CPU and then calling the numpy method. At this point, it's worth remembering that in our processed data set, we've stored an attribute called sample ID.

This is a unique string identifier corresponding to each question context pair. It's also worth remembering that by processing the data set, we expand the samples by splitting up context windows and as such some samples and hence sample IDs may be repeated. So if you check the length of the processed data set, it should be longer than the set of all unique sample IDs. We will demonstrate this in the notebook. Now. Why do we care about repeated sample IDs?

Well, as with many of the steps in this notebook, we just need to do some preliminary work that will all come together at the end. For now, we're just going to create a dictionary that maps each sample ID to all the possible indices of where that sample resides in the process data set. So for example, if the first sample gets split into four windows, then its sample ID would point to a list containing zero, one, two and three, which are the first four positions in the list containing all the processed samples.

This code you see here simply loops through all the sample IDs and stores the corresponding indices as values. Since there may be multiple index values per sample ID, we store them in a list. So now let's get back to our logits. At this point, we're going to tackle the big question how do we

convert these logits into an actual prediction for the answer as text? To answer this question, we're going to need to do a bit of math. Usually we save the math for the advanced theoretical sections, but in this case, it's necessary to compute the answer.

If this math is too much for you, please note that the actual computation is simple. So you can skip this if you like, but then you won't understand why your computation looks the way it does. Either way, up to you. The prerequisite for this is just a basic probability. So study up on that if you want to understand how this works. So suppose that we take the softmax of the start logits and end logits to get the corresponding probabilities. Our simplified notation for this will be p of s1, p of s2, and so forth up to p of s big T. For the end tokens,

the corresponding probabilities will be p of e1, p of e2, all the way up to p of e big T. Now we could add a given x to be more precise, but I'll assume you know that this is implicitly the case. This is just to keep the notation simpler. So p of s sub i is the probability that the answer start token is at position i and p of e sub j is the probability that the answer end token is at position j. Our goal, of course, is to find the most probable answer. That is, the values of I and J that lead to the maximum joint probability p of s sub i and e sub j.

In order to simplify this computation, we're going to make the assumption that the distributions of start and end tokens are independent. By doing this, we can factorize the joint distribution into two separate distributions, p of s sub i times p of e sub j. And obviously this is exactly what we need since this is exactly our neural network outputs. So a naive solution might look like this. Before we do anything, we initialize the best i and j to none and the best score to minus infinity.

And then we do a loop for all values of I from one up to big T, and then we do a nested for loop for all values of J from one up to big T. So we're searching through all combinations of I and J and note that since this is pseudocode, we indexed from one instead of zero, as we might do in real code. Inside the nested loop, we compute the joint probability by taking p of s sub i and multiplying it by p of e sub j. This is our score. If this is greater than our current best score, then we make this our new best score and we assign the current I and J values to be the best I and J we have found so far.

From here, we know the best start token and the best end token. So all we need to do is a few more additional steps to convert the answer back into text. However, before we do that, there are a few ways we can make this better and more correct. Firstly, note that looping through all big T squared values is a lot. Big T in our case is 384, and since this is a nested for loop where we loop through the range of Big T both times the number of iterations is 384 times 384.

This is 147,456. That's a lot of computations we need to do. And remember, we need to do this for every sample. Instead, we're going to use a possibly suboptimal heuristic and only search through the top 20 best start positions in the top 20 best end positions. As a result, we'll only need to iterate 20 times 20 or 400 times, which is about 0.3% of what we'd have to do if we did a full search. Now you might be wondering if we're assuming that the start and end probabilities are independent.

Why do we need to search at all? Why can't we just take the maximum p of s sub i and multiply it by the maximum p of e sub j. The answer is that this won't always lead to a valid answer. For instance, if I is bigger than J, then that means the start comes after the end, which doesn't make sense. Also recall that out of the big T input tokens, not all of them refer to the context. As you recall, we start with the CLS token, then the question, then the SEP token, and then the context, and then a final SEP.

In other words, there are a bunch of checks we need to do to make sure that the answer is actually valid. As such, it is not feasible to just take the maximum start and maximum end probabilities. To be more specific, there are three cases in which we will consider the answer to be invalid. The first case is when either i or j corresponds to a token outside the context. The second case is when I is greater than j, meaning that the start is past the end, which doesn't make sense.

The third case we haven't discussed yet, but this is when the answer is too long. In this case, we will assume that this is so unlikely that we will not consider these answers in the code. We'll set the maximum answer size to 30 tokens. So the final improvement we're going to make is that we are not going to work with probabilities at all. As you recall, in deep learning, we prefer to work with logits since they are more numerically stable. Furthermore, this often allows us to use addition instead of multiplication, which also tends to be more stable and more efficient.

So how do we compute the score from only the logits? Firstly, let's consider how we would write down p of s sub i times p of e sub j in terms of logits. We'll call the start logit l of s sub i and the end logit l of e sub j. If we take the softmax of the start and end logits separately, we would exponentiate the logits and divide by their sum. However, note that these sums are always the same no matter the value of i and j. This is because they are the sum over all values of i and all values of J.

We can just call them Z, sub s and Z sub E, which is the letter we would normally use for normalizing constants. As such, the argmax of p of s sub i times p of e sub j is the same as the argmax of e to the power logit s sub i times e to the power logit e sub j. This allows us to get rid of z sub s and z sub e completely. This is because z sub s and z sub e are both independent of I and J. In other words, they are constant with respect to I and J. Therefore, it doesn't matter which values of I and J we choose.

As such, we can simply remove them from the expression. The next step is to remember that taking the argmax of a function is the same as taking the max of the log of that function. This is because the log is monotonically increasing. What this means is that if A is a number bigger than B, it is also true that log A is also bigger than log B. So if we find the max of log of something, we also find the max of that something as such. If you're suspicious of this claim, I recommend picking various values of A and B and trying it yourself.

So what happens if we take the log of this function? Well, recall that the log of a product is the sum of logs. This means we can split up the product. Furthermore, recall that the log in the exponential are inverse functions, so the log of the exponential of any number is just that number. Therefore we will get the same answer if we use logit s sub i plus logit e sub j as our score. This makes the score much easier to compute as we can just add the neural network outputs directly.

So at this point we have everything we need to write some code that will take some neural network outputs and compute predicted answers from them. We'll begin by setting some variables. Specifically, we set n_largest to 20, which represents the maximum number of logits we want to consider. We set max answer length to 30, which represents the largest number of tokens that can constitute a valid answer, and we set predicted answers to an empty list. We will populate this list in the following loop.

So our loop will be through each sample in the original tokenized data set. In practice, this would be the validation set we loaded in, but for this example it will be the small subsample version of the validation set. Inside the loop we begin by grabbing the sample ID and the context from the ID and context fields for the current sample. Note that we use the context because this contains the answer as text. The next step is to initialize the best score, which will start as minus infinity and the best answer which will start as none.

Unlike the previous example, we will not bother to store the best data and we will instead skip straight to computing the answer text itself. The next step is to use our dictionary that we created earlier that tells us which expanded input samples correspond to this sample ID. In other words, we are using the sample_id2idxs mapping to tell us which of the tokenized outputs to look at. This will give us the indices into the processed samples, as you recall.

Now just at a high level. Let's consider why we are doing this. We are doing this because we need to loop through every context window. At least one of these will contain the predicted answer for which the corresponding score is highest. Some of these context windows will not contain the answer at all. So basically we're going to loop through each of these context windows to find the best answer. So inside the loop we find the start logits and end logits for this particular model input.

As you recall, the logits are of size N by T for N inputs and t time steps. This will give us a vector of size T for each of the logits. The next step is to grab the offset mapping for this particular model input. As you recall, we modified this in our tokenized function above. Specifically, it started out as a list of tuples where each tuple contain the character positions of the start and ends of each token. However, we modified it so that for any position that did not correspond to the context, we replace the value with none.

As such, this is an easy way to determine whether or not we are within the context. The next step is to call argsort on the start logit and end logit. Note that we negate these arrays before we call argsort, since by default it sorts in ascending order. Since we want the largest logits first we negate the array which will cause the sorting to be descending instead. The next step is to loop through the first n_largest of both the start and end indices.

This is basically our nested for loop from earlier in this lecture. Inside the loop, we check our three cases that result in invalid answers. The first case we check is that the answer is not contained within the context. To do this, we can index the modified offset mapping at start_idx and end_idx. Since the modified offset mapping contains none for any token not in the context, we just need to check if either of these is none. If that is the case, we say continue.

Since there is nothing more to do, as these indices do not correspond to a valid answer. The next scenario we check is if end_idx is less than start_idx. If this is the case, we continue. The third scenario we check is if the length of the answer is greater than the maximum allowed answer length. This is basically the difference between end_idx and start_idx. And if this is the case, again, we continue. Finally, if we reach this point and we haven't yet continued on to the next iteration, we can compute the score.

As you recall, the score is just the sum of the start and end logits. Then we check if this score is better than our best current score. If it is, we set a best score to the current score and we update the best answer. At this point, we actually convert the answer back into text. To do this, we use the offset mapping once again. Again, recall that the offset mapping contains tuples with the character positions of start and end tokens. As such, if we want the first character of the answer, we index the offset mapping at start_idx.

This will give us a tuple of the start and end locations of the first token. We index this tuple at position zero because that's where it starts. This will give us the first character position of the answer. Similarly to find the last character, we index the offset mapping at end_idx and we grab the second element of the tuple there. Finally, we can index the full context at the first character up to the last character, and this will give us the answer as text.

The final thing we do once we've determined the best answer for this sample ID is to save it as a dictionary to a list of predicted answers. As you recall, this is the expected format when we call metric compute. At this point we can compute the true answers as well, since they also require a special format. Specifically, we make sure that each answer is a dictionary which contains the sample ID and a key called Answers that stores the answers from the original data sets.

And finally, once we have a list of predicted answers and a list of true answers, we can call metric compute.

13 · From Logits to Answers in Python

So in this lecture, we will be continuing our notebook. As a reminder, our goal right now is to use metric compute. However, we can't use this until we are able to get the predicted answers into text format, since as you saw, that is the required input. Now, in order to develop our solution, we won't use the whole data set and we won't use our untrained model since we want to just see that it works. Instead, we'll take a small subsample of our validation set and we'll use a model already

fine-tuned on the SQuAD data set. To begin, let's create a small validation set, which will just be the first 100 samples from our full validation data set. The next step is to define a trained checkpoint, which will be DistilBERT fine-tuned on SQuAD. As you recall, we'll also need to load in the corresponding tokenizer since these are checkpoint specific. We'll call this tokenizer2. Now, one issue with this is that we use the tokenizer variable as a global variable in some of the functions above.

Instead of rewriting them or having to add new arguments. We'll just temporarily replace the existing tokenizer variable as we process the data. To do this, we'll save the existing tokenizer in a new variable called old_tokenizer. Then we set tokenizer to be tokenizer2. Next we map the tok-

enized FN validation function to the small validation data set. Once this is done, we change our tokenizer back to the old version. The next step is to pass this data through our trained model.

Interestingly, we haven't had to do this most of the time since to train the model. We just use the trainer and once training is complete, we just use the pipeline. So this should be interesting just to see how we actually use the model to get the outputs. To begin, we import torch and we also import a new kind of model, AutoModelForQuestionAnswering. The next step is to remove irrelevant columns that are not used by the model. These are sample ID and offset mapping.

We'll call the result small_model_inputs. We also set the format to torch. The next step is to get our GPU device. As you recall, when we use PyTorch, we need to explicitly move tensors to the GPU. The next step is to move our data tensors to the GPU. Recall that the model inputs acts like a dictionary. So that's why, as you can see here, we convert them to an actual dictionary. Basically what this is doing is looping through each column of the inputs, grabbing the tensor at that column.

Calling to device on that tensor and mapping it to the same key, which here is called K. This new dictionary is called small_model_inputs_gpu. The next step is to download our model from the trained checkpoint. Note that we also move the model to the GPU. Finally, now that everything is on the GPU, we call our trained model passing in the GPU inputs to get the output. Note that we use torch.no_grad because we don't want to compute gradients. The next step is to print out the outputs just to see what we get.

As you can see, we get an object of type question answering model outputs. As usual, this basically acts like a named tuple and we can see that it has two attributes which are start logits and end logits. The next step is to grab these start logits and end logits as NumPy arrays. Before we get the NumPy arrays, we must also move them back to the CPU. The next step is to remind ourselves what the sample IDs look like. As you can see, there are alphanumeric strings.

The next step is to check the length of our validation dataset's sample IDs to remind ourselves that they are not unique. As you can see, we have 10,822 sample IDs. Now to show that they are not unique. We'll put the sample IDs into a set, which is a data structure in which all values must be unique. We'll then print the length of this set. As you can see, when we make the sample IDs unique, we only have 10,570 sample IDs. Since the length of the set is less than the length of the original list, this means that some items are duplicates.

However, note that this is for the full validation set and not necessarily for the subsampled validation set. Either way, we should write code to handle the case where the sample IDs are not unique. What we need right now is a data structure that points from sample ID to all possible indices in the processed data set. That is to say, when the context is split up into multiple samples, we want to have some way of mapping to all the samples corresponding to the original sample ID in order to do this.

We begin by creating an empty dictionary which will store the sample ID as the key and sample indices as the value. The next step is to loop through all the sample IDs. We use enumerate to get the index at the same time. Inside the loop, we check whether or not the ID exists in our map-

ping. If it does not, then we create a new entry for this ID which will point to a list containing the current index i. Otherwise the sample ID already exists. In this case, we print a little message saying here and we append i to the existing list.

So as you can see, the word here never got printed out, meaning that there were no duplicate sample IDs in this list. The next step is to confirm the shape of our logits. As you recall, we expect them both to have the shape N by T where n is the number of samples and t is the padded sequence length. So as you can see, they both have the shape 100 by 384, which is what we expect. Note that the number of samples is still 100. Since none of the contexts were long enough to be expanded as we saw.

The next step is just a reminder of how to sort the indices of an array. Recall that we don't want to just sort the array because we don't care about the values themselves. Instead, we care about where the values are. For example, if time step five is the highest logit, then we want five, not the logit. This is why we call argsort instead of just sort. Note also that we negate the array first. Recall that when sorting we sort by ascending order by default.

If we negate the values first, this will effectively sort them in descending order. In this way, the largest values will always be at the front. So these are the sort indices. So let's see what happens if we plug the sort indices back into the original array. So as you can see, when we index the original array by the argsort indices, it is in fact sorted in descending order. The next step is to remind ourselves of the work we did to modify the offset mapping for the validation set.

As a reminder, since offset mapping is such an ambiguous name, we originally stored a list of tuples containing the start and end character positions of each token. We modified this for the validation set. The way we modified it was to get rid of any tuples not pertaining to the context. We set those values to none. So as you can see, we get a list containing many Nones and somewhere in the middle we start to see tuples of integers in increasing order.

These are the character positions of each token in the context which will be used in the next step. The next step is where we finally write some code to convert our model outputs into actual text answers. We begin by setting n_largest to 20. Recall that this is the number of start logits and end logits that we want to search. We also set an max answer length to 30, which is the maximum answer size we want to allow. Finally, we set predicted answers to be an empty list, which is where we will store the answers.

The next step is to loop through the original sub sampled validation set. Recall that this is tokenized inside the loop. We grab the sample ID and the context. Next we initialize best_score and best_answer. We will update these as we find better answers in the below loop. The next step is to loop through the expanded input samples. These are the samples that have been tokenized in order to retrieve the correct samples. We use our dictionary, which points from sample ID to the index of the tokenized samples that came from this sample ID.

Inside the loop. We obtain the start logits and end logits for this particular index. These are both size T arrays. The next step is to grab the offset mapping for this index. Recall that this is the processed and modified offset mapping which contains none for any position, not in the context. The

next step is to sort the logits. The next step is to loop through the n_largest start logits and the n_largest end logits. Inside the loop we check the three cases for which an answer does not exist.

The first case we check is that the answer is not contained within the context. For this, we can use the offset mapping. Recall that the offset mapping is none for any token, not in the context. So if we index the context at start_idx and it's none, then we are not in the context. If we index the context at end_idx and that is none, then we are also not in the context. In either case, we continue. The next case to check is, if end_idx is less than start_idx.

In this case, the answer has negative length, which doesn't make sense. So we continue again. The third case we check is if the answer is too long. That is longer than max answer length. If this is the case, we continue. Finally, we handle the case where we do have a legitimate answer. In this case. We start by computing the score for this answer, which is the start logit plus the end logit. If you need to review why this is the score, please check the previous lecture.

The next step is to check whether or not this score is better than our current score. If it is, we'll save this score and compute the answer. So we set best score to be the current score. Next we grab the positions of the first character of the start token and the last character of the end token. As you recall, we can use the offset mapping since this is the information it stores. Finally, we retrieve the answer as actual text by indexing the context from the range first to last.

Once the loop is complete, we save the best answer using the format we learned about above. As you recall, this is a dictionary that stores the sample ID and prediction_text. The next step is to just remind ourselves of the format of the true answers. As you can see, they are dictionaries containing the answer_start character indices in the corresponding text. So this is the exact format we need when we call metric compute. Aside from putting this into a parent dictionary containing the sample ID as well.

The next step is to format the true answers as described above. We want a list of dictionaries, each of which stores the ID and the answers formatted like how we just saw. Once this is complete, we can call metric compute. So as you can see, this is the score for the trained checkpoint. We get 83 for exact match in 88.25 for F1.

14 · Computing Metrics

So now that we know how to actually convert our model predictions into actual answers such that we can call metric compute, we are now going to write our function for compute metrics. Now normally in this course, you've seen that we can pass compute metrics into the trainer object, at which point the function would be called at the end of each training epoch. In this section, that won't be the case. Instead, we'll be calling the compute metrics function manually after training is complete.

Furthermore, we won't be following the usual API for compute metrics where the input is just a tuple of logits and labels. This would allow us to compute a validation loss, but in this case we'd prefer to look at the actual answers themselves. If you'd like to compute a validation loss based on the logits. You're welcome to do that as well as an exercise. So we'll begin by importing tqdm, which allows us to display a progress bar as our function loops through the data set.

The next step is to define our compute metrics function, which will not follow the same structure as we saw earlier in the course. As input into this function, we will take in an array of start logits, an array of end logits, the processed data set and the original data set. Recall that the process data set is the one where we've applied the tokenizer and the context. Windows have been expanded into multiple samples. Note that essentially all of this function will be a review of the previous lectures, so we'll go over this quickly.

We begin by creating a mapping from sample ID to the row index of the processed data set. This will be a dictionary called sample_id2idxs. To populate this dictionary. We loop through the sample IDs from the processed data set. We use enumerate so that we get the index as well, which we call i. Inside the loop, we check whether or not the ID is already in the dictionary. If it is not, we create a new entry where the value is a list containing the current index i.

Otherwise if the list already exists, then we append i to the existing list. The next step is to compute the predicted answers. We start by defining an empty list. The next step is to loop through the original data set using tqdm. Inside this loop we begin by grabbing the current samples ID as well as the context. The next step is to initialize the best score and the best answer. The next step is to loop through all the indices from the processed input samples that correspond with the current sample ID.

Inside this loop we grab the start and end logits at position index. We also grab the offset mapping at position index. At this point, it should be noted that the order in which you index the process data set is very important. Although both ways will work, one way is very slow and will not complete in a feasible amount of time. This is just due to how the data set object is implemented. So the better way to index the offset mapping is to first grab the desired row by using index and then grabbing the offset mapping attributes, not in the reverse order.

The next step is to sort the logits in descending order, as you've seen. Once we've done that, we can do a nested loop over the n_largest indices for both the start and end logits inside these nested for loops, we check the three conditions where no answer exists to review. This is what they are. Number one, we want to skip answers that are not fully contained within the context window. We know this is the case. If we index the offset, mapping at either start or end and it returns none.

Number two, we skip answers where the end position is less than the start position, which is impossible. And number three, we skip answers that exceed the maximum length we set above. If none of these three conditions are triggered, we compute the score for the current answer. We then check whether this score is better than our current best score. If it is, we make this our new best score and we also save the answer in a variable called best answer. We do this by indexing the context at the first character up to the last character, which we retrieve from the offset mappings.

Once we've completed the nested loop, we add our predicted answer to our list of predicted answers. And once we've populated all the predicted answers, we collect the true answers in another list in the format that metric compute expects. Finally, we call metric compute and return the result.

15 · Computing Metrics in Python

So this lecture will be pretty simple since all we're going to do is take what we did previously and put it into a function which we will call compute metrics. The important thing to remember is that this is not like the compute metrics functions that we wrote earlier in the course. We will not be passing this into the trainer and the arguments are different. We will only run this function manually after training is complete. We'll begin by importing tqdm, which will display a progress bar as the function runs.

The next step is to define the compute metrics function as input. It takes in the start logits, end logits, the processed data set and the original data set. As you recall, the process data set is the one where the context has been split up into multiple windows. Inside the function. We begin by creating a mapping from sample ID to all the corresponding indices in the processed data set. This way, given any particular sample ID, we can find all the processed samples that belong to it.

Since you've seen this code before, I won't explain it again. The next step is to create an empty list where we will store our predicted answers. The next step is to loop over the original data set. Note that we use tqdm at this point. Inside the loop. We grab the sample ID for the sample and the context. The next step is to initialize the best score to minus infinity and the best answer to none. The next step is to find all the processed samples that come from this sample ID.

At this point, we use our mapping to get all the corresponding indices in the process data set, and we loop through this list of indices. Inside the loop. We use the current index to find the start logit array and the end logit array. Recall that these are both size T vectors. The next step is to retrieve the offset mapping for this sample. One thing that becomes very important at this point is to index the processed data set in the right order. Because of the way the dataset object is implemented.

It will be very slow if you retrieve the offset mapping first and the index second. On the other hand, if you want it to go faster, then you should index by index first and the offset mapping second. The next step is to sort the logits in descending order. The next step is to loop through the n_largest logit indices for both start and end. Inside this loop, we check the three cases for invalid answers once again. To review the first case is where the answer does not fully lie within the context window.

The second case is where the end is less than the start, which wouldn't make sense. And the third case is where the answer is too long. If we reach the next part of the loop, then the next thing to do is compute the score for this pair of logits. The next step is to check whether the score is better than our current best score. If it is, we first update the best score to be the current score. The next step is to use the offset mapping to find the first character and the last character of the answer within the full context.

Finally, we update the best answer to store the answer given by these character positions. After we exit the loop, we save the best answer we found along with the corresponding sample ID as a side note. Notice that for any particular sample ID there will always be one best answer. This is because we loop through all context windows and pick the best one. Once we finish populating the predicted answers list, we then compile the true answers like we did above.

Finally, we call metrics compute to return the metrics for this task. The next step is to test our function to make sure that it works. Note that since we haven't yet trained our model, we'll just use the same values for our subsampled data set that we used above. This is good also so that we can ensure that we get the same result. So as you can see, we still get the same result. 83 for exact match and 88.25 for F1.

16 · Train and Evaluate

So at this point, we finally have all the pieces we need in order to conclude this notebook. All that's left to do is train and evaluate this model. As usual, we begin by loading up a pre-trained model, but we need to make sure that we use the correct type of model. In this case, we'll use the class AutoModelForQuestionAnswering. As usual, we get a warning that the model head has not been trained. The next step, as usual, is to create a training arguments object.

None of these inputs should be particularly surprising at this point. Note that we use FP16 in order to speed up training. The next step is to create a trainer object. Again, none of these inputs should be particularly surprising at this point. After we create the trainer objects we call trainer.train to begin the training process. Note that the training process will take quite some time to complete. If you use BERT, this will take about two and a half hours on the full training set.

If you use DistilBERT, it takes about half this time. After training. What we would like to do is call our compute metrics function on the validation sets. However, this requires retrieving the model outputs and specifically the logits. To do this we call trainer.predict passing in the processed validation set. This will give us a prediction output object. Interestingly, although it's not obvious, this object can be split into a triple. The first item in this triple has the data we need, specifically a tuple of start and end logits represented as numpy arrays.

At this point, we can call the compute metrics function passing in these logits along with the processed and original validation data sets. So once you've trained the model, you might want to actually use it in the real world. As usual, we can make this easy by using pipelines. We begin by calling trainer.save_model to save the model to some desired location. Once we've done this, we create a new pipeline object specifying the task as question answering.

As you learned earlier in this course, we can then use this pipeline by passing in any context and question pair. The result will be a dictionary containing the answer along with the corresponding score.

17 · Train and Evaluate in Python

So in this lecture we will continue our notebook with the train and evaluation steps. We'll begin by calling from_pretrained and passing in our model checkpoints. Recall that this is a model with randomly initialized weights of the head, which we are going to fine tune. The next step is to create our training arguments object. As you can see, evaluation strategy has been set to no since we will call compute metrics manually, we'll save it every epoch and we set FP16 to true to speed up the training process.

The next step is to instantiate a trainer object. Since you've seen this before, there's not much to explain. Note that I've commented out a line of code which you can use if you just want to test this out quickly. This will randomly select 1000 samples out of the whole train set, so the training process will complete in a shorter amount of time. Also note that this takes about two and one half hours if you use BERT and about half that time if you use DistilBERT.

For this reason, we've used DistilBERT in this notebook. However, you're encouraged to try BERT by yourself. So let's call trainer.train. The next step is to evaluate our model. In order to do this, we need to get the predictions for the validation set. We can do this by calling trainer.predict. Now it's not clear what the output is, so let's check its type. As you can see, it's an object of type prediction output. The next step is to print the object itself just to see what it looks like.

So it seems to hold some arrays and some information about how long it took to compute. Now, it's not at all obvious that you can or should do this, but basically you can expand this trainer output object just like you would if it were a tuple. By doing this, we get three items, the first of which are the predictions we need. Let's now print out the predictions. So as you can see, we get a tuple of two arrays. These are the start logits and the end logits.

Let's now assign the start logits and end logits to their own variable names. The next step is to call the compute_metrics function passing in the logits we just got back. As you can see, the last two arguments are the processed data set and the original data set, both of which need to be passed in. So we get an exact match score of about 77% and an F1 score of about 85%. Note that if you use BERT, you can improve this result. The next step is to call trainer.save_model

to save the model to disk. The next step is to demonstrate how to use the fine tune model. As you recall, pipelines make this convenient. We'll create a pipeline for the question answering task, passing in the saved model directory that we just created above. We'll also pass in device equals zero to use the GPU. The next step is to test our pipeline with a simple question. The context states that today I went to the store to purchase a carton of milk, and the question is, what did I buy?

So as you can see, our model correctly answers that I purchased a carton of milk.

18 · Question-Answering Section Summary

So in this lecture, we will be summarizing everything we learned in this section. This section at a high level was relatively straightforward. We have a new task question answering, and we wanted to learn how to fine tune a model for this task. It involves all the usual steps, the tokenize, the metrics, trainer pipeline and so forth. What made it more challenging was all the manual coding we had to do in order to glue all the pieces together. As you saw, it was definitely not as straightforward as the earlier fine tuning examples in this course.

So the first challenge we encountered was how to deal with long contacts. Although we could have extended the context window size, there would still be contexts that would have to be truncated. Furthermore, recall that transformers have quadratic time complexity. In other words, making the input length longer by some amount does not increase the processing time in proportion. Instead, it increases the time squared. As such smaller context, windows may be desirable.

The challenge with splitting up the context into multiple samples is that the answers become misaligned. Some context windows might not even contain the answer at all. Therefore, a lot of the effort we put into this section was to deal with this issue. The second challenge we encountered was how to compute metrics. We do have a function that will compute the metrics for us, but the challenge was actually converting the transformer outputs into a format that would be appropriate for the metric compute function.

We had to look at an algorithm to convert the output logits into an answer represented as text. This involved a bit of probability and a lot of coding. One of the challenging aspects of this section was keeping track of which variables look at character positions and which variables look at token positions. Also challenging is that many different things are called IDs with generic names like sequence ID. Now, one question to consider after going through this section might be How do I apply this to my own data sets?

Perhaps you'd like to build a question answering system for biology or a question answering system for history or a question answering system for legal documents. That would be nice since these kinds of documents tend to be convoluted and difficult to read. Well, at this point it would be helpful to remember my rule. All data is the same. By recalling this rule, you'll realize that nothing actually changes at this point in the course. We've had so much practice working with custom data sets, you will have no problem creating your own custom data set in the same format as what we looked at for this task.

Now, this always reminds me of a question I get sometimes, which is how do I actually get the data? Well, the reality is you have to create the data yourself or pay someone to collect it for you. The follow up question is, Well, why can't I automate it? Of course, this is sort of a circular question. Remember that what we just built is a question answering system. If we already had a computer program that could automate question answering perfectly such that the process could be done without a human, then we wouldn't be building such a system in the first place.

Of course, you may find data that already exists that you can make use of, but this would be the exception and not the rule. So as it stands, we still need human experts for this process.

1 · Theory Section Introduction

So in this lecture we will be introducing the next section of this course, which is all about the theory behind Transformers. This section is where you will learn how Transformers actually work. To contrast this with the previous sections. Note that they were purely about how to use pre-existing libraries. Everything was done with just a few function calls. And thanks to this, we were able to accomplish a lot, even without knowing anything at all about how Transformers work.

Of course, there may come a time where you realize that you do in fact need to know how Transformers work. And if that's the case, then this section is for you. So let's look at a brief outline of this section and what you can expect to learn. Firstly, note that this section is purely theoretical. There will be no coding since it will take a significant amount of time just to understand the concepts. The next section will look at implementations. So you can go there if you want to see how to actually build a transformer from scratch.

However, do note that everything you need to build a transformer will be covered in this section. This section is code agnostic, so it doesn't matter if you use TensorFlow, PyTorch, or any other deep learning library. You should be able to take this theory and go implement transformers yourself. This is really the exercise for this section. If you can go directly from theory to code, then you can be sure that you understand transformers. Now, since this section is all conceptual, it's worth talking about how to make sure you are prepared.

Essentially, if you want to understand Transformers, it would be very helpful to have a good understanding of CNNs and RNNs. We'll be drawing on that knowledge multiple times throughout these lectures. You'll want to understand how convolutional filters work and specifically why it acts like a feature finder by using a simple dot product. You'll also want to understand how we can stack convolutional filters together so that we can find multiple features in an image.

These stacked filters are what give us the eventual convolutional layer that we used in CNNs. You'll want to understand some advanced CNN models, specifically the ResNet, which uses what we call skip connections. In addition, it would be helpful to understand other layers we use in CNNs other than convolution, such as pooling and batch norm. In terms of RNNs, it would be very useful to understand how RNNs can be used for NLP specifically for the same tasks we covered earlier in this course.

Like classifying text, classifying each token and sequence to sequence prediction. Furthermore, you'll want to understand how to track the shapes in such RNNs. Understand the shape of the inputs, the shape of the embeddings, and the shape of the hidden states. If you can do all of these things, then understanding transformers will be very simple. In terms of actual topics, we'll begin with a very basic form of self attention. This will lay the groundwork for the scaled dot-product attention, which will be discussed next.

After this, we'll add the attention mask, which you've seen earlier in this course. So by learning the theory, you will understand exactly what this does and how it's used. The next step will be to combine multiple attention heads together to form what we call multi head attention. However, even this is incomplete. It's merely a building block in a larger layer. In the next lecture, we'll discuss all the other building blocks we need and then combine them all to form the transformer block.

To give you an analogy, the transformer block is to transformers as the linear layer is to ANNs, meaning just as we build ANNs by stacking linear layers, we build transformers by stacking transformer blocks. From there, we'll go directly into how to build an encoder. As you recall, BERT is an example of an encoder. So at this point you'll understand how to build a BERT from scratch. The next step will be to look at how to build a decoder. As you recall, GPT is an example of that.

And the next step, which should come as no surprise, is how to combine the encoder and decoder to build a seq2seq transformer. As you recall, these types of transformers can be used for things like machine translation. After understanding the general architecture of Transformers, we will then look at specific instances of Transformers and the features that make them unique. Specifically, we'll look at BERT, GPT, GPT-2 and GPT-3, which have been the most popular and noteworthy transformers thus far.

2 · Basic Self-Attention

So in this lecture we will begin discussing a basic form of self attention. The goal of this lecture is to gain an intuition for how attention works and then expand on this idea in the subsequent lectures. We'll begin this lecture by briefly reviewing how attention works in seq2seq. At this point, you may want to review the beginners corner in more detail. So the main thing we're trying to compute in RNNs with attention is the context vector.

As you recall, the context vector is the input to the decoder RNN in seq2seq. Looking at the context vector in more detail, we see that it's simply a weighted sum of the hidden states computed on the input sequence that is for each output time step t, the context vector for this time step, c of t, is the weighted sum of all the input hidden states across the input time steps t prime. This makes sense for the output time step t we want to know which input time step t prime to pay attention to, and because it's deep learning, we don't just pick a one time step, but we use a soft selection so that gradients can pass through.

As you recall, the weights for each of the input time steps are called the attention weights denoted by the symbol Alpha. Alpha is indexed by both t and t prime since it tells us how much input hidden state t prime matters to output time step t. Now briefly, how do we compute these attention weights? Well, as you recall, this is a weighted sum, so it makes sense that they should all be positive and sum to one. Luckily, we know a function that can meet this constraint and it's called the softmax.

So the attention weights are the softmax over all input time steps t prime over some score value, also indexed by t and t prime. Now how do we compute this score? The answer for this lecture is not too important, but if you're curious, it's just the output of a feedforward neural network that takes as input the hidden states of the encoder and concatenate them with the previous hidden state of the decoder s of t minus one. Now, if that sounds complicated, don't worry, as it's not really necessary to understand the self attention for Transformers.

The critical part to understand is we get the weights by taking a softmax. Okay. So with this background in place, we can now begin to understand a self attention. Basically, it's almost like RNN attention, but without the RNN. So suppose we have a sequence of feature vectors x1, x2 all the way up to x big T in this case, big T equals five. Now suppose we'd like to compute a new context vector, which is the weighted sum of these x's. In practice, we're going to compute a context vector for every time step, so we'll have A1, A2 all the way up to a big T since the computation for each time step is the same.

We can consider just one of these context vectors, a_i. As a side note, please be aware that the reason I've decided to use the index I instead of the index t as we did before with RNNs is because this tends to be the convention when describing self attention. Now, since this is for intuition, let's consider the meaning of these X's and these A's. Since we're doing NLP, you can think of the X's as embedding vectors, just like you would have for a CNN or RNN.

This is where we just map each word in our vocabulary to a vector. But these vectors are not context aware. For instance, the word bank always gets mapped to the same embedding vector, whether you mean river bank or whether you mean a financial institution. However, the A's,

on the other hand, are context aware embeddings. These also can be thought of as embedding vectors, since they are, after all, still vectors. And furthermore we have one of them for each time step.

So for each word in the input sentence we have a context aware embedding vector a_i for i from one up to big T. As you can see, the way we compute a_i is similar to how we compute the context vector in attention for seq2seq RNNs. It's the weighted sum of x_j's where the weights are called alpha ij. Now the question is how do we get alpha ij? Well, in this simple form of self attention alpha is easy to compute relative to the RNN. In any case, let's first begin with a basic premise which you should understand well.

If you've studied convolutional neural networks or deep learning more broadly, the premise is that the dot product is like a measure of similarity or affinity. To give you some intuition for why this is the case, recall that the dot product is just an element wise product and summation of two vectors. However, you'll recall that if the two vectors both have a length of one, then this is equal to the cosine of the angle between the two vectors. As you recall, this is because the dot product is equivalent to the magnitude of x times, the magnitude of Y times, the cosine of the angle between x and Y.

But what do you get when you take the cosine of the angle between two vectors? Well, that's simply the cosine similarity. So the dot product is almost like the cosine similarity, except that we ignore the magnitudes of the vectors. Recall that cosine similarity is also closely related to the Pearson correlation coefficient. The equation for that is shown here, but please don't be worried by all the symbols. Basically what you want to be able to see is that the cosine similarity and the Pearson correlation are nearly the same, except that the Pearson correlation also subtracts the mean at every point.

Now, why is this important to know? Well, because it gives us yet another way of thinking of cosine similarity and in turn, another way of thinking of the dot product. To say two vectors are similar is almost like saying that they are correlated. In other words, these are all different ways of measuring a degree of sameness. Now. Why do we care about that? Well, another way to think of the dot product is in terms of convolutional neural networks. Basically, convolution is simply a sliding dot product where we slide some filter over an input image.

And at each point we take the dot product. Plotting all the resulting dot products gives us another image. This new image, which is in black and white, tells us where in the original image some feature was found. As an example, suppose my filter is an eye. If we slide this filter around an image of a face, we should find mostly small values close to zero after doing convolution, except where the eye is found. Where the AI is found, we should find large values since the filter will match the image in that particular location.

This creates a spike, or in terms of an image, a bright spot showing that this is where the eye can be found in the input image. That is to say, we find spots in the image where there is a match between the filter and the image itself. You can think of it like if the image matches the filter at this point, return one, otherwise return zero. In other words, dot products are nice because they are like feature finders or feature matchers. Using convolution.

We can train a neural network to find all kinds of features like eyes, noses, ears, glasses and so forth. After training a CNN, you can look at the filters to interpret what they have learned to find. Okay. So hopefully you're convinced that dot products are a way to measure similarity. Using this knowledge, we can now form our attention weights. You recall that we first started with a score. There's no constraint on the score, so it can be positive or negative and large or small.

The score between input i and j, as you may have guessed, is simply the dot product between x_i and x_j. You can think of this as how similar are word i and word j in terms of their embeddings. We call these scores e_ij. In order to convert these into attention weights, we take the softmax over the j index. This will ensure that the weights are all positive and sum to one. And so now we know how to fully calculate the simple form of self attention to obtain context aware embeddings from a sequence of input vectors.

We first compute the attention weights as previously described. Then the context vector simply becomes the weighted average of input embeddings weighted by these alphas.

3 · Self-Attention & Scaled Dot-Product Attention

So in this lecture we will continue our discussion of self attention by adding a few new components to the simple form of self attention we just described. So let's consider why we might want to do this. Recall that in the previous lecture we computed the attention weights based on the input of vectors themselves. This means that for fixed inputs, these attention weights would always be fixed. In other words, there are no learnable parameters. In this lecture we will introduce some learnable parameters which will make the self attention mechanism more flexible and tunable for various tasks.

Now, one common way for people to introduce transformers is to immediately jump to this concept of queries, keys and values. Personally, I found this to be very confusing because the concept of queries, keys and values bears no relation to anything that came before in deep learning. It's not clear how we can use our existing knowledge of deep learning to derive this concept. In fact, the very first equation that shows up in the paper called Attention is All you need is this attention expression involving a softmax and matrices Q, K and V representing queries, keys and values.

In this lecture, I will show you where this concept comes from and how introducing new learnable parameters helps us to implement this idea. Our goal in this lecture will be to understand this attention matrix expression. We'll also see how this is easily built on our basic form of self attention that we looked at in the previous lecture. We'll begin by discussing where this concept of queries, keys and values come from. Please note that this is optional.

There is no need to understand how this works. In order to understand self attention. But it could help. In fact, you are already familiar with the concept of keys and values from dictionaries in Python. Dictionaries allow us to store data by using key value pairs. Each key points to a corresponding value. Of course, there are actual database technologies which use the same key value paradigm. Some popular examples are Redis, Memcached and Amazon's DynamoDB.

And of course, when you have a database, you often want to query the database to obtain data you might need for your application. So for example, the keys might be user IDs and the values might be your user preferences like video speed, language, whether captions should be on or off and so forth. And when we want to look up certain data in the database, we would query it with the user ID we are looking for. The database will return the data for which the query and key match.

This should sound familiar. The goal of introducing learnable parameters in self attention is to transform the input vectors. That is, all the x's into their respective roles as queries, keys and values. Okay. So what does self-attention look like when we introduce these parameters or weights? Well, as you may have expected, it's really nothing more than matrix multiplication. This is deep learning, after all. So we obtain the query vector. When we multiply X by WQ, we obtain the key vector.

When we multiply x by w k and we obtain the value vector when we multiply x by WV. And of course, we do this for all the X's from time step one up to Big T, so we'll have corresponding sequences of Qs, Ks and Vs. So what do we do with these Qs, Ks and Vs? Well, we do the exact same thing we saw in the basic form of self attention in order to make this less abstract. Let's use a running example. Suppose our input sentence is Bob cashed the check at the bank.

This is a useful example because as you know, the word check can have multiple meanings. It can mean, as in this example, a bill of exchange. Check can also be a verb. As in double check your homework. The word check has yet another meaning in the context of chess. Note that the word bank also has multiple meanings, but for the purpose of this lecture, let's just think of the word check. So check will correspond to our query vector q_i. Our model wants to know in what sense do we mean the word check?

In order to do that, we might want to pay attention to other words in the sentence, such as cashed and bank. So in order to compute these attention weights, we're going to dot this query vector q_i with all other key vectors k_j for j from one up to big T. This will give us big T attention scores. As you know, we then take the softmax, which will then give us big T attention weights. These attention weights are then used to form a weighted sum of the values.

Again, these are all vectors and we have one of them for each word in our sentence. So our attention vector for the word check will be the weighted sum of the value vectors. Now, in order to make this a bit less abstract, let's try to relate this back to our database analogy with graphics instead of equations. Firstly, as you can see here, we can imagine a vector space where all the key vectors live. These are denoted in blue. As you recall, there will be a key vector for every word in the input sentence, but only some of them are shown here to save space.

The purple vector represents our query, which is check. Importantly, note that the same word can be both the query and the key. Since it's possible for a word to pay attention to itself. So that's why Check can show up twice, once as a key and once as a query. Now you can see what's happening. What we're trying to do is find which key vector matches with the query. In vector space, we can think of a match as the vector that yields the largest dot product.

And in this case, that would be the word bank. As you can see, bank and check are very close together. Hence they are similar and would be considered the best match. So now we can think of what the attention weights might be. In this table we can see the dot products between the query and each key. Note that these are just made up numbers so they aren't designed to accurately reflect the graph on the left. Intuitively, the dot product between the query and the keys other than bank might be either very small or negative.

In other words, insignificant. The dot product between the query and bank might be large and positive. Now what happens when we take the softmax of these numbers? Well, the softmax will yield a number close to one. If an input is large relative to the others and a number close to zero otherwise. And again, these are made up numbers, but we'll say that the attention weight for bank is 0.94 and the attention weight for all the other words is 0.01.

As you can see, they add up to one. So what will the result be? Well, remember that after finding the attention weights, our attention output vector will be the weighted sum of value vectors weighted by these weights. In other words, we'll have 0.94 of the value vector for bank plus 0.01 of all the other value vectors in the sentence. Effectively, the attention output is almost just the value vector for bank, plus a bit of other stuff. Compare this to if we had taken the hard max instead of the softmax.

If that were the case, we would simply select whichever word gave us the highest dot product and use that value vector. Of course, in deep learning we don't do that. Now, although we technically answered this question on the previous slide, I want to state it more explicitly. Why do we want self attention? Admittedly, it seems kind of weird that we would want the input sentence to pay attention to itself. This is a bit different from how we introduced attention for RNNs, where each token in the output

had to decide which token from the input to pay attention to in order to generate a translation or a chatbot response? In this section, it appears that the input sequence merely pays attention to itself. And well, we've actually already seen why this is not weird. The example we saw was with word sense disambiguation. In other words, the word check with no context is ambiguous. We can disambiguate it by using the other words in the sentence. In particular, the words cashed and bank are clues that when we say check in this context, we mean the financial instrument and not, for instance, check your homework.

You can also imagine in your mind that the neural network can use this information to answer questions about the input where these answers appear as feature vectors. As an example, consider the sentence Alice drove to the store. Suppose the word store pays attention to the word drove. This might answer the question. How did Alice arrive at the store? Perhaps store pays attention to the word Alice. This might answer the question Who was at the store? Perhaps Alice pays attention to the word store.

This might answer the question. Where did Alice go? So you get the idea. This is just like how in convolutional neural networks we are looking to find certain features. Like, is there an eye in this image? Is there a nose? And if so, maybe this is a face. Now at this point, it's a good idea to introduce the concept of understanding shapes. If you've taken my courses before, then you know that this is a very important aspect of deep learning. It's important.

From linear regression up to ANNs, CNNs and RNNs, transformers are no different. While it's good to understand the intuition, if you ever want to actually build this stuff in code, you need to answer some basic questions. For instance, how big are the Q, K and V vectors? How big are WQ, WK and WV? Are there any constraints? Well, there must be, since we dot Q and K together implying that Q and K must have the same shape using the conventions of the Attention Is All You Need

paper, Q and K both have size d sub k. The value vector V has dimension D sub v. Importantly, both the input and output of a full attention layer have the same dimension, which they call d model. The reason for this will become clear when we discuss how to build the transformer block. But for now, just accept it as fact. Because of this, WQ and WK both have the shape d model by d_k and WV has the shape d model by d_v. In the Attention Is All You Need paper,

they set d_k and d_v both to 64 while D model is 512. In other words, while the input vectors are very large, they are compressed into relatively small dimension vectors when they are converted into queries, keys and values. So now that you know what the shapes are, let's do some math. So suppose that all the Xs have size 512. After mapping them to Qs, Ks and Vs, we get vectors of size 64. When we take the dot product between Q and K, we get a scalar and taking the softmax of several of these just gives us another scalar.

As such, our context aware embedding vectors will be the sum of Vs and hence will have the same size as the Vs, which is 64. Now you recall that I said the output of a full attention layer should still be d model. This will be addressed when we discuss multi-head attention. For now our output will have size d_v. Now there's one detail we haven't mentioned yet, which is that before we apply the softmax, we actually scale the product between the query and the key for some intuition on why we do this.

You want to look back at our study of linear regression and neural networks. You recall that when we initialize our weights, we divide by the square root of the dimensionality of w and x, which we normally call D. By doing this, we ensure that the output of the dot product has variance one if we assume that both the inputs and the weights also have variance one and are independent. In other words, this scales the variance of the output down, which keeps it from exploding.

Similarly, in this case, we divide Q dotted with K by the square root of D sub k, which is the dimensionality of both Q and K. And so this is why you may see the attention operation referred to as scaled dot product attention. We're doing a dot product between Q and K and then we're scaling by the square root of D sub k. Now. I promised you that by the end of this lecture we would understand the expression from the Attention Is All You Need paper. We are not quite there yet, but we are close.

The final step is to realize that we have a bunch of summations over element wise products. As I taught you in the prerequisites, whenever you see a summation over element wise products, you know that it's just matrix multiplication and hence can be vectorized. So let's see how to do that. Firstly, let's suppose our input x is a sequence of vectors stored as a matrix with the shape t by D model. Note that in this case each row of x is a feature vector for a specific time step.

Using this, we can get corresponding matrices for the queries, keys and values which we will call Big Q, Big K and Big V. We get these as usual, by multiplying by the corresponding W matrices, since WQ has the shape d model by d_k, Q will have the shape T by d_k. As you recall, when you do matrix multiplication, the inner dimension disappears. Similarly, K also has the shape T by d_k and V has the shape T by DV. The next step is to multiply Q by K transpose.

Now, why does this make sense? Well, since K has the shape T by d_k, K transpose will have the shape d_k by t since Q has the shape T by d_k and K transpose has the shape d_k by T. The result will be T by T. Mathematically, you should convince yourself that this results in multiplying every row of Q by every row of K. These rows are the different query and key vectors at different time steps. And this makes perfect sense, since as you recall, for every query vector we have t attention weights one for every input.

Furthermore, we have t query vectors for each of the t inputs making up t times t attention weights in total. So this perfectly matches what we did earlier. Note that after we take the softmax, the shape doesn't change. Dividing by the square root of d_k also does not change the shape. The final step is to multiply the attention weights by V, which has the shape T by d_v. So if we have a T by T multiplied by a T by d_v, again the inner dimension disappears and we get a T by d_v result.

This makes sense since we should end up with t attention vectors each with size dv. So using these shape calculations, you should be convinced that this matrix expression does in fact compute self attention like we previously described. Finally, note that in code we further vectorize the attention computation by handling multiple samples at once. So instead of just a T by D input, we have N by T by D. Now, one more thing before we conclude this lecture.

What is the difference between self attention and scaled dot product attention? In this lecture I've introduced both of these concepts at the same time, so it's important not to get them confused. To be clear, scaled dot product attention is the attention operation which can act on any queries, keys and values as described in this lecture. Self attention is where the queries, keys and values are all computed on the same input sequence. Hence the queries and keys both come from the same sequence and hence whichever key the query is paying attention to, it came from its own self sequence.

4 · Attention Efficiency

Now, although our previous computation looked at a specific query word at location i, it should be clear that we are going to do this for all values of i from one up to big t, so every word in the input sentence will pay attention to some other word, possibly itself in the same input sentence. Because of this, it should be clear that we have a t squared computation for each word in the sentence. We have to compute attention weights for every word in the sentence.

That's a double for loop where both loops go from one up to Big T. As you know, t squared is quadratic in T, and if you plot the function t squared, hopefully you know that you will get a parabola. What you should recall is that parabolas grow fast. If our sequence length is t equals ten,

then we have to compute 100 attention weights. If T equals 50, then we have to compute 2500 attention weights. If T equals 100, then we have to compute 10,000 attention weights.

In other words, the number of computations we need to do grows faster than the number of inputs we have. Eventually, the number of computations will get so large that we will need to limit the size of the inputs at some point. At the same time. Note that attention is still superior to RNNs for multiple reasons. Firstly, recognize that in our computation of the attention weights for q_i, we didn't have to consider the attention weights for any other query.

Q sub j where j is not equal to i. There should be a sense that we can parallelize this computation. For instance, if we have seven workers, they can compute all seven of the attention vectors at the same time. We don't have to wait for one to complete before we can compute the next. This is unlike RNNs, where for each hidden state h of T, we must have finished computing the previous hidden state h of t minus one since h of t depends on that value.

Another reason attention is superior to RNNs is that RNNs have trouble with long sequences, but not in the same way that attention does. Earnings have trouble remembering things too far in the past due to vanishing gradients. RNN units with gates such as the GRU and LSTM were created to fix this issue. But as we've demonstrated in my earlier courses, there is still a limit where these advanced units stop performing well. In fact, this was already demonstrated in the original RNN and attention paper as well.

It showed that as the number of tokens increased, RNNs without attention dropped in performance, while RNNs with attention maintained its performance. Here's another interesting fact to consider. Recall that RNNs are able to handle variable length sequences. This is possible since each h of t depends on h of t minus one. And there's no point at which we need to stop all the weights of an RNN unit do not depend on sequence length.

However, note that this is also true of the self attention weights. WQ and WK both have sized d model by DK and WV has D model by DV. The input into self attention has the shape T by D model and the output has the shape T by DV using only matrix multiplies. In other words, self attention is also capable of handling variable length sequences. Now, at this point, this is so advanced you probably forgot why this is significant. If that's the case, consider any model in scikit-learn.

These models require that all your inputs have the same dimension. Consider a regular feedforward ANN. This also requires that your inputs have the same dimension. Consider even a CNN without special modifications like global max pooling. This requires that all your images are the same size. So considering this, it's pretty neat that self attention can handle sequences of varying length. Although do note that because we want to use matrix multiplication, every input in the same batch must have the same length.

5 · Attention Mask

So in this lecture, we will be discussing one final addition to the self-attention mechanism, which is masking. You've already seen masking in this course since whenever we apply the tokenizer to a sentence, we always get back not just the token IDs but also the corresponding attention mask. Let's recall why we need the attention mask. There are two reasons. The second of which we'll un-

derstand in more detail later in the course. Reason number one is that, as you know, we need to pad our sequences so that each sequence in a single batch has the same sequence length based on the previous lectures.

You know that we must do this so that we can perform self attention using matrix multiplication, which is much faster because it is vectorized. When we introduce these pad tokens, we do not want our model to pay any attention to them as they are not relevant for understanding the sentence. Reason number two is related to the decoder. As you recall, the decoder is auto regressive, meaning it only looks at past tokens in order to predict the next token.

One way to accomplish this is to hide all the tokens beyond the last time step by using a mask. Intuitively, the function of the attention mask is very simple. Essentially, we want some way to set the attention weights for irrelevant tokens, such as padding to be zero. Now, how do we accomplish this? Well, at this point, we are ready to look at the original schematic for the self attention mechanism. This is from the paper. Attention is all you need. As you can see, we begin by taking a Q, K and V as inputs.

We then do a matrix multiply on Q and K. As you know, the next step is to scale the result. As you recall, we do this by dividing by the square root of d_k. After scaling, there is an optional step which is to apply the mask. After applying the mask, we apply the softmax. And after we apply the softmax, we do one final matrix multiply with V. Now, those of you paying close attention might notice something weird. What's weird about this is that the attention mask is applied before the softmax and not after.

So why is this weird? Well, we've seen in the code that in the attention mask, we use a one for locations where we do want to pay attention and we use a zero for locations where we do not want to pay attention. However, the softmax of zero is not zero and therefore it won't result in the corresponding attention weight being zero. As you recall, when we take the softmax, we first exponentiate all the values, but the exponent of zero is actually one.

The resolution to this is that in this formulation, the values in the attention mask are not actually zero for the locations where we do not want to pay attention. Instead, they are actually set to minus infinity, which is stated in the paper. Now just a minor note on convention. Notice how in this diagram no weights are included. Instead, queries, keys and values can be any matrices of inputs. Currently we are only looking at self attention, so it might not be clear why this is needed.

Since the queries, keys and values are all computed from the same x matrix. However, later in this section you'll see where this is not the case. And so this way of representing the attention operation becomes useful.

6 · Multi-Head Attention

So in this lecture we will be continuing our discussion of the self attention mechanism. In particular, we're going to use the scaled dot product attention we previously discussed as a building block for a new block called Multi Head Attention. The best format for this lecture will be to start

with how it works, followed by why it works. So how does multi head attention work currently? As it stands, our self attention mechanism only allows each token to pay attention to one thing using one set of attention weights.

But what if we would like our model to allow tokens to pay attention to multiple things? For instance, consider the sentence Bob cashed the check at the bank. Perhaps in one case, bank pays attention to the word cash so that we know it's a financial bank and not a river bank. In another instance, it might pay attention to Bob telling us who went to the bank. In this case, we create a new self attention layer with different weights. Specifically, suppose that we would like to compute this self-attention operation h times.

If we call the output of each individual self-attention a head, this means that we would like to have h heads. Then we would just need to have h different sets of weights. We'll call them W1Q, W1K and W1V, W2Q, W2K, W2V and so forth. In the Attention Is All You Need paper, they set the value of H to eight. Now of course after computing self-attention h times, we still need a way to combine the result back into a single block of sequence data which can then be passed into the next layer of the neural network.

The short answer is we can do that by concatenating all the heads and then multiplying by an output weight which we call w o. Of course the devil is in the details. As usual, it always helps to understand the shapes. We already know that the input to each self attention layer will be T by D model. The output will be T by DV as we previously showed. So what does it mean to concatenate these outputs? Well, we specifically concatenate them along the feature dimension rather than the time dimension.

So the result will be a bigger matrix of size t by h dv. Visually, you can imagine that if the output of each self attention is a small block after concatenation, these blocks will be lined up side by side. After doing this we multiply by w o which has the shape hd v by d model. In other words, we project the output back into its original dimensions. If we multiply a matrix of size T by hd v with another matrix of size DV by D model, the inner dimension will disappear and we will get back T by d model.

This also answers the question we previously posed, which was how can we make it so that the input and output both have the same size? The key is that the final step is to multiply by w o, which ensures that the output has the shape t by d model, which is the same as the input. Now just a minor note on convention: in the original paper, the diagram for a scaled dot product attention does not use any weights at all. Instead, the weights are included only in the multi head attention block, separate from the self attention operation.

Knowing this, we can understand the next diagram from the paper which shows a schematic for multi head attention. Firstly, you can see that there are h scaled dot product attention layers going in the depth direction. There are also as many linear layers which represent the weights. After going through each attention layers, we then concatenate the result and go through one final linear layer which represents w o. It's important to note again that just like attention itself, the operations we just discussed can be done in parallel.

Each self attention, it depends only on its own weights, and hence each head can be computed at the same time, leading to more efficient computation. This allows transformers to be faster, even though they may use more compute. Okay. So that's all there is to say about how multi-head attention works. But the important question now is why does it work? What is the point of repeating this self attention operation h times? Well, recall a running example. Bob cashed the check at the bank.

We've discussed that we can think of attention outputs as features that tell us something about the sentence. For example, when Bob pays attention to bank, the feature might be Where did Bob go? But we might also be interested in what Bob did. He cashed the check. We can only do this if we allow our layer to produce multiple features, each of which [unclear] to pay attention to different tokens in the sentence. One good analogy if you have past experience with deep learning is with CNNs.

We know that in CNNs we use convolutional layers to produce features from images. We already discussed how convolution is basically a feature finder. For example, if the filter is shaped like an eye, then we will find eyes. If the filter is shaped like an ear, it will find ears. But what if we would like to find eyes and ears? The answer is we need to pass our image through multiple convolution filters at once. We then concatenate the results which become image representations of where all these features were found.

At the end of the Attention Is All You Need paper. There are some nice visualizations that show what the attention weights have learned to do in practice. This lecture discussed some intuition, but the reality will always be more complex. In this case, the sentence is The law will never be perfect, but its application should be just. This is what we are missing, in my opinion. The figure shows the weights from two different attention heads and we can see that in both cases they allow the model to learn different aspects of the sentence.

In the bottom image, we can see that the attention is very localized, where the words pay attention only to its immediate neighbors. In the upper image, we can see that the attention is more spread out, allowing the words to pay attention to more context. The authors of the paper surmised that the attention weights seem to be learning about the structure of the sentence.

7 · Transformer Block

So in the previous lecture we learned how to use scaled dot-product attention as a building block for multi-head attention. In this lecture, we will take things to the next step. In particular, even the multi-head attention layer is just a building block of an even bigger building block. In this course, we'll refer to this block we're about to create as the transformer block. And it's the final block we need in order to build encoder models like BERT, decoder models like GPT, and a sequence to sequence models for each of these models, they really just take the transformer block and repeat it multiple times.

So once you understand this lecture, you'll understand the exact building block we use to build these more complex models. The first thing we're going to add is a feedforward ANN. As you recall, our multi head attention layer really only consists of linear transformations. But we know

that neural networks are nonlinear, which is why they are able to learn such rich representations of our data. One way we can accomplish this is by passing the output of multi-head attention through a feedforward ANN. In the Attention Is All You Need paper, they use an ANN with one hidden layer and a ReLU activation in the middle.

The output layer has no activation, but note that this is not the case for all transformers. For instance, in the BERT paper they use the GELU activation, which stands for Gaussian Error Linear Unit. It basically looks kind of like the ReLU, except that it has a small dip on the left side and doesn't have a sharp elbow. The next detail we're going to discuss is the layer normalization block. This appears both after the multi-head attention and the feedforward ANN.

And so you can imagine this like four sequential steps. First, we have multi-head attention. Then we have layer normalization. Then we have the feedforward ANN and then we have another layer normalization. So what is layer normalization? Well, it's not my goal in this course to go into great detail about layer norm. But if you've ever heard of batch norm, then you should already have some intuition about what this does. Basically layer norm works the same as batch norm, except that instead of normalizing across the samples, it normalizes across the features.

The computations are still the same. We still begin by subtracting the mean and dividing by the standard deviation. But again, we do this across the features and not the samples. After this, we then apply gain and an adaptive bias, both of which are learnable parameters. Now, for those of you who have never heard of either batch norm nor layer norm, some basic intuition is this. We know that for neural networks, it often helps to normalize our data before passing it into a neural network.

If the values are too small or too large, the performance degrades. You can do some experiments to confirm this yourself. Thus, normalizing your data keeps it within a standard range of values. However, instead of just performing this operation at the input, we've turned it into a layer so that it can be repeatedly applied even between intermediate layers of the network. And if you don't care about theory, basically the practical view is that it helps neural networks train faster.

The third and final modification we are going to make is to add skip connections, also known as residual connections. As with the other topics we've discussed in this section, it would be really helpful if you have past experience with CNNs. Specifically, residual connections were first introduced with CNNs with a specific CNN called the ResNet, short for Residual Network. The intuition is this. Basically, researchers notice that if you keep adding more layers to a neural network, performance does not improve, but instead it degrades.

This seems very weird since theoretically, given some performance level at some number of layers, you can just add an arbitrary number of new layers, but just copy the input to the output. In other words, all of these new layers could just be identity functions. By doing that, the neural networks performance would remain the same. But instead they found that when they trained a large neural network with more layers, performance got worse. As such, they decided to explicitly add the identity function.

We call these skip connections because they're like a wire that connects the input directly to the output. Without making any changes, it skips the layer in between. Another way to think of it is like a residual connection. Since the neural network layer only has to learn what is left after subtracting the input from the output. And again, if you don't care about the theory, all you have to know is that this helps the neural network learn better. By the way, note that this also explains why the output shape of multi-head attention must be equal to the input shape.

If this were not the case then it would not be possible to add the input to the output. As you recall, if you want to add two matrices together, they both must have the same size. Furthermore, this also implies that the output of the feedforward ANN does not change the size. So in fact the input to the whole transformer block will be of shaped t by d model, as will the output. Okay. So now that we have our building blocks, we can look at the final transformer block.

Again, there are essentially four steps. We begin with multi-head attention. Note that this takes in three inputs, which are the queries, keys and values. But all of them are actually the same input, which is the input sequence, at least in the scenarios we've discussed so far. Then we have the add & norm block. This first adds the skip connection to the output of the multi head attention block and then applies layer norm. Next we pass this through a feed forward and at the same time this also has a skip connection so that after the ANN we have another add & norm block.

Personally, I think writing this out in pseudocode helps. On the first line we pass through a multi-head attention layer, add it to itself, and then apply layer norm. On the second line, we pass x through the ANN and add it to itself and again apply layer norm.

8 · Positional Encodings

So in this lecture, we'll be discussing one final component of the Transformer network. This is separate from the previous lectures. Since the previous lectures were all about the transformer block, whereas this lecture is about a separate component. This lecture is all about the positional encoding, which solves one issue. You may have noticed the issue is this. Consider what would happen if we randomly rearranged all the words in our input sentence. Intuitively, this should confuse our model, just as it would confuse us.

When I speak these sentences to you, they make sense because the words are in a particular order. In other words, the ordering of the words coming out of my mouth right now matters. But note that this is not how self attention works. Self attention is what we call permutation invariant. It doesn't matter how we order the words in our input sequence because the amount of attention any token pays to another is purely a function of their embeddings and the weight matrices.

Although transformers are a sequence model, it appears that this crucial detail has somehow been lost. The answer to this problem is to simply add inputs that are position specific. That is to say, by looking at these new inputs, we can tell where in the sequence they came from. We call these positional encoding. Now, before we get into how positional encoding work, let's look at this from a higher level and talk about where they actually go. So far in this section, we've discussed the transformer block, but we haven't yet discussed what comes before the transformer block.

If you studied NLP with deep learning in the past, then you already know the answer. The input into our network is a sequence of token IDs and the first thing we do is map each of these token IDs to embeddings. In TensorFlow and pytorch, we would get these by using an embedding layer. Mathematically, all we are doing is creating a big table of embedding vectors where each row belongs to a different word or a different token. So if your input is word i, then we simply grab the ith row of the embedding table and that becomes our vector for word i.

These vectors are each of size D model, and so if we pass in a sequence of T tokens, they will first be converted into a sequence of T vectors. Each of size D model. In other words, a matrix of size T by D model, which is what we looked at earlier in the section. Of course, if we only do this and we pass this directly into the transformer block, the transformer will have no idea about the order of the tokens. The positional encodings simply add positional information to the existing embeddings.

So we're going to come up with a unique set of numbers to add at each position of the existing embeddings, such that this new set of numbers can uniquely identify which position they are located at. Now, how does this work in practice? In practice, we're going to create these positional encoding by using a set of sines and cosines at different frequencies. Notice that if you look at the original diagram for the transformer, it shows these positional encoding using a little circle with a sine wave inside it.

So that's what that represents. Let's now go into a little more detail about why this works. Firstly, let's begin with a single sine wave. Suppose our token is right at the beginning of the sentence at position zero. Since the sine of zero is zero, the value for the positional encoding would be zero. However, note that a single sine wave is not enough. If we go far enough, the sine wave simply repeats. So we can't tell the difference between position zero and whatever position is at two pi radians.

But let's consider what would happen if we include a cosine wave at the same frequency. Again, consider the word at the first position. The cosine of zero is one, so the positional encoding for this position would be a vector (0, 1): 0 from the sine and one from the cosine. Of course, this still doesn't lead to a unique encoding since at two pi radians cosine goes back to one. But now let's consider what if we just added more and more sines and cosines at different frequencies?

Even though sine and cosine repeats, they won't repeat at the same positions. So the combination of points from all of the sines and cosines will give us a unique vector for each position in the sequence. Now, just for completion sake, I'm going to show you the exact formulas they used for the positional encodings. Although this is not necessary for you to know unless you want to implement it yourself. Basically for even dimensions we use the sine and for odd dimensions we use the cosine. Inside the sine or cosine we take the position and divided by 10,000 raised to some power.

The power is a function of I divided by d model so that you can get different frequencies for different dimensions as shown in the previous slide. By the way, note that i goes from zero up to D model minus one, so that in total we have D model different sines and cosines. This must be the case since as you recall, we're going to add the positional encoding to the original embeddings. And in order to add a matrix to another matrix, they must have the same shape.

Now one question you might have is why not make the positional encoding learnable embeddings? The answer is that the authors tried this, but it didn't lead to any increase in performance. So one neat way to visualize how the positional encoding works, which I haven't seen anywhere else, is to use PCA to reduce the dimensionality of the embeddings. When we do this, we can see that in fact the positional encoding really do map each position to a unique location, which we can confirm by looking at the encoding in two dimensions.

Note that I've added a color gradient to each data point so that we can follow the positions from beginning to end. So it starts with red and then pink, and then purple, blue, green, yellow, and then back to red. In other words, the points are not scattered around randomly, but they follow a shape. This is not required, but it does show that positional encoding give us a sane representation of position in a sequence. In that way, we always know where we are in the sequence by looking at these points in a dimensional space.

And as you can see in this case, even a two dimensional space.

9 · Encoder Architecture

So in this lecture, we will be looking at how to take everything you just learned about the different building blocks of a transformer and build an actual encoder. Now, don't be fooled by its name. Saying Encoder doesn't imply that we only have half of a model. As you recall, BERT is an example of an encoder only network, and it's one of the most popular transformers that exists. Encoder really just qualifies what tasks this type of neural network can be used for.

Other types of architectures are used for other types of tasks. I also want to give you an analogy which should be helpful. If you've taken my deep learning courses in the past, in particular, think of how we learned about CNNs. This uses a similar format. When we learned about CNN's, we first learned about convolution in this course. We first learned about self attention. Then we learned about how to build the convolution layer which involved multiple convolutions stacked together.

That's like multi-head attention. Then we learned about other layers that work well with convolution such as pooling and batch norm. In this course that's like the ANN, skip connections and layer norm. Finally, after learning about these components, we learned how to group them together and form a full neural network. So the encoder architecture is actually very simple. First, let's suppose our input is a sequence of T word IDs or token IDs. Then we put this through an embedding layer which you've seen before.

If you studied deep learning for NLP after the embedding layer, we then add positional encoding, which you just learned about after adding positional encodings. We then apply several transformer blocks. How many you add is obviously up to you, and as usual, you can consider that as a hyper parameter. As an example, the base version of BERT has 12 layers of transformer blocks and in the Attention Is All You Need paper. The encoder part of the model has six layers of transformer blocks.

From there you can add any model head you like. For instance, a binary layer for doing next sentence prediction. Now, as usual, it's always important to think about shapes. Luckily, thanks to the groundwork we've done already, this is going to be easy. So after passing our T token IDs into the embedding layer, we'll have a sequence of embeddings shaped like a matrix of size T by d model. Since the positional encodings have the same size after adding them, we will still have T by d model.

As you know, the transformer blocks don't change the shape of the data from input to output. So the input to every transformer block is T by d model and the output is the same effectively. What we end up with is t vectors each of size d model. Now when we're working with RNNs, we thought of these as hidden states, which we called H of T. There's nothing stopping you from doing that with Transformers. But this terminology is typically not used. Instead, we normally refer to these vectors as contextual embeddings.

The reason for this is notice how they are just like the first layer of embeddings in terms of their shape. Whether you're at the beginning of the network or at the end, you still have t embeddings of size d model. The difference is that at the beginning of the model, those embeddings are just from a lookup table. These embeddings do not take context into account. So the embedding for bank will be the same whether you mean a river bank or a financial institution.

However, this changes when you get to the end of the network, which does take context into account. Thanks to attention. Now when we look at the corresponding embedding for the word bank, it will have been transformed based on the surrounding context. Hence, we call them contextual embeddings. At this point, you have everything you need to build an encoder to perform the task we studied earlier in this course for completion sake, we'll look at how to do this conceptually.

So suppose we want to do text classification which involves assigning a single label to an entire text document. With RNNs, we normally take the final hidden state, push that through a final dense layer, and then apply the softmax. With transformers such as BERT. We actually take the first output embedding, push that through a final dense layer and so forth. So why is this the case? Well, remember that with Transformers Order doesn't matter. Everything can pay attention to everything else.

This idea actually shows up in the name of BERT, which stands for bidirectional encoder representations from Transformers, the key word being bidirectional. What this means is that every embedding can pay attention not just to tokens that came before, but also tokens that came after. As such, using the first output works just fine. This is opposed to a typical RNN which only pays attention to the past hidden state. Another task you can do as you recall.

Called token classification where we classify every token in the document. In this case, we take every contextual embedding and pass each of them through a final, dense layer with a softmax. When we do this, that it becomes just like an RNN we would use for the same task. So hopefully, you know what we would do from here to train the model, make predictions and so forth. So as an exercise, I would recommend taking what you just learned and implementing a transformer from scratch for both text and token classification.

Note that both TensorFlow and PyTorch contain a multi-head attention layers, but you could also build them yourself. Since they're just simple matrix multiplies.

10 · Decoder Architecture

So now that we've looked at how the encoder works, let's look at how the decoder works. The structure of this lecture will be a bit different from the encoder, since the key points are not really analogous to what we previously did. For this lecture, we'll begin with the main task of a decoder. Unlike the encoder, there is really only one task. The architecture is also pretty simple. In fact, it looks pretty much the same as the encoder for classifying text.

The main difference, which in my opinion is the most challenging part to understand, is how we get this model to perform the task we want. Specifically, we want a model that can predict the next token from the previous tokens in a sequence. You could always just do this naively, as we will see. But for Transformers, there is a very specific method that we will learn about. Note that this is more of an implementation detail, so feel free to skip it if you like.

But I think that it will be useful if you plan to move on to implementing Transformers yourself. I've learned that repetition is always useful, so since this is a challenging concept, I believe it's worth it to be introduced to it now and then to repeat it again when you actually implement a transformer. Okay. So let's begin with the task that the decoder performs. As you may have learned earlier in this course, the decoders job is to simply predict the next token in a sequence given the previous tokens.

So as an example, I might pass in the input. The quick brown fox jumps over the lazy and I would train the transformer to output dog. We would do this for all documents in our data sets. And in addition, note that each document turns into multiple effective samples. For instance, we want to predict quick given just the we want to predict brown given the quick, we want to predict fox, given the quick brown and so on and so forth. So all the tokens in a document have a chance to be the target given just the previous tokens.

As you recall, we call this a language model. It is a model that estimates the probability of token T, given token T, minus one, token t, minus two all the way down to token one. So it's worth noting at this point why this task is sufficiently generic. Consider some use cases for full sequence to sequence models like chatbots and language translation. Recall that the decoder is the second part of such a model. In both of these cases, we need to generate text for a chatbot.

We would condition on the encoder output after feeding it the prompts we want to respond to. For language translation, we would condition on the encoder output after feeding it the input sentence we want to translate, but in both cases after we read in this encoder output, what do we need to do is generate a new sequence which basically amounts to the task of generating text. It's simply that when we're talking about the decoder only, that means we're not conditioning on anything except past tokens.

And there's a further side note, which we'll discuss more when we talk about GPT. Even the decoder by itself can do some pretty amazing things. For instance, with just a decoder, we can still design a prompt such that the decoder will automatically know that it should perform the task of

a chatbot or to translate text. And amazingly, we can do this even without explicitly training the decoder for those tasks. We can induce this behavior from the decoder by designing the right prompts which will be presented to the decoder as past tokens from which to continue generating texts.

This is sometimes called prompt engineering. In any case, that will be discussed later on in the section. Okay. So the next question is, what is the architecture of such a decoder? Interestingly, it looks a lot like the encoder. Specifically, it is a series of stacked transformer blocks. As input. We have our usual input sequence of token IDs which get converted into embedding vectors and combined with positional encoding. We then pass this through the series of transformer blocks at the output.

We have a final linear layer. Intuitively, we know that this final linear layer should be used for classification and it should have V output nodes where V is our token vocabulary size. In other words, this is just a standard sequence classifier which you have already seen. The main difference is that instead of using plain multi head attention, we use multi head attention with masking. This is also known as causal self attention. It is causal because it requires the model to only look at the past.

However, one crucial point is that this isn't technically part of the model itself. Instead, we induce this causality by inserting a specific mask ourselves, as mentioned at the start of this lecture. I think this is the most challenging part to understand. So one thing I want to mention before we start to discuss how masking is done is that we don't really need to do things this way. Let's think about this from a more first principles perspective. In order to build a causal language model, special masking is not technically needed.

We simply pass in as input a batch of partial sequences of token IDs and we make the target the next token ID in those sequences. As mentioned previously, one sentence say the quick brown fox jumps over the lazy dog actually results in multiple effective samples. We have "the" going to quick we have the quick going to brown and so forth. We can just pass in each of these inputs and targets into the model separately, no masking needed. So let's make sure we understand this point before we move on.

The special type of masking we're about to discuss is not really fundamental, but it is a neat trick. We don't need it to build a language model. We just have to make sure that we create our inputs and targets in the correct manner. So the trick is to understand how the attention weights can be modified to force the network to only pay attention to the past. Even if we pass in the whole sequence. Recall that our attention weights are a T by T square. Alpha ij is the amount that output i pays attention to input token j.

Consequently, we only want alpha ij to be greater than zero when i is greater than or equal to J. And we can accomplish this by setting the values of the mask appropriately. Note that equality is allowed because we also shift the sequence so the target is a shifted version of the input. What is important and useful to know is that when we implement this, notice how we would like only the lower triangular portion of the attention matrix to have non-zero values.

Now there's one really cool thing that I like about this trick. Let's think about what it allows us to do. To understand this. It is again helpful to return to RNNs. Consider how we might train an RNN to predict the next token in a sequence. One method is to use a simple text classification model where the input is a sequence and the output is the next token in that input sequence. However, as mentioned, this requires us to be intelligent about how we pass in the data.

For instance, consider the document. The quick brown fox jumps over the lazy dog. This can be split into multiple samples. For the first sample, the input is the and the target is quick. For the next sample, the input is the quick and the target is brown. You've seen this before. But can we do this more efficiently? The answer is yes. Suppose that instead of just a text classification model with one output, we have an RNN with T outputs. That is for an input of length t.

The target is also of length T For a regular RNN, this is causal by default because the information always passes from beginning to end. There's nothing special we have to do. What makes this more efficient is that we no longer have to bother splitting up our document into multiple samples. The first target will be quick and it can only consider the input token "the". The second target will be brown and it can only consider the input tokens "the" and "quick".

This is because RNNs only know about the current input and the past inputs. But now let's consider why this doesn't work for a transformer. Well, the reason it doesn't work for a transformer is also one of the reasons why transformers are so powerful. It's because by default, every output node can pay attention to every input token. Therefore, by default, we allow the output at a specific time step to look at tokens from future time steps. And of course, the way to fix this is to mask the attention weights to prevent it from happening.

In other words, what we have accomplished with our special masking is this. The awkward and cumbersome method is to have a text classification transformer with just one output where we have to manually create input samples. But with masking we can have a token classification transformer with t outputs. Any input sequence only needs to be passed in once. We don't have to create separate input samples from this one pass. We can simultaneously learn all of the sub sequences.

So at this point, you technically know everything you need to know in order to build a decoder language model yourself. That's not to say it wouldn't be challenging, but it is possible. You now know more than I did when I first started studying the code for how to do it. If you want to give it a try, that would be great. But if not, that's okay too. It would be interesting if you did because you can make a note of where you got stuck and then compare your thinking now to what it will be after you look at the code in the next section of the course.

11 · Encoder-Decoder Architecture

So now that you understand both the encoder and the decoder in isolation, it's time to combine them into a full sequence to sequence transformer. Now, you technically already know most of the broad strokes. This is basically as far as most resources and books will go. You take the en-

coder, plug the output into the decoder, and simply put, you now have a transformer. But as with the decoder lecture, I want to go deeper than that, especially to discuss details that will actually help you implement this yourself.

So at this point you've probably seen this diagram many times, but it's worth going over. As promised, a transformer is made up of an encoder connected to a decoder. The encoder and decoder transformer blocks are as shown. Notice how the decoder transformer block is not the same as the encoder transformer block. Specifically, it seems to have an extra multi head attention layer. Why is this? Well, if we look closely, we can see that the first multi head attention is of the kind we discussed in the decoder lecture.

Specifically it is a causal self attention using a mask. However, the second multi head attention is just a regular attention. Importantly, we see that it takes an input from the encoder output. In particular, the encoder output becomes V and K for the decoder while Q comes from the previous attention. Now we know that in general the sequence length of the input is not equal to the sequence length of the output. This is one reason why we need sequence to sequence models in the first place.

So how can it be that we can mix a sequence output from the encoder with a sequence from the decoder? Let's convince ourself that it still works. As you recall, the values V and the keys k come from the encoder. The queries Q come from the decoder. Suppose that V has sized T input by DV and K has size T input by DQ. Note that they both have the same sequence length of T input. Q has the size t output by DQ. Now let's consider the first operation to compute the attention output.

We have Q times k transpose. This gives us a matrix of size t output by t input. This is because the inner dimensions disappear. In fact, we don't even need to go past this first step. Note that scaling by the square root of DQ doesn't change the shape, and applying softmax also does not change the shape. Therefore, the attention weights will have the shape t output by t input. As you recall for the attention weight alpha ij, this tells us how much output token I should pay attention to input token j.

In the case of translation, this makes perfect sense. When we are generating the i-th output word, we want to know which of the input words to pay attention to. How many input words are there? Well, there are t input input words for each output. We need to know how much to pay attention to each input. Hence an attention matrix of size t output by t input is exactly what we need. So another important fact about the transformer architecture, which is really not clear from the typical diagram shown before, is which encoder output is connected to which decoder input.

Firstly, notice that beside each of the transformer blocks there is a little index. This means that we are stacking the transformer block N times. In the Attention Is All You Need paper, they set N equal to six. But here's one point of confusion or ambiguity. It's not clear whether we take, say, the first encoder output and connect that to the first decoder input, then take the second encoder output and connect that to the second decoder input and so on and so forth.

Or is it the case that we simply go through all six encoder layers and then take the final encoder output and plug that into each decoder input? The answer turns out to be the latter. So here is an expanded diagram which shows that the data passes through all the encoder layers. First, only

the final encoder output gets passed to the decoder, and it's this same output that gets passed into each decoder input. Another issue we haven't yet discussed is the topic of prediction or inference.

This is more of a topic for the decoder specifically, but since that lecture was pretty long, I've saved it for this one. Note that if you've already studied advanced RNNs with me in the past, then you already know how this works. So the issue is that during training time you've seen that we always pass in a shifted version of the true target into the decoder input. So if the sentence is the quick brown fox, the input becomes start tag the quick brown and fox, while the output becomes the quick brown fox end tag.

However, we have to recognize that this is not how the model will make predictions because when it's inference time we don't know the true target. Instead, it looks a lot like how we forecast a time series. We simply predict one token at a time and then use our previously predicted tokens as input into the next time step. So as an example, on time step one, the decoder input is just start tag and we have to predict "the". On time step two, the decoder input is start tag and the and we have to predict quick.

Now let's say on time step three, the decoder input is start tag the quick. But our model produces the word orange instead of brown. In that case, the next decoder input now becomes start tag the quick orange. We just use whatever the model predicted last. As a side note, there are other algorithms such as beam search which can improve the quality of the output, but that won't be discussed in this course. So that's pretty much it for the full sequence to sequence transformer.

Note that it was relatively short because we already spent a lot of effort building up the groundwork. By doing the groundwork first, the actual transformer itself becomes very simple. You can also imagine how we would train such a model. There would now be two inputs one for the encoder and one for the decoder. For instance, for translation, the encoder input would be the input sentence, while the decoder input would be the target sentence shifted to the right so that it starts with the start token.

The target would be the translated sentence not shifted, but offset by one relative to the input. So one single instance of a training sample actually involves a triple of items instead of just an input and output pair. To give you an example of this, consider the input. I am lazy, which we want to translate to French. The raw data is just the English sentence and the French sentence. So you might think the input is the English sentence and the target is the French sentence.

But when we actually train the model, we need an additional input in addition to this encoder input. We also have the French translation offset by a start token, which becomes the decoder input. The French translation then appears again in the target, but with the end token at the end. Again, this is pretty simple given what we know so far. Now, as a side note, this method is called teacher forcing. With teacher forcing, we pass in the true target into the decoder.

So the decoder only has to predict the next output token. Knowing the ground truth translation of the previous tokens. But recall that during inference time the model works differently because we do not have access to the true translation. In that case, the decoder works in true auto regressive fashion by predicting the next token using its own past predicted tokens.

12 · BERT

So in this lecture, we will be looking at some of the specifics of BERT. Note that you already know many of these details having been through the previous lectures in the course. We know that BERT stands for bidirectional encoder representations from Transformers. We know that BERT is an instance of the encoder transformer and we know how it is. Pre-trained And then fine tuned for specific tasks. This lecture is designed to summarize the key points you've already learned and to provide a few more interesting details.

So let's begin by discussing the architecture of BERT. As you know, it's an encoder only transformer. We know what such a transformer looks like. We have the input tokens which are converted into embeddings. To this we add positional encoding. We then go through a series of transformer blocks which contain a multi-head attention and a few other bells and whistles. After these transformer blocks, we have prediction heads which are specified based on the task. To get into specifics.

There are two main model sizes which are compared in the BERT paper, BERT base and BERT large. BERT base was designed to have the same number of parameters as GPT, which came out prior to BERT. This is so that the comparison between them would be more fair for BERT base. We have 12 transformer blocks, a hidden vector size of 768 and 12 attention heads for BERT large. We have 24 transformer blocks, a hidden vector size of 1024 and 16 attention heads.

So as you know, one of the central ideas of the BERT paper is that they used unsupervised pre-training and then fine tuning to produce state of the art results. There were two pre-training tasks which are masked language modeling and next sentence prediction for masked language modeling. Here's how they generated the data. So basically for each token in a document, they give it a 15% chance of being selected for prediction. If a token is selected, then multiple things can happen.

Firstly, there is an 80% chance that the input for that token will be replaced with the mask token. Then there is a 10% chance that the token will be replaced with a random word from the corpus. Finally, there is a 10% chance that the token remains unchanged. So not only does the model have to learn how to replace missing tokens when they are masked, it also has to learn how to detect incorrect tokens. For next sentence prediction, there is also a little detail here that we didn't discuss.

The detail is that it actually includes masked tokens as well. Another detail is that although there are many possible combinations of sentences where the second sentence does not follow the first. The authors created a balanced data set, so the number of positive and negative samples were equal at a 50/50 split. So in terms of the data set, BERT was trained on a data set called BooksCorpus, which has 800 million words and English Wikipedia, which has 2.5 billion words.

This is not so relevant in isolation, but it's useful when you compare it with other models like GPT. The final topic of this lecture is to discuss some of the results from the BERT paper. Students of my courses are familiar with my motto Machine learning is experimentation and not philoso-

phy. When you have questions like Is next sentence prediction a good task? Or is the fact that BERT is bidirectional relevant? The best way to answer these questions is not by theory, but by doing experiments and checking how they perform.

And of course, that's exactly what the authors have done. Looking at this table from the paper, we can see that when we remove the next sentence prediction pre-training task, the performance drops. Furthermore, when we remove bidirectionality, the performance also drops. Note that LTR stands for left to right, so they make the transformer read each input from beginning to end or left to right. Finally, they add a bidirectional LSTM and when they do this, performance drops even further.

So it appears that even the bidirectionality induced by the LSTM does not make up for the fact that it is missing from the transformer.

13 · GPT

So in this lecture we will be discussing the GPT series of models. As with the BERT lecture, we've already been exposed to many of the key points earlier in the course. However, unlike the BERT lecture, there are many details about the GPT series which are very different from what you have learned in the past, namely how we deal with downstream tasks after pre-training for example, like language translation. Now, before we begin this lecture, I want to remind you that this is just a summary.

There are many details we won't cover, but you can always look at extra_reading.txt and look up these details yourself. The GPT-3 paper is 75 pages long, so you can imagine that those details could make up a course all by itself. So let's start by reviewing the main points. The GPT models are decoder only models, which is evident from the acronym GPT. GPT stands for Generative Pre-trained Transformer. So again, we follow this paradigm of pre-training a transformer using an unsupervised task.

In this case, the task is to simply predict the next token, given the previous tokens in a document. What really makes GPT so interesting is that this task is so simple. We don't need a labeled data set. We can just pass in unlabeled text, which is readily found on the internet. The authors describe the lack of labeled data as a bottleneck and present GPT as a way to get around this bottleneck by having it learn useful things using only unsupervised training.

In this case, the simple task of predicting the next word in a sentence. And despite the simplicity of this task, you will see that the GPT models, especially GPT-2 and three, can do some amazing things. Now, as a side note, I want to mention that GPT version one actually came out before BERT. This is why BERT emphasizes that it is bidirectional, whereas GPT only reads documents from left to right. However, as you'll see, GPT-2 and GPT-3 are way more powerful and yet still only read documents from left to right and still only use the same simple pre-training task of predicting the next word in a document.

In terms of the architecture, the original GPT is just like the decoder we described earlier. As input, we have a sequence of token IDs. We convert these into embeddings and we add a positional encoding. It then goes through several transformer blocks. Note that these transformer blocks

use causal self attention so each token can only pay attention to past tokens. In the case of GPT, we have 12 transformer blocks. Finally, we have a prediction head, which is a classifier used to model the next token probability.

So in the GPT paper they provide a diagram that summarizes the architecture of the model for different tasks during the fine tuning stage. It also describes how to structure the input, which you've seen some examples of earlier in this course. For example, with the plain text classification task, we have a start token, then the text we want to classify. Then the extract token, which is really just the end token. We then put this through the transformer and add a final linear layer For the entailment task,

we have the exact same architecture. The difference is that the input is structured differently. Specifically, we start with a premise followed by a hypothesis, but it's still just simple text classification where the input is a single string of text and the output is a class. For the text similarity task, we actually use the same transformer twice. This is perhaps to ensure that the model doesn't use any information about the ordering of the sentences.

So in the first pass, sentence one comes before sentence two, and in the second pass, sentence two comes before sentence one. Therefore, the output of this overall model is symmetric in the inputs. Once we have the output from the transformer, we then add them together and the result goes through a final linear layer. Finally we have the multiple choice task. This is similar to the last task in that we pass multiple different inputs through the same transformer body.

In this case, we format the text so that the question comes first and then one possible answer. So if we have N possible answers, we will have N inputs to pass into the transformer. After going through the transformer, we have one more linear layer, after which we take the softmax over the outputs for each answer. So nothing you haven't seen before. And if you're taking this course, then I'm sure you're comfortable with these concepts. Another interesting thing about GPT is how they did fine tuning.

During the fine tuning stage. They didn't just train the model to perform the specific task. Instead, they trained the model to perform the task while simultaneously also doing more language modeling. We call this the auxiliary language model. So you can imagine a model with two heads, one for language modeling and one for the task. The final loss was the loss for the task, plus the loss for the language model multiplied by some small constant to balance the losses.

Now, as usual, you might be wondering, does this actually work? And the answer is as usual. Machine learning is experimentation, not philosophy. It turns out that it only performs better sometimes. Other times, the transformer without an auxiliary language model performs best. However, in all cases, we see that a transformer without pre-training did not perform as well, and an LSTM with an auxiliary language model also did not perform as well. So here are some details about the original GPT. To pre-train the model,

the authors used the BooksCorpus data. As you recall, this was also used for BERT, but BERT was also trained on Wikipedia. The GPT-1 model had 110 million parameters.

14 · GPT-2

So now let's talk about GPT-2 on a surface level. GPT-2 makes some basic changes. It adds more data and more parameters. But you recall that when the GPT-2 first came out, it was big news. It was seen as so powerful that OpenAI was hesitant to release it to the public. For the first time ever, machine learning was able to generate long strings of very coherent text. Although this sounds almost commonplace today, it's important to keep in mind that this was simply not possible, even during the first few years of the existence of Transformers.

But that's only one of the amazing things about GPT-2. The other amazing fact about GPT-2 is that when the authors measured the performance of GPT-2 on downstream tasks, they did not use any fine tuning. In fact, they did not have to update the model at all. That is to say, they only trained the model to predict the next word in a sentence and magically it was able to perform different tasks like machine translation and text summarization. Now it didn't reach state of the art performance, but the relevant fact is that the model was even able to do this at all.

So how did the authors manage to get GPT-2 to perform tasks that it wasn't even trained to do? The answer lies in how we format the input, sometimes called the prompt. Remember that all GPT-2 does is essentially sentence completion. So given some prompts, the model tries to predict what comes next. One task that might seem familiar to you is text summarization. In order to encourage the model to perform this task. What we can do is after the text we want to summarize add a TL;DR and a colon.

When GPT-2 tries to continue this text, it will naturally produce a summary. So why does this work? Well, perhaps you have seen after browsing online forums such as Reddit, this is what people often do when writing their posts. If you're wondering, TL;DR stands for "too long; didn't read". And since GPT-2 is trained on a large portion of the internet, it has seen this and can replicate that behavior. The same goes for tasks like language translation. In order to perform this task, they prime the model with several example translations in the format.

English sentence equals French sentence. They end the prompt with a final English sentence that they actually want to translate, followed by an equal sign. GPT-2 is smart enough to figure out that it should continue by translating the final English sentence. However, what is pretty cool is that because the inputs are essentially just words, there was a lot of flexibility in terms of how you represent the prompt. For instance, we can prompt the model to do translation simply by saying translate from English to French, and then following that with the English sentence

we want to translate. The authors reason that GPT-2 learns to do this because there are several examples of English to French translations in the Pre-training text corpus. Some examples are shown in the table on this slide, which is taken from the paper. So let's go through some specifics about GPT-2. Firstly, GPT-2 has an order of magnitude more parameters than GPT-1: 1.5 billion versus 110 million. The reason for this will become clear shortly.

Another difference is that GPT-2 doubles the context size or sequence length going from 512 to 1024 tokens. Finally, in addition to a much larger model, they also use a much larger data set. In particular, the authors noted that although there's a large dataset called Common Crawl, which

essentially covers the whole Internet, it was of poor quality. So they created their own dataset called WebText. This data set is based on links from Reddit posts that have achieved three or more karma, basically upvotes.

This results in 8 million documents and 40 gigabytes of data. Now one question we should ask about GPT-2 is why is it so large? Why did the authors decide to create a model with ten times more parameters? The answer is that it simply worked better. Lets recall my famous and annoying rule. Machine learning is experimentation, not philosophy. So what did the authors do? Well, they created multiple different models with different sizes. What they found was that over essentially all tasks, models with more parameters worked better than models with less.

In these charts, the horizontal axis represents the number of parameters, while the vertical axis measures performance. As you can see, performance generally increases as we increase the parameters. So what have we learned? We've learned that there is a whole new topic of study thanks to these new transformer models, which one might call prompt engineering. Essentially, it's a matter of figuring out how to talk to the AI to get it to do what you want it to do.

So to summarize the key points of GPT-2. Firstly, it is still a left to right causal language model like GPT-1. Compared to GPT-1, it uses much more data for Pre-training and has many more parameters, whereas GPT-1 has 110 million parameters. GPT-2 has 1.5 billion parameters, which is more than ten times larger. It also uses a larger context size or sequence length. GPT-1 uses a sequence length of 512 tokens, while GPT-2 has a sequence length of 1024 tokens.

And finally, and most importantly, GPT-2 does not use fine tuning. Instead, the authors show that it can perform specific tasks given a correctly formatted prompts.

15 · GPT-3

Okay. So now let's move on to GPT-3. Basically, it's the same story, more data and more parameters. But whereas GPT-2 was about ten times larger than GPT-1, GPT-3 is over 100 times bigger than GPT-2. It has 175 billion parameters. It also doubles the context size, again, going from 1024 to 2048. In terms of data. You'll recall that the authors of GPT-2 considered using Common Crawl, but found that performance suffered due to quality issues.

In this paper, the authors decided to use it anyway, but to filter it to improve its quality. In addition, they also used an expanded version of WebText called WebText2. They also add two book data sets and English Wikipedia. Now, this might not seem like much since listing out the number of parameters and the data sets that were used really do not portray the scale of this experiment. To give you some sense of this, the authors show a chart comparing the compute required to train several well known models.

So we have BERT Base on the left, followed by BERT Large and then the full GPT-3 on the right. Importantly, note that this is a log scale. So every time you move up a bar you are multiplying by ten. If you move up three bars, you are multiplying by 1000 or ten to the power of three. So to give you some sense of how large that is, consider a model that takes one day to train. Not so bad. Now, suppose it takes 1000 days to train. That is now nearly three years.

Of course, the authors use distributed training over many GPUs, which reduces the time. So as with GPT-2, the only training that happens is during the unsupervised training phase where the model learns to predict the next token in a sequence given the previous tokens. Again, there is no fine tuning step. However, the authors hypothesize that the model will indirectly learn to do various different tasks simply because they show up in the data set. In the paper they show a diagram where they call unsupervised pre training the outer loop.

Within this, the authors hypothesize that some in context learning is happening at the same time. For example, the model will see examples of arithmetic, and that explains why the model can do arithmetic. The model will see examples of spelling correction, and that's why the model can do spelling correction. The third example is with translation. So the authors call this the inner loop. So this explains how models like GPT-3 are able to perform tasks without any task specific data set or fine tuning.

So you'll recall that with GPT-2, the authors primarily used zero-shot transfer. That is, they prompted the model with a task but did not provide any examples of how to do that task. With GPT-3, they extend this by testing zero shot, one shot and few shot transfer. Basically, these names refer to the number of examples that are given in the prompts. So they are not training examples, they are prompt examples. So as an example, they use language translation for zero shot.

They simply say Translate English to French. Start a new line, say cheese and then use an arrow. And then the model completes the rest for one shot learning. You can see that they extend this by adding a new line before the final line, which is an example of what to do. In this example, we can see that it's a translation of sea otter. Finally for a few shot, they extend that again simply by adding more examples. So how well does this work? Well, it turns out that this works surprisingly well.

Keep in mind that GPT-2 worked well, but generally not as well as existing state of the art methods that were trained specifically for the given task. What was surprising about GPT-2 was that it was even able to do that to begin with. With GPT-3, there are even more surprises for a task called TriviaQA, which is a question answering task. We see the performance for a zero shot, one shot and few shot. On the horizontal axis, we have the number of parameters.

Observe that few-shot learning in the largest GPT-3 model actually beats the existing fine tuned state of the art. This means that GPT-3 being trained only to predict the next word in a sentence was able to perform better on a question answering task compared to a model trained specifically for that task.

16 · Theory Section Summary

So in this lecture we will summarize what we learned in this section of the course. This section was all about the theory behind attention and transformers about how they work and what makes them work so well. To that end, we discovered two key characteristics that make Transformers work well compared to previous deep learning technologies. Firstly, we use the new operation called Self Attention, which allows the neural network to model long range dependencies.

This solves the problem of vanishing gradients that we see in RNNs. Secondly, we leverage massive amounts of data by using unsupervised pre-training. By doing this, we can use more data compared to what we would have if we only used a supervised data set. In this section, we learn how to build a transformer from the ground up using only basic operations like multiplication, a softmax and so forth. We learned about the basic scaled dot-product attention and how to combine these to get multi head attention.

We also learned about how to add masking to give us causal self attention which is used in the decoder. One important fact we learned about attention is that the operation is quadratic. We now understand why this is and what it means. We also learned about the positional encoding, which is necessary since attention has no concept of order. It is permutation invariant. Once we learned about these fundamental building blocks, we learned about different ways to combine them.

Specifically, we learned about the encoder, which is similar in functionality to a plain vanilla RNN that we might use for classifying text. We then learned about the decoder, which is similar in functionality to a language modeling and text generation RNN. And then we learned about the encoder decoder, which is similar in functionality to the sequence to sequence RNN. We then learned about specific models, namely BERT, GPT-1, GPT-2 and GPT-3.

BERT is an encoder model and we've already seen that it's useful for tasks like text classification, token classification and question answering. But importantly, we now understand why based on how the model is actually built. We then saw that all the GPTs are decoder models. We learned the surprising fact, which is that GPT models only learn to do a very simple task, which is to predict the next word in a sentence. Perhaps the most surprising fact is that GPT-2 and GPT-3 do not use any fine tuning at all.

It appears to simply know how to do specific tasks based on the prompt you give it. Now it's important to consider the implications of a model with these abilities. I think most people agree that this is not necessarily an example of reasoning. That is, GPT-2 and three did not learn how to add or answer questions, but rather it's more likely that these models are so large they learn a distributed representation of the training dataset. In other words, they've memorized the training corpus.

But this is not memory as in a direct copy in a database, but rather memory in terms of neural network weights. Finally, it's helpful to note that, as with CNN's, there are many different variations of transformers, so many, in fact, that it's impossible to study them all. However, the goal of this section is to provide you with the fundamentals such that exploring those variations will be easy. So at this point, if you want to go ahead and read the literature to see if there's a transformer that fits your specific needs, you can do that.

One example of this is the Longformer, which we did not use in this course. As you recall, attention has quadratic time complexity, so it has a limit in terms of the sequence length it can handle. The Longformer tries to solve this by introducing a variant of attention that is linear complexity. Consequently, it can handle longer sequences. Another example is the sparse transformer, which has complexity of N times the square root of N. So if you're interested in learning more about different kinds of transformers, you are now ready to read some papers.

And if you want to learn more about any of the models discussed in this section, check out the papers listed in extra_reading.txt.

1 · Implementation Section Introduction

So in this lecture, we will be introduced to the next section of the course, which is all about how to implement transformers from scratch. Now, sometimes students get confused about what from scratch really means. I once came across a blog article that stated it was going to implement Decision Trees from scratch. But then in the actual code they used scikit-learn. Now, since this is an advanced section of the course, I will assume that you are an advanced student.

As such, I think you should have a pretty good idea of what we mean by from scratch. So maybe let's present this as a little quiz. Please take some time to consider. What do we mean by from scratch and come back when you've written down some ideas? For instance, consider whether we'll be using pure numpy or should we be using a deep learning library like TensorFlow or PyTorch? Also, consider what level of operations we're going to implement in terms of high level or low level.

Should we implement multi-head attention or should we use a multi head attention layer? Or better yet, perhaps we should find a transformer layer which already has multi head attention within it for comparison. In some prerequisite deep learning courses we use the LSTM and GRU layers for RNNs. So consider what we mean by from scratch. And I'll see you on the next slide. Okay. So in this course, when we say from scratch, we definitely do not mean using a library like numpy at this level.

It would simply be infeasible not to use a modern deep learning library like PyTorch. In this section we'll be using PyTorch. As such. It should go without saying that PyTorch basics are a prerequisite to this section of the course. You should know how to create different types of neural networks and how to train them on a data set. However, one thing we won't be doing is implementing tokenisation from scratch. As you recall, these follow algorithms of their own, which we haven't studied in this course at this point.

So instead, note that Hugging Face actually uses PyTorch as a back end so most things can interoperate. Now in terms of whether this will be high level or low level, we will not be using any pre-made layers such as multi-head attention or encoders or decoders. Instead, we will build all of these from scratch. This is the same kind of strategy we used earlier in this deep learning series, for example, with LSTMs. At the same time, this shouldn't be intimidating because, as you recall, attention really only involves operations we already know about like matrix multiplication and softmax.

We will use layers like the linear layer to build ANNs, activation functions, layer norm and dropouts. Of course we don't want to implement these from scratch, so as long as you understand how to do matrix multiplication and softmax, you should be okay. Now recall that one of the most important concepts I emphasize when we work with images and sequences is understand-

ing shapes. This will be especially true in the following lectures. So if you haven't been keeping up in your understanding of the shapes of all the objects we are working with, it may cause you some trouble.

At the same time, there's nothing to worry about since if you get stuck, you can always revisit the previous sections of the course. So let's go over a basic outline for this section and what to expect. Basically, in this section we will implement three different models. I think you have a pretty good idea of what they might be. Number one will be the encoder. Number two will be the decoder. And number three will be the encoder and decoder together. Very simple.

If you've been through the previous section, then this probably seems obvious. And if it does, that's a good sign. Now, each of these three exercises will present different challenges. The encoder by itself is the simplest model, so the challenge is really just context. Switching from what we were doing earlier in this course to what we are doing now. It's very different. Earlier in this course it was all about how to use an API, but in this part of the course it's all about understanding how things work.

So just getting your mind to switch gears might be a challenge all by itself. Another challenge is actually building all the parts from multi-head attention to the positional encoding to the transformer block. This will really apply what you learned in the earlier lectures. However, building all these components for the encoder makes it very easy to create the decoder. This is because most of it will be identical. The reason the decoder is challenging is because it requires a completely different training and inference paradigm.

And finally, the encoder decoder will also be challenging because it requires us to combine the two networks together. Conceptually, that sounds easy, but once you try yourself, you'll see that there are details you haven't considered. For all of these examples, note that we will be training them all from scratch with no fine tuning. This should be sufficient to understand how they work. And keep in mind that what we build is much smaller than, say, BERT or GPT, which we are definitely not trying to recreate.

Remember that these models take significant resources to train and the free version of CoLab would simply not be enough. At the same time, this is still far beyond what you would see in a typical blog post, which might show you the basics of how to build the model but not how to train it or use it, which I think is pretty important.

2 · Encoder Implementation Plan & Outline

So in this lecture, we will be discussing a plan for how we will approach building the encoder transformer from scratch. Now, as you recall, PyTorch is very modular. In fact, there is a class which will be using called nn.Module. This can represent any size component from a simple activation function or a dense layer to an LSTM or transformer block. The key point being no matter how high up the hierarchy you go, everything is still just a module.

To that end, let's discuss some of the modules we might need to build. Firstly, we're going to need a module for multi head attention. This is where all the math we learned in the theory section will reside. Now please note that although PyTorch does implement multi head attention and

even the whole encoder and decoder layers, that is not what we want to use as that would kind of destroy the purpose of this exercise. Now as you know, multi head attention is just part of the transformer block.

Other components involve layer norm and our feedforward ANN. And so the transformer block is also a module we need to build. Another component will need is the positional encoding. Luckily this can be directly reused in all the subsequent examples. And finally, we'll want to create an encoder module which will consist of transformer blocks, a positional encoding and a few other basic modules. So that seems like a pretty simple list. Multi-head attention, transformer block, positional encoding and finally the encoder itself.

Okay, so I realize that some of you might prefer to think of this using a top down approach, while others might use a bottom up approach. I myself kind of went back and forth, so I want to present both approaches in this course. In the following lectures, we'll follow a mainly bottom up approach building the main components and then putting them together. But in this lecture, I want to give you a better idea of how we will use those components at a high level.

So if you prefer the top down approach, this is for you. So firstly, let's recognize what is a low level component that includes basically multi-head attention and positional encoding. Inside these modules. We're just doing math. The high level components include these components and other built in modules. So suppose that we have a multi-head attention layer. How can we build a transformer block? Well, that's pretty simple. A transformer block is just a multi-head attention followed by layer norm after adding a residual connection.

And then once we've done this, we pass the result through an ANN, followed by another layer norm again after adding a residual connection. So we need to create four components in the init method: multi-head attention, two layer norms and an ANN. In the forward method, we just implement the operations we just described. As a side note, we've assumed that multi-head attention takes in three separate inputs for Q, k, and V, but for self attention these are all the same inputs.

Okay, so that's the transformer block. Once we have that, we can build the whole encoder. So what makes up an encoder? Well, we have some input sequence of categorical objects. We pass that through an embedding and we add the positional encoding. Once we have that, we have a few transformer blocks, which we just described, and then we have, say, a final linear layer if we want to classify text. So how do we implement this? Again, it's pretty simple. Inside the init method,

Again, we just construct the modules we need. That's the embedding, positional encoding, which we assume we've created, a list of transformer blocks and a final linear layer. Inside the forward method, we simply apply those modules in sequence. Now, once we've built the model, we aren't done. We still need to make sure it actually works. And we can do that by training it on a data set and checking its performance. Now, if you're thinking about how you'd actually do this, which I hope you are, you'll realize that something is missing.

We know where to get text data sets since we've used them earlier in the course and we know how to build a transformer, presumably. But we still need to preprocess the text and convert it into a numerical format. We've called this tokenisation, although technically it involves more steps than just tokenizing the text. In fact, we're in luck. We're in luck because Hugging Face actually uses PyTorch in its back end. So many of its components are compatible with PyTorch.

As such, we'll be using the Hugging Face tokenizers and datasets APIs throughout this section. Otherwise, we would also need to learn about how tokenizers work if we wanted to build that from scratch too, and that would be another course all by itself. In any case, using Hugging Face datasets and tokenizers, we'll be able to get our data into a format that we can pass into any PyTorch model. The next challenge will be to write a training loop. In TensorFlow,

this is easy because you can just call model fit, but in PyTorch you need to do this manually. This is where your knowledge of the prerequisites is required in this course. While I will explain everything line by line like I normally do, I probably won't be going into as much detail as I did in my PyTorch course, since that won't make sense at this stage. In any case, that just about covers all of the steps of the upcoming notebook. So I'll see you there.

3 · How to Implement Multihead Attention From Scratch

So in this lecture, we will be describing some of the key details behind the multi-head attention implementation in PyTorch. The reason we're doing this lecture is because it doesn't follow directly from the theory, as you might expect. I think the simplest way to do this will be to just show you what the code looks like and then to dissect it piece by piece. So I'm sure you're familiar with these imports. No need to explain these. So now we have the multi head attention class.

So we'll begin with the init function which takes in d_k, d_model and n_heads, which is the number of attention heads. Note that we will assume that d_k will be the same as d_v, so there's no need to pass in DV as well. Now just remember what we're trying to implement. We're trying to implement scaled dot-product attention, which is Q times K transpose, scale that, take the softmax and multiply by V. In this case, Q, K and V are just matrices multiplied by the same inputs.

And remember that we actually want to do this n_heads times. However, you'll recall that in Python and numpy and so forth, we always try to avoid for loops. So we don't want to do something like create a list of n_heads matrices for q, k and V do attention of that many times and then concatenate the results. Instead, it would be better to do all of the attentions all at once. To that end, we're going to create three linear layers, one for the key, one for the query and one for the value.

These represent our matrices from the theory lectures which are WQ, WK and WV. Now, you'll recall that linear layers don't just multiply by a matrix. They also add a bias vector. Technically, you can turn this off if you want to match the paper. Exactly. However, it's actually customary to just leave this in. Now, the shapes that get passed into these linear layers are important to make note of. So in theory, our input is of size d_model and the output is of size

d_k, and we want to do this n_heads times. So all of our attention operations happen on vectors of size d_k. But in the code you can see that the output is of size d_k times n_heads. This is because we're going to do all n_heads attentions all at once. The next step is to create the final linear layer, which, as you recall, is like a final projection after we concatenate all of the heads' attention outputs. Note again that we keep the bias vector. Also notice that the shapes are the opposite of what we have above.

The input is of size d_k times n_heads while the output is a size D model. This makes sense since before applying the final projection, each attention will output a vector of size DK and we have n_heads attention heads. The output should be of size d_model, since that's the size of every input and output in our transformer. So now let's have a look at the forward function. I would say that this is one of the most challenging parts of this section, but it's also one of the most crucial parts.

If you understand this, then you really understand attention and how to implement deep learning concepts in general. Firstly, note that as input into this function, we take in four values. Q, K, V and the mask. Note that these Q, k, and V values are not the ones that go into the attention equation, but the values we have before we multiply by the W matrices. If we're talking about the simple encoder with self attention. Q, k, and V would all be the same X input.

Also note that the mask is used since the input may have padding. So we begin by transforming Q, K and V, but now they represent the Q, K and V in our attention equation. As usual, it's important to track the shapes if you really want to understand what's going on. Firstly, the input is a batch of sequence data, so that will have the shape N by T by d_model. After going through these linear layers, they will be N by T by d_k times h. As you recall, we use the letter H to mean the number of attention heads and also remember that DQ is the same as DV, although I've written them separately.

The next step is to just store some of the shaped dimensions, namely the batch size and the sequence length. We'll call these n and T. Okay, so now this is the hard part. Our goal right now is to perform our self attention operation, which is the softmax of Q times K transpose scaled and then multiplied by V. What's confusing is how to do this when we don't actually have matrices. In the theory lecture we had matrices, but now we have sequences of size N by T by d_k.

In fact, we're just about to make things worse. Instead of three dimensional tensors, what we actually need are four dimensional tensors. You'll just have to trust me at this point and we'll see how it all works out at a later step. Note that Q, K and V are all reshaped using the same method, so it suffices to explain this once. The first thing we do is call the view function. This lets us reshape our data. Previously we had N by T by h times d_k. This reshaping splits up the different heads so we get N by T

by h by d_k. Now, in the next step, we're going to do a matrix multiply. But in order for this to work correctly, we need to change the order of the dimensions. Specifically, we need the h dimension to be second and the T dimension to be third. Consequently, we call transpose passing in one and two, which will switch the dimensions at index one and index two. And so the result is N by h by T by d_k. So the next step is where we compute the first matrix multiply that appears in attention, which is Q times K transpose.

You'll see that this is almost what we've written. Recall that the @ symbol is a new notation we use for the dot product, and the dot product is basically a more general version of matrix multiplication that also does broadcasting. So what's happening here? Firstly, you can see that we are transposing yet again. Specifically, we are transposing the last two dimensions of K at index minus one and index minus two, meaning we are switching these two dimensions.

We'll explain why that's needed soon. But first let's look at what the resulting shape will be. So remember that currently, Q, k, and V all have the same shape, N by h by T by d_k. If we switch the last two dimensions, we get N by h by d_k by T. So why do we want to do this? Well, basically just know that the dot product is like a matrix multiply and it also broadcasts. So how does the broadcasting work? Essentially, you can think of the last two dimensions as the matrices we're going to multiply and think of the first two dimensions component wise.

So the first two dimensions are N and H. So for all of the samples in all of the heads, we're going to do a matrix multiply between a matrix of size T by d_k and a matrix of size d_k by T. Now what would we get if we multiply two matrices of this size together? The result would be a T by T matrix. As you recall, the inner dimension always disappears and the inner dimensions always must match. In this case, the inner dimension is d k. So this is why we needed to do that transpose on K.

Now what is the result? The result is a new four dimensional tensor of size N by h by T by T. We then divide by the square root of d_k, which is just a scalar. The next line is a bit challenging, but not as challenging as the previous step. What we'd like to do next is apply the mask. Now, as you recall, the mask is a tensor of ones and zeros of size N by T. For each of the N samples,. We need to know which of the T tokens are actual tokens and which are just padding.

So it's two dimensional instead of four dimensional. So we can't apply the mask directly, say by multiplying the attention scores by the mask. But what we can do is add Nones to the middle dimensions, which will introduce superfluous dimensions of size one. So what we would end up with is going from N by T to N by one by one by T. This allows us to broadcast correctly. And so when we call the masked fill function, it applies the mask in the appropriate places.

Basically, wherever the first argument is true, apply the value of the second argument, which in this case is minus infinity. As you recall, we use minus infinity because the next step is to take the softmax. When we take the softmax, we exponentiate, and the exponent of minus infinity is zero, which will give us an attention weight of zero. As you can see, the next step is indeed to take the softmax of the attention scores, to get the attention weights.

So now that we have the attention weights, the next step is to multiply by the values V. Luckily, this can be done directly without any reshaping or transposing. Again, we apply the same rule as we learned above. The first two dimensions are component wise. So for each of the samples and for each of the attention heads do this multiply. And what do we want to multiply? We want to multiply a matrix of attention weights of size T by T by a sequence of value vectors of size T by d_k, and the result is T by d_k, or in full

it would be N by h by T by d_k. So now the next step is to do our final projection. But in order to do that, our data needs to be in the right shape. Basically, we're now going to do the reverse of what we did above. So we first transposed index one and index two. So the result is N by T by h by d_k. We want to have N and T be the first indices because all of our sequences have the shape N by T by something. The next step is to reshape this and combine the final two dimensions.

In other words, we get n by t by h times d k. Note that we also use contiguous. Don't worry too much about this as it's a lower level operation. Basically, it ensures that the data is laid out in memory correctly. And so the final step is to apply our projection layer, at which point we can return the results.

4 · How to Implement the Transformer Block From Scratch

So in this lecture, we'll continue our notebook with the transformer block. This will be pretty simple since we covered the basics in the intro. So now all we have to do is create the layers we need and then apply those layers in the right order. So first we have the init method. This takes in d_k, d_model and n_heads, all of which you've seen. We also have dropout, so this will take in the dropout probability. Note that this will be a recurring theme in this section.

Dropout will appear essentially everywhere, even though we didn't discuss it in the theory lectures. As you recall, dropout is used for regularization. In fact, there are even more places I could have added dropout. But I didn't want to make this too complicated. So inside the init method we begin by creating our two layer norms. The next step is to create a multi head attention layer, which we just wrote. Next we have the ANN. This has a hidden unit size of D model times four and a ReLU activation. In addition,

note that this also ends with a dropout. Finally, we create one last dropout. So the forward function is quite simple as well. This takes in some input sequence of size N by T by d_model and some mask of size N by T. Inside the function, we first apply multi-head attention passing in x as the query key and value along with the mask. We then add the residual connection and then apply the first layer norm. Next we pass the result through our ANN, then add the residual connection and apply the second layer norm.

Finally, we add dropout and then we return the result.

5 · How to Implement Positional Encoding From Scratch

So in this lecture, we will look at how to implement the positional encoding from scratch as before. This is a module, so we'll need the init method and the forward method. Note that this code has been adapted from one of the official PyTorch tutorials, except the dimensions have been transposed to be in the order that we're used to. That is N by T by D, where the batch size comes first and the time dimension comes second. In the PyTorch tutorials they tend to put the time dimension first.

So the init method takes in three arguments. d_model, max_len and dropout probability. Note that max_len is needed because that determines how many values we're going to precompute. The only actual layer we need is dropout, which we create first. The next step is just some computation to implement the positional encoding formulas. We'll explain it, since it's not exactly trivial. So we begin by creating a variable called position, which is just the list of integers from zero up to max_len.

Basically, this represents the pos variable in the formula. We also call unsqueeze(1). So this adds a superfluous dimension of size one at the end. So instead of a 1D array of size max_len we have a 2D array of size max_len by one. The next step is to create a variable called exp_term. This is just the integers from zero up to d_model, but counting up by two. Basically, this represents the 2i term in the exponent of the denominator. So, whereas the position variable counts across time, the exp_term variable counts across the feature dimension.

Next we have the div term which completes the denominator. Now, this is probably the most challenging part to understand. So in the equation we have 10,000 to the power of 2i divided by d_model. But in the code we have this very weird looking thing with the exp and the log and so forth. So why is this? Basically, this just makes things more numerically stable. 10,000 is a very big number, whereas E is not. It's about 2.7. And note that we also take the log of 10,000 and the log of a big number is usually a much more reasonable number.

Essentially, if you do the math, everything works out. So first we have exp of two times minus log of 10,000 divided by d_model. Now suppose that we bring the minus 2i divided by d_model outside. Then on the inside we have exp of log of 10,000, but exp and log are inverse functions. So we just end up with 10,000. And this is closer to the equation we have in the paper. So next we have pe, which we initialize to an array of zeroes of size one by max_len by d_model.

Basically, the first dimension of one makes this broadcastable because remember, we are going to add it to the embeddings of size N by T by d_model. Next, we assign the correct values to pe based on our formula. As you can see, it's sine of position times div term and cosine of position times div term. Note that we multiply instead of divide because we put a negative sign in the exponent. Okay. So one part that might be a little confusing about this is how we index pe, specifically the zero colon colon two and one colon colon two.

Basically, as you recall from your python studies, the syntax is start index, colon end index, colon step size and end index is optional. So for the sine portion, this is going to index zero two, four, six, eight and so forth. For the cosine portion, this will be one three, five, seven, nine and so forth. And finally, we call a register_buffer on pe. Basically, this is to be able to save and load the model correctly. So the module knows that this variable should be saved.

So next we have the forward function. This takes in an input of size N by T by D. Now, as you recall, PyTorch models can accept variable size sequences, so t might be different in each batch. As such, we can obtain the current sequence length by calling x.size(1) and we only need to index pe up to this point. Once we have that, we add it to x, call dropout and return the results.

6 · How to Implement Transformer Encoder From Scratch

So in this lecture, now that we have created all the building blocks we need, we can now build a full encoder. As usual, this amounts to filling in the init method. In the forward method. In the init method we take in the vocab size, maximum sequence length, d_k, d_model, number of attention heads, number of transformer blocks, number of classes at the output and dropout probability. Inside the init method. We create all the modules we need. We begin by creating our embedding which will convert our tokens into vectors.

As usual, this has the shape vocab size by d_model. The next step is to create our positional encoding, which we just built. The next step is to create our transformer blocks. We'll also encapsulate these transformer blocks in an nn.Sequential. Finally, we create one last layer norm and one last linear layer, which will both be used after the final transformer block where the final layer has n_classes outputs, because that's how many classes we have to choose from.

Okay, So next we have the forward method. This takes in some sequence input x and a corresponding mask. Inside the function. We first pass our sequence of tokens through an embedding. We then add the positional encoding. Next, we go through each of the transformer blocks, passing the mask through each one. Note that we couldn't have done the alternative, which is to pass X directly into the transformer blocks. This is because the mask needs to be passed through each block.

Next, we have an optional step, which depends on the type of task you are working on. In this case, we'll assume we're just doing text classification and so we only need to keep one of the final hidden vectors. As you know, by convention, we keep the first. So this goes from N by T by D to N by D. Finally, we apply a layer norm and one final linear layer, and then we return the result. Okay, so now let's test our encoder. We'll create an encoder with some dummy values.

Here, we've set the vocab size to 20,000, the max length to 1024, d_k to 16, d_model to 64, n_heads to four, n_layers to two, n_classes to five and dropout probability to 0.1. So that works. The next step is to test our encoder with some actual inputs to see if it runs without any errors. We'll begin by moving our model to the GPU if it exists. The next step is to create a random sequence of integers, which represents a sequence of tokens. So this will give us a sequence of tokens of size eight by 512 containing values from zero up to our vocab size.

We'll then convert this array to a torch tensor and move it to the GPU. The next step is to create a corresponding mask. Note that it has the same shape as the input, which is N by T. We set a few values to zero and then we convert it to a torch tensor and move it to the GPU. So the next step is to pass our data along with the mask through the encoder. So that works. Now let's check the shape of the output. So it's eight by five. So does this make sense?

Yes, this makes sense since we have a batch size of eight and five classes.

7 · Train and Evaluate Encoder From Scratch

Okay, So in this lecture, we will actually use our encoder on a real data set. Now, technically, we could just load in any data set and use any tokenizer, for instance, like what we did in the PyTorch course. In fact, you could even use TensorFlow's tokenizer if you wanted, which is actually much nicer than what comes with PyTorch. However, since we know that Hugging Face uses more modern tokenization methods, we'll use this along with data sets package as well, which works nicely with PyTorch.

So we begin by installing transformers and data sets. Next, we import AutoTokenizer and DataCollatorWithPadding. So next we set our checkpoint to distilbert-base-cased, and we load the tokenizer from this checkpoint. Now, you might wonder, why did I choose DistilBERT and not just BERT. So one reason, as you recall, is BERT also uses segment embeddings, and it uses these to distinguish between the first sentence and the second sentence in the case where two sentences are passed in.

However, we didn't build any such embeddings, and as such, it's simpler to just use DistilBERT, which also does not use segment embeddings. So the next step is to import the load_dataset method. And the next step is to load in our data set. So for this example, we'll keep things simple and use SST-2, which you've already seen. As you recall, this is a data set for a sentiment analysis, and it's part of the GLUE benchmark. So the next step is to remind ourselves what this data set looks like.

So as you can see, we have a train set, a validation set and a test set. And each data set comes with three columns: sentence, label and idx. So the next step is to create our tokenized function, which applies the tokenizer to the sentence column of a batch of data. The next step is to apply our tokenized function to our data sets. We'll also create a data collator at this point. So the next step is to just print out the data collator, just to remind ourselves what's in it.

So as you can see, it includes information like vocab size, maximum sequence length, which side the padding goes on, which side we truncate on, and so forth. So the next step is to look at our tokenized data sets. So as you can see, some new columns have been added. There are now two new columns called Input IDs and Attention Mask. Input IDs are the token indices and Attention Mask tells us which tokens are real tokens and which are padding. So the next step is to remove obsolete columns from our data sets, including sentence and index.

We'll also rename label to labels to work with the PyTorch data loader, which we'll see very soon. So the next step is to print out our modified tokenized data sets. And as you can see, we now have three columns which are labels, input IDs and Attention mask. So the next step is to return to the PyTorch world and create data loader objects. The main difference between a train and validation is that for training we set shuffle equal to true. Also, note that we can pass in our data collator at this step.

So now let's look at what our data loader returns by printing out the shape of each item in the first batch. So we have a loop here and then we just break on the first iteration. So as you can see, the input IDs are shaped 32 by T and the attention mask has the same shape. The labels are

just a 1D array of size 32, which makes sense since it's just sentiment analysis. So the next step is to check some of the values we'll need to pass into the encoder when we create one for this data sets.

We'll first check the number of distinct labels by casting the training labels to a set. So as you can see, we get back only zero and one. Which are the two classes? So the next step is to check the vocab size. As you can see, we have 28,996 tokens. The next step is to check the maximum sequence length. So basically for DistilBERT, it's always 512. So the next step is to actually build our encoder. Note that for some of these values, like DK, d_model and heads and so forth, I've just chosen these randomly, so feel free to play around with them yourself.

And again, we move the model to the GPU. So the next step is to create a loss and optimizer objects. Note that Hugging Face also includes an optimizer called AdamW, which is commonly used for Transformers. However, to keep things simple, we're just going to use regular Adam. The next step is to import datetime, which we'll use during training. So the next step is to define our training function as input. This takes in our model, criterion, optimizer, train loader, test loader and number of epochs.

Inside the function, we begin by initializing arrays to store our train losses and test losses. Next we enter a loop that runs epochs times. Inside the loop, we set the model to training mode and get the current time. We also initialize the train loss to zero and number of training samples to zero. Next we enter another loop that will loop through each batch in the train loader. Inside the loop, we begin by moving each tensor in the batch to the GPU. The next step is to zero the gradients.

The next step is to perform the forward pass and compute the loss. As you recall, the model takes in two inputs, which are the token IDs and corresponding mask. The next step is to call loss.backward() to compute gradients, and then optimizer.step() to do one step of gradient descent. Finally, we add the loss to our train loss. Now, importantly, note that the loss computed by PyTorch is the average cross entropy over the batch. However, in order to compute the average across the whole training set, we need to convert this back into a sum and then divide by the total number of training samples once the loop is complete.

So this is why we multiply by the batch size and also add the batch size to n_train. Once we've completed the loop, we then compute the average train loss. So the next step is to put the model into eval mode. We then initialize the test loss to zero and the number of test samples to zero. Then we loop through each batch in the valid loader. Inside this loop we go through all the steps again except for the validation set instead of the train set. In addition, we don't compute gradients, nor do we do any gradient descent, since we just want to evaluate the model on the validation set.

So once we finished evaluating the model, we then save the train loss and the test loss just in case you want to plot them later. We also print some information, including the epoch number, the train loss, the test loss, and the time it took for the epoch. And finally we return the train losses and test losses. So now let's run the train method. Okay. So as you can see, this completed relatively fast. The loss has decreased mostly in the train set and only slightly on the test set.

Now, since this is a classifier, what we are really interested in is how accurate the model is. We begin by putting the model into eval mode once again, although this is redundant since we did that on the second part of the training loop. The next step is to initialize n_correct and n_total to zero. As you recall, the accuracy is just n_correct divided by n_total. Next, we enter a loop that goes through each batch of the training set. Inside the loop, we move each item in the batch to the GPU.

The next step is to get our model outputs, which, as you recall, our logits. The next step is to get the prediction from the logits. Note that we can use the max function for this, which returns both the max and the argmax. The next step is to update n_correct and n_total. For n_correct, we just add the number of predictions that equal the labels and for n_total, We just add the number of samples in the batch. Once we're outside the loop, we compute the train accuracy by dividing n_correct by n_total.

The next step is to do the same thing for the test set. And lastly, we print our accuracies. So as you can see, we get about 93% on the train set and 80% on the test set. As an exercise, you might want to try computing other metrics like the F1 and AUC, since it could be that this data set is imbalanced.

8 · How to Implement Causal Self-Attention From Scratch

So in this lecture, we will be moving on to the decoder. As you'll see, most of this is surprisingly similar to the encoder with just a few minor changes. We'll begin with the same imports as the previous notebook. So the next step is to implement a new module which will call causal self attention. You'll notice that this is very similar to the multi head attention we previously implemented. The main difference is that we need a special mask so that each output can only pay attention to past tokens.

So firstly, notice that the first few parts of this are exactly the same as the encoder. We still have the same inputs, except that we also take in max len, which is needed to build our causal mask. However, DK is the same and heads is the same and all the layers are the same. Remember that the architecture of causal self attention is exactly the same. We're really just adding a mask before we apply the softmax. So this part is the same. The layers are the same. The final layer is the same.

And then we have something new. So you'll recall that we discussed in the theory lectures what this mask should look like. Basically, it's a mask where the lower left triangle is all ones and the diagonal is all ones, and then the upper right triangle is all zeros. It's also a size T by T, or in this case, max_len by max_len. Note that we can create such a matrix by using the PyTorch function tril, which means the lower triangular. There's also a function for upper triangular, which does the opposite.

So as with positional encoding, we call register_buffer so that this tensor is saved if we ever save the model. And notice that we also reshape the matrix to be of size one by one by max_len by max_len. Basically this is because, as you recall, during attention, our data is four dimensional

and so the mask also needs to be four dimensional in order to broadcast correctly. So now we have the forward method, which is almost the same as before. Note that we also take in a mask, but this is a different mask than the one we just created.

The mask we just created was to make sure that the outputs only pay attention to past inputs. So that is our causal mask. On the other hand, the mask we take in as input is our padding mask, which is used to handle padding in the inputs. So most of this is the same as the encoder. We basically just have one new line where we apply the causal mask. And so just to go through this very quickly, so we have Q, K and V going through the Q, K and V linear layers. We get N and T, which are the first two parts of the shape. We reshape and transpose Q, K and V, and then we compute the attention weights.

So it's Q times K transpose over here, then we add the padding mask. So the padding mask is exactly the same as the encoder mask that we just looked at in the previous lectures. And so the new part is where we do the causal mask. So as you can see for the causal mask, we index the causal mask so that it only goes up to T, which is the sequence length of the current batch. And we still use the masked_fill function and we still do the softmax afterwards.

So everything other than this part where we do the causal mask is still the same. So since this lecture has been pretty short, we'll move on to the next few parts, which are also pretty simple. So the next module is the transformer block, which is nearly identical to the encoder. The main difference is that we also need to accept max_len as input since this needs to be passed on to the causal self attention module. In the forward method. We've also renamed the mask to pad_mask just to make things a bit more clear.

And next we have the positional encoding, which is also exactly the same.

9 · How to Implement a Transformer Decoder (GPT) From Scratch

So in this lecture, we will continue looking at how to implement the transformer decoder. At this point we have all our component modules and all that is left is to build the decoder module itself. So first we have the init method. This takes in the vocab size, maximum sequence length, d_k, d_model, number of heads, number of transformer layers and dropout probability. Note that the number of classes is not needed because the number of classes at the output is just the vocab size.

As you recall, the job of the decoder is to predict the next token in a sequence. So inside the init method, we create all of the modules we need. We begin with the embedding. Then we have the positional encoding. Then we have the transformer blocks. Then we have Layer Norm and then the final fully connected layer. Notice that this is pretty much the same as the encoder. And so next we have the forward method. This takes in some batch of input sequences called X and the corresponding pad mask.

Inside the function we pass X through all the usual layers. Note again that this is basically the same as the encoder, except that there is one crucial difference with the encoder. You'll recall that we used a many-to-one task. In other words, the input sequence was a sequence of many to-

kens while the output was a single class. As such, after going through the transformer blocks, we selected only the output at a single time step to be passed on to layer norm and then the final fully connected layer.

In this case we do not have a many-to-one task, but a many-to-many task and recall that we discussed this in the theory lectures. Although we could treat language modeling as a many-to-one task, this would require us to break up the input data such that a single sequence turns into multiple samples, but using a many-to-many task, we only have to pass in each sequence once and the model learns all of the sub sequences simultaneously. So let's run this.

And so the next step is to test our decoder with some dummy inputs just to make sure that it works without any errors as before. Note that these values are completely made up, so feel free to change them. The next step is to create our device object for the GPU and move the model to the GPU. So the next step is to create a random dummy input. As before. We'll create a sequence of integers with a batch size of eight and a sequence length of 512. Since the vocab size is 20,000, the token IDs can be anywhere from 0 to 20000.

Exclusive. We also move the resulting array to the GPU. The next step is to pass our input into the model. Note that we are not using padding for this first test. So as you can see, the result has the shape eight by 512 by 20,000, which makes sense. There are eight samples with a sequence length of 512 and each prediction can belong to one of 20,000 possible tokens. So the next step is to create a random padding mask of size N by T and to also move this to the GPU.

The next step is to test our model again with our mask. So as you can see, the model is able to produce an output without any errors. And as expected, it has the same shape.

10 · How to Train a Causal Language Model From Scratch

So the next step is to work on training a decoder as a causal language model. We'll begin by installing transformers and data sets, since we'll be using the Hugging Face tokenizers and datasets packages once again. The next step is to import AutoTokenizer and DataCollatorWithPadding. The next step is to define our checkpoints, which will again be distilbert-base-cased. We'll also create our tokenizer at this point. The next step is to import the load_dataset method.

Now, for this example, all we really need is some unlabeled text, but to make it as simple as possible, we'll just use the same data set that we used for the previous lecture, except that we'll ignore the labels. The next step is to print out the raw data sets, just in case you forgot what they look like. So again, we have the sentence column, the label column and the idx column. For this notebook, we only need the sentence. So the next step is to define our tokenized function, which will call our tokenizer on a batch of sentences and also truncate the result if needed.

The next step is to map the tokenize function to a raw data sets. We'll also create our data collator at this point. The next step is to print out our tokenized data sets. So as you can see, we now have a new column for the input IDs and the attention mask. Note that the attention mask is the pad mask. So the next step is to remove unnecessary columns, which in this case are sentence index and label. So the next step is to import the data loader class and to create a data loader object.

Now, technically, we could have kept the validation loader in this notebook, but for some reason I decided to remove it. In fact, it could still be useful because we still would like to see the loss go down for the out of sample data as well. So perhaps as an exercise you can try putting it back. But for this notebook, we'll create a train loader only. Notice again that the data loader takes in our data collator. So this will do stuff like truncation and converting the result into a torch tensor.

So the next step is just a copy of the previous notebook where we check how the train loader works. Inside the loop, we simply go through each item in the batch and print out its shape. We then break so that it only does this for the first step. Okay. So as expected, both the input IDs and mask are torch tensors of size N by T. Note that there are no labels. So the next step is to figure out what our pad token is, since this will be important later on in this notebook.

As you can see, this is stored as an attribute of the tokenizer, so we can just call tokenizer pad token ID. As you can see, the value is zero. So the next step is to create our actual decoder model with the correct size for the vocab and sequence length. Again, all of these hyper parameters are completely arbitrary, so feel free to choose your own. So the next step is to create our loss and optimizer objects. Since our task is still classification, we can still use the cross entropy loss.

However, there's a difference with this and the previous notebook. So unlike the encoder notebook, this adds a new argument called ignore index. So why do we have this? Well, as you recall, our transformer now is many-to-many using the data collator, every input will be padded to have the same sequence length as the longest sequence in the current batch. As such, although every input and hence every target will be of length T, not all of the T tokens will be actual tokens from the sequence.

Some of them will just be padding. But we don't want to include padding tokens in the loss. Otherwise, our model may be biased to produce padding tokens as outputs. In order to avoid that situation, we use the ignore index argument and we pass in the padding token ID so that padding does not contribute to the loss. The next step is to import datetime, which will be used to time the duration of each epoch during the training loop. So the next step is to implement our train method.

This takes as input the model, the loss function, the optimizer, the data loader and the number of epochs. Inside the function, we initialize the train losses to be an empty array. Next, we enter a loop for each epoch. Inside the loop, we set our model to train mode. We then capture the current time and we set our train loss to be an empty list. So in this notebook we'll compute the train loss per epoch a bit more simply, instead of adding all the losses together and dividing by the total number of samples, at the end, we'll simply take the average of the train loss per batch.

This is not as accurate since not all batches will have the same number of samples, but it keeps things simple. Next, we move through each batch in the train loader. Inside this loop, we move each item in the batch to the GPU. The next step is to call zero_grad to zero the parameter gradients. The next step is the most important step, which is to create the targets. Now, as you recall, all we currently have are the input IDs and the mask. However, this is unsupervised training.

This is a language model. The target is simply a shifted version of the input so that the transformer is learning to predict the next token in a sequence to compute the targets. We'll first make a copy of the input IDs. The next step is to shift the targets to the left. Note that this is okay and we're not losing the first token in each sentence. This is because we use the DistilBERT tokenizer. So the first token is always the special CLS token, and that's obviously not needed as a target.

Now, to do this shifting efficiently, what we can do is call torch.roll. So you can think of our tensor as like a circular buffer. So if everything gets shifted to the left, whatever is at the first position on the left, we'll get carried over to the end. Now, because we don't want to have CLS tokens at the end of each sequence in our targets, we will set the last time step in the targets to be the pad token, which we know will be ignored in the loss.

So the next step is to perform our forward pass to get the model outputs. Note that the model takes in the input IDs and the corresponding mask. Now, the loss is a bit complicated to compute because our targets are not a single label per sample, but a whole sequence of labels. So one simple way to do this, which I've commented out, is just to flatten the outputs in the targets. So while the output is initially N by T by V, where V is the vocab size, we can flatten the first two dimensions so that it becomes N times T by V for the targets which were a 2D array of size N by T, we make them into a 1D array of size N times

T. And so in that way the outputs are 2D logits, and the targets are just 1D labels. And so this is the same situation that we had before. But another way to do this, which is the one we're actually using, is to transpose the dimensions of the input so that the time dimension comes last. This is what PyTorch expects if your targets are more than two dimensions. So instead of n by t by V, it expects N by V by T. So the next few steps are just boilerplate code, which we've already seen.

We call loss.backward() to compute the gradients and then we call optimizer step to do one step of gradient descent. We then append our loss to the list of train losses. Once we get outside the loop, we then compute the train loss for the whole epoch to be the mean of the train losses list that we collected for each batch. Next, we save the loss for this epoch to our list of losses. Finally, we compute the time once again and the time difference between the start and end of this epoch.

And then we print the result. Once the outer loop is complete, we return the train losses, which can be plotted later if needed. So the next step is to call our train function. Please note that I've chosen the number of epochs arbitrarily, so feel free to play around. So the next step is to just play around with our trained model to see what it can do. At this point, we'll create the data loader for the validation set with a batch size of one. The next step is to put our model into eval mode and to get the output for the first sample in the validation set.

The next step is to check the shape of the output. As you can see, it's one by 12 by vocab size. The next step is to take the argmax over the vocab size, since that will tell us which tokens our model is trying to predict. As you can see, this returns a list of token IDs. The next step is to save these predictions to a new variable called prediction IDs. Now, since the prediction IDs are just integers, we would like to convert them into actual tokens so that they can be more easily read.

So as you can see, the result doesn't make a lot of sense. The next step is to check what the input for the sample actually was. So it says it's a charming and often affecting journey. Note that it's difficult to tell from this whether our model's predictions make sense. So to sort of check this, we can decode the first few input IDs concatenated with the prediction for only the next token. So as you can see, it says it's a good which is a totally reasonable guess.

So the next step is to look at how to actually use our model to generate text. We begin by creating an initial prompt, which is just it. In order to get our model to predict the next token, we need to think through what steps have to be done. The first step is to tokenize the prompt in order to convert the text into a sequence of integers that our model can read. So as you can see, we get our input IDs along with the corresponding mask. So the next step is to pass these input IDs and the mask into our model to get a corresponding output.

Now, importantly, note that we cut off the final time step from the input IDs. This is because with DistilBERT, every input sequence ends with the SEP token. But we don't care what our model predicts should come after the SEP token, as that is already the end of a sentence. Instead, we care about what our model predicts for the actual last token in our prompt. So as you can see, the result is one by four by vocab size. Note that the sequence length is four because the prompts got converted into five tokens.

So the next step is to look at how to get an actual prediction. So to get a prediction, we need to get the correct output, the correct output for the next token, and our sequence will be at the final time step. So we index the outputs at minus one in the time dimension. We'll call the result prediction IDs. So the next optional step is to decode the prediction ID. In practice, what we're going to do next is to put all of this into a loop so that we get a full sequence of text.

Only after we've gathered the full sequence will we bother to decode the tokens into anything we can read. So it's a because our first word was it's so it's it's a and then the model will generate something after that. So as promised, we're going to take all the code we just wrote and put it into a loop in order to generate a full sequence of text. This time we'll begin with the prompt. It's a as before. We'll begin by tokenizing the prompt. And also as before, we remove the final SEP token at the end and we also move the two tensors to the GPU.

The next step is to enter a loop that iterates a maximum of 20 steps. Note that I've chosen this number arbitrarily, so if you'd like to go longer than that, please feel free to do so. Inside the loop we pass in our input IDs along with the mask into the model to get the outputs. Now remember that the outputs have the shape N by T by V, but we only care about the final time step. So we index the outputs at the final time step and then we take the argmax to get the predicted token ID.

From there we concatenate the prediction with the current input IDs and make this the new sequence of input IDs. Actually, let's scroll up so we can see the whole thing. And remember that this is how we generate text. The model simply uses its past predictions as inputs during the next step. And although it's not really necessary, we can also keep creating the mask. Note that the default is no mask, which would have the same effect as a mask with all ones.

Finally, we check if the current prediction ID is equal to the SEP token ID, which signals the end of a sentence. If we find this, then we break out of the loop. So let's run this. Oh, and you could also put this into a function to generate text from an arbitrary prompt. So the next step is to decode the input IDs, since it now contains both the prompt and all the models generated words. So as you can see, the result is it's a good time, which kind of makes sense since our data set is a set of movie reviews.

And please note that the result will be different every time you run this. So one time I saw it's a good movie, which also makes sense.

11 · Implement a Seq2Seq Transformer From Scratch for Language Translation (pt 1)

So in this lecture, we will move on to our next implementation, which is a full encoder decoder sequence to sequence transformer. At this point we've already built an encoder and we've already built a decoder. So now all we need to do is figure out how to put them together. So we begin with our usual imports. So the next step is to implement our multi-head attention class. Now you'll recognize that in the previous two notebooks we had slightly different versions of this class with the encoder.

There was no need for a causal mask, but for the decoder there was always a causal mask. In this version, since we need it to handle both cases, we'll need an option to use a causal mask. So that's this option right here. And note that the default value is false. Now, if we look at the init method, you'll see that all of the layers we create are the same. And at the end we only create a causal mask if causal has been set to true. So next we have the forward method.

Now, this will have a few major differences worth pointing out. So the inputs are the same. We still have Q, K, V and the padding mask and we still pass Q, k, and V through their respective linear layers. However, the next step is where things are different. As you recall, in the sequence to sequence transformer, it is possible to apply attention where we want to know which decoder output should pay attention to which encoder inputs. This corresponds to the part where the encoder is connected to the decoder.

So inside this attention layer, K and V come from the encoder while Q comes from the decoder. And because of this, they can have different sequence lengths, unlike the previous notebooks. So t output will come from Q while t input will come from K. The next step is to reshape Q, K and V. So what changes about this is that whereas before where we just had t, now we have either a t output or t input. The next step is to compute the attention scores. Note that this does not change.

In addition, note that the padding mask also does not change. So what does change a bit is how we apply the causal mask. In this case, when we index the causal mask, the third index goes up to t output, while the fourth index goes up to t input. Next, we compute the softmax, which is the same. Next, we multiply the attention weights by the values, which is the same. The next step is to reshape the attention outputs before going through the final fully connected layer.

In this case, the T dimension corresponds with T output. And finally, we do the final projection layer. So the next step is to implement the encoder block. This is what we call the transformer block in the encoder notebook, and it's pretty much the same as that notebook. Aside from the fact that when we create the multi head attention layer, we set causal to false. Okay, So next we have the decoder block, which is the transformer block from the decoder notebook.

Now, this is going to be very different from the previous notebook. So as you recall, when we discuss decoder only models, the decoder block only contains one causal self attention. But in the sequence to sequence transformer, the decoder block actually contains two attentions one for causal self attention and one for taking the input from the encoders outputs. Overall, we have three main blocks, two multi-head attentions and one ANN. As such, we'll need three layer norms.

Next, we have our two multi-head attentions, one with causal equal to true, and one with causal equal to false. The next step is to create our ANN layers, which is the same as before, and a dropout, which is the same as before. So the next step is to define the forward method. Now, note that this has many more inputs. It takes in the encoder output, the decoder input, the encoder mask and the decoder mask. So these are the padding masks. Note that this just follows the diagram from the Attention Is All You Need paper.

So the first step is to pass the decoder input through causal self attention. We then add the residual connection and layer norm as usual. The second step is to process the encoder output with another multi head attention. We then add the residual connection and layer norm as usual. The third step is to go through the ANN, and we then add the residual connection and layer norm as usual. Finally, we apply dropout and we return the results. And next we have positional encoding, which is exactly the same as before.

So the next step is to create our encoder. Note that this is slightly different from our first notebook. To make the differences more clear. Note that I tried not to delete anything. Instead, I simply commented out what used to be there. And we don't need any more. For example, we don't need any classes anymore because we no longer have any classifier head. So the next thing we don't need is inside the init method. And of course this is the classifier head which I just mentioned.

So the final thing that changes is inside the forward method. In our previous notebook we did a many-to-one task, but in this case we want all of the hidden vectors for every time step. So now there's no need to index the output at any particular time step. We do still perform the final layer norm, but we do not perform the final fully connected layer, which we also deleted from the init method. So the next step is to look at the decoder. Note that the inputs into the init method are the same as the encoder.

Okay, so inside the init method, we create all the layers you'd expect. So that's the embedding, positional encoding, decoder transformer blocks, layer norm and a final linear layer. So inside the forward method, we now take in many more inputs compared to the previous notebook. This takes in the encoder output, the decoder input, the encoder padding mask and the decoder padding mask. Inside the method, we first passed the decoder input through the embedding and positional encoding.

Next, we pass the result into the transformer blocks in addition to the decoder input. We also pass in the encoder output and the padding masks. Finally, we end with layer norm and a fully connected layer. It's good to keep in mind again that this is a many-to-many transformer since the output of our model is for generating text, as in the previous lecture. Okay, So next we have the full transformer class. This is pretty simple since we're going to use the encoder and decoder to do most of the work.

Inside the init method, we just take in an encoder and decoder we've already created and save them as instance variables. Inside the forward method, we take in all the model inputs and corresponding masks. Then we pass the data through the encoder, which gives us the encoder output. We then pass the encoder output into the decoder along with the decoder inputs and corresponding masks. This gives us the decoder output which we then return. So the next step is to test our model with some dummy inputs.

We'll begin by creating the encoder and decoder objects again with made up values for the inputs. We also create a transformer object passing in the encoder and decoder created above. The next step is to move both the encoder and decoder to the GPU. So the next step is to create some dummy data. This is useful because it forces us to think about what kinds of data need to be passed in. So we begin by creating an encoder input. We'll say it has a batch size of eight and a sequence length of 512.

The vocab size is 20,000, so we generate integers between zero and 20,000. Next, we create the decoder input. Note that this must have the same batch size, but it is allowed to have a different sequence length, which we should test to make sure that everything works as expected. So this time we set the sequence length to 256. Note that the vocab size can also be different. So we've set this to 10,000, so we generate integers between zero and 10,000. The next step is to create padding masks for both the encoder and decoder inputs.

In this case, the only requirement is that they have the same size as their corresponding inputs and contain only zeros and ones. So I've set the first half of the values to one and the second half to zero. Finally, we pass everything through the transformer and check the shape of the output. Okay. So as you can see, our model works as expected. We get a batch size of eight, a sequence length of 256 and a vocab size of 10,000, which matches what we set for the decoder.

The next step is to look at some values in the output, which might be helpful if you're getting bugs like nans. So as far as I can see, these are all real numbers. So everything is working as expected.

12 · Implement a Seq2Seq Transformer From Scratch for Language Translation (pt 2)

So in this lecture, we'll continue looking at our notebook where we will implement an encoder decoder transformer for translating language. At this point, we've already built our model and tested it on some dummy data. In this lecture we're going to download our actual data and train

our model to produce useful translations. So we begin by downloading an English to Spanish dataset, which we originally used in my third NLP course. So hopefully this offers a nice point of comparison for those of you who took that course.

And if you didn't take that course, don't worry. The next step is to check out the first few lines of our file. So as you can see, each line contains two columns and it is a tab separated text file with English on the left and Spanish on the right. So the next step is to modify our data just a bit to work better with the Hugging Face data sets package. We'll begin by downloading our data using pd.read_csv. So as you can see, our data was loaded in correctly.

The next step is to check the shape of our data set. So as you can see, we have over 100,000 translation examples. Now in order to make this notebook run in a more reasonable amount of time, we're going to keep only the first 30,000 samples. It's also helpful that this data set is approximately sorted by sentence length. So by taking the first 30,000 samples, we are getting the shortest samples. And since transformers have quadratic time complexity, this will keep the computation down.

As an exercise, you might want to try selecting samples at random or even trying the whole data set, which will probably improve the model's performance. So the next step is to give our DataFrame column names which are used by the Hugging Face data sets package. We'll also write our dataframe out to a new CSV. The next step is to check the contents of our CSV just to ensure that they are what we expect. So as you can see, our CSV looks as expected. Importantly, we now have the column names en and es.

So the next step is to install Hugging Face transformers and other packages which will be used in the following code. The next step is to load in our data set using the load_dataset method. The next step is to print out our data set just to see what we got back. So as you can see, we get a data set of 30,000 samples with two columns called en and es as expected. So the next step is to do a train test split. So we have 21,000 training samples and 9000 test samples.

So the next step is to create our tokenizer. Now, theoretically, you could have just found tokenizers for Spanish and English and use them both separately. This would be a completely valid approach, and it's the first one I thought of when making this notebook. However, what I eventually decided to do was to just use the same tokenizer we used earlier in the course when we looked at translation fine tuning. So depending on which approach you take, there might be subtle differences in the upcoming code.

So the next step is to just test our tokenizer on the first training sample of our data set. So as you recall, with this tokenizer, when we want to tokenize the targets, we need to specify that this is the case. In order to do that, we use the text_target argument. We'll print out the tokens as well, just to remind ourselves that they do not look like BERT. So as you can see, the tokens do not look like how they do with BERT and BERT like models.

Importantly, note that unlike BERT, there is no start token. Recall that this is useful as the first input into the decoder. So we'll need to handle this particular issue later in this notebook. The next step is to check the original Spanish sentence. So as you can see, we get the same sentence as

above, but without the special tokens and underscores at the beginning of every word. So the next step is to implement the preprocess function. As you recall, this will process both the inputs and the targets.

So we begin by setting the maximum sequence length for both the inputs and the targets. Note that I did not check these values, so you might want to do that yourself as an exercise. Inside the preprocess function, we first begin by tokenizing the English inputs which are located in the en column of the batch. We'll call the result model inputs. The next step is to tokenize the targets which come from the column es. We'll call the result labels. Importantly, we know that the tokenized target is located in the input IDs field of the labels variable, but what we need is to have everything in the same object so we save the target tokens to the model inputs variable using the field labels.

At this point, the English tokens are stored in a field called input IDs and the target tokens are stored in a field called labels. Finally, we return the model inputs. So the next step is to apply the preprocess function to our data sets. The next step is to look at our tokenized data sets. So as you can see, we have input IDs which correspond to the English inputs, the attention mask, which also corresponds to the English inputs and the labels. Now, you might find it odd that the labels don't have any mask, which I realized after writing this notebook.

I suppose you could have saved the mask from the tokenizer, but I never actually bothered to check. What we will do later is create our own mask on the fly. In fact, we still haven't created the decoder inputs which the mask should correspond with. So in other words, even if we did get the mask from the tokenizer, we couldn't use that because it's aligned with the targets and not the decoder inputs. So the next step is to create our data collator.

As you recall, there is a special kind of data collator for seq2seq. The next step is to test our data collator to remind ourselves how it works. We'll test it on the first five training samples. As you can see, we get a dictionary like object with the same keys as above. The next step is to look at the input IDs. So as you can see, the input IDs might be a little different than what you expect if you're used to working with BERT. Note that the padding value is not zero, which we can tell since most of the inputs end with 65,000.

It seems that zero is used for the end of sentence token, while 65,000 is used to pad. The next step is to look at the attention mask. So if we compare this to what we had above, we can see that it is in fact the case that 65,000 is the pad token. So the next step is to check the labels. So as you recall, one thing that's pretty weird about the tokenizer is that it pads targets differently than it pads the inputs. As you can see, the targets are padded with -100.

So the next step is to check which special IDs exist in our tokenizer. This will be useful in the coming steps. So as you can see, we have zero, one and 65,000. We know that 65,000 is padding and we know that zero is end of sentence, but we do not know what the one means. So the next step is to print out the special tokens. So as you can see, the ID one corresponds with the unknown token. This also confirms that zero is end of sentence and 65,000 is padding.

Now, one important thing to remember is that the tokenizer always adds an end of sentence token to everything you pass in to demonstrate this will tokenize the pad token. So as you can see, we get back two IDs corresponding to padding and end of sentence. So end of sentence is always added. The next step is to create a data loader object. As input we pass in the tokenized data set as well as the data collator we created above. So the next step is to check how our data loader works.

As before, we'll print out the shape of each item in the first batch and then break out of the loop. So as you can see, we get input IDs, attention mask and labels all of shape N by T. However, note that t for the input does not necessarily have to be the same as t for the output. So the next step is to check our tokenizer vocab size, which, as you recall, is one of the inputs into our model constructor. So the vocab size is 65,001. Now just for fun,

I wanted to see what some of these tokens were at the higher indices. So as you can see, it's a lot of weird Unicode. Please feel free to try other values for yourself. Now, one thing our tokenizer doesn't have, which is why I was doing some of this exploration is the start of sentence token. You'll recall that we need this when we form our decoder inputs, since this is how our model knows that it is at the start of a sentence, it's not possible for the decoder to have no decoder input.

I decided to add my own special token for CLS, which to my knowledge isn't used in this tokenizer elsewhere. As you can see, I've set it to be like an ~~tag, which hopefully makes sense since the end of sentence token is the corresponding end tag. Okay, so now let's check what happens if we tokenize our new token. So as you can see, this results in a new ID 65,001. Now, oddly, when you check the vocab size, you can see that it's not updated. So it still says 65,001 when it should really be one higher than our maximum token ID.~~

~~So the next step is to create our encoder decoder and transformer objects. As you can see, I've set the vocab size to be the tokenizer vocab size plus one for the reasons we just mentioned. The rest of the values are completely made up, so feel free to change them as you see fit. The next step is to move our model to the GPU. The next step is to create our loss and optimizer as before. We need the loss function to ignore padding tokens, which in this case is represented with -100.~~

~~So the next step is to implement our training loop. Now, since this lecture has been quite long already, we're going to take a break. In the next lecture, we'll continue on from this point.~~

~~13 · Implement a Seq2Seq Transformer From Scratch for Language Translation (pt 3)~~

~~So in this lecture, we will continue looking at our notebook for a sequence to sequence transformer. So we're just at the point where we are about to implement the train method as input into this method, we take in the model, criterion, optimizer, train loader, validation loader and number of epochs. Inside the function. We create empty arrays to store the loss at each epoch for both train and test. Next we enter a loop for each epoch. Inside the loop, we set the model to train mode.~~

We capture the current time and we instantiate an empty list to store the loss for each batch. The next step is to loop through each batch in the data set. Inside this loop, we take each item in the batch and move it to the GPU. Next we zero the parameter gradients. So the next step is to create a model inputs and targets. We begin by grabbing each item from our batch, which includes the encoder input IDs, the attention mask, and the labels which represent the target.

At this point, we do not yet have the decoder inputs. So the next step is to create these decoder inputs. So we begin by first cloning the targets. As you recall, the decoder inputs should simply be a shifted version of the targets. Now, to contrast this with the last lecture, we actually had the decoder inputs and we wanted to create the targets. But in this lecture we have the targets and we're trying to create the decoder inputs. As such, we need to shift the targets to the right in order to get the decoder inputs.

To do this, we set shifts equal to one. This is the opposite of what we did before, where we set it to minus one. We also set dims equal to one, since that corresponds to the time dimension. Now recall that we've just used the roll function to shift, which will bring anything that was at the end of the targets to the front of the decoder inputs. But of course, the decoder input should start with the start tag. So in the next step we set all the decoder inputs at time index zero to be the start token ID, which is 65,001.

Now, you'll also recall that the model has an input for the decoder mask. This is because not every sequence in the decoder input will have the same length without padding, so the decoder mask is needed. HOwever, before we create the decoder mask, we also have to modify the decoder input. As you recall, for some reason the tokenizer uses -100 as the padding token for the targets, but -100 is not a valid input into the transformer because these values need to go through an embedding matrix, and the embedding matrix can only be indexed from zero up to vocab size and hence -100 is not a valid input.

Therefore, what we need to do is change any values which are -100 into the tokenizer pad token ID. So the next step is to finally create the decoder mask. In order to create the decoder mask. We first create an array of ones with the same size as the decoder input. Then we set any value where we have padding to be zero. So it's zero where we have padding and ones everywhere else. The next step is to do our forward pass to get the outputs. We call our model with the encoder input, the decoder input encoder mask and decoder mask.

As before, when we compute the loss, the outputs have the shape N by T by vocab size, but we need vocab size to be in the second dimension and time to be last. So we transpose the output at index two and index one. The next step is to call loss.backward() and optimizer.step() as we normally do. We also append the loss for this batch to the train loss list. Then we compute the average train loss for the batch. So the next step is to do the same thing for the test set.

Since this code is essentially the same as above, there is no need to explain it again, except note that we put the model into eval mode before we begin. So once we have both our train loss and test loss, we save them to our arrays to store these losses. Finally, we print some information about the loss and duration of each epoch, and then we return the train losses and test losses. So the next step is to run our train method. Okay, So it takes about 20 seconds per epoch.

So the next step is to try our trained model as before. We'll go through the steps one by one first to show you how a translation might be generated. After that, we'll write a whole function that does all the work inside. So if we want to translate some arbitrary text, all we need to do is pass it into this function. So we'll begin by choosing an arbitrary sentence from our test set. Please feel free to change the index to translate some other sentence if you prefer.

We'll also print out the sentence we chose. So it says, Can I take a day off? The next step is to tokenize the input sentence. We'll call the result enc_input. Okay, so this is the encoder input. Now remember that our model has two inputs, one for the encoder and one for the decoder. So now we need to create the decoder input. Since we're now running the transformer in generative mode, the decoder input is just the sentence start tag again. We use the tokenizer to get the model inputs, but this time we use target mode.

So note again that when we tokenize any text, the end of sentence tag always gets added. So the next step is to move our encoder input and decoder input to the GPU. We then pass these to the transformer to get our output. Note that we index the decoder input and decoder mask only up to time step minus one instead of the whole array since the last value is the end of sentence tag. So the next step is to check the shape of our output. So as you can see, it's one by one by 65,002 as expected.

One sample, one time step and 65,002 is our vocab size. Now in practice, how we actually want to use the model is to use the encoder and decoder separately. To do this, we begin by obtaining the encoder output, which only looks at the encoder input along with its attention mask. In other words, just the English sentence. So note that the result has the shape of one by eight by 64. This means one sample, eight time steps and 64 hidden size. The next step is to pass the encoder output into the decoder, along with the actual decoder inputs.

So as you can see, the result is one by one by vocab size as before. Now just as a sanity check. It would be nice to confirm that what we just did by looking at the encoder and decoder in separate steps is equivalent to going through the whole transformer. So in order to check this, we'll check whether the two outputs are close in value. Note that we use the allclose function since floating point numbers are usually never exactly equal. So this confirms that the encoder and decoder separately produce the same output as the whole transformer model.

So the next step is to demonstrate how we will actually use the decoder to generate the rest of the translation. To do this, we first begin by setting some variables for the decoder input IDs and the decoder attention mask. As above, we leave out the last value, which is just the end of sentence tag that the tokenizer adds. The next step is to enter a loop that goes for a maximum of 32 steps. This means if we try to generate more than 32 tokens, we'll just stop.

Of course, you can make this longer if you choose. So inside the loop, we first pass our data through the decoder to get the decoder output. Remember that we've already obtained the encoder output. Now, you recall that I said that this process is done in an auto regressive fashion. So the whole idea is take this output, convert it into a prediction, append it to the decoder inputs, and then run the whole thing again to get the next output. In other words, generate one token at a time and then on the next round the token we just generated becomes part of the input.

So the next step is to get the prediction ID by taking the argmax over the decoder outputs at the final time step. So this is the same as the previous notebook. The next step is to append the prediction ID to the existing decoder input IDs. So the decoder input IDs have grown by one token, which is the latest predicted token. Next, we recreate the mask. Since we only have one sample and every decoder input is a relevant token, we can just create an array of ones that is the same shape as the decoder input IDs and also note that this is optional since it will work without a mask anyway.

Finally, at the end of the loop, we check if our prediction ID is equal to the end of sentence tag. If it is, we break out of the loop. since the model thinks we're done. So the next step is to check the translation. We can do this by decoding the decoder input IDs, which now contain every token generated by the model. So this is the translation. Note that it will be different every time you run this, but what I've noticed is that it's usually pretty close to being correct if it's not already fully correct.

So the next step is to check the ground truth translation. So as you can see, it's pretty close. Note that it's not necessary for the translation to be exactly the same as the prediction. Sometimes the prediction is different, but still a valid translation. One thing we can do actually is paste this into Google Translate and see what it gives us. So this gives us the correct original English sentence. So the next step is to take everything we just wrote and put it into a function called translate.

This demonstrates how you might do this in the real world where you just have a convenient function you can call whenever you want to translate text. As input into this function, we take in a single input sentence represented as a string. The first thing we do inside the function is to get the encoder outputs. To do this, we first tokenize the input sentence and move the result to the GPU. We then pass the tokenized inputs into the encoder along with its mask.

Note that the mask is just all ones since we only have one sentence and hence it is optional. So the next step is to set up the decoder inputs. Since we know what the first value should be, we can just create the inputs directly without tokenizing the start tag. As you recall, the start tag has the ID 65,001. So the next step is to enter the decoder loop where we will generate the output tokens one step at a time. As before, we'll go through 32 steps, but feel free to change this as needed.

So inside the loop, we begin by passing our current data through the decoder. This gives us the decoder output. Next, we take the argmax of the decoder output at the final time step. This gives us our prediction ID, the ID of the predicted token for this step. The next step is to combine the newest prediction with the existing decoder input IDs. Once we've done that, we can create our Decoder Attention mask. Again, it's all ones since we only have one sample.

Finally, we check whether the prediction ID is equal to the end of sentence tag, in which case we break out of the loop. So at the end of our function, we decode whatever is inside the decoder input IDs. Since we don't want the start tag to be included, we can simply start at time index one. We then end the function by printing out the translated text. So to test out our new function, we're going to translate a sentence. Okay, So to my knowledge, this is pretty close to the true answer.

In fact, we can just test it ourselves. So it seems to be correct.

14 · Implementation Section Summary

So in this lecture, we will summarize what we learned in this section. This section was all about how to implement transformers from scratch. In this section we implemented multi-head attention, the positional encoding, and three different types of transformer models. Specifically, we learn how to implement an encoder, a decoder and an encoder decoder combined. We also learn how to train them on actual data sets and use them to make predictions which many resources are missing.

What is interesting about this section is that we were able to build these models with only basic parts. For example, the linear layer dropout, residual connections and layer norm. In other words, all stuff that existed in 2016. So technically you could have built transformers using 2016 technology in 2016 had you only invented the attention mechanism. So I think that's a crucial point to keep in mind in order to do really cool things in machine learning.

It's not just a matter of using a high level library and doing stuff people have already done. Instead, it requires us to think of new approaches and implement them ourselves using existing tools and not wait around for someone else to build the tools for us. You could have been the one to implement Transformers if you invented them first. So keep inventing, keep trying new things, and I'll see you in the next lecture.

1 · Anaconda Environment Setup

Everyone and welcome back to this class in this lecture I'm going to go over a better way to install data science and machine learning libraries for Python for Windows users. HIstorically Windows users have had a lot of problems installing this stuff. Luckily these days there is an option that makes things very painless and just as easy as they are on Linux or Mac that is Anaconda. In fact even if you're not on Windows you can still use Anaconda. It's nice because it isolates your environment from the defaults provided on your system.

So for example you can have Python 3 in Anaconda but Python 2 as your system default. When I first started these courses I wasn't keen on Windows since there were a few essential libraries that couldn't be installed on Windows without a significant amount of effort. If at all in my view anything beyond a couple of lines in the console or clicking an install file is too much. And believe me some students even have trouble with that so it's good not to make things too complicated before you can even begin the course.

Nowadays that has changed. It's a lot easier to install things on Windows in large part thanks to Anaconda and so this lecture is all about how to install all the data science and machine learning libraries you'll need on Windows using Anaconda. So in this lecture I'm going to walk you through how to install Anaconda as well as some of the libraries you might need that don't already come with Anaconda. You'll find that most of the common libraries such as NumPy and SciPy are already included.

So if that's all you want to use then for you it's just a one click install on this slide. I'm going to give you a super short summarized version of this lecture so you don't have to walk through the installation with me if you don't want to. For some people that really helps since you can see it.

But if you can do it on your own feel free so number one. Download and install Anaconda. This is just a one click install. It already includes NumPy, SciPy, Matplotlib and Pandas.

That's all we need for The Numpy Stack in Python, Linear regression and logistic regression and a few more courses. It also comes with NLTK, which is what we use for NLP, and scikit-learn which has some pre-built machine learning models. Now even though this stuff comes by default you can still update them if you want so you can do conda update numpy for example and that will update NumPy. Number two, install deep learning libraries. We've got pip install tensorflow, that's going to install TensorFlow, and if you want to install Keras you have to first do conda install pip which is going to update pip and then you can install Keras using pip install keras if you don't update pip first,

you might get an error. Next we have CNTK, which is Microsoft's deep learning library. So you do pip install and then the CNTK URL which you can get from Microsoft's Web site. So I'm not posting any URL here because the version could likely change in the future. And so you can Google search how to install CNTK and you can get a URL just like this. Next we have PyTorch that's conda install -c peterjc123 pytorch. After that we have Theano. So that's conda install theano or conda install theano pygpu

if you have an Nvidia GPU and you've installed the CUDA toolkit already. Number three, install OpenAI Gym. That's just pip install gym if you want to be able to play Atari games also then that's more involved so just skip to the end of this lecture where I walk through that if you want to play and save videos using OpenAI Gym then you also have to install FFmpeg. So the first thing we're going to do is go over to the Anaconda Web site. That's anaconda.com/download.

Scroll down to the Windows section and click on either Python 3.6 or Python 2.7. Or if you're watching this lecture in the future and there's a new version. Get that. The code in my courses is compatible with both Python 2 and Python 3. So in that regard it doesn't really matter which one you get. In the lectures you might see Python 2 code. But the best way to make sure you're seeing the latest version is to git pull inside the course repo so make sure you're always doing that because I'm constantly making new updates.

Now though Python 3 is newer, there are still reasons to use Python 2. For example in your work you might use Python 2 or certain platforms like Google App Engine only support Python 2. So if you are running a web app that means you're stuck with Python 2. It does have great scalability features so there are many good reasons to use Google App Engine if you want to get more insight on whether to choose Python 2 or Python 3. Just check out the appendix lecture Python 2 vs Python 3.

So now that we've downloaded the install file all we need to do is click on it. That's what I mean by one click install you click on this here OK a few times and everything is done. Unfortunately I gave my username a space which kind of sucks but that's what happened. So I'm sure some of you have a space in your username too. So if I come across any issues at least you'll know what to do. Alright so everything's installed. So essentially everything except the deep learning libraries have already been automatically installed so you don't need to manually install them NumPy, SciPy, Matplotlib, Pandas, IPython or scikit-learn.

So if you're taking my Numpy course or any course that doesn't use modern deep learning libraries you already have everything you need. So let's go into Python and make sure that's the case. To start Python I go to the start menu type in Anaconda and then this anaconda prompt application should pop up. Actually it should pop up before you even finish typing Anaconda. So we go into there and this brings up a command line terminal next. Just type in Python.

After that we can import all the libraries I mentioned earlier. If we don't get an error that means they've been installed successfully. So let's try something simple like generating some random numbers and making a plot so that's a plot of random noise. Let's make a histogram to it so we see a normal curve just like we expect. Now you can see that TensorFlow is not installed which is why we get this error. But we can install it very easily by exiting Python and then typing in pip install TensorFlow next.

Let's try to install Keras so we get this error. So I looked this up and determine that we need to update pip so let's do that by typing in conda install pip conda kind of works like pip that way. They are all just tools for installing stuff. Now let's try pip install Keras again. All right. So everything works next let's try to install NLTK. This is used in my NLP courses so it looks like it's already installed so there's nothing more to do.

Just keep in mind if we come across a library you don't care about. Feel free to ignore it. I find it's useful just to install everything at the same time so that when you're deep in the code later you don't have to think about stuff like this. So the next thing we'll install is CNTK. This is Microsoft's deep learning library. Notice how it's not part of pip. So you need to grab the URL manually from Microsoft's Web site. Unfortunately it's a little hard to find because there are many pages that deal with how to install CNTK on Microsoft's Web site.

But what you're looking for is a page that has a link to a WHL file. So copy and paste that after pip install this is a good example because it shows you another way you can use pip by doing pip install and then a URL next. Let's install PyTorch. This requires us to use a custom source so we have to specify the option -c peterjc123. So that's conda install -c peterjc123 pytorch. This is because a very nice guy called peterjc123 has provided us with a version of PyTorch that works on Windows.

And once that's all done we can verify that they've been installed correctly by going back to Python. All right. So TensorFlow works. Keras works, NLTK works, CNTK works and torch works. Next let's install Theano. Theano has historically been pretty difficult to install on Windows but nowadays that's not the case. So if you go to their Web site you'll see a bunch of instructions. If you don't want to use the GPU or you don't have a GPU then the instructions will be very easy.

I don't have a GPU on this machine so I'm going to do the easy version. Keep in mind that Theano is really great for learning purposes. So it's totally fine. Even if you have a GPU, to just install the CPU version for now and then use the GPU version of TensorFlow for now. I ended up upgrading mkl-service and libpython since that's what they told me to do on Theano's Web site. But it looks like these were already installed. In fact updating mkl-service gave me an issue later.

So we'll have to fix that. So if you want to install Theano for CPU only, that's conda install theano. If you want to install Theano for the GPU then do conda install theano pygpu. Now let me go into Python and check if Theano works. So we get an error because of this mkl-service thing that I mentioned earlier. So let's set this environment variable. It's telling me to set. By the way this is great to know if you don't yet know how to set environment variables on Windows.

We can also check that it works by using the echo command. So let's try that again. Let's do a simple example of adding two numbers in Theano. Just to make sure everything's working. If you want to do something even more complicated you can run this script from deep learning part 2 which doesn't require any external data set. So just go over to the folder and in ann_class2, type in python grid_search.py. So that's going to look for hyper parameters using cross-validation.

Now in this last section of this lecture we're going to talk about reinforcement learning when we started studying reinforcement learning. There is yet another library will need to install called OpenAI Gym. If you don't plan on learning reinforcement learning you can skip this part of the lecture. This has also historically been very difficult but luckily the open source community has put in the work. So you don't have to. You're welcome to read through the GitHub issue on this.

If you want but I'm going to just do the simplest thing that works. So let's first do pip install gym. Great, so that worked. Now the second command is a bit longer. So let's go to the actual GitHub issue and copy and paste the command the easiest way to get there is just to go to Google and type in install openai gym windows anaconda or something along those lines. So let's paste that in. And notice how it fails since I haven't yet installed git. So we can install it by doing conda install git.

Now let's try it again. Unfortunately this fails again because we need gcc which is a C compiler. Now one way to get gcc is to do conda install m2w64-toolchain. But unfortunately I tried this and it also doesn't work. In fact I think this toolchain was installed already. So I tried quite a few things that didn't work. So in order to save you some time I'm going to recommend you only try this stuff on this page. If everything else doesn't work for you.

So what worked for me was to just grab the pre-compiled binary directly. Now in order to do that you want to go to [unclear] GitHub repo directly. So that's the URL here: github.com/[unclear]/atari-py/releases. Next you'll want to download the WHL file that matches your environment. So I have Python 3.6 on a 64 bit installation of Windows. So this is a file I want. Luckily we already discussed earlier in this lecture how to install a WHL file.

That's just pip. And then the path to the file. So let's do that. Now we can test our installation by running a script that requires an Atari game. So let's cd over to rl2 and then atari. And now let's run dqn_tf.py. Cool, so everything's good. Now the final thing we want to do for open gym is if you want to play a video or save a video you want to install FFmpeg, so to do that you want to type in conda install -c menpo ffmpeg.

Once you've done that you can go to the cartpole folder and type in python save_a_video.py and this will run a script that will play the CartPole game show a plot and then save a video and of course you can also play this video by just navigating to the file and clicking on it. So for now

that's everything. If I need to add new libraries or updates to this lecture they will be appended at the end.

2 · How to install Numpy, Scipy, Matplotlib, Pandas, IPython, Theano, and TensorFlow

Hey, guys. In this lecture, I'm going to show you how to set up your development environment so that you can follow along in these courses. First, let's talk about operating systems. The three big ones are Windows, Linux and Mac. Most, but not all of my courses are part of the deep learning series. Eventually, the deep learning models we're going to build will become so complex that we will need specialized libraries like Theano and TensorFlow to implement them.

Now, as of today, Theano and TensorFlow are not officially supported on Windows. So if you are interested in deep learning, you will discover pretty soon that there is not much you can do without a ton of work. Even working with non GPU enabled libraries like numpy and matplotlib can be a challenge, but these are at least possible on windows. If you really must use windows and have your python environment inside of windows and you don't want to try the virtual machine method, which I'm going to discuss next.

Then one good library I know of is called Anaconda. You can get Anaconda at Continuum I o slash downloads. I can't vouch for this 100% because I haven't used it myself, but I've known others who have found it at least usable. Now I'm telling you this from experience, I've worked with a ton of clients one on one, and Python development, especially when it involves the NumPy/SciPy stack is not easy when it's on windows. The method I'm going to describe using virtual machines will work on most modern computers.

And as of right now, if you want to do any deep learning stuff, whether that be with Theano or TensorFlow, you cannot do this on Windows. With that out of the way, I'm going to quickly go over which courses I've released so far or plan to release in the near future that do not require Theano and TensorFlow. So it's possible to do on Windows. With these in mind, you can decide if you want to install a virtual machine or not. But I would highly recommend it because it's free and everyone can follow the same instructions.

So we've got linear regression in Python, logistic regression in Python, which are the prerequisites to deep learning part one. And we have deep learning in Python Part one, which is mostly in NumPy and a little bit of TensorFlow. We've got easy natural language processing in Python. Data Analytics: SQL for Newbies, beginners and marketers cluster analysis and unsupervised machine learning in Python and unsupervised machine learning hidden Markov models in Python.

This course is going to have a little bit of Theano in it. Next, here are some courses that depend heavily upon Theano or TensorFlow or both. We've got practical, deep learning in Theano and TensorFlow, convolutional neural networks in Python, unsupervised deep learning in Python and recurrent neural networks in Python. As you can see, a lot of interesting and complex stuff can be done in Theano and TensorFlow. All right. So hopefully I've convinced you that using a virtual machine is a good idea.

~~Windows is a great operating system, but it tends to not be as developer friendly since it tries to be more consumer friendly. Note that if you're on a mac, this probably isn't necessary. You can just install numpy, scipy, pandas, matplotlib, theano and tensorflow using easy_install or pip. Sometimes this stuff can fail depending on the many combinations of versions of each thing that are possible. But usually just googling your error message will lead you to the right solution on StackOverflow.~~

~~If you're on a mac, you can just do sudo pip install numpy scipy ipython pandas matplotlib and theano. Alternatively, you can use easy install, which might have a more recent version. You may need to use easy install to install PIP itself, which is just sudo easy_install pip if you want to install tensorflow. Just copy the command at tensorflow.org. It's just a pip install command that points to their custom installation location. I won't put it here since it corresponds to a specific version and that version may become out of date by the time you watch this.~~

~~That is good news because the people working on TensorFlow are updating it all the time. All right. So if you made it this far, that means you want to install a virtual machine with Linux on Windows or that you're already using Linux and you want to know what commands to use to install these libraries. We are going to need two things to start for this tutorial. VirtualBox and a lightweight version of Ubuntu. I would recommend Xubuntu or Lubuntu.~~

~~I'm going to use the 64 bit version of ubuntu for this tutorial, so download these first and then return to the tutorial. All right. So now that you've got virtualbox installed, we are going to create a new machine with 64 bit ubuntu. So you want to hit new type in a name for your machine? And this stuff is already correct. I'm going to choose. Two gigs of memory. You can choose more memory if your computer has more memory. I'm going to create a virtual hard disk now.~~

~~I'm going to choose dynamically allocated. And eight gigs is good enough for this example. All right. So I'm going to go to settings if we go to storage. You can choose. The ISO file that you downloaded. So I'm going to load up the Lubuntu ISO that you downloaded. Once you've done that. All right. So hit. Okay. And it start. Okay. So it's going to take you through the installation of ubuntu. So you want to hit install? All right. So Lubuntu is going to take you through some prompts.~~

~~All right. So you want to erase disk and install Ubuntu. And I'm not going to check any of these continue. All right. So once that's done, it's going to ask you to restart. So just restart now. All right, now that you're in the machine, what I always like to do, if you notice how this window is too big for my Mac window, you can make the inner window resizable by installing guest additions. And this also lets you do useful things like cut and paste between machines.~~

~~Okay. So you want to open a terminal, you go to system tools, select LXTerminal. Now we cd into that folder. Cool. So the correct command was ./VBoxLinuxAdditions.run. Notice how there was something that failed in there. So we need to install gcc. So you want to run sudo apt-get update. And then run sudo apt-get upgrade. So the next thing is you want to sudo apt-get install build-essential. All right, once you've done that, we can try to run the VBox additions again, so that's sudo~~

~~./VBoxLinuxAdditions.run. All right, so everything worked this time. So we're going to restart this machine. All right, so now the window is only as big as I wanted to drag it. Okay. So now we can install the actual data science stuff. So you want to open again terminal and do a sudo apt-get~~

update again. Now you want to install. So it's a long list. So Python numpy, python-scipy, python-matplotlib, ipython, python-pip, python-dev and python-setuptools. Yes. All right.

So now that that's installed, you can test it out, can type in ipython. So IPython is open. Import numpy. NumPy is working. Import scipy. Import pandas. Right. We don't have pandas yet. We will install that next import matplotlib. Cool. So we have everything we've installed so far. So now we're going to install pandas and Theano. So that's pip install --upgrade pandas theano. All right, so now we've got Theano. The last thing we need is TensorFlow.

To get this. You guys want to go to the TensorFlow website and just grab the latest command since they're updating the package all the time. So let's open a browser. Let's search for install TensorFlow. All right, you take this code. Copy it when you paste it in here. All right, now you've got Theano and TensorFlow. You can test them out. Go to github.com/lazyprogrammer/machine_learning_examples. All right. So you need to get if you want to check out this code with git, so let's install git.

All right, so let's run our clone command again. Let's try the HTTPS version. Okay. So this one works. All right. So you cd into machine_learning_examples. I'm going to cd into ann_class2. So this has some introductory. TensorFlow and Theano code. So just run python theano1.py. Since this does not require any data. Cool. So it seems everything is working. Now let's try the TensorFlow example. So it's TensorFlow one. All right. So you want to make sure you install the correct TensorFlow.

You want the CPU only version. Hopefully it just overwrites the other version without any trouble. So let's try the example again. Super. So now everything is working. Now, if you want, there is a text editor that I highly recommend. It's called Sublime Text. So you hit download. You go to Ubuntu 64 bit. You save this? And it will automatically be installed. All right, so now you have the exact same text editor that I use in my lectures. So you open a machine learning examples and this is all of our code.

1 · How to Code by Yourself (part 1)

Everyone. And welcome back to this class. This lecture is about how to write code independently when you're doing machine learning. You always hear me say that the first thing you should try to do when you learn an algorithm is to go and write it up in code by yourself before looking at someone else's code. For some students who are new to machine learning, it's not quite clear how to do that. So this lecture is going to explain the thought process you should go through in hopes that it encourages more people to try to code by themselves.

I like to encapsulate this in the phrase When it's time to code, you must code. So remember this whenever you see some code that demonstrates the concepts we learned in this course, even if you don't fully understand what's going on, coding and copying examples from others helps you build muscle memory. Sometimes people act like they are going to learn tennis by reading a book about tennis. This is, of course, obviously not possible. You need to learn some things about tennis which are pure fact and reside in your brain as conscious facts.

But then you have to go out and practice those concepts on the tennis court. Once you've exhausted and mastered everything you know so far, then you can learn new techniques. So you have this continuous cycle of learning new techniques and then practicing them. Why is this? Because learning the technique as pure fact doesn't mean you truly understand it. Practicing the technique helps you think about the technique from a different perspective. Practicing again and again leads to new understanding, to learning in the subconscious, which is muscle memory.

You must go through this cycle. But some students naively think they are going to read an entire book about tennis and then be a tennis master the first time they try playing. This is very common, especially when all you really need to do is sit there and watch a video and no one can force you to type. A lot of people don't even think to try. So just remember this. Whenever you see code, when it's time to code, you must code. First we are going to talk a little bit about why you want to code by yourself.

A lot of the time, what an algorithm does or how it works isn't quite clear the first time you learn about it. Every individual will have their own unique background, so they might be familiar with the patterns and know exactly what to do. While some individuals might have questions that no one else has. Sometimes you gloss over details because you assume you know what's going on when really there are important things you're not considering. So this is how trying to code by yourself can help you.

Coding by yourself forces you to think about each and every detail. It forces you to think line by line. You have to be familiar with the data types and the shapes of all your variables, and they all have to fit together properly. Kind of like Lego blocks. As an example, you know that in order to do element wise matrix addition, both matrices have to have the exact same shape. So if you find that they are not the same shape, then one of your previous assumptions was incorrect.

So you should go back and correct it. LEGO blocks have to follow a specific set of rules in order to fit together properly. If you make incorrect assumptions about how Lego blocks fit together and you try to join them, it's not going to work without trying to build things by yourself. You will never discover these details. Let's now talk about how to code by yourself. Consider the supervised machine learning scenario. We know that we're going to have some inputs and some targets, and we want to try to make predictions from the inputs that are very close to the targets.

We typically call the inputs X and the targets Y. Sometimes we call the targets T, but for the purpose of this lecture, we'll call the targets Y. Now, this next point is a key point. It doesn't actually matter what X is and what Y is. It doesn't matter if you're looking at an eCommerce data set and the columns of X might be time on, say, time of day, how many pages the user looked at and so on. You could just as well be looking at a data set of x ray images where each column is the pixel intensity of an image.

This is the key point. We say all data is the same. When we do linear regression, we don't have different kinds of linear regression for e-commerce and x-ray images. Linear regression is the same algorithm no matter what your data set is. So we say all data is the same. Theoretically, I could give you a data set of X's and y's, and you could train a classification algorithm on it without me even telling you what the X's and Y's mean. You should get very comfortable with this idea.

Sometimes people say, well, this isn't practical because I want to do concrete examples. But that's because they're not thinking intelligently. In fact, this is the most practical thing we can do because it means everything we learn. We can apply it to any data set that exists. It means that I can train a model on an e-commerce dataset, but I can also train a model on an online advertising dataset without learning anything new at all. It's truly the greatest form of lazy programming.

Learn something once and apply it to every industry. One great consequence of all data is the same is that there is an unlimited amount of practice opportunity for you. You can download data sets from Kaggle, from Google, from Wikipedia, from Amazon or from anywhere else, and try the algorithms you learned. What this means is let's say you have a dataset you care about more than something we do in class. In class, we have to use datasets that everybody can understand.

For example, text and images. Everybody should know what text and images are. Text and images are just fundamental data types on the web. You probably don't even think about the fact that text and images are data because it's so trivial. In any case, we work with text in images a lot because everybody understands them. But let's say you're a biologist and you know about DNA. You find DNA and genomics really interesting. So you want to use the algorithms.

We learn on that. Well, that's great. And you should totally do it. Remember, all data is the same. So all you need to do is convert your data into the appropriate X's and Y's to plug into our machine learning algorithms. The downside is we most likely won't talk about DNA in class unless it's a very simple example, because most computer scientists are not biologists too, so they don't know what DNA is. They don't understand the specifics. So an example wouldn't make much sense to them.

This is opposed to text and images which make sense to everybody. Same goes with any other specialized field like finance, computer networking, cosmology, and so on. So now we can start out from the right place to talk about supervised learning. We have our X's and we have our Y's. What do we do with them in supervised learning? We know that there are two main things we want to do. We want to do training, and we want to do prediction. In scikit-learn, all the models have the same API.

It doesn't matter if you're doing logistic regression or decision trees or random forests. All supervised models in scikit-learn have the same two functions fit and predict. Fitting a model is just a synonym for training a model. If you don't believe me, you can go look at the scikit-learn documentation yourself. When you're learning about supervised machine learning, what you're really learning is what goes inside these two functions. What are the parameters and how are those parameters learned?

That's really what all supervised machine learning is filling in these two functions. Taking this perspective is going to give you a code structure, and it's going to make things much easier to visualize in your head. Let's now do a simple example to solidify this idea and this example, I'm going to give you an algorithm and you will implement it in code. There are two key points I want to make before I give you the algorithm. First, I'm not going to give you much intuition about how it works.

Second, I'm not going to derive the theory behind why it works. The reason I want to mention these two key points is that you should realize you don't need these two pieces of information in order to translate pseudocode into code. A lot of time these three concepts intuition, theory and implementation work to reinforce each other. The reason I'm focusing on implementation right now is because it's the one people most often miss or think they don't need at all, when really it's an extremely important part of the learning process.

Okay. So the pseudocode is as follows. In my predict function, as you already know, I'm going to take in some input data x. My predictions will be y hat equals x times w. You might already recognize this as linear regression. HOwever, forget what you know about linear regression and pretend that this is just some formula I gave you. What I can tell you is that we are creating some kind of regression model. It should be clear from this equation that the parameters of the model are contained in this vector of weights.

W I would probably also tell you that w is a vector that's the same size as the number of feature vectors in x. Of course this must be true in order for the matrix multiply to be valid. This acts as kind of a sanity check so you can make sure everything is making sense. In my fit function. I want to perform this loop some number of times, since we know from the previous slide that the parameters of the model are contained in w, then it should be clear that inside the fit function what we want to do is update w not surprisingly, that's what's happening here.

Another good sanity check now because you're already familiar with linear regression, you probably already recognize this as gradient descent. Again, I want you to pretend that you don't know that, but what do you know? You know that iterative algorithms are common in machine learning and that gradient descent is among the most common. You could probably infer that this is some form of gradient descent, but you don't need to know what it is in order to write it in code.

What else do we know? You know what a for loop is and you know how to write one in Python. You know that X is an N by D matrix containing the input data and that Y is an N-size vector containing the targets. You know that Y hat is an N-size vector containing the predictions. You know that W is a D-size vector containing the model parameters. And you know how to add, subtract and multiply these objects. So we can write this algorithm without even knowing how it works.

Of course, a lot about how it works can just be inferred from this formula. HEre is what you don't know. You don't know t the number of times the loop is supposed to run. You don't know eta, the learning rate. This is completely fine. You can't let this lack of knowledge stop you in your tracks. The truth of the matter is, there are going to be situations where you're not told the exact numbers to plug in. In many cases, the answer is going to be it depends on the problem.

The important thing is you have to get used to the idea of trial and error, and you have to make educated guesses based on what you already know. So what can you do when you know that the learning rate is supposed to be a small number? HOw small? It depends on the problem. Typically, it's a number less than one, like 0.1. If that doesn't work, you might try lowering it. Typically we lower it by factors of ten. So for example, the next learning rate we would try is ten to the minus two, ten to the minus three and so on.

Typically in machine learning, whether you're looking at a supervised or unsupervised algorithm, there is an associated cost or objective function that you're trying to minimize. Let's suppose I tell you it's squared error, which is typically what we would use for regression. Well, now you have a way to choose the number of iterations of the loop and the learning rate. How do we do this? So inside your fit function you would plot the cost as a function of iteration.

I always recommend doing this no matter what algorithm you're looking at. In general, you always want the cost to converge. The pattern you typically see is that there is a steep drop at the beginning and it slowly flattens out as the number of iterations increases. To choose the number of iterations you want to stop when the curve is sufficiently flat. You might recognize this as a diminishing returns scenario. The returns diminish because at the beginning we reduce the cost by a lot in just a few iterations.

At the end, we can do a lot of iterations, but we reduce the cost by almost nothing. An alternative is you could use a separate validation set and stop when the validation cost increases. But that is not the focus of this lecture. Unlike the training cost, the validation cost is not guaranteed to decrease at every round because that is not what we are minimizing with respect to. You can also use the same plot to help improve your learning rate if the cost explodes or becomes not a number.

Your learning rate is probably too high if your cost converges too slowly. You can try increasing your learning rate. So what does your final code look like? Well, we know we want to create a class with at least the predict and fit functions. You don't have to use a class, but I find that it encapsulates the code nicely and provides useful structure. Notice that once you have the algorithm in math, there isn't much work to convert it into numpy code. You know how to do matrix multiplication.

You know how to add? You know how to subtract. These are basic arithmetic operations that I'm sure you know already. If you don't know how to do these in NumPy, then I have a completely free NumPy course on the NumPy stack that you can take. Here is the second part of the code where we actually create an instance of the class and then use it. I've also included the squared error cost function just for completion sake. What's cool about this template is that it doesn't really change from one algorithm to the next.

The structure is basically always going to be this way, at least for supervised learning and unsupervised learning. So as we discussed before, it doesn't matter what algorithm you're implementing, whether it be linear regression, logistic regression, a neural network and so on, they always have the predict function and the fit function. And learning what goes inside these functions is equivalent to learning the algorithm. So in fact, one way to begin coding is to start with just this boilerplate code and then fill in the blanks.

2 · How to Code by Yourself (part 2)

This is what is known in software engineering as test driven development or TDD in test driven development. We start by running tests because, number one, your code should always have tests. And number two, running the tests first forces you to think about how you expect your

API to work, and it allows you to make sensible design decisions without being distracted by implementation details. So you see how this applies to every machine learning algorithm, not just linear regression.

In a lot of my older coding videos, I unfortunately decided to just write the code straight through. This helps some people but is not ideal for others. In addition, the coding videos will appear without warning. Generally, the lectures appear in the same pattern. First, we do theory and then we do the corresponding code. It pretty much always works this way. So after the theory lecture, you can be pretty sure we are going to implement that theory in code soon, unless it's so simple that it can be an exercise.

Now, to avoid treating everyone like they're in kindergarten, I don't have videos telling you that it's time to code. You're all proactive and independent adults, so when you learn some theory, you already know that it's time to try to put that into code by yourself. Sometimes you might get stuck and have no idea what to write in code or what the dataset is. In that case, it's fine to watch the next coding lecture, but don't watch it all the way through.

Watch just enough so that you know what the general idea is and what we're trying to accomplish, and then try to finish the rest yourself. Now, let's say you've watched just enough so that you know what we're trying to do. You know the theory, but you still can't complete the code. At this point, you may want to revisit the lecture theory to make sure you really understood each part. Failing that, you may want to try and get clarification on the discussion board.

Failing that, you may want to look at external resources as well. Sometimes you might be missing some important computer science background that I've assumed everyone already knows. That's perfectly fine because everyone has a different background. It's highly encouraged to do your own research, and this is also perfectly normal. It's not a bad thing. It's a good thing. Ask any university student if this is what they do, and they'll tell you that they do it all the time.

This builds a level of mental toughness and initiative taking that's important if you want to learn this stuff. There's a lot to learn and I myself am still learning new things every day. So let's say you've rewatched the theory lecture, you've looked at some external resources on the theory and you still don't understand how you would put it into code. Well, at this point, there are some great exercises you can do. Watch the coding lecture or look at the full code on GitHub.

Go through it and make sure you understand each line. The next step is without looking at the code again, try to implement it yourself. This by itself is a great learning tool. Many of the algorithms I've learned, I didn't understand them the first time. I learned them by looking at someone else's implementation and dissecting it. I looked at each part and I asked myself, Which parts did I get right and which parts did I get wrong? This helps you understand the mistakes you were making previously.

The cool thing about implementation is it works backwards too. We go from theory to code. But thinking in the reverse direction, the code actually helps us understand the theory better also. I always say that if you can't implement the code, then you don't understand the theory. So if you

can learn the theory 50% of the way, try to implement the code, look at an existing implementation, and then finally get it working. Then you can be guaranteed that you now know more than just 50% of the theory.

The act of coding helps you understand more of the theory than if you had just learned the theory by itself. So implementation helps you understand the theory, but it also helps you understand the intuition as well. Intuition is a tricky thing. In fact, I would go so far as to say it's a trap in many or maybe even most cases. Intuition is built on experience. That means deriving the theory from first principles, or in other words, deriving the theory from facts you already know to be true and from implementation.

Students often want to hear intuition from people with lots of experience. But the problem is that this intuition is built on hindsight. Intuition is often both the first thing and the last thing you want to focus on. You want to have a little bit of intuition at the beginning, so you have some general idea of what you want to accomplish. Then after you learn the theory and implementation, you can reflect on the details and summarize them at a high level.

This is real intuition built on hindsight and experience. The big mistake beginners make is that they read some intuition written by someone else who actually has experience when they themselves do not. From an outsider's perspective, it's hard to tell the difference. Why is that? Well, if the beginner can regurgitate the same high level ideas as the expert, then it would sound like they are also an expert when in fact they are not. There are, unfortunately, too many courses out there that do this.

They want you to feel good that you learn something and they want to avoid you having to do anything too difficult. And because you can repeat the same high level intuitions, it actually sounds like you're an expert. So this is why I call it a trap. Knowing intuition not only makes you feel like you know something, it also makes you really confident about this fact. And then because your ego wants to maintain this confidence, you'll start making crazy statements like the theory and implementation are not important because you already understand the intuition.

But as you know, this is completely backwards. Let's summarize this lecture, since that was a lot of information to take in. Remember, this lecture is all about how and why you should code by yourself. First we talked about why this is because it ensures you are thinking about the details and filling in the blanks using your own pattern recognition skills. It ensures you don't just assume you know what's going on, but that you actually know what's going on.

There's a big difference, especially if you're the type of person that likes to assume you know everything. Next, we talked about how we talked about how all data is the same and it doesn't really matter what domain or industry your data is from, the same algorithms all apply. Next, we looked at the interfaces to the algorithms which are also conveniently all the same. So we have a kind of mix and match type of scenario where we can input any kind of data into any kind of model.

For example, you can use linear regression with financial data, or you can use a neural network on ecological data. All data is the same in all models have the same interface. Next, we talked about a strategy for how to actually apply this. You've probably already seen that we always alter-

nate between theory and code. First we discuss the theory and then we implement that in code. Sometimes you might have to go back and revisit the theory, or sometimes you might have to look things up on the Internet.

That's perfectly normal. This is pretty much the day in the life of a university student. We're trying to build the same level of mental toughness and initiative taking in this course. So the strategy is stop after you learn the theory and try to code yourself. Sometimes you need a little more detail, so you might want to watch a little bit of the coding lectures to see what's going on and then try to implement the rest yourself. If you can't get it by yourself, then maybe try watching the whole thing or looking at the expected result and then trying to implement it from memory.

This by itself is a great learning tool because it allows you to ask yourself, What did I get right and what did I get wrong? So you can understand the mistakes you were making. Finally, it's important to understand the difference between intuition, theory and implementation and how they reinforce each other. It's important to not fall into the trap of only understanding intuition written by someone else, and then assuming that this equates to actually understanding everything.

In many cases, it's not about waiting for someone to tell you some nice phrases that trick you into thinking you understand the intuition, but rather you should seek intuition by working through the theory and the implementation. And lastly, don't forget when it's time to code, you must code.

1 · How to Succeed in this Course (Long Version)

Everyone and welcome back to this class this is a short lecture I like to give in each of my courses on how to succeed in this course. This is very important to talk about because you're learning a difficult technical subject. It's easy to get stuck. Or even worse give up because you're just not getting things. Maybe there are too many gaps in your knowledge so you can't connect the dots. Maybe some assumption you've made is incorrect. And keep in mind there is a lot of material in this course.

It can be overwhelming if you're new to this field of study and you're not familiar with the conventions and the typical patterns we use. Now if you've already seen this lecture in the past you're more than welcome to skip it if you remember what's in it and you're already using these guidelines. The first guideline is this. Remember that I want you to succeed. In fact that's why I'm giving you these guidelines in the first place. So if at any point when you watching these lectures and you become confused just ask on the Q&A.

Also known as the discussion forum Typically I respond within hours sometimes even within minutes if I happen to be on the site and I see a notification. I answer any and all questions. So even if you think it's stupid just ask. Anyway one example I get from time to time is where do I get the code for this course. And of course my answer is typically Well it's in a lecture called where to get the code for this course. At the end of the day no one cares if your question was trivial or silly.

If it's something keeping you from progressing in the course I want to answer it. Now Very rarely I get people saying well I was so confused that I couldn't even think of the right question ask and this is where you have to take some responsibility for your own learning. You have to ask yourself what did the instructor assume I know that I didn't actually know. Maybe I can look into those things and that will help me formulate a question. Maybe there's some background knowledge I'm missing a lot of people like to have exercises to practice when they're taking a course the act of thinking of the right question to ask is itself an exercise.

It might not be the type of exercise you thought you'd be doing but in many ways this is more profound. This is an exercise that requires you to think and to change how you think and ultimately that is the best value you can get out of a course. Speaking of background knowledge this is one of the most important guidelines in this lecture. You may have noticed that in the course description I've listed some prerequisites or in other words things you should know before starting this course.

Now you might wonder why do I ask you to know these things. Why don't I just teach them in the course and while we do review some topics in this course it's not the focus. There is such a wide variety of students in this Course students from almost every country in the world. Students of all ages students of all backgrounds. What does this mean. That means what's review for one person is going to be totally new for another. Some people are going to tell me they don't know calculus.

Some people are going to be very far behind with their Python programming skills. But if we start teaching Python programming skills other people are going to say hey why are we doing Python isn't the topic of this course deep learning. And I tend to agree that we should stay on topic. So how can we reconcile this well it doesn't make sense for me to give you an entire computer science education in this one course because a lot of that would be irrelevant for a lot of people.

The only logical thing to do is for all of us to meet at a common meeting point. HOw is that point defined. Well it's defined by the course prerequisites. In fact they are so important that in certain instances I've listed the course prerequisites twice just in case you missed it the first time. And I'm also reminding you now for good measure. Now one pitfall is that people sometimes assume they know things that they don't really know. So if you took calculus 20 years ago and you got a C and you skipped class and the times that you didn't show up for class you were sleeping well probably you don't know that much calculus.

But remember I want you to succeed. I want you to be able to learn this material. I can point you to resources where you can pick up this missing background knowledge but the key is you have to ask. In some cases there are certain places where I'm not allowed to post certain links so we have to be mindful of the rules of the platform we're using. Now occasionally people ask me Well why don't you just teach a course for pure beginners Why don't you teach these courses in such a way that they don't require lots of math and lots of prerequisites While I don't want to turn this into an easy beginner's type of course because for one that's not what I'm interested in personally.

And number two don't forget that this is State of the art stuff. State of the art generally isn't easy and if it were easy that would mean everyone is doing it. And if a thousand other people can do the same thing as you well what's a potential employer going to think what are your chances of getting that job when all 1000 of you have the exact same skills. So we don't want skills that are easy to get. We want skills that are hard to get skills that puts you at the top of your game skills that not many people have.

If I'm hiring someone I want the guy who stands out not the guy who says the same thing as everyone else. Finally the final guideline for how to succeed in this course. I mentioned earlier that there is a lot of material so much that it can be hard to keep track of. In addition to taking your own notes and doing all the derivations yourself how can you be absolutely sure you're understanding this material. I would in fact consider this even more important than the derivations.

Not everyone is going to be a pro at math so it's understandable if you want to skip a few steps. Sometimes it's possible to compartmentalize some concepts or delegate learning them for some other time but still understanding the high level picture and how it fits in with everything else. What you cannot skip is the implementation. Implementing these algorithms in code is the real test for whether or not you understand the material and this is not about reinventing the wheel or that you shouldn't roll your own whatever because someone else out there can do it better than you.

That is a software engineering principle not a principle of learning. In production, You don't want to write your own hash map or your own string class. And yet what do we usually do as an exercise in a typical intro to computer science class where we do precisely that. We implemented hash maps and we implemented strings. And why do we do this. Because we want to have a better understanding of how these things work. If you make an assumption that's incorrect while your code is probably just not going to work merely using a hash map and using a string only gives you a very superficial understanding that's beginner stuff that's stuff everyone knows that's not stuff that's going to help you stand out from the competition.

And it's true that you'll never use your own homemade hash map or your own homemade string in any code you write. Unless of course you are a professional that focuses on that sort of thing. They'll probably be inefficient and they'll probably have bugs but that's OK because you're not doing it to make it perfect. You're doing it to learn. You'll notice that the general pattern that this course follows is that we always discuss theory and then we implement that theory in code.

So we're always going from theory to code implementing everything that we learned. If you can try to implement something before watching the coding lecture in the appendix I have a lecture called How to code by yourself. That goes much more in-depth into this topic along with multiple strategies for how to approach implementation exercises. So to summarize these are the guidelines I think are going to help you succeed in this course. Number one make generous use of the Q&A even when you can't think of the right question ask.

Number two make sure you meet the prerequisites to this course. If you're not sure what they are or you're not sure how, see guideline number one. Number three try your best. Implement everything we learn. This is the true test for whether or not you understand something. Remember

that it's perfectly normal to get stuck once in a while. I get stuck all the time when I'm learning. That doesn't mean I'm not smart. Nor does it mean the book or video I'm learning from is bad.

It means that there is a disconnect between what the author believes I knew and what I actually know and as a learner I do my best to close that gap a good skill to have is learning how to compartmentalize concepts. So here are the things I understand here are the things I don't understand but here are the things I understand about the things I don't understand. And these are the holes. So you know what you're missing and you know where they fit and you can work to fill in those gaps over time.

One thing I've been doing quite a bit recently is adding review lectures to my courses. These are not meant to teach you everything from scratch since that's going to take you a long time but they are meant to help you recall certain things you may have forgotten. So if you have an idea for a review lecture that you'd like to see added to this course please feel free to make a suggestion using the Q&A.

2 · Is this for Beginners or Experts Academic or Practical Fast or slow-paced

Everyone. And welcome back to this class. This lecture is designed to answer some common questions I get about how difficult this course is and where it fits in terms of academia, the professional world, and its practical applicability. So rather than answering this question individually over and over again, it seems like a better idea to make a lecture about it and answer everybody at the same time. A lot of people become confused because this is their first machine learning course and they just don't know where it fits in the grand scheme of things.

So is this course academic or practical, and is it for beginners or experts? Is it fast paced or slow paced? I'm going to answer the second question first. Is this course for beginners or experts? Interestingly, the problem with this question is a problem for machine learning, too. Let me explain why this question is itself problematic. One of the challenges in natural language processing is that of ambiguity. When someone says some words, the problem is those words can mean multiple things.

And it just so happens that I have some courses on NLP, so this is a problem I think about often. So how does that apply here? Well, when we use the words beginner and expert, we have to ask a beginner from his perspective, an expert from his perspective. For some of you, you might think that a beginner is a random person. You take off a street who knows nothing. But even that is ambiguous. There are people out there who do not even really know how to use a computer beyond turning it on and checking their email and browsing the web.

There are people out there who are pros at Microsoft Office so they know how to use a computer and make a microsoft Word document or a spreadsheet. But they've never done any programming before. There are people out there who are very good developers and have lots of programming experience, but they have no idea what machine learning is. So all of these people are beginners in some way, but each of them would require a vastly different amount of work to prepare for this course.

~~It's the same situation with experts. Most people consider anyone who knows more than them to be an expert. But of course, how much you know and how much some other person knows is relative. If you know how to plug and play into scikit-learn and nothing else. You might seem like an expert to someone who only knows Excel, but you'll seem like a beginner to someone who's read Bishop's pattern recognition and machine learning book. So just to finalize our answer to this question, is this course for beginners or experts?~~

~~This question is misinformed to begin with because beginners and experts are not well-defined terms. Instead, we have to go back to what is the specific skill set you need before starting this course? And of course, that's defined by the prerequisites which are listed in the course description. So precision is the key here. We want to know what is the precise list of skills that I need before taking this course? This is much better than ambiguous terms like beginner and expert.~~

~~Another matter is that people are going to measure their own skills differently. That's a much tougher issue to deal with. Sure, I can say you need to know Python coding before starting this course, but you can be sure that there's a big difference between someone who just learn hello world and for loops versus someone working at a big financial institution. Now, even in those big financial institutions, there could be big gaps in performance even within your own team.~~

~~There can be big gaps in performance. I've seen guys who have been working for 15 years do worse than guys who've been working less than one year. So even if I say you need to know Python coding, there can be a lot of variation in skill among those who consider themselves good at it. A similar situation happens with deep learning. I can say you need to know how neural networks work, but there's a big difference between someone who can code them from scratch and has experience using them versus someone who just watch some online animations for a few hours.~~

~~If you had to bet on who knows neural networks better. Who would you pick? So to really answer this question, we have to turn to the course itself. If I say you need to know Python coding and you claim that you do, but you don't understand some Python code in this course, then simply put, your python coding skills are not yet up to par. So you have to improve your python coding skills by asking questions on the Q&A and taking the appropriate steps to cover your gaps.~~

~~Quite often I get people who are just too confident in their own abilities when unfortunately the reality is there is a huge discrepancy between what they think they know and what they actually know. What's funny is these people sometimes say this course requires Ph.D. math. I assure you there is absolutely no math in any of my courses that requires a Ph.D. or that only a Ph.D. can understand. My courses contain only undergraduate math at most. Sometimes people even claim to have a Ph.D.~~

~~and then claim that it's still too hard. Which is funny because that essentially invalidates your entire Ph.D. The entire point of a Ph.D. is to turn you into an independent researcher. And so if you're supposed to be at the level of doing independent research, but you can't understand some undergraduate math, well, that's not very good. Essentially what it boils down to is people are really trying to say everything I don't understand is not important.~~

~~Everything I do understand is important. And therefore, everything you say that I don't understand, I don't need to know. You can see why this thinking is dangerous. If you truly believe that everything you do not know is not important, then you'll never learn anything new. Now this brings us to our third question. Is this fast paced or slow paced? And I think you can see where I'm going with this. This is yet another example of ambiguity. Fast paced to whom?~~

~~Whether it's fast paced or not, depends on which person is taking the course. So if someone is well prepared, then things will be just right. If someone already knows most of these topics, then it will probably be too slow. If someone chose not to prepare for the course or they have an inadequate understanding of the prerequisites, then it's going to be too fast. So if I expected you to know something and you didn't. Then you have to look it up. Which is going to make the course feel like it's going too fast.~~

~~So here is the real test. If I say something depends on calculus and probability, and you claim to know calculus and probability, but you don't understand the thing that depends on calculus and probability. You have to ask yourself, honestly, do I really understand calculus and probability? Or did the instructor invent his own kind of calculus and probability that I don't yet understand? Well, of course, that's not very likely. It's most likely that you thought you understood calculus and probability, but you really don't.~~

~~Now on to our initial question. What kind of course is this? Is it academic? Is it for professionals? Is it practical? So to understand this, you have to understand where machine learning is usually taught and how it is usually taught. Firstly, machine learning is usually taught in upper year computer science and engineering programs in university and college. So these are students who already took calculus one, two, three, linear algebra, differential equations, probability and statistics, discrete math programming, and possibly more all in their first two or three years of college.~~

~~So by the time they get to third and fourth year, they know all this stuff and they're ready to apply it. So a typical academic course might cover a whole library of machine learning algorithms. So you might learn k-nearest neighbor, logistic regression, k-means clustering, pca and so on all in the same course. Typically you'd cover one algorithm per lecture, which is about 2 hours. Overall, you might have something like 12 or maybe 20 of those lectures in a term.~~

~~And by cover, I mean mostly via math, geometry and derivations. These courses don't typically have you at your computer doing programming work. If you do end up doing programming, it's usually part of a lab assignment, which is actually just a very small part of the course compared to the rest. So it's mostly theoretical dealing with equations, solving equations, reasoning about equations. Those equations typically involve probabilities and information theory.~~

~~So you have 2 hours to go through the theory for one algorithm. And then if there's coding involved, which there usually isn't, you do it on your own time. The lectures themselves do not involve any coding. But there's an even bigger problem with these academic courses. And what is that? What happens is that the grades turn out to be very low. Out of 100%, the average might be around 25% or up to 40%. Now, you might think, wow, so everyone fails. And of course, that's not the case since everyone gets shifted based on the average.~~

So what is the real problem here? A lot of people ask me, well, why don't you cover every machine learning algorithm in one big massive course and just call it machine learning? Well, the answer is that if you try to do too much, you only end up understanding 25% of the material, which I think is clearly demonstrated here. And as they say, the proof is in the pudding. If you look at these courses where students are trying to digest ten different algorithms at the same time, what is the result?

The result is they don't understand any of them. If you try to understand too many things, you end up understanding nothing. I take the perspective that if you're going to take a course, you should aim to understand 100% of the material. Each course is broken up into appropriate levels based on how difficult they are and by topic. So for example, you can typically take calculus one, which focuses on differential calculus, calculus two, which focuses on integral calculus and Calculus three, which focuses on multivariable calculus.

You would never learn all of these in just one huge course called math, because that's just too much stuff to understand all at once. I want you to understand 100% of the material, even 50% or 75% is not good enough, in my opinion. If you only understand 75% of the material and the next course depends on the 25% that you didn't understand, well then you're not going to make it to the next level. Now let's look at the opposite end of the spectrum, what most people might consider to be a practical course.

I don't consider these to be practical courses, and I'll explain why later. But I use the word practical here in quotes, since that's what some of the more beginner students will refer to them as. So how does a practical course typically proceed? Well, first, you are never really taught how the algorithms work, other than at a very high level. You might learn how they work by way of analogies and metaphors. So for example, suppose you are learning about gradient descent.

Well, the instructor might relate this to a ball rolling down a hill. Now some beginner students, they might conclude that I understand a ball rolling down a hill. Therefore, I understand gradient descent. But they neglect to understand how to take derivatives. They neglect to understand how to choose a good learning rate. And most importantly, they neglect how to implement gradient descent in code. So it's totally wrong to assume that just because you can visualize a ball rolling down a hill that you understand gradient descent.

Why? Well, the problem is, everybody can imagine a ball rolling down a hill. Your grandparents, who can't even turn on a computer, can understand a ball rolling down a hill. So don't equate understanding an analogy to understanding the real topic. So what's the next thing these practical courses do? The next thing they usually teach you is how to plug into an API or in other words, make use of a library someone else wrote. This is also not ideal. Typically, this involves just two or three lines of code.

You have to wonder if machine learning just boils down to two or three lines of code. Why is it such a big deal these days? Why did it take decades for people to realize its usefulness? And of course, the answer is that there's much more to machine learning than just these two or three lines of code. When you use a deep learning library like Keras, you're not doing deep learning. You are using a deep learning interface. You're simply programming. It's more accurate to say I can program with deep learning libraries rather than saying I actually know deep learning.

But some of you might say, well, that's all I really want to do. All I need is an API and for the API to give me an answer. I don't care about how it works. But then here is where you run into trouble. Recall that in addition to all data is the same. We know that all machine learning interfaces are the same. So the question becomes which API do you choose? If they're all the same. How can you make an informed choice about which one to use? Well, the reality is, if you don't know how these things work, then you can't answer this question.

This is why some of these so-called practical courses can cover 20 algorithms in the same course. It's because they never actually talk about how the algorithms work. Since the API is the same every time, it's very easy to just do the same thing 20 times and then not bother with the details. So this gives people a lot of confidence that they were able to do the same thing 20 times. But it's dangerous thinking because you don't realize you really only learned one thing.

You just repeated it 20 times. So how about when it comes to the professional world? Sometimes people tend to believe that in your profession you'll be leaning more toward the practical end of the spectrum, where you don't really need to know how a machine learning model works. You just have to know its API. But what it really comes down to is what your job actually is. If you're a regular, everyday programmer and you just want to see how a machine learning model might perform on your data, then you're going to use a pre-built library, most likely.

That's just the two or three lines of code I referenced earlier. However, that does not mean you don't still have to conform to best practices. Make sure you're not mixing your test set with your train set. Make sure you're doing proper validation and so on. And when it comes to tuning your model, then you have to be able to dig down into the details and see which hyper parameters are possible to change and which are likely to have the most impact. So only knowing how to use an API is only going to get you past the first stage.

You can plug in your data and at least your code has the right syntax and doesn't crash. But of course your code not crashing doesn't necessarily mean it's right. On the other end of the spectrum, you have programmers and scientists whose sole job it is to do machine learning. These guys are machine learning experts. They know the math inside and out and have a full computer science education. And so to even be able to communicate your ideas effectively, you have to be comfortable with the theory and not just how to plug into an API with two lines of code.

These scientists are building custom models, really digging down into the math to not only use but invent new things and to be able to invent new things. You really need to know the math very well. And so if you're on a team like this, you can't just know how to write two lines of code. Your skill set has to be far beyond that. So your theoretical background has to be very solid if you want to be able to communicate effectively with other scientists. Now a lot of people say, well, I don't ever want to be a scientist, so I don't really care how these algorithms work.

Unfortunately, that's a very bad attitude to have in the workplace. Suppose, for example, I am running a business. Now an employee of mine, he wants to show off and say he's learning about machine learning. He says, Oh, I'm taking this machine learning course online. Therefore, I will become a machine learning master. But his approach to machine learning is to learn as little as possible. He says he doesn't care about the math. He only wants the practical stuff.

He only wants to know the bare minimum. He tells me he doesn't care how it works. He's just going to plug into some API because that's what his buddy told him he could do. As a business owner, I really don't want this guy working for me. I run a business that sells something and the life of the company itself and the livelihoods of the employees all depend on our products being the best they can possibly be. So if someone comes to me and says, I don't care about being the best, I just care about doing the bare minimum.

My response is, I don't want that guy on my team. In fact, he should be fired. So be careful if this is your approach to machine learning. No business owner wants to hear that your approach to the business is to do the bare minimum and to put off some math because you think it's too hard. That's a scary prospect for a business owner. Now, if we go back to our original question, maybe you've realized we haven't answered it yet. What kind of course is this?

Well, you can think of it as a marriage between these approaches. We want to cover all the math and derivations, because that's very important. If you want to be able to communicate effectively with other scientists, to be able to debug your code effectively and to be able to invent new approaches. This is also what we mean by machine learning. We mean really knowing machine learning and not just programming with an API. So the theoretical background is very important.

At the same time, we're not going to introduce 20 different algorithms in the same course because unlike a typical academic course where you can pass just by understanding 25% of the material in this course, it's structured in a way that if you have any missing gaps, it's going to be impossible for you to move on to the next stage. But academic courses don't contain much coding, if at all. So in this course, every algorithm we learn about will be implemented in code.

My motto is, if you can't implement it, then you don't understand it. You can pretend to understand it. And when you talk about it, you might sound like you understand it. But implementation is the true test. It's like a math exam rather than writing an essay. When you write an essay. You might be able to fool the teacher into thinking you understand something. But in a math exam, either your answers are right or your answers are wrong. In this context, either your code works or it doesn't.

It's a very clear cut. When it comes to practicality, we cover that, too. We demonstrate our algorithms both on real world data like text and images, and we demonstrate our algorithms on two dimensional datasets so that you can visualize what an algorithm is doing. Both of these are important. Text and images are quite possibly the most practical data there is. Some billion dollar companies exist and thrive because of those capabilities. 2D data sets are critical as well because as humans, that's the only thing we can see.

But at the same time, we don't try to pull the wool over your eyes and pretend like you need 20 different datasets to practice on. Instead, we learn something much more valuable how to use an algorithm on an infinite number of datasets because we are able to abstract the idea that all data is the same. Remember that all the computer sees is a list of numbers. It doesn't know that those numbers represent height measurements or an image pixel. And of course, by experiencing both ends of the spectrum, both the theory and the implementation, and then finally plugging data into your model, that's going to make you very well rounded professionally.

To conclude this lecture, I want to pose a final question. How does all this knowledge help you? Why does understanding the approach of this course make you better at machine learning in general? Well, my hope is that this gives you a very high level bird's eye view of the landscape. You can see the different approaches of people from different backgrounds. Academic people typically don't see what other non-academic people are doing. And those people who hate math typically don't see what people who love math are doing.

So with this bird's eye view, you can see everybody. If you're learning machine learning, it helps you understand who am I competing with when I apply for a job and what kinds of jobs are out there and what would be adequate preparation. If you're hiring for a machine learning team, it helps you understand the different backgrounds of individuals. You have to compare to one another to understand their strengths and weaknesses and what they bring to the table.

3 · Machine Learning and AI Prerequisite Roadmap (pt 1)

Everyone. And welcome back to this class. In this lecture, I'm going to answer a question that I get pretty much multiple times per day, and that is what order should I take your courses in? This is a great question, and it's something that even I have to think about sometimes. What is the best way to learn all this stuff? So I think that what you'll find is this ordering makes a lot of sense in that we're always building on skills we learned before. So it's just like learning calculus.

First you learn calculus one, then you learn calculus two, which is kind of the opposite of calculus one. Then you learn Calculus three, which expands on the ideas from both Calculus one and two. And then from there, you can go on to even more advanced topics like probability. So this is a process based on skill building. Now it's important to keep in mind that this lecture is not just about my courses and how to navigate them, although that is the question I'm answering, since that's a question I get multiple times per day.

But this is also a lecture on how different machine learning topics are related and what depends on what else. So even if you end up not taking any of my courses, it doesn't change the fact that these topics are still related to each other in these ways. So in some sense, this lecture isn't really about my courses at all. It's a general lecture about how different types of machine learning models are related and in what order you might want to learn them.

From a skill building perspective. Now I want to show you this stuff in a visual way. I've actually set up a webpage where you can go to and look at the chart I made for yourself. So what I want you to do is go to this website. deeplearningcourses.com. Now I want you to click on the catalog link. And from there you want to click on the click here link. Okay. So this brings us to the course order web page. So at the top you have sort of this linear chart, right?

It's just one course after the other. After the other. But this is not really what I want to look at in this lecture. This linear chart gives you a basic overview, but it's not ideal. The reason is the relationships between courses are way more complex than this. Sometimes one course gives you the skills you need for two other courses. Sometimes there will be a course that's so complex that it depends on multiple different courses. So this is the perfect opportunity to use a graph.

So if you scroll up at the top, you can use this link to just jump down to the bottom. And we have this graph where you can see the dependencies between each course. So what I'm going to do is I'm going to go through each link in this graph and explain to you why it exists and why it's important. So let's look at the SQL course first, since that has no links. So it might show up in the middle so you can just drag it out to the side. That's the coolest thing about this graph because you can just pick stuff up and drag it around.

It's very interactive, so SQL has no links. This isn't a prerequisite to any other course, nor is any other course a prerequisite to it. It is stand alone. You can take it at any time. Next, you'll notice that there are courses that have no links going into them. So, for example, the Numpy course. This is because none of my other courses are a prerequisite to this course. This course was just designed to give you a basic understanding of the syntax and tools we use in data science and machine learning.

The goal is actually to not do any machine learning in this course. Believe me, it was actually quite difficult to accomplish that because it's so easy to use something in machine learning as an example. It was very hard to avoid talking about it. You'll also notice that there are some courses with no outgoing edges. You can think of these as the most advanced courses. They are at the top of the ladder. So, for example, this one right here. Of course, that could change in the future as I create more courses, in which case I will update this lecture.

So let's start with what's coming out of NumPy. You'll see that there are two links. One goes to linear regression and one goes to Bayesian machine learning AB testing. And by the way, you can zoom into this graph. So that's what I'm going to do right now. Okay. So here's linear regression. Here's Bayesian machine learning. So while NumPy goes to both of these courses, I actually strongly recommend linear regression as the next step. Bayesian machine learning doesn't depend on any other courses per se, but it's still more advanced.

It requires a more advanced understanding of probability and of real world engineering problems, like optimizing the click through rate of a link on your website. So linear regression would be my number one recommendation for your foray into machine learning. Now after linear regression, we have logistic regression. So why do we go from linear regression to logistic regression? Well, the basic idea is both of these are linear models. In other words, the model is of the form Y equals M X plus B, which is a line.

The difference is that linear regression does regression while logistic regression does classification. We want to learn classification because that's one of the essential tasks of a neural network, which is what comes after. And you might notice that linear regression also has two outgoing links. So there's one to logistic regression and one to reinforcement learning. This is because my reinforcement learning class makes use of linear regression. So naturally, if you don't understand linear regression, you won't understand that part.

But again, this is a much weaker link when going from linear regression to logistic regression. This is because reinforcement learning is actually conceptually more difficult. It doesn't depend on the content of any other courses directly other than what we've shown. But the ideas behind

it are more advanced. You really want to have some experience with both supervised and unsupervised learning before jumping into reinforcement learning. It helps to build the proper perspective on the subject.

We will get back to reinforcement learning later, but for now, let's go along the deep learning path. So on the deep learning path, we last discussed logistic regression. This is also known as a neuron. This is very important because in order to build a neural network, we must first know how to build a neuron. A neural network is just a bunch of neurons stuck together. So this leads us to deep learning and Python part one, which is the bright green logo.

This course teaches you how neural networks work and basic fundamental operations like back propagation. Once you know the basics of deep learning, you can move on to deep learning. Part two or modern deep learning in Python. This is the same image, but in a darker color to signify that it's a continuation of the previous course, as the title suggests. This course is all about modern developments in deep learning, so we go over ways we've improved basic back propagation like momentum and adaptive learning rate techniques like RMSprop and Adam.

We talk about modern regularization techniques like dropout and batch normalization. And most importantly, we look at modern, deep learning libraries like Theano and TensorFlow. We also take a look at Keras, PyTorch, MXNet and CNTK. You'll see that once you know the fundamentals, things don't change that much from one library to the next. It's all the same concept. Now learning about these modern libraries sets the stage for the next level, which is applying deep learning to certain special data formats.

The two fundamental types of data are text and images. These are special because they have unique structural properties and we can make use of our knowledge of those properties in order to build better neural networks. For example, we know that text is made up of sentences, and sentences are sequences of words. So sequential modeling is important. We know that images are two dimensional objects, but importantly, each pixel is very likely to be the same as nearby pixels.

For example, if you pick a random pixel on an image of a red car, if that pixel is red, then most likely the pixels around it are also going to be red. There is also no question about the practical applicability of text and images. The Internet is made up of text. So by doing machine learning on text, you're learning techniques that essentially allow you to write models of the entire world's knowledge. Images are also everywhere. Snapchat. Instagram. Facebook.

Self-driving cars. Home security systems. Knowing how to deal with images is super important, and billion dollar companies exist because they are good at it. So that brings us to the next course in the deep learning series. Convolutional Neural Networks or Deep Learning in Python Part three. This is all about how to incorporate convolution into neural networks and what convolution is in the first place, and how that helps us make use of the structure of images.

Let's scroll up a bit. So after convolutional neural networks, we take a detour into unsupervised, deep learning. Unsupervised, deep learning isn't very popular with beginners, but it should be. If you look at the experts of deep learning, like Yann LeCun, Yoshua Bengio and Geoffrey HIinton,

they all talk about how important unsupervised, deep learning is. This course introduces us to several new ideas. First, the idea of latent variables or latent causes.

These are factors that you don't directly observe in your data, but that you can learn about by studying the structure of your data. Second, modern techniques for data visualization like t-SNE. This will be very important since as I discussed earlier, along with images, text is a really important data format. In order to visualize text in later courses, we will be making use of t-SNE. So it's good to know what it is and why it's useful. Third Unsupervised Pre-Training Unsupervised Pre-Training is the foundation of modern ideas like transfer learning, and we make use of it both in NLP and with convolutional neural networks.

Fourth, the vanishing gradient problem. We demonstrate the vanishing gradient problem directly, so you understand where it comes from. Vanishing gradients are especially important in the context of recurrent neural networks. And it just so happens that recurrent neural networks is the next course in the Deep Learning series, Deep Learning in Python Part five. So recall that our two fundamentally interesting data types are text and images. We've looked at images and in this course we start to look at text.

Why might that be? Well, recurrent neural networks specialize in modeling sequences, and text is just the sequence of words. So it's the best type of data to use to demonstrate the principles of RNNs.

4 · Machine Learning and AI Prerequisite Roadmap (pt 2)

Once we've studied RNNs, we can go even deeper into how deep learning can be used on text. That's the next course. Natural language processing with deep learning in Python or deep learning in Python. Part six. In this course, we look at a very important concepts called word embeddings. These allow us to turn words which are categorical variables into vectors, which are numbers that a neural network can read. This is why it's very important to have studied unsupervised learning before this, because finding word embeddings is actually an unsupervised task.

Word embeddings also allow us to make use of pre-training, which was discussed in deep learning in Python four. In this course. We also look at a very advanced model for doing sentiment analysis called a recursive neural network or a tree neural network. This is an example of a dynamic neural network because it changes its structure based on what input you give it. Most deep learning libraries are not equipped to handle dynamic neural networks, and I demonstrate what happens if you try to build one the naive way.

You basically end up building a separate neural network for each of your training samples, and it's going to eat up all your RAM and make your computer slow down to a crawl. So you get a first-hand perspective of why working with dynamic neural networks is not easy. But luckily we are able to make use of our knowledge of recurrent neural networks. And remember, that's the prerequisite for this course in order to convert a tree into a sequence which an RNN is capable of handling.

So that's why RNNs is a prerequisite to this course. Now you'll notice that this course Deep NLP has no outgoing edges. This is because it's the most advanced course that I have on this path. For now, I certainly plan on expanding in this direction in the future. However, this is not the last deep learning course in the series. After this, we have deep reinforcement learning, which is deep learning in Python Part seven. With that said, I think now is a good time to go back and start exploring the reinforcement learning path.

So you'll notice that I have Bayesian machine learning, feeding into reinforcement learning. On the surface, these two courses might seem unrelated, but there is a very important concept you'll learn that applies to both. called the explore-exploit dilemma. In Bayesian machine learning, you learn this idea in the context of trying to optimize a click through rate or a conversion rate, or, in other words, the number of times people buy things from your website versus the number of times people have visited your website.

Very practical concept, I think if you do anything related to e-commerce. Now in reinforcement learning, we look at the explore exploit dilemma again, but in the context of playing games. Reinforcement learning is like a third branch of machine learning, whereas the other two are supervised and unsupervised learning. The main difference is that supervised and unsupervised learning look at static data. In reinforcement learning, the idea is more like you have a robot living in the real world.

It can take the experiences it had today and based on them, behave more intelligently tomorrow. So the learning paradigm in reinforcement learning is sequential. This is opposed to supervised and unsupervised learning, where your data set usually resides in some file on your hard drive. So in reinforcement learning part one, we get all the basics, as you might expect. This is a prerequisite to deep reinforcement learning, since deep reinforcement learning applies those concepts to more difficult games.

But you'll notice that this is not the only prerequisite to deep reinforcement learning. As the title suggests, this is also dependent on knowledge of deep learning, in particular convolutional neural networks. But as we know, in order to build a CNN, we have to know how to build a regular neural network, which means we have to know what a neural network is and why it's useful and so on. So in deep reinforcement learning, these two paths converge. You combine your knowledge of both reinforcement learning and deep learning in this course.

Now the reason it depends on CNN's and not say RNNs, which is over here, is because it will be learning to play visual games. So, for example, we can learn how to play a video game like Pong and Breakout, which are classic Atari games from the old days. And so those are images because they're basically screenshots from the screen. In the future, we might end up applying RNNs, in which case RNNs will become a prerequisite to that course.

So that's the end of the reinforcement learning path for now. I'm very excited to bring you more updates in this area in the future. Let's now jump back to logistic regression. Where we can see another outgoing edge to supervised machine learning. So why is this edge here? Well, you may recall that linear regression and logistic regression are both linear models that do regression and

classification respectively. These are both supervised learning tasks. And so it makes sense that now that, you know, one model for regression and one model for classification, it's time to dig deeper into supervised learning.

The thing with linear regression and logistic regression is that they aren't really different models. They're both just a line because they do different tasks. The techniques and interpretations are slightly different, though. And there are, of course, different models that are not linear models that can do these tasks. And that's what this course is all about. In this course, we look at classic supervised machine learning techniques like k-nearest neighbor decision trees, the perceptron and the Bayes classifier.

Much like how logistic regression was the basic building block of the neural network. Classic models like decision trees are the basic building block of ensemble methods. So that's why this course is a prerequisite to ensemble machine learning. Again, we use the same logo with a different color to signify that these two courses are very closely related. In ensemble machine learning, we learn how to combine multiple decision trees in different ways in order to make some very powerful classifiers.

What's really remarkable about these methods is that they are very easy to plug and play on data. So if you're looking for a plug and play solution without having to learn a lot of theory, then deep learning is most likely not for you. But ensemble methods are a great fit. Deep learning is very dependent on hyper parameters, and if you choose incorrectly, your model will perform very poorly. Sometimes it requires immense computing power to find good hyper parameters.

This is an active area of research that has not yet solved. This is why you can implement what you see in a deep learning paper. But suppose the author left out some seemingly insignificant detail. So you end up having to make an assumption and then your results end up totally different. So deep learning is fragile. But luckily, ensemble methods are not. We focus on two very famous ensemble methods The Random Forest and AdaBoost. So that's everything on the supervised machine learning track.

For now. Next, we see an edge going from supervised machine learning to unsupervised machine learning, in particular cluster analysis. The reason we study supervised learning before unsupervised learning is because unsupervised learning is a little more abstract. It takes more effort on the student's part to realize why it's practical and what it can be used for. Cluster analysis shows us how to model data that does not come with targets, as you might guess.

We do this in the form of clustering. The idea behind clustering is very simple. We want to know how many naturally occurring groups of data there are and what are the relationships between the data in these clusters? So for example, if you were clustering books, you might find a book about Steve Jobs and a book about Elon Musk in the same cluster. This cluster is probably about tech companies in Silicon Valley. But you don't need a label in your data to tell you that.

You can discover it yourself by looking at how the data naturally groups together. Now, as I mentioned earlier, I think once you learn about both supervised and unsupervised learning, you'll be ready to jump into reinforcement learning. I haven't made cluster analysis a prerequisite

to reinforcement learning, since none of the material depends on this course. But it's good to know about these techniques so that you have a more mature and experienced view on machine learning.

What is sort of a sequel to cluster analysis is Hidden Markov Models. The reason might not be clearer first. So let me give you two reasons. Number one, they're both unsupervised machine learning models. Just that HMM is harder, so it's natural to learn about clustering first. Clustering is also about static data, whereas HMMs are about sequences. So it's similar to the process we did in deep learning. We looked at static data like images, then sequential data like text.

Reason number two, in cluster analysis, we learn about a technique called the Gaussian mixture model, which we make use of in the HMM course. One key point is they both learn by using the expectation maximization algorithm. So it's good to first see the EM algorithm on a simple model. And then when you see it again on a more complicated model like the HMM, it won't be as intimidating. One key concept you learn in HMMs is the Markov assumption. That just means the current state depends only on the previous state, but not any states before it.

This is a simplifying assumption that usually makes the math easier to work with. You will also notice that we encounter the Markov assumption in reinforcement learning. HOwever, it's not too hard to learn it from scratch. And so for that reason, I do not consider HMMs to be a prerequisite to reinforcement learning. The Markov assumption is really the only thing they have in common. There is also a slight connection between cluster analysis and unsupervised deep learning.

So I'm not going to draw the link right now, but I sometimes consider this to be unsupervised machine learning part one and this to be unsupervised machine learning part two. We also see that HMMs feed into the RNN course, which is about deep learning. So why might that be? This is, of course, because both these models are models that can learn about sequences in particular. In both these courses, we model text as sequences. But whereas the HMM makes use of the Markov assumption, the RNN does not.

And hence the RNN is a more powerful model. And so this just goes along with the main theme that we always go from simple basic models to more complex models. This is also something you should do in your work as well. If you start with a simple model, you often find that it is faster and more robust. Complex models sometimes break down, but they are also more difficult to implement and might not even be fast enough. Of course, that's just a generalization.

So you always want to analyze engineering tradeoffs individually for every problem you have. Now there is one last link in this part of the graph here that I want to explain, and that's the first NLP course. You can see that it depends on supervised machine learning and feeds into deep NLP. The main purpose of this basic NLP course is to apply basic machine learning models to text. So that's why supervised machine learning comes before. It is because that this course was all about basic machine learning models.

The important skill for NLP was not the implementation of those models, but rather a bigger picture perspective on how machine learning is used. What is the interface between the data and the model? What does the model do? What is its input and output? HOw is the output interpreted?

And so we take those principles and we apply them to text. In this way, we can see that text can be treated in such a way that you don't have to think about it any differently than any other data.

This reinforces the principle that all data is the same. The machine learning model doesn't care what your data is. All it sees is a table of numbers. It doesn't care if it's text or images or radio signals from space. The model just does what it was designed to do on the numbers that you give it. So this course gives you a high level systems perspective on working with machine learning models in text. This easy NLP course also feeds into deep NLP, which is of course not so easy because it depends on a lot of background in deep learning.

One of the main questions I get in the NLP course is how do I improve the results of these basic models? And a lot of the time the answer to that is, well, you have to use a more complex model, but of course that necessitates learning how that complex model works. And Deep NLP is an example of that because we learn a state of the art method for sentiment analysis, whereas in easy NLP we used only a linear model. So it's important to understand that while yes, it's possible to improve the predictive ability on simple basic models, as you can see, it's not always an easy path to get there, so you have to make sure you're prepared.

Case in point, just look at all the time spent just to get to deep NLP. It's not an easy task. Let's now scroll over to GANs and variational auto encoders, just like how deep reinforcement learning is not related to Deep NLP. This isn't really related to deep reinforcement learning either. This is deep learning part eight by order of creation only. It is the spiritual sequel to Unsupervised Deep Learning, which was deep learning in Python Part four. Okay.

So just to reiterate, this is part six, part seven and part eight by order only. But they are not related to each other conceptually, although it's always nice to know all these things because more context makes future things easier to learn. So the reason this is linked to this is because GANs and variational auto encoders are also unsupervised, deep learning models. But whereas unsupervised, deep learning was all about how to improve supervised learning, GANs And variational auto encoders don't have any direct benefit to supervised learning at all, although we do make use of supervised learning within the course.

In this course, the focus is on generating images. We've seen that GANs can create photorealistic images based on a dual neural network system. That's pretty cool because before GANs, we didn't have any kind of machine learning model that could generate real looking images. Nowadays, GANs are able to generate high resolution, high quality images of people that you can't even tell are not real people. It certainly makes the idea of The Matrix seem very possible.

All right. So I hope you found this lecture helpful. We saw that these courses are related to each other in some pretty complicated ways. Learning machine learning is not exactly linear. Sometimes you have to take one course before you can take the next. Sometimes one course might just be related to another course by some key concept, but maybe in one context it's a lot easier to understand. So remember to keep in mind that these arrows do not all indicate strong prerequisites, or rather, there's just a relationship between the two courses.

I hope that this lecture explained any nuances between what is a prerequisite and what is not. And I hope I did a good job of answering which order should you take these courses in and why?

1 · What is the Appendix

Everyone. And welcome back to this class. In this section, I'm going to go over some common questions that students have and some general tips that have helped students in the past. Please keep in mind that this is not a core part of the course. These are just common general questions that students have had about machine learning or about my courses in general. I realized that it's not useful for everyone, but please keep in mind that it's useful for someone.

There are a few people out there that tell me they think all the lectures should only answer their questions but not anyone else's questions. So just keep in mind that many of your fellow students have asked me these questions in the past, and in order to answer those questions in a scalable way, I've created this section. Please try not to hold it against me, as this may not directly help you, but it may help your fellow students. Please also keep in mind that the appendix section in no way takes away from the core content.

All the material provided for the topic matter is exactly what I wanted to provide. These appendix lectures are simply extra stuff, so these appendix lectures do not displace any technical content. If the appendix section did not exist, the technical part of this course would be exactly the same as it is now. So just remember this. The appendix is in addition to the technical content. It does not displace any technical content. You get all the technical content that I intended.

Also keep in mind that you're always able to select the lecture you want using the user interface. That's this thing. If you don't know how to use the video player to select a lecture, please ask me how on the Q&A or contact Udemy support. So you're encouraged to select the lectures you want to watch manually and skip lectures you don't want to watch. Also, keep in mind, however, that a common mistake is in thinking that this material is not going to help you, when in reality it will.

For example, I come across students all the time who have no idea how to get started implementing machine learning algorithms in code or why we are doing so in the first place. Then I come to find that they have never watched how to code by yourself. So I think it goes without saying that if you don't know how to code by yourself, you should probably watch the lecture. HIow to code by yourself. Same thing goes for the other lectures in this section. All right, let's get to the lectures.

2 · BONUS

Hey, everyone, and welcome back to this class. In this lecture, I'm going to answer one of the most common questions I get. Where can I get discount coupons and free deep learning material? Let's start with coupons. I have several ways for you to keep up to date with me. The absolute number one best way for you to keep up to date with newly released discount coupons is to subscribe to my newsletter. There are several ways you can do this. First, you can visit my website lazyprogrammer.me.

At the top of the page there is a box where you can enter your email and sign up for the newsletter. Another website I own and operate is deeplearningcourses.com. This website largely contains the same courses as you see on this platform, but it also contains extra VIP material. More

on that later. So if you scroll to the bottom of this website, you'll find a box to enter your email which will sign you up for the same newsletter as you would on lazy programmer.

Mhm. So you only have to do one of these. Now let's do a small digression because this is another common question I get. What's this VIP material all about and how can I get it? So here's how the VIP thing works. Usually when I release a course, I'll release it with temporary VIP material, which is exclusive for those early birds who signed up for the course during my announcement. This is a nice little reward for those of you who stay up to date with my announcements and of course, actually read them.

It's important to note that VIP material can come out at any time. For example, I could make major updates to a course three years after starting it and do another VIP release. The purpose of deeplearningcourses.com is to have a permanent home for these VIP materials. So even though it could be temporary on the platform you signed up on, if you signed up for the VIP version of the course, then you'll get access to the VIP materials on deeplearningcourses.com permanently upon request.

Here are some examples of materials you might find in the VIP sections in my TensorFlow two course. There are three extra hours of material on Deep Dream and object localization. Usually I don't release the VIP content in video format, but this was an exception. Another example in my cutting edge AI course was an extra written section on the TD3 algorithm. This course covered three algorithms in total, so the extra section gives you one more, or in other words, 33% more material.

Another example in my advanced NLP and RNNs course is a section on speech recognition using deep learning. In addition, there is an all new section of the course on stock predictions or memory networks, depending on which version of the course you are taking. The reason for this is I might release slightly different versions of each course on different platforms because of how the rules on all these platforms work. I must differentiate the courses. However, since I own deeplearningcourses.com,

this is the only platform that contains the most complete version of the course, which includes all the sections. Please note that this is rare, so depending on what course you are taking, it might not affect you. All right. So let's get back to discount coupons and free material. Other places where I announce discount coupons are Facebook, Twitter and YouTube. You might want to pause this video so you can go to these URLs and follow me or subscribe to me on these sites if they are websites that you use regularly.

So for Facebook, that's facebook.com/lazyprogrammer.me. For Twitter, that's twitter.com/lazy_scientist. For YouTube that's youtube.com/c/lazyprogrammerx. Occasionally I still release completely free material. This is nice. If I want to just talk about a singular topic without having to make an entire course for it. For example, I just released a video on stock market predictions and why most other blogs and courses approach this problem completely wrong.

That's another benefit of signing up for these things. I get to expose fake data scientists who are really marketers, whereas I wouldn't ever make an entire course about that. Sometimes this can be in written form and sometimes it can be in video form. If it's in written form, it will either be

~~on lazyprogrammer.me or deeplearningcourses.com. If it's a video, it will be on YouTube. So make sure you subscribe to me on YouTube. If I release a video, I may also make a post about it on lazyprogrammer.me and I may also announce it using all the methods I previously discussed.~~

~~So that's the newsletter, Facebook, Twitter and obviously YouTube itself. Now I realize that's a lot of stuff and you probably don't use all these platforms. I certainly don't, at least not regularly. So if you want to do the bare minimum, here's what you should do. First sign up for my newsletter. Remember, you can do that either on lazyprogrammer.me or deeplearningcourses.com. Second, subscribe to my youtube channel at youtube.com/c/lazyprogrammerx.~~

~~Thanks for listening and I'll see you in the next lecture.~~